

Correctness and Complexity of BMSSP in Isabelle/HOL

Arthur Freitas Ramos David Barros Hulak
Ruy J. G. B. de Queiroz

June 17, 2026

Abstract

This development formalizes the bounded multi-source shortest path (BMSSP) algorithm underlying the deterministic directed single-source shortest path algorithm of Duan, Mao, Mao, Shu, and Yin [2]. It proves five explicit claims. First, the executable `bmssp_distances` computes the exact reachable shortest-distance map for well-formed finite natural weighted digraphs. Second, a costed BMSSP recurrence satisfies the $O(m \log^{2/3} n)$ bound under the abstract Insert/BatchPrepend/Pull operation-cost interface. Third, the bucketed partition operations realize the primitive costs required by that interface. Fourth, the costed recurrence and the correctness statement are joined on a single object: the running time bounded by the headline is, for all but finitely many sizes, the cost of a concrete BMSSP run that provably returns the exact single-source shortest-path distances on the whole vertex set. Fifth, for the unbounded family of unit-weight directed paths P_k (with $|E| = k$ and $|V| = k + 1$ both growing with k), the closed BMSSP cost *expression* is proved *unconditionally* to be $O(k \log^{2/3} k)$ in the true graph size k . We are deliberately explicit about a limitation we uncovered and machine-checked: the companion *algorithm-level* size-parametric statement carries a run-existence hypothesis that is in fact unsatisfiable for P_k at $k \geq 2$ — a sound exact-concrete loop is forced into at most one genuine step, which cannot cover the whole path — so that statement, though valid, is vacuous on its own family and is retained only as a disclosed obstruction. The formalization does not state a single generated-code machine-step theorem for the executable. The entry also exports executable SML and includes a checked worked shortest-path example whose evaluated output is $[(0, 0), (1, 3), (2, 5), (3, 8), (4, 6)]$.

AI assistance was used for proof engineering. The final definitions, statements, and proofs are checked by Isabelle.

Contents

1 Background

4

2	Roadmap of the Formalization	4
3	Assumptions and Scope	6
4	The Headline Theorems	6
5	The Bucketed Partition Data Structure	7
6	Executable Export and Worked Example	8
7	Checked Source	9
8	Correctness Interface for BMSSP	9
9	Abstract BMSSP Correctness	15
10	Shortest-Path Prefix Facts	26
11	Concrete Base Case	50
12	Partition Data-Structure Interface	58
13	Bucketed Partition Data Structure	76
14	Cost Budgets for the Bucketed Structure	187
14.1	Paper-Facing Costed Operations	254
15	Bucketed Partition	263
16	Executable Bucketed BMSSP Smoke Test	263
17	Executable Base-Case Ordering	278
18	Unique Shortest-Path Tree	297
19	FindPivots Core Relaxation Lemmas	304
20	Concrete FindPivots Rounds	310
21	Pull Minimum	355
22	Partition Pull to BMSSP Pull Minimum	377
23	Initial Labels	380
24	Concrete One-Step BMSSP Assembly	383
25	Concrete Top-Level Correctness	386

26 Level-Indexed BMSSP Relation	388
27 Operational Pull Step	390
28 Core Cost Accounting	415
29 Logarithmic Parameter Schedule	435
30 Range-Synchronised Costed Runs	516
31 Exact Split Range-Costed Runs	614
32 Direct-Insert Costed Runs	638
33 Strict Tie-Breaking Consequences	673
34 Concrete Partition Data Structure	761
35 Exact Concrete Cost Runs	778
36 Top-Level BMSSP Theorems	823
37 Reduction Contract for the Asymptotic Bound	855
38 Executable Graph Refinement Bridge	857
39 Executable Refinement	992
40 Executable Headline	992
41 Runtime Bound for a Verified-Correct BMSSP Run	995
42 A Concrete Non-Vacuous Instance of the Runtime Locale	998
43 A Second, Structurally Different Non-Vacuous Instance	1002
44 A Size-Parametric BMSSP Running-Time Bound over a Growing Graph Family	1005
44.1 The path family	1005
44.2 Structural and cardinality lemmas	1006
44.3 Easy locale obligations, uniform in k	1006
44.4 The unconditional cost-recurrence bound (FLOOR)	1007
44.5 The forced-degenerate (empty-pivot) regime is provably avoided	1008
44.6 Unique shortest walks on the path family	1009
44.7 The verified algorithm-level running-time bound (GOLD) . .	1013
45 On the Totality of the Exact-Concrete Cost Relation	1015

1 Background

The single-source shortest path problem asks for the distance from one source vertex to every reachable vertex of a directed graph with non-negative edge weights. Dijkstra’s algorithm [1] is the classical starting point: it maintains tentative labels and repeatedly settles a vertex of minimum tentative distance. With Fibonacci heaps, Fredman and Tarjan [3] obtained the well-known $O(m + n \log n)$ implementation for graphs with n vertices and m edges. For sparse directed graphs this bound became the standard deterministic barrier: the logarithmic sorting-like term is paid by the priority-queue discipline that extracts vertices one by one.

Duan, Mao, Mao, Shu, and Yin [2] break this barrier by changing the recursive unit of work. Their algorithm is not simply Dijkstra with a faster heap. The central object is BMSSP, a bounded multi-source shortest path subproblem. A BMSSP call starts from a set of already relevant sources and an upper bound B . It only has to complete vertices whose shortest distances fall below the bound and whose shortest paths pass through the current source set. Recursive calls therefore work on ranges of distances rather than on one global queue of all unsettled vertices.

The formalization follows that structure. At the correctness level, the algorithm is expressed in terms of finite directed graphs, simple walks, shortest-path distances, and BMSSP pre/postconditions. At the cost level, the recursive proof is parameterized by an abstract partition interface with Insert, BatchPrepend, and Pull operations. The runtime theorem is proved once against that interface and the paper’s logarithmic schedule. The concrete bucketed data structure is then shown to realize the interface with the critical operation bounds. This cost statement is separate from the functional-correctness theorem for the exported executable.

The essential improvement is the ratio inside the logarithm. A sorted-list or ordinary balanced-tree representation searches among all N live entries and naturally gives a $\log N$ term. The paper’s partition structure searches among bucket boundaries, so Insert pays $O(\max(1, \log(N/M)))$, where M is the block-size parameter. With the schedule used by the recursive BMSSP analysis, this distinction is exactly what preserves the $O(m \log^{2/3} n)$ headline.

2 Roadmap of the Formalization

The formal development is organized by proof layer. The table below gives one or two primary entry points for each layer and the checked surface that a reader should look for there; supporting theories appear nearby in the session order.

Layer	Start here	Checked surface
Semantic target	BMSSP_Correctness	Finite weighted digraphs, shortest-path distances, BMSSP pre/postconditions, and root-call correctness for ordinary SSSP.
Recursive BMSSP	BMSSP_Algorithm_Correctness; BMSSP_Recursive	Correctness of FindPivots, base cases, partition-loop traces, recursive calls, and final assembly of completed vertices.
Partition interface	BMSSP_Partition_Interface; BMSSP_Partition_Pull_Bridge	Abstract Insert, BatchPrepend, and Pull contracts, with the algebraic cost predicates used by the recurrence proof.
Cost recurrence	BMSSP_Top_Level_Bounds	The logarithmic parameter schedule and the cost-model theorem <code>bmssp_runtime_bigo_target</code> .
Verified-runtime bridge	BMSSP_Verified_Runtime	Couples the asymptotic bound with correctness on one object: <code>verified_bmssp_runtime_bigo_target</code> shows the bounded running time is, eventually, the cost of a run proved <code>sssp_correct</code> on the whole vertex set.
Size-parametric family	BMSSP_Path_Family	For the growing path family P_k , <code>path_size_cost_bigo_size</code> (unconditional) proves the closed cost <i>expression</i> is $O(k \log^{2/3} k)$ in the true graph size k ; <code>path_k_bounded_instance</code> certifies locale inhabitation for every k . The algorithm-level companion <code>bmssp_path_family_runtime_bigo_size</code> is conditional and, for this family, vacuous (see the obstruction row).
Totality obstruction	BMSSP_Cost_Totality	Machine-checked record that a sound exact-concrete loop is forced into at most one genuine step (<code>exact_concrete_step_forces_empty_prefix</code> , <code>no_genuine_exact_concrete_step_at_positive_lower_bound</code>), so no complete run exists on P_k for $k \geq 2$: the <code>runs_exist</code> hypothesis of the algorithm-level size-parametric bound is unsatisfiable for the path family.
Bucketed data structure	BMSSP_Bucketed_Partition	Concrete bucketed operations satisfying <code>bp_insert_cost_bound</code> , <code>bp_batch_prepend_cost_bound</code> , <code>bp_pull_cost_bound</code> , and the interface facts <code>bp_realises_partition_insert_cost_bound</code> , <code>bp_realises_partition_batch_cost_bound</code> , and <code>bp_realises_partition_pull_cost_bound</code> .
Executable	BMSSP_Executable_	The executable <code>bmssp_distances</code> ,

Several additional theories separate the mathematical layers used below: shortest-path semantics, abstract BMSSP correctness, costed recurrences, partition-interface costs, bucketed primitive operations, executable code, and the final executable wrapper. The cost-model theorems and the executable correctness theorem are deliberately separate claims.

3 Assumptions and Scope

The entry proves three related, but not identical, layers.

First, the semantic BMSSP development starts from finite weighted digraphs, simple walks, shortest-path distances, and a bounded BMSSP pre/postcondition. The recursive correctness proof is organized through FindPivots, partition loop obligations, complete sources, and bounded distance ranges.

Second, the asymptotic development is a cost-model proof. It solves the costed BMSSP recurrence under the abstract Insert/BatchPrepend/Pull interface, uses the logarithmic parameter schedule from the paper, and proves the `bmssp_runtime_bigo_target` theorem in the corresponding runtime locale. This is not stated as a machine-step bound for generated SML.

Third, the executable development proves functional correctness for the exported finite natural-weight graph representation. The public theorem `bmssp_correct_strong` applies to every G satisfying `nat_graph_well_formed` G and every source `src` in `nat_graph_vertices` G . It does not assume unique shortest paths. The well-formedness predicate is a representation-side condition: the directed edge list is duplicate-free and the total natural edge weight is below the executable sentinel `bmssp_infinity`.

4 The Headline Theorems

The primary executable correctness statement is

`bmssp_correct_strong` in `BMSSP_Executable_Headline`.

The theorem fixes a finite directed graph G of natural-numbered vertices with non-negative natural edge weights and a source vertex `src`. The statement is

For every G with `nat_graph_well_formed` G and every `src` in `nat_graph_vertices` G :

- the keys of `bmssp_distances` G `src` are distinct;
- the key set is exactly the set of vertices reachable from `src`;
- every pair (v, d) returned by `bmssp_distances` G `src` satisfies $d = \text{nat_graph_dist } G \text{ src } v$ after coercion to real numbers.

Thus the executable theorem is not conditional on a unique-shortest-path or tie-breaking assumption. Its displayed hypotheses are the finite natural-graph representation condition and the source-membership condition.

The runtime and bucketed-interface statements are separate public claims. `bmssp_runtime_bigo_target` is the cost-model theorem for the abstract BMSSP recurrence and logarithmic parameter schedule. Its closed assumption-free form is stated inside the runtime locale; interpreting that locale supplies the reduced positive instance, reachability, and bounded outdegree hypotheses. To certify that this locale is inhabited rather than vacuous, the theory `BMSSP_Runtime_Instance` interprets it on a concrete unit-weight directed path $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$: it discharges unique shortest walks (from the verified simple-walk enumerator), positive weights, full reachability, and bounded outdegree, and thereby obtains `path_runtime_bigo_target`, the same logarithmic running-time bound as a closed, assumption-free statement for an explicit non-trivial graph. The corollaries `bp_realises_partition_insert_cost_bound`, `bp_realises_partition_batch_cost_bound`, and `bp_realises_partition_pull_cost_bound` show that the bucketed Insert, BatchPrepend, and Pull operations realize the abstract primitive-cost predicates used by that recurrence. The more concrete theorems `bp_insert_cost_bound`, `bp_batch_prepend_cost_bound`, and `bp_pull_cost_bound` expose the amortised budgets for the bucketed implementation itself.

The session build checks all of these claims. The executable smoke test `example_bmssp_correct` also executes the generated code equations on a worked five-vertex example with `bmssp_distances example_graph 0 = [(0, 0), (1, 3), (2, 5), (3, 8), (4, 6)]`, proved by `by eval` inside the session build itself.

5 The Bucketed Partition Data Structure

The partition data structure is the main concrete contribution of the executable layer. It stores tentative labels as key/value pairs grouped into buckets. Each bucket contains an unsorted list of at most M entries in the regular state, and the bucket markers form a monotone directory of lower bounds. The directory, rather than the full set of entries, is the object searched by Insert. If N entries have been inserted since the last Pull, there are only on the order of N/M buckets, which gives the $\log(N/M)$ search term.

The formal model separates several invariants. The ordinary invariant states that bucket sizes are within the block-size bound, markers are sorted, markers are lower bounds for their bucket entries, keys are distinct, and the functional value memory agrees with the entries stored in buckets. The ordered invariant additionally states that adjacent bucket boundaries separate

the value ranges. The lazy invariant relaxes the size side condition: a bucket may temporarily grow up to the lazy threshold before a split is forced.

The amortised analysis is deliberately close to the paper’s intuition. A lazy Insert searches the bucket directory, inserts into one bucket, and only splits when the bucket has accumulated enough excess entries. The potential function records overflow beyond M together with a small slack term for the number of buckets. The proofs use a scaled credit invariant, so the actual cost of a split is paid by the overflow accumulated by previous insertions. After the split the potential drops, and the amortised Insert cost is the directory-search charge plus a constant amount of update work.

BatchPrepend receives a list of new candidates that belong before the currently exposed range. The concrete operation rebuckets that batch and prepends the resulting buckets in one operation, which is why its accounting is per item rather than a repeated ordinary Insert cost. Pull removes a first bucket when the first-bucket side conditions hold, and its scanned work is bounded by the block size. The theorem names reflect these three conclusions: `bp_insert_cost_bound`, `bp_batch_prepend_cost_bound`, and `bp_pull_cost_bound`.

The session also contains a baseline sorted-list partition model used by the exact recurrence and abstraction proofs. Sorted lists refine the same abstract contracts, but they search on N entries and therefore do not supply the paper’s N/M ratio. The bucketed structure is the implementation connected to the active partition-interface realization facts.

6 Executable Export and Worked Example

The final theory gives a concrete executable surface. Vertices and edge weights are natural numbers, graphs are finite edge lists, and the work-list used by the executable loop is the bucketed partition from `BMSSP_Bucketed_Partition`. The entry point `bmssp_distances` computes a finite association list of distances for the example graph.

The executable entry point is a finite natural-weight instantiation using the bucketed work-list. Its functional correctness is proved independently against the shortest-path semantics. The asymptotic result applies to the costed BMSSP model and bucketed operation interface; the two layers share the same partition primitives but are distinct artifacts. The executable layer uses a finite upper sentinel for natural labels; the abstract correctness layer uses the `bound = Fin real | Infinity` datatype.

The example has source 0 and five vertices. The hand-computed distances are:

$$d(0) = 0, \quad d(1) = 3, \quad d(2) = 5, \quad d(3) = 8, \quad d(4) = 6.$$

The Isabelle theorem

`example_bmssp_correct`

states that the executable function returns exactly this association list. Its proof is by `eval`. Thus the bundled Poly/ML evaluator executes the generated code equations during the build; if the executable algorithm or the expected map is wrong, the theory does not check.

The same theory contains a `value` command for human inspection and an `export_code` command that emits SML module `BMSSP`. The session `ROOT` file includes an export directive so the generated code can be materialized as `generated/BMSSP.ML`.

7 Checked Source

The remainder of this document is the checked Isabelle source. Definitions, theorem statements, proof scripts, and document text are all processed by the same session build.

```
theory BMSSP-Correctness
imports Complex-Main
begin
```

8 Correctness Interface for BMSSP

This is the entry point of the correctness layer. It deliberately avoids committing to the paper’s data structure, cost accounting, perturbation reduction, or executable representation. The purpose here is to state the mathematical contract that every later algorithmic refinement has to meet.

The ambient object is a finite directed graph with non-negative real edge weights and a distinguished source vertex. Distances are not imported from a library shortest-path algorithm. Instead, they are defined directly as minima of finite sets of simple-walk weights. This gives the later proofs a small, transparent semantic target: a label is correct exactly when it agrees with the minimum simple-walk distance from the source.

BMSSP means bounded multi-source shortest path. A recursive BMSSP call is not asked to solve all of single-source shortest paths. It receives a set of sources, a bound, and a current label function. It must complete the vertices whose shortest paths pass through the source set and whose distances lie below the bound returned by the call. This bounded contract is the reason the paper can recurse on distance ranges rather than repeatedly extracting one global minimum vertex.

The definitions below therefore introduce three pieces of vocabulary that recur throughout the development. A bound can be finite or infinite; a bound tree is the set of vertices relevant to a source set below such a bound; and the BMSSP postcondition says that the output labels are complete on exactly that tree. Later theories refine how the tree is assembled: `FindPivots` identifies useful sources, the partition loop splits the distance range, and

the bucketed data structure realizes the required Insert, BatchPrepend, and Pull operations. The final theorems of this file show that a successful root BMSSP call is already enough to establish ordinary single-source shortest-path correctness.

datatype *bound* = *Fin real* | *Infinity*

fun *below-bound* :: *real* $\Rightarrow$ *bound* $\Rightarrow$ *bool* **where**
 below-bound *x* *Infinity* $\longleftrightarrow$ *True*
 | *below-bound* *x* (*Fin* *b*) $\longleftrightarrow$ *x* < *b*

fun *bound-le* :: *bound* $\Rightarrow$ *bound* $\Rightarrow$ *bool* **where**
 bound-le *Infinity* *Infinity* $\longleftrightarrow$ *True*
 | *bound-le* *Infinity* (*Fin* *-*) $\longleftrightarrow$ *False*
 | *bound-le* (*Fin* *a*) (*Fin* *b*) $\longleftrightarrow$ *a* $\leq$ *b*

The datatype *bound* is intentionally small. The predicate *below-bound* is strict for finite bounds, matching the paper's range convention, while *bound-le* is the preorder used when a recursive call returns a possibly smaller upper bound. Keeping this type separate from raw reals makes the top-level call with *Infinity* explicit and avoids ad hoc sentinel values in the correctness proof.

locale *finite-weighted-digraph* =
 fixes *V* :: '*v* set
 and *E* :: ('*v* $\times$ '*v*) set
 and *w* :: '*v* $\Rightarrow$ '*v* $\Rightarrow$ *real*
 and *s* :: '*v*
 assumes *finite-V*: *finite* *V*
 and *source-in-V*: *s* $\in$ *V*
 and *edge-in-V*: $(u, v) \in E \implies u \in V \wedge v \in V$
 and *nonneg-weight*: $(u, v) \in E \implies 0 \leq w\ u\ v$
begin

fun *walk* :: '*v* list $\Rightarrow$ *bool* **where**
 walk [] $\longleftrightarrow$ *False*
 | *walk* [*x*] $\longleftrightarrow$ *x* $\in$ *V*
 | *walk* (*x* # *y* # *xs*) $\longleftrightarrow$ *x* $\in$ *V* $\wedge$ $(x, y) \in E \wedge$ *walk* (*y* # *xs*)

fun *walk-weight* :: '*v* list $\Rightarrow$ *real* **where**
 walk-weight [] = 0
 | *walk-weight* [*x*] = 0
 | *walk-weight* (*x* # *y* # *xs*) = *w* *x* *y* + *walk-weight* (*y* # *xs*)

definition *walk-betw* :: '*v* $\Rightarrow$ '*v* list $\Rightarrow$ '*v* $\Rightarrow$ *bool* **where**
 walk-betw *a* *p* *b* $\longleftrightarrow$ *p* $\neq$ [] $\wedge$ *hd* *p* = *a* $\wedge$ *last* *p* = *b* $\wedge$ *walk* *p*

definition *simple-walk-betw* :: '*v* $\Rightarrow$ '*v* list $\Rightarrow$ '*v* $\Rightarrow$ *bool* **where**
 simple-walk-betw *a* *p* *b* $\longleftrightarrow$ *walk-betw* *a* *p* *b* $\wedge$ *distinct* *p*

definition *reachable* :: $'v \Rightarrow 'v \Rightarrow \text{bool}$ **where**
 $\text{reachable } a \ b \longleftrightarrow (\exists p. \text{simple-walk-betw } a \ p \ b)$

definition *simple-walk-weights* :: $'v \Rightarrow 'v \Rightarrow \text{real set}$ **where**
 $\text{simple-walk-weights } a \ b = \text{walk-weight } \{p. \text{simple-walk-betw } a \ p \ b\}$

definition *dist* :: $'v \Rightarrow 'v \Rightarrow \text{real}$ **where**
 $\text{dist } a \ b = \text{Min } (\text{simple-walk-weights } a \ b)$

definition *shortest-walk* :: $'v \Rightarrow 'v \text{ list} \Rightarrow 'v \Rightarrow \text{bool}$ **where**
 $\text{shortest-walk } a \ p \ b \longleftrightarrow \text{simple-walk-betw } a \ p \ b \wedge \text{walk-weight } p = \text{dist } a \ b$

definition *through* :: $'v \text{ set} \Rightarrow 'v \Rightarrow \text{bool}$ **where**
 $\text{through } S \ v \longleftrightarrow (\exists u \in S. \exists p. \text{shortest-walk } s \ p \ v \wedge u \in \text{set } p)$

definition *bound-tree* :: $'v \text{ set} \Rightarrow \text{bound} \Rightarrow 'v \text{ set}$ **where**
 $\text{bound-tree } S \ B =$
 $\{v \in V. \text{reachable } s \ v \wedge \text{through } S \ v \wedge \text{below-bound } (\text{dist } s \ v) \ B\}$

definition *complete-on* :: $('v \Rightarrow \text{real}) \Rightarrow 'v \text{ set} \Rightarrow \text{bool}$ **where**
 $\text{complete-on } d \ U \longleftrightarrow (\forall v \in U. \text{reachable } s \ v \longrightarrow d \ v = \text{dist } s \ v)$

definition *sssp-correct* :: $('v \Rightarrow \text{real}) \Rightarrow \text{bool}$ **where**
 $\text{sssp-correct } d \longleftrightarrow (\forall v \in V. \text{reachable } s \ v \longrightarrow d \ v = \text{dist } s \ v)$

definition *bmssp-pre* :: $('v \Rightarrow \text{real}) \Rightarrow 'v \text{ set} \Rightarrow \text{bound} \Rightarrow \text{bool}$ **where**
 $\text{bmssp-pre } d \ S \ B \longleftrightarrow$
 $S \subseteq V \wedge$
 $(\forall v \in V. \text{reachable } s \ v \longrightarrow \text{below-bound } (\text{dist } s \ v) \ B \longrightarrow$
 $d \ v \neq \text{dist } s \ v \longrightarrow \text{through } S \ v)$

definition *bmssp-post* ::
 $('v \Rightarrow \text{real}) \Rightarrow 'v \text{ set} \Rightarrow \text{bound} \Rightarrow ('v \Rightarrow \text{real}) \Rightarrow \text{bound} \Rightarrow 'v \text{ set} \Rightarrow \text{bool}$ **where**
 $\text{bmssp-post } d \ S \ B \ d' \ B' \ U \longleftrightarrow$
 $\text{bound-le } B' \ B \wedge U = \text{bound-tree } S \ B' \wedge \text{complete-on } d' \ U$

The graph locale makes the semantic model finite at the point where finiteness is needed. A walk is a list of vertices following edges; a simple walk is a walk with no repeated vertices; and *local.dist* is the minimum weight over simple walks. Because all edge weights are non-negative, restricting to simple walks preserves shortest distances while making the set minimized by *local.dist* finite.

The central BMSSP contract is expressed by *bmssp-pre* and *bmssp-post*. The precondition says that every not-yet-complete reachable vertex below the input bound must be reachable through the current source set, as formalized by *through*. The postcondition returns a new bound, an output set, and an updated label function; it demands that the output set is exactly *bound-tree* for the returned bound and that labels are complete there via *complete-*

on. This is intentionally weaker than global SSSP correctness, because the recursive algorithm only solves the range assigned to the current call.

lemma *walk-set-subset*:

assumes *walk p*
shows $\text{set } p \subseteq V$
using *assms edge-in-V*
by (*induction p rule: walk.induct*) *auto*

lemma *simple-walk-length-le-card*:

assumes *simple-walk-betw a p b*
shows $\text{length } p \leq \text{card } V$

proof –

have *walk p* **and** *distinct p*
using *assms unfolding simple-walk-betw-def walk-betw-def* **by** *auto*
then have $\text{set } p \subseteq V$
using *walk-set-subset* **by** *blast*
with $\langle \text{distinct } p \rangle$ **have** $\text{length } p = \text{card } (\text{set } p)$
by (*simp add: distinct-card*)
also have $\dots \leq \text{card } V$
using $\langle \text{set } p \subseteq V \rangle$ *finite-V* **by** (*simp add: card-mono*)
finally show *?thesis* .

qed

lemma *finite-simple-walks*:

shows *finite* $\{p. \text{simple-walk-betw } a \ p \ b\}$

proof (*rule finite-subset*)

show $\{p. \text{simple-walk-betw } a \ p \ b\}$
 $\subseteq \{p. \text{set } p \subseteq V \wedge \text{length } p \leq \text{card } V\}$
using *simple-walk-length-le-card walk-set-subset*
unfolding *simple-walk-betw-def walk-betw-def* **by** *auto*
show *finite* $\{p. \text{set } p \subseteq V \wedge \text{length } p \leq \text{card } V\}$
using *finite-lists-length-le[OF finite-V]* .

qed

lemma *finite-simple-walk-weights*:

shows *finite* (*simple-walk-weights a b*)
unfolding *simple-walk-weights-def*
using *finite-simple-walks* **by** *simp*

lemma *simple-walk-weights-nonempty*:

assumes *reachable a b*
shows *simple-walk-weights a b* $\neq \{\}$
using *assms unfolding reachable-def simple-walk-weights-def* **by** *blast*

lemma *dist-is-simple-walk-weight*:

assumes *reachable a b*
shows $\text{dist } a \ b \in \text{simple-walk-weights } a \ b$
unfolding *dist-def*
using *finite-simple-walk-weights simple-walk-weights-nonempty[OF assms]* **by**

(rule *Min-in*)

lemma *shortest-walk-exists*:

assumes *reachable a b*

obtains *p* where *shortest-walk a p b*

proof –

have *dist a b ∈ simple-walk-weights a b*

using *assms* by (rule *dist-is-simple-walk-weight*)

then have $\exists p. \text{simple-walk-betw } a \ p \ b \wedge \text{walk-weight } p = \text{dist } a \ b$

unfolding *simple-walk-weights-def* by (auto simp: *image-iff*)

then obtain *p* where *simple-walk-betw a p b walk-weight p = dist a b*
by *blast*

then have *shortest-walk a p b*

unfolding *shortest-walk-def* by *blast*

then show *thesis*

using *that* by *blast*

qed

lemma *shortest-walk-hd*:

assumes *shortest-walk a p b*

shows $p \neq [] \text{hd } p = a \text{last } p = b$

using *assms* unfolding *shortest-walk-def simple-walk-betw-def walk-betw-def* by
auto

lemma *reachable-refl*:

assumes $v \in V$

shows *reachable v v*

using *assms* unfolding *reachable-def simple-walk-betw-def walk-betw-def* by *auto*

lemma *reachable-source-through*:

assumes *reachable s v*

shows *through {s} v*

proof –

obtain *p* where *p: shortest-walk s p v*

using *assms* by (rule *shortest-walk-exists*)

then have $p \neq [] \text{hd } p = s$

using *shortest-walk-hd* by *blast+*

then have $s \in \text{set } p$

by (*metis hd-in-set*)

with *p* show *?thesis*

unfolding *through-def* by *blast*

qed

lemma *top-bmssp-pre*:

assumes $\bigwedge v. v \in V \implies \text{reachable } s \ v$

shows *bmssp-pre d {s} Infinity*

using *assms* *reachable-source-through source-in-V*

unfolding *bmssp-pre-def* by *auto*

lemma *bound-tree-source-infinity*:
assumes $\bigwedge v. v \in V \implies \text{reachable } s \ v$
shows *bound-tree* $\{s\}$ *Infinity* = V
using *assms reachable-source-through*
unfolding *bound-tree-def* **by** *auto*

lemma *bound-tree-source-infinity-reachable*:
shows *bound-tree* $\{s\}$ *Infinity* = $\{v \in V. \text{reachable } s \ v\}$
using *reachable-source-through*
unfolding *bound-tree-def* **by** *auto*

lemma *bmssp-post-complete-bound-tree*:
assumes *bmssp-post* $d \ S \ B \ d' \ B' \ U$
and $v \in \text{bound-tree } S \ B'$
shows $d' \ v = \text{dist } s \ v$
proof –
have $U = \text{bound-tree } S \ B' \text{ and } \text{complete-on } d' \ U$
using *assms(1)* **unfolding** *bmssp-post-def* **by** *auto*
then have $v \in U$
using *assms(2)* **by** *simp*
moreover have *reachable* $s \ v$
using *assms(2)* **unfolding** *bound-tree-def* **by** *auto*
then show *?thesis*
using $\langle \text{complete-on } d' \ U \rangle$ *calculation* **unfolding** *complete-on-def* **by** *auto*
qed

lemma *bmssp-success-completes-requested-tree*:
assumes *bmssp-post* $d \ S \ B \ d' \ B' \ U$
and $v \in \text{bound-tree } S \ B$
shows $d' \ v = \text{dist } s \ v$
using *bmssp-post-complete-bound-tree[OF assms]* .

lemma *sssp-correctI*:
assumes $\bigwedge v. \llbracket v \in V; \text{reachable } s \ v \rrbracket \implies d \ v = \text{dist } s \ v$
shows *sssp-correct* d
using *assms* **unfolding** *sssp-correct-def* **by** *blast*

The remaining lemmas discharge the semantic obligations introduced above. Finiteness of simple walks justifies the use of *Min*; reachability of the source shows that the root source set covers every reachable vertex; and $\llbracket \text{bmssp-post } ?d \ ?S \ ?B \ ?d' \ ?B' \ ?U; ?v \in \text{bound-tree } ?S \ ?B \rrbracket \implies ?d' \ ?v = \text{local.dist } s \ ?v$ extracts a pointwise distance equality from the BMSSP postcondition. The final two theorems are the bridge from a root BMSSP result to the ordinary SSSP predicate *sssp-correct*. Later files prove BMSSP postconditions for concrete recursive executions, and then reuse these root theorems unchanged.

theorem *successful-root-bmssp-correct*:
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$

```

    and post: bmssp-post d {s} Infinity d' Infinity U
  shows sssp-correct d'
proof -
  have U = V
    using post bound-tree-source-infinity[OF all-reachable]
    unfolding bmssp-post-def by auto
  moreover have complete-on d' U
    using post unfolding bmssp-post-def by auto
  ultimately show ?thesis
    unfolding complete-on-def sssp-correct-def by auto
qed

theorem successful-root-bmssp-sssp-correct:
  assumes post: bmssp-post d {s} Infinity d' Infinity U
  shows sssp-correct d'
proof (rule sssp-correctI)
  fix v
  assume vV: v ∈ V and reach-v: reachable s v
  have v ∈ bound-tree {s} Infinity
    using vV reach-v reachable-source-through unfolding bound-tree-def by auto
  then show d' v = dist s v
    by (rule bmssp-post-complete-bound-tree[OF post])
qed

end

end

theory BMSSP-Algorithm-Correctness
  imports BMSSP-Correctness
begin

```

9 Abstract BMSSP Correctness

This theory is the correctness skeleton of the BMSSP development. The previous entry-point theory defines distances, bounds, and the basic bounded multi-source postcondition. Here we refine that interface just far enough to express the recursive algorithm described in the paper, while still abstracting from running time and from the concrete partition data structure.

The key strengthening is that sources used by a recursive call are required to be complete sources: their current labels already equal their true shortest-path distances. This matters because BMSSP does not merely carry a set of vertices; it carries a set of vertices whose shortest-path trees may certify the rest of the current bounded range. A vertex that is not solved yet must either be solved directly by the current round or lie on a shortest path through one of those complete sources.

At this level the algorithm is decomposed into three abstract obligations:

- *FindPivots* must either complete a bounded vertex or delegate it to a complete pivot.
- The partition loop driven by *Pull*, recursive BMSSP calls, edge relaxation, and *BatchPrepend* must solve the bounded problem induced by the pivot set.
- The final assembly of recursive-loop vertices with the completed *FindPivots* vertices is then proved correct.

The main theorem of the file, *abstract-bmssp-correct*, says that any implementation satisfying those obligations establishes the full BMSSP postcondition. Later theories instantiate the obligations with a concrete recursive loop, then the bucketed partition structure, and finally the executable example. None of the runtime arguments appear here; this file is the semantic hinge that lets the later cost proofs remain separate from the shortest-path correctness proof.

Two abstractions matter for navigating the rest of the development. First, this layer abstracts from the partition data structure entirely: any implementation satisfying the three obligations above is acceptable, so the later sorted-list and bucketed-partition refinements both plug in here without disturbing the correctness proof. Second, this layer abstracts from running time entirely: the cost predicates and amortized budgets live one layer further down, in *BMSSP_Top_Level_Bounds.thy* and *BMSSP_Bucketed_Partition.thy*, so that a reader interested only in correctness can stop here.

context *finite-weighted-digraph*
begin

definition *complete-vertices* :: ('v $\Rightarrow$ real) $\Rightarrow$ 'v set **where**
complete-vertices d = {v $\in$ V. *reachable* s v $\wedge$ d v = *dist* s v}

definition *through-complete* :: ('v $\Rightarrow$ real) $\Rightarrow$ 'v set $\Rightarrow$ 'v $\Rightarrow$ bool **where**
through-complete d S v $\longleftrightarrow$ *through* (S $\cap$ *complete-vertices* d) v

definition *bmssp-pre-full* :: ('v $\Rightarrow$ real) $\Rightarrow$ 'v set $\Rightarrow$ bound $\Rightarrow$ bool **where**
bmssp-pre-full d S B $\longleftrightarrow$
 S $\subseteq$ V $\wedge$
 ($\forall v \in V. \text{reachable } s \ v \longrightarrow \text{below-bound } (\text{dist } s \ v) \ B \longrightarrow$
 d v $\neq$ *dist* s v $\longrightarrow$ *through-complete* d S v)

definition *bmssp-post-full* ::
 ('v $\Rightarrow$ real) $\Rightarrow$ 'v set $\Rightarrow$ bound $\Rightarrow$ ('v $\Rightarrow$ real) $\Rightarrow$ bound $\Rightarrow$ 'v set $\Rightarrow$ bool **where**
bmssp-post-full d S B d' B' U $\longleftrightarrow$
 bound-le B' B $\wedge$ U = bound-tree S B' $\wedge$ complete-on d' U

definition *find-pivots-post* ::
 ('v $\Rightarrow$ real) $\Rightarrow$ 'v set $\Rightarrow$ bound $\Rightarrow$ ('v $\Rightarrow$ real) $\Rightarrow$ 'v set $\Rightarrow$ 'v set $\Rightarrow$ bool **where**

$$\begin{aligned}
& \text{find-pivots-post } d \ S \ B \ d' \ P \ W \longleftrightarrow \\
& S \subseteq V \wedge \\
& P \subseteq S \wedge \\
& W \subseteq \text{bound-tree } S \ B \wedge \\
& \text{complete-on } d' \ W \wedge \\
& (\forall v \in V. \text{reachable } s \ v \longrightarrow \text{below-bound } (\text{dist } s \ v) \ B \longrightarrow \\
& \quad d \ v = \text{dist } s \ v \longrightarrow d' \ v = \text{dist } s \ v) \wedge \\
& (\forall v \in \text{bound-tree } S \ B. v \in W \vee \text{through-complete } d' \ P \ v)
\end{aligned}$$

definition *partition-loop-post* ::

$('v \Rightarrow \text{real}) \Rightarrow 'v \text{ set} \Rightarrow \text{bound} \Rightarrow ('v \Rightarrow \text{real}) \Rightarrow \text{bound} \Rightarrow 'v \text{ set} \Rightarrow \text{bool}$ **where**
 $\text{partition-loop-post } d \ P \ B \ d' \ B' \ U \longleftrightarrow \text{bmssp-post-full } d \ P \ B \ d' \ B' \ U$

definition *final-bmssp-assembly* ::

$'v \text{ set} \Rightarrow \text{bound} \Rightarrow 'v \text{ set} \Rightarrow 'v \text{ set} \Rightarrow 'v \text{ set} \Rightarrow \text{bool}$ **where**
 $\text{final-bmssp-assembly } S \ B' \ W \ U\text{-loop } U \longleftrightarrow$
 $U = U\text{-loop} \cup \{v \in W. \text{below-bound } (\text{dist } s \ v) \ B'\}$

definition *abstract-bmssp-step* ::

$('v \Rightarrow \text{real}) \Rightarrow 'v \text{ set} \Rightarrow \text{bound} \Rightarrow ('v \Rightarrow \text{real}) \Rightarrow \text{bound} \Rightarrow 'v \text{ set} \Rightarrow \text{bool}$ **where**
 $\text{abstract-bmssp-step } d \ S \ B \ d' \ B' \ U \longleftrightarrow$
 $(\exists d\text{-fp } P \ W \ U\text{-loop}. \\
\text{find-pivots-post } d \ S \ B \ d\text{-fp } P \ W \wedge \\
\text{partition-loop-post } d\text{-fp } P \ B \ d' \ B' \ U\text{-loop} \wedge \\
\text{complete-on } d' \ W \wedge \\
\text{final-bmssp-assembly } S \ B' \ W \ U\text{-loop } U)$

The definitions above separate the proof into named contracts. The set *complete-vertices* records exactly the vertices solved by a label function, and *through-complete* refines the earlier *through*-predicate by requiring the intermediary source to be solved. The strengthened precondition *bmssp-pre-full* is therefore the recursive invariant: every unresolved bounded vertex is certified by a shortest path through a complete source.

The predicate *find-pivots-post* is intentionally asymmetric. It returns a new label function, a pivot subset, and a set of vertices completed directly by the pivot search. Vertices in the current bounded tree are then either in that completed set or are delegated to the pivot subset via *through-complete*. The partition loop contract *partition-loop-post* is just the full BMSSP postcondition specialized to those pivots. Finally, *final-bmssp-assembly* describes the set union that reconstructs the caller's solved range from the recursive-loop output and the vertices completed by FindPivots.

This decomposition is useful because each later theory can prove one small interface. The algorithmic theory proves an instance of *abstract-bmssp-step*; the data-structure theories prove that the partition operations realize the abstract loop assumptions; the top-level theory only needs the postcondition produced here.

lemma *bmssp-post-full-imp-post*:

assumes *bmssp-post-full* $d \ S \ B \ d' \ B' \ U$
shows *bmssp-post* $d \ S \ B \ d' \ B' \ U$
using *assms* **unfolding** *bmssp-post-full-def* *bmssp-post-def* **by** *blast*

lemma *top-bmssp-pre-full*:
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *source-complete*: $d \ s = \text{dist } s \ s$
shows *bmssp-pre-full* $d \ \{s\} \ \text{Infinity}$
proof –
have $s \in \text{complete-vertices } d$
using *source-in-V* *source-complete* *reachable-refl*
unfolding *complete-vertices-def* **by** *blast*
then have $\bigwedge v. \text{reachable } s \ v \implies \text{through-complete } d \ \{s\} \ v$
using *reachable-source-through* **unfolding** *through-complete-def* **by** *auto*
then show *?thesis*
using *all-reachable* *source-in-V* **unfolding** *bmssp-pre-full-def* **by** *auto*
qed

lemma *top-bmssp-pre-full-reachable*:
assumes *source-complete*: $d \ s = \text{dist } s \ s$
shows *bmssp-pre-full* $d \ \{s\} \ \text{Infinity}$
proof –
have $s \in \text{complete-vertices } d$
using *source-in-V* *source-complete* *reachable-refl*
unfolding *complete-vertices-def* **by** *blast*
then have $\bigwedge v. \text{reachable } s \ v \implies \text{through-complete } d \ \{s\} \ v$
using *reachable-source-through* **unfolding** *through-complete-def* **by** *auto*
then show *?thesis*
using *source-in-V* **unfolding** *bmssp-pre-full-def* **by** *auto*
qed

lemma *through-complete-imp-through*:
assumes *through-complete* $d \ S \ v$
shows *through* $S \ v$
using *assms* **unfolding** *through-complete-def* *through-def* **by** *blast*

lemma *bound-tree-mono*:
assumes $S \subseteq T$
shows *bound-tree* $S \ B \subseteq \text{bound-tree } T \ B$
using *assms* **unfolding** *bound-tree-def* *through-def* **by** *blast*

lemma *bound-tree-bound-mono*:
assumes *bound-le* $B' \ B$
shows *bound-tree* $S \ B' \subseteq \text{bound-tree } S \ B$
using *assms* **unfolding** *bound-tree-def*
by (*cases* B' ; *cases* B) *auto*

lemma *complete-on-subset*:
assumes *complete-on* $d \ T \ S \subseteq T$

shows *complete-on* d S
using *assms* **unfolding** *complete-on-def* **by** *blast*

lemma *source-in-own-bound-tree*:
assumes xS : $x \in S$
and *reach-x*: *reachable* s x
and *below-x*: *below-bound* ($\text{dist } s$ x) B
shows $x \in \text{bound-tree } S$ B
proof –
obtain p **where** p : *shortest-walk* s p x
using *reach-x* **by** (*rule shortest-walk-exists*)
have $x\text{-in-}p$: $x \in \text{set } p$
using *shortest-walk-hd*(1,3)[*OF* p] **by** (*metis last-in-set*)
then have *through* S x
using xS p **unfolding** *through-def* **by** *blast*
moreover have $x \in V$
proof –
have *walk* p
using p **unfolding** *shortest-walk-def* *simple-walk-betw-def*
walk-betw-def **by** *blast*
then have $\text{set } p \subseteq V$
by (*rule walk-set-subset*)
then show *?thesis*
using $x\text{-in-}p$ **by** *blast*
qed
ultimately show *?thesis*
using *reach-x* *below-x* **unfolding** *bound-tree-def* **by** *blast*
qed

lemma *sources-subset-bound-tree*:
assumes *reaches*: $\bigwedge x. x \in S \implies \text{reachable } s$ x
and *below*: $\bigwedge x. x \in S \implies \text{below-bound } (\text{dist } s$ $x)$ B
shows $S \subseteq \text{bound-tree } S$ B
using *source-in-own-bound-tree*[*OF* - *reaches below*] **by** *blast*

lemma *sources-subset-bound-tree-if-label-below*:
assumes *complete*: *complete-on* d S
and *reaches*: $\bigwedge x. x \in S \implies \text{reachable } s$ x
and *below*: $\bigwedge x. x \in S \implies \text{below-bound } (d$ $x)$ B
shows $S \subseteq \text{bound-tree } S$ B
proof (*rule sources-subset-bound-tree*)
fix x
assume xS : $x \in S$
then show *reachable* s x
by (*rule reaches*)
have d $x = \text{dist } s$ x
using *complete reaches*[*OF* xS] xS **unfolding** *complete-on-def* **by** *blast*
then show *below-bound* ($\text{dist } s$ x) B
using *below*[*OF* xS] **by** *simp*

qed

lemma *bmssp-post-full-source-card-le-if-label-below-output:*

assumes *post*: *bmssp-post-full* *d S B d' B' U*
and *complete*: *complete-on* *d S*
and *reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
and *below*: $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B'$
shows *card* *S* $\leq$ *card* *U*

proof –

have *U-eq*: *U* = *bound-tree* *S B'*
using *post* **unfolding** *bmssp-post-full-def* **by** *blast*
have *S-subset-U*: *S* $\subseteq$ *U*
using *sources-subset-bound-tree-if-label-below*[*OF complete reaches below*]
unfolding *U-eq* .
have *finite-U*: *finite* *U*
unfolding *U-eq bound-tree-def* **using** *finite-V* **by** *auto*
show *?thesis*
by (*rule card-mono*[*OF finite-U S-subset-U*])

qed

The first group of lemmas is structural. It relates the strengthened complete-source invariant to the base BMSSP vocabulary and proves the monotonicity facts needed when the recursive call returns a smaller bound. The source-subset lemmas are small but important: they show that a complete source whose label is below the returned bound belongs to the returned *bound-tree*. That is the formal reason the recursive call cannot discard the pivots it was asked to solve.

lemma *find-pivots-establishes-pivot-pre:*

assumes *pre*: *bmssp-pre-full* *d S B*
and *fp*: *find-pivots-post* *d S B d' P W*
shows *bmssp-pre-full* *d' P B*

proof –

have *P* $\subseteq$ *V*
using *fp* **unfolding** *find-pivots-post-def bound-tree-def* **by** *auto*
moreover **have** $\bigwedge v. \llbracket v \in V; \text{reachable } s \ v; \text{below-bound } (dist \ s \ v) \ B; d' \ v \neq dist \ s \ v \rrbracket$
 $\implies \text{through-complete } d' \ P \ v$

proof –

fix *v*
assume *v*: *v* $\in$ *V* *reachable* *s v* *below-bound* $(dist \ s \ v) \ B \ d' \ v \neq dist \ s \ v$
show *through-complete* *d' P v*
proof (*cases through S v*)
case *True*
then **have** *v* $\in$ *bound-tree* *S B*
using *v* **unfolding** *bound-tree-def* **by** *blast*
then **have** *v* $\in$ *W* $\vee$ *through-complete* *d' P v*
using *fp* **unfolding** *find-pivots-post-def* **by** *blast*
then **show** *?thesis*
proof

```

    assume  $v \in W$ 
    then have  $d' v = \text{dist } s v$ 
      using  $\text{fp } v$  unfolding find-pivots-post-def complete-on-def by blast
    with  $v$  show ?thesis by blast
  qed blast
next
case False
have  $d v = \text{dist } s v$ 
proof (rule ccontr)
  assume  $d v \neq \text{dist } s v$ 
  then have through-complete  $d S v$ 
    using  $\text{pre } v$  unfolding bmssp-pre-full-def by blast
  then have through  $S v$ 
    by (rule through-complete-imp-through)
  with False show False by blast
qed
then have  $d' v = \text{dist } s v$ 
  using  $\text{fp } v$  unfolding find-pivots-post-def by blast
with  $v$  show ?thesis by blast
qed
ultimately show ?thesis
  unfolding bmssp-pre-full-def by blast
qed

lemma conservative-find-pivots-post:
  assumes  $\text{pre: } \text{bmssp-pre-full } d S B$ 
  defines  $W \equiv \{v \in \text{bound-tree } S B. d v = \text{dist } s v\}$ 
  shows find-pivots-post  $d S B d S W$ 
proof -
  have  $S: S \subseteq V$ 
    using  $\text{pre}$  unfolding bmssp-pre-full-def by blast
  have  $W\text{-subset}: W \subseteq \text{bound-tree } S B$ 
    unfolding W-def by blast
  have  $\text{comp}W: \text{complete-on } d W$ 
    unfolding W-def complete-on-def by blast
  have  $\text{preserve}: \forall v \in V. \text{reachable } s v \longrightarrow \text{below-bound } (\text{dist } s v) B \longrightarrow$ 
     $d v = \text{dist } s v \longrightarrow d v = \text{dist } s v$ 
    by blast
  have  $\text{cover}: \forall v \in \text{bound-tree } S B. v \in W \vee \text{through-complete } d S v$ 
proof
  fix  $v$ 
  assume  $v: v \in \text{bound-tree } S B$ 
  show  $v \in W \vee \text{through-complete } d S v$ 
  proof (cases  $d v = \text{dist } s v$ )
    case True
    then show ?thesis
      using  $v$  unfolding W-def by blast
  next

```

```

    case False
  then show ?thesis
    using pre v unfolding bmssp-pre-full-def bound-tree-def by blast
  qed
qed
show ?thesis
  using S W-subset compW preserve cover
  unfolding find-pivots-post-def by blast
qed

```

The next two lemmas explain the handoff from FindPivots to recursion. The theorem $\llbracket \text{bmssp-pre-full } ?d \text{ } ?S \text{ } ?B; \text{find-pivots-post } ?d \text{ } ?S \text{ } ?B \text{ } ?d' \text{ } ?P \text{ } ?W \rrbracket \implies \text{bmssp-pre-full } ?d' \text{ } ?P \text{ } ?B$ is the crucial invariant transfer: after FindPivots, the pivot set satisfies *bmssp-pre-full* under the updated labels. If a vertex was already outside the old source tree, the preservation clause of *find-pivots-post* keeps its label correct; if it was inside the tree and not directly completed, the cover clause delegates it through a complete pivot.

The conservative postcondition is a sanity instance of the same interface. It says the abstract FindPivots contract is not empty: keeping the same sources and naming the already-complete part of the tree is sufficient. The executable development uses stronger concrete pivot selection, but this lemma is helpful for understanding the shape of the contract independently of any particular implementation.

lemma *final-assembly-bound-tree*:

```

  assumes fp: find-pivots-post d S B d-fp P W
    and loop: partition-loop-post d-fp P B d' B' U-loop
    and asm: final-bmssp-assembly S B' W U-loop U
  shows U = bound-tree S B'

```

proof –

```

  have loop-post: bmssp-post-full d-fp P B d' B' U-loop
    using loop unfolding partition-loop-post-def .
  then have le: bound-le B' B and Uloop: U-loop = bound-tree P B'
    unfolding bmssp-post-full-def by auto
  have P-subset: P ⊆ S
    using fp unfolding find-pivots-post-def by blast
  have W-subset: W ⊆ bound-tree S B
    using fp unfolding find-pivots-post-def by blast

```

show *?thesis*

proof

```

  show U ⊆ bound-tree S B'

```

proof

```

  fix v

```

```

  assume v ∈ U

```

```

  then consider v ∈ U-loop | v ∈ W below-bound (dist s v) B'

```

```

    using asm unfolding final-bmssp-assembly-def by blast

```

```

  then show v ∈ bound-tree S B'

```

```

  proof cases

```

```

    case 1
    then have  $v \in \text{bound-tree } P \ B'$ 
      using Uloop by simp
    then show ?thesis
      using bound-tree-mono[OF P-subset, of B'] by blast
  next
    case 2
    then show ?thesis
      using W-subset unfolding bound-tree-def by blast
  qed
next
show bound-tree  $S \ B' \subseteq U$ 
proof
  fix  $v$ 
  assume  $v: v \in \text{bound-tree } S \ B'$ 
  then have  $v \in \text{bound-tree } S \ B$ 
    using bound-tree-bound-mono[OF le, of S] by blast
  then have cover:  $v \in W \vee \text{through-complete d-fp } P \ v$ 
    using fp unfolding find-pivots-post-def by blast
  then show  $v \in U$ 
  proof
    assume  $v \in W$ 
    moreover have below-bound (dist s v)  $B'$ 
      using  $v$  unfolding bound-tree-def by blast
    ultimately show ?thesis
      using asm unfolding final-bmssp-assembly-def by blast
  next
    assume through-complete d-fp  $P \ v$ 
    then have through  $P \ v$ 
      by (rule through-complete-imp-through)
    then have  $v \in \text{bound-tree } P \ B'$ 
      using  $v$  unfolding bound-tree-def by blast
    then have  $v \in U\text{-loop}$ 
      using Uloop by simp
    then show ?thesis
      using asm unfolding final-bmssp-assembly-def by blast
  qed
qed
qed
qed
qed

lemma final-assembly-complete:
  assumes fp: find-pivots-post  $d \ S \ B \ \text{d-fp } P \ W$ 
    and loop: partition-loop-post d-fp  $P \ B \ d' \ B' \ U\text{-loop}$ 
    and compW: complete-on  $d' \ W$ 
    and asm: final-bmssp-assembly  $S \ B' \ W \ U\text{-loop } U$ 
  shows complete-on  $d' \ U$ 
proof –

```

```

have complete-on  $d'$  U-loop
  using loop unfolding partition-loop-post-def bmssp-post-full-def by blast
moreover have  $\{v \in W. \text{below-bound } (\text{dist } s \ v) \ B'\} \subseteq W$ 
  by blast
then have complete-on  $d'$   $\{v \in W. \text{below-bound } (\text{dist } s \ v) \ B'\}$ 
  using complete-on-subset[OF compW] by blast
ultimately show ?thesis
  using asm unfolding final-bmssp-assembly-def complete-on-def by blast
qed

```

Final assembly is where the two solved regions are reconciled. The loop has solved *bound-tree* for the pivot set below the returned bound. The FindPivots phase may also have solved vertices in the caller's original tree; only those below the returned bound are retained in the caller's output set. Lemma $\llbracket \text{find-pivots-post } ?d \ ?S \ ?B \ ?d\text{-fp } ?P \ ?W; \text{partition-loop-post } ?d\text{-fp } ?P \ ?B \ ?d' \ ?B' \ ?U\text{-loop}; \text{final-bmssp-assembly } ?S \ ?B' \ ?W \ ?U\text{-loop } ?U \rrbracket \implies ?U = \text{bound-tree } ?S \ ?B'$ proves that this union is not merely contained in the desired set but exactly equals the caller's *bound-tree*. Lemma $\llbracket \text{find-pivots-post } ?d \ ?S \ ?B \ ?d\text{-fp } ?P \ ?W; \text{partition-loop-post } ?d\text{-fp } ?P \ ?B \ ?d' \ ?B' \ ?U\text{-loop}; \text{complete-on } ?d' \ ?W; \text{final-bmssp-assembly } ?S \ ?B' \ ?W \ ?U\text{-loop } ?U \rrbracket \implies \text{complete-on } ?d' \ ?U$ then proves that labels are complete on that exact set.

theorem *abstract-bmssp-correct:*

assumes *step: abstract-bmssp-step* $d \ S \ B \ d' \ B' \ U$

shows *bmssp-post-full* $d \ S \ B \ d' \ B' \ U$

proof –

obtain $d\text{-fp } P \ W \ U\text{-loop}$ **where**

fp: find-pivots-post $d \ S \ B \ d\text{-fp } P \ W$ **and**

loop: partition-loop-post $d\text{-fp } P \ B \ d' \ B' \ U\text{-loop}$ **and**

compW: complete-on $d' \ W$ **and**

asm: final-bmssp-assembly $S \ B' \ W \ U\text{-loop } U$

using *step unfolding* *abstract-bmssp-step-def* **by** *blast*

have *le: bound-le* $B' \ B$

using *loop unfolding* *partition-loop-post-def* *bmssp-post-full-def* **by** *blast*

have $U: U = \text{bound-tree } S \ B'$

using *final-assembly-bound-tree*[*OF fp loop asm*] .

have *comp: complete-on* $d' \ U$

using *final-assembly-complete*[*OF fp loop compW asm*] .

show *?thesis*

using *le U comp unfolding* *bmssp-post-full-def* **by** *blast*

qed

theorem *abstract-bmssp-correct-with-pre:*

assumes *pre: bmssp-pre-full* $d \ S \ B$

and *step: abstract-bmssp-step* $d \ S \ B \ d' \ B' \ U$

shows *bmssp-post-full* $d \ S \ B \ d' \ B' \ U$

using *abstract-bmssp-correct*[*OF step*] .

lemma *abstract-bmssp-step-exposes-pivot-pre:*

assumes *pre*: *bmssp-pre-full* *d S B*
and *step*: *abstract-bmssp-step* *d S B d' B' U*
obtains *d-fp P W U-loop* **where**
find-pivots-post *d S B d-fp P W*
bmssp-pre-full *d-fp P B*
partition-loop-post *d-fp P B d' B' U-loop*
complete-on *d' W*
final-bmssp-assembly *S B' W U-loop U*
proof –
obtain *d-fp P W U-loop* **where**
fp: *find-pivots-post* *d S B d-fp P W* **and**
loop: *partition-loop-post* *d-fp P B d' B' U-loop* **and**
compW: *complete-on* *d' W* **and**
asm: *final-bmssp-assembly* *S B' W U-loop U*
using *step* **unfolding** *abstract-bmssp-step-def* **by** *blast*
have *bmssp-pre-full* *d-fp P B*
using *find-pivots-establishes-pivot-pre*[*OF pre fp*] .
then show *thesis*
using *that fp loop compW asm* **by** *blast*
qed

corollary *abstract-bmssp-correct-weak*:
assumes *abstract-bmssp-step* *d S B d' B' U*
shows *bmssp-post* *d S B d' B' U*
using *bmssp-post-full-imp-post* *abstract-bmssp-correct* *assms* **by** *blast*

The theorem *abstract-bmssp-step* *?d ?S ?B ?d' ?B' ?U* $\implies$ *bmssp-post-full* *?d ?S ?B ?d' ?B' ?U* is the file's main assembly result: from an *abstract-bmssp-step* witness it extracts the FindPivots contract, the recursive partition-loop contract, and the final set assembly, then rebuilds *bmssp-post-full*. The variant $\llbracket \text{bmssp-pre-full } ?d ?S ?B; \text{abstract-bmssp-step } ?d ?S ?B ?d' ?B' ?U \rrbracket \implies \text{bmssp-post-full } ?d ?S ?B ?d' ?B' ?U$ keeps the precondition in its statement for clients whose proof scripts naturally carry it, while $\llbracket \text{bmssp-pre-full } ?d ?S ?B; \text{abstract-bmssp-step } ?d ?S ?B ?d' ?B' ?U; \bigwedge d\text{-fp } P W U\text{-loop}. \llbracket \text{find-pivots-post } ?d ?S ?B d\text{-fp } P W; \text{bmssp-pre-full } d\text{-fp } P ?B; \text{partition-loop-post } d\text{-fp } P ?B ?d' ?B' U\text{-loop}; \text{complete-on } ?d' W; \text{final-bmssp-assembly } ?S ?B' W U\text{-loop } ?U \rrbracket \implies ?thesis \rrbracket \implies ?thesis$ exposes the pivot precondition needed by the recursive implementation.

The final top-level theorems connect this abstract step to ordinary single-source shortest paths. Once the root call uses source set $\{s\}$ and bound *Infinity*, the root correctness lemmas from the entry-point theory turn the BMSSP postcondition into *sssp-correct*.

theorem *abstract-top-level-sssp-correct*:
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s v$
and *step*: *abstract-bmssp-step* *d {s} Infinity d' Infinity U*
shows *sssp-correct* *d'*
proof –

```

have bmssp-post  $d \{s\} \text{ Infinity } d' \text{ Infinity } U$ 
  using abstract-bmssp-correct-weak[OF step] .
then show ?thesis
  using successful-root-bmssp-correct[OF all-reachable] by blast
qed

```

```

theorem abstract-top-level-reachable-sssp-correct:
  assumes step: abstract-bmssp-step  $d \{s\} \text{ Infinity } d' \text{ Infinity } U$ 
  shows sssp-correct  $d'$ 
proof -
  have bmssp-post  $d \{s\} \text{ Infinity } d' \text{ Infinity } U$ 
    using abstract-bmssp-correct-weak[OF step] .
  then show ?thesis
    by (rule successful-root-bmssp-sssp-correct)
qed

```

```

theorem initialized-abstract-top-level-sssp-correct:
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
  and source-complete:  $d \ s = \text{dist } s \ s$ 
  and step: abstract-bmssp-step  $d \{s\} \text{ Infinity } d' \text{ Infinity } U$ 
  shows sssp-correct  $d'$ 
  using abstract-top-level-sssp-correct[OF all-reachable step] .

```

```

theorem initialized-abstract-top-level-reachable-sssp-correct:
  assumes source-complete:  $d \ s = \text{dist } s \ s$ 
  and step: abstract-bmssp-step  $d \{s\} \text{ Infinity } d' \text{ Infinity } U$ 
  shows sssp-correct  $d'$ 
  using abstract-top-level-reachable-sssp-correct[OF step] .

```

end

end

theory *BMSSP-Shortest-Path-Lemmas*

imports *BMSSP-Correctness*

begin

10 Shortest-Path Prefix Facts

The algorithm's tree-based correctness arguments repeatedly use the fact that vertices appearing earlier on a shortest path have no larger true distance. This theory proves that fact from the finite non-negative weighted graph model, without assuming uniqueness of shortest paths.

```

context finite-weighted-digraph
begin

```

```

lemma walk-nonempty:
  assumes walk  $p$ 
  shows  $p \neq []$ 

```

```

using assms by (cases p) auto

lemma walk-weight-nonneg:
  assumes walk p
  shows  $0 \leq \text{walk-weight } p$ 
  using assms
proof (induction p)
  case Nil
  then show ?case by simp
next
  case (Cons x ps)
  show ?case
  proof (cases ps)
    case Nil
    then show ?thesis by simp
  next
    case (Cons y ys)
    then have edge:  $(x, y) \in E$  and tail:  $\text{walk } (y \# ys)$ 
      using Cons.prems by auto
    have wx:  $0 \leq w \ x \ y$ 
      using edge nonneg-weight by auto
    have tail-nonneg:  $0 \leq \text{walk-weight } (y \# ys)$ 
      using Cons.IH tail Cons by simp
    show ?thesis
      using wx tail-nonneg Cons by simp
  qed
qed

lemma dist-nonneg:
  assumes reachable a b
  shows  $0 \leq \text{dist } a \ b$ 
proof -
  have dist a b  $\in \text{simple-walk-weights } a \ b$ 
    by (rule dist-is-simple-walk-weight[OF assms])
  then obtain p where p: simple-walk-betw a p b
    and dist-eq:  $\text{walk-weight } p = \text{dist } a \ b$ 
    unfolding simple-walk-weights-def by auto
  have walk p
    using p unfolding simple-walk-betw-def walk-betw-def by blast
  then have  $0 \leq \text{walk-weight } p$ 
    by (rule walk-weight-nonneg)
  then show ?thesis
    using dist-eq by simp
qed

lemma walk-take-Suc:
  assumes walk p i < length p
  shows walk (take (Suc i) p)
  using assms

```

```

proof (induction p arbitrary: i)
  case Nil
  then show ?case by simp
next
  case (Cons x ps)
  show ?case
  proof (cases ps)
    case Nil
    then show ?thesis
    using Cons by (cases i) auto
  next
  case (Cons y ys)
  then have walk-tail: walk (y # ys)
    using Cons.prems by auto
  show ?thesis
  proof (cases i)
    case 0
    then show ?thesis
    using Cons.prems Cons by auto
  next
  case (Suc j)
  then have j < length (y # ys)
    using Cons.prems Cons by simp
  then have walk (take (Suc j) (y # ys))
    using Cons.IH walk-tail Cons by simp
  then show ?thesis
    using Cons.prems Cons Suc by auto
  qed
qed
qed

lemma walk-weight-take-Suc-le:
  assumes walk p i < length p
  shows walk-weight (take (Suc i) p) ≤ walk-weight p
  using assms
proof (induction p arbitrary: i)
  case Nil
  then show ?case by simp
next
  case (Cons x ps)
  show ?case
  proof (cases ps)
    case Nil
    then show ?thesis
    using Cons by (cases i) auto
  next
  case (Cons y ys)
  then have edge: (x, y) ∈ E and walk-tail: walk (y # ys)
    using Cons.prems by auto

```

```

show ?thesis
proof (cases i)
  case 0
  have  $0 \leq w \ x \ y$ 
    using edge nonneg-weight by auto
  moreover have  $0 \leq \text{walk-weight } (y \ \# \ ys)$ 
    using walk-weight-nonneg[OF walk-tail] .
  ultimately show ?thesis
    using Cons 0 by auto
next
case (Suc j)
then have  $j < \text{length } (y \ \# \ ys)$ 
  using Cons.prem Cons by simp
have  $\text{walk-weight } (\text{take } (Suc \ j) \ (y \ \# \ ys)) \leq \text{walk-weight } (y \ \# \ ys)$ 
  using Cons.IH walk-tail j Cons by simp
then show ?thesis
  using Cons Suc by auto
qed
qed
qed

lemma simple-walk-prefix:
  assumes  $p$ : simple-walk-betw  $a \ p \ b$ 
  and  $i$ :  $i < \text{length } p$ 
  shows simple-walk-betw  $a \ (\text{take } (Suc \ i) \ p) \ (p \ ! \ i)$ 
proof -
  have nonempty:  $p \neq []$  and hd:  $\text{hd } p = a$  and walk-p: walk  $p$  and distinct-p:
distinct  $p$ 
  using p unfolding simple-walk-betw-def walk-betw-def by auto
  have take-nonempty:  $\text{take } (Suc \ i) \ p \neq []$ 
  using i by auto
  have  $\text{hd } (\text{take } (Suc \ i) \ p) = a$ 
  using hd i by (simp add: hd-take)
  moreover have  $\text{last } (\text{take } (Suc \ i) \ p) = p \ ! \ i$ 
proof -
  have len-take:  $\text{length } (\text{take } (Suc \ i) \ p) = Suc \ i$ 
  using i by simp
  have  $\text{last } (\text{take } (Suc \ i) \ p) =$ 
 $\text{take } (Suc \ i) \ p \ ! \ (\text{length } (\text{take } (Suc \ i) \ p) - 1)$ 
  using take-nonempty by (rule last-conv-nth)
  also have  $\dots = \text{take } (Suc \ i) \ p \ ! \ i$ 
  using len-take by simp
  finally have  $\text{last } (\text{take } (Suc \ i) \ p) = \text{take } (Suc \ i) \ p \ ! \ i$  .
  then show ?thesis
  using i by simp
qed
moreover have walk  $(\text{take } (Suc \ i) \ p)$ 
  using walk-take-Suc[OF walk-p i] .
moreover have distinct  $(\text{take } (Suc \ i) \ p)$ 

```

using *distinct-p* by *simp*
 ultimately show *?thesis*
 using *take-nonempty* **unfolding** *simple-walk-betw-def* *walk-betw-def* by *auto*
 qed

lemma *dist-le-simple-walk-weight*:
 assumes *p*: *simple-walk-betw* *a* *p* *b*
 shows *dist* *a* *b* $\leq$ *walk-weight* *p*
proof –
 have *mem*: *walk-weight* *p* $\in$ *simple-walk-weights* *a* *b*
 using *p* **unfolding** *simple-walk-weights-def* by *blast*
 have *fin*: *finite* (*simple-walk-weights* *a* *b*)
 using *finite-simple-walk-weights* .
 show *?thesis*
unfolding *dist-def* using *fin* *mem* by (rule *Min-le*)
 qed

lemma *shortest-walk-prefix-dist-le*:
 assumes *p*: *shortest-walk* *a* *p* *b*
 and *u*: *u* $\in$ *set* *p*
 shows *dist* *a* *u* $\leq$ *dist* *a* *b*
proof –
 have *simple-p*: *simple-walk-betw* *a* *p* *b*
 using *p* **unfolding** *shortest-walk-def* by *blast*
 then have *walk-p*: *walk* *p*
unfolding *simple-walk-betw-def* *walk-betw-def* by *auto*
 obtain *i* where *i*: *i* < *length* *p* $\wedge$ *i* = *u*
 using *u* by (auto *simp*: *in-set-conv-nth*)
 let *?q* = *take* (*Suc* *i*) *p*
 have *simple-q*: *simple-walk-betw* *a* *?q* *u*
 using *simple-walk-prefix*[*OF* *simple-p* *i*(1)] *i*(2) by *simp*
 have *dist* *a* *u* $\leq$ *walk-weight* *?q*
 using *dist-le-simple-walk-weight*[*OF* *simple-q*] .
 also have $\dots \leq$ *walk-weight* *p*
 using *walk-weight-take-Suc-le*[*OF* *walk-p* *i*(1)] .
 also have $\dots =$ *dist* *a* *b*
 using *p* **unfolding** *shortest-walk-def* by *simp*
 finally show *?thesis* .
 qed

lemma *through-witness-dist-le*:
 assumes *through* *S* *v*
 obtains *u* *p* where *u* $\in$ *S* *shortest-walk* *s* *p* *v* $\wedge$ *u* $\in$ *set* *p* $\wedge$ *dist* *s* *u* $\leq$ *dist* *s* *v*
proof –
 obtain *u* *p* where *u*: *u* $\in$ *S* *shortest-walk* *s* *p* *v* $\wedge$ *u* $\in$ *set* *p*
 using *assms* **unfolding** *through-def* by *blast*
 then have *dist* *s* *u* $\leq$ *dist* *s* *v*
 using *shortest-walk-prefix-dist-le* by *blast*
 then show *thesis*

using *that u* by *blast*
qed

lemma *walk-snoc*:

assumes *walk-p*: *walk p*
and *nonempty*: $p \neq []$
and *last-p*: *last p = u*
and *edge*: $(u, v) \in E$
shows *walk* ($p @ [v]$)
using *walk-p nonempty last-p edge*
proof (*induction p arbitrary: u*)
case *Nil*
then show ?*case* by *simp*

next

case (*Cons x ps*)
show ?*case*
proof (*cases ps*)
case *Nil*
then have *xu*: $x = u$
using *Cons.prem*s by *simp*
have *xv*: $(x, v) \in E$
using *Cons.prem*s *xu* by *simp*
have *xV*: $x \in V$
using *Cons.prem*s *Nil* by *simp*
have *vV*: $v \in V$
using *edge edge-in-V* by *blast*
show ?*thesis*
using *xV vV xv Nil* by *auto*

next

case (*Cons y ys*)
have *ps-walk*: *walk ps*
using *Cons.prem*s *Cons* by *auto*
have *ps-nonempty*: $ps \neq []$
using *Cons* by *simp*
have *ps-last*: *last ps = u*
using *Cons.prem*s *ps-nonempty* by *simp*
have *tail-edge*: $(u, v) \in E$
using *Cons.prem*s by *simp*
have *walk* ($ps @ [v]$)
using *Cons.IH*[*OF ps-walk ps-nonempty ps-last tail-edge*] .
then show ?*thesis*
using *Cons.prem*s *Cons* by *auto*

qed

qed

lemma *walk-weight-snoc*:

assumes $p \neq []$ *last p = u*
shows *walk-weight* ($p @ [v]$) = *walk-weight p* + *w u v*
using *assms*

```

proof (induction p arbitrary: u)
  case Nil
  then show ?case by simp
next
  case (Cons x ps)
  show ?case
  proof (cases ps)
    case Nil
    then show ?thesis
    using Cons by simp
  next
  case (Cons y ys)
  have ps-ne: ps  $\neq$  []
  using Cons by simp
  have ps-last: last ps = u
  using Cons.prems ps-ne by simp
  have walk-weight (ps @ [v]) = walk-weight ps + w u v
  using Cons.IH[OF ps-ne ps-last] .
  then show ?thesis
  using Cons by simp
qed
qed

```

```

lemma walk-append-tl:
  assumes walk-p: walk p
  and walk-q: walk q
  and p-ne: p  $\neq$  []
  and q-ne: q  $\neq$  []
  and join: last p = hd q
  shows walk (p @ tl q)
  using walk-p walk-q p-ne q-ne join
proof (induction p arbitrary: q)
  case Nil
  then show ?case by simp
next
  case (Cons x ps)
  show ?case
  proof (cases ps)
    case Nil
    have q-ne': q  $\neq$  []
    using Cons.prems by blast
    have walk-q': walk q
    using Cons.prems by blast
    have x = hd q
    using Cons.prems Nil by simp
    then have x # tl q = q
    using q-ne' by (cases q) auto
    then show ?thesis
    using walk-q' Nil by simp
  qed

```



```

next
  case (Cons y ys)
  have tail-walk: walk (y # ys)
    using Cons.premys Cons by auto
  have tail-walk-ps: walk ps
    using tail-walk Cons by simp
  have tail-ne: y # ys ≠ []
    by simp
  have tail-ne-ps: ps ≠ []
    using Cons by simp
  have tail-join: last (y # ys) = hd q
    using Cons.premys Cons by simp
  have tail-join-ps: last ps = hd q
    using tail-join Cons by simp
  have walk-q': walk q
    using Cons.premys by blast
  have q-ne': q ≠ []
    using Cons.premys by blast
  have walk (ps @ tl q)
    using Cons.IH[OF tail-walk-ps walk-q' tail-ne-ps q-ne' tail-join-ps] .
  then have walk ((y # ys) @ tl q)
    using Cons by simp
  then show ?thesis
    using Cons.premys Cons by auto
qed
qed

lemma walk-weight-append-tl:
  assumes p-ne: p ≠ []
  and q-ne: q ≠ []
  and join: last p = hd q
  shows walk-weight (p @ tl q) = walk-weight p + walk-weight q
  using p-ne q-ne join
proof (induction p arbitrary: q)
  case Nil
  then show ?case by simp
next
  case (Cons x ps)
  show ?case
  proof (cases ps)
    case Nil
    have q-ne': q ≠ []
      using Cons.premys by blast
    have x = hd q
      using Cons.premys Nil by simp
    then have x # tl q = q
      using q-ne' by (cases q) auto
    then show ?thesis
      using Nil by simp
  
```

```

next
  case (Cons y ys)
  have tail-ne: y # ys ≠ []
    by simp
  have tail-ne-ps: ps ≠ []
    using Cons by simp
  have tail-join: last (y # ys) = hd q
    using Cons.prem1 Cons by simp
  have tail-join-ps: last ps = hd q
    using tail-join Cons by simp
  have q-ne': q ≠ []
    using Cons.prem1 by blast
  have walk-weight (ps @ tl q) = walk-weight ps + walk-weight q
    using Cons.IH[OF tail-ne-ps q-ne' tail-join-ps] .
  then have walk-weight ((y # ys) @ tl q) =
    walk-weight (y # ys) + walk-weight q
    using Cons by simp
  then show ?thesis
    using Cons by simp
qed
qed

lemma walk-betw-append-tl:
  assumes p: walk-betw a p b
  and q: walk-betw b q c
  shows walk-betw a (p @ tl q) c
proof -
  have p-ne: p ≠ [] and q-ne: q ≠ []
    and hd-p: hd p = a and last-p: last p = b
    and hd-q: hd q = b and last-q: last q = c
    and walk-p: walk p and walk-q: walk q
    using p q unfolding walk-betw-def by auto
  have join: last p = hd q
    using last-p hd-q by simp
  have walk: walk (p @ tl q)
    using walk-append-tl[OF walk-p walk-q p-ne q-ne join] .
  have nonempty: p @ tl q ≠ []
    using p-ne by simp
  have hd: hd (p @ tl q) = a
    using hd-p p-ne by simp
  have last: last (p @ tl q) = c
  proof (cases tl q = [])
    case True
    then have q = [hd q]
      using q-ne by (cases q) auto
    then show ?thesis
      using last-p hd-q last-q by simp
  next
    case False

```

```

    then show ?thesis
      using last-q q-ne by (cases q) auto
  qed
  show ?thesis
    using nonempty hd last walk unfolding walk-betw-def by blast
qed

```

```

lemma walk-weight-append-tl-betw:
  assumes p: walk-betw a p b
    and q: walk-betw b q c
  shows walk-weight (p @ tl q) = walk-weight p + walk-weight q
proof -
  have p ≠ [] q ≠ [] last p = hd q
    using p q unfolding walk-betw-def by auto
  then show ?thesis
    by (rule walk-weight-append-tl)
qed

```

```

lemma walk-suffix-append:
  assumes walk (xs @ ys) ys ≠ []
  shows walk ys
  using assms
proof (induction xs)
  case Nil
    then show ?case by simp
  next
    case (Cons x xs)
      show ?case
      proof (cases xs)
        case Nil
          then show ?thesis
            using Cons by (cases ys) auto
        next
          case (Cons y zs)
            then have tail-walk: walk ((y # zs) @ ys)
              using Cons.prem by auto
            have tail-walk-ps: walk (xs @ ys)
              using tail-walk Cons by simp
            then show ?thesis
              using Cons.IH[OF tail-walk-ps Cons.prem(2)] by blast
      qed
    qed
qed

```

```

lemma walk-prefix-append:
  assumes walk (xs @ ys) xs ≠ []
  shows walk xs
  using assms
proof (induction xs arbitrary: ys)
  case Nil

```

```

    then show ?case by simp
next
  case (Cons x xs)
  show ?case
  proof (cases xs)
    case Nil
    then have  $x \in V$ 
      using Cons.prem by (cases xs) auto
    then show ?thesis
      using Nil by simp
  next
    case (Cons y zs)
    then have  $xV$ :  $x \in V$  and edge:  $(x, y) \in E$  and tail-walk:  $\text{walk } ((y \# zs) @$ 
 $ys)$ 
      using Cons.prem by auto
    have tail-walk-ps:  $\text{walk } (xs @ ys)$ 
      using tail-walk Cons by simp
    have tail-ne:  $xs \neq []$ 
      using Cons by simp
    have walk xs
      using Cons.IH[OF tail-walk-ps tail-ne] .
    then have walk  $(y \# zs)$ 
      using Cons by simp
    then show ?thesis
      using Cons  $xV$  edge by auto
  qed
qed

lemma walk-remove-cycle:
  assumes walk-p:  $\text{walk } (xs @ y \# ys @ y \# zs)$ 
  shows  $\text{walk } (xs @ y \# zs)$ 
proof -
  let ?pref =  $xs @ [y]$ 
  let ?cycle-suffix =  $y \# ys @ y \# zs$ 
  let ?suffix =  $y \# zs$ 
  have pref-walk:  $\text{walk } ?pref$ 
    using walk-prefix-append[of ?pref  $ys @ y \# zs$ ] walk-p by simp
  have suffix-walk:  $\text{walk } ?suffix$ 
    using walk-suffix-append[of  $xs @ y \# ys$  ?suffix] walk-p by simp
  have pref-ne:  $?pref \neq []$ 
    by simp
  have suffix-ne:  $?suffix \neq []$ 
    by simp
  have join:  $\text{last } ?pref = \text{hd } ?suffix$ 
    by simp
  have walk  $(?pref @ \text{tl } ?suffix)$ 
    using walk-append-tl[OF pref-walk suffix-walk pref-ne suffix-ne join] .
  then show ?thesis
    by simp

```

qed

lemma *walk-weight-remove-cycle-le*:

assumes *walk-p*: $\text{walk } (xs @ y \# ys @ y \# zs)$

shows $\text{walk-weight } (xs @ y \# zs) \leq \text{walk-weight } (xs @ y \# ys @ y \# zs)$

proof –

let *?pref* = $xs @ [y]$

let *?cycle* = $y \# ys @ [y]$

let *?suffix* = $y \# zs$

let *?cycle-suffix* = $y \# ys @ y \# zs$

have *pref-walk*: $\text{walk } ?pref$

using *walk-prefix-append*[of *?pref* *ys @ y \# zs*] *walk-p* **by** *simp*

have *cycle-suffix-walk*: $\text{walk } ?cycle\text{-suffix}$

using *walk-suffix-append*[of *xs* *?cycle-suffix*] *walk-p* **by** *simp*

have *cycle-walk*: $\text{walk } ?cycle$

using *walk-prefix-append*[of *?cycle* *zs*] *cycle-suffix-walk* **by** *simp*

have *suffix-walk*: $\text{walk } ?suffix$

using *walk-suffix-append*[of *xs @ y \# ys* *?suffix*] *walk-p* **by** *simp*

have *pref-ne*: $?pref \neq []$ **and** *cycle-suffix-ne*: $?cycle\text{-suffix} \neq []$

and *cycle-ne*: $?cycle \neq []$ **and** *suffix-ne*: $?suffix \neq []$

by *simp-all*

have *pref-join-cycle-suffix*: $\text{last } ?pref = \text{hd } ?cycle\text{-suffix}$

by *simp*

have *pref-join-suffix*: $\text{last } ?pref = \text{hd } ?suffix$

by *simp*

have *cycle-join-suffix*: $\text{last } ?cycle = \text{hd } ?suffix$

by *simp*

have *new-weight*:

$\text{walk-weight } (xs @ y \# zs) = \text{walk-weight } ?pref + \text{walk-weight } ?suffix$

using *walk-weight-append-tl*[OF *pref-ne* *suffix-ne* *pref-join-suffix*] **by** *simp*

have *original-weight*:

$\text{walk-weight } (xs @ y \# ys @ y \# zs) =$

$\text{walk-weight } ?pref + \text{walk-weight } ?cycle\text{-suffix}$

using *walk-weight-append-tl*[OF *pref-ne* *cycle-suffix-ne* *pref-join-cycle-suffix*] **by**

simp

have *cycle-suffix-weight*:

$\text{walk-weight } ?cycle\text{-suffix} = \text{walk-weight } ?cycle + \text{walk-weight } ?suffix$

using *walk-weight-append-tl*[OF *cycle-ne* *suffix-ne* *cycle-join-suffix*] **by** *simp*

have *cycle-nonneg*: $0 \leq \text{walk-weight } ?cycle$

using *walk-weight-nonneg*[OF *cycle-walk*] .

show *?thesis*

using *new-weight* *original-weight* *cycle-suffix-weight* *cycle-nonneg* **by** *linarith*

qed

lemma *walk-betw-remove-cycle*:

assumes *p*: $\text{walk-betw } a (xs @ y \# ys @ y \# zs) b$

shows $\text{walk-betw } a (xs @ y \# zs) b$

proof –

let *?p* = $xs @ y \# ys @ y \# zs$

```

let ?p' = xs @ y # zs
have walk-p: walk ?p
  using p unfolding walk-betw-def by blast
have walk-p': walk ?p'
  using walk-remove-cycle[OF walk-p] .
have nonempty: ?p' ≠ []
  by simp
have hd-p: hd ?p = a and last-p: last ?p = b
  using p unfolding walk-betw-def by auto
have hd-p': hd ?p' = a
  using hd-p by (cases xs) auto
have last-p': last ?p' = b
  using last-p by (cases zs) auto
show ?thesis
  using nonempty hd-p' last-p' walk-p' unfolding walk-betw-def by blast
qed

```

```

lemma walk-betw-to-simple-walk-le-exists:
  assumes p: walk-betw a p b
  shows  $\exists q. \text{simple-walk-betw } a \ q \ b \wedge \text{walk-weight } q \leq \text{walk-weight } p$ 
  using p
proof (induction length p arbitrary: p rule: less-induct)
  case less
  show ?case
  proof (cases distinct p)
    case True
    then have simple-walk-betw a p b
      using less.prem1 unfolding simple-walk-betw-def by blast
    then show ?thesis
      by blast
  next
    case False
    then obtain xs ys zs y where decomp:  $p = xs @ [y] @ ys @ [y] @ zs$ 
      using not-distinct-decomp by blast
    let ?p' = xs @ y # zs
    have p-form:  $p = xs @ y \# ys @ y \# zs$ 
      using decomp by simp
    have p'-betw: walk-betw a ?p' b
      using walk-betw-remove-cycle[of a xs y ys zs b] less.prem1 p-form by simp
    have p'-shorter:  $\text{length } ?p' < \text{length } p$ 
      using p-form by simp
    obtain q where q: simple-walk-betw a q b walk-weight q ≤ walk-weight ?p'
      using less.hyps[OF p'-shorter p'-betw] by blast
    have walk-weight ?p' ≤ walk-weight p
      using walk-weight-remove-cycle-le[of xs y ys zs] less.prem1 p-form
      unfolding walk-betw-def by simp
    then have walk-weight q ≤ walk-weight p
      using q(2) by linarith
    then show ?thesis

```

using $q(1)$ by blast
 qed
 qed

lemma *walk-betw-to-simple-walk-le*:
 assumes p : *walk-betw* a p b
 obtains q **where** *simple-walk-betw* a q b *walk-weight* $q \leq$ *walk-weight* p
 using *walk-betw-to-simple-walk-le-exists*[*OF* p] **that** by blast

lemma *walk-suffix-from-index*:
 assumes p : *simple-walk-betw* a p b
 and i : $i < \text{length } p$
 shows *walk-betw* $(p ! i)$ $(\text{drop } i \ p)$ b
proof –
 have *walk-p*: *walk* p **and** *last-p*: *last* $p = b$
 using p **unfolding** *simple-walk-betw-def* *walk-betw-def* **by** *auto*
 have *drop-ne*: *drop* $i \ p \neq []$
 using i **by** *auto*
 have *hd-drop*: *hd* $(\text{drop } i \ p) = p ! i$
 using i **by** (*simp add: hd-drop-conv-nth*)
 have *last-drop*: *last* $(\text{drop } i \ p) = b$
 using *last-p* i **by** (*simp add: last-drop*)
 have *p-decomp*: $p = \text{take } i \ p @ \text{drop } i \ p$
by *simp*
 have *walk-drop*: *walk* $(\text{drop } i \ p)$
 using *walk-suffix-append*[*of* $\text{take } i \ p \ \text{drop } i \ p$] *walk-p* *drop-ne* *p-decomp* **by** *simp*
 show *?thesis*
 using *drop-ne* *hd-drop* *last-drop* *walk-drop* **unfolding** *walk-betw-def* **by** *blast*
 qed

lemma *take-Suc-append-tl-drop*:
 assumes $i < \text{length } p$
 shows *take* $(\text{Suc } i) \ p @ \text{tl } (\text{drop } i \ p) = p$
proof –
 have *drop-i*: *drop* $i \ p = p ! i \# \text{drop } (\text{Suc } i) \ p$
 using *Cons-nth-drop-Suc*[*OF* *assms*] **by** *simp*
 have *take* $(\text{Suc } i) \ p @ \text{tl } (\text{drop } i \ p) =$
 $\text{take } i \ p @ [p ! i] @ \text{drop } (\text{Suc } i) \ p$
 using *assms* *drop-i* **by** (*simp add: take-Suc-conv-app-nth*)
 also have $\dots = \text{take } i \ p @ \text{drop } i \ p$
 using *drop-i* **by** *simp*
 finally show *?thesis*
by *simp*
 qed

lemma *shortest-walk-prefix-shortest*:
 assumes sp : *shortest-walk* a p b
 and i : $i < \text{length } p$
 shows *shortest-walk* a $(\text{take } (\text{Suc } i) \ p)$ $(p ! i)$

```

proof –
  let ?u = p ! i
  let ?pref = take (Suc i) p
  let ?suf = drop i p
  have simple-p: simple-walk-betw a p b
    using sp unfolding shortest-walk-def by blast
  have weight-p: walk-weight p = dist a b
    using sp unfolding shortest-walk-def by blast
  have pref-simple: simple-walk-betw a ?pref ?u
    using simple-walk-prefix[OF simple-p i] .
  have pref-walk-betw: walk-betw a ?pref ?u
    using pref-simple unfolding simple-walk-betw-def by blast
  have suf-walk-betw: walk-betw ?u ?suf b
    using walk-suffix-from-index[OF simple-p i] .
  have split-p: ?pref @ tl ?suf = p
    using take-Suc-append-tl-drop[OF i] .
  have weight-split: walk-weight p = walk-weight ?pref + walk-weight ?suf
    using walk-weight-append-tl-betw[OF pref-walk-betw suf-walk-betw] split-p by
simp
  have dist-pref-le: dist a ?u ≤ walk-weight ?pref
    using dist-le-simple-walk-weight[OF pref-simple] .
  have no-strict: ¬ dist a ?u < walk-weight ?pref
proof
  assume strict: dist a ?u < walk-weight ?pref
  have reach-u: reachable a ?u
    using pref-simple unfolding reachable-def by blast
  obtain r where r-short: shortest-walk a r ?u
    using shortest-walk-exists[OF reach-u] by blast
  have r-simple: simple-walk-betw a r ?u
    using r-short unfolding shortest-walk-def by blast
  have r-walk-betw: walk-betw a r ?u
    using r-simple unfolding simple-walk-betw-def by blast
  have r-weight: walk-weight r = dist a ?u
    using r-short unfolding shortest-walk-def by blast
  have concat-walk: walk-betw a (r @ tl ?suf) b
    using walk-betw-append-tl[OF r-walk-betw suf-walk-betw] .
  have concat-weight: walk-weight (r @ tl ?suf) = walk-weight r + walk-weight
?suf
    using walk-weight-append-tl-betw[OF r-walk-betw suf-walk-betw] .
  obtain q where q: simple-walk-betw a q b
    walk-weight q ≤ walk-weight (r @ tl ?suf)
    using walk-betw-to-simple-walk-le[OF concat-walk] by blast
  have walk-weight (r @ tl ?suf) < walk-weight p
    using strict r-weight weight-split concat-weight by linarith
  then have walk-weight q < dist a b
    using q(2) weight-p by linarith
  moreover have dist a b ≤ walk-weight q
    using dist-le-simple-walk-weight[OF q(1)] .
  ultimately show False

```



```

    by linarith
qed
have walk-weight ?pref = dist a ?u
  using dist-pref-le no-strict by linarith
then show ?thesis
  using pref-simple unfolding shortest-walk-def by blast
qed

```

```

lemma walk-nth-edge:
  assumes walk p
  and Suc i < length p
  shows (p ! i, p ! Suc i) ∈ E
  using assms
proof (induction p arbitrary: i)
  case Nil
  then show ?case by simp
next
  case (Cons x xs)
  show ?case
  proof (cases i)
    case 0
    then show ?thesis
      using Cons.prem1 by (cases xs) auto
  next
    case (Suc j)
    then have Suc j < length xs
      using Cons.prem1 by simp
    moreover have walk xs
      using Cons.prem1 Suc by (cases xs) auto
    ultimately have (xs ! j, xs ! Suc j) ∈ E
      using Cons.IH by blast
    then show ?thesis
      using Suc by simp
  qed
qed

```

```

lemma simple-walk-snoc:
  assumes p: simple-walk-betw s p u
  and edge: (u, v) ∈ E
  and fresh: v ∉ set p
  shows simple-walk-betw s (p @ [v]) v
proof -
  have nonempty: p ≠ [] and hd: hd p = s and last-p: last p = u
  and walk-p: walk p and distinct-p: distinct p
  using p unfolding simple-walk-betw-def walk-betw-def by auto
  have walk (p @ [v])
  using walk-snoc[OF walk-p nonempty last-p edge] .
  moreover have p @ [v] ≠ [] hd (p @ [v]) = s last (p @ [v]) = v distinct (p @ [v])
  using nonempty hd fresh distinct-p by auto

```

```

ultimately show ?thesis
  unfolding simple-walk-betw-def walk-betw-def by auto
qed

lemma dist-triangle-edge:
  assumes edge:  $(u, v) \in E$ 
  and reach-u: reachable  $s\ u$ 
  shows reachable  $s\ v$   $\text{dist } s\ v \leq \text{dist } s\ u + w\ u\ v$ 
proof -
  obtain  $p$  where  $p$ : shortest-walk  $s\ p\ u$ 
  using reach-u by (rule shortest-walk-exists)
  have simple-p: simple-walk-betw  $s\ p\ u$ 
  using  $p$  unfolding shortest-walk-def by blast
  have weight-p: walk-weight  $p = \text{dist } s\ u$ 
  using  $p$  unfolding shortest-walk-def by blast
  show reach-v: reachable  $s\ v$ 
proof (cases  $v \in \text{set } p$ )
  case True
  then obtain  $i$  where  $i$ :  $i < \text{length } p \wedge i = v$ 
  by (auto simp: in-set-conv-nth)
  have simple-walk-betw  $s\ (\text{take } (\text{Suc } i)\ p)\ v$ 
  using simple-walk-prefix[OF simple-p  $i(1)$ ]  $i(2)$  by simp
  then show ?thesis
  unfolding reachable-def by blast
next
  case False
  have simple-walk-betw  $s\ (p @ [v])\ v$ 
  using simple-walk-snoc[OF simple-p edge False] .
  then show ?thesis
  unfolding reachable-def by blast
qed
show  $\text{dist } s\ v \leq \text{dist } s\ u + w\ u\ v$ 
proof (cases  $v \in \text{set } p$ )
  case True
  have  $\text{dist } s\ v \leq \text{dist } s\ u$ 
  using shortest-walk-prefix-dist-le[OF  $p$  True] .
  moreover have  $0 \leq w\ u\ v$ 
  using edge nonneg-weight by auto
  ultimately show ?thesis
  by linarith
next
  case False
  have simple-q: simple-walk-betw  $s\ (p @ [v])\ v$ 
  using simple-walk-snoc[OF simple-p edge False] .
  have  $\text{dist } s\ v \leq \text{walk-weight } (p @ [v])$ 
  using dist-le-simple-walk-weight[OF simple-q] .
  also have  $\dots = \text{dist } s\ u + w\ u\ v$ 
  using simple-p weight-p unfolding simple-walk-betw-def walk-betw-def
  by (simp add: walk-weight-snoc)

```

finally show ?thesis .
qed
qed

definition *sound-label* **where**
sound-label $d \longleftrightarrow (\forall v \in V. \text{reachable } s \ v \longrightarrow \text{dist } s \ v \leq d \ v)$

definition *relax-edge* **where**
relax-edge $d \ u \ v = (\lambda x. \text{if } x = v \text{ then } \min (d \ v) (d \ u + w \ u \ v) \text{ else } d \ x)$

definition *tight-edge-step* **where**
tight-edge-step $u \ v \longleftrightarrow$
 $(u, v) \in E \wedge \text{reachable } s \ u \wedge \text{dist } s \ v = \text{dist } s \ u + w \ u \ v$

lemma *relax-edge-le*:
relax-edge $d \ u \ v \ x \leq d \ x$
unfolding *relax-edge-def* **by** *simp*

lemma *shortest-walk-successively-tight*:
assumes *sp*: *shortest-walk* $s \ p \ v$
shows *successively tight-edge-step* p
unfolding *successively-conv-nth*
proof *clarify*
fix i
assume i : $\text{Suc } i < \text{length } p$
have *simple-p*: *simple-walk-betw* $s \ p \ v$
using *sp* **unfolding** *shortest-walk-def* **by** *blast*
have *walk-p*: *walk* p
using *simple-p* **unfolding** *simple-walk-betw-def* *walk-betw-def* **by** *blast*
have *edge*: $(p \ ! \ i, p \ ! \ \text{Suc } i) \in E$
using *walk-nth-edge*[*OF walk-p i*] .
have *pref-i*: *shortest-walk* $s \ (\text{take } (\text{Suc } i) \ p) \ (p \ ! \ i)$
using *shortest-walk-prefix-shortest*[*OF sp*] i **by** *simp*
have *pref-Suc-i*: *shortest-walk* $s \ (\text{take } (\text{Suc } (\text{Suc } i)) \ p) \ (p \ ! \ \text{Suc } i)$
using *shortest-walk-prefix-shortest*[*OF sp i*] .
have *reach-i*: *reachable* $s \ (p \ ! \ i)$
using *pref-i* **unfolding** *shortest-walk-def* *reachable-def* **by** *blast*
have *pref-i-ne*: $\text{take } (\text{Suc } i) \ p \neq []$
using i **by** (*cases p*) *auto*
have *last-pref-i*: $\text{last } (\text{take } (\text{Suc } i) \ p) = p \ ! \ i$
proof –
have *len-take*: $\text{length } (\text{take } (\text{Suc } i) \ p) = \text{Suc } i$
using i **by** *simp*
have *last* $(\text{take } (\text{Suc } i) \ p) = \text{take } (\text{Suc } i) \ p \ ! \ i$
using *last-conv-nth*[*OF pref-i-ne*] *len-take* **by** *simp*
then show ?thesis
using i **by** *simp*
qed
have *take-next*:

```

    take (Suc (Suc i)) p = take (Suc i) p @ [p ! Suc i]
    using i by (simp add: take-Suc-conv-app-nth)
  have weight-step:
    walk-weight (take (Suc (Suc i)) p) =
      walk-weight (take (Suc i) p) + w (p ! i) (p ! Suc i)
    using walk-weight-snoc[OF pref-i-ne last-pref-i, of p ! Suc i] take-next by simp
  have dist-i: walk-weight (take (Suc i) p) = dist s (p ! i)
    using pref-i unfolding shortest-walk-def by blast
  have dist-Suc-i: walk-weight (take (Suc (Suc i)) p) = dist s (p ! Suc i)
    using pref-Suc-i unfolding shortest-walk-def by blast
  show tight-edge-step (p ! i) (p ! Suc i)
    using edge reach-i weight-step dist-i dist-Suc-i
    unfolding tight-edge-step-def by simp
qed

lemma relax-edge-sound:
  assumes sound: sound-label d
    and edge: (u, v) ∈ E
    and reach-u: reachable s u
  shows sound-label (relax-edge d u v)
proof -
  have uV: u ∈ V
    using edge edge-in-V by blast
  have dist-u-le: dist s u ≤ d u
    using sound uV reach-u unfolding sound-label-def by blast
  have triangle: dist s v ≤ dist s u + w u v
    using dist-triangle-edge[OF edge reach-u] by blast
  show ?thesis
    unfolding sound-label-def relax-edge-def
  proof clarify
    fix x
    assume xV: x ∈ V and reach-x: reachable s x
    show dist s x ≤ (if x = v then min (d v) (d u + w u v) else d x)
    proof (cases x = v)
      case True
      have dist s v ≤ d v
        using sound xV reach-x True unfolding sound-label-def by blast
      moreover have dist s v ≤ d u + w u v
        using dist-u-le triangle by linarith
      ultimately show ?thesis
        using True by simp
    next
      case False
      then show ?thesis
        using sound xV reach-x unfolding sound-label-def by simp
    qed
  qed
qed

```

lemma *relax-tight-edge-completes*:
assumes *sound*: *sound-label* *d*
and *complete-u*: $d\ u = \text{dist}\ s\ u$
and *tight*: *tight-edge-step* *u v*
shows *sound-label* (*relax-edge* *d u v*)
and *relax-edge* *d u v v* = *dist s v*
proof –
have *edge*: $(u, v) \in E$ **and** *reach-u*: *reachable* *s u*
and *dist-v*: $\text{dist}\ s\ v = \text{dist}\ s\ u + w\ u\ v$
using *tight* **unfolding** *tight-edge-step-def* **by** *auto*
show *sound-label* (*relax-edge* *d u v*)
using *relax-edge-sound*[*OF sound edge reach-u*] **.**
have *vV*: $v \in V$
using *edge edge-in-V* **by** *blast*
have *reach-v*: *reachable* *s v*
using *dist-triangle-edge*[*OF edge reach-u*] **by** *blast*
have $\text{dist}\ s\ v \leq d\ v$
using *sound vV reach-v* **unfolding** *sound-label-def* **by** *blast*
then show *relax-edge* *d u v v* = *dist s v*
using *complete-u dist-v* **unfolding** *relax-edge-def* **by** *simp*
qed

fun *relax-path* **where**
relax-path *d []* = *d*
| *relax-path* *d [x]* = *d*
| *relax-path* *d (x # y # xs)* = *relax-path* (*relax-edge* *d x y*) (*y # xs*)

fun *relax-edges* **where**
relax-edges *d []* = *d*
| *relax-edges* *d ((u, v) # es)* = *relax-edges* (*relax-edge* *d u v*) *es*

fun *path-edges* **where**
path-edges [] = []
| *path-edges* [*x*] = []
| *path-edges* (*x # y # xs*) = (*x, y*) # *path-edges* (*y # xs*)

lemma *relax-edges-append*:
relax-edges *d (xs @ ys)* = *relax-edges* (*relax-edges* *d xs*) *ys*
by (*induction xs arbitrary: d*) (*auto split: prod.splits*)

lemma *relax-edges-le*:
relax-edges *d es x* $\leq d\ x$
proof (*induction es arbitrary: d*)
case *Nil*
then show ?*case* **by** *simp*
next
case (*Cons e es*)
obtain *u v* **where** *e*: $e = (u, v)$
by *force*

have $\text{relax-edges } (\text{relax-edge } d \ u \ v) \text{ es } x \leq \text{relax-edge } d \ u \ v \ x$
by (rule *Cons.IH*)
also have $\dots \leq d \ x$
by (rule *relax-edge-le*)
finally show ?case
unfolding e **by** *simp*
qed

lemma *relax-path-eq-relax-edges*:
 $\text{relax-path } d \ p = \text{relax-edges } d \ (\text{path-edges } p)$
by (induction p arbitrary: d rule: *relax-path.induct*) *simp-all*

lemma *relax-edge-preserves-other*:
assumes $x \neq v$
shows $\text{relax-edge } d \ u \ v \ x = d \ x$
using *assms* **unfolding** *relax-edge-def* **by** *simp*

lemma *relax-edge-preserves-complete-other*:
assumes $x \neq v \ d \ x = \text{dist } s \ x$
shows $\text{relax-edge } d \ u \ v \ x = \text{dist } s \ x$
using *assms* **by** (*simp add: relax-edge-preserves-other*)

lemma *relax-edge-preserves-complete-sound*:
assumes *sound: sound-label d*
and *complete-x: d x = dist s x*
and *edge: (u, v) ∈ E*
and *reach-u: reachable s u*
shows $\text{relax-edge } d \ u \ v \ x = \text{dist } s \ x$
proof (*cases x = v*)
case *False*
then show ?thesis
using *complete-x* **by** (*simp add: relax-edge-preserves-other*)
next
case *True*
have $vV: v \in V$
using *edge edge-in-V* **by** *blast*
have *reach-v: reachable s v*
using *dist-triangle-edge[OF edge reach-u]* **by** *blast*
have *dist-v-le-dv: dist s v ≤ d v*
using *sound vV reach-v* **unfolding** *sound-label-def* **by** *blast*
have *dist-v-le-candidate: dist s v ≤ d u + w u v*
proof –
have $uV: u \in V$
using *edge edge-in-V* **by** *blast*
have $\text{dist } s \ u \leq d \ u$
using *sound uV reach-u* **unfolding** *sound-label-def* **by** *blast*
moreover have $\text{dist } s \ v \leq \text{dist } s \ u + w \ u \ v$
using *dist-triangle-edge[OF edge reach-u]* **by** *blast*
ultimately show ?thesis

```

    by linarith
qed
show ?thesis
  using True complete-x dist-v-le-dv dist-v-le-candidate unfolding relax-edge-def
by simp
qed

```

```

lemma relax-edges-sound:
  assumes sound: sound-label d
    and edges:  $\bigwedge u v. (u, v) \in \text{set } es \implies (u, v) \in E$ 
    and reaches:  $\bigwedge u v. (u, v) \in \text{set } es \implies \text{reachable } s u$ 
  shows sound-label (relax-edges d es)
  using sound edges reaches
proof (induction es arbitrary: d)
  case Nil
  then show ?case by simp
next
  case (Cons e es)
  obtain u v where e:  $e = (u, v)$ 
  by force
  have edge:  $(u, v) \in E$ 
  using Cons.prem1 e by auto
  have reach:  $\text{reachable } s u$ 
  using Cons.prem2 e by auto
  have sound1: sound-label (relax-edge d u v)
  using relax-edge-sound[OF Cons.prem1 edge reach] .
  have edges-tail:  $\bigwedge a b. (a, b) \in \text{set } es \implies (a, b) \in E$ 
  using Cons.prem3 by auto
  have reaches-tail:  $\bigwedge a b. (a, b) \in \text{set } es \implies \text{reachable } s a$ 
  using Cons.prem4 by auto
  show ?case
  using Cons.IH[OF sound1 edges-tail reaches-tail] e by simp
qed

```

```

lemma relax-edges-preserves-complete-if-not-targeted:
  assumes complete-x:  $d x = \text{dist } s x$ 
    and not-target:  $\bigwedge u. (u, x) \notin \text{set } es$ 
  shows relax-edges d es  $x = \text{dist } s x$ 
  using complete-x not-target
proof (induction es arbitrary: d)
  case Nil
  then show ?case by simp
next
  case (Cons e es)
  obtain u v where e:  $e = (u, v)$ 
  by force
  have x-ne-v:  $x \neq v$ 
  using Cons.prem1[of u] e by auto
  have complete-after: relax-edge d u v  $x = \text{dist } s x$ 

```

```

    using x-ne-v Cons.premis(1) unfolding relax-edge-def by simp
  have not-target-tail:  $\bigwedge a. (a, x) \notin \text{set } es$ 
    using Cons.premis e by auto
  have relax-edges (relax-edge d u v) es x = dist s x
    using Cons.IH[of relax-edge d u v] complete-after not-target-tail by blast
  show ?case
    using <relax-edges (relax-edge d u v) es x = dist s x> e by simp
qed

```

lemma *relax-edges-preserves-complete-sound*:

```

  assumes sound: sound-label d
    and complete-x: d x = dist s x
    and edges:  $\bigwedge u v. (u, v) \in \text{set } es \implies (u, v) \in E$ 
    and reaches:  $\bigwedge u v. (u, v) \in \text{set } es \implies \text{reachable } s u$ 
  shows relax-edges d es x = dist s x
    using sound complete-x edges reaches
  proof (induction es arbitrary: d)
    case Nil
    then show ?case by simp
  next
    case (Cons e es)
    obtain u v where e: e = (u, v)
      by force
    have edge: (u, v)  $\in E$ 
      using Cons.premis e by auto
    have reach: reachable s u
      using Cons.premis e by auto
    have sound1: sound-label (relax-edge d u v)
      using relax-edge-sound[OF Cons.premis(1) edge reach] .
    have complete-after: relax-edge d u v x = dist s x
      using relax-edge-preserves-complete-sound[OF Cons.premis(1) Cons.premis(2)
edge reach] .
    have edges-tail:  $\bigwedge a b. (a, b) \in \text{set } es \implies (a, b) \in E$ 
      using Cons.premis by auto
    have reaches-tail:  $\bigwedge a b. (a, b) \in \text{set } es \implies \text{reachable } s a$ 
      using Cons.premis by auto
    have relax-edges (relax-edge d u v) es x = dist s x
      using Cons.IH[OF sound1 complete-after edges-tail reaches-tail] .
    then show ?case
      using e by simp
  qed

```

lemma *relax-path-tight-sound-complete*:

```

  assumes nonempty: p  $\neq []$ 
    and sound: sound-label d
    and complete-hd: d (hd p) = dist s (hd p)
    and tight: successively tight-edge-step p
  shows sound-label (relax-path d p)
    and relax-path d p (last p) = dist s (last p)

```



```

proof –
  have combined:  $\bigwedge d. p \neq [] \implies \text{sound-label } d \implies$ 
     $d (\text{hd } p) = \text{dist } s (\text{hd } p) \implies \text{successively tight-edge-step } p \implies$ 
     $\text{sound-label } (\text{relax-path } d \ p) \wedge$ 
     $\text{relax-path } d \ p (\text{last } p) = \text{dist } s (\text{last } p)$ 
  proof (induction p)
    case Nil
    then show ?case by simp
  next
    case (Cons x ps)
    show ?case
    proof (cases ps = [])
      case True
      then show ?thesis
      using Cons.prems by simp
    next
      case False
      then obtain y ys where ps = y # ys
      by (cases ps) auto
      have step: tight-edge-step x y
      using Cons.prems ps by (simp add: successively-Cons)
      have tight-tail: successively tight-edge-step (y # ys)
      using Cons.prems ps by (simp add: successively-Cons)
      let ?d1 = relax-edge d x y
      have complete-x:  $d \ x = \text{dist } s \ x$ 
      using Cons.prems by simp
      have sound1: sound-label ?d1
      using relax-tight-edge-completes(1)[OF Cons.prems(2) complete-x step] .
      have complete-y:  $?d1 \ y = \text{dist } s \ y$ 
      using relax-tight-edge-completes(2)[OF Cons.prems(2) complete-x step] .
      have complete-hd-ps:  $?d1 \ (\text{hd } ps) = \text{dist } s \ (\text{hd } ps)$ 
      using complete-y ps by simp
      have tight-ps: successively tight-edge-step ps
      using tight-tail ps by simp
      have tail: sound-label (relax-path ?d1 ps)  $\wedge$ 
        relax-path ?d1 ps (last ps) = dist s (last ps)
      using Cons.IH[of ?d1] False sound1 complete-hd-ps tight-ps by blast
      then show ?thesis
      using ps by simp
    qed
  qed
  then show sound-label (relax-path d p)
  using assms by blast
  from combined show relax-path d p (last p) = dist s (last p)
  using assms by blast
qed

```

lemma *relax-edges-with-tight-path-prefix-complete*:
assumes nonempty: p $\neq []$

```

    and sound: sound-label d
    and complete-hd: d (hd p) = dist s (hd p)
    and tight: successively tight-edge-step p
    and es: es = path-edges p @ extra
    and extra-edges:  $\bigwedge u v. (u, v) \in \text{set extra} \implies (u, v) \in E$ 
    and extra-reaches:  $\bigwedge u v. (u, v) \in \text{set extra} \implies \text{reachable } s \ u$ 
    shows sound-label (relax-edges d es)
    and relax-edges d es (last p) = dist s (last p)
  proof -
    let ?d1 = relax-edges d (path-edges p)
    have path-eq: ?d1 = relax-path d p
    using relax-path-eq-relax-edges[of d p] by simp
    have sound1: sound-label ?d1
    using relax-path-tight-sound-complete(1)[OF nonempty sound complete-hd tight]
  path-eq by simp
    have complete-last: ?d1 (last p) = dist s (last p)
    using relax-path-tight-sound-complete(2)[OF nonempty sound complete-hd tight]
  path-eq by simp
    have full: relax-edges d es = relax-edges ?d1 extra
    using es relax-edges-append[of d path-edges p extra] by simp
    show sound-label (relax-edges d es)
    using relax-edges-sound[OF sound1 extra-edges extra-reaches] full by simp
    show relax-edges d es (last p) = dist s (last p)
    using relax-edges-preserves-complete-sound[OF sound1 complete-last extra-edges
  extra-reaches] full by simp
  qed

end

end

theory BMSSP-Base-Case
  imports BMSSP-Algorithm-Correctness BMSSP-Shortest-Path-Lemmas
begin

```

11 Concrete Base Case

The algorithm's level-0 BMSSP call is a bounded Dijkstra-style search from a singleton source. It explores the bounded tree below the input bound and stops once the next extracted distance would exceed the first k vertices. Operationally, the paper describes this with a priority queue. For correctness, only one property of that queue matters here: vertices are extracted in nondecreasing true distance order.

This theory turns that observation into a compact semantic base case. Since the graph is finite, the bounded tree can be listed in sorted distance order. If the list has at most k vertices, the base case completes the whole bounded tree and returns the original bound. If the list is longer, the k -th distance becomes the returned finite bound and the completed set is exactly

the part of the bounded tree strictly below that bound. The final lemma packages this as a BMSSP postcondition without assuming any recursive postcondition for the base case itself.

context *finite-weighted-digraph*
begin

definition *min-dist-vertex* :: 'v set $\Rightarrow$ 'v **where**
min-dist-vertex A = (SOME x. x $\in$ A $\wedge$ dist s x = Min (dist s ' A))

lemma *finite-bound-tree* [simp]: *finite* (bound-tree S B)
unfolding *bound-tree-def* **using** *finite-V* **by** *auto*

lemma *min-dist-vertex-prop*:
assumes *finite* A A $\neq$ {}
shows *min-dist-vertex* A $\in$ A $\wedge$ dist s (*min-dist-vertex* A) = Min (dist s ' A)
proof –
obtain a **where** a $\in$ A
using *assms* **by** *auto*
then have Min (dist s ' A) $\in$ dist s ' A
using *assms* **by** (intro Min-in) *auto*
then have $\exists x. x \in A \wedge \text{dist } s \ x = \text{Min } (\text{dist } s \ ' \ A)$
by *auto*
then have (SOME x. x $\in$ A $\wedge$ dist s x = Min (dist s ' A)) $\in$ A $\wedge$
dist s (SOME x. x $\in$ A $\wedge$ dist s x = Min (dist s ' A)) = Min (dist s ' A)
by (rule someI-ex)
then show ?thesis
unfolding *min-dist-vertex-def* .
qed

lemma *min-dist-vertex-in*:
assumes *finite* A A $\neq$ {}
shows *min-dist-vertex* A $\in$ A
using *min-dist-vertex-prop*[OF *assms*] **by** *blast*

lemma *min-dist-vertex-min*:
assumes *finite* A A $\neq$ {} y $\in$ A
shows dist s (*min-dist-vertex* A) $\leq$ dist s y
proof –
have dist s (*min-dist-vertex* A) = Min (dist s ' A)
using *min-dist-vertex-prop*[OF *assms*(1,2)] **by** *blast*
also have ... $\leq$ dist s y
using *assms* **by** *auto*
finally show ?thesis .
qed

lemma *ordered-extraction-exists*:
assumes *finite* A
shows $\exists xs. \text{set } xs = A \wedge \text{distinct } xs \wedge \text{sorted-wrt } (\lambda u \ v. \text{dist } s \ u \leq \text{dist } s \ v) \ xs$
proof –

```

obtain xs where xs: set xs = A distinct xs
  using finite-distinct-list[OF assms] by blast
let ?ys = sort-key (dist s) xs
have set ?ys = A
  using xs(1) by simp
moreover have distinct ?ys
  using xs(2) by simp
moreover have sorted-wrt (λu v. dist s u ≤ dist s v) ?ys
proof –
  have sorted (map (dist s) ?ys)
    by (rule sorted-sort-key)
  then show ?thesis
    by (simp add: sorted-map)
qed
ultimately show ?thesis
  by blast
qed

```

definition *closest-vertices* :: '*v* set $\Rightarrow$ '*v* list **where**
closest-vertices *A* =
 (*SOME xs. set xs = A $\wedge$ distinct xs $\wedge$ sorted-wrt (λu v. dist s u ≤ dist s v) xs*)

lemma *closest-vertices-properties*:
assumes *finite A*
shows *set (closest-vertices A) = A*
and *distinct (closest-vertices A)*
and *sorted-wrt (λu v. dist s u ≤ dist s v) (closest-vertices A)*
proof –
have $\exists xs. set xs = A \wedge distinct xs \wedge$
 sorted-wrt (λu v. dist s u ≤ dist s v) xs
using *ordered-extraction-exists[OF assms]* .
then have *cv-props: set (closest-vertices A) = A $\wedge$*
 distinct (closest-vertices A) $\wedge$
 sorted-wrt (λu v. dist s u ≤ dist s v) (closest-vertices A)
unfolding *closest-vertices-def* **by** *(rule someI-ex)*
then show *set (closest-vertices A) = A*
by *blast*
from *cv-props* **show** *distinct (closest-vertices A)*
by *blast*
from *cv-props* **show** *sorted-wrt (λu v. dist s u ≤ dist s v) (closest-vertices A)*
by *blast*
qed

definition *base-case-order* :: '*v* $\Rightarrow$ bound $\Rightarrow$ '*v* list **where**
base-case-order *x B* = *closest-vertices (bound-tree {x} B)*

definition *base-case-vertices* :: nat $\Rightarrow$ '*v* $\Rightarrow$ bound $\Rightarrow$ '*v* set **where**
base-case-vertices *k x B* =
 (*let xs = base-case-order x B in*

if length $xs \leq k$ then set xs
 else $\{v \in \text{set } (\text{take } (\text{Suc } k) \text{ } xs). \text{dist } s \ v < \text{dist } s \ (xs \ ! \ k)\}$

definition *base-case-bound* :: $\text{nat} \Rightarrow 'v \Rightarrow \text{bound} \Rightarrow \text{bound}$ **where**
base-case-bound $k \ x \ B =$
 (let $xs = \text{base-case-order } x \ B$ in
 if length $xs \leq k$ then B else $\text{Fin } (\text{dist } s \ (xs \ ! \ k))$)

definition *base-case-result* :: $\text{nat} \Rightarrow 'v \Rightarrow \text{bound} \Rightarrow \text{bound} \times 'v \text{ set}$ **where**
base-case-result $k \ x \ B =$
 (*base-case-bound* $k \ x \ B$, *base-case-vertices* $k \ x \ B$)

The helper *closest-vertices* is a choice of a distinct list sorted by true distance. It is intentionally non-executable: the base-case proof is semantic, and later executable theories provide their own finite procedure. The public base-case result is described by *base-case-result*, whose components are *base-case-bound* and *base-case-vertices*. The strict inequality in *base-case-vertices* mirrors the main BMSSP convention for finite bounds.

lemma *base-case-order-set*:
 set (*base-case-order* $x \ B$) = *bound-tree* $\{x\} \ B$
unfolding *base-case-order-def*
using *closest-vertices-properties*(1)[*OF finite-bound-tree*] .

lemma *base-case-order-sorted*:
sorted-wrt $(\lambda u \ v. \text{dist } s \ u \leq \text{dist } s \ v) \ (\text{base-case-order } x \ B)$
unfolding *base-case-order-def*
using *closest-vertices-properties*(3)[*OF finite-bound-tree*] .

lemma *base-case-order-distinct*:
distinct (*base-case-order* $x \ B$)
unfolding *base-case-order-def*
using *closest-vertices-properties*(2)[*OF finite-bound-tree*] .

lemma *in-set-take-dist-lt-nth*:
assumes *sorted*: *sorted-wrt* $(\lambda u \ v. \text{dist } s \ u \leq \text{dist } s \ v) \ xs$
and $y: y \in \text{set } xs$
and $lt: \text{dist } s \ y < \text{dist } s \ (xs \ ! \ k)$
and $k: k < \text{length } xs$
shows $y \in \text{set } (\text{take } k \ xs)$

proof –
obtain i **where** $i: i < \text{length } xs \ xs \ ! \ i = y$
using y **by** (*auto simp: in-set-conv-nth*)
have $i < k$
proof (*rule ccontr*)
assume $\neg i < k$
then have $k \leq i$
by *simp*
show *False*
proof (*cases* $k = i$)

```

    case True
    then show ?thesis
      using lt i(2) by simp
  next
  case False
  with k-le-i have  $k < i$ 
    by simp
  then have  $\text{dist } s \ (xs \ ! \ k) \leq \text{dist } s \ (xs \ ! \ i)$ 
    using sorted-wrt-nth-less[OF sorted, of k i] i(1) by simp
  then show ?thesis
    using lt i(2) by simp
qed
qed
then have  $i < \text{length } (\text{take } k \ xs) \ \text{take } k \ xs \ ! \ i = y$ 
  using i by auto
then show ?thesis
  using nth-mem by metis
qed

lemma base-case-success:
  assumes  $\text{length } (\text{base-case-order } x \ B) \leq k$ 
  shows base-case-vertices  $k \ x \ B = \text{bound-tree } \{x\} \ B$ 
  using assms base-case-order-set[of x B]
  unfolding base-case-vertices-def by (simp add: Let-def)

lemma below-bound-less-trans:
  assumes  $a < b$  below-bound b B
  shows below-bound a B
  using assms by (cases B) auto

lemma Fin-below-bound-le:
  assumes below-bound a B
  shows bound-le (Fin a) B
  using assms by (cases B) auto

lemma base-case-partial:
  assumes len: k < length (base-case-order x B)
  shows base-case-vertices  $k \ x \ B = \text{bound-tree } \{x\} \ (\text{Fin } (\text{dist } s \ ((\text{base-case-order } x \ B) \ ! \ k)))$ 
proof -
  let ?xs = base-case-order x B
  let ?b =  $\text{dist } s \ (?xs \ ! \ k)$ 
  have set-xs:  $\text{set } ?xs = \text{bound-tree } \{x\} \ B$ 
    using base-case-order-set[of x B] .
  have sorted: sorted-wrt  $(\lambda u \ v. \text{dist } s \ u \leq \text{dist } s \ v) \ ?xs$ 
    using base-case-order-sorted[of x B] .
  have kth-in:  $?xs \ ! \ k \in \text{bound-tree } \{x\} \ B$ 
    using len set-xs nth-mem by metis
  show ?thesis

```

```

proof
  show base-case-vertices  $k\ x\ B \subseteq \text{bound-tree } \{x\} \text{ (Fin } ?b)$ 
proof
  fix  $v$ 
  assume  $v \in \text{base-case-vertices } k\ x\ B$ 
  moreover have base-case-vertices  $k\ x\ B =$ 
     $\{v \in \text{set } (\text{take } (\text{Suc } k) \ ?xs). \text{dist } s\ v < ?b\}$ 
  using len unfolding base-case-vertices-def by (simp add: Let-def)
  ultimately have  $v: v \in \text{set } (\text{take } (\text{Suc } k) \ ?xs) \text{dist } s\ v < ?b$ 
  by auto
  have  $v \in \text{set } ?xs$ 
  using  $v(1)$  by (meson in-set-takeD)
  then have  $v \in \text{bound-tree } \{x\} \ B$ 
  using set-xs by simp
  then show  $v \in \text{bound-tree } \{x\} \text{ (Fin } ?b)$ 
  using  $v(2)$  unfolding bound-tree-def by auto
qed
next
show bound-tree  $\{x\} \text{ (Fin } ?b) \subseteq \text{base-case-vertices } k\ x\ B$ 
proof
  fix  $v$ 
  assume  $v: v \in \text{bound-tree } \{x\} \text{ (Fin } ?b)$ 
  then have lt:  $\text{dist } s\ v < ?b$ 
  unfolding bound-tree-def by auto
  have kth-below: below-bound  $(\text{dist } s\ (?xs\ !\ k))\ B$ 
  using kth-in unfolding bound-tree-def by auto
  have below-bound  $(\text{dist } s\ v)\ B$ 
  using below-bound-less-trans[OF lt kth-below] .
  then have  $v \in \text{bound-tree } \{x\} \ B$ 
  using  $v$  unfolding bound-tree-def by auto
  then have  $v \in \text{set } ?xs$ 
  using set-xs by simp
  then have  $v \in \text{set } (\text{take } k\ ?xs)$ 
  using in-set-take-dist-lt-nth[OF sorted - lt len] by blast
  then have  $v \in \text{set } (\text{take } (\text{Suc } k) \ ?xs)$ 
  using set-take-subset-set-take[of k Suc k ?xs] by auto
  then show  $v \in \text{base-case-vertices } k\ x\ B$ 
  using len lt unfolding base-case-vertices-def by (simp add: Let-def)
qed
qed
qed

lemma base-case-result-correct:
  assumes  $S = \{x\}$ 
  shows case base-case-result  $k\ x\ B$  of  $(B', U) \Rightarrow$ 
     $U = \text{bound-tree } S\ B' \wedge \text{complete-on } (\lambda v. \text{if } v \in U \text{ then } \text{dist } s\ v \text{ else } d\ v) \ U$ 
proof (cases length (base-case-order x B) ≤ k)
  case True
  then have  $U: \text{base-case-vertices } k\ x\ B = \text{bound-tree } \{x\} \ B$ 

```

```

    using base-case-success by blast
  have B': base-case-bound k x B = B
    using True unfolding base-case-bound-def by (simp add: Let-def)
  have complete-on ( $\lambda v. \text{if } v \in \text{base-case-vertices } k \ x \ B \text{ then } \text{dist } s \ v \text{ else } d \ v$ )
    (base-case-vertices k x B)
    unfolding complete-on-def by simp
  then show ?thesis
    using assms U B' unfolding base-case-result-def by simp
next
case False
then have len:  $k < \text{length } (\text{base-case-order } x \ B)$ 
  by simp
let ?B = Fin ( $\text{dist } s \ ((\text{base-case-order } x \ B) ! k)$ )
have U: base-case-vertices k x B = bound-tree {x} ?B
  using base-case-partial[OF len] .
have B': base-case-bound k x B = ?B
  using False unfolding base-case-bound-def by (simp add: Let-def)
have complete-on ( $\lambda v. \text{if } v \in \text{base-case-vertices } k \ x \ B \text{ then } \text{dist } s \ v \text{ else } d \ v$ )
  (base-case-vertices k x B)
  unfolding complete-on-def by simp
then show ?thesis
  using assms U B' unfolding base-case-result-def by simp
qed

lemma base-case-bound-le:
  bound-le (base-case-bound k x B) B
proof (cases  $\text{length } (\text{base-case-order } x \ B) \leq k$ )
case True
then show ?thesis
  unfolding base-case-bound-def by (cases B) (simp-all add: Let-def)
next
case False
then have len:  $k < \text{length } (\text{base-case-order } x \ B)$ 
  by simp
let ?xs = base-case-order x B
have ?xs !  $k \in \text{bound-tree } \{x\} \ B$ 
  using len base-case-order-set[of x B] nth-mem by metis
then have below-bound ( $\text{dist } s \ (?xs ! k)$ ) B
  unfolding bound-tree-def by auto
then show ?thesis
  using False Fin-below-bound-le unfolding base-case-bound-def by (simp add:
Let-def)
qed

lemma base-case-result-bound-le:
  case base-case-result k x B of (B', U)  $\Rightarrow \text{bound-le } B' \ B$ 
  unfolding base-case-result-def using base-case-bound-le by simp

lemma base-case-result-bmssp-post:

```



```

assumes  $S = \{x\}$ 
shows case base-case-result  $k \ x \ B$  of  $(B', U) \Rightarrow$ 
  bmssp-post  $d \ S \ B \ (\lambda v. \text{if } v \in U \text{ then } \text{dist } s \ v \text{ else } d \ v) \ B' \ U$ 
proof -
  have corr: case base-case-result  $k \ x \ B$  of  $(B', U) \Rightarrow$ 
     $U = \text{bound-tree } S \ B' \wedge \text{complete-on } (\lambda v. \text{if } v \in U \text{ then } \text{dist } s \ v \text{ else } d \ v) \ U$ 
  using base-case-result-correct[OF assms, where  $k=k$  and  $B=B$  and  $d=d$ ] .
  have le: case base-case-result  $k \ x \ B$  of  $(B', U) \Rightarrow \text{bound-le } B' \ B$ 
  using base-case-result-bound-le[where  $k=k$  and  $x=x$  and  $B=B$ ] .
  show ?thesis
  using corr le unfolding bmssp-post-def by (cases base-case-result  $k \ x \ B$ ) auto
qed

```

The proof splits on whether the sorted bounded tree has already fit inside the base-case budget. In the success case, $\text{length } (\text{base-case-order } ?x \ ?B) \leq ?k \Rightarrow \text{base-case-vertices } ?k \ ?x \ ?B = \text{bound-tree } \{?x\} \ ?B$ says the returned set is the entire input tree. In the partial case, $?k < \text{length } (\text{base-case-order } ?x \ ?B) \Rightarrow \text{base-case-vertices } ?k \ ?x \ ?B = \text{bound-tree } \{?x\} \ (\text{Fin } (\text{local.dist } s \ (\text{base-case-order } ?x \ ?B \ ! \ ?k)))$ identifies the returned set with the same tree under the newly returned finite bound. Lemma $?S = \{?x\} \Rightarrow \text{case base-case-result } ?k \ ?x \ ?B \text{ of } (B', U) \Rightarrow \text{bmssp-post } ?d \ ?S \ ?B \ (\lambda v. \text{if } v \in U \text{ then } \text{local.dist } s \ v \text{ else } ?d \ v) \ B' \ U$ is the result consumed by the recursive relation: updating labels to *local.dist* on the returned set establishes the BMSSP postcondition.

```

lemma base-case-label-sound:
  assumes sound: sound-label  $d$ 
  shows sound-label  $(\lambda v. \text{if } v \in \text{base-case-vertices } k \ x \ B \text{ then } \text{dist } s \ v \text{ else } d \ v)$ 
  using sound unfolding sound-label-def by auto

```

```

lemma finite-base-case-vertices [simp]:
  finite  $(\text{base-case-vertices } k \ x \ B)$ 
proof (cases length (base-case-order  $x \ B) \leq k$ )
  case True
  then show ?thesis
    using base-case-success[of  $x \ B \ k$ ] by simp
next
  case False
  then show ?thesis
    unfolding base-case-vertices-def by (simp add: Let-def)
qed

```

```

lemma card-base-case-vertices-le:
  card  $(\text{base-case-vertices } k \ x \ B) \leq k$ 
proof (cases length (base-case-order  $x \ B) \leq k$ )
  case True
  have card  $(\text{base-case-vertices } k \ x \ B) = \text{length } (\text{base-case-order } x \ B)$ 
  using True base-case-order-distinct[of  $x \ B$ ]
  unfolding base-case-vertices-def by (simp add: Let-def distinct-card)

```

```

    then show ?thesis
      using True by simp
next
case False
then have len:  $k < \text{length } (\text{base-case-order } x \ B)$ 
  by simp
let ?xs = base-case-order  $x \ B$ 
let ?U = base-case-vertices  $k \ x \ B$ 
have U-subset:  $?U \subseteq \text{set } (\text{take } k \ ?xs)$ 
proof
  fix v
  assume  $v \in ?U$ 
  moreover have  $?U = \{v \in \text{set } (\text{take } (\text{Suc } k) \ ?xs). \text{dist } s \ v < \text{dist } s \ (?xs \ ! \ k)\}$ 
    using len unfolding base-case-vertices-def by (simp add: Let-def)
  ultimately have v:  $v \in \text{set } (\text{take } (\text{Suc } k) \ ?xs) \ \text{dist } s \ v < \text{dist } s \ (?xs \ ! \ k)$ 
    by auto
  then have v-set:  $v \in \text{set } ?xs$ 
    by (meson in-set-takeD)
  then show  $v \in \text{set } (\text{take } k \ ?xs)$ 
    using in-set-take-dist-lt-nth[OF base-case-order-sorted[of  $x \ B$ ] v-set v(2) len]
by blast
qed
have card-U-le:  $\text{card } ?U \leq \text{card } (\text{set } (\text{take } k \ ?xs))$ 
  using U-subset by (simp add: card-mono)
have card-take-le:  $\text{card } (\text{set } (\text{take } k \ ?xs)) \leq \text{length } (\text{take } k \ ?xs)$ 
  by (rule card-length)
have length (take  $k \ ?xs$ ) =  $k$ 
  using len by simp
then show ?thesis
  using card-U-le card-take-le by linarith
qed

```

The remaining facts are bookkeeping for the costed and recursive layers. The base-case label update preserves *sound-label*, the returned set is finite, and $\text{card } (\text{base-case-vertices } ?k \ ?x \ ?B) \leq ?k$ records the intended cap. The strict cutoff in the partial case is what makes this cardinality bound hold: ties at the k -th distance are not included.

end

end

theory *BMSSP-Partition-Interface*

imports *BMSSP-Correctness*

begin

12 Partition Data-Structure Interface

The algorithm's non-base BMSSP loop depends on a partition data structure with *Insert*, *BatchPrepend*, and *Pull*. This theory records the mathematical

contract of those operations. It also gives a sorted reference implementation of the functional *Pull* contract. The separate concrete partition-state theory refines this view with explicit active entries, retained value memory, costed operations, and pull-to-split bridge theorems.

The interface is intentionally small. A partition view is just a finite set of keys with a real-valued label for each key. In the algorithm those labels are tentative distances, but the interface does not mention graphs. This separation is what lets the correctness proof talk about the mathematical effect of a pull, while the bucketed refinement later proves that a concrete representation implements the same effect with the desired costs.

Three design choices are worth making explicit. First, inserts are minimum-updates: inserting an existing key keeps the smaller label. Second, batch prepends are admissible only when every prepended label is below every remaining partition label. This is the abstract form of the "prepend a lower block" operation in the paper. Third, a pull does not merely return any small set. It returns a prefix of the partition order, a separating bound, and a residual view whose values are unchanged.

The sorted reference implementation at the end of the file is not the efficient implementation used for the final headline. It is a specification device. Sorting the keys by label makes the pull contract transparent, and the bucketed structure is later shown to realise the same contract while satisfying the paper-tight amortised bounds.

record *'k partition-view* =
keys-of :: *'k set*
value-of :: *'k ⇒ real*

definition *min-update* **where**
min-update *D x b* =
 (| *keys-of* = *insert x (keys-of D)*,
 value-of = (*value-of D*)(*x* := if *x* ∈ *keys-of D* then min (*value-of D x*) *b* else *b*) |)

definition *insert-spec* **where**
insert-spec *D x b D'* $\longleftrightarrow$
keys-of *D'* = *insert x (keys-of D)* $\wedge$
value-of *D' x* = (if *x* ∈ *keys-of D* then min (*value-of D x*) *b* else *b*) $\wedge$
 ($\forall y. y \neq x \longrightarrow \text{value-of } D' y = \text{value-of } D y$)

definition *batch-min-update* **where**
batch-min-update *D xs* = fold ($\lambda(x, b) D'. \text{min-update } D' x b$) *xs D*

definition *pull-separates* **where**
pull-separates *D M B S x D'* $\longleftrightarrow$
S $\subseteq \text{keys-of } D \wedge$
card *S* $\leq M \wedge$
keys-of *D'* = *keys-of D* − *S* $\wedge$

$$\begin{aligned}
& \text{value-of } D' = \text{value-of } D \wedge \\
& (\forall u \in S. \forall v \in \text{keys-of } D'. \text{value-of } D \ u \leq \text{value-of } D' \ v) \wedge \\
& (\text{if } \text{keys-of } D' = \{\} \text{ then } x = B \\
& \text{else } (\forall u \in S. \text{value-of } D \ u < x) \wedge (\forall v \in \text{keys-of } D'. x \leq \text{value-of } D' \ v))
\end{aligned}$$

definition *batch-prepend-admissible* **where**

$$\begin{aligned}
& \text{batch-prepend-admissible } D \ xs \longleftrightarrow \\
& (\forall (x, b) \in \text{set } xs. \forall y \in \text{keys-of } D. b \leq \text{value-of } D \ y)
\end{aligned}$$

definition *partition-upper-bound* **where**

$$\text{partition-upper-bound } D \ B \longleftrightarrow (\forall u \in \text{keys-of } D. \text{value-of } D \ u < B)$$

definition *partition-insert-cost-bound* **where**

$$\text{partition-insert-cost-bound } c \ t \longleftrightarrow c \leq t$$

definition *partition-batch-cost-bound* **where**

$$\text{partition-batch-cost-bound } c \ t \ xs \longleftrightarrow c \leq t * \text{length } xs$$

definition *partition-pull-cost-bound* **where**

$$\text{partition-pull-cost-bound } c \ S \longleftrightarrow c \leq \text{card } S$$

The three cost predicates are deliberately parametric and algebraic. They do not prescribe an implementation, only the local budget that an implementation must provide to plug into the runtime recurrence. The sorted reference pull has a linear cost, while the bucketed implementation later supplies the stronger logarithmic and amortised bounds required by the paper.

The summation lemmas below are the interface's first payoff: once individual operations satisfy their local predicates, loop-level proofs can sum costs by list induction without opening the implementation of the data structure.

lemma *partition-insert-costs-sum-bound*:

assumes *list-all* $(\lambda c. \text{partition-insert-cost-bound } c \ t) \ cs$

shows $\text{sum-list } cs \leq t * \text{length } cs$

using *assms*

proof (*induction cs*)

case *Nil*

then show *?case* **by** *simp*

next

case $(\text{Cons } c \ cs)$

then have $c: c \leq t$

unfolding *partition-insert-cost-bound-def* **by** *simp*

have *tail*: $\text{sum-list } cs \leq t * \text{length } cs$

using *Cons.IH Cons.prem* **by** *simp*

show *?case*

using *c tail* **by** *simp*

qed

lemma *partition-batch-costs-sum-bound*:

```

assumes list-all2 ( $\lambda c\ xs.\ \text{partition-batch-cost-bound}\ c\ t\ xs$ ) cs xss
shows  $\text{sum-list}\ cs \leq t * \text{sum-list}\ (\text{map}\ \text{length}\ xss)$ 
using assms
proof (induction cs arbitrary: xss)
  case Nil
  then show ?case
    by (cases xss) simp-all
next
  case (Cons c cs)
  then obtain xs xss' where xss: xss = xs # xss'
    and  $c \leq t * \text{length}\ xs$ 
    and rest: list-all2 ( $\lambda c\ xs.\ \text{partition-batch-cost-bound}\ c\ t\ xs$ ) cs xss'
    by (cases xss) (auto simp: partition-batch-cost-bound-def)
  have tail: sum-list cs  $\leq t * \text{sum-list}\ (\text{map}\ \text{length}\ xss')$ 
    using Cons.IH[OF rest] .
  show ?case
  proof -
    have  $c + \text{sum-list}\ cs \leq$ 
       $t * \text{length}\ xs + t * \text{sum-list}\ (\text{map}\ \text{length}\ xss')$ 
      using c tail by simp
    also have  $\dots = t * (\text{length}\ xs + \text{sum-list}\ (\text{map}\ \text{length}\ xss'))$ 
      by (simp add: algebra-simps)
    finally show ?thesis
      unfolding xss by simp
  qed
qed

```

```

lemma partition-pull-costs-sum-bound:
  assumes list-all2 ( $\lambda c\ S.\ \text{partition-pull-cost-bound}\ c\ S$ ) cs Ss
  shows  $\text{sum-list}\ cs \leq \text{sum-list}\ (\text{map}\ \text{card}\ Ss)$ 
  using assms
proof (induction cs arbitrary: Ss)
  case Nil
  then show ?case
    by (cases Ss) simp-all
next
  case (Cons c cs)
  then obtain S Ss' where Ss: Ss = S # Ss'
    and  $c \leq \text{card}\ S$ 
    and rest: list-all2 ( $\lambda c\ S.\ \text{partition-pull-cost-bound}\ c\ S$ ) cs Ss'
    by (cases Ss) (auto simp: partition-pull-cost-bound-def)
  have tail: sum-list cs  $\leq \text{sum-list}\ (\text{map}\ \text{card}\ Ss')$ 
    using Cons.IH[OF rest] .
  show ?case
    using c tail unfolding Ss by simp
  qed

```

```

lemma partition-batch-cost-bound-append:
  assumes xs: partition-batch-cost-bound c-x t xs

```

```

    and ys: partition-batch-cost-bound c-y h ys
    and h-le-t: h ≤ t
  shows partition-batch-cost-bound (c-x + c-y) t (xs @ ys)
proof -
  have c-x: c-x ≤ t * length xs
    using xs unfolding partition-batch-cost-bound-def by simp
  have c-y: c-y ≤ h * length ys
    using ys unfolding partition-batch-cost-bound-def by simp
  have h * length ys ≤ t * length ys
    using h-le-t by simp
  then have c-x + c-y ≤ t * length xs + t * length ys
    using c-x c-y by linarith
  also have ... = t * length (xs @ ys)
    by (simp add: algebra-simps)
  finally show ?thesis
    unfolding partition-batch-cost-bound-def .
qed

```

```

lemma batch-prepend-admissibleI:
  assumes  $\bigwedge x\ b\ y. [(x, b) \in \text{set } xs; y \in \text{keys-of } D] \implies$ 
     $b \leq \text{value-of } D\ y$ 
  shows batch-prepend-admissible D xs
  using assms unfolding batch-prepend-admissible-def by auto

```

```

lemma batch-prepend-admissible-below-all:
  assumes vals:  $\bigwedge x\ b. (x, b) \in \text{set } xs \implies b \leq L$ 
    and lower:  $\bigwedge y. y \in \text{keys-of } D \implies L \leq \text{value-of } D\ y$ 
  shows batch-prepend-admissible D xs
proof (rule batch-prepend-admissibleI)
  fix x b y
  assume xb:  $(x, b) \in \text{set } xs$  and y:  $y \in \text{keys-of } D$ 
  have b ≤ L
    by (rule vals[OF xb])
  moreover have L ≤ value-of D y
    by (rule lower[OF y])
  then show b ≤ value-of D y
    using calculation by linarith
qed

```

```

lemma sorted-value-less-nth-in-take:
  fixes f :: 'a ⇒ 'b::linorder
  assumes sorted: sorted-wrt (λu v. f u ≤ f v) xs
    and y: y ∈ set xs
    and lt: f y < f (xs ! k)
    and k: k < length xs
  shows y ∈ set (take k xs)
proof -
  obtain i where i: i < length xs xs ! i = y
    using y by (auto simp: in-set-conv-nth)

```

```

have  $i < k$ 
proof (rule ccontr)
  assume  $\neg i < k$ 
  then have  $k \leq i$ 
  by simp
  show False
proof (cases  $k = i$ )
  case True
  have  $f\ y < f\ y$ 
  using  $lt\ i(2)\ True$  by simp
  then show ?thesis
  using less-irrefl by blast
next
  case False
  with  $k \leq i$  have  $k < i$ 
  by simp
  then have  $f\ (xs\ !\ k) \leq f\ (xs\ !\ i)$ 
  using sorted-wrt-nth-less[OF sorted, of k i]  $i(1)$  by simp
  then show ?thesis
  using  $lt\ i(2)$  by simp
qed
qed
then have  $i < \text{length}\ (\text{take}\ k\ xs)\ \text{take}\ k\ xs\ !\ i = y$ 
  using  $i$  by auto
then show ?thesis
  using nth-mem by metis
qed

```

The sorted reference view chooses one canonical order of the partition keys: any distinct list sorted by *value-of*. Isabelle’s Hilbert choice is used only to avoid committing to a concrete sorting algorithm in the specification. The theorem *partition-key-order-properties* immediately recovers the facts needed by later proofs: the order contains exactly the keys, has no duplicates, and is sorted by label.

The reference pull then takes the first block of that order. If at most M keys remain, the pull consumes everything and returns the outer bound. Otherwise it uses the M -th key as the separating bound and pulls exactly the keys with smaller labels. This strict comparison matches the BMSSP lower split used by the correctness proof.

definition *partition-key-order* **where**
partition-key-order $D =$
 $(\text{SOME } xs. \text{ set } xs = \text{keys-of } D \wedge \text{distinct } xs \wedge$
 $\text{sorted-wrt } (\lambda u\ v. \text{value-of } D\ u \leq \text{value-of } D\ v)\ xs)$

definition *sorted-pull-cost* **where**
 $\text{sorted-pull-cost } D = \text{length}\ (\text{partition-key-order } D)$

lemma *partition-key-order-properties*:

```

assumes finite-keys: finite (keys-of D)
shows set (partition-key-order D) = keys-of D
  and distinct (partition-key-order D)
  and sorted-wrt ( $\lambda u v. \text{value-of } D \ u \leq \text{value-of } D \ v$ ) (partition-key-order D)
proof –
  obtain xs where xs: set xs = keys-of D distinct xs
    using finite-distinct-list[OF finite-keys] by blast
  let ?ys = sort-key (value-of D) xs
  have exists:
     $\exists \text{ys. set ys = keys-of } D \wedge \text{distinct ys} \wedge$ 
     $\text{sorted-wrt } (\lambda u v. \text{value-of } D \ u \leq \text{value-of } D \ v) \ \text{ys}$ 
  proof (intro exI[of - ?ys] conjI)
    show set ?ys = keys-of D
      using xs by simp
    show distinct ?ys
      using xs by simp
    show sorted-wrt ( $\lambda u v. \text{value-of } D \ u \leq \text{value-of } D \ v$ ) ?ys
  proof –
    have sorted (map (value-of D) ?ys)
      by (rule sorted-sort-key)
    then show ?thesis
      by (simp add: sorted-map)
  qed
qed
have props: set (partition-key-order D) = keys-of D  $\wedge$ 
  distinct (partition-key-order D)  $\wedge$ 
  sorted-wrt ( $\lambda u v. \text{value-of } D \ u \leq \text{value-of } D \ v$ ) (partition-key-order D)
  unfolding partition-key-order-def by (rule someI-ex[OF exists])
  then show set (partition-key-order D) = keys-of D
    by blast
  from props show distinct (partition-key-order D)
    by blast
  from props show sorted-wrt ( $\lambda u v. \text{value-of } D \ u \leq \text{value-of } D \ v$ ) (partition-key-order D)
    by blast
qed

```

```

definition sorted-pull-set where
  sorted-pull-set M D =
    (let xs = partition-key-order D in
      if length xs  $\leq M$  then set xs
      else  $\{u \in \text{keys-of } D. \text{value-of } D \ u < \text{value-of } D \ (xs \ ! \ M)\}$ )

```

```

definition sorted-pull-bound where
  sorted-pull-bound M B D =
    (let xs = partition-key-order D in
      if length xs  $\leq M$  then B else value-of D (xs ! M))

```

```

definition remove-keys-view where

```


$\text{remove-keys-view } D \ S =$
 $\langle \text{keys-of} = \text{keys-of } D - S, \text{value-of} = \text{value-of } D \rangle$

lemma *min-update-keys [simp]*:
 $\text{keys-of } (\text{min-update } D \ x \ b) = \text{insert } x \ (\text{keys-of } D)$
unfolding *min-update-def* **by** *simp*

lemma *min-update-value-same*:
assumes $y \neq x$
shows $\text{value-of } (\text{min-update } D \ x \ b) \ y = \text{value-of } D \ y$
using *assms* **unfolding** *min-update-def* **by** *simp*

lemma *min-update-value-key-new*:
assumes $x \notin \text{keys-of } D$
shows $\text{value-of } (\text{min-update } D \ x \ b) \ x = b$
using *assms* **unfolding** *min-update-def* **by** *simp*

lemma *min-update-value-key-old*:
assumes $x \in \text{keys-of } D$
shows $\text{value-of } (\text{min-update } D \ x \ b) \ x = \min (\text{value-of } D \ x) \ b$
using *assms* **unfolding** *min-update-def* **by** *simp*

lemma *min-update-value-le-old*:
assumes $x \in \text{keys-of } D$
shows $\text{value-of } (\text{min-update } D \ y \ b) \ x \leq \text{value-of } D \ x$
proof (*cases* $x = y$)
case *True*
then show *?thesis*
using *assms* **unfolding** *min-update-def* **by** *simp*
next
case *False*
then show *?thesis*
using *min-update-value-same*[*of* $x \ y \ D \ b$] **by** *simp*
qed

lemma *min-update-value-le-inserted*:
 $\text{value-of } (\text{min-update } D \ x \ b) \ x \leq b$
unfolding *min-update-def* **by** *simp*

theorem *min-update-insert-spec*:
 $\text{insert-spec } D \ x \ b \ (\text{min-update } D \ x \ b)$
unfolding *insert-spec-def* *min-update-def* **by** *auto*

lemma *min-update-preserves-upper-bound*:
assumes *upper*: *partition-upper-bound* $D \ B$
and *b-lt*: $b < B$
shows *partition-upper-bound* $(\text{min-update } D \ x \ b) \ B$
proof –
have $\bigwedge u. u \in \text{keys-of } (\text{min-update } D \ x \ b) \implies$

```

    value-of (min-update D x b) u < B
  proof -
    fix u
    assume u: u ∈ keys-of (min-update D x b)
    show value-of (min-update D x b) u < B
    proof (cases u = x)
      case True
      then show ?thesis
      proof (cases x ∈ keys-of D)
        case True
        then have min (value-of D x) b < B
          using upper b-lt unfolding partition-upper-bound-def by simp
        then show ?thesis
          using ⟨u = x⟩ True unfolding min-update-def by simp
      next
        case False
        then show ?thesis
          using ⟨u = x⟩ b-lt unfolding min-update-def by simp
      qed
    next
      case False
      then have u ∈ keys-of D
        using u unfolding min-update-def by simp
      then have value-of D u < B
        using upper unfolding partition-upper-bound-def by blast
      then show ?thesis
        using False min-update-value-same[of u x D b] by simp
      qed
    qed
  then show ?thesis
    unfolding partition-upper-bound-def by blast
qed

lemma batch-min-update-keys-subset:
  keys-of (batch-min-update D xs) ⊆ keys-of D ∪ fst ` set xs
proof (induction xs arbitrary: D)
  case Nil
  then show ?case
    unfolding batch-min-update-def by simp
next
  case (Cons xb xs)
  obtain x b where xb: xb = (x, b)
  by force
  have keys-of (batch-min-update (min-update D x b) xs)
    ⊆ keys-of (min-update D x b) ∪ fst ` set xs
    using Cons.IH[of min-update D x b] .
  also have ... ⊆ keys-of D ∪ fst ` set (xb # xs)
    using xb by auto
  finally show ?case

```

```

    unfolding batch-min-update-def xb by simp
qed

lemma batch-min-update-keys:
  keys-of (batch-min-update D xs) = keys-of D  $\cup$  fst ` set xs
proof (induction xs arbitrary: D)
  case Nil
  then show ?case
    unfolding batch-min-update-def by simp
next
  case (Cons xb xs)
  obtain x b where xb: xb = (x, b)
  by force
  have keys-of (batch-min-update (min-update D x b) xs) =
    keys-of (min-update D x b)  $\cup$  fst ` set xs
  using Cons.IH[of min-update D x b] .
  then show ?case
    unfolding batch-min-update-def xb by auto
qed

lemma batch-min-update-value-le-old:
  assumes x  $\in$  keys-of D
  shows value-of (batch-min-update D xs) x  $\leq$  value-of D x
  using assms
proof (induction xs arbitrary: D)
  case Nil
  then show ?case
    unfolding batch-min-update-def by simp
next
  case (Cons xb xs)
  obtain y b where xb: xb = (y, b)
  by force
  have x-in-step: x  $\in$  keys-of (min-update D y b)
  using Cons.prem by simp
  have tail:
    value-of (batch-min-update (min-update D y b) xs) x  $\leq$ 
    value-of (min-update D y b) x
  by (rule Cons.IH[OF x-in-step])
  have step: value-of (min-update D y b) x  $\leq$  value-of D x
  by (rule min-update-value-le-old[OF Cons.prem])
  have tail-fold:
    value-of (fold ( $\lambda(x, b)$  D'. min-update D' x b) xs (min-update D y b)) x  $\leq$ 
    value-of (min-update D y b) x
  using tail unfolding batch-min-update-def .
  show ?case
    unfolding batch-min-update-def xb
    using tail-fold step by (simp add: case-prod-beta)
qed

```

```

lemma batch-min-update-value-le-pair:
  assumes  $(x, b) \in \text{set } xs$ 
  shows  $\text{value-of } (\text{batch-min-update } D \text{ } xs) \ x \leq b$ 
  using assms
proof (induction xs arbitrary: D)
  case Nil
  then show ?case by simp
next
  case (Cons xb xs)
  obtain y c where xb:  $xb = (y, c)$ 
  by force
  show ?case
  proof (cases  $(x, b) = (y, c)$ )
    case True
    have x-in-step:  $x \in \text{keys-of } (\text{min-update } D \ y \ c)$ 
    using True by simp
    have tail:
       $\text{value-of } (\text{batch-min-update } (\text{min-update } D \ y \ c) \ xs) \ x \leq$ 
       $\text{value-of } (\text{min-update } D \ y \ c) \ x$ 
      by (rule batch-min-update-value-le-old[OF x-in-step])
    have step:  $\text{value-of } (\text{min-update } D \ y \ c) \ x \leq b$ 
    using True min-update-value-le-inserted[of D y c] by simp
    have tail-fold:
       $\text{value-of } (\text{fold } (\lambda(x, b) \ D'. \ \text{min-update } D' \ x \ b) \ xs \ (\text{min-update } D \ y \ c)) \ x \leq$ 
       $\text{value-of } (\text{min-update } D \ y \ c) \ x$ 
    using tail unfolding batch-min-update-def .
    show ?thesis
    unfolding batch-min-update-def xb
    using tail-fold step by (simp add: case-prod-beta)
  next
  case False
  then have  $(x, b) \in \text{set } xs$ 
  using Cons.premys xb by auto
  then have tail:
     $\text{value-of } (\text{batch-min-update } (\text{min-update } D \ y \ c) \ xs) \ x \leq b$ 
    by (rule Cons.IH)
  have tail-fold:
     $\text{value-of } (\text{fold } (\lambda(x, b) \ D'. \ \text{min-update } D' \ x \ b) \ xs \ (\text{min-update } D \ y \ c)) \ x \leq b$ 
    using tail unfolding batch-min-update-def .
  show ?thesis
  unfolding batch-min-update-def xb
  using tail-fold by (simp add: case-prod-beta)
qed
qed

lemma sorted-pull-cost-linear:
  assumes finite-keys: finite (keys-of D)
  shows  $\text{sorted-pull-cost } D = \text{card } (\text{keys-of } D)$ 
proof –

```

```

have  $\text{length } (\text{partition-key-order } D) = \text{card } (\text{set } (\text{partition-key-order } D))$ 
  using  $\text{partition-key-order-properties}(2)[\text{OF } \text{finite-keys}]$  by  $(\text{simp add: distinct-card})$ 
also have  $\dots = \text{card } (\text{keys-of } D)$ 
  using  $\text{partition-key-order-properties}(1)[\text{OF } \text{finite-keys}]$  by  $\text{simp}$ 
finally show  $?thesis$ 
  unfolding  $\text{sorted-pull-cost-def}$  .
qed

```

The minimum-update lemmas establish the functional behaviour of inserts and batches. They are intentionally elementary: keys are added, old values are never increased, inserted values are respected, and an upper bound is preserved when every inserted value is below that bound. These facts are used by the operational refresh step after a recursive child call has returned.

lemma $\text{batch-min-update-preserves-upper-bound}$:

```

assumes  $\text{upper: partition-upper-bound } D \ B$ 
  and  $\text{values-lt: } \bigwedge x \ b. (x, b) \in \text{set } xs \implies b < B$ 
shows  $\text{partition-upper-bound } (\text{batch-min-update } D \ xs) \ B$ 
using  $\text{assms}$ 
proof  $(\text{induction } xs \text{ arbitrary: } D)$ 
  case  $\text{Nil}$ 
  then show  $?case$ 
    unfolding  $\text{batch-min-update-def}$  by  $\text{simp}$ 
next
  case  $(\text{Cons } xb \ xs)$ 
  obtain  $x \ b$  where  $xb: xb = (x, b)$ 
  by  $\text{force}$ 
  have  $b\text{-lt: } b < B$ 
  using  $\text{Cons.prem}(2)[\text{of } x \ b] \ xb$  by  $\text{simp}$ 
  have  $\text{upper-step: partition-upper-bound } (\text{min-update } D \ x \ b) \ B$ 
  using  $\text{min-update-preserves-upper-bound}[\text{OF } \text{Cons.prem}(1) \ b\text{-lt}]$  .
  have  $\text{tail-lt: } \bigwedge y \ c. (y, c) \in \text{set } xs \implies c < B$ 
  proof –
  fix  $y \ c$ 
  assume  $(y, c) \in \text{set } xs$ 
  then show  $c < B$ 
    using  $\text{Cons.prem}(2)[\text{of } y \ c]$  by  $\text{simp}$ 
  qed
  have  $\text{ih: partition-upper-bound } (\text{batch-min-update } (\text{min-update } D \ x \ b) \ xs) \ B$ 
  by  $(\text{rule } \text{Cons.IH}[\text{OF } \text{upper-step } \text{tail-lt}])$ 
  then show  $?case$ 
    unfolding  $\text{batch-min-update-def } xb$  by  $(\text{simp add: case-prod-beta})$ 
qed

```

lemma $\text{pull-separates-pulled-smallest}$:

```

assumes  $\text{pull-separates } D \ M \ B \ S \ x \ D'$ 
  and  $u \in S$ 
  and  $v \in \text{keys-of } D'$ 
shows  $\text{value-of } D \ u \leq \text{value-of } D' \ v$ 

```

```

using assms unfolding pull-separates-def by blast

lemma pull-separates-remaining-keys:
  assumes pull-separates D M B S x D'
  shows keys-of D' = keys-of D - S
  using assms unfolding pull-separates-def by blast

lemma pull-separates-card:
  assumes pull-separates D M B S x D'
  shows card S ≤ M
  using assms unfolding pull-separates-def by blast

lemma pull-separates-subset:
  assumes pull-separates D M B S x D'
  shows S ⊆ keys-of D
  using assms unfolding pull-separates-def by blast

lemma pull-separates-preserves-upper-bound:
  assumes pull: pull-separates D M B0 S x D'
    and upper: partition-upper-bound D B
  shows partition-upper-bound D' B
    using pull upper unfolding pull-separates-def partition-upper-bound-def by
auto

  The pull lemmas unpack the abstract separator contract. A successful
  pull gives a source set of size at most  $M$ , a residual key set with the pulled
  keys removed, and a bound that lies between the pulled prefix and the
  residual suffix. The admissibility lemma for batch prepend is the property
  needed by the BMSSP loop: if every new batch value is below the separator,
  then the batch may be prepended in front of the residual partition.

lemma pull-separates-empty-bound:
  assumes pull-separates D M B S x D'
    and keys-of D' = {}
  shows x = B
  using assms unfolding pull-separates-def by simp

lemma pull-separates-nonempty-bound:
  assumes pull-separates D M B S x D'
    and keys-of D' ≠ {}
    and v ∈ keys-of D'
  shows x ≤ value-of D' v
  using assms unfolding pull-separates-def by auto

lemma pull-separates-bound-le:
  assumes pull: pull-separates D M B S x D'
    and upper: partition-upper-bound D B
  shows x ≤ B
proof (cases keys-of D' = {})
  case True

```

```

    then show ?thesis
      using pull-separates-empty-bound[OF pull True] by simp
next
case False
then obtain v where v: v ∈ keys-of D'
  by blast
have x-le-v: x ≤ value-of D' v
  by (rule pull-separates-nonempty-bound[OF pull False v])
have v-old: v ∈ keys-of D
  using pull v unfolding pull-separates-def by auto
have val-eq: value-of D' = value-of D
  using pull unfolding pull-separates-def by blast
have value-of D v < B
  using upper v-old unfolding partition-upper-bound-def by blast
then show ?thesis
  using x-le-v val-eq by auto
qed

```

lemma *pull-separates-batch-prepend-admissible:*

```

  assumes pull: pull-separates D M B S x D'
    and vals:  $\bigwedge u b. (u, b) \in \text{set } xs \implies b \leq x$ 
  shows batch-prepend-admissible D' xs
proof (cases keys-of D' = {})
case True
  then show ?thesis
    unfolding batch-prepend-admissible-def by simp
next
case False
  show ?thesis
  proof (rule batch-prepend-admissible-below-all[OF vals])
    fix y
    assume y ∈ keys-of D'
    then show x ≤ value-of D' y
      by (rule pull-separates-nonempty-bound[OF pull False])
  qed
qed

```

theorem *pull-separates-exact-split:*

```

  assumes pull: pull-separates D M B S x D'
    and upper:  $\bigwedge u. u \in \text{keys-of } D \implies \text{value-of } D u < B$ 
  shows S = {u ∈ keys-of D. value-of D u < x}
proof -
  have S-subset: S ⊆ keys-of D
    using pull-separates-subset[OF pull] .
  have D'-keys: keys-of D' = keys-of D - S
    using pull unfolding pull-separates-def by blast
  show ?thesis
  proof (cases keys-of D' = {})
  case True

```

```

have x-B:  $x = B$ 
  using pull True by (rule pull-separates-empty-bound)
have S-keys:  $S = \text{keys-of } D$ 
  using S-subset D'-keys True by blast
have  $\{u \in \text{keys-of } D. \text{value-of } D \ u < x\} = \text{keys-of } D$ 
  using upper unfolding x-B by blast
then show ?thesis
  using S-keys by simp
next
case False
have S-to-split:  $S \subseteq \{u \in \text{keys-of } D. \text{value-of } D \ u < x\}$ 
proof
  fix u
  assume uS:  $u \in S$ 
  then have value-of D u < x
    using pull False unfolding pull-separates-def by auto
  then show  $u \in \{u \in \text{keys-of } D. \text{value-of } D \ u < x\}$ 
    using uS S-subset by blast
qed
have split-to-S:  $\{u \in \text{keys-of } D. \text{value-of } D \ u < x\} \subseteq S$ 
proof
  fix u
  assume u:  $u \in \{u \in \text{keys-of } D. \text{value-of } D \ u < x\}$ 
  show  $u \in S$ 
  proof (rule ccontr)
    assume u  $\notin S$ 
    then have uD':  $u \in \text{keys-of } D'$ 
      using u D'-keys by blast
    have  $x \leq \text{value-of } D' \ u$ 
      by (rule pull-separates-nonempty-bound[OF pull False uD'])
    moreover have  $\text{value-of } D' \ u = \text{value-of } D \ u$ 
      using pull unfolding pull-separates-def by simp
    ultimately show False
      using u by simp
  qed
qed
show ?thesis
  using S-to-split split-to-S by blast
qed
qed

```

The final theorems connect the sorted reference operation to the abstract pull contract. *sorted-pull-separates* proves that the reference operation really returns a separating prefix, and *sorted-pull-exact-split* proves that the same prefix is exactly the strict lower split at the returned bound whenever all labels are below the outer bound.

These two facts are specification tools. The executable bucketed partition does not sort all keys on every pull; instead, it proves the same separator and split properties for its bucket state and then plugs those properties into

the generic BMSSP correctness and cost layers.

theorem *sorted-pull-separates*:

fixes $M :: \text{nat}$

and $B :: \text{real}$

assumes *finite-keys*: $\text{finite } (\text{keys-of } D)$

and $S\text{-def}$: $S = \text{sorted-pull-set } M \ D$

and $x\text{-def}$: $x = \text{sorted-pull-bound } M \ B \ D$

and $D'\text{-def}$: $D' = \text{remove-keys-view } D \ S$

shows *pull-separates* $D \ M \ B \ S \ x \ D'$

proof –

let $?xs = \text{partition-key-order } D$

have $\text{set-}xs$: $\text{set } ?xs = \text{keys-of } D$

using *partition-key-order-properties*(1)[*OF finite-keys*] .

have $\text{distinct-}xs$: $\text{distinct } ?xs$

using *partition-key-order-properties*(2)[*OF finite-keys*] .

have $\text{sorted-}xs$: $\text{sorted-wrt } (\lambda u \ v. \text{value-of } D \ u \leq \text{value-of } D \ v) \ ?xs$

using *partition-key-order-properties*(3)[*OF finite-keys*] .

show $?thesis$

proof (*cases length ?xs ≤ M*)

case *True*

have $S\text{-all}$: $S = \text{keys-of } D$

using *True set-xs unfolding S-def sorted-pull-set-def* **by** (*simp add: Let-def*)

have $D\text{-empty}$: $\text{keys-of } D' = \{\}$

unfolding $D'\text{-def}$ *remove-keys-view-def S-all* **by** *simp*

have $x = B$

using *True unfolding x-def sorted-pull-bound-def* **by** (*simp add: Let-def*)

moreover **have** $\text{card } S \leq M$

proof –

have $\text{card } S = \text{card } (\text{set } ?xs)$

using $S\text{-all set-}xs$ **by** *simp*

also **have** $\dots = \text{length } ?xs$

using $\text{distinct-}xs$ **by** (*simp add: distinct-card*)

finally **have** $\text{card } S = \text{length } ?xs$.

then **show** $?thesis$

using *True* **by** *simp*

qed

ultimately **show** $?thesis$

unfolding $\text{pull-separates-def } D'\text{-def}$ *remove-keys-view-def*

using $S\text{-all } D\text{-empty}$ **by** *auto*

next

case *False*

then **have** $M\text{-lt}$: $M < \text{length } ?xs$

by *simp*

let $?beta = \text{value-of } D \ (?xs \ ! \ M)$

have $x\text{-beta}$: $x = ?beta$

using *False unfolding x-def sorted-pull-bound-def* **by** (*simp add: Let-def*)

have $S\text{-lt}$: $S = \{u \in \text{keys-of } D. \text{value-of } D \ u < ?beta\}$

using *False unfolding S-def sorted-pull-set-def* **by** (*simp add: Let-def*)

have $S\text{-subset}$: $S \subseteq \text{keys-of } D$

```

  unfolding S-lt by blast
have S-take:  $S \subseteq \text{set } (\text{take } M \text{ } ?xs)$ 
proof
  fix u
  assume uS:  $u \in S$ 
  then have u-keys:  $u \in \text{keys-of } D$  and lt:  $\text{value-of } D \text{ } u < ?\text{beta}$ 
    unfolding S-lt by auto
  then have  $u \in \text{set } ?xs$ 
    using set-xs by simp
  then show  $u \in \text{set } (\text{take } M \text{ } ?xs)$ 
    using sorted-value-less-nth-in-take[OF sorted-xs - lt M-lt] by blast
qed
have card-S:  $\text{card } S \leq M$ 
proof -
  have  $\text{card } S \leq \text{card } (\text{set } (\text{take } M \text{ } ?xs))$ 
    using S-take by (intro card-mono) simp-all
  also have  $\dots \leq \text{length } (\text{take } M \text{ } ?xs)$ 
    by (rule card-length)
  also have  $\dots = M$ 
    using M-lt by simp
  finally show ?thesis .
qed
have D'-keys:  $\text{keys-of } D' = \text{keys-of } D - S$ 
  unfolding D'-def remove-keys-view-def by simp
have D'-value:  $\text{value-of } D' = \text{value-of } D$ 
  unfolding D'-def remove-keys-view-def by simp
have remaining-nonempty:  $\text{keys-of } D' \neq \{\}$ 
proof -
  have nth-in:  $?xs ! M \in \text{keys-of } D$ 
    using set-xs M-lt nth-mem by metis
  have  $?xs ! M \notin S$ 
    unfolding S-lt by simp
  then have  $?xs ! M \in \text{keys-of } D - S$ 
    using nth-in by blast
  then show ?thesis
    using D'-keys by blast
qed
have pulled-before-remaining:
 $\bigwedge u \ v. \llbracket u \in S; v \in \text{keys-of } D' \rrbracket \implies \text{value-of } D \text{ } u \leq \text{value-of } D' \text{ } v$ 
proof -
  fix u v
  assume uS:  $u \in S$  and vD:  $v \in \text{keys-of } D'$ 
  have  $\text{value-of } D \text{ } u < ?\text{beta}$ 
    using uS unfolding S-lt by blast
  moreover have  $\neg \text{value-of } D \text{ } v < ?\text{beta}$ 
    using vD D'-keys unfolding S-lt by blast
  ultimately have  $\text{value-of } D \text{ } u \leq \text{value-of } D \text{ } v$ 
    by simp
  then show  $\text{value-of } D \text{ } u \leq \text{value-of } D' \text{ } v$ 

```

```

    using D'-value by simp
  qed
  have lower-remaining:
     $\bigwedge v. v \in \text{keys-of } D' \implies x \leq \text{value-of } D' v$ 
  proof -
    fix v
    assume vD:  $v \in \text{keys-of } D'$ 
    have  $\neg \text{value-of } D v < ?\text{beta}$ 
      using vD D'-keys unfolding S-lt by blast
    then show  $x \leq \text{value-of } D' v$ 
      using x-beta D'-value by simp
  qed
  have upper-pulled:
     $\bigwedge u. u \in S \implies \text{value-of } D u < x$ 
    using x-beta unfolding S-lt by blast
  show ?thesis
    unfolding pull-separates-def
    using S-subset card-S D'-keys D'-value remaining-nonempty
      pulled-before-remaining lower-remaining upper-pulled
    by auto
  qed
qed

theorem sorted-pull-exact-split:
  assumes finite-keys: finite (keys-of D)
  and upper:  $\bigwedge u. u \in \text{keys-of } D \implies \text{value-of } D u < B$ 
  shows sorted-pull-set M D =
     $\{u \in \text{keys-of } D. \text{value-of } D u < \text{sorted-pull-bound } M B D\}$ 
  proof -
    let ?xs = partition-key-order D
    have set-xs: set ?xs = keys-of D
      using partition-key-order-properties(1)[OF finite-keys] .
    show ?thesis
      proof (cases length ?xs  $\leq M$ )
      case True
      then have sorted-pull-set M D = keys-of D
        using set-xs unfolding sorted-pull-set-def by (simp add: Let-def)
      moreover have sorted-pull-bound M B D = B
        using True unfolding sorted-pull-bound-def by (simp add: Let-def)
      ultimately show ?thesis
        proof -
          have  $\{u \in \text{keys-of } D. \text{value-of } D u < \text{sorted-pull-bound } M B D\} = \text{keys-of } D$ 
          proof
            show  $\{u \in \text{keys-of } D. \text{value-of } D u < \text{sorted-pull-bound } M B D\} \subseteq \text{keys-of } D$ 
              by blast
          next
            show keys-of D  $\subseteq \{u \in \text{keys-of } D. \text{value-of } D u < \text{sorted-pull-bound } M B D\}$ 
              proof
                fix u

```

```

    assume  $u: u \in \text{keys-of } D$ 
    then have  $\text{value-of } D \ u < B$ 
      by (rule upper)
    then show  $u \in \{u \in \text{keys-of } D. \text{value-of } D \ u < \text{sorted-pull-bound } M \ B \ D\}$ 
      using  $u \ \langle \text{sorted-pull-bound } M \ B \ D = B \rangle$  by simp
    qed
  qed
  then show ?thesis
    using  $\langle \text{sorted-pull-set } M \ D = \text{keys-of } D \rangle$  by simp
  qed
next
case False
then show ?thesis
  unfolding sorted-pull-set-def sorted-pull-bound-def by (simp add: Let-def)
qed
qed

end
theory BMSSP-Bucketed-Partition-Internal
imports BMSSP-Partition-Interface
        HOL-Library.Discrete-Functions
        HOL-Library.Landau-Symbols
begin

```

13 Bucketed Partition Data Structure

This is the main data-structure theory for the paper-faithful partition layer. The BMSSP paper uses a bucketed structure whose operation bounds are expressed in terms of the ratio between the number of inserted elements and the block size. That ratio is the essential distinction from a sorted-list model: Insert must pay for a search over buckets, not for a search over all stored elements.

The formal model is functional. A partition state contains a block size, an upper bound, a list of buckets, and a value memory. Each bucket has a marker and an unsorted list of key/value entries. The marker is a lower bound for the values in the bucket, and adjacent markers delimit the value ranges used by Pull. The list-level operations below first establish a simple exact model, then a lazy model that permits buckets to grow up to the split threshold before they are rebuilt.

The theory has three layers. The first layer defines the functional state, invariants, Insert, BatchPrepend, and Pull, and proves that their views refine the abstract partition interface. The second layer adds primitive step counters and a potential function for lazy splitting. The third layer names the paper-facing operations and proves the three exported cost bounds: Insert at the ratio-log budget, BatchPrepend at the matching batched budget, and Pull at an amortized linear-in- M budget. The baseline sorted-list partition

state used by the recurrence proofs is not used here.

The amortized analysis uses a scaled credit potential $\Phi(P) = 4 \cdot \sum_b \max 0 (\text{length } (bp\text{-bucket-entries } b) - bp\text{-block-size } P)$, summed over the buckets of P . The factor of four is the constant that absorbs the per-Insert charge: each Insert raises Φ by at most four credits, which is exactly the budget that pays for the $O(M)$ rebucket work performed during a Pull when a bucket has overflowed past the lazy split threshold. This bookkeeping is what makes the bound paper-tight rather than merely amortized-but-loose: an Insert that does not split the bucket pays only for the bucket walk $O(\log(N / M))$, an Insert that triggers a rebucket pays the same physical cost plus a refund of credits to Φ , and Pull discharges the credits accumulated by its bucket's prior Inserts to fund the rebucket step. Without the factor four the per-Insert charge would not cover the rebucket, and the resulting Insert bound would degrade to $O(N / M)$; with the factor four it stays at the paper rate.

record 'k bp-bucket =
 bp-marker :: real
 bp-bucket-entries :: ('k $\times$ real) list

record 'k bucketed-partition =
 bp-block-size :: nat
 bp-upper-bound :: real
 bp-buckets :: 'k bp-bucket list
 bp-values :: 'k $\Rightarrow$ real

definition bp-bucket-entries-flat :: 'k bp-bucket list $\Rightarrow$ ('k $\times$ real) list **where**
 bp-bucket-entries-flat bs = concat (map bp-bucket-entries bs)

definition bp-entries :: 'k bucketed-partition $\Rightarrow$ ('k $\times$ real) list **where**
 bp-entries P = bp-bucket-entries-flat (bp-buckets P)

definition bp-entry-keys :: ('k $\times$ real) list $\Rightarrow$ 'k set **where**
 bp-entry-keys xs = fst ' set xs

definition bp-bucket-keys :: 'k bp-bucket $\Rightarrow$ 'k set **where**
 bp-bucket-keys b = bp-entry-keys (bp-bucket-entries b)

definition bp-view :: 'k bucketed-partition $\Rightarrow$ 'k partition-view **where**
 bp-view P =
 [keys-of = bp-entry-keys (bp-entries P),
 value-of = bp-values P]

definition bp-bucket-sizes-ok :: 'k bucketed-partition $\Rightarrow$ bool **where**
 bp-bucket-sizes-ok P $\longleftrightarrow$
 ($\forall b \in \text{set } (bp\text{-buckets } P).$
 length (bp-bucket-entries b) $\leq$ bp-block-size P)

definition *bp-lazy-bucket-sizes-ok* :: 'k bucketed-partition $\Rightarrow$ bool **where**
bp-lazy-bucket-sizes-ok $P \longleftrightarrow$
 $(\forall b \in \text{set } (\text{bp-buckets } P)).$
 $\text{length } (\text{bp-bucket-entries } b) \leq 2 * \text{bp-block-size } P)$

definition *bp-bucket-markers-sorted* :: 'k bucketed-partition $\Rightarrow$ bool **where**
bp-bucket-markers-sorted $P \longleftrightarrow$
 $\text{sorted-wrt } (\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c) (\text{bp-buckets } P)$

definition *bp-bucket-markers-lower-bound* :: 'k bucketed-partition $\Rightarrow$ bool **where**
bp-bucket-markers-lower-bound $P \longleftrightarrow$
 $(\forall b \in \text{set } (\text{bp-buckets } P)).$
 $\forall p \in \text{set } (\text{bp-bucket-entries } b). \text{bp-marker } b \leq \text{snd } p)$

definition *bp-values-consistent* :: 'k bucketed-partition $\Rightarrow$ bool **where**
bp-values-consistent $P \longleftrightarrow$
 $(\forall p \in \text{set } (\text{bp-entries } P). \text{bp-values } P (\text{fst } p) = \text{snd } p)$

definition *bp-distinct-keys* :: 'k bucketed-partition $\Rightarrow$ bool **where**
bp-distinct-keys $P \longleftrightarrow \text{distinct } (\text{map fst } (\text{bp-entries } P))$

fun *bp-bucket-boundaries-ok* :: 'k bp-bucket list $\Rightarrow$ bool **where**
bp-bucket-boundaries-ok [] $\longleftrightarrow \text{True}$
| *bp-bucket-boundaries-ok* [b] $\longleftrightarrow \text{True}$
| *bp-bucket-boundaries-ok* (b # c # bs) $\longleftrightarrow$
 $(\forall p \in \text{set } (\text{bp-bucket-entries } b). \text{snd } p \leq \text{bp-marker } c) \wedge$
bp-bucket-boundaries-ok (c # bs)

definition *bp-bucket-boundaries-state-ok* ::
'k bucketed-partition $\Rightarrow$ bool **where**
bp-bucket-boundaries-state-ok $P \longleftrightarrow$
bp-bucket-boundaries-ok (*bp-buckets* P)

definition *bp-invariant* :: 'k bucketed-partition $\Rightarrow$ bool **where**
bp-invariant $P \longleftrightarrow$
 $0 < \text{bp-block-size } P \wedge$
bp-distinct-keys $P \wedge$
bp-bucket-sizes-ok $P \wedge$
bp-bucket-markers-sorted $P \wedge$
bp-bucket-markers-lower-bound $P \wedge$
bp-values-consistent P

definition *bp-ordered-invariant* :: 'k bucketed-partition $\Rightarrow$ bool **where**
bp-ordered-invariant $P \longleftrightarrow$
bp-invariant $P \wedge \text{bp-bucket-boundaries-state-ok } P$

definition *bp-lazy-invariant* :: 'k bucketed-partition $\Rightarrow$ bool **where**
bp-lazy-invariant $P \longleftrightarrow$
 $0 < \text{bp-block-size } P \wedge$

$bp\text{-distinct-keys } P \wedge$
 $bp\text{-lazy-bucket-sizes-ok } P \wedge$
 $bp\text{-bucket-markers-sorted } P \wedge$
 $bp\text{-bucket-markers-lower-bound } P \wedge$
 $bp\text{-values-consistent } P$

definition $bp\text{-lazy-ordered-invariant} :: 'k \text{ bucketed-partition} \Rightarrow \text{bool}$ **where**
 $bp\text{-lazy-ordered-invariant } P \longleftrightarrow$
 $bp\text{-lazy-invariant } P \wedge bp\text{-bucket-boundaries-state-ok } P$

The invariant vocabulary separates three concerns. The basic view of a state is *bp-view*: it exposes only the key set and the value memory required by the abstract partition interface. The structural invariant *bp-invariant* says that the block size is positive, keys are unique, strict bucket sizes are at most *bp-block-size*, markers are sorted, markers are lower bounds for their entries, and the stored value memory agrees with the bucket entries. The ordered invariant *bp-ordered-invariant* adds the boundary condition between adjacent buckets, which is what makes a first-bucket Pull sound.

The lazy variants are used for the amortized Insert proof. In *bp-lazy-invariant*, buckets may temporarily contain up to twice the block size. This slack is exactly what lets Insert avoid rebuilding on every operation. Once a bucket crosses the lazy threshold, the state is rebuilt into strict buckets and the potential drops enough to pay for the split.

definition $bp\text{-empty} :: \text{nat} \Rightarrow \text{real} \Rightarrow 'k \text{ bucketed-partition}$ **where**
 $bp\text{-empty } M \ B =$
 $\langle \langle bp\text{-block-size} = M,$
 $bp\text{-upper-bound} = B,$
 $bp\text{-buckets} = [],$
 $bp\text{-values} = (\lambda\cdot. B) \rangle \rangle$

definition $bp\text{-singleton-bucket} :: 'k \times \text{real} \Rightarrow 'k \text{ bp-bucket}$ **where**
 $bp\text{-singleton-bucket } p =$
 $\langle \langle bp\text{-marker} = \text{snd } p, bp\text{-bucket-entries} = [p] \rangle \rangle$

lemma $bp\text{-singleton-bucket-simps}$ $[simp]:$
 $bp\text{-marker } (bp\text{-singleton-bucket } p) = \text{snd } p$
 $bp\text{-bucket-entries } (bp\text{-singleton-bucket } p) = [p]$
unfolding $bp\text{-singleton-bucket-def}$ **by** $simp\text{-all}$

definition $bp\text{-make-bucket} :: ('k \times \text{real}) \text{ list} \Rightarrow 'k \text{ bp-bucket}$ **where**
 $bp\text{-make-bucket } xs =$
 $\langle \langle bp\text{-marker} = \text{snd } (\text{hd } xs), bp\text{-bucket-entries} = xs \rangle \rangle$

fun $bp\text{-bucketize-sorted-entries-aux} ::$
 $\text{nat} \Rightarrow \text{nat} \Rightarrow ('k \times \text{real}) \text{ list} \Rightarrow 'k \text{ bp-bucket list}$ **where**
 $bp\text{-bucketize-sorted-entries-aux } 0 \ M \ xs = []$
 $| bp\text{-bucketize-sorted-entries-aux } (\text{Suc } \text{fuel}) \ M \ xs =$
 $(\text{if } M = 0 \vee xs = []$

```

then []
else bp-make-bucket (take M xs) #
  bp-bucketize-sorted-entries-aux fuel M (drop M xs))

```

definition *bp-bucketize-sorted-entries* ::
 $\text{nat} \Rightarrow ('k \times \text{real}) \text{ list} \Rightarrow 'k \text{ bp-bucket list}$ **where**
bp-bucketize-sorted-entries M xs =
bp-bucketize-sorted-entries-aux (length xs) M xs

definition *bp-bucketize-entries* ::
 $\text{nat} \Rightarrow ('k \times \text{real}) \text{ list} \Rightarrow 'k \text{ bp-bucket list}$ **where**
bp-bucketize-entries M xs = *bp-bucketize-sorted-entries* M (sort-key snd xs)

lemma *bp-bucketize-sorted-entries-aux-empty* [simp]:
bp-bucketize-sorted-entries-aux fuel M [] = []
by (cases fuel) simp-all

definition *bp-rebucket* :: $'k \text{ bucketed-partition} \Rightarrow 'k \text{ bucketed-partition}$ **where**
bp-rebucket P =
P(*bp-buckets* := *bp-bucketize-entries* (bp-block-size P) (bp-entries P))

fun *bp-local-insert-bucket* ::
 $\text{nat} \Rightarrow 'k \times \text{real} \Rightarrow 'k \text{ bp-bucket list} \Rightarrow 'k \text{ bp-bucket list}$ **where**
bp-local-insert-bucket M p [] = *bp-bucketize-entries* M [p]
| *bp-local-insert-bucket* M p [b] =
 (if snd p < bp-marker b
 then *bp-bucketize-entries* M [p] @ [b]
 else *bp-bucketize-entries* M (p # bp-bucket-entries b))
| *bp-local-insert-bucket* M p (b # c # bs) =
 (if snd p < bp-marker b
 then *bp-bucketize-entries* M [p] @ b # c # bs
 else if snd p < bp-marker c
 then *bp-bucketize-entries* M (p # bp-bucket-entries b) @ c # bs
 else b # *bp-local-insert-bucket* M p (c # bs))

definition *bp-lazy-bucket-insert-entries* ::
 $\text{nat} \Rightarrow 'k \times \text{real} \Rightarrow 'k \text{ bp-bucket} \Rightarrow 'k \text{ bp-bucket list}$ **where**
bp-lazy-bucket-insert-entries M p b =
 (if length (bp-bucket-entries b) < 2 * M
 then [b(*bp-bucket-entries* := p # bp-bucket-entries b)]
 else *bp-bucketize-entries* M (p # bp-bucket-entries b))

fun *bp-lazy-insert-bucket* ::
 $\text{nat} \Rightarrow 'k \times \text{real} \Rightarrow 'k \text{ bp-bucket list} \Rightarrow 'k \text{ bp-bucket list}$ **where**
bp-lazy-insert-bucket M p [] = *bp-bucketize-entries* M [p]
| *bp-lazy-insert-bucket* M p [b] =
 (if snd p < bp-marker b
 then *bp-bucketize-entries* M [p] @ [b]
 else *bp-lazy-bucket-insert-entries* M p b)

| *bp-lazy-insert-bucket* $M\ p\ (b\ \# \ c\ \# \ bs) =$
 (*if* $snd\ p < bp-marker\ b$
 then *bp-bucketize-entries* $M\ [p]\ @\ b\ \# \ c\ \# \ bs$
 else if $snd\ p < bp-marker\ c$
 then *bp-lazy-bucket-insert-entries* $M\ p\ b\ @\ c\ \# \ bs$
 else $b\ \# \ bp-lazy-insert-bucket\ M\ p\ (c\ \# \ bs)$)

fun *bp-insert-bucket* :: $'k \times real \Rightarrow 'k\ bp\ bucket\ list \Rightarrow 'k\ bp\ bucket\ list$ **where**
 bp-insert-bucket $p\ [] = [bp-singleton-bucket\ p]$
 | *bp-insert-bucket* $p\ (b\ \# \ bs) =$
 (*if* $snd\ p \leq bp-marker\ b$
 then *bp-singleton-bucket* $p\ \# \ b\ \# \ bs$
 else $b\ \# \ bp-insert-bucket\ p\ bs$)

definition *bp-delete-key-from-bucket* :: $'k \Rightarrow 'k\ bp\ bucket \Rightarrow 'k\ bp\ bucket$ **where**
 bp-delete-key-from-bucket $x\ b =$
 $P(\lambda p. fst\ p \neq x)\ (bp-bucket-entries\ b)$

definition *bp-delete-key* :: $'k \Rightarrow 'k\ bucketed-partition \Rightarrow 'k\ bucketed-partition$ **where**
 bp-delete-key $x\ P =$
 $P(\lambda bp-buckets := map\ (bp-delete-key-from-bucket\ x)\ (bp-buckets\ P))$

definition *bp-insert* ::
 $'k \Rightarrow real \Rightarrow 'k\ bucketed-partition \Rightarrow 'k\ bucketed-partition$ **where**
 bp-insert $x\ b\ P =$
 (*let* $b' = (if\ x \in bp-entry-keys\ (bp-entries\ P)$
 then $min\ (bp-values\ P\ x)\ b\ else\ b);$
 $P0 = bp-delete-key\ x\ P$
 in $P0(\lambda bp-buckets := bp-insert-bucket\ (x,\ b')\ (bp-buckets\ P0),$
 $bp-values := (bp-values\ P)(x := b'))$)

definition *bp-local-insert-state* ::
 $'k \Rightarrow real \Rightarrow 'k\ bucketed-partition \Rightarrow 'k\ bucketed-partition$ **where**
 bp-local-insert-state $x\ b\ P =$
 (*let* $b' = (if\ x \in bp-entry-keys\ (bp-entries\ P)$
 then $min\ (bp-values\ P\ x)\ b\ else\ b);$
 $P0 = bp-delete-key\ x\ P$
 in $P0(\lambda bp-buckets :=$
 $bp-local-insert-bucket\ (bp-block-size\ P0)\ (x,\ b')\ (bp-buckets\ P0),$
 $bp-values := (bp-values\ P)(x := b'))$)

definition *bp-lazy-insert-state* ::
 $'k \Rightarrow real \Rightarrow 'k\ bucketed-partition \Rightarrow 'k\ bucketed-partition$ **where**
 bp-lazy-insert-state $x\ b\ P =$
 (*let* $b' = (if\ x \in bp-entry-keys\ (bp-entries\ P)$
 then $min\ (bp-values\ P\ x)\ b\ else\ b);$
 $P0 = bp-delete-key\ x\ P$
 in $P0(\lambda bp-buckets :=$
 $bp-lazy-insert-bucket\ (bp-block-size\ P0)\ (x,\ b')\ (bp-buckets\ P0),$

$bp\text{-}values := (bp\text{-}values\ P)(x := b')$

definition $bp\text{-}batch\text{-}prepend ::$

$('k \times real) list \Rightarrow 'k\ bucketed\text{-}partition \Rightarrow 'k\ bucketed\text{-}partition$ **where**
 $bp\text{-}batch\text{-}prepend\ xs\ P =$
 $fold\ (\lambda(x, b)\ P'.\ bp\text{-}insert\ x\ b\ P')\ xs\ P$

fun $bp\text{-}batch\text{-}value\text{-}update ::$

$('k \times real) list \Rightarrow ('k \Rightarrow real) \Rightarrow 'k \Rightarrow real$ **where**
 $bp\text{-}batch\text{-}value\text{-}update\ []\ f = f$
 $| bp\text{-}batch\text{-}value\text{-}update\ ((x, b) \# xs)\ f =$
 $bp\text{-}batch\text{-}value\text{-}update\ xs\ (f(x := b))$

fun $bp\text{-}batch\text{-}max\text{-}value :: real \Rightarrow ('k \times real) list \Rightarrow real$ **where**

$bp\text{-}batch\text{-}max\text{-}value\ beta\ [] = beta$
 $| bp\text{-}batch\text{-}max\text{-}value\ beta\ ((x, b) \# xs) =$
 $bp\text{-}batch\text{-}max\text{-}value\ (max\ beta\ b)\ xs$

fun $bp\text{-}rebase\text{-}first\text{-}bucket\text{-}marker ::$

$real \Rightarrow 'k\ bp\text{-}bucket\ list \Rightarrow 'k\ bp\text{-}bucket\ list$ **where**
 $bp\text{-}rebase\text{-}first\text{-}bucket\text{-}marker\ beta\ [] = []$
 $| bp\text{-}rebase\text{-}first\text{-}bucket\text{-}marker\ beta\ (b \# bs) =$
 $b(bp\text{-}marker := beta) \# bs$

fun $bp\text{-}drop\text{-}empty\text{-}prefix :: 'k\ bp\text{-}bucket\ list \Rightarrow 'k\ bp\text{-}bucket\ list$ **where**

$bp\text{-}drop\text{-}empty\text{-}prefix\ [] = []$
 $| bp\text{-}drop\text{-}empty\text{-}prefix\ (b \# bs) =$
 $(if\ bp\text{-}bucket\text{-}entries\ b = []\ then\ bp\text{-}drop\text{-}empty\text{-}prefix\ bs\ else\ b \# bs)$

definition $bp\text{-}bucketed\text{-}batch\text{-}prepend\text{-}state ::$

$('k \times real) list \Rightarrow 'k\ bucketed\text{-}partition \Rightarrow$
 $'k\ bucketed\text{-}partition$ **where**
 $bp\text{-}bucketed\text{-}batch\text{-}prepend\text{-}state\ xs\ P =$
 $(case\ xs\ of$
 $\quad [] \Rightarrow P$
 $\quad | p \# ps \Rightarrow$
 $\quad (let\ beta = bp\text{-}batch\text{-}max\text{-}value\ (snd\ p)\ ps;$
 $\quad \quad new = bp\text{-}bucketize\text{-}entries\ (bp\text{-}block\text{-}size\ P)\ xs;$
 $\quad \quad old = bp\text{-}rebase\text{-}first\text{-}bucket\text{-}marker\ beta$
 $\quad \quad (bp\text{-}drop\text{-}empty\text{-}prefix\ (bp\text{-}buckets\ P))$
 $\quad in\ P(bp\text{-}buckets := new\ @\ old,$
 $\quad \quad bp\text{-}values := bp\text{-}batch\text{-}value\text{-}update\ xs\ (bp\text{-}values\ P)))$

definition $bp\text{-}delete\text{-}keys\text{-}from\text{-}bucket :: 'k\ set \Rightarrow 'k\ bp\text{-}bucket \Rightarrow 'k\ bp\text{-}bucket$ **where**

$bp\text{-}delete\text{-}keys\text{-}from\text{-}bucket\ S\ b =$
 $b(bp\text{-}bucket\text{-}entries := filter\ (\lambda p.\ fst\ p \notin S)\ (bp\text{-}bucket\text{-}entries\ b))$

definition $bp\text{-}delete\text{-}keys :: 'k\ set \Rightarrow 'k\ bucketed\text{-}partition \Rightarrow 'k\ bucketed\text{-}partition$
where

$bp\text{-delete-keys } S \ P =$
 $P(\lambda bp\text{-buckets} := \text{map } (bp\text{-delete-keys-from-bucket } S) \ (bp\text{-buckets } P))$

fun $bp\text{-min-value} :: \text{real} \Rightarrow ('k \times \text{real}) \text{ list} \Rightarrow \text{real}$ **where**
 $bp\text{-min-value } B \ [] = B$
 $| bp\text{-min-value } B \ ((x, b) \# xs) = \min b \ (bp\text{-min-value } B \ xs)$

definition $bp\text{-bucket-below-bound} :: 'k \ bp\text{-bucket} \Rightarrow \text{real} \Rightarrow \text{bool}$ **where**
 $bp\text{-bucket-below-bound } b \ \text{beta} \longleftrightarrow$
 $(\forall p \in \text{set } (bp\text{-bucket-entries } b). \text{snd } p < \text{beta})$

definition $bp\text{-first-bucket-pull} ::$
 $\text{nat} \Rightarrow \text{real} \Rightarrow 'k \ \text{bucketed-partition} \Rightarrow$
 $'k \ \text{set} \times \text{real} \times 'k \ \text{bucketed-partition}$ **where**
 $bp\text{-first-bucket-pull } M \ B \ P =$
 $(\text{case } bp\text{-buckets } P \ \text{of}$
 $\quad b \# c \# bs \Rightarrow$
 $\quad (\text{let } S = bp\text{-bucket-keys } b$
 $\quad \quad \text{in } (S, bp\text{-marker } c, bp\text{-delete-keys } S \ P))$
 $| _ \Rightarrow (\{\}, B, P))$

definition $bp\text{-pull-set} :: \text{nat} \Rightarrow 'k \ \text{bucketed-partition} \Rightarrow 'k \ \text{set}$ **where**
 $bp\text{-pull-set } M \ P =$
 $(\text{if } \text{length } (bp\text{-entries } P) \leq M$
 $\quad \text{then } bp\text{-entry-keys } (bp\text{-entries } P)$
 $\quad \text{else } \{\})$

definition $bp\text{-pull-bound} :: \text{nat} \Rightarrow \text{real} \Rightarrow 'k \ \text{bucketed-partition} \Rightarrow \text{real}$ **where**
 $bp\text{-pull-bound } M \ B \ P =$
 $(\text{if } \text{length } (bp\text{-entries } P) \leq M$
 $\quad \text{then } B$
 $\quad \text{else } bp\text{-min-value } B \ (bp\text{-entries } P))$

definition $bp\text{-conservative-pull} ::$
 $\text{nat} \Rightarrow \text{real} \Rightarrow 'k \ \text{bucketed-partition} \Rightarrow$
 $'k \ \text{set} \times \text{real} \times 'k \ \text{bucketed-partition}$ **where**
 $bp\text{-conservative-pull } M \ B \ P =$
 $(\text{let } S = bp\text{-pull-set } M \ P;$
 $\quad \text{beta} = bp\text{-pull-bound } M \ B \ P$
 $\quad \text{in } (S, \text{beta}, bp\text{-delete-keys } S \ P))$

definition $bp\text{-can-first-bucket-pull} ::$
 $\text{nat} \Rightarrow 'k \ \text{bucketed-partition} \Rightarrow \text{bool}$ **where**
 $bp\text{-can-first-bucket-pull } M \ P \longleftrightarrow$
 $(\text{case } bp\text{-buckets } P \ \text{of}$
 $\quad b \# c \# bs \Rightarrow$
 $\quad \text{length } (bp\text{-entries } P) > M \wedge$
 $\quad \text{length } (bp\text{-bucket-entries } b) \leq M \wedge$
 $\quad bp\text{-bucket-below-bound } b \ (bp\text{-marker } c) \wedge$

$bp\text{-bucket-entries-flat } (c \# bs) \neq []$
 $| - \Rightarrow \text{False})$

definition *bp-pull* ::
 $\text{nat} \Rightarrow \text{real} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow$
 $'k \text{ set} \times \text{real} \times 'k \text{ bucketed-partition}$ **where**
bp-pull *M B P* =
 (if *bp-can-first-bucket-pull* *M P*
 then *bp-first-bucket-pull* *M B P*
 else *bp-conservative-pull* *M B P*)

lemma *bp-can-first-bucket-pullE*:
assumes *bp-can-first-bucket-pull* *M P*
obtains *b c bs* **where**
 $bp\text{-buckets } P = b \# c \# bs$
 $\text{length } (bp\text{-entries } P) > M$
 $\text{length } (bp\text{-bucket-entries } b) \leq M$
 $bp\text{-bucket-below-bound } b \text{ (bp-marker } c)$
 $bp\text{-bucket-entries-flat } (c \# bs) \neq []$
using *assms*
proof (cases *bp-buckets* *P*)
 case *Nil*
 then show ?thesis
 using *assms* **unfolding** *bp-can-first-bucket-pull-def* **by** *simp*
next
 case (Cons *b rest*)
 note *buckets-outer* = Cons
 then show ?thesis
 proof (cases *rest*)
 case *Nil*
 then show ?thesis
 using *assms* *buckets-outer* **unfolding** *bp-can-first-bucket-pull-def* **by** *simp*
next
 case (Cons *c bs*)
 have *buckets*: $bp\text{-buckets } P = b \# c \# bs$
 using *buckets-outer* Cons **by** *simp*
 have *len-entries*: $\text{length } (bp\text{-entries } P) > M$
 using *assms* *buckets* **unfolding** *bp-can-first-bucket-pull-def* **by** *simp*
 have *len-b*: $\text{length } (bp\text{-bucket-entries } b) \leq M$
 using *assms* *buckets* **unfolding** *bp-can-first-bucket-pull-def* **by** *simp*
 have *below*: $bp\text{-bucket-below-bound } b \text{ (bp-marker } c)$
 using *assms* *buckets* **unfolding** *bp-can-first-bucket-pull-def* **by** *simp*
 have *tail-nonempty*: $bp\text{-bucket-entries-flat } (c \# bs) \neq []$
 using *assms* *buckets* **unfolding** *bp-can-first-bucket-pull-def* **by** *simp*
 show ?thesis
 by (rule that[OF *buckets len-entries len-b below tail-nonempty*])
qed
qed

lemma *bp-entry-keys-filter-neq* [simp]:
 $bp_entry_keys\ (filter\ (\lambda p.\ fst\ p \neq x)\ xs) = bp_entry_keys\ xs - \{x\}$
unfolding *bp-entry-keys-def* **by** *auto*

lemma *bp-entry-keys-filter-notin* [simp]:
 $bp_entry_keys\ (filter\ (\lambda p.\ fst\ p \notin S)\ xs) = bp_entry_keys\ xs - S$
unfolding *bp-entry-keys-def* **by** *auto*

lemma *bp-bucket-keys-alt* [simp]:
 $bp_bucket_keys\ b = fst\ 'set\ (bp_bucket_entries\ b)$
unfolding *bp-bucket-keys-def bp-entry-keys-def* **by** *simp*

lemma *bp-entries-empty* [simp]:
 $bp_entries\ (bp_empty\ M\ B) = []$
unfolding *bp-entries-def bp-empty-def bp-bucket-entries-flat-def* **by** *simp*

lemma *bp-bucket-entries-flat-append* [simp]:
 $bp_bucket_entries_flat\ (bs\ @\ cs) =$
 $bp_bucket_entries_flat\ bs\ @\ bp_bucket_entries_flat\ cs$
unfolding *bp-bucket-entries-flat-def* **by** *simp*

lemma *bp-bucket-entries-flat-rebase-first-bucket-marker* [simp]:
 $bp_bucket_entries_flat\ (bp_rebase_first_bucket_marker\ beta\ bs) =$
 $bp_bucket_entries_flat\ bs$
by (*cases bs*) (*simp-all add: bp-bucket-entries-flat-def*)

lemma *bp-bucket-entries-flat-drop-empty-prefix* [simp]:
 $bp_bucket_entries_flat\ (bp_drop_empty_prefix\ bs) =$
 $bp_bucket_entries_flat\ bs$
by (*induction bs*) (*simp-all add: bp-bucket-entries-flat-def*)

lemma *length-bp-rebase-first-bucket-marker* [simp]:
 $length\ (bp_rebase_first_bucket_marker\ beta\ bs) = length\ bs$
by (*cases bs*) *simp-all*

lemma *length-bp-drop-empty-prefix-le* [simp]:
 $length\ (bp_drop_empty_prefix\ bs) \leq length\ bs$
by (*induction bs*) *auto*

lemma *bp-entry-keys-rebase-first-bucket-marker* [simp]:
 bp_entry_keys
 $(bp_bucket_entries_flat\ (bp_rebase_first_bucket_marker\ beta\ bs)) =$
 $bp_entry_keys\ (bp_bucket_entries_flat\ bs)$
by *simp*

lemma *bp-entry-keys-drop-empty-prefix* [simp]:
 $bp_entry_keys\ (bp_bucket_entries_flat\ (bp_drop_empty_prefix\ bs)) =$
 $bp_entry_keys\ (bp_bucket_entries_flat\ bs)$
by *simp*

```

lemma bp-drop-empty-prefix-set-subset:
  set (bp-drop-empty-prefix bs) ⊆ set bs
by (induction bs) auto

lemma bp-drop-empty-prefix-head-nonempty:
  assumes bp-drop-empty-prefix bs = b # bs'
  shows bp-bucket-entries b ≠ []
  using assms
proof (induction bs)
  case Nil
  then show ?case by simp
next
  case (Cons a bs)
  show ?case
  proof (cases bp-bucket-entries a = [])
    case True
    then show ?thesis
    using Cons.IH Cons.prems by simp
  next
  case False
  then have a = b
    using Cons.prems by simp
  then show ?thesis
    using False by simp
  qed
qed

lemma bp-bucket-boundaries-ok-drop-empty-prefix:
  assumes bp-bucket-boundaries-ok bs
  shows bp-bucket-boundaries-ok (bp-drop-empty-prefix bs)
  using assms
proof (induction bs)
  case Nil
  then show ?case by simp
next
  case (Cons b bs)
  show ?case
  proof (cases bp-bucket-entries b = [])
    case True
    have bp-bucket-boundaries-ok bs
    using Cons.prems by (cases bs) simp-all
    then show ?thesis
    using True Cons.IH by simp
  next
  case False
  then show ?thesis
    using Cons.prems by simp
  qed

```

qed

lemma *bp-bucket-markers-sorted-drop-empty-prefix*:
 assumes *sorted-wrt* ($\lambda b\ c. \text{bp-marker } b \leq \text{bp-marker } c$) *bs*
 shows *sorted-wrt* ($\lambda b\ c. \text{bp-marker } b \leq \text{bp-marker } c$)
 (*bp-drop-empty-prefix bs*)
 using *assms*
proof (*induction bs*)
 case *Nil*
 then show ?case **by** *simp*
next
 case (*Cons b bs*)
 show ?case
 proof (*cases bp-bucket-entries b = []*)
 case *True*
 have *sorted-wrt* ($\lambda b\ c. \text{bp-marker } b \leq \text{bp-marker } c$) *bs*
 using *Cons.prem*s **by** *simp*
 then show ?thesis
 using *True Cons.IH* **by** *simp*
next
 case *False*
 then show ?thesis
 using *Cons.prem*s **by** *simp*
qed
qed

lemma *bp-bucket-boundaries-ok-rebase-first-bucket-marker* [*simp*]:
 bp-bucket-boundaries-ok (*bp-rebase-first-bucket-marker beta bs*) =
 bp-bucket-boundaries-ok bs
proof (*cases bs*)
 case *Nil*
 then show ?thesis **by** *simp*
next
 case (*Cons b rest*)
 then show ?thesis
 by (*cases rest*) *simp-all*
qed

lemma *bp-batch-value-update-notin*:
 assumes $x \notin \text{fst } \text{'set } xs$
 shows *bp-batch-value-update xs f x = f x*
 using *assms* **by** (*induction xs arbitrary: f*) *auto*

lemma *bp-batch-value-update-in-set-distinct*:
 assumes *distinct*: *distinct* (*map fst xs*)
 and *member*: $(x, b) \in \text{set } xs$
 shows *bp-batch-value-update xs f x = b*
 using *assms*
proof (*induction xs arbitrary: f*)

```

    case Nil
    then show ?case by simp
next
case (Cons p xs)
obtain y c where p-def: p = (y, c)
  by force
show ?case
proof (cases x = y)
  case True
  have x-not-tail:  $x \notin \text{fst } \text{'set } xs$ 
    using True Cons.prem p-def by auto
  then have (x, b)  $\notin \text{set } xs$ 
    by force
  then have (x, b) = (y, c)
    using Cons.prem p-def by auto
  then have bc: b = c
    by simp
  have y-not-tail:  $y \notin \text{fst } \text{'set } xs$ 
    using x-not-tail True by simp
  have unchanged:
    bp-batch-value-update xs (f(y := c)) y = c
    using bp-batch-value-update-notin[OF y-not-tail, of f(y := c)]
    by simp
  then show ?thesis
    unfolding p-def True bc
    by simp
next
case False
then have (x, b)  $\in \text{set } xs$ 
  using Cons.prem p-def by auto
moreover have distinct (map fst xs)
  using Cons.prem p-def by simp
ultimately show ?thesis
  unfolding p-def by (simp add: Cons.IH)
qed
qed

```

```

lemma batch-min-update-value-distinct-disjoint:
  assumes distinct: distinct (map fst xs)
    and disjoint:  $\text{fst } \text{'set } xs \cap \text{keys-of } D = \{\}$ 
  shows value-of (batch-min-update D xs) =
    bp-batch-value-update xs (value-of D)
  using distinct disjoint
proof (induction xs arbitrary: D)
  case Nil
  then show ?case
    unfolding batch-min-update-def by simp
next
case (Cons p xs)

```



```

obtain  $x\ b$  where  $p\text{-def}: p = (x, b)$ 
  by force
have  $\text{distinct-tail}: \text{distinct } (\text{map } \text{fst } xs)$ 
  using  $\text{Cons.premis } p\text{-def}$  by simp
have  $x\text{-not-tail}: x \notin \text{fst } \text{'set } xs$ 
  using  $\text{Cons.premis } p\text{-def}$  by auto
have  $x\text{-not-}D: x \notin \text{keys-of } D$ 
  using  $\text{Cons.premis } p\text{-def}$  by auto
have  $\text{disjoint-tail}: \text{fst } \text{'set } xs \cap \text{keys-of } (\text{min-update } D\ x\ b) = \{\}$ 
  using  $\text{Cons.premis } x\text{-not-tail } p\text{-def}$ 
  unfolding  $\text{min-update-def}$  by auto
have  $\text{tail}: \text{value-of } (\text{batch-min-update } (\text{min-update } D\ x\ b)\ xs) =$ 
   $\text{bp-batch-value-update } xs\ (\text{value-of } (\text{min-update } D\ x\ b))$ 
  by  $(\text{rule } \text{Cons.IH}[\text{OF } \text{distinct-tail } \text{disjoint-tail}])$ 
have  $\text{fold-tail}: \text{value-of } (\text{fold } (\lambda(x, b)\ D'. \text{min-update } D'\ x\ b)\ xs\ (\text{min-update } D\ x\ b)) =$ 
   $\text{bp-batch-value-update } xs\ (\text{value-of } (\text{min-update } D\ x\ b))$ 
  using  $\text{tail unfolding batch-min-update-def .}$ 
have  $\text{step-values}: \text{value-of } (\text{min-update } D\ x\ b) = (\text{value-of } D)(x := b)$ 
  using  $x\text{-not-}D$  unfolding  $\text{min-update-def}$  by simp
show  $?case$ 
  unfolding  $p\text{-def}$ 
  by  $(\text{simp add: batch-min-update-def fold-tail step-values})$ 
qed

lemma  $\text{bp-batch-max-value-upper}: \text{assumes } \bigwedge p. p \in \text{set } xs \implies \text{snd } p \leq \text{gamma}$ 
  and  $\text{beta} \leq \text{gamma}$ 
  shows  $\text{bp-batch-max-value } \text{beta } xs \leq \text{gamma}$ 
  using  $\text{assms}$ 
proof  $(\text{induction } xs\ \text{arbitrary: beta})$ 
  case  $\text{Nil}$ 
  then show  $?case$  by simp
next
  case  $(\text{Cons } p\ xs)$ 
  obtain  $x\ b$  where  $p\text{-def}: p = (x, b)$ 
  by force
  have  $\text{head-le}: b \leq \text{gamma}$ 
  using  $\text{Cons.premis}(1)[\text{of } p]\ p\text{-def}$  by simp
  have  $\text{max-le}: \text{max } \text{beta } b \leq \text{gamma}$ 
  using  $\text{Cons.premis}(2)\ \text{head-le}$  by simp
  have  $\text{bp-batch-max-value } (\text{max } \text{beta } (\text{snd } p))\ xs \leq \text{gamma}$ 
  proof  $(\text{rule } \text{Cons.IH})$ 
  fix  $q$ 
  assume  $q \in \text{set } xs$ 

```

```

    then show  $\text{snd } q \leq \text{gamma}$ 
    using Cons.prems by simp
qed (use max-le p-def in simp)
then show ?case
    unfolding p-def by simp
qed

```

```

lemma bp-batch-max-value-ge-initial:
   $\text{beta} \leq \text{bp-batch-max-value } \text{beta } xs$ 
proof (induction xs arbitrary: beta)
  case Nil
  then show ?case by simp
next
  case (Cons p xs)
  obtain x b where p-def:  $p = (x, b)$ 
  by force
  have  $\text{beta} \leq \text{max } \text{beta } b$ 
  by simp
  moreover have  $\text{max } \text{beta } b \leq \text{bp-batch-max-value } (\text{max } \text{beta } b) xs$ 
  by (rule Cons.IH)
  ultimately have  $\text{beta} \leq \text{bp-batch-max-value } (\text{max } \text{beta } b) xs$ 
  by linarith
  then show ?case
    unfolding p-def by simp
qed

```

```

lemma bp-batch-max-value-ge-member:
  assumes  $p \in \text{set } xs$ 
  shows  $\text{snd } p \leq \text{bp-batch-max-value } \text{beta } xs$ 
  using assms
proof (induction xs arbitrary: beta)
  case Nil
  then show ?case by simp
next
  case (Cons q xs)
  obtain y c where q-def:  $q = (y, c)$ 
  by force
  show ?case
  proof (cases  $p = q$ )
    case True
    have  $\text{snd } p \leq \text{max } \text{beta } c$ 
    using True q-def by simp
    moreover have  $\text{max } \text{beta } c \leq \text{bp-batch-max-value } (\text{max } \text{beta } c) xs$ 
    by (rule bp-batch-max-value-ge-initial)
    ultimately have  $\text{snd } p \leq \text{bp-batch-max-value } (\text{max } \text{beta } c) xs$ 
    by linarith
  then show ?thesis
    unfolding q-def by simp
  next

```

```

    case False
    then have  $p \in \text{set } xs$ 
      using Cons.prems by simp
    then show ?thesis
      unfolding q-def by (simp add: Cons.IH)
  qed
qed

lemma bp-batch-max-value-ge-member-Cons:
  assumes  $q \in \text{set } (p \# ps)$ 
  shows  $\text{snd } q \leq \text{bp-batch-max-value } (\text{snd } p) \text{ } ps$ 
  using assms
  by (cases q = p)
    (simp-all add: bp-batch-max-value-ge-initial bp-batch-max-value-ge-member)

lemma bp-bucket-entries-flat-bucketize-sorted-entries-aux:
  assumes  $0 < M$ 
    and  $\text{length } xs \leq \text{fuel}$ 
  shows bp-bucket-entries-flat
    (bp-bucketize-sorted-entries-aux fuel M xs) = xs
  using assms
proof (induction fuel arbitrary: xs)
  case 0
  then have  $xs = []$ 
    by simp
  then show ?case
    by (simp add: bp-bucket-entries-flat-def)
next
  case (Suc fuel)
  show ?case
  proof (cases xs = [])
    case True
    then show ?thesis
      by (simp add: bp-bucket-entries-flat-def)
  next
    case False
    have len-drop:  $\text{length } (\text{drop } M \text{ } xs) \leq \text{fuel}$ 
      using Suc.prems False by (cases M) auto
    have tail:
      bp-bucket-entries-flat
        (bp-bucketize-sorted-entries-aux fuel M (drop M xs)) =
        drop M xs
      by (rule Suc.IH[OF Suc.prems(1) len-drop])
    show ?thesis
      using Suc.prems False tail
      by (simp add: bp-bucket-entries-flat-def bp-make-bucket-def)
  qed
qed

```

lemma *bp-bucket-entries-flat-bucketize-sorted-entries*:
assumes $0 < M$
shows *bp-bucket-entries-flat* (*bp-bucketize-sorted-entries* M xs) = xs
using *assms*
unfolding *bp-bucketize-sorted-entries-def*
by (*simp add: bp-bucket-entries-flat-bucketize-sorted-entries-aux*)

lemma *bp-bucket-entries-flat-bucketize-entries*:
assumes $0 < M$
shows *bp-bucket-entries-flat* (*bp-bucketize-entries* M xs) = *sort-key snd* xs
using *assms*
unfolding *bp-bucketize-entries-def*
by (*simp add: bp-bucket-entries-flat-bucketize-sorted-entries*)

lemma *set-bp-bucket-entries-flat-bucketize-entries*:
assumes $0 < M$
shows *set* (*bp-bucket-entries-flat* (*bp-bucketize-entries* M xs)) = *set* xs
using *assms*
by (*simp add: bp-bucket-entries-flat-bucketize-entries*)

lemma *bp-entry-keys-bucketize-entries [simp]*:
assumes $0 < M$
shows *bp-entry-keys* (*bp-bucket-entries-flat* (*bp-bucketize-entries* M xs)) =
bp-entry-keys xs
using *assms*
unfolding *bp-entry-keys-def*
by (*simp add: set-bp-bucket-entries-flat-bucketize-entries*)

lemma *bp-bucketize-sorted-entries-aux-sizes-ok*:
assumes $0 < M$
shows $\forall b \in \text{set } (bp\text{-bucketize-sorted-entries-aux fuel } M \text{ } xs).$
length (*bp-bucket-entries* b) $\leq M$
using *assms*
by (*induction fuel arbitrary: xs*) (*auto simp: bp-make-bucket-def*)

lemma *bp-bucketize-sorted-entries-sizes-ok*:
assumes $0 < M$
shows $\forall b \in \text{set } (bp\text{-bucketize-sorted-entries } M \text{ } xs).$
length (*bp-bucket-entries* b) $\leq M$
using *assms*
unfolding *bp-bucketize-sorted-entries-def*
by (*rule bp-bucketize-sorted-entries-aux-sizes-ok*)

lemma *bp-bucketize-entries-sizes-ok*:
assumes $0 < M$
shows $\forall b \in \text{set } (bp\text{-bucketize-entries } M \text{ } xs).$
length (*bp-bucket-entries* b) $\leq M$
using *assms*
unfolding *bp-bucketize-entries-def*

```

    by (rule bp-bucketize-sorted-entries-sizes-ok)

lemma bp-sorted-wrt-snd-sort-key [simp]:
  sorted-wrt ( $\lambda p q. \text{snd } p \leq \text{snd } q$ ) (sort-key snd xs)
proof -
  have sorted (map snd (sort-key snd xs))
    by (rule sorted-sort-key)
  then show ?thesis
    by (simp add: sorted-map)
qed

lemma sorted-wrt-hd-snd-le:
  fixes xs :: ('k  $\times$  real) list
  assumes sorted: sorted-wrt ( $\lambda p q. \text{snd } p \leq \text{snd } q$ ) xs
    and xs: xs  $\neq []$ 
    and p: p  $\in \text{set } xs$ 
  shows snd (hd xs)  $\leq \text{snd } p$ 
proof (cases xs)
  case Nil
  then show ?thesis
    using xs by simp
next
  case (Cons q qs)
  show ?thesis
  proof (cases p = q)
    case True
    have snd (hd xs) = snd p
      using Cons True by simp
    then show ?thesis
      by simp
  next
    case False
    then have p  $\in \text{set } qs$ 
      using p Cons by simp
    moreover have  $\forall r \in \text{set } qs. \text{snd } q \leq \text{snd } r$ 
      using sorted Cons by simp
    ultimately show ?thesis
      using Cons by simp
  qed
qed

lemma sorted-wrt-snd-take-drop-le:
  fixes xs :: ('k  $\times$  real) list
  assumes sorted: sorted-wrt ( $\lambda p q. \text{snd } p \leq \text{snd } q$ ) xs
    and p: p  $\in \text{set } (\text{take } n \text{ } xs)$ 
    and q: q  $\in \text{set } (\text{drop } n \text{ } xs)$ 
  shows snd p  $\leq \text{snd } q$ 
proof -
  have sorted-wrt ( $\lambda p q. \text{snd } p \leq \text{snd } q$ ) (take n xs @ drop n xs)

```

using *sorted* by *simp*
 then have $\forall p \in \text{set } (\text{take } n \text{ } xs). \forall q \in \text{set } (\text{drop } n \text{ } xs). \text{snd } p \leq \text{snd } q$
 unfolding *sorted-wrt-append* by *blast*
 then show *?thesis*
 using *p q* by *blast*
 qed

lemma *bp-bucketize-sorted-entries-aux-marker-in-set*:
 assumes $b \in \text{set } (\text{bp-bucketize-sorted-entries-aux fuel } M \text{ } xs)$
 shows $\exists p \in \text{set } xs. \text{bp-marker } b = \text{snd } p$
 using *assms*
proof (*induction fuel arbitrary: xs*)
 case 0
 then show *?case*
 by *simp*
next
 case (*Suc fuel*)
 show *?case*
proof (*cases M = 0 $\vee$ xs = []*)
 case *True*
 then show *?thesis*
 using *Suc.prem*s by *simp*
next
 case *False*
 have *M-pos*: $0 < M$
 using *False* by *simp*
 have *xs-nonempty*: $xs \neq []$
 using *False* by *simp*
 from *Suc.prem*s have *b-cases*:
 $b = \text{bp-make-bucket } (\text{take } M \text{ } xs) \vee$
 $b \in \text{set } (\text{bp-bucketize-sorted-entries-aux fuel } M \text{ } (\text{drop } M \text{ } xs))$
 using *False* by *simp*
 from *b-cases* show *?thesis*
proof
 assume *b-def*: $b = \text{bp-make-bucket } (\text{take } M \text{ } xs)$
 have *hd* $(\text{take } M \text{ } xs) \in \text{set } xs$
 using *M-pos xs-nonempty* by (*cases xs*) *auto*
 then show *?thesis*
 unfolding *b-def bp-make-bucket-def* by *auto*
next
 assume *b-tail*:
 $b \in \text{set } (\text{bp-bucketize-sorted-entries-aux fuel } M \text{ } (\text{drop } M \text{ } xs))$
 obtain *p* where *p-drop*: $p \in \text{set } (\text{drop } M \text{ } xs)$
 and *marker*: $\text{bp-marker } b = \text{snd } p$
 using *Suc.IH[OF b-tail]* by *auto*
 have $p \in \text{set } xs$
 by (*meson in-set-dropD p-drop*)
 then show *?thesis*
 using *marker* by *auto*

```

    qed
  qed
qed

lemma bp-bucketize-sorted-entries-aux-markers-lower-bound:
  fixes xs :: ('k × real) list
  assumes M-pos: 0 < M
    and sorted: sorted-wrt (λp q. snd p ≤ snd q) xs
  shows ∀ b ∈ set (bp-bucketize-sorted-entries-aux fuel M xs).
    ∀ p ∈ set (bp-bucket-entries b). bp-marker b ≤ snd p
  using M-pos sorted
proof (induction fuel arbitrary: xs)
  case 0
  then show ?case
    by simp
next
  case (Suc fuel)
  show ?case
  proof (cases xs = [])
    case True
    then show ?thesis
      by simp
  next
    case False
    have current:
      ∀ p ∈ set (bp-bucket-entries (bp-make-bucket (take M xs))).
        bp-marker (bp-make-bucket (take M xs)) ≤ snd p
    proof
      fix p
      assume p: p ∈ set (bp-bucket-entries (bp-make-bucket (take M xs)))
      then have p-take: p ∈ set (take M xs)
        unfolding bp-make-bucket-def by simp
      have sorted-take: sorted-wrt (λp q. snd p ≤ snd q) (take M xs)
        using Suc.prem by simp
      have take-nonempty: take M xs ≠ []
        using p-take by (cases take M xs) auto
      have snd (hd (take M xs)) ≤ snd p
        by (rule sorted-wrt-hd-snd-le[OF sorted-take take-nonempty p-take])
      then show bp-marker (bp-make-bucket (take M xs)) ≤ snd p
        unfolding bp-make-bucket-def by simp
    qed
  have tail:
    ∀ b ∈ set (bp-bucketize-sorted-entries-aux fuel M (drop M xs)).
      ∀ p ∈ set (bp-bucket-entries b). bp-marker b ≤ snd p
    by (rule Suc.IH) (use Suc.prem in simp-all)
  show ?thesis
    using False current tail Suc.prem by simp
  qed
qed

```

lemma *bp-bucketize-sorted-entries-markers-lower-bound*:
fixes $xs :: ('k \times \text{real}) \text{ list}$
assumes $0 < M$
and *sorted-wrt* $(\lambda p q. \text{snd } p \leq \text{snd } q) \text{ } xs$
shows $\forall b \in \text{set } (bp\text{-bucketize-sorted-entries } M \text{ } xs).$
 $\forall p \in \text{set } (bp\text{-bucket-entries } b). \text{bp-marker } b \leq \text{snd } p$
using *assms*
unfolding *bp-bucketize-sorted-entries-def*
by (rule *bp-bucketize-sorted-entries-aux-markers-lower-bound*)

lemma *bp-bucketize-entries-markers-lower-bound*:
assumes $0 < M$
shows $\forall b \in \text{set } (bp\text{-bucketize-entries } M \text{ } xs).$
 $\forall p \in \text{set } (bp\text{-bucket-entries } b). \text{bp-marker } b \leq \text{snd } p$
using *assms*
unfolding *bp-bucketize-entries-def*
by (rule *bp-bucketize-sorted-entries-markers-lower-bound*) *simp*

lemma *bp-bucketize-sorted-entries-aux-markers-sorted*:
fixes $xs :: ('k \times \text{real}) \text{ list}$
assumes *M-pos*: $0 < M$
and *sorted*: *sorted-wrt* $(\lambda p q. \text{snd } p \leq \text{snd } q) \text{ } xs$
shows *sorted-wrt* $(\lambda b c. \text{bp-marker } b \leq \text{bp-marker } c) \text{ } (bp\text{-bucketize-sorted-entries-aux fuel } M \text{ } xs)$
using *M-pos sorted*
proof (*induction fuel arbitrary: xs*)
case 0
then show ?*case*
by *simp*
next
case (*Suc fuel*)
show ?*case*
proof (*cases xs = []*)
case *True*
then show ?*thesis*
by *simp*
next
case *False*
let ?*tail* = *bp-bucketize-sorted-entries-aux fuel M (drop M xs)*
have *tail-sorted*:
 $\text{sorted-wrt } (\lambda b c. \text{bp-marker } b \leq \text{bp-marker } c) \text{ } ?tail$
by (rule *Suc.IH*) (use *Suc.prem*s **in** *simp-all*)
have *current-le-tail*:
 $\forall c \in \text{set } ?tail.$
 $\text{bp-marker } (bp\text{-make-bucket } (take \text{ } M \text{ } xs)) \leq \text{bp-marker } c$
proof
fix *c*
assume *c*: $c \in \text{set } ?tail$


```

obtain  $p$  where  $p\text{-drop}$ :  $p \in \text{set } (\text{drop } M \text{ } xs)$ 
  and  $c\text{-marker}$ :  $\text{bp-marker } c = \text{snd } p$ 
  using  $\text{bp-bucketize-sorted-entries-aux-marker-in-set}[OF \ c]$  by  $\text{auto}$ 
have  $p\text{-xs}$ :  $p \in \text{set } xs$ 
  by  $(\text{meson in-set-dropD } p\text{-drop})$ 
have  $\text{head-le}$ :  $\text{snd } (\text{hd } xs) \leq \text{snd } p$ 
  by  $(\text{rule sorted-wrt-hd-snd-le}[OF \ \text{Suc.prems}(2) \ \text{False } p\text{-xs}])$ 
have  $\text{marker-eq}$ :  $\text{bp-marker } (\text{bp-make-bucket } (\text{take } M \text{ } xs)) = \text{snd } (\text{hd } xs)$ 
  using  $\text{Suc.prems} \ \text{False}$  by  $(\text{cases } xs) \ (\text{auto simp: bp-make-bucket-def})$ 
show  $\text{bp-marker } (\text{bp-make-bucket } (\text{take } M \text{ } xs)) \leq \text{bp-marker } c$ 
  using  $\text{head-le marker-eq } c\text{-marker}$  by  $\text{simp}$ 
qed
show  $?thesis$ 
  using  $\text{False tail-sorted current-le-tail } \text{Suc.prems}$  by  $\text{simp}$ 
qed
qed

```

```

lemma  $\text{bp-bucketize-sorted-entries-markers-sorted}$ :
  fixes  $xs :: ('k \times \text{real}) \text{ list}$ 
  assumes  $0 < M$ 
  and  $\text{sorted-wrt } (\lambda p \ q. \text{snd } p \leq \text{snd } q) \ xs$ 
  shows  $\text{sorted-wrt } (\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c)$ 
     $(\text{bp-bucketize-sorted-entries } M \text{ } xs)$ 
  using  $\text{assms}$ 
  unfolding  $\text{bp-bucketize-sorted-entries-def}$ 
  by  $(\text{rule bp-bucketize-sorted-entries-aux-markers-sorted})$ 

```

```

lemma  $\text{bp-bucketize-entries-markers-sorted}$ :
  assumes  $0 < M$ 
  shows  $\text{sorted-wrt } (\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c)$ 
     $(\text{bp-bucketize-entries } M \text{ } xs)$ 
  using  $\text{assms}$ 
  unfolding  $\text{bp-bucketize-entries-def}$ 
  by  $(\text{rule bp-bucketize-sorted-entries-markers-sorted}) \ \text{simp}$ 

```

```

lemma  $\text{bp-bucketize-entries-marker-in-set}$ :
  assumes  $b \in \text{set } (\text{bp-bucketize-entries } M \text{ } xs)$ 
  shows  $\exists p \in \text{set } xs. \text{bp-marker } b = \text{snd } p$ 
proof –
  obtain  $p$  where  $p\text{-sort}$ :  $p \in \text{set } (\text{sort-key snd } xs)$ 
  and  $\text{marker}$ :  $\text{bp-marker } b = \text{snd } p$ 
  using  $\text{assms}$ 
  unfolding  $\text{bp-bucketize-entries-def bp-bucketize-sorted-entries-def}$ 
  by  $(\text{auto dest: bp-bucketize-sorted-entries-aux-marker-in-set})$ 
  then have  $p \in \text{set } xs$ 
  by  $\text{simp}$ 
  then show  $?thesis$ 
  using  $\text{marker}$  by  $\text{auto}$ 
qed

```

```

lemma bp-bucketize-sorted-entries-aux-boundaries-ok:
  fixes xs :: ('k × real) list
  assumes M-pos: 0 < M
    and sorted: sorted-wrt (λp q. snd p ≤ snd q) xs
  shows bp-bucket-boundaries-ok
    (bp-bucketize-sorted-entries-aux fuel M xs)
  using M-pos sorted
proof (induction fuel arbitrary: xs)
  case 0
  then show ?case
    by simp
next
  case (Suc fuel)
  show ?case
  proof (cases xs = [])
    case True
    then show ?thesis
      by simp
  next
  case False
  let ?tail = bp-bucketize-sorted-entries-aux fuel M (drop M xs)
  have tail-boundaries: bp-bucket-boundaries-ok ?tail
    by (rule Suc.IH) (use Suc.prems in simp-all)
  show ?thesis
  proof (cases ?tail)
    case Nil
    then show ?thesis
      using False Suc.prems by simp
  next
  case (Cons c cs)
  have c-in: c ∈ set ?tail
    using Cons by simp
  obtain q where q-drop: q ∈ set (drop M xs)
    and c-marker: bp-marker c = snd q
    using bp-bucketize-sorted-entries-aux-marker-in-set[OF c-in] by auto
  have current-before-c:
    ∀ p ∈ set (bp-bucket-entries (bp-make-bucket (take M xs))).
      snd p ≤ bp-marker c
  proof
    fix p
    assume p:
      p ∈ set (bp-bucket-entries (bp-make-bucket (take M xs)))
    then have p-take: p ∈ set (take M xs)
      unfolding bp-make-bucket-def by simp
    have snd p ≤ snd q
      by (rule sorted-wrt-snd-take-drop-le[OF Suc.prems(2) p-take q-drop])
    then show snd p ≤ bp-marker c
      using c-marker by simp

```

```

    qed
    show ?thesis
    using False Cons tail-boundaries current-before-c Suc.prem by simp
  qed
qed
qed

```

```

lemma bp-bucketize-sorted-entries-boundaries-ok:
  fixes xs :: ('k × real) list
  assumes 0 < M
    and sorted-wrt (λp q. snd p ≤ snd q) xs
  shows bp-bucket-boundaries-ok (bp-bucketize-sorted-entries M xs)
  using assms
  unfolding bp-bucketize-sorted-entries-def
  by (rule bp-bucketize-sorted-entries-aux-boundaries-ok)

```

```

lemma bp-bucketize-entries-boundaries-ok:
  assumes 0 < M
  shows bp-bucket-boundaries-ok (bp-bucketize-entries M xs)
  using assms
  unfolding bp-bucketize-entries-def
  by (rule bp-bucketize-sorted-entries-boundaries-ok) simp

```

```

lemma distinct-map-fst-sort-key:
  assumes distinct (map fst xs)
  shows distinct (map fst (sort-key f xs))
proof -
  have distinct-sort: distinct (sort-key f xs)
    using assms unfolding distinct-map by simp
  have inj: inj-on fst (set (sort-key f xs))
    using assms unfolding distinct-map by simp
  show ?thesis
    unfolding distinct-map using distinct-sort inj by simp
qed

```

```

lemma bp-rebucket-view [simp]:
  assumes 0 < bp-block-size P
  shows bp-view (bp-rebucket P) = bp-view P
  using assms
  unfolding bp-rebucket-def bp-view-def bp-entries-def
  by simp

```

```

lemma bp-rebucket-invariant:
  assumes inv: bp-invariant P
  shows bp-invariant (bp-rebucket P)
proof -
  let ?M = bp-block-size P
  let ?xs = bp-entries P
  have M-pos: 0 < ?M

```

```

using inv unfolding bp-invariant-def by blast
have distinct-xs: distinct (map fst ?xs)
using inv unfolding bp-invariant-def bp-distinct-keys-def by blast
have vals:
   $\forall p \in \text{set } ?xs. \text{bp-values } P (\text{fst } p) = \text{snd } p$ 
using inv unfolding bp-invariant-def bp-values-consistent-def by blast
have flat:
  bp-bucket-entries-flat (bp-bucketize-entries ?M ?xs) = sort-key snd ?xs
by (rule bp-bucket-entries-flat-bucketize-entries[OF M-pos])
have flat-rebucket:
  bp-entries (bp-rebucket P) = sort-key snd ?xs
using flat unfolding bp-rebucket-def bp-entries-def by simp
have sizes-ok:
   $\forall b \in \text{set } (\text{bp-bucketize-entries } ?M ?xs).$ 
     $\text{length } (\text{bp-bucket-entries } b) \leq ?M$ 
by (rule bp-bucketize-entries-sizes-ok[OF M-pos])
have markers-sorted:
  sorted-wrt ( $\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c$ )
    (bp-bucketize-entries ?M ?xs)
by (rule bp-bucketize-entries-markers-sorted[OF M-pos])
have markers-lower:
   $\forall b \in \text{set } (\text{bp-bucketize-entries } ?M ?xs).$ 
     $\forall p \in \text{set } (\text{bp-bucket-entries } b). \text{bp-marker } b \leq \text{snd } p$ 
by (rule bp-bucketize-entries-markers-lower-bound[OF M-pos])
have set-flat:
  set (bp-bucket-entries-flat (bp-bucketize-entries ?M ?xs)) = set ?xs
by (rule set-bp-bucket-entries-flat-bucketize-entries[OF M-pos])
show ?thesis
  unfolding bp-invariant-def
proof (intro conjI)
  show  $0 < \text{bp-block-size } (\text{bp-rebucket } P)$ 
    using M-pos unfolding bp-rebucket-def by simp
  show bp-distinct-keys (bp-rebucket P)
    using distinct-map-fst-sort-key[OF distinct-xs, of snd]
    unfolding bp-distinct-keys-def flat-rebucket by simp
  show bp-bucket-sizes-ok (bp-rebucket P)
    using sizes-ok
    unfolding bp-rebucket-def bp-bucket-sizes-ok-def by simp
  show bp-bucket-markers-sorted (bp-rebucket P)
    using markers-sorted
    unfolding bp-rebucket-def bp-bucket-markers-sorted-def by simp
  show bp-bucket-markers-lower-bound (bp-rebucket P)
    using markers-lower
    unfolding bp-rebucket-def bp-bucket-markers-lower-bound-def by simp
  show bp-values-consistent (bp-rebucket P)
    using vals set-flat
    unfolding bp-rebucket-def bp-values-consistent-def bp-entries-def
      bp-entry-keys-def
    by auto

```

qed
qed

lemma *bp-rebucket-boundaries-state-ok*:
assumes *inv*: *bp-invariant P*
shows *bp-bucket-boundaries-state-ok (bp-rebucket P)*
proof –
have *M-pos*: $0 < \text{bp-block-size } P$
using *inv* **unfolding** *bp-invariant-def* **by** *blast*
show *?thesis*
using *bp-bucketize-entries-boundaries-ok*[*OF M-pos*, *of bp-entries P*]
unfolding *bp-bucket-boundaries-state-ok-def bp-rebucket-def* **by** *simp*
qed

lemma *bp-rebucket-ordered-invariant*:
assumes *inv*: *bp-invariant P*
shows *bp-ordered-invariant (bp-rebucket P)*
unfolding *bp-ordered-invariant-def*
using *bp-rebucket-invariant*[*OF inv*] *bp-rebucket-boundaries-state-ok*[*OF inv*]
by *blast*

Rebucketing is the canonical repair operation. It sorts the flat entry list by value, chunks it into blocks of size *bp-block-size*, and rebuilds markers from the first entry of each block. The view theorem $0 < \text{bp-block-size } ?P \implies \text{bp-view } (\text{bp-rebucket } ?P) = \text{bp-view } ?P$ says this repair is observationally invisible at the abstract partition interface: keys and remembered values do not change. The invariant theorems above then show that rebucketing restores strict bucket sizes, sorted markers, lower-bound markers, and adjacent bucket boundaries. Later regularized operations use this fact when a lazy step has accumulated enough debt to warrant a rebuild.

lemma *bp-ordered-invariant-invariant*:
assumes *bp-ordered-invariant P*
shows *bp-invariant P*
using *assms* **unfolding** *bp-ordered-invariant-def* **by** *blast*

lemma *bp-ordered-invariant-boundaries-state-ok*:
assumes *bp-ordered-invariant P*
shows *bp-bucket-boundaries-state-ok P*
using *assms* **unfolding** *bp-ordered-invariant-def* **by** *blast*

lemma *bp-invariant-lazy-invariant*:
assumes *inv*: *bp-invariant P*
shows *bp-lazy-invariant P*
proof –
have *lazy-sizes*: *bp-lazy-bucket-sizes-ok P*
proof –
have $\forall b \in \text{set } (\text{bp-buckets } P).$
 $\text{length } (\text{bp-bucket-entries } b) \leq 2 * \text{bp-block-size } P$

```

    using inv unfolding bp-invariant-def bp-bucket-sizes-ok-def
    by auto
  then show ?thesis
    unfolding bp-lazy-bucket-sizes-ok-def .
qed
show ?thesis
  using inv lazy-sizes
  unfolding bp-invariant-def bp-lazy-invariant-def
  by blast
qed

lemma bp-ordered-invariant-lazy-ordered-invariant:
  assumes ord: bp-ordered-invariant P
  shows bp-lazy-ordered-invariant P
  using bp-invariant-lazy-invariant[OF bp-ordered-invariant-invariant[OF ord]]
    bp-ordered-invariant-boundaries-state-ok[OF ord]
  unfolding bp-lazy-ordered-invariant-def by blast

lemma bp-lazy-ordered-invariant-lazy-invariant:
  assumes bp-lazy-ordered-invariant P
  shows bp-lazy-invariant P
  using assms unfolding bp-lazy-ordered-invariant-def by blast

lemma bp-lazy-ordered-invariant-boundaries-state-ok:
  assumes bp-lazy-ordered-invariant P
  shows bp-bucket-boundaries-state-ok P
  using assms unfolding bp-lazy-ordered-invariant-def by blast

lemma bp-rebucket-invariant-from-lazy:
  assumes inv: bp-lazy-invariant P
  shows bp-invariant (bp-rebucket P)
proof -
  let ?M = bp-block-size P
  let ?xs = bp-entries P
  have M-pos: 0 < ?M
    using inv unfolding bp-lazy-invariant-def by blast
  have distinct-xs: distinct (map fst ?xs)
    using inv unfolding bp-lazy-invariant-def bp-distinct-keys-def by blast
  have vals:
     $\forall p \in \text{set } ?xs. \text{bp-values } P (\text{fst } p) = \text{snd } p$ 
    using inv unfolding bp-lazy-invariant-def bp-values-consistent-def
    by blast
  have flat:
    bp-bucket-entries-flat (bp-bucketize-entries ?M ?xs) = sort-key snd ?xs
    by (rule bp-bucket-entries-flat-bucketize-entries[OF M-pos])
  have flat-rebucket:
    bp-entries (bp-rebucket P) = sort-key snd ?xs
    using flat unfolding bp-rebucket-def bp-entries-def by simp
  have sizes-ok:

```

```

   $\forall b \in \text{set } (\text{bp-bucketize-entries } ?M \text{ } ?xs).$ 
   $\text{length } (\text{bp-bucket-entries } b) \leq ?M$ 
  by (rule bp-bucketize-entries-sizes-ok[OF M-pos])
have markers-sorted:
  sorted-wrt ( $\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c$ )
  ( $\text{bp-bucketize-entries } ?M \text{ } ?xs$ )
  by (rule bp-bucketize-entries-markers-sorted[OF M-pos])
have markers-lower:
   $\forall b \in \text{set } (\text{bp-bucketize-entries } ?M \text{ } ?xs).$ 
   $\forall p \in \text{set } (\text{bp-bucket-entries } b). \text{bp-marker } b \leq \text{snd } p$ 
  by (rule bp-bucketize-entries-markers-lower-bound[OF M-pos])
have set-flat:
   $\text{set } (\text{bp-bucket-entries-flat } (\text{bp-bucketize-entries } ?M \text{ } ?xs)) = \text{set } ?xs$ 
  by (rule set-bp-bucket-entries-flat-bucketize-entries[OF M-pos])
show ?thesis
  unfolding bp-invariant-def
proof (intro conjI)
  show  $0 < \text{bp-block-size } (\text{bp-rebucket } P)$ 
    using M-pos unfolding bp-rebucket-def by simp
  show bp-distinct-keys ( $\text{bp-rebucket } P$ )
    using distinct-map-fst-sort-key[OF distinct-xs, of snd]
    unfolding bp-distinct-keys-def flat-rebucket by simp
  show bp-bucket-sizes-ok ( $\text{bp-rebucket } P$ )
    using sizes-ok
    unfolding bp-rebucket-def bp-bucket-sizes-ok-def by simp
  show bp-bucket-markers-sorted ( $\text{bp-rebucket } P$ )
    using markers-sorted
    unfolding bp-rebucket-def bp-bucket-markers-sorted-def by simp
  show bp-bucket-markers-lower-bound ( $\text{bp-rebucket } P$ )
    using markers-lower
    unfolding bp-rebucket-def bp-bucket-markers-lower-bound-def by simp
  show bp-values-consistent ( $\text{bp-rebucket } P$ )
    using vals set-flat
    unfolding bp-rebucket-def bp-values-consistent-def bp-entries-def
    bp-entry-keys-def
    by auto
qed
qed

lemma bp-rebucket-boundaries-state-ok-from-lazy:
  assumes inv: bp-lazy-invariant P
  shows bp-bucket-boundaries-state-ok ( $\text{bp-rebucket } P$ )
proof -
  have M-pos:  $0 < \text{bp-block-size } P$ 
    using inv unfolding bp-lazy-invariant-def by blast
  show ?thesis
    using bp-bucketize-entries-boundaries-ok[OF M-pos, of bp-entries P]
    unfolding bp-bucket-boundaries-state-ok-def bp-rebucket-def by simp
qed

```

```

lemma bp-rebucket-ordered-invariant-from-lazy:
  assumes inv: bp-lazy-invariant P
  shows bp-ordered-invariant (bp-rebucket P)
  unfolding bp-ordered-invariant-def
  using bp-rebucket-invariant-from-lazy[OF inv]
  bp-rebucket-boundaries-state-ok-from-lazy[OF inv]
  by blast

lemma set-bp-bucket-entries-flat-local-insert-bucket:
  assumes  $0 < M$ 
  shows  $\text{set } (\text{bp-bucket-entries-flat } (\text{bp-local-insert-bucket } M \ p \ bs)) =$ 
     $\text{insert } p \ (\text{set } (\text{bp-bucket-entries-flat } bs))$ 
  using assms
proof (induction bs arbitrary: p)
  case Nil
  have  $\text{set } (\text{bp-bucket-entries-flat } (\text{bp-bucketize-entries } M \ [p])) = \text{set } [p]$ 
    by (rule set-bp-bucket-entries-flat-bucketize-entries[OF Nil.prem])
  then show ?case
    by (simp add: bp-bucket-entries-flat-def)
next
  case (Cons b bs)
  note IH = Cons.IH
  note prems = Cons.prems
  show ?case
  proof (cases bs)
    case Nil
    have single:
       $\text{set } (\text{bp-bucket-entries-flat } (\text{bp-bucketize-entries } M \ [p])) = \{p\}$ 
      using set-bp-bucket-entries-flat-bucketize-entries[OF prems, of [p]]
      by simp
    have inserted:
       $\text{set } (\text{bp-bucket-entries-flat } (\text{bp-bucketize-entries } M \ (p \# \text{bp-bucket-entries } b))) =$ 
         $\text{insert } p \ (\text{set } (\text{bp-bucket-entries-flat } b))$ 
      using set-bp-bucket-entries-flat-bucketize-entries
        [OF prems, of p # bp-bucket-entries b]
      by simp
    then show ?thesis
      using Nil single inserted
      by (auto simp: bp-bucket-entries-flat-def)
  next
  case (Cons c cs)
  have single:
     $\text{set } (\text{bp-bucket-entries-flat } (\text{bp-bucketize-entries } M \ [p])) = \{p\}$ 
    using set-bp-bucket-entries-flat-bucketize-entries[OF prems, of [p]]
    by simp
  have inserted:
     $\text{set } (\text{bp-bucket-entries-flat } (\text{bp-bucketize-entries } M \ (p \# \text{bp-bucket-entries } b))) =$ 
       $\text{insert } p \ (\text{set } (\text{bp-bucket-entries-flat } b))$ 
    using set-bp-bucket-entries-flat-bucketize-entries
      [OF prems, of p # bp-bucket-entries b]
    by simp
  then show ?thesis
    using single inserted
    by (auto simp: bp-bucket-entries-flat-def)

```



```

      (bp-bucketize-entries M (p # bp-bucket-entries b))) =
      insert p (set (bp-bucket-entries b))
    using set-bp-bucket-entries-flat-bucketize-entries
      [OF prems, of p # bp-bucket-entries b]
    by simp
  have tail:
    set (bp-bucket-entries-flat (bp-local-insert-bucket M p bs)) =
    insert p (set (bp-bucket-entries-flat bs))
  by (rule IH[OF prems])
  then show ?thesis
    using Cons single inserted tail
    by (auto simp: bp-bucket-entries-flat-def)
qed
qed

lemma set-bp-bucket-entries-flat-lazy-bucket-insert-entries:
  assumes 0 < M
  shows set (bp-bucket-entries-flat
    (bp-lazy-bucket-insert-entries M p b)) =
    insert p (set (bp-bucket-entries b))
proof (cases length (bp-bucket-entries b) < 2 * M)
  case True
  then show ?thesis
    unfolding bp-lazy-bucket-insert-entries-def bp-bucket-entries-flat-def
    by simp
next
  case False
  have set (bp-bucket-entries-flat
    (bp-bucketize-entries M (p # bp-bucket-entries b))) =
    set (p # bp-bucket-entries b)
  by (rule set-bp-bucket-entries-flat-bucketize-entries[OF assms])
  then show ?thesis
    using False unfolding bp-lazy-bucket-insert-entries-def by simp
qed

lemma set-bp-bucket-entries-flat-lazy-insert-bucket:
  assumes 0 < M
  shows set (bp-bucket-entries-flat (bp-lazy-insert-bucket M p bs)) =
    insert p (set (bp-bucket-entries-flat bs))
  using assms
proof (induction bs arbitrary: p)
  case Nil
  have set (bp-bucket-entries-flat (bp-bucketize-entries M [p])) = set [p]
  by (rule set-bp-bucket-entries-flat-bucketize-entries[OF Nil.prems])
  then show ?case
    by (simp add: bp-bucket-entries-flat-def)
next
  case (Cons b bs)
  note IH = Cons.IH

```

```

note  $M\text{-pos} = \text{Cons.prem}$ s
show ?case
proof (cases bs)
  case Nil
  have single:
    set (bp-bucket-entries-flat (bp-bucketize-entries M [p])) = {p}
    using set-bp-bucket-entries-flat-bucketize-entries[OF M-pos, of [p]]
    by simp
  have inserted:
    set (bp-bucket-entries-flat
      (bp-lazy-bucket-insert-entries M p b)) =
      insert p (set (bp-bucket-entries b))
    by (rule set-bp-bucket-entries-flat-lazy-bucket-insert-entries
      [OF M-pos])
  show ?thesis
    using Nil single inserted by (auto simp: bp-bucket-entries-flat-def)
next
  case (Cons c cs)
  have single:
    set (bp-bucket-entries-flat (bp-bucketize-entries M [p])) = {p}
    using set-bp-bucket-entries-flat-bucketize-entries[OF M-pos, of [p]]
    by simp
  have inserted:
    set (bp-bucket-entries-flat
      (bp-lazy-bucket-insert-entries M p b)) =
      insert p (set (bp-bucket-entries b))
    by (rule set-bp-bucket-entries-flat-lazy-bucket-insert-entries
      [OF M-pos])
  have tail:
    set (bp-bucket-entries-flat (bp-lazy-insert-bucket M p bs)) =
      insert p (set (bp-bucket-entries-flat bs))
    by (rule IH[OF M-pos])
  show ?thesis
    using Cons single inserted tail
    by (auto simp: bp-bucket-entries-flat-def)
qed
qed

lemma length-bp-bucket-entries-flat-lazy-bucket-insert-entries [simp]:
  assumes  $0 < M$ 
  shows length (bp-bucket-entries-flat
    (bp-lazy-bucket-insert-entries M p b)) =
    Suc (length (bp-bucket-entries b))
proof (cases length (bp-bucket-entries b) < 2 * M)
  case True
  then show ?thesis
    unfolding bp-lazy-bucket-insert-entries-def bp-bucket-entries-flat-def
    by simp
next

```

```

case False
have length (bp-bucket-entries-flat
  (bp-bucketize-entries M (p # bp-bucket-entries b))) =
  length (p # bp-bucket-entries b)
using bp-bucket-entries-flat-bucketize-entries
  [OF assms, of p # bp-bucket-entries b]
by simp
then show ?thesis
  using False unfolding bp-lazy-bucket-insert-entries-def by simp
qed

lemma length-bp-bucket-entries-flat-lazy-insert-bucket [simp]:
  assumes 0 < M
  shows length (bp-bucket-entries-flat (bp-lazy-insert-bucket M p bs)) =
    Suc (length (bp-bucket-entries-flat bs))
  using assms
proof (induction bs arbitrary: p)
  case Nil
  have len:
    length (bp-bucket-entries-flat (bp-bucketize-entries M [p])) =
      length [p]
  using bp-bucket-entries-flat-bucketize-entries [OF Nil.prems, of [p]]
  by simp
  show ?case
    using len by (simp add: bp-bucket-entries-flat-def)
next
  case (Cons b bs)
  note IH = Cons.IH
  note M-pos = Cons.prems
  show ?case
  proof (cases bs)
    case Nil
    have len-single:
      length (bp-bucket-entries-flat (bp-bucketize-entries M [p])) =
        length [p]
    using bp-bucket-entries-flat-bucketize-entries [OF M-pos, of [p]]
    by simp
    have len-inserted:
      length (bp-bucket-entries-flat
        (bp-lazy-bucket-insert-entries M p b)) =
        Suc (length (bp-bucket-entries b))
    by (rule length-bp-bucket-entries-flat-lazy-bucket-insert-entries
      [OF M-pos])
    show ?thesis
      using Nil len-single len-inserted
      by (auto simp: bp-bucket-entries-flat-def)
  next
  case (Cons c cs)
  have len-single:

```

```

length (bp-bucket-entries-flat (bp-bucketize-entries M [p])) =
  length [p]
using bp-bucket-entries-flat-bucketize-entries[OF M-pos, of [p]]
by simp
have len-inserted:
  length (bp-bucket-entries-flat
    (bp-lazy-bucket-insert-entries M p b)) =
    Suc (length (bp-bucket-entries b))
by (rule length-bp-bucket-entries-flat-lazy-bucket-insert-entries
  [OF M-pos])
have tail-len:
  length (bp-bucket-entries-flat
    (bp-lazy-insert-bucket M p (c # cs))) =
    Suc (length (bp-bucket-entries-flat (c # cs)))
using IH[OF M-pos, of p] Cons by simp
show ?thesis
proof (cases snd p < bp-marker b)
  case True
  then show ?thesis
    using Cons len-single by (simp add: bp-bucket-entries-flat-def)
next
  case False
  show ?thesis
proof (cases snd p < bp-marker c)
  case True
  then show ?thesis
    using Cons False len-inserted
    by (simp add: bp-bucket-entries-flat-def)
next
  case False
  then show ?thesis
    using Cons <¬ snd p < bp-marker b> tail-len
    by (simp add: bp-bucket-entries-flat-def)
qed
qed
qed
qed

```

```

lemma bp-lazy-insert-bucket-distinct-keys:
  assumes M-pos: 0 < M
  and distinct: distinct (map fst (bp-bucket-entries-flat bs))
  and fresh: fst p ∉ bp-entry-keys (bp-bucket-entries-flat bs)
  shows distinct
    (map fst (bp-bucket-entries-flat (bp-lazy-insert-bucket M p bs)))
proof (rule card-distinct)
  let ?old = map fst (bp-bucket-entries-flat bs)
  let ?new =
    map fst (bp-bucket-entries-flat (bp-lazy-insert-bucket M p bs))
  have set-new: set ?new = insert (fst p) (set ?old)

```

```

    using set-bp-bucket-entries-flat-lazy-insert-bucket[OF M-pos, of p bs]
    by auto
  have fresh-set: fst p  $\notin$  set ?old
    using fresh unfolding bp-entry-keys-def by simp
  have card (set ?new) = Suc (card (set ?old))
    unfolding set-new using fresh-set by simp
  also have ... = Suc (length ?old)
    using distinct-card[OF distinct] by simp
  also have ... = length ?new
    using length-bp-bucket-entries-flat-lazy-insert-bucket[OF M-pos, of p bs]
    by simp
  finally show card (set ?new) = length ?new .
qed

```

```

lemma bp-lazy-bucket-insert-entries-sizes-ok:
  assumes M-pos: 0 < M
    and size: length (bp-bucket-entries b)  $\leq$  2 * M
  shows  $\forall c \in \text{set } (bp\text{-lazy-bucket-insert-entries } M\ p\ b).$ 
    length (bp-bucket-entries c)  $\leq$  2 * M
proof (cases length (bp-bucket-entries b) < 2 * M)
case True
  then have length (p # bp-bucket-entries b)  $\leq$  2 * M
    by simp
  then show ?thesis
    using True unfolding bp-lazy-bucket-insert-entries-def by simp
next
case False
  have strict-sizes:
     $\forall c \in \text{set } (bp\text{-bucketize-entries } M\ (p \# bp\text{-bucket-entries } b)).$ 
    length (bp-bucket-entries c)  $\leq$  M
    by (rule bp-bucketize-entries-sizes-ok[OF M-pos])
  have  $\forall c \in \text{set } (bp\text{-bucketize-entries } M\ (p \# bp\text{-bucket-entries } b)).$ 
    length (bp-bucket-entries c)  $\leq$  2 * M
    using strict-sizes by auto
  then show ?thesis
    using False unfolding bp-lazy-bucket-insert-entries-def by simp
qed

```

```

lemma bp-lazy-insert-bucket-sizes-ok:
  assumes M-pos: 0 < M
    and sizes:  $\forall b \in \text{set } bs. \text{length } (bp\text{-bucket-entries } b) \leq 2 * M$ 
  shows  $\forall b \in \text{set } (bp\text{-lazy-insert-bucket } M\ p\ bs).$ 
    length (bp-bucket-entries b)  $\leq$  2 * M
  using sizes
proof (induction bs arbitrary: p)
case Nil
  have strict:
     $\forall b \in \text{set } (bp\text{-bucketize-entries } M\ [p]).$ 
    length (bp-bucket-entries b)  $\leq$  M

```

```

    by (rule bp-bucketize-entries-sizes-ok[OF M-pos])
  then show ?case
    using M-pos by auto
next
case (Cons b bs)
note IH = Cons.IH
note prems = Cons.prems
show ?case
proof (cases bs)
case Nil
have single:
   $\forall b \in \text{set } (bp\text{-bucketize-entries } M [p]).$ 
   $\text{length } (bp\text{-bucket-entries } b) \leq 2 * M$ 
  using bp-bucketize-entries-sizes-ok[OF M-pos, of [p]] by auto
have inserted:
   $\forall c \in \text{set } (bp\text{-lazy-bucket-insert-entries } M p b).$ 
   $\text{length } (bp\text{-bucket-entries } c) \leq 2 * M$ 
  by (rule bp-lazy-bucket-insert-entries-sizes-ok[OF M-pos])
  (use prems Nil in simp)
have b-size:  $\text{length } (bp\text{-bucket-entries } b) \leq 2 * M$ 
  using prems by simp
then show ?thesis
  using Nil single inserted b-size by auto
next
case (Cons c cs)
have tail-sizes:
   $\forall b \in \text{set } bs. \text{length } (bp\text{-bucket-entries } b) \leq 2 * M$ 
  using prems by simp
have single:
   $\forall b \in \text{set } (bp\text{-bucketize-entries } M [p]).$ 
   $\text{length } (bp\text{-bucket-entries } b) \leq 2 * M$ 
  using bp-bucketize-entries-sizes-ok[OF M-pos, of [p]] by auto
have inserted:
   $\forall d \in \text{set } (bp\text{-lazy-bucket-insert-entries } M p b).$ 
   $\text{length } (bp\text{-bucket-entries } d) \leq 2 * M$ 
  by (rule bp-lazy-bucket-insert-entries-sizes-ok[OF M-pos])
  (use prems Cons in simp)
show ?thesis
  using IH[of p, OF tail-sizes] prems Cons single inserted by auto
qed
qed

lemma bp-local-insert-bucket-sizes-ok:
  assumes M-pos:  $0 < M$ 
  and sizes:  $\forall b \in \text{set } bs. \text{length } (bp\text{-bucket-entries } b) \leq M$ 
  shows  $\forall b \in \text{set } (bp\text{-local-insert-bucket } M p bs).$ 
   $\text{length } (bp\text{-bucket-entries } b) \leq M$ 
  using sizes
proof (induction bs arbitrary: p)

```

```

case Nil
have  $\forall b \in \text{set } (bp\text{-bucketize-entries } M \ [p]).$ 
   $\text{length } (bp\text{-bucket-entries } b) \leq M$ 
  by (rule bp-bucketize-entries-sizes-ok[OF M-pos])
then show ?case
  by simp
next
case (Cons b bs)
note IH = Cons.IH
note prems = Cons.prems
show ?case
proof (cases bs)
  case Nil
  then show ?thesis
    using prems bp-bucketize-entries-sizes-ok[OF M-pos, of [p]]
    bp-bucketize-entries-sizes-ok[OF M-pos, of p # bp-bucket-entries b]
    by auto
  next
  case (Cons c cs)
  have tail-sizes:
     $\forall b \in \text{set } bs. \text{length } (bp\text{-bucket-entries } b) \leq M$ 
    using prems by simp
  show ?thesis
    using IH[of p, OF tail-sizes] prems Cons
    bp-bucketize-entries-sizes-ok[OF M-pos, of [p]]
    bp-bucketize-entries-sizes-ok[OF M-pos, of p # bp-bucket-entries b]
    by auto
  qed
qed

lemma bp-local-insert-bucket-markers-lower-bound:
assumes M-pos: 0 < M
  and lower:  $\forall b \in \text{set } bs. \forall p \in \text{set } (bp\text{-bucket-entries } b).$ 
    bp-marker b  $\leq$  snd p
shows  $\forall b \in \text{set } (bp\text{-local-insert-bucket } M \ p \ bs).$ 
   $\forall q \in \text{set } (bp\text{-bucket-entries } b). \text{bp-marker } b \leq \text{snd } q$ 
using lower
proof (induction bs arbitrary: p)
  case Nil
  have  $\forall b \in \text{set } (bp\text{-bucketize-entries } M \ [p]).$ 
     $\forall q \in \text{set } (bp\text{-bucket-entries } b). \text{bp-marker } b \leq \text{snd } q$ 
    by (rule bp-bucketize-entries-markers-lower-bound[OF M-pos])
  then show ?case
    by simp
  next
  case (Cons b bs)
  note IH = Cons.IH
  note prems = Cons.prems
  show ?case

```

```

proof (cases bs)
  case Nil
  then show ?thesis
    using prems bp-bucketize-entries-markers-lower-bound[OF M-pos, of [p]]
      bp-bucketize-entries-markers-lower-bound
      [OF M-pos, of p # bp-bucket-entries b]
    by auto
  next
    case (Cons c cs)
    have tail-lower:
       $\forall b \in \text{set } bs. \forall p \in \text{set } (bp\text{-bucket-entries } b).$ 
       $bp\text{-marker } b \leq \text{snd } p$ 
    using prems by simp
    show ?thesis
      using IH[of p, OF tail-lower] prems Cons
      bp-bucketize-entries-markers-lower-bound[OF M-pos, of [p]]
      bp-bucketize-entries-markers-lower-bound
      [OF M-pos, of p # bp-bucket-entries b]
    by auto
  qed
qed

```

```

lemma bp-lazy-bucket-insert-entries-markers-lower-bound:
  assumes M-pos:  $0 < M$ 
  and lower:  $\forall q \in \text{set } (bp\text{-bucket-entries } b). bp\text{-marker } b \leq \text{snd } q$ 
  and p-ge:  $bp\text{-marker } b \leq \text{snd } p$ 
  shows  $\forall c \in \text{set } (bp\text{-lazy-bucket-insert-entries } M p b).$ 
   $\forall q \in \text{set } (bp\text{-bucket-entries } c). bp\text{-marker } c \leq \text{snd } q$ 
proof (cases length (bp-bucket-entries b) < 2 * M)
  case True
  then show ?thesis
    using lower p-ge unfolding bp-lazy-bucket-insert-entries-def by auto
  next
    case False
    have  $\forall c \in \text{set } (bp\text{-bucketize-entries } M (p \# bp\text{-bucket-entries } b)).$ 
     $\forall q \in \text{set } (bp\text{-bucket-entries } c). bp\text{-marker } c \leq \text{snd } q$ 
    by (rule bp-bucketize-entries-markers-lower-bound[OF M-pos])
    then show ?thesis
      using False unfolding bp-lazy-bucket-insert-entries-def by simp
  qed

```

```

lemma bp-lazy-insert-bucket-markers-lower-bound:
  assumes M-pos:  $0 < M$ 
  and lower:  $\forall b \in \text{set } bs. \forall p \in \text{set } (bp\text{-bucket-entries } b).$ 
   $bp\text{-marker } b \leq \text{snd } p$ 
  shows  $\forall b \in \text{set } (bp\text{-lazy-insert-bucket } M p bs).$ 
   $\forall q \in \text{set } (bp\text{-bucket-entries } b). bp\text{-marker } b \leq \text{snd } q$ 
  using lower
proof (induction bs arbitrary: p)

```



```

case Nil
show ?case
  by (simp add: bp-bucketize-entries-markers-lower-bound[OF M-pos])
next
case (Cons b bs)
note IH = Cons.IH
note prems = Cons.prems
show ?case
proof (cases bs)
  case Nil
  have single:
     $\forall b \in \text{set } (bp\text{-bucketize-entries } M \ [p]).$ 
     $\forall q \in \text{set } (bp\text{-bucket-entries } b). \text{ bp-marker } b \leq \text{snd } q$ 
    by (rule bp-bucketize-entries-markers-lower-bound[OF M-pos])
  have b-lower:
     $\forall q \in \text{set } (bp\text{-bucket-entries } b). \text{ bp-marker } b \leq \text{snd } q$ 
    using prems by simp
  have inserted:
     $\text{bp-marker } b \leq \text{snd } p \implies$ 
     $\forall c \in \text{set } (bp\text{-lazy-bucket-insert-entries } M \ p \ b).$ 
     $\forall q \in \text{set } (bp\text{-bucket-entries } c). \text{ bp-marker } c \leq \text{snd } q$ 
    by (rule bp-lazy-bucket-insert-entries-markers-lower-bound
      [OF M-pos b-lower])
  show ?thesis
proof (cases snd p < bp-marker b)
  case True
  show ?thesis
    using Nil True single b-lower by auto
next
  case False
  have p-ge-b:  $\text{bp-marker } b \leq \text{snd } p$ 
    using False by simp
  show ?thesis
    using Nil False inserted[OF p-ge-b] by simp
qed
next
case (Cons c cs)
have tail-lower:
   $\forall b \in \text{set } bs. \forall p \in \text{set } (bp\text{-bucket-entries } b).$ 
   $\text{bp-marker } b \leq \text{snd } p$ 
  using prems by simp
have single:
   $\forall b \in \text{set } (bp\text{-bucketize-entries } M \ [p]).$ 
   $\forall q \in \text{set } (bp\text{-bucket-entries } b). \text{ bp-marker } b \leq \text{snd } q$ 
  by (rule bp-bucketize-entries-markers-lower-bound[OF M-pos])
have b-lower:
   $\forall q \in \text{set } (bp\text{-bucket-entries } b). \text{ bp-marker } b \leq \text{snd } q$ 
  using prems by simp
have inserted:

```

```

    bp-marker b ≤ snd p ⇒
      ∀ d ∈ set (bp-lazy-bucket-insert-entries M p b).
        ∀ q ∈ set (bp-bucket-entries d). bp-marker d ≤ snd q
    by (rule bp-lazy-bucket-insert-entries-markers-lower-bound
        [OF M-pos b-lower])
  have tail: ∀ b ∈ set (bp-lazy-insert-bucket M p bs).
    ∀ q ∈ set (bp-bucket-entries b). bp-marker b ≤ snd q
  by (rule IH[OF tail-lower])
  show ?thesis
  proof (cases snd p < bp-marker b)
    case True
    have result:
      bp-lazy-insert-bucket M p (b # bs) =
        bp-bucketize-entries M [p] @ b # bs
    using True Cons by simp
    show ?thesis
    unfolding result
    using single prems Cons by auto
  next
    case False
    have p-ge-b: bp-marker b ≤ snd p
    using False by simp
    show ?thesis
    proof (cases snd p < bp-marker c)
      case True
      have result:
        bp-lazy-insert-bucket M p (b # bs) =
          bp-lazy-bucket-insert-entries M p b @ bs
      using False True Cons by simp
      show ?thesis
      unfolding result
      using inserted[OF p-ge-b] prems Cons by auto
    next
      case False
      have result:
        bp-lazy-insert-bucket M p (b # bs) =
          b # bp-lazy-insert-bucket M p bs
      using ‹¬ snd p < bp-marker b› False Cons by simp
      show ?thesis
      unfolding result
      using tail prems Cons by auto
    qed
  qed
qed
qed
qed

lemma bp-bucketize-entries-markers-le:
  fixes beta :: real
  assumes ∀ q ∈ set xs. snd q ≤ beta

```

shows $\forall b \in \text{set } (bp\text{-bucketize-entries } M \text{ } xs). bp\text{-marker } b \leq \text{beta}$
proof
 fix b
 assume $b: b \in \text{set } (bp\text{-bucketize-entries } M \text{ } xs)$
 obtain q where $q: q \in \text{set } xs$ and $\text{marker}: bp\text{-marker } b = \text{snd } q$
 using $bp\text{-bucketize-entries-marker-in-set}[OF \text{ } b]$ by auto
 have $\text{snd } q \leq \text{beta}$
 using $\text{assms } q$ by blast
 show $bp\text{-marker } b \leq \text{beta}$
 using $\text{marker } \langle \text{snd } q \leq \text{beta} \rangle$ by simp
qed

lemma $bp\text{-bucketize-entries-markers-ge}$:
 fixes $\alpha :: \text{real}$
 assumes $\forall q \in \text{set } xs. \alpha \leq \text{snd } q$
 shows $\forall b \in \text{set } (bp\text{-bucketize-entries } M \text{ } xs). \alpha \leq bp\text{-marker } b$
proof
 fix b
 assume $b: b \in \text{set } (bp\text{-bucketize-entries } M \text{ } xs)$
 obtain q where $q: q \in \text{set } xs$ and $\text{marker}: bp\text{-marker } b = \text{snd } q$
 using $bp\text{-bucketize-entries-marker-in-set}[OF \text{ } b]$ by auto
 have $\alpha \leq \text{snd } q$
 using $\text{assms } q$ by blast
 show $\alpha \leq bp\text{-marker } b$
 using $\text{marker } \langle \alpha \leq \text{snd } q \rangle$ by simp
qed

lemma $bp\text{-bucketize-entries-entry-in-source}$:
 assumes $M\text{-pos}: 0 < M$
 and $b: b \in \text{set } (bp\text{-bucketize-entries } M \text{ } xs)$
 and $q: q \in \text{set } (bp\text{-bucket-entries } b)$
 shows $q \in \text{set } xs$
proof –
 have $q \in \text{set } (bp\text{-bucket-entries-flat } (bp\text{-bucketize-entries } M \text{ } xs))$
 using $b \text{ } q$ unfolding $bp\text{-bucket-entries-flat-def}$ by auto
 then show ?thesis
 using $\text{set-}bp\text{-bucket-entries-flat-bucketize-entries}[OF \text{ } M\text{-pos}, of \text{ } xs]$
 by simp
qed

lemma $bp\text{-bucketize-entries-nonempty}$:
 assumes $0 < M$ and $xs \neq []$
 shows $bp\text{-bucketize-entries } M \text{ } xs \neq []$
proof
 assume $\text{empty}: bp\text{-bucketize-entries } M \text{ } xs = []$
 have $\text{set } (bp\text{-bucket-entries-flat } (bp\text{-bucketize-entries } M \text{ } xs)) = \text{set } xs$
 by (rule $\text{set-}bp\text{-bucket-entries-flat-bucketize-entries}[OF \text{ } \text{assms}(1)]$)
 then have $\text{set } xs = \{\}$
 using empty unfolding $bp\text{-bucket-entries-flat-def}$ by simp

```

    then show False
      using assms(2) by auto
qed

lemma bp-local-insert-bucket-nonempty:
  assumes  $0 < M$ 
  shows bp-local-insert-bucket  $M\ p\ bs \neq []$ 
  using assms
proof (induction bs arbitrary: p)
  case Nil
  then show ?case
    by (simp add: bp-bucketize-entries-nonempty)
next
  case (Cons b bs)
  then show ?case
  proof (cases bs)
    case Nil
    then show ?thesis
      using Cons.prems by (simp add: bp-bucketize-entries-nonempty)
  next
    case (Cons c cs)
    then show ?thesis
      using Cons by (simp add: bp-bucketize-entries-nonempty)
  qed
qed

lemma bp-bucket-boundaries-ok-append:
  assumes left: bp-bucket-boundaries-ok xs
  and right: bp-bucket-boundaries-ok ys
  and cross:  $ys \neq [] \implies$ 
     $\forall b \in \text{set } xs. \forall p \in \text{set } (bp\text{-bucket-entries } b).$ 
     $\text{snd } p \leq bp\text{-marker } (hd\ ys)$ 
  shows bp-bucket-boundaries-ok ( $xs @ ys$ )
  using left cross
proof (induction xs)
  case Nil
  then show ?case
    using right by simp
next
  case (Cons b xs)
  note outer-left = Cons.prems(1)
  note outer-cross = Cons.prems(2)
  note outer-IH = Cons.IH
  show ?case
  proof (cases xs)
    case Nil
    show ?thesis
  proof (cases ys)
    case Nil

```

```

    then show ?thesis
      using  $\langle xs = [] \rangle$  by simp
  next
    case (Cons c cs)
    have cross-all:
       $\forall d \in \text{set } (b \# xs). \forall p \in \text{set } (\text{bp-bucket-entries } d).$ 
       $\text{snd } p \leq \text{bp-marker } (\text{hd } ys)$ 
      using outer-cross Cons by simp
    have  $\forall p \in \text{set } (\text{bp-bucket-entries } b). \text{snd } p \leq \text{bp-marker } c$ 
      using cross-all Cons unfolding  $\langle xs = [] \rangle$  by simp
    then show ?thesis
      using right Cons  $\langle xs = [] \rangle$  by simp
  qed
next
  case (Cons c cs)
  have head-boundary:
     $\forall p \in \text{set } (\text{bp-bucket-entries } b). \text{snd } p \leq \text{bp-marker } c$ 
    using outer-left Cons by simp
  have tail-boundaries: bp-bucket-boundaries-ok (xs @ ys)
  proof (rule outer-IH)
    show bp-bucket-boundaries-ok xs
      using outer-left Cons by simp
    show  $ys \neq [] \implies$ 
       $\forall b \in \text{set } xs. \forall p \in \text{set } (\text{bp-bucket-entries } b).$ 
       $\text{snd } p \leq \text{bp-marker } (\text{hd } ys)$ 
      using outer-cross by simp
  qed
  show ?thesis
    using Cons head-boundary tail-boundaries by simp
qed
qed

```

lemma bp-bucket-boundaries-ok-bucketize-append-Cons:

```

  assumes M-pos:  $0 < M$ 
  and tail: bp-bucket-boundaries-ok (c # bs)
  and upper:  $\forall q \in \text{set } xs. \text{snd } q \leq \text{bp-marker } c$ 
  shows bp-bucket-boundaries-ok (bp-bucketize-entries M xs @ c # bs)
  proof (rule bp-bucket-boundaries-ok-append)
    show bp-bucket-boundaries-ok (bp-bucketize-entries M xs)
      by (rule bp-bucketize-entries-boundaries-ok[OF M-pos])
    show bp-bucket-boundaries-ok (c # bs)
      by (rule tail)
    show  $c \# bs \neq [] \implies$ 
       $\forall b \in \text{set } (\text{bp-bucketize-entries } M xs).$ 
       $\forall p \in \text{set } (\text{bp-bucket-entries } b). \text{snd } p \leq \text{bp-marker } (\text{hd } (c \# bs))$ 
  proof
    fix b
    assume b:  $b \in \text{set } (\text{bp-bucketize-entries } M xs)$ 
    show  $\forall p \in \text{set } (\text{bp-bucket-entries } b). \text{snd } p \leq \text{bp-marker } (\text{hd } (c \# bs))$ 

```

```

proof
  fix  $p$ 
  assume  $p: p \in \text{set } (bp\text{-bucket-entries } b)$ 
  have  $p \in \text{set } xs$ 
    by (rule  $bp\text{-bucketize-entries-entry-in-source}[OF\ M\text{-pos } b\ p]$ )
  then show  $snd\ p \leq bp\text{-marker } (hd\ (c \# bs))$ 
    using upper by simp
  qed
qed
qed

lemma  $bp\text{-lazy-bucket-insert-entries-nonempty}$ :
  assumes  $M\text{-pos}: 0 < M$ 
  shows  $bp\text{-lazy-bucket-insert-entries } M\ p\ b \neq []$ 
proof (cases  $\text{length } (bp\text{-bucket-entries } b) < 2 * M$ )
  case True
    then show ?thesis
      unfolding  $bp\text{-lazy-bucket-insert-entries-def}$  by simp
  next
    case False
    have  $bp\text{-bucketize-entries } M\ (p \# bp\text{-bucket-entries } b) \neq []$ 
      by (rule  $bp\text{-bucketize-entries-nonempty}[OF\ M\text{-pos}]$ ) simp
    then show ?thesis
      using False unfolding  $bp\text{-lazy-bucket-insert-entries-def}$  by simp
  qed

lemma  $bp\text{-lazy-insert-bucket-nonempty}$ :
  assumes  $M\text{-pos}: 0 < M$ 
  shows  $bp\text{-lazy-insert-bucket } M\ p\ bs \neq []$ 
  using assms
proof (induction  $bs$  arbitrary:  $p$ )
  case Nil
    then show ?case
      by (simp add:  $bp\text{-bucketize-entries-nonempty}$ )
  next
    case (Cons  $b\ bs$ )
    note  $IH = \text{Cons}.IH$ 
    note  $M\text{-pos} = \text{Cons}.prems$ 
    show ?case
    proof (cases  $bs$ )
    case Nil
      show ?thesis
    proof (cases  $snd\ p < bp\text{-marker } b$ )
    case True
      then show ?thesis
        using Nil by (simp add:  $bp\text{-bucketize-entries-nonempty}[OF\ M\text{-pos}]$ )
    next
      case False
      then show ?thesis

```

```

    using Nil bp-lazy-bucket-insert-entries-nonempty[OF M-pos, of p b]
    by simp
  qed
next
  case (Cons c cs)
  show ?thesis
  proof (cases snd p < bp-marker b)
    case True
    then show ?thesis
    using Cons by (simp add: bp-bucketize-entries-nonempty[OF M-pos])
  next
    case False
    show ?thesis
    proof (cases snd p < bp-marker c)
      case True
      then show ?thesis
      using Cons False bp-lazy-bucket-insert-entries-nonempty
        [OF M-pos, of p b]
      by simp
    next
      case False
      then show ?thesis
      using Cons <¬ snd p < bp-marker b> IH[OF M-pos, of p]
      by simp
    qed
  qed
qed
qed
qed

```

```

lemma bp-lazy-bucket-insert-entries-markers-sorted:
  assumes M-pos: 0 < M
  shows sorted-wrt (λb c. bp-marker b ≤ bp-marker c)
    (bp-lazy-bucket-insert-entries M p b)
  proof (cases length (bp-bucket-entries b) < 2 * M)
    case True
    then show ?thesis
    unfolding bp-lazy-bucket-insert-entries-def by simp
  next
    case False
    have sorted-wrt (λb c. bp-marker b ≤ bp-marker c)
      (bp-bucketize-entries M (p # bp-bucket-entries b))
    by (rule bp-bucketize-entries-markers-sorted[OF M-pos])
    then show ?thesis
    using False unfolding bp-lazy-bucket-insert-entries-def by simp
  qed

```

```

lemma bp-lazy-bucket-insert-entries-markers-ge:
  fixes alpha :: real
  assumes M-pos: 0 < M

```

```

    and lower:  $\forall q \in \text{set } (\text{bp-bucket-entries } b). \text{bp-marker } b \leq \text{snd } q$ 
    and p-ge:  $\text{bp-marker } b \leq \text{snd } p$ 
    and alpha-le:  $\alpha \leq \text{bp-marker } b$ 
  shows  $\forall c \in \text{set } (\text{bp-lazy-bucket-insert-entries } M \ p \ b).$ 
     $\alpha \leq \text{bp-marker } c$ 
proof (cases length (bp-bucket-entries b) < 2 * M)
  case True
  then show ?thesis
    using alpha-le unfolding bp-lazy-bucket-insert-entries-def by simp
next
  case False
  have entries-ge:
     $\forall q \in \text{set } (p \# \text{bp-bucket-entries } b). \alpha \leq \text{snd } q$ 
    using lower p-ge alpha-le by auto
  have  $\forall c \in \text{set } (\text{bp-bucketize-entries } M \ (p \# \text{bp-bucket-entries } b)).$ 
     $\alpha \leq \text{bp-marker } c$ 
    by (rule bp-bucketize-entries-markers-ge[OF entries-ge])
  then show ?thesis
    using False unfolding bp-lazy-bucket-insert-entries-def by simp
qed

```

```

lemma bp-lazy-bucket-insert-entries-markers-le:
  fixes beta :: real
  assumes M-pos:  $0 < M$ 
    and marker-le:  $\text{bp-marker } b \leq \text{beta}$ 
    and upper:  $\forall q \in \text{set } (p \# \text{bp-bucket-entries } b). \text{snd } q \leq \text{beta}$ 
  shows  $\forall c \in \text{set } (\text{bp-lazy-bucket-insert-entries } M \ p \ b).$ 
     $\text{bp-marker } c \leq \text{beta}$ 
proof (cases length (bp-bucket-entries b) < 2 * M)
  case True
  then show ?thesis
    using marker-le unfolding bp-lazy-bucket-insert-entries-def by simp
next
  case False
  have  $\forall c \in \text{set } (\text{bp-bucketize-entries } M \ (p \# \text{bp-bucket-entries } b)).$ 
     $\text{bp-marker } c \leq \text{beta}$ 
    by (rule bp-bucketize-entries-markers-le[OF upper])
  then show ?thesis
    using False unfolding bp-lazy-bucket-insert-entries-def by simp
qed

```

```

lemma bp-lazy-bucket-insert-entries-entries-le:
  fixes beta :: real
  assumes M-pos:  $0 < M$ 
    and upper:  $\forall q \in \text{set } (p \# \text{bp-bucket-entries } b). \text{snd } q \leq \text{beta}$ 
  shows  $\forall c \in \text{set } (\text{bp-lazy-bucket-insert-entries } M \ p \ b).$ 
     $\forall q \in \text{set } (\text{bp-bucket-entries } c). \text{snd } q \leq \text{beta}$ 
proof (cases length (bp-bucket-entries b) < 2 * M)
  case True

```



```

show ?thesis
proof
  fix c
  assume c: c ∈ set (bp-lazy-bucket-insert-entries M p b)
  show ∀ q ∈ set (bp-bucket-entries c). snd q ≤ beta
  proof
    fix q
    assume q: q ∈ set (bp-bucket-entries c)
    have q ∈ set (p # bp-bucket-entries b)
      using True c q unfolding bp-lazy-bucket-insert-entries-def by auto
    then show snd q ≤ beta
      using upper by blast
  qed
qed
next
case False
show ?thesis
proof
  fix c
  assume c:
    c ∈ set (bp-lazy-bucket-insert-entries M p b)
  show ∀ q ∈ set (bp-bucket-entries c). snd q ≤ beta
  proof
    fix q
    assume q: q ∈ set (bp-bucket-entries c)
    have c-bucketize:
      c ∈ set (bp-bucketize-entries M (p # bp-bucket-entries b))
      using c False unfolding bp-lazy-bucket-insert-entries-def by simp
    have q ∈ set (p # bp-bucket-entries b)
      by (rule bp-bucketize-entries-entry-in-source
        [OF M-pos c-bucketize q])
    then show snd q ≤ beta
      using upper by blast
  qed
qed
qed

lemma bp-lazy-bucket-insert-entries-boundaries-ok:
  assumes M-pos: 0 < M
  shows bp-bucket-boundaries-ok (bp-lazy-bucket-insert-entries M p b)
proof (cases length (bp-bucket-entries b) < 2 * M)
  case True
  then show ?thesis
    unfolding bp-lazy-bucket-insert-entries-def by simp
next
case False
have bp-bucket-boundaries-ok
  (bp-bucketize-entries M (p # bp-bucket-entries b))
  by (rule bp-bucketize-entries-boundaries-ok[OF M-pos])

```

```

then show ?thesis
  using False unfolding bp-lazy-bucket-insert-entries-def by simp
qed

lemma bp-lazy-insert-bucket-markers-ge-hd:
  assumes M-pos:  $0 < M$ 
  and sorted: sorted-wrt  $(\lambda b c. \text{bp-marker } b \leq \text{bp-marker } c) \text{ } bs$ 
  and lower:  $\forall b \in \text{set } bs. \forall q \in \text{set } (\text{bp-bucket-entries } b).$ 
     $\text{bp-marker } b \leq \text{snd } q$ 
  and nonempty:  $bs \neq []$ 
  and p-ge:  $\text{bp-marker } (\text{hd } bs) \leq \text{snd } p$ 
  shows  $\forall b \in \text{set } (\text{bp-lazy-insert-bucket } M \text{ } p \text{ } bs).$ 
     $\text{bp-marker } (\text{hd } bs) \leq \text{bp-marker } b$ 
  using sorted lower nonempty p-ge
proof (induction bs arbitrary: p)
  case Nil
  then show ?case
    by simp
next
  case (Cons b bs)
  note IH = Cons.IH
  note sorted = Cons.prem1
  note lower = Cons.prem2
  note p-ge = Cons.prem4
  show ?case
  proof (cases bs)
    case Nil
    have p-ge-b:  $\text{bp-marker } b \leq \text{snd } p$ 
      using p-ge by simp
    have inserted-ge:
       $\forall c \in \text{set } (\text{bp-lazy-bucket-insert-entries } M \text{ } p \text{ } b).$ 
         $\text{bp-marker } b \leq \text{bp-marker } c$ 
    proof (rule bp-lazy-bucket-insert-entries-markers-ge
      [where alpha =  $\text{bp-marker } b$ , OF M-pos - p-ge-b])
      show  $\forall q \in \text{set } (\text{bp-bucket-entries } b). \text{bp-marker } b \leq \text{snd } q$ 
        using lower Nil by simp
      show  $\text{bp-marker } b \leq \text{bp-marker } b$ 
        by simp
    qed
  qed
  show ?thesis
  proof (cases  $\text{snd } p < \text{bp-marker } b$ )
    case True
    then show ?thesis
      using p-ge by simp
  next
    case False
    then show ?thesis
      using Nil inserted-ge by simp
  qed

```

```

next
  case (Cons c cs)
  have sorted-tail:
    sorted-wrt ( $\lambda b c. \text{bp-marker } b \leq \text{bp-marker } c$ ) ( $c \# cs$ )
    using sorted Cons by simp
  have lower-tail:
     $\forall b \in \text{set } (c \# cs). \forall q \in \text{set } (\text{bp-bucket-entries } b).$ 
     $\text{bp-marker } b \leq \text{snd } q$ 
    using lower Cons by simp
  have b-le-c:  $\text{bp-marker } b \leq \text{bp-marker } c$ 
    using sorted Cons by simp
  show ?thesis
  proof (cases  $\text{snd } p < \text{bp-marker } b$ )
    case True
    then show ?thesis
      using p-ge by simp
  next
    case False
    show ?thesis
    proof (cases  $\text{snd } p < \text{bp-marker } c$ )
      case True
      have p-ge-b:  $\text{bp-marker } b \leq \text{snd } p$ 
        using False by simp
      have inserted-ge:
         $\forall d \in \text{set } (\text{bp-lazy-bucket-insert-entries } M p b).$ 
         $\text{bp-marker } b \leq \text{bp-marker } d$ 
      proof (rule bp-lazy-bucket-insert-entries-markers-ge
        [where  $\alpha = \text{bp-marker } b$ , OF M-pos])
        show  $\forall q \in \text{set } (\text{bp-bucket-entries } b). \text{bp-marker } b \leq \text{snd } q$ 
          using lower Cons by simp
        show  $\text{bp-marker } b \leq \text{snd } p$ 
          by (rule p-ge-b)
        show  $\text{bp-marker } b \leq \text{bp-marker } b$ 
          by simp
      qed
      have tail-ge:
         $\forall d \in \text{set } (c \# cs). \text{bp-marker } b \leq \text{bp-marker } d$ 
        using sorted Cons by auto
      show ?thesis
        using Cons False True inserted-ge tail-ge by auto
    next
      case False
      case False
      have p-ge-c:  $\text{bp-marker } c \leq \text{snd } p$ 
        using False by simp
      have tail-ge-c:
         $\forall d \in \text{set } (\text{bp-lazy-insert-bucket } M p (c \# cs)).$ 
         $\text{bp-marker } c \leq \text{bp-marker } d$ 
        using IH[of p] sorted-tail lower-tail p-ge-c Cons by auto
      have tail-ge-b:

```

```

       $\forall d \in \text{set } (\text{bp-lazy-insert-bucket } M \ p \ (c \ \# \ cs)).$ 
       $\text{bp-marker } b \leq \text{bp-marker } d$ 
      using  $b\text{-le-}c \ \text{tail-ge-}c$  by force
    show ?thesis
      using  $\text{Cons } \langle \neg \text{snd } p < \text{bp-marker } b \rangle \ \text{False tail-ge-}b$  by simp
  qed
qed
qed
qed

lemma bp-lazy-insert-bucket-markers-sorted:
  assumes  $M\text{-pos}: 0 < M$ 
  and sorted:  $\text{sorted-wrt } (\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c) \ bs$ 
  and lower:  $\forall b \in \text{set } bs. \forall q \in \text{set } (\text{bp-bucket-entries } b).$ 
     $\text{bp-marker } b \leq \text{snd } q$ 
  and boundaries: bp-bucket-boundaries-ok  $bs$ 
shows  $\text{sorted-wrt } (\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c)$ 
   $(\text{bp-lazy-insert-bucket } M \ p \ bs)$ 
using sorted lower boundaries
proof (induction bs arbitrary: p)
  case Nil
  then show ?case
    by (simp add: bp-bucketize-entries-markers-sorted[OF M-pos])
next
  case ( $\text{Cons } b \ bs$ )
  note  $IH = \text{Cons.IH}$ 
  note  $\text{sorted} = \text{Cons.prems}(1)$ 
  note  $\text{lower} = \text{Cons.prems}(2)$ 
  note  $\text{boundaries} = \text{Cons.prems}(3)$ 
  show ?case
  proof (cases bs)
    case Nil
    show ?thesis
    proof (cases snd p < bp-marker b)
      case True
      have left-sorted:
         $\text{sorted-wrt } (\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c)$ 
         $(\text{bp-bucketize-entries } M \ [p])$ 
        by (rule bp-bucketize-entries-markers-sorted[OF M-pos])
      have left-le-b:
         $\forall x \in \text{set } (\text{bp-bucketize-entries } M \ [p]).$ 
         $\text{bp-marker } x \leq \text{bp-marker } b$ 
        by (rule bp-bucketize-entries-markers-le) (use True in simp)
      show ?thesis
        using Nil True left-sorted left-le-b
        by (simp add: sorted-wrt-append)
    next
    case False
    show ?thesis

```

```

    using Nil False
    bp-lazy-bucket-insert-entries-markers-sorted[OF M-pos, of p b]
    by simp
qed
next
case (Cons c cs)
have sorted-tail:
  sorted-wrt ( $\lambda b c. \text{bp-marker } b \leq \text{bp-marker } c$ ) ( $c \# cs$ )
  using sorted Cons by simp
have lower-tail:
   $\forall b \in \text{set } (c \# cs). \forall q \in \text{set } (\text{bp-bucket-entries } b).$ 
   $\text{bp-marker } b \leq \text{snd } q$ 
  using lower Cons by simp
have boundaries-tail: bp-bucket-boundaries-ok ( $c \# cs$ )
  using boundaries Cons by simp
have b-le-c:  $\text{bp-marker } b \leq \text{bp-marker } c$ 
  using sorted Cons by simp
have b-le-all:
   $\forall d \in \text{set } (b \# c \# cs). \text{bp-marker } b \leq \text{bp-marker } d$ 
  using sorted Cons by auto
show ?thesis
proof (cases  $\text{snd } p < \text{bp-marker } b$ )
case True
have left-sorted:
  sorted-wrt ( $\lambda b c. \text{bp-marker } b \leq \text{bp-marker } c$ )
  ( $\text{bp-bucketize-entries } M [p]$ )
  by (rule bp-bucketize-entries-markers-sorted[OF M-pos])
have right-sorted:
  sorted-wrt ( $\lambda b c. \text{bp-marker } b \leq \text{bp-marker } c$ ) ( $b \# c \# cs$ )
  using sorted Cons by simp
have left-le-b:
   $\forall x \in \text{set } (\text{bp-bucketize-entries } M [p]).$ 
   $\text{bp-marker } x \leq \text{bp-marker } b$ 
  by (rule bp-bucketize-entries-markers-le) (use True in simp)
have cross:
   $\forall x \in \text{set } (\text{bp-bucketize-entries } M [p]).$ 
   $\forall y \in \text{set } (b \# c \# cs). \text{bp-marker } x \leq \text{bp-marker } y$ 
  using left-le-b b-le-all by force
show ?thesis
  using Cons True left-sorted right-sorted cross
  by (simp add: sorted-wrt-append)
next
case False
show ?thesis
proof (cases  $\text{snd } p < \text{bp-marker } c$ )
case True
have left-sorted:
  sorted-wrt ( $\lambda b c. \text{bp-marker } b \leq \text{bp-marker } c$ )
  ( $\text{bp-lazy-bucket-insert-entries } M p b$ )

```

```

  by (rule bp-lazy-bucket-insert-entries-markers-sorted[OF M-pos])
have left-le-c:
   $\forall x \in \text{set } (bp\text{-lazy-bucket-insert-entries } M \ p \ b).$ 
   $bp\text{-marker } x \leq bp\text{-marker } c$ 
proof (rule bp-lazy-bucket-insert-entries-markers-le
  [OF M-pos b-le-c])
  show  $\forall q \in \text{set } (p \ \# \ bp\text{-bucket-entries } b).$ 
     $snd \ q \leq bp\text{-marker } c$ 
  using boundaries Cons True by auto
qed
have c-le-all:
   $\forall d \in \text{set } (c \ \# \ cs). \ bp\text{-marker } c \leq bp\text{-marker } d$ 
  using sorted-tail by auto
have cross:
   $\forall x \in \text{set } (bp\text{-lazy-bucket-insert-entries } M \ p \ b).$ 
   $\forall y \in \text{set } (c \ \# \ cs). \ bp\text{-marker } x \leq bp\text{-marker } y$ 
  using left-le-c c-le-all by force
show ?thesis
  using Cons False True left-sorted sorted-tail cross
  by (simp add: sorted-wrt-append)
next
case False
have p-ge-c:  $bp\text{-marker } c \leq snd \ p$ 
  using False by simp
have tail-sorted-result:
   $sorted\text{-wrt } (\lambda b \ c. \ bp\text{-marker } b \leq bp\text{-marker } c)$ 
   $(bp\text{-lazy-insert-bucket } M \ p \ (c \ \# \ cs))$ 
  using IH[of p] sorted-tail lower-tail boundaries-tail Cons by auto
have tail-ge-hd:
   $\forall d \in \text{set } (bp\text{-lazy-insert-bucket } M \ p \ (c \ \# \ cs)).$ 
   $bp\text{-marker } (hd \ (c \ \# \ cs)) \leq bp\text{-marker } d$ 
proof (rule bp-lazy-insert-bucket-markers-ge-hd
  [where bs = c # cs and p = p and M = M])
  show  $0 < M$ 
    by (rule M-pos)
  show  $sorted\text{-wrt } (\lambda b \ c. \ bp\text{-marker } b \leq bp\text{-marker } c) \ (c \ \# \ cs)$ 
    by (rule sorted-tail)
  show  $\forall b \in \text{set } (c \ \# \ cs). \ \forall q \in \text{set } (bp\text{-bucket-entries } b).$ 
     $bp\text{-marker } b \leq snd \ q$ 
    by (rule lower-tail)
  show  $c \ \# \ cs \neq []$ 
    by simp
  show  $bp\text{-marker } (hd \ (c \ \# \ cs)) \leq snd \ p$ 
    using p-ge-c by simp
qed
have tail-ge-c:
   $\forall d \in \text{set } (bp\text{-lazy-insert-bucket } M \ p \ (c \ \# \ cs)).$ 
   $bp\text{-marker } c \leq bp\text{-marker } d$ 
  using tail-ge-hd by simp

```

```

    have tail-ge-b:
       $\forall d \in \text{set } (\text{bp-lazy-insert-bucket } M \ p \ (c \ \# \ cs)).$ 
       $\text{bp-marker } b \leq \text{bp-marker } d$ 
      using b-le-c tail-ge-c by force
    show ?thesis
      using Cons  $\langle \neg \text{snd } p < \text{bp-marker } b \rangle$  False
      tail-sorted-result tail-ge-b by simp
  qed
qed
qed
qed

lemma bp-lazy-insert-bucket-boundaries-ok:
  assumes M-pos:  $0 < M$ 
  and sorted: sorted-wrt  $(\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c)$  bs
  and lower:  $\forall b \in \text{set } bs. \forall q \in \text{set } (\text{bp-bucket-entries } b).$ 
     $\text{bp-marker } b \leq \text{snd } q$ 
  and boundaries: bp-bucket-boundaries-ok bs
  shows bp-bucket-boundaries-ok (bp-lazy-insert-bucket M p bs)
  using sorted lower boundaries
proof (induction bs arbitrary: p)
  case Nil
  then show ?case
    by (simp add: bp-bucketize-entries-boundaries-ok[OF M-pos])
next
  case (Cons b bs)
  note IH = Cons.IH
  note sorted = Cons.prem1
  note lower = Cons.prem2
  note boundaries = Cons.prem3
  show ?case
  proof (cases bs)
    case Nil
    show ?thesis
  proof (cases snd p < bp-marker b)
    case True
    have tail: bp-bucket-boundaries-ok [b]
    by simp
    have upper:  $\forall q \in \text{set } [p]. \text{snd } q \leq \text{bp-marker } b$ 
    using True by simp
    show ?thesis
    using Nil True
    by (simp add: bp-bucket-boundaries-ok-bucketize-append-Cons
      [OF M-pos tail upper])
  next
    case False
    show ?thesis
    using Nil False
    by (simp add: bp-lazy-bucket-insert-entries-boundaries-ok[OF M-pos, of p b])
  qed
  qed

```



```

    by (rule boundaries-tail)
  show  $c \# cs \neq [] \implies$ 
     $\forall b \in \text{set } (\text{bp-lazy-bucket-insert-entries } M \ p \ b).$ 
     $\forall p \in \text{set } (\text{bp-bucket-entries } b).$ 
     $\text{snd } p \leq \text{bp-marker } (\text{hd } (c \# cs))$ 
    using cross-entries by simp
qed
show ?thesis
  using Cons False True append-ok by simp
next
case False
have p-ge-c:  $\text{bp-marker } c \leq \text{snd } p$ 
  using False by simp
have tail-boundaries:
   $\text{bp-bucket-boundaries-ok } (\text{bp-lazy-insert-bucket } M \ p \ (c \# cs))$ 
  using IH[of p] sorted-tail lower-tail boundaries-tail Cons by auto
have tail-ge-hd:
   $\forall d \in \text{set } (\text{bp-lazy-insert-bucket } M \ p \ (c \# cs)).$ 
   $\text{bp-marker } (\text{hd } (c \# cs)) \leq \text{bp-marker } d$ 
proof (rule bp-lazy-insert-bucket-markers-ge-hd
  [where bs =  $c \# cs$  and  $p = p$  and  $M = M$ ])
  show  $0 < M$ 
    by (rule M-pos)
  show sorted-wrt  $(\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c) \ (c \# cs)$ 
    by (rule sorted-tail)
  show  $\forall b \in \text{set } (c \# cs). \forall q \in \text{set } (\text{bp-bucket-entries } b).$ 
     $\text{bp-marker } b \leq \text{snd } q$ 
    by (rule lower-tail)
  show  $c \# cs \neq []$ 
    by simp
  show  $\text{bp-marker } (\text{hd } (c \# cs)) \leq \text{snd } p$ 
    using p-ge-c by simp
qed
have tail-ge-c:
   $\forall d \in \text{set } (\text{bp-lazy-insert-bucket } M \ p \ (c \# cs)).$ 
   $\text{bp-marker } c \leq \text{bp-marker } d$ 
  using tail-ge-hd by simp
have tail-nonempty-result:
   $\text{bp-lazy-insert-bucket } M \ p \ (c \# cs) \neq []$ 
  by (rule bp-lazy-insert-bucket-nonempty[OF M-pos])
obtain d ds where tail-eq:
   $\text{bp-lazy-insert-bucket } M \ p \ (c \# cs) = d \# ds$ 
  using tail-nonempty-result
  by (cases  $\text{bp-lazy-insert-bucket } M \ p \ (c \# cs)$ ) auto
have c-le-d:  $\text{bp-marker } c \leq \text{bp-marker } d$ 
  using tail-ge-c tail-eq by simp
have head-boundary:
   $\forall q \in \text{set } (\text{bp-bucket-entries } b). \text{snd } q \leq \text{bp-marker } d$ 
proof

```

```

    fix q
    assume q: q ∈ set (bp-bucket-entries b)
    have snd q ≤ bp-marker c
      using boundaries Cons q by auto
    then show snd q ≤ bp-marker d
      using c-le-d by linarith
  qed
show ?thesis
  using Cons ⟨¬ snd p < bp-marker b⟩ False tail-eq
    tail-boundaries head-boundary by simp
qed
qed
qed
qed

```

lemma *bp-lazy-insert-bucket-preserves-bucket-shape:*

```

assumes M-pos: 0 < M
  and sizes: ∀ b ∈ set bs. length (bp-bucket-entries b) ≤ 2 * M
  and sorted: sorted-wrt (λb c. bp-marker b ≤ bp-marker c) bs
  and lower: ∀ b ∈ set bs. ∀ q ∈ set (bp-bucket-entries b).
    bp-marker b ≤ snd q
  and boundaries: bp-bucket-boundaries-ok bs
shows (∀ b ∈ set (bp-lazy-insert-bucket M p bs).
  length (bp-bucket-entries b) ≤ 2 * M) ∧
  sorted-wrt (λb c. bp-marker b ≤ bp-marker c)
  (bp-lazy-insert-bucket M p bs) ∧
  (∀ b ∈ set (bp-lazy-insert-bucket M p bs).
    ∀ q ∈ set (bp-bucket-entries b). bp-marker b ≤ snd q) ∧
  bp-bucket-boundaries-ok (bp-lazy-insert-bucket M p bs)
using bp-lazy-insert-bucket-sizes-ok[OF M-pos sizes, of p]
  bp-lazy-insert-bucket-markers-sorted
  [OF M-pos sorted lower boundaries, of p]
  bp-lazy-insert-bucket-markers-lower-bound[OF M-pos lower, of p]
  bp-lazy-insert-bucket-boundaries-ok
  [OF M-pos sorted lower boundaries, of p]
by blast

```

lemma *bp-local-insert-bucket-markers-ge-hd:*

```

assumes M-pos: 0 < M
  and sorted: sorted-wrt (λb c. bp-marker b ≤ bp-marker c) bs
  and lower: ∀ b ∈ set bs. ∀ q ∈ set (bp-bucket-entries b).
    bp-marker b ≤ snd q
  and nonempty: bs ≠ []
  and p-ge: bp-marker (hd bs) ≤ snd p
shows ∀ b ∈ set (bp-local-insert-bucket M p bs).
  bp-marker (hd bs) ≤ bp-marker b
using sorted lower nonempty p-ge
proof (induction bs arbitrary: p)
case Nil

```

```

then show ?case
  by simp
next
case (Cons b bs)
note IH = Cons.IH
note sorted = Cons.prem1
note lower = Cons.prem2
note p-ge = Cons.prem4
show ?case
proof (cases bs)
  case Nil
  have markers-ge:
     $\forall c \in \text{set } (\text{bp-bucketize-entries } M \ (p \# \text{bp-bucket-entries } b)).$ 
     $\text{bp-marker } b \leq \text{bp-marker } c$ 
  proof (rule bp-bucketize-entries-markers-ge)
    show  $\forall q \in \text{set } (p \# \text{bp-bucket-entries } b). \text{bp-marker } b \leq \text{snd } q$ 
    using lower p-ge by auto
  qed
  show ?thesis
  using Nil p-ge markers-ge by auto
next
case (Cons c cs)
have sorted-tail:
   $\text{sorted-wrt } (\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c) \ (c \# \text{cs})$ 
  using sorted Cons by simp
have lower-tail:
   $\forall b \in \text{set } (c \# \text{cs}). \forall q \in \text{set } (\text{bp-bucket-entries } b).$ 
   $\text{bp-marker } b \leq \text{snd } q$ 
  using lower Cons by simp
have b-le-c:  $\text{bp-marker } b \leq \text{bp-marker } c$ 
  using sorted Cons by simp
show ?thesis
proof (cases  $\text{snd } p < \text{bp-marker } b$ )
  case True
  then show ?thesis
  using p-ge by simp
next
case False
note not-before-b = False
show ?thesis
proof (cases  $\text{snd } p < \text{bp-marker } c$ )
  case True
  have markers-ge:
     $\forall d \in \text{set } (\text{bp-bucketize-entries } M \ (p \# \text{bp-bucket-entries } b)).$ 
     $\text{bp-marker } b \leq \text{bp-marker } d$ 
  proof (rule bp-bucketize-entries-markers-ge)
    show  $\forall q \in \text{set } (p \# \text{bp-bucket-entries } b). \text{bp-marker } b \leq \text{snd } q$ 
    using lower p-ge by auto
  qed

```

```

    have tail-ge:
       $\forall d \in \text{set } (c \# cs). \text{bp-marker } b \leq \text{bp-marker } d$ 
      using sorted Cons by auto
    show ?thesis
      using Cons False True markers-ge tail-ge by auto
  next
    case False
    have p-ge-c:  $\text{bp-marker } c \leq \text{snd } p$ 
      using False by simp
    have tail-ge-c:
       $\forall d \in \text{set } (\text{bp-local-insert-bucket } M \text{ } p \text{ } (c \# cs)).$ 
       $\text{bp-marker } c \leq \text{bp-marker } d$ 
      using IH[of p] sorted-tail lower-tail p-ge-c Cons by auto
    show ?thesis
      using Cons False b-le-c tail-ge-c by auto
  qed
qed
qed
qed

lemma bp-local-insert-bucket-markers-sorted:
  assumes M-pos:  $0 < M$ 
  and sorted:  $\text{sorted-wrt } (\lambda b \text{ } c. \text{bp-marker } b \leq \text{bp-marker } c) \text{ } bs$ 
  and lower:  $\forall b \in \text{set } bs. \forall q \in \text{set } (\text{bp-bucket-entries } b).$ 
   $\text{bp-marker } b \leq \text{snd } q$ 
  and boundaries:  $\text{bp-bucket-boundaries-ok } bs$ 
  shows  $\text{sorted-wrt } (\lambda b \text{ } c. \text{bp-marker } b \leq \text{bp-marker } c)$ 
   $(\text{bp-local-insert-bucket } M \text{ } p \text{ } bs)$ 
  using sorted lower boundaries
proof (induction bs arbitrary: p)
  case Nil
  then show ?case
    by (simp add: bp-bucketize-entries-markers-sorted[OF M-pos])
next
  case (Cons b bs)
  note IH = Cons.IH
  note sorted = Cons.prem1
  note lower = Cons.prem2
  note boundaries = Cons.prem3
  show ?case
  proof (cases bs)
    case Nil
    show ?thesis
  proof (cases  $\text{snd } p < \text{bp-marker } b$ )
    case True
    have left-sorted:
       $\text{sorted-wrt } (\lambda b \text{ } c. \text{bp-marker } b \leq \text{bp-marker } c)$ 
       $(\text{bp-bucketize-entries } M \text{ } [p])$ 
      by (rule bp-bucketize-entries-markers-sorted[OF M-pos])

```

```

have left-le-b:
   $\forall x \in \text{set } (\text{bp-bucketize-entries } M \ [p]).$ 
   $\text{bp-marker } x \leq \text{bp-marker } b$ 
proof (rule bp-bucketize-entries-markers-le)
  show  $\forall q \in \text{set } [p]. \text{snd } q \leq \text{bp-marker } b$ 
  using True by simp
qed
show ?thesis
  using Nil True left-sorted left-le-b
  by (simp add: sorted-wrt-append)
next
case False
show ?thesis
  using Nil False
  by (simp add: bp-bucketize-entries-markers-sorted[OF M-pos])
qed
next
case (Cons c cs)
have sorted-tail:
   $\text{sorted-wrt } (\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c) \ (c \ \# \ cs)$ 
  using sorted Cons by simp
have lower-tail:
   $\forall b \in \text{set } (c \ \# \ cs). \forall q \in \text{set } (\text{bp-bucket-entries } b).$ 
   $\text{bp-marker } b \leq \text{snd } q$ 
  using lower Cons by simp
have boundaries-tail: bp-bucket-boundaries-ok (c # cs)
  using boundaries Cons by simp
have b-le-c: bp-marker b ≤ bp-marker c
  using sorted Cons by simp
have b-le-all-tail:
   $\forall d \in \text{set } (b \ \# \ c \ \# \ cs). \text{bp-marker } b \leq \text{bp-marker } d$ 
  using sorted Cons by auto
show ?thesis
proof (cases snd p < bp-marker b)
case True
have left-sorted:
   $\text{sorted-wrt } (\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c)$ 
   $(\text{bp-bucketize-entries } M \ [p])$ 
  by (rule bp-bucketize-entries-markers-sorted[OF M-pos])
have right-sorted:
   $\text{sorted-wrt } (\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c) \ (b \ \# \ c \ \# \ cs)$ 
  using sorted Cons by simp
have left-le-b:
   $\forall x \in \text{set } (\text{bp-bucketize-entries } M \ [p]).$ 
   $\text{bp-marker } x \leq \text{bp-marker } b$ 
proof (rule bp-bucketize-entries-markers-le)
  show  $\forall q \in \text{set } [p]. \text{snd } q \leq \text{bp-marker } b$ 
  using True by simp
qed

```

```

have cross:
   $\forall x \in \text{set } (bp\text{-bucketize-entries } M [p]).$ 
   $\forall y \in \text{set } (b \# c \# cs). bp\text{-marker } x \leq bp\text{-marker } y$ 
  using left-le-b b-le-all-tail by force
show ?thesis
  using Cons True left-sorted right-sorted cross
  by (simp add: sorted-wrt-append)
next
case False
note not-before-b = False
show ?thesis
proof (cases snd p < bp-marker c)
  case True
  have left-sorted:
    sorted-wrt ( $\lambda b c. bp\text{-marker } b \leq bp\text{-marker } c$ )
    (bp-bucketize-entries M (p # bp-bucket-entries b))
    by (rule bp-bucketize-entries-markers-sorted[OF M-pos])
  have left-le-c:
     $\forall x \in \text{set } (bp\text{-bucketize-entries } M (p \# bp\text{-bucket-entries } b)).$ 
     $bp\text{-marker } x \leq bp\text{-marker } c$ 
  proof (rule bp-bucketize-entries-markers-le)
    show  $\forall q \in \text{set } (p \# bp\text{-bucket-entries } b). snd\ q \leq bp\text{-marker } c$ 
    using boundaries Cons True by auto
  qed
  have c-le-all:
     $\forall d \in \text{set } (c \# cs). bp\text{-marker } c \leq bp\text{-marker } d$ 
    using sorted-tail by auto
  have cross:
     $\forall x \in \text{set } (bp\text{-bucketize-entries } M (p \# bp\text{-bucket-entries } b)).$ 
     $\forall y \in \text{set } (c \# cs). bp\text{-marker } x \leq bp\text{-marker } y$ 
    using left-le-c c-le-all by force
  show ?thesis
    using Cons False True left-sorted sorted-tail cross
    by (simp add: sorted-wrt-append)
next
case False
have p-ge-c:  $bp\text{-marker } c \leq snd\ p$ 
  using False by simp
have p-ge-hd:  $bp\text{-marker } (hd\ (c \# cs)) \leq snd\ p$ 
  using p-ge-c by simp
have tail-nonempty:  $c \# cs \neq []$ 
  by simp
have tail-sorted-result:
  sorted-wrt ( $\lambda b c. bp\text{-marker } b \leq bp\text{-marker } c$ )
  (bp-local-insert-bucket M p (c # cs))
  using IH[of p] sorted-tail lower-tail boundaries-tail Cons by auto
have tail-ge-hd:
   $\forall d \in \text{set } (bp\text{-local-insert-bucket } M\ p\ (c \# cs)).$ 
   $bp\text{-marker } (hd\ (c \# cs)) \leq bp\text{-marker } d$ 

```

```

    by (rule bp-local-insert-bucket-markers-ge-hd
      [where bs = c # cs and p = p and M = M,
        OF M-pos sorted-tail lower-tail tail-nonempty p-ge-hd])
  have tail-ge-c:
     $\forall d \in \text{set } (bp\text{-local-insert-bucket } M \ p \ (c \ \# \ cs)).$ 
     $bp\text{-marker } c \leq bp\text{-marker } d$ 
    using tail-ge-hd by simp
  have tail-ge-b:
     $\forall d \in \text{set } (bp\text{-local-insert-bucket } M \ p \ (c \ \# \ cs)).$ 
     $bp\text{-marker } b \leq bp\text{-marker } d$ 
    using b-le-c tail-ge-c by force
  show ?thesis
    using Cons not-before-b False tail-sorted-result tail-ge-b by simp
qed
qed
qed
qed

lemma bp-local-insert-bucket-boundaries-ok:
  assumes M-pos:  $0 < M$ 
  and sorted: sorted-wrt  $(\lambda b \ c. bp\text{-marker } b \leq bp\text{-marker } c) \ bs$ 
  and lower:  $\forall b \in \text{set } bs. \forall q \in \text{set } (bp\text{-bucket-entries } b).$ 
     $bp\text{-marker } b \leq \text{snd } q$ 
  and boundaries: bp-bucket-boundaries-ok bs
  shows bp-bucket-boundaries-ok (bp-local-insert-bucket M p bs)
  using sorted lower boundaries
proof (induction bs arbitrary: p)
  case Nil
  then show ?case
    by (simp add: bp-bucketize-entries-boundaries-ok[OF M-pos])
next
  case (Cons b bs)
  note IH = Cons.IH
  note sorted = Cons.prem1
  note lower = Cons.prem2
  note boundaries = Cons.prem3
  show ?case
  proof (cases bs)
    case Nil
    show ?thesis
  proof (cases snd p < bp-marker b)
    case True
    have tail: bp-bucket-boundaries-ok [b]
      by simp
    have upper:  $\forall q \in \text{set } [p]. \text{snd } q \leq bp\text{-marker } b$ 
      using True by simp
    show ?thesis
      using Nil True
    by (simp add: bp-bucket-boundaries-ok-bucketize-append-Cons

```

```

      [OF M-pos tail upper])
next
  case False
  show ?thesis
  using Nil False by (simp add: bp-bucketize-entries-boundaries-ok[OF M-pos])
qed
next
  case (Cons c cs)
  have sorted-tail:
    sorted-wrt ( $\lambda b c. \text{bp-marker } b \leq \text{bp-marker } c$ ) ( $c \# cs$ )
  using sorted Cons by simp
  have lower-tail:
     $\forall b \in \text{set } (c \# cs). \forall q \in \text{set } (\text{bp-bucket-entries } b).$ 
     $\text{bp-marker } b \leq \text{snd } q$ 
  using lower Cons by simp
  have boundaries-tail: bp-bucket-boundaries-ok ( $c \# cs$ )
  using boundaries Cons by simp
  show ?thesis
  proof (cases  $\text{snd } p < \text{bp-marker } b$ )
  case True
  have tail-all: bp-bucket-boundaries-ok ( $b \# c \# cs$ )
  using boundaries Cons by simp
  have upper:  $\forall q \in \text{set } [p]. \text{snd } q \leq \text{bp-marker } b$ 
  using True by simp
  show ?thesis
  using Cons True
  by (simp add: bp-bucket-boundaries-ok-bucketize-append-Cons
    [where  $c = b$  and  $bs = c \# cs$  and  $xs = [p]$ ,
      OF M-pos tail-all upper])
next
  case False
  note not-before-b = False
  show ?thesis
  proof (cases  $\text{snd } p < \text{bp-marker } c$ )
  case True
  have upper:
     $\forall q \in \text{set } (p \# \text{bp-bucket-entries } b). \text{snd } q \leq \text{bp-marker } c$ 
  using boundaries Cons True by auto
  show ?thesis
  using Cons not-before-b True
  by (simp add: bp-bucket-boundaries-ok-bucketize-append-Cons
    [OF M-pos boundaries-tail upper])
next
  case False
  have p-ge-c:  $\text{bp-marker } c \leq \text{snd } p$ 
  using False by simp
  have p-ge-hd:  $\text{bp-marker } (\text{hd } (c \# cs)) \leq \text{snd } p$ 
  using p-ge-c by simp
  have tail-nonempty:  $c \# cs \neq []$ 

```



```

    by simp
  have tail-boundaries:
    bp-bucket-boundaries-ok (bp-local-insert-bucket M p (c # cs))
    using IH[of p] sorted-tail lower-tail boundaries-tail Cons by auto
  have tail-ge-hd:
     $\forall d \in \text{set } (bp\text{-local-insert-bucket } M \text{ } p \text{ } (c \# cs)).$ 
    bp-marker (hd (c # cs))  $\leq$  bp-marker d
    by (rule bp-local-insert-bucket-markers-ge-hd
      [where bs = c # cs and p = p and M = M,
        OF M-pos sorted-tail lower-tail tail-nonempty p-ge-hd])
  have tail-nonempty-result:
    bp-local-insert-bucket M p (c # cs)  $\neq$  []
    by (rule bp-local-insert-bucket-nonempty[OF M-pos])
  obtain d ds where tail-eq:
    bp-local-insert-bucket M p (c # cs) = d # ds
    using tail-nonempty-result by (cases bp-local-insert-bucket M p (c # cs))
auto
  have c-le-d: bp-marker c  $\leq$  bp-marker d
    using tail-ge-hd tail-eq by simp
  have head-boundary:
     $\forall q \in \text{set } (bp\text{-bucket-entries } b). \text{snd } q \leq bp\text{-marker } d$ 
  proof
    fix q
    assume q:  $q \in \text{set } (bp\text{-bucket-entries } b)$ 
    have snd q  $\leq$  bp-marker c
      using boundaries Cons q by auto
    then show snd q  $\leq$  bp-marker d
      using c-le-d by linarith
  qed
  show ?thesis
    using Cons not-before-b False tail-eq tail-boundaries head-boundary by simp
  qed
qed
qed
qed
qed

```

lemma *bp-local-insert-bucket-preserves-bucket-shape*:

```

  assumes M-pos:  $0 < M$ 
  and sizes:  $\forall b \in \text{set } bs. \text{length } (bp\text{-bucket-entries } b) \leq M$ 
  and sorted: sorted-wrt ( $\lambda b \ c. bp\text{-marker } b \leq bp\text{-marker } c$ ) bs
  and lower:  $\forall b \in \text{set } bs. \forall q \in \text{set } (bp\text{-bucket-entries } b).$ 
    bp-marker b  $\leq$  snd q
  and boundaries: bp-bucket-boundaries-ok bs
  shows ( $\forall b \in \text{set } (bp\text{-local-insert-bucket } M \text{ } p \text{ } bs).$ 
    length (bp-bucket-entries b)  $\leq$  M)  $\wedge$ 
    sorted-wrt ( $\lambda b \ c. bp\text{-marker } b \leq bp\text{-marker } c$ )
    (bp-local-insert-bucket M p bs)  $\wedge$ 
    ( $\forall b \in \text{set } (bp\text{-local-insert-bucket } M \text{ } p \text{ } bs).$ 
       $\forall q \in \text{set } (bp\text{-bucket-entries } b). bp\text{-marker } b \leq \text{snd } q$ )  $\wedge$ 

```

$bp\text{-bucket-boundaries-ok } (bp\text{-local-insert-bucket } M \ p \ bs)$
using $bp\text{-local-insert-bucket-sizes-ok}[OF \ M\text{-pos sizes, of } p]$
 $bp\text{-local-insert-bucket-markers-sorted}[OF \ M\text{-pos sorted lower boundaries, of } p]$
 $bp\text{-local-insert-bucket-markers-lower-bound}[OF \ M\text{-pos lower, of } p]$
 $bp\text{-local-insert-bucket-boundaries-ok}[OF \ M\text{-pos sorted lower boundaries, of } p]$
by *blast*

The local-insert proof is deliberately split into many list-level lemmas because each invariant component has a different reason for being preserved. Size preservation depends on re-chunking only the affected bucket. Marker ordering depends on inserting into the unique marker interval selected by the value of the new entry. Boundary preservation depends on the fact that every entry of the previous bucket remains below the next bucket marker. The bundled lemma $\llbracket 0 < ?M; \forall b \in \text{set } ?bs. \text{length } (bp\text{-bucket-entries } b) \leq ?M; \text{sorted-wrt } (\lambda b \ c. bp\text{-marker } b \leq bp\text{-marker } c) \ ?bs; \forall b \in \text{set } ?bs. \forall q \in \text{set } (bp\text{-bucket-entries } b). bp\text{-marker } b \leq \text{snd } q; bp\text{-bucket-boundaries-ok } ?bs \rrbracket \implies (\forall b \in \text{set } (bp\text{-local-insert-bucket } ?M \ ?p \ ?bs). \text{length } (bp\text{-bucket-entries } b) \leq ?M) \wedge \text{sorted-wrt } (\lambda b \ c. bp\text{-marker } b \leq bp\text{-marker } c) (bp\text{-local-insert-bucket } ?M \ ?p \ ?bs) \wedge (\forall b \in \text{set } (bp\text{-local-insert-bucket } ?M \ ?p \ ?bs). \forall q \in \text{set } (bp\text{-bucket-entries } b). bp\text{-marker } b \leq \text{snd } q) \wedge bp\text{-bucket-boundaries-ok } (bp\text{-local-insert-bucket } ?M \ ?p \ ?bs)$ is the high-level summary used by the state-level Insert proof.

lemma $\text{length-bp-bucket-entries-flat-bucketize-entries } [simp]:$
assumes $0 < M$
shows $\text{length } (bp\text{-bucket-entries-flat } (bp\text{-bucketize-entries } M \ xs)) = \text{length } xs$
using $bp\text{-bucket-entries-flat-bucketize-entries}[OF \ \text{assms, of } xs]$ **by** *simp*

lemma $\text{length-bp-bucketize-sorted-entries-aux-le-ratio}:$
assumes $M\text{-pos}: 0 < M$
and $\text{fuel}: \text{length } xs \leq \text{fuel}$
shows $\text{length } (bp\text{-bucketize-sorted-entries-aux } \text{fuel } M \ xs) \leq \text{Suc } (\text{length } xs \text{ div } M)$
using $M\text{-pos fuel}$

proof (*induction fuel arbitrary: xs*)
case 0
then show $?case$
by *simp*
next
case (Suc fuel)
show $?case$
proof ($\text{cases } xs = []$)
case *True*
then show $?thesis$
by *simp*
next
case *False*
show $?thesis$

```

proof (cases length xs < M)
  case True
  then have drop M xs = []
    by simp
  then show ?thesis
    using Suc.prem1s False True by simp
next
  case False
  have M-le:  $M \leq \text{length } xs$ 
    using False by simp
  have len-drop:  $\text{length } (\text{drop } M \text{ } xs) \leq \text{fuel}$ 
    using Suc.prem1s False by simp
  have tail:
     $\text{length } (\text{bp-bucketize-sorted-entries-aux fuel } M (\text{drop } M \text{ } xs)) \leq$ 
     $\text{Suc } (\text{length } (\text{drop } M \text{ } xs) \text{ div } M)$ 
    by (rule Suc.IH[OF Suc.prem1s(1) len-drop])
  have len-xs:
     $\text{length } xs = M + \text{length } (\text{drop } M \text{ } xs)$ 
    using M-le by simp
  have div-eq:
     $\text{length } xs \text{ div } M = \text{Suc } (\text{length } (\text{drop } M \text{ } xs) \text{ div } M)$ 
  proof -
    have  $\text{length } xs \text{ div } M = (M + \text{length } (\text{drop } M \text{ } xs)) \text{ div } M$ 
      using len-xs by simp
    also have  $\dots = \text{length } (\text{drop } M \text{ } xs) \text{ div } M + 1$ 
      using Suc.prem1s(1) by (simp add: div-add-self1)
    finally show ?thesis
      by simp
  qed
  show ?thesis
    using Suc.prem1s False tail div-eq by simp
qed
qed
qed

lemma length-bp-bucketize-sorted-entries-le-ratio:
  assumes  $0 < M$ 
  shows  $\text{length } (\text{bp-bucketize-sorted-entries } M \text{ } xs) \leq$ 
   $\text{Suc } (\text{length } xs \text{ div } M)$ 
  unfolding bp-bucketize-sorted-entries-def
  by (rule length-bp-bucketize-sorted-entries-aux-le-ratio[OF assms]) simp

lemma length-bp-bucketize-entries-le-ratio:
  assumes  $0 < M$ 
  shows  $\text{length } (\text{bp-bucketize-entries } M \text{ } xs) \leq$ 
   $\text{Suc } (\text{length } xs \text{ div } M)$ 
  unfolding bp-bucketize-entries-def
  using length-bp-bucketize-sorted-entries-le-ratio[OF assms, of sort-key snd xs]
  by simp

```

```

lemma length-bp-bucketize-entries-singleton [simp]:
  assumes  $0 < M$ 
  shows  $\text{length } (\text{bp-bucketize-entries } M \ [p]) = 1$ 
  using assms
  unfolding bp-bucketize-entries-def bp-bucketize-sorted-entries-def
  by (cases M) simp-all

lemma length-bp-bucketize-sorted-entries-aux-le-length:
  assumes M-pos:  $0 < M$ 
  and fuel:  $\text{length } xs \leq \text{fuel}$ 
  shows  $\text{length } (\text{bp-bucketize-sorted-entries-aux fuel } M \ xs) \leq$ 
     $\text{length } xs$ 
  using M-pos fuel
proof (induction fuel arbitrary: xs)
  case 0
  then show ?case
    by simp
next
  case (Suc fuel)
  show ?case
  proof (cases xs = [])
    case True
    then show ?thesis by simp
  next
  case False
  have len-drop:  $\text{length } (\text{drop } M \ xs) \leq \text{fuel}$ 
    using Suc.prem1 False by (cases M) auto
  have tail:
     $\text{length } (\text{bp-bucketize-sorted-entries-aux fuel } M \ (\text{drop } M \ xs)) \leq$ 
       $\text{length } (\text{drop } M \ xs)$ 
    by (rule Suc.IH[OF Suc.prem1(1) len-drop])
  have drop-less:  $\text{length } (\text{drop } M \ xs) < \text{length } xs$ 
    using Suc.prem1 False by simp
  have drop-le:  $\text{Suc } (\text{length } (\text{drop } M \ xs)) \leq \text{length } xs$ 
    using drop-less by simp
  have unfold:
     $\text{length } (\text{bp-bucketize-sorted-entries-aux } (\text{Suc fuel}) \ M \ xs) =$ 
       $\text{Suc } (\text{length } (\text{bp-bucketize-sorted-entries-aux fuel } M \ (\text{drop } M \ xs)))$ 
    using Suc.prem1 False by simp
  show ?thesis
    using tail drop-le unfolding unfold by simp
qed
qed

lemma length-bp-bucketize-entries-le-length:
  assumes  $0 < M$ 
  shows  $\text{length } (\text{bp-bucketize-entries } M \ xs) \leq \text{length } xs$ 
  unfolding bp-bucketize-entries-def bp-bucketize-sorted-entries-def

```

```

using length-bp-bucketize-sorted-entries-aux-le-length
  [OF assms, of sort-key snd xs length xs]
by simp

lemma length-bp-bucketize-entries-le-three:
  assumes M-pos:  $0 < M$ 
    and len:  $\text{length } xs \leq \text{Suc } (2 * M)$ 
  shows  $\text{length } (\text{bp-bucketize-entries } M \ xs) \leq 3$ 
proof (cases  $M = 1$ )
  case True
    have  $\text{length } (\text{bp-bucketize-entries } M \ xs) \leq \text{length } xs$ 
      by (rule length-bp-bucketize-entries-le-length[OF M-pos])
    also have  $\dots \leq 3$ 
      using len True by simp
    finally show ?thesis .
  next
    case False
    have M-ge2:  $2 \leq M$ 
      using M-pos False by simp
    have div-bound:  $\text{length } xs \text{ div } M \leq 2$ 
    proof -
      have  $\text{length } xs \text{ div } M \leq \text{Suc } (2 * M) \text{ div } M$ 
        by (rule div-le-mono[OF len])
      moreover have  $\text{Suc } (2 * M) \text{ div } M < 3$ 
    proof -
      have  $\text{Suc } (2 * M) < 3 * M$ 
        using M-ge2 by simp
      then show ?thesis
        using M-pos by (simp add: div-less-iff-less-mult)
    qed
    ultimately show ?thesis
      by simp
  qed
  have  $\text{length } (\text{bp-bucketize-entries } M \ xs) \leq \text{Suc } (\text{length } xs \text{ div } M)$ 
    by (rule length-bp-bucketize-entries-le-ratio[OF M-pos])
  also have  $\dots \leq 3$ 
    using div-bound by simp
  finally show ?thesis .
qed

lemma length-bp-lazy-bucket-insert-entries-le-three:
  assumes M-pos:  $0 < M$ 
    and size:  $\text{length } (\text{bp-bucket-entries } b) \leq 2 * M$ 
  shows  $\text{length } (\text{bp-lazy-bucket-insert-entries } M \ p \ b) \leq 3$ 
proof (cases  $\text{length } (\text{bp-bucket-entries } b) < 2 * M$ )
  case True
    then show ?thesis
      unfolding bp-lazy-bucket-insert-entries-def by simp
  next

```

```

case False
have len:  $\text{length } (p \# \text{bp-bucket-entries } b) \leq \text{Suc } (2 * M)$ 
  using size by simp
show ?thesis
  using False length-bp-bucketize-entries-le-three[OF M-pos len]
  unfolding bp-lazy-bucket-insert-entries-def by simp
qed

lemma length-bp-lazy-insert-bucket-le:
  assumes M-pos:  $0 < M$ 
    and sizes:  $\forall b \in \text{set } bs. \text{length } (\text{bp-bucket-entries } b) \leq 2 * M$ 
  shows  $\text{length } (\text{bp-lazy-insert-bucket } M \ p \ bs) \leq \text{length } bs + 2$ 
  using sizes
proof (induction bs arbitrary: p)
  case Nil
  then show ?case
    using length-bp-bucketize-entries-singleton[OF M-pos, of p] by simp
next
  case (Cons b bs)
  note IH = Cons.IH
  note sizes = Cons.prems
  have b-size:  $\text{length } (\text{bp-bucket-entries } b) \leq 2 * M$ 
    using sizes by simp
  show ?case
  proof (cases bs)
  case Nil
  show ?thesis
  proof (cases snd p < bp-marker b)
  case True
  then show ?thesis
    using Nil length-bp-bucketize-entries-singleton[OF M-pos, of p]
    by simp
  next
  case False
  have  $\text{length } (\text{bp-lazy-bucket-insert-entries } M \ p \ b) \leq 3$ 
    by (rule length-bp-lazy-bucket-insert-entries-le-three
      [OF M-pos b-size])
  then show ?thesis
    using Nil False by simp
  qed
next
  case (Cons c cs)
  have tail-sizes:
     $\forall b \in \text{set } bs. \text{length } (\text{bp-bucket-entries } b) \leq 2 * M$ 
    using sizes by simp
  show ?thesis
  proof (cases snd p < bp-marker b)
  case True
  then show ?thesis

```

```

    using Cons length-bp-bucketize-entries-singleton[OF M-pos, of p]
    by simp
next
case False
show ?thesis
proof (cases snd p < bp-marker c)
case True
have length (bp-lazy-bucket-insert-entries M p b) ≤ 3
  by (rule length-bp-lazy-bucket-insert-entries-le-three
      [OF M-pos b-size])
then show ?thesis
  using Cons False True by simp
next
case False
have tail:
  length (bp-lazy-insert-bucket M p bs) ≤ length bs + 2
  by (rule IH[OF tail-sizes])
show ?thesis
  using Cons <¬ snd p < bp-marker b> False tail by simp
qed
qed
qed
qed

lemma length-bp-bucket-entries-flat-local-insert-bucket [simp]:
  assumes 0 < M
  shows length (bp-bucket-entries-flat (bp-local-insert-bucket M p bs)) =
    Suc (length (bp-bucket-entries-flat bs))
  using assms
proof (induction bs arbitrary: p)
case Nil
have len:
  length (bp-bucket-entries-flat (bp-bucketize-entries M [p])) =
    length [p]
  by (rule length-bp-bucket-entries-flat-bucketize-entries[OF Nil.prem])
show ?case
  using len by (simp add: bp-bucket-entries-flat-def)
next
case (Cons b bs)
then show ?case
proof (cases bs)
case Nil
have len-single:
  length (bp-bucket-entries-flat (bp-bucketize-entries M [p])) =
    length [p]
  by (rule length-bp-bucket-entries-flat-bucketize-entries[OF Cons.prem])
have len-inserted:
  length
    (bp-bucket-entries-flat

```

```

      (bp-bucketize-entries M (p # bp-bucket-entries b))) =
      length (p # bp-bucket-entries b)
    by (rule length-bp-bucket-entries-flat-bucketize-entries[OF Cons.prem])
  then show ?thesis
    using Nil len-single len-inserted
    by (auto simp: bp-bucket-entries-flat-def)
next
case (Cons c cs)
have len-single:
  length (bp-bucket-entries-flat (bp-bucketize-entries M [p])) =
  length [p]
  by (rule length-bp-bucket-entries-flat-bucketize-entries[OF Cons.prem])
have len-inserted:
  length
    (bp-bucket-entries-flat
      (bp-bucketize-entries M (p # bp-bucket-entries b))) =
  length (p # bp-bucket-entries b)
  by (rule length-bp-bucket-entries-flat-bucketize-entries[OF Cons.prem])
have tail-len:
  length (bp-bucket-entries-flat
    (bp-local-insert-bucket M p (c # cs))) =
  Suc (length (bp-bucket-entries-flat (c # cs)))
  using Cons.IH[OF Cons.prem, of p] Cons by simp
show ?thesis
proof (cases snd p < bp-marker b)
case True
  then show ?thesis
    using Cons len-single by (simp add: bp-bucket-entries-flat-def)
next
case False
  note not-before-b = False
  show ?thesis
  proof (cases snd p < bp-marker c)
  case True
    then show ?thesis
      using Cons not-before-b len-inserted
      by (simp add: bp-bucket-entries-flat-def)
  next
  case False
    then show ?thesis
      using Cons not-before-b tail-len
      by (simp add: bp-bucket-entries-flat-def)
  qed
qed
qed
qed

```

lemma *bp-local-insert-bucket-distinct-keys*:
 assumes *M-pos*: $0 < M$


```

    and distinct: distinct (map fst (bp-bucket-entries-flat bs))
    and fresh:  $\text{fst } p \notin \text{bp-entry-keys } (\text{bp-bucket-entries-flat } bs)$ 
  shows distinct
    (map fst (bp-bucket-entries-flat (bp-local-insert-bucket M p bs)))
proof (rule card-distinct)
  let ?old = map fst (bp-bucket-entries-flat bs)
  let ?new =
    map fst (bp-bucket-entries-flat (bp-local-insert-bucket M p bs))
  have set-new:  $\text{set } ?new = \text{insert } (\text{fst } p) (\text{set } ?old)$ 
    using set-bp-bucket-entries-flat-local-insert-bucket[OF M-pos, of p bs]
    by auto
  have fresh-set:  $\text{fst } p \notin \text{set } ?old$ 
    using fresh unfolding bp-entry-keys-def by simp
  have card ( $\text{set } ?new$ ) = Suc (card ( $\text{set } ?old$ ))
    unfolding set-new using fresh-set by simp
  also have ... = Suc (length ?old)
    using distinct-card[OF distinct] by simp
  also have ... = length ?new
    using length-bp-bucket-entries-flat-local-insert-bucket[OF M-pos, of p bs]
    by simp
  finally show card ( $\text{set } ?new$ ) = length ?new .
qed

```

lemma *bp-local-insert-bucket-values-consistent*:

```

  assumes M-pos:  $0 < M$ 
    and vals:  $\forall q \in \text{set } (\text{bp-bucket-entries-flat } bs). f (\text{fst } q) = \text{snd } q$ 
    and fresh:  $\text{fst } p \notin \text{bp-entry-keys } (\text{bp-bucket-entries-flat } bs)$ 
  shows  $\forall q \in \text{set } (\text{bp-bucket-entries-flat } (\text{bp-local-insert-bucket } M p bs)).$ 
    ( $f(\text{fst } p := \text{snd } p)$ ) ( $\text{fst } q$ ) = snd q
proof
  fix q
  assume q:  $q \in$ 
     $\text{set } (\text{bp-bucket-entries-flat } (\text{bp-local-insert-bucket } M p bs))$ 
  then have q-cases:  $q = p \vee q \in \text{set } (\text{bp-bucket-entries-flat } bs)$ 
    using set-bp-bucket-entries-flat-local-insert-bucket[OF M-pos, of p bs]
    by auto
  from q-cases show ( $f(\text{fst } p := \text{snd } p)$ ) ( $\text{fst } q$ ) = snd q
proof
  assume  $q = p$ 
  then show ?thesis by simp
next
  assume q-old:  $q \in \text{set } (\text{bp-bucket-entries-flat } bs)$ 
  have  $\text{fst } q \in \text{bp-entry-keys } (\text{bp-bucket-entries-flat } bs)$ 
    using q-old unfolding bp-entry-keys-def by auto
  then have  $\text{fst } q \neq \text{fst } p$ 
    using fresh by auto
  moreover have  $f (\text{fst } q) = \text{snd } q$ 
    using vals q-old by blast
  ultimately show ?thesis

```

by *simp*
qed
qed

lemma *bp-empty-invariant*:
 assumes $0 < M$
 shows *bp-invariant* (*bp-empty* $M B$)
 using *assms*
 unfolding *bp-invariant-def bp-empty-def bp-entries-def*
 bp-bucket-entries-flat-def bp-bucket-sizes-ok-def
 bp-bucket-markers-sorted-def bp-bucket-markers-lower-bound-def
 bp-values-consistent-def bp-distinct-keys-def
 by *simp*

lemma *bp-empty-boundaries-state-ok* [*simp*]:
bp-bucket-boundaries-state-ok (*bp-empty* $M B$)
 unfolding *bp-bucket-boundaries-state-ok-def bp-empty-def* by *simp*

lemma *bp-empty-ordered-invariant*:
 assumes $0 < M$
 shows *bp-ordered-invariant* (*bp-empty* $M B$)
 unfolding *bp-ordered-invariant-def*
 using *bp-empty-invariant*[*OF assms*] by *simp*

lemma *bp-empty-view* [*simp*]:
bp-view (*bp-empty* $M B$) = (*keys-of* = {}, *value-of* = (λ -. B))
 unfolding *bp-view-def bp-empty-def bp-entries-def*
 bp-bucket-entries-flat-def bp-entry-keys-def
 by *simp*

lemma *bp-empty-partition-upper-bound* [*simp*]:
partition-upper-bound (*bp-view* (*bp-empty* $M B0$)) B
 unfolding *partition-upper-bound-def* by *simp*

lemma *set-bp-insert-bucket*:
 $\text{set } (\text{bp-insert-bucket } p \text{ } bs) = \text{insert } (\text{bp-singleton-bucket } p) (\text{set } bs)$
 by (*induction bs*) *auto*

lemma *set-bp-bucket-entries-flat-insert-bucket*:
 $\text{set } (\text{bp-bucket-entries-flat } (\text{bp-insert-bucket } p \text{ } bs)) =$
 $\text{insert } p (\text{set } (\text{bp-bucket-entries-flat } bs))$
 unfolding *bp-bucket-entries-flat-def*
 by (*induction bs*) *auto*

lemma *length-bp-bucket-entries-flat-insert-bucket* [*simp*]:
 $\text{length } (\text{bp-bucket-entries-flat } (\text{bp-insert-bucket } p \text{ } bs)) =$
 $\text{Suc } (\text{length } (\text{bp-bucket-entries-flat } bs))$
 unfolding *bp-bucket-entries-flat-def*
 by (*induction bs*) *auto*

```

lemma bp-entry-keys-insert-bucket [simp]:
  bp-entry-keys (bp-bucket-entries-flat (bp-insert-bucket p bs)) =
    insert (fst p) (bp-entry-keys (bp-bucket-entries-flat bs))
unfolding bp-entry-keys-def
using set-bp-bucket-entries-flat-insert-bucket[of p bs] by auto

lemma bp-insert-bucket-sizes-ok:
assumes  $\forall b \in \text{set } bs. \text{length } (\text{bp-bucket-entries } b) \leq M$ 
and  $0 < M$ 
shows  $\forall b \in \text{set } (\text{bp-insert-bucket } p \text{ } bs).$ 
   $\text{length } (\text{bp-bucket-entries } b) \leq M$ 
using assms unfolding bp-singleton-bucket-def
by (induction bs) auto

lemma bp-insert-bucket-markers-sorted:
assumes sorted-wrt ( $\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c$ ) bs
shows sorted-wrt ( $\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c$ )
  (bp-insert-bucket p bs)
using assms
proof (induction bs)
case Nil
then show ?case
  unfolding bp-singleton-bucket-def by simp
next
case (Cons b bs)
show ?case
proof (cases snd p  $\leq \text{bp-marker } b$ )
case True
then show ?thesis
  using Cons.prems by auto
next
case False
have tail-sorted:
  sorted-wrt ( $\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c$ ) bs
using Cons.prems by simp
have inserted-sorted:
  sorted-wrt ( $\lambda b \ c. \text{bp-marker } b \leq \text{bp-marker } c$ )
    (bp-insert-bucket p bs)
by (rule Cons.IH[OF tail-sorted])
have lower:
 $\bigwedge c. c \in \text{set } (\text{bp-insert-bucket } p \text{ } bs) \implies$ 
   $\text{bp-marker } b \leq \text{bp-marker } c$ 
using Cons.prems False
by (auto simp: set-bp-insert-bucket bp-singleton-bucket-def)
show ?thesis
using False inserted-sorted lower by simp
qed
qed

```

lemma *bp-insert-bucket-markers-lower-bound*:
assumes $\forall b \in \text{set } bs. \forall p \in \text{set } (bp\text{-bucket-entries } b).$
 $bp\text{-marker } b \leq \text{snd } p$
shows $\forall b \in \text{set } (bp\text{-insert-bucket } p \text{ } bs). \forall p \in \text{set } (bp\text{-bucket-entries } b).$
 $bp\text{-marker } b \leq \text{snd } p$
using *assms*
by (*induction bs*) *auto*

lemma *bp-insert-bucket-values-consistent*:
assumes $\forall q \in \text{set } (bp\text{-bucket-entries-flat } bs). f (\text{fst } q) = \text{snd } q$
and $\text{fst } p \notin bp\text{-entry-keys } (bp\text{-bucket-entries-flat } bs)$
shows $\forall q \in \text{set } (bp\text{-bucket-entries-flat } (bp\text{-insert-bucket } p \text{ } bs)).$
 $(f(\text{fst } p := \text{snd } p)) (\text{fst } q) = \text{snd } q$

proof
fix *q*
assume *q*: $q \in \text{set } (bp\text{-bucket-entries-flat } (bp\text{-insert-bucket } p \text{ } bs))$
then have *q-cases*: $q = p \vee q \in \text{set } (bp\text{-bucket-entries-flat } bs)$
using *set-bp-bucket-entries-flat-insert-bucket[of p bs]* **by** *auto*
from *q-cases* **show** $(f(\text{fst } p := \text{snd } p)) (\text{fst } q) = \text{snd } q$

proof
assume $q = p$
then show *?thesis* **by** *simp*

next
assume *q-old*: $q \in \text{set } (bp\text{-bucket-entries-flat } bs)$
have $\text{fst } q \in bp\text{-entry-keys } (bp\text{-bucket-entries-flat } bs)$
using *q-old* **unfolding** *bp-entry-keys-def* **by** *auto*
then have $\text{fst } q \neq \text{fst } p$
using *assms(2)* **by** *auto*
moreover have $f (\text{fst } q) = \text{snd } q$
using *assms(1)* *q-old* **by** *blast*
ultimately show *?thesis*
by *simp*

qed
qed

lemma *bp-insert-bucket-distinct-keys*:
assumes *distinct*: $\text{distinct } (\text{map } \text{fst } (bp\text{-bucket-entries-flat } bs))$
and *fresh*: $\text{fst } p \notin bp\text{-entry-keys } (bp\text{-bucket-entries-flat } bs)$
shows $\text{distinct } (\text{map } \text{fst } (bp\text{-bucket-entries-flat } (bp\text{-insert-bucket } p \text{ } bs)))$

proof (*rule card-distinct*)
let *?old* = $\text{map } \text{fst } (bp\text{-bucket-entries-flat } bs)$
let *?new* = $\text{map } \text{fst } (bp\text{-bucket-entries-flat } (bp\text{-insert-bucket } p \text{ } bs))$
have *set-new*: $\text{set } ?new = \text{insert } (\text{fst } p) (\text{set } ?old)$
using *set-bp-bucket-entries-flat-insert-bucket[of p bs]* **by** *auto*
have *fresh-set*: $\text{fst } p \notin \text{set } ?old$
using *fresh* **unfolding** *bp-entry-keys-def* **by** *simp*
have $\text{card } (\text{set } ?new) = \text{Suc } (\text{card } (\text{set } ?old))$
unfolding *set-new* **using** *fresh-set* **by** *simp*

also have $\dots = \text{Suc } (\text{length } ?old)$
using *distinct-card*[*OF distinct*] **by** *simp*
also have $\dots = \text{length } ?new$
by *simp*
finally show $\text{card } (\text{set } ?new) = \text{length } ?new$.
qed

lemma *distinct-map-fst-filter-neq*:
assumes *distinct* (*map fst xs*)
shows *distinct* (*map fst (filter ($\lambda p. \text{fst } p \neq x$) xs)*)
using *assms* **by** (*induction xs*) *auto*

lemma *distinct-map-fst-filter-notin*:
assumes *distinct* (*map fst xs*)
shows *distinct* (*map fst (filter ($\lambda p. \text{fst } p \notin S$) xs)*)
using *assms* **by** (*induction xs*) *auto*

lemma *bp-bucket-entries-flat-delete-key*:
 $\text{bp-bucket-entries-flat } (\text{map } (\text{bp-delete-key-from-bucket } x) \text{ bs}) =$
 $\text{filter } (\lambda p. \text{fst } p \neq x) (\text{bp-bucket-entries-flat bs})$
unfolding *bp-bucket-entries-flat-def bp-delete-key-from-bucket-def*
by (*induction bs*) *simp-all*

lemma *bp-bucket-entries-flat-delete-keys*:
 $\text{bp-bucket-entries-flat } (\text{map } (\text{bp-delete-keys-from-bucket } S) \text{ bs}) =$
 $\text{filter } (\lambda p. \text{fst } p \notin S) (\text{bp-bucket-entries-flat bs})$
unfolding *bp-bucket-entries-flat-def bp-delete-keys-from-bucket-def*
by (*induction bs*) *simp-all*

lemma *bp-entries-delete-key*:
 $\text{bp-entries } (\text{bp-delete-key } x \text{ } P) =$
 $\text{filter } (\lambda p. \text{fst } p \neq x) (\text{bp-entries } P)$
unfolding *bp-entries-def bp-delete-key-def*
by (*simp add: bp-bucket-entries-flat-delete-key*)

lemma *bp-entries-delete-keys*:
 $\text{bp-entries } (\text{bp-delete-keys } S \text{ } P) =$
 $\text{filter } (\lambda p. \text{fst } p \notin S) (\text{bp-entries } P)$
unfolding *bp-entries-def bp-delete-keys-def*
by (*simp add: bp-bucket-entries-flat-delete-keys*)

lemma *length-filter-fst-neq-ge-pred*:
assumes *distinct*: *distinct* (*map fst xs*)
shows $\text{length } xs \leq \text{Suc } (\text{length } (\text{filter } (\lambda p. \text{fst } p \neq x) xs))$
using *distinct*
proof (*induction xs*)
case *Nil*
then show *?case* **by** *simp*
next

```

case (Cons p xs)
show ?case
proof (cases fst p = x)
  case True
  have fst p  $\notin$  fst ' set xs
    using Cons.prems by simp
  have  $\forall q \in \text{set } xs. \text{fst } q \neq x$ 
  proof
    fix q
    assume q  $\in$  set xs
    then have fst q  $\in$  fst ' set xs
      by blast
    have fst q  $\neq$  fst p
    proof
      assume fst q = fst p
      then have fst p  $\in$  fst ' set xs
        using  $\langle \text{fst } q \in \text{fst ' set } xs \rangle$  by simp
      then show False
        using  $\langle \text{fst } p \notin \text{fst ' set } xs \rangle$  by simp
    qed
    then show fst q  $\neq$  x
      using True by simp
  qed
then have filter ( $\lambda p. \text{fst } p \neq x$ ) xs = xs
  by (simp add: filter-id-conv)
then show ?thesis
  using True by simp
next
  case False
  have tail: length xs  $\leq$  Suc (length (filter ( $\lambda p. \text{fst } p \neq x$ ) xs))
    by (rule Cons.IH) (use Cons.prems in simp)
  show ?thesis
    using False tail by simp
  qed
qed

```

lemma *length-bp-entries-delete-key-ge-pred*:

```

assumes bp-distinct-keys P
shows length (bp-entries P)  $\leq$ 
  Suc (length (bp-entries (bp-delete-key x P)))
using length-filter-fst-neq-ge-pred
  [of bp-entries P x] assms
unfolding bp-entries-delete-key bp-distinct-keys-def by simp

```

lemma *bp-entry-keys-delete-key [simp]*:

```

bp-entry-keys (bp-entries (bp-delete-key x P)) =
  bp-entry-keys (bp-entries P) - {x}
by (simp add: bp-entries-delete-key)

```

lemma *bp-entry-keys-delete-keys* [simp]:
 $bp_entry_keys\ (bp_entries\ (bp_delete_keys\ S\ P)) =$
 $bp_entry_keys\ (bp_entries\ P) - S$
by (simp add: bp-entries-delete-keys)

lemma *bp-values-delete-key* [simp]:
 $bp_values\ (bp_delete_key\ x\ P) = bp_values\ P$
unfolding bp-delete-key-def **by** simp

lemma *bp-values-delete-keys* [simp]:
 $bp_values\ (bp_delete_keys\ S\ P) = bp_values\ P$
unfolding bp-delete-keys-def **by** simp

lemma *bp-delete-key-markers-sorted-list*:
assumes sorted-wrt ($\lambda b\ c.\ bp_marker\ b \leq bp_marker\ c$) *bs*
shows sorted-wrt ($\lambda b\ c.\ bp_marker\ b \leq bp_marker\ c$)
 $(map\ (bp_delete_key_from_bucket\ x)\ bs)$
using assms **unfolding** bp-delete-key-from-bucket-def
by (induction *bs*) auto

lemma *bp-delete-keys-markers-sorted-list*:
assumes sorted-wrt ($\lambda b\ c.\ bp_marker\ b \leq bp_marker\ c$) *bs*
shows sorted-wrt ($\lambda b\ c.\ bp_marker\ b \leq bp_marker\ c$)
 $(map\ (bp_delete_keys_from_bucket\ S)\ bs)$
using assms **unfolding** bp-delete-keys-from-bucket-def
by (induction *bs*) auto

lemma *bp-delete-key-markers-sorted*:
assumes bp-bucket-markers-sorted *P*
shows bp-bucket-markers-sorted (bp-delete-key *x P*)
using assms
unfolding bp-bucket-markers-sorted-def bp-delete-key-def
bp-delete-key-from-bucket-def
by (simp add: bp-delete-key-markers-sorted-list
[unfolded bp-delete-key-from-bucket-def])

lemma *bp-delete-keys-markers-sorted*:
assumes bp-bucket-markers-sorted *P*
shows bp-bucket-markers-sorted (bp-delete-keys *S P*)
using assms
unfolding bp-bucket-markers-sorted-def bp-delete-keys-def
bp-delete-keys-from-bucket-def
by (simp add: bp-delete-keys-markers-sorted-list
[unfolded bp-delete-keys-from-bucket-def])

lemma *bp-delete-key-bucket-sizes-ok*:
assumes bp-bucket-sizes-ok *P*
shows bp-bucket-sizes-ok (bp-delete-key *x P*)
unfolding bp-bucket-sizes-ok-def

proof
fix b
assume b : $b \in \text{set } (bp\text{-buckets } (bp\text{-delete-key } x \ P))$
then obtain $b0$ **where** $b0$: $b0 \in \text{set } (bp\text{-buckets } P)$
and $b\text{-def}$: $b = bp\text{-delete-key-from-bucket } x \ b0$
unfolding $bp\text{-delete-key-def}$ **by** *auto*
have $\text{length } (bp\text{-bucket-entries } b) \leq \text{length } (bp\text{-bucket-entries } b0)$
unfolding $b\text{-def}$ $bp\text{-delete-key-from-bucket-def}$ **by** *simp*
also have $\dots \leq bp\text{-block-size } P$
using *assms* $b0$ **unfolding** $bp\text{-bucket-sizes-ok-def}$ **by** *blast*
finally show $\text{length } (bp\text{-bucket-entries } b) \leq$
 $bp\text{-block-size } (bp\text{-delete-key } x \ P)$
unfolding $bp\text{-delete-key-def}$ **by** *simp*
qed

lemma $bp\text{-delete-keys-bucket-sizes-ok}$:
assumes $bp\text{-bucket-sizes-ok } P$
shows $bp\text{-bucket-sizes-ok } (bp\text{-delete-keys } S \ P)$
unfolding $bp\text{-bucket-sizes-ok-def}$

proof
fix b
assume b : $b \in \text{set } (bp\text{-buckets } (bp\text{-delete-keys } S \ P))$
then obtain $b0$ **where** $b0$: $b0 \in \text{set } (bp\text{-buckets } P)$
and $b\text{-def}$: $b = bp\text{-delete-keys-from-bucket } S \ b0$
unfolding $bp\text{-delete-keys-def}$ **by** *auto*
have $\text{length } (bp\text{-bucket-entries } b) \leq \text{length } (bp\text{-bucket-entries } b0)$
unfolding $b\text{-def}$ $bp\text{-delete-keys-from-bucket-def}$ **by** *simp*
also have $\dots \leq bp\text{-block-size } P$
using *assms* $b0$ **unfolding** $bp\text{-bucket-sizes-ok-def}$ **by** *blast*
finally show $\text{length } (bp\text{-bucket-entries } b) \leq$
 $bp\text{-block-size } (bp\text{-delete-keys } S \ P)$
unfolding $bp\text{-delete-keys-def}$ **by** *simp*
qed

lemma $bp\text{-delete-key-lazy-bucket-sizes-ok}$:
assumes $bp\text{-lazy-bucket-sizes-ok } P$
shows $bp\text{-lazy-bucket-sizes-ok } (bp\text{-delete-key } x \ P)$
unfolding $bp\text{-lazy-bucket-sizes-ok-def}$

proof
fix b
assume b : $b \in \text{set } (bp\text{-buckets } (bp\text{-delete-key } x \ P))$
then obtain $b0$ **where** $b0$: $b0 \in \text{set } (bp\text{-buckets } P)$
and $b\text{-def}$: $b = bp\text{-delete-key-from-bucket } x \ b0$
unfolding $bp\text{-delete-key-def}$ **by** *auto*
have $\text{length } (bp\text{-bucket-entries } b) \leq \text{length } (bp\text{-bucket-entries } b0)$
unfolding $b\text{-def}$ $bp\text{-delete-key-from-bucket-def}$ **by** *simp*
also have $\dots \leq 2 * bp\text{-block-size } P$
using *assms* $b0$ **unfolding** $bp\text{-lazy-bucket-sizes-ok-def}$ **by** *blast*
finally show $\text{length } (bp\text{-bucket-entries } b) \leq$


```

    2 * bp-block-size (bp-delete-key x P)
  unfolding bp-delete-key-def by simp
qed

lemma bp-delete-keys-lazy-bucket-sizes-ok:
  assumes bp-lazy-bucket-sizes-ok P
  shows bp-lazy-bucket-sizes-ok (bp-delete-keys S P)
  unfolding bp-lazy-bucket-sizes-ok-def
proof
  fix b
  assume b: b ∈ set (bp-buckets (bp-delete-keys S P))
  then obtain b0 where b0: b0 ∈ set (bp-buckets P)
    and b-def: b = bp-delete-keys-from-bucket S b0
    unfolding bp-delete-keys-def by auto
  have length (bp-bucket-entries b) ≤ length (bp-bucket-entries b0)
    unfolding b-def bp-delete-keys-from-bucket-def by simp
  also have ... ≤ 2 * bp-block-size P
    using assms b0 unfolding bp-lazy-bucket-sizes-ok-def by blast
  finally show length (bp-bucket-entries b) ≤
    2 * bp-block-size (bp-delete-keys S P)
    unfolding bp-delete-keys-def by simp
qed

lemma bp-delete-key-markers-lower-bound:
  assumes bp-bucket-markers-lower-bound P
  shows bp-bucket-markers-lower-bound (bp-delete-key x P)
  using assms
  unfolding bp-bucket-markers-lower-bound-def bp-delete-key-def
    bp-delete-key-from-bucket-def
  by auto

lemma bp-delete-keys-markers-lower-bound:
  assumes bp-bucket-markers-lower-bound P
  shows bp-bucket-markers-lower-bound (bp-delete-keys S P)
  using assms
  unfolding bp-bucket-markers-lower-bound-def bp-delete-keys-def
    bp-delete-keys-from-bucket-def
  by auto

lemma bp-delete-key-values-consistent:
  assumes bp-values-consistent P
  shows bp-values-consistent (bp-delete-key x P)
  using assms
  unfolding bp-values-consistent-def
  by (auto simp: bp-entries-delete-key)

lemma bp-delete-keys-values-consistent:
  assumes bp-values-consistent P
  shows bp-values-consistent (bp-delete-keys S P)

```

```

using assms
unfolding bp-values-consistent-def
by (auto simp: bp-entries-delete-keys)

lemma bp-delete-key-distinct-keys:
  assumes bp-distinct-keys P
  shows bp-distinct-keys (bp-delete-key x P)
  using assms
  unfolding bp-distinct-keys-def
  by (simp add: bp-entries-delete-key distinct-map-fst-filter-neq)

lemma bp-delete-keys-distinct-keys:
  assumes bp-distinct-keys P
  shows bp-distinct-keys (bp-delete-keys S P)
  using assms
  unfolding bp-distinct-keys-def
  by (simp add: bp-entries-delete-keys distinct-map-fst-filter-notin)

lemma bp-bucket-boundaries-ok-delete-key-from-bucket:
  assumes bp-bucket-boundaries-ok bs
  shows bp-bucket-boundaries-ok (map (bp-delete-key-from-bucket x) bs)
  using assms
proof (induction bs)
  case Nil
  then show ?case
    by simp
next
  case (Cons b bs)
  then show ?case
    proof (cases bs)
    case Nil
    then show ?thesis
      by simp
    next
    case (Cons c cs)
    have head:
       $\forall p \in \text{set } (bp\text{-bucket-entries } b). \text{snd } p \leq bp\text{-marker } c$ 
      using Cons.prem1 Cons by simp
    have tail: bp-bucket-boundaries-ok
      (map (bp-delete-key-from-bucket x) (c # cs))
    using Cons.IH Cons.prem1 Cons by simp
    show ?thesis
      using Cons head tail
      unfolding bp-delete-key-from-bucket-def by auto
  qed
qed

lemma bp-bucket-boundaries-ok-delete-keys-from-bucket:
  assumes bp-bucket-boundaries-ok bs

```

```

shows bp-bucket-boundaries-ok (map (bp-delete-keys-from-bucket S) bs)
using assms
proof (induction bs)
  case Nil
  then show ?case
    by simp
next
  case (Cons b bs)
  then show ?case
  proof (cases bs)
    case Nil
    then show ?thesis
      by simp
  next
    case (Cons c cs)
    have head:
       $\forall p \in \text{set } (\text{bp-bucket-entries } b). \text{snd } p \leq \text{bp-marker } c$ 
      using Cons.prem Cons by simp
    have tail: bp-bucket-boundaries-ok
      (map (bp-delete-keys-from-bucket S) (c # cs))
      using Cons.IH Cons.prem Cons by simp
    show ?thesis
      using Cons head tail
      unfolding bp-delete-keys-from-bucket-def by auto
  qed
qed

```

```

lemma bp-delete-key-boundaries-state-ok:
  assumes bp-bucket-boundaries-state-ok P
  shows bp-bucket-boundaries-state-ok (bp-delete-key x P)
  using assms
  unfolding bp-bucket-boundaries-state-ok-def bp-delete-key-def
  by (simp add: bp-bucket-boundaries-ok-delete-key-from-bucket)

```

```

lemma bp-delete-keys-boundaries-state-ok:
  assumes bp-bucket-boundaries-state-ok P
  shows bp-bucket-boundaries-state-ok (bp-delete-keys S P)
  using assms
  unfolding bp-bucket-boundaries-state-ok-def bp-delete-keys-def
  by (simp add: bp-bucket-boundaries-ok-delete-keys-from-bucket)

```

```

lemma bp-delete-key-invariant:
  assumes bp-invariant P
  shows bp-invariant (bp-delete-key x P)
proof -
  have block:  $0 < \text{bp-block-size } (\text{bp-delete-key } x P)$ 
    using assms unfolding bp-invariant-def bp-delete-key-def by simp
  have distinct: bp-distinct-keys (bp-delete-key x P)
    using assms unfolding bp-invariant-def

```

```

  by (auto intro: bp-delete-key-distinct-keys)
have sizes: bp-bucket-sizes-ok (bp-delete-key x P)
  using assms unfolding bp-invariant-def
  by (auto intro: bp-delete-key-bucket-sizes-ok)
have sorted: bp-bucket-markers-sorted (bp-delete-key x P)
  using assms unfolding bp-invariant-def
  by (auto intro: bp-delete-key-markers-sorted)
have markers: bp-bucket-markers-lower-bound (bp-delete-key x P)
  using assms unfolding bp-invariant-def
  by (auto intro: bp-delete-key-markers-lower-bound)
have vals: bp-values-consistent (bp-delete-key x P)
  using assms unfolding bp-invariant-def
  by (auto intro: bp-delete-key-values-consistent)
show ?thesis
  unfolding bp-invariant-def using block distinct sizes sorted markers vals by
blast
qed

```

```

lemma bp-delete-key-ordered-invariant:
  assumes bp-ordered-invariant P
  shows bp-ordered-invariant (bp-delete-key x P)
  unfolding bp-ordered-invariant-def
  using bp-delete-key-invariant[OF bp-ordered-invariant-invariant[OF assms]]
    bp-delete-key-boundaries-state-ok
    [OF bp-ordered-invariant-boundaries-state-ok[OF assms]]
  by blast

```

```

lemma bp-delete-key-lazy-invariant:
  assumes bp-lazy-invariant P
  shows bp-lazy-invariant (bp-delete-key x P)
proof -
  have block: 0 < bp-block-size (bp-delete-key x P)
    using assms unfolding bp-lazy-invariant-def bp-delete-key-def by simp
  have distinct: bp-distinct-keys (bp-delete-key x P)
    using assms unfolding bp-lazy-invariant-def
    by (auto intro: bp-delete-key-distinct-keys)
  have sizes: bp-lazy-bucket-sizes-ok (bp-delete-key x P)
    using assms unfolding bp-lazy-invariant-def
    by (auto intro: bp-delete-key-lazy-bucket-sizes-ok)
  have sorted: bp-bucket-markers-sorted (bp-delete-key x P)
    using assms unfolding bp-lazy-invariant-def
    by (auto intro: bp-delete-key-markers-sorted)
  have markers: bp-bucket-markers-lower-bound (bp-delete-key x P)
    using assms unfolding bp-lazy-invariant-def
    by (auto intro: bp-delete-key-markers-lower-bound)
  have vals: bp-values-consistent (bp-delete-key x P)
    using assms unfolding bp-lazy-invariant-def
    by (auto intro: bp-delete-key-values-consistent)
show ?thesis

```

unfolding *bp-lazy-invariant-def*
 using *block distinct sizes sorted markers vals* by *blast*
 qed

lemma *bp-delete-key-lazy-ordered-invariant*:
 assumes *bp-lazy-ordered-invariant P*
 shows *bp-lazy-ordered-invariant (bp-delete-key x P)*
 unfolding *bp-lazy-ordered-invariant-def*
 using *bp-delete-key-lazy-invariant*
 [*OF bp-lazy-ordered-invariant-lazy-invariant[OF assms]*]
 bp-delete-key-boundaries-state-ok
 [*OF bp-lazy-ordered-invariant-boundaries-state-ok[OF assms]*]
 by *blast*

lemma *bp-delete-keys-invariant*:
 assumes *bp-invariant P*
 shows *bp-invariant (bp-delete-keys S P)*
proof –
 have *block*: $0 < \text{bp-block-size } (bp\text{-delete-keys } S \ P)$
 using *assms* unfolding *bp-invariant-def bp-delete-keys-def* by *simp*
 have *distinct*: *bp-distinct-keys (bp-delete-keys S P)*
 using *assms* unfolding *bp-invariant-def*
 by (auto intro: *bp-delete-keys-distinct-keys*)
 have *sizes*: *bp-bucket-sizes-ok (bp-delete-keys S P)*
 using *assms* unfolding *bp-invariant-def*
 by (auto intro: *bp-delete-keys-bucket-sizes-ok*)
 have *sorted*: *bp-bucket-markers-sorted (bp-delete-keys S P)*
 using *assms* unfolding *bp-invariant-def*
 by (auto intro: *bp-delete-keys-markers-sorted*)
 have *markers*: *bp-bucket-markers-lower-bound (bp-delete-keys S P)*
 using *assms* unfolding *bp-invariant-def*
 by (auto intro: *bp-delete-keys-markers-lower-bound*)
 have *vals*: *bp-values-consistent (bp-delete-keys S P)*
 using *assms* unfolding *bp-invariant-def*
 by (auto intro: *bp-delete-keys-values-consistent*)
 show ?thesis
 unfolding *bp-invariant-def* using *block distinct sizes sorted markers vals* by
blast
 qed

lemma *bp-delete-keys-ordered-invariant*:
 assumes *bp-ordered-invariant P*
 shows *bp-ordered-invariant (bp-delete-keys S P)*
 unfolding *bp-ordered-invariant-def*
 using *bp-delete-keys-invariant[OF bp-ordered-invariant-invariant[OF assms]]*
 bp-delete-keys-boundaries-state-ok
 [*OF bp-ordered-invariant-boundaries-state-ok[OF assms]*]
 by *blast*

lemma *bp-delete-keys-lazy-invariant*:
assumes *bp-lazy-invariant P*
shows *bp-lazy-invariant (bp-delete-keys S P)*
proof –
have *block*: $0 < \text{bp-block-size } (\text{bp-delete-keys } S \ P)$
using *assms unfolding bp-lazy-invariant-def bp-delete-keys-def* **by** *simp*
have *distinct*: *bp-distinct-keys (bp-delete-keys S P)*
using *assms unfolding bp-lazy-invariant-def*
by (*auto intro: bp-delete-keys-distinct-keys*)
have *sizes*: *bp-lazy-bucket-sizes-ok (bp-delete-keys S P)*
using *assms unfolding bp-lazy-invariant-def*
by (*auto intro: bp-delete-keys-lazy-bucket-sizes-ok*)
have *sorted*: *bp-bucket-markers-sorted (bp-delete-keys S P)*
using *assms unfolding bp-lazy-invariant-def*
by (*auto intro: bp-delete-keys-markers-sorted*)
have *markers*: *bp-bucket-markers-lower-bound (bp-delete-keys S P)*
using *assms unfolding bp-lazy-invariant-def*
by (*auto intro: bp-delete-keys-markers-lower-bound*)
have *vals*: *bp-values-consistent (bp-delete-keys S P)*
using *assms unfolding bp-lazy-invariant-def*
by (*auto intro: bp-delete-keys-values-consistent*)
show *?thesis*
unfolding *bp-lazy-invariant-def*
using *block distinct sizes sorted markers vals* **by** *blast*
qed

lemma *bp-delete-keys-view*:
 $\text{bp-view } (\text{bp-delete-keys } S \ P) =$
 $\langle \text{keys-of} = \text{keys-of } (\text{bp-view } P) - S,$
 $\text{value-of} = \text{value-of } (\text{bp-view } P) \rangle$
unfolding *bp-view-def* **by** (*simp add: bp-entries-delete-keys*)

lemma *bp-entry-keys-insert-bucket-state [simp]*:
 $\text{bp-entry-keys } (\text{bp-entries } (P \langle \text{bp-buckets} := \text{bp-insert-bucket } p \ (\text{bp-buckets } P) \rangle)) =$
 $\text{insert } (\text{fst } p) \ (\text{bp-entry-keys } (\text{bp-entries } P))$
unfolding *bp-entries-def* **by** *simp*

lemma *bp-entry-keys-insert-bucket-state-values [simp]*:
 $\text{bp-entry-keys } (\text{bp-entries } (P \langle \text{bp-buckets} := \text{bp-insert-bucket } p \ (\text{bp-buckets } P),$
 $\text{bp-values} := f \rangle)) =$
 $\text{insert } (\text{fst } p) \ (\text{bp-entry-keys } (\text{bp-entries } P))$
unfolding *bp-entries-def* **by** *simp*

lemma *bp-insert-invariant*:
assumes *inv: bp-invariant P*
shows *bp-invariant (bp-insert x b P)*
proof –

```

let ?old-keys = bp-entry-keys (bp-entries P)
let ?b = if  $x \in ?old\text{-}keys$  then min (bp-values P x) b else b
let ?P0 = bp-delete-key x P
have inv0: bp-invariant ?P0
  by (rule bp-delete-key-invariant[OF inv])
have block-pos:  $0 < bp\text{-}block\text{-}size\ ?P0$ 
  using inv0 unfolding bp-invariant-def by blast
have distinct0:
  distinct (map fst (bp-entries ?P0))
  using inv0 unfolding bp-invariant-def bp-distinct-keys-def by blast
have sizes0:
   $\forall c \in set\ (bp\text{-}buckets\ ?P0).$ 
  length (bp-bucket-entries c)  $\leq bp\text{-}block\text{-}size\ ?P0$ 
  using inv0 unfolding bp-invariant-def bp-bucket-sizes-ok-def by blast
have sorted0:
  sorted-wrt ( $\lambda c\ d.\ bp\text{-}marker\ c \leq bp\text{-}marker\ d$ ) (bp-buckets ?P0)
  using inv0 unfolding bp-invariant-def bp-bucket-markers-sorted-def by blast
have markers0:
   $\forall c \in set\ (bp\text{-}buckets\ ?P0).$   $\forall p \in set\ (bp\text{-}bucket\text{-}entries\ c).$ 
  bp-marker c  $\leq snd\ p$ 
  using inv0 unfolding bp-invariant-def bp-bucket-markers-lower-bound-def by
blast
have values0:
   $\forall p \in set\ (bp\text{-}entries\ ?P0).$  bp-values ?P0 (fst p) = snd p
  using inv0 by (auto simp: bp-invariant-def bp-values-consistent-def)
have fresh:  $x \notin bp\text{-}entry\text{-}keys\ (bp\text{-}entries\ ?P0)$ 
  unfolding bp-entries-delete-key by simp
have distinct-insert:
  distinct (map fst
    (bp-bucket-entries-flat (bp-insert-bucket (x, ?b) (bp-buckets ?P0))))
  by (rule bp-insert-bucket-distinct-keys)
  (use distinct0 fresh in <simp-all add: bp-entries-def>)
have values0-P:
   $\forall p \in set\ (bp\text{-}entries\ ?P0).$  bp-values P (fst p) = snd p
  using values0 unfolding bp-delete-key-def by simp
have values-insert:
   $\forall p \in set$ 
  (bp-bucket-entries-flat (bp-insert-bucket (x, ?b) (bp-buckets ?P0))).
  ((bp-values P)(x := ?b)) (fst p) = snd p
proof
fix p
assume p:
   $p \in set\ (bp\text{-}bucket\text{-}entries\text{-}flat$ 
    (bp-insert-bucket (x, ?b) (bp-buckets ?P0)))
then have p-cases:  $p = (x, ?b) \vee$ 
   $p \in set\ (bp\text{-}bucket\text{-}entries\text{-}flat\ (bp\text{-}buckets\ ?P0))$ 
  using set-bp-bucket-entries-flat-insert-bucket[of (x, ?b) bp-buckets ?P0]
  by auto
from p-cases show ((bp-values P)(x := ?b)) (fst p) = snd p

```

```

proof
  assume  $p = (x, ?b)$ 
  then show  $?thesis$  by simp
next
  assume  $p\text{-flat}$ :  $p \in \text{set } (bp\text{-bucket-entries-flat } (bp\text{-buckets } ?P0))$ 
  then have  $p\text{-old}$ :  $p \in \text{set } (bp\text{-entries } ?P0)$ 
    unfolding bp-entries-def .
  have  $\text{fst } p \neq x$ 
  proof
    assume  $\text{fst } p = x$ 
    then have  $x \in bp\text{-entry-keys } (bp\text{-entries } ?P0)$ 
      using  $p\text{-old}$  unfolding bp-entry-keys-def by auto
    then show False
      using fresh by simp
  qed
  moreover have  $bp\text{-values } P (\text{fst } p) = \text{snd } p$ 
    using values0-P  $p\text{-old}$  by blast
  ultimately show  $?thesis$ 
    by simp
qed
qed
show  $?thesis$ 
  unfolding bp-insert-def Let-def bp-invariant-def
proof (intro conjI)
  show  $0 < bp\text{-block-size}$ 
    ( $?P0 \mid bp\text{-buckets} := bp\text{-insert-bucket } (x, ?b) (bp\text{-buckets } ?P0),$ 
      $bp\text{-values} := (bp\text{-values } P)(x := ?b) \mid \mid$ )
    using block-pos by simp
  show  $bp\text{-distinct-keys}$ 
    ( $?P0 \mid bp\text{-buckets} := bp\text{-insert-bucket } (x, ?b) (bp\text{-buckets } ?P0),$ 
      $bp\text{-values} := (bp\text{-values } P)(x := ?b) \mid \mid$ )
    using distinct-insert
    unfolding bp-distinct-keys-def bp-entries-def by simp
  show  $bp\text{-bucket-sizes-ok}$ 
    ( $?P0 \mid bp\text{-buckets} := bp\text{-insert-bucket } (x, ?b) (bp\text{-buckets } ?P0),$ 
      $bp\text{-values} := (bp\text{-values } P)(x := ?b) \mid \mid$ )
    using  $bp\text{-insert-bucket-sizes-ok}[OF \text{ sizes0 } block\text{-pos}]$ 
    unfolding bp-bucket-sizes-ok-def by simp
  show  $bp\text{-bucket-markers-sorted}$ 
    ( $?P0 \mid bp\text{-buckets} := bp\text{-insert-bucket } (x, ?b) (bp\text{-buckets } ?P0),$ 
      $bp\text{-values} := (bp\text{-values } P)(x := ?b) \mid \mid$ )
    using  $bp\text{-insert-bucket-markers-sorted}[OF \text{ sorted0}]$ 
    unfolding bp-bucket-markers-sorted-def by simp
  show  $bp\text{-bucket-markers-lower-bound}$ 
    ( $?P0 \mid bp\text{-buckets} := bp\text{-insert-bucket } (x, ?b) (bp\text{-buckets } ?P0),$ 
      $bp\text{-values} := (bp\text{-values } P)(x := ?b) \mid \mid$ )
    using  $bp\text{-insert-bucket-markers-lower-bound}[OF \text{ markers0}]$ 
    unfolding bp-bucket-markers-lower-bound-def by simp
  show  $bp\text{-values-consistent}$ 

```



```

    (?P0(|bp-buckets := bp-insert-bucket (x, ?b) (bp-buckets ?P0),
      bp-values := (bp-values P)(x := ?b)))
  using values-insert
  unfolding bp-values-consistent-def bp-entries-def by simp
qed
qed

lemma bp-insert-keys [simp]:
  bp-entry-keys (bp-entries (bp-insert x b P)) =
    insert x (bp-entry-keys (bp-entries P))
proof -
  let ?b = if x ∈ bp-entry-keys (bp-entries P)
    then min (bp-values P x) b else b
  let ?P0 = bp-delete-key x P
  have bp-entry-keys
    (bp-entries
      (?P0(|bp-buckets := bp-insert-bucket (x, ?b) (bp-buckets ?P0),
        bp-values := (bp-values P)(x := ?b))) =
      insert x (bp-entry-keys (bp-entries ?P0))
    by simp
  also have ... = insert x (bp-entry-keys (bp-entries P))
    by simp
  finally show ?thesis
    unfolding bp-insert-def Let-def .
qed

lemma bp-insert-values [simp]:
  bp-values (bp-insert x b P) =
    (bp-values P)
    (x := if x ∈ bp-entry-keys (bp-entries P)
      then min (bp-values P x) b else b)
  unfolding bp-insert-def Let-def by simp

theorem bp-insert-refines-min-update:
  bp-view (bp-insert x b P) = min-update (bp-view P) x b
  unfolding bp-view-def min-update-def by simp

theorem bp-insert-refines-insert-spec:
  insert-spec (bp-view P) x b (bp-view (bp-insert x b P))
  unfolding bp-insert-refines-min-update
  by (rule min-update-insert-spec)

theorem bp-insert-preserves-upper-bound:
  assumes upper: partition-upper-bound (bp-view P) B
  and b-lt: b < B
  shows partition-upper-bound (bp-view (bp-insert x b P)) B
  unfolding bp-insert-refines-min-update
  by (rule min-update-preserves-upper-bound[OF upper b-lt])

```

lemma *bp-local-insert-state-keys*:
assumes *M-pos*: $0 < \text{bp-block-size } P$
shows $\text{bp-entry-keys } (\text{bp-entries } (\text{bp-local-insert-state } x \ b \ P)) =$
 $\text{insert } x \ (\text{bp-entry-keys } (\text{bp-entries } P))$
proof –
let $?b = \text{if } x \in \text{bp-entry-keys } (\text{bp-entries } P)$
 $\text{then } \min (\text{bp-values } P \ x) \ b \text{ else } b$
let $?P0 = \text{bp-delete-key } x \ P$
have $M0: 0 < \text{bp-block-size } ?P0$
using *M-pos* **unfolding** *bp-delete-key-def* **by** *simp*
have *keys-step*:
 bp-entry-keys
 $(\text{bp-entries}$
 $(?P0 \setminus \text{bp-buckets} :=$
 $\text{bp-local-insert-bucket } (\text{bp-block-size } ?P0) \ (x, ?b)$
 $(\text{bp-buckets } ?P0),$
 $\text{bp-values} := (\text{bp-values } P)(x := ?b))) =$
 $\text{insert } x \ (\text{bp-entry-keys } (\text{bp-entries } ?P0))$
using *set-bp-bucket-entries-flat-local-insert-bucket*
 $[OF \ M0, \text{of } (x, ?b) \ \text{bp-buckets } ?P0]$
unfolding *bp-entries-def* *bp-entry-keys-def* **by** *auto*
show *?thesis*
unfolding *bp-local-insert-state-def* *Let-def*
using *keys-step* **by** *simp*
qed

lemma *bp-local-insert-state-values [simp]*:
 $\text{bp-values } (\text{bp-local-insert-state } x \ b \ P) =$
 $(\text{bp-values } P)$
 $(x := \text{if } x \in \text{bp-entry-keys } (\text{bp-entries } P)$
 $\text{then } \min (\text{bp-values } P \ x) \ b \text{ else } b)$
unfolding *bp-local-insert-state-def* *Let-def* **by** *simp*

theorem *bp-local-insert-state-refines-min-update*:
assumes $0 < \text{bp-block-size } P$
shows $\text{bp-view } (\text{bp-local-insert-state } x \ b \ P) = \text{min-update } (\text{bp-view } P) \ x \ b$
using *bp-local-insert-state-keys* $[OF \ \text{assms}, \text{of } x \ b]$
unfolding *bp-view-def* *min-update-def* **by** *simp*

theorem *bp-local-insert-state-refines-insert-spec*:
assumes $0 < \text{bp-block-size } P$
shows $\text{insert-spec } (\text{bp-view } P) \ x \ b$
 $(\text{bp-view } (\text{bp-local-insert-state } x \ b \ P))$
unfolding *bp-local-insert-state-refines-min-update* $[OF \ \text{assms}]$
by *(rule min-update-insert-spec)*

theorem *bp-local-insert-state-preserves-upper-bound*:
assumes *M-pos*: $0 < \text{bp-block-size } P$
and *upper*: $\text{partition-upper-bound } (\text{bp-view } P) \ B$

and $b\text{-lt}$: $b < B$
shows $\text{partition-upper-bound } (\text{bp-view } (\text{bp-local-insert-state } x \ b \ P)) \ B$
unfolding $\text{bp-local-insert-state-refines-min-update}[OF \ M\text{-pos}]$
by $(\text{rule } \text{min-update-preserves-upper-bound}[OF \ \text{upper } b\text{-lt}])$

lemma $\text{bp-lazy-insert-state-keys}$:
assumes $M\text{-pos}$: $0 < \text{bp-block-size } P$
shows $\text{bp-entry-keys } (\text{bp-entries } (\text{bp-lazy-insert-state } x \ b \ P)) =$
 $\text{insert } x \ (\text{bp-entry-keys } (\text{bp-entries } P))$

proof –
let $?b = \text{if } x \in \text{bp-entry-keys } (\text{bp-entries } P)$
 $\text{then } \min (\text{bp-values } P \ x) \ b \ \text{else } b$
let $?P0 = \text{bp-delete-key } x \ P$
have $M0$: $0 < \text{bp-block-size } ?P0$
using $M\text{-pos}$ **unfolding** bp-delete-key-def **by** simp
have keys-step :
 bp-entry-keys
 $(\text{bp-entries}$
 $(?P0 \ (\text{bp-buckets } :=$
 $\text{bp-lazy-insert-bucket } (\text{bp-block-size } ?P0) \ (x, \ ?b)$
 $(\text{bp-buckets } ?P0),$
 $\text{bp-values } := (\text{bp-values } P)(x := ?b))) =$
 $\text{insert } x \ (\text{bp-entry-keys } (\text{bp-entries } ?P0))$
using $\text{set-bp-bucket-entries-flat-lazy-insert-bucket}$
 $[OF \ M0, \ \text{of } (x, \ ?b) \ \text{bp-buckets } ?P0]$
unfolding bp-entries-def bp-entry-keys-def **by** auto
show $?thesis$
unfolding $\text{bp-lazy-insert-state-def}$ Let-def
using keys-step **by** simp
qed

lemma $\text{bp-lazy-insert-state-values } [simp]$:
 $\text{bp-values } (\text{bp-lazy-insert-state } x \ b \ P) =$
 $(\text{bp-values } P)$
 $(x := \text{if } x \in \text{bp-entry-keys } (\text{bp-entries } P)$
 $\text{then } \min (\text{bp-values } P \ x) \ b \ \text{else } b)$
unfolding $\text{bp-lazy-insert-state-def}$ Let-def **by** simp

theorem $\text{bp-lazy-insert-state-refines-min-update}$:
assumes $0 < \text{bp-block-size } P$
shows $\text{bp-view } (\text{bp-lazy-insert-state } x \ b \ P) =$
 $\text{min-update } (\text{bp-view } P) \ x \ b$
using $\text{bp-lazy-insert-state-keys}[OF \ \text{assms}, \ \text{of } x \ b]$
unfolding bp-view-def min-update-def **by** simp

theorem $\text{bp-lazy-insert-state-refines-insert-spec}$:
assumes $0 < \text{bp-block-size } P$
shows $\text{insert-spec } (\text{bp-view } P) \ x \ b$
 $(\text{bp-view } (\text{bp-lazy-insert-state } x \ b \ P))$

unfolding *bp-lazy-insert-state-refines-min-update*[*OF assms*]
by (*rule min-update-insert-spec*)

theorem *bp-lazy-insert-state-preserves-upper-bound*:

assumes *M-pos*: $0 < \text{bp-block-size } P$
and *upper*: *partition-upper-bound* (*bp-view* *P*) *B*
and *b-lt*: $b < B$
shows *partition-upper-bound* (*bp-view* (*bp-lazy-insert-state* *x b P*)) *B*
unfolding *bp-lazy-insert-state-refines-min-update*[*OF M-pos*]
by (*rule min-update-preserves-upper-bound*[*OF upper b-lt*])

lemma *bp-lazy-insert-state-lazy-bucket-sizes-ok*:

assumes *lazy*: *bp-lazy-ordered-invariant* *P*
shows *bp-lazy-bucket-sizes-ok* (*bp-lazy-insert-state* *x b P*)

proof –

let *?old-keys* = *bp-entry-keys* (*bp-entries* *P*)
let *?b* = *if* $x \in ?old-keys$ *then* *min* (*bp-values* *P x*) *b* *else* *b*
let *?P0* = *bp-delete-key* *x P*
have *inv*: *bp-lazy-invariant* *P*
by (*rule bp-lazy-ordered-invariant-lazy-invariant*[*OF lazy*])
have *inv0*: *bp-lazy-invariant* *?P0*
by (*rule bp-delete-key-lazy-invariant*[*OF inv*])
have *M-pos*: $0 < \text{bp-block-size } ?P0$
using *inv0* **unfolding** *bp-lazy-invariant-def* **by** *blast*
have *sizes0*:
 $\forall c \in \text{set } (\text{bp-buckets } ?P0).$
 $\text{length } (\text{bp-bucket-entries } c) \leq 2 * \text{bp-block-size } ?P0$
using *inv0* **unfolding** *bp-lazy-invariant-def*
bp-lazy-bucket-sizes-ok-def **by** *blast*
have *sizes*:
 $\forall c \in \text{set}$
 $(\text{bp-lazy-insert-bucket } (\text{bp-block-size } ?P0) (x, ?b)$
 $(\text{bp-buckets } ?P0)).$
 $\text{length } (\text{bp-bucket-entries } c) \leq 2 * \text{bp-block-size } ?P0$
by (*rule bp-lazy-insert-bucket-sizes-ok*[*OF M-pos sizes0*])
show *?thesis*
unfolding *bp-lazy-insert-state-def* *Let-def* *bp-lazy-bucket-sizes-ok-def*
using *sizes* **by** *simp*

qed

lemma *bp-lazy-insert-state-markers-lower-bound*:

assumes *lazy*: *bp-lazy-ordered-invariant* *P*
shows *bp-bucket-markers-lower-bound* (*bp-lazy-insert-state* *x b P*)

proof –

let *?old-keys* = *bp-entry-keys* (*bp-entries* *P*)
let *?b* = *if* $x \in ?old-keys$ *then* *min* (*bp-values* *P x*) *b* *else* *b*
let *?P0* = *bp-delete-key* *x P*
have *inv*: *bp-lazy-invariant* *P*
by (*rule bp-lazy-ordered-invariant-lazy-invariant*[*OF lazy*])

```

have inv0: bp-lazy-invariant ?P0
  by (rule bp-delete-key-lazy-invariant[OF inv])
have M-pos: 0 < bp-block-size ?P0
  using inv0 unfolding bp-lazy-invariant-def by blast
have lower0:
   $\forall c \in \text{set } (\text{bp-buckets } ?P0). \forall p \in \text{set } (\text{bp-bucket-entries } c).$ 
   $\text{bp-marker } c \leq \text{snd } p$ 
  using inv0 unfolding bp-lazy-invariant-def
  bp-bucket-markers-lower-bound-def by blast
have lower:
   $\forall c \in \text{set}$ 
     $(\text{bp-lazy-insert-bucket } (\text{bp-block-size } ?P0) (x, ?b)$ 
       $(\text{bp-buckets } ?P0)).$ 
     $\forall q \in \text{set } (\text{bp-bucket-entries } c). \text{bp-marker } c \leq \text{snd } q$ 
  by (rule bp-lazy-insert-bucket-markers-lower-bound[OF M-pos lower0])
show ?thesis
  unfolding bp-lazy-insert-state-def Let-def
  bp-bucket-markers-lower-bound-def
  using lower by simp
qed

lemma bp-lazy-insert-state-values-consistent:
  assumes lazy: bp-lazy-ordered-invariant P
  shows bp-values-consistent (bp-lazy-insert-state x b P)
proof –
  let ?old-keys = bp-entry-keys (bp-entries P)
  let ?b = if x ∈ ?old-keys then min (bp-values P x) b else b
  let ?P0 = bp-delete-key x P
  let ?M = bp-block-size ?P0
  let ?bs = bp-buckets ?P0
  let ?new-bs = bp-lazy-insert-bucket ?M (x, ?b) ?bs
  let ?P' = ?P0 (| bp-buckets := ?new-bs,
    bp-values := (bp-values P)(x := ?b) |)
  have inv: bp-lazy-invariant P
    by (rule bp-lazy-ordered-invariant-lazy-invariant[OF lazy])
  have inv0: bp-lazy-invariant ?P0
    by (rule bp-delete-key-lazy-invariant[OF inv])
  have M-pos: 0 < ?M
    using inv0 unfolding bp-lazy-invariant-def by blast
  have values0:
     $\forall p \in \text{set } (\text{bp-entries } ?P0). \text{bp-values } ?P0 (\text{fst } p) = \text{snd } p$ 
    using inv0 unfolding bp-lazy-invariant-def bp-values-consistent-def
    by blast
  have values0-P:
     $\forall p \in \text{set } (\text{bp-entries } ?P0). \text{bp-values } P (\text{fst } p) = \text{snd } p$ 
    using values0 unfolding bp-delete-key-def by simp
  have fresh: x ∉ bp-entry-keys (bp-entries ?P0)
    by simp
  have values-insert:

```

```

   $\forall p \in \text{set } (\text{bp-bucket-entries-flat } ?\text{new-bs}).$ 
   $((\text{bp-values } P)(x := ?b)) (\text{fst } p) = \text{snd } p$ 
proof
  fix  $p$ 
  assume  $p: p \in \text{set } (\text{bp-bucket-entries-flat } ?\text{new-bs})$ 
  then have  $p\text{-cases}$ :
     $p = (x, ?b) \vee p \in \text{set } (\text{bp-bucket-entries-flat } ?\text{bs})$ 
    using  $\text{set-bp-bucket-entries-flat-lazy-insert-bucket}$ 
       $[OF\ M\text{-pos, of } (x, ?b)\ ?\text{bs}]$ 
    by  $\text{auto}$ 
  from  $p\text{-cases}$  show  $((\text{bp-values } P)(x := ?b)) (\text{fst } p) = \text{snd } p$ 
proof
  assume  $p = (x, ?b)$ 
  then show  $?thesis$  by  $\text{simp}$ 
next
  assume  $p\text{-flat}: p \in \text{set } (\text{bp-bucket-entries-flat } ?\text{bs})$ 
  then have  $p\text{-old}: p \in \text{set } (\text{bp-entries } ?P0)$ 
    unfolding  $\text{bp-entries-def}$  .
  have  $\text{fst } p \neq x$ 
  proof
    assume  $\text{fst } p = x$ 
    then have  $x \in \text{bp-entry-keys } (\text{bp-entries } ?P0)$ 
      using  $p\text{-old}$  unfolding  $\text{bp-entry-keys-def}$  by  $\text{auto}$ 
    then show  $\text{False}$ 
      using  $\text{fresh}$  by  $\text{simp}$ 
  qed
  moreover have  $\text{bp-values } P (\text{fst } p) = \text{snd } p$ 
    using  $\text{values0-P } p\text{-old}$  by  $\text{blast}$ 
  ultimately show  $?thesis$ 
    by  $\text{simp}$ 
qed
qed
have  $\text{state}: \text{bp-lazy-insert-state } x\ b\ P = ?P'$ 
  unfolding  $\text{bp-lazy-insert-state-def Let-def}$  by  $\text{simp}$ 
have  $\text{bp-values-consistent } ?P'$ 
  using  $\text{values-insert}$ 
  unfolding  $\text{bp-values-consistent-def bp-entries-def}$  by  $\text{simp}$ 
show  $?thesis$ 
  unfolding  $\text{state}$  by  $(\text{rule } \langle \text{bp-values-consistent } ?P' \rangle)$ 
qed

lemma  $\text{bp-lazy-insert-state-distinct-keys}$ :
  assumes  $\text{lazy}: \text{bp-lazy-ordered-invariant } P$ 
  shows  $\text{bp-distinct-keys } (\text{bp-lazy-insert-state } x\ b\ P)$ 
proof –
  let  $?old\text{-keys} = \text{bp-entry-keys } (\text{bp-entries } P)$ 
  let  $?b = \text{if } x \in ?old\text{-keys} \text{ then } \min (\text{bp-values } P\ x)\ b \text{ else } b$ 
  let  $?P0 = \text{bp-delete-key } x\ P$ 
  let  $?M = \text{bp-block-size } ?P0$ 

```

```

let ?bs = bp-buckets ?P0
let ?new-bs = bp-lazy-insert-bucket ?M (x, ?b) ?bs
let ?P' = ?P0 (|bp-buckets := ?new-bs,
    bp-values := (bp-values P)(x := ?b)|)
have inv: bp-lazy-invariant P
  by (rule bp-lazy-ordered-invariant-lazy-invariant[OF lazy])
have inv0: bp-lazy-invariant ?P0
  by (rule bp-delete-key-lazy-invariant[OF inv])
have M-pos: 0 < ?M
  using inv0 unfolding bp-lazy-invariant-def by blast
have distinct0:
  distinct (map fst (bp-entries ?P0))
  using inv0 unfolding bp-lazy-invariant-def bp-distinct-keys-def
  by blast
have fresh: x ∉ bp-entry-keys (bp-entries ?P0)
  by simp
have distinct-lazy:
  distinct (map fst (bp-bucket-entries-flat ?new-bs))
  by (rule bp-lazy-insert-bucket-distinct-keys[OF M-pos])
  (use distinct0 fresh in ⟨simp-all add: bp-entries-def⟩)
have state: bp-lazy-insert-state x b P = ?P'
  unfolding bp-lazy-insert-state-def Let-def by simp
have bp-distinct-keys ?P'
  using distinct-lazy unfolding bp-distinct-keys-def bp-entries-def
  by simp
show ?thesis
  unfolding state by (rule ⟨bp-distinct-keys ?P'⟩)
qed

```

```

lemma bp-lazy-insert-state-markers-sorted:
  assumes lazy: bp-lazy-ordered-invariant P
  shows bp-bucket-markers-sorted (bp-lazy-insert-state x b P)
proof –
  let ?old-keys = bp-entry-keys (bp-entries P)
  let ?b = if x ∈ ?old-keys then min (bp-values P x) b else b
  let ?P0 = bp-delete-key x P
  let ?M = bp-block-size ?P0
  let ?bs = bp-buckets ?P0
  let ?new-bs = bp-lazy-insert-bucket ?M (x, ?b) ?bs
  have inv: bp-lazy-invariant P
    by (rule bp-lazy-ordered-invariant-lazy-invariant[OF lazy])
  have inv0: bp-lazy-invariant ?P0
    by (rule bp-delete-key-lazy-invariant[OF inv])
  have boundaries0-state: bp-bucket-boundaries-state-ok ?P0
    by (rule bp-delete-key-boundaries-state-ok
      [OF bp-lazy-ordered-invariant-boundaries-state-ok[OF lazy]])
  have M-pos: 0 < ?M
    using inv0 unfolding bp-lazy-invariant-def by blast
  have sorted0:

```

```

    sorted-wrt ( $\lambda c d. \text{bp-marker } c \leq \text{bp-marker } d$ ) ?bs
  using inv0 unfolding bp-lazy-invariant-def
    bp-bucket-markers-sorted-def by blast
have lower0:
   $\forall c \in \text{set } ?bs. \forall p \in \text{set } (\text{bp-bucket-entries } c).$ 
   $\text{bp-marker } c \leq \text{snd } p$ 
  using inv0 unfolding bp-lazy-invariant-def
    bp-bucket-markers-lower-bound-def by blast
have boundaries0: bp-bucket-boundaries-ok ?bs
  using boundaries0-state unfolding bp-bucket-boundaries-state-ok-def .
have sorted-new:
  sorted-wrt ( $\lambda c d. \text{bp-marker } c \leq \text{bp-marker } d$ ) ?new-bs
  by (rule bp-lazy-insert-bucket-markers-sorted
    [OF M-pos sorted0 lower0 boundaries0])
show ?thesis
  unfolding bp-lazy-insert-state-def Let-def
    bp-bucket-markers-sorted-def
  using sorted-new by simp
qed

```

```

lemma bp-lazy-insert-state-boundaries-state-ok:
  assumes lazy: bp-lazy-ordered-invariant P
  shows bp-bucket-boundaries-state-ok (bp-lazy-insert-state x b P)
proof -
  let ?old-keys = bp-entry-keys (bp-entries P)
  let ?b = if  $x \in ?old-keys$  then  $\min (\text{bp-values } P \ x)$  b else b
  let ?P0 = bp-delete-key x P
  let ?M = bp-block-size ?P0
  let ?bs = bp-buckets ?P0
  let ?new-bs = bp-lazy-insert-bucket ?M (x, ?b) ?bs
  have inv: bp-lazy-invariant P
    by (rule bp-lazy-ordered-invariant-lazy-invariant[OF lazy])
  have inv0: bp-lazy-invariant ?P0
    by (rule bp-delete-key-lazy-invariant[OF inv])
  have boundaries0-state: bp-bucket-boundaries-state-ok ?P0
    by (rule bp-delete-key-boundaries-state-ok
      [OF bp-lazy-ordered-invariant-boundaries-state-ok[OF lazy]])
  have M-pos:  $0 < ?M$ 
    using inv0 unfolding bp-lazy-invariant-def by blast
  have sorted0:
    sorted-wrt ( $\lambda c d. \text{bp-marker } c \leq \text{bp-marker } d$ ) ?bs
    using inv0 unfolding bp-lazy-invariant-def
      bp-bucket-markers-sorted-def by blast
  have lower0:
     $\forall c \in \text{set } ?bs. \forall p \in \text{set } (\text{bp-bucket-entries } c).$ 
     $\text{bp-marker } c \leq \text{snd } p$ 
    using inv0 unfolding bp-lazy-invariant-def
      bp-bucket-markers-lower-bound-def by blast
  have boundaries0: bp-bucket-boundaries-ok ?bs

```



```

    using boundaries0-state unfolding bp-bucket-boundaries-state-ok-def .
  have boundaries-new: bp-bucket-boundaries-ok ?new-bs
  by (rule bp-lazy-insert-bucket-boundaries-ok
      [OF M-pos sorted0 lower0 boundaries0])
  show ?thesis
    unfolding bp-lazy-insert-state-def Let-def
      bp-bucket-boundaries-state-ok-def
    using boundaries-new by simp
qed

```

```

lemma bp-lazy-insert-state-lazy-invariant:
  assumes lazy: bp-lazy-ordered-invariant P
  shows bp-lazy-invariant (bp-lazy-insert-state x b P)
proof -
  have inv: bp-lazy-invariant P
  by (rule bp-lazy-ordered-invariant-lazy-invariant[OF lazy])
  have M-pos: 0 < bp-block-size P
  using inv unfolding bp-lazy-invariant-def by blast
  have block:
    bp-block-size (bp-lazy-insert-state x b P) = bp-block-size P
  unfolding bp-lazy-insert-state-def bp-delete-key-def
  by (simp add: Let-def split: if-splits)
  show ?thesis
    unfolding bp-lazy-invariant-def
  proof (intro conjI)
    show 0 < bp-block-size (bp-lazy-insert-state x b P)
    using M-pos block by simp
    show bp-distinct-keys (bp-lazy-insert-state x b P)
    by (rule bp-lazy-insert-state-distinct-keys[OF lazy])
    show bp-lazy-bucket-sizes-ok (bp-lazy-insert-state x b P)
    by (rule bp-lazy-insert-state-lazy-bucket-sizes-ok[OF lazy])
    show bp-bucket-markers-sorted (bp-lazy-insert-state x b P)
    by (rule bp-lazy-insert-state-markers-sorted[OF lazy])
    show bp-bucket-markers-lower-bound (bp-lazy-insert-state x b P)
    by (rule bp-lazy-insert-state-markers-lower-bound[OF lazy])
    show bp-values-consistent (bp-lazy-insert-state x b P)
    by (rule bp-lazy-insert-state-values-consistent[OF lazy])
  qed
qed

```

```

lemma bp-lazy-insert-state-lazy-ordered-invariant:
  assumes lazy: bp-lazy-ordered-invariant P
  shows bp-lazy-ordered-invariant (bp-lazy-insert-state x b P)
  unfolding bp-lazy-ordered-invariant-def
  using bp-lazy-insert-state-lazy-invariant[OF lazy]
    bp-lazy-insert-state-boundaries-state-ok[OF lazy]
  by blast

```

```

lemma length-bp-lazy-insert-state-entries:

```

assumes $M\text{-pos}$: $0 < \text{bp-block-size } P$
shows $\text{length } (\text{bp-entries } (\text{bp-lazy-insert-state } x \ b \ P)) =$
 $\text{Suc } (\text{length } (\text{bp-entries } (\text{bp-delete-key } x \ P)))$
proof –
let $?old\text{-keys} = \text{bp-entry-keys } (\text{bp-entries } P)$
let $?b = \text{if } x \in ?old\text{-keys} \text{ then } \min (\text{bp-values } P \ x) \ b \text{ else } b$
let $?P0 = \text{bp-delete-key } x \ P$
let $?M = \text{bp-block-size } ?P0$
let $?new\text{-bs} = \text{bp-lazy-insert-bucket } ?M \ (x, ?b) \ (\text{bp-buckets } ?P0)$
let $?P' = ?P0 \langle \text{bp-buckets} := ?new\text{-bs},$
 $\text{bp-values} := (\text{bp-values } P)(x := ?b) \rangle$
have $M0$: $0 < ?M$
using $M\text{-pos}$ **unfolding** bp-delete-key-def **by** simp
have len :
 $\text{length } (\text{bp-bucket-entries-flat } ?new\text{-bs}) =$
 $\text{Suc } (\text{length } (\text{bp-bucket-entries-flat } (\text{bp-buckets } ?P0)))$
by $(\text{rule } \text{length-bp-bucket-entries-flat-lazy-insert-bucket} [OF \ M0])$
have state : $\text{bp-lazy-insert-state } x \ b \ P = ?P'$
unfolding $\text{bp-lazy-insert-state-def}$ Let-def **by** simp
have $\text{length } (\text{bp-entries } ?P') =$
 $\text{Suc } (\text{length } (\text{bp-entries } ?P0))$
using len **unfolding** bp-entries-def **by** simp
show $?thesis$
unfolding state **by** $(\text{rule } \langle \text{length } (\text{bp-entries } ?P') =$
 $\text{Suc } (\text{length } (\text{bp-entries } ?P0)) \rangle)$
qed

lemma $\text{length-bp-lazy-insert-state-entries-ge}$:
assumes lazy : $\text{bp-lazy-ordered-invariant } P$
shows $\text{length } (\text{bp-entries } P) \leq$
 $\text{length } (\text{bp-entries } (\text{bp-lazy-insert-state } x \ b \ P))$
proof –
have inv : $\text{bp-lazy-invariant } P$
by $(\text{rule } \text{bp-lazy-ordered-invariant-lazy-invariant} [OF \ \text{lazy}])$
have $M\text{-pos}$: $0 < \text{bp-block-size } P$
using inv **unfolding** $\text{bp-lazy-invariant-def}$ **by** blast
have distinct : $\text{bp-distinct-keys } P$
using inv **unfolding** $\text{bp-lazy-invariant-def}$ **by** blast
have delete-ge :
 $\text{length } (\text{bp-entries } P) \leq$
 $\text{Suc } (\text{length } (\text{bp-entries } (\text{bp-delete-key } x \ P)))$
by $(\text{rule } \text{length-bp-entries-delete-key-ge-pred} [OF \ \text{distinct}])$
have insert-len :
 $\text{length } (\text{bp-entries } (\text{bp-lazy-insert-state } x \ b \ P)) =$
 $\text{Suc } (\text{length } (\text{bp-entries } (\text{bp-delete-key } x \ P)))$
by $(\text{rule } \text{length-bp-lazy-insert-state-entries} [OF \ M\text{-pos}])$
show $?thesis$
using delete-ge **unfolding** insert-len .
qed

lemma *length-bp-lazy-insert-state-buckets-le*:
assumes *lazy*: *bp-lazy-ordered-invariant* *P*
shows $\text{length } (\text{bp-buckets } (\text{bp-lazy-insert-state } x \ b \ P)) \leq$
 $\text{length } (\text{bp-buckets } P) + 2$
proof –
let *?old-keys* = *bp-entry-keys* (*bp-entries* *P*)
let *?b* = if *x* ∈ *?old-keys* then *min* (*bp-values* *P* *x*) *b* else *b*
let *?P0* = *bp-delete-key* *x* *P*
let *?M* = *bp-block-size* *?P0*
let *?bs* = *bp-buckets* *?P0*
let *?new-bs* = *bp-lazy-insert-bucket* *?M* (*x*, *?b*) *?bs*
let *?P'* = *?P0* (*bp-buckets* := *?new-bs*,
 $\text{bp-values} := (\text{bp-values } P)(x := ?b)$)
have *inv*: *bp-lazy-invariant* *P*
by (rule *bp-lazy-ordered-invariant-lazy-invariant*[*OF lazy*])
have *inv0*: *bp-lazy-invariant* *?P0*
by (rule *bp-delete-key-lazy-invariant*[*OF inv*])
have *M-pos*: $0 < ?M$
using *inv0* **unfolding** *bp-lazy-invariant-def* **by** *blast*
have *sizes0*:
 $\forall b \in \text{set } ?bs. \text{length } (\text{bp-bucket-entries } b) \leq 2 * ?M$
using *inv0* **unfolding** *bp-lazy-invariant-def*
 $\text{bp-lazy-bucket-sizes-ok-def}$ **by** *blast*
have *len-new*:
 $\text{length } ?new-bs \leq \text{length } ?bs + 2$
by (rule *length-bp-lazy-insert-bucket-le*[*OF M-pos sizes0*])
have *state*: *bp-lazy-insert-state* *x* *b* *P* = *?P'*
unfolding *bp-lazy-insert-state-def* *Let-def* **by** *simp*
have $\text{length } (\text{bp-buckets } ?P') \leq \text{length } (\text{bp-buckets } P) + 2$
using *len-new* **unfolding** *bp-delete-key-def* **by** *simp*
show *?thesis*
unfolding *state* **by** (rule $\langle \text{length } (\text{bp-buckets } ?P') \leq$
 $\text{length } (\text{bp-buckets } P) + 2 \rangle$)
qed

lemma *bp-local-insert-state-invariant*:
assumes *ord*: *bp-ordered-invariant* *P*
shows *bp-invariant* (*bp-local-insert-state* *x* *b* *P*)
proof –
let *?old-keys* = *bp-entry-keys* (*bp-entries* *P*)
let *?b* = if *x* ∈ *?old-keys* then *min* (*bp-values* *P* *x*) *b* else *b*
let *?P0* = *bp-delete-key* *x* *P*
let *?M* = *bp-block-size* *?P0*
let *?bs* = *bp-buckets* *?P0*
let *?new-bs* = *bp-local-insert-bucket* *?M* (*x*, *?b*) *?bs*
let *?P'* = *?P0* (*bp-buckets* := *?new-bs*,
 $\text{bp-values} := (\text{bp-values } P)(x := ?b)$)
have *inv*: *bp-invariant* *P*

```

  by (rule bp-ordered-invariant-invariant[OF ord])
have inv0: bp-invariant ?P0
  by (rule bp-delete-key-invariant[OF inv])
have boundaries0-state: bp-bucket-boundaries-state-ok ?P0
  by (rule bp-delete-key-boundaries-state-ok
      [OF bp-ordered-invariant-boundaries-state-ok[OF ord]])
have M-pos: 0 < ?M
  using inv0 unfolding bp-invariant-def by blast
have distinct0: distinct (map fst (bp-entries ?P0))
  using inv0 unfolding bp-invariant-def bp-distinct-keys-def by blast
have sizes0:
   $\forall c \in \text{set } ?bs. \text{length } (bp\text{-bucket-entries } c) \leq ?M$ 
  using inv0 unfolding bp-invariant-def bp-bucket-sizes-ok-def by blast
have sorted0:
  sorted-wrt ( $\lambda c d. bp\text{-marker } c \leq bp\text{-marker } d$ ) ?bs
  using inv0 unfolding bp-invariant-def bp-bucket-markers-sorted-def by blast
have markers0:
   $\forall c \in \text{set } ?bs. \forall p \in \text{set } (bp\text{-bucket-entries } c). \\ bp\text{-marker } c \leq \text{snd } p$ 
  using inv0 unfolding bp-invariant-def bp-bucket-markers-lower-bound-def
  by blast
have boundaries0: bp-bucket-boundaries-ok ?bs
  using boundaries0-state unfolding bp-bucket-boundaries-state-ok-def .
have shape:
  ( $\forall c \in \text{set } ?new\text{-bs}. \text{length } (bp\text{-bucket-entries } c) \leq ?M$ )  $\wedge$ 
  sorted-wrt ( $\lambda c d. bp\text{-marker } c \leq bp\text{-marker } d$ ) ?new-bs  $\wedge$ 
  ( $\forall c \in \text{set } ?new\text{-bs}. \forall q \in \text{set } (bp\text{-bucket-entries } c). \\ bp\text{-marker } c \leq \text{snd } q$ )  $\wedge$ 
  bp-bucket-boundaries-ok ?new-bs
  by (rule bp-local-insert-bucket-preserves-bucket-shape
      [OF M-pos sizes0 sorted0 markers0 boundaries0])
have fresh:  $x \notin bp\text{-entry-keys } (bp\text{-entries } ?P0)$ 
  by simp
have distinct-local:
  distinct (map fst (bp-bucket-entries-flat ?new-bs))
  by (rule bp-local-insert-bucket-distinct-keys[OF M-pos])
  (use distinct0 fresh in <simp-all add: bp-entries-def>)
have values0:
   $\forall p \in \text{set } (bp\text{-entries } ?P0). bp\text{-values } ?P0 (\text{fst } p) = \text{snd } p$ 
  using inv0 unfolding bp-invariant-def bp-values-consistent-def by blast
have values0-P:
   $\forall p \in \text{set } (bp\text{-entries } ?P0). bp\text{-values } P (\text{fst } p) = \text{snd } p$ 
  using values0 unfolding bp-delete-key-def by simp
have values-insert:
   $\forall p \in \text{set } (bp\text{-bucket-entries-flat } ?new\text{-bs}). \\ ((bp\text{-values } P)(x := ?b)) (\text{fst } p) = \text{snd } p$ 
proof
  fix p
  assume p:  $p \in \text{set } (bp\text{-bucket-entries-flat } ?new\text{-bs})$ 

```

```

then have p-cases:
   $p = (x, ?b) \vee p \in \text{set } (bp\text{-bucket-entries-flat } ?bs)$ 
  using set-bp-bucket-entries-flat-local-insert-bucket
  [OF M-pos, of  $(x, ?b) ?bs$ ]
  by auto
from p-cases show  $((bp\text{-values } P)(x := ?b)) (fst\ p) = snd\ p$ 
proof
  assume  $p = (x, ?b)$ 
  then show ?thesis by simp
next
  assume p-old:  $p \in \text{set } (bp\text{-bucket-entries-flat } ?bs)$ 
  then have p-old-entry:  $p \in \text{set } (bp\text{-entries } ?P0)$ 
    unfolding bp-entries-def .
  have  $fst\ p \neq x$ 
  proof
    assume  $fst\ p = x$ 
    then have  $x \in bp\text{-entry-keys } (bp\text{-entries } ?P0)$ 
      using p-old-entry unfolding bp-entry-keys-def by auto
    then show False
      using fresh by simp
  qed
  moreover have  $bp\text{-values } P (fst\ p) = snd\ p$ 
    using values0-P p-old-entry by blast
  ultimately show ?thesis
    by simp
qed
qed
show ?thesis
  unfolding bp-local-insert-state-def Let-def bp-invariant-def
proof (intro conjI)
  show  $0 < bp\text{-block-size } ?P'$ 
    using M-pos by simp
  show bp-distinct-keys  $?P'$ 
    using distinct-local unfolding bp-distinct-keys-def bp-entries-def
    by simp
  show bp-bucket-sizes-ok  $?P'$ 
    using shape unfolding bp-bucket-sizes-ok-def by simp
  show bp-bucket-markers-sorted  $?P'$ 
    using shape unfolding bp-bucket-markers-sorted-def by simp
  show bp-bucket-markers-lower-bound  $?P'$ 
    using shape unfolding bp-bucket-markers-lower-bound-def by simp
  show bp-values-consistent  $?P'$ 
    using values-insert unfolding bp-values-consistent-def bp-entries-def
    by simp
qed
qed
lemma bp-local-insert-state-boundaries-state-ok:
  assumes ord: bp-ordered-invariant P

```

shows *bp-bucket-boundaries-state-ok* (*bp-local-insert-state* *x b P*)
proof –
let *?old-keys* = *bp-entry-keys* (*bp-entries* *P*)
let *?b* = *if* *x* ∈ *?old-keys* *then* *min* (*bp-values* *P x*) *b* *else* *b*
let *?P0* = *bp-delete-key* *x P*
let *?M* = *bp-block-size* *?P0*
let *?bs* = *bp-buckets* *?P0*
let *?new-bs* = *bp-local-insert-bucket* *?M (x, ?b) ?bs*
have *inv*: *bp-invariant* *P*
by (*rule* *bp-ordered-invariant-invariant*[*OF ord*])
have *inv0*: *bp-invariant* *?P0*
by (*rule* *bp-delete-key-invariant*[*OF inv*])
have *boundaries0-state*: *bp-bucket-boundaries-state-ok* *?P0*
by (*rule* *bp-delete-key-boundaries-state-ok*
[*OF bp-ordered-invariant-boundaries-state-ok*[*OF ord*]])
have *M-pos*: *0* < *?M*
using *inv0* **unfolding** *bp-invariant-def* **by** *blast*
have *sizes0*:
 $\forall c \in \text{set } ?bs. \text{length } (\text{bp-bucket-entries } c) \leq ?M$
using *inv0* **unfolding** *bp-invariant-def* *bp-bucket-sizes-ok-def* **by** *blast*
have *sorted0*:
sorted-wrt ($\lambda c d. \text{bp-marker } c \leq \text{bp-marker } d$) *?bs*
using *inv0* **unfolding** *bp-invariant-def* *bp-bucket-markers-sorted-def* **by** *blast*
have *markers0*:
 $\forall c \in \text{set } ?bs. \forall p \in \text{set } (\text{bp-bucket-entries } c).$
 $\text{bp-marker } c \leq \text{snd } p$
using *inv0* **unfolding** *bp-invariant-def* *bp-bucket-markers-lower-bound-def*
by *blast*
have *boundaries0*: *bp-bucket-boundaries-ok* *?bs*
using *boundaries0-state* **unfolding** *bp-bucket-boundaries-state-ok-def* .
have *shape*:
 $(\forall c \in \text{set } ?new-bs. \text{length } (\text{bp-bucket-entries } c) \leq ?M) \wedge$
sorted-wrt ($\lambda c d. \text{bp-marker } c \leq \text{bp-marker } d$) *?new-bs* $\wedge$
 $(\forall c \in \text{set } ?new-bs. \forall q \in \text{set } (\text{bp-bucket-entries } c).$
 $\text{bp-marker } c \leq \text{snd } q) \wedge$
bp-bucket-boundaries-ok *?new-bs*
by (*rule* *bp-local-insert-bucket-preserves-bucket-shape*
[*OF M-pos sizes0 sorted0 markers0 boundaries0*])
show *?thesis*
unfolding *bp-local-insert-state-def* *Let-def*
bp-bucket-boundaries-state-ok-def
using *shape* **by** *simp*
qed

lemma *bp-local-insert-state-ordered-invariant*:
assumes *ord*: *bp-ordered-invariant* *P*
shows *bp-ordered-invariant* (*bp-local-insert-state* *x b P*)
unfolding *bp-ordered-invariant-def*
using *bp-local-insert-state-invariant*[*OF ord*]

```

    bp-local-insert-state-boundaries-state-ok[OF ord]
  by blast

lemma bp-batch-prepend-invariant:
  assumes bp-invariant P
  shows bp-invariant (bp-batch-prepend xs P)
  using assms
proof (induction xs arbitrary: P)
  case Nil
  then show ?case
    unfolding bp-batch-prepend-def by simp
next
  case (Cons xb xs)
  obtain x b where xb: xb = (x, b)
  by force
  have step: bp-invariant (bp-insert x b P)
  by (rule bp-insert-invariant[OF Cons.prems])
  have unfold-step:
    bp-batch-prepend (xb # xs) P =
    bp-batch-prepend xs (bp-insert x b P)
  unfolding bp-batch-prepend-def xb by simp
  show ?case
    unfolding unfold-step by (rule Cons.IH[OF step])
qed

theorem bp-batch-prepend-refines-batch-min-update:
  assumes bp-invariant P
  shows bp-view (bp-batch-prepend xs P) =
    batch-min-update (bp-view P) xs
  using assms
proof (induction xs arbitrary: P)
  case Nil
  then show ?case
    unfolding bp-batch-prepend-def batch-min-update-def by simp
next
  case (Cons xb xs)
  obtain x b where xb: xb = (x, b)
  by force
  have step-inv: bp-invariant (bp-insert x b P)
  by (rule bp-insert-invariant[OF Cons.prems])
  have step-view: bp-view (bp-insert x b P) = min-update (bp-view P) x b
  by (rule bp-insert-refines-min-update)
  have unfold-state:
    bp-batch-prepend (xb # xs) P =
    bp-batch-prepend xs (bp-insert x b P)
  unfolding bp-batch-prepend-def xb by simp
  have unfold-view:
    batch-min-update (bp-view P) (xb # xs) =
    batch-min-update (min-update (bp-view P) x b) xs

```

unfolding *batch-min-update-def* *xb* **by** *simp*
show *?case*
unfolding *unfold-state* *unfold-view*
using *Cons.IH*[*OF step-inv*] *step-view* **by** *simp*
qed

theorem *bp-batch-prepend-preserves-upper-bound*:
assumes *inv*: *bp-invariant* *P*
and *upper*: *partition-upper-bound* (*bp-view* *P*) *B*
and *values-lt*: $\bigwedge x b. (x, b) \in \text{set } xs \implies b < B$
shows *partition-upper-bound* (*bp-view* (*bp-batch-prepend* *xs* *P*)) *B*
unfolding *bp-batch-prepend-refines-batch-min-update*[*OF inv*]
by (*rule* *batch-min-update-preserves-upper-bound*[*OF upper values-lt*])

definition *bp-rebucketed-insert* ::
 $'k \Rightarrow \text{real} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow 'k \text{ bucketed-partition}$ **where**
 $\text{bp-rebucketed-insert } x \ b \ P = \text{bp-rebucket } (\text{bp-insert } x \ b \ P)$

definition *bp-rebucketed-batch-prepend* ::
 $('k \times \text{real}) \text{ list} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow$
 $'k \text{ bucketed-partition}$ **where**
 $\text{bp-rebucketed-batch-prepend } xs \ P = \text{bp-rebucket } (\text{bp-batch-prepend } xs \ P)$

lemma *bp-rebucketed-insert-invariant*:
assumes *inv*: *bp-invariant* *P*
shows *bp-invariant* (*bp-rebucketed-insert* *x* *b* *P*)
unfolding *bp-rebucketed-insert-def*
by (*rule* *bp-rebucket-invariant*[*OF bp-insert-invariant*[*OF inv*]])

lemma *bp-rebucketed-insert-ordered-invariant*:
assumes *inv*: *bp-invariant* *P*
shows *bp-ordered-invariant* (*bp-rebucketed-insert* *x* *b* *P*)
unfolding *bp-rebucketed-insert-def*
by (*rule* *bp-rebucket-ordered-invariant*[*OF bp-insert-invariant*[*OF inv*]])

theorem *bp-rebucketed-insert-refines-min-update*:
assumes *inv*: *bp-invariant* *P*
shows *bp-view* (*bp-rebucketed-insert* *x* *b* *P*) = *min-update* (*bp-view* *P*) *x* *b*
proof –
have *step-inv*: *bp-invariant* (*bp-insert* *x* *b* *P*)
by (*rule* *bp-insert-invariant*[*OF inv*])
then have *block*: $0 < \text{bp-block-size } (\text{bp-insert } x \ b \ P)$
unfolding *bp-invariant-def* **by** *blast*
have *rebucket-view*:
 $\text{bp-view } (\text{bp-rebucket } (\text{bp-insert } x \ b \ P)) = \text{bp-view } (\text{bp-insert } x \ b \ P)$
by (*rule* *bp-rebucket-view*[*OF block*])
show *?thesis*
unfolding *bp-rebucketed-insert-def*
using *rebucket-view bp-insert-refines-min-update* **by** *simp*

qed

theorem *bp-rebucketed-insert-refines-insert-spec:*

assumes *inv*: *bp-invariant P*

shows *insert-spec (bp-view P) x b (bp-view (bp-rebucketed-insert x b P))*

unfolding *bp-rebucketed-insert-refines-min-update[OF inv]*

by (*rule min-update-insert-spec*)

theorem *bp-rebucketed-insert-preserves-upper-bound:*

assumes *inv*: *bp-invariant P*

and *upper*: *partition-upper-bound (bp-view P) B*

and *b-lt*: *b < B*

shows *partition-upper-bound (bp-view (bp-rebucketed-insert x b P)) B*

unfolding *bp-rebucketed-insert-refines-min-update[OF inv]*

by (*rule min-update-preserves-upper-bound[OF upper b-lt]*)

lemma *bp-rebucketed-batch-prepend-invariant:*

assumes *inv*: *bp-invariant P*

shows *bp-invariant (bp-rebucketed-batch-prepend xs P)*

unfolding *bp-rebucketed-batch-prepend-def*

by (*rule bp-rebucket-invariant[OF bp-batch-prepend-invariant[OF inv]]*)

lemma *bp-rebucketed-batch-prepend-ordered-invariant:*

assumes *inv*: *bp-invariant P*

shows *bp-ordered-invariant (bp-rebucketed-batch-prepend xs P)*

unfolding *bp-rebucketed-batch-prepend-def*

by (*rule bp-rebucket-ordered-invariant[OF bp-batch-prepend-invariant[OF inv]]*)

theorem *bp-rebucketed-batch-prepend-refines-batch-min-update:*

assumes *inv*: *bp-invariant P*

shows *bp-view (bp-rebucketed-batch-prepend xs P) =*

batch-min-update (bp-view P) xs

proof –

have *step-inv*: *bp-invariant (bp-batch-prepend xs P)*

by (*rule bp-batch-prepend-invariant[OF inv]*)

then have *block*: *0 < bp-block-size (bp-batch-prepend xs P)*

unfolding *bp-invariant-def* **by** *blast*

have *rebucket-view*:

bp-view (bp-rebucket (bp-batch-prepend xs P)) =

bp-view (bp-batch-prepend xs P)

by (*rule bp-rebucket-view[OF block]*)

show *?thesis*

unfolding *bp-rebucketed-batch-prepend-def*

using *rebucket-view bp-batch-prepend-refines-batch-min-update[OF inv]*

by *simp*

qed

theorem *bp-rebucketed-batch-prepend-preserves-upper-bound:*

assumes *inv*: *bp-invariant P*

and *upper*: *partition-upper-bound* (*bp-view* *P*) *B*
and *values-lt*: $\bigwedge x\ b. (x, b) \in \text{set } xs \implies b < B$
shows *partition-upper-bound*
(bp-view (bp-rebucketed-batch-prepend xs P)) B
unfolding *bp-rebucketed-batch-prepend-refines-batch-min-update*[*OF inv*]
by (*rule batch-min-update-preserves-upper-bound*[*OF upper values-lt*])

lemma *bp-min-value-le-entry*:
assumes $(x, b) \in \text{set } xs$
shows *bp-min-value* *B xs* $\leq b$
using *assms* **by** (*induction xs*) *auto*

lemma *bp-entries-Cons*:
assumes *bp-buckets* *P* = *b # bs*
shows *bp-entries* *P* = *bp-bucket-entries* *b @ bp-bucket-entries-flat bs*
using *assms* **unfolding** *bp-entries-def bp-bucket-entries-flat-def* **by** *simp*

lemma *bp-tail-entry-ge-head-marker*:
assumes *sorted*: *sorted-wrt* $(\lambda b\ c. \text{bp-marker } b \leq \text{bp-marker } c)$ $(c \# bs)$
and *lower*: $\bigwedge d\ p. \llbracket d \in \text{set } (c \# bs); p \in \text{set } (\text{bp-bucket-entries } d) \rrbracket$
 $\implies \text{bp-marker } d \leq \text{snd } p$
and *p*: *p* $\in \text{set } (\text{bp-bucket-entries-flat } (c \# bs))$
shows *bp-marker* *c* $\leq \text{snd } p$
proof –
obtain *d* **where** *d*: *d* $\in \text{set } (c \# bs)$
and *p-d*: *p* $\in \text{set } (\text{bp-bucket-entries } d)$
using *p* **unfolding** *bp-bucket-entries-flat-def* **by** *auto*
have *c-le-d*: *bp-marker* *c* $\leq \text{bp-marker } d$
using *sorted d* **by** (*cases d = c*) *auto*
have *d-le-p*: *bp-marker* *d* $\leq \text{snd } p$
by (*rule lower*[*OF d p-d*])
show *?thesis*
using *c-le-d d-le-p* **by** *linarith*
qed

lemma *bp-first-bucket-pull-invariant*:
assumes *inv*: *bp-invariant* *P*
and *pull*: *bp-first-bucket-pull* *M B P* = (*S*, *beta*, *P'*)
shows *bp-invariant* *P'*
proof (*cases bp-buckets P*)
case *Nil*
then **have** *P'* = *P*
using *pull* **unfolding** *bp-first-bucket-pull-def* **by** *simp*
then **show** *?thesis*
using *inv* **by** *simp*
next
case (*Cons b rest*)
then **have** *buckets-outer*: *bp-buckets* *P* = *b # rest* .
then **show** *?thesis*

```

proof (cases rest)
  case Nil
  then have  $P' = P$ 
    using buckets-outer pull unfolding bp-first-bucket-pull-def by simp
  then show ?thesis
    using inv by simp
next
  case (Cons c bs)
  let ?S = bp-bucket-keys b
  have pull-eval:
    bp-first-bucket-pull M B P = (?S, bp-marker c, bp-delete-keys ?S P)
    using buckets-outer Cons
    unfolding bp-first-bucket-pull-def by (simp add: Let-def)
  have tuple-eq: (?S, bp-marker c, bp-delete-keys ?S P) = (S, beta, P')
    using pull-eval pull by simp
  have S-eq: S = ?S
    using tuple-eq by simp
  have P'-eq-raw: bp-delete-keys ?S P = P'
    using tuple-eq by (metis prod.inject)
  then have  $P' = \text{bp-delete-keys } ?S P$ 
    by simp
  then have  $P' = \text{bp-delete-keys } S P$ 
    using S-eq by simp
  then show ?thesis
    by (simp add: bp-delete-keys-invariant[OF inv])
qed
qed

theorem bp-first-bucket-pull-refines-pull-separates:
  assumes inv: bp-invariant P
  and buckets: bp-buckets P = b # c # bs
  and len-b: length (bp-bucket-entries b) ≤ M
  and below: bp-bucket-below-bound b (bp-marker c)
  and tail-nonempty: bp-bucket-entries-flat (c # bs) ≠ []
  and pull: bp-first-bucket-pull M B P = (S, beta, P')
  shows pull-separates (bp-view P) M B S beta (bp-view P')
proof –
  let ?tail = bp-bucket-entries-flat (c # bs)
  have S-def: S = bp-bucket-keys b
  and beta-def: beta = bp-marker c
  and P'-def: P' = bp-delete-keys S P
  using pull buckets unfolding bp-first-bucket-pull-def by (auto simp: Let-def)
  have entries-P: bp-entries P = bp-bucket-entries b @ ?tail
    using buckets by (simp add: bp-entries-Cons)
  have distinct-P: distinct (map fst (bp-entries P))
    using inv unfolding bp-invariant-def bp-distinct-keys-def by blast
  have values-P:
     $\bigwedge p. p \in \text{set } (\text{bp-entries } P) \implies \text{bp-values } P (\text{fst } p) = \text{snd } p$ 
    using inv unfolding bp-invariant-def bp-values-consistent-def by blast

```

```

have sorted-tail:
  sorted-wrt ( $\lambda x y. \text{bp-marker } x \leq \text{bp-marker } y$ ) ( $c \# bs$ )
using inv buckets unfolding bp-invariant-def bp-bucket-markers-sorted-def by
simp
have lower-tail:
   $\bigwedge d p. \llbracket d \in \text{set } (c \# bs); p \in \text{set } (\text{bp-bucket-entries } d) \rrbracket$ 
   $\implies \text{bp-marker } d \leq \text{snd } p$ 
using inv buckets unfolding bp-invariant-def
  bp-bucket-markers-lower-bound-def by auto
have S-subset:  $S \subseteq \text{keys-of } (\text{bp-view } P)$ 
unfolding S-def bp-view-def bp-bucket-keys-def bp-entry-keys-def entries-P
by auto
have card-S:  $\text{card } S \leq M$ 
proof -
  have  $\text{card } S \leq \text{length } (\text{bp-bucket-entries } b)$ 
  proof -
    have  $\text{card } S = \text{card } (\text{fst } ' \text{set } (\text{bp-bucket-entries } b))$ 
    unfolding S-def bp-bucket-keys-def bp-entry-keys-def by simp
    also have  $\dots \leq \text{card } (\text{set } (\text{bp-bucket-entries } b))$ 
    by (rule card-image-le) simp
    also have  $\dots \leq \text{length } (\text{bp-bucket-entries } b)$ 
    by (rule card-length)
    finally show ?thesis .
  qed
then show ?thesis
  using len-b by linarith
qed
have P'-keys:  $\text{keys-of } (\text{bp-view } P') = \text{keys-of } (\text{bp-view } P) - S$ 
unfolding P'-def bp-delete-keys-view by simp
have P'-values:  $\text{value-of } (\text{bp-view } P') = \text{value-of } (\text{bp-view } P)$ 
unfolding P'-def bp-delete-keys-view by simp
have pulled-lt-beta:
   $\bigwedge u. u \in S \implies \text{value-of } (\text{bp-view } P) u < \text{beta}$ 
proof -
  fix u
  assume u:  $u \in S$ 
  then obtain p where p-b:  $p \in \text{set } (\text{bp-bucket-entries } b)$ 
  and fst-p:  $\text{fst } p = u$ 
  unfolding S-def bp-bucket-keys-def bp-entry-keys-def by auto
  have  $\text{snd } p < \text{beta}$ 
  using below p-b unfolding bp-bucket-below-bound-def beta-def by blast
  moreover have bp-values P u =  $\text{snd } p$ 
  using values-P[of p] p-b fst-p entries-P by simp
  ultimately show  $\text{value-of } (\text{bp-view } P) u < \text{beta}$ 
  unfolding bp-view-def by simp
qed
have tail-key-notin-S:
   $\bigwedge p. p \in \text{set } ?\text{tail} \implies \text{fst } p \notin S$ 
proof -

```

```

fix p
assume p-tail: p ∈ set ?tail
show fst p ∉ S
proof
  assume fst p ∈ S
  then obtain q where q-b: q ∈ set (bp-bucket-entries b)
    and fst-q: fst q = fst p
    unfolding S-def bp-bucket-keys-def bp-entry-keys-def by auto
  have distinct-append: distinct (map fst (bp-bucket-entries b @ ?tail))
    using distinct-P unfolding entries-P .
  have disjoint:
    set (map fst (bp-bucket-entries b)) ∩ set (map fst ?tail) = {}
    using distinct-append by simp
  have fst p ∈ set (map fst ?tail)
    using p-tail by force
  then have fst p ∉ set (map fst (bp-bucket-entries b))
    using disjoint by blast
  moreover have fst p ∈ set (map fst (bp-bucket-entries b))
  proof -
    have fst q ∈ set (map fst (bp-bucket-entries b))
      using q-b by force
    then show ?thesis
      using fst-q by simp
  qed
  ultimately show False
    by blast
qed
qed
have tail-key-remaining:
  ∧p. p ∈ set ?tail ⇒ fst p ∈ keys-of (bp-view P')
proof -
  fix p
  assume p-tail: p ∈ set ?tail
  have old: fst p ∈ keys-of (bp-view P)
    unfolding bp-view-def bp-entry-keys-def entries-P using p-tail by auto
  have not-S: fst p ∉ S
    by (rule tail-key-notin-S[OF p-tail])
  show fst p ∈ keys-of (bp-view P')
    using old not-S unfolding P'-keys by blast
qed
have remaining-nonempty: keys-of (bp-view P') ≠ {}
proof -
  obtain p where p-tail: p ∈ set ?tail
    using tail-nonempty by (cases ?tail) auto
  then have fst p ∈ keys-of (bp-view P')
    by (rule tail-key-remaining)
  then show ?thesis by blast
qed
have lower-remaining:

```

$\bigwedge v. v \in \text{keys-of } (\text{bp-view } P') \implies \text{beta} \leq \text{value-of } (\text{bp-view } P') v$
proof –
 fix v
 assume $v: v \in \text{keys-of } (\text{bp-view } P')$
 then have $v\text{-old}: v \in \text{keys-of } (\text{bp-view } P)$
 using $P'\text{-keys}$ by blast
 then obtain p where $p\text{-entries}: p \in \text{set } (\text{bp-entries } P)$
 and $\text{fst-}p: \text{fst } p = v$
 unfolding $\text{bp-view-def bp-entry-keys-def}$ by auto
 have $v\text{-not-}S: v \notin S$
 using $v P'\text{-keys}$ by blast
 have $p\text{-tail}: p \in \text{set } ?\text{tail}$
proof –
 have $p \notin \text{set } (\text{bp-bucket-entries } b)$
 using $v\text{-not-}S \text{ fst-}p$ unfolding $S\text{-def bp-bucket-keys-def bp-entry-keys-def}$
 by auto
 then show $?thesis$
 using $p\text{-entries}$ unfolding $\text{entries-}P$ by auto
 qed
 have $\text{beta} \leq \text{snd } p$
 unfolding beta-def
 by (rule $\text{bp-tail-entry-ge-head-marker}[OF \text{ sorted-tail lower-tail } p\text{-tail}]$)
 moreover have $\text{value-of } (\text{bp-view } P') v = \text{snd } p$
proof –
 have $\text{value-of } (\text{bp-view } P) v = \text{snd } p$
 using $\text{values-}P[OF p\text{-entries}] \text{ fst-}p$ unfolding bp-view-def by simp
 moreover have $\text{value-of } (\text{bp-view } P') v = \text{value-of } (\text{bp-view } P) v$
 using $P'\text{-values}$ by simp
 ultimately show $?thesis$ by simp
 qed
 ultimately show $\text{beta} \leq \text{value-of } (\text{bp-view } P') v$
 by simp
 qed
 have $\text{pulled-before-remaining}$:
 $\bigwedge u v. \llbracket u \in S; v \in \text{keys-of } (\text{bp-view } P') \rrbracket$
 $\implies \text{value-of } (\text{bp-view } P) u \leq \text{value-of } (\text{bp-view } P') v$
proof –
 fix $u v$
 assume $u: u \in S$ and $v: v \in \text{keys-of } (\text{bp-view } P')$
 have $\text{value-of } (\text{bp-view } P) u < \text{beta}$
 by (rule $\text{pulled-lt-beta}[OF u]$)
 moreover have $\text{beta} \leq \text{value-of } (\text{bp-view } P') v$
 by (rule $\text{lower-remaining}[OF v]$)
 ultimately show $\text{value-of } (\text{bp-view } P) u \leq \text{value-of } (\text{bp-view } P') v$
 by simp
 qed
 show $?thesis$
 unfolding $\text{pull-separates-def}$
 using $S\text{-subset card-}S P'\text{-keys } P'\text{-values remaining-nonempty}$

$\text{pulled-before-remaining pulled-lt-beta lower-remaining}$
 by *auto*
 qed

lemma *bp-conservative-pull-invariant*:
 assumes *inv*: *bp-invariant P*
 and *pull*: *bp-conservative-pull M B P = (S, beta, P')*
 shows *bp-invariant P'*
proof –
 have $P' = \text{bp-delete-keys } S \ P$
 using *pull* **unfolding** *bp-conservative-pull-def* **by** (*auto simp: Let-def*)
 then **show** *?thesis*
 by (*simp add: bp-delete-keys-invariant[OF inv]*)
 qed

theorem *bp-conservative-pull-refines-pull-separates*:
 assumes *inv*: *bp-invariant P*
 and *upper*: $\bigwedge u. u \in \text{keys-of } (\text{bp-view } P) \implies \text{value-of } (\text{bp-view } P) \ u < B$
 and *pull*: *bp-conservative-pull M B P = (S, beta, P')*
 shows *pull-separates (bp-view P) M B S beta (bp-view P')*
proof –
 have *S-def*: $S = \text{bp-pull-set } M \ P$
 and *beta-def*: $\text{beta} = \text{bp-pull-bound } M \ B \ P$
 and *P'-def*: $P' = \text{bp-delete-keys } S \ P$
 using *pull* **unfolding** *bp-conservative-pull-def* **by** (*auto simp: Let-def*)
show *?thesis*
proof (*cases length (bp-entries P) ≤ M*)
 case *True*
 have *S-keys*: $S = \text{keys-of } (\text{bp-view } P)$
 using *True* **unfolding** *S-def bp-pull-set-def bp-view-def* **by** *simp*
 have *beta-B*: $\text{beta} = B$
 using *True* **unfolding** *beta-def bp-pull-bound-def* **by** *simp*
 have *keys-empty*: $\text{keys-of } (\text{bp-view } P') = \{\}$
unfolding *P'-def bp-delete-keys-view S-keys* **by** *simp*
 have *card-S*: $\text{card } S \leq M$
proof –
 have $\text{card } S = \text{card } (\text{fst ' set } (\text{bp-entries } P))$
unfolding *S-keys bp-view-def bp-entry-keys-def* **by** *simp*
 also have $\dots \leq \text{card } (\text{set } (\text{bp-entries } P))$
 by (*rule card-image-le*) *simp*
 also have $\dots \leq \text{length } (\text{bp-entries } P)$
 by (*rule card-length*)
 also have $\dots \leq M$
 by (*rule True*)
 finally **show** *?thesis* .
 qed
show *?thesis*
unfolding *pull-separates-def P'-def bp-delete-keys-view*

```

    using S-keys beta-B keys-empty card-S by auto
next
case False
have S-empty:  $S = \{\}$ 
  using False unfolding S-def bp-pull-set-def by simp
have beta-min:  $\text{beta} = \text{bp-min-value } B \text{ (bp-entries } P\text{)}$ 
  using False unfolding beta-def bp-pull-bound-def by simp
have P'-same:  $P' = P$ 
  unfolding P'-def S-empty bp-delete-keys-def bp-delete-keys-from-bucket-def
  by simp
have keys-nonempty:  $\text{keys-of (bp-view } P') \neq \{\}$ 
proof
  assume empty:  $\text{keys-of (bp-view } P') = \{\}$ 
  then have set  $(\text{bp-entries } P) = \{\}$ 
    unfolding P'-same bp-view-def bp-entry-keys-def by auto
  then have length  $(\text{bp-entries } P) = 0$ 
    by simp
  then show False
    using False by simp
qed
have lower:
 $\bigwedge v. v \in \text{keys-of (bp-view } P') \implies \text{beta} \leq \text{value-of (bp-view } P') v$ 
proof -
  fix v
  assume v:  $v \in \text{keys-of (bp-view } P')$ 
  then obtain b where vb:  $(v, b) \in \text{set (bp-entries } P)$ 
    unfolding P'-same bp-view-def bp-entry-keys-def by auto
  have bp-min-value  $B \text{ (bp-entries } P) \leq b$ 
    by (rule bp-min-value-le-entry[OF vb])
  moreover have bp-values  $P v = b$ 
    using inv vb by (auto simp: bp-invariant-def bp-values-consistent-def)
  ultimately show  $\text{beta} \leq \text{value-of (bp-view } P') v$ 
    unfolding beta-min P'-same bp-view-def by simp
qed
have keys-nonempty-P:  $\text{keys-of (bp-view } P) \neq \{\}$ 
  using keys-nonempty unfolding P'-same .
have lower-P:
 $\bigwedge v. v \in \text{keys-of (bp-view } P) \implies \text{beta} \leq \text{value-of (bp-view } P) v$ 
  using lower unfolding P'-same .
show ?thesis
  unfolding pull-separates-def P'-same S-empty
  using lower-P keys-nonempty-P by auto
qed
qed

lemma bp-pull-invariant:
  assumes inv: bp-invariant P
  and pull: bp-pull M B P = (S, beta, P')
  shows bp-invariant P'

```



```

proof (cases bp-can-first-bucket-pull M P)
  case True
    have first-pull: bp-first-bucket-pull M B P = (S, beta, P')
      using pull True unfolding bp-pull-def by simp
    show ?thesis
      by (rule bp-first-bucket-pull-invariant[OF inv first-pull])
  next
    case False
    have conservative-pull: bp-conservative-pull M B P = (S, beta, P')
      using pull False unfolding bp-pull-def by simp
    show ?thesis
      by (rule bp-conservative-pull-invariant[OF inv conservative-pull])
qed

```

```

lemma bp-first-bucket-pull-ordered-invariant:
  assumes ord: bp-ordered-invariant P
  and pull: bp-first-bucket-pull M B P = (S, beta, P')
  shows bp-ordered-invariant P'
proof -
  have cases: P' = P  $\vee$  ( $\exists T. P' = \text{bp-delete-keys } T P$ )
    using pull unfolding bp-first-bucket-pull-def
    by (cases bp-buckets P; auto split: list.splits)
  then show ?thesis
    proof
      assume P' = P
      then show ?thesis
        using ord by simp
    next
      assume  $\exists T. P' = \text{bp-delete-keys } T P$ 
      then obtain T where P' = bp-delete-keys T P
        by blast
      then show ?thesis
        using bp-delete-keys-ordered-invariant[OF ord] by simp
    qed
qed

```

```

lemma bp-conservative-pull-ordered-invariant:
  assumes ord: bp-ordered-invariant P
  and pull: bp-conservative-pull M B P = (S, beta, P')
  shows bp-ordered-invariant P'
proof -
  have P' = bp-delete-keys S P
    using pull unfolding bp-conservative-pull-def by (auto simp: Let-def)
  then show ?thesis
    using bp-delete-keys-ordered-invariant[OF ord] by simp
qed

```

```

lemma bp-pull-ordered-invariant:
  assumes ord: bp-ordered-invariant P

```

and *pull*: $bp\text{-}pull\ M\ B\ P = (S, \beta, P')$
shows *bp-ordered-invariant* P'
proof (*cases bp-can-first-bucket-pull* $M\ P$)
 case *True*
 have *first-pull*: $bp\text{-}first\text{-}bucket\text{-}pull\ M\ B\ P = (S, \beta, P')$
 using *pull* *True* **unfolding** *bp-pull-def* **by** *simp*
 show *?thesis*
 by (*rule bp-first-bucket-pull-ordered-invariant* [*OF ord first-pull*])
 next
 case *False*
 have *conservative-pull*: $bp\text{-}conservative\text{-}pull\ M\ B\ P = (S, \beta, P')$
 using *pull* *False* **unfolding** *bp-pull-def* **by** *simp*
 show *?thesis*
 by (*rule bp-conservative-pull-ordered-invariant*
 [*OF ord conservative-pull*])
qed

theorem *bp-pull-refines-pull-separates*:

assumes *inv*: *bp-invariant* P
 and *upper*: $\bigwedge u. u \in \text{keys-of } (bp\text{-}view\ P) \implies$
 value-of $(bp\text{-}view\ P)\ u < B$
 and *pull*: $bp\text{-}pull\ M\ B\ P = (S, \beta, P')$
shows *pull-separates* $(bp\text{-}view\ P)\ M\ B\ S\ \beta\ (bp\text{-}view\ P')$
proof (*cases bp-can-first-bucket-pull* $M\ P$)
 case *True*
 obtain $b\ c\ bs$ **where** *buckets*: $bp\text{-}buckets\ P = b \# c \# bs$
 and *len-b*: $\text{length } (bp\text{-}bucket\text{-}entries\ b) \leq M$
 and *below*: $bp\text{-}bucket\text{-}below\text{-}bound\ b\ (bp\text{-}marker\ c)$
 and *tail-nonempty*: $bp\text{-}bucket\text{-}entries\text{-}flat\ (c \# bs) \neq []$
 by (*rule bp-can-first-bucket-pullE* [*OF True*])
 have *first-pull*: $bp\text{-}first\text{-}bucket\text{-}pull\ M\ B\ P = (S, \beta, P')$
 using *pull* *True* **unfolding** *bp-pull-def* **by** *simp*
 show *?thesis*
 by (*rule bp-first-bucket-pull-refines-pull-separates*
 [*OF inv buckets len-b below tail-nonempty first-pull*])
 next
 case *False*
 have *conservative-pull*: $bp\text{-}conservative\text{-}pull\ M\ B\ P = (S, \beta, P')$
 using *pull* *False* **unfolding** *bp-pull-def* **by** *simp*
 show *?thesis*
 by (*rule bp-conservative-pull-refines-pull-separates*
 [*OF inv upper conservative-pull*])
qed

theorem *bp-pull-preserves-upper-bound*:

assumes *inv*: *bp-invariant* P
 and *pull-upper*: *partition-upper-bound* $(bp\text{-}view\ P)\ Bpull$
 and *upper*: *partition-upper-bound* $(bp\text{-}view\ P)\ B$
 and *pull*: $bp\text{-}pull\ M\ Bpull\ P = (S, \beta, P')$

```

shows partition-upper-bound (bp-view  $P'$ )  $B$ 
proof –
  have sep: pull-separates (bp-view  $P$ )  $M$   $B_{pull}$   $S$  beta (bp-view  $P'$ )
  proof (rule bp-pull-refines-pull-separates[OF inv - pull])
    fix  $u$ 
    assume  $u \in \text{keys-of } (bp\text{-view } P)$ 
    then show value-of (bp-view  $P$ )  $u < B_{pull}$ 
      using pull-upper unfolding partition-upper-bound-def by blast
    qed
  show ?thesis
    by (rule pull-separates-preserves-upper-bound[OF sep upper])
  qed

```

The preceding block completes the functional refinement layer. Insert and BatchPrepend update the view by minimum-value update, while Pull satisfies *pull-separates*: it returns at most the requested block, removes those keys, and establishes the returned boundary between pulled and remaining entries. The implementation attempts a cheap first-bucket pull when the ordered boundary proves it is safe; otherwise it uses the conservative pull specification. Both branches preserve *bp-invariant* and the abstract upper-bound condition.

14 Cost Budgets for the Bucketed Structure

The cost layer counts primitive functional steps by pairing each operation result with a natural number. The budget definitions isolate the paper-critical logarithmic search term as a function of N / M , where N is represented by an entry-list length and M by the block size. The potential terms record the credits that pay for lazy bucket splitting and for restoring the bucket-count ratio after a rebuild.

definition *bp-steps-of* :: $'a \times nat \Rightarrow nat$ **where**
bp-steps-of $r = \text{snd } r$

definition *bp-result-of* :: $'a \times nat \Rightarrow 'a$ **where**
bp-result-of $r = \text{fst } r$

lemma *bp-steps-of-pair* [*simp*]:
bp-steps-of $(x, c) = c$
unfolding *bp-steps-of-def* **by** *simp*

lemma *bp-result-of-pair* [*simp*]:
bp-result-of $(x, c) = x$
unfolding *bp-result-of-def* **by** *simp*

lemma *bp-result-of-add-cost-case* [*simp*]:
bp-result-of (*case* r *of* $(x, c') \Rightarrow (x, c + c')$) = *bp-result-of* r
by (*cases* r) *simp*

lemma *bp-steps-of-add-cost-case* [simp]:

bp-steps-of (case *r* of (*x*, *c'*) $\Rightarrow$ (*x*, *c* + *c'*)) =
c + *bp-steps-of* *r*
by (cases *r*) *simp*

definition *bp-ratio-log-budget* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ *nat* **where**
bp-ratio-log-budget *N* *M* = *Suc* (*floor-log* (*Suc* (*N* *div* *M*)))

definition *bp-insert-search-budget* :: *nat* $\Rightarrow$ *nat* $\Rightarrow$ *nat* **where**
bp-insert-search-budget *N* *M* = *Suc* (*bp-ratio-log-budget* *N* *M*)

definition *bp-lazy-insert-amortized-budget* ::
'k *bucketed-partition* $\Rightarrow$ *nat* **where**
bp-lazy-insert-amortized-budget *P* =
bp-insert-search-budget (*length* (*bp-entries* *P*)) (*bp-block-size* *P*) + 7

definition *bp-batch-prepend-log-budget* :: (*'k* $\times$ *real*) *list* $\Rightarrow$ *nat* $\Rightarrow$ *nat* **where**
bp-batch-prepend-log-budget *xs* *M* =
length *xs* * *bp-ratio-log-budget* (*length* *xs*) *M*

definition *bp-batch-prepend-per-item-budget* ::
(*'k* $\times$ *real*) *list* $\Rightarrow$ *nat* $\Rightarrow$ *nat* **where**
bp-batch-prepend-per-item-budget *xs* *M* =
Suc (*bp-ratio-log-budget* (*length* *xs*) *M*)

definition *bp-batch-prepend-amortized-budget* ::
(*'k* $\times$ *real*) *list* $\Rightarrow$ *nat* $\Rightarrow$ *nat* **where**
bp-batch-prepend-amortized-budget *xs* *M* =
length *xs* + *bp-batch-prepend-log-budget* *xs* *M*

definition *bp-pull-amortized-budget* :: *nat* $\Rightarrow$ *nat* **where**
bp-pull-amortized-budget *M* = 2 * *M*

definition *bp-local-split-budget* :: *nat* $\Rightarrow$ *nat* **where**
bp-local-split-budget *M* = *Suc* (2 * *Suc* *M*)

definition *bp-lazy-split-budget* :: *nat* $\Rightarrow$ *nat* **where**
bp-lazy-split-budget *M* = 5 + 4 * *M*

definition *bp-bucket-overflow* :: *nat* $\Rightarrow$ *'k* *bp-bucket* $\Rightarrow$ *nat* **where**
bp-bucket-overflow *M* *b* = *length* (*bp-bucket-entries* *b*) - *M*

definition *bp-bucket-overflow-sum* ::
nat $\Rightarrow$ *'k* *bp-bucket* *list* $\Rightarrow$ *nat* **where**
bp-bucket-overflow-sum *M* *bs* =
sum-list (*map* (*bp-bucket-overflow* *M*) *bs*)

definition *bp-overflow-potential* :: *'k* *bucketed-partition* $\Rightarrow$ *nat* **where**
bp-overflow-potential *P* =

$sum-list \ (map \ (bp_bucket_overflow \ (bp_block_size \ P)) \ (bp_buckets \ P))$

definition $bp_bucket_count_slack :: 'k \ bucketed_partition \Rightarrow nat$ **where**
 $bp_bucket_count_slack \ P =$
 $length \ (bp_buckets \ P) -$
 $Suc \ (length \ (bp_entries \ P) \ div \ bp_block_size \ P)$

definition $bp_split_potential :: 'k \ bucketed_partition \Rightarrow nat$ **where**
 $bp_split_potential \ P =$
 $4 * bp_overflow_potential \ P + bp_bucket_count_slack \ P$

definition $bp_amortized_step_bound ::$
 $nat \Rightarrow 'k \ bucketed_partition \Rightarrow 'k \ bucketed_partition \Rightarrow nat \Rightarrow bool$ **where**
 $bp_amortized_step_bound \ c \ P \ P' \ t \longleftrightarrow$
 $c + bp_split_potential \ P' \leq t + bp_split_potential \ P$

definition $bp_pull_state_of ::$
 $'k \ set \times real \times 'k \ bucketed_partition \Rightarrow 'k \ bucketed_partition$ **where**
 $bp_pull_state_of \ R = (case \ R \ of \ (-, -, P') \Rightarrow P')$

definition $bp_pull_amortized_step_bound ::$
 $nat \Rightarrow 'k \ bucketed_partition \Rightarrow$
 $('k \ set \times real \times 'k \ bucketed_partition) \Rightarrow nat \Rightarrow bool$ **where**
 $bp_pull_amortized_step_bound \ c \ P \ R \ t \longleftrightarrow$
 $c + bp_split_potential \ (bp_pull_state_of \ R) \leq$
 $t + bp_split_potential \ P$

definition $bp_bucket_count_ratio_ok :: 'k \ bucketed_partition \Rightarrow bool$ **where**
 $bp_bucket_count_ratio_ok \ P \longleftrightarrow$
 $0 < bp_block_size \ P \wedge$
 $length \ (bp_buckets \ P) \leq$
 $Suc \ (length \ (bp_entries \ P) \ div \ bp_block_size \ P)$

definition $bp_regular_invariant :: 'k \ bucketed_partition \Rightarrow bool$ **where**
 $bp_regular_invariant \ P \longleftrightarrow$
 $bp_ordered_invariant \ P \wedge bp_bucket_count_ratio_ok \ P$

definition $bp_bucket_count_loaded :: 'k \ bucketed_partition \Rightarrow bool$ **where**
 $bp_bucket_count_loaded \ P \longleftrightarrow$
 $bp_block_size \ P * (length \ (bp_buckets \ P) - 1) \leq$
 $length \ (bp_entries \ P)$

definition $bp_bucket_search_steps :: 'k \ bucketed_partition \Rightarrow nat$ **where**
 $bp_bucket_search_steps \ P = Suc \ (floor_log \ (length \ (bp_buckets \ P)))$

fun $bp_halving_search_steps :: nat \Rightarrow nat$ **where**
 $[simp \ del]: bp_halving_search_steps \ n =$
 $(if \ n < 2 \ then \ 1 \ else \ Suc \ (bp_halving_search_steps \ (n \ div \ 2)))$

definition *bp-local-insert-search-charge* :: 'k bucketed-partition $\Rightarrow$ nat **where**
bp-local-insert-search-charge P = Suc (*bp-bucket-search-steps* P)

definition *c-bp-bucket-directory-search* ::
'k bp-bucket list $\Rightarrow$ real $\Rightarrow$ unit $\times$ nat **where**
c-bp-bucket-directory-search bs b =
((), *bp-halving-search-steps* (length bs))

fun *c-bp-bucketize-sorted-entries-aux* ::
nat $\Rightarrow$ nat $\Rightarrow$ ('k $\times$ real) list $\Rightarrow$ 'k bp-bucket list $\times$ nat **where**
c-bp-bucketize-sorted-entries-aux 0 M xs = ([], 0)
| *c-bp-bucketize-sorted-entries-aux* (Suc fuel) M xs =
(if M = 0 $\vee$ xs = []
then ([], 0)
else (let (bs, c) =
c-bp-bucketize-sorted-entries-aux fuel M (drop M xs)
in (bp-make-bucket (take M xs) # bs,
Suc (length (take M xs) + c))))

definition *c-bp-bucketize-sorted-entries* ::
nat $\Rightarrow$ ('k $\times$ real) list $\Rightarrow$ 'k bp-bucket list $\times$ nat **where**
c-bp-bucketize-sorted-entries M xs =
c-bp-bucketize-sorted-entries-aux (length xs) M xs

definition *c-bp-bucketize-entries* ::
nat $\Rightarrow$ ('k $\times$ real) list $\Rightarrow$ 'k bp-bucket list $\times$ nat **where**
c-bp-bucketize-entries M xs =
c-bp-bucketize-sorted-entries M (sort-key snd xs)

definition *c-bp-rebucket-build* ::
'k bucketed-partition $\Rightarrow$ 'k bucketed-partition $\times$ nat **where**
c-bp-rebucket-build P =
(case *c-bp-bucketize-entries* (bp-block-size P) (bp-entries P) of
(bs, c) $\Rightarrow$ (P([bp-buckets := bs], c))

The definitions above introduce the amortized accounting used throughout the cost proof. *bp-ratio-log-budget* is the formal ratio-log term: it applies *floor-log* to *Suc* (*N div M*), avoiding division by zero corner cases at the statement level. The actual bucket-directory search is modeled by *bp-halving-search-steps*, and *bp-bucket-search-steps* records the resulting cost for a state.

The potential *bp-split-potential* has two components. The weighted overflow component *bp-overflow-potential* counts entries above the strict block size, multiplied by four in the combined potential. The *bp-bucket-count-slack* component measures how far the bucket count is above the target ratio. A regular state, captured by *bp-regular-invariant*, has no such debt: it is ordered and satisfies *bp-bucket-count-ratio-ok*. The amortized predicates *bp-amortized-step-bound* and *bp-pull-amortized-step-bound* express the usual inequality *actual* + *Phi*(*after*) $\leq$ *budget* + *Phi*(*before*).

definition *c-bp-local-insert-search* ::
 $'k \Rightarrow \text{real} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow$
 $'k \text{ bucketed-partition} \times \text{nat}$ **where**
c-bp-local-insert-search $x \ b \ P =$
 $(\text{bp-local-insert-state } x \ b \ P, \text{ bp-local-insert-search-charge } P)$

definition *c-bp-local-insert-split* ::
 $'k \Rightarrow \text{real} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow$
 $'k \text{ bucketed-partition} \times \text{nat}$ **where**
c-bp-local-insert-split $x \ b \ P =$
 $(\text{bp-local-insert-state } x \ b \ P,$
 $\text{bp-local-insert-search-charge } P +$
 $\text{bp-local-split-budget } (\text{bp-block-size } P))$

definition *c-bp-lazy-insert* ::
 $'k \Rightarrow \text{real} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow$
 $'k \text{ bucketed-partition} \times \text{nat}$ **where**
c-bp-lazy-insert $x \ b \ P =$
 $(\text{bp-lazy-insert-state } x \ b \ P,$
 $\text{bp-local-insert-search-charge } P +$
 $\text{bp-lazy-split-budget } (\text{bp-block-size } P))$

definition *bp-lazy-bucket-insert-charge* ::
 $\text{nat} \Rightarrow 'k \text{ bp-bucket} \Rightarrow \text{nat}$ **where**
bp-lazy-bucket-insert-charge $M \ b =$
 $(\text{if length } (\text{bp-bucket-entries } b) < 2 * M$
 $\text{then } 1 \text{ else bp-lazy-split-budget } M)$

fun *bp-lazy-insert-bucket-charge* ::
 $\text{nat} \Rightarrow 'k \times \text{real} \Rightarrow 'k \text{ bp-bucket list} \Rightarrow \text{nat}$ **where**
bp-lazy-insert-bucket-charge $M \ p \ [] = 1$
 $| \text{ bp-lazy-insert-bucket-charge } M \ p \ [b] =$
 $(\text{if snd } p < \text{bp-marker } b \text{ then } 1$
 $\text{else bp-lazy-bucket-insert-charge } M \ b)$
 $| \text{ bp-lazy-insert-bucket-charge } M \ p \ (b \# c \# bs) =$
 $(\text{if snd } p < \text{bp-marker } b \text{ then } 1$
 $\text{else if snd } p < \text{bp-marker } c$
 $\text{then bp-lazy-bucket-insert-charge } M \ b$
 $\text{else bp-lazy-insert-bucket-charge } M \ p \ (c \# bs))$

definition *bp-lazy-insert-charge* ::
 $'k \Rightarrow \text{real} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow \text{nat}$ **where**
bp-lazy-insert-charge $x \ b \ P =$
 $(\text{let } b' = (\text{if } x \in \text{bp-entry-keys } (\text{bp-entries } P)$
 $\text{then min } (\text{bp-values } P \ x) \ b \text{ else } b);$
 $P0 = \text{bp-delete-key } x \ P$
 $\text{in bp-lazy-insert-bucket-charge } (\text{bp-block-size } P0) \ (x, \ b')$
 $(\text{bp-buckets } P0))$

definition *c-bp-lazy-insert-precise* ::
 $'k \Rightarrow \text{real} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow$
 $'k \text{ bucketed-partition} \times \text{nat}$ **where**
c-bp-lazy-insert-precise $x \ b \ P =$
 $(\text{bp-lazy-insert-state } x \ b \ P,$
 $\text{bp-local-insert-search-charge } P + \text{bp-lazy-insert-charge } x \ b \ P)$

fun *bp-lazy-batch-prepend-state* ::
 $('k \times \text{real}) \text{ list} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow$
 $'k \text{ bucketed-partition}$ **where**
bp-lazy-batch-prepend-state $\square P = P$
 $| \text{bp-lazy-batch-prepend-state } ((x, b) \# xs) \ P =$
 $\text{bp-lazy-batch-prepend-state } xs \ (\text{bp-lazy-insert-state } x \ b \ P)$

fun *c-bp-lazy-batch-prepend-precise* ::
 $('k \times \text{real}) \text{ list} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow$
 $'k \text{ bucketed-partition} \times \text{nat}$ **where**
c-bp-lazy-batch-prepend-precise $\square P = (P, 0)$
 $| \text{c-bp-lazy-batch-prepend-precise } ((x, b) \# xs) \ P =$
 $(\text{case } \text{c-bp-lazy-insert-precise } x \ b \ P \text{ of}$
 $(P1, c1) \Rightarrow$
 $(\text{case } \text{c-bp-lazy-batch-prepend-precise } xs \ P1 \text{ of}$
 $(P2, c2) \Rightarrow (P2, c1 + c2)))$

fun *bp-lazy-batch-ratio-ok* ::
 $('k \times \text{real}) \text{ list} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow \text{bool}$ **where**
bp-lazy-batch-ratio-ok $\square P \longleftrightarrow \text{True}$
 $| \text{bp-lazy-batch-ratio-ok } ((x, b) \# xs) \ P \longleftrightarrow$
 $\text{bp-bucket-count-ratio-ok } P \wedge$
 $\text{bp-lazy-batch-ratio-ok } xs$
 $(\text{bp-result-of } (\text{c-bp-lazy-insert-precise } x \ b \ P))$

fun *bp-lazy-batch-insert-budget-sum* ::
 $('k \times \text{real}) \text{ list} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow \text{nat}$ **where**
bp-lazy-batch-insert-budget-sum $\square P = 0$
 $| \text{bp-lazy-batch-insert-budget-sum } ((x, b) \# xs) \ P =$
 $\text{bp-lazy-insert-amortized-budget } P +$
 $\text{bp-lazy-batch-insert-budget-sum } xs$
 $(\text{bp-result-of } (\text{c-bp-lazy-insert-precise } x \ b \ P))$

lemma *c-bp-lazy-insert-precise-eq* [simp]:
c-bp-lazy-insert-precise $x \ b \ P =$
 $(\text{bp-lazy-insert-state } x \ b \ P,$
 $\text{bp-local-insert-search-charge } P + \text{bp-lazy-insert-charge } x \ b \ P)$
unfolding *c-bp-lazy-insert-precise-def* **by** *simp*

definition *bp-regularized-local-insert* ::
 $'k \Rightarrow \text{real} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow 'k \text{ bucketed-partition}$ **where**
bp-regularized-local-insert $x \ b \ P =$

bp-rebucket (*bp-local-insert-state* *x b P*)

definition *c-bp-regularized-local-insert* ::

'*k* $\Rightarrow$ *real* $\Rightarrow$ '*k* *bucketed-partition* $\Rightarrow$
'*k* *bucketed-partition* $\times$ *nat* **where**
c-bp-regularized-local-insert *x b P* =
(*bp-regularized-local-insert* *x b P*, *bp-local-insert-search-charge* *P*)

This wrapper counts the concrete rebuilding pass explicitly. It is an accounting bridge for the later lazy-split analysis, not the final insert operation bound: rebuilding every bucket has a linear term.

definition *c-bp-regularized-local-insert-build* ::

'*k* $\Rightarrow$ *real* $\Rightarrow$ '*k* *bucketed-partition* $\Rightarrow$
'*k* *bucketed-partition* $\times$ *nat* **where**
c-bp-regularized-local-insert-build *x b P* =
(case *c-bp-rebucket-build* (*bp-local-insert-state* *x b P*) of
(*P'*, *c*) $\Rightarrow$ (*P'*, *bp-local-insert-search-charge* *P* + *c*))

definition *c-bp-rebucketed-batch-prepend-bulk* ::

('*k* $\times$ *real*) *list* $\Rightarrow$ '*k* *bucketed-partition* $\Rightarrow$
'*k* *bucketed-partition* $\times$ *nat* **where**
c-bp-rebucketed-batch-prepend-bulk *xs P* =
(*bp-rebucketed-batch-prepend* *xs P*,
bp-batch-prepend-log-budget *xs* (*bp-block-size* *P*))

definition *c-bp-bucketed-batch-prepend-direct* ::

('*k* $\times$ *real*) *list* $\Rightarrow$ '*k* *bucketed-partition* $\Rightarrow$
'*k* *bucketed-partition* $\times$ *nat* **where**
c-bp-bucketed-batch-prepend-direct *xs P* =
(*bp-bucketed-batch-prepend-state* *xs P*,
bp-batch-prepend-amortized-budget *xs* (*bp-block-size* *P*))

definition *c-bp-bucketed-batch-prepend-direct-actual* ::

('*k* $\times$ *real*) *list* $\Rightarrow$ '*k* *bucketed-partition* $\Rightarrow$
'*k* *bucketed-partition* $\times$ *nat* **where**
c-bp-bucketed-batch-prepend-direct-actual *xs P* =
(*bp-bucketed-batch-prepend-state* *xs P*,
bp-batch-prepend-log-budget *xs* (*bp-block-size* *P*))

definition *c-bp-first-bucket-pull* ::

nat $\Rightarrow$ *real* $\Rightarrow$ '*k* *bucketed-partition* $\Rightarrow$
(''*k* *set* $\times$ *real* $\times$ '*k* *bucketed-partition*) $\times$ *nat* **where**
c-bp-first-bucket-pull *M B P* = (*bp-first-bucket-pull* *M B P*, *M*)

definition *c-bp-first-bucket-pull-scan* ::

nat $\Rightarrow$ *real* $\Rightarrow$ '*k* *bucketed-partition* $\Rightarrow$
(''*k* *set* $\times$ *real* $\times$ '*k* *bucketed-partition*) $\times$ *nat* **where**
c-bp-first-bucket-pull-scan *M B P* =
(case *bp-first-bucket-pull* *M B P* of

$$(S, \text{beta}, P') \Rightarrow ((S, \text{beta}, P'), \text{card } S))$$

lemma *bp-ratio-log-budget-pos* [simp]:

$0 < \text{bp-ratio-log-budget } N \ M$

unfolding *bp-ratio-log-budget-def* **by** *simp*

lemma *bp-insert-search-budget-pos* [simp]:

$0 < \text{bp-insert-search-budget } N \ M$

unfolding *bp-insert-search-budget-def* **by** *simp*

lemma *bp-lazy-insert-amortized-budget-ratio*:

bp-lazy-insert-amortized-budget $P =$

bp-ratio-log-budget (*length* (*bp-entries* P)) (*bp-block-size* P) + 8

unfolding *bp-lazy-insert-amortized-budget-def* *bp-insert-search-budget-def*
by *simp*

lemma *bp-batch-prepend-amortized-budget-alt*:

bp-batch-prepend-amortized-budget $xs \ M =$

bp-batch-prepend-per-item-budget $xs \ M * \text{length } xs$

unfolding *bp-batch-prepend-amortized-budget-def*

bp-batch-prepend-log-budget-def

bp-batch-prepend-per-item-budget-def

by (*simp* *add: algebra-simps*)

lemma *bp-batch-prepend-log-budget-le-amortized-budget*:

bp-batch-prepend-log-budget $xs \ M \leq$

bp-batch-prepend-amortized-budget $xs \ M$

unfolding *bp-batch-prepend-amortized-budget-def* **by** *simp*

lemma *bp-halving-search-steps-eq-floor-log*:

bp-halving-search-steps $n = \text{Suc } (\text{floor-log } n)$

by (*induction* n *rule: bp-halving-search-steps.induct*)

(*simp* *add: bp-halving-search-steps.simps floor-log.simps*)

lemma *c-bp-bucket-directory-search-result* [simp]:

bp-result-of (*c-bp-bucket-directory-search* $bs \ b$) = ()

unfolding *bp-result-of-def* *c-bp-bucket-directory-search-def* **by** *simp*

lemma *c-bp-bucket-directory-search-steps* [simp]:

bp-steps-of (*c-bp-bucket-directory-search* $bs \ b$) =

bp-halving-search-steps (*length* bs)

unfolding *bp-steps-of-def* *c-bp-bucket-directory-search-def* **by** *simp*

lemma *c-bp-bucket-directory-search-steps-floor-log*:

bp-steps-of (*c-bp-bucket-directory-search* $bs \ b$) =

Suc (*floor-log* (*length* bs))

by (*simp* *add: bp-halving-search-steps-eq-floor-log*)

lemma *c-bp-bucket-directory-search-state-steps*:

$bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}bucket\text{-}directory\text{-}search\ (bp\text{-}buckets\ P)\ b) =$
 $bp\text{-}bucket\text{-}search\text{-}steps\ P$
proof –
have $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}bucket\text{-}directory\text{-}search\ (bp\text{-}buckets\ P)\ b) =$
 $Suc\ (floor\text{-}log\ (length\ (bp\text{-}buckets\ P)))$
by (*simp add: bp-halving-search-steps-eq-floor-log*)
also have $\dots = bp\text{-}bucket\text{-}search\text{-}steps\ P$
unfolding *bp-bucket-search-steps-def* **by** *simp*
finally show *?thesis* .
qed

lemma *c-bp-bucket-directory-search-ratio-bound*:
assumes *count-le*: $length\ bs \leq Suc\ (N\ div\ M)$
shows $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}bucket\text{-}directory\text{-}search\ bs\ b) \leq$
 $bp\text{-}ratio\text{-}log\text{-}budget\ N\ M$
proof –
have $floor\text{-}log\ (length\ bs) \leq floor\text{-}log\ (Suc\ (N\ div\ M))$
by (*rule floor-log-le-iff[OF count-le]*)
then show *?thesis*
unfolding *bp-ratio-log-budget-def*
by (*simp add: bp-halving-search-steps-eq-floor-log*)
qed

lemma *bp-local-insert-search-charge-search-steps*:
 $bp\text{-}local\text{-}insert\text{-}search\text{-}charge\ P =$
 $Suc\ (bp\text{-}steps\text{-}of$
 $(c\text{-}bp\text{-}bucket\text{-}directory\text{-}search\ (bp\text{-}buckets\ P)\ b))$
proof –
have *search*:
 $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}bucket\text{-}directory\text{-}search\ (bp\text{-}buckets\ P)\ b) =$
 $bp\text{-}bucket\text{-}search\text{-}steps\ P$
by (*rule c-bp-bucket-directory-search-state-steps*)
show *?thesis*
unfolding *bp-local-insert-search-charge-def* **using** *search* **by** *simp*
qed

lemma *c-bp-bucketize-sorted-entries-aux-empty* [*simp*]:
 $c\text{-}bp\text{-}bucketize\text{-}sorted\text{-}entries\text{-}aux\ fuel\ M\ [] = ([],\ 0)$
by (*cases fuel*) *simp-all*

lemma *c-bp-bucketize-sorted-entries-aux-result* [*simp*]:
 $bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}bucketize\text{-}sorted\text{-}entries\text{-}aux\ fuel\ M\ xs) =$
 $bp\text{-}bucketize\text{-}sorted\text{-}entries\text{-}aux\ fuel\ M\ xs$
proof (*induction fuel arbitrary: xs*)
case 0
then show *?case*
by (*simp add: bp-result-of-def*)
next
case (*Suc fuel*)

```

show ?case
proof (cases  $M = 0 \vee xs = []$ )
  case True
  then show ?thesis
    by (simp add: bp-result-of-def)
next
  case False
  obtain bs c where rec:
    c-bp-bucketize-sorted-entries-aux fuel M (drop M xs) = (bs, c)
  by (cases c-bp-bucketize-sorted-entries-aux fuel M (drop M xs))
    simp
  have bs:
    bs = bp-bucketize-sorted-entries-aux fuel M (drop M xs)
  using Suc.IH[of drop M xs] rec
  by (simp add: bp-result-of-def)
  show ?thesis
    using False rec bs by (simp add: bp-result-of-def Let-def)
qed
qed

lemma c-bp-bucketize-sorted-entries-result [simp]:
  bp-result-of (c-bp-bucketize-sorted-entries M xs) =
    bp-bucketize-sorted-entries M xs
unfolding c-bp-bucketize-sorted-entries-def
  bp-bucketize-sorted-entries-def by simp

lemma c-bp-bucketize-entries-result [simp]:
  bp-result-of (c-bp-bucketize-entries M xs) = bp-bucketize-entries M xs
unfolding c-bp-bucketize-entries-def bp-bucketize-entries-def by simp

lemma c-bp-bucketize-sorted-entries-aux-steps-le:
  assumes M-pos:  $0 < M$ 
  and fuel:  $\text{length } xs \leq \text{fuel}$ 
  shows bp-steps-of (c-bp-bucketize-sorted-entries-aux fuel M xs)  $\leq$ 
    length xs + Suc (length xs div M)
  using M-pos fuel
proof (induction fuel arbitrary: xs)
  case 0
  then show ?case
    by (simp add: bp-steps-of-def)
next
  case (Suc fuel)
  show ?case
  proof (cases xs = [])
  case True
  then show ?thesis
    by (simp add: bp-steps-of-def)
  next
  case False

```

```

note xs-nonempty = False
show ?thesis
proof (cases length xs < M)
  case True
  then have drop M xs = []
  by simp
  then show ?thesis
  using Suc.prems xs-nonempty True by (simp add: bp-steps-of-def)
next
case False
have M-le:  $M \leq \text{length } xs$ 
  using False by simp
have drop-fuel:  $\text{length } (\text{drop } M \text{ } xs) \leq \text{fuel}$ 
  using Suc.prems False by simp
have tail:
  bp-steps-of
    (c-bp-bucketize-sorted-entries-aux fuel M (drop M xs))  $\leq$ 
     $\text{length } (\text{drop } M \text{ } xs) + \text{Suc } (\text{length } (\text{drop } M \text{ } xs) \text{ div } M)$ 
  by (rule Suc.IH[OF Suc.prems(1) drop-fuel])
have len-xs:  $\text{length } xs = M + \text{length } (\text{drop } M \text{ } xs)$ 
  using M-le by simp
have take-len:  $\text{length } (\text{take } M \text{ } xs) = M$ 
  using M-le by simp
have div-eq:
   $\text{length } xs \text{ div } M = \text{Suc } (\text{length } (\text{drop } M \text{ } xs) \text{ div } M)$ 
proof –
  have  $\text{length } xs \text{ div } M = (M + \text{length } (\text{drop } M \text{ } xs)) \text{ div } M$ 
  using len-xs by simp
  also have  $\dots = \text{length } (\text{drop } M \text{ } xs) \text{ div } M + 1$ 
  using Suc.prems(1) by (simp add: div-add-self1)
  finally show ?thesis by simp
qed
obtain bs c where rec:
  c-bp-bucketize-sorted-entries-aux fuel M (drop M xs) = (bs, c)
  by (cases c-bp-bucketize-sorted-entries-aux fuel M (drop M xs))
  simp
have c-le:
   $c \leq \text{length } (\text{drop } M \text{ } xs) + \text{Suc } (\text{length } (\text{drop } M \text{ } xs) \text{ div } M)$ 
  using tail rec unfolding bp-steps-of-def by simp
have step-eq:
  bp-steps-of
    (c-bp-bucketize-sorted-entries-aux (Suc fuel) M xs) =
     $\text{Suc } (M + c)$ 
  using Suc.prems xs-nonempty False rec take-len
  by (simp add: bp-steps-of-def Let-def)
have  $\text{Suc } (M + c) \leq$ 
   $M + \text{length } (\text{drop } M \text{ } xs) +$ 
   $\text{Suc } (\text{Suc } (\text{length } (\text{drop } M \text{ } xs) \text{ div } M))$ 
  using c-le by linarith

```

```

show ?thesis
using step-eq len-xs div-eq
  ⟨Suc (M + c) ≤
    M + length (drop M xs) +
    Suc (Suc (length (drop M xs) div M))⟩
by simp
qed
qed
qed

lemma c-bp-bucketize-sorted-entries-steps-le:
  assumes 0 < M
  shows bp-steps-of (c-bp-bucketize-sorted-entries M xs) ≤
    length xs + Suc (length xs div M)
  unfolding c-bp-bucketize-sorted-entries-def
  by (rule c-bp-bucketize-sorted-entries-aux-steps-le[OF assms]) simp

lemma c-bp-bucketize-entries-steps-le:
  assumes 0 < M
  shows bp-steps-of (c-bp-bucketize-entries M xs) ≤
    length xs + Suc (length xs div M)
  unfolding c-bp-bucketize-entries-def
  using c-bp-bucketize-sorted-entries-steps-le[OF assms,
    of sort-key snd xs]
  by simp

lemma c-bp-bucketize-entries-steps-le-linear-length:
  assumes 0 < M
  shows bp-steps-of (c-bp-bucketize-entries M xs) ≤
    Suc (2 * length xs)
proof –
  have length xs div M ≤ length xs
  by simp
  then have length xs + Suc (length xs div M) ≤
    Suc (2 * length xs)
  by linarith
  moreover have bp-steps-of (c-bp-bucketize-entries M xs) ≤
    length xs + Suc (length xs div M)
  by (rule c-bp-bucketize-entries-steps-le[OF assms])
  ultimately show ?thesis
  by linarith
qed

lemma c-bp-bucketize-entries-steps-le-local-split-budget:
  assumes M-pos: 0 < M
  and len: length xs ≤ Suc M
  shows bp-steps-of (c-bp-bucketize-entries M xs) ≤
    bp-local-split-budget M
proof –

```

have *steps*:
 $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}bucketize\text{-}entries\ M\ xs) \leq$
 $Suc\ (2 * length\ xs)$
by (rule *c-bp-bucketize-entries-steps-le-linear-length*[*OF M-pos*])
have $Suc\ (2 * length\ xs) \leq bp\text{-}local\text{-}split\text{-}budget\ M$
using *len* **unfolding** *bp-local-split-budget-def* **by** *simp*
then show *?thesis*
using *steps* **by** *linarith*
qed

lemma *bp-local-split-budget-le-lazy-split-budget*:
 $bp\text{-}local\text{-}split\text{-}budget\ M \leq bp\text{-}lazy\text{-}split\text{-}budget\ M$
unfolding *bp-local-split-budget-def* *bp-lazy-split-budget-def* **by** *simp*

lemma *c-bp-bucketize-entries-steps-le-lazy-split-budget*:
assumes *M-pos*: $0 < M$
and *len*: $length\ xs \leq Suc\ (2 * M)$
shows $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}bucketize\text{-}entries\ M\ xs) \leq$
 $bp\text{-}lazy\text{-}split\text{-}budget\ M$
proof –
have *steps*:
 $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}bucketize\text{-}entries\ M\ xs) \leq$
 $Suc\ (2 * length\ xs)$
by (rule *c-bp-bucketize-entries-steps-le-linear-length*[*OF M-pos*])
have $Suc\ (2 * length\ xs) \leq bp\text{-}lazy\text{-}split\text{-}budget\ M$
using *len* **unfolding** *bp-lazy-split-budget-def* **by** *simp*
then show *?thesis*
using *steps* **by** *linarith*
qed

lemma *bp-lazy-split-budget-paid-by-bucket-overflow*:
assumes *large*: $2 * M \leq length\ (bp\text{-}bucket\text{-}entries\ b)$
shows $bp\text{-}lazy\text{-}split\text{-}budget\ M \leq$
 $5 + 4 * bp\text{-}bucket\text{-}overflow\ M\ b$
proof –
have $M \leq bp\text{-}bucket\text{-}overflow\ M\ b$
using *large* **unfolding** *bp-bucket-overflow-def* **by** *linarith*
then show *?thesis*
unfolding *bp-lazy-split-budget-def* **by** *simp*
qed

lemma *bp-bucket-overflow-sum-simps* [*simp*]:
 $bp\text{-}bucket\text{-}overflow\text{-}sum\ M\ [] = 0$
 $bp\text{-}bucket\text{-}overflow\text{-}sum\ M\ (b \# bs) =$
 $bp\text{-}bucket\text{-}overflow\ M\ b + bp\text{-}bucket\text{-}overflow\text{-}sum\ M\ bs$
 $bp\text{-}bucket\text{-}overflow\text{-}sum\ M\ (bs @ cs) =$
 $bp\text{-}bucket\text{-}overflow\text{-}sum\ M\ bs + bp\text{-}bucket\text{-}overflow\text{-}sum\ M\ cs$
unfolding *bp-bucket-overflow-sum-def* **by** *simp-all*

```

lemma bp-overflow-potential-alt:
  bp-overflow-potential  $P =$ 
    bp-bucket-overflow-sum (bp-block-size  $P$ ) (bp-buckets  $P$ )
  unfolding bp-overflow-potential-def bp-bucket-overflow-sum-def by simp

lemma bp-bucket-overflow-sum-zero-if-sizes-ok:
  assumes sizes:  $\forall b \in \text{set } bs. \text{length } (\text{bp-bucket-entries } b) \leq M$ 
  shows bp-bucket-overflow-sum  $M$   $bs = 0$ 
  using sizes unfolding bp-bucket-overflow-sum-def bp-bucket-overflow-def
  by (induction  $bs$ ) auto

lemma bp-bucket-overflow-cons-credit:
  fixes  $b :: 'k \text{ bp-bucket}$ 
  shows  $1 + 4 * \text{bp-bucket-overflow } M$ 
     $(b(\text{bp-bucket-entries} := p \# \text{bp-bucket-entries } b)) \leq$ 
     $5 + 4 * \text{bp-bucket-overflow } M \ b$ 
proof -
  let  $?l = \text{length } (\text{bp-bucket-entries } b)$ 
  show ?thesis
  proof (cases  $?l < M$ )
    case True
    then have Suc  $?l \leq M$ 
    by simp
    then have after: bp-bucket-overflow  $M$ 
       $(b(\text{bp-bucket-entries} := p \# \text{bp-bucket-entries } b)) = 0$ 
    unfolding bp-bucket-overflow-def by simp
    have before: bp-bucket-overflow  $M \ b = 0$ 
    using True unfolding bp-bucket-overflow-def by simp
    show ?thesis
    unfolding after before by simp
  next
  case False
  then have M-le:  $M \leq ?l$ 
  by simp
  have after: bp-bucket-overflow  $M$ 
     $(b(\text{bp-bucket-entries} := p \# \text{bp-bucket-entries } b)) =$ 
    Suc (bp-bucket-overflow  $M \ b$ )
  using M-le unfolding bp-bucket-overflow-def by simp
  show ?thesis
  unfolding after by simp
qed
qed

lemma bp-lazy-bucket-insert-entries-weighted-overflow-credit:
  assumes M-pos:  $0 < M$ 
  and size:  $\text{length } (\text{bp-bucket-entries } b) \leq 2 * M$ 
  shows bp-lazy-bucket-insert-charge  $M \ b +$ 
     $4 * \text{bp-bucket-overflow-sum } M$ 
     $(\text{bp-lazy-bucket-insert-entries } M \ p \ b) \leq$ 

```


$5 + 4 * \text{bp-bucket-overflow } M \ b$
proof (cases length (bp-bucket-entries b) < 2 * M)
case True
have charge: bp-lazy-bucket-insert-charge M b = 1
using True **unfolding** bp-lazy-bucket-insert-charge-def **by** simp
have result:
 bp-lazy-bucket-insert-entries M p b =
 [b⟦bp-bucket-entries := p # bp-bucket-entries b⟧]
using True **unfolding** bp-lazy-bucket-insert-entries-def **by** simp
have credit:
 1 + 4 * bp-bucket-overflow M
 (b⟦bp-bucket-entries := p # bp-bucket-entries b⟧) ≤
 5 + 4 * bp-bucket-overflow M b
by (rule bp-bucket-overflow-cons-credit)
show ?thesis
using credit **unfolding** charge result **by** simp
next
case False
have len-eq: length (bp-bucket-entries b) = 2 * M
using False size **by** simp
have charge: bp-lazy-bucket-insert-charge M b = bp-lazy-split-budget M
using False **unfolding** bp-lazy-bucket-insert-charge-def **by** simp
have result:
 bp-lazy-bucket-insert-entries M p b =
 bp-bucketize-entries M (p # bp-bucket-entries b)
using False **unfolding** bp-lazy-bucket-insert-entries-def **by** simp
have sizes:
 $\forall c \in \text{set } (bp\text{-bucketize-entries } M \ (p \# bp\text{-bucket-entries } b)).$
 $\text{length } (bp\text{-bucket-entries } c) \leq M$
by (rule bp-bucketize-entries-sizes-ok[OF M-pos])
have overflow-zero:
 bp-bucket-overflow-sum M
 (bp-bucketize-entries M (p # bp-bucket-entries b)) = 0
by (rule bp-bucket-overflow-sum-zero-if-sizes-ok[OF sizes])
have before: bp-bucket-overflow M b = M
using len-eq **unfolding** bp-bucket-overflow-def **by** simp
show ?thesis
unfolding charge result overflow-zero before bp-lazy-split-budget-def
by simp
qed

lemma bp-lazy-insert-bucket-weighted-overflow-credit:
assumes M-pos: 0 < M
and sizes: $\forall b \in \text{set } bs. \text{length } (bp\text{-bucket-entries } b) \leq 2 * M$
shows bp-lazy-insert-bucket-charge M p bs +
 4 * bp-bucket-overflow-sum M (bp-lazy-insert-bucket M p bs) ≤
 5 + 4 * bp-bucket-overflow-sum M bs
using sizes
proof (induction bs arbitrary: p)

```

case Nil
have result: bp-lazy-insert-bucket M p [] = bp-bucketize-entries M [p]
  by simp
have sizes-result:
   $\forall c \in \text{set } (\text{bp-bucketize-entries } M [p]).$ 
     $\text{length } (\text{bp-bucket-entries } c) \leq M$ 
  by (rule bp-bucketize-entries-sizes-ok[OF M-pos])
have overflow-zero:
  bp-bucket-overflow-sum M (bp-bucketize-entries M [p]) = 0
  by (rule bp-bucket-overflow-sum-zero-if-sizes-ok[OF sizes-result])
show ?case
  unfolding result overflow-zero by simp
next
case (Cons b bs)
note IH = Cons.IH
note sizes = Cons.prem
have b-size:  $\text{length } (\text{bp-bucket-entries } b) \leq 2 * M$ 
  using sizes by simp
show ?case
proof (cases bs)
  case Nil
  show ?thesis
  proof (cases snd p < bp-marker b)
  case True
  have result:
    bp-lazy-insert-bucket M p (b # bs) =
      bp-bucketize-entries M [p] @ [b]
    using Nil True by simp
  have charge: bp-lazy-insert-bucket-charge M p (b # bs) = 1
    using Nil True by simp
  have sizes-single:
     $\forall c \in \text{set } (\text{bp-bucketize-entries } M [p]).$ 
       $\text{length } (\text{bp-bucket-entries } c) \leq M$ 
    by (rule bp-bucketize-entries-sizes-ok[OF M-pos])
  have overflow-single:
    bp-bucket-overflow-sum M (bp-bucketize-entries M [p]) = 0
    by (rule bp-bucket-overflow-sum-zero-if-sizes-ok[OF sizes-single])
  show ?thesis
    using Nil True overflow-single unfolding result charge by simp
  next
  case False
  have result:
    bp-lazy-insert-bucket M p (b # bs) =
      bp-lazy-bucket-insert-entries M p b
    using Nil False by simp
  have charge:
    bp-lazy-insert-bucket-charge M p (b # bs) =
      bp-lazy-bucket-insert-charge M b
    using Nil False by simp

```

```

show ?thesis
  unfolding result charge using Nil
  by (simp add: bp-lazy-bucket-insert-entries-weighted-overflow-credit
    [OF M-pos b-size, of p])
qed
next
case (Cons c cs)
have tail-sizes:
   $\forall b \in \text{set } bs. \text{length } (bp\text{-bucket-entries } b) \leq 2 * M$ 
  using sizes by simp
show ?thesis
proof (cases snd p < bp-marker b)
case True
  have result:
     $bp\text{-lazy-insert-bucket } M \ p \ (b \# \text{bs}) =$ 
     $bp\text{-bucketize-entries } M \ [p] \ @ \ b \# \text{bs}$ 
    using Cons True by simp
  have charge:  $bp\text{-lazy-insert-bucket-charge } M \ p \ (b \# \text{bs}) = 1$ 
    using Cons True by simp
  have sizes-single:
     $\forall c \in \text{set } (bp\text{-bucketize-entries } M \ [p]).$ 
     $\text{length } (bp\text{-bucket-entries } c) \leq M$ 
    by (rule bp-bucketize-entries-sizes-ok[OF M-pos])
  have overflow-single:
     $bp\text{-bucket-overflow-sum } M \ (bp\text{-bucketize-entries } M \ [p]) = 0$ 
    by (rule bp-bucket-overflow-sum-zero-if-sizes-ok[OF sizes-single])
  show ?thesis
    using Cons True overflow-single unfolding result charge by simp
next
case False
show ?thesis
proof (cases snd p < bp-marker c)
case True
  have result:
     $bp\text{-lazy-insert-bucket } M \ p \ (b \# \text{bs}) =$ 
     $bp\text{-lazy-bucket-insert-entries } M \ p \ b \ @ \ \text{bs}$ 
    using Cons False True by simp
  have charge:
     $bp\text{-lazy-insert-bucket-charge } M \ p \ (b \# \text{bs}) =$ 
     $bp\text{-lazy-bucket-insert-charge } M \ b$ 
    using Cons False True by simp
  have bucket-credit:
     $bp\text{-lazy-bucket-insert-charge } M \ b +$ 
     $4 * bp\text{-bucket-overflow-sum } M$ 
     $(bp\text{-lazy-bucket-insert-entries } M \ p \ b) \leq$ 
     $5 + 4 * bp\text{-bucket-overflow } M \ b$ 
    by (rule bp-lazy-bucket-insert-entries-weighted-overflow-credit
      [OF M-pos b-size])
  show ?thesis

```

```

    using bucket-credit unfolding result charge
    by simp
  next
  case False
  have result:
    bp-lazy-insert-bucket M p (b # bs) =
      b # bp-lazy-insert-bucket M p bs
    using Cons ⟨¬ snd p < bp-marker b⟩ False by simp
  have charge:
    bp-lazy-insert-bucket-charge M p (b # bs) =
      bp-lazy-insert-bucket-charge M p bs
    using Cons ⟨¬ snd p < bp-marker b⟩ False by simp
  have tail-credit:
    bp-lazy-insert-bucket-charge M p bs +
      4 * bp-bucket-overflow-sum M
      (bp-lazy-insert-bucket M p bs) ≤
      5 + 4 * bp-bucket-overflow-sum M bs
    by (rule IH[OF tail-sizes])
  show ?thesis
    using tail-credit unfolding result charge by simp
qed
qed
qed
qed

lemma bp-delete-key-from-bucket-overflow-le:
  bp-bucket-overflow M (bp-delete-key-from-bucket x b) ≤
  bp-bucket-overflow M b
proof -
  have len-le:
    length (bp-bucket-entries (bp-delete-key-from-bucket x b)) ≤
    length (bp-bucket-entries b)
  unfolding bp-delete-key-from-bucket-def by simp
  show ?thesis
    using len-le unfolding bp-bucket-overflow-def by simp
qed

lemma bp-bucket-overflow-sum-map-delete-key-le:
  bp-bucket-overflow-sum M (map (bp-delete-key-from-bucket x) bs) ≤
  bp-bucket-overflow-sum M bs
proof (induction bs)
  case Nil
  then show ?case by simp
next
  case (Cons b bs)
  have head:
    bp-bucket-overflow M (bp-delete-key-from-bucket x b) ≤
    bp-bucket-overflow M b
  by (rule bp-delete-key-from-bucket-overflow-le)

```

show ?case
 using head Cons.IH **by** simp
qed

lemma bp-delete-key-overflow-potential-le:
 $bp\text{-}overflow\text{-}potential\ (bp\text{-}delete\text{-}key\ x\ P) \leq bp\text{-}overflow\text{-}potential\ P$
proof –
 have bp-bucket-overflow-sum (bp-block-size P)
 (map (bp-delete-key-from-bucket x) (bp-buckets P)) $\leq$
 bp-bucket-overflow-sum (bp-block-size P) (bp-buckets P)
 by (rule bp-bucket-overflow-sum-map-delete-key-le)
 then show ?thesis
 unfolding bp-delete-key-def bp-overflow-potential-alt **by** simp
qed

lemma bp-delete-keys-from-bucket-overflow-le:
 $bp\text{-}bucket\text{-}overflow\ M\ (bp\text{-}delete\text{-}keys\text{-}from\text{-}bucket\ S\ b) \leq$
 $bp\text{-}bucket\text{-}overflow\ M\ b$
proof –
 have len-le:
 length (bp-bucket-entries (bp-delete-keys-from-bucket S b)) $\leq$
 length (bp-bucket-entries b)
 unfolding bp-delete-keys-from-bucket-def **by** simp
 show ?thesis
 using len-le unfolding bp-bucket-overflow-def **by** simp
qed

lemma bp-bucket-overflow-sum-map-delete-keys-le:
 $bp\text{-}bucket\text{-}overflow\text{-}sum\ M\ (map\ (bp\text{-}delete\text{-}keys\text{-}from\text{-}bucket\ S)\ bs) \leq$
 $bp\text{-}bucket\text{-}overflow\text{-}sum\ M\ bs$
proof (induction bs)
 case Nil
 then show ?case **by** simp
next
 case (Cons b bs)
 have head:
 $bp\text{-}bucket\text{-}overflow\ M\ (bp\text{-}delete\text{-}keys\text{-}from\text{-}bucket\ S\ b) \leq$
 $bp\text{-}bucket\text{-}overflow\ M\ b$
 by (rule bp-delete-keys-from-bucket-overflow-le)
 show ?case
 using head Cons.IH **by** simp
qed

lemma bp-delete-keys-overflow-potential-le:
 $bp\text{-}overflow\text{-}potential\ (bp\text{-}delete\text{-}keys\ S\ P) \leq bp\text{-}overflow\text{-}potential\ P$
proof –
 have bp-bucket-overflow-sum (bp-block-size P)
 (map (bp-delete-keys-from-bucket S) (bp-buckets P)) $\leq$
 bp-bucket-overflow-sum (bp-block-size P) (bp-buckets P)

by (rule bp-bucket-overflow-sum-map-delete-keys-le)
 then show ?thesis
 unfolding bp-delete-keys-def bp-overflow-potential-alt by simp
 qed

lemma *length-filter-fst-notin-le-card*:
 assumes *distinct*: *distinct* (map fst *xs*)
 and *finite*: *finite* *S*
 shows $\text{length } xs \leq \text{length } (\text{filter } (\lambda p. \text{fst } p \notin S) \text{ } xs) + \text{card } S$
proof –
 let ?*removed* = *filter* ($\lambda p. \text{fst } p \in S$) *xs*
 have *len-part*:
 $\text{length } xs = \text{length } (\text{filter } (\lambda p. \text{fst } p \notin S) \text{ } xs) + \text{length } ?\text{removed}$
 by (*induction* *xs*) *auto*
 have *distinct-removed*: *distinct* (map fst ?*removed*)
 using *distinct* by (*induction* *xs*) *auto*
 have $\text{length } ?\text{removed} = \text{card } (\text{set } (\text{map fst } ?\text{removed}))$
 using *distinct-card*[*OF distinct-removed*] by simp
 also have $\dots \leq \text{card } S$
 using *finite* by (*intro card-mono*) *auto*
 finally have $\text{length } ?\text{removed} \leq \text{card } S$.
 then show ?thesis
 using *len-part* by *linarith*
 qed

lemma *div-add-right-le-div-plus*:
 fixes *n k M* :: *nat*
 assumes *M-pos*: $0 < M$
 shows $(n + k) \text{ div } M \leq n \text{ div } M + k$
proof –
 have *n-less*: $n < \text{Suc } (n \text{ div } M) * M$
proof –
 have $n \text{ div } M < \text{Suc } (n \text{ div } M)$
 by *simp*
 moreover have
 $n \text{ div } M < \text{Suc } (n \text{ div } M) \longleftrightarrow n < \text{Suc } (n \text{ div } M) * M$
 by (rule *div-less-iff-less-mult*[*OF M-pos*])
 then show ?thesis
 using *calculation* by *simp*
 qed
 have *k-le*: $k \leq k * M$
 using *M-pos* by (*cases* *M*) *auto*
 have $n + k < \text{Suc } (n \text{ div } M) * M + k * M$
 using *n-less k-le* by *linarith*
 also have $\dots = (\text{Suc } (n \text{ div } M) + k) * M$
 by (*simp add: algebra-simps*)
 finally have $(n + k) \text{ div } M < \text{Suc } (n \text{ div } M) + k$
 using *M-pos* by (*simp add: div-less-iff-less-mult*)
 then show ?thesis by *simp*

qed

lemma *div-bound-after-delete*:
assumes $M\text{-pos}$: $0 < M$
and len : $n \leq n' + k$
shows $Suc\ (n\ div\ M) \leq Suc\ (n'\ div\ M) + k$
proof –
have $n\ div\ M \leq (n' + k)\ div\ M$
by (*rule div-le-mono*[*OF len*])
also have $\dots \leq n'\ div\ M + k$
by (*rule div-add-right-le-div-plus*[*OF M-pos*])
finally show *?thesis* **by** *simp*
qed

lemma *nat-diff-le-diff-add-right*:
fixes $a\ c\ d\ k :: nat$
assumes $c \leq d + k$
shows $a - d \leq (a - c) + k$
proof –
have *base*: $a - d \leq (a - c) + (c - d)$
by *simp*
have $c - d \leq k$
using *assms* **by** *simp*
then show *?thesis*
using *base* **by** *linarith*
qed

lemma *nat-diff-add-right-le*:
fixes $a\ b\ c\ d\ k :: nat$
assumes $a\text{-le}$: $a \leq k + b$
and $d\text{-le-c}$: $d \leq c$
shows $a - c \leq k + (b - d)$
proof –
have $a - c \leq (k + b) - c$
using $a\text{-le}$ **by** *simp*
also have $\dots \leq k + (b - c)$
by *arith*
also have $\dots \leq k + (b - d)$
using $d\text{-le-c}$ **by** *simp*
finally show *?thesis* .
qed

lemma *bp-delete-keys-entries-length-le*:
assumes *distinct*: *bp-distinct-keys* P
and *finite*: *finite* S
shows $length\ (bp\text{-entries}\ P) \leq$
 $length\ (bp\text{-entries}\ (bp\text{-delete-keys}\ S\ P)) + card\ S$
using *length-filter-fst-notin-le-card*[*OF - finite, of bp-entries P*]
distinct

unfolding *bp-entries-delete-keys bp-distinct-keys-def* **by** *simp*

lemma *bp-delete-keys-bucket-count-slack-le*:
assumes *M-pos*: $0 < \text{bp-block-size } P$
and *distinct*: *bp-distinct-keys* *P*
and *finite*: *finite* *S*
shows $\text{bp-bucket-count-slack } (\text{bp-delete-keys } S \ P) \leq$
 $\text{bp-bucket-count-slack } P + \text{card } S$
proof –
let $?M = \text{bp-block-size } P$
let $?N = \text{length } (\text{bp-entries } P)$
let $?N' = \text{length } (\text{bp-entries } (\text{bp-delete-keys } S \ P))$
let $?K = \text{length } (\text{bp-buckets } P)$
have *len*: $?N \leq ?N' + \text{card } S$
by (*rule* *bp-delete-keys-entries-length-le*[*OF distinct finite*])
have *divs*: $\text{Suc } (?N \text{ div } ?M) \leq \text{Suc } (?N' \text{ div } ?M) + \text{card } S$
by (*rule* *div-bound-after-delete*[*OF M-pos len*])
have *slack*:
 $?K - \text{Suc } (?N' \text{ div } ?M) \leq (?K - \text{Suc } (?N \text{ div } ?M)) + \text{card } S$
by (*rule* *nat-diff-le-diff-add-right*[*OF divs*])
show *?thesis*
using *slack unfolding bp-bucket-count-slack-def bp-delete-keys-def* **by** *simp*
qed

lemma *bp-delete-keys-split-potential-le*:
assumes *M-pos*: $0 < \text{bp-block-size } P$
and *distinct*: *bp-distinct-keys* *P*
and *finite*: *finite* *S*
shows $\text{bp-split-potential } (\text{bp-delete-keys } S \ P) \leq$
 $\text{bp-split-potential } P + \text{card } S$
proof –
have *overflow*:
 $\text{bp-overflow-potential } (\text{bp-delete-keys } S \ P) \leq \text{bp-overflow-potential } P$
by (*rule* *bp-delete-keys-overflow-potential-le*)
have *slack*:
 $\text{bp-bucket-count-slack } (\text{bp-delete-keys } S \ P) \leq$
 $\text{bp-bucket-count-slack } P + \text{card } S$
by (*rule* *bp-delete-keys-bucket-count-slack-le*[*OF M-pos distinct finite*])
show *?thesis*
using *overflow slack unfolding bp-split-potential-def* **by** *linarith*
qed

lemma *bp-lazy-insert-state-weighted-overflow-credit-after-delete*:
assumes *lazy*: *bp-lazy-ordered-invariant* *P*
shows $\text{bp-lazy-insert-charge } x \ b \ P +$
 $4 * \text{bp-overflow-potential } (\text{bp-lazy-insert-state } x \ b \ P) \leq$
 $5 + 4 * \text{bp-overflow-potential } (\text{bp-delete-key } x \ P)$
proof –
let $?old\text{-keys} = \text{bp-entry-keys } (\text{bp-entries } P)$


```

let ?b = if x ∈ ?old-keys then min (bp-values P x) b else b
let ?P0 = bp-delete-key x P
let ?M = bp-block-size ?P0
let ?bs = bp-buckets ?P0
let ?new-bs = bp-lazy-insert-bucket ?M (x, ?b) ?bs
let ?P' = ?P0 (| bp-buckets := ?new-bs,
    bp-values := (bp-values P)(x := ?b) |)
have inv: bp-lazy-invariant P
  by (rule bp-lazy-ordered-invariant-lazy-invariant[OF lazy])
have inv0: bp-lazy-invariant ?P0
  by (rule bp-delete-key-lazy-invariant[OF inv])
have M-pos: 0 < ?M
  using inv0 unfolding bp-lazy-invariant-def by blast
have sizes0:
  ∀ c ∈ set ?bs. length (bp-bucket-entries c) ≤ 2 * ?M
  using inv0 unfolding bp-lazy-invariant-def
    bp-lazy-bucket-sizes-ok-def by blast
have list-credit:
  bp-lazy-insert-bucket-charge ?M (x, ?b) ?bs +
    4 * bp-bucket-overflow-sum ?M ?new-bs ≤
    5 + 4 * bp-bucket-overflow-sum ?M ?bs
  by (rule bp-lazy-insert-bucket-weighted-overflow-credit
    [OF M-pos sizes0])
have state: bp-lazy-insert-state x b P = ?P'
  unfolding bp-lazy-insert-state-def Let-def by simp
have charge:
  bp-lazy-insert-charge x b P =
    bp-lazy-insert-bucket-charge ?M (x, ?b) ?bs
  unfolding bp-lazy-insert-charge-def Let-def by simp
show ?thesis
  using list-credit
  unfolding state charge bp-overflow-potential-alt by simp
qed

```

```

lemma bp-lazy-insert-state-weighted-overflow-credit:
  assumes lazy: bp-lazy-ordered-invariant P
  shows bp-lazy-insert-charge x b P +
    4 * bp-overflow-potential (bp-lazy-insert-state x b P) ≤
    5 + 4 * bp-overflow-potential P
proof -
  have after-delete:
    bp-lazy-insert-charge x b P +
      4 * bp-overflow-potential (bp-lazy-insert-state x b P) ≤
      5 + 4 * bp-overflow-potential (bp-delete-key x P)
    by (rule bp-lazy-insert-state-weighted-overflow-credit-after-delete
      [OF lazy])
  have delete-le:
    bp-overflow-potential (bp-delete-key x P) ≤ bp-overflow-potential P
    by (rule bp-delete-key-overflow-potential-le)

```

```

show ?thesis
  using after-delete delete-le by linarith
qed

```

```

lemma nat-diff-le-diff-add:
  fixes a b c d k :: nat
  assumes a-le: a ≤ b + k
  and c-le: c ≤ d
  shows a - d ≤ (b - c) + k
proof -
  have a - d ≤ a - c
    using c-le by simp
  also have ... ≤ (b + k) - c
    using a-le by simp
  also have ... ≤ (b - c) + k
    by simp
  finally show ?thesis .
qed

```

```

lemma bp-lazy-insert-state-bucket-count-slack-le:
  assumes lazy: bp-lazy-ordered-invariant P
  shows bp-bucket-count-slack (bp-lazy-insert-state x b P) ≤
    bp-bucket-count-slack P + 2
proof -
  let ?P' = bp-lazy-insert-state x b P
  let ?M = bp-block-size P
  have inv: bp-lazy-invariant P
    by (rule bp-lazy-ordered-invariant-lazy-invariant[OF lazy])
  have M-pos: 0 < ?M
    using inv unfolding bp-lazy-invariant-def by blast
  have block: bp-block-size ?P' = ?M
    unfolding bp-lazy-insert-state-def bp-delete-key-def
    by (simp add: Let-def split: if-splits)
  have buckets:
    length (bp-buckets ?P') ≤ length (bp-buckets P) + 2
    by (rule length-bp-lazy-insert-state-buckets-le[OF lazy])
  have entries:
    length (bp-entries P) ≤ length (bp-entries ?P')
    by (rule length-bp-lazy-insert-state-entries-ge[OF lazy])
  have divs:
    Suc (length (bp-entries P) div ?M) ≤
      Suc (length (bp-entries ?P') div ?M)
    using entries by (simp add: div-le-mono)
  have slack:
    length (bp-buckets ?P') -
      Suc (length (bp-entries ?P') div ?M) ≤
      (length (bp-buckets P) -
        Suc (length (bp-entries P) div ?M)) + 2
    by (rule nat-diff-le-diff-add[OF buckets divs])

```

```

show ?thesis
  using slack unfolding bp-bucket-count-slack-def block by simp
qed

lemma bp-lazy-insert-state-split-potential-credit:
  assumes lazy: bp-lazy-ordered-invariant P
  shows bp-lazy-insert-charge x b P +
    bp-split-potential (bp-lazy-insert-state x b P) ≤
    7 + bp-split-potential P
proof –
  have overflow:
    bp-lazy-insert-charge x b P +
      4 * bp-overflow-potential (bp-lazy-insert-state x b P) ≤
      5 + 4 * bp-overflow-potential P
  by (rule bp-lazy-insert-state-weighted-overflow-credit[OF lazy])
  have slack:
    bp-bucket-count-slack (bp-lazy-insert-state x b P) ≤
    bp-bucket-count-slack P + 2
  by (rule bp-lazy-insert-state-bucket-count-slack-le[OF lazy])
  show ?thesis
    using overflow slack unfolding bp-split-potential-def by linarith
qed

```

The lazy Insert credit proof is the core amortized argument. A local insert either adds an entry to a bucket below the lazy threshold or splits the bucket into strict chunks. The weighted overflow lemma pays for the expensive case: any actual lazy-insert charge plus four times the post-state overflow is at most a constant plus four times the pre-state overflow. The bucket-count slack can increase by only a small constant. Combining those facts yields $bp\text{-}lazy\text{-}ordered\text{-}invariant\ ?P \implies bp\text{-}lazy\text{-}insert\text{-}charge\ ?x\ ?b\ ?P + bp\text{-}split\text{-}potential\ (bp\text{-}lazy\text{-}insert\text{-}state\ ?x\ ?b\ ?P) \leq 7 + bp\text{-}split\text{-}potential\ ?P$, which is the precise credit inequality used by the costed Insert operation.

```

lemma c-bp-rebucket-build-result [simp]:
  bp-result-of (c-bp-rebucket-build P) = bp-rebucket P
proof –
  obtain bs c where build:
    c-bp-bucketize-entries (bp-block-size P) (bp-entries P) = (bs, c)
  by (cases c-bp-bucketize-entries (bp-block-size P) (bp-entries P))
    simp
  have bs:
    bs = bp-bucketize-entries (bp-block-size P) (bp-entries P)
  using c-bp-bucketize-entries-result[of bp-block-size P bp-entries P]
    build
  by (simp add: bp-result-of-def)
  show ?thesis
    using build bs unfolding bp-result-of-def c-bp-rebucket-build-def
      bp-rebucket-def
    by simp

```

qed

lemma *c-bp-rebucket-build-steps-le*:

assumes *inv*: *bp-invariant P*

shows *bp-steps-of (c-bp-rebucket-build P) ≤*
length (bp-entries P) +
Suc (length (bp-entries P) div bp-block-size P)

proof –

have *M-pos*: *0 < bp-block-size P*

using *inv* **unfolding** *bp-invariant-def* **by** *blast*

have *steps*:

bp-steps-of

(c-bp-bucketize-entries (bp-block-size P) (bp-entries P)) ≤
length (bp-entries P) +
Suc (length (bp-entries P) div bp-block-size P)

by (rule *c-bp-bucketize-entries-steps-le[OF M-pos]*)

show *?thesis*

using *steps* **unfolding** *bp-steps-of-def c-bp-rebucket-build-def*

by (*auto split: prod.splits*)

qed

lemma *bp-bucket-overflow-zero*:

assumes *length (bp-bucket-entries b) ≤ M*

shows *bp-bucket-overflow M b = 0*

using *assms* **unfolding** *bp-bucket-overflow-def* **by** *simp*

lemma *bp-overflow-potential-zero-if-sizes-ok*:

assumes *bp-bucket-sizes-ok P*

shows *bp-overflow-potential P = 0*

proof –

have $\forall b \in \text{set } (bp\text{-buckets } P). \text{ bp-bucket-overflow } (bp\text{-block-size } P) \text{ } b = 0$

using *assms*

unfolding *bp-bucket-sizes-ok-def*

by (*simp add: bp-bucket-overflow-zero*)

then show *?thesis*

unfolding *bp-overflow-potential-def* **by** (*induction bp-buckets P*) *auto*

qed

lemma *bp-invariant-overflow-potential-zero*:

assumes *bp-invariant P*

shows *bp-overflow-potential P = 0*

by (rule *bp-overflow-potential-zero-if-sizes-ok*)

(*use assms in <simp add: bp-invariant-def>*)

lemma *bp-bucket-count-slack-zero-if-ratio-ok*:

assumes *bp-bucket-count-ratio-ok P*

shows *bp-bucket-count-slack P = 0*

using *assms* **unfolding** *bp-bucket-count-ratio-ok-def*

bp-bucket-count-slack-def **by** *simp*

```

lemma bp-split-potential-zero-if-regular:
  assumes inv: bp-invariant P
    and ratio: bp-bucket-count-ratio-ok P
  shows bp-split-potential P = 0
  using bp-invariant-overflow-potential-zero[OF inv]
    bp-bucket-count-slack-zero-if-ratio-ok[OF ratio]
  unfolding bp-split-potential-def by simp

lemma bp-bucket-search-steps-ratio-bound:
  assumes ratio: bp-bucket-count-ratio-ok P
  shows bp-bucket-search-steps P ≤
    bp-ratio-log-budget (length (bp-entries P)) (bp-block-size P)
proof –
  have count-le:
    length (bp-buckets P) ≤
      Suc (length (bp-entries P) div bp-block-size P)
    using ratio unfolding bp-bucket-count-ratio-ok-def by blast
  have floor-log (length (bp-buckets P)) ≤
    floor-log (Suc (length (bp-entries P) div bp-block-size P))
    by (rule floor-log-le-iff[OF count-le])
  then show ?thesis
    unfolding bp-bucket-search-steps-def bp-ratio-log-budget-def by simp
qed

lemma bp-bucket-count-ratio-okI-loaded:
  assumes M-pos: 0 < bp-block-size P
    and loaded: bp-bucket-count-loaded P
  shows bp-bucket-count-ratio-ok P
proof –
  let ?M = bp-block-size P
  let ?k = length (bp-buckets P)
  let ?N = length (bp-entries P)
  have count-le: ?k ≤ Suc (?N div ?M)
  proof (cases ?k)
    case 0
      then show ?thesis by simp
    next
      case (Suc j)
        have ?M * j ≤ ?N
        using loaded Suc unfolding bp-bucket-count-loaded-def by simp
        then have j ≤ ?N div ?M
        proof –
          have j = (?M * j) div ?M
          using M-pos by simp
          also have ... ≤ ?N div ?M
          by (rule div-le-mono) (rule ⟨?M * j ≤ ?N⟩)
          finally show ?thesis .
        qed
  qed

```

```

    then show ?thesis
      using Suc by simp
  qed
  show ?thesis
    unfolding bp-bucket-count-ratio-ok-def using M-pos count-le by blast
  qed

```

```

lemma bp-empty-bucket-count-ratio-ok:
  assumes 0 < M
  shows bp-bucket-count-ratio-ok (bp-empty M B)
  using assms unfolding bp-bucket-count-ratio-ok-def bp-empty-def
    bp-entries-def bp-bucket-entries-flat-def by simp

```

```

lemma bp-regular-invariant-ordered-invariant:
  assumes bp-regular-invariant P
  shows bp-ordered-invariant P
  using assms unfolding bp-regular-invariant-def by blast

```

```

lemma bp-regular-invariant-ratio-ok:
  assumes bp-regular-invariant P
  shows bp-bucket-count-ratio-ok P
  using assms unfolding bp-regular-invariant-def by blast

```

```

lemma bp-empty-regular-invariant:
  assumes 0 < M
  shows bp-regular-invariant (bp-empty M B)
  unfolding bp-regular-invariant-def
  using bp-empty-ordered-invariant[OF assms]
    bp-empty-bucket-count-ratio-ok[OF assms]
  by blast

```

```

lemma bp-rebucket-bucket-count-ratio-ok:
  assumes inv: bp-invariant P
  shows bp-bucket-count-ratio-ok (bp-rebucket P)
proof -
  let ?M = bp-block-size P
  let ?xs = bp-entries P
  have M-pos: 0 < ?M
    using inv unfolding bp-invariant-def by blast
  have count:
    length (bp-bucketize-entries ?M ?xs) ≤
      Suc (length ?xs div ?M)
    by (rule length-bp-bucketize-entries-le-ratio[OF M-pos])
  have entries-len:
    length (bp-entries (bp-rebucket P)) = length ?xs
    using bp-bucket-entries-flat-bucketize-entries[OF M-pos, of ?xs]
    unfolding bp-rebucket-def bp-entries-def by simp
  have block-eq [simp]: bp-block-size (bp-rebucket P) = ?M
    unfolding bp-rebucket-def by simp

```

have *buckets-eq* [*simp*]:
bp-buckets (*bp-rebucket* *P*) = *bp-bucketize-entries* ?*M* ?*xs*
unfolding *bp-rebucket-def* **by** *simp*
show ?*thesis*
unfolding *bp-bucket-count-ratio-ok-def*
using *M-pos* *count* *entries-len* **by** *simp*
qed

lemma *bp-rebucket-bucket-count-ratio-ok-from-lazy*:
assumes *inv*: *bp-lazy-invariant* *P*
shows *bp-bucket-count-ratio-ok* (*bp-rebucket* *P*)
proof –
let ?*M* = *bp-block-size* *P*
let ?*xs* = *bp-entries* *P*
have *M-pos*: $0 < ?M$
using *inv* **unfolding** *bp-lazy-invariant-def* **by** *blast*
have *count*:
length (*bp-bucketize-entries* ?*M* ?*xs*) $\leq$
Suc (*length* ?*xs* *div* ?*M*)
by (*rule* *length-bp-bucketize-entries-le-ratio*[*OF* *M-pos*])
have *entries-len*:
length (*bp-entries* (*bp-rebucket* *P*)) = *length* ?*xs*
using *bp-bucket-entries-flat-bucketize-entries*[*OF* *M-pos*, *of* ?*xs*]
unfolding *bp-rebucket-def* *bp-entries-def* **by** *simp*
have *block-eq* [*simp*]: *bp-block-size* (*bp-rebucket* *P*) = ?*M*
unfolding *bp-rebucket-def* **by** *simp*
have *buckets-eq* [*simp*]:
bp-buckets (*bp-rebucket* *P*) = *bp-bucketize-entries* ?*M* ?*xs*
unfolding *bp-rebucket-def* **by** *simp*
show ?*thesis*
unfolding *bp-bucket-count-ratio-ok-def*
using *M-pos* *count* *entries-len* **by** *simp*
qed

lemma *bp-rebucket-regular-split-potential-zero*:
assumes *inv*: *bp-invariant* *P*
shows *bp-split-potential* (*bp-rebucket* *P*) = 0
proof (*rule* *bp-split-potential-zero-if-regular*)
show *bp-invariant* (*bp-rebucket* *P*)
by (*rule* *bp-rebucket-invariant*[*OF* *inv*])
show *bp-bucket-count-ratio-ok* (*bp-rebucket* *P*)
by (*rule* *bp-rebucket-bucket-count-ratio-ok*[*OF* *inv*])
qed

lemma *bp-rebucket-regular-invariant*:
assumes *inv*: *bp-invariant* *P*
shows *bp-regular-invariant* (*bp-rebucket* *P*)
unfolding *bp-regular-invariant-def*
using *bp-rebucket-ordered-invariant*[*OF* *inv*]

$bp_rebucket_bucket_count_ratio_ok[OF\ inv]$
by *blast*

lemma *bp-rebucket-regular-split-potential-zero-from-lazy*:
assumes *inv*: *bp-lazy-invariant P*
shows *bp-split-potential (bp-rebucket P) = 0*
proof (*rule bp-split-potential-zero-if-regular*)
show *bp-invariant (bp-rebucket P)*
by (*rule bp-rebucket-invariant-from-lazy[OF inv]*)
show *bp-bucket-count-ratio-ok (bp-rebucket P)*
by (*rule bp-rebucket-bucket-count-ratio-ok-from-lazy[OF inv]*)
qed

lemma *bp-rebucket-regular-invariant-from-lazy*:
assumes *inv*: *bp-lazy-invariant P*
shows *bp-regular-invariant (bp-rebucket P)*
unfolding *bp-regular-invariant-def*
using *bp-rebucket-ordered-invariant-from-lazy[OF inv]*
bp-rebucket-bucket-count-ratio-ok-from-lazy[OF inv]
by *blast*

lemma *c-bp-rebucket-build-steps-le-from-lazy*:
assumes *inv*: *bp-lazy-invariant P*
shows *bp-steps-of (c-bp-rebucket-build P) ≤*
length (bp-entries P) +
Suc (length (bp-entries P) div bp-block-size P)
proof –
have *M-pos*: *0 < bp-block-size P*
using *inv* **unfolding** *bp-lazy-invariant-def* **by** *blast*
have *steps*:
bp-steps-of
(c-bp-bucketize-entries (bp-block-size P) (bp-entries P)) ≤
length (bp-entries P) +
Suc (length (bp-entries P) div bp-block-size P)
by (*rule c-bp-bucketize-entries-steps-le[OF M-pos]*)
show *?thesis*
using *steps* **unfolding** *bp-steps-of-def c-bp-rebucket-build-def*
by (*auto split: prod.splits*)
qed

lemma *c-bp-rebucket-build-regular-invariant-from-lazy*:
assumes *inv*: *bp-lazy-invariant P*
shows *bp-regular-invariant (bp-result-of (c-bp-rebucket-build P))*
by (*simp add: bp-rebucket-regular-invariant-from-lazy[OF inv]*)

lemma *c-bp-rebucket-build-split-potential-zero-from-lazy*:
assumes *inv*: *bp-lazy-invariant P*
shows *bp-split-potential (bp-result-of (c-bp-rebucket-build P)) = 0*
by (*simp add: bp-rebucket-regular-split-potential-zero-from-lazy[OF inv]*)

lemma *c-bp-rebucket-build-regular-invariant*:
assumes *inv*: *bp-invariant P*
shows *bp-regular-invariant (bp-result-of (c-bp-rebucket-build P))*
by (*simp add: bp-rebucket-regular-invariant[OF inv]*)

lemma *c-bp-rebucket-build-split-potential-zero*:
assumes *inv*: *bp-invariant P*
shows *bp-split-potential (bp-result-of (c-bp-rebucket-build P)) = 0*
by (*simp add: bp-rebucket-regular-split-potential-zero[OF inv]*)

lemma *bp-rebucketed-insert-regular-invariant*:
assumes *inv*: *bp-invariant P*
shows *bp-regular-invariant (bp-rebucketed-insert x b P)*
unfolding *bp-rebucketed-insert-def*
by (*rule bp-rebucket-regular-invariant[OF bp-insert-invariant[OF inv]]*)

lemma *bp-rebucketed-batch-prepend-regular-invariant*:
assumes *inv*: *bp-invariant P*
shows *bp-regular-invariant (bp-rebucketed-batch-prepend xs P)*
unfolding *bp-rebucketed-batch-prepend-def*
by (*rule bp-rebucket-regular-invariant[OF bp-batch-prepend-invariant[OF inv]]*)

lemma *bp-rebucketed-insert-split-potential-zero*:
assumes *inv*: *bp-invariant P*
shows *bp-split-potential (bp-rebucketed-insert x b P) = 0*
unfolding *bp-rebucketed-insert-def*
by (*rule bp-rebucket-regular-split-potential-zero*
[OF bp-insert-invariant[OF inv]])

lemma *bp-rebucketed-batch-prepend-split-potential-zero*:
assumes *inv*: *bp-invariant P*
shows *bp-split-potential (bp-rebucketed-batch-prepend xs P) = 0*
unfolding *bp-rebucketed-batch-prepend-def*
by (*rule bp-rebucket-regular-split-potential-zero*
[OF bp-batch-prepend-invariant[OF inv]])

lemma *bp-regularized-local-insert-invariant*:
assumes *ord*: *bp-ordered-invariant P*
shows *bp-invariant (bp-regularized-local-insert x b P)*
unfolding *bp-regularized-local-insert-def*
by (*rule bp-rebucket-invariant[OF bp-local-insert-state-invariant[OF ord]]*)

lemma *bp-regularized-local-insert-ordered-invariant*:
assumes *ord*: *bp-ordered-invariant P*
shows *bp-ordered-invariant (bp-regularized-local-insert x b P)*
unfolding *bp-regularized-local-insert-def*
by (*rule bp-rebucket-ordered-invariant*
[OF bp-local-insert-state-invariant[OF ord]])

lemma *bp-regularized-local-insert-regular-invariant*:
assumes *ord*: *bp-ordered-invariant P*
shows *bp-regular-invariant (bp-regularized-local-insert x b P)*
unfolding *bp-regularized-local-insert-def*
by (*rule bp-rebucket-regular-invariant*
 $[OF\ bp\text{-local-insert-state-invariant}[OF\ ord]])$

lemma *bp-regularized-local-insert-refines-min-update*:
assumes *ord*: *bp-ordered-invariant P*
shows *bp-view (bp-regularized-local-insert x b P) =*
 $\text{min-update (bp-view P) } x\ b$
proof –
have *inv*: *bp-invariant P*
by (*rule bp-ordered-invariant-invariant*[*OF ord*])
then have *M-pos*: $0 < bp\text{-block-size } P$
unfolding *bp-invariant-def* **by** *blast*
have *step-inv*: *bp-invariant (bp-local-insert-state x b P)*
by (*rule bp-local-insert-state-invariant*[*OF ord*])
then have *step-M-pos*:
 $0 < bp\text{-block-size (bp-local-insert-state } x\ b\ P)$
unfolding *bp-invariant-def* **by** *blast*
have *view-rebucket*:
 $bp\text{-view (bp-rebucket (bp-local-insert-state } x\ b\ P)) =$
 $bp\text{-view (bp-local-insert-state } x\ b\ P)$
by (*rule bp-rebucket-view*[*OF step-M-pos*])
show *?thesis*
unfolding *bp-regularized-local-insert-def*
using *view-rebucket bp-local-insert-state-refines-min-update*[*OF M-pos, of x b*]
by *simp*
qed

lemma *bp-regularized-local-insert-refines-insert-spec*:
assumes *ord*: *bp-ordered-invariant P*
shows *insert-spec (bp-view P) x b*
 $(bp\text{-view (bp-regularized-local-insert } x\ b\ P))$
unfolding *bp-regularized-local-insert-refines-min-update*[*OF ord*]
by (*rule min-update-insert-spec*)

lemma *bp-regularized-local-insert-split-potential-zero*:
assumes *ord*: *bp-ordered-invariant P*
shows *bp-split-potential (bp-regularized-local-insert x b P) = 0*
unfolding *bp-regularized-local-insert-def*
by (*rule bp-rebucket-regular-split-potential-zero*
 $[OF\ bp\text{-local-insert-state-invariant}[OF\ ord]])$

lemma *bp-local-insert-search-charge-ratio-bound*:
assumes *ratio*: *bp-bucket-count-ratio-ok P*
shows *bp-local-insert-search-charge P* $\leq$

bp-insert-search-budget (*length* (*bp-entries* *P*)) (*bp-block-size* *P*)
using *bp-bucket-search-steps-ratio-bound*[*OF ratio*]
unfolding *bp-local-insert-search-charge-def* *bp-insert-search-budget-def*
by *simp*

lemma *c-bp-local-insert-search-result* [*simp*]:
bp-result-of (*c-bp-local-insert-search* *x b P*) =
bp-local-insert-state *x b P*
unfolding *bp-result-of-def* *c-bp-local-insert-search-def* **by** *simp*

lemma *c-bp-local-insert-search-steps* [*simp*]:
bp-steps-of (*c-bp-local-insert-search* *x b P*) =
bp-local-insert-search-charge *P*
unfolding *bp-steps-of-def* *c-bp-local-insert-search-def* **by** *simp*

lemma *c-bp-local-insert-search-refines-min-update*:
assumes $0 < \text{bp-block-size } P$
shows *bp-view* (*bp-result-of* (*c-bp-local-insert-search* *x b P*)) =
min-update (*bp-view* *P*) *x b*
unfolding *c-bp-local-insert-search-result*
by (*rule* *bp-local-insert-state-refines-min-update*[*OF assms*])

lemma *c-bp-local-insert-search-invariant*:
assumes *bp-ordered-invariant* *P*
shows *bp-invariant* (*bp-result-of* (*c-bp-local-insert-search* *x b P*))
unfolding *c-bp-local-insert-search-result*
by (*rule* *bp-local-insert-state-invariant*[*OF assms*])

lemma *c-bp-local-insert-search-ordered-invariant*:
assumes *bp-ordered-invariant* *P*
shows *bp-ordered-invariant*
(bp-result-of (c-bp-local-insert-search x b P))
unfolding *c-bp-local-insert-search-result*
by (*rule* *bp-local-insert-state-ordered-invariant*[*OF assms*])

lemma *c-bp-local-insert-search-refines-insert-spec*:
assumes $0 < \text{bp-block-size } P$
shows *insert-spec* (*bp-view* *P*) *x b*
(bp-view (bp-result-of (c-bp-local-insert-search x b P)))
unfolding *c-bp-local-insert-search-refines-min-update*[*OF assms*]
by (*rule* *min-update-insert-spec*)

lemma *c-bp-local-insert-search-ratio-bound*:
assumes *ratio: bp-bucket-count-ratio-ok* *P*
shows *bp-steps-of* (*c-bp-local-insert-search* *x b P*) $\leq$
bp-insert-search-budget (*length* (*bp-entries* *P*)) (*bp-block-size* *P*)
by (*simp* *add: bp-local-insert-search-charge-ratio-bound*[*OF ratio*])

lemma *c-bp-local-insert-search-partition-insert-cost-bound*:

assumes *ratio*: *bp-bucket-count-ratio-ok* *P*
shows *partition-insert-cost-bound*
 $(bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}local\text{-}insert\text{-}search\ x\ b\ P))$
 $(bp\text{-}insert\text{-}search\text{-}budget\ (length\ (bp\text{-}entries\ P))\ (bp\text{-}block\text{-}size\ P))$
using *c-bp-local-insert-search-ratio-bound*[*OF ratio*]
unfolding *partition-insert-cost-bound-def* .

lemma *c-bp-local-insert-split-result* [*simp*]:
 $bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}local\text{-}insert\text{-}split\ x\ b\ P) =$
 $bp\text{-}local\text{-}insert\text{-}state\ x\ b\ P$
unfolding *bp-result-of-def c-bp-local-insert-split-def* **by** *simp*

lemma *c-bp-local-insert-split-steps* [*simp*]:
 $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}local\text{-}insert\text{-}split\ x\ b\ P) =$
 $bp\text{-}local\text{-}insert\text{-}search\text{-}charge\ P +$
 $bp\text{-}local\text{-}split\text{-}budget\ (bp\text{-}block\text{-}size\ P)$
unfolding *bp-steps-of-def c-bp-local-insert-split-def* **by** *simp*

lemma *c-bp-local-insert-split-refines-min-update*:
assumes $0 < bp\text{-}block\text{-}size\ P$
shows $bp\text{-}view\ (bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}local\text{-}insert\text{-}split\ x\ b\ P)) =$
 $min\text{-}update\ (bp\text{-}view\ P)\ x\ b$
unfolding *c-bp-local-insert-split-result*
by (rule *bp-local-insert-state-refines-min-update*[*OF assms*])

lemma *c-bp-local-insert-split-invariant*:
assumes *bp-ordered-invariant* *P*
shows *bp-invariant* $(bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}local\text{-}insert\text{-}split\ x\ b\ P))$
unfolding *c-bp-local-insert-split-result*
by (rule *bp-local-insert-state-invariant*[*OF assms*])

lemma *c-bp-local-insert-split-ordered-invariant*:
assumes *bp-ordered-invariant* *P*
shows *bp-ordered-invariant*
 $(bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}local\text{-}insert\text{-}split\ x\ b\ P))$
unfolding *c-bp-local-insert-split-result*
by (rule *bp-local-insert-state-ordered-invariant*[*OF assms*])

lemma *c-bp-local-insert-split-refines-insert-spec*:
assumes $0 < bp\text{-}block\text{-}size\ P$
shows *insert-spec* $(bp\text{-}view\ P)\ x\ b$
 $(bp\text{-}view\ (bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}local\text{-}insert\text{-}split\ x\ b\ P)))$
unfolding *c-bp-local-insert-split-refines-min-update*[*OF assms*]
by (rule *min-update-insert-spec*)

lemma *c-bp-local-insert-split-steps-le*:
assumes *ratio*: *bp-bucket-count-ratio-ok* *P*
shows $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}local\text{-}insert\text{-}split\ x\ b\ P) \leq$
 $bp\text{-}insert\text{-}search\text{-}budget\ (length\ (bp\text{-}entries\ P))\ (bp\text{-}block\text{-}size\ P) +$

```

    bp-local-split-budget (bp-block-size P)
  using bp-local-insert-search-charge-ratio-bound[OF ratio]
  by simp

lemma c-bp-local-insert-split-steps-le-lazy-budget:
  assumes ratio: bp-bucket-count-ratio-ok P
  shows bp-steps-of (c-bp-local-insert-split x b P) ≤
    bp-insert-search-budget (length (bp-entries P)) (bp-block-size P) +
    bp-lazy-split-budget (bp-block-size P)
proof –
  have local:
    bp-steps-of (c-bp-local-insert-split x b P) ≤
      bp-insert-search-budget (length (bp-entries P)) (bp-block-size P) +
      bp-local-split-budget (bp-block-size P)
  by (rule c-bp-local-insert-split-steps-le[OF ratio])
  have bp-local-split-budget (bp-block-size P) ≤
    bp-lazy-split-budget (bp-block-size P)
  by (rule bp-local-split-budget-le-lazy-split-budget)
  then show ?thesis
  using local by linarith
qed

lemma c-bp-local-insert-split-amortized-if-split-credit:
  assumes ratio: bp-bucket-count-ratio-ok P
  and paid:
    bp-local-split-budget (bp-block-size P) +
    bp-split-potential
    (bp-result-of (c-bp-local-insert-split x b P)) ≤
    bp-split-potential P
  shows bp-amortized-step-bound
    (bp-steps-of (c-bp-local-insert-split x b P)) P
    (bp-result-of (c-bp-local-insert-split x b P))
    (bp-insert-search-budget (length (bp-entries P)) (bp-block-size P))
proof –
  have search:
    bp-local-insert-search-charge P ≤
      bp-insert-search-budget (length (bp-entries P)) (bp-block-size P)
  by (rule bp-local-insert-search-charge-ratio-bound[OF ratio])
  have steps:
    bp-steps-of (c-bp-local-insert-split x b P) =
      bp-local-insert-search-charge P +
      bp-local-split-budget (bp-block-size P)
  by simp
  show ?thesis
  using search paid steps
  unfolding bp-amortized-step-bound-def
  by linarith
qed

```

lemma *c-bp-lazy-insert-result* [simp]:
 $bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\ x\ b\ P) =$
 $bp\text{-}lazy\text{-}insert\text{-}state\ x\ b\ P$
unfolding *bp-result-of-def c-bp-lazy-insert-def* **by** *simp*

lemma *c-bp-lazy-insert-steps* [simp]:
 $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\ x\ b\ P) =$
 $bp\text{-}local\text{-}insert\text{-}search\text{-}charge\ P +$
 $bp\text{-}lazy\text{-}split\text{-}budget\ (bp\text{-}block\text{-}size\ P)$
unfolding *bp-steps-of-def c-bp-lazy-insert-def* **by** *simp*

lemma *c-bp-lazy-insert-refines-min-update*:
assumes $0 < bp\text{-}block\text{-}size\ P$
shows $bp\text{-}view\ (bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\ x\ b\ P)) =$
 $min\text{-}update\ (bp\text{-}view\ P)\ x\ b$
unfolding *c-bp-lazy-insert-result*
by (rule *bp-lazy-insert-state-refines-min-update*[*OF assms*])

lemma *c-bp-lazy-insert-lazy-ordered-invariant*:
assumes *bp-lazy-ordered-invariant* P
shows *bp-lazy-ordered-invariant*
 $(bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\ x\ b\ P))$
unfolding *c-bp-lazy-insert-result*
by (rule *bp-lazy-insert-state-lazy-ordered-invariant*[*OF assms*])

lemma *c-bp-lazy-insert-refines-insert-spec*:
assumes $0 < bp\text{-}block\text{-}size\ P$
shows *insert-spec* $(bp\text{-}view\ P)\ x\ b$
 $(bp\text{-}view\ (bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\ x\ b\ P)))$
unfolding *c-bp-lazy-insert-refines-min-update*[*OF assms*]
by (rule *min-update-insert-spec*)

lemma *c-bp-lazy-insert-steps-le-lazy-budget*:
assumes *ratio: bp-bucket-count-ratio-ok* P
shows $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\ x\ b\ P) \leq$
 $bp\text{-}insert\text{-}search\text{-}budget\ (length\ (bp\text{-}entries\ P))\ (bp\text{-}block\text{-}size\ P) +$
 $bp\text{-}lazy\text{-}split\text{-}budget\ (bp\text{-}block\text{-}size\ P)$
using *bp-local-insert-search-charge-ratio-bound*[*OF ratio*]
by *simp*

lemma *c-bp-lazy-insert-amortized-if-split-credit*:
assumes *ratio: bp-bucket-count-ratio-ok* P
and paid:
 $bp\text{-}lazy\text{-}split\text{-}budget\ (bp\text{-}block\text{-}size\ P) +$
 $bp\text{-}split\text{-}potential\ (bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\ x\ b\ P)) \leq$
 $bp\text{-}split\text{-}potential\ P$
shows *bp-amortized-step-bound*
 $(bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\ x\ b\ P))\ P$
 $(bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\ x\ b\ P))$

```

    (bp-insert-search-budget (length (bp-entries P)) (bp-block-size P))
  proof -
    have search:
      bp-local-insert-search-charge P ≤
        bp-insert-search-budget (length (bp-entries P)) (bp-block-size P)
    by (rule bp-local-insert-search-charge-ratio-bound[OF ratio])
    have steps:
      bp-steps-of (c-bp-lazy-insert x b P) =
        bp-local-insert-search-charge P +
        bp-lazy-split-budget (bp-block-size P)
    by simp
    show ?thesis
      using search paid steps
    unfolding bp-amortized-step-bound-def
    by linarith
  qed

lemma bp-lazy-bucket-insert-charge-le:
  bp-lazy-bucket-insert-charge M b ≤ bp-lazy-split-budget M
  unfolding bp-lazy-bucket-insert-charge-def bp-lazy-split-budget-def
  by simp

lemma bp-lazy-insert-bucket-charge-le:
  bp-lazy-insert-bucket-charge M p bs ≤ bp-lazy-split-budget M
  proof (induction bs arbitrary: p)
    case Nil
    then show ?case
      unfolding bp-lazy-split-budget-def by simp
  next
    case (Cons b bs)
    note IH = Cons.IH
    show ?case
    proof (cases bs)
      case Nil
      then show ?thesis
        using bp-lazy-bucket-insert-charge-le[of M b]
        unfolding bp-lazy-split-budget-def by auto
    next
      case (Cons c cs)
      have one-le: 1 ≤ bp-lazy-split-budget M
        unfolding bp-lazy-split-budget-def by simp
      then show ?thesis
        using Cons IH[of p] bp-lazy-bucket-insert-charge-le[of M b] one-le
        by auto
    qed
  qed
qed

lemma bp-lazy-insert-charge-le:
  bp-lazy-insert-charge x b P ≤ bp-lazy-split-budget (bp-block-size P)

```

proof –

let $?old\text{-}keys = bp\text{-}entry\text{-}keys\ (bp\text{-}entries\ P)$
let $?b = \text{if } x \in ?old\text{-}keys \text{ then } \min\ (bp\text{-}values\ P\ x)\ b \text{ else } b$
let $?P0 = bp\text{-}delete\text{-}key\ x\ P$
have *charge*:
 $bp\text{-}lazy\text{-}insert\text{-}bucket\text{-}charge\ (bp\text{-}block\text{-}size\ ?P0)\ (x, ?b)$
 $(bp\text{-}buckets\ ?P0) \leq bp\text{-}lazy\text{-}split\text{-}budget\ (bp\text{-}block\text{-}size\ ?P0)$
by $(rule\ bp\text{-}lazy\text{-}insert\text{-}bucket\text{-}charge\text{-}le)$
have *block*: $bp\text{-}block\text{-}size\ ?P0 = bp\text{-}block\text{-}size\ P$
unfolding $bp\text{-}delete\text{-}key\text{-}def$ **by** *simp*
show $?thesis$
using *charge* **unfolding** $bp\text{-}lazy\text{-}insert\text{-}charge\text{-}def$ *Let-def* *block* **by** *simp*
qed

lemma $c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\text{-}result\ [simp]$:
 $bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P) =$
 $bp\text{-}lazy\text{-}insert\text{-}state\ x\ b\ P$
unfolding $bp\text{-}result\text{-}of\text{-}def$ $c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\text{-}def$ **by** *simp*

lemma $c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\text{-}steps\ [simp]$:
 $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P) =$
 $bp\text{-}local\text{-}insert\text{-}search\text{-}charge\ P + bp\text{-}lazy\text{-}insert\text{-}charge\ x\ b\ P$
unfolding $bp\text{-}steps\text{-}of\text{-}def$ $c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\text{-}def$ **by** *simp*

lemma $c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\text{-}refines\text{-}min\text{-}update$:
assumes $0 < bp\text{-}block\text{-}size\ P$
shows $bp\text{-}view\ (bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P)) =$
 $min\text{-}update\ (bp\text{-}view\ P)\ x\ b$
unfolding $c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\text{-}result$
by $(rule\ bp\text{-}lazy\text{-}insert\text{-}state\text{-}refines\text{-}min\text{-}update[OF\ assms])$

lemma $c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\text{-}lazy\text{-}ordered\text{-}invariant$:
assumes $bp\text{-}lazy\text{-}ordered\text{-}invariant\ P$
shows $bp\text{-}lazy\text{-}ordered\text{-}invariant$
 $(bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P))$
unfolding $c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\text{-}result$
by $(rule\ bp\text{-}lazy\text{-}insert\text{-}state\text{-}lazy\text{-}ordered\text{-}invariant[OF\ assms])$

lemma $c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\text{-}refines\text{-}insert\text{-}spec$:
assumes $0 < bp\text{-}block\text{-}size\ P$
shows $insert\text{-}spec\ (bp\text{-}view\ P)\ x\ b$
 $(bp\text{-}view\ (bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P)))$
unfolding $c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\text{-}refines\text{-}min\text{-}update[OF\ assms]$
by $(rule\ min\text{-}update\text{-}insert\text{-}spec)$

lemma $c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\text{-}steps\text{-}le\text{-}lazy\text{-}budget$:
assumes $ratio: bp\text{-}bucket\text{-}count\text{-}ratio\text{-}ok\ P$
shows $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P) \leq$
 $bp\text{-}insert\text{-}search\text{-}budget\ (length\ (bp\text{-}entries\ P))\ (bp\text{-}block\text{-}size\ P) +$

$bp\text{-}lazy\text{-}split\text{-}budget\ (bp\text{-}block\text{-}size\ P)$
proof –
have *search*:
 $bp\text{-}local\text{-}insert\text{-}search\text{-}charge\ P \leq$
 $bp\text{-}insert\text{-}search\text{-}budget\ (length\ (bp\text{-}entries\ P))\ (bp\text{-}block\text{-}size\ P)$
by (rule $bp\text{-}local\text{-}insert\text{-}search\text{-}charge\text{-}ratio\text{-}bound[OF\ ratio]$)
have *charge*:
 $bp\text{-}lazy\text{-}insert\text{-}charge\ x\ b\ P \leq$
 $bp\text{-}lazy\text{-}split\text{-}budget\ (bp\text{-}block\text{-}size\ P)$
by (rule $bp\text{-}lazy\text{-}insert\text{-}charge\text{-}le$)
show *?thesis*
using *search charge* **by** *simp*
qed

lemma $c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\text{-}steps\text{-}le\text{-}coarse$:
 $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P) \leq$
 $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\ x\ b\ P)$
using $bp\text{-}lazy\text{-}insert\text{-}charge\text{-}le[of\ x\ b\ P]$ **by** *simp*

lemma $c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\text{-}amortized\text{-}if\text{-}credit$:
assumes *ratio*: $bp\text{-}bucket\text{-}count\text{-}ratio\text{-}ok\ P$
and *paid*:
 $bp\text{-}lazy\text{-}insert\text{-}charge\ x\ b\ P +$
 $bp\text{-}split\text{-}potential$
 $(bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P)) \leq$
 $extra + bp\text{-}split\text{-}potential\ P$
shows $bp\text{-}amortized\text{-}step\text{-}bound$
 $(bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P))\ P$
 $(bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P))$
 $(bp\text{-}insert\text{-}search\text{-}budget\ (length\ (bp\text{-}entries\ P))\ (bp\text{-}block\text{-}size\ P) +$
 $extra)$
proof –
have *search*:
 $bp\text{-}local\text{-}insert\text{-}search\text{-}charge\ P \leq$
 $bp\text{-}insert\text{-}search\text{-}budget\ (length\ (bp\text{-}entries\ P))\ (bp\text{-}block\text{-}size\ P)$
by (rule $bp\text{-}local\text{-}insert\text{-}search\text{-}charge\text{-}ratio\text{-}bound[OF\ ratio]$)
show *?thesis*
using *search paid*
unfolding $bp\text{-}amortized\text{-}step\text{-}bound\text{-}def$
by *simp*
qed

lemma $c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\text{-}amortized\text{-}bound$:
assumes *lazy*: $bp\text{-}lazy\text{-}ordered\text{-}invariant\ P$
and *ratio*: $bp\text{-}bucket\text{-}count\text{-}ratio\text{-}ok\ P$
shows $bp\text{-}amortized\text{-}step\text{-}bound$
 $(bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P))\ P$
 $(bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P))$
 $(bp\text{-}insert\text{-}search\text{-}budget\ (length\ (bp\text{-}entries\ P))\ (bp\text{-}block\text{-}size\ P) + 7)$

proof (rule *c-bp-lazy-insert-precise-amortized-if-credit*[*OF ratio*])
show *bp-lazy-insert-charge* *x b P* +
bp-split-potential (*bp-result-of* (*c-bp-lazy-insert-precise* *x b P*)) $\leq$
 $7 + \text{bp-split-potential } P$
using *bp-lazy-insert-state-split-potential-credit*[*OF lazy, of x b*]
by *simp*
qed

lemma *c-bp-lazy-insert-precise-amortized-ratio-budget*:
assumes *lazy*: *bp-lazy-ordered-invariant* *P*
and *ratio*: *bp-bucket-count-ratio-ok* *P*
shows *bp-amortized-step-bound*
(*bp-steps-of* (*c-bp-lazy-insert-precise* *x b P*)) *P*
(*bp-result-of* (*c-bp-lazy-insert-precise* *x b P*))
(*bp-lazy-insert-amortized-budget* *P*)
unfolding *bp-lazy-insert-amortized-budget-def*
by (rule *c-bp-lazy-insert-precise-amortized-bound*[*OF lazy ratio*])

lemma *c-bp-lazy-insert-precise-regular-ratio-bound*:
assumes *reg*: *bp-regular-invariant* *P*
shows *bp-steps-of* (*c-bp-lazy-insert-precise* *x b P*) $\leq$
bp-lazy-insert-amortized-budget *P*
proof –
have *ord*: *bp-ordered-invariant* *P*
by (rule *bp-regular-invariant-ordered-invariant*[*OF reg*])
have *lazy*: *bp-lazy-ordered-invariant* *P*
by (rule *bp-ordered-invariant-lazy-ordered-invariant*[*OF ord*])
have *ratio*: *bp-bucket-count-ratio-ok* *P*
by (rule *bp-regular-invariant-ratio-ok*[*OF reg*])
have *am*: *bp-amortized-step-bound*
(*bp-steps-of* (*c-bp-lazy-insert-precise* *x b P*)) *P*
(*bp-result-of* (*c-bp-lazy-insert-precise* *x b P*))
(*bp-lazy-insert-amortized-budget* *P*)
by (rule *c-bp-lazy-insert-precise-amortized-ratio-budget*[*OF lazy ratio*])
have *pot*: *bp-split-potential* *P* = 0
by (rule *bp-split-potential-zero-if-regular*
[*OF bp-ordered-invariant-invariant*[*OF ord*] *ratio*])
show *?thesis*
using *am pot unfolding bp-amortized-step-bound-def* **by** *linarith*
qed

lemma *c-bp-lazy-insert-precise-regular-partition-insert-cost-bound*:
assumes *reg*: *bp-regular-invariant* *P*
shows *partition-insert-cost-bound*
(*bp-steps-of* (*c-bp-lazy-insert-precise* *x b P*))
(*bp-lazy-insert-amortized-budget* *P*)
using *c-bp-lazy-insert-precise-regular-ratio-bound*[*OF reg*]
unfolding *partition-insert-cost-bound-def* .

lemma *c-bp-lazy-batch-prepend-precise-result* [simp]:
 $bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}batch\text{-}prepend\text{-}precise\ xs\ P) =$
 $bp\text{-}lazy\text{-}batch\text{-}prepend\text{-}state\ xs\ P$
proof (induction xs arbitrary: P)
case Nil
then show ?case by simp
next
case (Cons xb xs)
obtain x b where xb: $xb = (x, b)$
by force
show ?case
unfolding xb by (simp add: Cons.IH)
qed

lemma *c-bp-lazy-batch-prepend-precise-steps-Nil* [simp]:
 $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}batch\text{-}prepend\text{-}precise\ []\ P) = 0$
unfolding bp-steps-of-def by simp

lemma *c-bp-lazy-batch-prepend-precise-steps-Cons* [simp]:
 $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}batch\text{-}prepend\text{-}precise\ ((x, b) \# xs)\ P) =$
 $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P) +$
 $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}batch\text{-}prepend\text{-}precise\ xs$
 $(bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P)))$
unfolding bp-steps-of-def bp-result-of-def
by (simp split: prod.splits)

lemma *c-bp-lazy-batch-prepend-precise-lazy-ordered-invariant*:
assumes lazy: *bp-lazy-ordered-invariant* P
shows *bp-lazy-ordered-invariant*
 $(bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}batch\text{-}prepend\text{-}precise\ xs\ P))$
using lazy
proof (induction xs arbitrary: P)
case Nil
then show ?case by simp
next
case (Cons xb xs)
obtain x b where xb: $xb = (x, b)$
by force
have step-lazy: *bp-lazy-ordered-invariant*
 $(bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}insert\text{-}precise\ x\ b\ P))$
by (rule c-bp-lazy-insert-precise-lazy-ordered-invariant[OF Cons.prem])
show ?case
unfolding xb using Cons.IH[OF step-lazy] by simp
qed

lemma *c-bp-lazy-batch-prepend-precise-refines-batch-min-update*:
assumes lazy: *bp-lazy-ordered-invariant* P
shows *bp-view* $(bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}batch\text{-}prepend\text{-}precise\ xs\ P)) =$

```

    batch-min-update (bp-view P) xs
  using lazy
proof (induction xs arbitrary: P)
  case Nil
  then show ?case
    unfolding batch-min-update-def by simp
next
  case (Cons xb xs)
  obtain x b where xb: xb = (x, b)
  by force
  have M-pos: 0 < bp-block-size P
  using Cons.premis unfolding bp-lazy-ordered-invariant-def
    bp-lazy-invariant-def by blast
  have step-view:
    bp-view (bp-result-of (c-bp-lazy-insert-precise x b P)) =
      min-update (bp-view P) x b
  by (rule c-bp-lazy-insert-precise-refines-min-update[OF M-pos])
  have step-lazy: bp-lazy-ordered-invariant
    (bp-result-of (c-bp-lazy-insert-precise x b P))
  by (rule c-bp-lazy-insert-precise-lazy-ordered-invariant[OF Cons.premis])
  show ?case
    unfolding xb
    using Cons.IH[OF step-lazy] step-view
    by (simp add: batch-min-update-def)
qed

```

lemma *c-bp-lazy-batch-prepend-precise-amortized-bound:*

```

  assumes lazy: bp-lazy-ordered-invariant P
  and ratios: bp-lazy-batch-ratio-ok xs P
  shows bp-amortized-step-bound
    (bp-steps-of (c-bp-lazy-batch-prepend-precise xs P)) P
    (bp-result-of (c-bp-lazy-batch-prepend-precise xs P))
    (bp-lazy-batch-insert-budget-sum xs P)
  using lazy ratios
proof (induction xs arbitrary: P)
  case Nil
  then show ?case
    unfolding bp-amortized-step-bound-def by simp
next
  case (Cons xb xs)
  obtain x b where xb: xb = (x, b)
  by force
  let ?I = c-bp-lazy-insert-precise x b P
  let ?P1 = bp-result-of ?I
  have ratio: bp-bucket-count-ratio-ok P
  using Cons.premis(2) unfolding xb by simp
  have tail-ratios: bp-lazy-batch-ratio-ok xs ?P1
  using Cons.premis(2) unfolding xb by simp
  have step-lazy: bp-lazy-ordered-invariant ?P1

```

by (rule *c-bp-lazy-insert-precise-lazy-ordered-invariant*[*OF Cons.prem*s(1)])
have *insert-bound*:
 $bp\text{-}steps\text{-}of\ ?I + bp\text{-}split\text{-}potential\ ?P1 \leq$
 $bp\text{-}lazy\text{-}insert\text{-}amortized\text{-}budget\ P + bp\text{-}split\text{-}potential\ P$
using *c-bp-lazy-insert-precise-amortized-ratio-budget*
 $[OF\ Cons.prem$ s(1) *ratio*, of *x b*]
unfolding *bp-amortized-step-bound-def* .
have *tail-bound*:
 $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}batch\text{-}prepend\text{-}precise\ xs\ ?P1) +$
 $bp\text{-}split\text{-}potential$
 $(bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}batch\text{-}prepend\text{-}precise\ xs\ ?P1)) \leq$
 $bp\text{-}lazy\text{-}batch\text{-}insert\text{-}budget\text{-}sum\ xs\ ?P1 + bp\text{-}split\text{-}potential\ ?P1$
using *Cons.IH*[*OF step-lazy tail-ratios*]
unfolding *bp-amortized-step-bound-def* .
show *?case*
unfolding *xb bp-amortized-step-bound-def*
using *insert-bound tail-bound* **by** *simp*
qed

lemma *c-bp-lazy-batch-prepend-precise-regular-steps-le-budget-sum*:
assumes *reg: bp-regular-invariant P*
and *ratios: bp-lazy-batch-ratio-ok xs P*
shows $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}batch\text{-}prepend\text{-}precise\ xs\ P) \leq$
 $bp\text{-}lazy\text{-}batch\text{-}insert\text{-}budget\text{-}sum\ xs\ P$
proof –
have *ord: bp-ordered-invariant P*
by (rule *bp-regular-invariant-ordered-invariant*[*OF reg*])
have *lazy: bp-lazy-ordered-invariant P*
by (rule *bp-ordered-invariant-lazy-ordered-invariant*[*OF ord*])
have *am: bp-amortized-step-bound*
 $(bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}batch\text{-}prepend\text{-}precise\ xs\ P))\ P$
 $(bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}lazy\text{-}batch\text{-}prepend\text{-}precise\ xs\ P))$
 $(bp\text{-}lazy\text{-}batch\text{-}insert\text{-}budget\text{-}sum\ xs\ P)$
by (rule *c-bp-lazy-batch-prepend-precise-amortized-bound*[*OF lazy ratios*])
have *pot: bp-split-potential P = 0*
by (rule *bp-split-potential-zero-if-regular*
 $[OF\ bp\text{-}ordered\text{-}invariant\text{-}invariant[OF\ ord]$
 $bp\text{-}regular\text{-}invariant\text{-}ratio\text{-}ok[OF\ reg]]$)
show *?thesis*
using *am pot* **unfolding** *bp-amortized-step-bound-def* **by** *linarith*
qed

lemma *c-bp-regularized-local-insert-result* [*simp*]:
 $bp\text{-}result\text{-}of\ (c\text{-}bp\text{-}regularized\text{-}local\text{-}insert\ x\ b\ P) =$
 $bp\text{-}regularized\text{-}local\text{-}insert\ x\ b\ P$
unfolding *bp-result-of-def c-bp-regularized-local-insert-def* **by** *simp*

lemma *c-bp-regularized-local-insert-steps* [*simp*]:
 $bp\text{-}steps\text{-}of\ (c\text{-}bp\text{-}regularized\text{-}local\text{-}insert\ x\ b\ P) =$

$bp\text{-local-insert-search-charge } P$
unfolding $bp\text{-steps-of-def } c\text{-bp-regularized-local-insert-def}$ **by** $simp$

lemma $c\text{-bp-regularized-local-insert-regular-invariant}$:
assumes $reg: bp\text{-regular-invariant } P$
shows $bp\text{-regular-invariant}$
 $(bp\text{-result-of } (c\text{-bp-regularized-local-insert } x \ b \ P))$
unfolding $c\text{-bp-regularized-local-insert-result}$
by $(rule \ bp\text{-regularized-local-insert-regular-invariant}$
 $[OF \ bp\text{-regular-invariant-ordered-invariant}[OF \ reg]])$

lemma $c\text{-bp-regularized-local-insert-refines-min-update}$:
assumes $reg: bp\text{-regular-invariant } P$
shows $bp\text{-view } (bp\text{-result-of } (c\text{-bp-regularized-local-insert } x \ b \ P)) =$
 $min\text{-update } (bp\text{-view } P) \ x \ b$
unfolding $c\text{-bp-regularized-local-insert-result}$
by $(rule \ bp\text{-regularized-local-insert-refines-min-update}$
 $[OF \ bp\text{-regular-invariant-ordered-invariant}[OF \ reg]])$

lemma $c\text{-bp-regularized-local-insert-refines-insert-spec}$:
assumes $reg: bp\text{-regular-invariant } P$
shows $insert\text{-spec } (bp\text{-view } P) \ x \ b$
 $(bp\text{-view } (bp\text{-result-of } (c\text{-bp-regularized-local-insert } x \ b \ P)))$
unfolding $c\text{-bp-regularized-local-insert-refines-min-update}[OF \ reg]$
by $(rule \ min\text{-update-insert-spec})$

lemma $c\text{-bp-regularized-local-insert-ratio-bound}$:
assumes $reg: bp\text{-regular-invariant } P$
shows $bp\text{-steps-of } (c\text{-bp-regularized-local-insert } x \ b \ P) \leq$
 $bp\text{-insert-search-budget } (length \ (bp\text{-entries } P)) \ (bp\text{-block-size } P)$
by $(simp \ add: \ bp\text{-local-insert-search-charge-ratio-bound}$
 $[OF \ bp\text{-regular-invariant-ratio-ok}[OF \ reg]])$

lemma $c\text{-bp-regularized-local-insert-split-potential-zero}$:
assumes $reg: bp\text{-regular-invariant } P$
shows $bp\text{-split-potential}$
 $(bp\text{-result-of } (c\text{-bp-regularized-local-insert } x \ b \ P)) = 0$
unfolding $c\text{-bp-regularized-local-insert-result}$
by $(rule \ bp\text{-regularized-local-insert-split-potential-zero}$
 $[OF \ bp\text{-regular-invariant-ordered-invariant}[OF \ reg]])$

lemma $bp\text{-regular-invariant-split-potential-zero}$:
assumes $reg: bp\text{-regular-invariant } P$
shows $bp\text{-split-potential } P = 0$
proof $(rule \ bp\text{-split-potential-zero-if-regular})$
show $bp\text{-invariant } P$
by $(rule \ bp\text{-ordered-invariant-invariant}$
 $[OF \ bp\text{-regular-invariant-ordered-invariant}[OF \ reg]])$
show $bp\text{-bucket-count-ratio-ok } P$

by (rule *bp-regular-invariant-ratio-ok*[*OF reg*])
qed

lemma *c-bp-regularized-local-insert-amortized-ratio-bound*:

assumes *reg*: *bp-regular-invariant P*

shows *bp-amortized-step-bound*

(*bp-steps-of* (*c-bp-regularized-local-insert x b P*)) *P*
(*bp-result-of* (*c-bp-regularized-local-insert x b P*))
(*bp-insert-search-budget* (*length* (*bp-entries P*)) (*bp-block-size P*))

using *c-bp-regularized-local-insert-ratio-bound*[*OF reg*]

c-bp-regularized-local-insert-split-potential-zero[*OF reg, of x b*]

bp-regular-invariant-split-potential-zero[*OF reg*]

unfolding *bp-amortized-step-bound-def* by *simp*

lemma *c-bp-regularized-local-insert-partition-insert-cost-bound*:

assumes *reg*: *bp-regular-invariant P*

shows *partition-insert-cost-bound*

(*bp-steps-of* (*c-bp-regularized-local-insert x b P*))
(*bp-insert-search-budget* (*length* (*bp-entries P*)) (*bp-block-size P*))

using *c-bp-regularized-local-insert-ratio-bound*[*OF reg*]

unfolding *partition-insert-cost-bound-def* .

lemma *c-bp-regularized-local-insert-build-result* [*simp*]:

bp-result-of (*c-bp-regularized-local-insert-build x b P*) =

bp-regularized-local-insert x b P

proof –

let *?Q* = *bp-local-insert-state x b P*

obtain *P' c* where *build*: *c-bp-rebucket-build ?Q* = (*P'*, *c*)

by (*cases c-bp-rebucket-build ?Q*)

have *P'*: *P' = bp-rebucket ?Q*

using *c-bp-rebucket-build-result*[*of ?Q*] *build*

unfolding *bp-result-of-def* by *simp*

show *?thesis*

using *build P'*

unfolding *c-bp-regularized-local-insert-build-def*

bp-regularized-local-insert-def bp-result-of-def

by *simp*

qed

lemma *c-bp-regularized-local-insert-build-steps*:

bp-steps-of (*c-bp-regularized-local-insert-build x b P*) =

bp-local-insert-search-charge P +

bp-steps-of (*c-bp-rebucket-build* (*bp-local-insert-state x b P*))

unfolding *c-bp-regularized-local-insert-build-def*

by (*auto simp: bp-steps-of-def split: prod.splits*)

lemma *c-bp-regularized-local-insert-build-regular-invariant*:

assumes *reg*: *bp-regular-invariant P*

shows *bp-regular-invariant*

(bp-result-of (c-bp-regularized-local-insert-build x b P))
unfolding c-bp-regularized-local-insert-build-result
by (rule bp-regularized-local-insert-regular-invariant
 [OF bp-regular-invariant-ordered-invariant[OF reg]])

lemma c-bp-regularized-local-insert-build-refines-min-update:

assumes reg: bp-regular-invariant P
shows bp-view (bp-result-of
 (c-bp-regularized-local-insert-build x b P)) =
 min-update (bp-view P) x b
unfolding c-bp-regularized-local-insert-build-result
by (rule bp-regularized-local-insert-refines-min-update
 [OF bp-regular-invariant-ordered-invariant[OF reg]])

lemma c-bp-regularized-local-insert-build-refines-insert-spec:

assumes reg: bp-regular-invariant P
shows insert-spec (bp-view P) x b
 (bp-view (bp-result-of
 (c-bp-regularized-local-insert-build x b P)))
unfolding c-bp-regularized-local-insert-build-refines-min-update[OF reg]
by (rule min-update-insert-spec)

lemma c-bp-regularized-local-insert-build-split-potential-zero:

assumes reg: bp-regular-invariant P
shows bp-split-potential
 (bp-result-of (c-bp-regularized-local-insert-build x b P)) = 0
unfolding c-bp-regularized-local-insert-build-result
by (rule bp-regularized-local-insert-split-potential-zero
 [OF bp-regular-invariant-ordered-invariant[OF reg]])

lemma c-bp-regularized-local-insert-build-rebuild-steps-le:

assumes ord: bp-ordered-invariant P
shows bp-steps-of (c-bp-regularized-local-insert-build x b P) ≤
 bp-local-insert-search-charge P +
 length (bp-entries (bp-local-insert-state x b P)) +
 Suc (length (bp-entries (bp-local-insert-state x b P)) div
 bp-block-size P)

proof –

have step-inv: bp-invariant (bp-local-insert-state x b P)
by (rule bp-local-insert-state-invariant[OF ord])
have build:
 bp-steps-of (c-bp-rebucket-build (bp-local-insert-state x b P)) ≤
 length (bp-entries (bp-local-insert-state x b P)) +
 Suc (length (bp-entries (bp-local-insert-state x b P)) div
 bp-block-size (bp-local-insert-state x b P))
by (rule c-bp-rebucket-build-steps-le[OF step-inv])
have block:
 bp-block-size (bp-local-insert-state x b P) = bp-block-size P
unfolding bp-local-insert-state-def bp-delete-key-def


```

    by (simp add: Let-def split: if-splits)
  show ?thesis
    using build
    unfolding c-bp-regularized-local-insert-build-steps block
    by simp
qed

lemma c-bp-regularized-local-insert-build-steps-le:
  assumes reg: bp-regular-invariant P
  shows bp-steps-of (c-bp-regularized-local-insert-build x b P) ≤
    bp-insert-search-budget (length (bp-entries P)) (bp-block-size P) +
    length (bp-entries (bp-local-insert-state x b P)) +
    Suc (length (bp-entries (bp-local-insert-state x b P)) div
      bp-block-size P)
proof -
  have ord: bp-ordered-invariant P
  by (rule bp-regular-invariant-ordered-invariant[OF reg])
  have search:
    bp-local-insert-search-charge P ≤
    bp-insert-search-budget (length (bp-entries P)) (bp-block-size P)
  by (rule bp-local-insert-search-charge-ratio-bound
    [OF bp-regular-invariant-ratio-ok[OF reg]])
  have build:
    bp-steps-of (c-bp-regularized-local-insert-build x b P) ≤
    bp-local-insert-search-charge P +
    length (bp-entries (bp-local-insert-state x b P)) +
    Suc (length (bp-entries (bp-local-insert-state x b P)) div
      bp-block-size P)
  by (rule c-bp-regularized-local-insert-build-rebuild-steps-le[OF ord])
  show ?thesis
    using search build by linarith
qed

lemma c-bp-regularized-local-insert-build-amortized-rebuild-bound:
  assumes reg: bp-regular-invariant P
  shows bp-amortized-step-bound
    (bp-steps-of (c-bp-regularized-local-insert-build x b P)) P
    (bp-result-of (c-bp-regularized-local-insert-build x b P))
    (bp-insert-search-budget (length (bp-entries P)) (bp-block-size P) +
    length (bp-entries (bp-local-insert-state x b P)) +
    Suc (length (bp-entries (bp-local-insert-state x b P)) div
      bp-block-size P))
  using c-bp-regularized-local-insert-build-steps-le[OF reg, of x b]
    c-bp-regularized-local-insert-build-split-potential-zero[OF reg, of x b]
    bp-regular-invariant-split-potential-zero[OF reg]
  unfolding bp-amortized-step-bound-def by simp

lemma c-bp-rebucketed-batch-prepend-bulk-result [simp]:
  bp-result-of (c-bp-rebucketed-batch-prepend-bulk xs P) =

```

$bp\text{-rebucketed-batch-prepend } xs \ P$
unfolding $bp\text{-result-of-def } c\text{-bp-rebucketed-batch-prepend-bulk-def}$ **by** $simp$

lemma $c\text{-bp-rebucketed-batch-prepend-bulk-steps}$ $[simp]$:
 $bp\text{-steps-of } (c\text{-bp-rebucketed-batch-prepend-bulk } xs \ P) =$
 $bp\text{-batch-prepend-log-budget } xs \ (bp\text{-block-size } P)$
unfolding $bp\text{-steps-of-def } c\text{-bp-rebucketed-batch-prepend-bulk-def}$ **by** $simp$

lemma $c\text{-bp-bucketed-batch-prepend-direct-result}$ $[simp]$:
 $bp\text{-result-of } (c\text{-bp-bucketed-batch-prepend-direct } xs \ P) =$
 $bp\text{-bucketed-batch-prepend-state } xs \ P$
unfolding $c\text{-bp-bucketed-batch-prepend-direct-def}$ **by** $simp$

lemma $c\text{-bp-bucketed-batch-prepend-direct-steps}$ $[simp]$:
 $bp\text{-steps-of } (c\text{-bp-bucketed-batch-prepend-direct } xs \ P) =$
 $bp\text{-batch-prepend-amortized-budget } xs \ (bp\text{-block-size } P)$
unfolding $c\text{-bp-bucketed-batch-prepend-direct-def}$ **by** $simp$

lemma $c\text{-bp-bucketed-batch-prepend-direct-actual-result}$ $[simp]$:
 $bp\text{-result-of } (c\text{-bp-bucketed-batch-prepend-direct-actual } xs \ P) =$
 $bp\text{-bucketed-batch-prepend-state } xs \ P$
unfolding $c\text{-bp-bucketed-batch-prepend-direct-actual-def}$ **by** $simp$

lemma $c\text{-bp-bucketed-batch-prepend-direct-actual-steps}$ $[simp]$:
 $bp\text{-steps-of } (c\text{-bp-bucketed-batch-prepend-direct-actual } xs \ P) =$
 $bp\text{-batch-prepend-log-budget } xs \ (bp\text{-block-size } P)$
unfolding $c\text{-bp-bucketed-batch-prepend-direct-actual-def}$ **by** $simp$

lemma $c\text{-bp-bucketed-batch-prepend-direct-paper-batch-cost-bound}$:
 $partition\text{-batch-cost-bound}$
 $(bp\text{-steps-of } (c\text{-bp-bucketed-batch-prepend-direct } xs \ P))$
 $(bp\text{-batch-prepend-per-item-budget } xs \ (bp\text{-block-size } P)) \ xs$
unfolding $partition\text{-batch-cost-bound-def}$
by $(simp \text{ add: } bp\text{-batch-prepend-amortized-budget-alt})$

lemma $bp\text{-bucketed-batch-prepend-state-empty}$ $[simp]$:
 $bp\text{-bucketed-batch-prepend-state } [] \ P = P$
unfolding $bp\text{-bucketed-batch-prepend-state-def}$ **by** $simp$

lemma $bp\text{-bucketed-batch-prepend-state-entries-nonempty}$:
assumes $M\text{-pos}$: $0 < bp\text{-block-size } P$
and xs : $xs \neq []$
shows $bp\text{-entries } (bp\text{-bucketed-batch-prepend-state } xs \ P) =$
 $sort\text{-key } snd \ xs \ @ \ bp\text{-entries } P$
proof –
obtain $p \ ps$ **where** $xs\text{-def}$: $xs = p \ # \ ps$
using xs **by** $(cases \ xs) \ auto$
have $flat$:
 $bp\text{-bucket-entries-flat}$

(bp-bucketize-entries (bp-block-size P) xs) = sort-key snd xs
by (rule bp-bucket-entries-flat-bucketize-entries[OF M-pos])
have flat':
 bp-bucket-entries-flat
 (bp-bucketize-entries (bp-block-size P) (p # ps)) =
 sort-key snd (p # ps)
using flat **unfolding** xs-def .
show ?thesis
unfolding bp-bucketed-batch-prepend-state-def xs-def bp-entries-def
using flat' **by** simp
qed

lemma bp-bucketed-batch-prepend-state-keys-nonempty:
assumes M-pos: $0 < \text{bp-block-size } P$
and xs: $xs \neq []$
shows keys-of (bp-view (bp-bucketed-batch-prepend-state xs P)) =
 bp-entry-keys xs $\cup$ keys-of (bp-view P)
unfolding bp-view-def
 bp-bucketed-batch-prepend-state-entries-nonempty[OF M-pos xs]
 bp-entry-keys-def
by auto

lemma bp-bucketed-batch-prepend-state-values:
assumes distinct: distinct (map fst xs)
and disjoint: $\text{bp-entry-keys } xs \cap \text{keys-of (bp-view P)} = \{\}$
shows value-of (bp-view (bp-bucketed-batch-prepend-state xs P)) =
 value-of (batch-min-update (bp-view P) xs)
proof –
have disjoint': $\text{fst 'set } xs \cap \text{keys-of (bp-view P)} = \{\}$
using disjoint **unfolding** bp-entry-keys-def **by** simp
have vals-abs: value-of (batch-min-update (bp-view P) xs) =
 bp-batch-value-update xs (value-of (bp-view P))
by (rule batch-min-update-value-distinct-disjoint
 [OF distinct disjoint'])
show ?thesis
using vals-abs
unfolding bp-view-def bp-bucketed-batch-prepend-state-def
by (cases xs) simp-all
qed

theorem bp-bucketed-batch-prepend-state-refines-batch-min-update:
assumes inv: bp-invariant P
and distinct: distinct (map fst xs)
and disjoint: $\text{bp-entry-keys } xs \cap \text{keys-of (bp-view P)} = \{\}$
shows bp-view (bp-bucketed-batch-prepend-state xs P) =
 batch-min-update (bp-view P) xs
proof (cases xs = [])
case True
then show ?thesis

```

    unfolding batch-min-update-def by simp
next
case False
have M-pos: 0 < bp-block-size P
  using inv unfolding bp-invariant-def by blast
have keys: keys-of (bp-view (bp-bucketed-batch-prepend-state xs P)) =
  keys-of (batch-min-update (bp-view P) xs)
proof -
  have state: keys-of (bp-view (bp-bucketed-batch-prepend-state xs P)) =
    bp-entry-keys xs  $\cup$  keys-of (bp-view P)
  by (rule bp-bucketed-batch-prepend-state-keys-nonempty[OF M-pos False])
  have abstract: keys-of (batch-min-update (bp-view P) xs) =
    keys-of (bp-view P)  $\cup$  bp-entry-keys xs
  by (simp add: batch-min-update-keys bp-entry-keys-def)
  show ?thesis
    using state abstract by auto
qed
have vals-state: value-of (bp-view (bp-bucketed-batch-prepend-state xs P)) =
  value-of (batch-min-update (bp-view P) xs)
  by (rule bp-bucketed-batch-prepend-state-values
    [OF distinct disjoint])
show ?thesis
  using keys vals-state
  by (cases bp-view (bp-bucketed-batch-prepend-state xs P);
    cases batch-min-update (bp-view P) xs; simp)
qed

lemma bp-rebase-first-bucket-marker-markers-lower-bound:
  assumes lower:
     $\forall b \in \text{set } bs. \forall p \in \text{set } (bp\text{-bucket-entries } b). bp\text{-marker } b \leq \text{snd } p$ 
    and beta-le:  $\forall p \in \text{set } (bp\text{-bucket-entries-flat } bs). \text{beta} \leq \text{snd } p$ 
  shows  $\forall b \in \text{set } (bp\text{-rebase-first-bucket-marker } \text{beta } bs).$ 
     $\forall p \in \text{set } (bp\text{-bucket-entries } b). bp\text{-marker } b \leq \text{snd } p$ 
  using assms
  by (cases bs) (auto simp: bp-bucket-entries-flat-def)

lemma bp-rebase-first-bucket-marker-markers-sorted:
  assumes sorted: sorted-wrt ( $\lambda b c. bp\text{-marker } b \leq bp\text{-marker } c$ ) bs
    and boundaries: bp-bucket-boundaries-ok bs
    and head-nonempty:
       $\bigwedge b bs'. bp\text{-drop-empty-prefix } bs = b \# bs' \implies$ 
        bp-bucket-entries b  $\neq []$ 
    and beta-le:
       $\forall p \in \text{set } (bp\text{-bucket-entries-flat } (bp\text{-drop-empty-prefix } bs)).$ 
        beta  $\leq \text{snd } p$ 
  shows sorted-wrt ( $\lambda b c. bp\text{-marker } b \leq bp\text{-marker } c$ )
    (bp-rebase-first-bucket-marker beta (bp-drop-empty-prefix bs))
  proof (cases bp-drop-empty-prefix bs)
  case Nil

```

```

    then show ?thesis by simp
next
case (Cons b rest)
note old0 = Cons
have old0-sorted: sorted-wrt ( $\lambda b\ c. \text{bp-marker } b \leq \text{bp-marker } c$ )
  (bp-drop-empty-prefix bs)
  by (rule bp-bucket-markers-sorted-drop-empty-prefix[OF sorted])
have old0-boundaries:
  bp-bucket-boundaries-ok (bp-drop-empty-prefix bs)
  by (rule bp-bucket-boundaries-ok-drop-empty-prefix[OF boundaries])
have b-nonempty: bp-bucket-entries b  $\neq$  []
  using head-nonempty Cons by blast
show ?thesis
proof (cases rest)
case Nil
  then show ?thesis
    using old0 by simp
next
case (Cons c cs)
obtain q where q:  $q \in \text{set } (\text{bp-bucket-entries } b)$ 
  using b-nonempty by (cases bp-bucket-entries b) auto
have beta-le-q:  $\beta \leq \text{snd } q$ 
  using beta-le old0 q unfolding bp-bucket-entries-flat-def by auto
have q-le-c:  $\text{snd } q \leq \text{bp-marker } c$ 
  using old0-boundaries old0 Cons q by simp
have beta-le-c:  $\beta \leq \text{bp-marker } c$ 
  using beta-le-q q-le-c by linarith
have tail-sorted:
  sorted-wrt ( $\lambda b\ c. \text{bp-marker } b \leq \text{bp-marker } c$ ) (c # cs)
  using old0-sorted old0 Cons by simp
have beta-le-tail:
   $\forall d \in \text{set } (c \# \text{cs}). \beta \leq \text{bp-marker } d$ 
proof
fix d
assume d:  $d \in \text{set } (c \# \text{cs})$ 
have bp-marker c  $\leq \text{bp-marker } d$ 
  using tail-sorted d by (cases d = c) auto
then show  $\beta \leq \text{bp-marker } d$ 
  using beta-le-c by linarith
qed
show ?thesis
  using old0 Cons tail-sorted beta-le-tail by simp
qed
qed

lemma bp-batch-max-value-le-old-entry:
  assumes xs:  $xs = p \# ps$ 
  and admissible: batch-prepend-admissible (bp-view P) xs
  and inv: bp-invariant P

```

```

    and  $r$ :  $r \in \text{set } (\text{bp-entries } P)$ 
  shows  $\text{bp-batch-max-value } (\text{snd } p) \text{ } ps \leq \text{snd } r$ 
proof -
  have  $r\text{-key}$ :  $\text{fst } r \in \text{keys-of } (\text{bp-view } P)$ 
    using  $r$  unfolding  $\text{bp-view-def}$   $\text{bp-entry-keys-def}$  by auto
  have  $r\text{-value}$ :  $\text{value-of } (\text{bp-view } P) (\text{fst } r) = \text{snd } r$ 
    using  $\text{inv } r$  unfolding  $\text{bp-invariant-def}$   $\text{bp-values-consistent-def}$ 
     $\text{bp-view-def}$  by simp
  have  $\text{all-le}$ :  $\bigwedge q. q \in \text{set } xs \implies \text{snd } q \leq \text{snd } r$ 
proof -
  fix  $q$ 
  assume  $q$ :  $q \in \text{set } xs$ 
  then have  $\text{snd } q \leq \text{value-of } (\text{bp-view } P) (\text{fst } r)$ 
    using  $\text{admissible } r\text{-key}$ 
    unfolding  $\text{batch-prepend-admissible-def}$  by force
  then show  $\text{snd } q \leq \text{snd } r$ 
    using  $r\text{-value}$  by simp
qed
have  $\text{ps-le}$ :  $\bigwedge q. q \in \text{set } ps \implies \text{snd } q \leq \text{snd } r$ 
  using  $\text{all-le } xs$  by auto
have  $\text{p-le}$ :  $\text{snd } p \leq \text{snd } r$ 
  using  $\text{all-le } xs$  by simp
show ?thesis
  by (rule  $\text{bp-batch-max-value-upper}[OF \text{ ps-le } \text{ p-le}]$ )
qed

```

```

lemma  $\text{bp-bucketed-batch-prepend-state-invariant}$ :
  assumes  $\text{ord}$ :  $\text{bp-ordered-invariant } P$ 
    and  $\text{distinct}$ :  $\text{distinct } (\text{map } \text{fst } xs)$ 
    and  $\text{disjoint}$ :  $\text{bp-entry-keys } xs \cap \text{keys-of } (\text{bp-view } P) = \{\}$ 
    and  $\text{admissible}$ :  $\text{batch-prepend-admissible } (\text{bp-view } P) \text{ } xs$ 
  shows  $\text{bp-invariant } (\text{bp-bucketed-batch-prepend-state } xs \text{ } P)$ 
proof (cases  $xs$ )
  case Nil
  then show ?thesis
    using  $\text{bp-ordered-invariant-invariant}[OF \text{ ord}]$  by simp
next
  case ( $\text{Cons } p \text{ } ps$ )
  let  $?M$  =  $\text{bp-block-size } P$ 
  let  $?beta$  =  $\text{bp-batch-max-value } (\text{snd } p) \text{ } ps$ 
  let  $?new$  =  $\text{bp-bucketize-entries } ?M \text{ } xs$ 
  let  $?trimmed$  =  $\text{bp-drop-empty-prefix } (\text{bp-buckets } P)$ 
  let  $?old$  =  $\text{bp-rebase-first-bucket-marker } ?beta \text{ } ?trimmed$ 
  let  $?P'$  =  $\text{bp-bucketed-batch-prepend-state } xs \text{ } P$ 
  have  $\text{inv}$ :  $\text{bp-invariant } P$ 
    by (rule  $\text{bp-ordered-invariant-invariant}[OF \text{ ord}]$ )
  have  $\text{boundaries}$ :  $\text{bp-bucket-boundaries-ok } (\text{bp-buckets } P)$ 
    using  $\text{bp-ordered-invariant-boundaries-state-ok}[OF \text{ ord}]$ 
    unfolding  $\text{bp-bucket-boundaries-state-ok-def}$  .

```

```

have M-pos: 0 < ?M
  using inv unfolding bp-invariant-def by blast
have entries-state: bp-entries ?P' = sort-key snd xs @ bp-entries P
  by (rule bp-bucketed-batch-prepend-state-entries-nonempty[OF M-pos])
  (simp add: Cons)
have old-distinct: distinct (map fst (bp-entries P))
  using inv unfolding bp-invariant-def bp-distinct-keys-def by blast
have new-distinct: distinct (map fst (sort-key snd xs))
  by (rule distinct-map-fst-sort-key[OF distinct])
have disj-keys:
  set (map fst (sort-key snd xs)) ∩ set (map fst (bp-entries P)) = {}
  using disjoint unfolding bp-entry-keys-def bp-view-def by auto
have distinct-entries:
  distinct (map fst (sort-key snd xs @ bp-entries P))
  using new-distinct old-distinct disj-keys by simp
have beta-le-old:
  ∀ r ∈ set (bp-entries P). ?beta ≤ snd r
proof
  fix r
  assume r: r ∈ set (bp-entries P)
  show ?beta ≤ snd r
    by (rule bp-batch-max-value-le-old-entry
        [OF Cons admissible inv r])
qed
have beta-le-trimmed:
  ∀ r ∈ set (bp-bucket-entries-flat ?trimmed). ?beta ≤ snd r
  using beta-le-old unfolding bp-entries-def by simp
have new-sizes:
  ∀ b ∈ set ?new. length (bp-bucket-entries b) ≤ ?M
  by (rule bp-bucketize-entries-sizes-ok[OF M-pos])
have trimmed-subset: set ?trimmed ⊆ set (bp-buckets P)
  by (rule bp-drop-empty-prefix-set-subset)
have trimmed-sizes:
  ∀ b ∈ set ?trimmed. length (bp-bucket-entries b) ≤ ?M
  using inv trimmed-subset unfolding bp-invariant-def bp-bucket-sizes-ok-def
  by blast
have old-sizes:
  ∀ b ∈ set ?old. length (bp-bucket-entries b) ≤ ?M
proof (cases ?trimmed)
  case Nil
  then show ?thesis by simp
next
  case (Cons b bs)
  then show ?thesis
    using trimmed-sizes by simp
qed
have new-sorted:
  sorted-wrt (λ b c. bp-marker b ≤ bp-marker c) ?new
  by (rule bp-bucketize-entries-markers-sorted[OF M-pos])

```

```

have old-sorted:
  sorted-wrt ( $\lambda b\ c. \text{bp-marker } b \leq \text{bp-marker } c$ ) ?old
proof (rule bp-rebase-first-bucket-marker-markers-sorted)
  show sorted-wrt ( $\lambda b\ c. \text{bp-marker } b \leq \text{bp-marker } c$ ) (bp-buckets P)
    using inv unfolding bp-invariant-def bp-bucket-markers-sorted-def
    by blast
  show bp-bucket-boundaries-ok (bp-buckets P)
    by (rule boundaries)
  show  $\bigwedge b\ bs'. \text{bp-drop-empty-prefix } (\text{bp-buckets } P) = b \# bs' \implies$ 
     $\text{bp-bucket-entries } b \neq []$ 
    by (rule bp-drop-empty-prefix-head-nonempty)
  show  $\forall r \in \text{set } (\text{bp-bucket-entries-flat}$ 
     $(\text{bp-drop-empty-prefix } (\text{bp-buckets } P))). \text{?beta} \leq \text{snd } r$ 
    by (rule beta-le-trimmed)
qed
have new-marker-le-beta:
   $\forall b \in \text{set } \text{?new}. \text{bp-marker } b \leq \text{?beta}$ 
proof (rule bp-bucketize-entries-markers-le)
  show  $\forall q \in \text{set } xs. \text{snd } q \leq \text{?beta}$ 
    using Cons
    by (auto simp: bp-batch-max-value-ge-member-Cons)
qed
have beta-le-old-markers:
   $\forall b \in \text{set } \text{?old}. \text{?beta} \leq \text{bp-marker } b$ 
proof (cases ?old)
  case Nil
  then show ?thesis by simp
next
  case (Cons b bs)
  have head:  $\text{bp-marker } b = \text{?beta}$ 
    using Cons by (cases ?trimmed) auto
  have tail-ge:  $\forall c \in \text{set } bs. \text{bp-marker } b \leq \text{bp-marker } c$ 
    using old-sorted Cons by simp
  show ?thesis
    using Cons head tail-ge by auto
qed
have sorted-all:
  sorted-wrt ( $\lambda b\ c. \text{bp-marker } b \leq \text{bp-marker } c$ ) (?new @ ?old)
  unfolding sorted-wrt-append
proof (intro conjI ballI)
  show sorted-wrt ( $\lambda b\ c. \text{bp-marker } b \leq \text{bp-marker } c$ ) ?new
    by (rule new-sorted)
  show sorted-wrt ( $\lambda b\ c. \text{bp-marker } b \leq \text{bp-marker } c$ ) ?old
    by (rule old-sorted)
  fix b c
  assume b:  $b \in \text{set } \text{?new}$  and c:  $c \in \text{set } \text{?old}$ 
  have  $\text{bp-marker } b \leq \text{?beta}$ 
    using new-marker-le-beta b by blast
  moreover have  $\text{?beta} \leq \text{bp-marker } c$ 

```



```

    using beta-le-old-markers c by blast
    ultimately show bp-marker b ≤ bp-marker c
    by linarith
qed
have new-lower:
  ∀ b ∈ set ?new. ∀ q ∈ set (bp-bucket-entries b).
    bp-marker b ≤ snd q
  by (rule bp-bucketize-entries-markers-lower-bound[OF M-pos])
have trimmed-lower:
  ∀ b ∈ set ?trimmed. ∀ q ∈ set (bp-bucket-entries b).
    bp-marker b ≤ snd q
  using inv trimmed-subset
  unfolding bp-invariant-def bp-bucket-markers-lower-bound-def by blast
have old-lower:
  ∀ b ∈ set ?old. ∀ q ∈ set (bp-bucket-entries b).
    bp-marker b ≤ snd q
  by (rule bp-rebase-first-bucket-marker-markers-lower-bound
    [OF trimmed-lower beta-le-trimmed])
have lower-all:
  ∀ b ∈ set (?new @ ?old). ∀ q ∈ set (bp-bucket-entries b).
    bp-marker b ≤ snd q
  using new-lower old-lower by auto
have values-entries:
  ∀ r ∈ set (sort-key snd xs @ bp-entries P).
    bp-batch-value-update xs (bp-values P) (fst r) = snd r
proof
  fix r
  assume r: r ∈ set (sort-key snd xs @ bp-entries P)
  show bp-batch-value-update xs (bp-values P) (fst r) = snd r
  proof (cases r ∈ set (sort-key snd xs))
    case True
    then have r ∈ set xs
    by simp
    obtain x b where r-def: r = (x, b)
    by force
    then have (x, b) ∈ set xs
    using ⟨r ∈ set xs⟩ by simp
    then have bp-batch-value-update xs (bp-values P) x = b
    by (rule bp-batch-value-update-in-set-distinct[OF distinct])
    then show ?thesis
    unfolding r-def by simp
  next
  case False
  then have r-old: r ∈ set (bp-entries P)
  using r by simp
  have fst r ∉ fst `set xs
  using disjoint r-old unfolding bp-entry-keys-def bp-view-def
  by auto
  then have unchanged:

```

```

    bp-batch-value-update xs (bp-values P) (fst r) =
      bp-values P (fst r)
    by (rule bp-batch-value-update-notin)
  have bp-values P (fst r) = snd r
    using inv r-old unfolding bp-invariant-def bp-values-consistent-def
    by blast
  then show ?thesis
    using unchanged by simp
qed
qed
have flat-new: bp-bucket-entries-flat ?new = sort-key snd xs
  by (rule bp-bucket-entries-flat-bucketize-entries[OF M-pos])
have flat-old: bp-bucket-entries-flat ?old = bp-entries P
  unfolding bp-entries-def by simp
show ?thesis
  using M-pos distinct-entries new-sizes old-sizes sorted-all lower-all
    values-entries flat-new flat-old
  unfolding bp-bucketed-batch-prepend-state-def Cons Let-def
    bp-invariant-def bp-distinct-keys-def bp-bucket-sizes-ok-def
    bp-bucket-markers-sorted-def bp-bucket-markers-lower-bound-def
    bp-values-consistent-def bp-entries-def
  by auto
qed

lemma bp-bucketed-batch-prepend-state-boundaries-state-ok:
  assumes ord: bp-ordered-invariant P
  and admissible: batch-prepend-admissible (bp-view P) xs
  shows bp-bucket-boundaries-state-ok
    (bp-bucketed-batch-prepend-state xs P)
proof (cases xs)
  case Nil
  then show ?thesis
    using bp-ordered-invariant-boundaries-state-ok[OF ord] by simp
next
  case (Cons p ps)
  let ?M = bp-block-size P
  let ?beta = bp-batch-max-value (snd p) ps
  let ?new = bp-bucketize-entries ?M xs
  let ?trimmed = bp-drop-empty-prefix (bp-buckets P)
  let ?old = bp-rebase-first-bucket-marker ?beta ?trimmed
  have inv: bp-invariant P
    by (rule bp-ordered-invariant-invariant[OF ord])
  have M-pos: 0 < ?M
    using inv unfolding bp-invariant-def by blast
  have trimmed-boundaries:
    bp-bucket-boundaries-ok ?trimmed
  by (rule bp-bucket-boundaries-ok-drop-empty-prefix)
  (use bp-ordered-invariant-boundaries-state-ok[OF ord] in
    ‹simp add: bp-bucket-boundaries-state-ok-def›)

```

```

have old-boundaries: bp-bucket-boundaries-ok ?old
proof (cases ?trimmed)
  case Nil
  then show ?thesis by simp
next
case (Cons b bs)
then show ?thesis
proof (cases bs)
  case Nil
  then show ?thesis
  using Cons by simp
next
case (Cons c cs)
then show ?thesis
  using Cons trimmed-boundaries by simp
qed
qed
have new-boundaries: bp-bucket-boundaries-ok ?new
  by (rule bp-bucketize-entries-boundaries-ok[OF M-pos])
have new-entries-le-beta:
   $\forall b \in \text{set } ?new. \forall q \in \text{set } (\text{bp-bucket-entries } b).$ 
   $\text{snd } q \leq ?beta$ 
proof (intro ballI)
  fix b q
  assume b:  $b \in \text{set } ?new$ 
  and q:  $q \in \text{set } (\text{bp-bucket-entries } b)$ 
  have  $q \in \text{set } xs$ 
  by (rule bp-bucketize-entries-entry-in-source[OF M-pos b q])
  then show  $\text{snd } q \leq ?beta$ 
  using Cons by (auto simp: bp-batch-max-value-ge-member-Cons)
qed
have cross:
   $?old \neq [] \implies$ 
   $\forall b \in \text{set } ?new. \forall q \in \text{set } (\text{bp-bucket-entries } b).$ 
   $\text{snd } q \leq \text{bp-marker } (\text{hd } ?old)$ 
proof -
  assume old-nonempty:  $?old \neq []$ 
  have bp-marker (hd ?old) = ?beta
  using old-nonempty by (cases ?trimmed) auto
  then show  $\forall b \in \text{set } ?new. \forall q \in \text{set } (\text{bp-bucket-entries } b).$ 
   $\text{snd } q \leq \text{bp-marker } (\text{hd } ?old)$ 
  using new-entries-le-beta by simp
qed
have appended: bp-bucket-boundaries-ok (?new @ ?old)
  by (rule bp-bucket-boundaries-ok-append
    [OF new-boundaries old-boundaries cross])
show ?thesis
  using appended Cons
  unfolding bp-bucketed-batch-prepend-state-def Let-def

```

$bp_bucket_boundaries_state_ok_def$
 by *simp*
 qed

lemma *bp-bucketed-batch-prepend-state-ordered-invariant*:
 assumes *ord*: *bp-ordered-invariant* P
 and *distinct*: *distinct* (*map fst* xs)
 and *disjoint*: $bp_entry_keys\ xs \cap keys_of\ (bp_view\ P) = \{\}$
 and *admissible*: *batch-prepend-admissible* (*bp-view* P) xs
 shows *bp-ordered-invariant* (*bp-bucketed-batch-prepend-state* $xs\ P$)
 unfolding *bp-ordered-invariant-def*
 using *bp-bucketed-batch-prepend-state-invariant*
 [*OF ord distinct disjoint admissible*]
bp-bucketed-batch-prepend-state-boundaries-state-ok
 [*OF ord admissible*]
 by *blast*

Bucketed BatchPrepend is not implemented as repeated Insert. The incoming batch is first bucketized on its own, then placed before the existing buckets. To preserve the boundary condition, the old prefix of empty buckets is dropped and the first surviving old bucket is rebased to the maximum value of the batch. The admissibility assumption is the abstract promise that every old entry is above the batch range. Together with distinctness and disjointness of keys, this yields both the view refinement $\llbracket bp_invariant\ ?P; distinct\ (map\ fst\ ?xs); bp_entry_keys\ ?xs \cap keys_of\ (bp_view\ ?P) = \{\} \rrbracket \implies bp_view\ (bp_bucketed_batch_prepend_state\ ?xs\ ?P) = batch_min_update\ (bp_view\ ?P)\ ?xs$ and the ordered invariant theorem above.

lemma *c-bp-bucketed-batch-prepend-direct-invariant*:
 assumes *ord*: *bp-ordered-invariant* P
 and *distinct*: *distinct* (*map fst* xs)
 and *disjoint*: $bp_entry_keys\ xs \cap keys_of\ (bp_view\ P) = \{\}$
 and *admissible*: *batch-prepend-admissible* (*bp-view* P) xs
 shows *bp-invariant*
 (*bp-result-of* (*c-bp-bucketed-batch-prepend-direct* $xs\ P$))
 unfolding *c-bp-bucketed-batch-prepend-direct-result*
 by (*rule bp-bucketed-batch-prepend-state-invariant*
 [*OF ord distinct disjoint admissible*])

lemma *c-bp-bucketed-batch-prepend-direct-ordered-invariant*:
 assumes *ord*: *bp-ordered-invariant* P
 and *distinct*: *distinct* (*map fst* xs)
 and *disjoint*: $bp_entry_keys\ xs \cap keys_of\ (bp_view\ P) = \{\}$
 and *admissible*: *batch-prepend-admissible* (*bp-view* P) xs
 shows *bp-ordered-invariant*
 (*bp-result-of* (*c-bp-bucketed-batch-prepend-direct* $xs\ P$))
 unfolding *c-bp-bucketed-batch-prepend-direct-result*
 by (*rule bp-bucketed-batch-prepend-state-ordered-invariant*
 [*OF ord distinct disjoint admissible*])

lemma *c-bp-bucketed-batch-prepend-direct-refines-batch-min-update*:
assumes *ord*: *bp-ordered-invariant P*
and *distinct*: *distinct (map fst xs)*
and *disjoint*: *bp-entry-keys xs ∩ keys-of (bp-view P) = {}*
shows *bp-view (bp-result-of (c-bp-bucketed-batch-prepend-direct xs P)) =*
batch-min-update (bp-view P) xs
unfolding *c-bp-bucketed-batch-prepend-direct-result*
by (*rule bp-bucketed-batch-prepend-state-refines-batch-min-update*
[OF bp-ordered-invariant-invariant[OF ord] distinct disjoint])

lemma *c-bp-bucketed-batch-prepend-direct-preserves-upper-bound*:
assumes *ord*: *bp-ordered-invariant P*
and *distinct*: *distinct (map fst xs)*
and *disjoint*: *bp-entry-keys xs ∩ keys-of (bp-view P) = {}*
and *upper*: *partition-upper-bound (bp-view P) B*
and *values-lt*: $\bigwedge x b. (x, b) \in \text{set } xs \implies b < B$
shows *partition-upper-bound*
(bp-view (bp-result-of (c-bp-bucketed-batch-prepend-direct xs P))) B
unfolding *c-bp-bucketed-batch-prepend-direct-refines-batch-min-update*
[OF ord distinct disjoint]
by (*rule batch-min-update-preserves-upper-bound[OF upper values-lt]*)

lemma *c-bp-bucketed-batch-prepend-direct-batch-cost-bound*:
partition-batch-cost-bound
(bp-steps-of (c-bp-bucketed-batch-prepend-direct xs P))
(bp-batch-prepend-per-item-budget xs (bp-block-size P)) xs
by (*rule c-bp-bucketed-batch-prepend-direct-paper-batch-cost-bound*)

lemma *bp-bucketed-batch-prepend-state-buckets-length-nonempty*:
assumes *M-pos*: $0 < \text{bp-block-size } P$
and *xs*: $xs \neq []$
shows $\text{length } (\text{bp-buckets } (\text{bp-bucketed-batch-prepend-state } xs P)) \leq$
 $\text{length } xs + \text{length } (\text{bp-buckets } P)$
proof –
obtain *p ps* **where** *xs-def*: $xs = p \# ps$
using *xs* **by** (*cases xs*) *auto*
have *new*:
 $\text{length } (\text{bp-bucketize-entries } (\text{bp-block-size } P) xs) \leq \text{length } xs$
by (*rule length-bp-bucketize-entries-le-length[OF M-pos]*)
have *new'*:
 $\text{length } (\text{bp-bucketize-entries } (\text{bp-block-size } P) (p \# ps)) \leq$
 $\text{Suc } (\text{length } ps)$
using *new* **unfolding** *xs-def* **by** *simp*
have *old*:
 $\text{length } (\text{bp-rebase-first-bucket-marker}$
 $(\text{bp-batch-max-value } (\text{snd } p) ps)$
 $(\text{bp-drop-empty-prefix } (\text{bp-buckets } P))) \leq$
 $\text{length } (\text{bp-buckets } P)$

```

    by simp
  have old':
    length (bp-drop-empty-prefix (bp-buckets P)) ≤
      length (bp-buckets P)
    by simp
  have total:
    length (bp-bucketize-entries (bp-block-size P) (p # ps)) +
      length (bp-drop-empty-prefix (bp-buckets P)) ≤
        Suc (length ps) + length (bp-buckets P)
    using new' old' by linarith
  show ?thesis
    unfolding bp-bucketed-batch-prepend-state-def xs-def Let-def
    using total by simp
qed

lemma bp-bucketed-batch-prepend-state-bucket-count-slack-le:
  assumes inv: bp-invariant P
  shows bp-bucket-count-slack (bp-bucketed-batch-prepend-state xs P) ≤
    length xs + bp-bucket-count-slack P
proof (cases xs = [])
  case True
    then show ?thesis by simp
  next
  case False
    let ?P' = bp-bucketed-batch-prepend-state xs P
    let ?M = bp-block-size P
    let ?N = length (bp-entries P)
    let ?N' = length (bp-entries ?P')
    let ?K = length (bp-buckets P)
    let ?K' = length (bp-buckets ?P')
    let ?L = length xs
    have M-pos: 0 < ?M
      using inv unfolding bp-invariant-def by blast
    have entries-len: ?N' = ?L + ?N
      using bp-bucketed-batch-prepend-state-entries-nonempty[OF M-pos False]
      by simp
    have buckets-len: ?K' ≤ ?L + ?K
      by (rule bp-bucketed-batch-prepend-state-buckets-length-nonempty
        [OF M-pos False])
    have div-le: Suc (?N div ?M) ≤ Suc (?N' div ?M)
      unfolding entries-len by (simp add: div-le-mono)
    have block: bp-block-size ?P' = ?M
      unfolding bp-bucketed-batch-prepend-state-def
      by (cases xs) (simp-all add: Let-def)
    have diff:
      ?K' - Suc (?N' div ?M) ≤
        ?L + (?K - Suc (?N div ?M))
      by (rule nat-diff-add-right-le[OF buckets-len div-le])
    show ?thesis

```

using *diff*
 unfolding *bp-bucket-count-slack-def entries-len block*
 by *simp*
 qed

lemma *bp-bucketed-batch-prepend-state-split-potential-le:*
 assumes *ord: bp-ordered-invariant P*
 and *distinct: distinct (map fst xs)*
 and *disjoint: bp-entry-keys xs $\cap$ keys-of (bp-view P) = {}*
 and *admissible: batch-prepend-admissible (bp-view P) xs*
 shows *bp-split-potential (bp-bucketed-batch-prepend-state xs P) $\leq$*
length xs + bp-split-potential P

proof –
 have *inv: bp-invariant P*
 by (rule *bp-ordered-invariant-invariant[OF ord]*)
 have *inv'*:
bp-invariant (bp-bucketed-batch-prepend-state xs P)
 by (rule *bp-bucketed-batch-prepend-state-invariant*
[OF ord distinct disjoint admissible])
 have *overflow'*:
bp-overflow-potential (bp-bucketed-batch-prepend-state xs P) = 0
 by (rule *bp-invariant-overflow-potential-zero[OF inv']*)
 have *slack*:
bp-bucket-count-slack (bp-bucketed-batch-prepend-state xs P) $\leq$
length xs + bp-bucket-count-slack P
 by (rule *bp-bucketed-batch-prepend-state-bucket-count-slack-le[OF inv]*)
 have *bp-split-potential (bp-bucketed-batch-prepend-state xs P) =*
bp-bucket-count-slack (bp-bucketed-batch-prepend-state xs P)
 unfolding *bp-split-potential-def* using *overflow'* by *simp*
 also have *... $\leq$ length xs + bp-bucket-count-slack P*
 by (rule *slack*)
 also have *... $\leq$ length xs + bp-split-potential P*
 unfolding *bp-split-potential-def* by *simp*
 finally show *?thesis* .
 qed

lemma *c-bp-bucketed-batch-prepend-direct-actual-refines-batch-min-update:*
 assumes *ord: bp-ordered-invariant P*
 and *distinct: distinct (map fst xs)*
 and *disjoint: bp-entry-keys xs $\cap$ keys-of (bp-view P) = {}*
 shows *bp-view*
(bp-result-of (c-bp-bucketed-batch-prepend-direct-actual xs P)) =
batch-min-update (bp-view P) xs
 unfolding *c-bp-bucketed-batch-prepend-direct-actual-result*
 by (rule *bp-bucketed-batch-prepend-state-refines-batch-min-update*
[OF bp-ordered-invariant-invariant[OF ord] distinct disjoint])

lemma *c-bp-bucketed-batch-prepend-direct-actual-invariant:*
 assumes *ord: bp-ordered-invariant P*

and *distinct*: *distinct* (map fst xs)
and *disjoint*: *bp-entry-keys* xs $\cap$ *keys-of* (bp-view P) = {}
and *admissible*: *batch-prepend-admissible* (bp-view P) xs
shows *bp-invariant*
 (*bp-result-of* (c-bp-bucketed-batch-prepend-direct-actual xs P))
unfolding *c-bp-bucketed-batch-prepend-direct-actual-result*
by (rule *bp-bucketed-batch-prepend-state-invariant*
 [OF ord distinct disjoint admissible])

lemma *c-bp-bucketed-batch-prepend-direct-actual-ordered-invariant*:
assumes ord: *bp-ordered-invariant* P
and *distinct*: *distinct* (map fst xs)
and *disjoint*: *bp-entry-keys* xs $\cap$ *keys-of* (bp-view P) = {}
and *admissible*: *batch-prepend-admissible* (bp-view P) xs
shows *bp-ordered-invariant*
 (*bp-result-of* (c-bp-bucketed-batch-prepend-direct-actual xs P))
unfolding *c-bp-bucketed-batch-prepend-direct-actual-result*
by (rule *bp-bucketed-batch-prepend-state-ordered-invariant*
 [OF ord distinct disjoint admissible])

lemma *c-bp-bucketed-batch-prepend-direct-actual-preserves-upper-bound*:
assumes ord: *bp-ordered-invariant* P
and *distinct*: *distinct* (map fst xs)
and *disjoint*: *bp-entry-keys* xs $\cap$ *keys-of* (bp-view P) = {}
and *upper*: *partition-upper-bound* (bp-view P) B
and *values-lt*: $\bigwedge x b. (x, b) \in \text{set } xs \implies b < B$
shows *partition-upper-bound*
 (*bp-view*
 (*bp-result-of* (c-bp-bucketed-batch-prepend-direct-actual xs P))) B
unfolding *c-bp-bucketed-batch-prepend-direct-actual-refines-batch-min-update*
 [OF ord distinct disjoint]
by (rule *batch-min-update-preserves-upper-bound*[OF upper values-lt])

lemma *c-bp-bucketed-batch-prepend-direct-actual-amortized-paper-bound*:
assumes ord: *bp-ordered-invariant* P
and *distinct*: *distinct* (map fst xs)
and *disjoint*: *bp-entry-keys* xs $\cap$ *keys-of* (bp-view P) = {}
and *admissible*: *batch-prepend-admissible* (bp-view P) xs
shows *bp-amortized-step-bound*
 (*bp-steps-of* (c-bp-bucketed-batch-prepend-direct-actual xs P)) P
 (*bp-result-of* (c-bp-bucketed-batch-prepend-direct-actual xs P))
 (*bp-batch-prepend-amortized-budget* xs (bp-block-size P))
proof –
have *pot*:
 bp-split-potential
 (*bp-bucketed-batch-prepend-state* xs P) $\leq$
 length xs + *bp-split-potential* P
by (rule *bp-bucketed-batch-prepend-state-split-potential-le*
 [OF ord distinct disjoint admissible])

show *?thesis*
using *pot*
unfolding *bp-amortized-step-bound-def*
bp-batch-prepend-amortized-budget-def
by *simp*
qed

lemma *c-bp-bucketed-batch-prepend-direct-actual-batch-cost-bound:*
partition-batch-cost-bound
(bp-steps-of (c-bp-bucketed-batch-prepend-direct-actual xs P))
(bp-ratio-log-budget (length xs) (bp-block-size P)) xs
unfolding *partition-batch-cost-bound-def*
c-bp-bucketed-batch-prepend-direct-actual-steps
bp-batch-prepend-log-budget-def
by (*simp add: mult.commute*)

lemma *c-bp-rebucketed-batch-prepend-bulk-regular-invariant:*
assumes *reg: bp-regular-invariant P*
shows *bp-regular-invariant*
(bp-result-of (c-bp-rebucketed-batch-prepend-bulk xs P))
unfolding *c-bp-rebucketed-batch-prepend-bulk-result*
by (*rule bp-rebucketed-batch-prepend-regular-invariant*
[OF bp-ordered-invariant-invariant
[OF bp-regular-invariant-ordered-invariant[OF reg]]])

lemma *c-bp-rebucketed-batch-prepend-bulk-refines-batch-min-update:*
assumes *reg: bp-regular-invariant P*
shows *bp-view (bp-result-of (c-bp-rebucketed-batch-prepend-bulk xs P)) =*
batch-min-update (bp-view P) xs
unfolding *c-bp-rebucketed-batch-prepend-bulk-result*
by (*rule bp-rebucketed-batch-prepend-refines-batch-min-update*
[OF bp-ordered-invariant-invariant
[OF bp-regular-invariant-ordered-invariant[OF reg]]])

lemma *c-bp-rebucketed-batch-prepend-bulk-preserves-upper-bound:*
assumes *reg: bp-regular-invariant P*
and *upper: partition-upper-bound (bp-view P) B*
and *values-lt: $\bigwedge x b. (x, b) \in \text{set } xs \implies b < B$*
shows *partition-upper-bound*
(bp-view (bp-result-of (c-bp-rebucketed-batch-prepend-bulk xs P))) B
unfolding *c-bp-rebucketed-batch-prepend-bulk-result*
by (*rule bp-rebucketed-batch-prepend-preserves-upper-bound*
[OF bp-ordered-invariant-invariant
[OF bp-regular-invariant-ordered-invariant[OF reg]]
upper values-lt])

lemma *c-bp-rebucketed-batch-prepend-bulk-split-potential-zero:*
assumes *reg: bp-regular-invariant P*
shows *bp-split-potential*

$(\text{bp-result-of } (c\text{-bp-rebucketed-batch-prepend-bulk } xs \ P)) = 0$
unfolding $c\text{-bp-rebucketed-batch-prepend-bulk-result}$
by (rule $\text{bp-rebucketed-batch-prepend-split-potential-zero}$
 $[OF \ \text{bp-ordered-invariant-invariant}$
 $[OF \ \text{bp-regular-invariant-ordered-invariant}[OF \ \text{reg}]]])$

lemma $c\text{-bp-rebucketed-batch-prepend-bulk-amortized-bound}$:
assumes $\text{reg: bp-regular-invariant } P$
shows $\text{bp-amortized-step-bound}$
 $(\text{bp-steps-of } (c\text{-bp-rebucketed-batch-prepend-bulk } xs \ P)) \ P$
 $(\text{bp-result-of } (c\text{-bp-rebucketed-batch-prepend-bulk } xs \ P))$
 $(\text{bp-batch-prepend-log-budget } xs \ (\text{bp-block-size } P))$
using $c\text{-bp-rebucketed-batch-prepend-bulk-split-potential-zero}[OF \ \text{reg}, \ \text{of } xs]$
 $\text{bp-regular-invariant-split-potential-zero}[OF \ \text{reg}]$
unfolding $\text{bp-amortized-step-bound-def}$ **by** simp

lemma $c\text{-bp-rebucketed-batch-prepend-bulk-steps-le-amortized-budget}$:
 $\text{bp-steps-of } (c\text{-bp-rebucketed-batch-prepend-bulk } xs \ P) \leq$
 $\text{bp-batch-prepend-amortized-budget } xs \ (\text{bp-block-size } P)$
using $\text{bp-batch-prepend-log-budget-le-amortized-budget}[\text{of } xs \ \text{bp-block-size } P]$
by simp

lemma $c\text{-bp-rebucketed-batch-prepend-bulk-amortized-paper-bound}$:
assumes $\text{reg: bp-regular-invariant } P$
shows $\text{bp-amortized-step-bound}$
 $(\text{bp-steps-of } (c\text{-bp-rebucketed-batch-prepend-bulk } xs \ P)) \ P$
 $(\text{bp-result-of } (c\text{-bp-rebucketed-batch-prepend-bulk } xs \ P))$
 $(\text{bp-batch-prepend-amortized-budget } xs \ (\text{bp-block-size } P))$
proof –
have $\text{base: bp-amortized-step-bound}$
 $(\text{bp-steps-of } (c\text{-bp-rebucketed-batch-prepend-bulk } xs \ P)) \ P$
 $(\text{bp-result-of } (c\text{-bp-rebucketed-batch-prepend-bulk } xs \ P))$
 $(\text{bp-batch-prepend-log-budget } xs \ (\text{bp-block-size } P))$
by (rule $c\text{-bp-rebucketed-batch-prepend-bulk-amortized-bound}[OF \ \text{reg}]$)
have budget:
 $\text{bp-batch-prepend-log-budget } xs \ (\text{bp-block-size } P) \leq$
 $\text{bp-batch-prepend-amortized-budget } xs \ (\text{bp-block-size } P)$
by (rule $\text{bp-batch-prepend-log-budget-le-amortized-budget}$)
show $?thesis$
using $\text{base budget unfolding bp-amortized-step-bound-def by linarith}$
qed

lemma $c\text{-bp-rebucketed-batch-prepend-bulk-batch-cost-bound}$:
 $\text{partition-batch-cost-bound}$
 $(\text{bp-steps-of } (c\text{-bp-rebucketed-batch-prepend-bulk } xs \ P))$
 $(\text{bp-ratio-log-budget } (\text{length } xs) \ (\text{bp-block-size } P)) \ xs$
unfolding $\text{partition-batch-cost-bound-def}$
by ($\text{simp add: } c\text{-bp-rebucketed-batch-prepend-bulk-steps}$
 $\text{bp-batch-prepend-log-budget-def mult.commute}$)

lemma *c-bp-rebucketed-batch-prepend-bulk-paper-batch-cost-bound*:
partition-batch-cost-bound
 (*bp-steps-of* (*c-bp-rebucketed-batch-prepend-bulk* *xs P*))
 (*bp-batch-prepend-per-item-budget* *xs* (*bp-block-size P*)) *xs*

proof –
have *steps*:
 bp-steps-of (*c-bp-rebucketed-batch-prepend-bulk* *xs P*) ≤
 bp-batch-prepend-amortized-budget *xs* (*bp-block-size P*)
by (*rule c-bp-rebucketed-batch-prepend-bulk-steps-le-amortized-budget*)
show *?thesis*
using *steps*
unfolding *partition-batch-cost-bound-def*
 bp-batch-prepend-amortized-budget-alt
by *simp*

qed

lemma *c-bp-first-bucket-pull-result* [*simp*]:
bp-result-of (*c-bp-first-bucket-pull* *M B P*) =
bp-first-bucket-pull *M B P*
unfolding *bp-result-of-def c-bp-first-bucket-pull-def* **by** *simp*

lemma *c-bp-first-bucket-pull-steps* [*simp*]:
bp-steps-of (*c-bp-first-bucket-pull* *M B P*) = *M*
unfolding *bp-steps-of-def c-bp-first-bucket-pull-def* **by** *simp*

lemma *c-bp-first-bucket-pull-M-bound*:
bp-steps-of (*c-bp-first-bucket-pull* *M B P*) ≤ *M*
by *simp*

lemma *c-bp-first-bucket-pull-scan-result* [*simp*]:
bp-result-of (*c-bp-first-bucket-pull-scan* *M B P*) =
bp-first-bucket-pull *M B P*
unfolding *bp-result-of-def c-bp-first-bucket-pull-scan-def*
by (*auto split: prod.splits*)

lemma *c-bp-first-bucket-pull-scan-steps*:
assumes *bp-first-bucket-pull* *M B P* = (*S*, *beta*, *P'*)
shows *bp-steps-of* (*c-bp-first-bucket-pull-scan* *M B P*) = *card S*
using *assms* **unfolding** *bp-steps-of-def c-bp-first-bucket-pull-scan-def*
by *simp*

lemma *bp-bucket-keys-card-le-length*:
card (*bp-bucket-keys* *b*) ≤ *length* (*bp-bucket-entries* *b*)
unfolding *bp-bucket-keys-def bp-entry-keys-def*
by (*metis card-length length-map set-map*)

lemma *c-bp-first-bucket-pull-scan-M-bound*:
assumes *can*: *bp-can-first-bucket-pull* *M P*

shows $bp\text{-steps-of } (c\text{-bp-first-bucket-pull-scan } M \ B \ P) \leq M$
proof –
obtain $b \ c \ bs$ **where** $bp\text{-buckets } P = b \ \# \ c \ \# \ bs$
and $len\text{-}b$: $length \ (bp\text{-bucket-entries } b) \leq M$
by (rule $bp\text{-can-first-bucket-pullE}[OF \ can]$)
have $pull$:
 $bp\text{-first-bucket-pull } M \ B \ P =$
 $(bp\text{-bucket-keys } b, bp\text{-marker } c, bp\text{-delete-keys } (bp\text{-bucket-keys } b) \ P)$
unfolding $bp\text{-first-bucket-pull-def}$ $bp\text{-buckets}$ **by** $simp$
have $bp\text{-steps-of } (c\text{-bp-first-bucket-pull-scan } M \ B \ P) =$
 $card \ (bp\text{-bucket-keys } b)$
by (rule $c\text{-bp-first-bucket-pull-scan-steps}[OF \ pull]$)
also have $\dots \leq length \ (bp\text{-bucket-entries } b)$
by (rule $bp\text{-bucket-keys-card-le-length}$)
also have $\dots \leq M$
by (rule $len\text{-}b$)
finally show $?thesis$.
qed

lemma $c\text{-bp-first-bucket-pull-scan-partition-pull-cost-bound}$:
assumes $pull$: $bp\text{-first-bucket-pull } M \ B \ P = (S, \beta, P')$
shows $partition\text{-pull-cost-bound}$
 $(bp\text{-steps-of } (c\text{-bp-first-bucket-pull-scan } M \ B \ P)) \ S$
using $c\text{-bp-first-bucket-pull-scan-steps}[OF \ pull]$
unfolding $partition\text{-pull-cost-bound-def}$ **by** $simp$

lemma $c\text{-bp-first-bucket-pull-scan-partition-pull-cost-bound-from-result}$:
assumes $pull$:
 $bp\text{-result-of } (c\text{-bp-first-bucket-pull-scan } M \ B \ P) = (S, \beta, P')$
shows $partition\text{-pull-cost-bound}$
 $(bp\text{-steps-of } (c\text{-bp-first-bucket-pull-scan } M \ B \ P)) \ S$
proof –
have $pull'$: $bp\text{-first-bucket-pull } M \ B \ P = (S, \beta, P')$
using $pull$ **by** $simp$
show $?thesis$
by (rule $c\text{-bp-first-bucket-pull-scan-partition-pull-cost-bound}[OF \ pull']$)
qed

lemma $c\text{-bp-first-bucket-pull-scan-invariant}$:
assumes inv : $bp\text{-invariant } P$
shows $bp\text{-invariant}$
 $(bp\text{-pull-state-of } (bp\text{-result-of } (c\text{-bp-first-bucket-pull-scan } M \ B \ P)))$
proof –
obtain $S \ \beta \ P'$ **where** $pull$: $bp\text{-first-bucket-pull } M \ B \ P = (S, \beta, P')$
by (cases $bp\text{-first-bucket-pull } M \ B \ P$)
have $bp\text{-invariant } P'$
by (rule $bp\text{-first-bucket-pull-invariant}[OF \ inv \ pull]$)
then show $?thesis$
using $pull$ **unfolding** $bp\text{-pull-state-of-def}$ **by** $simp$

qed

lemma *c-bp-first-bucket-pull-scan-amortized-M-bound:*

assumes *inv*: *bp-invariant P*

and *can*: *bp-can-first-bucket-pull M P*

shows *bp-pull-amortized-step-bound*

(bp-steps-of (c-bp-first-bucket-pull-scan M B P)) P

*(bp-result-of (c-bp-first-bucket-pull-scan M B P)) (2 * M)*

proof –

obtain *b c bs* **where** *buckets*: *bp-buckets P = b # c # bs*

and *len-b*: *length (bp-bucket-entries b) ≤ M*

by (*rule bp-can-first-bucket-pullE[OF can]*)

let *?S* = *bp-bucket-keys b*

have *pull*:

bp-first-bucket-pull M B P =

(?S, bp-marker c, bp-delete-keys ?S P)

unfolding *bp-first-bucket-pull-def buckets* **by** *simp*

have *state*:

bp-pull-state-of

(bp-result-of (c-bp-first-bucket-pull-scan M B P)) =

bp-delete-keys ?S P

unfolding *bp-pull-state-of-def* **using** *pull* **by** *simp*

have *steps*: *bp-steps-of (c-bp-first-bucket-pull-scan M B P) ≤ M*

by (*rule c-bp-first-bucket-pull-scan-M-bound[OF can]*)

have *card-S*: *card ?S ≤ M*

proof –

have *card ?S ≤ length (bp-bucket-entries b)*

by (*rule bp-bucket-keys-card-le-length*)

also have *... ≤ M*

by (*rule len-b*)

finally show *?thesis* .

qed

have *finite-S*: *finite ?S*

unfolding *bp-bucket-keys-def bp-entry-keys-def* **by** *simp*

have *potential*:

bp-split-potential (bp-delete-keys ?S P) ≤

bp-split-potential P + card ?S

proof (*rule bp-delete-keys-split-potential-le*)

show *0 < bp-block-size P*

using *inv* **unfolding** *bp-invariant-def* **by** *blast*

show *bp-distinct-keys P*

using *inv* **unfolding** *bp-invariant-def* **by** *blast*

show *finite ?S*

by (*rule finite-S*)

qed

show *?thesis*

using *steps card-S potential* **unfolding** *bp-pull-amortized-step-bound-def state*

by *linarith*

qed

lemma *c-bp-first-bucket-pull-scan-amortized-paper-bound*:

assumes *inv*: *bp-invariant P*

and *can*: *bp-can-first-bucket-pull M P*

shows *bp-pull-amortized-step-bound*

(*bp-steps-of* (*c-bp-first-bucket-pull-scan M B P*)) *P*

(*bp-result-of* (*c-bp-first-bucket-pull-scan M B P*))

(*bp-pull-amortized-budget M*)

unfolding *bp-pull-amortized-budget-def*

by (*rule c-bp-first-bucket-pull-scan-amortized-M-bound[OF inv can]*)

lemma *c-bp-first-bucket-pull-scan-refines-pull-separates*:

assumes *inv*: *bp-invariant P*

and *can*: *bp-can-first-bucket-pull M P*

and *pull*:

bp-result-of (*c-bp-first-bucket-pull-scan M B P*) = (*S*, *beta*, *P'*)

shows *pull-separates* (*bp-view P*) *M B S beta* (*bp-view P'*)

proof –

obtain *b c bs* **where** *buckets*: *bp-buckets P* = *b # c # bs*

and *len-b*: *length* (*bp-bucket-entries b*) ≤ *M*

and *below*: *bp-bucket-below-bound b* (*bp-marker c*)

and *tail-nonempty*: *bp-bucket-entries-flat* (*c # bs*) ≠ []

by (*rule bp-can-first-bucket-pullE[OF can]*)

have *pull'*: *bp-first-bucket-pull M B P* = (*S*, *beta*, *P'*)

using *pull* **by** *simp*

show *?thesis*

by (*rule bp-first-bucket-pull-refines-pull-separates*

[*OF inv buckets len-b below tail-nonempty pull'*])

qed

Pull is charged by the size of the set it returns. The first-bucket policy is sound when *bp-can-first-bucket-pull* exposes two adjacent nonempty ranges and the first bucket lies entirely below the next marker. In that case the returned boundary is the next marker, the deleted set is the key set of the first bucket, and *pull-separates* follows from the bucket boundary invariant. The scan-cost lemmas count exactly the returned keys, so the primitive cost is at most *M*; the amortized Pull bound additionally allows potential to change after deleting those keys.

14.1 Paper-Facing Costed Operations

The following aliases are the operations exported to the rest of the BMSSP development. They choose the lazy precise Insert, the direct bucketed BatchPrepend, and the first-bucket Pull scan. The aliases keep the public layer small: clients do not need to know which internal costed variant proved the bound, only that the selected operation refines the abstract interface and has the paper budget.

definition *c-bp-paper-insert* ::
 $'k \Rightarrow \text{real} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow$
 $'k \text{ bucketed-partition} \times \text{nat}$ **where**
c-bp-paper-insert $x \ b \ P = \text{c-bp-lazy-insert-precise } x \ b \ P$

definition *c-bp-paper-batch-prepend* ::
 $('k \times \text{real}) \text{ list} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow$
 $'k \text{ bucketed-partition} \times \text{nat}$ **where**
c-bp-paper-batch-prepend $xs \ P =$
c-bp-bucketed-batch-prepend-direct-actual $xs \ P$

definition *c-bp-paper-pull* ::
 $\text{nat} \Rightarrow \text{real} \Rightarrow 'k \text{ bucketed-partition} \Rightarrow$
 $('k \text{ set} \times \text{real} \times 'k \text{ bucketed-partition}) \times \text{nat}$ **where**
c-bp-paper-pull $M \ B \ P = \text{c-bp-first-bucket-pull-scan } M \ B \ P$

definition *bp-insert-paper-budget* :: $\text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat}$ **where**
bp-insert-paper-budget $N \ M =$
 $9 + \text{floor-log } (\text{Suc } (N \text{ div } M))$

definition *bp-batch-prepend-paper-budget* :: $\text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat}$ **where**
bp-batch-prepend-paper-budget $N \ M =$
 $2 + \text{floor-log } (\text{Suc } (N \text{ div } M))$

definition *bp-pull-paper-budget* :: $\text{nat} \Rightarrow \text{nat}$ **where**
bp-pull-paper-budget $M = 2 * M$

lemma *bp-lazy-insert-amortized-budget-paper-form*:
bp-lazy-insert-amortized-budget $P =$
bp-insert-paper-budget $(\text{length } (\text{bp-entries } P)) \ (\text{bp-block-size } P)$
unfolding *bp-lazy-insert-amortized-budget-def*
bp-insert-search-budget-def *bp-ratio-log-budget-def*
bp-insert-paper-budget-def
by *simp*

lemma *bp-batch-prepend-amortized-budget-paper-form*:
bp-batch-prepend-amortized-budget $xs \ M =$
 $\text{length } xs * \text{bp-batch-prepend-paper-budget } (\text{length } xs) \ M$
unfolding *bp-batch-prepend-amortized-budget-def*
bp-batch-prepend-log-budget-def *bp-ratio-log-budget-def*
bp-batch-prepend-paper-budget-def
by $(\text{simp add: algebra-simps})$

lemma *bp-ratio-log-budget-le-insert-paper-budget*:
bp-ratio-log-budget $N \ M \leq \text{bp-insert-paper-budget } N \ M$
unfolding *bp-ratio-log-budget-def* *bp-insert-paper-budget-def* **by** *simp*

lemma *bp-ratio-log-budget-le-batch-prepend-paper-budget*:
bp-ratio-log-budget $N \ M \leq \text{bp-batch-prepend-paper-budget } N \ M$

unfolding *bp-ratio-log-budget-def bp-batch-prepend-paper-budget-def* **by** *simp*

lemma *bp-insert-paper-budget-bigo-ratio-log*:
 $(\lambda N. \text{real } (bp\text{-insert-paper-budget } N \ M)) \in O(\lambda N. \text{real } (bp\text{-ratio-log-budget } N \ M))$

proof –
have *bound*:
 $\bigwedge N. \text{norm } (\text{real } (bp\text{-insert-paper-budget } N \ M)) \leq 9 * \text{norm } (\text{real } (bp\text{-ratio-log-budget } N \ M))$

proof –
fix *N*
have *bp-insert-paper-budget* *N M* $\leq 9 * bp\text{-ratio-log-budget } N \ M$
unfolding *bp-insert-paper-budget-def bp-ratio-log-budget-def* **by** *simp*
then show $\text{norm } (\text{real } (bp\text{-insert-paper-budget } N \ M)) \leq 9 * \text{norm } (\text{real } (bp\text{-ratio-log-budget } N \ M))$
by *simp*

qed
show *?thesis*
unfolding *bigo-def*
by (*intro CollectI exI[of - 9] eventuallyI*)
(simp add: bp-insert-paper-budget-def bp-ratio-log-budget-def)

qed

lemma *bp-batch-prepend-paper-budget-bigo-ratio-log*:
 $(\lambda N. \text{real } (bp\text{-batch-prepend-paper-budget } N \ M)) \in O(\lambda N. \text{real } (bp\text{-ratio-log-budget } N \ M))$

proof –
have *bound*:
 $\bigwedge N. \text{norm } (\text{real } (bp\text{-batch-prepend-paper-budget } N \ M)) \leq 2 * \text{norm } (\text{real } (bp\text{-ratio-log-budget } N \ M))$

proof –
fix *N*
have *bp-batch-prepend-paper-budget* *N M* $\leq 2 * bp\text{-ratio-log-budget } N \ M$
unfolding *bp-batch-prepend-paper-budget-def bp-ratio-log-budget-def*
by *simp*
then show $\text{norm } (\text{real } (bp\text{-batch-prepend-paper-budget } N \ M)) \leq 2 * \text{norm } (\text{real } (bp\text{-ratio-log-budget } N \ M))$
by *simp*

qed
show *?thesis*
unfolding *bigo-def*
by (*intro CollectI exI[of - 2] eventuallyI*)
(simp add: bp-batch-prepend-paper-budget-def bp-ratio-log-budget-def)

qed

lemma *bp-pull-paper-budget-bigo-linear*:
 $(\lambda M. \text{real } (bp\text{-pull-paper-budget } M)) \in O(\lambda M. \text{real } M)$

unfolding *bp-pull-paper-budget-def bigo-def*
by (*intro CollectI exI[of - 2] eventuallyI*) *simp*

The named paper budgets are intentionally simple. The Insert budget *bp-insert-paper-budget* is a constant plus the ratio-log term over stored entries and block size. The BatchPrepend budget *bp-batch-prepend-paper-budget* is the corresponding per-item budget; the abstract batch predicate then multiplies it by the batch length. Pull uses *bp-pull-paper-budget*, linear in the requested block size. The Big-O lemmas above record exactly the envelopes consumed by the headline bridge.

lemma *c-bp-paper-insert-result [simp]:*
bp-result-of (c-bp-paper-insert x b P) =
bp-lazy-insert-state x b P
unfolding *c-bp-paper-insert-def* **by** *simp*

lemma *c-bp-paper-insert-steps [simp]:*
bp-steps-of (c-bp-paper-insert x b P) =
bp-local-insert-search-charge P + bp-lazy-insert-charge x b P
unfolding *c-bp-paper-insert-def* **by** *simp*

lemma *c-bp-paper-insert-refines-min-update:*
assumes *lazy: bp-lazy-ordered-invariant P*
shows *bp-view (bp-result-of (c-bp-paper-insert x b P)) =*
min-update (bp-view P) x b

proof –
have *M-pos: 0 < bp-block-size P*
using *lazy* **unfolding** *bp-lazy-ordered-invariant-def*
bp-lazy-invariant-def **by** *blast*
show *?thesis*
unfolding *c-bp-paper-insert-def*
by (*rule c-bp-lazy-insert-precise-refines-min-update[OF M-pos]*)
qed

lemma *c-bp-paper-insert-refines-insert-spec:*
assumes *lazy: bp-lazy-ordered-invariant P*
shows *insert-spec (bp-view P) x b*
(bp-view (bp-result-of (c-bp-paper-insert x b P)))
unfolding *c-bp-paper-insert-refines-min-update[OF lazy]*
by (*rule min-update-insert-spec*)

lemma *c-bp-paper-insert-lazy-ordered-invariant:*
assumes *lazy: bp-lazy-ordered-invariant P*
shows *bp-lazy-ordered-invariant*
(bp-result-of (c-bp-paper-insert x b P))
unfolding *c-bp-paper-insert-def*
by (*rule c-bp-lazy-insert-precise-lazy-ordered-invariant[OF lazy]*)

lemma *c-bp-paper-insert-amortized-paper-bound:*
assumes *lazy: bp-lazy-ordered-invariant P*

and *ratio*: *bp-bucket-count-ratio-ok* *P*
shows *bp-amortized-step-bound*
(bp-steps-of (c-bp-paper-insert x b P)) P
(bp-result-of (c-bp-paper-insert x b P))
(bp-lazy-insert-amortized-budget P)
unfolding *c-bp-paper-insert-def*
by (*rule c-bp-lazy-insert-precise-amortized-ratio-budget*
[OF lazy ratio])

lemma *c-bp-paper-insert-regular-partition-insert-cost-bound*:
assumes *reg*: *bp-regular-invariant P*
shows *partition-insert-cost-bound*
(bp-steps-of (c-bp-paper-insert x b P))
(bp-lazy-insert-amortized-budget P)
unfolding *c-bp-paper-insert-def*
by (*rule c-bp-lazy-insert-precise-regular-partition-insert-cost-bound*
[OF reg])

lemma *c-bp-paper-batch-prepend-result [simp]*:
bp-result-of (c-bp-paper-batch-prepend xs P) =
bp-bucketed-batch-prepend-state xs P
unfolding *c-bp-paper-batch-prepend-def* **by** *simp*

lemma *c-bp-paper-batch-prepend-steps [simp]*:
bp-steps-of (c-bp-paper-batch-prepend xs P) =
bp-batch-prepend-log-budget xs (bp-block-size P)
unfolding *c-bp-paper-batch-prepend-def* **by** *simp*

lemma *c-bp-paper-batch-prepend-refines-batch-min-update*:
assumes *ord*: *bp-ordered-invariant P*
and *distinct*: *distinct (map fst xs)*
and *disjoint*: *bp-entry-keys xs ∩ keys-of (bp-view P) = {}*
shows *bp-view (bp-result-of (c-bp-paper-batch-prepend xs P)) =*
batch-min-update (bp-view P) xs
unfolding *c-bp-paper-batch-prepend-def*
by (*rule c-bp-bucketed-batch-prepend-direct-actual-refines-batch-min-update*
[OF ord distinct disjoint])

lemma *c-bp-paper-batch-prepend-ordered-invariant*:
assumes *ord*: *bp-ordered-invariant P*
and *distinct*: *distinct (map fst xs)*
and *disjoint*: *bp-entry-keys xs ∩ keys-of (bp-view P) = {}*
and *admissible*: *batch-prepend-admissible (bp-view P) xs*
shows *bp-ordered-invariant*
(bp-result-of (c-bp-paper-batch-prepend xs P))
unfolding *c-bp-paper-batch-prepend-def*
by (*rule c-bp-bucketed-batch-prepend-direct-actual-ordered-invariant*
[OF ord distinct disjoint admissible])

lemma *c-bp-paper-batch-prepend-preserves-upper-bound*:
assumes *ord*: *bp-ordered-invariant P*
and *distinct*: *distinct (map fst xs)*
and *disjoint*: *bp-entry-keys xs $\cap$ keys-of (bp-view P) = {}*
and *upper*: *partition-upper-bound (bp-view P) B*
and *values-lt*: $\bigwedge x b. (x, b) \in \text{set } xs \implies b < B$
shows *partition-upper-bound*
(bp-view (bp-result-of (c-bp-paper-batch-prepend xs P))) B
unfolding *c-bp-paper-batch-prepend-def*
by (*rule c-bp-bucketed-batch-prepend-direct-actual-preserves-upper-bound*
 $[OF \text{ } ord \text{ } distinct \text{ } disjoint \text{ } upper \text{ } values-lt]$)

lemma *c-bp-paper-batch-prepend-amortized-paper-bound*:
assumes *ord*: *bp-ordered-invariant P*
and *distinct*: *distinct (map fst xs)*
and *disjoint*: *bp-entry-keys xs $\cap$ keys-of (bp-view P) = {}*
and *admissible*: *batch-prepend-admissible (bp-view P) xs*
shows *bp-amortized-step-bound*
(bp-steps-of (c-bp-paper-batch-prepend xs P)) P
(bp-result-of (c-bp-paper-batch-prepend xs P))
(bp-batch-prepend-amortized-budget xs (bp-block-size P))
unfolding *c-bp-paper-batch-prepend-def*
by (*rule c-bp-bucketed-batch-prepend-direct-actual-amortized-paper-bound*
 $[OF \text{ } ord \text{ } distinct \text{ } disjoint \text{ } admissible]$)

lemma *c-bp-paper-batch-prepend-batch-cost-bound*:
partition-batch-cost-bound
(bp-steps-of (c-bp-paper-batch-prepend xs P))
(bp-ratio-log-budget (length xs) (bp-block-size P)) xs
unfolding *c-bp-paper-batch-prepend-def*
by (*rule c-bp-bucketed-batch-prepend-direct-actual-batch-cost-bound*)

lemma *c-bp-paper-pull-result [simp]*:
bp-result-of (c-bp-paper-pull M B P) =
bp-first-bucket-pull M B P
unfolding *c-bp-paper-pull-def* **by** *simp*

lemma *c-bp-paper-pull-M-bound*:
assumes *can*: *bp-can-first-bucket-pull M P*
shows *bp-steps-of (c-bp-paper-pull M B P) $\leq$ M*
unfolding *c-bp-paper-pull-def*
by (*rule c-bp-first-bucket-pull-scan-M-bound* $[OF \text{ } can]$)

lemma *c-bp-paper-pull-partition-pull-cost-bound*:
assumes *pull*:
bp-result-of (c-bp-paper-pull M B P) = (S, beta, P')
shows *partition-pull-cost-bound*
(bp-steps-of (c-bp-paper-pull M B P)) S
unfolding *c-bp-paper-pull-def*

by (rule *c-bp-first-bucket-pull-scan-partition-pull-cost-bound-from-result*
[*OF pull[unfolded c-bp-paper-pull-def]*])

lemma *c-bp-paper-pull-invariant*:
assumes *inv*: *bp-invariant P*
shows *bp-invariant*
 (*bp-pull-state-of (bp-result-of (c-bp-paper-pull M B P))*)
unfolding *c-bp-paper-pull-def*
by (rule *c-bp-first-bucket-pull-scan-invariant[OF inv]*)

lemma *c-bp-paper-pull-amortized-paper-bound*:
assumes *inv*: *bp-invariant P*
and *can*: *bp-can-first-bucket-pull M P*
shows *bp-pull-amortized-step-bound*
 (*bp-steps-of (c-bp-paper-pull M B P)*) *P*
 (*bp-result-of (c-bp-paper-pull M B P)*)
 (*bp-pull-amortized-budget M*)
unfolding *c-bp-paper-pull-def*
by (rule *c-bp-first-bucket-pull-scan-amortized-paper-bound[OF inv can]*)

lemma *c-bp-paper-pull-refines-pull-separates*:
assumes *inv*: *bp-invariant P*
and *can*: *bp-can-first-bucket-pull M P*
and *pull*:
 (*bp-result-of (c-bp-paper-pull M B P) = (S, beta, P')*)
shows *pull-separates (bp-view P) M B S beta (bp-view P')*
unfolding *c-bp-paper-pull-def*
by (rule *c-bp-first-bucket-pull-scan-refines-pull-separates*
[*OF inv can pull[unfolded c-bp-paper-pull-def]*])

corollary *bp-realises-partition-insert-cost-bound*:
assumes *reg*: *bp-regular-invariant P*
shows $\exists t.$ *partition-insert-cost-bound*
 (*bp-steps-of (c-bp-paper-insert x b P)*) *t* $\wedge$
 $t = \text{bp-insert-paper-budget } (\text{length } (\text{bp-entries } P)) (\text{bp-block-size } P)$
proof (*intro exI conjI*)
show *partition-insert-cost-bound*
 (*bp-steps-of (c-bp-paper-insert x b P)*)
 (*bp-insert-paper-budget (length (bp-entries P)) (bp-block-size P)*)
using *c-bp-paper-insert-regular-partition-insert-cost-bound[OF reg, of x b]*
by (*simp add: bp-lazy-insert-amortized-budget-paper-form*)
qed *simp*

corollary *bp-realises-partition-batch-cost-bound*:
shows $\exists t.$ *partition-batch-cost-bound*
 (*bp-steps-of (c-bp-paper-batch-prepend xs P)*) *t* $\wedge$
 $t = \text{bp-batch-prepend-paper-budget } (\text{length } xs) (\text{bp-block-size } P)$
proof (*intro exI conjI*)
have *cost*:

```

partition-batch-cost-bound
  (bp-steps-of (c-bp-paper-batch-prepend xs P))
  (bp-ratio-log-budget (length xs) (bp-block-size P)) xs
  by (rule c-bp-paper-batch-prepend-batch-cost-bound)
have le:
  bp-ratio-log-budget (length xs) (bp-block-size P) ≤
  bp-batch-prepend-paper-budget (length xs) (bp-block-size P)
  by (rule bp-ratio-log-budget-le-batch-prepend-paper-budget)
show partition-batch-cost-bound
  (bp-steps-of (c-bp-paper-batch-prepend xs P))
  (bp-batch-prepend-paper-budget (length xs) (bp-block-size P)) xs
proof -
  have steps-le:
    bp-steps-of (c-bp-paper-batch-prepend xs P) ≤
    bp-ratio-log-budget (length xs) (bp-block-size P) * length xs
    using cost unfolding partition-batch-cost-bound-def .
  have budget-le:
    bp-ratio-log-budget (length xs) (bp-block-size P) * length xs ≤
    bp-batch-prepend-paper-budget (length xs) (bp-block-size P) *
    length xs
    using le by simp
  show ?thesis
    using steps-le budget-le unfolding partition-batch-cost-bound-def
    by linarith
qed
qed simp

```

corollary *bp-realises-partition-pull-cost-bound*:

```

assumes pull:
  bp-result-of (c-bp-paper-pull M B P) = (S, beta, P')
shows partition-pull-cost-bound
  (bp-steps-of (c-bp-paper-pull M B P)) S
by (rule c-bp-paper-pull-partition-pull-cost-bound[OF pull])

```

The realization corollaries are the interface-facing summary. They convert the concrete step counters of *c-bp-paper-insert*, *c-bp-paper-batch-prepend*, and *c-bp-paper-pull* into the abstract predicates *partition-insert-cost-bound*, *partition-batch-cost-bound*, and *partition-pull-cost-bound*. These are the facts imported by the headline bridge; the operation-specific theorems below keep the sharper amortized statements visible for readers who want to inspect the data-structure proof itself.

theorem *bp-insert-cost-bound*:

```

assumes lazy: bp-lazy-ordered-invariant P
and ratio: bp-bucket-count-ratio-ok P
shows bp-amortized-step-bound
  (bp-steps-of (c-bp-paper-insert x b P)) P
  (bp-result-of (c-bp-paper-insert x b P))
  (9 + floor-log
    (Suc (length (bp-entries P) div bp-block-size P)))

```

using *c-bp-paper-insert-amortized-paper-bound*[*OF* lazy ratio, of *x b*]
by (*simp add: bp-lazy-insert-amortized-budget-paper-form*
bp-insert-paper-budget-def)

theorem *bp-batch-prepend-cost-bound*:
assumes *ord*: *bp-ordered-invariant P*
and *distinct*: *distinct (map fst xs)*
and *disjoint*: *bp-entry-keys xs ∩ keys-of (bp-view P) = {}*
and *admissible*: *batch-prepend-admissible (bp-view P) xs*
shows *bp-amortized-step-bound*
(bp-steps-of (c-bp-paper-batch-prepend xs P)) P
(bp-result-of (c-bp-paper-batch-prepend xs P))
*(length xs * (2 + floor-log (Suc (length xs div bp-block-size P))))*
using *c-bp-paper-batch-prepend-amortized-paper-bound*
[*OF ord distinct disjoint admissible*]
by (*simp add: bp-batch-prepend-amortized-budget-paper-form*
bp-batch-prepend-paper-budget-def)

theorem *bp-pull-cost-bound*:
assumes *inv*: *bp-invariant P*
and *can*: *bp-can-first-bucket-pull M P*
shows *bp-pull-amortized-step-bound*
(bp-steps-of (c-bp-paper-pull M B P)) P
(bp-result-of (c-bp-paper-pull M B P))
*(2 * M)*
using *c-bp-paper-pull-amortized-paper-bound*[*OF inv can, of B*]
unfolding *bp-pull-amortized-budget-def* .

The three theorems above are the exported paper-tight cost bounds for the bucketed structure. $\llbracket bp\text{-lazy-ordered-invariant } ?P; bp\text{-bucket-count-ratio-ok } ?P \rrbracket \implies bp\text{-amortized-step-bound } (bp\text{-steps-of } (c\text{-bp-paper-insert } ?x ?b ?P)) ?P (bp\text{-result-of } (c\text{-bp-paper-insert } ?x ?b ?P)) (9 + floor\text{-log } (Suc (length (bp\text{-entries } ?P) \text{ div } bp\text{-block-size } ?P)))$ combines a ratio-log bucket search with the lazy split potential, giving an amortized Insert budget of a constant plus *floor-log* of *Suc (N div M)*. This is the formal $\log(N/M)$ term required by the BMSSP parameter schedule. The batch theorem $\llbracket bp\text{-ordered-invariant } ?P; distinct (map fst ?xs); bp\text{-entry-keys } ?xs \cap keys\text{-of } (bp\text{-view } ?P) = \{\}; batch\text{-prepend-admissible } (bp\text{-view } ?P) ?xs \rrbracket \implies bp\text{-amortized-step-bound } (bp\text{-steps-of } (c\text{-bp-paper-batch-prepend } ?xs ?P)) ?P (bp\text{-result-of } (c\text{-bp-paper-batch-prepend } ?xs ?P)) (length ?xs * (2 + floor\text{-log } (Suc (length ?xs \text{ div } bp\text{-block-size } ?P))))$ charges the same ratio-log scale per incoming item, matching the paper's batched prepend cost. Finally, $\llbracket bp\text{-invariant } ?P; bp\text{-can-first-bucket-pull } ?M ?P \rrbracket \implies bp\text{-pull-amortized-step-bound } (bp\text{-steps-of } (c\text{-bp-paper-pull } ?M ?B ?P)) ?P (bp\text{-result-of } (c\text{-bp-paper-pull } ?M ?B ?P)) (2 * ?M)$ gives the amortized linear Pull budget $2 * M$. These statements are intentionally stronger and more concrete than the abstract cost-interface

corollaries: they expose the primitive-step and potential accounting that justifies plugging the structure into the top-level runtime proof.

```

end
theory BMSSP-Bucketed-Partition
  imports BMSSP-Bucketed-Partition-Internal
begin

```

15 Bucketed Partition

This theory is the public import point for the bucketed partition implementation. The definitions and proof-engineering lemmas live in *BMSSP-Bucketed-Partition-Internal*; importing this wrapper exports the same constants and facts to the rest of the BMSSP development while keeping the documented surface short.

```

end
theory BMSSP-Code-Export
  imports BMSSP-Bucketed-Partition
begin

```

16 Executable Bucketed BMSSP Smoke Test

This theory gives the development an executable surface. All preceding imported theory proves the paper-faithful bucketed operation costs and the concrete partition API. Those facts are essential, but they do not by themselves show a human reader that the formalized program can be run on an ordinary graph. The purpose of this file is to close that gap with a small concrete instantiation and a proof by evaluation.

The executable graph representation is deliberately plain: vertices are natural numbers, edges are triples of natural numbers, and distances are association lists sorted by vertex at the end of the run. The work-list is the paper-style bucketed partition structure imported from the bucketed theory, not the baseline sorted-list partition model. The loop below uses Pull to obtain the next bucket of unsettled vertices, relaxes outgoing edges from that batch, and feeds discovered or improved labels through BatchPrepend for an empty partition and Insert otherwise. This shape is chosen to exercise all three bucketed operations in the worked example.

The fuel argument is an executable guard, not part of the mathematical correctness theorem. For the small smoke graph it is chosen large enough to allow all relaxations needed by the hand calculation. The checked statement at the bottom is the important artifact: *example-bmssp-correct* is proved by Isabelle’s evaluator, so the bundled Poly/ML runtime executes the exported equations during the build and compares the result with the literal expected distance map.

type-synonym *nat-edge* = *nat* × *nat* × *nat*
type-synonym *nat-graph* = *nat-edge* *list*
type-synonym *nat-dist* = (*nat* × *nat*) *list*

definition *bmssp-block-size* :: *nat* **where**
bmssp-block-size = 1

definition *bmssp-infinity* :: *nat* **where**
bmssp-infinity = 1000000

definition *bmssp-bound* :: *real* **where**
bmssp-bound = *real* *bmssp-infinity*

fun *bmssp-edge-vertices* :: *nat-edge* ⇒ *nat* *list* **where**
bmssp-edge-vertices (*u*, *v*, *w*) = [*u*, *v*]

definition *bmssp-vertices* :: *nat-graph* ⇒ *nat* ⇒ *nat* *list* **where**
bmssp-vertices *G* *s* = *sort* (*remdups* (*s* # *concat* (*map* *bmssp-edge-vertices* *G*)))

fun *bmssp-lookup-dist* :: *nat-dist* ⇒ *nat* ⇒ *nat* *option* **where**
bmssp-lookup-dist [] *x* = *None*
| *bmssp-lookup-dist* ((*y*, *d*) # *ds*) *x* =
(*if* *x* = *y* *then* *Some* *d* *else* *bmssp-lookup-dist* *ds* *x*)

fun *bmssp-set-dist* :: *nat* ⇒ *nat* ⇒ *nat-dist* ⇒ *nat-dist* **where**
bmssp-set-dist *x* *d* [] = [(*x*, *d*)]
| *bmssp-set-dist* *x* *d* ((*y*, *e*) # *ds*) =
(*if* *x* = *y* *then* (*x*, *d*) # *ds* *else* (*y*, *e*) # *bmssp-set-dist* *x* *d* *ds*)

definition *bmssp-normalize-dist* :: *nat-dist* ⇒ *nat-dist* **where**
bmssp-normalize-dist *ds* = *sort-key* *fst* *ds*

definition *bmssp-partition-key* :: *nat* ⇒ *nat* ⇒ *real* **where**
bmssp-partition-key *x* *d* = *real* *d* + *real* *x* / *real* (*Suc* *x*)

lemma *bmssp-partition-key-fraction-nonneg*:
0 ≤ *real* *x* / *real* (*Suc* *x*)
by *simp*

lemma *bmssp-partition-key-fraction-lt-one*:
real *x* / *real* (*Suc* *x*) < 1

proof –
have *real* *x* < *real* (*Suc* *x*)
by *simp*
then show ?thesis
by (*simp* *add*: *divide-less-eq*)
qed

lemma *bmssp-partition-key-ge-distance*:


```

    real d ≤ bmssp-partition-key x d
  unfolding bmssp-partition-key-def by simp

lemma bmssp-partition-key-lt-suc-distance:
  bmssp-partition-key x d < real (Suc d)
proof -
  have frac-lt: real x / real (Suc x) < 1
    by (rule bmssp-partition-key-fraction-lt-one)
  have bmssp-partition-key x d < real d + 1
    unfolding bmssp-partition-key-def using frac-lt by linarith
  then show ?thesis
    by simp
qed

lemma bmssp-partition-key-floor:
  nat (floor (bmssp-partition-key x d)) = d
proof (rule floor-eq4)
  show real d ≤ bmssp-partition-key x d
    by (rule bmssp-partition-key-ge-distance)
  show bmssp-partition-key x d < real (Suc d)
    by (rule bmssp-partition-key-lt-suc-distance)
qed

lemma bmssp-partition-key-strict-mono-distance:
  assumes d < e
  shows bmssp-partition-key x d < bmssp-partition-key y e
proof -
  have bmssp-partition-key x d < real (Suc d)
    by (rule bmssp-partition-key-lt-suc-distance)
  also have ... ≤ real e
    using assms by simp
  also have ... ≤ bmssp-partition-key y e
    by (rule bmssp-partition-key-ge-distance)
  finally show ?thesis .
qed

fun bmssp-insert-updates ::
  (nat × real) list ⇒ nat bucketed-partition ⇒
  nat bucketed-partition where
  bmssp-insert-updates [] P = P
| bmssp-insert-updates ((x, b) # xs) P =
  bmssp-insert-updates xs
  (bp-result-of (c-bp-regularized-local-insert x b P))

lemma bmssp-insert-updates-regular-invariant:
  assumes bp-regular-invariant P
  shows bp-regular-invariant (bmssp-insert-updates xs P)
  using assms
proof (induction xs arbitrary: P)

```

```

    case Nil
    then show ?case by simp
next
case (Cons p xs)
then obtain x b where p: p = (x, b)
  by force
have step:
  bp-regular-invariant
  (bp-result-of (c-bp-regularized-local-insert x b P))
  by (rule c-bp-regularized-local-insert-regular-invariant[OF Cons.prem])
show ?case
  unfolding p bmssp-insert-updates.simps
  by (rule Cons.IH[OF step])
qed

lemma bmssp-insert-updates-ordered-invariant:
  assumes bp-regular-invariant P
  shows bp-ordered-invariant (bmssp-insert-updates xs P)
  using bp-regular-invariant-ordered-invariant
    [OF bmssp-insert-updates-regular-invariant[OF assms]]
  .

lemma bmssp-insert-updates-refines-batch-min-update:
  assumes reg: bp-regular-invariant P
  shows bp-view (bmssp-insert-updates xs P) =
    batch-min-update (bp-view P) xs
  using reg
proof (induction xs arbitrary: P)
  case Nil
  then show ?case
    unfolding batch-min-update-def by simp
next
case (Cons p xs)
then obtain x b where p: p = (x, b)
  by force
let ?P1 = bp-result-of (c-bp-regularized-local-insert x b P)
have step-reg: bp-regular-invariant ?P1
  by (rule c-bp-regularized-local-insert-regular-invariant[OF Cons.prem])
have step-view: bp-view ?P1 = min-update (bp-view P) x b
  by (rule c-bp-regularized-local-insert-refines-min-update[OF Cons.prem])
have tail:
  bp-view (bmssp-insert-updates xs ?P1) =
    batch-min-update (bp-view ?P1) xs
  by (rule Cons.IH[OF step-reg])
have head:
  batch-min-update (bp-view P) ((x, b) # xs) =
    batch-min-update (min-update (bp-view P) x b) xs
  unfolding batch-min-update-def by simp
show ?case

```

```

    unfolding p bmssp-insert-updates.simps
    using tail step-view head by simp
qed

lemma bmssp-insert-updates-ordered-invariant-from-ordered:
  assumes ord: bp-ordered-invariant P
  shows bp-ordered-invariant (bmssp-insert-updates xs P)
proof (cases xs)
  case Nil
  then show ?thesis
    using ord by simp
next
  case (Cons p ys)
  then obtain x b where p: p = (x, b)
    by force
  let ?P1 = bp-result-of (c-bp-regularized-local-insert x b P)
  have step-reg: bp-regular-invariant ?P1
    unfolding c-bp-regularized-local-insert-result
    by (rule bp-regularized-local-insert-regular-invariant[OF ord])
  have tail-ord: bp-ordered-invariant (bmssp-insert-updates ys ?P1)
    by (rule bmssp-insert-updates-ordered-invariant[OF step-reg])
  show ?thesis
    unfolding Cons p bmssp-insert-updates.simps
    by (rule tail-ord)
qed

lemma bmssp-insert-updates-refines-batch-min-update-from-ordered:
  assumes ord: bp-ordered-invariant P
  shows bp-view (bmssp-insert-updates xs P) =
    batch-min-update (bp-view P) xs
proof (cases xs)
  case Nil
  then show ?thesis
    unfolding batch-min-update-def by simp
next
  case (Cons p ys)
  then obtain x b where p: p = (x, b)
    by force
  let ?P1 = bp-result-of (c-bp-regularized-local-insert x b P)
  have step-reg: bp-regular-invariant ?P1
    unfolding c-bp-regularized-local-insert-result
    by (rule bp-regularized-local-insert-regular-invariant[OF ord])
  have step-view: bp-view ?P1 = min-update (bp-view P) x b
    unfolding c-bp-regularized-local-insert-result
    by (rule bp-regularized-local-insert-refines-min-update[OF ord])
  have tail:
    bp-view (bmssp-insert-updates ys ?P1) =
      batch-min-update (bp-view ?P1) ys
  by (rule bmssp-insert-updates-refines-batch-min-update[OF step-reg])

```

```

have head:
  batch-min-update (bp-view P) ((x, b) # ys) =
    batch-min-update (min-update (bp-view P) x b) ys
  unfolding batch-min-update-def by simp
show ?thesis
  unfolding Cons p bmssp-insert-updates.simps
  using tail step-view head by simp
qed

definition bmssp-trim-empty-prefix ::
  nat bucketed-partition  $\Rightarrow$  nat bucketed-partition where
  bmssp-trim-empty-prefix P =
    P( $\lfloor$ bp-buckets := bp-drop-empty-prefix (bp-buckets P) $\rfloor$ )

lemma bp-bucket-entries-flat-drop-empty-prefix [simp]:
  bp-bucket-entries-flat (bp-drop-empty-prefix bs) =
    bp-bucket-entries-flat bs
  by (induction bs) (simp-all add: bp-bucket-entries-flat-def)

lemma bp-drop-empty-prefix-set-subset:
  set (bp-drop-empty-prefix bs)  $\subseteq$  set bs
  by (induction bs) auto

lemma bp-drop-empty-prefix-length-le [simp]:
  length (bp-drop-empty-prefix bs)  $\leq$  length bs
proof (induction bs)
  case Nil
  then show ?case by simp
next
  case (Cons b bs)
  show ?case
  proof (cases bp-bucket-entries b = [])
  case True
  have length (bp-drop-empty-prefix bs)  $\leq$  Suc (length bs)
    using Cons.IH by linarith
  then show ?thesis
    using True by simp
  next
  case False
  then show ?thesis by simp
qed
qed

lemma bp-drop-empty-prefix-sorted-wrt:
  assumes sorted-wrt R bs
  shows sorted-wrt R (bp-drop-empty-prefix bs)
  using assms
  by (induction bs) auto

```

```

lemma bp-bucket-boundaries-ok-drop-empty-prefix:
  assumes bp-bucket-boundaries-ok bs
  shows bp-bucket-boundaries-ok (bp-drop-empty-prefix bs)
  using assms
proof (induction bs)
  case Nil
  then show ?case by simp
next
  case (Cons b bs)
  show ?case
  proof (cases bp-bucket-entries b = [])
    case True
    have bp-bucket-boundaries-ok bs
    using Cons.premis by (cases bs) auto
    then show ?thesis
    using True Cons.IH by simp
  next
  case False
  then show ?thesis
  using Cons.premis by simp
qed
qed

lemma bmssp-trim-empty-prefix-entries [simp]:
  bp-entries (bmssp-trim-empty-prefix P) = bp-entries P
  unfolding bmssp-trim-empty-prefix-def bp-entries-def by simp

lemma bmssp-trim-empty-prefix-view [simp]:
  bp-view (bmssp-trim-empty-prefix P) = bp-view P
  unfolding bmssp-trim-empty-prefix-def bp-view-def bp-entries-def by simp

lemma bmssp-trim-empty-prefix-ordered-invariant:
  assumes bp-ordered-invariant P
  shows bp-ordered-invariant (bmssp-trim-empty-prefix P)
proof –
  have inv: bp-invariant P
  using assms unfolding bp-ordered-invariant-def by blast
  have boundaries: bp-bucket-boundaries-state-ok P
  using assms unfolding bp-ordered-invariant-def by blast
  have set-subset:
    set (bp-buckets (bmssp-trim-empty-prefix P)) ⊆ set (bp-buckets P)
  unfolding bmssp-trim-empty-prefix-def
  by (simp add: bp-drop-empty-prefix-set-subset)
  have bp-invariant (bmssp-trim-empty-prefix P)
proof –
  have 0 < bp-block-size (bmssp-trim-empty-prefix P)
  using inv unfolding bmssp-trim-empty-prefix-def bp-invariant-def by simp
  moreover have bp-distinct-keys (bmssp-trim-empty-prefix P)
  using inv unfolding bp-invariant-def bp-distinct-keys-def by simp

```

```

moreover have bp-bucket-sizes-ok (bmssp-trim-empty-prefix P)
proof (unfold bp-bucket-sizes-ok-def, intro ballI)
  fix b
  assume b: b ∈ set (bp-buckets (bmssp-trim-empty-prefix P))
  then have b ∈ set (bp-buckets P)
    using set-subset by blast
  then have length (bp-bucket-entries b) ≤ bp-block-size P
    using inv unfolding bp-invariant-def bp-bucket-sizes-ok-def by blast
  then show length (bp-bucket-entries b) ≤
    bp-block-size (bmssp-trim-empty-prefix P)
    unfolding bmssp-trim-empty-prefix-def by simp
qed
moreover have bp-bucket-markers-sorted (bmssp-trim-empty-prefix P)
  using inv
  unfolding bmssp-trim-empty-prefix-def bp-invariant-def
    bp-bucket-markers-sorted-def
  by (simp add: bp-drop-empty-prefix-sorted-wrt)
moreover have bp-bucket-markers-lower-bound (bmssp-trim-empty-prefix P)
  using inv set-subset
  unfolding bp-invariant-def bp-bucket-markers-lower-bound-def by auto
moreover have bp-values-consistent (bmssp-trim-empty-prefix P)
  using inv
  unfolding bmssp-trim-empty-prefix-def bp-invariant-def
    bp-values-consistent-def bp-entries-def
  by simp
ultimately show ?thesis
  unfolding bp-invariant-def by blast
qed
moreover have bp-bucket-boundaries-state-ok (bmssp-trim-empty-prefix P)
  using boundaries
  unfolding bmssp-trim-empty-prefix-def bp-bucket-boundaries-state-ok-def
  by (simp add: bp-bucket-boundaries-ok-drop-empty-prefix)
ultimately show ?thesis
  unfolding bp-ordered-invariant-def by blast
qed

lemma bmssp-trim-empty-prefix-regular-invariant:
  assumes bp-regular-invariant P
  shows bp-regular-invariant (bmssp-trim-empty-prefix P)
proof –
  have ordered: bp-ordered-invariant P
    by (rule bp-regular-invariant-ordered-invariant[OF assms])
  have ratio: bp-bucket-count-ratio-ok P
    by (rule bp-regular-invariant-ratio-ok[OF assms])
  have bp-bucket-count-ratio-ok (bmssp-trim-empty-prefix P)
proof –
  have len-le:
    length (bp-drop-empty-prefix (bp-buckets P)) ≤ length (bp-buckets P)
    by (rule bp-drop-empty-prefix-length-le)

```

```

have block-pos: 0 < bp-block-size P
  using ratio unfolding bp-bucket-count-ratio-ok-def by blast
have orig-len:
  length (bp-buckets P) ≤
    Suc (length (bp-entries P) div bp-block-size P)
  using ratio unfolding bp-bucket-count-ratio-ok-def by blast
have drop-len:
  length (bp-drop-empty-prefix (bp-buckets P)) ≤
    Suc (length (bp-entries P) div bp-block-size P)
  using len-le orig-len by (rule order-trans)
show ?thesis
  using block-pos drop-len
  unfolding bmssp-trim-empty-prefix-def bp-bucket-count-ratio-ok-def
    bp-entries-def
  by simp
qed
then show ?thesis
  using bmssp-trim-empty-prefix-ordered-invariant[OF ordered]
  unfolding bp-regular-invariant-def by blast
qed

```

definition *bmssp-apply-updates* ::
 $(\text{nat} \times \text{real}) \text{ list} \Rightarrow \text{nat bucketed-partition} \Rightarrow$
 $\text{nat bucketed-partition}$ **where**
bmssp-apply-updates *xs* *P* =
 (let *P0* = *bmssp-trim-empty-prefix* *P* in
 if *bp-entries* *P0* = []
 then *bp-result-of* (*c-bp-paper-batch-prepend* *xs* *P0*)
 else *bmssp-insert-updates* *xs* *P0*)

lemma *bmssp-apply-updates-refines-batch-min-update*:

assumes *reg*: *bp-regular-invariant* *P*
and *distinct*: *distinct* (*map fst xs*)
shows *bp-view* (*bmssp-apply-updates* *xs* *P*) =
batch-min-update (*bp-view* *P*) *xs*

proof –

let *?P0* = *bmssp-trim-empty-prefix* *P*
have *reg0*: *bp-regular-invariant* *?P0*
by (rule *bmssp-trim-empty-prefix-regular-invariant*[*OF reg*])
have *ord0*: *bp-ordered-invariant* *?P0*
by (rule *bp-regular-invariant-ordered-invariant*[*OF reg0*])
show ?thesis
proof (cases *bp-entries* *?P0* = [])
case *True*
have *disjoint*: *bp-entry-keys* *xs* $\cap$ *keys-of* (*bp-view* *?P0*) = {}
using *True* **unfolding** *bp-view-def* *bp-entry-keys-def* **by** *simp*
have *bp-view* (*bp-result-of* (*c-bp-paper-batch-prepend* *xs* *?P0*)) =
batch-min-update (*bp-view* *?P0*) *xs*
by (rule *c-bp-paper-batch-prepend-refines-batch-min-update*)

```

      [OF ord0 distinct disjoint])
    then show ?thesis
      using True unfolding bmssp-apply-updates-def by simp
  next
    case False
    have bp-view (bmssp-insert-updates xs ?P0) =
      batch-min-update (bp-view ?P0) xs
      by (rule bmssp-insert-updates-refines-batch-min-update[OF reg0])
    then show ?thesis
      using False unfolding bmssp-apply-updates-def by simp
  qed
qed

lemma bmssp-apply-updates-ordered-invariant:
  assumes reg: bp-regular-invariant P
  and distinct: distinct (map fst xs)
  and admissible:
    batch-prepend-admissible (bp-view (bmssp-trim-empty-prefix P)) xs
  shows bp-ordered-invariant (bmssp-apply-updates xs P)
proof -
  let ?P0 = bmssp-trim-empty-prefix P
  have reg0: bp-regular-invariant ?P0
    by (rule bmssp-trim-empty-prefix-regular-invariant[OF reg])
  have ord0: bp-ordered-invariant ?P0
    by (rule bp-regular-invariant-ordered-invariant[OF reg0])
  show ?thesis
  proof (cases bp-entries ?P0 = [])
    case True
    have disjoint: bp-entry-keys xs  $\cap$  keys-of (bp-view ?P0) = {}
      using True unfolding bp-view-def bp-entry-keys-def by simp
    have bp-ordered-invariant
      (bp-result-of (c-bp-paper-batch-prepend xs ?P0))
      by (rule c-bp-paper-batch-prepend-ordered-invariant
        [OF ord0 distinct disjoint admissible])
    then show ?thesis
      using True unfolding bmssp-apply-updates-def by simp
  next
    case False
    have bp-ordered-invariant (bmssp-insert-updates xs ?P0)
      by (rule bmssp-insert-updates-ordered-invariant[OF reg0])
    then show ?thesis
      using False unfolding bmssp-apply-updates-def by simp
  qed
qed

lemma bmssp-apply-updates-refines-batch-min-update-from-ordered:
  assumes ord: bp-ordered-invariant P
  and distinct: distinct (map fst xs)
  shows bp-view (bmssp-apply-updates xs P) =

```



```

    batch-min-update (bp-view P) xs
proof –
  let ?P0 = bmssp-trim-empty-prefix P
  have ord0: bp-ordered-invariant ?P0
    by (rule bmssp-trim-empty-prefix-ordered-invariant[OF ord])
  show ?thesis
proof (cases bp-entries ?P0 = [])
  case True
    have disjoint: bp-entry-keys xs  $\cap$  keys-of (bp-view ?P0) = {}
      using True unfolding bp-view-def bp-entry-keys-def by simp
    have bp-view (bp-result-of (c-bp-paper-batch-prepend xs ?P0)) =
      batch-min-update (bp-view ?P0) xs
      by (rule c-bp-paper-batch-prepend-refines-batch-min-update
        [OF ord0 distinct disjoint])
    then show ?thesis
      using True unfolding bmssp-apply-updates-def by simp
  next
    case False
    have bp-view (bmssp-insert-updates xs ?P0) =
      batch-min-update (bp-view ?P0) xs
      by (rule bmssp-insert-updates-refines-batch-min-update-from-ordered
        [OF ord0])
    then show ?thesis
      using False unfolding bmssp-apply-updates-def by simp
qed
qed

lemma bmssp-apply-updates-ordered-invariant-from-ordered:
  assumes ord: bp-ordered-invariant P
    and distinct: distinct (map fst xs)
  shows bp-ordered-invariant (bmssp-apply-updates xs P)
proof –
  let ?P0 = bmssp-trim-empty-prefix P
  have ord0: bp-ordered-invariant ?P0
    by (rule bmssp-trim-empty-prefix-ordered-invariant[OF ord])
  show ?thesis
proof (cases bp-entries ?P0 = [])
  case True
    have disjoint: bp-entry-keys xs  $\cap$  keys-of (bp-view ?P0) = {}
      using True unfolding bp-view-def bp-entry-keys-def by simp
    have admissible: batch-prepend-admissible (bp-view ?P0) xs
      using True
      unfolding batch-prepend-admissible-def bp-view-def bp-entry-keys-def
      by simp
    have bp-ordered-invariant
      (bp-result-of (c-bp-paper-batch-prepend xs ?P0))
      by (rule c-bp-paper-batch-prepend-ordered-invariant
        [OF ord0 distinct disjoint admissible])
    then show ?thesis

```

```

    using True unfolding bmssp-apply-updates-def by simp
  next
    case False
    have bp-ordered-invariant (bmssp-insert-updates xs ?P0)
      by (rule bmssp-insert-updates-ordered-invariant-from-ordered[OF ord0])
    then show ?thesis
      using False unfolding bmssp-apply-updates-def by simp
  qed
qed

```

```

lemma bmssp-apply-updates-preserves-upper-bound:
  assumes reg: bp-regular-invariant P
    and upper: partition-upper-bound (bp-view P) B
    and distinct: distinct (map fst xs)
    and values-lt:  $\bigwedge x b. (x, b) \in \text{set } xs \implies b < B$ 
  shows partition-upper-bound (bp-view (bmssp-apply-updates xs P)) B
  unfolding bmssp-apply-updates-refines-batch-min-update[OF reg distinct]
  by (rule batch-min-update-preserves-upper-bound[OF upper values-lt])

```

```

lemma bmssp-apply-updates-preserves-upper-bound-from-ordered:
  assumes ord: bp-ordered-invariant P
    and upper: partition-upper-bound (bp-view P) B
    and distinct: distinct (map fst xs)
    and values-lt:  $\bigwedge x b. (x, b) \in \text{set } xs \implies b < B$ 
  shows partition-upper-bound (bp-view (bmssp-apply-updates xs P)) B
  unfolding bmssp-apply-updates-refines-batch-min-update-from-ordered
    [OF ord distinct]
  by (rule batch-min-update-preserves-upper-bound[OF upper values-lt])

```

```

lemma bmssp-pull-refines-pull-separates:
  assumes ord: bp-ordered-invariant P
    and upper: partition-upper-bound (bp-view P) B
    and pull: bp-pull M B P = (S, beta, P')
  shows pull-separates (bp-view P) M B S beta (bp-view P')
proof (rule bp-pull-refines-pull-separates[OF - - pull])
  show bp-invariant P
    by (rule bp-ordered-invariant-invariant[OF ord])
next
  fix u
  assume u  $\in$  keys-of (bp-view P)
  then show value-of (bp-view P) u < B
    using upper unfolding partition-upper-bound-def by blast
qed

```

```

lemma bmssp-pull-ordered-invariant:
  assumes ord: bp-ordered-invariant P
    and pull: bp-pull M B P = (S, beta, P')
  shows bp-ordered-invariant P'
  by (rule bp-pull-ordered-invariant[OF ord pull])

```

lemma *bmssp-pull-preserves-upper-bound*:
assumes *ord*: *bp-ordered-invariant P*
and *upper*: *partition-upper-bound (bp-view P) B*
and *pull*: *bp-pull M B P = (S, beta, P')*
shows *partition-upper-bound (bp-view P') B*
proof –
have *inv*: *bp-invariant P*
by (*rule bp-ordered-invariant-invariant[OF ord]*)
show *?thesis*
by (*rule bp-pull-preserves-upper-bound[OF inv upper upper pull]*)
qed

lemma *bmssp-pull-step-bridge*:
assumes *ord*: *bp-ordered-invariant P*
and *upper*: *partition-upper-bound (bp-view P) B*
and *pull*: *bp-pull M B P = (S, beta, P')*
shows *pull-separates (bp-view P) M B S beta (bp-view P')*
and *bp-ordered-invariant P'*
and *partition-upper-bound (bp-view P') B*
using *bmssp-pull-refines-pull-separates[OF assms]*
bmssp-pull-ordered-invariant[OF ord pull]
bmssp-pull-preserves-upper-bound[OF assms]
by *blast+*

lemma *bmssp-vertices-distinct [simp]*:
distinct (bmssp-vertices G s)
unfolding *bmssp-vertices-def* **by** *simp*

lemma *finite-keys-of-bp-view [simp]*:
finite (keys-of (bp-view P))
unfolding *bp-view-def bp-entry-keys-def* **by** *simp*

lemma *bmssp-pulled-length-le-block-size*:
assumes *distinct-vertices*: *distinct vertices*
and *pull*: *pull-separates (bp-view P) bmssp-block-size B S beta D'*
shows *length (filter (λx. x ∈ S ∧ x ∉ set settled) vertices)*
 $\leq$ *bmssp-block-size*
proof –
let *?pulled* = *filter (λx. x ∈ S ∧ x ∉ set settled) vertices*
have *S-subset*: *S ⊆ keys-of (bp-view P)*
by (*rule pull-separates-subset[OF pull]*)
then have *finite-S*: *finite S*
by (*rule finite-subset*) *simp*
have *set-pulled*: *set ?pulled ⊆ S*
by *auto*
have *distinct-pulled*: *distinct ?pulled*
using *distinct-vertices* **by** *simp*
have *length ?pulled = card (set ?pulled)*

```

    using distinct-card[OF distinct-pulled] by simp
  also have ... ≤ card S
    by (rule card-mono[OF finite-S set-pulled])
  also have ... ≤ bmssp-block-size
    by (rule pull-separates-card[OF pull])
  finally show ?thesis .
qed

```

```

lemma bmssp-pulled-length-le-one:
  assumes distinct-vertices: distinct vertices
    and pull: pull-separates (bp-view P) bmssp-block-size B S beta D'
  shows length (filter (λx. x ∈ S ∧ x ∉ set settled) vertices)
    ≤ 1
  using bmssp-pulled-length-le-block-size[OF distinct-vertices pull]
  unfolding bmssp-block-size-def .

```

```

fun bmssp-relax-edges ::
  nat-graph ⇒ nat list ⇒ nat ⇒ nat ⇒ nat-dist ⇒
  (nat × real) list × nat-dist where
  bmssp-relax-edges [] settled u du ds = ([], ds)
| bmssp-relax-edges ((a, b, w) # es) settled u du ds =
  (case bmssp-relax-edges es settled u du ds of
    (updates, ds1) ⇒
    (if a = u ∧ b ∉ set settled
    then
      (let nd = du + w in
       case bmssp-lookup-dist ds1 b of
         None ⇒
           ((b, bmssp-partition-key b nd) # updates,
            bmssp-set-dist b nd ds1)
         | Some old ⇒
           (if nd < old
            then ((b, bmssp-partition-key b nd) # updates,
                  bmssp-set-dist b nd ds1)
            else (updates, ds1))))
    else (updates, ds1)))

```

```

fun bmssp-relax-vertices ::
  nat-graph ⇒ nat list ⇒ nat list ⇒ nat-dist ⇒
  (nat × real) list × nat-dist where
  bmssp-relax-vertices G settled [] ds = ([], ds)
| bmssp-relax-vertices G settled (u # us) ds =
  (case bmssp-lookup-dist ds u of
    None ⇒ bmssp-relax-vertices G settled us ds
  | Some du ⇒
    (case bmssp-relax-edges G settled u du ds of
      (updates-u, ds1) ⇒
      (case bmssp-relax-vertices G settled us ds1 of
        (updates-us, ds2) ⇒ (updates-u @ updates-us, ds2))))

```

```

fun bmssp-loop ::
  nat  $\Rightarrow$  nat-graph  $\Rightarrow$  nat list  $\Rightarrow$  nat list  $\Rightarrow$ 
    nat-dist  $\Rightarrow$  nat bucketed-partition  $\Rightarrow$  nat-dist where
    bmssp-loop 0 G vertices settled ds P = bmssp-normalize-dist ds
  | bmssp-loop (Suc fuel) G vertices settled ds P =
    (case bp-pull bmssp-block-size bmssp-bound P of
      (S, beta, P1)  $\Rightarrow$ 
        (let pulled = filter ( $\lambda x. x \in S \wedge x \notin \text{set settled}$ ) vertices in
          if pulled = []
            then bmssp-normalize-dist ds
          else
            (let settled' = pulled @ settled;
              relaxed = bmssp-relax-vertices G settled' pulled ds;
              updates = fst relaxed;
              ds' = snd relaxed;
              P2 = bmssp-apply-updates updates P1
            in bmssp-loop fuel G vertices settled' ds' P2)))

```

definition *bmssp-distances* :: nat-graph $\Rightarrow$ nat $\Rightarrow$ nat-dist **where**
bmssp-distances *G* *s* =
 (let vertices = *bmssp-vertices* *G* *s*;
P0 = *bp-empty* *bmssp-block-size* *bmssp-bound*;
P1 = *bp-result-of*
 (c-bp-regularized-local-insert *s* (*bmssp-partition-key* *s* 0) *P0*);
 fuel = Suc (length vertices * Suc (length *G*))
 in *bmssp-loop* fuel *G* vertices [] [(*s*, 0)] *P1*)

The entry point *bmssp-distances* initializes the bucketed partition with the source at distance zero and then iterates *bmssp-loop*. The helper *bmssp-apply-updates* makes the example exercise both bulk and singleton update paths: an empty work-list receives a batch prepend, while a non-empty work-list receives local inserts. The bucketed Pull operation is invoked in every loop iteration through *bp-pull*.

definition *example-graph* :: nat-graph **where**

```

example-graph =
  [(0, 1, 3), (0, 4, 6), (0, 2, 10),
   (1, 2, 2), (1, 4, 4), (1, 3, 9),
   (2, 3, 3), (2, 4, 1), (4, 3, 5)]

```

From source 0: d(0) = 0 d(1) = 3 via 0 -> 1 d(2) = 5 via 0 -> 1 -> 2
 d(3) = 8 via 0 -> 1 -> 2 -> 3 d(4) = 6 via 0 -> 4, tied by 0 -> 1 -> 2 -> 4

definition *example-expected-dist* :: nat-dist **where**

```

example-expected-dist = [(0, 0), (1, 3), (2, 5), (3, 8), (4, 6)]

```

The graph is small enough to audit by hand but nontrivial enough to check the executable plumbing. Vertex 0 starts the run. Vertex 1 is reached directly with cost 3; vertex 2 improves from the direct edge of cost 10 to cost 5 via 1; vertex 3 is then reached at cost 8 via 2; and vertex 4 has final

cost 6, tied between the direct edge and the path through 1 and 2. The constant *example-expected-dist* records that hand calculation as a literal sorted association list.

lemma *example-bmssp-correct*:
bmssp-distances example-graph 0 = example-expected-dist
by *eval*

lemma *bmssp-counterexample-fixed*:
bmssp-distances [(0::nat, 1::nat, 1::nat), (0, 2, 1), (1, 3, 2)] 0
= [(0, 0), (1, 1), (2, 1), (3, 3)]
by *eval*

lemma *bmssp-line-graph-fixed*:
bmssp-distances [(0::nat, 1::nat, 1::nat), (1, 2, 1), (2, 3, 1)] 0
= [(0, 0), (1, 1), (2, 2), (3, 3)]
by *eval*

lemma *bmssp-empty-prefix-progress-fixed*:
bmssp-distances
[(0::nat, 1::nat, 1::nat), (0, 2, 2), (0, 3, 3), (2, 4, 1)] 0
= [(0, 0), (1, 1), (2, 2), (3, 3), (4, 3)]
by *eval*

The theorem *bmssp-distances example-graph 0 = example-expected-dist* is the end-to-end smoke test. The regression lemmas above cover equal-distance frontier updates, a multi-hop line graph, and progress after a pull leaves an empty bucket in front of pending vertices. The proof method *eval* runs the generated equations inside Isabelle and proves that the result equals the expected literal distance map. The following **value** command is intentionally kept in the theory so the build log prints the computed list for human inspection, and the final **export-code** declaration emits the same executable entry point as SML in the generated directory.

value *bmssp-distances example-graph 0*

export-code *bmssp-distances example-graph example-expected-dist*
in *SML module-name BMSSP file-prefix generated/BMSSP*

end

theory *BMSSP-Executable-Base-Case*

imports *BMSSP-Base-Case BMSSP-Code-Export HOL-Library.Multiset*
begin

17 Executable Base-Case Ordering

The semantic base-case theory orders a finite set by *finite-weighted-digraph.dist*. The functions below provide an executable finite counterpart: enumerate simple walks over explicit vertex and edge lists, compute their

weights by a list minimum, and sort the target vertices by the resulting finite distance scan.

```
fun exec-walk :: 'v list  $\Rightarrow$  ('v  $\times$  'v) list  $\Rightarrow$  'v list  $\Rightarrow$  bool where
  exec-walk vs es []  $\longleftrightarrow$  False
| exec-walk vs es [x]  $\longleftrightarrow$  x  $\in$  set vs
| exec-walk vs es (x # y # xs)  $\longleftrightarrow$ 
  x  $\in$  set vs  $\wedge$  (x, y)  $\in$  set es  $\wedge$  exec-walk vs es (y # xs)
```

```
fun exec-walk-weight :: ('v  $\Rightarrow$  'v  $\Rightarrow$  real)  $\Rightarrow$  'v list  $\Rightarrow$  real where
  exec-walk-weight W [] = 0
| exec-walk-weight W [x] = 0
| exec-walk-weight W (x # y # xs) = W x y + exec-walk-weight W (y # xs)
```

```
definition exec-simple-walks-betw ::
  'v list  $\Rightarrow$  ('v  $\times$  'v) list  $\Rightarrow$  'v  $\Rightarrow$  'v  $\Rightarrow$  'v list list where
  exec-simple-walks-betw vs es a b =
    filter ( $\lambda p$ . p  $\neq$  []  $\wedge$  hd p = a  $\wedge$  last p = b  $\wedge$ 
      exec-walk vs es p  $\wedge$  distinct p)
      (concat (map ( $\lambda n$ . List.n-lists n vs) [1.. $\text{Suc}$  (length vs)]))
```

```
definition exec-walk-weights ::
  'v list  $\Rightarrow$  ('v  $\times$  'v) list  $\Rightarrow$  ('v  $\Rightarrow$  'v  $\Rightarrow$  real)  $\Rightarrow$  'v  $\Rightarrow$  'v  $\Rightarrow$  real list where
  exec-walk-weights vs es W a b =
    map (exec-walk-weight W) (exec-simple-walks-betw vs es a b)
```

```
definition exec-dist ::
  'v list  $\Rightarrow$  ('v  $\times$  'v) list  $\Rightarrow$  ('v  $\Rightarrow$  'v  $\Rightarrow$  real)  $\Rightarrow$  'v  $\Rightarrow$  'v  $\Rightarrow$  real where
  exec-dist vs es W a b =
    (case exec-walk-weights vs es W a b of []  $\Rightarrow$  0 | ws  $\Rightarrow$  min-list ws)
```

```
definition exec-closest-vertices ::
  'v list  $\Rightarrow$  ('v  $\times$  'v) list  $\Rightarrow$  ('v  $\Rightarrow$  'v  $\Rightarrow$  real)  $\Rightarrow$  'v  $\Rightarrow$  'v list  $\Rightarrow$  'v list where
  exec-closest-vertices vs es W src xs =
    sort-key (exec-dist vs es W src) (remdups xs)
```

```
definition exec-reachable ::
  'v list  $\Rightarrow$  ('v  $\times$  'v) list  $\Rightarrow$  'v  $\Rightarrow$  'v  $\Rightarrow$  bool where
  exec-reachable vs es a b  $\longleftrightarrow$  exec-simple-walks-betw vs es a b  $\neq$  []
```

```
definition exec-shortest-through ::
  'v list  $\Rightarrow$  ('v  $\times$  'v) list  $\Rightarrow$  ('v  $\Rightarrow$  'v  $\Rightarrow$  real)  $\Rightarrow$  'v  $\Rightarrow$  'v  $\Rightarrow$  'v  $\Rightarrow$  bool where
  exec-shortest-through vs es W src x v  $\longleftrightarrow$ 
    ( $\exists p \in$  set (exec-simple-walks-betw vs es src v).
      exec-walk-weight W p = exec-dist vs es W src v  $\wedge$  x  $\in$  set p)
```

```
definition exec-bound-tree-vertices ::
  'v list  $\Rightarrow$  ('v  $\times$  'v) list  $\Rightarrow$  ('v  $\Rightarrow$  'v  $\Rightarrow$  real)  $\Rightarrow$  'v  $\Rightarrow$  'v  $\Rightarrow$  bound  $\Rightarrow$  'v list where
  exec-bound-tree-vertices vs es W src x B =
    filter ( $\lambda v$ . exec-reachable vs es src v  $\wedge$ 
```

exec-shortest-through *vs es W src x v* $\wedge$
below-bound (*exec-dist vs es W src v*) *B*)
(*remdups vs*)

definition *exec-base-case-order* ::

(*'v* $\Rightarrow$ *'v* $\Rightarrow$ *real*) $\Rightarrow$ *'v* $\Rightarrow$ *'v list* $\Rightarrow$ (*'v* $\times$ *'v*) *list* $\Rightarrow$
'v $\Rightarrow$ *bound* $\Rightarrow$ *'v list* **where**
exec-base-case-order W src vs es x B =
exec-closest-vertices vs es W src (exec-bound-tree-vertices vs es W src x B)

definition *exec-base-case-vertices* ::

(*'v* $\Rightarrow$ *'v* $\Rightarrow$ *real*) $\Rightarrow$ *'v* $\Rightarrow$ *'v list* $\Rightarrow$ (*'v* $\times$ *'v*) *list* $\Rightarrow$
nat $\Rightarrow$ *'v* $\Rightarrow$ *bound* $\Rightarrow$ *'v set* **where**
exec-base-case-vertices W src vs es k x B =
(let *xs* = *exec-base-case-order W src vs es x B* in
if *length xs* $\leq$ *k* then *set xs*
else *set (filter (λv . *exec-dist vs es W src v* < *exec-dist vs es W src (xs ! k)*)*
(*take (Suc k) xs*)))

definition *exec-base-case-bound* ::

(*'v* $\Rightarrow$ *'v* $\Rightarrow$ *real*) $\Rightarrow$ *'v* $\Rightarrow$ *'v list* $\Rightarrow$ (*'v* $\times$ *'v*) *list* $\Rightarrow$
nat $\Rightarrow$ *'v* $\Rightarrow$ *bound* $\Rightarrow$ *bound* **where**
exec-base-case-bound W src vs es k x B =
(let *xs* = *exec-base-case-order W src vs es x B* in
if *length xs* $\leq$ *k* then *B* else *Fin (exec-dist vs es W src (xs ! k))*)

definition *exec-base-case-result* ::

(*'v* $\Rightarrow$ *'v* $\Rightarrow$ *real*) $\Rightarrow$ *'v* $\Rightarrow$ *'v list* $\Rightarrow$ (*'v* $\times$ *'v*) *list* $\Rightarrow$
nat $\Rightarrow$ *'v* $\Rightarrow$ *bound* $\Rightarrow$ *bound* $\times$ *'v set* **where**
exec-base-case-result W src vs es k x B =
(*exec-base-case-bound W src vs es k x B*,
exec-base-case-vertices W src vs es k x B)

context *finite-weighted-digraph*

begin

lemma *exec-walk-iff-walk*:

assumes *set vs* = *V* *set es* = *E*
shows *exec-walk vs es p* $\longleftrightarrow$ *walk p*
using *assms* **by** (*induction p* rule: *exec-walk.induct*) *simp-all*

lemma *exec-walk-weight-eq-walk-weight* [*simp*]:

exec-walk-weight w p = *walk-weight p*
by (*induction p* rule: *walk-weight.induct*) *simp-all*

lemma *set-exec-simple-walks-betw*:

assumes *vs*: *set vs* = *V*
and *es*: *set es* = *E*
shows *set (exec-simple-walks-betw vs es a b)* = {*p*. *simple-walk-betw a p b*}


```

proof
  show  $\text{set } (\text{exec-simple-walks-betw } vs \text{ es } a \ b) \subseteq \{p. \text{simple-walk-betw } a \ p \ b\}$ 
    unfolding exec-simple-walks-betw-def simple-walk-betw-def walk-betw-def
    using exec-walk-iff-walk[OF vs es] by auto
next
  show  $\{p. \text{simple-walk-betw } a \ p \ b\} \subseteq \text{set } (\text{exec-simple-walks-betw } vs \text{ es } a \ b)$ 
  proof
    fix p
    assume p:  $p \in \{p. \text{simple-walk-betw } a \ p \ b\}$ 
    then have simple: simple-walk-betw a p b
      by simp
    then have walk-p: walk p and distinct-p: distinct p
      and p-ne:  $p \neq []$  and hd-p: hd p = a and last-p: last p = b
      unfolding simple-walk-betw-def walk-betw-def by auto
    have len-le-card: length p  $\leq \text{card } V$ 
      by (rule simple-walk-length-le-card[OF simple])
    have card-le-len:  $\text{card } V \leq \text{length } vs$ 
      using card-length[of vs] vs by simp
    have len-le: length p  $\leq \text{length } vs$ 
      using len-le-card card-le-len by linarith
    have len-pos:  $0 < \text{length } p$ 
      using p-ne by (cases p) auto
    have p-in-lists:  $p \in \text{set } (\text{List.n-lists } (\text{length } p) \text{ vs})$ 
      using walk-set-subset[OF walk-p] vs
      by (auto simp: set-n-lists)
    have len-mem:  $\text{length } p \in \text{set } [1..<\text{Suc } (\text{length } vs)]$ 
  proof –
    have  $1 \leq \text{length } p$ 
      using len-pos by linarith
    moreover have  $\text{length } p < \text{Suc } (\text{length } vs)$ 
      using len-le by simp
    ultimately show ?thesis
      by auto
  qed
  have enum:  $p \in \text{set } (\text{concat } (\text{map } (\lambda n. \text{List.n-lists } n \text{ vs}) [1..<\text{Suc } (\text{length } vs)]))$ 
    using p-in-lists len-mem by auto
  have exec: exec-walk vs es p
    using exec-walk-iff-walk[OF vs es] walk-p by simp
  show  $p \in \text{set } (\text{exec-simple-walks-betw } vs \text{ es } a \ b)$ 
    unfolding exec-simple-walks-betw-def
    using enum exec distinct-p hd-p last-p p-ne by auto
qed
qed

lemma set-exec-walk-weights:
  assumes vs:  $\text{set } vs = V$ 
    and es:  $\text{set } es = E$ 
  shows  $\text{set } (\text{exec-walk-weights } vs \text{ es } w \ a \ b) = \text{simple-walk-weights } a \ b$ 
  unfolding exec-walk-weights-def simple-walk-weights-def

```

```

using set-exec-simple-walks-betw[OF vs es, of a b] by auto

lemma exec-walk-weights-nonempty:
  assumes vs: set vs = V
    and es: set es = E
    and reach: reachable a b
  shows exec-walk-weights vs es w a b  $\neq$  []
proof
  assume empty: exec-walk-weights vs es w a b = []
  then have simple-walk-weights a b = {}
    using set-exec-walk-weights[OF vs es, of a b] by simp
  then show False
    using simple-walk-weights-nonempty[OF reach] by simp
qed

lemma exec-dist-eq-dist:
  assumes vs: set vs = V
    and es: set es = E
    and reach: reachable a b
  shows exec-dist vs es w a b = dist a b
proof -
  have nonempty: exec-walk-weights vs es w a b  $\neq$  []
    using exec-walk-weights-nonempty[OF vs es reach] .
  have exec-dist vs es w a b = min-list (exec-walk-weights vs es w a b)
    using nonempty unfolding exec-dist-def by (cases exec-walk-weights vs es w a
b) simp-all
  also have ... = Min (set (exec-walk-weights vs es w a b))
    using min-list-Min[OF nonempty] .
  also have ... = Min (simple-walk-weights a b)
    using set-exec-walk-weights[OF vs es, of a b] by simp
  also have ... = dist a b
    unfolding dist-def by simp
  finally show ?thesis .
qed

lemma exec-closest-vertices-properties:
  assumes vs: set vs = V
    and es: set es = E
    and xs: set xs = A
    and reach-A: A  $\subseteq$  {v. reachable s v}
  shows set (exec-closest-vertices vs es w s xs) = A
    and distinct (exec-closest-vertices vs es w s xs)
    and sorted-wrt ( $\lambda u v. \text{dist } s u \leq \text{dist } s v$ )
      (exec-closest-vertices vs es w s xs)
proof -
  let ?ys = exec-closest-vertices vs es w s xs
  have set-ys: set ?ys = A
    unfolding exec-closest-vertices-def using xs by simp
  then show set ?ys = A .

```

```

show distinct ?ys
  unfolding exec-closest-vertices-def by simp
have eq-on:  $\bigwedge v. v \in \text{set } ?ys \implies \text{exec-dist } vs \text{ es } w \text{ s } v = \text{dist } s \text{ v}$ 
  using exec-dist-eq-dist[OF vs es] reach-A set-ys by blast
have sorted (map (exec-dist vs es w s) ?ys)
  unfolding exec-closest-vertices-def by (rule sorted-sort-key)
then have sorted (map (dist s) ?ys)
  by (simp add: map-cong[OF refl eq-on])
then show sorted-wrt ( $\lambda u v. \text{dist } s \text{ u} \leq \text{dist } s \text{ v}$ ) ?ys
  by (simp add: sorted-map)
qed

```

```

lemma closest-vertices-executable:
  assumes vs: set vs = V
    and es: set es = E
    and xs: set xs = A
    and reach-A:  $A \subseteq \{v. \text{reachable } s \text{ v}\}$ 
  shows set (exec-closest-vertices vs es w s xs) = A
    and distinct (exec-closest-vertices vs es w s xs)
    and sorted-wrt ( $\lambda u v. \text{dist } s \text{ u} \leq \text{dist } s \text{ v}$ )
      (exec-closest-vertices vs es w s xs)
  using exec-closest-vertices-properties[OF assms] by blast+

```

```

lemma closest-vertices-eq-exec-if-inj:
  assumes vs: set vs = V
    and es: set es = E
    and xs: set xs = A
    and reach-A:  $A \subseteq \{v. \text{reachable } s \text{ v}\}$ 
    and inj: inj-on (dist s) A
  shows closest-vertices A = exec-closest-vertices vs es w s xs
proof -
  have finite-A: finite A
    using xs by auto
  have cv-set: set (closest-vertices A) = A
    by (rule closest-vertices-properties(1)[OF finite-A])
  have cv-distinct: distinct (closest-vertices A)
    by (rule closest-vertices-properties(2)[OF finite-A])
  have cv-sorted: sorted-wrt ( $\lambda u v. \text{dist } s \text{ u} \leq \text{dist } s \text{ v}$ ) (closest-vertices A)
    by (rule closest-vertices-properties(3)[OF finite-A])
  have ex-set: set (exec-closest-vertices vs es w s xs) = A
    by (rule exec-closest-vertices-properties(1)[OF vs es xs reach-A])
  have ex-distinct: distinct (exec-closest-vertices vs es w s xs)
    by (rule exec-closest-vertices-properties(2)[OF vs es xs reach-A])
  have ex-sorted:
    sorted-wrt ( $\lambda u v. \text{dist } s \text{ u} \leq \text{dist } s \text{ v}$ )
      (exec-closest-vertices vs es w s xs)
    by (rule exec-closest-vertices-properties(3)[OF vs es xs reach-A])
  have inj-union:
    inj-on (dist s)

```

```

    (set (closest-vertices A)  $\cup$  set (exec-closest-vertices vs es w s xs))
  using inj cv-set ex-set by simp
  have cv-sorted-map: sorted (map (dist s) (closest-vertices A))
    using cv-sorted by (simp add: sorted-map)
  have ex-sorted-map: sorted (map (dist s) (exec-closest-vertices vs es w s xs))
    using ex-sorted by (simp add: sorted-map)
  have cv-distinct-map: distinct (map (dist s) (closest-vertices A))
    using cv-distinct inj cv-set by (simp add: distinct-map)
  have ex-distinct-map: distinct (map (dist s) (exec-closest-vertices vs es w s xs))
    using ex-distinct inj ex-set by (simp add: distinct-map)
  show ?thesis
    by (rule map-sorted-distinct-set-unique
      [OF inj-union cv-sorted-map cv-distinct-map ex-sorted-map ex-distinct-map])
      (simp add: cv-set ex-set)
qed

```

```

lemma exec-reachable-iff-reachable:
  assumes vs: set vs = V
    and es: set es = E
  shows exec-reachable vs es a b  $\longleftrightarrow$  reachable a b
proof -
  have exec-reachable vs es a b  $\longleftrightarrow$ 
    set (exec-simple-walks-betw vs es a b)  $\neq$  {}
  unfolding exec-reachable-def by auto
  also have ...  $\longleftrightarrow$  {p. simple-walk-betw a p b}  $\neq$  {}
    using set-exec-simple-walks-betw[OF vs es, of a b] by simp
  also have ...  $\longleftrightarrow$  reachable a b
    unfolding reachable-def by auto
  finally show ?thesis .
qed

```

```

lemma exec-shortest-through-iff-through-single:
  assumes vs: set vs = V
    and es: set es = E
  shows exec-shortest-through vs es w s x v  $\longleftrightarrow$  through {x} v
proof
  assume exec: exec-shortest-through vs es w s x v
  then obtain p where p-exec:
    p  $\in$  set (exec-simple-walks-betw vs es s v)
    exec-walk-weight w p = exec-dist vs es w s v
    x  $\in$  set p
  unfolding exec-shortest-through-def by blast
  have p-simple: simple-walk-betw s p v
    using p-exec(1) set-exec-simple-walks-betw[OF vs es, of s v] by blast
  then have reach-v: reachable s v
    unfolding reachable-def by blast
  have walk-weight p = dist s v
    using p-exec(2) exec-dist-eq-dist[OF vs es reach-v] by simp
  then have shortest-walk s p v

```

using *p-simple* **unfolding** *shortest-walk-def* **by** *blast*
 then **show** *through {x} v*
 using *p-exec(3)* **unfolding** *through-def* **by** *blast*
next
 assume *through: through {x} v*
 then **obtain** *p* **where** *p: shortest-walk s p v x ∈ set p*
unfolding *through-def* **by** *blast*
 have *p-simple: simple-walk-betw s p v*
 using *p(1)* **unfolding** *shortest-walk-def* **by** *blast*
 have *reach-v: reachable s v*
 using *p-simple* **unfolding** *reachable-def* **by** *blast*
 have *p ∈ set (exec-simple-walks-betw vs es s v)*
 using *p-simple* *set-exec-simple-walks-betw[OF vs es, of s v]* **by** *blast*
moreover have *exec-walk-weight w p = exec-dist vs es w s v*
 using *p(1)* *exec-dist-eq-dist[OF vs es reach-v]*
unfolding *shortest-walk-def* **by** *simp*
ultimately **show** *exec-shortest-through vs es w s x v*
 using *p(2)* **unfolding** *exec-shortest-through-def* **by** *blast*
qed

lemma *set-exec-bound-tree-vertices:*
 assumes *vs: set vs = V*
 and *es: set es = E*
 shows *set (exec-bound-tree-vertices vs es w s x B) = bound-tree {x} B*
proof
 show *set (exec-bound-tree-vertices vs es w s x B) ⊆ bound-tree {x} B*
proof
 fix *v*
 assume *v: v ∈ set (exec-bound-tree-vertices vs es w s x B)*
 then have *vV: v ∈ V*
 using *vs* **unfolding** *exec-bound-tree-vertices-def* **by** *auto*
 have *reach-v: reachable s v*
 using *v* *exec-reachable-iff-reachable[OF vs es, of s v]*
unfolding *exec-bound-tree-vertices-def* **by** *auto*
 have *through-v: through {x} v*
 using *v* *exec-shortest-through-iff-through-single[OF vs es, of x v]*
unfolding *exec-bound-tree-vertices-def* **by** *auto*
 have *below-v: below-bound (dist s v) B*
 using *v* *exec-dist-eq-dist[OF vs es reach-v]*
unfolding *exec-bound-tree-vertices-def* **by** *auto*
 show *v ∈ bound-tree {x} B*
 using *vV* *reach-v* *through-v* *below-v* **unfolding** *bound-tree-def* **by** *blast*
qed

next
 show *bound-tree {x} B ⊆ set (exec-bound-tree-vertices vs es w s x B)*
proof
 fix *v*
 assume *v: v ∈ bound-tree {x} B*
 then have *vV: v ∈ V* **and** *reach-v: reachable s v*

and *through-v*: *through* $\{x\}$ *v* **and** *below-v*: *below-bound* (*dist* *s* *v*) *B*
unfolding *bound-tree-def* **by** *auto*
have *exec-reachable* *vs es s v*
using *exec-reachable-iff-reachable*[*OF vs es, of s v*] *reach-v* **by** *simp*
moreover **have** *exec-shortest-through* *vs es w s x v*
using *exec-shortest-through-iff-through-single*[*OF vs es, of x v*] *through-v* **by**
simp
moreover **have** *below-bound* (*exec-dist* *vs es w s v*) *B*
using *below-v exec-dist-eq-dist*[*OF vs es reach-v*] **by** *simp*
ultimately **show** $v \in \text{set } (\text{exec-bound-tree-vertices } vs \text{ es } w \text{ s } x \text{ } B)$
using *vV vs* **unfolding** *exec-bound-tree-vertices-def* **by** *auto*
qed
qed

definition *base-case-order-impl* ::
 $'v \text{ list} \Rightarrow ('v \times 'v) \text{ list} \Rightarrow 'v \Rightarrow \text{bound} \Rightarrow 'v \text{ list}$ **where**
base-case-order-impl *vs es x B* =
exec-closest-vertices *vs es w s* (*exec-bound-tree-vertices* *vs es w s x B*)

definition *base-case-vertices-impl* ::
 $'v \text{ list} \Rightarrow ('v \times 'v) \text{ list} \Rightarrow \text{nat} \Rightarrow 'v \Rightarrow \text{bound} \Rightarrow 'v \text{ set}$ **where**
base-case-vertices-impl *vs es k x B* =
 (let *xs* = *base-case-order-impl* *vs es x B* in
 if *length xs* $\leq k$ then *set xs*
 else *set (filter* ($\lambda v. \text{exec-dist } vs \text{ es } w \text{ s } v < \text{exec-dist } vs \text{ es } w \text{ s } (xs ! k)$)
 (*take* (*Suc k*) *xs*)))

definition *base-case-bound-impl* ::
 $'v \text{ list} \Rightarrow ('v \times 'v) \text{ list} \Rightarrow \text{nat} \Rightarrow 'v \Rightarrow \text{bound} \Rightarrow \text{bound}$ **where**
base-case-bound-impl *vs es k x B* =
 (let *xs* = *base-case-order-impl* *vs es x B* in
 if *length xs* $\leq k$ then *B* else *Fin* (*exec-dist* *vs es w s* (*xs* ! *k*)))

definition *base-case-result-impl* ::
 $'v \text{ list} \Rightarrow ('v \times 'v) \text{ list} \Rightarrow \text{nat} \Rightarrow 'v \Rightarrow \text{bound} \Rightarrow \text{bound} \times 'v \text{ set}$ **where**
base-case-result-impl *vs es k x B* =
 (*base-case-bound-impl* *vs es k x B*, *base-case-vertices-impl* *vs es k x B*)

declare *base-case-order-impl-def* [*code*]
declare *base-case-vertices-impl-def* [*code*]
declare *base-case-bound-impl-def* [*code*]
declare *base-case-result-impl-def* [*code*]

lemma *base-case-order-impl-set*:
assumes *vs*: *set vs* = *V*
and *es*: *set es* = *E*
shows *set* (*base-case-order-impl* *vs es x B*) = *bound-tree* $\{x\}$ *B*
unfolding *base-case-order-impl-def*
by (*rule exec-closest-vertices-properties*(1))

[*OF vs es set-exec-bound-tree-vertices*[*OF vs es*]])
 (auto simp: bound-tree-def)

lemma *base-case-order-impl-eq-if-inj*:

assumes *vs*: set *vs* = *V*
 and *es*: set *es* = *E*
 and *inj*: *inj-on* (*dist s*) (bound-tree {*x*} *B*)
 shows *base-case-order* *x B* = *base-case-order-impl* *vs es x B*
 unfolding *base-case-order-def* *base-case-order-impl-def*
 by (rule *closest-vertices-eq-exec-if-inj*)
 [*OF vs es set-exec-bound-tree-vertices*[*OF vs es*]])
 (use *inj* in ⟨auto simp: bound-tree-def⟩)

lemma *base-case-order-impl-distinct*:

distinct (*base-case-order-impl* *vs es x B*)
 unfolding *base-case-order-impl-def* *exec-closest-vertices-def* by simp

lemma *base-case-order-impl-sorted*:

assumes *vs*: set *vs* = *V*
 and *es*: set *es* = *E*
 shows *sorted-wrt* ($\lambda u v. \text{dist } s \ u \leq \text{dist } s \ v$) (*base-case-order-impl* *vs es x B*)
 unfolding *base-case-order-impl-def*
 by (rule *exec-closest-vertices-properties*(3))
 [*OF vs es set-exec-bound-tree-vertices*[*OF vs es*]])
 (auto simp: bound-tree-def)

lemma *base-case-order-impl-length-eq*:

assumes *vs*: set *vs* = *V*
 and *es*: set *es* = *E*
 shows *length* (*base-case-order-impl* *vs es x B*) = *length* (*base-case-order* *x B*)
 proof –
 have *length* (*base-case-order-impl* *vs es x B*) =
 card (set (*base-case-order-impl* *vs es x B*))
 using *base-case-order-impl-distinct*[*of vs es x B*] by (simp add: *distinct-card*)
 also have ... = card (bound-tree {*x*} *B*)
 using *base-case-order-impl-set*[*OF vs es, of x B*] by simp
 also have ... = card (set (*base-case-order* *x B*))
 using *base-case-order-set*[*of x B*] by simp
 also have ... = *length* (*base-case-order* *x B*)
 using *base-case-order-distinct*[*of x B*] by (simp add: *distinct-card*)
 finally show ?thesis .
 qed

lemma *dist-map-eq-if-same-sorted-set*:

assumes *sorted-x*: *sorted-wrt* ($\lambda u v. \text{dist } s \ u \leq \text{dist } s \ v$) *xs*
 and *sorted-y*: *sorted-wrt* ($\lambda u v. \text{dist } s \ u \leq \text{dist } s \ v$) *ys*
 and *distinct-x*: *distinct* *xs*
 and *distinct-y*: *distinct* *ys*
 and *set-eq*: set *xs* = set *ys*

shows $\text{map } (\text{dist } s) \text{ } xs = \text{map } (\text{dist } s) \text{ } ys$
proof –
have $\text{sorted-}x'$: $\text{sorted } (\text{map } (\text{dist } s) \text{ } xs)$
using $\text{sorted-}x$ **by** $(\text{simp add: sorted-map})$
have $\text{sorted-}y'$: $\text{sorted } (\text{map } (\text{dist } s) \text{ } ys)$
using $\text{sorted-}y$ **by** $(\text{simp add: sorted-map})$
have $\text{mset } xs = \text{mset } ys$
using $\text{distinct-}x \text{ distinct-}y \text{ set-eq}$
by $(\text{simp add: set-eq-iff-mset-eq-distinct})$
then have mset-dist : $\text{mset } (\text{map } (\text{dist } s) \text{ } xs) = \text{mset } (\text{map } (\text{dist } s) \text{ } ys)$
by simp
have $\text{sort } (\text{map } (\text{dist } s) \text{ } ys) = \text{map } (\text{dist } s) \text{ } xs$
by $(\text{rule properties-for-sort}[OF \text{ mset-dist sorted-}x'])$
moreover have $\text{sort } (\text{map } (\text{dist } s) \text{ } ys) = \text{map } (\text{dist } s) \text{ } ys$
using $\text{sorted-}y'$ **by** $(\text{simp add: sort-key-id-if-sorted})$
ultimately show $?thesis$
by simp
qed

lemma $\text{base-case-order-impl-kth-dist-eq}$:
assumes vs : $\text{set } vs = V$
and es : $\text{set } es = E$
and k : $k < \text{length } (\text{base-case-order } x \text{ } B)$
shows $\text{dist } s ((\text{base-case-order } x \text{ } B) ! k) =$
 $\text{dist } s ((\text{base-case-order-impl } vs \text{ } es \text{ } x \text{ } B) ! k)$
proof –
let $?xs = \text{base-case-order } x \text{ } B$
let $?ys = \text{base-case-order-impl } vs \text{ } es \text{ } x \text{ } B$
have $\text{len-}y$: $k < \text{length } ?ys$
using k $\text{base-case-order-impl-length-eq}[OF \text{ } vs \text{ } es, \text{ of } x \text{ } B]$ **by** simp
have map-eq : $\text{map } (\text{dist } s) \text{ } ?xs = \text{map } (\text{dist } s) \text{ } ?ys$
by $(\text{rule dist-map-eq-if-same-sorted-set})$
 $(\text{use } \text{base-case-order-sorted } \text{base-case-order-impl-sorted}[OF \text{ } vs \text{ } es]$
 $\text{base-case-order-distinct } \text{base-case-order-impl-distinct}$
 $\text{base-case-order-set } \text{base-case-order-impl-set}[OF \text{ } vs \text{ } es] \text{ in } \text{auto})$
have $\text{map } (\text{dist } s) \text{ } ?xs ! k = \text{map } (\text{dist } s) \text{ } ?ys ! k$
using map-eq **by** simp
then show $?thesis$
using $k \text{ len-}y$ **by** simp
qed

lemma $\text{exec-dist-eq-dist-on-base-case-order-impl}$:
assumes vs : $\text{set } vs = V$
and es : $\text{set } es = E$
and v : $v \in \text{set } (\text{base-case-order-impl } vs \text{ } es \text{ } x \text{ } B)$
shows $\text{exec-dist } vs \text{ } es \text{ } w \text{ } s \text{ } v = \text{dist } s \text{ } v$
proof –
have $v \in \text{bound-tree } \{x\} \text{ } B$
using v $\text{base-case-order-impl-set}[OF \text{ } vs \text{ } es, \text{ of } x \text{ } B]$ **by** simp


```

    then have reachable s v
      unfolding bound-tree-def by blast
    then show ?thesis
      by (rule exec-dist-eq-dist[OF vs es])
qed

lemma base-case-bound-impl-eq:
  assumes vs: set vs = V
    and es: set es = E
  shows base-case-bound-impl vs es k x B = base-case-bound k x B
proof (cases length (base-case-order x B) ≤ k)
  case True
  then have length (base-case-order-impl vs es x B) ≤ k
    using base-case-order-impl-length-eq[OF vs es, of x B] by simp
  then show ?thesis
    using True unfolding base-case-bound-impl-def base-case-bound-def by (simp
add: Let-def)
next
  case False
  then have k < length (base-case-order x B)
    by simp
  let ?ys = base-case-order-impl vs es x B
  have len-y: k < length ?ys
    using k base-case-order-impl-length-eq[OF vs es, of x B] by simp
  have y-mem: ?ys ! k ∈ set ?ys
    using len-y by simp
  have exec-eq: exec-dist vs es w s (?ys ! k) = dist s (?ys ! k)
    using exec-dist-eq-dist-on-base-case-order-impl[OF vs es y-mem] .
  have dist-eq: dist s ((base-case-order x B) ! k) = dist s (?ys ! k)
    by (rule base-case-order-impl-kth-dist-eq[OF vs es k])
  show ?thesis
    using False len-y exec-eq dist-eq
    unfolding base-case-bound-impl-def base-case-bound-def
    by (simp add: Let-def)
qed

lemma base-case-vertices-impl-success:
  assumes vs: set vs = V
    and es: set es = E
    and len: length (base-case-order-impl vs es x B) ≤ k
  shows base-case-vertices-impl vs es k x B = bound-tree {x} B
  using len base-case-order-impl-set[OF vs es, of x B]
  unfolding base-case-vertices-impl-def by (simp add: Let-def)

lemma base-case-vertices-impl-partial:
  assumes vs: set vs = V
    and es: set es = E
    and len: k < length (base-case-order-impl vs es x B)
  shows base-case-vertices-impl vs es k x B =

```

```

    bound-tree {x} (Fin (exec-dist vs es w s ((base-case-order-impl vs es x B) ! k)))
  proof -
    let ?xs = base-case-order-impl vs es x B
    let ?b = dist s (?xs ! k)
    have set-xs: set ?xs = bound-tree {x} B
      using base-case-order-impl-set[OF vs es, of x B] .
    have sorted: sorted-wrt ( $\lambda u v. \text{dist } s \ u \leq \text{dist } s \ v$ ) ?xs
      using base-case-order-impl-sorted[OF vs es, of x B] .
    have kth-in: ?xs ! k  $\in$  bound-tree {x} B
      using len set-xs nth-mem by metis
    have kth-exec: exec-dist vs es w s (?xs ! k) = ?b
      using exec-dist-eq-dist-on-base-case-order-impl[OF vs es] len nth-mem by blast
    have exec-on-xs:  $\bigwedge v. v \in \text{set } ?xs \implies \text{exec-dist vs es w s } v = \text{dist } s \ v$ 
      by (rule exec-dist-eq-dist-on-base-case-order-impl[OF vs es])
    have exec-on-take:  $\bigwedge v. v \in \text{set } (\text{take } (\text{Suc } k) \ ?xs) \implies$ 
      exec-dist vs es w s v = dist s v
  proof -
    fix v
    assume v  $\in$  set (take (Suc k) ?xs)
    then have v  $\in$  set ?xs
      by (rule in-set-takeD)
    then show exec-dist vs es w s v = dist s v
      by (rule exec-on-xs)
  qed
  have vertices-eq:
    base-case-vertices-impl vs es k x B =
      {v  $\in$  set (take (Suc k) ?xs). dist s v < ?b}
    using len kth-exec exec-on-take
    unfolding base-case-vertices-impl-def by (auto simp: Let-def)
  have base-case-vertices-impl vs es k x B = bound-tree {x} (Fin ?b)
  proof
    show base-case-vertices-impl vs es k x B  $\subseteq$  bound-tree {x} (Fin ?b)
  proof
    fix v
    assume v  $\in$  base-case-vertices-impl vs es k x B
    then have v: v  $\in$  set (take (Suc k) ?xs) dist s v < ?b
      using vertices-eq by auto
    have v  $\in$  set ?xs
      using v(1) by (meson in-set-takeD)
    then have v  $\in$  bound-tree {x} B
      using set-xs by simp
    then show v  $\in$  bound-tree {x} (Fin ?b)
      using v(2) unfolding bound-tree-def by auto
  qed
  next
    show bound-tree {x} (Fin ?b)  $\subseteq$  base-case-vertices-impl vs es k x B
  proof
    fix v
    assume v: v  $\in$  bound-tree {x} (Fin ?b)

```

```

then have lt:  $\text{dist } s \ v < ?b$ 
  unfolding bound-tree-def by auto
have kth-below: below-bound ( $\text{dist } s \ (?xs \ ! \ k)$ )  $B$ 
  using kth-in unfolding bound-tree-def by auto
have below-bound ( $\text{dist } s \ v$ )  $B$ 
  using below-bound-less-trans[OF lt kth-below] .
then have  $v \in \text{bound-tree } \{x\} \ B$ 
  using v unfolding bound-tree-def by auto
then have  $v \in \text{set } ?xs$ 
  using set-xs by simp
then have  $v \in \text{set } (\text{take } k \ ?xs)$ 
  using in-set-take-dist-lt-nth[OF sorted - lt len] by blast
then have  $v \in \text{set } (\text{take } (\text{Suc } k) \ ?xs)$ 
  using set-take-subset-set-take[of k Suc k ?xs] by auto
then show  $v \in \text{base-case-vertices-impl } vs \ es \ k \ x \ B$ 
  using vertices-eq lt by simp
qed
qed
then show ?thesis
  using kth-exec by simp
qed

lemma base-case-vertices-impl-eq:
  assumes vs:  $\text{set } vs = V$ 
  and es:  $\text{set } es = E$ 
  shows  $\text{base-case-vertices-impl } vs \ es \ k \ x \ B = \text{base-case-vertices } k \ x \ B$ 
proof (cases  $\text{length } (\text{base-case-order } x \ B) \leq k$ )
case True
  then have len-impl:  $\text{length } (\text{base-case-order-impl } vs \ es \ x \ B) \leq k$ 
    using base-case-order-impl-length-eq[OF vs es, of x B] by simp
  show ?thesis
    using base-case-success[OF True] base-case-vertices-impl-success[OF vs es len-impl] by simp
next
case False
  then have k:  $k < \text{length } (\text{base-case-order } x \ B)$ 
    by simp
  have len-impl:  $k < \text{length } (\text{base-case-order-impl } vs \ es \ x \ B)$ 
    using k base-case-order-impl-length-eq[OF vs es, of x B] by simp
  let ?ys =  $\text{base-case-order-impl } vs \ es \ x \ B$ 
  have y-mem:  $?ys \ ! \ k \in \text{set } ?ys$ 
    using len-impl by simp
  have exec-eq:  $\text{exec-dist } vs \ es \ w \ s \ (?ys \ ! \ k) = \text{dist } s \ (?ys \ ! \ k)$ 
    using exec-dist-eq-dist-on-base-case-order-impl[OF vs es y-mem] .
  have dist-eq:  $\text{dist } s \ ((\text{base-case-order } x \ B) \ ! \ k) = \text{dist } s \ (?ys \ ! \ k)$ 
    by (rule base-case-order-impl-kth-dist-eq[OF vs es k])
  have impl:  $\text{base-case-vertices-impl } vs \ es \ k \ x \ B =$ 
     $\text{bound-tree } \{x\} \ (\text{Fin } (\text{exec-dist } vs \ es \ w \ s \ (?ys \ ! \ k)))$ 
    by (rule base-case-vertices-impl-partial[OF vs es len-impl])

```

have *sem*: *base-case-vertices* $k \ x \ B =$
 $\text{bound-tree } \{x\} \ (Fin \ (dist \ s \ ((base\text{-}case\text{-}order \ x \ B) \ ! \ k)))$
by (*rule base-case-partial*[*OF* k])
show *?thesis*
using *impl sem exec-eq dist-eq* **by** *simp*
qed

lemma *base-case-result-impl-eq*:
assumes *vs*: *set vs* = V
and *es*: *set es* = E
shows *base-case-result-impl* *vs es k x B* = *base-case-result* $k \ x \ B$
using *base-case-bound-impl-eq*[*OF* *vs es*, *of k x B*]
base-case-vertices-impl-eq[*OF* *vs es*, *of k x B*]
unfolding *base-case-result-impl-def base-case-result-def* **by** *simp*

lemma *base-case-result-impl-bmssp-post*:
assumes *vs*: *set vs* = V
and *es*: *set es* = E
and *S*: $S = \{x\}$
shows *case base-case-result-impl vs es k x B of* $(B', U) \Rightarrow$
 $bmssp\text{-}post \ d \ S \ B \ (\lambda v. \text{if } v \in U \text{ then } dist \ s \ v \text{ else } d \ v) \ B' \ U$
using *base-case-result-bmssp-post*[*OF* *S*, **where** $k=k$ **and** $B=B$ **and** $d=d$]
base-case-result-impl-eq[*OF* *vs es*, *of k x B*]
by *simp*

lemma *base-case-order-impl-eq-exec-base-case-order*:
base-case-order-impl *vs es x B* = *exec-base-case-order* $w \ s \ vs \ es \ x \ B$
unfolding *base-case-order-impl-def exec-base-case-order-def* **by** *simp*

lemma *base-case-vertices-impl-eq-exec-base-case-vertices*:
base-case-vertices-impl *vs es k x B* = *exec-base-case-vertices* $w \ s \ vs \ es \ k \ x \ B$
unfolding *base-case-vertices-impl-def exec-base-case-vertices-def*
base-case-order-impl-eq-exec-base-case-order **by** *simp*

lemma *base-case-bound-impl-eq-exec-base-case-bound*:
base-case-bound-impl *vs es k x B* = *exec-base-case-bound* $w \ s \ vs \ es \ k \ x \ B$
unfolding *base-case-bound-impl-def exec-base-case-bound-def*
base-case-order-impl-eq-exec-base-case-order **by** *simp*

lemma *base-case-result-impl-eq-exec-base-case-result*:
base-case-result-impl *vs es k x B* = *exec-base-case-result* $w \ s \ vs \ es \ k \ x \ B$
unfolding *base-case-result-impl-def exec-base-case-result-def*
base-case-bound-impl-eq-exec-base-case-bound
base-case-vertices-impl-eq-exec-base-case-vertices **by** *simp*

lemma *exec-base-case-bound-eq*:
assumes *vs*: *set vs* = V
and *es*: *set es* = E
shows *exec-base-case-bound* $w \ s \ vs \ es \ k \ x \ B = base\text{-}case\text{-}bound \ k \ x \ B$

```

using base-case-bound-impl-eq[OF vs es, of k x B]
unfolding base-case-bound-impl-eq-exec-base-case-bound .

lemma exec-base-case-vertices-eq:
  assumes vs: set vs = V
    and es: set es = E
  shows exec-base-case-vertices w s vs es k x B = base-case-vertices k x B
  using base-case-vertices-impl-eq[OF vs es, of k x B]
  unfolding base-case-vertices-impl-eq-exec-base-case-vertices .

lemma exec-base-case-result-eq:
  assumes vs: set vs = V
    and es: set es = E
  shows exec-base-case-result w s vs es k x B = base-case-result k x B
  using base-case-result-impl-eq[OF vs es, of k x B]
  unfolding base-case-result-impl-eq-exec-base-case-result .

lemmas base-case-bound-exec-code [code] = exec-base-case-bound-eq[symmetric]
lemmas base-case-vertices-exec-code [code] = exec-base-case-vertices-eq[symmetric]
lemmas base-case-result-exec-code [code] = exec-base-case-result-eq[symmetric]

end

declare finite-weighted-digraph.base-case-bound-exec-code [code]
declare finite-weighted-digraph.base-case-vertices-exec-code [code]
declare finite-weighted-digraph.base-case-result-exec-code [code]

definition example-edges :: (nat × nat) list where
  example-edges = map ( $\lambda(u, v, c). (u, v)$ ) example-graph

definition example-vertices :: nat list where
  example-vertices = bmssp-vertices example-graph 0

definition example-V :: nat set where
  example-V = set example-vertices

definition example-E :: (nat × nat) set where
  example-E = set example-edges

definition example-weight :: nat ⇒ nat ⇒ real where
  example-weight u v =
    (case map-of (map ( $\lambda(a, b, c). ((a, b), \text{real } c)$ ) example-graph) (u, v) of
      None ⇒ 0
    | Some c ⇒ c)

lemma example-vertices-set [simp]: set example-vertices = example-V
  unfolding example-vertices-def example-V-def by simp

lemma example-edges-set [simp]: set example-edges = example-E

```

```

unfolding example-edges-def example-E-def by simp

lemma example-finite-weighted-digraph:
  finite-weighted-digraph example-V example-E example-weight 0
unfolding finite-weighted-digraph-def
  example-V-def example-E-def example-vertices-def example-edges-def
  bmssp-vertices-def example-graph-def example-weight-def
by (auto split: prod.splits option.splits)

interpretation example-fw: finite-weighted-digraph example-V example-E example-
weight 0
by (rule example-finite-weighted-digraph)

lemma example-exec-simple-walk-0:
   $[0] \in \text{set } (\text{exec-simple-walks-betw } \text{example-vertices } \text{example-edges } 0 \ 0)$ 
by eval

lemma example-exec-simple-walk-1:
   $[0, 1] \in \text{set } (\text{exec-simple-walks-betw } \text{example-vertices } \text{example-edges } 0 \ 1)$ 
by eval

lemma example-exec-simple-walk-2:
   $[0, 1, 2] \in \text{set } (\text{exec-simple-walks-betw } \text{example-vertices } \text{example-edges } 0 \ 2)$ 
by eval

lemma example-exec-simple-walk-3:
   $[0, 1, 2, 3] \in \text{set } (\text{exec-simple-walks-betw } \text{example-vertices } \text{example-edges } 0 \ 3)$ 
by eval

lemma example-exec-simple-walk-4:
   $[0, 4] \in \text{set } (\text{exec-simple-walks-betw } \text{example-vertices } \text{example-edges } 0 \ 4)$ 
by eval

lemma example-exec-reachable-if-in-V:
  assumes  $v \in \text{example-V}$ 
  shows exec-reachable example-vertices example-edges 0 v
proof –
  have  $v = 0 \vee v = 1 \vee v = 2 \vee v = 3 \vee v = 4$ 
  using assms
  unfolding example-V-def example-vertices-def bmssp-vertices-def example-graph-
def
  by auto
  then show ?thesis
  using example-exec-simple-walk-0 example-exec-simple-walk-1
  example-exec-simple-walk-2 example-exec-simple-walk-3
  example-exec-simple-walk-4
  unfolding exec-reachable-def by auto
qed

```

```

lemma example-dist-inj-on-V:
  inj-on (finite-weighted-digraph.dist example-V example-E example-weight 0)
example-V
proof (rule inj-onI)
  fix u v
  assume uV: u ∈ example-V and vV: v ∈ example-V
  and dist-eq:
    finite-weighted-digraph.dist example-V example-E example-weight 0 u =
    finite-weighted-digraph.dist example-V example-E example-weight 0 v
  have reach-u: finite-weighted-digraph.reachable example-V example-E 0 u
  using example-exec-reachable-if-in-V[OF uV]
    example-fw.exec-reachable-iff-reachable[of example-vertices example-edges 0 u]
  by simp
  have reach-v: finite-weighted-digraph.reachable example-V example-E 0 v
  using example-exec-reachable-if-in-V[OF vV]
    example-fw.exec-reachable-iff-reachable[of example-vertices example-edges 0 v]
  by simp
  have u-eq:
    exec-dist example-vertices example-edges example-weight 0 u =
    finite-weighted-digraph.dist example-V example-E example-weight 0 u
  using example-fw.exec-dist-eq-dist[OF example-vertices-set example-edges-set
reach-u] .
  have v-eq:
    exec-dist example-vertices example-edges example-weight 0 v =
    finite-weighted-digraph.dist example-V example-E example-weight 0 v
  using example-fw.exec-dist-eq-dist[OF example-vertices-set example-edges-set
reach-v] .
  have exec-inj:
    inj-on (exec-dist example-vertices example-edges example-weight 0) example-V
  by eval
  show u = v
  using inj-onD[OF exec-inj - ] uV vV dist-eq u-eq v-eq by metis
qed

lemma example-base-case-order-code [code-unfold]:
  finite-weighted-digraph.base-case-order example-V example-E example-weight 0 x
B =
  exec-base-case-order example-weight 0 example-vertices example-edges x B
proof –
  have tree-subset:
    finite-weighted-digraph.bound-tree example-V example-E example-weight 0 {x} B
  ⊆
  example-V
  unfolding example-fw.bound-tree-def by auto
  have inj:
    inj-on (finite-weighted-digraph.dist example-V example-E example-weight 0)
    (finite-weighted-digraph.bound-tree example-V example-E example-weight 0 {x}
B)
  by (rule inj-on-subset[OF example-dist-inj-on-V tree-subset])

```

```

have finite-weighted-digraph.base-case-order example-V example-E example-weight
0 x B =
  finite-weighted-digraph.base-case-order-impl example-weight 0
  example-vertices example-edges x B
using example-fw.base-case-order-impl-eq-if-inj
  [OF example-vertices-set example-edges-set inj] .
then show ?thesis
using example-fw.base-case-order-impl-eq-exec-base-case-order
  [of example-vertices example-edges x B]
by simp
qed

```

```

lemma example-base-case-vertices-code [code-unfold]:
  finite-weighted-digraph.base-case-vertices example-V example-E example-weight 0
k x B =
  exec-base-case-vertices example-weight 0 example-vertices example-edges k x B
using example-fw.exec-base-case-vertices-eq[of example-vertices example-edges k x
B]
by simp

```

```

lemma example-base-case-result-code [code-unfold]:
  finite-weighted-digraph.base-case-result example-V example-E example-weight 0 k x
B =
  exec-base-case-result example-weight 0 example-vertices example-edges k x B
using example-fw.exec-base-case-result-eq[of example-vertices example-edges k x
B]
by simp

```

```

value exec-closest-vertices
  (bmssp-vertices example-graph 0) example-edges example-weight 0
  (bmssp-vertices example-graph 0)

```

```

value exec-base-case-order
  example-weight 0 (bmssp-vertices example-graph 0) example-edges 0 (Fin 100)

```

```

value exec-base-case-result
  example-weight 0 (bmssp-vertices example-graph 0) example-edges 3 0 (Fin 100)
::
  bound × nat set

```

```

value finite-weighted-digraph.base-case-vertices
  example-V example-E example-weight 0 3 0 (Fin 100) :: nat set

```

```

value finite-weighted-digraph.base-case-order
  example-V example-E example-weight 0 0 (Fin 100)

```

```

value finite-weighted-digraph.base-case-result
  example-V example-E example-weight 0 3 0 (Fin 100) :: bound × nat set

```



```

end
theory BMSSP-Unique-Shortest-Tree
  imports BMSSP-Shortest-Path-Lemmas
begin

```

18 Unique Shortest-Path Tree

After tie-breaking, shortest paths form a tree rooted at the source. This locale makes that assumption explicit and derives the tree notions used by later BMSSP arguments from the graph model.

```

locale unique-shortest-digraph = finite-weighted-digraph +
  assumes unique-shortest-walk:
     $\llbracket \text{shortest-walk } s \ p \ v; \text{shortest-walk } s \ q \ v \rrbracket \implies p = q$ 
begin

```

```

definition shortest-path-to where
  shortest-path-to  $v = (THE \ p. \text{shortest-walk } s \ p \ v)$ 

```

```

definition tree-path where
  tree-path  $u \ v \longleftrightarrow \text{reachable } s \ v \wedge u \in \text{set } (\text{shortest-path-to } v)$ 

```

```

definition tree-of where
  tree-of  $u = \{v \in V. \text{tree-path } u \ v\}$ 

```

```

definition tree-set where
  tree-set  $S = \{v \in V. \exists u \in S. \text{tree-path } u \ v\}$ 

```

```

definition tree-antichain where
  tree-antichain  $S \longleftrightarrow (\forall u \in S. \forall v \in S. \text{tree-path } u \ v \longrightarrow u = v)$ 

```

```

lemma tree-antichain-subset:
  assumes anti: tree-antichain  $T$ 
  and subset:  $S \subseteq T$ 
  shows tree-antichain  $S$ 
  using anti subset unfolding tree-antichain-def by blast

```

```

lemma tree-antichain-singleton [simp]:
  tree-antichain  $\{x\}$ 
  unfolding tree-antichain-def by blast

```

```

lemma tree-pathD:
  assumes tree-path  $u \ v$ 
  shows reachable  $s \ v \wedge u \in \text{set } (\text{shortest-path-to } v)$ 
  using assms unfolding tree-path-def by auto

```

```

lemma tree-pathI:
  assumes reachable  $s \ v \wedge u \in \text{set } (\text{shortest-path-to } v)$ 
  shows tree-path  $u \ v$ 

```

```

using assms unfolding tree-path-def by auto

lemma shortest-path-to-ex1:
  assumes reachable s v
  shows  $\exists! p. \text{shortest-walk } s \ p \ v$ 
proof -
  obtain p where p: shortest-walk s p v
  using assms by (rule shortest-walk-exists)
  have  $\bigwedge q. \text{shortest-walk } s \ q \ v \implies q = p$ 
  using p unique-shortest-walk by blast
  then show ?thesis
  using p by blast
qed

lemma shortest-path-to-shortest:
  assumes reachable s v
  shows shortest-walk s (shortest-path-to v) v
  unfolding shortest-path-to-def
  using shortest-path-to-ex1[OF assms] by (rule theI')

lemma shortest-path-to-successively-tight:
  assumes reachable s v
  shows successively tight-edge-step (shortest-path-to v)
  using shortest-walk-successively-tight[OF shortest-path-to-shortest[OF assms]] .

lemma shortest-path-prefix-eq:
  assumes reach-v: reachable s v
  and i:  $i < \text{length } (\text{shortest-path-to } v)$ 
  shows shortest-path-to (shortest-path-to v ! i) =
    take (Suc i) (shortest-path-to v)
proof -
  let ?p = shortest-path-to v
  have sp: shortest-walk s ?p v
  using shortest-path-to-shortest[OF reach-v] .
  have pref-short: shortest-walk s (take (Suc i) ?p) (?p ! i)
  using shortest-walk-prefix-shortest[OF sp i] .
  have reach-i: reachable s (?p ! i)
  using pref-short unfolding shortest-walk-def reachable-def by blast
  have canon-short: shortest-walk s (shortest-path-to (?p ! i)) (?p ! i)
  using shortest-path-to-shortest[OF reach-i] .
  show ?thesis
  using unique-shortest-walk[OF canon-short pref-short] by simp
qed

lemma tree-path-between-shortest-path-indices:
  assumes reach-v: reachable s v
  and ij:  $i \leq j$ 
  and j:  $j < \text{length } (\text{shortest-path-to } v)$ 
  shows tree-path (shortest-path-to v ! i) (shortest-path-to v ! j)

```

```

proof –
  let ?p = shortest-path-to v
  have sp: shortest-walk s ?p v
    using shortest-path-to-shortest[OF reach-v] .
  have pref-j: shortest-walk s (take (Suc j) ?p) (?p ! j)
    using shortest-walk-prefix-shortest[OF sp j] .
  have reach-j: reachable s (?p ! j)
    using pref-j unfolding shortest-walk-def reachable-def by blast
  have i-take: i < length (take (Suc j) ?p)
    using ij j by simp
  have take-i: take (Suc j) ?p ! i = ?p ! i
    using ij by simp
  have ?p ! i ∈ set (take (Suc j) ?p)
    using nth-mem[OF i-take] take-i by metis
  then have in-canon: ?p ! i ∈ set (shortest-path-to (?p ! j))
    using shortest-path-prefix-eq[OF reach-v j] by simp
  show ?thesis
    using reach-j in-canon by (rule tree-pathI)
qed

lemma tree-prefix-of-shortest-path-suffix:
  assumes reach-v: reachable s v
    and i: i < length (shortest-path-to v)
  shows set (take k (drop i (shortest-path-to v))) ⊆
    tree-of (shortest-path-to v ! i)
proof
  let ?p = shortest-path-to v
  fix x
  assume x-take: x ∈ set (take k (drop i ?p))
  obtain r where r: r < length (take k (drop i ?p))
    take k (drop i ?p) ! r = x
    using x-take by (auto simp: in-set-conv-nth)
  have r-drop: r < length (drop i ?p)
    using r(1) by simp
  have j-len: i + r < length ?p
    using r-drop i by simp
  have x-drop: x = drop i ?p ! r
    using r by simp
  have x-eq: x = ?p ! (i + r)
    using x-drop r-drop by simp
  have tree: tree-path (?p ! i) x
    using tree-path-between-shortest-path-indices[OF reach-v - j-len, of i]
      x-eq by simp
  have xV: x ∈ V
proof –
  have shortest-walk s ?p v
    using shortest-path-to-shortest[OF reach-v] .
  then have walk ?p
    unfolding shortest-walk-def simple-walk-betw-def walk-betw-def by blast

```

```

    moreover have  $x \in \text{set } ?p$ 
      using  $x\text{-eq } nth\text{-mem}[OF\ j\text{-len}]$  by simp
    ultimately show  $?thesis$ 
      using  $walk\text{-set-subset}$  by blast
  qed
  show  $x \in \text{tree-of } (?p ! i)$ 
    using  $xV\ \text{tree}\ \text{unfolding}\ \text{tree-of-def}$  by blast
qed

```

```

lemma tree-path-dist-le:
  assumes  $\text{tree-path } u\ v$ 
  shows  $\text{dist } s\ u \leq \text{dist } s\ v$ 
proof -
  have  $\text{reach: reachable } s\ v$  and  $u: u \in \text{set } (\text{shortest-path-to } v)$ 
    using  $\text{assms}$  by (auto dest:  $\text{tree-pathD}$ )
  have  $\text{shortest-walk } s\ (\text{shortest-path-to } v)\ v$ 
    using  $\text{shortest-path-to-shortest}[OF\ \text{reach}]$  .
  then show  $?thesis$ 
    using  $\text{shortest-walk-prefix-dist-le } u$  by blast
qed

```

```

lemma tree-path-root-reachable:
  assumes  $\text{tree-path } u\ v$ 
  shows  $\text{reachable } s\ u$ 
proof -
  have  $\text{reach-v: reachable } s\ v$  and  $u\text{-path: } u \in \text{set } (\text{shortest-path-to } v)$ 
    using  $\text{assms}$  by (auto dest:  $\text{tree-pathD}$ )
  obtain  $i$  where  $i: i < \text{length } (\text{shortest-path-to } v)\ \text{shortest-path-to } v ! i = u$ 
    using  $u\text{-path}$  by (auto simp:  $\text{in-set-conv-nth}$ )
  have  $\text{sp: shortest-walk } s\ (\text{shortest-path-to } v)\ v$ 
    using  $\text{shortest-path-to-shortest}[OF\ \text{reach-v}]$  .
  have  $\text{shortest-walk } s\ (\text{take } (\text{Suc } i)\ (\text{shortest-path-to } v))\ u$ 
    using  $\text{shortest-walk-prefix-shortest}[OF\ \text{sp } i(1)]\ i(2)$  by simp
  then show  $?thesis$ 
    unfolding  $\text{shortest-walk-def}\ \text{reachable-def}$  by blast
qed

```

```

lemma tree-path-root-in-V:
  assumes  $\text{tree-path } u\ v$ 
  shows  $u \in V$ 
proof -
  have  $\text{reachable } s\ u$ 
    using  $\text{tree-path-root-reachable}[OF\ \text{assms}]$  .
  then obtain  $p$  where  $p: \text{simple-walk-betw } s\ p\ u$ 
    unfolding  $\text{reachable-def}$  by blast
  then have  $\text{walk } p$  and  $u \in \text{set } p$ 
    unfolding  $\text{simple-walk-betw-def}\ \text{walk-betw-def}$ 
    by (auto intro:  $\text{last-in-set}$ )
  then show  $?thesis$ 

```

using *walk-set-subset* by *blast*
qed

lemma *tree-path-trans*:
 assumes *uv*: *tree-path* *u* *v*
 and *xu*: *tree-path* *x* *u*
 shows *tree-path* *x* *v*
proof –
 have *reach-v*: *reachable* *s* *v* and *u-path*: $u \in \text{set } (\text{shortest-path-to } v)$
 using *uv* by (auto dest: *tree-pathD*)
 have *x-path-u*: $x \in \text{set } (\text{shortest-path-to } u)$
 using *xu* by (auto dest: *tree-pathD*)
 obtain *i* where *i*: $i < \text{length } (\text{shortest-path-to } v)$ *shortest-path-to* *v* ! *i* = *u*
 using *u-path* by (auto simp: *in-set-conv-nth*)
 have *shortest-path-to* *u* = *take* (*Suc* *i*) (*shortest-path-to* *v*)
 using *shortest-path-prefix-eq*[*OF reach-v* *i*(1)] *i*(2) by *simp*
 then have $x \in \text{set } (\text{take } (\text{Suc } i) (\text{shortest-path-to } v))$
 using *x-path-u* by *simp*
 then have $x \in \text{set } (\text{shortest-path-to } v)$
 by (*meson in-set-takeD*)
 then show ?*thesis*
 by (*rule tree-pathI*[*OF reach-v*])
qed

lemma *tree-path-comparable*:
 assumes *ux*: *tree-path* *u* *x*
 and *vx*: *tree-path* *v* *x*
 shows *tree-path* *u* *v* $\vee$ *tree-path* *v* *u*
proof –
 have *reach-x*: *reachable* *s* *x*
 using *ux* by (auto dest: *tree-pathD*)
 have *u-path*: $u \in \text{set } (\text{shortest-path-to } x)$
 and *v-path*: $v \in \text{set } (\text{shortest-path-to } x)$
 using *ux vx* by (auto dest: *tree-pathD*)
 obtain *i* where *i*: $i < \text{length } (\text{shortest-path-to } x)$ *shortest-path-to* *x* ! *i* = *u*
 using *u-path* by (auto simp: *in-set-conv-nth*)
 obtain *j* where *j*: $j < \text{length } (\text{shortest-path-to } x)$ *shortest-path-to* *x* ! *j* = *v*
 using *v-path* by (auto simp: *in-set-conv-nth*)
 show ?*thesis*
proof (*cases* $i \leq j$)
 case *True*
 have *tree-path* *u* *v*
 using *tree-path-between-shortest-path-indices*[*OF reach-x* *True* *j*(1)] *i*(2) *j*(2)
 by *simp*
 then show ?*thesis*
 by *blast*
 next
 case *False*
 then have $j \leq i$

```

    by simp
  have tree-path v u
    using tree-path-between-shortest-path-indices[OF reach-x <j ≤ i> i(1)] i(2)
j(2)
    by simp
  then show ?thesis
    by blast
qed
qed

```

```

lemma tree-antichainD:
  assumes tree-antichain S
    and u ∈ S v ∈ S tree-path u v
  shows u = v
  using assms unfolding tree-antichain-def by blast

```

```

lemma tree-of-disjoint-if-antichain:
  assumes anti: tree-antichain S
    and uS: u ∈ S
    and vS: v ∈ S
    and neq: u ≠ v
  shows tree-of u ∩ tree-of v = {}
proof
  show tree-of u ∩ tree-of v ⊆ {}
  proof
    fix x
    assume x: x ∈ tree-of u ∩ tree-of v
    then have ux: tree-path u x and vx: tree-path v x
      unfolding tree-of-def by auto
    have tree-path u v ∨ tree-path v u
      using tree-path-comparable[OF ux vx] .
    then have u = v
    proof
      assume uv': tree-path u v
      then show ?thesis
        using tree-antichainD[OF anti uS vS uv'] by simp
    next
      assume vu': tree-path v u
      then show ?thesis
        using tree-antichainD[OF anti vS uS vu'] by simp
    qed
    with neq show x ∈ {}
    by blast
  qed
qed simp

```

```

lemma through-iff-tree-path:
  assumes reachable s v
  shows through S v ⟷ (∃ u ∈ S. tree-path u v)

```

```

proof
  assume through S v
  then obtain u p where u: u ∈ S shortest-walk s p v u ∈ set p
    unfolding through-def by blast
  have p = shortest-path-to v
    using shortest-path-to-shortest[OF assms] u(2) unique-shortest-walk by blast
  then have tree-path u v
    using assms u by (auto intro: tree-pathI)
  then show  $\exists u \in S. \text{tree-path } u \ v$ 
    using u(1) by blast
next
  assume  $\exists u \in S. \text{tree-path } u \ v$ 
  then obtain u where u: u ∈ S tree-path u v
    by blast
  then have u ∈ set (shortest-path-to v)
    by (auto dest: tree-pathD)
  moreover have shortest-walk s (shortest-path-to v) v
    using shortest-path-to-shortest[OF assms] .
  ultimately show through S v
    using u(1) unfolding through-def by blast
qed

lemma bound-tree-eq-tree-set:
  bound-tree S B = {v ∈ tree-set S. below-bound (dist s v) B}
proof
  show bound-tree S B ⊆ {v ∈ tree-set S. below-bound (dist s v) B}
  proof
    fix v
    assume v: v ∈ bound-tree S B
    then have reach: reachable s v and through S v and below: below-bound (dist s
v) B and v ∈ V
      unfolding bound-tree-def by auto
    then obtain u where u ∈ S tree-path u v
      using through-iff-tree-path[OF reach, of S] by blast
    then have v ∈ tree-set S
      using  $\langle v \in V \rangle$  unfolding tree-set-def by blast
    then show v ∈ {v ∈ tree-set S. below-bound (dist s v) B}
      using below by simp
  qed
next
  show  $\{v \in \text{tree-set } S. \text{below-bound } (\text{dist } s \ v) \ B\} \subseteq \text{bound-tree } S \ B$ 
  proof
    fix v
    assume v: v ∈ {v ∈ tree-set S. below-bound (dist s v) B}
    then obtain u where vV: v ∈ V and uS: u ∈ S and tp: tree-path u v
      and below: below-bound (dist s v) B
      unfolding tree-set-def by blast
    then have reach: reachable s v
      by (auto dest: tree-pathD)

```

```

    have through S v
      using through-iff-tree-path[OF reach, of S] uS tp by blast
    then show v ∈ bound-tree S B
      using vV reach below unfolding bound-tree-def by blast
  qed
qed

end

end
theory BMSSP-Find-Pivots-Core
  imports BMSSP-Unique-Shortest-Tree
begin

```

19 FindPivots Core Relaxation Lemmas

The concrete FindPivots procedure performs bounded Bellman-Ford-style relaxations. This theory packages the generic relaxation facts in the form needed for that proof: vertices reached by a bounded tight path from a complete source are complete after the corresponding path relaxations, and remain complete through additional valid relaxations.

```

context unique-shortest-digraph
begin

```

```

lemma finite-E [simp]: finite E
proof –
  have  $E \subseteq V \times V$ 
    using edge-in-V by auto
  moreover have finite ( $V \times V$ )
    using finite-V by simp
  ultimately show ?thesis
    by (rule finite-subset)
qed

```

```

definition outgoing-edges where
  outgoing-edges F =  $\{(u, v) \in E. u \in F\}$ 

```

```

lemma finite-outgoing-edges [simp]:
  finite (outgoing-edges F)
proof –
  have outgoing-edges F  $\subseteq E$ 
    unfolding outgoing-edges-def by blast
  then show ?thesis
    by (rule finite-subset) simp
qed

```

```

definition edge-list-of where
  edge-list-of A = (SOME xs. set xs = A  $\wedge$  distinct xs)

```


lemma *edge-list-of-properties*:
 assumes *finite A*
 shows *set (edge-list-of A) = A*
 and *distinct (edge-list-of A)*
proof –
 have $\exists xs. \text{set } xs = A \wedge \text{distinct } xs$
 using *finite-distinct-list[OF assms]* .
 then have *props: set (edge-list-of A) = A $\wedge$ distinct (edge-list-of A)*
 unfolding *edge-list-of-def* **by** (*rule someI-ex*)
 then show *set (edge-list-of A) = A*
by *blast*
 from *props* show *distinct (edge-list-of A)*
by *blast*
qed

definition *relax-frontier* **where**
 $\text{relax-frontier } d \ F = \text{relax-edges } d \ (\text{edge-list-of } (\text{outgoing-edges } F))$

lemma *relax-frontier-le*:
 $\text{relax-frontier } d \ F \ x \leq d \ x$
 unfolding *relax-frontier-def* **by** (*rule relax-edges-le*)

lemma *relax-frontier-sound*:
 assumes *sound: sound-label d*
 and *frontier-reaches: $\bigwedge u. u \in F \implies \text{reachable } s \ u$*
 shows *sound-label (relax-frontier d F)*
proof –
 let *?es* = *edge-list-of (outgoing-edges F)*
 have *set-es: set ?es = outgoing-edges F*
 using *edge-list-of-properties(1)[OF finite-outgoing-edges]* .
 have *edge: $\bigwedge u \ v. (u, v) \in \text{set } ?es \implies (u, v) \in E$*
 using *set-es* unfolding *outgoing-edges-def* **by** *blast*
 have *reach: $\bigwedge u \ v. (u, v) \in \text{set } ?es \implies \text{reachable } s \ u$*
 using *set-es* *frontier-reaches* unfolding *outgoing-edges-def* **by** *blast*
 show *?thesis*
 unfolding *relax-frontier-def* using *relax-edges-sound[OF sound edge reach]* .
qed

fun *relax-rounds* **where**
 $\text{relax-rounds } d \ [] = d$
 $|\ \text{relax-rounds } d \ (r \ \# \ rs) = \text{relax-rounds } (\text{relax-edges } d \ r) \ rs$

definition *within-k-edges* **where**
 $\text{within-k-edges } p \ k \longleftrightarrow p \neq [] \wedge \text{length } (\text{path-edges } p) \leq k$

lemma *relax-rounds-concat*:
 $\text{relax-rounds } d \ \text{rounds} = \text{relax-edges } d \ (\text{concat } \text{rounds})$
by (*induction rounds arbitrary: d*) (*simp-all add: relax-edges-append*)

lemma *relax-rounds-sound*:

assumes *sound*: *sound-label* *d*
 and *edges*: $\bigwedge u v. (u, v) \in \text{set } (\text{concat } \text{rounds}) \implies (u, v) \in E$
 and *reaches*: $\bigwedge u v. (u, v) \in \text{set } (\text{concat } \text{rounds}) \implies \text{reachable } s u$
 shows *sound-label* (*relax-rounds* *d* *rounds*)
proof –
 have *sound-label* (*relax-edges* *d* (*concat rounds*))
 by (rule *relax-edges-sound*[*OF sound edges reaches*])
 then show ?thesis
 by (simp add: *relax-rounds-concat*)
qed

lemma *relax-rounds-preserves-complete-sound*:

assumes *sound*: *sound-label* *d*
 and *complete-x*: *d* *x* = *dist* *s* *x*
 and *edges*: $\bigwedge u v. (u, v) \in \text{set } (\text{concat } \text{rounds}) \implies (u, v) \in E$
 and *reaches*: $\bigwedge u v. (u, v) \in \text{set } (\text{concat } \text{rounds}) \implies \text{reachable } s u$
 shows *relax-rounds* *d* *rounds* *x* = *dist* *s* *x*
proof –
 have *relax-edges* *d* (*concat rounds*) *x* = *dist* *s* *x*
 by (rule *relax-edges-preserves-complete-sound*[*OF sound complete-x edges reaches*])
 then show ?thesis
 by (simp add: *relax-rounds-concat*)
qed

lemma *path-edges-length*:

length (*path-edges* *p*) = *length* *p* – 1
 by (induction *p* rule: *path-edges.induct*) auto

lemma *within-k-edgesD*:

assumes *within-k-edges* *p* *k*
 shows *p* ≠ [] *length* (*path-edges* *p*) ≤ *k*
 using *assms* **unfolding** *within-k-edges-def* **by** auto

lemma *find-pivots-tight-path-prefix-completion*:

assumes *bounded*: *within-k-edges* *p* *k*
 and *sound*: *sound-label* *d*
 and *complete-source*: *d* (*hd* *p*) = *dist* *s* (*hd* *p*)
 and *tight*: *successively tight-edge-step* *p*
 and *es*: *es* = *path-edges* *p* @ *extra*
 and *extra-edges*: $\bigwedge u v. (u, v) \in \text{set } \text{extra} \implies (u, v) \in E$
 and *extra-reaches*: $\bigwedge u v. (u, v) \in \text{set } \text{extra} \implies \text{reachable } s u$
 shows *sound-label* (*relax-edges* *d* *es*)
 and *relax-edges* *d* *es* (*last* *p*) = *dist* *s* (*last* *p*)
proof –
 have *nonempty*: *p* ≠ []
 using *within-k-edgesD*(1)[*OF bounded*] .

```

show sound-label (relax-edges d es)
  using relax-edges-with-tight-path-prefix-complete(1)
  [OF nonempty sound complete-source tight es extra-edges extra-reaches] .
show relax-edges d es (last p) = dist s (last p)
  using relax-edges-with-tight-path-prefix-complete(2)
  [OF nonempty sound complete-source tight es extra-edges extra-reaches] .
qed

```

lemma find-pivots-tight-path-block-completion:

```

assumes bounded: within-k-edges p k
  and sound: sound-label d
  and complete-source: d (hd p) = dist s (hd p)
  and tight: successively tight-edge-step p
  and es: es = pre @ path-edges p @ extra
  and pre-edges:  $\bigwedge u v. (u, v) \in \text{set pre} \implies (u, v) \in E$ 
  and pre-reaches:  $\bigwedge u v. (u, v) \in \text{set pre} \implies \text{reachable } s u$ 
  and extra-edges:  $\bigwedge u v. (u, v) \in \text{set extra} \implies (u, v) \in E$ 
  and extra-reaches:  $\bigwedge u v. (u, v) \in \text{set extra} \implies \text{reachable } s u$ 
shows sound-label (relax-edges d es)
  and relax-edges d es (last p) = dist s (last p)
proof –
  let ?d0 = relax-edges d pre
  have nonempty:  $p \neq []$ 
    using within-k-edgesD(1)[OF bounded] .
  have sound0: sound-label ?d0
    using relax-edges-sound[OF sound pre-edges pre-reaches] .
  have complete0: ?d0 (hd p) = dist s (hd p)
    using relax-edges-preserves-complete-sound[OF sound complete-source pre-edges
pre-reaches] .
  have tail-sound: sound-label (relax-edges ?d0 (path-edges p @ extra))
    using relax-edges-with-tight-path-prefix-complete(1)
    [OF nonempty sound0 complete0 tight refl extra-edges extra-reaches] .
  have tail-complete: relax-edges ?d0 (path-edges p @ extra) (last p) = dist s (last
p)
    using relax-edges-with-tight-path-prefix-complete(2)
    [OF nonempty sound0 complete0 tight refl extra-edges extra-reaches] .
  have unfold-es: relax-edges d es = relax-edges ?d0 (path-edges p @ extra)
    using es relax-edges-append[of d pre path-edges p @ extra] by simp
  show sound-label (relax-edges d es)
    using tail-sound unfold-es by simp
  show relax-edges d es (last p) = dist s (last p)
    using tail-complete unfold-es by simp
qed

```

lemma find-pivots-rounds-block-completion:

```

assumes bounded: within-k-edges p k
  and sound: sound-label d
  and complete-source: d (hd p) = dist s (hd p)
  and tight: successively tight-edge-step p

```

and *rounds*: *concat rounds* = *pre* @ *path-edges p* @ *extra*
and *pre-edges*: $\bigwedge u v. (u, v) \in \text{set } \text{pre} \implies (u, v) \in E$
and *pre-reaches*: $\bigwedge u v. (u, v) \in \text{set } \text{pre} \implies \text{reachable } s u$
and *extra-edges*: $\bigwedge u v. (u, v) \in \text{set } \text{extra} \implies (u, v) \in E$
and *extra-reaches*: $\bigwedge u v. (u, v) \in \text{set } \text{extra} \implies \text{reachable } s u$
shows *sound-label* (*relax-rounds d rounds*)
and *relax-rounds d rounds* (*last p*) = *dist s* (*last p*)
proof –
have *edge-sound*: *sound-label* (*relax-edges d* (*concat rounds*))
using *find-pivots-tight-path-block-completion*(1)
[*OF bounded sound complete-source tight rounds pre-edges pre-reaches extra-edges extra-reaches*].
have *endpoint*: *relax-edges d* (*concat rounds*) (*last p*) = *dist s* (*last p*)
using *find-pivots-tight-path-block-completion*(2)
[*OF bounded sound complete-source tight rounds pre-edges pre-reaches extra-edges extra-reaches*].
show *sound-label* (*relax-rounds d rounds*)
using *edge-sound* **by** (*simp add: relax-rounds-concat*)
show *relax-rounds d rounds* (*last p*) = *dist s* (*last p*)
using *endpoint* **by** (*simp add: relax-rounds-concat*)
qed

lemma *relax-frontier-tight-successor-complete*:

assumes *sound*: *sound-label d*
and *uF*: $u \in F$
and *frontier-reaches*: $\bigwedge x. x \in F \implies \text{reachable } s x$
and *complete-u*: $d u = \text{dist } s u$
and *tight*: *tight-edge-step u v*
shows *sound-label* (*relax-frontier d F*)
and *relax-frontier d F v* = *dist s v*
proof –
let *?es* = *edge-list-of* (*outgoing-edges F*)
have *set-es*: *set ?es* = *outgoing-edges F*
using *edge-list-of-properties*(1)[*OF finite-outgoing-edges*].
have *edge-uv*: $(u, v) \in E$
using *tight unfolding tight-edge-step-def* **by** *blast*
have *mem-uv*: $(u, v) \in \text{set } ?es$
using *set-es edge-uv uF unfolding outgoing-edges-def* **by** *blast*
obtain *pre extra* **where** *es-split*: $?es = \text{pre} @ (u, v) \# \text{extra}$
using *split-list*[*OF mem-uv*] **by** *blast*
have *bounded*: *within-k-edges* [*u, v*] (*Suc 0*)
unfolding *within-k-edges-def* **by** *simp*
have *tight-path*: *successively tight-edge-step* [*u, v*]
using *tight* **by** *simp*
have *es-path*: $?es = \text{pre} @ \text{path-edges } [u, v] @ \text{extra}$
using *es-split* **by** *simp*
have *pre-edges*: $\bigwedge a b. (a, b) \in \text{set } \text{pre} \implies (a, b) \in E$
using *set-es es-split unfolding outgoing-edges-def* **by** *auto*
have *pre-reaches*: $\bigwedge a b. (a, b) \in \text{set } \text{pre} \implies \text{reachable } s a$

```

    using set-es es-split frontier-reaches unfolding outgoing-edges-def by auto
  have extra-edges:  $\bigwedge a b. (a, b) \in \text{set extra} \implies (a, b) \in E$ 
    using set-es es-split unfolding outgoing-edges-def by auto
  have extra-reaches:  $\bigwedge a b. (a, b) \in \text{set extra} \implies \text{reachable } s a$ 
    using set-es es-split frontier-reaches unfolding outgoing-edges-def by auto
  have core1-path: sound-label (relax-edges d (pre @ path-edges [u, v] @ extra))
  proof (rule find-pivots-tight-path-block-completion(1))
    [where p = [u, v] and k = Suc 0 and pre = pre and extra = extra]
    show within-k-edges [u, v] (Suc 0)
      using bounded .
    show sound-label d
      using sound .
    show d (hd [u, v]) = dist s (hd [u, v])
      using complete-u by simp
    show successively tight-edge-step [u, v]
      using tight-path .
    show pre @ path-edges [u, v] @ extra = pre @ path-edges [u, v] @ extra
      by simp
    show  $\bigwedge a b. (a, b) \in \text{set pre} \implies (a, b) \in E$ 
      using pre-edges .
    show  $\bigwedge a b. (a, b) \in \text{set pre} \implies \text{reachable } s a$ 
      using pre-reaches .
    show  $\bigwedge a b. (a, b) \in \text{set extra} \implies (a, b) \in E$ 
      using extra-edges .
    show  $\bigwedge a b. (a, b) \in \text{set extra} \implies \text{reachable } s a$ 
      using extra-reaches .
  qed
  have core2-path: relax-edges d (pre @ path-edges [u, v] @ extra) v = dist s v
  proof -
    have relax-edges d (pre @ path-edges [u, v] @ extra) (last [u, v]) =
      dist s (last [u, v])
    proof (rule find-pivots-tight-path-block-completion(2))
      [where p = [u, v] and k = Suc 0 and pre = pre and extra = extra]
      show within-k-edges [u, v] (Suc 0)
        using bounded .
      show sound-label d
        using sound .
      show d (hd [u, v]) = dist s (hd [u, v])
        using complete-u by simp
      show successively tight-edge-step [u, v]
        using tight-path .
      show pre @ path-edges [u, v] @ extra = pre @ path-edges [u, v] @ extra
        by simp
      show  $\bigwedge a b. (a, b) \in \text{set pre} \implies (a, b) \in E$ 
        using pre-edges .
      show  $\bigwedge a b. (a, b) \in \text{set pre} \implies \text{reachable } s a$ 
        using pre-reaches .
      show  $\bigwedge a b. (a, b) \in \text{set extra} \implies (a, b) \in E$ 
        using extra-edges .
    qed
  qed

```

```

    show  $\bigwedge a b. (a, b) \in \text{set extra} \implies \text{reachable } s a$ 
      using extra-reaches .
  qed
  then show ?thesis
    by simp
  qed
  have core1: sound-label (relax-edges d ?es)
    using core1-path es-path by simp
  have core2: relax-edges d ?es v = dist s v
    using core2-path es-path by simp
  show sound-label (relax-frontier d F)
    using core1 unfolding relax-frontier-def .
  show relax-frontier d F v = dist s v
    using core2 unfolding relax-frontier-def .
  qed

end

end
theory BMSSP-Find-Pivots
  imports BMSSP-Find-Pivots-Core BMSSP-Algorithm-Correctness
begin

```

20 Concrete FindPivots Rounds

FindPivots is the first nontrivial subroutine in a BMSSP recursive step. Conceptually, it starts from a current source frontier, performs a bounded number of relaxation rounds, records the vertices reached by those rounds, and chooses a smaller set of pivots whose shortest-path trees are large enough to justify the next recursive partitioning phase.

This theory gives the concrete round-by-round model used by the later correctness and cost proofs. The round state consists of a label function, a frontier, and a seen set. One round relaxes edges out of the frontier, selects the next frontier under the current upper bound, and extends the seen set. The capped variant stops early once the seen set exceeds a cardinality cap. That cap is the formal hook for the paper’s parameter schedule: if the local search grows too quickly, the original source set is already large enough to serve as the next pivot set.

The proof obligations in this file fall into three groups. First, the frontier and seen sets stay inside the finite vertex set and have the expected cardinality bounds under an outdegree assumption. Second, relaxation preserves the BMSSP source precondition and label soundness. Third, if the capped search does not overflow, every short tight path starting from a complete source is completed, and long tight trees expose a pivot. These facts are assembled into the concrete FindPivots postcondition consumed by the abstract algorithm-correctness theory.

context *unique-shortest-digraph*
begin

definition *fp-next* **where**

fp-next $d\ F\ B =$
 $\{v \in V. \exists u \in F. (u, v) \in E \wedge d\ u + w\ u\ v \leq d\ v \wedge$
 $\text{below-bound } (d\ u + w\ u\ v)\ B\}$

fun *fp-iter* **where**

fp-iter $0\ d\ F\ W\ B = (d, F, W)$
 $| \text{fp-iter } (Suc\ n)\ d\ F\ W\ B =$
 $(\text{let } d' = \text{relax-frontier } d\ F;$
 $F' = \text{fp-next } d\ F\ B;$
 $W' = W \cup F'$
 $\text{in fp-iter } n\ d'\ F'\ W'\ B)$

fun *fp-iter-capped* **where**

fp-iter-capped $0\ \text{cap}\ d\ F\ W\ B = (d, F, W)$
 $| \text{fp-iter-capped } (Suc\ n)\ \text{cap}\ d\ F\ W\ B =$
 $(\text{let } d' = \text{relax-frontier } d\ F;$
 $F' = \text{fp-next } d\ F\ B;$
 $W' = W \cup F'$
 $\text{in if card } W' > \text{cap then } (d', F', W')$
 $\text{else fp-iter-capped } n\ \text{cap}\ d'\ F'\ W'\ B)$

fun *fp-iter-capped-scan-cost* **where**

fp-iter-capped-scan-cost $0\ \text{cap}\ d\ F\ W\ B = 0$
 $| \text{fp-iter-capped-scan-cost } (Suc\ n)\ \text{cap}\ d\ F\ W\ B =$
 $(\text{let } d' = \text{relax-frontier } d\ F;$
 $F' = \text{fp-next } d\ F\ B;$
 $W' = W \cup F';$
 $\text{round} = \text{card } (\text{outgoing-edges } F)$
 $\text{in round} + (\text{if card } W' > \text{cap then } 0$
 $\text{else fp-iter-capped-scan-cost } n\ \text{cap}\ d'\ F'\ W'\ B))$

definition *fp-label* **where**

fp-label $st = \text{fst } st$

definition *fp-frontier* **where**

fp-frontier $st = \text{fst } (\text{snd } st)$

definition *fp-seen* **where**

fp-seen $st = \text{snd } (\text{snd } st)$

definition *find-pivots-label* **where**

find-pivots-label $k\ d\ S\ B = \text{fp-label } (\text{fp-iter } k\ d\ S\ S\ B)$

definition *find-pivots-seen* **where**

find-pivots-seen $k\ d\ S\ B = \text{fp-seen } (\text{fp-iter } k\ d\ S\ S\ B)$

definition *find-pivots-seen-capped* **where**

find-pivots-seen-capped $k \text{ cap } d \ S \ B = \text{fp-seen } (\text{fp-iter-capped } k \text{ cap } d \ S \ S \ B)$

definition *find-pivots-label-capped* **where**

find-pivots-label-capped $k \text{ cap } d \ S \ B = \text{fp-label } (\text{fp-iter-capped } k \text{ cap } d \ S \ S \ B)$

definition *find-pivots-pivots-capped* **where**

find-pivots-pivots-capped $k \text{ cap } d \ S \ B =$
 (if $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \ S \ B) > \text{cap}$ then S
 else $\{u \in S. k \leq \text{card } (\text{tree-of } u \cap \text{find-pivots-seen-capped } k \text{ cap } d \ S \ B)\}$)

definition *find-pivots-pivots* **where**

find-pivots-pivots $k \ d \ S \ B =$
 $\{u \in S. k \leq \text{card } (\text{tree-of } u \cap \text{find-pivots-seen } k \ d \ S \ B)\}$

definition *short-tight-witness* **where**

short-tight-witness $k \ d \ S \ B \ x \longleftrightarrow$
 $(\exists p. p \neq [] \wedge \text{hd } p \in S \wedge \text{last } p = x \wedge$
 $d \ (\text{hd } p) = \text{dist } s \ (\text{hd } p) \wedge \text{successively tight-edge-step } p \wedge$
 $(\forall y \in \text{set } p. \text{below-bound } (\text{dist } s \ y) \ B) \wedge$
 $\text{length } (\text{path-edges } p) \leq k)$

definition *find-pivots-short-vertices* **where**

find-pivots-short-vertices $k \ d \ S \ B =$
 $\{x \in V. \text{short-tight-witness } k \ d \ S \ B \ x\}$

definition *out-neighbors* **where**

out-neighbors $F = \{v \in V. \exists u \in F. (u, v) \in E\}$

definition *outdegree-le* **where**

outdegree-le $\Delta \longleftrightarrow (\forall u \in V. \text{card } (\text{out-neighbors } \{u\}) \leq \Delta)$

definition *edge-outdegree-le* **where**

edge-outdegree-le $\Delta \longleftrightarrow (\forall u \in V. \text{card } (\text{outgoing-edges } \{u\}) \leq \Delta)$

The definitions above mirror the paper’s informal description. The set *fp-next* is the next bounded frontier: it contains vertices reached by a currently tight relaxation whose value remains below the call bound. The iterator *fp-iter* performs exactly k such rounds, while *fp-iter-capped* returns early when the accumulated seen set exceeds the cap. The projections *find-pivots-label-capped*, *find-pivots-seen-capped*, and *find-pivots-pivots-capped* are the concrete objects used by later BMSSP steps.

Two degree predicates are kept separate. *outdegree-le* bounds the number of distinct out-neighbours and is convenient for bounding the size of the seen set. *edge-outdegree-le* bounds outgoing edge records and is used for scan-cost accounting. The distinction matters in multigraph-style encodings where several edges may point to the same neighbour.

lemma *finite-tree-of* [*simp*]:
finite (*tree-of* *u*)
unfolding *tree-of-def* **using** *finite-V* **by** *auto*

lemma *fp-next-reaches*:
assumes *reaches*: $\bigwedge u. u \in F \implies \text{reachable } s \ u$
and *v*: $v \in \text{fp-next } d \ F \ B$
shows *reachable* *s v*
proof –
obtain *u* **where** $u \in F \ (u, v) \in E$
using *v* **unfolding** *fp-next-def* **by** *blast*
then show *?thesis*
using *reaches dist-triangle-edge* **by** *blast*
qed

lemma *fp-next-subset-V*:
 $\text{fp-next } d \ F \ B \subseteq V$
unfolding *fp-next-def* **by** *blast*

lemma *fp-next-subset-out-neighbors*:
 $\text{fp-next } d \ F \ B \subseteq \text{out-neighbors } F$
unfolding *fp-next-def out-neighbors-def* **by** *blast*

lemma *out-neighbors-subset-V*:
 $\text{out-neighbors } F \subseteq V$
unfolding *out-neighbors-def* **by** *blast*

lemma *finite-out-neighbors* [*simp*]:
finite (*out-neighbors* *F*)
using *finite-subset*[*OF out-neighbors-subset-V finite-V*] .

lemma *out-neighbors-UN-singleton*:
 $\text{out-neighbors } F = (\bigcup u \in F. \text{out-neighbors } \{u\})$
unfolding *out-neighbors-def* **by** *auto*

lemma *outgoing-edges-UN-singleton*:
 $\text{outgoing-edges } F = (\bigcup u \in F. \text{outgoing-edges } \{u\})$
unfolding *outgoing-edges-def* **by** *auto*

lemma *card-out-neighbors-le*:
assumes *F-subset*: $F \subseteq V$
and *degree*: *outdegree-le* Δ
shows $\text{card } (\text{out-neighbors } F) \leq \Delta * \text{card } F$
proof –
have *finite-F*: *finite* *F*
using *finite-subset*[*OF F-subset finite-V*] .
have *card-union*: $\text{card } (\text{out-neighbors } F) \leq (\sum u \in F. \text{card } (\text{out-neighbors } \{u\}))$
proof –
have $\text{card } (\text{out-neighbors } F) = \text{card } (\bigcup u \in F. \text{out-neighbors } \{u\})$

using *out-neighbors-UN-singleton*[*of F*] **by** *simp*
 also have $\dots \leq (\sum_{u \in F}. \text{card } (\text{out-neighbors } \{u\}))$
 using *card-UN-le*[*OF finite-F, of $\lambda u. \text{out-neighbors } \{u\}$*] .
 finally show *?thesis* .
 qed
 have *sum-bound*: $(\sum_{u \in F}. \text{card } (\text{out-neighbors } \{u\})) \leq (\sum_{u \in F}. \Delta)$
 proof (*rule sum-mono*)
 fix *u*
 assume $u \in F$
 then have $u \in V$
 using *F-subset* **by** *blast*
 then show $\text{card } (\text{out-neighbors } \{u\}) \leq \Delta$
 using *degree unfolding outdegree-le-def* **by** *blast*
 qed
 also have $\dots = \Delta * \text{card } F$
 by (*simp add: mult.commute*)
 finally have $(\sum_{u \in F}. \text{card } (\text{out-neighbors } \{u\})) \leq \Delta * \text{card } F$.
 then show *?thesis*
 using *card-union* **by** *linarith*
 qed

 lemma *card-fp-next-le*:
 assumes *F-subset*: $F \subseteq V$
 and *degree*: *outdegree-le* Δ
 shows $\text{card } (\text{fp-next } d \ F \ B) \leq \Delta * \text{card } F$
 proof –
 have $\text{card } (\text{fp-next } d \ F \ B) \leq \text{card } (\text{out-neighbors } F)$
 by (*rule card-mono*) (*simp-all add: fp-next-subset-out-neighbors*)
 also have $\dots \leq \Delta * \text{card } F$
 using *card-out-neighbors-le*[*OF F-subset degree*] .
 finally show *?thesis* .
 qed

 lemma *card-outgoing-edges-le*:
 assumes *F-subset*: $F \subseteq V$
 and *degree*: *edge-outdegree-le* Δ
 shows $\text{card } (\text{outgoing-edges } F) \leq \Delta * \text{card } F$
 proof –
 have *finite-F*: *finite* F
 using *finite-subset*[*OF F-subset finite-V*] .
 have *card-union*: $\text{card } (\text{outgoing-edges } F) \leq$
 $(\sum_{u \in F}. \text{card } (\text{outgoing-edges } \{u\}))$
 proof –
 have $\text{card } (\text{outgoing-edges } F) = \text{card } (\bigcup_{u \in F}. \text{outgoing-edges } \{u\})$
 using *outgoing-edges-UN-singleton*[*of F*] **by** *simp*
 also have $\dots \leq (\sum_{u \in F}. \text{card } (\text{outgoing-edges } \{u\}))$
 using *card-UN-le*[*OF finite-F, of $\lambda u. \text{outgoing-edges } \{u\}$*] .
 finally show *?thesis* .
 qed

```

have sum-bound: ( $\sum u \in F. \text{card } (\text{outgoing-edges } \{u\})$ )  $\leq$  ( $\sum u \in F. \Delta$ )
proof (rule sum-mono)
  fix u
  assume u  $\in$  F
  then have u  $\in$  V
    using F-subset by blast
  then show  $\text{card } (\text{outgoing-edges } \{u\}) \leq \Delta$ 
    using degree unfolding edge-outdegree-le-def by blast
qed
also have ... =  $\Delta * \text{card } F$ 
  by (simp add: mult.commute)
finally have ( $\sum u \in F. \text{card } (\text{outgoing-edges } \{u\})$ )  $\leq \Delta * \text{card } F$  .
then show ?thesis
  using card-union by linarith
qed

```

```

lemma card-seen-after-fp-step-le:
  assumes W-subset:  $W \subseteq V$ 
  and F-subset:  $F \subseteq V$ 
  and degree: outdegree-le  $\Delta$ 
  shows  $\text{card } (W \cup \text{fp-next } d \ F \ B) \leq \text{card } W + \Delta * \text{card } F$ 
proof -
  have  $\text{card } (W \cup \text{fp-next } d \ F \ B) \leq \text{card } W + \text{card } (\text{fp-next } d \ F \ B)$ 
    by (rule card-Un-le)
  also have ...  $\leq \text{card } W + \Delta * \text{card } F$ 
    using card-fp-next-le[OF F-subset degree] by simp
  finally show ?thesis .
qed

```

```

lemma card-seen-after-fp-step-cap-le:
  assumes F-subset-W:  $F \subseteq W$ 
  and W-subset:  $W \subseteq V$ 
  and degree: outdegree-le  $\Delta$ 
  and W-cap:  $\text{card } W \leq \text{cap}$ 
  shows  $\text{card } (W \cup \text{fp-next } d \ F \ B) \leq \text{Suc } \Delta * \text{cap}$ 
proof -
  have F-subset:  $F \subseteq V$ 
    using F-subset-W W-subset by blast
  have finite-W: finite W
    using finite-subset[OF W-subset finite-V] .
  have card-F-le-W:  $\text{card } F \leq \text{card } W$ 
    using card-mono[OF finite-W F-subset-W] .
  have card-F-le-cap:  $\text{card } F \leq \text{cap}$ 
    using card-F-le-W W-cap by linarith
  have  $\text{card } (W \cup \text{fp-next } d \ F \ B) \leq \text{card } W + \Delta * \text{card } F$ 
    using card-seen-after-fp-step-le[OF W-subset F-subset degree] .
  also have ...  $\leq \text{cap} + \Delta * \text{cap}$ 
    using W-cap card-F-le-cap by (intro add-mono mult-left-mono) simp-all
  also have ... =  $\text{Suc } \Delta * \text{cap}$ 

```

by *simp*
 finally show *?thesis* .
 qed

lemma *fp-iter-capped-seen-subset-V*:
 assumes *F-subset*: $F \subseteq V$
 and *W-subset*: $W \subseteq V$
 shows *fp-seen* (*fp-iter-capped* *n cap d F W B*) $\subseteq V$
 using *assms*
 proof (*induction n arbitrary: d F W*)
 case 0
 then show *?case*
 unfolding *fp-seen-def* by *simp*
 next
 case (*Suc n*)
 let *?F'* = *fp-next d F B*
 let *?W'* = $W \cup ?F'$
 have *F'-subset*: $?F' \subseteq V$
 using *fp-next-subset-V* .
 have *W'-subset*: $?W' \subseteq V$
 using *Suc.premis F'-subset* by *blast*
 show *?case*
 proof (*cases card ?W' > cap*)
 case *True*
 then show *?thesis*
 using *W'-subset* by (*simp add: fp-seen-def Let-def*)
 next
 case *False*
 have *fp-seen* (*fp-iter-capped n cap (relax-frontier d F) ?F' ?W' B*) $\subseteq V$
 using *Suc.IH[OF F'-subset W'-subset]* .
 then show *?thesis*
 using *False* by (*simp add: fp-seen-def Let-def*)
 qed
 qed

lemma *fp-iter-capped-frontier-subset-seen*:
 assumes $F \subseteq W$
 shows *fp-frontier* (*fp-iter-capped n cap d F W B*) $\subseteq$
 fp-seen (*fp-iter-capped n cap d F W B*)
 using *assms*
 proof (*induction n arbitrary: d F W*)
 case 0
 then show *?case*
 unfolding *fp-frontier-def fp-seen-def* by *simp*
 next
 case (*Suc n*)
 let *?F'* = *fp-next d F B*
 let *?W'* = $W \cup ?F'$
 have *F'-W'*: $?F' \subseteq ?W'$

```

    by blast
  show ?case
  proof (cases card ?W' > cap)
    case True
    then show ?thesis
      using F'-W' by (simp add: fp-frontier-def fp-seen-def Let-def)
  next
    case False
    show ?thesis
      using Suc.IH[OF F'-W'] False by (simp add: fp-frontier-def fp-seen-def
Let-def)
  qed
qed

```

```

lemma fp-iter-capped-seen-initial-subset:
  W  $\subseteq$  fp-seen (fp-iter-capped n cap d F W B)
proof (induction n arbitrary: d F W)
  case 0
  then show ?case
    unfolding fp-seen-def by simp
next
  case (Suc n)
  let ?d' = relax-frontier d F
  let ?F' = fp-next d F B
  let ?W' = W  $\cup$  ?F'
  show ?case
  proof (cases card ?W' > cap)
    case True
    then show ?thesis
      unfolding fp-seen-def by (simp add: Let-def)
  next
    case False
    have W  $\subseteq$  ?W'
    by blast
    also have ...  $\subseteq$  fp-seen (fp-iter-capped n cap ?d' ?F' ?W' B)
    by (rule Suc.IH)
    finally show ?thesis
      using False unfolding fp-seen-def by (simp add: Let-def)
  qed
qed

```

```

lemma fp-iter-capped-seen-reaches:
  assumes F-reaches:  $\bigwedge u. u \in F \implies \text{reachable } s \ u$ 
  and W-reaches:  $\bigwedge u. u \in W \implies \text{reachable } s \ u$ 
  shows  $\forall u \in \text{fp-seen } (\text{fp-iter-capped } n \ \text{cap } d \ F \ W \ B). \text{reachable } s \ u$ 
  using assms
proof (induction n arbitrary: d F W)
  case 0
  then show ?case

```

```

    unfolding fp-seen-def by simp
next
case (Suc n)
let ?F' = fp-next d F B
let ?W' = W  $\cup$  ?F'
have F'-reaches:  $\bigwedge u. u \in ?F' \implies \text{reachable } s \ u$ 
proof -
  fix u
  assume uF':  $u \in ?F'$ 
  then obtain x where xF:  $x \in F$  and edge:  $(x, u) \in E$ 
  unfolding fp-next-def by blast
  have reachable s x
  by (rule Suc.prem1[OF xF])
  then show reachable s u
  using dist-triangle-edge[OF edge] by blast
qed
have W'-reaches:  $\bigwedge u. u \in ?W' \implies \text{reachable } s \ u$ 
  using Suc.prem2 F'-reaches by blast
show ?case
proof (cases card ?W' > cap)
  case True
  then show ?thesis
  using W'-reaches by (simp add: fp-seen-def Let-def)
next
case False
have tail:
 $\forall u \in \text{fp-seen } (\text{fp-iter-capped } n \ \text{cap } (\text{relax-frontier } d \ F) \ ?F' \ ?W' \ B).$ 
 $\text{reachable } s \ u$ 
  by (rule Suc.IH
    [where d = relax-frontier d F and F = ?F' and W = ?W',
      OF F'-reaches W'-reaches])
  show ?thesis
  using tail False by (simp add: fp-seen-def Let-def)
qed
qed

lemma find-pivots-seen-capped-reaches:
  assumes S-reaches:  $\bigwedge u. u \in S \implies \text{reachable } s \ u$ 
  and u:  $u \in \text{find-pivots-seen-capped } k \ \text{cap } d \ S \ B$ 
  shows reachable s u
  using fp-iter-capped-seen-reaches
  [where F = S and W = S and n = k and cap = cap and d = d and B = B,
    OF S-reaches S-reaches]
  u
  unfolding find-pivots-seen-capped-def by blast

theorem fp-iter-capped-seen-card-le:
  assumes F-subset-W:  $F \subseteq W$ 
  and W-subset:  $W \subseteq V$ 

```

```

    and degree: outdegree-le  $\Delta$ 
    and W-cap: card  $W \leq \text{cap}$ 
    shows card (fp-seen (fp-iter-capped n cap d F W B))  $\leq \text{Suc } \Delta * \text{cap}$ 
    using assms
  proof (induction n arbitrary: d F W)
    case 0
    have card  $W \leq \text{Suc } \Delta * \text{cap}$ 
    using 0.premis(4) by simp
    then show ?case
    unfolding fp-seen-def by simp
  next
    case (Suc n)
    let ?d' = relax-frontier d F
    let ?F' = fp-next d F B
    let ?W' =  $W \cup ?F'$ 
    have F-subset:  $F \subseteq V$ 
    using Suc.premis(1,2) by blast
    have F'-subset:  $?F' \subseteq V$ 
    using fp-next-subset-V .
    have W'-subset:  $?W' \subseteq V$ 
    using Suc.premis(2) F'-subset by blast
    have F'-W':  $?F' \subseteq ?W'$ 
    by blast
    have step-cap: card  $?W' \leq \text{Suc } \Delta * \text{cap}$ 
    using card-seen-after-fp-step-cap-le
    [OF Suc.premis(1,2,3,4), of d B] .
    show ?case
    proof (cases card  $?W' > \text{cap}$ )
      case True
      then show ?thesis
      using step-cap by (simp add: fp-seen-def Let-def)
    next
      case False
      have W'-cap: card  $?W' \leq \text{cap}$ 
      using False by simp
      show ?thesis
      using Suc.IH[OF F'-W' W'-subset Suc.premis(3) W'-cap] False
      by (simp add: fp-seen-def Let-def)
    qed
  qed
qed

theorem find-pivots-seen-capped-card-le:
  assumes S-subset:  $S \subseteq V$ 
  and degree: outdegree-le  $\Delta$ 
  and S-cap: card  $S \leq \text{cap}$ 
  shows card (find-pivots-seen-capped k cap d S B)  $\leq \text{Suc } \Delta * \text{cap}$ 
  unfolding find-pivots-seen-capped-def
  by (rule fp-iter-capped-seen-card-le[OF subset-reft S-subset degree S-cap])

```

```

theorem fp-iter-capped-scan-cost-le:
  assumes F-subset-W:  $F \subseteq W$ 
    and W-subset:  $W \subseteq V$ 
    and degree: edge-outdegree-le  $\Delta$ 
    and W-cap:  $\text{card } W \leq \text{cap}$ 
  shows fp-iter-capped-scan-cost  $n \text{ cap } d \ F \ W \ B \leq n * \Delta * \text{cap}$ 
  using assms
proof (induction n arbitrary: d F W)
  case 0
  then show ?case
    by simp
next
  case (Suc n)
  let ?d' = relax-frontier  $d \ F$ 
  let ?F' = fp-next  $d \ F \ B$ 
  let ?W' =  $W \cup ?F'$ 
  have F-subset:  $F \subseteq V$ 
    using Suc.prems(1,2) by blast
  have finite-W: finite  $W$ 
    using finite-subset[OF Suc.prems(2) finite-V] .
  have card-F-le-W:  $\text{card } F \leq \text{card } W$ 
    using card-mono[OF finite-W Suc.prems(1)] .
  have card-F-le-cap:  $\text{card } F \leq \text{cap}$ 
    using card-F-le-W Suc.prems(4) by linarith
  have round-le:  $\text{card } (\text{outgoing-edges } F) \leq \Delta * \text{cap}$ 
  proof -
    have  $\text{card } (\text{outgoing-edges } F) \leq \Delta * \text{card } F$ 
      using card-outgoing-edges-le[OF F-subset Suc.prems(3)] .
    also have  $\dots \leq \Delta * \text{cap}$ 
      using card-F-le-cap by simp
    finally show ?thesis .
  qed
show ?case
proof (cases card ?W' > cap)
  case True
  have  $\text{card } (\text{outgoing-edges } F) \leq \text{Suc } n * \Delta * \text{cap}$ 
  proof -
    have  $\Delta * \text{cap} \leq \text{Suc } n * (\Delta * \text{cap})$ 
      by simp
    also have  $\dots = \text{Suc } n * \Delta * \text{cap}$ 
      by (simp add: algebra-simps)
    finally show ?thesis
      using round-le by linarith
  qed
  then show ?thesis
    using True by (simp add: Let-def)
next
  case False
  have  $F' - W'$ : ?F'  $\subseteq$  ?W'

```



```

    by blast
  have  $F'-subset: ?F' \subseteq V$ 
    using  $fp\_next\_subset-V$  .
  have  $W'-subset: ?W' \subseteq V$ 
    using  $Suc.prem(2) F'-subset$  by blast
  have  $W'-cap: card\ ?W' \leq cap$ 
    using  $False$  by simp
  have  $tail-le$ :
     $fp\_iter\_capped\_scan\_cost\ n\ cap\ ?d'\ ?F'\ ?W'\ B \leq n * \Delta * cap$ 
    using  $Suc.IH[OF\ F'-W'\ W'-subset\ Suc.prem(3)\ W'-cap]$  .
  have  $card\ (outgoing\_edges\ F) +$ 
     $fp\_iter\_capped\_scan\_cost\ n\ cap\ ?d'\ ?F'\ ?W'\ B$ 
     $\leq \Delta * cap + n * \Delta * cap$ 
    using  $round-le\ tail-le$  by simp
  also have  $\dots = Suc\ n * \Delta * cap$ 
    by ( $simp\ add: algebra-simps$ )
  finally show  $?thesis$ 
    using  $False$  by ( $simp\ add: Let-def$ )
qed
qed

theorem  $find\_pivots\_capped\_scan\_cost-le$ :
  assumes  $S-subset: S \subseteq V$ 
    and  $degree: edge-outdegree-le\ \Delta$ 
    and  $S-cap: card\ S \leq cap$ 
  shows  $fp\_iter\_capped\_scan\_cost\ k\ cap\ d\ S\ S\ B \leq k * \Delta * cap$ 
  by ( $rule\ fp\_iter\_capped\_scan\_cost-le[OF\ subset-refl\ S-subset\ degree\ S-cap]$ )

theorem  $fp\_iter\_capped\_scan\_cost-le-seen$ :
  assumes  $F-subset-W: F \subseteq W$ 
    and  $W-subset: W \subseteq V$ 
    and  $degree: edge-outdegree-le\ \Delta$ 
  shows  $fp\_iter\_capped\_scan\_cost\ n\ cap\ d\ F\ W\ B \leq$ 
     $n * \Delta * card\ (fp\_seen\ (fp\_iter\_capped\ n\ cap\ d\ F\ W\ B))$ 
  using  $F-subset-W\ W-subset$ 
proof ( $induction\ n\ arbitrary: d\ F\ W$ )
  case 0
  then show  $?case$ 
    unfolding  $fp\_iter\_capped\_scan\_cost.simps$  by simp
next
  case ( $Suc\ n$ )
  let  $?d' = relax-frontier\ d\ F$ 
  let  $?F' = fp\_next\ d\ F\ B$ 
  let  $?W' = W \cup ?F'$ 
  let  $?final = fp\_seen\ (fp\_iter\_capped\ (Suc\ n)\ cap\ d\ F\ W\ B)$ 
  have  $F-subset-V: F \subseteq V$ 
    using  $Suc.prem$  by blast
  have  $finite-final: finite\ ?final$ 
    using  $fp\_iter\_capped\_seen\_subset-V[OF\ F-subset-V\ Suc.prem(2)]\ finite-V$ 

```

```

    by (rule finite-subset)
  have W-final:  $W \subseteq ?final$ 
    by (rule fp-iter-capped-seen-initial-subset)
  have F-final:  $F \subseteq ?final$ 
    using Suc.premis(1) W-final by blast
  have card-F-le-final:  $\text{card } F \leq \text{card } ?final$ 
    by (rule card-mono[OF finite-final F-final])
  have round-le:
     $\text{card } (\text{outgoing-edges } F) \leq \Delta * \text{card } ?final$ 
  proof -
    have  $\text{card } (\text{outgoing-edges } F) \leq \Delta * \text{card } F$ 
      by (rule card-outgoing-edges-le[OF F-subset-V degree])
    also have  $\dots \leq \Delta * \text{card } ?final$ 
      using card-F-le-final by simp
    finally show ?thesis .
  qed
show ?case
proof (cases  $\text{card } ?W' > \text{cap}$ )
  case True
  have  $\text{card } (\text{outgoing-edges } F) \leq \text{Suc } n * \Delta * \text{card } ?final$ 
  proof -
    have  $\Delta * \text{card } ?final \leq \text{Suc } n * (\Delta * \text{card } ?final)$ 
      by simp
    then show ?thesis
      using round-le by linarith
  qed
  then show ?thesis
    using True by (simp add: Let-def)
next
  case False
  have F'-subset-W':  $?F' \subseteq ?W'$ 
    by blast
  have W'-subset:  $?W' \subseteq V$ 
    using Suc.premis fp-next-subset-V by blast
  have tail:
     $\text{fp-iter-capped-scan-cost } n \text{ cap } ?d' ?F' ?W' B \leq$ 
     $n * \Delta * \text{card } (\text{fp-seen } (\text{fp-iter-capped } n \text{ cap } ?d' ?F' ?W' B))$ 
    by (rule Suc.IH[OF F'-subset-W' W'-subset])
  have final-eq:
     $?final = \text{fp-seen } (\text{fp-iter-capped } n \text{ cap } ?d' ?F' ?W' B)$ 
    using False by (simp add: fp-seen-def Let-def)
  have  $\text{card } (\text{outgoing-edges } F) +$ 
     $\text{fp-iter-capped-scan-cost } n \text{ cap } ?d' ?F' ?W' B \leq$ 
     $\Delta * \text{card } ?final + n * \Delta * \text{card } ?final$ 
    using round-le tail final-eq by simp
  also have  $\dots = \text{Suc } n * \Delta * \text{card } ?final$ 
    by (simp add: algebra-simps)
  finally show ?thesis
    using False by (simp add: Let-def)

```

qed
qed

theorem *find-pivots-capped-scan-cost-le-seen*:
 assumes *S-subset*: $S \subseteq V$
 and *degree*: *edge-outdegree-le* Δ
 shows *fp-iter-capped-scan-cost* $k \text{ cap } d \ S \ S \ B \leq$
 $k * \Delta * \text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \ S \ B)$
 unfolding *find-pivots-seen-capped-def*
 by (rule *fp-iter-capped-scan-cost-le-seen*[*OF subset-refl S-subset degree*])

theorem *fp-iter-capped-scan-cost-le-outgoing-seen*:
 assumes *F-subset-W*: $F \subseteq W$
 shows *fp-iter-capped-scan-cost* $n \text{ cap } d \ F \ W \ B \leq$
 $n * \text{card } (\text{outgoing-edges } (\text{fp-seen } (\text{fp-iter-capped } n \text{ cap } d \ F \ W \ B)))$
 using *F-subset-W*
proof (*induction n arbitrary: d F W*)
 case 0
 then show ?case
 by simp
next
 case (*Suc n*)
 let ?*d'* = *relax-frontier* $d \ F$
 let ?*F'* = *fp-next* $d \ F \ B$
 let ?*W'* = $W \cup ?F'$
 let ?*final* = *fp-seen* (*fp-iter-capped* (*Suc n*) *cap d F W B*)
 have *W-final*: $W \subseteq ?final$
 by (rule *fp-iter-capped-seen-initial-subset*)
 have *F-final*: $F \subseteq ?final$
 using *Suc.prem*s *W-final* by blast
 have *outgoing-subset*: *outgoing-edges* $F \subseteq \text{outgoing-edges } ?final$
 using *F-final* unfolding *outgoing-edges-def* by blast
 have *round-le*:
 $\text{card } (\text{outgoing-edges } F) \leq \text{card } (\text{outgoing-edges } ?final)$
 by (rule *card-mono*[*OF finite-outgoing-edges outgoing-subset*])
 show ?case
proof (*cases card ?W' > cap*)
 case True
 have $\text{card } (\text{outgoing-edges } F) \leq$
 $\text{Suc } n * \text{card } (\text{outgoing-edges } ?final)$
proof –
 have $\text{card } (\text{outgoing-edges } ?final) \leq$
 $\text{Suc } n * \text{card } (\text{outgoing-edges } ?final)$
 by simp
 then show ?thesis
 using *round-le* by linarith
qed
 then show ?thesis
 using True by (simp add: *Let-def*)

```

next
  case False
  have tail:
    fp-iter-capped-scan-cost n cap ?d' ?F' ?W' B ≤
      n * card (outgoing-edges (fp-seen (fp-iter-capped n cap ?d' ?F' ?W' B)))
  by (rule Suc.IH) blast
  have final-eq:
    ?final = fp-seen (fp-iter-capped n cap ?d' ?F' ?W' B)
  using False by (simp add: fp-seen-def Let-def)
  have card (outgoing-edges F) +
    fp-iter-capped-scan-cost n cap ?d' ?F' ?W' B ≤
    card (outgoing-edges ?final) +
    n * card (outgoing-edges ?final)
  using round-le tail final-eq by simp
  also have ... = Suc n * card (outgoing-edges ?final)
  by (simp add: algebra-simps)
  finally show ?thesis
  using False by (simp add: Let-def)
qed
qed

```

theorem *find-pivots-capped-scan-cost-le-outgoing-seen*:
shows $fp\text{-}iter\text{-}capped\text{-}scan\text{-}cost\ k\ cap\ d\ S\ S\ B \leq$
 $k * card\ (outgoing\text{-}edges\ (find\text{-}pivots\text{-}seen\text{-}capped\ k\ cap\ d\ S\ B))$
unfolding *find-pivots-seen-capped-def*
by (rule *fp-iter-capped-scan-cost-le-outgoing-seen*[*OF subset-refl*])

The capped search scans only edges leaving frontiers that have already been absorbed into the final seen set. The theorem *fp-iter-capped-scan-cost* $?k\ ?cap\ ?d\ ?S\ ?S\ ?B \leq ?k * card\ (outgoing\text{-}edges\ (find\text{-}pivots\text{-}seen\text{-}capped\ ?k\ ?cap\ ?d\ ?S\ ?B))$ is the compact cost statement used later: *fp-iter-capped-scan-cost* is bounded by the number of rounds times the outgoing-edge volume of *find-pivots-seen-capped*. This is deliberately stated in terms of the final seen set rather than a sum over intermediate frontiers, because the top-level analysis controls the size of that set through the cap.

lemma *fp-iter-capped-eq-fp-iter-if-final-within-cap*:
assumes *final-cap*: $card\ (fp\text{-}seen\ (fp\text{-}iter\text{-}capped\ n\ cap\ d\ F\ W\ B)) \leq cap$
shows $fp\text{-}iter\text{-}capped\ n\ cap\ d\ F\ W\ B = fp\text{-}iter\ n\ d\ F\ W\ B$
using *final-cap*

proof (induction *n* arbitrary: $d\ F\ W$)

```

case 0
then show ?case
  by simp

```

```

next
case (Suc n)
let ?d' = relax-frontier d F
let ?F' = fp-next d F B
let ?W' = W ∪ ?F'
show ?case

```

```

proof (cases card ?W' > cap)
  case True
  then have fp-seen (fp-iter-capped (Suc n) cap d F W B) = ?W'
    by (simp add: fp-seen-def Let-def)
  then have False
    using Suc.premis True by simp
  then show ?thesis
    by blast
next
  case False
  have tail-cap:
    card (fp-seen (fp-iter-capped n cap ?d' ?F' ?W' B)) ≤ cap
    using Suc.premis False by (simp add: fp-seen-def Let-def)
  have fp-iter-capped n cap ?d' ?F' ?W' B = fp-iter n ?d' ?F' ?W' B
    using Suc.IH[OF tail-cap] .
  then show ?thesis
    using False by (simp add: Let-def)
qed
qed

lemma fp-iter-seen-subset-V:
  assumes F-subset:  $F \subseteq V$ 
  and W-subset:  $W \subseteq V$ 
  shows fp-seen (fp-iter n d F W B)  $\subseteq V$ 
  using assms
proof (induction n arbitrary: d F W)
  case 0
  then show ?case
    unfolding fp-seen-def by simp
next
  case (Suc n)
  let ?F' = fp-next d F B
  let ?W' =  $W \cup ?F'$ 
  have ?F'  $\subseteq V$ 
    using fp-next-subset-V .
  then have ?W'  $\subseteq V$ 
    using Suc.premis by blast
  then have fp-seen (fp-iter n (relax-frontier d F) ?F' ?W' B)  $\subseteq V$ 
    using Suc.IH[OF «?F'  $\subseteq V$ »] by blast
  then show ?case
    by (simp add: fp-seen-def Let-def)
qed

lemma find-pivots-seen-subset-V:
  assumes S  $\subseteq V$ 
  shows find-pivots-seen k d S B  $\subseteq V$ 
  unfolding find-pivots-seen-def
  using fp-iter-seen-subset-V[OF assms assms] .

```

lemma *finite-find-pivots-seen* [*simp*]:
 assumes $S \subseteq V$
 shows *finite* (*find-pivots-seen* k d S B)
 using *finite-subset*[*OF find-pivots-seen-subset-V* [*OF assms*] *finite-V*] .

lemma *fp-next-tight-successor*:
 assumes *sound*: *sound-label* d
 and *uF*: $u \in F$
 and *complete-u*: $d\ u = \text{dist}\ s\ u$
 and *tight*: *tight-edge-step* $u\ v$
 and *below-v*: *below-bound* ($\text{dist}\ s\ v$) B
 shows $v \in \text{fp-next}\ d\ F\ B$
proof –
 have *edge*: $(u, v) \in E$ and *reach-u*: *reachable* $s\ u$
 and *dist-v*: $\text{dist}\ s\ v = \text{dist}\ s\ u + w\ u\ v$
 using *tight* **unfolding** *tight-edge-step-def* **by** *auto*
 have $v \in V$: $v \in V$
 using *edge* *edge-in-V* **by** *blast*
 have *reach-v*: *reachable* $s\ v$
 using *dist-triangle-edge*[*OF edge reach-u*] **by** *blast*
 have $\text{dist}\ s\ v \leq d\ v$
 using *sound* $v \in V$ *reach-v* **unfolding** *sound-label-def* **by** *blast*
 then have $d\ u + w\ u\ v \leq d\ v$
 using *complete-u* *dist-v* **by** *simp*
 moreover have *below-bound* ($d\ u + w\ u\ v$) B
 using *complete-u* *dist-v* *below-v* **by** *simp*
 ultimately show *?thesis*
 using *uF* *edge* $v \in V$ **unfolding** *fp-next-def* **by** *blast*
qed

lemma *fp-step-tight-successor-complete*:
 assumes *sound*: *sound-label* d
 and *uF*: $u \in F$
 and *frontier-reaches*: $\bigwedge x. x \in F \implies \text{reachable}\ s\ x$
 and *complete-u*: $d\ u = \text{dist}\ s\ u$
 and *tight*: *tight-edge-step* $u\ v$
 and *below-v*: *below-bound* ($\text{dist}\ s\ v$) B
 shows *sound-label* (*relax-frontier* $d\ F$)
 and *relax-frontier* $d\ F\ v = \text{dist}\ s\ v$
 and $v \in \text{fp-next}\ d\ F\ B$
proof –
 show *sound-label* (*relax-frontier* $d\ F$)
 using *relax-frontier-tight-successor-complete*(1)
 [*OF sound uF frontier-reaches complete-u tight*] .
 show *relax-frontier* $d\ F\ v = \text{dist}\ s\ v$
 using *relax-frontier-tight-successor-complete*(2)
 [*OF sound uF frontier-reaches complete-u tight*] .
 show $v \in \text{fp-next}\ d\ F\ B$
 using *fp-next-tight-successor*[*OF sound uF complete-u tight below-v*] .

qed

lemma *fp-iter-seen-mono*:

assumes $x \in W$

shows $x \in \text{fp-seen } (\text{fp-iter } n \ d \ F \ W \ B)$

using *assms*

proof (*induction n arbitrary: d F W*)

case 0

then show *?case*

unfolding *fp-seen-def* **by** *simp*

next

case (*Suc n*)

let $?d' = \text{relax-frontier } d \ F$

let $?F' = \text{fp-next } d \ F \ B$

let $?W' = W \cup ?F'$

have $x \in ?W'$

using *Suc.prem*s **by** *blast*

then have $x \in \text{fp-seen } (\text{fp-iter } n \ ?d' \ ?F' \ ?W' \ B)$

using *Suc.IH* **by** *blast*

then show *?case*

by (*simp add: fp-seen-def Let-def*)

qed

lemma *relax-frontier-preserves-complete-sound*:

assumes *sound: sound-label d*

and *complete-x: d x = dist s x*

and *reaches: $\bigwedge u. u \in F \implies \text{reachable } s \ u$*

shows $\text{relax-frontier } d \ F \ x = \text{dist } s \ x$

proof –

let $?es = \text{edge-list-of } (\text{outgoing-edges } F)$

have *set-es: set ?es = outgoing-edges F*

using *edge-list-of-properties(1)[OF finite-outgoing-edges]* .

have *edges: $\bigwedge u \ v. (u, v) \in \text{set } ?es \implies (u, v) \in E$*

using *set-es unfolding outgoing-edges-def* **by** *blast*

have *reaches-es: $\bigwedge u \ v. (u, v) \in \text{set } ?es \implies \text{reachable } s \ u$*

using *set-es reaches unfolding outgoing-edges-def* **by** *blast*

show *?thesis*

unfolding *relax-frontier-def*

using *relax-edges-preserves-complete-sound[OF sound complete-x edges reaches-es]*

.

qed

lemma *relax-frontier-preserves-bmssp-pre-full*:

assumes *sound: sound-label d*

and *pre: bmssp-pre-full d S B*

and *reaches: $\bigwedge u. u \in F \implies \text{reachable } s \ u$*

shows *bmssp-pre-full (relax-frontier d F) S B*

proof –

have *S-subset: $S \subseteq V$*

```

    using pre unfolding bmssp-pre-full-def by blast
  have cover:
     $\bigwedge v. \llbracket v \in V; \text{reachable } s \ v; \text{below-bound } (\text{dist } s \ v) \ B;$ 
     $\text{relax-frontier } d \ F \ v \neq \text{dist } s \ v \rrbracket$ 
     $\implies \text{through-complete } (\text{relax-frontier } d \ F) \ S \ v$ 
  proof -
    fix v
    assume vV:  $v \in V$  and reach-v:  $\text{reachable } s \ v$ 
    and below-v:  $\text{below-bound } (\text{dist } s \ v) \ B$ 
    and incomplete-v:  $\text{relax-frontier } d \ F \ v \neq \text{dist } s \ v$ 
    have d-incomplete:  $d \ v \neq \text{dist } s \ v$ 
    proof
      assume complete-v:  $d \ v = \text{dist } s \ v$ 
      have relax-frontier  $d \ F \ v = \text{dist } s \ v$ 
        by (rule relax-frontier-preserves-complete-sound
            [OF sound complete-v reaches])
      then show False
        using incomplete-v by simp
    qed
    have through-d:  $\text{through-complete } d \ S \ v$ 
      using pre vV reach-v below-v d-incomplete
      unfolding bmssp-pre-full-def by blast
    then obtain u p where uS:  $u \in S$ 
      and u-complete:  $u \in \text{complete-vertices } d$ 
      and sp:  $\text{shortest-walk } s \ p \ v$ 
      and up:  $u \in \text{set } p$ 
      unfolding through-complete-def through-def by blast
    have uV:  $u \in V$  and reach-u:  $\text{reachable } s \ u$ 
      and du:  $d \ u = \text{dist } s \ u$ 
      using u-complete unfolding complete-vertices-def by auto
    have relax-u:  $\text{relax-frontier } d \ F \ u = \text{dist } s \ u$ 
      by (rule relax-frontier-preserves-complete-sound [OF sound du reaches])
    have  $u \in \text{complete-vertices } (\text{relax-frontier } d \ F)$ 
      using uV reach-u relax-u unfolding complete-vertices-def by blast
    then have through  $(S \cap \text{complete-vertices } (\text{relax-frontier } d \ F)) \ v$ 
      using uS sp up unfolding through-def by blast
    then show  $\text{through-complete } (\text{relax-frontier } d \ F) \ S \ v$ 
      unfolding through-complete-def .
  qed
  show ?thesis
    using S-subset cover unfolding bmssp-pre-full-def by blast
qed

lemma fp-iter-capped-preserves-bmssp-pre-full:
  assumes sound: sound-label d
  and pre: bmssp-pre-full d S B0
  and reaches:  $\bigwedge u. u \in F \implies \text{reachable } s \ u$ 
  shows bmssp-pre-full (fp-label (fp-iter-capped n cap d F W B)) S B0
  using assms

```



```

proof (induction n arbitrary: d F W)
  case 0
  then show ?case
    unfolding fp-label-def by simp
next
  case (Suc n)
  let ?d' = relax-frontier d F
  let ?F' = fp-next d F B
  let ?W' = W  $\cup$  ?F'
  have sound': sound-label ?d'
    using relax-frontier-sound[OF Suc.prem(1,3)] .
  have pre': bmssp-pre-full ?d' S B0
    by (rule relax-frontier-preserves-bmssp-pre-full
      [OF Suc.prem(1,2,3)])
  have reaches':  $\bigwedge u. u \in ?F' \implies \text{reachable } s \ u$ 
    using fp-next-reaches[OF Suc.prem(3)] by blast
  show ?case
  proof (cases card ?W' > cap)
    case True
    then show ?thesis
      using pre' by (simp add: fp-label-def Let-def)
  next
    case False
    show ?thesis
      using Suc.IH[OF sound' pre' reaches'] False
      by (simp add: fp-label-def Let-def)
  qed
qed

```

```

lemma find-pivots-label-capped-preserves-source-pre:
  assumes sound: sound-label d
    and pre: bmssp-pre-full d S B
    and S-reaches:  $\bigwedge u. u \in S \implies \text{reachable } s \ u$ 
  shows bmssp-pre-full (find-pivots-label-capped k cap d S B) S B
  unfolding find-pivots-label-capped-def
  by (rule fp-iter-capped-preserves-bmssp-pre-full
    [OF sound pre S-reaches])

```

```

lemma fp-iter-capped-label-le:
  fp-label (fp-iter-capped n cap d F W B) x  $\leq$  d x
proof (induction n arbitrary: d F W)
  case 0
  then show ?case
    unfolding fp-label-def by simp
next
  case (Suc n)
  let ?d' = relax-frontier d F
  let ?F' = fp-next d F B
  let ?W' = W  $\cup$  ?F'

```

```

show ?case
proof (cases card ?W' > cap)
  case True
  then show ?thesis
    using relax-frontier-le[of d F x]
    by (simp add: fp-label-def Let-def)
next
  case False
  have tail: fp-label (fp-iter-capped n cap ?d' ?F' ?W' B) x ≤ ?d' x
    by (rule Suc.IH)
  also have ... ≤ d x
    by (rule relax-frontier-le)
  finally show ?thesis
    using False by (simp add: fp-label-def Let-def)
qed
qed

lemma find-pivots-label-capped-le:
  find-pivots-label-capped k cap d S B x ≤ d x
  unfolding find-pivots-label-capped-def
  by (rule fp-iter-capped-label-le)

lemma fp-iter-capped-label-sound:
  assumes sound: sound-label d
  and reaches:  $\bigwedge u. u \in F \implies \text{reachable } s \ u$ 
  shows sound-label (fp-label (fp-iter-capped n cap d F W B))
  using assms
proof (induction n arbitrary: d F W)
  case 0
  then show ?case
    unfolding fp-label-def by simp
next
  case (Suc n)
  let ?d' = relax-frontier d F
  let ?F' = fp-next d F B
  let ?W' = W  $\cup$  ?F'
  have sound': sound-label ?d'
    using relax-frontier-sound[OF Suc.prem] .
  have reaches':  $\bigwedge u. u \in ?F' \implies \text{reachable } s \ u$ 
    using fp-next-reaches[OF Suc.prem(2)] by blast
  show ?case
  proof (cases card ?W' > cap)
  case True
  then show ?thesis
    using sound' by (simp add: fp-label-def Let-def)
  next
  case False
  show ?thesis
    using Suc.IH[OF sound' reaches'] False by (simp add: fp-label-def Let-def)

```

qed
qed

lemma *fp-iter-capped-preserves-complete-sound*:
 assumes *sound*: *sound-label* *d*
 and *complete-x*: $d\ x = \text{dist}\ s\ x$
 and *reaches*: $\bigwedge u. u \in F \implies \text{reachable}\ s\ u$
 shows *fp-label* (*fp-iter-capped* *n cap d F W B*) $x = \text{dist}\ s\ x$
 using *assms*
proof (*induction n arbitrary: d F W*)
 case 0
 then show ?case
 unfolding *fp-label-def* by *simp*
next
 case (*Suc n*)
 let ?*d'* = *relax-frontier* *d F*
 let ?*F'* = *fp-next* *d F B*
 let ?*W'* = *W* $\cup$?*F'*
 have *sound'*: *sound-label* ?*d'*
 using *relax-frontier-sound* [*OF Suc.prems*(1,3)] .
 have *complete'*: ?*d'* $x = \text{dist}\ s\ x$
 using *relax-frontier-preserves-complete-sound* [*OF Suc.prems*] .
 have *reaches'*: $\bigwedge u. u \in ?F' \implies \text{reachable}\ s\ u$
 using *fp-next-reaches* [*OF Suc.prems*(3)] by *blast*
 show ?case
proof (*cases card ?W' > cap*)
 case *True*
 then show ?thesis
 using *complete'* by (*simp add: fp-label-def Let-def*)
next
 case *False*
 show ?thesis
 using *Suc.IH* [*OF sound' complete' reaches'*] *False*
 by (*simp add: fp-label-def Let-def*)
 qed
 qed

theorem *find-pivots-capped-overflow-establishes-pre*:
 assumes *sound*: *sound-label* *d*
 and *pre*: *bmssp-pre-full* *d S B*
 and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable}\ s\ x$
 and *overflow*: *card* (*find-pivots-seen-capped* *k cap d S B*) > *cap*
 shows *bmssp-pre-full*
 (*find-pivots-label-capped* *k cap d S B*)
 (*find-pivots-pivots-capped* *k cap d S B*) *B*
proof –
 let ?*d'* = *find-pivots-label-capped* *k cap d S B*
 have *pivots-eq*: *find-pivots-pivots-capped* *k cap d S B* = *S*
 using *overflow* unfolding *find-pivots-pivots-capped-def* by *simp*

```

have S-subset:  $S \subseteq V$ 
  using pre unfolding bmssp-pre-full-def by blast
have preserve-complete:
   $\bigwedge x. d\ x = \text{dist}\ s\ x \implies ?d'\ x = \text{dist}\ s\ x$ 
  unfolding find-pivots-label-capped-def
  by (rule fp-iter-capped-preserves-complete-sound[OF sound - S-reaches])
have cover:
   $\bigwedge v. \llbracket v \in V; \text{reachable}\ s\ v; \text{below-bound}\ (\text{dist}\ s\ v)\ B; \\ ?d'\ v \neq \text{dist}\ s\ v \rrbracket \implies \text{through-complete}\ ?d'\ S\ v$ 
proof -
  fix v
  assume vV:  $v \in V$ 
  and reach-v: reachable s v
  and below-v: below-bound (dist s v) B
  and d'-incomplete:  $?d'\ v \neq \text{dist}\ s\ v$ 
  have d-incomplete:  $d\ v \neq \text{dist}\ s\ v$ 
  proof
    assume  $d\ v = \text{dist}\ s\ v$ 
    then have  $?d'\ v = \text{dist}\ s\ v$ 
      by (rule preserve-complete)
    with d'-incomplete show False
      by simp
  qed
  have through-complete d S v
    using pre vV reach-v below-v d-incomplete unfolding bmssp-pre-full-def by
blast
  then obtain u p where uS:  $u \in S$  and ucomp:  $u \in \text{complete-vertices}\ d$ 
    and sp: shortest-walk s p v and u-path:  $u \in \text{set}\ p$ 
    unfolding through-complete-def through-def by blast
  have uV:  $u \in V$  and reach-u: reachable s u and complete-u:  $d\ u = \text{dist}\ s\ u$ 
    using ucomp unfolding complete-vertices-def by auto
  have complete-u':  $?d'\ u = \text{dist}\ s\ u$ 
    using preserve-complete[OF complete-u] .
  have  $u \in \text{complete-vertices}\ ?d'$ 
    using uV reach-u complete-u' unfolding complete-vertices-def by blast
  then have through ( $S \cap \text{complete-vertices}\ ?d'$ ) v
    using uS sp u-path unfolding through-def by blast
  then show through-complete  $?d'\ S\ v$ 
    unfolding through-complete-def .
qed
show ?thesis
  unfolding bmssp-pre-full-def
proof
  show find-pivots-pivots-capped k cap d S B  $\subseteq V$ 
    using pivots-eq S-subset by simp
next
  show  $\forall v \in V. \text{reachable}\ s\ v \longrightarrow \text{below-bound}\ (\text{dist}\ s\ v)\ B \longrightarrow \\ ?d'\ v \neq \text{dist}\ s\ v \longrightarrow \\ \text{through-complete}\ ?d'\ (\text{find-pivots-pivots-capped}\ k\ \text{cap}\ d\ S\ B)\ v$ 

```

```

proof (intro ballI impI)
  fix v
  assume vV:  $v \in V$ 
  and reach-v: reachable s v
  and below-v: below-bound (dist s v) B
  and incomplete-v:  $?d' v \neq \text{dist } s \ v$ 
  have through-complete  $?d' \ S \ v$ 
  using cover[OF vV reach-v below-v incomplete-v] .
  then show through-complete  $?d' \ (\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ v$ 
  using pivots-eq by simp
qed
qed
qed

lemma fp-iter-preserves-complete-sound:
  assumes sound: sound-label d
  and complete-x:  $d \ x = \text{dist } s \ x$ 
  and reaches:  $\bigwedge u. u \in F \implies \text{reachable } s \ u$ 
  shows fp-label (fp-iter n d F W B)  $x = \text{dist } s \ x$ 
  using sound complete-x reaches
proof (induction n arbitrary: d F W)
  case 0
  then show ?case
    unfolding fp-label-def by simp
next
  case (Suc n)
  let ?d' = relax-frontier d F
  let ?F' = fp-next d F B
  let ?W' =  $W \cup ?F'$ 
  have sound': sound-label ?d'
  using relax-frontier-sound[OF Suc.prem1(1,3)] .
  have complete':  $?d' \ x = \text{dist } s \ x$ 
  using relax-frontier-preserves-complete-sound[OF Suc.prem1] .
  have reaches':  $\bigwedge u. u \in ?F' \implies \text{reachable } s \ u$ 
  using fp-next-reaches[OF Suc.prem1(3)] by blast
  show ?case
    using Suc.IH[OF sound' complete' reaches'] by (simp add: fp-label-def Let-def)
qed

lemma fp-iter-one-tight-step-completion:
  assumes sound: sound-label d
  and uF:  $u \in F$ 
  and frontier-reaches:  $\bigwedge x. x \in F \implies \text{reachable } s \ x$ 
  and complete-u:  $d \ u = \text{dist } s \ u$ 
  and tight: tight-edge-step u v
  and below-v: below-bound (dist s v) B
  shows fp-label (fp-iter (Suc n) d F W B)  $v = \text{dist } s \ v$ 
  and  $v \in \text{fp-seen } (\text{fp-iter } (\text{Suc } n) \ d \ F \ W \ B)$ 
proof –

```

```

let ?d' = relax-frontier d F
let ?F' = fp-next d F B
let ?W' = W  $\cup$  ?F'
have sound': sound-label ?d'
  using fp-step-tight-successor-complete(1)
  [OF sound uF frontier-reaches complete-u tight below-v] .
have complete-v: ?d' v = dist s v
  using fp-step-tight-successor-complete(2)
  [OF sound uF frontier-reaches complete-u tight below-v] .
have vF': v  $\in$  ?F'
  using fp-step-tight-successor-complete(3)
  [OF sound uF frontier-reaches complete-u tight below-v] .
have reaches':  $\bigwedge x. x \in ?F' \implies \text{reachable } s x$ 
  using fp-next-reaches[OF frontier-reaches] by blast
have tail-complete:
  fp-label (fp-iter n ?d' ?F' ?W' B) v = dist s v
  by (rule fp-iter-preserves-complete-sound[OF sound' complete-v reaches'])
show fp-label (fp-iter (Suc n) d F W B) v = dist s v
  using tail-complete by (simp add: fp-label-def Let-def)
have v  $\in$  ?W'
  using vF' by blast
then have tail-seen: v  $\in$  fp-seen (fp-iter n ?d' ?F' ?W' B)
  using fp-iter-seen-mono[of v ?W' n ?d' ?F' B] by blast
then show v  $\in$  fp-seen (fp-iter (Suc n) d F W B)
  using tail-seen by (simp add: fp-seen-def Let-def)
qed

```

```

lemma fp-iter-tight-path-completion:
  assumes sound: sound-label d
    and frontier-reaches:  $\bigwedge x. x \in F \implies \text{reachable } s x$ 
    and F-seen:  $F \subseteq W$ 
    and nonempty:  $p \neq []$ 
    and start:  $\text{hd } p \in F$ 
    and complete-start:  $d (\text{hd } p) = \text{dist } s (\text{hd } p)$ 
    and tight: successively tight-edge-step p
    and below:  $\bigwedge x. x \in \text{set } p \implies \text{below-bound } (\text{dist } s x) B$ 
    and len:  $\text{length } (\text{path-edges } p) \leq n$ 
  shows fp-label (fp-iter n d F W B) (last p) = dist s (last p)  $\wedge$ 
    last p  $\in$  fp-seen (fp-iter n d F W B)
  using assms
proof (induction p arbitrary: n d F W rule: path-edges.induct)
  case 1
  then show ?case
    by simp
  next
  case (2 x)
  have sound0: sound-label d
    using 2.prem1 by blast
  have reaches0:  $\bigwedge z. z \in F \implies \text{reachable } s z$ 

```

```

    using 2.premis by blast
  have complete0:  $d\ x = \text{dist}\ s\ x$ 
    using 2.premis by simp
  have complete-x: fp-label (fp-iter n d F W B)  $x = \text{dist}\ s\ x$ 
    using fp-iter-preserves-complete-sound[OF sound0 complete0 reaches0] .
  have seen-x:  $x \in \text{fp-seen}\ (\text{fp-iter}\ n\ d\ F\ W\ B)$ 
proof -
  have  $x \in W$ 
    using 2.premis by auto
  then show ?thesis
    using fp-iter-seen-mono by blast
qed
show ?case
  using complete-x seen-x by simp
next
case ( $\exists\ x\ y\ xs$ )
show ?case
proof (cases n)
  case 0
  then show ?thesis
    using 3.premis(9) by simp
next
case (Suc m)
let ?d' = relax-frontier d F
let ?F' = fp-next d F B
let ?W' =  $W \cup ?F'$ 
have sound0: sound-label d
  using 3.premis by blast
have reaches0:  $\bigwedge z. z \in F \implies \text{reachable}\ s\ z$ 
  using 3.premis by blast
have xF:  $x \in F$ 
  using 3.premis by simp
have complete-x:  $d\ x = \text{dist}\ s\ x$ 
  using 3.premis by simp
have step: tight-edge-step x y
  using 3.premis by (simp add: successively-Cons)
have below-y: below-bound (dist s y) B
  using 3.premis by simp
have sound': sound-label ?d'
  using fp-step-tight-successor-complete(1)
    [OF sound0 xF reaches0 complete-x step below-y] .
have complete-y:  $?d'\ y = \text{dist}\ s\ y$ 
  using fp-step-tight-successor-complete(2)
    [OF sound0 xF reaches0 complete-x step below-y] .
have yF':  $y \in ?F'$ 
  using fp-step-tight-successor-complete(3)
    [OF sound0 xF reaches0 complete-x step below-y] .
have reaches':  $\bigwedge z. z \in ?F' \implies \text{reachable}\ s\ z$ 
  using fp-next-reaches[OF reaches0] by blast

```

```

have  $F\text{-seen}'$ :  $?F' \subseteq ?W'$ 
  by blast
have  $\text{tail-nonempty}$ :  $y \# xs \neq []$ 
  by simp
have  $\text{tail-start}$ :  $\text{hd } (y \# xs) \in ?F'$ 
  using  $yF'$  by simp
have  $\text{tail-complete-start}$ :  $?d' (\text{hd } (y \# xs)) = \text{dist } s (\text{hd } (y \# xs))$ 
  using  $\text{complete-}y$  by simp
have  $\text{tail-tight}$ :  $\text{successively tight-edge-step } (y \# xs)$ 
  using  $\mathcal{P}.\text{prems}$  by (simp add:  $\text{successively-Cons}$ )
have  $\text{tail-below}$ :  $\bigwedge z. z \in \text{set } (y \# xs) \implies \text{below-bound } (\text{dist } s \ z) \ B$ 
  using  $\mathcal{P}.\text{prems}$  by simp
have  $\text{tail-len}$ :  $\text{length } (\text{path-edges } (y \# xs)) \leq m$ 
  using  $\mathcal{P}.\text{prems}$   $\text{Suc}$  by simp
have  $\text{tail}$ :
   $\text{fp-label } (\text{fp-iter } m \ ?d' \ ?F' \ ?W' \ B) (\text{last } (y \# xs)) =$ 
     $\text{dist } s (\text{last } (y \# xs)) \wedge$ 
     $\text{last } (y \# xs) \in \text{fp-seen } (\text{fp-iter } m \ ?d' \ ?F' \ ?W' \ B)$ 
  using  $\mathcal{P}.\text{IH}[\text{of } ?d' \ ?F' \ ?W' \ m,$ 
     $\text{OF sound' reaches' } F\text{-seen' tail-nonempty tail-start tail-complete-start}$ 
     $\text{tail-tight tail-below tail-len}]$ 
  by simp
then show  $?thesis$ 
  using  $\text{Suc}$  by (simp add:  $\text{fp-label-def fp-seen-def Let-def}$ )
qed
qed

corollary  $\text{find-pivots-short-tight-path-complete}$ :
  assumes  $\text{sound}$ :  $\text{sound-label } d$ 
  and  $S\text{-reaches}$ :  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
  and  $\text{nonempty}$ :  $p \neq []$ 
  and  $\text{start}$ :  $\text{hd } p \in S$ 
  and  $\text{complete-start}$ :  $d (\text{hd } p) = \text{dist } s (\text{hd } p)$ 
  and  $\text{tight}$ :  $\text{successively tight-edge-step } p$ 
  and  $\text{below}$ :  $\bigwedge x. x \in \text{set } p \implies \text{below-bound } (\text{dist } s \ x) \ B$ 
  and  $\text{len}$ :  $\text{length } (\text{path-edges } p) \leq k$ 
  shows  $\text{find-pivots-label } k \ d \ S \ B (\text{last } p) = \text{dist } s (\text{last } p)$ 
  and  $\text{last } p \in \text{find-pivots-seen } k \ d \ S \ B$ 
proof -
  have  $\text{combined}$ :
     $\text{fp-label } (\text{fp-iter } k \ d \ S \ S \ B) (\text{last } p) = \text{dist } s (\text{last } p) \wedge$ 
     $\text{last } p \in \text{fp-seen } (\text{fp-iter } k \ d \ S \ S \ B)$ 
  using  $\text{fp-iter-tight-path-completion}$ 
     $[\text{OF sound } S\text{-reaches subset-refl nonempty start complete-start tight below len}]$  .
  show  $\text{find-pivots-label } k \ d \ S \ B (\text{last } p) = \text{dist } s (\text{last } p)$ 
    using  $\text{combined}$  unfolding  $\text{find-pivots-label-def}$  by blast
  show  $\text{last } p \in \text{find-pivots-seen } k \ d \ S \ B$ 
    using  $\text{combined}$  unfolding  $\text{find-pivots-seen-def}$  by blast
qed

```


lemma *find-pivots-short-paths-complete-on*:

assumes *sound*: *sound-label* *d*

and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$

and *witnesses*:

$\bigwedge x. x \in X \implies \exists p. p \neq [] \wedge \text{hd } p \in S \wedge \text{last } p = x \wedge$
 $d (\text{hd } p) = \text{dist } s (\text{hd } p) \wedge \text{successively tight-edge-step } p \wedge$
 $(\forall y \in \text{set } p. \text{below-bound } (\text{dist } s \ y) \ B) \wedge \text{length } (\text{path-edges } p) \leq k$

shows *complete-on* (*find-pivots-label* *k d S B*) *X* $\wedge$
 $X \subseteq \text{find-pivots-seen } k \ d \ S \ B$

proof –

have *complete*: *complete-on* (*find-pivots-label* *k d S B*) *X*

proof (*unfold complete-on-def*, *intro ballI impI*)

fix *x*

assume *xX*: $x \in X$

obtain *p* **where** *p*:

$p \neq [] \wedge \text{hd } p \in S \wedge \text{last } p = x$
 $d (\text{hd } p) = \text{dist } s (\text{hd } p)$
successively tight-edge-step *p*
 $\forall y \in \text{set } p. \text{below-bound } (\text{dist } s \ y) \ B$
 $\text{length } (\text{path-edges } p) \leq k$

using *witnesses*[*OF xX*] **by** *blast*

have *find-pivots-label* *k d S B* (*last p*) = *dist s* (*last p*)

using *find-pivots-short-tight-path-complete*(1)
 $[OF \ \text{sound } S\text{-reaches } p(1,2,4,5) - p(7)]$
p(6) **by** *blast*

then show *reachable s x* $\implies \text{find-pivots-label } k \ d \ S \ B \ x = \text{dist } s \ x$

using *p*(3) **by** *simp*

qed

have *seen*: $X \subseteq \text{find-pivots-seen } k \ d \ S \ B$

proof

fix *x*

assume *xX*: $x \in X$

obtain *p* **where** *p*:

$p \neq [] \wedge \text{hd } p \in S \wedge \text{last } p = x$
 $d (\text{hd } p) = \text{dist } s (\text{hd } p)$
successively tight-edge-step *p*
 $\forall y \in \text{set } p. \text{below-bound } (\text{dist } s \ y) \ B$
 $\text{length } (\text{path-edges } p) \leq k$

using *witnesses*[*OF xX*] **by** *blast*

have *last p* $\in \text{find-pivots-seen } k \ d \ S \ B$

using *find-pivots-short-tight-path-complete*(2)
 $[OF \ \text{sound } S\text{-reaches } p(1,2,4,5) - p(7)]$
p(6) **by** *blast*

then show $x \in \text{find-pivots-seen } k \ d \ S \ B$

using *p*(3) **by** *simp*

qed

show *?thesis*

using *complete seen* **by** *blast*

qed

The path-completion lemmas are the semantic core of FindPivots. A short tight witness is a shortest-path prefix, already correct at its start, whose edges remain below the current bound. The round induction proves that such a witness is followed by the relaxation process: $\llbracket \text{sound-label } ?d; \bigwedge x. x \in ?S \implies \text{reachable } s \ x; ?p \neq \square; \text{hd } ?p \in ?S; ?d (\text{hd } ?p) = \text{local.dist } s (\text{hd } ?p); \text{successively tight-edge-step } ?p; \bigwedge x. x \in \text{set } ?p \implies \text{below-bound } (\text{local.dist } s \ x) \ ?B; \text{length } (\text{path-edges } ?p) \leq ?k \rrbracket \implies \text{find-pivots-label } ?k \ ?d \ ?S \ ?B (\text{last } ?p) = \text{local.dist } s (\text{last } ?p)$

$\llbracket \text{sound-label } ?d; \bigwedge x. x \in ?S \implies \text{reachable } s \ x; ?p \neq \square; \text{hd } ?p \in ?S; ?d (\text{hd } ?p) = \text{local.dist } s (\text{hd } ?p); \text{successively tight-edge-step } ?p; \bigwedge x. x \in \text{set } ?p \implies \text{below-bound } (\text{local.dist } s \ x) \ ?B; \text{length } (\text{path-edges } ?p) \leq ?k \rrbracket \implies \text{last } ?p \in \text{find-pivots-seen } ?k \ ?d \ ?S \ ?B$ records both the resulting label equality and membership in the seen set. The set-level lemma above packages the same fact for any family of witnesses. The next theorem instantiates that package with *find-pivots-short-vertices*, the set defined exactly by *short-tight-witness*, yielding completeness via *complete-on*.

theorem *find-pivots-completes-short-vertices*:

assumes *sound*: *sound-label* *d*

and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$

shows *complete-on* (*find-pivots-label* *k d S B*)

(*find-pivots-short-vertices* *k d S B*) $\wedge$

find-pivots-short-vertices *k d S B* $\subseteq$ *find-pivots-seen* *k d S B*

proof (rule *find-pivots-short-paths-complete-on*[*OF sound S-reaches*])

fix *x*

assume $x \in \text{find-pivots-short-vertices } k \ d \ S \ B$

then show $\exists p. p \neq \square \wedge \text{hd } p \in S \wedge \text{last } p = x \wedge$

$d (\text{hd } p) = \text{dist } s (\text{hd } p) \wedge \text{successively tight-edge-step } p \wedge$

$(\forall y \in \text{set } p. \text{below-bound } (\text{dist } s \ y) \ B) \wedge \text{length } (\text{path-edges } p) \leq k$

unfolding *find-pivots-short-vertices-def short-tight-witness-def* **by** *blast*

qed

lemma *card-set-take-distinct*:

assumes *distinct* *xs* $k \leq \text{length } xs$

shows $\text{card } (\text{set } (\text{take } k \ xs)) = k$

using *assms* **by** (*simp add: distinct-card*)

lemma *successively-take*:

assumes *successively* *R xs*

shows *successively* *R (take n xs)*

unfolding *successively-conv-nth*

proof *clarify*

fix *i*

assume *i-take*: $\text{Suc } i < \text{length } (\text{take } n \ xs)$

have *i-xs*: $\text{Suc } i < \text{length } xs$

using *i-take* **by** *simp*

```

have si-n: Suc i < n
  using i-take by simp
have step: R (xs ! i) (xs ! Suc i)
  using assms i-xs unfolding successively-conv-nth by blast
have take n xs ! i = xs ! i
  using si-n by simp
moreover have take n xs ! Suc i = xs ! Suc i
  using si-n by simp
ultimately show R (take n xs ! i) (take n xs ! Suc i)
  using step by simp
qed

```

```

lemma successively-drop:
  assumes successively R xs
  shows successively R (drop n xs)
  unfolding successively-conv-nth
proof clarify
  fix i
  assume i-drop: Suc i < length (drop n xs)
  have i-xs: Suc (n + i) < length xs
    using i-drop by simp
  have step: R (xs ! (n + i)) (xs ! Suc (n + i))
    using assms i-xs unfolding successively-conv-nth by blast
  have drop n xs ! i = xs ! (n + i)
    using i-drop by simp
  moreover have drop n xs ! Suc i = xs ! Suc (n + i)
    using i-drop by simp
  ultimately show R (drop n xs ! i) (drop n xs ! Suc i)
    using step by simp
qed

```

```

lemma below-bound-le-trans:
  assumes a ≤ b below-bound b B
  shows below-bound a B
  using assms by (cases B) auto

```

```

lemma find-pivots-pivots-capped-label-below:
  assumes below:  $\bigwedge x. x \in S \implies \text{below-bound } (d\ x)\ B$ 
  and xP:  $x \in \text{find-pivots-pivots-capped } k\ \text{cap } d\ S\ B$ 
  shows below-bound (find-pivots-label-capped k cap d S B x) B
proof -
  have xS:  $x \in S$ 
    using xP unfolding find-pivots-pivots-capped-def
    by (auto split: if-splits)
  have le:  $\text{find-pivots-label-capped } k\ \text{cap } d\ S\ B\ x \leq d\ x$ 
    by (rule find-pivots-label-capped-le)
  show ?thesis
    by (rule below-bound-le-trans[OF le below[OF xS]])
qed

```

lemma *find-pivots-completes-short-tree-suffix*:

assumes *sound*: *sound-label* *d*

and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$

and *reach-v*: *reachable* *s v*

and *below-v*: *below-bound* (*dist* *s v*) *B*

and *i*: *i* < *length* (*shortest-path-to* *v*)

and *source-in-S*: *shortest-path-to* *v* ! *i* ∈ *S*

and *source-complete*: *d* (*shortest-path-to* *v* ! *i*) = *dist* *s* (*shortest-path-to* *v* ! *i*)

and *suffix-len*: *length* (*path-edges* (*drop* *i* (*shortest-path-to* *v*))) ≤ *k*

shows *find-pivots-label* *k d S B v* = *dist* *s v*

and *v* ∈ *find-pivots-seen* *k d S B*

proof –

let *?p* = *shortest-path-to* *v*

let *?q* = *drop* *i* *?p*

have *sp*: *shortest-walk* *s* *?p v*

using *shortest-path-to-shortest*[*OF reach-v*] .

have *tight-p*: *successively tight-edge-step* *?p*

using *shortest-path-to-successively-tight*[*OF reach-v*] .

have *q-nonempty*: *?q* ≠ []

using *i* **by** *auto*

have *hd-q*: *hd* *?q* = *?p* ! *i*

using *i* **by** (*simp add*: *hd-drop-conv-nth*)

have *last-q*: *last* *?q* = *v*

using *sp i* **unfolding** *shortest-walk-def simple-walk-betw-def walk-betw-def*

by (*simp add*: *last-drop*)

have *tight-q*: *successively tight-edge-step* *?q*

using *successively-drop*[*OF tight-p, of i*] .

have *below-q*: $\bigwedge x. x \in \text{set } ?q \implies \text{below-bound } (\text{dist } s \ x) \ B$

proof –

fix *x*

assume *xq*: *x* ∈ *set* *?q*

have *x* ∈ *set* *?p*

using *in-set-dropD*[*OF xq*] .

then have *tree-path* *x v*

using *reach-v* **unfolding** *tree-path-def* **by** *blast*

then have *dist* *s x* ≤ *dist* *s v*

by (*rule tree-path-dist-le*)

then show *below-bound* (*dist* *s x*) *B*

using *below-bound-le-trans*[*OF - below-v*] **by** *blast*

qed

have *hd-q-S*: *hd* *?q* ∈ *S*

using *hd-q source-in-S* **by** *simp*

have *hd-q-complete*: *d* (*hd* *?q*) = *dist* *s* (*hd* *?q*)

using *hd-q source-complete* **by** *simp*

have *label-last*: *find-pivots-label* *k d S B* (*last* *?q*) = *dist* *s* (*last* *?q*)

using *find-pivots-short-tight-path-complete*(1)

[*OF sound S-reaches q-nonempty hd-q-S hd-q-complete tight-q below-q suffix-len*]

.

have *seen-last*: $\text{last } ?q \in \text{find-pivots-seen } k \ d \ S \ B$
using *find-pivots-short-tight-path-complete*(2)
 $[OF \ \text{sound } S\text{-reaches } q\text{-nonempty } \text{hd-}q\text{-}S \ \text{hd-}q\text{-complete } \text{tight-}q \ \text{below-}q \ \text{suffix-len}]$
.

show *find-pivots-label* $k \ d \ S \ B \ v = \text{dist } s \ v$
using *label-last last-q* **by** *simp*
show $v \in \text{find-pivots-seen } k \ d \ S \ B$
using *seen-last last-q* **by** *simp*
qed

lemma *find-pivots-pivot-if-k-seen-tree-vertices*:
assumes *uS*: $u \in S$
and *distinct*: *distinct* xs
and *len*: $k \leq \text{length } xs$
and *in-tree*: $\text{set } (\text{take } k \ xs) \subseteq \text{tree-of } u$
and *seen*: $\text{set } (\text{take } k \ xs) \subseteq \text{find-pivots-seen } k \ d \ S \ B$
shows $u \in \text{find-pivots-pivots } k \ d \ S \ B$
proof –
let $?A = \text{set } (\text{take } k \ xs)$
have *card-A*: $\text{card } ?A = k$
using *card-set-take-distinct*[*OF distinct len*] .
have *A-subset*: $?A \subseteq \text{tree-of } u \cap \text{find-pivots-seen } k \ d \ S \ B$
using *in-tree seen* **by** *blast*
have $\text{card } ?A \leq \text{card } (\text{tree-of } u \cap \text{find-pivots-seen } k \ d \ S \ B)$
using *A-subset* **by** (*intro card-mono*) *simp-all*
then show *?thesis*
using *uS card-A unfolding find-pivots-pivots-def* **by** *simp*
qed

lemma *finite-find-pivots-pivots* [*simp*]:
assumes $S \subseteq V$
shows *finite* (*find-pivots-pivots* $k \ d \ S \ B$)
proof –
have *find-pivots-pivots* $k \ d \ S \ B \subseteq V$
using *assms unfolding find-pivots-pivots-def* **by** *blast*
then show *?thesis*
using *finite-subset*[*OF - finite-V*] **by** *blast*
qed

lemma *find-pivots-pivots-subset*:
 $\text{find-pivots-pivots } k \ d \ S \ B \subseteq S$
unfolding *find-pivots-pivots-def* **by** *blast*

theorem *find-pivots-pivots-card-times-le-seen*:
assumes *S-subset*: $S \subseteq V$
and *anti*: *tree-antichain* S
shows $k * \text{card } (\text{find-pivots-pivots } k \ d \ S \ B) \leq$
 $\text{card } (\text{find-pivots-seen } k \ d \ S \ B)$
proof –

```

let ?P = find-pivots-pivots k d S B
let ?W = find-pivots-seen k d S B
let ?A =  $\lambda u. \text{tree-of } u \cap ?W$ 
have finite-P: finite ?P
  using finite-find-pivots-pivots[OF S-subset] .
have finite-W: finite ?W
  using finite-find-pivots-seen[OF S-subset] .
have finite-A:  $\bigwedge u. u \in ?P \implies \text{finite } (?A \ u)$ 
  using finite-W by blast
have disjoint-A:
   $\bigwedge u \ v. \llbracket u \in ?P; v \in ?P; u \neq v \rrbracket \implies ?A \ u \cap ?A \ v = \{\}$ 
proof –
  fix u v
  assume uP:  $u \in ?P$  and vP:  $v \in ?P$  and neg:  $u \neq v$ 
  have uS:  $u \in S$  and vS:  $v \in S$ 
    using uP vP unfolding find-pivots-pivots-def by auto
  have tree-of  $u \cap \text{tree-of } v = \{\}$ 
    using tree-of-disjoint-if-antichain[OF anti uS vS neg] .
  then show  $?A \ u \cap ?A \ v = \{\}$ 
    by blast
qed
have finite-A-all:  $\forall u \in ?P. \text{finite } (?A \ u)$ 
  using finite-A by blast
have disjoint-A-all:  $\forall u \in ?P. \forall v \in ?P. u \neq v \longrightarrow ?A \ u \cap ?A \ v = \{\}$ 
  using disjoint-A by blast
have card-union-image:
   $\text{card } (\bigcup (?A \ ' \ ?P)) = (\sum u \in ?P. \text{card } (?A \ u))$ 
  by (rule card-UN-disjoint[OF finite-P finite-A-all disjoint-A-all])
have union-eq:  $(\bigcup u \in ?P. ?A \ u) = \bigcup (?A \ ' \ ?P)$ 
  by blast
have card-union:
   $\text{card } (\bigcup u \in ?P. ?A \ u) = (\sum u \in ?P. \text{card } (?A \ u))$ 
  using card-union-image union-eq by simp
have lower:  $(\sum u \in ?P. k) \leq (\sum u \in ?P. \text{card } (?A \ u))$ 
proof (rule sum-mono)
  fix u
  assume  $u \in ?P$ 
  then show  $k \leq \text{card } (?A \ u)$ 
    unfolding find-pivots-pivots-def by simp
qed
have const-sum:  $(\sum u \in ?P. k) = k * \text{card } ?P$ 
  by (simp add: mult.commute)
have  $k * \text{card } ?P \leq \text{card } (\bigcup u \in ?P. ?A \ u)$ 
proof –
  have  $k * \text{card } ?P \leq (\sum u \in ?P. \text{card } (?A \ u))$ 
    using lower const-sum by (simp add: mult.commute)
  also have  $\dots = \text{card } (\bigcup u \in ?P. ?A \ u)$ 
    using card-union by simp
  finally show ?thesis .

```

```

qed
also have ... ≤ card ?W
proof (rule card-mono[OF finite-W])
  show (⋃ u∈?P. ?A u) ⊆ ?W
  by blast
qed
finally show ?thesis .
qed

```

```

theorem find-pivots-pivots-capped-card-times-le-seen-or-overflow:
  assumes S-subset: S ⊆ V
  and anti: tree-antichain S
  shows k * card (find-pivots-pivots-capped k cap d S B) ≤
    card (find-pivots-seen-capped k cap d S B) ∨
    (find-pivots-pivots-capped k cap d S B = S ∧
     card (find-pivots-seen-capped k cap d S B) > cap)
proof (cases card (find-pivots-seen-capped k cap d S B) > cap)
  case True
  then show ?thesis
    unfolding find-pivots-pivots-capped-def by simp
next
  case False
  have cap-le: card (find-pivots-seen-capped k cap d S B) ≤ cap
    using False by simp
  have iter-eq: fp-iter-capped k cap d S S B = fp-iter k d S S B
    using fp-iter-capped-eq-fp-iter-if-final-within-cap[of k cap d S S B]
    cap-le unfolding find-pivots-seen-capped-def by simp
  have seen-eq: find-pivots-seen-capped k cap d S B = find-pivots-seen k d S B
    using iter-eq unfolding find-pivots-seen-capped-def find-pivots-seen-def by simp
  have pivots-eq:
    find-pivots-pivots-capped k cap d S B = find-pivots-pivots k d S B
    using False seen-eq
    unfolding find-pivots-pivots-capped-def find-pivots-pivots-def by simp
  have k * card (find-pivots-pivots k d S B) ≤
    card (find-pivots-seen k d S B)
    by (rule find-pivots-pivots-card-times-le-seen[OF S-subset anti])
  then show ?thesis
    unfolding seen-eq pivots-eq by blast
qed

```

```

lemma find-pivots-long-tight-path-root-pivot:
  assumes sound: sound-label d
  and S-reaches: ⋀x. x ∈ S ⇒ reachable s x
  and uS: u ∈ S
  and complete-u: d u = dist s u
  and p-nonempty: p ≠ []
  and hd-p: hd p = u
  and distinct-p: distinct p
  and len: k ≤ length p

```

```

and tight: successively tight-edge-step p
and below-prefix:
   $\bigwedge z. z \in \text{set } (\text{take } k \ p) \implies \text{below-bound } (\text{dist } s \ z) \ B$ 
and tree-prefix:  $\text{set } (\text{take } k \ p) \subseteq \text{tree-of } u$ 
shows  $u \in \text{find-pivots-pivots } k \ d \ S \ B$ 
proof (rule find-pivots-pivot-if-k-seen-tree-vertices[OF uS distinct-p len tree-prefix])
show  $\text{set } (\text{take } k \ p) \subseteq \text{find-pivots-seen } k \ d \ S \ B$ 
proof
  fix z
  assume z-take:  $z \in \text{set } (\text{take } k \ p)$ 
  obtain i where i:  $i < \text{length } (\text{take } k \ p) \ \text{take } k \ p \ ! \ i = z$ 
    using z-take by (auto simp: in-set-conv-nth)
  have len-take:  $\text{length } (\text{take } k \ p) = k$ 
    using len by simp
  have i-k:  $i < k$ 
    using i(1) len-take by simp
  have i-p:  $i < \text{length } p$ 
    using i-k len by linarith
  have p-i:  $p \ ! \ i = z$ 
    using i i-k by simp
  let ?q = take (Suc i) p
  have q-nonempty:  $?q \neq []$ 
    using i-p by (cases p) auto
  have hd-q:  $\text{hd } ?q = u$ 
    using hd-p p-nonempty by (cases p) auto
  have hd-q-S:  $\text{hd } ?q \in S$ 
    using hd-q uS by simp
  have complete-hd-q:  $d (\text{hd } ?q) = \text{dist } s (\text{hd } ?q)$ 
    using hd-q complete-u by simp
  have last-q:  $\text{last } ?q = z$ 
  proof –
    have len-q:  $\text{length } ?q = \text{Suc } i$ 
      using i-p by simp
    have last ?q =  $?q \ ! \ i$ 
      using len-q last-conv-nth[OF q-nonempty] by simp
    also have  $\dots = z$ 
      using p-i i-p by simp
    finally show ?thesis .
  qed
  have tight-q: successively tight-edge-step ?q
    using successively-take[OF tight, of Suc i] .
  have below-q:  $\bigwedge y. y \in \text{set } ?q \implies \text{below-bound } (\text{dist } s \ y) \ B$ 
  proof –
    fix y
    assume yq:  $y \in \text{set } ?q$ 
    have  $\text{set } ?q \subseteq \text{set } (\text{take } k \ p)$ 
      using i-k set-take-subset-set-take[of Suc i k p] by auto
    then show  $\text{below-bound } (\text{dist } s \ y) \ B$ 
      using below-prefix yq by blast

```



```

qed
have len-q: length (path-edges ?q) ≤ k
proof -
  have length (path-edges ?q) = length ?q - 1
    by (rule path-edges-length)
  also have ... = i
    using i-p by simp
  also have ... ≤ k
    using i-k by simp
  finally show ?thesis .
qed
have last ?q ∈ find-pivots-seen k d S B
  using find-pivots-short-tight-path-complete(2)
  [OF sound S-reaches q-nonempty hd-q-S complete-hd-q tight-q below-q len-q] .
then show z ∈ find-pivots-seen k d S B
  using last-q by simp
qed
qed

lemma find-pivots-long-tree-suffix-root-pivot:
  assumes sound: sound-label d
    and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
    and reach-v:  $\text{reachable } s \ v$ 
    and below-v:  $\text{below-bound } (dist \ s \ v) \ B$ 
    and i:  $i < \text{length } (\text{shortest-path-to } v)$ 
    and source-in-S:  $\text{shortest-path-to } v \ ! \ i \in S$ 
    and source-complete:  $d \ (\text{shortest-path-to } v \ ! \ i) = dist \ s \ (\text{shortest-path-to } v \ ! \ i)$ 
    and long:  $k \leq \text{length } (\text{drop } i \ (\text{shortest-path-to } v))$ 
    and tree-prefix:
      set (take k (drop i (shortest-path-to v))) ⊆
      tree-of (shortest-path-to v ! i)
  shows shortest-path-to v ! i ∈ find-pivots-pivots k d S B
proof -
  let ?p = shortest-path-to v
  let ?q = drop i ?p
  let ?u = ?p ! i
  have sp: shortest-walk s ?p v
    using shortest-path-to-shortest[OF reach-v] .
  have simple-p: simple-walk-betw s ?p v
    using sp unfolding shortest-walk-def by blast
  have distinct-p: distinct ?p
    using simple-p unfolding simple-walk-betw-def by blast
  have distinct-q: distinct ?q
    using distinct-p by simp
  have q-nonempty: ?q ≠ []
    using i by auto
  have hd-q: hd ?q = ?u
    using i by (simp add: hd-drop-conv-nth)
  have tight-q: successively tight-edge-step ?q

```

```

    using successively-drop[OF shortest-path-to-successively-tight[OF reach-v], of i] .
  have below-prefix:  $\bigwedge x. x \in \text{set } (\text{take } k \text{ } ?q) \implies \text{below-bound } (\text{dist } s \ x) \ B$ 
proof -
  fix x
  assume x-take:  $x \in \text{set } (\text{take } k \text{ } ?q)$ 
  have xq:  $x \in \text{set } ?q$ 
  using x-take by (meson in-set-takeD)
  have x  $\in \text{set } ?p$ 
  using in-set-dropD[OF xq] .
  then have tree-path x v
  using reach-v unfolding tree-path-def by blast
  then have  $\text{dist } s \ x \leq \text{dist } s \ v$ 
  by (rule tree-path-dist-le)
  then show below-bound  $(\text{dist } s \ x) \ B$ 
  using below-bound-le-trans[OF - below-v] by blast
qed
show ?thesis
using find-pivots-long-tight-path-root-pivot
[OF sound S-reaches source-in-S source-complete q-nonempty hd-q
distinct-q long tight-q below-prefix tree-prefix] .
qed

corollary find-pivots-one-tight-step-complete:
assumes sound: sound-label d
and uS:  $u \in S$ 
and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
and complete-u:  $d \ u = \text{dist } s \ u$ 
and tight: tight-edge-step u v
and below-v: below-bound  $(\text{dist } s \ v) \ B$ 
shows find-pivots-label  $(\text{Suc } n) \ d \ S \ B \ v = \text{dist } s \ v$ 
and  $v \in \text{find-pivots-seen } (\text{Suc } n) \ d \ S \ B$ 
using fp-iter-one-tight-step-completion
[OF sound uS S-reaches complete-u tight below-v, of n S]
unfolding find-pivots-label-def find-pivots-seen-def by auto

theorem find-pivots-covers-bound-tree-complete-sources:
assumes sound: sound-label d
and S-complete: complete-on d S
and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
and v-bound:  $v \in \text{bound-tree } S \ B$ 
shows
 $(\text{find-pivots-label } k \ d \ S \ B \ v = \text{dist } s \ v \wedge$ 
 $v \in \text{find-pivots-seen } k \ d \ S \ B) \vee$ 
 $\text{through-complete } (\text{find-pivots-label } k \ d \ S \ B)$ 
 $(\text{find-pivots-pivots } k \ d \ S \ B) \ v$ 
proof -
let ?p = shortest-path-to v
let ?d' = find-pivots-label k d S B
let ?P = find-pivots-pivots k d S B

```

```

have reach-v: reachable s v
  and through-S: through S v
  and below-v: below-bound (dist s v) B
  using v-bound unfolding bound-tree-def by auto
obtain u where uS:  $u \in S$  and tree-uv: tree-path u v
  using through-iff-tree-path[OF reach-v, of S] through-S by blast
then have u-in-path:  $u \in \text{set } ?p$ 
  by (auto dest: tree-pathD)
have sp-v: shortest-walk s ?p v
  using shortest-path-to-shortest[OF reach-v] .
have uV:  $u \in V$ 
proof -
  have walk ?p
    using sp-v unfolding shortest-walk-def simple-walk-betw-def walk-betw-def by
blast
  then show ?thesis
    using u-in-path walk-set-subset by blast
qed
obtain i where i:  $i < \text{length } ?p$  ?p ! i = u
  using u-in-path by (auto simp: in-set-conv-nth)
have complete-u:  $d\ u = \text{dist } s\ u$ 
  using S-complete S-reaches[OF uS] uS unfolding complete-on-def by blast
have source-in: ?p ! i  $\in S$ 
  using i(2) uS by simp
have source-complete:  $d\ (?p\ !\ i) = \text{dist } s\ (?p\ !\ i)$ 
  using i(2) complete-u by simp

show ?thesis
proof (cases length (path-edges (drop i ?p))  $\leq k$ )
case True
  have complete-v: ?d' v = dist s v
    by (rule find-pivots-completes-short-tree-suffix(1)
      [OF sound S-reaches reach-v below-v i(1) source-in source-complete True])
  have seen-v:  $v \in \text{find-pivots-seen } k\ d\ S\ B$ 
    by (rule find-pivots-completes-short-tree-suffix(2)
      [OF sound S-reaches reach-v below-v i(1) source-in source-complete True])
  then show ?thesis
    using complete-v by blast
next
case False
  let ?q = drop i ?p
  have q-nonempty: ?q  $\neq []$ 
    using i(1) by auto
  have long:  $k \leq \text{length } ?q$ 
  proof -
    have  $k < \text{length } (\text{path-edges } ?q)$ 
      using False by simp
    also have  $\dots = \text{length } ?q - 1$ 
      by (rule path-edges-length)
  end

```

```

    finally show ?thesis
      by linarith
qed
have source-tree-prefix:
  set (take k ?q)  $\subseteq$  tree-of (?p ! i)
  using tree-prefix-of-shortest-path-suffix[OF reach-v i(1), of k] .
have source-pivot: ?p ! i  $\in$  ?P
  by (rule find-pivots-long-tree-suffix-root-pivot
      [OF sound S-reaches reach-v below-v i(1) source-in source-complete long
        source-tree-prefix])
have u-pivot: u  $\in$  ?P
  using source-pivot i(2) by simp
have complete-u-after: ?d' u = dist s u
  unfolding find-pivots-label-def
  by (rule fp-iter-preserves-complete-sound[OF sound complete-u S-reaches])
have u-complete-vertex: u  $\in$  complete-vertices ?d'
  using uV S-reaches[OF uS] complete-u-after
  unfolding complete-vertices-def by blast
have through (?P  $\cap$  complete-vertices ?d') v
  using u-pivot u-complete-vertex sp-v u-in-path unfolding through-def by blast
then show ?thesis
  unfolding through-complete-def by blast
qed
qed

theorem find-pivots-post-complete-sources:
  assumes sound: sound-label d
    and S-subset:  $S \subseteq V$ 
    and S-complete: complete-on d S
    and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
  shows find-pivots-post d S B
    (find-pivots-label k d S B)
    (find-pivots-pivots k d S B)
    {v  $\in$  bound-tree S B. find-pivots-label k d S B v = dist s v}
  proof -
    let ?d' = find-pivots-label k d S B
    let ?P = find-pivots-pivots k d S B
    let ?W = {v  $\in$  bound-tree S B. ?d' v = dist s v}
    have P-subset: ?P  $\subseteq$  S
      unfolding find-pivots-pivots-def by blast
    have W-subset: ?W  $\subseteq$  bound-tree S B
      by blast
    have complete-W: complete-on ?d' ?W
      unfolding complete-on-def by blast
    have preserve:
       $\forall v \in V. \text{reachable } s \ v \implies \text{below-bound } (\text{dist } s \ v) \ B \implies$ 
       $d \ v = \text{dist } s \ v \implies ?d' \ v = \text{dist } s \ v$ 
    proof clarify
      fix v

```

```

assume v-complete:  $d\ v = \text{dist}\ s\ v$ 
show  $?d'\ v = \text{dist}\ s\ v$ 
  unfolding find-pivots-label-def
  by (rule fp-iter-preserves-complete-sound[OF sound v-complete S-reaches])
qed
have cover:  $\forall v \in \text{bound-tree}\ S\ B. v \in ?W \vee \text{through-complete}\ ?d'\ ?P\ v$ 
proof
  fix v
  assume v:  $v \in \text{bound-tree}\ S\ B$ 
  have cov:
    ( $?d'\ v = \text{dist}\ s\ v \wedge v \in \text{find-pivots-seen}\ k\ d\ S\ B$ )  $\vee$ 
     $\text{through-complete}\ ?d'\ ?P\ v$ 
  using find-pivots-covers-bound-tree-complete-sources
    [OF sound S-complete S-reaches v]
  .
  then show  $v \in ?W \vee \text{through-complete}\ ?d'\ ?P\ v$ 
    using v by blast
qed
show ?thesis
  using S-subset P-subset W-subset complete-W preserve cover
  unfolding find-pivots-post-def by blast
qed

theorem find-pivots-establishes-pivot-pre-concrete:
  assumes sound: sound-label d
  and pre: bmssp-pre-full d S B
  and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable}\ s\ x$ 
  shows bmssp-pre-full
    (find-pivots-label k d S B)
    (find-pivots-pivots k d S B) B
proof –
  let  $?d' = \text{find-pivots-label}\ k\ d\ S\ B$ 
  let  $?P = \text{find-pivots-pivots}\ k\ d\ S\ B$ 
  have P-subset-S:  $?P \subseteq S$ 
  unfolding find-pivots-pivots-def by blast
  have P-subset-V:  $?P \subseteq V$ 
  using P-subset-S pre unfolding bmssp-pre-full-def by blast
  have preserve-complete:  $\bigwedge x. d\ x = \text{dist}\ s\ x \implies ?d'\ x = \text{dist}\ s\ x$ 
  unfolding find-pivots-label-def
  by (rule fp-iter-preserves-complete-sound[OF sound - S-reaches])
  have cover:
     $\bigwedge v. \llbracket v \in V; \text{reachable}\ s\ v; \text{below-bound}\ (\text{dist}\ s\ v)\ B; \\ ?d'\ v \neq \text{dist}\ s\ v \rrbracket \implies \text{through-complete}\ ?d'\ ?P\ v$ 
proof –
  fix v
  assume vV:  $v \in V$ 
  and reach-v:  $\text{reachable}\ s\ v$ 
  and below-v:  $\text{below-bound}\ (\text{dist}\ s\ v)\ B$ 
  and d'-incomplete:  $?d'\ v \neq \text{dist}\ s\ v$ 

```

```

have d-incomplete:  $d\ v \neq \text{dist } s\ v$ 
proof
  assume  $d\ v = \text{dist } s\ v$ 
  then have  $?d'\ v = \text{dist } s\ v$ 
    by (rule preserve-complete)
  with d'-incomplete show False
    by simp
qed
have through-comp: through-complete  $d\ S\ v$ 
  using pre  $vV\ \text{reach-}v\ \text{below-}v\ d\text{-incomplete}$ 
  unfolding bmssp-pre-full-def by blast
obtain  $u\ p$  where  $uS$ :  $u \in S$  and ucomp:  $u \in \text{complete-vertices } d$ 
  and sp: shortest-walk  $s\ p\ v$  and u-in-p:  $u \in \text{set } p$ 
  using through-comp unfolding through-complete-def through-def by blast
let  $?p = \text{shortest-path-to } v$ 
have sp-canon: shortest-walk  $s\ ?p\ v$ 
  using shortest-path-to-shortest[OF reach-v] .
have p-eq:  $p = ?p$ 
  using unique-shortest-walk[OF sp sp-canon] .
have u-in-path:  $u \in \text{set } ?p$ 
  using u-in-p p-eq by simp
obtain  $i$  where  $i$ :  $i < \text{length } ?p$   $?p\ !\ i = u$ 
  using u-in-path by (auto simp: in-set-conv-nth)
have complete-u:  $d\ u = \text{dist } s\ u$ 
  using ucomp unfolding complete-vertices-def by blast
have source-in:  $?p\ !\ i \in S$ 
  using  $i(2)\ uS$  by simp
have source-complete:  $d\ (?p\ !\ i) = \text{dist } s\ (?p\ !\ i)$ 
  using  $i(2)\ \text{complete-u}$  by simp

show through-complete  $?d'\ ?P\ v$ 
proof (cases  $\text{length } (\text{path-edges } (\text{drop } i\ ?p)) \leq k$ )
  case True
    have  $?d'\ v = \text{dist } s\ v$ 
      by (rule find-pivots-completes-short-tree-suffix(1)
        [OF sound S-reaches reach-v below-v i(1) source-in source-complete True])
    with d'-incomplete have False
      by simp
    then show ?thesis
      by blast
  next
  case False
    let  $?q = \text{drop } i\ ?p$ 
    have  $\text{long}$ :  $k \leq \text{length } ?q$ 
    proof -
      have  $k < \text{length } (\text{path-edges } ?q)$ 
        using False by simp
      also have  $\dots = \text{length } ?q - 1$ 
        by (rule path-edges-length)
    qed

```

```

    finally show ?thesis
      by linarith
  qed
  have source-tree-prefix:
    set (take k ?q)  $\subseteq$  tree-of (?p ! i)
    using tree-prefix-of-shortest-path-suffix[OF reach-v i(1), of k] .
  have source-pivot: ?p ! i  $\in$  ?P
    by (rule find-pivots-long-tree-suffix-root-pivot
        [OF sound S-reaches reach-v below-v i(1) source-in source-complete long
         source-tree-prefix])
  have u-pivot: u  $\in$  ?P
    using source-pivot i(2) by simp
  have uV: u  $\in$  V and reach-u: reachable s u
    using ucomp unfolding complete-vertices-def by auto
  have complete-u-after: ?d' u = dist s u
    by (rule preserve-complete[OF complete-u])
  have u-complete-after: u  $\in$  complete-vertices ?d'
    using uV reach-u complete-u-after unfolding complete-vertices-def by blast
  have through (?P  $\cap$  complete-vertices ?d') v
    using u-pivot u-complete-after sp-canon u-in-path unfolding through-def by
blast
  then show ?thesis
    unfolding through-complete-def .
  qed
  show ?thesis
    using P-subset-V cover unfolding bmssp-pre-full-def by blast
  qed

theorem find-pivots-post-from-pre-concrete:
  assumes sound: sound-label d
    and pre: bmssp-pre-full d S B
    and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
  shows find-pivots-post d S B
    (find-pivots-label k d S B)
    (find-pivots-pivots k d S B)
    {v  $\in$  bound-tree S B. find-pivots-label k d S B v = dist s v}
  proof -
    let ?d' = find-pivots-label k d S B
    let ?P = find-pivots-pivots k d S B
    let ?W = {v  $\in$  bound-tree S B. ?d' v = dist s v}
    have S-subset: S  $\subseteq$  V
      using pre unfolding bmssp-pre-full-def by blast
    have P-subset: ?P  $\subseteq$  S
      unfolding find-pivots-pivots-def by blast
    have W-subset: ?W  $\subseteq$  bound-tree S B
      by blast
    have complete-W: complete-on ?d' ?W
      unfolding complete-on-def by blast

```

```

have preserve:
   $\forall v \in V. \text{reachable } s \ v \longrightarrow \text{below-bound } (\text{dist } s \ v) \ B \longrightarrow$ 
   $d \ v = \text{dist } s \ v \longrightarrow ?d' \ v = \text{dist } s \ v$ 
proof clarify
  fix v
  assume complete-v:  $d \ v = \text{dist } s \ v$ 
  show  $?d' \ v = \text{dist } s \ v$ 
    unfolding find-pivots-label-def
    by (rule fp-iter-preserves-complete-sound[OF sound complete-v S-reaches])
qed
have pivot-pre: bmssp-pre-full  $?d' \ ?P \ B$ 
  by (rule find-pivots-establishes-pivot-pre-concrete
    [OF sound pre S-reaches])
have cover:  $\forall v \in \text{bound-tree } S \ B. v \in ?W \vee \text{through-complete } ?d' \ ?P \ v$ 
proof
  fix v
  assume v-bound:  $v \in \text{bound-tree } S \ B$ 
  show  $v \in ?W \vee \text{through-complete } ?d' \ ?P \ v$ 
  proof (cases  $?d' \ v = \text{dist } s \ v$ )
    case True
    then show ?thesis
      using v-bound by blast
    next
    case False
    have vV:  $v \in V$  and reach-v: reachable  $s \ v$ 
      and below-v: below-bound  $(\text{dist } s \ v) \ B$ 
      using v-bound unfolding bound-tree-def by auto
    have through-complete  $?d' \ ?P \ v$ 
      using pivot-pre vV reach-v below-v False
      unfolding bmssp-pre-full-def by blast
    then show ?thesis
      by blast
  qed
qed
show ?thesis
  using S-subset P-subset W-subset complete-W preserve cover
  unfolding find-pivots-post-def by blast
qed

theorem find-pivots-capped-no-overflow-establishes-pre:
  assumes sound: sound-label d
  and pre: bmssp-pre-full  $d \ S \ B$ 
  and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
  and no-overflow:  $\neg \text{card } (\text{find-pivots-seen-capped } k \ \text{cap } d \ S \ B) > \text{cap}$ 
  shows bmssp-pre-full
    (find-pivots-label-capped  $k \ \text{cap } d \ S \ B$ )
    (find-pivots-pivots-capped  $k \ \text{cap } d \ S \ B$ ) B
  proof –
    have cap-le:  $\text{card } (\text{find-pivots-seen-capped } k \ \text{cap } d \ S \ B) \leq \text{cap}$ 

```



```

    using no-overflow by simp
  have iter-eq: fp-iter-capped k cap d S S B = fp-iter k d S S B
    using fp-iter-capped-eq-fp-iter-if-final-within-cap[of k cap d S S B]
    cap-le unfolding find-pivots-seen-capped-def by simp
  have label-eq: find-pivots-label-capped k cap d S B = find-pivots-label k d S B
    using iter-eq unfolding find-pivots-label-capped-def find-pivots-label-def by simp
  have seen-eq: find-pivots-seen-capped k cap d S B = find-pivots-seen k d S B
    using iter-eq unfolding find-pivots-seen-capped-def find-pivots-seen-def by simp
  have pivots-eq:
    find-pivots-pivots-capped k cap d S B = find-pivots-pivots k d S B
    using no-overflow seen-eq
    unfolding find-pivots-pivots-capped-def find-pivots-pivots-def by simp
  show ?thesis
    using find-pivots-establishes-pivot-pre-concrete[OF sound pre S-reaches]
    unfolding label-eq pivots-eq .
qed

theorem find-pivots-capped-establishes-pivot-pre-concrete:
  assumes sound: sound-label d
    and pre: bmssp-pre-full d S B
    and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
  shows bmssp-pre-full
    (find-pivots-label-capped k cap d S B)
    (find-pivots-pivots-capped k cap d S B) B
proof (cases card (find-pivots-seen-capped k cap d S B) > cap)
  case True
  show ?thesis
    by (rule find-pivots-capped-overflow-establishes-pre[OF sound pre S-reaches True])
  next
  case False
  show ?thesis
    by (rule find-pivots-capped-no-overflow-establishes-pre[OF sound pre S-reaches False])
qed

```

The capped variant has two proof branches but one external contract. If the cap overflows, the paper keeps the original source set as the pivot set; the existing BMSSP precondition is therefore preserved. If it does not overflow, $\text{card } (\text{fp-seen } (\text{fp-iter-capped } ?n \ ?cap \ ?d \ ?F \ ?W \ ?B)) \leq ?cap \implies \text{fp-iter-capped } ?n \ ?cap \ ?d \ ?F \ ?W \ ?B = \text{fp-iter } ?n \ ?d \ ?F \ ?W \ ?B$ identifies the capped execution with the uncapped one, so the pivot-precondition proof can reuse the short-path completeness argument. The theorem above hides that case split and exposes a single *bmssp-pre-full* fact for the capped labels and *find-pivots-pivots-capped*.

```

theorem find-pivots-capped-post-from-pre-concrete:
  assumes sound: sound-label d
    and pre: bmssp-pre-full d S B

```

and $S\text{-reaches}: \bigwedge x. x \in S \implies \text{reachable } s \ x$
shows $\text{find-pivots-post } d \ S \ B$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B)$
 $\{v \in \text{bound-tree } S \ B.$
 $\quad \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\}$
proof –
let $?d' = \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B$
let $?P = \text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B$
let $?W = \{v \in \text{bound-tree } S \ B. ?d' \ v = \text{dist } s \ v\}$
have $S\text{-subset}: S \subseteq V$
using $\text{pre unfolding } \text{bmssp-pre-full-def}$ **by** blast
have $P\text{-subset}: ?P \subseteq S$
unfolding $\text{find-pivots-pivots-capped-def}$ **by** auto
have $W\text{-subset}: ?W \subseteq \text{bound-tree } S \ B$
by blast
have $\text{complete-}W: \text{complete-on } ?d' \ ?W$
unfolding complete-on-def **by** blast
have $\text{preserve}:$
 $\forall v \in V. \text{reachable } s \ v \longrightarrow \text{below-bound } (\text{dist } s \ v) \ B \longrightarrow$
 $\quad d \ v = \text{dist } s \ v \longrightarrow ?d' \ v = \text{dist } s \ v$
proof clarify
fix v
assume $\text{complete-}v: d \ v = \text{dist } s \ v$
show $?d' \ v = \text{dist } s \ v$
unfolding $\text{find-pivots-label-capped-def}$
by $(\text{rule } \text{fp-iter-capped-preserves-complete-sound}$
 $\quad [\text{OF } \text{sound } \text{complete-}v \ S\text{-reaches}])$
qed
have $\text{pivot-pre}: \text{bmssp-pre-full } ?d' \ ?P \ B$
by $(\text{rule } \text{find-pivots-capped-establishes-pivot-pre-concrete}$
 $\quad [\text{OF } \text{sound } \text{pre } S\text{-reaches}])$
have $\text{cover}: \forall v \in \text{bound-tree } S \ B. v \in ?W \vee \text{through-complete } ?d' \ ?P \ v$
proof
fix v
assume $v\text{-bound}: v \in \text{bound-tree } S \ B$
show $v \in ?W \vee \text{through-complete } ?d' \ ?P \ v$
proof $(\text{cases } ?d' \ v = \text{dist } s \ v)$
case True
then show $?thesis$
using $v\text{-bound}$ **by** blast
next
case False
have $vV: v \in V$ **and** $\text{reach-}v: \text{reachable } s \ v$
and $\text{below-}v: \text{below-bound } (\text{dist } s \ v) \ B$
using $v\text{-bound}$ **unfolding** bound-tree-def **by** auto
have $\text{through-complete } ?d' \ ?P \ v$
using $\text{pivot-pre } vV \ \text{reach-}v \ \text{below-}v \ \text{False}$
unfolding $\text{bmssp-pre-full-def}$ **by** blast

```

    then show ?thesis
      by blast
  qed
qed
show ?thesis
  using S-subset P-subset W-subset complete-W preserve cover
  unfolding find-pivots-post-def by blast
qed

end

end
theory BMSSP-Pull-Minimum
  imports BMSSP-Find-Pivots
begin

```

21 Pull Minimum

This theory starts replacing the abstract partition-loop contract by concrete range-splitting lemmas. In the BMSSP loop, a partition Pull returns a boundary and a lower batch of candidate sources. The recursive child call is made below that boundary. To justify that call, we need to know that every incomplete vertex below the boundary is still covered by a complete source in the pulled lower part.

The rest of the theory is range algebra. It defines the half-open range tree between two distance boundaries, proves that a monotone boundary list partitions a bounded tree into disjoint child ranges, and assembles completeness of the lower prefix and every child range into the BMSSP postcondition. These lemmas are the semantic counterpart of the operational partition loop and are later used by the concrete one-step assembly theory.

```

context unique-shortest-digraph
begin

```

```

definition split-below where
  split-below d S beta = {x ∈ S. d x < beta}

```

```

lemma split-below-label-below:
  assumes x ∈ split-below d S beta
  shows below-bound (d x) (Fin beta)
  using assms unfolding split-below-def by simp

```

```

lemma split-below-tree-antichain:
  assumes anti: tree-antichain S
  shows tree-antichain (split-below d S beta)
  by (rule tree-antichain-subset[OF anti]) (simp add: split-below-def)

```

```

lemma split-below-scaled-card-le:

```

```

assumes  $S\text{-subset}: S \subseteq V$ 
and  $S\text{-}k\text{-cap}: k * \text{card } S \leq \text{cap}$ 
shows  $k * \text{card } (\text{split-below } d \ S \ \text{beta}) \leq \text{cap}$ 
proof –
  have  $\text{finite-}S: \text{finite } S$ 
  using  $\text{finite-subset}[OF \ S\text{-subset } \text{finite-}V]$  .
  have  $\text{card } (\text{split-below } d \ S \ \text{beta}) \leq \text{card } S$ 
  by  $(\text{rule } \text{card-mono}[OF \ \text{finite-}S]) (\text{simp add: } \text{split-below-def})$ 
  then have  $k * \text{card } (\text{split-below } d \ S \ \text{beta}) \leq k * \text{card } S$ 
  by  $\text{simp}$ 
  then show  $?thesis$ 
  using  $S\text{-}k\text{-cap}$  by  $\text{linarith}$ 
qed

```

The set *split-below* is the lower part produced by a Pull boundary: it keeps exactly those sources whose current labels are below *beta*. The first lemmas show that this split respects the finite bound, preserves the tree-antichain property, and cannot increase the scaled cardinality used by the FindPivots cap. These are small facts, but they are the side conditions needed before the lower recursive call can be formed.

definition *complete-tree-cover* **where**

$\text{complete-tree-cover } d \ S \ B \longleftrightarrow (\forall v \in \text{bound-tree } S \ B. \text{through-complete } d \ S \ v)$

definition *range-tree* **where**

$\text{range-tree } S \ a \ B =$

$\{v \in \text{tree-set } S. a \leq \text{dist } s \ v \wedge \text{below-bound } (\text{dist } s \ v) \ B\}$

fun *nondecreasing-from* **where**

$\text{nondecreasing-from } a \ [] \longleftrightarrow \text{True}$

$| \text{nondecreasing-from } a \ (b \# \text{bs}) \longleftrightarrow a \leq b \wedge \text{nondecreasing-from } b \ \text{bs}$

fun *bounds-le* **where**

$\text{bounds-le } B \ [] \longleftrightarrow \text{True}$

$| \text{bounds-le } B \ (b \# \text{bs}) \longleftrightarrow \text{bound-le } (\text{Fin } b) \ B \wedge \text{bounds-le } B \ \text{bs}$

fun *range-tree-chain* **where**

$\text{range-tree-chain } S \ a \ [] \ B = \text{range-tree } S \ a \ B$

$| \text{range-tree-chain } S \ a \ (b \# \text{bs}) \ B =$
 $\text{range-tree } S \ a \ (\text{Fin } b) \cup \text{range-tree-chain } S \ b \ \text{bs} \ B$

fun *range-tree-chain-list* **where**

$\text{range-tree-chain-list } S \ a \ [] \ B = [\text{range-tree } S \ a \ B]$

$| \text{range-tree-chain-list } S \ a \ (b \# \text{bs}) \ B =$
 $\text{range-tree } S \ a \ (\text{Fin } b) \# \text{range-tree-chain-list } S \ b \ \text{bs} \ B$

The predicates *complete-tree-cover* and *range-tree* describe the two main geometric facts used by the loop. A complete tree cover says every vertex in a bounded tree is certified by a complete source. A range tree is the part of a source tree whose true distances lie between a lower boundary and an upper

bound. The functions *range-tree-chain* and *range-tree-chain-list* extend this to a monotone list of child boundaries; the list form is convenient for relating one child postcondition to one child range.

lemma *below-bound-strict-trans*:
assumes $a < b$ *below-bound* b B
shows *below-bound* a B
using *assms* **by** (*cases* B) *auto*

lemma *complete-verticesD*:
assumes $u \in \text{complete-vertices } d$
shows $u \in V$ *reachable* s u d $u = \text{dist } s$ u
using *assms* **unfolding** *complete-vertices-def* **by** *auto*

lemma *bound-tree-complete-vertices-eq*:
assumes *cover*:
 $\bigwedge u. [\![u \in S; \text{reachable } s \ u; \text{below-bound } (\text{dist } s \ u) \ B; \\ d \ u \neq \text{dist } s \ u]\!] \implies \text{through-complete } d \ S \ u$
shows *bound-tree* $S \ B = \text{bound-tree } (S \cap \text{complete-vertices } d) \ B$
proof
show *bound-tree* $(S \cap \text{complete-vertices } d) \ B \subseteq \text{bound-tree } S \ B$
using *bound-tree-mono*[*of* $S \cap \text{complete-vertices } d \ S \ B$] **by** *blast*
next
show *bound-tree* $S \ B \subseteq \text{bound-tree } (S \cap \text{complete-vertices } d) \ B$
proof
fix v
assume $v\text{-bound}$: $v \in \text{bound-tree } S \ B$
then have vV : $v \in V$ **and** $\text{reach-}v$: *reachable* s v
and $\text{through-}S$: *through* S v **and** $\text{below-}v$: *below-bound* $(\text{dist } s \ v)$ B
unfolding *bound-tree-def* **by** *auto*
obtain u **where** uS : $u \in S$ **and** uv : *tree-path* u v
using *through-iff-tree-path*[*OF* $\text{reach-}v$, *of* S] *through-}S* **by** *blast*
have $\text{reach-}u$: *reachable* s u
using *tree-path-root-reachable*[*OF* uv] .
have $u\text{-below}$: *below-bound* $(\text{dist } s \ u)$ B
using *tree-path-dist-le*[*OF* uv] $\text{below-}v$
by (*cases* B) *auto*
have *through* $(S \cap \text{complete-vertices } d)$ v
proof (*cases* $d \ u = \text{dist } s \ u$)
case *True*
have $u\text{-complete}$: $u \in \text{complete-vertices } d$
using *tree-path-root-in-}V*[*OF* uv] $\text{reach-}u$ *True*
unfolding *complete-vertices-def* **by** *blast*
have $u\text{-path}$: $u \in \text{set } (\text{shortest-path-to } v)$
using uv **by** (*auto* $\text{dest: tree-path}D$)
have $sp\text{-}v$: *shortest-walk* s $(\text{shortest-path-to } v)$ v
using *shortest-path-to-shortest*[*OF* $\text{reach-}v$] .
show *?thesis*
using uS $u\text{-complete}$ $sp\text{-}v$ $u\text{-path}$ **unfolding** *through-def* **by** *blast*
next

```

case False
have through-complete d S u
  using cover[OF uS reach-u u-below False] .
then have through (S ∩ complete-vertices d) u
  unfolding through-complete-def .
then obtain q where qS: q ∈ S ∩ complete-vertices d
  and qu: tree-path q u
  using through-iff-tree-path[OF reach-u, of S ∩ complete-vertices d]
  by blast
have qv: tree-path q v
  using tree-path-trans[OF uv qu] .
have q-path: q ∈ set (shortest-path-to v)
  using qv by (auto dest: tree-pathD)
have sp-v: shortest-walk s (shortest-path-to v) v
  using shortest-path-to-shortest[OF reach-v] .
show ?thesis
  using qS sp-v q-path unfolding through-def by blast
qed
then show v ∈ bound-tree (S ∩ complete-vertices d) B
  using vV reach-v below-v unfolding bound-tree-def by blast
qed
qed

lemma bmssp-pre-full-bound-tree-complete-vertices-eq:
  assumes pre: bmssp-pre-full d S B
  shows bound-tree S B = bound-tree (S ∩ complete-vertices d) B
proof (rule bound-tree-complete-vertices-eq)
  fix u
  assume uS: u ∈ S and reach-u: reachable s u
  and below-u: below-bound (dist s u) B
  and incomplete-u: d u ≠ dist s u
  have u ∈ V
  using uS pre unfolding bmssp-pre-full-def by blast
  then show through-complete d S u
    using uS reach-u below-u incomplete-u pre unfolding bmssp-pre-full-def by
blast
qed

lemma bound-tree-complete-vertices-eq-imp-cover:
  assumes eq: bound-tree S B = bound-tree (S ∩ complete-vertices d) B
  shows complete-tree-cover d S B
proof (unfold complete-tree-cover-def, intro ballI)
  fix v
  assume v ∈ bound-tree S B
  then have v ∈ bound-tree (S ∩ complete-vertices d) B
  using eq by simp
  then have through (S ∩ complete-vertices d) v
  unfolding bound-tree-def by blast
  then show through-complete d S v

```

unfolding through-complete-def by simp
qed

lemma *bmssp-pre-full-complete-tree-cover*:
 assumes *pre*: *bmssp-pre-full* *d S B*
 shows *complete-tree-cover* *d S B*
 using *bound-tree-complete-vertices-eq-imp-cover*
 [OF *bmssp-pre-full-bound-tree-complete-vertices-eq* [OF *pre*]] .

lemma *bmssp-pre-full-bound-mono*:
 assumes *pre*: *bmssp-pre-full* *d S B*
 and *le*: *bound-le* *B' B*
 shows *bmssp-pre-full* *d S B'*
proof –
 have *S-subset*: $S \subseteq V$
 using *pre* unfolding *bmssp-pre-full-def* by blast
 have *cover*:
 $\bigwedge v. \llbracket v \in V; \text{reachable } s \ v; \text{below-bound } (dist \ s \ v) \ B';$
 $d \ v \neq dist \ s \ v \rrbracket \implies \text{through-complete } d \ S \ v$
proof –
 fix *v*
 assume *v*: $v \in V \text{ reachable } s \ v \text{ below-bound } (dist \ s \ v) \ B'$
 $d \ v \neq dist \ s \ v$
 have *below-bound* $(dist \ s \ v) \ B$
 using *le* *v*(3) by (cases *B'*; cases *B*) auto
 then show *through-complete* *d S v*
 using *pre* *v* unfolding *bmssp-pre-full-def* by blast
 qed
 show ?thesis
 using *S-subset* *cover* unfolding *bmssp-pre-full-def* by blast
 qed

theorem *concrete-find-pivots-complete-pivot-tree*:
 assumes *sound*: *sound-label* *d*
 and *pre*: *bmssp-pre-full* *d S B*
 and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
 shows *bound-tree* $(\text{find-pivots-pivots } k \ d \ S \ B) \ B =$
 bound-tree
 $(\text{find-pivots-pivots } k \ d \ S \ B \cap$
 $\text{complete-vertices } (\text{find-pivots-label } k \ d \ S \ B)) \ B$
 using *bmssp-pre-full-bound-tree-complete-vertices-eq*
 [OF *find-pivots-establishes-pivot-pre-concrete* [OF *sound pre S-reaches*]] .

theorem *concrete-find-pivots-pivot-tree-cover*:
 assumes *sound*: *sound-label* *d*
 and *pre*: *bmssp-pre-full* *d S B*
 and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
 shows *complete-tree-cover*
 $(\text{find-pivots-label } k \ d \ S \ B)$

$(\text{find-pivots-pivots } k \ d \ S \ B) \ B$
using *bmssp-pre-full-complete-tree-cover*
 $[OF \ \text{find-pivots-establishes-pivot-pre-concrete}[OF \ \text{sound} \ pre \ S\text{-reaches}]] \ .$

theorem *concrete-capped-find-pivots-complete-pivot-tree:*
assumes *sound: sound-label d*
and *pre: bmssp-pre-full d S B*
and *S-reaches: $\bigwedge x. x \in S \implies \text{reachable } s \ x$*
shows *bound-tree (find-pivots-pivots-capped k cap d S B) B =*
bound-tree
(find-pivots-pivots-capped k cap d S B $\cap$
complete-vertices (find-pivots-label-capped k cap d S B)) B
using *bmssp-pre-full-bound-tree-complete-vertices-eq*
 $[OF \ \text{find-pivots-capped-establishes-pivot-pre-concrete}[OF \ \text{sound} \ pre \ S\text{-reaches}]] \ .$

theorem *concrete-capped-find-pivots-pivot-tree-cover:*
assumes *sound: sound-label d*
and *pre: bmssp-pre-full d S B*
and *S-reaches: $\bigwedge x. x \in S \implies \text{reachable } s \ x$*
shows *complete-tree-cover*
(find-pivots-label-capped k cap d S B)
(find-pivots-pivots-capped k cap d S B) B
using *bmssp-pre-full-complete-tree-cover*
 $[OF \ \text{find-pivots-capped-establishes-pivot-pre-concrete}[OF \ \text{sound} \ pre \ S\text{-reaches}]] \ .$

lemma *through-complete-obtain:*
assumes *through-complete d S v*
obtains *u p where*
 $u \in S$
 $u \in \text{complete-vertices } d$
 $\text{shortest-walk } s \ p \ v$
 $u \in \text{set } p$
 $d \ u = \text{dist } s \ u$
 $\text{dist } s \ u \leq \text{dist } s \ v$
proof –
obtain *u p where* $u: u \in S \ u \in \text{complete-vertices } d \ \text{shortest-walk } s \ p \ v \ u \in \text{set } p$
using *assms unfolding through-complete-def through-def by blast*
have $d \ u = \text{dist } s \ u$
using *u(2) unfolding complete-vertices-def by blast*
moreover have $\text{dist } s \ u \leq \text{dist } s \ v$
using *shortest-walk-prefix-dist-le u by blast*
ultimately show *thesis*
using *that u by blast*
qed

lemma *pull-minimum-incomplete-covered:*
assumes *pre: bmssp-pre-full d S B*
and *beta: below-bound beta B*
and *vV: v $\in V$*

and *reach*: *reachable s v*
and *lt*: *dist s v < beta*
and *incomplete*: *d v ≠ dist s v*
shows *through-complete d (split-below d S beta) v*
proof –
have *below-v-B*: *below-bound (dist s v) B*
using *below-bound-strict-trans[OF lt beta]* .
have *cover*: *through-complete d S v*
using *pre vV reach below-v-B incomplete* **unfolding** *bmssp-pre-full-def* **by** *blast*
then obtain *u p* **where** *uS*: *u ∈ S* **and** *ucomp*: *u ∈ complete-vertices d*
and *p*: *shortest-walk s p v* **and** *up*: *u ∈ set p*
and *du*: *d u = dist s u* **and** *dist-le*: *dist s u ≤ dist s v*
by (*rule through-complete-obtain*)
have *d u < beta*
using *du dist-le lt* **by** *linarith*
then have *u ∈ split-below d S beta*
using *uS* **unfolding** *split-below-def* **by** *blast*
then have *through (split-below d S beta ∩ complete-vertices d) v*
using *ucomp p up* **unfolding** *through-def* **by** *blast*
then show *?thesis*
unfolding *through-complete-def* .
qed

lemma *pull-minimum-pre-for-lower-split*:
assumes *pre*: *bmssp-pre-full d S B*
and *beta*: *below-bound beta B*
shows *bmssp-pre-full d (split-below d S beta) (Fin beta)*
proof –
have *S-subset*: *split-below d S beta ⊆ V*
using *pre* **unfolding** *bmssp-pre-full-def split-below-def* **by** *blast*
have *cover*:
 $\bigwedge v. \llbracket v \in V; \text{reachable } s \ v; \text{below-bound } (dist \ s \ v) \ (Fin \ beta); \ d \ v \neq \text{dist } s \ v \rrbracket$
 $\implies \text{through-complete } d \ (\text{split-below } d \ S \ beta) \ v$
using *pull-minimum-incomplete-covered[OF pre beta]* **by** *auto*
show *?thesis*
using *S-subset cover* **unfolding** *bmssp-pre-full-def* **by** *blast*
qed

lemma *split-below-subset*:
split-below d S beta ⊆ S
unfolding *split-below-def* **by** *blast*

lemma *range-tree-eq-bound-diff*:
 $\text{range-tree } S \ a \ B = \text{bound-tree } S \ B - \text{bound-tree } S \ (Fin \ a)$
unfolding *range-tree-def bound-tree-eq-tree-set* **by** *auto*

lemma *range-tree-subset-bound-tree*:
 $\text{range-tree } S \ a \ B \subseteq \text{bound-tree } S \ B$
unfolding *range-tree-eq-bound-diff* **by** *blast*

```

lemma finite-bound-tree [simp]:
  finite (bound-tree S B)
proof –
  have bound-tree S B  $\subseteq$  V
    unfolding bound-tree-def by blast
  then show ?thesis
    using finite-subset[OF - finite-V] by blast
qed

lemma finite-range-tree [simp]:
  finite (range-tree S a B)
proof –
  have range-tree S a B  $\subseteq$  V
    unfolding range-tree-def tree-set-def by blast
  then show ?thesis
    using finite-subset[OF - finite-V] by blast
qed

lemma finite-range-tree-chain [simp]:
  finite (range-tree-chain S a bs B)
  by (induction bs arbitrary: a) simp-all

lemma finite-range-tree-chain-list-sets [simp]:
  assumes  $X \in \text{set } (\text{range-tree-chain-list } S \ a \ bs \ B)$ 
  shows finite X
  using assms by (induction bs arbitrary: a) auto

lemma Union-range-tree-chain-list:
   $\bigcup (\text{set } (\text{range-tree-chain-list } S \ a \ bs \ B)) = \text{range-tree-chain } S \ a \ bs \ B$ 
  by (induction bs arbitrary: a) auto

lemma range-tree-chain-lower-bound:
  assumes mono: nondecreasing-from a bs
  and  $x \in \text{range-tree-chain } S \ a \ bs \ B$ 
  shows  $a \leq \text{dist } s \ x$ 
  using assms
proof (induction bs arbitrary: a)
  case Nil
  then show ?case
    by (simp add: range-tree-def)
next
  case (Cons b bs)
  then consider  $x \in \text{range-tree } S \ a \ (\text{Fin } b) \mid$ 
     $x \in \text{range-tree-chain } S \ b \ bs \ B$ 
  by auto
  then show ?case
proof cases
  case 1

```

```

    then show ?thesis
      unfolding range-tree-def by blast
  next
    case 2
    have  $b \leq \text{dist } s \ x$ 
      using Cons.IH[of b] Cons.prem 2 by simp
    moreover have  $a \leq b$ 
      using Cons.prem by simp
    ultimately show ?thesis
      by linarith
  qed
qed

lemma range-tree-disjoint-lower:
  bound-tree S (Fin a)  $\cap$  range-tree S a B = {}
  unfolding range-tree-eq-bound-diff by blast

lemma bound-tree-disjoint-range-tree-chain:
  assumes mono: nondecreasing-from a bs
  shows bound-tree S (Fin a)  $\cap$  range-tree-chain S a bs B = {}
proof
  show bound-tree S (Fin a)  $\cap$  range-tree-chain S a bs B  $\subseteq$  {}
  proof
    fix x
    assume x:  $x \in \text{bound-tree } S \ (Fin \ a) \cap \text{range-tree-chain } S \ a \ bs \ B$ 
    then have  $\text{dist } s \ x < a$ 
      unfolding bound-tree-def by auto
    moreover have  $a \leq \text{dist } s \ x$ 
      using range-tree-chain-lower-bound[OF mono] x by blast
    ultimately show  $x \in \{\}$ 
      by simp
  qed
qed simp

lemma range-tree-disjoint-range-tree-chain-tail:
  assumes a-le-b:  $a \leq b$ 
  and mono-tail: nondecreasing-from b bs
  shows range-tree S a (Fin b)  $\cap$  range-tree-chain S b bs B = {}
proof
  show range-tree S a (Fin b)  $\cap$  range-tree-chain S b bs B  $\subseteq$  {}
  proof
    fix x
    assume x:  $x \in \text{range-tree } S \ a \ (Fin \ b) \cap \text{range-tree-chain } S \ b \ bs \ B$ 
    then have  $\text{dist } s \ x < b$ 
      unfolding range-tree-def by auto
    moreover have  $b \leq \text{dist } s \ x$ 
      using range-tree-chain-lower-bound[OF mono-tail] x by blast
    ultimately show  $x \in \{\}$ 
      by simp
  qed

```

```

qed
qed simp

lemma card-range-tree-chain-eq-sum-list:
  assumes mono: nondecreasing-from a bs
  shows card (range-tree-chain S a bs B) =
    sum-list (map card (range-tree-chain-list S a bs B))
  using mono
proof (induction bs arbitrary: a)
  case Nil
  then show ?case
    by simp
next
  case (Cons b bs)
  have a-le-b: a ≤ b
    using Cons.prem by simp
  have mono-tail: nondecreasing-from b bs
    using Cons.prem by simp
  have disjoint:
    range-tree S a (Fin b) ∩ range-tree-chain S b bs B = {}
    using range-tree-disjoint-range-tree-chain-tail[OF a-le-b mono-tail] .
  have card-union:
    card (range-tree S a (Fin b) ∪ range-tree-chain S b bs B) =
      card (range-tree S a (Fin b)) + card (range-tree-chain S b bs B)
    by (rule card-Un-disjoint) (simp-all add: disjoint)
  show ?case
    using card-union Cons.IH[OF mono-tail] by simp
qed

lemma sum-list-map-mult-left:
  fixes c :: nat
  and f :: 'a ⇒ nat
  shows sum-list (map (λx. c * f x) xs) = c * sum-list (map f xs)
  by (induction xs) (simp-all add: algebra-simps)

lemma sum-list-map-card-mult-left:
  fixes C :: nat
  and Xs :: 'v set list
  shows sum-list (map (λX. C * card X) Xs) = C * sum-list (map card Xs)
  by (induction Xs) (simp-all add: algebra-simps)

lemma sum-list-le-if-list-all2:
  assumes list-all2 (λc X. c ≤ C * card X) costs Xs
  shows sum-list costs ≤ sum-list (map (λX. C * card X) Xs)
  using assms
proof (induction rule: list-all2-induct)
  case Nil
  then show ?case by simp
next

```

case (*Cons* *c* *X* *costs* *Xs*)
then show ?*case* **by** *simp*
qed

lemma *child-costs-le-range-tree-chain-card*:
assumes *mono*: *nondecreasing-from* *a* *bs*
and *costs*: *list-all2* ($\lambda c\ X. c \leq C * \text{card } X$) *costs*
 (*range-tree-chain-list* *S* *a* *bs* *B*)
shows *sum-list* *costs* $\leq C * \text{card } (\text{range-tree-chain } S\ a\ bs\ B)$
proof –
have *sum-list* *costs* $\leq$
 sum-list (*map* ($\lambda X. C * \text{card } X$) (*range-tree-chain-list* *S* *a* *bs* *B*))
using *sum-list-le-if-list-all2*[*OF costs*] .
also have ... =
 $C * \text{sum-list } (\text{map } \text{card } (\text{range-tree-chain-list } S\ a\ bs\ B))$
by (*rule sum-list-map-card-mult-left*)
also have ... = $C * \text{card } (\text{range-tree-chain } S\ a\ bs\ B)$
using *card-range-tree-chain-eq-sum-list*[*OF mono, of S B*] **by** *simp*
finally show ?*thesis* .
qed

lemma *range-tree-adjacent-disjoint*:
 range-tree *S* *a* (*Fin* *b*) $\cap$ *range-tree* *S* *b* *B* = {}
unfolding *range-tree-def* **by** *auto*

lemma *range-tree-adjacent-union*:
assumes $a \leq b$
and *bound-le* (*Fin* *b*) *B*
shows *range-tree* *S* *a* (*Fin* *b*) $\cup$ *range-tree* *S* *b* *B* = *range-tree* *S* *a* *B*
using *assms* **unfolding** *range-tree-def* **by** (*cases* *B*) *auto*

lemma *bound-tree-split-at*:
assumes *bound-le* (*Fin* *a*) *B*
shows *bound-tree* *S* *B* = *bound-tree* *S* (*Fin* *a*) $\cup$ *range-tree* *S* *a* *B*
using *assms* **unfolding** *range-tree-def* *bound-tree-eq-tree-set*
by (*cases* *B*) *auto*

lemma *bound-tree-adjacent-union*:
assumes $a \leq b$
and *bound-le* (*Fin* *b*) *B*
shows *bound-tree* *S* (*Fin* *a*) $\cup$ *range-tree* *S* *a* (*Fin* *b*) $\cup$ *range-tree* *S* *b* *B* =
 bound-tree *S* *B*
proof –
have *union*: *range-tree* *S* *a* (*Fin* *b*) $\cup$ *range-tree* *S* *b* *B* = *range-tree* *S* *a* *B*
using *range-tree-adjacent-union*[*OF assms*] .
have *le-a-B*: *bound-le* (*Fin* *a*) *B*
using *assms* **by** (*cases* *B*) *auto*
show ?*thesis*
using *union* *bound-tree-split-at*[*OF le-a-B, of S*]

by (cases B) auto
qed

lemma complete-on-Un:
assumes complete-on d A complete-on d B
shows complete-on d (A $\cup$ B)
using assms unfolding complete-on-def by blast

lemma complete-on-Union-list:
assumes $\bigwedge X. X \in \text{set } Xs \implies \text{complete-on } d \ X$
shows complete-on d ($\bigcup (\text{set } Xs)$)
using assms unfolding complete-on-def by blast

lemma complete-on-bound-tree-split:
assumes bound-le (Fin a) B
and lower: complete-on d (bound-tree S (Fin a))
and upper: complete-on d (range-tree S a B)
shows complete-on d (bound-tree S B)
proof –
have complete-on d (bound-tree S (Fin a) $\cup$ range-tree S a B)
using complete-on-Un[OF lower upper] .
then show ?thesis
using bound-tree-split-at[OF assms(1), of S] by simp
qed

lemma bound-tree-range-chain-union:
assumes mono: nondecreasing-from a bs
and bounds: bounds-le B (a $\#$ bs)
shows bound-tree S (Fin a) $\cup$ range-tree-chain S a bs B = bound-tree S B
using assms
proof (induction bs arbitrary: a)
case Nil
then show ?case
using bound-tree-split-at[of a B S] by simp
next
case (Cons b bs)
have a-le-b: a $\leq$ b
using Cons.prem(1) by simp
have mono-tail: nondecreasing-from b bs
using Cons.prem(1) by simp
have b-le-B: bound-le (Fin b) B
using Cons.prem(2) by simp
have bounds-tail: bounds-le B (b $\#$ bs)
using Cons.prem(2) by simp
have split-ab:
bound-tree S (Fin b) = bound-tree S (Fin a) $\cup$ range-tree S a (Fin b)
using bound-tree-split-at[of a Fin b S] a-le-b by simp
have tail:
bound-tree S (Fin b) $\cup$ range-tree-chain S b bs B = bound-tree S B

using *Cons.IH*[*OF mono-tail bounds-tail*] .
 show ?case
 using *split-ab tail by auto*
 qed

lemma *card-bound-tree-range-chain-union*:
 assumes *mono*: *nondecreasing-from a bs*
 and *bounds*: *bounds-le B (a # bs)*
 shows *card (bound-tree S B) =*
 card (bound-tree S (Fin a)) + card (range-tree-chain S a bs B)
proof –
 have *union*: *bound-tree S (Fin a) $\cup$ range-tree-chain S a bs B = bound-tree S B*
 using *bound-tree-range-chain-union*[*OF mono bounds, of S*] .
 have *disjoint*: *bound-tree S (Fin a) $\cap$ range-tree-chain S a bs B = {}*
 using *bound-tree-disjoint-range-tree-chain*[*OF mono, of S*] .
 have *card (bound-tree S (Fin a) $\cup$ range-tree-chain S a bs B) =*
 card (bound-tree S (Fin a)) + card (range-tree-chain S a bs B)
 by (*rule card-Un-disjoint*) (*simp-all add: disjoint*)
 then show ?thesis
 using *union by simp*
 qed

lemma *complete-on-range-chain-union*:
 assumes *mono*: *nondecreasing-from a bs*
 and *bounds*: *bounds-le B (a # bs)*
 and *lower*: *complete-on d (bound-tree S (Fin a))*
 and *chain*: *complete-on d (range-tree-chain S a bs B)*
 shows *complete-on d (bound-tree S B)*
proof –
 have *complete-on d (bound-tree S (Fin a) $\cup$ range-tree-chain S a bs B)*
 using *complete-on-Un*[*OF lower chain*] .
 then show ?thesis
 using *bound-tree-range-chain-union*[*OF mono bounds, of S*] **by simp**
 qed

lemma *complete-on-range-tree-chain-list*:
 assumes $\bigwedge X. X \in \text{set } (\text{range-tree-chain-list } S \text{ a bs } B) \implies \text{complete-on } d \text{ } X$
 shows *complete-on d (range-tree-chain S a bs B)*
proof –
 have *complete-on d ($\bigcup (\text{set } (\text{range-tree-chain-list } S \text{ a bs } B))$)*
 using *complete-on-Union-list*[*OF assms*] .
 then show ?thesis
 using *Union-range-tree-chain-list*[*of S a bs B*] **by simp**
 qed

theorem *partition-loop-trace-assembly-post*:
 assumes *le*: *bound-le B' B*
 and *mono*: *nondecreasing-from a bs*
 and *bounds*: *bounds-le B' (a # bs)*

```

and lower: complete-on d' (bound-tree S (Fin a))
and children: list-all2 ( $\lambda U X. U = X \wedge$  complete-on d' U) Us
  (range-tree-chain-list S a bs B')
and U-def: U = bound-tree S (Fin a)  $\cup \bigcup$  (set Us)
shows bmssp-post-full d S B d' B' U
proof –
  have Union-eq:
     $\bigcup$  (set Us) =  $\bigcup$  (set (range-tree-chain-list S a bs B'))
    using children by (induction rule: list-all2-induct) auto
  have children-complete:  $\bigwedge X. X \in \text{set } Us \implies \text{complete-on } d' X$ 
    using children by (induction rule: list-all2-induct) auto
  have chain-complete: complete-on d' (range-tree-chain S a bs B')
proof –
    have complete-on d' ( $\bigcup$  (set Us))
      by (rule complete-on-Union-list[OF children-complete])
    then show ?thesis
      using Union-eq Union-range-tree-chain-list[of S a bs B'] by simp
qed
have U-tree: U = bound-tree S B'
  using U-def Union-eq Union-range-tree-chain-list[of S a bs B']
    bound-tree-range-chain-union[OF mono bounds, of S]
  by simp
have complete-on d' U
  unfolding U-def
  using complete-on-Un[OF lower]
    complete-on-Union-list[OF children-complete] by blast
then show ?thesis
  using le U-tree unfolding bmssp-post-full-def by blast
qed

```

The range-chain lemmas culminate in $\llbracket \text{bound-le } ?B' ?B; \text{nondecreasing-from } ?a ?bs; \text{bounds-le } ?B' (?a \# ?bs); \text{complete-on } ?d' (\text{bound-tree } ?S (\text{Fin } ?a)); \text{list-all2 } (\lambda U X. U = X \wedge \text{complete-on } ?d' U) ?Us (\text{range-tree-chain-list } ?S ?a ?bs ?B'); ?U = \text{bound-tree } ?S (\text{Fin } ?a) \cup \bigcup (\text{set } ?Us) \rrbracket \implies \text{bmssp-post-full } ?d ?S ?B ?d' ?B' ?U$. The hypotheses are exactly the certificate carried by the concrete partition trace: a returned bound below the input bound, nondecreasing child boundaries, child bounds below the returned bound, completeness of the lower prefix, and one completed set for each range in *range-tree-chain-list*. The theorem proves that their union is precisely *bound-tree* for the returned bound and that the final labels are complete there.

```

lemma pull-minimum-bound-tree-lower-split:
  assumes cover: complete-tree-cover d S (Fin beta)
    and beta'-le: beta'  $\leq$  beta
  shows bound-tree S (Fin beta') = bound-tree (split-below d S beta) (Fin beta')
proof
  show bound-tree S (Fin beta')  $\subseteq$  bound-tree (split-below d S beta) (Fin beta')
proof

```



```

fix v
assume v: v ∈ bound-tree S (Fin beta')
then have vV: v ∈ V and reach-v: reachable s v
  and through-S: through S v and lt-beta': dist s v < beta'
  unfolding bound-tree-def by auto
have v-beta: v ∈ bound-tree S (Fin beta)
  using v beta'-le unfolding bound-tree-def by auto
then have through-complete d S v
  using cover unfolding complete-tree-cover-def by blast
then obtain u p where uS: u ∈ S and ucomp: u ∈ complete-vertices d
  and p: shortest-walk s p v and up: u ∈ set p
  and du: d u = dist s u and dist-le: dist s u ≤ dist s v
  by (rule through-complete-obtain)
have d u < beta
  using du dist-le lt-beta' beta'-le by linarith
then have u ∈ split-below d S beta
  using uS unfolding split-below-def by blast
then have through (split-below d S beta) v
  using p up unfolding through-def by blast
then show v ∈ bound-tree (split-below d S beta) (Fin beta')
  using vV reach-v lt-beta' unfolding bound-tree-def by auto
qed
next
show bound-tree (split-below d S beta) (Fin beta') ⊆ bound-tree S (Fin beta')
  using bound-tree-mono[OF split-below-subset[of d S beta], of Fin beta'] by blast
qed

lemma pull-recursive-post-lifts-bound-tree:
  assumes cover: complete-tree-cover d S (Fin beta)
  and post: bmssp-post-full d (split-below d S beta) (Fin beta) d' B' U
  shows U = bound-tree S B'
proof -
  have le: bound-le B' (Fin beta)
  and U: U = bound-tree (split-below d S beta) B'
  using post unfolding bmssp-post-full-def by auto
show ?thesis
proof (cases B')
case Infinity
then have False
  using le by simp
then show ?thesis
  by blast
next
case (Fin beta')
then have beta'-le: beta' ≤ beta
  using le by simp
have bound-tree S (Fin beta') =
  bound-tree (split-below d S beta) (Fin beta')
  using pull-minimum-bound-tree-lower-split[OF cover beta'-le] .

```

then show *?thesis*
 using *U Fin* by *simp*
 qed
 qed

theorem *concrete-find-pivots-pull-pre*:
 assumes *sound*: *sound-label d*
 and *pre*: *bmssp-pre-full d S B*
 and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
 and *beta*: *below-bound beta B*
 shows *bmssp-pre-full*
 (*find-pivots-label k d S B*)
 (*split-below (find-pivots-label k d S B)*)
 (*find-pivots-pivots k d S B*) *beta*)
 (*Fin beta*)
proof –
 have *pivot-pre*:
bmssp-pre-full
 (*find-pivots-label k d S B*)
 (*find-pivots-pivots k d S B*) *B*
 using *find-pivots-establishes-pivot-pre-concrete*[*OF sound pre S-reaches*] .
 show *?thesis*
 using *pull-minimum-pre-for-lower-split*[*OF pivot-pre beta*] .
 qed

theorem *concrete-capped-find-pivots-pull-pre*:
 assumes *sound*: *sound-label d*
 and *pre*: *bmssp-pre-full d S B*
 and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
 and *beta*: *below-bound beta B*
 shows *bmssp-pre-full*
 (*find-pivots-label-capped k cap d S B*)
 (*split-below (find-pivots-label-capped k cap d S B)*)
 (*find-pivots-pivots-capped k cap d S B*) *beta*)
 (*Fin beta*)
proof –
 have *pivot-pre*:
bmssp-pre-full
 (*find-pivots-label-capped k cap d S B*)
 (*find-pivots-pivots-capped k cap d S B*) *B*
 using *find-pivots-capped-establishes-pivot-pre-concrete*[*OF sound pre S-reaches*] .
 show *?thesis*
 using *pull-minimum-pre-for-lower-split*[*OF pivot-pre beta*] .
 qed

theorem *concrete-find-pivots-pull-bound-tree-lower*:
 assumes *sound*: *sound-label d*
 and *pre*: *bmssp-pre-full d S B*
 and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$

and *beta*: below-bound *beta* *B*
and *beta'*-le: $\beta' \leq \beta$
shows *bound-tree* (*find-pivots-pivots* *k d S B*) (*Fin* *beta'*) =
bound-tree
(*split-below* (*find-pivots-label* *k d S B*)
(*find-pivots-pivots* *k d S B*) *beta*)
(*Fin* *beta'*)
proof –
let *?d'* = *find-pivots-label* *k d S B*
let *?P* = *find-pivots-pivots* *k d S B*
have *pivot-pre-B*: *bmssp-pre-full* *?d' ?P B*
using *find-pivots-establishes-pivot-pre-concrete*[*OF sound pre S-reaches*] .
have *pivot-pre-beta*: *bmssp-pre-full* *?d' ?P (Fin beta)*
using *bmssp-pre-full-bound-mono*[*OF pivot-pre-B*] *beta*
by (*cases B*) *auto*
have *cover-beta*: *complete-tree-cover* *?d' ?P (Fin beta)*
using *bmssp-pre-full-complete-tree-cover*[*OF pivot-pre-beta*] .
show *?thesis*
using *pull-minimum-bound-tree-lower-split*[*OF cover-beta beta'-le*] .
qed

theorem *concrete-capped-find-pivots-pull-bound-tree-lower*:
assumes *sound*: *sound-label* *d*
and *pre*: *bmssp-pre-full* *d S B*
and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
and *beta*: below-bound *beta* *B*
and *beta'*-le: $\beta' \leq \beta$
shows *bound-tree* (*find-pivots-pivots-capped* *k cap d S B*) (*Fin* *beta'*) =
bound-tree
(*split-below* (*find-pivots-label-capped* *k cap d S B*)
(*find-pivots-pivots-capped* *k cap d S B*) *beta*)
(*Fin* *beta'*)
proof –
let *?d'* = *find-pivots-label-capped* *k cap d S B*
let *?P* = *find-pivots-pivots-capped* *k cap d S B*
have *pivot-pre-B*: *bmssp-pre-full* *?d' ?P B*
using *find-pivots-capped-establishes-pivot-pre-concrete*[*OF sound pre S-reaches*] .
have *pivot-pre-beta*: *bmssp-pre-full* *?d' ?P (Fin beta)*
using *bmssp-pre-full-bound-mono*[*OF pivot-pre-B*] *beta*
by (*cases B*) *auto*
have *cover-beta*: *complete-tree-cover* *?d' ?P (Fin beta)*
using *bmssp-pre-full-complete-tree-cover*[*OF pivot-pre-beta*] .
show *?thesis*
using *pull-minimum-bound-tree-lower-split*[*OF cover-beta beta'-le*] .
qed

theorem *concrete-find-pivots-pull-recursive-post-lifts*:
assumes *sound*: *sound-label* *d*
and *pre*: *bmssp-pre-full* *d S B*

```

and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
and beta: below-bound beta B
and post: bmssp-post-full
  (find-pivots-label k d S B)
  (split-below (find-pivots-label k d S B)
    (find-pivots-pivots k d S B) beta)
  (Fin beta) d'' B' U
shows U = bound-tree (find-pivots-pivots k d S B) B'
proof –
  let ?d' = find-pivots-label k d S B
  let ?P = find-pivots-pivots k d S B
  have pivot-pre-B: bmssp-pre-full ?d' ?P B
    using find-pivots-establishes-pivot-pre-concrete[OF sound pre S-reaches] .
  have pivot-pre-beta: bmssp-pre-full ?d' ?P (Fin beta)
    using bmssp-pre-full-bound-mono[OF pivot-pre-B] beta
    by (cases B) auto
  have cover-beta: complete-tree-cover ?d' ?P (Fin beta)
    using bmssp-pre-full-complete-tree-cover[OF pivot-pre-beta] .
  show ?thesis
    using pull-recursive-post-lifts-bound-tree[OF cover-beta post] .
qed

theorem concrete-capped-find-pivots-pull-recursive-post-lifts:
  assumes sound: sound-label d
  and pre: bmssp-pre-full d S B
  and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
  and beta: below-bound beta B
  and post: bmssp-post-full
    (find-pivots-label-capped k cap d S B)
    (split-below (find-pivots-label-capped k cap d S B)
      (find-pivots-pivots-capped k cap d S B) beta)
    (Fin beta) d'' B' U
  shows U = bound-tree (find-pivots-pivots-capped k cap d S B) B'
proof –
  let ?d' = find-pivots-label-capped k cap d S B
  let ?P = find-pivots-pivots-capped k cap d S B
  have pivot-pre-B: bmssp-pre-full ?d' ?P B
    using find-pivots-capped-establishes-pivot-pre-concrete[OF sound pre S-reaches] .
  have pivot-pre-beta: bmssp-pre-full ?d' ?P (Fin beta)
    using bmssp-pre-full-bound-mono[OF pivot-pre-B] beta
    by (cases B) auto
  have cover-beta: complete-tree-cover ?d' ?P (Fin beta)
    using bmssp-pre-full-complete-tree-cover[OF pivot-pre-beta] .
  show ?thesis
    using pull-recursive-post-lifts-bound-tree[OF cover-beta post] .
qed

```

The Pull-minimum lemmas connect the range algebra to FindPivots. Once FindPivots establishes a pivot precondition, *split-below* can be used as the source set for a recursive call below a Pull boundary. If that recursive call

returns a BMSSP postcondition for the split set, the lifting lemmas identify its output tree with the corresponding pivot tree. Both capped and uncapped FindPivots variants have parallel theorems because later operational relations use the capped executable search.

theorem *concrete-find-pivots-final-tree-assembly:*

assumes *sound: sound-label d*
and *pre: bmssp-pre-full d S B*
and *S-reaches: $\bigwedge x. x \in S \implies \text{reachable } s \ x$*
and *loop: bmssp-post-full*
(find-pivots-label k d S B)
(find-pivots-pivots k d S B) B d-loop B' U-loop
shows *U-loop $\cup$*
{v $\in$ bound-tree S B'. find-pivots-label k d S B v = dist s v} =
bound-tree S B'

proof –

let *?d' = find-pivots-label k d S B*
let *?P = find-pivots-pivots k d S B*
let *?W = {v $\in$ bound-tree S B'. ?d' v = dist s v}*
have *loop-le: bound-le B' B* **and** *U-loop: U-loop = bound-tree ?P B'*
using *loop unfolding bmssp-post-full-def* **by** *auto*
have *P-subset-S: ?P $\subseteq$ S*
unfolding *find-pivots-pivots-def* **by** *blast*
have *pivot-pre: bmssp-pre-full ?d' ?P B*
using *find-pivots-establishes-pivot-pre-concrete[OF sound pre S-reaches]* .
show *?thesis*

proof

show *U-loop $\cup$?W $\subseteq$ bound-tree S B'*

proof

fix *v*

assume *v $\in$ U-loop $\cup$?W*

then show *v $\in$ bound-tree S B'*

proof

assume *v $\in$ U-loop*

then have *v $\in$ bound-tree ?P B'*

using *U-loop* **by** *simp*

then show *?thesis*

using *bound-tree-mono[OF P-subset-S, of B']* **by** *blast*

next

assume *v $\in$?W*

then show *?thesis*

by *blast*

qed

qed

next

show *bound-tree S B' $\subseteq$ U-loop $\cup$?W*

proof

fix *v*

assume *v-bound: v $\in$ bound-tree S B'*

then have *vV: v $\in$ V* **and** *reach-v: reachable s v*

```

    and below-B': below-bound (dist s v) B'
    unfolding bound-tree-def by auto
  show  $v \in U\text{-loop} \cup ?W$ 
  proof (cases ?d' v = dist s v)
    case True
    then show ?thesis
      using v-bound by blast
  next
    case False
    have below-B: below-bound (dist s v) B
      using loop-le below-B' by (cases B'; cases B) auto
    have through-complete ?d' ?P v
      using pivot-pre vV reach-v below-B False
      unfolding bmssp-pre-full-def by blast
    then have through ?P v
      by (rule through-complete-imp-through)
    then have  $v \in \text{bound-tree } ?P B'$ 
      using vV reach-v below-B' unfolding bound-tree-def by blast
    then show ?thesis
      using U-loop by blast
  qed
qed
qed
qed
qed

```

theorem *concrete-capped-find-pivots-final-tree-assembly:*

```

  assumes sound: sound-label d
  and pre: bmssp-pre-full d S B
  and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s x$ 
  and loop: bmssp-post-full
    (find-pivots-label-capped k cap d S B)
    (find-pivots-pivots-capped k cap d S B) B d-loop B' U-loop
  shows  $U\text{-loop} \cup$ 
     $\{v \in \text{bound-tree } S B'. \text{find-pivots-label-capped } k \text{ cap } d S B v = \text{dist } s v\} =$ 
     $\text{bound-tree } S B'$ 
  proof -
    let ?d' = find-pivots-label-capped k cap d S B
    let ?P = find-pivots-pivots-capped k cap d S B
    let ?W =  $\{v \in \text{bound-tree } S B'. ?d' v = \text{dist } s v\}$ 
    have loop-le: bound-le B' B and U-loop:  $U\text{-loop} = \text{bound-tree } ?P B'$ 
      using loop unfolding bmssp-post-full-def by auto
    have pivot-pre: bmssp-pre-full ?d' ?P B
      using find-pivots-capped-establishes-pivot-pre-concrete[OF sound pre S-reaches] .
    have P-subset-V:  $?P \subseteq V$ 
      using pivot-pre unfolding bmssp-pre-full-def by blast
    have P-through-S:
       $\bigwedge v. v \in \text{bound-tree } ?P B' \implies v \in \text{bound-tree } S B'$ 
    proof -
      fix v

```

```

assume  $vP$ :  $v \in \text{bound-tree } ?P \ B'$ 
then have  $vV$ :  $v \in V$  and  $\text{reach-}v$ :  $\text{reachable } s \ v$ 
  and  $\text{through-}P$ :  $\text{through } ?P \ v$  and  $\text{below-}v$ :  $\text{below-bound } (\text{dist } s \ v) \ B'$ 
  unfolding  $\text{bound-tree-def}$  by  $\text{auto}$ 
have  $\text{through } S \ v$ 
proof –
  obtain  $u \ p$  where  $uP$ :  $u \in ?P$  and  $sp$ :  $\text{shortest-walk } s \ p \ v$  and  $up$ :  $u \in \text{set } p$ 
    using  $\text{through-}P$  unfolding  $\text{through-def}$  by  $\text{blast}$ 
  have  $u \in S$ 
  proof ( $\text{cases } \text{card } (\text{find-pivots-seen-capped } k \ \text{cap } d \ S \ B) > \text{cap}$ )
    case  $\text{True}$ 
    then show  $?thesis$ 
      using  $uP$  unfolding  $\text{find-pivots-pivots-capped-def}$  by  $\text{simp}$ 
    next
    case  $\text{False}$ 
    then show  $?thesis$ 
      using  $uP$  unfolding  $\text{find-pivots-pivots-capped-def}$  by  $\text{auto}$ 
  qed
  then show  $?thesis$ 
    using  $sp \ up$  unfolding  $\text{through-def}$  by  $\text{blast}$ 
  qed
then show  $v \in \text{bound-tree } S \ B'$ 
  using  $vV \ \text{reach-}v \ \text{below-}v$  unfolding  $\text{bound-tree-def}$  by  $\text{blast}$ 
qed
show  $?thesis$ 
proof
  show  $U\text{-loop} \cup ?W \subseteq \text{bound-tree } S \ B'$ 
    using  $U\text{-loop } P\text{-through-}S$  by  $\text{blast}$ 
  next
  show  $\text{bound-tree } S \ B' \subseteq U\text{-loop} \cup ?W$ 
  proof
    fix  $v$ 
    assume  $v\text{-bound}$ :  $v \in \text{bound-tree } S \ B'$ 
    then have  $vV$ :  $v \in V$  and  $\text{reach-}v$ :  $\text{reachable } s \ v$ 
      and  $\text{below-}B'$ :  $\text{below-bound } (\text{dist } s \ v) \ B'$ 
      unfolding  $\text{bound-tree-def}$  by  $\text{auto}$ 
    show  $v \in U\text{-loop} \cup ?W$ 
    proof ( $\text{cases } ?d' \ v = \text{dist } s \ v$ )
      case  $\text{True}$ 
      then show  $?thesis$ 
        using  $v\text{-bound}$  by  $\text{blast}$ 
      next
      case  $\text{False}$ 
      have  $\text{below-}B$ :  $\text{below-bound } (\text{dist } s \ v) \ B$ 
        using  $\text{loop-le below-}B'$  by ( $\text{cases } B'$ ;  $\text{cases } B$ )  $\text{auto}$ 
      have  $\text{through-complete } ?d' \ ?P \ v$ 
        using  $\text{pivot-pre } vV \ \text{reach-}v \ \text{below-}B \ \text{False}$ 
        unfolding  $\text{bmssp-pre-full-def}$  by  $\text{blast}$ 
      then have  $\text{through } ?P \ v$ 

```

by (rule through-complete-imp-through)
 then have $v \in \text{bound-tree } ?P \ B'$
 using $vV \text{ reach-}v \text{ below-}B' \text{ unfolding bound-tree-def by blast}$
 then show $?thesis$
 using $U\text{-loop}$ by blast
 qed
 qed
 qed
 qed

theorem *concrete-find-pivots-final-assembly-post:*

assumes $\text{sound: sound-label } d$
 and $\text{pre: bmssp-pre-full } d \ S \ B$
 and $S\text{-reaches: } \bigwedge x. x \in S \implies \text{reachable } s \ x$
 and $\text{loop: bmssp-post-full}$
 ($\text{find-pivots-label } k \ d \ S \ B$)
 ($\text{find-pivots-pivots } k \ d \ S \ B$) $B \ d\text{-final } B' \ U\text{-loop}$
 and $\text{comp}W: \text{complete-on } d\text{-final}$
 $\{v \in \text{bound-tree } S \ B'. \text{find-pivots-label } k \ d \ S \ B \ v = \text{dist } s \ v\}$
 defines $U \equiv U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \text{find-pivots-label } k \ d \ S \ B \ v = \text{dist } s \ v\}$
 shows $\text{bmssp-post-full } d \ S \ B \ d\text{-final } B' \ U$
proof –
 let $?W = \{v \in \text{bound-tree } S \ B'. \text{find-pivots-label } k \ d \ S \ B \ v = \text{dist } s \ v\}$
 have $\text{le: bound-le } B' \ B$
 using loop unfolding $\text{bmssp-post-full-def}$ by blast
 have $U\text{-eq: } U = \text{bound-tree } S \ B'$
 using *concrete-find-pivots-final-tree-assembly*[$OF \ \text{sound pre } S\text{-reaches loop}$]
 unfolding $U\text{-def}$ by simp
 have $\text{loop-complete: complete-on } d\text{-final } U\text{-loop}$
 using loop unfolding $\text{bmssp-post-full-def}$ by blast
 have $\text{complete-on } d\text{-final } U$
 unfolding $U\text{-def}$ using complete-on-Un [$OF \ \text{loop-complete comp}W$] .
 then show $?thesis$
 using $\text{le } U\text{-eq}$ unfolding $\text{bmssp-post-full-def}$ by blast
 qed

theorem *concrete-capped-find-pivots-final-assembly-post:*

assumes $\text{sound: sound-label } d$
 and $\text{pre: bmssp-pre-full } d \ S \ B$
 and $S\text{-reaches: } \bigwedge x. x \in S \implies \text{reachable } s \ x$
 and $\text{loop: bmssp-post-full}$
 ($\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B$)
 ($\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B$) $B \ d\text{-final } B' \ U\text{-loop}$
 and $\text{comp}W: \text{complete-on } d\text{-final}$
 $\{v \in \text{bound-tree } S \ B'. \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\}$
 defines $U \equiv U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\}$
 shows $\text{bmssp-post-full } d \ S \ B \ d\text{-final } B' \ U$


```

proof –
  let ?W = {v ∈ bound-tree S B'. find-pivots-label-capped k cap d S B v = dist s v}
  have le: bound-le B' B
    using loop unfolding bmssp-post-full-def by blast
  have U-eq: U = bound-tree S B'
    using concrete-capped-find-pivots-final-tree-assembly[OF sound pre S-reaches
loop]
  unfolding U-def by simp
  have loop-complete: complete-on d-final U-loop
    using loop unfolding bmssp-post-full-def by blast
  have complete-on d-final U
    unfolding U-def using complete-on-Un[OF loop-complete comp W] .
  then show ?thesis
    using le U-eq unfolding bmssp-post-full-def by blast
qed

```

The final assembly theorems are the facts used by *concrete-capped-bmssp-step*. The loop solves the pivot tree, while FindPivots may have completed additional vertices in the caller's original tree. The assembly theorem proves that the union of those two regions is the caller's returned bounded tree. The postcondition theorems then add completeness of the explicitly completed FindPivots region and conclude *bmssp-post-full* for the whole non-base step.

end

end

theory BMSSP-Partition-Pull-Bridge

imports BMSSP-Pull-Minimum BMSSP-Partition-Interface

begin

22 Partition Pull to BMSSP Pull Minimum

The partition interface is graph-independent: it knows about keys and labels, but not about shortest-path trees. BMSSP, on the other hand, phrases a child recursive call as a lower split of the current source set by a tentative distance bound. This small bridge theory identifies those two views.

The conversion is simple but important. A label function d and a source set S induce a partition view whose keys are exactly S and whose values are the labels d . A pull from that view is therefore a pull by tentative distance. The theorems below prove that the pulled key set is exactly *split-below* at the returned bound, and that the BMSSP precondition descends from the parent problem to the child problem selected by the pull.

This file is used twice in the development. The sorted reference pull uses it to justify the executable specification path. The operational and bucketed theories use the abstract *pull-separates* statement, so they can reason about BMSSP recursion without mentioning the representation of the partition data structure.

context *unique-shortest-digraph*
begin

definition *label-partition-view* **where**
label-partition-view $d\ S = (\text{keys-of} = S, \text{value-of} = d)$

lemma *label-partition-view-keys* [*simp*]:
 $\text{keys-of} (\text{label-partition-view } d\ S) = S$
unfolding *label-partition-view-def* **by** *simp*

lemma *label-partition-view-value* [*simp*]:
 $\text{value-of} (\text{label-partition-view } d\ S) = d$
unfolding *label-partition-view-def* **by** *simp*

The first two theorems are the semantic translation. For the sorted reference operation, *sorted-pull-set-eq-split-below* unfolds the sorted pull and the label partition view to obtain the mathematical lower split. For any implementation satisfying the abstract separator predicate, *pull-separates-label-set-eq-split-below* gives the same conclusion from *pull-separates*. The latter is the theorem used by the bucketed implementation after it proves its pull operation realises the separator contract.

theorem *sorted-pull-set-eq-split-below*:
fixes $M :: \text{nat}$
and $Bmax :: \text{real}$
assumes *finite-S*: $\text{finite } S$
and *upper*: $\bigwedge u. u \in S \implies d\ u < Bmax$
and *D-def*: $D = \text{label-partition-view } d\ S$
and *beta-def*: $\text{beta} = \text{sorted-pull-bound } M\ Bmax\ D$
shows $\text{sorted-pull-set } M\ D = \text{split-below } d\ S\ \text{beta}$

proof –
have $\text{sorted-pull-set } M\ D =$
 $\{u \in \text{keys-of } D. \text{value-of } D\ u < \text{sorted-pull-bound } M\ Bmax\ D\}$
using *sorted-pull-exact-split*[*of* $D\ Bmax\ M$] *finite-S upper*
unfolding *D-def* **by** *simp*
then show *?thesis*
unfolding *D-def beta-def split-below-def* **by** *simp*
qed

theorem *pull-separates-label-set-eq-split-below*:
fixes $Bmax :: \text{real}$
assumes *pull*: $\text{pull-separates} (\text{label-partition-view } d\ S)\ M\ Bmax\ S'\ \text{beta}\ D'$
and *upper*: $\bigwedge u. u \in S \implies d\ u < Bmax$
shows $S' = \text{split-below } d\ S\ \text{beta}$

proof –
have $S' =$
 $\{u \in \text{keys-of} (\text{label-partition-view } d\ S). \text{value-of} (\text{label-partition-view } d\ S)\ u < \text{beta}\}$
by (*rule* *pull-separates-exact-split*[*OF* *pull*]) (*simp add: upper*)
then show *?thesis*

```

    unfolding split-below-def by simp
  qed

theorem sorted-pull-establishes-lower-pre:
  fixes M :: nat
  and Bmax :: real
  assumes pre: bmssp-pre-full d S (Fin Bmax)
  and upper:  $\bigwedge u. u \in S \implies d\ u < Bmax$ 
  and D-def:  $D = \text{label-partition-view } d\ S$ 
  and S'-def:  $S' = \text{sorted-pull-set } M\ D$ 
  and beta-def:  $\beta = \text{sorted-pull-bound } M\ Bmax\ D$ 
  shows bmssp-pre-full d S' (Fin beta)
proof -
  have S-subset:  $S \subseteq V$ 
  using pre unfolding bmssp-pre-full-def by blast
  have finite-S: finite S
  using finite-subset[OF S-subset finite-V] .
  let ?xs = partition-key-order D
  have set-xs: set ?xs = S
  using partition-key-order-properties(1)[of D] finite-S
  unfolding D-def by simp
  show ?thesis
proof (cases length ?xs ≤ M)
  case True
  have S'-eq:  $S' = S$ 
  using True set-xs unfolding S'-def sorted-pull-set-def by (simp add: Let-def)
  have beta-eq:  $\beta = Bmax$ 
  using True unfolding beta-def sorted-pull-bound-def by (simp add: Let-def)
  show ?thesis
  using pre unfolding S'-eq beta-eq .
next
  case False
  then have M-lt:  $M < \text{length } ?xs$ 
  by simp
  have beta-eq:  $\beta = d\ (?xs\ !\ M)$ 
  using False unfolding beta-def sorted-pull-bound-def D-def by (simp add:
Let-def)
  have nth-in:  $?xs\ !\ M \in S$ 
  using set-xs M-lt nth-mem by metis
  have beta-lt:  $\beta < Bmax$ 
  using upper[OF nth-in] beta-eq by simp
  have S'-split:  $S' = \text{split-below } d\ S\ \beta$ 
  using sorted-pull-set-eq-split-below[OF finite-S upper D-def beta-def]
  unfolding S'-def .
  show ?thesis
  unfolding S'-split
  by (rule pull-minimum-pre-for-lower-split[OF pre]) (simp add: beta-lt)
qed
qed

```

The final theorem packages the precondition transfer needed by a recursive BMSSP child call. If the parent problem is valid up to $Bmax$ and all current source labels lie below $Bmax$, then the set returned by a sorted pull is valid up to the returned child bound. The all-keys case is just the parent precondition again; the proper-prefix case uses the pull-minimum lemma for lower splits.

Later operational proofs use this theorem as a template for arbitrary partition implementations: once a pull is known to select the same lower split, the recursive child precondition follows from the same argument.

end

end

theory *BMSSP-Initialization*

imports *BMSSP-Shortest-Path-Lemmas*

begin

23 Initial Labels

BMSSP starts from the usual shortest-path initialization: the source is complete and all other labels are conservative upper bounds. In an imperative implementation one would normally write this as distance zero for the source and infinity for every other vertex. The abstract correctness layer in this entry uses real-valued labels rather than an extended-real datatype, so this theory gives a finite replacement for infinity.

The replacement is the maximum true source distance over the finite vertex set. This is not intended as an executable way to discover shortest paths; it is a semantic initial label used in the proof stack. Because the graph locale is finite, the maximum exists. Because every reachable vertex has distance at most that maximum, the label function is sound. The source remains exact because the distance from any vertex to itself is zero under non-negative edge weights.

The lemmas below isolate precisely the facts needed by later top-level theorems: the source label is complete, the whole label function is sound, and the root call with source set $\{s\}$ and bound *Infinity* satisfies the BMSSP precondition.

context *finite-weighted-digraph*

begin

definition *finite-initial-label* **where**

finite-initial-label $v = (if\ v = s\ then\ 0\ else\ Max\ (dist\ s\ 'V))$

The definition *finite-initial-label* is intentionally semantic. Its non-source value mentions *local.dist*, so it is not the data structure used by the exported example. Its role is to let the correctness proof start from a finite real-valued label function while preserving the same logical shape as Dijkstra's standard initialization.

```

lemma dist-refl-zero:
  assumes  $v \in V$ 
  shows  $\text{dist } v \ v = 0$ 
proof –
  have simple: simple-walk-betw  $v \ [v] \ v$ 
    using assms unfolding simple-walk-betw-def walk-betw-def by simp
  have  $\text{dist } v \ v \leq 0$ 
    using dist-le-simple-walk-weight [OF simple] by simp
  moreover have  $0 \leq \text{dist } v \ v$ 
  proof –
    have reachable  $v \ v$ 
      using reachable-refl [OF assms] .
    then have  $\text{dist } v \ v \in \text{simple-walk-weights } v \ v$ 
      using dist-is-simple-walk-weight by blast
    then obtain  $p$  where  $p$ : simple-walk-betw  $v \ p \ v$  walk-weight  $p = \text{dist } v \ v$ 
      unfolding simple-walk-weights-def by auto
    then have walk  $p$ 
      unfolding simple-walk-betw-def walk-betw-def by auto
    then have  $0 \leq \text{walk-weight } p$ 
      by (rule walk-weight-nonneg)
    then show ?thesis
      using  $p$  by simp
  qed
  ultimately show ?thesis
    by simp
qed

```

```

lemma finite-initial-label-source-complete:
  finite-initial-label  $s = \text{dist } s \ s$ 
  using dist-refl-zero [OF source-in-V] unfolding finite-initial-label-def by simp

```

The proof of $?v \in V \implies \text{local.dist } ?v \ ?v = 0$ uses the one-vertex simple walk for the upper bound and non-negativity of walk weights for the lower bound. From it, *finite-initial-label* $s = \text{local.dist } s \ s$ gives the exact source label required by root BMSSP initialization. The two soundness lemmas below differ only in whether reachability of every vertex is supplied explicitly or left as the local reachable premise of *sound-label*.

```

lemma finite-initial-label-sound:
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
  shows sound-label finite-initial-label
proof –
  have nonempty:  $\text{dist } s \ ' V \neq \{\}$ 
    using source-in-V by blast
  show ?thesis
    unfolding sound-label-def finite-initial-label-def
  proof clarify
    fix  $v$ 
    assume  $v \in V$ :  $v \in V$  and reach-v: reachable  $s \ v$ 
    show  $\text{dist } s \ v \leq (\text{if } v = s \text{ then } 0 \text{ else } \text{Max } (\text{dist } s \ ' V))$ 

```

```

proof (cases v = s)
  case True
  then show ?thesis
    using dist-refl-zero[OF source-in-V] by simp
next
  case False
  have dist s v ∈ dist s ‘ V
    using vV by blast
  then have dist s v ≤ Max (dist s ‘ V)
    using finite-V by simp
  then show ?thesis
    using False by simp
qed
qed
qed

lemma finite-initial-label-sound-reachable:
  shows sound-label finite-initial-label
proof –
  have nonempty: dist s ‘ V ≠ {}
    using source-in-V by blast
  show ?thesis
    unfolding sound-label-def finite-initial-label-def
  proof clarify
    fix v
    assume vV: v ∈ V and reach-v: reachable s v
    show dist s v ≤ (if v = s then 0 else Max (dist s ‘ V))
    proof (cases v = s)
      case True
      then show ?thesis
        using dist-refl-zero[OF source-in-V] by simp
    next
      case False
      have dist s v ∈ dist s ‘ V
        using vV by blast
      then have dist s v ≤ Max (dist s ‘ V)
        using finite-V nonempty by simp
      then show ?thesis
        using False by simp
    qed
  qed
qed

```

lemma initialized-root-pre-full:

assumes all-reachable: $\bigwedge v. v \in V \implies \text{reachable } s \ v$

shows bmssp-pre finite-initial-label {s} Infinity

using top-bmssp-pre[OF all-reachable, of finite-initial-label] .

Finally, $(\bigwedge v. v \in V \implies \text{reachable } s \ v) \implies \text{bmssp-pre finite-initial-label } \{s\} \text{ Infinity}$ packages the initialized labels for the root abstract BMSSP call.

The source set is the singleton $\{s\}$, the bound is *Infinity*, and every reachable vertex lies on a shortest path through the source. Later theories strengthen this precondition to the complete-source form and then discharge the full algorithmic step relation.

end

end

theory *BMSSP-Concrete-Step*

imports *BMSSP-Pull-Minimum*

begin

24 Concrete One-Step BMSSP Assembly

This theory packages the concrete part of a non-base BMSSP call. The abstract correctness skeleton states its recursive middle phase with the predicate *partition-loop-post*: after FindPivots has chosen a pivot set, the loop must solve the bounded tree induced by those pivots. That statement is intentionally compact, but it hides the range structure that the implementation actually follows.

The concrete step exposed here replaces that abstract loop contract by an explicit monotone trace of child ranges. A trace contains the first lower boundary, the list of subsequent boundaries, the label function after the loop, and the collection of vertex sets solved by the child calls. The trace proof checks that the boundaries are nondecreasing, remain below the returned BMSSP bound, and cover exactly the chain of bounded shortest-path trees that the partition loop is supposed to assemble.

The final step definitions then restore FindPivots. The uncapped version uses the conceptual FindPivots operation, while the capped version uses the executable bounded search from the paper. Both definitions have the same shape: perform FindPivots, solve the pivot loop, keep the vertices completed directly by FindPivots, and return their union. The two correctness theorems below show that this concrete assembly is still a valid *bmssp-post-full* result.

context *unique-shortest-digraph*

begin

definition *concrete-partition-loop-trace* **where**

concrete-partition-loop-trace $P \ B \ a \ bs \ d' \ B' \ Us \ U \longleftrightarrow$

bound-le $B' \ B \wedge$

nondecreasing-from $a \ bs \wedge$

bounds-le $B' \ (a \ \# \ bs) \wedge$

complete-on $d' \ (bound-tree \ P \ (Fin \ a)) \wedge$

list-all2 $(\lambda U \ X. \ U = X \wedge complete-on \ d' \ U) \ Us$

$(range-tree-chain-list \ P \ a \ bs \ B') \wedge$

$U = bound-tree \ P \ (Fin \ a) \cup \bigcup (set \ Us)$

theorem *concrete-partition-loop-trace-post*:
assumes *trace*: *concrete-partition-loop-trace* $P\ B\ a\ bs\ d'\ B'\ Us\ U$
shows *bmssp-post-full* $d\ P\ B\ d'\ B'\ U$
proof –
have *le*: *bound-le* $B'\ B$
and *mono*: *nondecreasing-from* $a\ bs$
and *bounds*: *bounds-le* $B'\ (a\ \# \ bs)$
and *lower*: *complete-on* $d'\ (\text{bound-tree } P\ (\text{Fin } a))$
and *children*: *list-all2* $(\lambda U\ X. U = X \wedge \text{complete-on } d'\ U)\ Us$
(range-tree-chain-list $P\ a\ bs\ B')$
and *U*: $U = \text{bound-tree } P\ (\text{Fin } a) \cup \bigcup (\text{set } Us)$
using *trace unfolding concrete-partition-loop-trace-def* **by** *blast+*
show *?thesis*
by (*rule partition-loop-trace-assembly-post*
 $[OF\ le\ mono\ bounds\ lower\ children\ U])$
qed

definition *concrete-bmssp-step* **where**
concrete-bmssp-step $k\ d\ S\ B\ a\ bs\ d'\ B'\ Us\ U \longleftrightarrow$
 $(\text{let } d\text{-fp} = \text{find-pivots-label } k\ d\ S\ B;$
 $P = \text{find-pivots-pivots } k\ d\ S\ B;$
 $W = \{v \in \text{bound-tree } S\ B'.\ d\text{-fp } v = \text{dist } s\ v\}$
in $\exists U\text{-loop}.$
 $\text{concrete-partition-loop-trace } P\ B\ a\ bs\ d'\ B'\ Us\ U\text{-loop} \wedge$
 $\text{complete-on } d'\ W \wedge$
 $U = U\text{-loop} \cup W)$

definition *concrete-capped-bmssp-step* **where**
concrete-capped-bmssp-step $k\ \text{cap}\ d\ S\ B\ a\ bs\ d'\ B'\ Us\ U \longleftrightarrow$
 $(\text{let } d\text{-fp} = \text{find-pivots-label-capped } k\ \text{cap}\ d\ S\ B;$
 $P = \text{find-pivots-pivots-capped } k\ \text{cap}\ d\ S\ B;$
 $W = \{v \in \text{bound-tree } S\ B'.\ d\text{-fp } v = \text{dist } s\ v\}$
in $\exists U\text{-loop}.$
 $\text{concrete-partition-loop-trace } P\ B\ a\ bs\ d'\ B'\ Us\ U\text{-loop} \wedge$
 $\text{complete-on } d'\ W \wedge$
 $U = U\text{-loop} \cup W)$

The definition *concrete-partition-loop-trace* is the local certificate for the recursive partition loop. It does not say how pulls are generated; it says what must be true of the resulting range sequence. The theorem *concrete-partition-loop-trace ?P ?B ?a ?bs ?d' ?B' ?Us ?U $\implies$ bmssp-post-full ?d ?P ?B ?d' ?B' ?U* then converts such a trace into *bmssp-post-full* for the pivot set. The step predicates *concrete-bmssp-step* and *concrete-capped-bmssp-step* wrap the trace with FindPivots and the final union of loop-completed and FindPivots-completed vertices.

theorem *concrete-bmssp-step-correct*:
assumes *sound*: *sound-label* d
and *pre*: *bmssp-pre-full* $d\ S\ B$
and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s\ x$

and *step*: *concrete-bmssp-step* $k\ d\ S\ B\ a\ bs\ d'\ B'\ Us\ U$
shows *bmssp-post-full* $d\ S\ B\ d'\ B'\ U$
proof –
let $?d\text{-fp} = \text{find-pivots-label}\ k\ d\ S\ B$
let $?P = \text{find-pivots-pivots}\ k\ d\ S\ B$
let $?W = \{v \in \text{bound-tree}\ S\ B'.\ ?d\text{-fp}\ v = \text{dist}\ s\ v\}$
obtain *U-loop* **where**
trace: *concrete-partition-loop-trace* $?P\ B\ a\ bs\ d'\ B'\ Us\ U\text{-loop}$
and *compW*: *complete-on* $d'\ ?W$
and $U: U = U\text{-loop} \cup ?W$
using *step unfolding* *concrete-bmssp-step-def* **by** (*auto simp: Let-def*)
have *loop*: *bmssp-post-full* $?d\text{-fp}\ ?P\ B\ d'\ B'\ U\text{-loop}$
using *concrete-partition-loop-trace-post*[*OF trace*] .
have *le*: *bound-le* $B'\ B$
using *loop unfolding* *bmssp-post-full-def* **by** *blast*
have *U-tree*: $U\text{-loop} \cup ?W = \text{bound-tree}\ S\ B'$
using *concrete-find-pivots-final-tree-assembly*[*OF sound pre S-reaches loop*] .
have *loop-complete*: *complete-on* $d'\ U\text{-loop}$
using *loop unfolding* *bmssp-post-full-def* **by** *blast*
have *complete*: *complete-on* $d'\ (U\text{-loop} \cup ?W)$
using *complete-on-Un*[*OF loop-complete compW*] .
have *bmssp-post-full* $d\ S\ B\ d'\ B'\ (U\text{-loop} \cup ?W)$
using *le U-tree complete* *unfolding* *bmssp-post-full-def* **by** *blast*
then show *?thesis*
using U **by** *simp*
qed

theorem *concrete-capped-bmssp-step-correct*:

assumes *sound*: *sound-label* d
and *pre*: *bmssp-pre-full* $d\ S\ B$
and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable}\ s\ x$
and *step*: *concrete-capped-bmssp-step* $k\ \text{cap}\ d\ S\ B\ a\ bs\ d'\ B'\ Us\ U$
shows *bmssp-post-full* $d\ S\ B\ d'\ B'\ U$
proof –
let $?d\text{-fp} = \text{find-pivots-label-capped}\ k\ \text{cap}\ d\ S\ B$
let $?P = \text{find-pivots-pivots-capped}\ k\ \text{cap}\ d\ S\ B$
let $?W = \{v \in \text{bound-tree}\ S\ B'.\ ?d\text{-fp}\ v = \text{dist}\ s\ v\}$
obtain *U-loop* **where**
trace: *concrete-partition-loop-trace* $?P\ B\ a\ bs\ d'\ B'\ Us\ U\text{-loop}$
and *compW*: *complete-on* $d'\ ?W$
and $U: U = U\text{-loop} \cup ?W$
using *step unfolding* *concrete-capped-bmssp-step-def* **by** (*auto simp: Let-def*)
have *loop*: *bmssp-post-full* $?d\text{-fp}\ ?P\ B\ d'\ B'\ U\text{-loop}$
using *concrete-partition-loop-trace-post*[*OF trace*] .
have *le*: *bound-le* $B'\ B$
using *loop unfolding* *bmssp-post-full-def* **by** *blast*
have *U-tree*: $U\text{-loop} \cup ?W = \text{bound-tree}\ S\ B'$
using *concrete-capped-find-pivots-final-tree-assembly*[*OF sound pre S-reaches loop*] .

```

have loop-complete: complete-on d' U-loop
  using loop unfolding bmssp-post-full-def by blast
have complete: complete-on d' (U-loop  $\cup$  ?W)
  using complete-on-Un[OF loop-complete comp W] .
have bmssp-post-full d S B d' B' (U-loop  $\cup$  ?W)
  using le U-tree complete unfolding bmssp-post-full-def by blast
then show ?thesis
  using U by simp
qed

end

end
theory BMSSP-Concrete-Top-Level
  imports BMSSP-Initialization BMSSP-Concrete-Step
begin

```

25 Concrete Top-Level Correctness

These theorems connect the finite initial labels to the concrete one-step BMSSP assembly relation. They avoid the abstract *abstract-bmssp-step* contract used by the abstract interface.

The top-level SSSP call is a BMSSP call with singleton source set $\{s\}$ and infinite bound. The finite initial label gives distance 0 at the source and an effectively infinite value elsewhere. Once a concrete BMSSP step proves the full BMSSP postcondition for this root problem, the infinite-bound tree is the whole vertex set, so completeness of the returned label function is exactly SSSP correctness.

Two entry points are provided because the development has both an uncapped concrete step and the capped step used by the paper's parameter schedule. They share the same proof: establish the root precondition, invoke the corresponding concrete-step correctness theorem, then collapse the root BMSSP postcondition to *sssp-correct*.

```

context unique-shortest-digraph
begin

```

```

theorem finite-initial-label-concrete-top-level-correct:
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
  and step: concrete-bmssp-step k finite-initial-label {s} Infinity a bs
  d' Infinity Us U
  shows sssp-correct d'
proof –
  have pre: bmssp-pre-full finite-initial-label {s} Infinity
  using all-reachable finite-initial-label-source-complete
  by (rule top-bmssp-pre-full)
  have sound: sound-label finite-initial-label
  using finite-initial-label-sound[OF all-reachable] .

```

```

have S-reaches:  $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$ 
  using all-reachable source-in-V by blast
have post-full:
  bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
  by (rule concrete-bmssp-step-correct[OF sound pre S-reaches step])
have bmssp-post finite-initial-label {s} Infinity d' Infinity U
  using bmssp-post-full-imp-post[OF post-full] .
then have U-V:  $U = V$ 
  using bound-tree-source-infinity[OF all-reachable]
  unfolding bmssp-post-def by auto
have complete: complete-on d' U
  using  $\langle \text{bmssp-post finite-initial-label } \{s\} \text{ Infinity } d' \text{ Infinity } U \rangle$ 
  unfolding bmssp-post-def by auto
then show ?thesis
  using U-V unfolding complete-on-def sssp-correct-def by auto
qed

theorem finite-initial-label-concrete-capped-top-level-correct:
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
  and step: concrete-capped-bmssp-step k cap finite-initial-label {s} Infinity a bs
    d' Infinity Us U
  shows sssp-correct d'
proof –
  have pre: bmssp-pre-full finite-initial-label {s} Infinity
    using all-reachable finite-initial-label-source-complete
    by (rule top-bmssp-pre-full)
  have sound: sound-label finite-initial-label
    using finite-initial-label-sound[OF all-reachable] .
  have S-reaches:  $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$ 
    using all-reachable source-in-V by blast
  have post-full:
    bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
    by (rule concrete-capped-bmssp-step-correct[OF sound pre S-reaches step])
  have bmssp-post finite-initial-label {s} Infinity d' Infinity U
    using bmssp-post-full-imp-post[OF post-full] .
  then have U-V:  $U = V$ 
    using bound-tree-source-infinity[OF all-reachable]
    unfolding bmssp-post-def by auto
  have complete: complete-on d' U
    using  $\langle \text{bmssp-post finite-initial-label } \{s\} \text{ Infinity } d' \text{ Infinity } U \rangle$ 
    unfolding bmssp-post-def by auto
  then show ?thesis
    using U-V unfolding complete-on-def sssp-correct-def by auto
qed

end

end
theory BMSSP-Recursive

```

imports *BMSSP-Base-Case BMSSP-Concrete-Top-Level*
begin

26 Level-Indexed BMSSP Relation

This theory introduces the first level-indexed BMSSP relation. Earlier theories prove a base case and a concrete non-base step in isolation. The inductive relation below is the small bridge that turns those one-step facts into a recursive BMSSP run indexed by a natural recursion level.

The relation is deliberately simple. At level zero it uses the checked base case for a singleton source set. At a successor level it accepts any concrete capped BMSSP step assembled from FindPivots and a monotone partition trace. This file does not yet model the operational generation of every pull and child call; that more detailed relation is proved separately in *BMSSP-Operational-Pull*. Here the purpose is to show that the base/step decomposition is already enough for semantic correctness.

The main theorem *concrete-bmssp-correct* below is an induction over the level-indexed relation. The base rule delegates to the base-case postcondition, and the step rule delegates to the concrete capped step theorem. The final theorem specializes the relation to the root source and infinite bound, obtaining ordinary single-source shortest-path correctness for the finite initial label.

context *unique-shortest-digraph*
begin

inductive *concrete-bmssp* **where**

Base:

$S = \{x\} \implies$
concrete-bmssp $k \text{ cap } 0 \ d \ S \ B$
 $(\lambda v. \text{ if } v \in \text{base-case-vertices } k \ x \ B \text{ then } \text{dist } s \ v \text{ else } d \ v)$
 $(\text{base-case-bound } k \ x \ B)$
 $(\text{base-case-vertices } k \ x \ B)$

| *Step:*

concrete-capped-bmssp-step $k \text{ cap } d \ S \ B \ a \ bs \ d' \ B' \ Us \ U \implies$
concrete-bmssp $k \text{ cap } (\text{Suc } l) \ d \ S \ B \ d' \ B' \ U$

The two rules of *concrete-bmssp* mirror the recursive structure of the paper. The base rule solves the bounded region reachable from one source by the base-case routine. The successor rule abstracts over the child runs already summarized by *concrete-capped-bmssp-step*. This keeps the induction proof short and makes clear which theorem is responsible for each layer of the algorithm.

lemma *bmssp-post-imp-post-full:*

assumes *bmssp-post* $d \ S \ B \ d' \ B' \ U$

shows *bmssp-post-full* $d \ S \ B \ d' \ B' \ U$

using *assms* **unfolding** *bmssp-post-def* *bmssp-post-full-def* **by** *blast*

theorem *concrete-bmssp-correct*:
assumes *sound*: *sound-label* *d*
and *pre*: *bmssp-pre-full* *d S B*
and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
and *run*: *concrete-bmssp* *k cap l d S B d' B' U*
shows *bmssp-post-full* *d S B d' B' U*
using *run sound pre S-reaches*
proof (*induction arbitrary*; *rule*: *concrete-bmssp.induct*)
case (*Base* *S x k cap d B*)
have *post*:
bmssp-post *d S B*
 $(\lambda v. \text{if } v \in \text{base-case-vertices } k \ x \ B \text{ then } \text{dist } s \ v \text{ else } d \ v)$
(base-case-bound *k x B*)
(base-case-vertices *k x B*)
using *base-case-result-bmssp-post* [*OF Base.hyps*, **where** *k = k* **and** *B = B* **and**
d = d]
unfolding *base-case-result-def* **by** *simp*
then show *?case*
by (*rule* *bmssp-post-imp-post-full*)
next
case (*Step* *k cap d S B a bs d' B' Us U l*)
show *?case*
by (*rule* *concrete-capped-bmssp-step-correct*
 $[OF \text{ Step.prem } (1) \text{ Step.prem } (2) \text{ Step.prem } (3) \text{ Step.hyps}]$)
qed

The proof of $\llbracket \text{sound-label } ?d; \text{bmssp-pre-full } ?d \ ?S \ ?B; \bigwedge x. x \in ?S \implies \text{reachable } s \ x; \text{concrete-bmssp } ?k \ ?cap \ ?l \ ?d \ ?S \ ?B \ ?d' \ ?B' \ ?U \rrbracket \implies \text{bmssp-post-full } ?d \ ?S \ ?B \ ?d' \ ?B' \ ?U$ is intentionally direct. All shortest-path reasoning has already been packaged in the base-case and concrete-step theories, so the induction only selects the appropriate correctness theorem for each rule. The remaining top-level theorem then supplies the standard root precondition, soundness of the initial label, and reachability of the singleton source set.

theorem *finite-initial-label-recursive-top-level-correct*:
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *run*: *concrete-bmssp* *k cap l finite-initial-label {s} Infinity d' Infinity U*
shows *sssp-correct* *d'*
proof –
have *pre*: *bmssp-pre-full* *finite-initial-label {s} Infinity*
using *all-reachable finite-initial-label-source-complete*
by (*rule* *top-bmssp-pre-full*)
have *sound*: *sound-label* *finite-initial-label*
using *finite-initial-label-sound* [*OF all-reachable*] .
have *S-reaches*: $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$
using *all-reachable source-in-V* **by** *blast*
have *post-full*:

```

    bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
  by (rule concrete-bmssp-correct[OF sound pre S-reaches run])
  have post: bmssp-post finite-initial-label {s} Infinity d' Infinity U
    using bmssp-post-full-imp-post[OF post-full] .
  then have U-V: U = V
    using bound-tree-source-infinity[OF all-reachable]
    unfolding bmssp-post-def by auto
  have complete: complete-on d' U
    using post unfolding bmssp-post-def by auto
  then show ?thesis
    using U-V unfolding complete-on-def sssp-correct-def by auto
qed

end

end
theory BMSSP-Operational-Pull
  imports BMSSP-Partition-Pull-Bridge BMSSP-Recursive
begin

```

27 Operational Pull Step

This theory is the bridge from the abstract recursive BMSSP specification to an operational loop that explicitly pulls a bounded prefix out of a partition, recurses on that prefix, and then pushes newly discovered labels into the remaining partition. Earlier theories prove the abstract correctness theorem for *concrete-bmssp*; here we expose the loop shape that the later cost relations charge.

The paper's partition loop is not merely a list traversal. Each iteration selects a child source set below a temporary bound, runs BMSSP recursively on that child problem, and then refreshes the parent partition with two kinds of labels: edge relaxations out of the completed child tree and old source labels that now fall into the open range just processed. The predicate *pull-separates* is the abstract statement that the pull operation has found exactly such a prefix and retained an ordered residual partition.

The main proof obligation is a lifting argument. If a child recursive call satisfies *bmssp-post-full* for the split prefix, and if the parent already satisfied *bmssp-pre-full* up to the larger bound, then the child result can be viewed as progress for the parent. The connective tissue is the lower split *split-below*: it identifies the prefix chosen by the data structure with the mathematical set of vertices whose current labels are below the recursive child bound.

Later in the file the same idea is rephrased as inductive operational traces. The traces remember the sequence of child output bounds, the range trees *range-tree* that those bounds cut out, and the final tree assembled from all slices. This is the form needed by the range-synchronised cost theories, where every loop iteration must be charged to the slice it actually completed.

context *unique-shortest-digraph*
begin

theorem *sorted-pull-recursive-child-lifts*:

assumes *sound*: *sound-label* *d*
and *pre*: *bmssp-pre-full* *d S (Fin Bmax)*
and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
and *upper*: $\bigwedge u. u \in S \implies d \ u < Bmax$
and *S'-def*: *S' = sorted-pull-set* *M (label-partition-view d S)*
and *beta-def*: *beta = sorted-pull-bound* *M Bmax (label-partition-view d S)*
and *run*: *concrete-bmssp* *k cap l d S' (Fin beta) d' B' U*
shows *bmssp-post-full* *d S (Fin Bmax) d' B' U*

proof –

have *S-subset*: $S \subseteq V$
using *pre* **unfolding** *bmssp-pre-full-def* **by** *blast*
have *finite-S*: *finite* *S*
using *finite-subset*[*OF S-subset finite-V*] .
let *?D* = *label-partition-view* *d S*
let *?xs* = *partition-key-order* *?D*
have *set-xs*: *set* *?xs = S*
using *partition-key-order-properties*(1)[*of ?D*] *finite-S* **by** *simp*
have *child-pre*: *bmssp-pre-full* *d S' (Fin beta)*
by (*rule sorted-pull-establishes-lower-pre*
[*OF pre upper refl S'-def beta-def*])
have *S'-split*: *S' = split-below* *d S beta*
using *sorted-pull-set-eq-split-below*[*OF finite-S upper refl beta-def*]
unfolding *S'-def* .
have *S'-reaches*: $\bigwedge x. x \in S' \implies \text{reachable } s \ x$
using *S-reaches S'-split* **unfolding** *split-below-def* **by** *blast*
have *child-post*: *bmssp-post-full* *d S' (Fin beta) d' B' U*
by (*rule concrete-bmssp-correct*[*OF sound child-pre S'-reaches run*])
have *beta-le*: $\beta \leq Bmax$
proof (*cases length ?xs ≤ M*)
case *True*
then show *?thesis*
unfolding *beta-def sorted-pull-bound-def* **by** (*simp add: Let-def*)
next
case *False*
then have *M-lt*: $M < \text{length } ?xs$
by *simp*
have *beta-eq*: $\beta = d \ (?xs ! M)$
using *False* **unfolding** *beta-def sorted-pull-bound-def* **by** (*simp add: Let-def*)
have $?xs ! M \in S$
using *set-xs M-lt nth-mem* **by** *metis*
then have $\beta < Bmax$
using *upper beta-eq* **by** *simp*
then show *?thesis*
by *simp*
qed

```

have pre-beta: bmssp-pre-full d S (Fin beta)
  using bmssp-pre-full-bound-mono[OF pre, of Fin beta] beta-le by simp
have cover-beta: complete-tree-cover d S (Fin beta)
  using bmssp-pre-full-complete-tree-cover[OF pre-beta] .
have child-post-split:
  bmssp-post-full d (split-below d S beta) (Fin beta) d' B' U
  using child-post unfolding S'-split .
have U-tree: U = bound-tree S B'
  using pull-recursive-post-lifts-bound-tree[OF cover-beta child-post-split] .
have le-child: bound-le B' (Fin beta)
  using child-post unfolding bmssp-post-full-def by blast
have le-parent: bound-le B' (Fin Bmax)
  using le-child beta-le by (cases B') auto
have complete: complete-on d' U
  using child-post unfolding bmssp-post-full-def by blast
show ?thesis
  using le-parent U-tree complete unfolding bmssp-post-full-def by blast
qed

```

theorem pull-separates-recursive-child-lifts:

```

fixes Bmax :: real
assumes sound: sound-label d
  and pre: bmssp-pre-full d S (Fin Bmax)
  and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
  and upper:  $\bigwedge u. u \in S \implies d \ u < Bmax$ 
  and pull: pull-separates (label-partition-view d S) M Bmax S' beta D'
  and run: concrete-bmssp k cap l d S' (Fin beta) d' B' U
shows bmssp-post-full d S (Fin Bmax) d' B' U
proof -
  have S'-split: S' = split-below d S beta
    using pull-separates-label-set-eq-split-below[OF pull upper] .
  have S'-subset: S'  $\subseteq$  S
    using pull-separates-subset[OF pull] by simp
  have beta-le: beta  $\leq$  Bmax
  proof (cases keys-of D' = {})
    case True
    then show ?thesis
      using pull-separates-empty-bound[OF pull True] by simp
  next
    case False
    then obtain v where vD': v  $\in$  keys-of D'
      by blast
    have D'-keys: keys-of D' = S - S'
      using pull unfolding pull-separates-def by simp
    then have vS: v  $\in$  S
      using vD' by blast
    have beta  $\leq$  value-of D' v
      by (rule pull-separates-nonempty-bound[OF pull False vD'])
    also have ... = d v

```



```

    using pull unfolding pull-separates-def by simp
  also have ... < Bmax
    using upper[OF vS] .
  finally show ?thesis
    by simp
qed
have child-pre: bmssp-pre-full d S' (Fin beta)
proof (cases beta = Bmax)
  case True
    have split-all: split-below d S beta = S
      unfolding split-below-def True using upper by blast
    show ?thesis
      using pre S'-split split-all True by simp
  next
    case False
      then have beta-lt: beta < Bmax
        using beta-le by simp
      have bmssp-pre-full d (split-below d S beta) (Fin beta)
        by (rule pull-minimum-pre-for-lower-split[OF pre]) (simp add: beta-lt)
      then show ?thesis
        using S'-split by simp
    qed
  have S'-reaches:  $\bigwedge x. x \in S' \implies \text{reachable } s \ x$ 
    using S'-subset S-reaches by blast
  have child-post: bmssp-post-full d S' (Fin beta) d' B' U
    by (rule concrete-bmssp-correct[OF sound child-pre S'-reaches run])
  have pre-beta: bmssp-pre-full d S (Fin beta)
proof -
  have bound-le (Fin beta) (Fin Bmax)
    using beta-le by simp
  then show ?thesis
    by (rule bmssp-pre-full-bound-mono[OF pre])
qed
have cover-beta: complete-tree-cover d S (Fin beta)
  using bmssp-pre-full-complete-tree-cover[OF pre-beta] .
have child-post-split:
  bmssp-post-full d (split-below d S beta) (Fin beta) d' B' U
  using child-post S'-split by simp
have U-tree: U = bound-tree S B'
  using pull-recursive-post-lifts-bound-tree[OF cover-beta child-post-split] .
have le-child: bound-le B' (Fin beta)
  using child-post unfolding bmssp-post-full-def by blast
have le-parent: bound-le B' (Fin Bmax)
  using le-child beta-le by (cases B') auto
have complete: complete-on d' U
  using child-post unfolding bmssp-post-full-def by blast
show ?thesis
  using le-parent U-tree complete unfolding bmssp-post-full-def by blast
qed

```

The first two lifting lemmas isolate the correctness content of a pull. The theorem $\llbracket \text{sound-label } ?d; \text{bmssp-pre-full } ?d \text{ } ?S \text{ (Fin } ?B\text{max}); \bigwedge x. x \in ?S \implies \text{reachable } s \text{ } x; \bigwedge u. u \in ?S \implies ?d \text{ } u < ?B\text{max}; ?S' = \text{sorted-pull-set } ?M \text{ (label-partition-view } ?d \text{ } ?S); ?\text{beta} = \text{sorted-pull-bound } ?M \text{ } ?B\text{max} \text{ (label-partition-view } ?d \text{ } ?S); \text{concrete-bmssp } ?k \text{ } ?\text{cap } ?l \text{ } ?d \text{ } ?S' \text{ (Fin } ?\text{beta}) } ?d' \text{ } ?B' \text{ } ?U \rrbracket \implies \text{bmssp-post-full } ?d \text{ } ?S \text{ (Fin } ?B\text{max}) } ?d' \text{ } ?B' \text{ } ?U$ is phrased for the executable sorted-list partition view: the child set and child bound are computed by sorting labels and taking the first M positions. The theorem $\llbracket \text{sound-label } ?d; \text{bmssp-pre-full } ?d \text{ } ?S \text{ (Fin } ?B\text{max}); \bigwedge x. x \in ?S \implies \text{reachable } s \text{ } x; \bigwedge u. u \in ?S \implies ?d \text{ } u < ?B\text{max}; \text{pull-separates (label-partition-view } ?d \text{ } ?S) \text{ } ?M \text{ } ?B\text{max} \text{ } ?S' \text{ } ?\text{beta} \text{ } ?D'; \text{concrete-bmssp } ?k \text{ } ?\text{cap } ?l \text{ } ?d \text{ } ?S' \text{ (Fin } ?\text{beta}) } ?d' \text{ } ?B' \text{ } ?U \rrbracket \implies \text{bmssp-post-full } ?d \text{ } ?S \text{ (Fin } ?B\text{max}) } ?d' \text{ } ?B' \text{ } ?U$ is the abstract version used by the bucketed partition and by the costed loop relations.

Both statements follow the same pattern. The pulled set is shown to equal the mathematical split below the child bound, the child precondition is inherited from the parent precondition, and the child postcondition is lifted through the parent's complete tree cover. The conclusion deliberately reuses *bmssp-post-full*; nothing about the later operational or costed presentation changes the semantic contract of BMSSP.

definition *complete-preserved where*

complete-preserved $d \text{ } d' \text{ } U \longleftrightarrow (\text{complete-on } d \text{ } U \longrightarrow \text{complete-on } d' \text{ } U)$

lemma *relax-edges-complete-preserved:*

assumes *sound: sound-label* d

and *edges:* $\bigwedge u \text{ } v. (u, v) \in \text{set } es \implies (u, v) \in E$

and *reaches:* $\bigwedge u \text{ } v. (u, v) \in \text{set } es \implies \text{reachable } s \text{ } u$

shows *complete-preserved* $d \text{ (relax-edges } d \text{ } es) \text{ } U$

proof –

have *preserve:* $\bigwedge x. d \text{ } x = \text{dist } s \text{ } x \implies \text{relax-edges } d \text{ } es \text{ } x = \text{dist } s \text{ } x$

proof –

fix x

assume *complete-x:* $d \text{ } x = \text{dist } s \text{ } x$

show *relax-edges* $d \text{ } es \text{ } x = \text{dist } s \text{ } x$

by (*rule relax-edges-preserves-complete-sound*

[*OF sound complete-x edges reaches*])

qed

show *?thesis*

unfolding *complete-preserved-def complete-on-def using preserve by blast*

qed

lemma *relax-frontier-complete-preserved:*

assumes *sound: sound-label* d

and *reaches:* $\bigwedge u. u \in F \implies \text{reachable } s \text{ } u$

shows *complete-preserved* $d \text{ (relax-frontier } d \text{ } F) \text{ } U$

proof –

```

have preserve:  $\bigwedge x. d\ x = \text{dist}\ s\ x \implies \text{relax-frontier}\ d\ F\ x = \text{dist}\ s\ x$ 
proof –
  fix x
  assume complete-x:  $d\ x = \text{dist}\ s\ x$ 
  show  $\text{relax-frontier}\ d\ F\ x = \text{dist}\ s\ x$ 
    by (rule relax-frontier-preserves-complete-sound
      [OF sound complete-x reaches])
  qed
  show ?thesis
  unfolding complete-preserved-def complete-on-def using preserve by blast
qed

```

After a child call returns, the parent loop relaxes edges out of the completed child tree and may also reinsert source labels from the pulled set. The small predicate *complete-preserved* packages the only semantic requirement imposed on that refresh: labels already complete on the child slice stay complete after the refresh.

The lemmas $\llbracket \text{sound-label } ?d; \bigwedge u\ v. (u, v) \in \text{set } ?es \implies (u, v) \in E; \bigwedge u\ v. (u, v) \in \text{set } ?es \implies \text{reachable } s\ u \rrbracket \implies \text{complete-preserved } ?d\ (\text{relax-edges } ?d\ ?es)\ ?U$ and $\llbracket \text{sound-label } ?d; \bigwedge u. u \in ?F \implies \text{reachable } s\ u \rrbracket \implies \text{complete-preserved } ?d\ (\text{relax-frontier } ?d\ ?F)\ ?U$ are deliberately local and modest. They do not prove a new shortest-path theorem; they just record that ordinary sound relaxation cannot destroy a vertex whose label has already reached its true distance. This lets the loop proofs compose recursive completion with parent-side batch updates without reopening the relaxation algebra every time.

definition *edge-relaxation-pairs-between* **where**

```

edge-relaxation-pairs-between d U L H =
  map ( $\lambda(u, v). (v, d\ u + w\ u\ v)$ )
    (filter ( $\lambda(u, v). L \leq d\ u + w\ u\ v \wedge d\ u + w\ u\ v < H$ )
      (edge-list-of (outgoing-edges U))))

```

definition *label-pairs-between* **where**

```

label-pairs-between d S L H =
  map ( $\lambda x. (x, d\ x)$ )
    (filter ( $\lambda x. L \leq d\ x \wedge d\ x < H$ )
      (partition-key-order (label-partition-view d S))))

```

lemma *edge-relaxation-pairs-between-value-le-high*:

```

assumes  $(x, b) \in \text{set } (\text{edge-relaxation-pairs-between } d\ U\ L\ H)$ 
shows  $b \leq H$ 
using assms unfolding edge-relaxation-pairs-between-def
by (auto split: prod.splits)

```

lemma *edge-relaxation-pairs-between-value-ge-low*:

```

assumes  $(x, b) \in \text{set } (\text{edge-relaxation-pairs-between } d\ U\ L\ H)$ 
shows  $L \leq b$ 
using assms unfolding edge-relaxation-pairs-between-def

```

```

by (auto split: prod.splits)

lemma label-pairs-between-value-le-high:
  assumes  $(x, b) \in \text{set } (\text{label-pairs-between } d \ S \ L \ H)$ 
  shows  $b \leq H$ 
  using assms unfolding label-pairs-between-def by auto

lemma label-pairs-between-value-ge-low:
  assumes  $(x, b) \in \text{set } (\text{label-pairs-between } d \ S \ L \ H)$ 
  shows  $L \leq b$ 
  using assms unfolding label-pairs-between-def by auto

lemma edge-relaxation-pairs-between-length-le-outgoing:
  length (edge-relaxation-pairs-between  $d \ U \ L \ H$ )  $\leq$ 
  card (outgoing-edges  $U$ )
proof -
  let ?es = edge-list-of (outgoing-edges  $U$ )
  have distinct-es: distinct ?es
    using edge-list-of-properties(2)[OF finite-outgoing-edges] .
  have length-es: length ?es = card (outgoing-edges  $U$ )
    using edge-list-of-properties(1)[OF finite-outgoing-edges] distinct-es
    by (metis distinct-card)
  have length (edge-relaxation-pairs-between  $d \ U \ L \ H$ )  $\leq$  length ?es
    unfolding edge-relaxation-pairs-between-def by simp
  then show ?thesis
    using length-es by simp
qed

lemma label-pairs-between-length-le-card:
  assumes finite-S: finite  $S$ 
  shows length (label-pairs-between  $d \ S \ L \ H$ )  $\leq$  card  $S$ 
proof -
  let ?xs = partition-key-order (label-partition-view  $d \ S$ )
  have set-xs: set ?xs =  $S$ 
    using partition-key-order-properties(1)[of label-partition-view  $d \ S$ ]
    finite-S by simp
  have distinct-xs: distinct ?xs
    using partition-key-order-properties(2)[of label-partition-view  $d \ S$ ]
    finite-S by simp
  have length-xs: length ?xs = card  $S$ 
    using set-xs distinct-xs by (metis distinct-card)
  have length (label-pairs-between  $d \ S \ L \ H$ )  $\leq$  length ?xs
    unfolding label-pairs-between-def by simp
  then show ?thesis
    using length-xs by simp
qed

theorem pull-separates-batch-prepend-for-relaxation-pairs:
  assumes pull: pull-separates  $D \ M \ B \ S$ -pull beta  $D'$ 

```

```

shows batch-prepend-admissible  $D'$ 
  (edge-relaxation-pairs-between  $d \ U \ L \ \text{beta}$  @ label-pairs-between  $d \ S \ L \ \text{beta}$ )
proof (rule pull-separates-batch-prepend-admissible[OF pull])
  fix  $x \ b$ 
  assume  $(x, b) \in \text{set}$ 
  (edge-relaxation-pairs-between  $d \ U \ L \ \text{beta}$  @ label-pairs-between  $d \ S \ L \ \text{beta}$ )
  then show  $b \leq \text{beta}$ 
    using edge-relaxation-pairs-between-value-le-high
      label-pairs-between-value-le-high by auto
qed

```

The two batch constructors separate the sources of new partition entries. *edge-relaxation-pairs-between* enumerates outgoing child-tree edges whose relaxed values lie in the current open range, while *label-pairs-between* keeps old source labels that also lie in that range. The length lemmas above connect these lists to the quantities charged later: outgoing edges for the first list and source cardinality for the second.

The admissibility theorem *pull-separates* $?D \ ?M \ ?B \ ?S\text{-pull} \ ?\text{beta} \ ?D' \implies \text{batch-prepend-admissible } ?D' \ (\text{edge-relaxation-pairs-between } ?d \ ?U \ ?L \ ?\text{beta} \ @ \ \text{label-pairs-between } ?d \ ?S \ ?L \ ?\text{beta})$ is the key local invariant for pushing the batch into the residual partition. It says that every generated value is still below the bound exposed by the pull, so the residual ordered partition can accept the batch without invalidating the next pull.

```

lemma range-tree-same-empty [simp]:
  range-tree  $S \ a \ (\text{Fin } a) = \{\}$ 
  unfolding range-tree-def by auto

```

The relation below is the first operational presentation of the parent loop. It abstracts away the concrete partition representation, but it records the same sequence of events as the algorithm: stop when the lower bound has already produced enough completed vertices, otherwise recurse on the split prefix, preserve completion through the parent refresh, and continue from the child output bound.

The lists *betas*, *bs*, and *Us* are not auxiliary clutter. They are the trace that later cost proofs consume. The child output bounds *bs* determine adjacent range slices, and each stored set in *Us* is identified with the corresponding *range-tree*. Proving this trace property once keeps the later runtime arguments independent of the details of the recursive correctness proof.

inductive *operational-partition-loop* **where**

Done:

```

  bound-le  $(\text{Fin } a) \ B \implies$ 
    complete-on  $d' \ (\text{bound-tree } P \ (\text{Fin } a)) \implies$ 
      operational-partition-loop  $k \ \text{cap } l \ d \ P \ B \ d' \ a \ [] \ [] \ (\text{Fin } a)$ 
        [range-tree  $P \ a \ (\text{Fin } a)$ ]
        ( $\text{bound-tree } P \ (\text{Fin } a) \cup \bigcup (\text{set } [\text{range-tree } P \ a \ (\text{Fin } a)])$ )

```

| *Step*:

```

  below-bound  $\text{beta} \ B \implies$ 

```

$a \leq b \implies$
complete-on d' (*bound-tree* P (*Fin* a)) $\implies$
concrete-bmssp k *cap* l d (*split-below* d P β) (*Fin* β)
d-child (*Fin* b) *U-child* $\implies$
complete-preserved *d-child* d' *U-child* $\implies$
operational-partition-loop k *cap* l d P B d' b β bs B' *Us-tail* *U-tail* $\implies$
operational-partition-loop k *cap* l d P B d' a ($\beta \# \beta$) ($b \# bs$) B'
(*range-tree* P a (*Fin* b) $\#$ *Us-tail*)
(*bound-tree* P (*Fin* a) $\cup \bigcup$ (*set* (*range-tree* P a (*Fin* b) $\#$ *Us-tail*)))

theorem *operational-partition-loop-trace*:

assumes *sound*: *sound-label* d
and *pre*: *bmssp-pre-full* d P B
and *P-reaches*: $\bigwedge x. x \in P \implies \text{reachable } s \ x$
and *loop*: *operational-partition-loop* k *cap* l d P B d' a β bs B' *Us* U
shows *concrete-partition-loop-trace* P B a bs d' B' *Us* U
using *loop* *sound* *pre* *P-reaches*
proof (*induction*)
case (*Done* a B d' P k *cap* l d)
then show ?*case*
unfolding *concrete-partition-loop-trace-def* *complete-on-def* **by** *simp*
next
case (*Step* β B a b d' P k *cap* l d *d-child* *U-child* β bs B' *Us-tail* *U-tail*)
have *beta-le*: *bound-le* (*Fin* β) B
using *Step.hyps*(1) **by** (*cases* B) *auto*
have *pre-beta*: *bmssp-pre-full* d P (*Fin* β)
using *bmssp-pre-full-bound-mono*[*OF* *Step.prem*s(2) *beta-le*] .
have *child-pre*: *bmssp-pre-full* d (*split-below* d P β) (*Fin* β)
using *pull-minimum-pre-for-lower-split*[*OF* *Step.prem*s(2) *Step.hyps*(1)] .
have *child-reaches*:
 $\bigwedge x. x \in \text{split-below } d \ P \ \beta \implies \text{reachable } s \ x$
using *Step.prem*s(3) **unfolding** *split-below-def* **by** *blast*
have *child-post*:
bmssp-post-full d (*split-below* d P β) (*Fin* β)
d-child (*Fin* b) *U-child*
by (*rule* *concrete-bmssp-correct*[*OF* *Step.prem*s(1) *child-pre* *child-reaches* *Step.hyps*(4)])
have *cover-beta*: *complete-tree-cover* d P (*Fin* β)
using *bmssp-pre-full-complete-tree-cover*[*OF* *pre-beta*] .
have *U-child-tree*: *U-child* = *bound-tree* P (*Fin* b)
using *pull-recursive-post-lifts-bound-tree*[*OF* *cover-beta* *child-post*] .
have *child-complete*: *complete-on* *d-child* *U-child*
using *child-post* **unfolding** *bmssp-post-full-def* **by** *blast*
have *child-complete-final*: *complete-on* d' *U-child*
using *Step.hyps*(5) *child-complete* **unfolding** *complete-preserved-def* **by** *blast*
have *head-complete*: *complete-on* d' (*range-tree* P a (*Fin* b))
proof –
have *range-tree* P a (*Fin* b) $\subseteq$ *U-child*
using *U-child-tree* *range-tree-subset-bound-tree*[*of* P a *Fin* b] **by** *simp*

```

    then show ?thesis
      using complete-on-subset[OF child-complete-final] by blast
qed
have tail-trace:
  concrete-partition-loop-trace P B b bs d' B' Us-tail U-tail
  using Step.IH[OF Step.prem1 Step.prem2 Step.prem3] .
have tail-le: bound-le B' B
  and tail-mono: nondecreasing-from b bs
  and tail-bounds: bounds-le B' (b # bs)
  and tail-children:
    list-all2 ( $\lambda U X. U = X \wedge \text{complete-on } d' U$ ) Us-tail
    (range-tree-chain-list P b bs B')
  using tail-trace unfolding concrete-partition-loop-trace-def by blast+
have a-bound: bound-le (Fin a) B'
proof -
  have bound-le (Fin b) B'
    using tail-bounds by simp
  then show ?thesis
    using Step.hyps(2) by (cases B') auto
qed
show ?case
  unfolding concrete-partition-loop-trace-def
  using tail-le Step.hyps(2) tail-mono tail-bounds a-bound
  Step.hyps(3) head-complete tail-children
  by auto
qed

definition operational-capped-bmssp-step where
  operational-capped-bmssp-step k cap l d S B a betas bs d' B' Us U  $\longleftrightarrow$ 
    (let d-fp = find-pivots-label-capped k cap d S B;
     P = find-pivots-pivots-capped k cap d S B;
     W = {v  $\in$  bound-tree S B'. d-fp v = dist s v}
    in  $\exists$  U-loop.
      operational-partition-loop k cap l d-fp P B d' a betas bs B' Us U-loop  $\wedge$ 
      complete-on d' W  $\wedge$ 
      U = U-loop  $\cup$  W)

theorem operational-capped-bmssp-step-correct:
  assumes sound: sound-label d
  and pre: bmssp-pre-full d S B
  and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s x$ 
  and step: operational-capped-bmssp-step k cap l d S B a betas bs d' B' Us U
  shows bmssp-post-full d S B d' B' U
proof -
  let ?d-fp = find-pivots-label-capped k cap d S B
  let ?P = find-pivots-pivots-capped k cap d S B
  let ?W = {v  $\in$  bound-tree S B'. ?d-fp v = dist s v}
  obtain U-loop where
    loop: operational-partition-loop k cap l ?d-fp ?P B d' a betas bs B' Us U-loop

```

```

and compW: complete-on d' ?W
and U: U = U-loop  $\cup$  ?W
using step unfolding operational-capped-bmssp-step-def by (auto simp: Let-def)
have sound-fp: sound-label ?d-fp
  unfolding find-pivots-label-capped-def
  by (rule fp-iter-capped-label-sound[OF sound S-reaches])
have pivot-pre: bmssp-pre-full ?d-fp ?P B
  using find-pivots-capped-establishes-pivot-pre-concrete[OF sound pre S-reaches] .
have P-subset-S: ?P  $\subseteq$  S
  unfolding find-pivots-pivots-capped-def by auto
have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
  using P-subset-S S-reaches by blast
have trace: concrete-partition-loop-trace ?P B a bs d' B' Us U-loop
  by (rule operational-partition-loop-trace[OF sound-fp pivot-pre P-reaches loop])
have concrete-step:
  concrete-capped-bmssp-step k cap d S B a bs d' B' Us U
  unfolding concrete-capped-bmssp-step-def
  using trace compW U by (auto simp: Let-def)
show ?thesis
  by (rule concrete-capped-bmssp-step-correct
    [OF sound pre S-reaches concrete-step])
qed

```

inductive operational-bmssp **where**

Base:

$S = \{x\} \implies$
 operational-bmssp k cap 0 d S B
 ($\lambda v. \text{if } v \in \text{base-case-vertices } k \ x \ B \text{ then } \text{dist } s \ v \text{ else } d \ v$)
 (base-case-bound k x B)
 (base-case-vertices k x B)

| *Step:*

operational-capped-bmssp-step k cap l d S B a betas bs d' B' Us U $\implies$
 operational-bmssp k cap (Suc l) d S B d' B' U

theorem operational-bmssp-correct:

assumes sound: sound-label d
and pre: bmssp-pre-full d S B
and S-reaches: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
and run: operational-bmssp k cap l d S B d' B' U
shows bmssp-post-full d S B d' B' U
using run sound pre S-reaches

proof (induction arbitrary: rule: operational-bmssp.induct)

case (Base S x k cap d B)

have post:

bmssp-post d S B
 ($\lambda v. \text{if } v \in \text{base-case-vertices } k \ x \ B \text{ then } \text{dist } s \ v \text{ else } d \ v$)
 (base-case-bound k x B)
 (base-case-vertices k x B)

using base-case-result-bmssp-post[OF Base.hyps, **where** k = k **and** B = B **and**


```

d = d]
  unfolding base-case-result-def by simp
  then show ?case
    by (rule bmssp-post-imp-post-full)
next
case (Step k cap l d S B a betas bs d' B' Us U)
show ?case
  by (rule operational-capped-bmssp-step-correct
    [OF Step.premis(1) Step.premis(2) Step.premis(3) Step.hyps])
qed

theorem finite-initial-label-operational-top-level-correct:
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
  and run: operational-bmssp k cap l finite-initial-label {s} Infinity d' Infinity U
  shows sssp-correct d'
proof -
  have pre: bmssp-pre-full finite-initial-label {s} Infinity
    using all-reachable finite-initial-label-source-complete
    by (rule top-bmssp-pre-full)
  have sound: sound-label finite-initial-label
    using finite-initial-label-sound[OF all-reachable] .
  have S-reaches:  $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$ 
    using all-reachable source-in-V by blast
  have post-full:
    bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
    by (rule operational-bmssp-correct[OF sound pre S-reaches run])
  have post: bmssp-post finite-initial-label {s} Infinity d' Infinity U
    using bmssp-post-full-imp-post[OF post-full] .
  then have U-V:  $U = V$ 
    using bound-tree-source-infinity[OF all-reachable]
    unfolding bmssp-post-def by auto
  have complete: complete-on d' U
    using post unfolding bmssp-post-def by auto
  then show ?thesis
    using U-V unfolding complete-on-def sssp-correct-def by auto
qed

```

The plain operational relation still calls *concrete-bmssp* in each loop step. The full relation below unfolds that recursive call as another inductive branch. This mutual presentation is more verbose, but it gives the cost layer a single derivation tree whose nodes are either BMSSP calls or partition-loop iterations.

The theorem $\llbracket \bigwedge v. v \in V \implies \text{reachable } s \ v; \text{operational-bmssp } ?k \ ?cap \ ?l \text{ finite-initial-label } \{s\} \text{ Infinity } ?d' \text{ Infinity } ?U \rrbracket \implies \text{sssp-correct } ?d'$ remains the semantic endpoint for this layer: starting from the finite initial label at the source and an infinite top-level bound, any operational run computes a full single-source shortest-path labelling. The later relations refine this one by adding costs and range synchronisation; they do not change this

correctness statement.

inductive *full-operational-partition-loop* **and** *full-operational-bmssp* **where**

Full-Loop-Done:

$\text{bound-le } (\text{Fin } a) \ B \implies$
 $\text{complete-on } d' \ (\text{bound-tree } P \ (\text{Fin } a)) \implies$
 $\text{full-operational-partition-loop } k \ \text{cap } l \ d \ P \ B \ d' \ a \ [] \ [] \ (\text{Fin } a)$
 $[\text{range-tree } P \ a \ (\text{Fin } a)]$
 $(\text{bound-tree } P \ (\text{Fin } a) \cup \bigcup (\text{set } [\text{range-tree } P \ a \ (\text{Fin } a)]))$

| *Full-Loop-Step:*

$\text{below-bound } \text{beta} \ B \implies$
 $a \leq b \implies$
 $\text{complete-on } d' \ (\text{bound-tree } P \ (\text{Fin } a)) \implies$
 $\text{full-operational-bmssp } k \ \text{cap } l \ d \ (\text{split-below } d \ P \ \text{beta}) \ (\text{Fin } \text{beta})$
 $d\text{-child } (\text{Fin } b) \ U\text{-child} \implies$
 $\text{complete-preserved } d\text{-child } d' \ U\text{-child} \implies$
 $\text{full-operational-partition-loop } k \ \text{cap } l \ d \ P \ B \ d' \ b \ \text{betas } bs \ B' \ Us\text{-tail } U\text{-tail} \implies$
 $\text{full-operational-partition-loop } k \ \text{cap } l \ d \ P \ B \ d' \ a \ (\text{beta} \# \text{betas}) \ (b \# bs) \ B'$
 $(\text{range-tree } P \ a \ (\text{Fin } b) \# Us\text{-tail})$
 $(\text{bound-tree } P \ (\text{Fin } a) \cup \bigcup (\text{set } (\text{range-tree } P \ a \ (\text{Fin } b) \# Us\text{-tail})))$

| *Full-Loop-Step-Pre:*

$\text{bound-le } (\text{Fin } \text{beta}) \ B \implies$
 $\text{bmssp-pre-full } d \ (\text{split-below } d \ P \ \text{beta}) \ (\text{Fin } \text{beta}) \implies$
 $a \leq b \implies$
 $\text{complete-on } d' \ (\text{bound-tree } P \ (\text{Fin } a)) \implies$
 $\text{full-operational-bmssp } k \ \text{cap } l \ d \ (\text{split-below } d \ P \ \text{beta}) \ (\text{Fin } \text{beta})$
 $d\text{-child } (\text{Fin } b) \ U\text{-child} \implies$
 $\text{complete-preserved } d\text{-child } d' \ U\text{-child} \implies$
 $\text{full-operational-partition-loop } k \ \text{cap } l \ d \ P \ B \ d' \ b \ \text{betas } bs \ B' \ Us\text{-tail } U\text{-tail} \implies$
 $\text{full-operational-partition-loop } k \ \text{cap } l \ d \ P \ B \ d' \ a \ (\text{beta} \# \text{betas}) \ (b \# bs) \ B'$
 $(\text{range-tree } P \ a \ (\text{Fin } b) \# Us\text{-tail})$
 $(\text{bound-tree } P \ (\text{Fin } a) \cup \bigcup (\text{set } (\text{range-tree } P \ a \ (\text{Fin } b) \# Us\text{-tail})))$

| *Full-Base:*

$S = \{x\} \implies$
 $\text{full-operational-bmssp } k \ \text{cap } 0 \ d \ S \ B$
 $(\lambda v. \text{ if } v \in \text{base-case-vertices } k \ x \ B \text{ then } \text{dist } s \ v \text{ else } d \ v)$
 $(\text{base-case-bound } k \ x \ B)$
 $(\text{base-case-vertices } k \ x \ B)$

| *Full-Step:*

$\text{full-operational-partition-loop } k \ \text{cap } l$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $\text{full-operational-bmssp } k \ \text{cap } (\text{Suc } l) \ d \ S \ B \ d' \ B' \ U$

theorem *full-operational-partition-loop-trace:*

$\text{full-operational-partition-loop } k \ \text{cap } l \ d \ P \ B \ d' \ a \ \text{betas } bs \ B' \ Us \ U \implies$

$\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ P \ B \implies$
 $(\bigwedge x. x \in P \implies \text{reachable } s \ x) \implies$
 $\text{concrete-partition-loop-trace } P \ B \ a \ bs \ d' \ B' \ Us \ U$
and $\text{full-operational-bmssp-correct:}$
 $\text{full-operational-bmssp } k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{bmssp-post-full } d \ S \ B \ d' \ B' \ U$
proof (*induction rule: full-operational-partition-loop-full-operational-bmssp.inducts*)
case ($\text{Full-Loop-Done } a \ B \ d' \ P \ k \ \text{cap } l \ d$)
then show $?case$
unfolding $\text{concrete-partition-loop-trace-def complete-on-def}$ **by** simp
next
case ($\text{Full-Loop-Step } \beta \ B \ a \ b \ d' \ P \ k \ \text{cap } l \ d \ d\text{-child } U\text{-child } \beta \text{as } bs \ B' \ Us\text{-tail}$
 $U\text{-tail}$)
have $\beta \text{-le: bound-le } (\text{Fin } \beta) \ B$
using $\text{Full-Loop-Step}(1)$ **by** ($\text{cases } B$) auto
have $\text{pre-}\beta \text{: bmssp-pre-full } d \ P \ (\text{Fin } \beta)$
using $\text{bmssp-pre-full-bound-mono}[OF \ \text{Full-Loop-Step.prem}(2) \ \beta \text{-le}]$.
have $\text{child-pre: bmssp-pre-full } d \ (\text{split-below } d \ P \ \beta) \ (\text{Fin } \beta)$
using $\text{pull-minimum-pre-for-lower-split}$
 $[OF \ \text{Full-Loop-Step.prem}(2) \ \text{Full-Loop-Step}(1)]$.
have child-reaches:
 $\bigwedge x. x \in \text{split-below } d \ P \ \beta \implies \text{reachable } s \ x$
using $\text{Full-Loop-Step.prem}(3)$ **unfolding** split-below-def **by** blast
have child-post:
 $\text{bmssp-post-full } d \ (\text{split-below } d \ P \ \beta) \ (\text{Fin } \beta)$
 $d\text{-child } (\text{Fin } b) \ U\text{-child}$
using $\text{Full-Loop-Step.IH } \text{Full-Loop-Step.prem}(1) \ \text{child-pre } \text{child-reaches}$
by blast
have $\text{cover-}\beta \text{: complete-tree-cover } d \ P \ (\text{Fin } \beta)$
using $\text{bmssp-pre-full-complete-tree-cover}[OF \ \text{pre-}\beta]$.
have $U\text{-child-tree: } U\text{-child} = \text{bound-tree } P \ (\text{Fin } b)$
using $\text{pull-recursive-post-lifts-bound-tree}[OF \ \text{cover-}\beta \ \text{child-post}]$.
have $\text{child-complete: complete-on } d\text{-child } U\text{-child}$
using child-post **unfolding** $\text{bmssp-post-full-def}$ **by** blast
have $\text{child-complete-final: complete-on } d' \ U\text{-child}$
using $\text{Full-Loop-Step } \text{child-complete}$
unfolding $\text{complete-preserved-def}$ **by** blast
have $\text{head-complete: complete-on } d' \ (\text{range-tree } P \ a \ (\text{Fin } b))$
proof –
have $\text{range-tree } P \ a \ (\text{Fin } b) \subseteq U\text{-child}$
using $U\text{-child-tree } \text{range-tree-subset-bound-tree}[of \ P \ a \ \text{Fin } b]$ **by** simp
then show $?thesis$
using $\text{complete-on-subset}[OF \ \text{child-complete-final}]$ **by** blast
qed
have tail-trace:

```

    concrete-partition-loop-trace  $P$   $B$   $b$   $bs$   $d'$   $B'$   $Us$ -tail  $U$ -tail
  using Full-Loop-Step.IH Full-Loop-Step.premis by blast
have tail-le: bound-le  $B'$   $B$ 
and tail-mono: nondecreasing-from  $b$   $bs$ 
and tail-bounds: bounds-le  $B'$  ( $b \neq bs$ )
and tail-children:
  list-all2 ( $\lambda U X. U = X \wedge \text{complete-on } d' U$ )  $Us$ -tail
  (range-tree-chain-list  $P$   $b$   $bs$   $B'$ )
  using tail-trace unfolding concrete-partition-loop-trace-def by blast+
have a-bound: bound-le ( $Fin$   $a$ )  $B'$ 
proof -
  have bound-le ( $Fin$   $b$ )  $B'$ 
  using tail-bounds by simp
  then show ?thesis
  using Full-Loop-Step(2) by (cases  $B'$ ) auto
qed
show ?case
  unfolding concrete-partition-loop-trace-def
  using tail-le Full-Loop-Step(2) tail-mono tail-bounds a-bound
  Full-Loop-Step(3) head-complete tail-children
  by auto
next
  case (Full-Loop-Step-Pre beta  $B$   $d$   $P$   $a$   $b$   $d'$   $k$  cap  $l$   $d$ -child  $U$ -child betas  $bs$   $B'$ 
   $Us$ -tail  $U$ -tail)
  have beta-le: bound-le ( $Fin$  beta)  $B$ 
  using Full-Loop-Step-Pre by blast
  have a-le-b:  $a \leq b$ 
  using Full-Loop-Step-Pre by blast
  have lower-complete: complete-on  $d'$  (bound-tree  $P$  ( $Fin$   $a$ ))
  using Full-Loop-Step-Pre by blast
  have pre-beta: bmssp-pre-full  $d$   $P$  ( $Fin$  beta)
  using bmssp-pre-full-bound-mono
  [OF Full-Loop-Step-Pre.premis(2) beta-le] .
  have child-pre: bmssp-pre-full  $d$  (split-below  $d$   $P$  beta) ( $Fin$  beta)
  using Full-Loop-Step-Pre by blast
  have child-reaches:
     $\bigwedge x. x \in \text{split-below } d P \text{ beta} \implies \text{reachable } s x$ 
  using Full-Loop-Step-Pre.premis(3) unfolding split-below-def by blast
  have child-post:
    bmssp-post-full  $d$  (split-below  $d$   $P$  beta) ( $Fin$  beta)
     $d$ -child ( $Fin$   $b$ )  $U$ -child
  using Full-Loop-Step-Pre.IH Full-Loop-Step-Pre.premis(1) child-pre child-reaches
  by blast
  have cover-beta: complete-tree-cover  $d$   $P$  ( $Fin$  beta)
  using bmssp-pre-full-complete-tree-cover[OF pre-beta] .
  have  $U$ -child-tree:  $U$ -child = bound-tree  $P$  ( $Fin$   $b$ )
  using pull-recursive-post-lifts-bound-tree[OF cover-beta child-post] .
  have child-complete: complete-on  $d$ -child  $U$ -child
  using child-post unfolding bmssp-post-full-def by blast

```

```

have child-complete-final: complete-on d' U-child
  using Full-Loop-Step-Pre child-complete
  unfolding complete-preserved-def by blast
have head-complete: complete-on d' (range-tree P a (Fin b))
proof -
  have range-tree P a (Fin b)  $\subseteq$  U-child
  using U-child-tree range-tree-subset-bound-tree[of P a Fin b] by simp
  then show ?thesis
  using complete-on-subset[OF child-complete-final] by blast
qed
have tail-trace:
  concrete-partition-loop-trace P B b bs d' B' Us-tail U-tail
  using Full-Loop-Step-Pre.IH Full-Loop-Step-Pre.premis by blast
have tail-le: bound-le B' B
  and tail-mono: nondecreasing-from b bs
  and tail-bounds: bounds-le B' (b # bs)
  and tail-children:
    list-all2 ( $\lambda U X. U = X \wedge \text{complete-on } d' U$ ) Us-tail
    (range-tree-chain-list P b bs B')
  using tail-trace unfolding concrete-partition-loop-trace-def by blast+
have a-bound: bound-le (Fin a) B'
proof -
  have bound-le (Fin b) B'
  using tail-bounds by simp
  then show ?thesis
  using a-le-b by (cases B') auto
qed
show ?case
  unfolding concrete-partition-loop-trace-def
  using tail-le a-le-b tail-mono tail-bounds a-bound
  lower-complete head-complete tail-children
  by auto
next
case (Full-Base S x k cap d B)
have post:
  bmssp-post d S B
  ( $\lambda v. \text{if } v \in \text{base-case-vertices } k \ x \ B \text{ then } \text{dist } s \ v \text{ else } d \ v$ )
  (base-case-bound k x B)
  (base-case-vertices k x B)
  using base-case-result-bmssp-post[OF Full-Base.hyps, where k = k and B = B
and d = d]
  unfolding base-case-result-def by simp
  then show ?case
  by (rule bmssp-post-imp-post-full)
next
case (Full-Step k cap l d S B d' a betas bs B' Us U-loop U)
let ?d-fp = find-pivots-label-capped k cap d S B
let ?P = find-pivots-pivots-capped k cap d S B
let ?W = {v  $\in$  bound-tree S B'. ?d-fp v = dist s v}

```

```

have sound-fp: sound-label ?d-fp
  unfolding find-pivots-label-capped-def
  by (rule fp-iter-capped-label-sound[OF Full-Step.prem(1) Full-Step.prem(3)])
have pivot-pre: bmssp-pre-full ?d-fp ?P B
  using find-pivots-capped-establishes-pivot-pre-concrete
  [OF Full-Step.prem(1) Full-Step.prem(2) Full-Step.prem(3)] .
have P-subset-S: ?P  $\subseteq$  S
  unfolding find-pivots-pivots-capped-def by auto
have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
  using P-subset-S Full-Step.prem(3) by blast
have trace: concrete-partition-loop-trace ?P B a bs d' B' Us U-loop
  using Full-Step.IH sound-fp pivot-pre P-reaches by blast
have concrete-step:
  concrete-capped-bmssp-step k cap d S B a bs d' B' Us U
  unfolding concrete-capped-bmssp-step-def
  using trace Full-Step by (auto simp: Let-def)
show ?case
  by (rule concrete-capped-bmssp-step-correct
    [OF Full-Step.prem(1) Full-Step.prem(2) Full-Step.prem(3) concrete-step])
qed

theorem finite-initial-label-full-operational-top-level-correct:
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
  and run: full-operational-bmssp k cap l finite-initial-label {s} Infinity d' Infinity
  U
  shows sssp-correct d'
proof -
  have pre: bmssp-pre-full finite-initial-label {s} Infinity
    using all-reachable finite-initial-label-source-complete
    by (rule top-bmssp-pre-full)
  have sound: sound-label finite-initial-label
    using finite-initial-label-sound[OF all-reachable] .
  have S-reaches:  $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$ 
    using all-reachable source-in-V by blast
  have post-full:
    bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
    by (rule full-operational-bmssp-correct[OF run sound pre S-reaches])
  have post: bmssp-post finite-initial-label {s} Infinity d' Infinity U
    using bmssp-post-full-imp-post[OF post-full] .
  then have U-V: U = V
    using bound-tree-source-infinity[OF all-reachable]
    unfolding bmssp-post-def by auto
  have complete: complete-on d' U
    using post unfolding bmssp-post-def by auto
  then show ?thesis
    using U-V unfolding complete-on-def sssp-correct-def by auto
qed

inductive full-operational-partition-loop-state where

```

State-Done:
keys-of $D = \{\}$ $\implies$
bound-le (*Fin* a) $B \implies$
complete-on d' (*bound-tree* P (*Fin* a)) $\implies$
full-operational-partition-loop-state M t k $\text{cap } l$ d P B d' D a $\square$ $\square$ (*Fin* a)
[range-tree P a (*Fin* a)]
(*bound-tree* P (*Fin* a) $\cup \bigcup (\text{set } [\text{range-tree } P \ a \ (\text{Fin } a)])$) 0

| *State-Step:*
pull-separates D M $B_{\max}$ S -pull β D -pull $\implies$
bound-le (*Fin* β) $B \implies$
bmssp-pre-full d (*split-below* d P β) (*Fin* β) $\implies$
S-pull = *split-below* d P β $\implies$
 $a \leq b \implies$
complete-on d' (*bound-tree* P (*Fin* a)) $\implies$
full-operational-bmssp k $\text{cap } l$ d S -pull (*Fin* β) d -child (*Fin* b) U -child $\implies$
complete-preserved d -child d' U -child $\implies$
batch =
edge-relaxation-pairs-between d -child U -child b β $@$
label-pairs-between d S -pull b β $\implies$
D-next = *batch-min-update* D -pull *batch* $\implies$
partition-pull-cost-bound c -pull S -pull $\implies$
partition-batch-cost-bound c -batch t *batch* $\implies$
full-operational-partition-loop-state M t k $\text{cap } l$ d P B d' D -next b β bs B'
Us-tail U -tail c -tail $\implies$
 $c = c\text{-pull} + c\text{-batch} + c\text{-child} + c\text{-tail} \implies$
full-operational-partition-loop-state M t k $\text{cap } l$ d P B d' D a (β $\#$ β as) (b
 $\#$ bs) B'
(*range-tree* P a (*Fin* b) $\#$ *Us-tail*)
(*bound-tree* P (*Fin* a) $\cup \bigcup (\text{set } (\text{range-tree } P \ a \ (\text{Fin } b) \ \# \ \text{Us-tail}))$) c

lemma *full-operational-partition-loop-state-lengths:*

assumes *run:*

full-operational-partition-loop-state M t k $\text{cap } l$ d P B d' D a
 β as B' Us U c

shows $\text{length } \beta$ $as = \text{length } bs$

and $\text{length } Us = \text{Suc } (\text{length } bs)$

proof –

have *both*: $\text{length } \beta$ $as = \text{length } bs \wedge \text{length } Us = \text{Suc } (\text{length } bs)$

using *run*

proof (*induction*)

case *State-Done*

then show ?*case* **by** *simp*

next

case (*State-Step* D M $B_{\max}$ S -pull β D -pull B d P a b d' k $\text{cap } l$

d -child U -child *batch* D -next c -pull c -batch t β as B'

Us -tail U -tail c -tail c -child c)

then show ?*case* **by** *simp*

qed

show $\text{length } \beta$ $as = \text{length } bs$

```

    using both by blast
  show length Us = Suc (length bs)
    using both by blast
qed

```

theorem *full-operational-partition-loop-state-refines:*

```

  assumes run:
    full-operational-partition-loop-state M t k cap l d P B d' D a betas bs B' Us U c
  shows full-operational-partition-loop k cap l d P B d' a betas bs B' Us U
  using run
proof (induction)
  case (State-Done D a B d' P M t k cap l d)
  show ?case
    by (rule Full-Loop-Done[OF State-Done.hyps(2,3)])
next
  case (State-Step D M Bmax S-pull beta D-pull B d P a b d' k cap l d-child U-child
    batch D-next c-pull c-batch t betas bs B' Us-tail U-tail c-tail c-step c-total)
  have child-run:
    full-operational-bmssp k cap l d (split-below d P beta) (Fin beta)
    d-child (Fin b) U-child
  using State-Step by blast
  show ?case
    by (rule Full-Loop-Step-Pre)
    (use State-Step child-run in auto)
qed

```

theorem *full-operational-partition-loop-state-cost-bound:*

```

  assumes run:
    full-operational-partition-loop-state M t k cap l d P B d' D a betas bs B' Us U c
  shows  $\exists$  child-costs pulls batches.
     $c \leq \text{sum-list child-costs} + \text{sum-list } (\text{map } (\lambda S. \text{card } (S :: 'a \text{ set})) \text{ pulls}) +$ 
     $t * \text{sum-list } (\text{map } (\lambda \text{batch}. \text{length } (\text{batch} :: ('a \times \text{real}) \text{ list})) \text{ batches})$ 
  using run
proof (induction)
  case (State-Done D a B d' P M t k cap l d)
  show ?case
    by (intro exI[of -] exI[of -] exI[of -]) simp
next
  case (State-Step D M Bmax S-pull beta D-pull B d P a b d' k cap l d-child U-child
    batch D-next c-pull c-batch t betas bs B' Us-tail U-tail c-tail c-step c-total)
  have tail-ex:  $\exists$  child-costs pulls batches.
     $c\text{-tail} \leq \text{sum-list child-costs} + \text{sum-list } (\text{map } (\lambda S. \text{card } (S :: 'a \text{ set})) \text{ pulls}) +$ 
     $t * \text{sum-list } (\text{map } (\lambda \text{batch}. \text{length } (\text{batch} :: ('a \times \text{real}) \text{ list})) \text{ batches})$ 
  using State-Step.IH by assumption
  obtain child-costs :: nat list and pulls :: 'a set list
    and batches :: ('a  $\times$  real) list list where tail:
     $c\text{-tail} \leq \text{sum-list child-costs} + \text{sum-list } (\text{map card pulls}) +$ 
     $t * \text{sum-list } (\text{map length batches})$ 
  using tail-ex by blast

```



```

have pull-bound:  $c\text{-pull} \leq \text{card } S\text{-pull}$ 
  using State-Step unfolding partition-pull-cost-bound-def by blast
have batch-bound:  $c\text{-batch} \leq t * \text{length } \text{batch}$ 
  using State-Step unfolding partition-batch-cost-bound-def by blast
have c-eq:  $c\text{-step} = c\text{-pull} + c\text{-batch} + c\text{-total} + c\text{-tail}$ 
  using State-Step by blast
let ?child-costs =  $c\text{-total} \# \text{child-costs}$ 
let ?pulls =  $S\text{-pull} \# \text{pulls}$ 
let ?batches =  $\text{batch} \# \text{batches}$ 
have c-step  $\leq \text{sum-list } ?\text{child-costs} + \text{sum-list } (\text{map } \text{card } ?\text{pulls}) +$ 
   $t * \text{sum-list } (\text{map } \text{length } ?\text{batches})$ 
proof -
  have  $c\text{-pull} + c\text{-batch} + c\text{-total} + c\text{-tail} \leq$ 
     $\text{card } S\text{-pull} + t * \text{length } \text{batch} + c\text{-total} +$ 
     $(\text{sum-list } \text{child-costs} + \text{sum-list } (\text{map } \text{card } \text{pulls}) +$ 
     $t * \text{sum-list } (\text{map } \text{length } \text{batches}))$ 
  using pull-bound batch-bound tail by simp
  also have ... =
     $\text{sum-list } ?\text{child-costs} + \text{sum-list } (\text{map } \text{card } ?\text{pulls}) +$ 
     $t * \text{sum-list } (\text{map } \text{length } ?\text{batches})$ 
  by (simp add: algebra-simps)
  finally show ?thesis
    using c-eq by simp
qed
then show ?case
  by blast
qed

theorem full-operational-partition-loop-state-trace-and-cost:
assumes run:
  full-operational-partition-loop-state  $M \ t \ k \ \text{cap} \ l \ d \ P \ B \ d' \ D \ a \ \text{betas} \ bs \ B' \ Us \ U \ c$ 
and sound: sound-label  $d$ 
and pre: bmssp-pre-full  $d \ P \ B$ 
and P-reaches:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
obtains child-costs pulls batches where
  concrete-partition-loop-trace  $P \ B \ a \ bs \ d' \ B' \ Us \ U$ 
   $c \leq \text{sum-list } \text{child-costs} + \text{sum-list } (\text{map } (\lambda S. \text{card } (S :: 'a \ \text{set})) \ \text{pulls}) +$ 
   $t * \text{sum-list } (\text{map } (\lambda \text{batch}. \text{length } (\text{batch} :: ('a \times \text{real}) \ \text{list})) \ \text{batches})$ 
proof -
have full-loop:
  full-operational-partition-loop  $k \ \text{cap} \ l \ d \ P \ B \ d' \ a \ \text{betas} \ bs \ B' \ Us \ U$ 
by (rule full-operational-partition-loop-state-refines[OF run])
have trace: concrete-partition-loop-trace  $P \ B \ a \ bs \ d' \ B' \ Us \ U$ 
by (rule full-operational-partition-loop-trace
  [OF full-loop sound pre P-reaches])
obtain child-costs pulls batches where cost:
   $c \leq \text{sum-list } \text{child-costs} + \text{sum-list } (\text{map } (\lambda S. \text{card } (S :: 'a \ \text{set})) \ \text{pulls}) +$ 
   $t * \text{sum-list } (\text{map } (\lambda \text{batch}. \text{length } (\text{batch} :: ('a \times \text{real}) \ \text{list})) \ \text{batches})$ 
using full-operational-partition-loop-state-cost-bound[OF run] by blast

```

```

then show thesis
  using that trace by blast
qed

theorem full-operational-partition-loop-state-cost-bound-by-child-edges:
  assumes run:
    full-operational-partition-loop-state  $M\ t\ k\ \text{cap}\ l\ d\ P\ B\ d'\ D\ a\ \text{betas}\ bs\ B'\ Us\ U\ c$ 
  and P-subset:  $P \subseteq V$ 
  shows  $\exists \text{child-costs child-sets}.$ 
     $\text{length child-costs} = \text{length child-sets} \wedge$ 
     $c \leq \text{sum-list child-costs} + M * \text{length child-costs} +$ 
     $t * (\text{sum-list} (\text{map} (\lambda X. \text{card} (\text{outgoing-edges } X)) \text{child-sets}) +$ 
     $M * \text{length child-costs})$ 
  using run P-subset
proof (induction)
  case (State-Done  $D\ a\ B\ d'\ P\ M\ t\ k\ \text{cap}\ l\ d$ )
  show ?case
    by (intro exI[of -] exI[of -] simp)
next
  case (State-Step  $D\ M\ B_{\max}\ S\text{-pull}\ \text{beta}\ D\text{-pull}\ B\ d\ P\ a\ b\ d'\ k\ \text{cap}\ l$ 
     $d\text{-child}\ U\text{-child}\ \text{batch}\ D\text{-next}\ c\text{-pull}\ c\text{-batch}\ t\ \text{betas}\ bs\ B'$ 
     $Us\text{-tail}\ U\text{-tail}\ c\text{-tail}\ c\text{-step}\ c\text{-total}$ )
  have tail-ex:  $\exists \text{child-costs child-sets}.$ 
     $\text{length child-costs} = \text{length child-sets} \wedge$ 
     $c\text{-tail} \leq \text{sum-list child-costs} + M * \text{length child-costs} +$ 
     $t * (\text{sum-list} (\text{map} (\lambda X. \text{card} (\text{outgoing-edges } X)) \text{child-sets}) +$ 
     $M * \text{length child-costs})$ 
  using State-Step.IH State-Step.prems by blast
obtain child-costs child-sets where len-tail:
     $\text{length child-costs} = \text{length child-sets}$ 
and tail:
     $c\text{-tail} \leq \text{sum-list child-costs} + M * \text{length child-costs} +$ 
     $t * (\text{sum-list} (\text{map} (\lambda X. \text{card} (\text{outgoing-edges } X)) \text{child-sets}) +$ 
     $M * \text{length child-costs})$ 
  using tail-ex by blast
have finite-P: finite  $P$ 
  using finite-subset[OF State-Step.prems(1) finite-V] .
have S-pull-subset:  $S\text{-pull} \subseteq P$ 
  using State-Step.hyps(4) unfolding split-below-def by blast
have finite-S-pull: finite  $S\text{-pull}$ 
  using finite-subset[OF S-pull-subset finite-P] .
have card-pull:  $\text{card } S\text{-pull} \leq M$ 
  using pull-separates-card[OF State-Step.hyps(1)] .
have pull-cost:  $c\text{-pull} \leq M$ 
  using State-Step.hyps(11) card-pull
unfolding partition-pull-cost-bound-def by linarith
have edge-len:
     $\text{length} (\text{edge-relaxation-pairs-between } d\text{-child } U\text{-child } b\ \text{beta}) \leq$ 
     $\text{card} (\text{outgoing-edges } U\text{-child})$ 

```

```

    by (rule edge-relaxation-pairs-between-length-le-outgoing)
  have label-len:
    length (label-pairs-between d S-pull b beta) ≤ card S-pull
  by (rule label-pairs-between-length-le-card[OF finite-S-pull])
  have batch-len:
    length batch ≤ card (outgoing-edges U-child) + M
  using State-Step.hyps(9) edge-len label-len card-pull by simp
  have batch-cost:
    c-batch ≤ t * (card (outgoing-edges U-child) + M)
  proof -
    have c-batch ≤ t * length batch
    using State-Step.hyps(12)
    unfolding partition-batch-cost-bound-def by simp
    also have ... ≤ t * (card (outgoing-edges U-child) + M)
    using batch-len by simp
    finally show ?thesis .
  qed
  let ?child-costs = c-total # child-costs
  let ?child-sets = U-child # child-sets
  have len: length ?child-costs = length ?child-sets
  using len-tail by simp
  have cost:
    c-step ≤ sum-list ?child-costs + M * length ?child-costs +
      t * (sum-list (map (λX. card (outgoing-edges X)) ?child-sets) +
        M * length ?child-costs)
  proof -
    have c-eq: c-step = c-pull + c-batch + c-total + c-tail
    using State-Step.hyps(14) .
    have c-pull + c-batch + c-total + c-tail ≤
      M + t * (card (outgoing-edges U-child) + M) + c-total +
      (sum-list child-costs + M * length child-costs +
        t * (sum-list (map (λX. card (outgoing-edges X)) child-sets) +
          M * length child-costs))
    using pull-cost batch-cost tail by simp
    also have ... =
      sum-list ?child-costs + M * length ?child-costs +
      t * (sum-list (map (λX. card (outgoing-edges X)) ?child-sets) +
        M * length ?child-costs)
    by (simp add: algebra-simps)
    finally show ?thesis
    using c-eq by simp
  qed
  show ?case
  using len cost by blast
  qed

theorem full-operational-partition-loop-state-trace-and-edge-cost:
  assumes run:
    full-operational-partition-loop-state M t k cap l d P B d' D a betas bs B' Us U c

```

and *sound*: *sound-label* *d*
and *pre*: *bmssp-pre-full* *d P B*
and *P-reaches*: $\bigwedge x. x \in P \implies \text{reachable } s \ x$
obtains *child-costs child-sets* **where**
concrete-partition-loop-trace *P B a bs d' B' Us U*
length child-costs = length child-sets
 $c \leq \text{sum-list } \text{child-costs} + M * \text{length } \text{child-costs} +$
 $t * (\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ child-sets}) +$
 $M * \text{length } \text{child-costs})$
proof –
have *P-subset*: $P \subseteq V$
using *pre* **unfolding** *bmssp-pre-full-def* **by** *blast*
have *full-loop*:
full-operational-partition-loop *k cap l d P B d' a betas bs B' Us U*
by (*rule full-operational-partition-loop-state-refines* [*OF run*])
have *trace*: *concrete-partition-loop-trace* *P B a bs d' B' Us U*
by (*rule full-operational-partition-loop-trace*
[*OF full-loop sound pre P-reaches*])
obtain *child-costs child-sets* **where** *len*:
length child-costs = length child-sets
and *cost*:
 $c \leq \text{sum-list } \text{child-costs} + M * \text{length } \text{child-costs} +$
 $t * (\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ child-sets}) +$
 $M * \text{length } \text{child-costs})$
using *full-operational-partition-loop-state-cost-bound-by-child-edges*
[*OF run P-subset*] **by** *blast*
then show *thesis*
using *that trace* **by** *blast*
qed

theorem *full-operational-partition-loop-state-cost-bound-by-child-edges-complete*:
assumes *run*:
full-operational-partition-loop-state *M t k cap l d P B d' D a betas bs B' Us U c*
and *sound*: *sound-label* *d*
and *pre*: *bmssp-pre-full* *d P B*
and *P-reaches*: $\bigwedge x. x \in P \implies \text{reachable } s \ x$
shows $\exists \text{ child-costs child-sets.}$
 $\text{length } \text{child-costs} = \text{length } \text{child-sets} \wedge$
 $(\forall X \in \text{set } \text{child-sets. } X \subseteq V) \wedge$
 $c \leq \text{sum-list } \text{child-costs} + M * \text{length } \text{child-costs} +$
 $t * (\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ child-sets}) +$
 $M * \text{length } \text{child-costs})$
using *run sound pre P-reaches*
proof (*induction*)
case (*State-Done* *D a B d' P M t k cap l d*)
show ?*case*
by (*intro exI* [*of* - []] *exI* [*of* - []]) *simp*
next
case (*State-Step* *D M Bmax S-pull beta D-pull B d P a b d' k cap l*)

$d\text{-child } U\text{-child batch } D\text{-next } c\text{-pull } c\text{-batch } t \text{ betas } bs \ B'$
 $Us\text{-tail } U\text{-tail } c\text{-tail } c\text{-step } c\text{-total})$
have $tail\text{-ex}$: $\exists \text{ child-costs child-sets.}$
 $length \text{ child-costs} = length \text{ child-sets} \wedge$
 $(\forall X \in \text{set child-sets. } X \subseteq V) \wedge$
 $c\text{-tail} \leq \text{sum-list child-costs} + M * length \text{ child-costs} +$
 $t * (\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ child-sets}) +$
 $M * length \text{ child-costs})$
using $State\text{-Step.IH } State\text{-Step.premis}$ **by** $blast$
obtain $child\text{-costs child-sets}$ **where** $len\text{-tail}$:
 $length \text{ child-costs} = length \text{ child-sets}$
and $sets\text{-tail}$: $\forall X \in \text{set child-sets. } X \subseteq V$
and $tail$:
 $c\text{-tail} \leq \text{sum-list child-costs} + M * length \text{ child-costs} +$
 $t * (\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ child-sets}) +$
 $M * length \text{ child-costs})$
using $tail\text{-ex}$ **by** $blast$
have $child\text{-reaches}$:
 $\bigwedge x. x \in S\text{-pull} \implies \text{reachable } s \ x$
using $State\text{-Step.premis}(3) \ State\text{-Step.hyps}(4)$
unfolding $split\text{-below-def}$ **by** $blast$
have $child\text{-pre}$: $bmssp\text{-pre-full } d \ S\text{-pull } (Fin \ beta)$
using $State\text{-Step.hyps}(3,4)$ **by** $simp$
have $child\text{-post}$:
 $bmssp\text{-post-full } d \ S\text{-pull } (Fin \ beta) \ d\text{-child } (Fin \ b) \ U\text{-child}$
using $full\text{-operational-bmssp-correct}$
 $[OF \ State\text{-Step.hyps}(7) \ State\text{-Step.premis}(1) \ child\text{-pre } child\text{-reaches}]$.
have $child\text{-set}$: $U\text{-child} \subseteq V$
using $child\text{-post}$ **unfolding** $bmssp\text{-post-full-def bound-tree-def}$ **by** $blast$
have $P\text{-subset}$: $P \subseteq V$
using $State\text{-Step.premis}(2)$ **unfolding** $bmssp\text{-pre-full-def}$ **by** $blast$
have $finite\text{-P}$: $finite \ P$
using $finite\text{-subset}[OF \ P\text{-subset } finite\text{-V}]$.
have $S\text{-pull-subset}$: $S\text{-pull} \subseteq P$
using $State\text{-Step.hyps}(4)$ **unfolding** $split\text{-below-def}$ **by** $blast$
have $finite\text{-S-pull}$: $finite \ S\text{-pull}$
using $finite\text{-subset}[OF \ S\text{-pull-subset } finite\text{-P}]$.
have $card\text{-pull}$: $card \ S\text{-pull} \leq M$
using $pull\text{-separates-card}[OF \ State\text{-Step.hyps}(1)]$.
have $pull\text{-cost}$: $c\text{-pull} \leq M$
using $State\text{-Step.hyps}(11) \ card\text{-pull}$
unfolding $partition\text{-pull-cost-bound-def}$ **by** $linarith$
have $edge\text{-len}$:
 $length \ (edge\text{-relaxation-pairs-between } d\text{-child } U\text{-child } b \ beta) \leq$
 $card \ (outgoing\text{-edges } U\text{-child})$
by $(rule \ edge\text{-relaxation-pairs-between-length-le-outgoing})$
have $label\text{-len}$:
 $length \ (label\text{-pairs-between } d \ S\text{-pull } b \ beta) \leq card \ S\text{-pull}$
by $(rule \ label\text{-pairs-between-length-le-card}[OF \ finite\text{-S-pull}])$

```

have batch-len:
  length batch ≤ card (outgoing-edges U-child) + M
  using State-Step.hyps(9) edge-len label-len card-pull by simp
have batch-cost:
  c-batch ≤ t * (card (outgoing-edges U-child) + M)
proof -
  have c-batch ≤ t * length batch
  using State-Step.hyps(12)
  unfolding partition-batch-cost-bound-def by simp
  also have ... ≤ t * (card (outgoing-edges U-child) + M)
  using batch-len by simp
  finally show ?thesis .
qed
let ?child-costs = c-total # child-costs
let ?child-sets = U-child # child-sets
have len: length ?child-costs = length ?child-sets
  using len-tail by simp
have sets: ∀ X ∈ set ?child-sets. X ⊆ V
  using child-set sets-tail by simp
have cost:
  c-step ≤ sum-list ?child-costs + M * length ?child-costs +
    t * (sum-list (map (λX. card (outgoing-edges X)) ?child-sets) +
      M * length ?child-costs)
proof -
  have c-eq: c-step = c-pull + c-batch + c-total + c-tail
  using State-Step.hyps(14) .
  have c-pull + c-batch + c-total + c-tail ≤
    M + t * (card (outgoing-edges U-child) + M) + c-total +
    (sum-list child-costs + M * length child-costs +
      t * (sum-list (map (λX. card (outgoing-edges X)) child-sets) +
        M * length child-costs))
  using pull-cost batch-cost tail by simp
  also have ... =
    sum-list ?child-costs + M * length ?child-costs +
    t * (sum-list (map (λX. card (outgoing-edges X)) ?child-sets) +
      M * length ?child-costs)
  by (simp add: algebra-simps)
  finally show ?thesis
  using c-eq by simp
qed
show ?case
  using len sets cost by blast
qed

```

```

theorem full-operational-partition-loop-state-trace-and-complete-edge-cost:
  assumes run:
    full-operational-partition-loop-state M t k cap l d P B d' D a betas bs B' Us U c
  and sound: sound-label d
  and pre: bmssp-pre-full d P B

```

```

and P-reaches:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
obtains child-costs child-sets where
  concrete-partition-loop-trace P B a bs d' B' Us U
  length child-costs = length child-sets
   $\bigwedge X. X \in \text{set } \text{child-sets} \implies X \subseteq V$ 
   $c \leq \text{sum-list } \text{child-costs} + M * \text{length } \text{child-costs} +$ 
     $t * (\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ child-sets}) +$ 
       $M * \text{length } \text{child-costs})$ 
proof –
  have full-loop:
    full-operational-partition-loop k cap l d P B d' a betas bs B' Us U
    by (rule full-operational-partition-loop-state-refines[OF run])
  have trace: concrete-partition-loop-trace P B a bs d' B' Us U
    by (rule full-operational-partition-loop-trace
      [OF full-loop sound pre P-reaches])
  obtain child-costs child-sets where len:
    length child-costs = length child-sets
  and sets:  $\forall X \in \text{set } \text{child-sets}. X \subseteq V$ 
  and cost:
     $c \leq \text{sum-list } \text{child-costs} + M * \text{length } \text{child-costs} +$ 
       $t * (\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ child-sets}) +$ 
         $M * \text{length } \text{child-costs})$ 
  using full-operational-partition-loop-state-cost-bound-by-child-edges-complete
    [OF run sound pre P-reaches] by blast
  then show thesis
    using that trace by blast
qed

end

end

theory BMSSP-Complexity
  imports HOL-Library.Landau-Symbols BMSSP-Operational-Pull
begin

```

28 Core Cost Accounting

This theory starts the checked complexity side of the development. Earlier theories prove that the recursive BMSSP algorithm returns the right vertices and labels. Here the same traces are interpreted as cost recurrences. The first part is deliberately arithmetic: logarithmic target functions, level widths, level capacities, and the Big-O envelopes that eventually become the SSSP headline.

The file then moves inside the graph locale and introduces the graph-size quantities used by the cost bounds. Vertex counts, edge counts, outgoing ranges, and range-tree chains are all finite combinatorial objects. The proofs establish that the lists produced by a partition loop are disjoint slices, so their

vertex and outgoing-edge costs can be summed without double counting.

The final part defines a costed operational relation for the abstract partition interface. It is intentionally not the bucketed implementation; it is the reusable recurrence layer that later theories refine. The bucketed proof plugs concrete local costs into these abstract cost expressions, while the top-level bounds instantiate the logarithmic parameter schedule.

definition *sssp-log-factor* **where**

$$\text{sssp-log-factor } n = (\ln (\text{real } n + 2)) \text{ powr } (2 / 3)$$

definition *sssp-log-factor-one-third* **where**

$$\text{sssp-log-factor-one-third } n = (\ln (\text{real } n + 2)) \text{ powr } (1 / 3)$$

definition *sssp-time-target* **where**

$$\text{sssp-time-target } m \ n = \text{real } (m \ n) * \text{sssp-log-factor } n$$

definition *bmssp-level-width* :: $\text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat}$ **where**

$$\text{bmssp-level-width } t \ l = 2^{\wedge} (l * t)$$

definition *bmssp-level-cap* :: $\text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat}$ **where**

$$\text{bmssp-level-cap } k \ t \ l = k * \text{bmssp-level-width } t \ l$$

lemma *bmssp-level-width-pos* [simp]:

$$0 < \text{bmssp-level-width } t \ l$$

unfolding *bmssp-level-width-def* **by** *simp*

lemma *bmssp-level-cap-eq*:

$$\text{bmssp-level-cap } k \ t \ l = k * 2^{\wedge} (l * t)$$

unfolding *bmssp-level-cap-def* *bmssp-level-width-def* **by** *simp*

lemma *bmssp-level-width-mono*:

assumes $l \leq l'$

shows $\text{bmssp-level-width } t \ l \leq \text{bmssp-level-width } t \ l'$

proof –

have $l * t \leq l' * t$

using *assms* **by** *simp*

then show *?thesis*

unfolding *bmssp-level-width-def* **by** (*simp add: power-increasing*)

qed

lemma *bmssp-level-width-Suc*:

$$\text{bmssp-level-width } t \ (\text{Suc } l) = \text{bmssp-level-width } t \ l * 2^{\wedge} t$$

unfolding *bmssp-level-width-def* **by** (*simp add: power-add algebra-simps*)

lemma *bmssp-level-width-prev-times-two-le*:

assumes $0 < t \ 0 < l$

shows $2 * \text{bmssp-level-width } t \ (l - 1) \leq \text{bmssp-level-width } t \ l$

proof (*cases l*)

case *0*


```

    then show ?thesis
      using assms by simp
next
case (Suc l')
have two-le: (2::nat) ≤ 2 ^ t
proof -
  have 2 ^ 1 ≤ (2::nat) ^ t
    by (rule power-increasing) (use assms(1) in simp-all)
  then show ?thesis
    by simp
qed
have 2 * bmssp-level-width t (l - 1) ≤
  (2 ^ t) * bmssp-level-width t (l - 1)
  by (rule mult-right-mono[OF two-le]) simp
also have ... = bmssp-level-width t l
  unfolding Suc bmssp-level-width-Suc by (simp add: algebra-simps)
finally show ?thesis .
qed

lemma bmssp-level-cap-mono:
  assumes l ≤ l'
  shows bmssp-level-cap k t l ≤ bmssp-level-cap k t l'
  unfolding bmssp-level-cap-def
  using bmssp-level-width-mono[OF assms, of t] by simp

lemma bmssp-level-cap-ge-level:
  k ≤ bmssp-level-cap k t l
  unfolding bmssp-level-cap-def
  using bmssp-level-width-pos[of t l]
  by (cases bmssp-level-width t l) simp-all

lemma bmssp-level-size-constraint-arithmetic:
  assumes t-pos: 0 < t
  and l-pos: 0 < l
  and before: acc < k * bmssp-level-width t l
  and child: child ≤ 4 * k * bmssp-level-width t (l - 1)
  and extra: extra ≤ k * bmssp-level-width t l
  and U-size: U-size ≤ acc + child + extra
  shows U-size ≤ 4 * k * bmssp-level-width t l
proof -
  have acc-le: acc ≤ k * bmssp-level-width t l
    using before by simp
  have child-le: child ≤ 2 * k * bmssp-level-width t l
  proof -
    have 4 * k * bmssp-level-width t (l - 1) ≤
      2 * k * bmssp-level-width t l
    proof -
      have 2 * bmssp-level-width t (l - 1) ≤ bmssp-level-width t l
        by (rule bmssp-level-width-prev-times-two-le[OF t-pos l-pos])

```

```

    then have  $2 * k * (2 * \text{bmssp-level-width } t \ (l - 1)) \leq$ 
       $2 * k * \text{bmssp-level-width } t \ l$ 
    by simp
    then show ?thesis
      by (simp add: algebra-simps)
  qed
  then show ?thesis
    using child by linarith
  qed
  have  $U\text{-size} \leq$ 
     $k * \text{bmssp-level-width } t \ l +$ 
     $2 * k * \text{bmssp-level-width } t \ l +$ 
     $k * \text{bmssp-level-width } t \ l$ 
  using  $U\text{-size acc-le child-le extra}$  by linarith
  also have  $\dots = 4 * k * \text{bmssp-level-width } t \ l$ 
    by (simp add: algebra-simps)
  finally show ?thesis .
qed

lemma sssp-log-factor-pos [simp]:
   $0 < \text{sssp-log-factor } n$ 
proof -
  have  $1 < \text{real } n + 2$ 
  by simp
  then have  $0 < \ln (\text{real } n + 2)$ 
    by (rule ln-gt-zero)
  then show ?thesis
    unfolding sssp-log-factor-def by simp
qed

lemma sssp-log-factor-one-third-pos [simp]:
   $0 < \text{sssp-log-factor-one-third } n$ 
proof -
  have  $1 < \text{real } n + 2$ 
  by simp
  then have  $0 < \ln (\text{real } n + 2)$ 
    by (rule ln-gt-zero)
  then show ?thesis
    unfolding sssp-log-factor-one-third-def by simp
qed

lemma sssp-log-factor-mono:
  assumes  $n \leq m$ 
  shows  $\text{sssp-log-factor } n \leq \text{sssp-log-factor } m$ 
proof -
  have  $\ln\text{-le}: \ln (\text{real } n + 2) \leq \ln (\text{real } m + 2)$ 
    by (rule ln-mono) (use assms in simp-all)
  have  $\ln\text{-nonneg}: 0 \leq \ln (\text{real } n + 2)$ 
    by simp

```

```

show ?thesis
  unfolding sssp-log-factor-def
  by (rule powr-mono2) (use ln-nonneg ln-le in simp-all)
qed

lemma sssp-log-factor-one-third-mono:
  assumes  $n \leq m$ 
  shows  $\text{sssp-log-factor-one-third } n \leq \text{sssp-log-factor-one-third } m$ 
proof -
  have ln-le:  $\ln (\text{real } n + 2) \leq \ln (\text{real } m + 2)$ 
    by (rule ln-mono) (use assms in simp-all)
  have ln-nonneg:  $0 \leq \ln (\text{real } n + 2)$ 
    by simp
  show ?thesis
    unfolding sssp-log-factor-one-third-def
    by (rule powr-mono2) (use ln-nonneg ln-le in simp-all)
qed

lemma sssp-log-factor-square-arg-le:
   $\text{sssp-log-factor } (n * n) \leq 2 * \text{sssp-log-factor } n$ 
proof -
  let ?a =  $\ln (\text{real } (n * n) + 2)$ 
  let ?b =  $\ln (\text{real } n + 2)$ 
  have arg-le:  $\text{real } (n * n) + 2 \leq (\text{real } n + 2) ^ 2$ 
    by (simp add: power2-eq-square algebra-simps)
  have arg-pos:  $0 < \text{real } (n * n) + 2$ 
  proof -
    have  $0 \leq \text{real } (n * n)$ 
    by simp
  then show ?thesis
    by linarith
qed
have ln-le:  $?a \leq 2 * ?b$ 
proof -
  have  $?a \leq \ln ((\text{real } n + 2) ^ 2)$ 
    by (rule ln-mono) (use arg-le in simp, rule arg-pos)
  also have  $\dots = 2 * ?b$ 
    by (simp add: ln-realpow)
  finally show ?thesis .
qed
have a-nonneg:  $0 \leq ?a$ 
  by simp
have b-nonneg:  $0 \leq ?b$ 
  by simp
have powr-le:  $?a \text{ powr } (2 / 3) \leq (2 * ?b) \text{ powr } (2 / 3)$ 
  by (rule powr-mono2) (use a-nonneg ln-le in simp-all)
have two-powr-le:  $(2::\text{real}) \text{ powr } (2 / 3) \leq 2$ 
proof -
  have  $(2::\text{real}) \text{ powr } (2 / 3) \leq 2 \text{ powr } 1$ 

```

```

    by (rule powr-mono) simp-all
  then show ?thesis
    by simp
qed
have (2 * ?b) powr (2 / 3) =
  (2::real) powr (2 / 3) * (?b powr (2 / 3))
  by (simp add: powr-mult)
also have ... ≤ 2 * (?b powr (2 / 3))
  by (rule mult-right-mono[OF two-powr-le]) (use b-nonneg in simp)
finally have (2 * ?b) powr (2 / 3) ≤ 2 * (?b powr (2 / 3)) .
then show ?thesis
  unfolding sssp-log-factor-def using powr-le by linarith
qed

lemma sssp-log-factor-one-third-square-arg-le:
  sssp-log-factor-one-third (n * n) ≤
    2 * sssp-log-factor-one-third n
proof -
  let ?a = ln (real (n * n) + 2)
  let ?b = ln (real n + 2)
  have arg-le: real (n * n) + 2 ≤ (real n + 2) ^ 2
    by (simp add: power2-eq-square algebra-simps)
  have arg-pos: 0 < real (n * n) + 2
  proof -
    have 0 ≤ real (n * n)
    by simp
  then show ?thesis
    by linarith
qed
have ln-le: ?a ≤ 2 * ?b
proof -
  have ?a ≤ ln ((real n + 2) ^ 2)
    by (rule ln-mono) (use arg-le in simp, rule arg-pos)
  also have ... = 2 * ?b
    by (simp add: ln-realpow)
  finally show ?thesis .
qed
have a-nonneg: 0 ≤ ?a
  by simp
have b-nonneg: 0 ≤ ?b
  by simp
have powr-le: ?a powr (1 / 3) ≤ (2 * ?b) powr (1 / 3)
  by (rule powr-mono2) (use a-nonneg ln-le in simp-all)
have two-powr-le: (2::real) powr (1 / 3) ≤ 2
proof -
  have (2::real) powr (1 / 3) ≤ 2 powr 1
    by (rule powr-mono) simp-all
  then show ?thesis
    by simp

```

```

qed
have (2 * ?b) powr (1 / 3) =
  (2::real) powr (1 / 3) * (?b powr (1 / 3))
  by (simp add: powr-mult)
also have ... ≤ 2 * (?b powr (1 / 3))
  by (rule mult-right-mono[OF two-powr-le]) (use b-nonneg in simp)
finally have (2 * ?b) powr (1 / 3) ≤ 2 * (?b powr (1 / 3)) .
then show ?thesis
  unfolding sssp-log-factor-one-third-def using powr-le by linarith
qed

```

```

lemma sssp-log-factor-one-third-square:
  sssp-log-factor-one-third n * sssp-log-factor-one-third n =
    sssp-log-factor n
proof -
  have 0 < ln (real n + 2)
  proof -
    have 1 < real n + 2
    by simp
    then show ?thesis
    by (rule ln-gt-zero)
  qed
  then show ?thesis
    unfolding sssp-log-factor-one-third-def sssp-log-factor-def
    by (simp add: powr-add [symmetric])
qed

```

```

lemma eventually-one-le-sssp-log-factor-one-third:
  eventually (λn. 1 ≤ sssp-log-factor-one-third n) at-top
proof (rule eventually-at-top-linorderI[of 1])
  fix n :: nat
  assume n-ge: 1 ≤ n
  have exp 1 ≤ (3::real)
    by (rule exp-le)
  also have ... ≤ real n + 2
    using n-ge by simp
  finally have exp-le-arg: exp 1 ≤ real n + 2 .
  have ln-ge: 1 ≤ ln (real n + 2)
    using exp-le-arg by (simp add: ln-ge-iff)
  show 1 ≤ sssp-log-factor-one-third n
    unfolding sssp-log-factor-one-third-def
    by (rule ge-one-powr-ge-zero[OF ln-ge]) simp
qed

```

```

lemma sssp-time-target-nonneg [simp]:
  0 ≤ sssp-time-target m n
proof -
  have 0 ≤ real (m n)
  by simp

```

moreover have $0 \leq \text{sssp-log-factor } n$
using *sssp-log-factor-pos*[of n] **by** *linarith*
ultimately show *?thesis*
unfolding *sssp-time-target-def* **by** (rule *mult-nonneg-nonneg*)
qed

theorem *sssp-time-target-bigoI-norm*:
assumes $C: 0 < C$
and *eventually-bound*:
eventually
 $(\lambda n. \text{norm } (\text{real } (T\ n))) \leq C * \text{norm } (\text{sssp-time-target } m\ n)) \text{ at-top}$
shows $(\lambda n. \text{real } (T\ n)) \in O(\lambda n. \text{sssp-time-target } m\ n)$
using C *eventually-bound* **unfolding** *bigo-def* **by** *blast*

theorem *sssp-time-target-bigoI*:
assumes $C: 0 < C$
and *eventually-bound*:
eventually $(\lambda n. \text{real } (T\ n)) \leq C * \text{sssp-time-target } m\ n) \text{ at-top}$
shows $(\lambda n. \text{real } (T\ n)) \in O(\lambda n. \text{sssp-time-target } m\ n)$
proof (rule *sssp-time-target-bigoI-norm*[OF C])
show *eventually*
 $(\lambda n. \text{norm } (\text{real } (T\ n))) \leq C * \text{norm } (\text{sssp-time-target } m\ n)) \text{ at-top}$
using *eventually-bound*
proof *eventually-elim*
case (*elim* n)
then show *?case*
by *simp*
qed
qed

definition *bmssp-graph-time-bound* **where**
 $\text{bmssp-graph-time-bound } A\ R\ l\ t\ m\ n =$
 $2 * A\ n * (2 * l\ n + 1) * n + (R\ n + l\ n * t\ n) * m\ n$

theorem *bmssp-graph-time-bound-bigo-sssp-time-target*:
assumes $Cn\text{-pos}: 0 < Cn$
and $Cv\text{-pos}: 0 < Cv$
and $Ce\text{-pos}: 0 < Ce$
and *vertex-count-dominated*:
eventually $(\lambda n. \text{real } n \leq Cn * \text{real } (m\ n)) \text{ at-top}$
and *vertex-factor*:
eventually
 $(\lambda n. \text{real } (2 * A\ n * (2 * l\ n + 1))) \leq$
 $Cv * \text{sssp-log-factor } n) \text{ at-top}$
and *edge-factor*:
eventually
 $(\lambda n. \text{real } (R\ n + l\ n * t\ n)) \leq$
 $Ce * \text{sssp-log-factor } n) \text{ at-top}$
shows $(\lambda n. \text{real } (\text{bmssp-graph-time-bound } A\ R\ l\ t\ m\ n))$

```

    ∈  $O(\lambda n. \text{sssp-time-target } m \ n)$ 
proof (rule sssp-time-target-bigoI [where  $C = C_v * C_n + C_e$ ])
  show  $0 < C_v * C_n + C_e$ 
proof –
  have  $0 < C_v * C_n$ 
    using Cv-pos Cn-pos by (rule mult-pos-pos)
  then show ?thesis
    using Ce-pos by linarith
qed
next
show eventually
  ( $\lambda n. \text{real } (\text{bmssp-graph-time-bound } A \ R \ l \ t \ m \ n) \leq$ 
   ( $C_v * C_n + C_e$ ) * sssp-time-target  $m \ n$ ) at-top
  using vertex-count-dominated vertex-factor edge-factor
proof eventually-elim
case (elim  $n$ )
  let ?vf = real ( $2 * A \ n * (2 * l \ n + 1)$ )
  let ?ef = real ( $R \ n + l \ n * t \ n$ )
  let ?log = sssp-log-factor  $n$ 
  have vertex:
    ?vf * real  $n \leq (C_v * ?log) * (C_n * \text{real } (m \ n))$ 
proof (rule mult-mono)
  show ?vf ≤  $C_v * ?log$ 
    using elim(2) .
  show real  $n \leq C_n * \text{real } (m \ n)$ 
    using elim(1) .
  show  $0 \leq \text{real } n$ 
    by simp
  show  $0 \leq C_v * ?log$ 
proof –
  have  $0 \leq C_v$ 
    using Cv-pos by linarith
  moreover have  $0 \leq ?log$ 
    using sssp-log-factor-pos [of  $n$ ] by linarith
  ultimately show ?thesis
    by (rule mult-nonneg-nonneg)
qed
qed
also have ... =  $(C_v * C_n) * (\text{real } (m \ n) * ?log)$ 
  by (simp add: algebra-simps)
finally have vertex':
  ?vf * real  $n \leq (C_v * C_n) * (\text{real } (m \ n) * ?log)$  .
have edge:
  ?ef * real  $(m \ n) \leq (C_e * ?log) * \text{real } (m \ n)$ 
  by (rule mult-right-mono [OF elim(3)]) simp
also have ... =  $C_e * (\text{real } (m \ n) * ?log)$ 
  by (simp add: algebra-simps)
finally have edge':
  ?ef * real  $(m \ n) \leq C_e * (\text{real } (m \ n) * ?log)$  .

```

```

have real (bmssp-graph-time-bound A R l t m n) =
  ?vf * real n + ?ef * real (m n)
  unfolding bmssp-graph-time-bound-def by simp
also have ... ≤
  (Cv * Cn) * (real (m n) * ?log) +
  Ce * (real (m n) * ?log)
  using vertex' edge' by linarith
also have ... = (Cv * Cn + Ce) * sssp-time-target m n
  unfolding sssp-time-target-def by (simp add: algebra-simps)
finally show ?case .
qed
qed

theorem bmssp-cost-bound-bigo-sssp-time-target:
  assumes Cn-pos: 0 < Cn
  and Cv-pos: 0 < Cv
  and Ce-pos: 0 < Ce
  and vertex-count-dominated:
    eventually (λn. real n ≤ Cn * real (m n)) at-top
  and vertex-factor:
    eventually
      (λn. real (2 * A n * (2 * l n + 1)) ≤
        Cv * sssp-log-factor n) at-top
  and edge-factor:
    eventually
      (λn. real (R n + l n * t n) ≤
        Ce * sssp-log-factor n) at-top
  and cost-bound:
    eventually (λn. T n ≤ bmssp-graph-time-bound A R l t m n) at-top
  shows (λn. real (T n)) ∈ O(λn. sssp-time-target m n)
proof -
  have graph:
    (λn. real (bmssp-graph-time-bound A R l t m n))
    ∈ O(λn. sssp-time-target m n)
  by (rule bmssp-graph-time-bound-bigo-sssp-time-target
    [OF Cn-pos Cv-pos Ce-pos vertex-count-dominated vertex-factor
      edge-factor])
  then obtain C where C-pos: 0 < C
  and eventually-graph:
    eventually
      (λn. norm (real (bmssp-graph-time-bound A R l t m n)) ≤
        C * norm (sssp-time-target m n)) at-top
  unfolding bigo-def by blast
  show ?thesis
proof (rule sssp-time-target-bigoI-norm[OF C-pos])
  show eventually
    (λn. norm (real (T n)) ≤ C * norm (sssp-time-target m n)) at-top
  using cost-bound eventually-graph
proof eventually-elim

```



```

    case (elim n)
    have norm (real (T n)) ≤
      norm (real (bmssp-graph-time-bound A R l t m n))
    using elim(1) by simp
    also have ... ≤ C * norm (sssp-time-target m n)
    using elim(2) .
    finally show ?case .
  qed
qed
qed

definition bmssp-refined-graph-time-bound where
  bmssp-refined-graph-time-bound A R H l t m n =
    2 * A n * (2 * l n + 1) * n +
    (R n + t n + l n * H n) * m n

theorem bmssp-refined-graph-time-bound-bigo-sssp-time-target:
  assumes Cn-pos: 0 < Cn
  and Cv-pos: 0 < Cv
  and Ce-pos: 0 < Ce
  and vertex-count-dominated:
    eventually (λn. real n ≤ Cn * real (m n)) at-top
  and vertex-factor:
    eventually
      (λn. real (2 * A n * (2 * l n + 1)) ≤
        Cv * sssp-log-factor n) at-top
  and edge-factor:
    eventually
      (λn. real (R n + t n + l n * H n) ≤
        Ce * sssp-log-factor n) at-top
  shows (λn. real (bmssp-refined-graph-time-bound A R H l t m n))
    ∈ O(λn. sssp-time-target m n)
proof (rule sssp-time-target-bigoI[where C = Cv * Cn + Ce])
  show 0 < Cv * Cn + Ce
  proof –
    have 0 < Cv * Cn
    using Cv-pos Cn-pos by (rule mult-pos-pos)
    then show ?thesis
    using Ce-pos by linarith
  qed
next
  show eventually
    (λn. real (bmssp-refined-graph-time-bound A R H l t m n) ≤
      (Cv * Cn + Ce) * sssp-time-target m n) at-top
  using vertex-count-dominated vertex-factor edge-factor
proof eventually-elim
  case (elim n)
  let ?vf = real (2 * A n * (2 * l n + 1))
  let ?ef = real (R n + t n + l n * H n)

```

```

let ?log = sssp-log-factor n
have vertex:
  ?vf * real n ≤ (Cv * ?log) * (Cn * real (m n))
proof (rule mult-mono)
  show ?vf ≤ Cv * ?log
    using elim(2) .
  show real n ≤ Cn * real (m n)
    using elim(1) .
  show 0 ≤ real n
    by simp
  show 0 ≤ Cv * ?log
proof -
  have 0 ≤ Cv
    using Cv-pos by linarith
  moreover have 0 ≤ ?log
    using sssp-log-factor-pos[of n] by linarith
  ultimately show ?thesis
    by (rule mult-nonneg-nonneg)
qed
qed
also have ... = (Cv * Cn) * (real (m n) * ?log)
  by (simp add: algebra-simps)
finally have vertex':
  ?vf * real n ≤ (Cv * Cn) * (real (m n) * ?log) .
have edge:
  ?ef * real (m n) ≤ (Ce * ?log) * real (m n)
  by (rule mult-right-mono[OF elim(3)]) simp
also have ... = Ce * (real (m n) * ?log)
  by (simp add: algebra-simps)
finally have edge':
  ?ef * real (m n) ≤ Ce * (real (m n) * ?log) .
have real (bmssp-refined-graph-time-bound A R H l t m n) =
  ?vf * real n + ?ef * real (m n)
  unfolding bmssp-refined-graph-time-bound-def by simp
also have ... ≤
  (Cv * Cn) * (real (m n) * ?log) +
  Ce * (real (m n) * ?log)
  using vertex' edge' by linarith
also have ... = (Cv * Cn + Ce) * sssp-time-target m n
  unfolding sssp-time-target-def by (simp add: algebra-simps)
finally show ?case .
qed
qed

theorem bmssp-refined-cost-bound-bigo-sssp-time-target:
  assumes Cn-pos: 0 < Cn
  and Cv-pos: 0 < Cv
  and Ce-pos: 0 < Ce
  and vertex-count-dominated:

```

eventually ($\lambda n. \text{real } n \leq Cn * \text{real } (m \ n)$) at-top
and vertex-factor:
 eventually
 ($\lambda n. \text{real } (2 * A \ n * (2 * l \ n + 1)) \leq$
 $Cv * \text{sssp-log-factor } n$) at-top
and edge-factor:
 eventually
 ($\lambda n. \text{real } (R \ n + t \ n + l \ n * H \ n) \leq$
 $Ce * \text{sssp-log-factor } n$) at-top
and cost-bound:
 eventually
 ($\lambda n. T \ n \leq \text{bmssp-refined-graph-time-bound } A \ R \ H \ l \ t \ m \ n$) at-top
shows ($\lambda n. \text{real } (T \ n) \in O(\lambda n. \text{sssp-time-target } m \ n)$)
proof –
have graph:
 ($\lambda n. \text{real } (\text{bmssp-refined-graph-time-bound } A \ R \ H \ l \ t \ m \ n)$
 $\in O(\lambda n. \text{sssp-time-target } m \ n)$)
by (rule *bmssp-refined-graph-time-bound-bigo-sssp-time-target*
 $[OF \ Cn\text{-pos} \ Cv\text{-pos} \ Ce\text{-pos} \ \text{vertex-count-dominated vertex-factor}$
 $\text{edge-factor}]$)
then obtain C **where** $C\text{-pos}: 0 < C$
and eventually-graph:
 eventually
 ($\lambda n. \text{norm } (\text{real } (\text{bmssp-refined-graph-time-bound } A \ R \ H \ l \ t \ m \ n)) \leq$
 $C * \text{norm } (\text{sssp-time-target } m \ n)$) at-top
unfolding *bigo-def* **by** *blast*
show *?thesis*
proof (rule *sssp-time-target-bigoI-norm* $[OF \ C\text{-pos}]$)
show eventually
 ($\lambda n. \text{norm } (\text{real } (T \ n)) \leq C * \text{norm } (\text{sssp-time-target } m \ n)$) at-top
using *cost-bound eventually-graph*
proof *eventually-elim*
case (*elim* n)
have $\text{norm } (\text{real } (T \ n)) \leq$
 $\text{norm } (\text{real } (\text{bmssp-refined-graph-time-bound } A \ R \ H \ l \ t \ m \ n))$
using *elim(1)* **by** *simp*
also have $\dots \leq C * \text{norm } (\text{sssp-time-target } m \ n)$
using *elim(2)* .
finally show *?case* .
qed
qed
qed

theorem *bmssp-refined-cost-bound-bigo-sssp-time-target-from-component-bounds*:
assumes $Cn\text{-pos}: 0 < Cn$
and $Ca\text{-pos}: 0 < Ca$
and $Cl\text{-nonneg}: 0 \leq Cl$
and $Cr\text{-pos}: 0 < Cr$
and $Ct\text{-pos}: 0 < Ct$

```

and Ch-pos:  $0 < Ch$ 
and vertex-count-dominated:
  eventually  $(\lambda n. \text{real } n \leq Cn * \text{real } (m \ n)) \text{ at-top}$ 
and A-bound:
  eventually
     $(\lambda n. \text{real } (A \ n) \leq Ca * \text{sssp-log-factor-one-third } n) \text{ at-top}$ 
and l-bound:
  eventually
     $(\lambda n. \text{real } (l \ n) \leq Cl * \text{sssp-log-factor-one-third } n) \text{ at-top}$ 
and R-bound:
  eventually
     $(\lambda n. \text{real } (R \ n) \leq Cr * \text{sssp-log-factor } n) \text{ at-top}$ 
and t-bound:
  eventually
     $(\lambda n. \text{real } (t \ n) \leq Ct * \text{sssp-log-factor } n) \text{ at-top}$ 
and H-bound:
  eventually
     $(\lambda n. \text{real } (H \ n) \leq Ch * \text{sssp-log-factor-one-third } n) \text{ at-top}$ 
and cost-bound:
  eventually
     $(\lambda n. T \ n \leq \text{bmssp-refined-graph-time-bound } A \ R \ H \ l \ t \ m \ n) \text{ at-top}$ 
shows  $(\lambda n. \text{real } (T \ n)) \in O(\lambda n. \text{sssp-time-target } m \ n)$ 
proof (rule bmssp-refined-cost-bound-bigo-sssp-time-target
  [where  $Cn = Cn$  and  $Cv = 2 * Ca * (2 * Cl + 1)$ 
    and  $Ce = Cr + Ct + Cl * Ch$ ])
show  $0 < Cn$ 
  using Cn-pos .
show  $0 < 2 * Ca * (2 * Cl + 1)$ 
  using Ca-pos Cl-nonneg by (intro mult-pos-pos) simp-all
show  $0 < Cr + Ct + Cl * Ch$ 
proof –
  have  $0 \leq Cl * Ch$ 
    using Cl-nonneg Ch-pos by simp
  then show ?thesis
    using Cr-pos Ct-pos by linarith
qed
show eventually  $(\lambda n. \text{real } n \leq Cn * \text{real } (m \ n)) \text{ at-top}$ 
  using vertex-count-dominated .
show eventually
   $(\lambda n. \text{real } (2 * A \ n * (2 * l \ n + 1)) \leq$ 
     $2 * Ca * (2 * Cl + 1) * \text{sssp-log-factor } n) \text{ at-top}$ 
  using A-bound l-bound eventually-one-le-sssp-log-factor-one-third
proof eventually-elim
  case (elim  $n$ )
  let  $?L = \text{sssp-log-factor-one-third } n$ 
  have L-nonneg:  $0 \leq ?L$ 
    using sssp-log-factor-one-third-pos[of  $n$ ] by linarith
  have A-le:  $\text{real } (A \ n) \leq Ca * ?L$ 
    using elim(1) .

```

```

have l-le: real (2 * l n + 1) ≤ (2 * Cl + 1) * ?L
proof -
  have real (2 * l n + 1) = 2 * real (l n) + 1
    by simp
  also have ... ≤ 2 * (Cl * ?L) + 1
    using elim(2) by linarith
  also have ... ≤ 2 * (Cl * ?L) + ?L
    using elim(3) by linarith
  also have ... = (2 * Cl + 1) * ?L
    by (simp add: algebra-simps)
  finally show ?thesis .
qed
have l-rhs-nonneg: 0 ≤ (2 * Cl + 1) * ?L
  using Cl-nonneg L-nonneg by (intro mult-nonneg-nonneg) simp-all
have prod:
  real (A n) * real (2 * l n + 1) ≤
    (Ca * ?L) * ((2 * Cl + 1) * ?L)
  by (rule mult-mono[OF A-le l-le]) (use L-nonneg Ca-pos l-rhs-nonneg in
simp-all)
  have real (2 * A n * (2 * l n + 1)) =
    2 * (real (A n) * real (2 * l n + 1))
  by (simp add: algebra-simps)
  also have ... ≤ 2 * ((Ca * ?L) * ((2 * Cl + 1) * ?L))
    using prod by simp
  also have ... = 2 * Ca * (2 * Cl + 1) * sssp-log-factor n
    using sssp-log-factor-one-third-square[of n]
  by (simp add: algebra-simps)
  finally show ?case .
qed
show eventually
  (λn. real (R n + t n + l n * H n) ≤
    (Cr + Ct + Cl * Ch) * sssp-log-factor n) at-top
  using R-bound t-bound l-bound H-bound
proof eventually-elim
  case (elim n)
  let ?L = sssp-log-factor-one-third n
  have L-nonneg: 0 ≤ ?L
    using sssp-log-factor-one-third-pos[of n] by linarith
  have lH:
    real (l n * H n) ≤ (Cl * Ch) * sssp-log-factor n
  proof -
    have prod:
      real (l n) * real (H n) ≤ (Cl * ?L) * (Ch * ?L)
      by (rule mult-mono[OF elim(3) elim(4)])
      (use Cl-nonneg Ch-pos L-nonneg in simp-all)
    have real (l n * H n) = real (l n) * real (H n)
      by simp
    also have ... ≤ (Cl * ?L) * (Ch * ?L)
      using prod .

```

```

    also have ... = (Cl * Ch) * sssp-log-factor n
      using sssp-log-factor-one-third-square[of n]
      by (simp add: algebra-simps)
    finally show ?thesis .
  qed
  have real (R n + t n + l n * H n) =
    real (R n) + real (t n) + real (l n * H n)
  by simp
  also have ... ≤
    Cr * sssp-log-factor n + Ct * sssp-log-factor n +
    (Cl * Ch) * sssp-log-factor n
  using elim(1) elim(2) lH by linarith
  also have ... = (Cr + Ct + Cl * Ch) * sssp-log-factor n
  by (simp add: algebra-simps)
  finally show ?case .
  qed
  show eventually
    (λn. T n ≤ bmssp-refined-graph-time-bound A R H l t m n) at-top
  using cost-bound .
  qed

theorem bmssp-refined-graph-time-bound-bigo-sssp-time-target-slack:
  assumes Cn-pos: 0 < Cn
  and Cm-pos: 0 < Cm
  and Cv-pos: 0 < Cv
  and Ce-pos: 0 < Ce
  and vertex-count-dominated:
    eventually (λn. real (v n) ≤ Cn * real (m n)) at-top
  and edge-count-dominated:
    eventually (λn. real (m' n) ≤ Cm * real (m n)) at-top
  and vertex-factor:
    eventually
      (λn. real (2 * A n * (2 * l n + 1)) ≤
        Cv * sssp-log-factor n) at-top
  and edge-factor:
    eventually
      (λn. real (R n + t n + l n * H n) ≤
        Ce * sssp-log-factor n) at-top
  shows (λn. real (bmssp-refined-graph-time-bound
    (λ-. A n) (λ-. R n) (λ-. H n) (λ-. l n) (λ-. t n)
    (λ-. m' n) (v n)))
    ∈ O(λn. sssp-time-target m n)
  proof (rule sssp-time-target-bigoI[where C = Cv * Cn + Ce * Cm])
    show 0 < Cv * Cn + Ce * Cm
  proof -
    have 0 < Cv * Cn
    using Cv-pos Cn-pos by (rule mult-pos-pos)
    moreover have 0 < Ce * Cm
    using Ce-pos Cm-pos by (rule mult-pos-pos)

```

```

ultimately show ?thesis
  by linarith
qed
next
show eventually
  ( $\lambda n. \text{real (bmssp-refined-graph-time-bound}$ 
    ( $\lambda-. A\ n$ ) ( $\lambda-. R\ n$ ) ( $\lambda-. H\ n$ ) ( $\lambda-. l\ n$ ) ( $\lambda-. t\ n$ )
    ( $\lambda-. m'\ n$ ) ( $v\ n$ ))  $\leq$ 
    ( $Cv * Cn + Ce * Cm$ ) *  $\text{sssp-time-target } m\ n$ ) at-top
  using vertex-count-dominated edge-count-dominated vertex-factor edge-factor
proof eventually-elim
  case (elim  $n$ )
  let ?vf =  $\text{real } (2 * A\ n * (2 * l\ n + 1))$ 
  let ?ef =  $\text{real } (R\ n + t\ n + l\ n * H\ n)$ 
  let ?log =  $\text{sssp-log-factor } n$ 
  have vertex:
    ?vf *  $\text{real } (v\ n) \leq (Cv * ?log) * (Cn * \text{real } (m\ n))$ 
  proof (rule mult-mono)
    show ?vf  $\leq Cv * ?log$ 
      using elim(3) .
    show  $\text{real } (v\ n) \leq Cn * \text{real } (m\ n)$ 
      using elim(1) .
    show  $0 \leq \text{real } (v\ n)$ 
      by simp
    show  $0 \leq Cv * ?log$ 
  proof -
    have  $0 \leq Cv$ 
      using Cv-pos by linarith
    moreover have  $0 \leq ?log$ 
      using sssp-log-factor-pos[of  $n$ ] by linarith
    ultimately show ?thesis
      by (rule mult-nonneg-nonneg)
  qed
qed
also have  $\dots = (Cv * Cn) * (\text{real } (m\ n) * ?log)$ 
  by (simp add: algebra-simps)
finally have vertex':
  ?vf *  $\text{real } (v\ n) \leq (Cv * Cn) * (\text{real } (m\ n) * ?log)$  .
have edge:
  ?ef *  $\text{real } (m'\ n) \leq (Ce * ?log) * (Cm * \text{real } (m\ n))$ 
proof (rule mult-mono)
  show ?ef  $\leq Ce * ?log$ 
    using elim(4) .
  show  $\text{real } (m'\ n) \leq Cm * \text{real } (m\ n)$ 
    using elim(2) .
  show  $0 \leq \text{real } (m'\ n)$ 
    by simp
  show  $0 \leq Ce * ?log$ 
proof -

```

```

have  $0 \leq Ce$ 
  using  $Ce\text{-pos}$  by linarith
moreover have  $0 \leq ?log$ 
  using  $sssp\text{-log-factor-pos}[of\ n]$  by linarith
ultimately show  $?thesis$ 
  by (rule mult-nonneg-nonneg)
qed
qed
also have  $\dots = (Ce * Cm) * (real\ (m\ n) * ?log)$ 
  by (simp add: algebra-simps)
finally have edge':
   $?ef * real\ (m' n) \leq (Ce * Cm) * (real\ (m\ n) * ?log)$  .
have real (bmssp-refined-graph-time-bound
   $(\lambda\cdot. A\ n)\ (\lambda\cdot. R\ n)\ (\lambda\cdot. H\ n)\ (\lambda\cdot. l\ n)\ (\lambda\cdot. t\ n)$ 
   $(\lambda\cdot. m' n)\ (v\ n)) =$ 
   $?vf * real\ (v\ n) + ?ef * real\ (m' n)$ 
  unfolding bmssp-refined-graph-time-bound-def by simp
also have  $\dots \leq$ 
   $(Cv * Cn) * (real\ (m\ n) * ?log) +$ 
   $(Ce * Cm) * (real\ (m\ n) * ?log)$ 
  using vertex' edge' by linarith
also have  $\dots = (Cv * Cn + Ce * Cm) * sssp\text{-time-target}\ m\ n$ 
  unfolding sssp-time-target-def by (simp add: algebra-simps)
finally show  $?case$  .
qed
qed

```

theorem *bmssp-refined-cost-bound-bigo-sssp-time-target-from-component-bounds-slack*:

```

assumes  $Cn\text{-pos}$ :  $0 < Cn$ 
and  $Cm\text{-pos}$ :  $0 < Cm$ 
and  $Ca\text{-pos}$ :  $0 < Ca$ 
and  $Cl\text{-nonneg}$ :  $0 \leq Cl$ 
and  $Cr\text{-pos}$ :  $0 < Cr$ 
and  $Ct\text{-pos}$ :  $0 < Ct$ 
and  $Ch\text{-pos}$ :  $0 < Ch$ 
and vertex-count-dominated:
  eventually  $(\lambda n. real\ (v\ n) \leq Cn * real\ (m\ n))$  at-top
and edge-count-dominated:
  eventually  $(\lambda n. real\ (m' n) \leq Cm * real\ (m\ n))$  at-top
and A-bound:
  eventually
     $(\lambda n. real\ (A\ n) \leq Ca * sssp\text{-log-factor-one-third}\ n)$  at-top
and l-bound:
  eventually
     $(\lambda n. real\ (l\ n) \leq Cl * sssp\text{-log-factor-one-third}\ n)$  at-top
and R-bound:
  eventually
     $(\lambda n. real\ (R\ n) \leq Cr * sssp\text{-log-factor}\ n)$  at-top

```



```

and t-bound:
  eventually
    ( $\lambda n. \text{real } (t \ n) \leq Ct * \text{sssp-log-factor } n$ ) at-top
and H-bound:
  eventually
    ( $\lambda n. \text{real } (H \ n) \leq Ch * \text{sssp-log-factor-one-third } n$ ) at-top
and cost-bound:
  eventually
    ( $\lambda n. T \ n \leq \text{bmssp-refined-graph-time-bound}$ 
      ( $\lambda-. A \ n$ ) ( $\lambda-. R \ n$ ) ( $\lambda-. H \ n$ ) ( $\lambda-. l \ n$ ) ( $\lambda-. t \ n$ )
      ( $\lambda-. m' \ n$ ) ( $v \ n$ )) at-top)
  shows ( $\lambda n. \text{real } (T \ n) \in O(\lambda n. \text{sssp-time-target } m \ n)$ )
proof -
  have Cv-pos:  $0 < 2 * Ca * (2 * Cl + 1)$ 
    using Ca-pos Cl-nonneg by (intro mult-pos-pos) simp-all
  have Ce-pos:  $0 < Cr + Ct + Cl * Ch$ 
proof -
  have  $0 \leq Cl * Ch$ 
    using Cl-nonneg Ch-pos by simp
  then show ?thesis
    using Cr-pos Ct-pos by linarith
qed
have vertex-factor:
  eventually
    ( $\lambda n. \text{real } (2 * A \ n * (2 * l \ n + 1)) \leq$ 
       $2 * Ca * (2 * Cl + 1) * \text{sssp-log-factor } n$ ) at-top
    using A-bound l-bound eventually-one-le-sssp-log-factor-one-third
proof eventually-elim
  case (elim n)
  let ?L = sssp-log-factor-one-third n
  have L-nonneg:  $0 \leq ?L$ 
    using sssp-log-factor-one-third-pos[of n] by linarith
  have A-le:  $\text{real } (A \ n) \leq Ca * ?L$ 
    using elim(1) .
  have l-le:  $\text{real } (2 * l \ n + 1) \leq (2 * Cl + 1) * ?L$ 
proof -
  have  $\text{real } (2 * l \ n + 1) = 2 * \text{real } (l \ n) + 1$ 
    by simp
  also have  $\dots \leq 2 * (Cl * ?L) + 1$ 
    using elim(2) by linarith
  also have  $\dots \leq 2 * (Cl * ?L) + ?L$ 
    using elim(3) by linarith
  also have  $\dots = (2 * Cl + 1) * ?L$ 
    by (simp add: algebra-simps)
  finally show ?thesis .
qed
have l-rhs-nonneg:  $0 \leq (2 * Cl + 1) * ?L$ 
  using Cl-nonneg L-nonneg by (intro mult-nonneg-nonneg) simp-all
have prod:

```

```

    real (A n) * real (2 * l n + 1) ≤
      (Ca * ?L) * ((2 * Cl + 1) * ?L)
  by (rule mult-mono[OF A-le l-le])
    (use L-nonneg Ca-pos l-rhs-nonneg in simp-all)
  have real (2 * A n * (2 * l n + 1)) =
    2 * (real (A n) * real (2 * l n + 1))
  by (simp add: algebra-simps)
  also have ... ≤ 2 * ((Ca * ?L) * ((2 * Cl + 1) * ?L))
  using prod by simp
  also have ... = 2 * Ca * (2 * Cl + 1) * sssp-log-factor n
  using sssp-log-factor-one-third-square[of n]
  by (simp add: algebra-simps)
  finally show ?case .
qed
have edge-factor:
  eventually
    (λn. real (R n + t n + l n * H n) ≤
      (Cr + Ct + Cl * Ch) * sssp-log-factor n) at-top
  using R-bound t-bound l-bound H-bound
proof eventually-elim
  case (elim n)
  let ?L = sssp-log-factor-one-third n
  have L-nonneg: 0 ≤ ?L
  using sssp-log-factor-one-third-pos[of n] by linarith
  have lH:
    real (l n * H n) ≤ (Cl * Ch) * sssp-log-factor n
  proof -
    have prod:
      real (l n) * real (H n) ≤ (Cl * ?L) * (Ch * ?L)
    by (rule mult-mono[OF elim(3) elim(4)])
      (use Cl-nonneg Ch-pos L-nonneg in simp-all)
    have real (l n * H n) = real (l n) * real (H n)
    by simp
    also have ... ≤ (Cl * ?L) * (Ch * ?L)
    using prod .
    also have ... = (Cl * Ch) * sssp-log-factor n
    using sssp-log-factor-one-third-square[of n]
    by (simp add: algebra-simps)
    finally show ?thesis .
  qed
  have real (R n + t n + l n * H n) =
    real (R n) + real (t n) + real (l n * H n)
  by simp
  also have ... ≤
    Cr * sssp-log-factor n + Ct * sssp-log-factor n +
    (Cl * Ch) * sssp-log-factor n
  using elim(1) elim(2) lH by linarith
  also have ... = (Cr + Ct + Cl * Ch) * sssp-log-factor n
  by (simp add: algebra-simps)

```

```

    finally show ?case .
qed
have graph:
  ( $\lambda n.$  real (bmssp-refined-graph-time-bound
    ( $\lambda-. A\ n$ ) ( $\lambda-. R\ n$ ) ( $\lambda-. H\ n$ ) ( $\lambda-. l\ n$ ) ( $\lambda-. t\ n$ )
    ( $\lambda-. m'\ n$ ) ( $v\ n$ )))
   $\in O(\lambda n.$  sssp-time-target  $m\ n$ )
by (rule bmssp-refined-graph-time-bound-bigo-sssp-time-target-slack
  [OF Cn-pos Cm-pos Cv-pos Ce-pos vertex-count-dominated
    edge-count-dominated vertex-factor edge-factor])
then obtain C where C-pos:  $0 < C$ 
and eventually-graph:
  eventually
    ( $\lambda n.$  norm (real (bmssp-refined-graph-time-bound
      ( $\lambda-. A\ n$ ) ( $\lambda-. R\ n$ ) ( $\lambda-. H\ n$ ) ( $\lambda-. l\ n$ ) ( $\lambda-. t\ n$ )
      ( $\lambda-. m'\ n$ ) ( $v\ n$ )))  $\leq$ 
      C * norm (sssp-time-target  $m\ n$ )) at-top
  unfolding bigo-def by blast
show ?thesis
proof (rule sssp-time-target-bigoI-norm[OF C-pos])
  show eventually
    ( $\lambda n.$  norm (real (T n))  $\leq$  C * norm (sssp-time-target  $m\ n$ )) at-top
  using cost-bound eventually-graph
proof eventually-elim
  case (elim n)
  have norm (real (T n))  $\leq$ 
    norm (real (bmssp-refined-graph-time-bound
      ( $\lambda-. A\ n$ ) ( $\lambda-. R\ n$ ) ( $\lambda-. H\ n$ ) ( $\lambda-. l\ n$ ) ( $\lambda-. t\ n$ )
      ( $\lambda-. m'\ n$ ) ( $v\ n$ )))
  using elim(1) by simp
  also have ...  $\leq$  C * norm (sssp-time-target  $m\ n$ )
  using elim(2) .
  finally show ?case .
qed
qed
qed

```

29 Logarithmic Parameter Schedule

The paper's schedule balances two logarithmic scales. The parameter usually called k is chosen on the order of $\log^{(1/3)} n$, while the recursion depth and partition width use the corresponding $\log^{(2/3)} n$ scale. The two definitions below use ceilings of the real-valued factors so that the executable and inductive parameters remain natural numbers.

The lemmas in this section are mostly analytic bookkeeping. They show that the ceiling parameters are eventually positive, monotone enough for the recurrence, and asymptotically within constant factors of the real logarithmic targets. Those facts are what let the refined graph-time expressions be

rewritten as $O(m * \log^{(2/3)} n)$.

definition *sssp-log-one-third-param* **where**

sssp-log-one-third-param $n = \text{nat } \lceil \text{sssp-log-factor-one-third } n \rceil$

definition *sssp-log-two-thirds-param* **where**

sssp-log-two-thirds-param $n = \text{nat } \lceil \text{sssp-log-factor } n \rceil$

lemma *sssp-log-one-third-param-pos* [simp]:

$0 < \text{sssp-log-one-third-param } n$

unfolding *sssp-log-one-third-param-def*

using *sssp-log-factor-one-third-pos*[of n] **by** *simp*

lemma *sssp-log-two-thirds-param-pos* [simp]:

$0 < \text{sssp-log-two-thirds-param } n$

unfolding *sssp-log-two-thirds-param-def*

using *sssp-log-factor-pos*[of n] **by** *simp*

lemma *real-cuberoom-at-top*:

filterlim $(\lambda x::\text{real}. \text{root } 3 \ x)$ *at-top at-top*

by (*rule filterlim-at-top-at-top*

[where $Q = \lambda x. \text{True}$ **and** $P = \lambda x. \text{True}$ **and** $g = \lambda x. x^3$])

(*auto intro: real-root-le-mono simp: odd-real-root-power-cancel*)

lemma *sssp-log-factor-one-third-at-top*:

filterlim *sssp-log-factor-one-third* *at-top at-top*

proof –

have *ln-at-top'*:

filterlim $(\lambda n::\text{nat}. \ln (\text{real } (n + 2)))$ *at-top at-top*

by (*intro filterlim-compose*[OF *ln-at-top*]

filterlim-compose[OF *filterlim-real-sequentially*]

filterlim-add-const-nat-at-top)

have *root-ln-at-top*:

filterlim $(\lambda n::\text{nat}. \text{root } 3 (\ln (\text{real } (n + 2))))$ *at-top at-top*

by (*rule filterlim-compose*[OF *real-cuberoom-at-top ln-at-top'*])

have *eq*:

$(\lambda n::\text{nat}. \text{root } 3 (\ln (\text{real } (n + 2)))) = \text{sssp-log-factor-one-third}$

proof

fix $n :: \text{nat}$

have *ln-nonneg*: $0 \leq \ln (\text{real } (n + 2))$

by *simp*

have *root 3* $(\ln (\text{real } (n + 2))) =$

$\ln (\text{real } (n + 2)) \text{ powr } (1 / 3)$

using *root-powr-inverse*[of $3 \ln (\text{real } (n + 2))$] *ln-nonneg*

by *simp*

then show *root 3* $(\ln (\text{real } (n + 2))) = \text{sssp-log-factor-one-third } n$

unfolding *sssp-log-factor-one-third-def*

by (*simp add: add.commute*)

qed

show *?thesis*

```

    using root-ln-at-top unfolding eq .
qed

lemma nat-ceiling-at-top:
  filterlim (λx::real. nat ⌈x⌉) at-top at-top
proof (rule filterlim-at-top-at-top
  [where Q = λx. 0 ≤ x and P = λ::nat. True and g = real])
  fix x y :: real
  assume x-nonneg: 0 ≤ x and y-nonneg: 0 ≤ y and xy: x ≤ y
  have ⌈x⌉ ≤ ⌈y⌉
    by (rule ceiling-mono[OF xy])
  then show nat ⌈x⌉ ≤ nat ⌈y⌉
    by (rule nat-mono)
next
  fix n :: nat
  show nat ⌈real n⌉ = n
    by simp
next
  fix n :: nat
  show 0 ≤ real n
    by simp
next
  show eventually (λx::real. 0 ≤ x) at-top
    by (rule eventually-ge-at-top)
next
  show eventually (λ::nat. True) at-top
    by simp
qed

```

```

lemma sssp-log-one-third-param-at-top:
  filterlim sssp-log-one-third-param at-top at-top
  unfolding sssp-log-one-third-param-def
  by (rule filterlim-compose
    [OF nat-ceiling-at-top sssp-log-factor-one-third-at-top])

```

```

lemma eventually-less-sssp-log-one-third-param:
  eventually (λn. K < sssp-log-one-third-param n) at-top
proof -
  have eventually (λn. Suc K ≤ sssp-log-one-third-param n) at-top
    using sssp-log-one-third-param-at-top
    unfolding filterlim-at-top by blast
  then show ?thesis
    by eventually-elim simp
qed

```

```

lemma sssp-log-two-thirds-param-le-one-third-square:
  sssp-log-two-thirds-param n ≤
  sssp-log-one-third-param n * sssp-log-one-third-param n
proof -

```

```

let ?L = sssp-log-factor-one-third n
let ?p = sssp-log-one-third-param n
have L-nonneg:  $0 \leq ?L$ 
  using sssp-log-factor-one-third-pos[of n] by linarith
have L-le-p:  $?L \leq \text{real } ?p$ 
  unfolding sssp-log-one-third-param-def
  using L-nonneg by simp
have sssp-log-factor n  $\leq \text{real } (?p * ?p)$ 
proof –
  have  $?L * ?L \leq \text{real } ?p * \text{real } ?p$ 
    by (rule mult-mono[OF L-le-p L-le-p])
    (use L-nonneg in simp-all)
  then show ?thesis
    using sssp-log-factor-one-third-square[of n] by simp
qed
then show ?thesis
  unfolding sssp-log-two-thirds-param-def by simp
qed

```

```

lemma real-nat-ceiling-le-twice:
  fixes x :: real
  assumes  $1 \leq x$ 
  shows  $\text{real } (\text{nat } \lceil x \rceil) \leq 2 * x$ 
proof –
  have x-nonneg:  $0 \leq x$ 
    using assms by linarith
  have ceil-nat:  $\text{real } (\text{nat } \lceil x \rceil) = \text{of-int } \lceil x \rceil$ 
    using of-nat-int-ceiling[OF x-nonneg] .
  have  $\text{of-int } \lceil x \rceil - 1 < x$ 
    using ceiling-correct[of x] by blast
  then have  $\text{of-int } \lceil x \rceil \leq x + 1$ 
    by linarith
  also have  $\dots \leq 2 * x$ 
    using assms by linarith
  finally show ?thesis
    unfolding ceil-nat .
qed

```

```

lemma eventually-one-le-sssp-log-factor:
  eventually ( $\lambda n. 1 \leq \text{sssp-log-factor } n$ ) at-top
  using eventually-one-le-sssp-log-factor-one-third
proof eventually-elim
  case (elim n)
  let ?L = sssp-log-factor-one-third n
  have  $1 \leq ?L * ?L$ 
proof –
  have L-nonneg:  $0 \leq ?L$ 
    using elim by linarith
  have  $(1::\text{real}) * 1 \leq ?L * ?L$ 

```

```

    by (rule mult-mono[OF elim elim]) (simp-all add: L-nonneg)
  then show ?thesis
    by simp
qed
then show ?case
  using sssp-log-factor-one-third-square[of n] by simp
qed

```

lemma *sssp-log-one-third-param-component-bound:*
eventually
 $(\lambda n. \text{real } (\text{sssp-log-one-third-param } n) \leq 2 * \text{sssp-log-factor-one-third } n) \text{ at-top}$
using *eventually-one-le-sssp-log-factor-one-third*
proof *eventually-elim*
case (*elim* *n*)
show ?case
unfolding *sssp-log-one-third-param-def*
by (rule *real-nat-ceiling-le-twice*[OF *elim*])
qed

lemma *sssp-log-two-thirds-param-component-bound:*
eventually
 $(\lambda n. \text{real } (\text{sssp-log-two-thirds-param } n) \leq 2 * \text{sssp-log-factor } n) \text{ at-top}$
using *eventually-one-le-sssp-log-factor*
proof *eventually-elim*
case (*elim* *n*)
show ?case
unfolding *sssp-log-two-thirds-param-def*
by (rule *real-nat-ceiling-le-twice*[OF *elim*])
qed

lemma *sssp-log-one-third-param-square-arg-component-bound:*
eventually
 $(\lambda n. \text{real } (\text{sssp-log-one-third-param } (n * n)) \leq 4 * \text{sssp-log-factor-one-third } n) \text{ at-top}$
proof (rule *eventually-at-top-linorderI*[of 1])
fix *n* :: *nat*
assume *n-ge*: $1 \leq n$
have *one-le*: $1 \leq \text{sssp-log-factor-one-third } (n * n)$
proof –
have $1 \leq \text{sssp-log-factor-one-third } n$
proof –
have $\exp 1 \leq (3::\text{real})$
by (rule *exp-le*)
also have $\dots \leq \text{real } n + 2$
using *n-ge* **by** *simp*
finally have *exp-le-arg*: $\exp 1 \leq \text{real } n + 2$.
have *ln-ge*: $1 \leq \ln (\text{real } n + 2)$

```

    using exp-le-arg by (simp add: ln-ge-iff)
  show ?thesis
    unfolding sssp-log-factor-one-third-def
    by (rule ge-one-powr-ge-zero[OF ln-ge]) simp
qed
also have ... ≤ sssp-log-factor-one-third (n * n)
  by (rule sssp-log-factor-one-third-mono) (use n-ge in simp)
finally show ?thesis .
qed
have real (sssp-log-one-third-param (n * n)) ≤
  2 * sssp-log-factor-one-third (n * n)
  unfolding sssp-log-one-third-param-def
  by (rule real-nat-ceiling-le-twice[OF one-le])
also have ... ≤ 2 * (2 * sssp-log-factor-one-third n)
  using sssp-log-factor-one-third-square-arg-le[of n] by simp
also have ... = 4 * sssp-log-factor-one-third n
  by simp
finally show real (sssp-log-one-third-param (n * n)) ≤
  4 * sssp-log-factor-one-third n .
qed

lemma sssp-log-two-thirds-param-square-arg-component-bound:
  eventually
    (λn. real (sssp-log-two-thirds-param (n * n)) ≤
      4 * sssp-log-factor n) at-top
proof (rule eventually-at-top-linorderI[of 1])
  fix n :: nat
  assume n-ge: 1 ≤ n
  have one-le: 1 ≤ sssp-log-factor (n * n)
  proof -
    have 1 ≤ sssp-log-factor n
  proof -
    have one-third: 1 ≤ sssp-log-factor-one-third n
  proof -
    have exp 1 ≤ (3::real)
      by (rule exp-le)
    also have ... ≤ real n + 2
      using n-ge by simp
    finally have exp-le-arg: exp 1 ≤ real n + 2 .
    have ln-ge: 1 ≤ ln (real n + 2)
      using exp-le-arg by (simp add: ln-ge-iff)
    show ?thesis
      unfolding sssp-log-factor-one-third-def
      by (rule ge-one-powr-ge-zero[OF ln-ge]) simp
  qed
  let ?L = sssp-log-factor-one-third n
  have 1 ≤ ?L * ?L
  proof -
    have L-nonneg: 0 ≤ ?L

```



```

      using one-third by linarith
    have (1::real) * 1 ≤ ?L * ?L
      by (rule mult-mono[OF one-third one-third])
        (simp-all add: L-nonneg)
    then show ?thesis
      by simp
  qed
  then show ?thesis
    using sssp-log-factor-one-third-square[of n] by simp
  qed
  also have ... ≤ sssp-log-factor (n * n)
    by (rule sssp-log-factor-mono) (use n-ge in simp)
  finally show ?thesis .
  qed
  have real (sssp-log-two-thirds-param (n * n)) ≤
    2 * sssp-log-factor (n * n)
    unfolding sssp-log-two-thirds-param-def
    by (rule real-nat-ceiling-le-twice[OF one-le])
  also have ... ≤ 2 * (2 * sssp-log-factor n)
    using sssp-log-factor-square-arg-le[of n] by simp
  also have ... = 4 * sssp-log-factor n
    by simp
  finally show real (sssp-log-two-thirds-param (n * n)) ≤
    4 * sssp-log-factor n .
  qed

theorem bmssp-refined-cost-bound-bigo-sssp-time-target-log-params:
  assumes Cn-pos: 0 < Cn
  and vertex-count-dominated:
    eventually (λn. real n ≤ Cn * real (m n)) at-top
  and cost-bound:
    eventually
      (λn. T n ≤
        bmssp-refined-graph-time-bound sssp-log-one-third-param
          sssp-log-two-thirds-param sssp-log-one-third-param
          sssp-log-one-third-param sssp-log-two-thirds-param m n) at-top
  shows (λn. real (T n)) ∈ O(λn. sssp-time-target m n)
proof (rule bmssp-refined-cost-bound-bigo-sssp-time-target-from-component-bounds
  [where Cn = Cn and Ca = 2 and Cl = 2 and Cr = 2 and Ct = 2 and Ch
    = 2])
  show 0 < Cn
    using Cn-pos .
  show 0 < (2::real)
    by simp
  show 0 ≤ (2::real)
    by simp
  show 0 < (2::real)
    by simp
  show 0 < (2::real)
    by simp

```

```

    by simp
  show 0 < (2::real)
    by simp
  show eventually (λn. real n ≤ Cn * real (m n)) at-top
    using vertex-count-dominated .
  show eventually
    (λn. real (sssp-log-one-third-param n) ≤
      2 * sssp-log-factor-one-third n) at-top
    by (rule sssp-log-one-third-param-component-bound)
  show eventually
    (λn. real (sssp-log-one-third-param n) ≤
      2 * sssp-log-factor-one-third n) at-top
    by (rule sssp-log-one-third-param-component-bound)
  show eventually
    (λn. real (sssp-log-two-thirds-param n) ≤
      2 * sssp-log-factor n) at-top
    by (rule sssp-log-two-thirds-param-component-bound)
  show eventually
    (λn. real (sssp-log-two-thirds-param n) ≤
      2 * sssp-log-factor n) at-top
    by (rule sssp-log-two-thirds-param-component-bound)
  show eventually
    (λn. real (sssp-log-one-third-param n) ≤
      2 * sssp-log-factor-one-third n) at-top
    by (rule sssp-log-one-third-param-component-bound)
  show eventually
    (λn. T n ≤
      bmssp-refined-graph-time-bound sssp-log-one-third-param
      sssp-log-two-thirds-param sssp-log-one-third-param
      sssp-log-one-third-param sssp-log-two-thirds-param m n) at-top
    using cost-bound .
qed

```

theorem *bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-degree:*

```

fixes D :: nat
assumes Cn-pos: 0 < Cn
and vertex-count-dominated:
  eventually (λn. real n ≤ Cn * real (m n)) at-top
and cost-bound:
  eventually
    (λn. T n ≤
      bmssp-refined-graph-time-bound
      (λn. Suc D * sssp-log-one-third-param n)
      sssp-log-two-thirds-param sssp-log-one-third-param
      sssp-log-one-third-param sssp-log-two-thirds-param m n) at-top
shows (λn. real (T n)) ∈ O(λn. sssp-time-target m n)
proof (rule bmssp-refined-cost-bound-bigo-sssp-time-target-from-component-bounds
  [where Cn = Cn and Ca = 2 * real (Suc D)
    and Cl = 2 and Cr = 2 and Ct = 2 and Ch = 2])

```

```

show 0 < Cn
  using Cn-pos .
show 0 < 2 * real (Suc D)
  by simp
show 0 ≤ (2::real)
  by simp
show 0 < (2::real)
  by simp
show 0 < (2::real)
  by simp
show 0 < (2::real)
  by simp
show eventually (λn. real n ≤ Cn * real (m n)) at-top
  using vertex-count-dominated .
show eventually
  (λn. real (Suc D * sssp-log-one-third-param n) ≤
    2 * real (Suc D) * sssp-log-factor-one-third n) at-top
  using sssp-log-one-third-param-component-bound
proof eventually-elim
  case (elim n)
  have real (Suc D * sssp-log-one-third-param n) =
    real (Suc D) * real (sssp-log-one-third-param n)
    by (simp add: algebra-simps)
  also have ... ≤ real (Suc D) *
    (2 * sssp-log-factor-one-third n)
    using elim by simp
  also have ... = 2 * real (Suc D) * sssp-log-factor-one-third n
    by (simp add: algebra-simps)
  finally show ?case .
qed
show eventually
  (λn. real (sssp-log-one-third-param n) ≤
    2 * sssp-log-factor-one-third n) at-top
  by (rule sssp-log-one-third-param-component-bound)
show eventually
  (λn. real (sssp-log-two-thirds-param n) ≤
    2 * sssp-log-factor n) at-top
  by (rule sssp-log-two-thirds-param-component-bound)
show eventually
  (λn. real (sssp-log-two-thirds-param n) ≤
    2 * sssp-log-factor n) at-top
  by (rule sssp-log-two-thirds-param-component-bound)
show eventually
  (λn. real (sssp-log-one-third-param n) ≤
    2 * sssp-log-factor-one-third n) at-top
  by (rule sssp-log-one-third-param-component-bound)
show eventually
  (λn. T n ≤
    bmssp-refined-graph-time-bound

```

```

      (λn. Suc D * sssp-log-one-third-param n)
      sssp-log-two-thirds-param sssp-log-one-third-param
      sssp-log-one-third-param sssp-log-two-thirds-param m n) at-top
    using cost-bound .
  qed

```

theorem *bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-degree-slack:*

```

  fixes D :: nat
  assumes Cn-pos: 0 < Cn
  and Cm-pos: 0 < Cm
  and vertex-count-dominated:
    eventually (λn. real (v n) ≤ Cn * real (m n)) at-top
  and edge-count-dominated:
    eventually (λn. real (m' n) ≤ Cm * real (m n)) at-top
  and cost-bound:
    eventually
      (λn. T n ≤
        bmssp-refined-graph-time-bound
        (λ-. Suc D * sssp-log-one-third-param n)
        (λ-. sssp-log-two-thirds-param n)
        (λ-. sssp-log-one-third-param n)
        (λ-. sssp-log-one-third-param n)
        (λ-. sssp-log-two-thirds-param n)
        (λ-. m' n) (v n)) at-top
      shows (λn. real (T n)) ∈ O(λn. sssp-time-target m n)
proof (rule bmssp-refined-cost-bound-bigo-sssp-time-target-from-component-bounds-slack
  [where Cn = Cn and Cm = Cm and Ca = 2 * real (Suc D)
    and Cl = 2 and Cr = 2 and Ct = 2 and Ch = 2])
  show 0 < Cn
    using Cn-pos .
  show 0 < Cm
    using Cm-pos .
  show 0 < 2 * real (Suc D)
    by simp
  show 0 ≤ (2::real)
    by simp
  show 0 < (2::real)
    by simp
  show 0 < (2::real)
    by simp
  show 0 < (2::real)
    by simp
  show eventually (λn. real (v n) ≤ Cn * real (m n)) at-top
    using vertex-count-dominated .
  show eventually (λn. real (m' n) ≤ Cm * real (m n)) at-top
    using edge-count-dominated .
  show eventually

```

```

    (λn. real (Suc D * sssp-log-one-third-param n) ≤
      2 * real (Suc D) * sssp-log-factor-one-third n) at-top
  using sssp-log-one-third-param-component-bound
proof eventually-elim
  case (elim n)
  have real (Suc D * sssp-log-one-third-param n) =
    real (Suc D) * real (sssp-log-one-third-param n)
  by (simp add: algebra-simps)
  also have ... ≤ real (Suc D) *
    (2 * sssp-log-factor-one-third n)
  using elim by simp
  also have ... = 2 * real (Suc D) * sssp-log-factor-one-third n
  by (simp add: algebra-simps)
  finally show ?case .
qed
show eventually
  (λn. real (sssp-log-one-third-param n) ≤
    2 * sssp-log-factor-one-third n) at-top
  by (rule sssp-log-one-third-param-component-bound)
show eventually
  (λn. real (sssp-log-two-thirds-param n) ≤
    2 * sssp-log-factor n) at-top
  by (rule sssp-log-two-thirds-param-component-bound)
show eventually
  (λn. real (sssp-log-two-thirds-param n) ≤
    2 * sssp-log-factor n) at-top
  by (rule sssp-log-two-thirds-param-component-bound)
show eventually
  (λn. real (sssp-log-one-third-param n) ≤
    2 * sssp-log-factor-one-third n) at-top
  by (rule sssp-log-one-third-param-component-bound)
show eventually
  (λn. T n ≤
    bmssp-refined-graph-time-bound
    (λ-. Suc D * sssp-log-one-third-param n)
    (λ-. sssp-log-two-thirds-param n)
    (λ-. sssp-log-one-third-param n)
    (λ-. sssp-log-one-third-param n)
    (λ-. sssp-log-two-thirds-param n)
    (λ-. m' n) (v n)) at-top
  using cost-bound .
qed

```

theorem *bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-degree-square-arg:*

```

  fixes D :: nat
  assumes Cn-pos: 0 < Cn
  and vertex-count-dominated:
    eventually (λn. real n ≤ Cn * real (m n)) at-top

```

```

and cost-bound:
  eventually
    ( $\lambda n. T\ n \leq$ 
      bmssp-refined-graph-time-bound
        ( $\lambda n. \text{Suc } D * \text{sssp-log-one-third-param } (n * n)$ )
        ( $\lambda n. \text{sssp-log-two-thirds-param } (n * n)$ )
        ( $\lambda n. \text{sssp-log-one-third-param } (n * n)$ )
        ( $\lambda n. \text{sssp-log-one-third-param } (n * n)$ )
        ( $\lambda n. \text{sssp-log-two-thirds-param } (n * n)$ ) m n) at-top
    shows ( $\lambda n. \text{real } (T\ n) \in O(\lambda n. \text{sssp-time-target } m\ n)$ )
proof (rule bmssp-refined-cost-bound-bigo-sssp-time-target-from-component-bounds
  [where Cn = Cn and Ca =  $4 * \text{real } (\text{Suc } D)$ 
    and Cl =  $4$  and Cr =  $4$  and Ct =  $4$  and Ch =  $4$ ])
show  $0 < Cn$ 
  using Cn-pos .
show  $0 < 4 * \text{real } (\text{Suc } D)$ 
  by simp
show  $0 \leq (4::\text{real})$ 
  by simp
show  $0 < (4::\text{real})$ 
  by simp
show  $0 < (4::\text{real})$ 
  by simp
show  $0 < (4::\text{real})$ 
  by simp
show eventually ( $\lambda n. \text{real } n \leq Cn * \text{real } (m\ n)$ ) at-top
  using vertex-count-dominated .
show eventually
  ( $\lambda n. \text{real } (\text{Suc } D * \text{sssp-log-one-third-param } (n * n)) \leq$ 
     $4 * \text{real } (\text{Suc } D) * \text{sssp-log-factor-one-third } n$ ) at-top
  using sssp-log-one-third-param-square-arg-component-bound
proof eventually-elim
  case (elim n)
  have  $\text{real } (\text{Suc } D * \text{sssp-log-one-third-param } (n * n)) =$ 
     $\text{real } (\text{Suc } D) * \text{real } (\text{sssp-log-one-third-param } (n * n))$ 
  by (simp add: algebra-simps)
  also have  $\dots \leq \text{real } (\text{Suc } D) *$ 
     $(4 * \text{sssp-log-factor-one-third } n)$ 
  using elim by simp
  also have  $\dots = 4 * \text{real } (\text{Suc } D) * \text{sssp-log-factor-one-third } n$ 
  by (simp add: algebra-simps)
  finally show ?case .
qed
show eventually
  ( $\lambda n. \text{real } (\text{sssp-log-one-third-param } (n * n)) \leq$ 
     $4 * \text{sssp-log-factor-one-third } n$ ) at-top
  by (rule sssp-log-one-third-param-square-arg-component-bound)
show eventually
  ( $\lambda n. \text{real } (\text{sssp-log-two-thirds-param } (n * n)) \leq$ 

```

$4 * \text{sssp-log-factor } n) \text{ at-top}$
by (rule *sssp-log-two-thirds-param-square-arg-component-bound*)
show eventually
 $(\lambda n. \text{real } (\text{sssp-log-two-thirds-param } (n * n)) \leq$
 $4 * \text{sssp-log-factor } n) \text{ at-top}$
by (rule *sssp-log-two-thirds-param-square-arg-component-bound*)
show eventually
 $(\lambda n. \text{real } (\text{sssp-log-one-third-param } (n * n)) \leq$
 $4 * \text{sssp-log-factor-one-third } n) \text{ at-top}$
by (rule *sssp-log-one-third-param-square-arg-component-bound*)
show eventually
 $(\lambda n. T \ n \leq$
 $\text{bmssp-refined-graph-time-bound}$
 $(\lambda n. \text{Suc } D * \text{sssp-log-one-third-param } (n * n))$
 $(\lambda n. \text{sssp-log-two-thirds-param } (n * n))$
 $(\lambda n. \text{sssp-log-one-third-param } (n * n))$
 $(\lambda n. \text{sssp-log-one-third-param } (n * n))$
 $(\lambda n. \text{sssp-log-two-thirds-param } (n * n)) \ m \ n) \text{ at-top}$
using *cost-bound* .
qed

theorem *bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-degree-square-arg-slack*:

fixes $D :: \text{nat}$
assumes $Cn\text{-pos}: 0 < Cn$
and $Cm\text{-pos}: 0 < Cm$
and *vertex-count-dominated*:
 $\text{eventually } (\lambda n. \text{real } (v \ n) \leq Cn * \text{real } (m \ n)) \text{ at-top}$
and *edge-count-dominated*:
 $\text{eventually } (\lambda n. \text{real } (m' \ n) \leq Cm * \text{real } (m \ n)) \text{ at-top}$
and *cost-bound*:
 eventually
 $(\lambda n. T \ n \leq$
 $\text{bmssp-refined-graph-time-bound}$
 $(\lambda-. \text{Suc } D * \text{sssp-log-one-third-param } (n * n))$
 $(\lambda-. \text{sssp-log-two-thirds-param } (n * n))$
 $(\lambda-. \text{sssp-log-one-third-param } (n * n))$
 $(\lambda-. \text{sssp-log-one-third-param } (n * n))$
 $(\lambda-. \text{sssp-log-two-thirds-param } (n * n))$
 $(\lambda-. m' \ n) \ (v \ n)) \text{ at-top}$
shows $(\lambda n. \text{real } (T \ n)) \in O(\lambda n. \text{sssp-time-target } m \ n)$

proof (rule *bmssp-refined-cost-bound-bigo-sssp-time-target-from-component-bounds-slack*)

$[\text{where } Cn = Cn \text{ and } Cm = Cm \text{ and } Ca = 4 * \text{real } (\text{Suc } D)$
 $\text{and } Cl = 4 \text{ and } Cr = 4 \text{ and } Ct = 4 \text{ and } Ch = 4]$
show $0 < Cn$
using *Cn-pos* .
show $0 < Cm$
using *Cm-pos* .

```

show 0 < 4 * real (Suc D)
  by simp
show 0 ≤ (4::real)
  by simp
show 0 < (4::real)
  by simp
show 0 < (4::real)
  by simp
show 0 < (4::real)
  by simp
show eventually (λn. real (v n) ≤ Cn * real (m n)) at-top
  using vertex-count-dominated .
show eventually (λn. real (m' n) ≤ Cm * real (m n)) at-top
  using edge-count-dominated .
show eventually
  (λn. real (Suc D * sssp-log-one-third-param (n * n)) ≤
    4 * real (Suc D) * sssp-log-factor-one-third n) at-top
  using sssp-log-one-third-param-square-arg-component-bound
proof eventually-elim
  case (elim n)
  have real (Suc D * sssp-log-one-third-param (n * n)) =
    real (Suc D) * real (sssp-log-one-third-param (n * n))
    by (simp add: algebra-simps)
  also have ... ≤ real (Suc D) *
    (4 * sssp-log-factor-one-third n)
    using elim by simp
  also have ... = 4 * real (Suc D) * sssp-log-factor-one-third n
    by (simp add: algebra-simps)
  finally show ?case .
qed
show eventually
  (λn. real (sssp-log-one-third-param (n * n)) ≤
    4 * sssp-log-factor-one-third n) at-top
  by (rule sssp-log-one-third-param-square-arg-component-bound)
show eventually
  (λn. real (sssp-log-two-thirds-param (n * n)) ≤
    4 * sssp-log-factor n) at-top
  by (rule sssp-log-two-thirds-param-square-arg-component-bound)
show eventually
  (λn. real (sssp-log-two-thirds-param (n * n)) ≤
    4 * sssp-log-factor n) at-top
  by (rule sssp-log-two-thirds-param-square-arg-component-bound)
show eventually
  (λn. real (sssp-log-one-third-param (n * n)) ≤
    4 * sssp-log-factor-one-third n) at-top
  by (rule sssp-log-one-third-param-square-arg-component-bound)
show eventually
  (λn. T n ≤
    bmssp-refined-graph-time-bound

```



```

      (λ-. Suc D * sssp-log-one-third-param (n * n))
      (λ-. sssp-log-two-thirds-param (n * n))
      (λ-. sssp-log-one-third-param (n * n))
      (λ-. sssp-log-one-third-param (n * n))
      (λ-. sssp-log-two-thirds-param (n * n))
      (λ-. m' n) (v n)) at-top
    using cost-bound .
qed

theorem bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-degree-
square-arg-real-slack:
  fixes D :: nat
  assumes Cn-pos: 0 < Cn
  and Ccost-pos: 0 < Ccost
  and vertex-count-dominated:
    eventually (λn. real n ≤ Cn * real (m n)) at-top
  and cost-bound:
    eventually
      (λn. real (T n) ≤ Ccost *
        real (bmssp-refined-graph-time-bound
          (λn. Suc D * sssp-log-one-third-param (n * n))
          (λn. sssp-log-two-thirds-param (n * n))
          (λn. sssp-log-one-third-param (n * n))
          (λn. sssp-log-one-third-param (n * n))
          (λn. sssp-log-two-thirds-param (n * n)) m n)) at-top
  shows (λn. real (T n)) ∈ O(λn. sssp-time-target m n)
proof -
  let ?G = λn. bmssp-refined-graph-time-bound
    (λn. Suc D * sssp-log-one-third-param (n * n))
    (λn. sssp-log-two-thirds-param (n * n))
    (λn. sssp-log-one-third-param (n * n))
    (λn. sssp-log-one-third-param (n * n))
    (λn. sssp-log-two-thirds-param (n * n)) m n
  have G-bigo: (λn. real (?G n)) ∈ O(λn. sssp-time-target m n)
  by (rule bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-
degree-square-arg
    [where D = D and T = ?G, OF Cn-pos vertex-count-dominated]) simp
  then obtain C where C-pos: 0 < C
  and G-bound:
    eventually
      (λn. norm (real (?G n)) ≤
        C * norm (sssp-time-target m n)) at-top
  unfolding bigo-def by blast
  have Cprod-pos: 0 < Ccost * C
  using Ccost-pos C-pos by simp
  show ?thesis
proof (rule sssp-time-target-bigoI-norm[OF Cprod-pos])
  show eventually
    (λn. norm (real (T n)) ≤

```

```

      Ccost * C * norm (sssp-time-target m n)) at-top
    using cost-bound G-bound
  proof eventually-elim
    case (elim n)
    have norm (real (T n)) = real (T n)
      by simp
    also have ... ≤ Ccost * real (?G n)
      using elim(1) .
    also have ... = Ccost * norm (real (?G n))
      by simp
    also have ... ≤ Ccost * (C * norm (sssp-time-target m n))
      using elim(2) Ccost-pos by simp
    also have ... = Ccost * C * norm (sssp-time-target m n)
      by (simp add: algebra-simps)
    finally show ?case .
  qed
qed
qed

```

theorem *bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-degree-real-slack:*

```

  fixes D :: nat
  assumes Cn-pos: 0 < Cn
  and Ccost-pos: 0 < Ccost
  and vertex-count-dominated:
    eventually (λn. real n ≤ Cn * real (m n)) at-top
  and cost-bound:
    eventually
      (λn. real (T n) ≤ Ccost *
        real (bmssp-refined-graph-time-bound
          (λn. Suc D * sssp-log-one-third-param n)
          sssp-log-two-thirds-param sssp-log-one-third-param
          sssp-log-one-third-param sssp-log-two-thirds-param m n)) at-top
  shows (λn. real (T n)) ∈ O(λn. sssp-time-target m n)
proof -
  let ?G = λn. bmssp-refined-graph-time-bound
    (λn. Suc D * sssp-log-one-third-param n)
    sssp-log-two-thirds-param sssp-log-one-third-param
    sssp-log-one-third-param sssp-log-two-thirds-param m n
  have G-bigo: (λn. real (?G n)) ∈ O(λn. sssp-time-target m n)
  by (rule bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-degree
    [where D = D and T = ?G, OF Cn-pos vertex-count-dominated]) simp
  then obtain C where C-pos: 0 < C
  and G-bound:
    eventually
      (λn. norm (real (?G n)) ≤
        C * norm (sssp-time-target m n)) at-top
  unfolding bigo-def by blast
  have Cprod-pos: 0 < Ccost * C

```

```

    using Ccost-pos C-pos by simp
  show ?thesis
proof (rule sssp-time-target-bigoI-norm[OF Cprod-pos])
  show eventually
    (λn. norm (real (T n)) ≤
      Ccost * C * norm (sssp-time-target m n)) at-top
  using cost-bound G-bound
proof eventually-elim
  case (elim n)
  have norm (real (T n)) = real (T n)
  by simp
  also have ... ≤ Ccost * real (?G n)
  using elim(1) .
  also have ... = Ccost * norm (real (?G n))
  by simp
  also have ... ≤ Ccost * (C * norm (sssp-time-target m n))
  using elim(2) Ccost-pos by simp
  also have ... = Ccost * C * norm (sssp-time-target m n)
  by (simp add: algebra-simps)
  finally show ?case .
qed
qed
qed

```

```

context unique-shortest-digraph
begin

```

From this point on the arithmetic bounds are tied to a concrete finite graph. The definitions *vertex-count* and *edge-count* are simple wrappers around the locale's finite vertex and edge sets, but naming them keeps the final theorems independent of internal set notation. The parent-edge construction is used to relate reachable vertices to edges, which is the standard sparse graph move behind replacing a vertex term by an edge term when all vertices are reachable.

```

definition vertex-count where
  vertex-count = card V

```

```

definition edge-count where
  edge-count = card E

```

```

definition parent-edge-to where
  parent-edge-to v = (last (butlast (shortest-path-to v)), v)

```

```

lemma vertex-count-pos [simp]:
  0 < vertex-count

```

```

proof -
  have V ≠ {}
  using source-in-V by blast
  then show ?thesis

```

```

      unfolding vertex-count-def using finite-V by (simp add: card-gt-0-iff)
    qed

lemma edge-count-le-vertex-count-square:
  edge-count ≤ vertex-count * vertex-count
proof -
  have E-subset:  $E \subseteq V \times V$ 
  using edge-in-V by auto
  have edge-count = card E
  unfolding edge-count-def by simp
  also have ... ≤ card (V × V)
  by (rule card-mono[OF E-subset]) (use finite-V in simp)
  also have ... = vertex-count * vertex-count
  unfolding vertex-count-def by (simp add: card-cartesian-product)
  finally show ?thesis .
qed

lemma parent-edge-to-in-E:
  assumes reach-v: reachable s v
  and v-ne:  $v \neq s$ 
  shows parent-edge-to  $v \in E$ 
proof -
  let ?p = shortest-path-to v
  have sp: shortest-walk s ?p v
  by (rule shortest-path-to-shortest[OF reach-v])
  have walk-p: walk ?p
  using sp unfolding shortest-walk-def simple-walk-betw-def walk-betw-def by
blast
  have p-ne:  $?p \neq []$ 
  using sp unfolding shortest-walk-def simple-walk-betw-def walk-betw-def by
blast
  have hd-p: hd ?p = s
  using sp unfolding shortest-walk-def simple-walk-betw-def walk-betw-def by
blast
  have last-p: last ?p = v
  using sp unfolding shortest-walk-def simple-walk-betw-def walk-betw-def by
blast
  have len-ge2:  $1 < \text{length } ?p$ 
  proof (rule ccontr)
    assume  $\neg 1 < \text{length } ?p$ 
    then have length ?p = 1
    using p-ne by (cases ?p) auto
    then have hd ?p = last ?p
    by (cases ?p) auto
    then show False
    using hd-p last-p v-ne by simp
  qed
  have but-ne: butlast ?p  $\neq []$ 
  using len-ge2 by (cases ?p) auto

```

obtain $q\ u$ **where** *but*: $\text{butlast } ?p = q @ [u]$
using *but-ne* **by** (*cases butlast ?p rule: rev-cases*) *auto*
have *p-decomp*: $?p = q @ [u, v]$
proof –
have $?p = \text{butlast } ?p @ [\text{last } ?p]$
using *p-ne* **by** *simp*
then show *?thesis*
using *but last-p* **by** *simp*
qed
have *i-lt*: $\text{Suc } (\text{length } q) < \text{length } ?p$
using *p-decomp* **by** *simp*
have *edge*: $(?p ! \text{length } q, ?p ! \text{Suc } (\text{length } q)) \in E$
by (*rule walk-nth-edge*[*OF walk-p i-lt*])
have $?p ! \text{length } q = u$
using *p-decomp* **by** (*simp add: nth-append*)
moreover have $?p ! \text{Suc } (\text{length } q) = v$
using *p-decomp* **by** (*simp add: nth-append*)
ultimately have $(u, v) \in E$
using *edge* **by** *simp*
moreover have $\text{last } (\text{butlast } ?p) = u$
using *but* **by** *simp*
ultimately show *?thesis*
unfolding *parent-edge-to-def* **by** *simp*
qed

theorem *vertex-count-le-Suc-edge-count-if-all-reachable*:
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s\ v$
shows $\text{vertex-count} \leq \text{Suc } \text{edge-count}$
proof –
have *image-subset*: $\text{parent-edge-to } ' (V - \{s\}) \subseteq E$
proof
fix e
assume $e \in \text{parent-edge-to } ' (V - \{s\})$
then obtain v **where** $v: v \in V\ v \neq s\ e = \text{parent-edge-to } v$
by *blast*
have $\text{reachable } s\ v$
by (*rule all-reachable*[*OF v(1)*])
then show $e \in E$
using *parent-edge-to-in-E*[*OF - v(2)*] $v(3)$ **by** *simp*
qed
have *inj*: $\text{inj-on } \text{parent-edge-to } (V - \{s\})$
unfolding *parent-edge-to-def inj-on-def* **by** *auto*
have *card-image*:
 $\text{card } (V - \{s\}) = \text{card } (\text{parent-edge-to } ' (V - \{s\}))$
using *inj finite-V* **by** (*simp add: card-image*)
also have $\dots \leq \text{edge-count}$
unfolding *edge-count-def*
by (*rule card-mono*[*OF finite-E image-subset*])
finally have $\text{without-}s: \text{card } (V - \{s\}) \leq \text{edge-count} .$

```

have V-decomp:  $V = \text{insert } s (V - \{s\})$ 
  using source-in-V by blast
have card V = card (insert s (V - {s}))
  using V-decomp by simp
also have ... = Suc (card (V - {s}))
  by (rule card-insert-disjoint) (use finite-V in auto)
finally have card V = Suc (card (V - {s})) .
then show ?thesis
  using without-s unfolding vertex-count-def by simp
qed

corollary vertex-count-le-twice-edge-count-if-all-reachable:
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s v$ 
  and nontrivial:  $0 < \text{edge-count}$ 
  shows vertex-count  $\leq 2 * \text{edge-count}$ 
proof -
  have vertex-count  $\leq \text{Suc edge-count}$ 
    by (rule vertex-count-le-Suc-edge-count-if-all-reachable[OF all-reachable])
  also have ...  $\leq 2 * \text{edge-count}$ 
    using nontrivial by simp
  finally show ?thesis .
qed

lemma card-subset-vertex-count:
  assumes  $S \subseteq V$ 
  shows card S  $\leq \text{vertex-count}$ 
  unfolding vertex-count-def
  by (rule card-mono[OF finite-V assms])

lemma eventually-top-level-threshold-exceeds-vertex-count:
  eventually
    ( $\lambda N. \text{vertex-count} <$ 
      sssp-log-one-third-param N *
      bmssp-level-cap (sssp-log-one-third-param N)
      (sssp-log-two-thirds-param N) (sssp-log-one-third-param N))
  at-top
  using eventually-less-sssp-log-one-third-param[of vertex-count]
proof eventually-elim
  case (elim N)
  let ?p = sssp-log-one-third-param N
  let ?q = sssp-log-two-thirds-param N
  let ?cap = bmssp-level-cap ?p ?q ?p
  have cap-pos:  $0 < ?cap$ 
    unfolding bmssp-level-cap-def by simp
  have ?p  $\leq ?p * ?cap$ 
    using cap-pos by (cases ?cap) simp-all
  then show ?case
    using elim by linarith
qed

```

```

lemma find-pivots-pivots-capped-singleton-empty-if-params-exceed-vertex-count:
  assumes k-gt:  $\text{vertex-count} < k$ 
    and cap-ge:  $\text{vertex-count} \leq \text{cap}$ 
  shows find-pivots-pivots-capped k cap d  $\{s\}$   $B = \{\}$ 
proof –
  let ?seen = find-pivots-seen-capped k cap d  $\{s\}$   $B$ 
  have source-subset:  $\{s\} \subseteq V$ 
    using source-in-V by simp
  have seen-subset:  $?seen \subseteq V$ 
    unfolding find-pivots-seen-capped-def
    by (rule fp-iter-capped-seen-subset-V[OF source-subset source-subset])
  have card-seen-le:  $\text{card } ?seen \leq \text{vertex-count}$ 
    by (rule card-subset-vertex-count[OF seen-subset])
  have no-overflow:  $\neg \text{card } ?seen > \text{cap}$ 
    using card-seen-le cap-ge by linarith
  have no-large-tree:
     $\{u \in \{s\}. k \leq \text{card } (\text{tree-of } u \cap ?seen)\} = \{\}$ 
  proof
    show  $\{u \in \{s\}. k \leq \text{card } (\text{tree-of } u \cap ?seen)\} \subseteq \{\}$ 
    proof
      fix u
      assume u-mem:  $u \in \{u \in \{s\}. k \leq \text{card } (\text{tree-of } u \cap ?seen)\}$ 
      have tree-seen-subset:  $\text{tree-of } u \cap ?seen \subseteq V$ 
        unfolding tree-of-def by blast
      have  $\text{card } (\text{tree-of } u \cap ?seen) \leq \text{vertex-count}$ 
        by (rule card-subset-vertex-count[OF tree-seen-subset])
      moreover have  $k \leq \text{card } (\text{tree-of } u \cap ?seen)$ 
        using u-mem by blast
      ultimately show  $u \in \{\}$ 
        using k-gt by linarith
    qed
  qed simp
  show ?thesis
    unfolding find-pivots-pivots-capped-def
    using no-overflow no-large-tree by simp
qed

lemma eventually-top-level-pivots-empty:
  eventually
    ( $\lambda N. \text{find-pivots-pivots-capped } (\text{sssp-log-one-third-param } N)$ 
      (bmssp-level-cap (sssp-log-one-third-param N)
        (sssp-log-two-thirds-param N) (sssp-log-one-third-param N))
       $d \{s\} B = \{\}$ )
  at-top
  using eventually-less-sssp-log-one-third-param[of vertex-count]
proof eventually-elim
  case (elim N)
  let ?p = sssp-log-one-third-param N

```

```

let ?q = sssp-log-two-thirds-param N
let ?cap = bmssp-level-cap ?p ?q ?p
have cap-ge-p: ?p ≤ ?cap
  by (rule bmssp-level-cap-ge-level)
have cap-ge-vertex: vertex-count ≤ ?cap
  using elim cap-ge-p by linarith
show ?case
  by (rule find-pivots-pivots-capped-singleton-empty-if-params-exceed-vertex-count
    [OF elim cap-ge-vertex])
qed

```

```

lemma edge-count-outgoing-bound:
  card (outgoing-edges X) ≤ edge-count
proof –
  have outgoing-edges X ⊆ E
  unfolding outgoing-edges-def by blast
  then show ?thesis
  unfolding edge-count-def by (rule card-mono[OF finite-E])
qed

```

```

lemma edge-count-outgoing-bound-tree:
  card (outgoing-edges (bound-tree S B)) ≤ edge-count
  by (rule edge-count-outgoing-bound)

```

The range-based edge definitions are the combinatorial support for the direct-insert and lower-batch split. An edge is charged to a range according to the relaxed value at its source, not merely according to the source vertex. This matches the algorithm: a completed child range may generate relaxations that fall below the pull bound, inside the parent output bound, or outside the current problem.

The length lemmas connect executable lists of relaxation pairs to the mathematical edge ranges. The disjointness and split lemmas that follow make it possible to sum per-child edge charges and bound them by the outgoing edges of the assembled parent range.

```

definition outgoing-edges-range where
  outgoing-edges-range U a B =
    {(u, v) ∈ E. u ∈ U ∧ a ≤ dist s u + w u v ∧
      below-bound (dist s u + w u v) B}

```

```

lemma outgoing-edges-range-subset-E:
  outgoing-edges-range U a B ⊆ E
  unfolding outgoing-edges-range-def by blast

```

```

lemma outgoing-edges-range-subset-outgoing-edges:
  outgoing-edges-range U a B ⊆ outgoing-edges U
  unfolding outgoing-edges-range-def outgoing-edges-def by blast

```

```

lemma finite-outgoing-edges-range [simp]:

```


finite (*outgoing-edges-range* *U a B*)
by (*rule finite-subset*[*OF outgoing-edges-range-subset-E finite-E*])

lemma *card-outgoing-edges-range-le-edge-count*:
card (*outgoing-edges-range* *U a B*) $\leq$ *edge-count*
unfolding *edge-count-def*
by (*rule card-mono*[*OF finite-E outgoing-edges-range-subset-E*])

definition *edge-relaxation-pairs-in-bound* **where**
edge-relaxation-pairs-in-bound *d U L B* =
map ($\lambda(u, v). (v, d\ u + w\ u\ v)$)
filter ($\lambda(u, v). L \leq d\ u + w\ u\ v \wedge$
below-bound (*d u + w u v*) *B*)
(edge-list-of (outgoing-edges U)))

lemma *edge-relaxation-pairs-in-bound-value-ge-low*:
assumes $(x, b) \in \text{set } (\text{edge-relaxation-pairs-in-bound } d\ U\ L\ B)$
shows $L \leq b$
using *assms* **unfolding** *edge-relaxation-pairs-in-bound-def* **by** *auto*

lemma *edge-relaxation-pairs-in-bound-value-below-bound*:
assumes $(x, b) \in \text{set } (\text{edge-relaxation-pairs-in-bound } d\ U\ L\ B)$
shows *below-bound* *b B*
using *assms* **unfolding** *edge-relaxation-pairs-in-bound-def* **by** *auto*

lemma *edge-relaxation-pairs-in-bound-length-le-outgoing-edges-range*:
assumes *complete*: *complete-on* *d U*
and *reaches*: $\bigwedge u. u \in U \implies \text{reachable } s\ u$
shows *length* (*edge-relaxation-pairs-in-bound* *d U L B*) $\leq$
card (*outgoing-edges-range* *U L B*)
proof –
let *?es* = *edge-list-of* (*outgoing-edges* *U*)
let *?fs* =
filter ($\lambda(u, v). L \leq d\ u + w\ u\ v \wedge$
below-bound (*d u + w u v*) *B*) *?es*
have *set-es*: *set* *?es* = *outgoing-edges* *U*
using *edge-list-of-properties*(1)[*OF finite-outgoing-edges*] .
have *distinct-fs*: *distinct* *?fs*
using *edge-list-of-properties*(2)[*OF finite-outgoing-edges*] **by** *simp*
have *subset*:
set *?fs* $\subseteq$ *outgoing-edges-range* *U L B*
proof
fix *e*
assume *e*: $e \in \text{set } ?fs$
then obtain *u v* **where** *ev*: $e = (u, v)$
by (*metis surj-pair*)
have *edge*: $(u, v) \in E$
and *uU*: $u \in U$
and *low*: $L \leq d\ u + w\ u\ v$

```

    and high: below-bound ( $d\ u + w\ u\ v$ )  $B$ 
    using  $e$  set-es unfolding  $ev$  outgoing-edges-def by auto
  have  $du$ :  $d\ u = dist\ s\ u$ 
    using complete reaches[ $OF\ uU$ ]  $uU$  unfolding complete-on-def by blast
  show  $e \in outgoing\ edges\ range\ U\ L\ B$ 
    using edge  $uU$  low high unfolding  $ev\ du$  outgoing-edges-range-def by simp
qed
have length (edge-relaxation-pairs-in-bound  $d\ U\ L\ B$ ) = length  $?fs$ 
  unfolding edge-relaxation-pairs-in-bound-def by simp
also have  $\dots = card\ (set\ ?fs)$ 
  using distinct-card[ $OF\ distinct\ fs$ ] by simp
also have  $\dots \leq card\ (outgoing\ edges\ range\ U\ L\ B)$ 
  by (rule card-mono[ $OF\ finite\ outgoing\ edges\ range\ subset$ ])
finally show  $?thesis$  .
qed

```

```

lemma partition-batch-cost-bound-edge-relaxation-pairs-in-bound:
  assumes cost:
    partition-batch-cost-bound  $c\ t$ 
    (edge-relaxation-pairs-in-bound  $d\ U\ L\ B$ )
  and complete: complete-on  $d\ U$ 
  and reaches:  $\bigwedge u. u \in U \implies reachable\ s\ u$ 
  shows  $c \leq t * card\ (outgoing\ edges\ range\ U\ L\ B)$ 
proof -
  have len-le: length (edge-relaxation-pairs-in-bound  $d\ U\ L\ B$ )  $\leq$ 
    card (outgoing-edges-range  $U\ L\ B$ )
  by (rule edge-relaxation-pairs-in-bound-length-le-outgoing-edges-range
    [OF complete reaches])
  have  $c \leq t * length\ (edge-relaxation-pairs-in-bound\ d\ U\ L\ B)$ 
    using cost unfolding partition-batch-cost-bound-def by simp
  also have  $\dots \leq t * card\ (outgoing\ edges\ range\ U\ L\ B)$ 
    using len-le by simp
  finally show  $?thesis$  .
qed

```

```

lemma edge-relaxation-pairs-between-length-le-outgoing-edges-range:
  assumes complete: complete-on  $d\ U$ 
  and reaches:  $\bigwedge u. u \in U \implies reachable\ s\ u$ 
  shows length (edge-relaxation-pairs-between  $d\ U\ L\ H$ )  $\leq$ 
    card (outgoing-edges-range  $U\ L\ (Fin\ H)$ )
proof -
  let  $?es = edge\ list\ of\ (outgoing\ edges\ U)$ 
  let  $?fs =$ 
    filter ( $\lambda(u, v). L \leq d\ u + w\ u\ v \wedge d\ u + w\ u\ v < H$ )  $?es$ 
  have set-es: set  $?es = outgoing\ edges\ U$ 
    using edge-list-of-properties(1)[OF finite-outgoing-edges] .
  have distinct-fs: distinct  $?fs$ 
    using edge-list-of-properties(2)[OF finite-outgoing-edges] by simp
  have subset:

```

```

    set ?fs  $\subseteq$  outgoing-edges-range  $U L (Fin H)$ 
  proof
    fix e
    assume e:  $e \in set\ ?fs$ 
    then obtain u v where ev:  $e = (u, v)$ 
      by (metis surj-pair)
    have edge:  $(u, v) \in E$ 
    and uU:  $u \in U$ 
    and low:  $L \leq d\ u + w\ u\ v$ 
    and high:  $d\ u + w\ u\ v < H$ 
    using e set-es unfolding ev outgoing-edges-def by auto
    have du:  $d\ u = dist\ s\ u$ 
    using complete reaches[OF uU] uU unfolding complete-on-def by blast
    show  $e \in outgoing-edges-range\ U L (Fin H)$ 
    using edge uU low high unfolding ev du outgoing-edges-range-def by simp
  qed
  have length (edge-relaxation-pairs-between  $d\ U L H$ ) = length ?fs
    unfolding edge-relaxation-pairs-between-def by simp
  also have ... = card (set ?fs)
    using distinct-card[OF distinct-fs] by simp
  also have ...  $\leq$  card (outgoing-edges-range  $U L (Fin H)$ )
    by (rule card-mono[OF finite-outgoing-edges-range subset])
  finally show ?thesis .
qed

lemma edge-relaxation-pairs-between-length-le-range-tree-outgoing-edges-range:
  assumes complete: complete-on  $d$  (range-tree  $S\ a\ B$ )
  shows length (edge-relaxation-pairs-between  $d$  (range-tree  $S\ a\ B$ )  $L H$ )  $\leq$ 
    card (outgoing-edges-range (range-tree  $S\ a\ B$ )  $L (Fin H)$ )
proof (rule edge-relaxation-pairs-between-length-le-outgoing-edges-range[OF complete])
  fix u
  assume u  $\in$  range-tree  $S\ a\ B$ 
  then show reachable  $s\ u$ 
    unfolding range-tree-def tree-set-def tree-path-def by blast
qed

lemma outgoing-edges-range-mono-sources:
  assumes  $U \subseteq W$ 
  shows outgoing-edges-range  $U\ a\ B \subseteq outgoing-edges-range\ W\ a\ B$ 
  using assms unfolding outgoing-edges-range-def by blast

lemma outgoing-edges-range-split:
  assumes a-le-b:  $a \leq b$ 
  and b-le-B: bound-le (Fin b) B
  shows outgoing-edges-range  $U\ a\ B =$ 
    outgoing-edges-range  $U\ a\ (Fin\ b) \cup outgoing-edges-range\ U\ b\ B$ 
  using assms unfolding outgoing-edges-range-def
  by (cases B) auto

```

lemma *outgoing-edges-range-split-disjoint*:
 $outgoing-edges-range\ U\ a\ (Fin\ b) \cap outgoing-edges-range\ U\ b\ B = \{\}$
unfolding *outgoing-edges-range-def* **by** *auto*

lemma *card-outgoing-edges-range-split*:
assumes *a-le-b*: $a \leq b$
and *b-le-B*: *bound-le* (*Fin* *b*) *B*
shows $card\ (outgoing-edges-range\ U\ a\ B) =$
 $card\ (outgoing-edges-range\ U\ a\ (Fin\ b)) +$
 $card\ (outgoing-edges-range\ U\ b\ B)$
proof –
have *split*:
 $outgoing-edges-range\ U\ a\ B =$
 $outgoing-edges-range\ U\ a\ (Fin\ b) \cup outgoing-edges-range\ U\ b\ B$
by (*rule* *outgoing-edges-range-split*[*OF* *a-le-b* *b-le-B*])
have *disj*:
 $outgoing-edges-range\ U\ a\ (Fin\ b) \cap outgoing-edges-range\ U\ b\ B = \{\}$
by (*rule* *outgoing-edges-range-split-disjoint*)
show *?thesis*
unfolding *split*
by (*rule* *card-Un-disjoint*) (*use* *disj* **in** *auto*)
qed

lemma *card-outgoing-edges-range-lower-direct-split*:
assumes *b-le-beta*: $b \leq beta$
and *beta-le-B*: *bound-le* (*Fin* *beta*) *B*
shows $card\ (outgoing-edges-range\ U\ b\ (Fin\ beta)) +$
 $card\ (outgoing-edges-range\ U\ beta\ B) =$
 $card\ (outgoing-edges-range\ U\ b\ B)$
using *card-outgoing-edges-range-split*[*OF* *b-le-beta* *beta-le-B*, *of* *U*]
by *simp*

lemma *card-outgoing-edges-range-zero-lower-direct-split-le*:
assumes *beta-le-B*: *bound-le* (*Fin* *beta*) *B*
and *nonneg*:
 $\bigwedge u\ v. \llbracket (u, v) \in E; u \in U \rrbracket \implies 0 \leq dist\ s\ u + w\ u\ v$
shows $card\ (outgoing-edges-range\ U\ 0\ (Fin\ beta)) +$
 $card\ (outgoing-edges-range\ U\ beta\ B) \leq$
 $card\ (outgoing-edges-range\ U\ 0\ B)$
proof –
let *?lower* = *outgoing-edges-range* *U* *0* (*Fin* *beta*)
let *?direct* = *outgoing-edges-range* *U* *beta* *B*
let *?full* = *outgoing-edges-range* *U* *0* *B*
have *disj*: *?lower* $\cap$ *?direct* = $\{\}$
by (*rule* *outgoing-edges-range-split-disjoint*)
have *union-subset*: *?lower* $\cup$ *?direct* $\subseteq$ *?full*
proof
fix *e*

```

assume  $e: e \in ?lower \cup ?direct$ 
show  $e \in ?full$ 
proof (cases  $e \in ?lower$ )
  case True
    then show ?thesis
      using beta-le-B unfolding outgoing-edges-range-def
      by (cases B) auto
  next
    case False
    then have direct:  $e \in ?direct$ 
      using e by blast
    then obtain  $u\ v$  where  $e_{uv}: e = (u, v)$ 
      by (metis surj-pair)
    have  $edge: (u, v) \in E$  and  $uU: u \in U$ 
      using direct unfolding e-uv outgoing-edges-range-def by auto
    have  $0 \leq dist\ s\ u + w\ u\ v$ 
      by (rule nonneg[OF edge uU])
    then show ?thesis
      using direct unfolding e-uv outgoing-edges-range-def by auto
  qed
qed
have card-union:
   $card\ (?lower \cup ?direct) = card\ ?lower + card\ ?direct$ 
  using disj by (simp add: card-Un-disjoint)
have  $card\ (?lower \cup ?direct) \leq card\ ?full$ 
  by (rule card-mono[OF finite-outgoing-edges-range union-subset])
then show ?thesis
  using card-union by linarith
qed

lemma outgoing-edges-range-disjoint-sources:
  assumes  $U \cap W = \{\}$ 
  shows  $outgoing-edges-range\ U\ a\ B \cap outgoing-edges-range\ W\ c\ D = \{\}$ 
  using assms unfolding outgoing-edges-range-def by blast

fun pairwise-disjoint-list where
   $pairwise-disjoint-list\ [] \longleftrightarrow True$ 
|  $pairwise-disjoint-list\ (X\ \# \ Xs) \longleftrightarrow$ 
   $(\forall Y \in set\ Xs.\ X \cap Y = \{\}) \wedge pairwise-disjoint-list\ Xs$ 

lemma pairwise-disjoint-list-outgoing-edges-range:
  assumes pairwise-disjoint-list Us
  shows  $pairwise-disjoint-list\ (map\ (\lambda U.\ outgoing-edges-range\ U\ a\ B)\ Us)$ 
  using assms
proof (induction Us)
  case Nil
  then show ?case by simp
next
  case (Cons U Us)

```

```

have disj-head:
   $\forall Y \in \text{set } (\text{map } (\lambda U. \text{outgoing-edges-range } U \ a \ B) \ Us).$ 
   $\text{outgoing-edges-range } U \ a \ B \cap Y = \{\}$ 
proof
  fix Y
  assume  $Y \in \text{set } (\text{map } (\lambda U. \text{outgoing-edges-range } U \ a \ B) \ Us)$ 
  then obtain W where  $W: W \in \text{set } Us \ Y = \text{outgoing-edges-range } W \ a \ B$ 
  by auto
  have  $U \cap W = \{\}$ 
  using Cons.prems W(1) by simp
  then show  $\text{outgoing-edges-range } U \ a \ B \cap Y = \{\}$ 
  unfolding W(2) by (rule outgoing-edges-range-disjoint-sources)
qed
have tail: pairwise-disjoint-list ( $\text{map } (\lambda U. \text{outgoing-edges-range } U \ a \ B) \ Us$ )
  using Cons.IH Cons.prems by simp
show ?case
  using disj-head tail by simp
qed

lemma edge-outdegree-le-edge-count:
  edge-outdegree-le edge-count
  unfolding edge-outdegree-le-def
  using edge-count-outgoing-bound by blast

lemma card-Union-pairwise-disjoint-list:
  assumes disj: pairwise-disjoint-list Xs
  and finite-sets:  $\bigwedge X. X \in \text{set } Xs \implies \text{finite } X$ 
  shows  $\text{card } (\bigcup (\text{set } Xs)) = \text{sum-list } (\text{map } \text{card } Xs)$ 
  using assms
proof (induction Xs)
  case Nil
  then show ?case by simp
next
  case (Cons X Xs)
  have finite-X: finite X
  using Cons.prems by simp
  have finite-tail: finite ( $\bigcup (\text{set } Xs)$ )
  using Cons.prems by auto
  have disjoint-tail:  $X \cap \bigcup (\text{set } Xs) = \{\}$ 
  using Cons.prems by auto
  have tail:
     $\text{card } (\bigcup (\text{set } Xs)) = \text{sum-list } (\text{map } \text{card } Xs)$ 
  using Cons.IH Cons.prems by auto
  have  $\text{card } (\bigcup (\text{set } (X \# Xs))) =$ 
     $\text{card } X + \text{card } (\bigcup (\text{set } Xs))$ 
  by (simp add: card-Un-disjoint finite-X finite-tail disjoint-tail)
  then show ?case
  using tail by simp
qed

```

```

lemma sum-card-outgoing-edges-range-disjoint-sources-le-edge-count:
  assumes disj: pairwise-disjoint-list Us
  shows sum-list (map ( $\lambda U$ . card (outgoing-edges-range U a B)) Us)  $\leq$  edge-count
proof –
  have disj-ranges:
    pairwise-disjoint-list (map ( $\lambda U$ . outgoing-edges-range U a B) Us)
  by (rule pairwise-disjoint-list-outgoing-edges-range[OF disj])
  have finite-ranges:
     $\bigwedge X. X \in \text{set } (\text{map } (\lambda U. \text{outgoing-edges-range } U \ a \ B) \ Us) \implies \text{finite } X$ 
  by auto
  have card-union:
    card ( $\bigcup (\text{set } (\text{map } (\lambda U. \text{outgoing-edges-range } U \ a \ B) \ Us))$ ) =
      sum-list (map card (map ( $\lambda U$ . outgoing-edges-range U a B) Us))
  by (rule card-Union-pairwise-disjoint-list[OF disj-ranges finite-ranges])
  have union-subset:
     $\bigcup (\text{set } (\text{map } (\lambda U. \text{outgoing-edges-range } U \ a \ B) \ Us)) \subseteq E$ 
proof
  fix e
  assume  $e \in \bigcup (\text{set } (\text{map } (\lambda U. \text{outgoing-edges-range } U \ a \ B) \ Us))$ 
  then obtain U where  $U \in \text{set } Us$  and  $e \in \text{outgoing-edges-range } U \ a \ B$ 
  by auto
  then show  $e \in E$ 
  unfolding outgoing-edges-range-def by auto
qed
have sum-maps:
  sum-list (map ( $\lambda U$ . card (outgoing-edges-range U a B)) Us) =
    sum-list (map card (map ( $\lambda U$ . outgoing-edges-range U a B) Us))
  by (induction Us) simp-all
have sum-list (map ( $\lambda U$ . card (outgoing-edges-range U a B)) Us) =
  card ( $\bigcup (\text{set } (\text{map } (\lambda U. \text{outgoing-edges-range } U \ a \ B) \ Us))$ )
  using sum-maps card-union by simp
also have  $\dots \leq \text{edge-count}$ 
unfolding edge-count-def by (rule card-mono[OF finite-E union-subset])
finally show ?thesis .
qed

lemma outgoing-edges-disjoint-if-sources-disjoint:
  assumes  $A \cap B = \{\}$ 
  shows outgoing-edges  $A \cap \text{outgoing-edges } B = \{\}$ 
  using assms unfolding outgoing-edges-def by auto

lemma pairwise-disjoint-list-outgoing-edges:
  assumes pairwise-disjoint-list Xs
  shows pairwise-disjoint-list (map outgoing-edges Xs)
  using assms
proof (induction Xs)
  case Nil
  then show ?case by simp

```

```

next
  case (Cons X Xs)
  have  $\bigwedge Y. Y \in \text{set } Xs \implies \text{outgoing-edges } X \cap \text{outgoing-edges } Y = \{\}$ 
    using Cons.premis outgoing-edges-disjoint-if-sources-disjoint by auto
  then show ?case
    using Cons by auto
qed

lemma outgoing-edges-Union-list:
  outgoing-edges ( $\bigcup (\text{set } Xs)$ ) =  $\bigcup (\text{set } (\text{map outgoing-edges } Xs))$ 
  unfolding outgoing-edges-def by auto

lemma outgoing-edges-mono:
  assumes  $A \subseteq B$ 
  shows outgoing-edges  $A \subseteq \text{outgoing-edges } B$ 
  using assms unfolding outgoing-edges-def by blast

fun range-tree-child-list where
  range-tree-child-list  $S a [] = []$ 
| range-tree-child-list  $S a (b \# bs) =$ 
  range-tree  $S a (\text{Fin } b) \# \text{range-tree-child-list } S b bs$ 

lemma length-range-tree-child-list [simp]:
  length (range-tree-child-list  $S a bs$ ) = length  $bs$ 
  by (induction bs arbitrary: a) simp-all

lemma range-tree-child-list-set-subset-chain-list:
  set (range-tree-child-list  $S a bs$ )  $\subseteq$  set (range-tree-chain-list  $S a bs B$ )
proof (induction bs arbitrary: a)
  case Nil
  then show ?case by simp
next
  case (Cons b bs)
  have set (range-tree-child-list  $S b bs$ )  $\subseteq$ 
    set (range-tree-chain-list  $S b bs B$ )
    using Cons.IH .
  then show ?case by auto
qed

lemma Union-range-tree-child-list-subset-chain:
   $\bigcup (\text{set } (\text{range-tree-child-list } S a bs)) \subseteq \text{range-tree-chain } S a bs B$ 
proof (induction bs arbitrary: a)
  case Nil
  then show ?case by simp
next
  case (Cons b bs)
  have  $\bigcup (\text{set } (\text{range-tree-child-list } S b bs)) \subseteq$ 
    range-tree-chain  $S b bs B$ 
    using Cons.IH .

```



```

    then show ?case by auto
qed

fun range-tree-child-edge-range-list where
  range-tree-child-edge-range-list S a [] = []
| range-tree-child-edge-range-list S a [] (b # bs) = []
| range-tree-child-edge-range-list S a (beta # betas) [] = []
| range-tree-child-edge-range-list S a (beta # betas) (b # bs) =
  outgoing-edges-range (range-tree S a (Fin b)) b (Fin beta) #
  range-tree-child-edge-range-list S b betas bs

lemma range-tree-child-edge-range-list-subset-outgoing-sources:
  assumes Y ∈ set (range-tree-child-edge-range-list S a betas bs)
  shows ∃ X ∈ set (range-tree-child-list S a bs). Y ⊆ outgoing-edges X
  using assms
proof (induction betas arbitrary: a bs)
  case Nil
  then show ?case
    by (cases bs) simp-all
next
  case (Cons beta betas)
  show ?case
  proof (cases bs)
    case Nil
    then show ?thesis
      using Cons.prem by simp
  next
    case (Cons b bs')
    let ?head = outgoing-edges-range (range-tree S a (Fin b)) b (Fin beta)
    let ?tail = range-tree-child-edge-range-list S b betas bs'
    have Y-cases: Y = ?head ∨ Y ∈ set ?tail
      using Cons.prem Cons by simp
    then show ?thesis
      proof
        assume Y-head: Y = ?head
        have ?head ⊆ outgoing-edges (range-tree S a (Fin b))
          by (rule outgoing-edges-range-subset-outgoing-edges)
        then show ?thesis
          unfolding Cons Y-head by auto
      next
        assume Y-tail: Y ∈ set ?tail
        obtain X where X: X ∈ set (range-tree-child-list S b bs')
          and YX: Y ⊆ outgoing-edges X
          using Cons.IH[OF Y-tail] by blast
        show ?thesis
          using X YX unfolding Cons by auto
      qed
    qed
  qed

```

```

lemma range-tree-child-edge-range-list-member-subset-E:
  assumes  $Y \in \text{set } (\text{range-tree-child-edge-range-list } S \text{ a betas } bs)$ 
  shows  $Y \subseteq E$ 
proof
  fix  $e$ 
  assume  $eY: e \in Y$ 
  obtain  $X$  where  $X \in \text{set } (\text{range-tree-child-list } S \text{ a } bs)$ 
    and  $Y\text{-subset}: Y \subseteq \text{outgoing-edges } X$ 
    using range-tree-child-edge-range-list-subset-outgoing-sources[OF assms]
    by blast
  then have  $e \in \text{outgoing-edges } X$ 
    using  $eY$  by blast
  then show  $e \in E$ 
    unfolding outgoing-edges-def by blast
qed

lemma pairwise-disjoint-list-range-tree-child-edge-range-list:
  assumes mono: nondecreasing-from  $a$   $bs$ 
  shows pairwise-disjoint-list  $(\text{range-tree-child-edge-range-list } S \text{ a betas } bs)$ 
  using mono
proof (induction betas arbitrary:  $a$   $bs$ )
  case Nil
  then show ?case
    by (cases  $bs$ ) simp-all
next
  case (Cons  $\beta$   $betas$ )
  show ?case
  proof (cases  $bs$ )
    case Nil
    then show ?thesis by simp
  next
  case (Cons  $b$   $bs'$ )
  let  $?head = \text{outgoing-edges-range } (\text{range-tree } S \text{ a } (Fin \ b)) \ b \ (Fin \ \beta)$ 
  let  $?tail = \text{range-tree-child-edge-range-list } S \ b \ betas \ bs'$ 
  have  $a \leq b$ 
    using Cons.prems Cons by simp
  have mono-tail: nondecreasing-from  $b$   $bs'$ 
    using Cons.prems Cons by simp
  have head-tail-disj:  $\forall Y \in \text{set } ?tail. ?head \cap Y = \{\}$ 
  proof
    fix  $Y$ 
    assume  $Y: Y \in \text{set } ?tail$ 
    obtain  $X$  where  $X: X \in \text{set } (\text{range-tree-child-list } S \ b \ bs')$ 
      and  $YX: Y \subseteq \text{outgoing-edges } X$ 
      using range-tree-child-edge-range-list-subset-outgoing-sources[OF  $Y$ ]
      by blast
    have  $X\text{-subset}: X \subseteq \text{range-tree-chain } S \ b \ bs' \ \text{Infinity}$ 
      using  $X$  Union-range-tree-child-list-subset-chain[of  $S \ b \ bs' \ \text{Infinity}$ ]

```

```

    by blast
  have source-disj: range-tree S a (Fin b)  $\cap$  X = {}
    using range-tree-disjoint-range-tree-chain-tail
      [OF a-le-b mono-tail, of S Infinity] X-subset by blast
  have out-disj:
    outgoing-edges (range-tree S a (Fin b))  $\cap$  outgoing-edges X = {}
    by (rule outgoing-edges-disjoint-if-sources-disjoint[OF source-disj])
  have head-subset: ?head  $\subseteq$  outgoing-edges (range-tree S a (Fin b))
    by (rule outgoing-edges-range-subset-outgoing-edges)
  show ?head  $\cap$  Y = {}
    using head-subset YX out-disj by blast
qed
have tail-disj: pairwise-disjoint-list ?tail
  by (rule Cons.IH[OF mono-tail])
show ?thesis
  unfolding Cons by (simp add: head-tail-disj tail-disj)
qed
qed

lemma sum-card-range-tree-child-edge-range-list-le-edge-count:
  assumes mono: nondecreasing-from a bs
  shows sum-list (map card (range-tree-child-edge-range-list S a betas bs))
     $\leq$  edge-count
proof -
  let ?Xs = range-tree-child-edge-range-list S a betas bs
  have disj: pairwise-disjoint-list ?Xs
    by (rule pairwise-disjoint-list-range-tree-child-edge-range-list[OF mono])
  have finite-sets:  $\bigwedge X. X \in \text{set } ?Xs \implies \text{finite } X$ 
    using range-tree-child-edge-range-list-member-subset-E
      by (meson finite-E finite-subset)
  have card-union:
    card ( $\bigcup (\text{set } ?Xs)$ ) = sum-list (map card ?Xs)
    by (rule card-Union-pairwise-disjoint-list[OF disj finite-sets])
  have union-subset:  $\bigcup (\text{set } ?Xs) \subseteq E$ 
proof
  fix e
  assume e  $\in \bigcup (\text{set } ?Xs)$ 
  then obtain X where X: X  $\in \text{set } ?Xs$  and eX: e  $\in$  X
    by blast
  show e  $\in$  E
    using range-tree-child-edge-range-list-member-subset-E[OF X] eX by blast
qed
have sum-list (map card ?Xs) = card ( $\bigcup (\text{set } ?Xs)$ )
  using card-union by simp
also have ...  $\leq$  edge-count
  unfolding edge-count-def by (rule card-mono[OF finite-E union-subset])
finally show ?thesis .
qed

```

```

lemma sum-card-range-tree-child-edge-range-list-le-outgoing-edges-chain:
  assumes mono: nondecreasing-from a bs
  shows sum-list (map card (range-tree-child-edge-range-list S a betas bs))
     $\leq$  card (outgoing-edges (range-tree-chain S a bs B))
proof –
  let ?Rs = range-tree-child-edge-range-list S a betas bs
  let ?Xs = range-tree-child-list S a bs
  have disj: pairwise-disjoint-list ?Rs
    by (rule pairwise-disjoint-list-range-tree-child-edge-range-list[OF mono])
  have finite-sets:  $\bigwedge R. R \in \text{set } ?Rs \implies \text{finite } R$ 
    using range-tree-child-edge-range-list-member-subset-E
    by (meson finite-E finite-subset)
  have card-union:
    card ( $\bigcup (\text{set } ?Rs)$ ) = sum-list (map card ?Rs)
    by (rule card-Union-pairwise-disjoint-list[OF disj finite-sets])
  have union-subset-children:  $\bigcup (\text{set } ?Rs) \subseteq \text{outgoing-edges } (\bigcup (\text{set } ?Xs))$ 
proof
  fix e
  assume  $e \in \bigcup (\text{set } ?Rs)$ 
  then obtain R where R:  $R \in \text{set } ?Rs$  and eR:  $e \in R$ 
    by blast
  obtain X where X:  $X \in \text{set } ?Xs$  and R-subset:  $R \subseteq \text{outgoing-edges } X$ 
    using range-tree-child-edge-range-list-subset-outgoing-sources[OF R]
    by blast
  show  $e \in \text{outgoing-edges } (\bigcup (\text{set } ?Xs))$ 
    using eR R-subset X unfolding outgoing-edges-def by blast
qed
have children-subset:  $\bigcup (\text{set } ?Xs) \subseteq \text{range-tree-chain } S \text{ a bs } B$ 
  by (rule Union-range-tree-child-list-subset-chain)
have union-subset:
   $\bigcup (\text{set } ?Rs) \subseteq \text{outgoing-edges } (\text{range-tree-chain } S \text{ a bs } B)$ 
  using union-subset-children outgoing-edges-mono[OF children-subset] by blast
have sum-list (map card ?Rs) = card ( $\bigcup (\text{set } ?Rs)$ )
  using card-union by simp
also have  $\dots \leq \text{card } (\text{outgoing-edges } (\text{range-tree-chain } S \text{ a bs } B))$ 
  by (rule card-mono) (simp-all add: union-subset)
finally show ?thesis .
qed

fun range-tree-child-direct-edge-range-list where
  range-tree-child-direct-edge-range-list S B a [] [] = []
| range-tree-child-direct-edge-range-list S B a [] (b # bs) = []
| range-tree-child-direct-edge-range-list S B a (beta # betas) [] = []
| range-tree-child-direct-edge-range-list S B a (beta # betas) (b # bs) =
  outgoing-edges-range (range-tree S a (Fin b)) beta B #
  range-tree-child-direct-edge-range-list S B b betas bs

fun range-tree-child-zero-edge-range-list where
  range-tree-child-zero-edge-range-list S a [] [] = []

```

```

| range-tree-child-zero-edge-range-list S a [] (b # bs) = []
| range-tree-child-zero-edge-range-list S a (beta # betas) [] = []
| range-tree-child-zero-edge-range-list S a (beta # betas) (b # bs) =
  outgoing-edges-range (range-tree S a (Fin b)) 0 (Fin beta) #
  range-tree-child-zero-edge-range-list S b betas bs

fun range-tree-child-full-edge-range-list where
  range-tree-child-full-edge-range-list S B a [] = []
| range-tree-child-full-edge-range-list S B a (b # bs) =
  outgoing-edges-range (range-tree S a (Fin b)) 0 B #
  range-tree-child-full-edge-range-list S B b bs

fun range-tree-child-bound-pair-list where
  range-tree-child-bound-pair-list S a [] [] = []
| range-tree-child-bound-pair-list S a [] (b # bs) = []
| range-tree-child-bound-pair-list S a (beta # betas) [] = []
| range-tree-child-bound-pair-list S a (beta # betas) (b # bs) =
  (range-tree S a (Fin b), Fin beta) #
  range-tree-child-bound-pair-list S b betas bs

lemma range-tree-child-direct-edge-range-list-subset-outgoing-sources:
  assumes Y ∈ set (range-tree-child-direct-edge-range-list S B a betas bs)
  shows ∃ X ∈ set (range-tree-child-list S a bs). Y ⊆ outgoing-edges X
  using assms
proof (induction betas arbitrary: a bs)
  case Nil
  then show ?case
    by (cases bs) simp-all
next
  case (Cons beta betas)
  show ?case
  proof (cases bs)
    case Nil
    then show ?thesis
      using Cons.prem by simp
  next
    case (Cons b bs')
    let ?head = outgoing-edges-range (range-tree S a (Fin b)) beta B
    let ?tail = range-tree-child-direct-edge-range-list S B b betas bs'
    have Y-cases: Y = ?head ∨ Y ∈ set ?tail
      using Cons.prem Cons by simp
    then show ?thesis
  proof
    assume Y-head: Y = ?head
    have ?head ⊆ outgoing-edges (range-tree S a (Fin b))
      by (rule outgoing-edges-range-subset-outgoing-edges)
    then show ?thesis
      unfolding Cons Y-head by auto
  next

```

```

    assume Y-tail:  $Y \in \text{set } ?tail$ 
    obtain X where X:  $X \in \text{set } (\text{range-tree-child-list } S \ b \ bs')$ 
    and YX:  $Y \subseteq \text{outgoing-edges } X$ 
    using Cons.IH[OF Y-tail] by blast
    show ?thesis
    using X YX unfolding Cons by auto
  qed
qed
qed

lemma range-tree-child-direct-edge-range-list-member-subset-E:
  assumes Y  $\in \text{set } (\text{range-tree-child-direct-edge-range-list } S \ B \ a \ \text{betas } bs)$ 
  shows  $Y \subseteq E$ 
proof
  fix e
  assume eY:  $e \in Y$ 
  obtain X where X  $\in \text{set } (\text{range-tree-child-list } S \ a \ bs)$ 
  and Y-subset:  $Y \subseteq \text{outgoing-edges } X$ 
  using range-tree-child-direct-edge-range-list-subset-outgoing-sources[OF asms]
  by blast
  then have  $e \in \text{outgoing-edges } X$ 
  using eY by blast
  then show  $e \in E$ 
  unfolding outgoing-edges-def by blast
qed

lemma pairwise-disjoint-list-range-tree-child-direct-edge-range-list:
  assumes mono: nondecreasing-from a bs
  shows pairwise-disjoint-list
    (range-tree-child-direct-edge-range-list S B a betas bs)
  using mono
proof (induction betas arbitrary: a bs)
  case Nil
  then show ?case
    by (cases bs) simp-all
next
  case (Cons beta betas)
  show ?case
  proof (cases bs)
    case Nil
    then show ?thesis by simp
  next
    case (Cons b bs')
    let ?head = outgoing-edges-range (range-tree S a (Fin b)) beta B
    let ?tail = range-tree-child-direct-edge-range-list S B b betas bs'
    have a-le-b:  $a \leq b$ 
    using Cons.prems Cons by simp
    have mono-tail: nondecreasing-from b bs'
    using Cons.prems Cons by simp

```

```

have head-tail-disj:  $\forall Y \in \text{set } ?\text{tail}. ?\text{head} \cap Y = \{\}$ 
proof
  fix Y
  assume Y:  $Y \in \text{set } ?\text{tail}$ 
  obtain X where X:  $X \in \text{set } (\text{range-tree-child-list } S \ b \ bs')$ 
  and YX:  $Y \subseteq \text{outgoing-edges } X$ 
  using range-tree-child-direct-edge-range-list-subset-outgoing-sources[OF Y]
  by blast
  have X-subset:  $X \subseteq \text{range-tree-chain } S \ b \ bs' \ \text{Infinity}$ 
  using X Union-range-tree-child-list-subset-chain[of S b bs' Infinity]
  by blast
  have source-disj:  $\text{range-tree } S \ a \ (\text{Fin } b) \cap X = \{\}$ 
  using range-tree-disjoint-range-tree-chain-tail
    [OF a-le-b mono-tail, of S Infinity] X-subset by blast
  have out-disj:
     $\text{outgoing-edges } (\text{range-tree } S \ a \ (\text{Fin } b)) \cap \text{outgoing-edges } X = \{\}$ 
  by (rule outgoing-edges-disjoint-if-sources-disjoint[OF source-disj])
  have head-subset:  $?\text{head} \subseteq \text{outgoing-edges } (\text{range-tree } S \ a \ (\text{Fin } b))$ 
  by (rule outgoing-edges-range-subset-outgoing-edges)
  show  $?\text{head} \cap Y = \{\}$ 
  using head-subset YX out-disj by blast
qed
have tail-disj: pairwise-disjoint-list ?tail
  by (rule Cons.IH[OF mono-tail])
show ?thesis
  unfolding Cons by (simp add: head-tail-disj tail-disj)
qed
qed

lemma sum-card-range-tree-child-direct-edge-range-list-le-edge-count:
  assumes mono: nondecreasing-from a bs
  shows sum-list
    ( $\text{map card } (\text{range-tree-child-direct-edge-range-list } S \ B \ a \ \text{betas } bs)$ )
     $\leq \text{edge-count}$ 
proof -
  let ?Xs = range-tree-child-direct-edge-range-list S B a betas bs
  have disj: pairwise-disjoint-list ?Xs
    by (rule pairwise-disjoint-list-range-tree-child-direct-edge-range-list
      [OF mono])
  have finite-sets:  $\bigwedge X. X \in \text{set } ?Xs \implies \text{finite } X$ 
  using range-tree-child-direct-edge-range-list-member-subset-E
  by (meson finite-E finite-subset)
  have card-union:
     $\text{card } (\bigcup (\text{set } ?Xs)) = \text{sum-list } (\text{map card } ?Xs)$ 
  by (rule card-Union-pairwise-disjoint-list[OF disj finite-sets])
  have union-subset:  $\bigcup (\text{set } ?Xs) \subseteq E$ 
proof
  fix e
  assume e  $e \in \bigcup (\text{set } ?Xs)$ 

```

then obtain X where $X: X \in \text{set } ?Xs$ and $eX: e \in X$
 by *blast*
 show $e \in E$
 using *range-tree-child-direct-edge-range-list-member-subset-E*[*OF* X] eX
 by *blast*
 qed
 have *sum-list* (*map card* $?Xs$) = *card* ($\bigcup (\text{set } ?Xs)$)
 using *card-union* by *simp*
 also have $\dots \leq \text{edge-count}$
 unfolding *edge-count-def* by (rule *card-mono*[*OF* *finite-E union-subset*])
 finally show *?thesis* .
 qed

lemma *sum-card-range-tree-child-direct-edge-range-list-le-outgoing-edges-chain*:

assumes *mono*: *nondecreasing-from* a bs

shows *sum-list*

(*map card* (*range-tree-child-direct-edge-range-list* S B' a *betas* bs))
 $\leq \text{card}$ (*outgoing-edges* (*range-tree-chain* S a bs B))

proof –

let $?Rs = \text{range-tree-child-direct-edge-range-list } S \ B' \ a \ \text{betas} \ bs$

let $?Xs = \text{range-tree-child-list } S \ a \ bs$

have *disj*: *pairwise-disjoint-list* $?Rs$

by (rule *pairwise-disjoint-list-range-tree-child-direct-edge-range-list*
[*OF* *mono*])

have *finite-sets*: $\bigwedge R. R \in \text{set } ?Rs \implies \text{finite } R$

using *range-tree-child-direct-edge-range-list-member-subset-E*

by (*meson* *finite-E finite-subset*)

have *card-union*:

card ($\bigcup (\text{set } ?Rs)$) = *sum-list* (*map card* $?Rs$)

by (rule *card-Union-pairwise-disjoint-list*[*OF* *disj finite-sets*])

have *union-subset-children*: $\bigcup (\text{set } ?Rs) \subseteq \text{outgoing-edges } (\bigcup (\text{set } ?Xs))$

proof

fix e

assume $e \in \bigcup (\text{set } ?Rs)$

then obtain R where $R: R \in \text{set } ?Rs$ and $eR: e \in R$

by *blast*

obtain X where $X: X \in \text{set } ?Xs$ and $R\text{-subset}: R \subseteq \text{outgoing-edges } X$

using *range-tree-child-direct-edge-range-list-subset-outgoing-sources*[*OF* R]

by *blast*

show $e \in \text{outgoing-edges } (\bigcup (\text{set } ?Xs))$

using eR $R\text{-subset } X$ unfolding *outgoing-edges-def* by *blast*

qed

have *children-subset*: $\bigcup (\text{set } ?Xs) \subseteq \text{range-tree-chain } S \ a \ bs \ B$

by (rule *Union-range-tree-child-list-subset-chain*)

have *union-subset*:

$\bigcup (\text{set } ?Rs) \subseteq \text{outgoing-edges } (\text{range-tree-chain } S \ a \ bs \ B)$

using *union-subset-children* *outgoing-edges-mono*[*OF* *children-subset*] by *blast*

have *sum-list* (*map card* $?Rs$) = *card* ($\bigcup (\text{set } ?Rs)$)

using *card-union* by *simp*

also have $\dots \leq \text{card } (\text{outgoing-edges } (\text{range-tree-chain } S \ a \ bs \ B))$
by *(rule card-mono) (simp-all add: union-subset)*
finally show *?thesis* .
qed

lemma *range-tree-nonneg-edge-value:*
assumes *edge: (u, v) ∈ E*
and *u-range: u ∈ range-tree S a B*
shows $0 \leq \text{dist } s \ u + w \ u \ v$
proof –
have *reach-u: reachable s u*
using *u-range unfolding range-tree-def tree-set-def tree-path-def* **by** *blast*
have $0 \leq \text{dist } s \ u$
by *(rule dist-nonneg[OF reach-u])*
moreover have $0 \leq w \ u \ v$
by *(rule nonneg-weight[OF edge])*
ultimately show *?thesis*
by *linarith*
qed

lemma *sum-card-range-tree-child-zero-direct-edge-ranges-le-full:*
assumes *bounds: bounds-le B betas*
shows
 $\text{sum-list } (\text{map card } (\text{range-tree-child-zero-edge-range-list } S \ a \ betas \ bs)) +$
 $\text{sum-list } (\text{map card } (\text{range-tree-child-direct-edge-range-list } S \ B \ a \ betas \ bs))$
 $\leq \text{sum-list } (\text{map card } (\text{range-tree-child-full-edge-range-list } S \ B \ a \ bs))$
using *bounds*
proof *(induction betas arbitrary: a bs)*
case *Nil*
then show *?case*
by *(cases bs) simp-all*
next
case *(Cons beta betas)*
show *?case*
proof *(cases bs)*
case *Nil*
then show *?thesis*
by *simp*
next
case *(Cons b bs')*
have *beta-le-B: bound-le (Fin beta) B*
using *Cons.prem*s **by** *simp*
have *bounds-tail: bounds-le B betas*
using *Cons.prem*s **by** *simp*
have *head:*
 $\text{card } (\text{outgoing-edges-range } (\text{range-tree } S \ a \ (\text{Fin } b)) \ 0 \ (\text{Fin } beta)) +$
 $\text{card } (\text{outgoing-edges-range } (\text{range-tree } S \ a \ (\text{Fin } b)) \ beta \ B)$
 $\leq \text{card } (\text{outgoing-edges-range } (\text{range-tree } S \ a \ (\text{Fin } b)) \ 0 \ B)$

```

    by (rule card-outgoing-edges-range-zero-lower-direct-split-le
        [OF beta-le-B range-tree-nonneg-edge-value])
  have tail:
    sum-list (map card (range-tree-child-zero-edge-range-list S b betas bs')) +
    sum-list
      (map card (range-tree-child-direct-edge-range-list S B b betas bs'))
    ≤ sum-list (map card
      (range-tree-child-full-edge-range-list S B b bs'))
    by (rule Cons.IH[OF bounds-tail])
  show ?thesis
    unfolding Cons by simp (use head tail in linarith)
qed
qed

lemma range-tree-child-full-edge-range-list-eq-map:
  range-tree-child-full-edge-range-list S B a bs =
    map (λU. outgoing-edges-range U 0 B) (range-tree-child-list S a bs)
  by (induction bs arbitrary: a) simp-all

lemma sum-card-range-tree-child-full-edge-range-list-le-outgoing-edges-range-chain:
  assumes mono: nondecreasing-from a bs
  shows sum-list (map card (range-tree-child-full-edge-range-list S B a bs))
    ≤ card (outgoing-edges-range (range-tree-chain S a bs C) 0 B)
proof -
  let ?Xs = range-tree-child-list S a bs
  let ?Rs = map (λU. outgoing-edges-range U 0 B) ?Xs
  have disj-children: pairwise-disjoint-list ?Xs
    using mono
  proof (induction bs arbitrary: a)
    case Nil
    then show ?case by simp
  next
    case (Cons b bs)
    have a-le-b: a ≤ b
      using Cons.prem1 by simp
    have mono-tail: nondecreasing-from b bs
      using Cons.prem2 by simp
    have head-tail-disj:
      ∧ Y. Y ∈ set (range-tree-child-list S b bs) ⇒
        range-tree S a (Fin b) ∩ Y = {}
    proof -
      fix Y
      assume Y: Y ∈ set (range-tree-child-list S b bs)
      have Y ⊆ range-tree-chain S b bs Infinity
        using Y Union-range-tree-child-list-subset-chain[of S b bs Infinity]
        by blast
      moreover have
        range-tree S a (Fin b) ∩ range-tree-chain S b bs Infinity = {}
        using range-tree-disjoint-range-tree-chain-tail[OF a-le-b mono-tail] .
    qed
  qed

```

```

ultimately show range-tree S a (Fin b) ∩ Y = {}
  by blast
qed
show ?case
  using head-tail-disj Cons.IH[OF mono-tail] by simp
qed
have disj: pairwise-disjoint-list ?Rs
  by (rule pairwise-disjoint-list-outgoing-edges-range[OF disj-children])
have finite-sets:  $\bigwedge R. R \in \text{set } ?Rs \implies \text{finite } R$ 
  by auto
have card-union:
  card ( $\bigcup (\text{set } ?Rs)$ ) = sum-list (map card ?Rs)
  by (rule card-Union-pairwise-disjoint-list[OF disj finite-sets])
have children-subset:  $\bigcup (\text{set } ?Xs) \subseteq \text{range-tree-chain } S \text{ a bs } C$ 
  by (rule Union-range-tree-child-list-subset-chain)
have union-subset:
 $\bigcup (\text{set } ?Rs) \subseteq \text{outgoing-edges-range } (\text{range-tree-chain } S \text{ a bs } C) \ 0 \ B$ 
proof
  fix e
  assume e ∈  $\bigcup (\text{set } ?Rs)$ 
  then obtain X where X: X ∈ set ?Xs
    and eX: e ∈ outgoing-edges-range X 0 B
  by auto
  have X-subset: X ⊆ range-tree-chain S a bs C
    using X children-subset by blast
  show e ∈ outgoing-edges-range (range-tree-chain S a bs C) 0 B
    using outgoing-edges-range-mono-sources[OF X-subset] eX by blast
qed
have sum-list (map card (range-tree-child-full-edge-range-list S B a bs)) =
  sum-list (map card ?Rs)
  by (simp add: range-tree-child-full-edge-range-list-eq-map)
also have ... = card ( $\bigcup (\text{set } ?Rs)$ )
  using card-union by simp
also have ... ≤
  card (outgoing-edges-range (range-tree-chain S a bs C) 0 B)
  by (rule card-mono[OF finite-outgoing-edges-range union-subset])
finally show ?thesis .
qed

```

lemma *sum-card-range-tree-child-zero-direct-edge-ranges-le-outgoing-edges-range-chain:*

```

assumes mono: nondecreasing-from a bs
  and bounds: bounds-le B betas
shows
  sum-list (map card (range-tree-child-zero-edge-range-list S a betas bs)) +
  sum-list
    (map card (range-tree-child-direct-edge-range-list S B a betas bs))
  ≤ card (outgoing-edges-range (range-tree-chain S a bs C) 0 B)
proof -

```

```

have split:
  sum-list (map card (range-tree-child-zero-edge-range-list S a betas bs)) +
  sum-list
    (map card (range-tree-child-direct-edge-range-list S B a betas bs))
  ≤ sum-list (map card (range-tree-child-full-edge-range-list S B a bs))
by (rule sum-card-range-tree-child-zero-direct-edge-ranges-le-full
    [OF bounds])
have full:
  sum-list (map card (range-tree-child-full-edge-range-list S B a bs))
  ≤ card (outgoing-edges-range (range-tree-chain S a bs C) 0 B)
by (rule sum-card-range-tree-child-full-edge-range-list-le-outgoing-edges-range-
chain
  [OF mono])
show ?thesis
  using split full by linarith
qed

lemma weighted-sum-child-lower-direct-edge-ranges-le:
assumes mono: nondecreasing-from a bs
shows
  h * sum-list (map card (range-tree-child-edge-range-list S a betas bs)) +
  t * sum-list
    (map card (range-tree-child-direct-edge-range-list S B a betas bs))
  ≤ (h + t) * edge-count
proof –
have lower:
  sum-list (map card (range-tree-child-edge-range-list S a betas bs))
  ≤ edge-count
by (rule sum-card-range-tree-child-edge-range-list-le-edge-count [OF mono])
have direct:
  sum-list
    (map card (range-tree-child-direct-edge-range-list S B a betas bs))
  ≤ edge-count
by (rule sum-card-range-tree-child-direct-edge-range-list-le-edge-count
  [OF mono])
have lower-w:
  h * sum-list (map card (range-tree-child-edge-range-list S a betas bs))
  ≤ h * edge-count
using lower by simp
have direct-w:
  t * sum-list
    (map card (range-tree-child-direct-edge-range-list S B a betas bs))
  ≤ t * edge-count
using direct by simp
have h * sum-list (map card (range-tree-child-edge-range-list S a betas bs)) +
  t * sum-list
    (map card (range-tree-child-direct-edge-range-list S B a betas bs))
  ≤ h * edge-count + t * edge-count
using lower-w direct-w by linarith

```

also have $\dots = (h + t) * \text{edge-count}$
by (*simp add: algebra-simps*)
finally show ?thesis .
qed

lemma *weighted-sum-child-lower-direct-edge-ranges-le-outgoing-edges-chain:*

assumes *mono: nondecreasing-from a bs*

shows

$h * \text{sum-list } (\text{map card } (\text{range-tree-child-edge-range-list } S \ a \ \text{betas } bs)) +$
 $t * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-direct-edge-range-list } S \ B' \ a \ \text{betas } bs))$
 $\leq (h + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } S \ a \ bs \ B))$

proof –

have *lower:*

$\text{sum-list } (\text{map card } (\text{range-tree-child-edge-range-list } S \ a \ \text{betas } bs))$
 $\leq \text{card } (\text{outgoing-edges } (\text{range-tree-chain } S \ a \ bs \ B))$
by (*rule sum-card-range-tree-child-edge-range-list-le-outgoing-edges-chain*
 $[OF \ \text{mono}]$)

have *direct:*

sum-list
 $(\text{map card } (\text{range-tree-child-direct-edge-range-list } S \ B' \ a \ \text{betas } bs))$
 $\leq \text{card } (\text{outgoing-edges } (\text{range-tree-chain } S \ a \ bs \ B))$
by (*rule sum-card-range-tree-child-direct-edge-range-list-le-outgoing-edges-chain*
 $[OF \ \text{mono}]$)

have *lower-w:*

$h * \text{sum-list } (\text{map card } (\text{range-tree-child-edge-range-list } S \ a \ \text{betas } bs))$
 $\leq h * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } S \ a \ bs \ B))$
using *lower* **by** *simp*

have *direct-w:*

$t * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-direct-edge-range-list } S \ B' \ a \ \text{betas } bs))$
 $\leq t * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } S \ a \ bs \ B))$
using *direct* **by** *simp*

have $h * \text{sum-list } (\text{map card } (\text{range-tree-child-edge-range-list } S \ a \ \text{betas } bs)) +$

$t * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-direct-edge-range-list } S \ B' \ a \ \text{betas } bs))$
 $\leq h * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } S \ a \ bs \ B)) +$
 $t * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } S \ a \ bs \ B))$

using *lower-w direct-w* **by** *linarith*

also have $\dots = (h + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } S \ a \ bs \ B))$

by (*simp add: algebra-simps*)

finally show ?thesis .

qed

lemma *sum-card-outgoing-edges-le-degree:*

assumes *degree: edge-outdegree-le Δ*

and sets: $\bigwedge X. X \in \text{set } Xs \implies X \subseteq V$

shows $\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \ Xs) \leq$

$\Delta * \text{sum-list } (\text{map card } Xs)$

```

using sets
proof (induction Xs)
  case Nil
  then show ?case by simp
next
  case (Cons X Xs)
  have X-subset:  $X \subseteq V$ 
    using Cons.prems by simp
  have X-bound:  $\text{card } (\text{outgoing-edges } X) \leq \Delta * \text{card } X$ 
    by (rule card-outgoing-edges-le[OF X-subset degree])
  have tail-bound:
     $\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ } Xs) \leq$ 
     $\Delta * \text{sum-list } (\text{map } \text{card } Xs)$ 
    using Cons.IH Cons.prems by simp
  have  $\text{card } (\text{outgoing-edges } X) +$ 
     $\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ } Xs) \leq$ 
     $\Delta * \text{card } X + \Delta * \text{sum-list } (\text{map } \text{card } Xs)$ 
    using X-bound tail-bound by simp
  also have  $\dots = \Delta * \text{sum-list } (\text{map } \text{card } (X \# Xs))$ 
    by (simp add: algebra-simps)
  finally show ?case
    by simp
qed

lemma card-outgoing-edges-Union-list-eq-sum:
  assumes disj: pairwise-disjoint-list Xs
  shows  $\text{card } (\text{outgoing-edges } (\bigcup (\text{set } Xs))) =$ 
     $\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ } Xs)$ 
proof –
  have disj-edges: pairwise-disjoint-list (map outgoing-edges Xs)
    using pairwise-disjoint-list-outgoing-edges[OF disj] .
  have finite-edges:  $\bigwedge X. X \in \text{set } (\text{map } \text{outgoing-edges } Xs) \implies \text{finite } X$ 
    by auto
  have  $\text{card } (\text{outgoing-edges } (\bigcup (\text{set } Xs))) =$ 
     $\text{card } (\bigcup (\text{set } (\text{map } \text{outgoing-edges } Xs)))$ 
    using outgoing-edges-Union-list[of Xs] by simp
  also have  $\dots = \text{sum-list } (\text{map } (\lambda Y. \text{card } Y) (\text{map } \text{outgoing-edges } Xs))$ 
    by (rule card-Union-pairwise-disjoint-list[OF disj-edges finite-edges])
  also have  $\dots = \text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ } Xs)$ 
    by (induction Xs) simp-all
  finally show ?thesis .
qed

lemma range-tree-chain-list-pairwise-disjoint:
  assumes mono: nondecreasing-from a bs
  shows pairwise-disjoint-list (range-tree-chain-list S a bs B)
  using assms
proof (induction bs arbitrary: a)
  case Nil

```

```

    then show ?case by simp
next
case (Cons b bs)
have a-le-b:  $a \leq b$ 
  using Cons.prem by simp
have mono-tail: nondecreasing-from b bs
  using Cons.prem by simp
have head-tail-disj:
 $\bigwedge Y. Y \in \text{set } (\text{range-tree-chain-list } S \ b \ bs \ B) \implies$ 
 $\text{range-tree } S \ a \ (\text{Fin } b) \cap Y = \{\}$ 
proof -
  fix Y
  assume Y:  $Y \in \text{set } (\text{range-tree-chain-list } S \ b \ bs \ B)$ 
  have  $Y \subseteq \text{range-tree-chain } S \ b \ bs \ B$ 
    using Y Union-range-tree-chain-list[of S b bs B] by blast
  moreover have
 $\text{range-tree } S \ a \ (\text{Fin } b) \cap \text{range-tree-chain } S \ b \ bs \ B = \{\}$ 
    using range-tree-disjoint-range-tree-chain-tail[OF a-le-b mono-tail] .
  ultimately show  $\text{range-tree } S \ a \ (\text{Fin } b) \cap Y = \{\}$ 
    by blast
qed
show ?case
  using head-tail-disj Cons.IH[OF mono-tail] by simp
qed

lemma range-tree-child-list-pairwise-disjoint:
  assumes mono: nondecreasing-from a bs
  shows pairwise-disjoint-list ( $\text{range-tree-child-list } S \ a \ bs$ )
  using assms
proof (induction bs arbitrary: a)
case Nil
  then show ?case by simp
next
case (Cons b bs)
have a-le-b:  $a \leq b$ 
  using Cons.prem by simp
have mono-tail: nondecreasing-from b bs
  using Cons.prem by simp
have head-tail-disj:
 $\bigwedge Y. Y \in \text{set } (\text{range-tree-child-list } S \ b \ bs) \implies$ 
 $\text{range-tree } S \ a \ (\text{Fin } b) \cap Y = \{\}$ 
proof -
  fix Y
  assume Y:  $Y \in \text{set } (\text{range-tree-child-list } S \ b \ bs)$ 
  have  $Y \subseteq \text{range-tree-chain } S \ b \ bs \ \text{Infinity}$ 
    using Y Union-range-tree-child-list-subset-chain[of S b bs Infinity] by blast
  moreover have
 $\text{range-tree } S \ a \ (\text{Fin } b) \cap \text{range-tree-chain } S \ b \ bs \ \text{Infinity} = \{\}$ 
    using range-tree-disjoint-range-tree-chain-tail[OF a-le-b mono-tail] .

```

```

    ultimately show range-tree S a (Fin b) ∩ Y = {}
    by blast
qed
show ?case
    using head-tail-disj Cons.IH[OF mono-tail] by simp
qed

theorem card-range-tree-child-list-le-chain:
  assumes mono: nondecreasing-from a bs
  shows sum-list (map card (range-tree-child-list S a bs)) ≤
    card (range-tree-chain S a bs B)
proof -
  let ?Xs = range-tree-child-list S a bs
  have disj: pairwise-disjoint-list ?Xs
    by (rule range-tree-child-list-pairwise-disjoint[OF mono])
  have finite-sets: ⋀X. X ∈ set ?Xs ⟹ finite X
    using range-tree-child-list-set-subset-chain-list
      finite-range-tree-chain-list-sets by blast
  have card-union:
    card (⋃ (set ?Xs)) = sum-list (map card ?Xs)
    by (rule card-Union-pairwise-disjoint-list[OF disj finite-sets])
  have subset: ⋃ (set ?Xs) ⊆ range-tree-chain S a bs B
    by (rule Union-range-tree-child-list-subset-chain)
  have sum-list (map card ?Xs) = card (⋃ (set ?Xs))
    using card-union by simp
  also have ... ≤ card (range-tree-chain S a bs B)
    by (rule card-mono) (simp-all add: subset)
  finally show ?thesis .
qed

lemma sum-card-list-all2-le:
  assumes list-all2 (λX Y. card X ≤ card Y) Xs Ys
  shows sum-list (map card Xs) ≤ sum-list (map card Ys)
  using assms
proof (induction rule: list-all2-induct)
  case Nil
  then show ?case by simp
next
  case (Cons X Y Xs Ys)
  then show ?case by simp
qed

lemma sum-card-dominated-by-disjoint-union:
  assumes disj: pairwise-disjoint-list Ys
    and finite-sets: ⋀Y. Y ∈ set Ys ⟹ finite Y
    and dominated: list-all2 (λX Y. card X ≤ card Y) Xs Ys
  shows sum-list (map card Xs) ≤ card (⋃ (set Ys))
proof -
  have sum-list (map card Xs) ≤ sum-list (map card Ys)

```


by (rule sum-card-list-all2-le[OF dominated])
 also have $\dots = \text{card } (\bigcup (\text{set } Ys))$
 using card-Union-pairwise-disjoint-list[OF disj finite-sets] by simp
 finally show ?thesis .
 qed

theorem sum-card-dominated-by-range-tree-child-list-le-chain:
 assumes mono: nondecreasing-from a bs
 and dominated:
 list-all2 $(\lambda X R. \text{card } X \leq \text{card } R) \ Xs$
 (range-tree-child-list S a bs)
 shows sum-list $(\text{map card } Xs) \leq \text{card } (\text{range-tree-chain } S \ a \ bs \ B)$
proof –
 have sum-list $(\text{map card } Xs) \leq$
 sum-list $(\text{map card } (\text{range-tree-child-list } S \ a \ bs))$
 by (rule sum-card-list-all2-le[OF dominated])
 also have $\dots \leq \text{card } (\text{range-tree-chain } S \ a \ bs \ B)$
 by (rule card-range-tree-child-list-le-chain[OF mono])
 finally show ?thesis .
 qed

theorem card-outgoing-edges-range-tree-chain-eq-sum-list:
 assumes mono: nondecreasing-from a bs
 shows card $(\text{outgoing-edges } (\text{range-tree-chain } S \ a \ bs \ B)) =$
 sum-list
 $(\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \ (\text{range-tree-chain-list } S \ a \ bs \ B))$
proof –
 have disj: pairwise-disjoint-list $(\text{range-tree-chain-list } S \ a \ bs \ B)$
 using range-tree-chain-list-pairwise-disjoint[OF mono] .
 have card $(\text{outgoing-edges } (\text{range-tree-chain } S \ a \ bs \ B)) =$
 card $(\text{outgoing-edges } (\bigcup (\text{set } (\text{range-tree-chain-list } S \ a \ bs \ B))))$
 using Union-range-tree-chain-list[of S a bs B] by simp
 also have $\dots =$
 sum-list
 $(\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \ (\text{range-tree-chain-list } S \ a \ bs \ B))$
 by (rule card-outgoing-edges-Union-list-eq-sum[OF disj])
 finally show ?thesis .
 qed

theorem card-outgoing-edges-range-tree-child-list-le-chain:
 assumes mono: nondecreasing-from a bs
 shows sum-list
 $(\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \ (\text{range-tree-child-list } S \ a \ bs)) \leq$
 card $(\text{outgoing-edges } (\text{range-tree-chain } S \ a \ bs \ B))$
proof –
 let ?Xs = range-tree-child-list S a bs
 have disj: pairwise-disjoint-list ?Xs
 by (rule range-tree-child-list-pairwise-disjoint[OF mono])
 have card-union:

```

    card (outgoing-edges (⋃ (set ?Xs))) =
      sum-list (map (λX. card (outgoing-edges X)) ?Xs)
  by (rule card-outgoing-edges-Union-list-eq-sum[OF disj])
  have subset: ⋃ (set ?Xs) ⊆ range-tree-chain S a bs B
  by (rule Union-range-tree-child-list-subset-chain)
  have outgoing-subset:
    outgoing-edges (⋃ (set ?Xs)) ⊆
      outgoing-edges (range-tree-chain S a bs B)
  by (rule outgoing-edges-mono[OF subset])
  have sum-list (map (λX. card (outgoing-edges X)) ?Xs) =
    card (outgoing-edges (⋃ (set ?Xs)))
  using card-union by simp
  also have ... ≤ card (outgoing-edges (range-tree-chain S a bs B))
  by (rule card-mono) (simp-all add: outgoing-subset)
  finally show ?thesis .
qed

lemma sum-outgoing-card-list-all2-le:
  assumes list-all2
    (λX Y. card (outgoing-edges X) ≤ card (outgoing-edges Y)) Xs Ys
  shows sum-list (map (λX. card (outgoing-edges X)) Xs) ≤
    sum-list (map (λY. card (outgoing-edges Y)) Ys)
  using assms
proof (induction rule: list-all2-induct)
  case Nil
  then show ?case by simp
next
  case (Cons X Y Xs Ys)
  then show ?case by simp
qed

theorem sum-outgoing-dominated-by-range-tree-child-list-le-chain:
  assumes mono: nondecreasing-from a bs
    and dominated:
      list-all2
        (λX R. card (outgoing-edges X) ≤ card (outgoing-edges R)) Xs
        (range-tree-child-list S a bs)
  shows sum-list (map (λX. card (outgoing-edges X)) Xs) ≤
    card (outgoing-edges (range-tree-chain S a bs B))
proof -
  have sum-list (map (λX. card (outgoing-edges X)) Xs) ≤
    sum-list
      (map (λR. card (outgoing-edges R)) (range-tree-child-list S a bs))
  by (rule sum-outgoing-card-list-all2-le[OF dominated])
  also have ... ≤ card (outgoing-edges (range-tree-chain S a bs B))
  by (rule card-outgoing-edges-range-tree-child-list-le-chain[OF mono])
  finally show ?thesis .
qed

```

```

lemma edge-costs-le-range-tree-chain-outgoing:
  assumes mono: nondecreasing-from a bs
    and costs: list-all2 ( $\lambda c X. c \leq C * \text{card} (\text{outgoing-edges } X)$ ) costs
      (range-tree-chain-list S a bs B)
  shows sum-list costs  $\leq C * \text{card} (\text{outgoing-edges} (\text{range-tree-chain } S a bs B))$ 
proof –
  have sum-list costs  $\leq$ 
    sum-list
      (map ( $\lambda X. C * \text{card} (\text{outgoing-edges } X)$ )
        (range-tree-chain-list S a bs B))
  using costs
proof (induction rule: list-all2-induct)
  case Nil
    then show ?case by simp
  next
    case (Cons c X costs Xs)
    then show ?case by simp
  qed
also have ... =
  C * sum-list
    (map ( $\lambda X. \text{card} (\text{outgoing-edges } X)$ ) (range-tree-chain-list S a bs B))
proof –
  define Xs where Xs = range-tree-chain-list S a bs B
  have sum-list (map ( $\lambda X. C * \text{card} (\text{outgoing-edges } X)$ ) Xs) =
    C * sum-list (map ( $\lambda X. \text{card} (\text{outgoing-edges } X)$ ) Xs)
  by (induction Xs) (simp-all add: algebra-simps)
  then show ?thesis
    unfolding Xs-def .
qed
also have ... = C * card (outgoing-edges (range-tree-chain S a bs B))
  using card-outgoing-edges-range-tree-chain-eq-sum-list[OF mono, of S B] by
simp
  finally show ?thesis .
qed

```

These two cost shapes are the local language of the runtime proof. The level bound *level-range-cost-bound* is the simple recurrence form: a vertex term scaled by the recursion level and an outgoing-edge term scaled by the range coefficient. The amortised bound *bmssp-amortized-cost-bound* adds the extra direct-edge range term used by the bucketed partition, where direct inserts have their own cost story.

The summation lemmas below say that if every recursive child call satisfies one of these bounds on its own range slice, then the sum of child costs can be rewritten as sums over the corresponding child range lists. Later theories collapse those sums using the disjointness lemmas above.

definition *level-range-cost-bound where*
level-range-cost-bound A R l X =
 $A * l * \text{card } X + R * \text{card} (\text{outgoing-edges } X)$

definition *bmssp-amortized-cost-bound* **where**
bmssp-amortized-cost-bound $A\ R\ h\ t\ q\ L\ B\ X =$
 $A * L * \text{card } X + (R + q * h) * \text{card } (\text{outgoing-edges } X) +$
 $t * \text{card } (\text{outgoing-edges-range } X\ 0\ B)$

lemma *sum-bmssp-amortized-child-bounds-le*:
assumes *bounds*:
 $\text{list-all2 } (\lambda c\text{-child } UB. \text{ case } UB \text{ of } (U\text{-child}, B\text{-child}) \Rightarrow$
 $c\text{-child} \leq$
 $\text{bmssp-amortized-cost-bound } A\ R\ h\ t\ q\ L\ B\text{-child } U\text{-child})$
 $\text{child-costs } (\text{range-tree-child-bound-pair-list } S\ a\ \text{betas } bs)$
shows $\text{sum-list child-costs} \leq$
 $A * L * \text{sum-list } (\text{map card } (\text{range-tree-child-list } S\ a\ bs)) +$
 $(R + q * h) *$
 $\text{sum-list } (\text{map } (\lambda U. \text{card } (\text{outgoing-edges } U))$
 $(\text{range-tree-child-list } S\ a\ bs)) +$
 $t * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-zero-edge-range-list } S\ a\ \text{betas } bs))$
using *bounds*
proof (*induction betas arbitrary: a bs child-costs*)
case *Nil*
then have *costs-nil*: $\text{child-costs} = []$
by (*cases bs; cases child-costs*) *auto*
show *?case*
using *costs-nil* **by** *simp*
next
case (*Cons beta betas*)
note *outer-prems* = *Cons.prems*
note *outer-IH* = *Cons.IH*
show *?case*
proof (*cases bs*)
case *Nil*
then have *costs-nil*: $\text{child-costs} = []$
using *outer-prems* **by** (*cases child-costs*) *auto*
show *?thesis*
using *Nil costs-nil* **by** *simp*
next
case *bs-Cons*: (*Cons b bs'*)
show *?thesis*
proof (*cases child-costs*)
case *Nil*
then show *?thesis*
by *simp*
next
case *costs-Cons*: (*Cons c-child child-costs-tail*)
have *head*:
 $c\text{-child} \leq$
 $\text{bmssp-amortized-cost-bound } A\ R\ h\ t\ q\ L\ (Fin\ beta)$

```

      (range-tree S a (Fin b))
    using outer-prems unfolding bs-Cons costs-Cons by simp
  have tail-bounds:
    list-all2 ( $\lambda c$ -child UB. case UB of (U-child, B-child)  $\Rightarrow$ 
      c-child  $\leq$ 
        bmssp-amortized-cost-bound A R h t q L B-child U-child)
      child-costs-tail (range-tree-child-bound-pair-list S b betas bs')
    using outer-prems unfolding bs-Cons costs-Cons by simp
  have tail:
    sum-list child-costs-tail  $\leq$ 
      A * L * sum-list (map card (range-tree-child-list S b bs')) +
      (R + q * h) *
        sum-list (map ( $\lambda U$ . card (outgoing-edges U))
          (range-tree-child-list S b bs')) +
      t * sum-list
        (map card (range-tree-child-zero-edge-range-list S b betas bs'))
    by (rule outer-IH[OF tail-bounds])
  have head-unfolded:
    c-child  $\leq$ 
      A * L * card (range-tree S a (Fin b)) +
      (R + q * h) * card (outgoing-edges (range-tree S a (Fin b))) +
      t * card (outgoing-edges-range (range-tree S a (Fin b)) 0
        (Fin beta))
    using head unfolding bmssp-amortized-cost-bound-def .
  show ?thesis
    unfolding bs-Cons costs-Cons
    using head-unfolded tail by (simp add: algebra-simps)
qed
qed
qed

lemma child-costs-le-level-range-chain-bound:
  assumes mono: nondecreasing-from a bs
  and costs: list-all2 ( $\lambda c$  X. c  $\leq$  level-range-cost-bound A R l X) costs
    (range-tree-chain-list S a bs B)
  shows sum-list costs  $\leq$ 
    A * l * card (range-tree-chain S a bs B) +
    R * card (outgoing-edges (range-tree-chain S a bs B))
proof -
  define Xs where Xs = range-tree-chain-list S a bs B
  have cost-sum:
    sum-list costs  $\leq$  sum-list (map (level-range-cost-bound A R l) Xs)
  using costs unfolding Xs-def
proof (induction rule: list-all2-induct)
  case Nil
  then show ?case by simp
next
  case (Cons c X costs Xs)
  then show ?case

```

```

    unfolding level-range-cost-bound-def by simp
  qed
  have split-sum:
    sum-list (map (level-range-cost-bound A R l) Xs) =
      A * l * sum-list (map (λX. card X) Xs) +
      R * sum-list (map (λX. card (outgoing-edges X)) Xs)
  proof (induction Xs)
    case Nil
    then show ?case
      unfolding level-range-cost-bound-def by simp
  next
    case (Cons X Xs)
    then show ?case
      unfolding level-range-cost-bound-def by (simp add: algebra-simps)
  qed
  have card-sum:
    sum-list (map (λX. card X) Xs) = card (range-tree-chain S a bs B)
  using card-range-tree-chain-eq-sum-list[OF mono, of S B]
  unfolding Xs-def by simp
  have edge-sum:
    sum-list (map (λX. card (outgoing-edges X)) Xs) =
      card (outgoing-edges (range-tree-chain S a bs B))
  using card-outgoing-edges-range-tree-chain-eq-sum-list[OF mono, of S B]
  unfolding Xs-def by simp
  show ?thesis
    using cost-sum split-sum card-sum edge-sum by simp
  qed

theorem concrete-partition-loop-trace-card-decomposition:
  assumes trace: concrete-partition-loop-trace P B a bs d' B' Us U
  shows card U =
    card (bound-tree P (Fin a)) + card (range-tree-chain P a bs B')
  proof -
    have mono: nondecreasing-from a bs
      and bounds: bounds-le B' (a # bs)
      using trace unfolding concrete-partition-loop-trace-def by blast+
    have post: bmssp-post-full d P B d' B' U
      using concrete-partition-loop-trace-post[OF trace] .
    then have U-eq: U = bound-tree P B'
      unfolding bmssp-post-full-def by blast
    show ?thesis
      using card-bound-tree-range-chain-union[OF mono bounds, of P] U-eq by simp
  qed

theorem concrete-partition-loop-trace-range-card-le:
  assumes trace: concrete-partition-loop-trace P B a bs d' B' Us U
  shows card (range-tree-chain P a bs B') ≤ card U
  using concrete-partition-loop-trace-card-decomposition[OF trace]
  by simp

```

definition *concrete-capped-step-core-cost* **where**
concrete-capped-step-core-cost $k \text{ cap } d \ S \ B \ \text{child-costs} =$
fp-iter-capped-scan-cost $k \text{ cap } d \ S \ S \ B + \text{sum-list } \text{child-costs}$

definition *base-case-scan-cost* **where**
base-case-scan-cost $\Delta \ k \ x \ B = \text{card } (\text{outgoing-edges } (\text{base-case-vertices } k \ x \ B))$

theorem *base-case-scan-cost-le*:
assumes *degree*: *edge-outdegree-le* Δ
shows *base-case-scan-cost* $\Delta \ k \ x \ B \leq \Delta * k$
proof –
have *order-subset*: *set* $(\text{base-case-order } x \ B) \subseteq V$
using *base-case-order-set*[*of* $x \ B$]
unfolding *bound-tree-def* **by** *auto*
have *vertices-subset*: *base-case-vertices* $k \ x \ B \subseteq V$
proof (*cases length* $(\text{base-case-order } x \ B) \leq k$)
case *True*
then show *?thesis*
using *order-subset* **unfolding** *base-case-vertices-def* **by** (*simp add*: *Let-def*)
next
case *False*
then show *?thesis*
using *order-subset* **unfolding** *base-case-vertices-def*
by (*auto simp*: *Let-def dest*: *in-set-takeD*)
qed
have *base-case-scan-cost* $\Delta \ k \ x \ B \leq$
 $\Delta * \text{card } (\text{base-case-vertices } k \ x \ B)$
unfolding *base-case-scan-cost-def*
by (*rule card-outgoing-edges-le*[*OF vertices-subset degree*])
also have $\dots \leq \Delta * k$
using *card-base-case-vertices-le*[*of* $k \ x \ B$] **by** *simp*
finally show *?thesis* .
qed

theorem *base-case-scan-cost-le-outgoing-edges*:
base-case-scan-cost $\Delta \ k \ x \ B \leq$
 $\text{card } (\text{outgoing-edges } (\text{base-case-vertices } k \ x \ B))$
unfolding *base-case-scan-cost-def*
by *simp*

theorem *base-case-scan-cost-le-edge-count*:
base-case-scan-cost *edge-count* $k \ x \ B \leq \text{edge-count} * k$
by (*rule base-case-scan-cost-le*[*OF edge-outdegree-le-edge-count*])

definition *concrete-capped-step-accounted-cost* **where**
concrete-capped-step-accounted-cost $k \text{ cap } d \ S \ B \ \text{child-costs} \ \text{edge-costs} =$
fp-iter-capped-scan-cost $k \text{ cap } d \ S \ S \ B +$
 $\text{sum-list } \text{child-costs} + \text{sum-list } \text{edge-costs}$

theorem *concrete-capped-step-core-cost-le-range:*
assumes *S-subset:* $S \subseteq V$
and *degree:* *edge-outdegree-le* Δ
and *S-cap:* $\text{card } S \leq \text{cap}$
and *trace:* *concrete-partition-loop-trace*
(find-pivots-pivots-capped k cap d S B) B a bs d' B' Us U-loop
and *child-costs:* *list-all2* $(\lambda c X. c \leq C * \text{card } X)$ *costs*
(range-tree-chain-list (find-pivots-pivots-capped k cap d S B) a bs B')
shows *concrete-capped-step-core-cost k cap d S B costs* $\leq$
 $k * \Delta * \text{cap} +$
 $C * \text{card } (\text{range-tree-chain}$
 $(\text{find-pivots-pivots-capped k cap d S B) a bs B'})$
proof –
let $?P = \text{find-pivots-pivots-capped k cap d S B}$
have *mono:* *nondecreasing-from a bs*
using *trace unfolding concrete-partition-loop-trace-def* **by** *blast*
have *scan:*
fp-iter-capped-scan-cost k cap d S S B $\leq k * \Delta * \text{cap}$
using *find-pivots-capped-scan-cost-le* $[OF \text{ } S\text{-subset degree } S\text{-cap}]$.
have *child:*
sum-list costs $\leq C * \text{card } (\text{range-tree-chain } ?P \text{ a bs } B')$
using *child-costs-le-range-tree-chain-card* $[OF \text{ } \text{mono child-costs}]$.
show *?thesis*
unfolding *concrete-capped-step-core-cost-def* **using** *scan child* **by** *linarith*
qed

theorem *concrete-capped-step-core-cost-le-level-chain:*
assumes *S-subset:* $S \subseteq V$
and *degree:* *edge-outdegree-le* Δ
and *S-cap:* $\text{card } S \leq \text{cap}$
and *trace:* *concrete-partition-loop-trace*
(find-pivots-pivots-capped k cap d S B) B a bs d' B' Us U-loop
and *child-costs:* *list-all2* $(\lambda c X. c \leq \text{level-range-cost-bound } A \text{ } R \text{ } l \text{ } X)$ *costs*
(range-tree-chain-list (find-pivots-pivots-capped k cap d S B) a bs B')
shows *concrete-capped-step-core-cost k cap d S B costs* $\leq$
 $k * \Delta * \text{cap} +$
 $A * l * \text{card } (\text{range-tree-chain } (\text{find-pivots-pivots-capped k cap d S B) a bs B') +$
 $R * \text{card } (\text{outgoing-edges}$
 $(\text{range-tree-chain } (\text{find-pivots-pivots-capped k cap d S B) a bs B'))$
proof –
let $?P = \text{find-pivots-pivots-capped k cap d S B}$
have *mono:* *nondecreasing-from a bs*
using *trace unfolding concrete-partition-loop-trace-def* **by** *blast*
have *scan:*
fp-iter-capped-scan-cost k cap d S S B $\leq k * \Delta * \text{cap}$
using *find-pivots-capped-scan-cost-le* $[OF \text{ } S\text{-subset degree } S\text{-cap}]$.
have *child:*


```

    sum-list costs ≤
      A * l * card (range-tree-chain ?P a bs B') +
      R * card (outgoing-edges (range-tree-chain ?P a bs B'))
    using child-costs-le-level-range-chain-bound[OF mono child-costs] .
  show ?thesis
    unfolding concrete-capped-step-core-cost-def using scan child by linarith
qed

```

theorem concrete-capped-step-core-cost-closes-level-bound:

```

  assumes S-subset:  $S \subseteq V$ 
  and degree: edge-outdegree-le  $\Delta$ 
  and S-cap: card  $S \leq \text{cap}$ 
  and trace: concrete-partition-loop-trace
    (find-pivots-pivots-capped k cap d S B) B a bs d' B' Us U-loop
  and child-costs: list-all2 ( $\lambda c X. c \leq \text{level-range-cost-bound } A R l X$ ) costs
    (range-tree-chain-list (find-pivots-pivots-capped k cap d S B) a bs B')
  and scan-budget:  $k * \Delta * \text{cap} \leq A * \text{card } U\text{-loop}$ 
  shows concrete-capped-step-core-cost k cap d S B costs ≤
    level-range-cost-bound A R (Suc l) U-loop
proof -
  let ?P = find-pivots-pivots-capped k cap d S B
  have children:
    list-all2 ( $\lambda U X. U = X \wedge \text{complete-on } d' U$ ) Us
    (range-tree-chain-list ?P a bs B')
  using trace unfolding concrete-partition-loop-trace-def by blast
  have U-def:  $U\text{-loop} = \text{bound-tree } ?P (\text{Fin } a) \cup \bigcup (\text{set } Us)$ 
  using trace unfolding concrete-partition-loop-trace-def by blast
  have Union-eq:
     $\bigcup (\text{set } Us) = \bigcup (\text{set } (\text{range-tree-chain-list } ?P a bs B'))$ 
  using children by (induction rule: list-all2-induct) auto
  have U-eq-chain:
     $U\text{-loop} = \text{bound-tree } ?P (\text{Fin } a) \cup \text{range-tree-chain } ?P a bs B'$ 
  using U-def Union-eq Union-range-tree-chain-list[of ?P a bs B'] by simp
  have finite-U: finite U-loop
  using U-eq-chain by simp
  have chain-subset:  $\text{range-tree-chain } ?P a bs B' \subseteq U\text{-loop}$ 
  using U-eq-chain by blast
  have card-chain-le:  $\text{card } (\text{range-tree-chain } ?P a bs B') \leq \text{card } U\text{-loop}$ 
  by (rule card-mono[OF finite-U chain-subset])
  have outgoing-subset:
     $\text{outgoing-edges } (\text{range-tree-chain } ?P a bs B') \subseteq \text{outgoing-edges } U\text{-loop}$ 
  using outgoing-edges-mono[OF chain-subset] .
  have finite-out-U: finite (outgoing-edges U-loop)
  by simp
  have card-out-le:
     $\text{card } (\text{outgoing-edges } (\text{range-tree-chain } ?P a bs B')) \leq$ 
     $\text{card } (\text{outgoing-edges } U\text{-loop})$ 
  by (rule card-mono[OF finite-out-U outgoing-subset])
  have step:

```

```

concrete-capped-step-core-cost  $k$  cap  $d$   $S$   $B$  costs  $\leq$ 
 $k * \Delta * \text{cap} +$ 
 $A * l * \text{card } (\text{range-tree-chain } ?P \text{ a bs } B') +$ 
 $R * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } ?P \text{ a bs } B'))$ 
by (rule concrete-capped-step-core-cost-le-level-chain
[OF  $S$ -subset degree  $S$ -cap trace child-costs])
also have ...  $\leq$ 
 $A * \text{card } U\text{-loop} + A * l * \text{card } U\text{-loop} +$ 
 $R * \text{card } (\text{outgoing-edges } U\text{-loop})$ 
using scan-budget card-chain-le card-out-le
by (intro add-mono mult-left-mono) simp-all
also have ... = level-range-cost-bound  $A$   $R$  (Suc  $l$ )  $U\text{-loop}$ 
unfolding level-range-cost-bound-def by (simp add: algebra-simps)
finally show ?thesis .
qed

```

```

lemma scan-budget-from-loop-card:
  assumes degree-factor:  $\Delta \leq A$ 
  and progress:  $k * \text{cap} \leq \text{card } U\text{-loop}$ 
  shows  $k * \Delta * \text{cap} \leq A * \text{card } U\text{-loop}$ 
proof -
  have  $k * \Delta * \text{cap} \leq k * A * \text{cap}$ 
  using degree-factor by (intro mult-left-mono mult-right-mono) simp-all
  also have ... =  $A * (k * \text{cap})$ 
  by (simp add: algebra-simps)
  also have ...  $\leq A * \text{card } U\text{-loop}$ 
  using progress by simp
  finally show ?thesis .
qed

```

```

lemma scan-insert-budget-from-loop-card:
  assumes degree-factor:  $\Delta \leq A$ 
  and insert-factor:  $t \leq A * k$ 
  and progress:  $k * \text{cap} \leq \text{card } U\text{-loop}$ 
  shows  $k * \Delta * \text{cap} + t * \text{cap} \leq 2 * A * \text{card } U\text{-loop}$ 
proof -
  have scan:  $k * \Delta * \text{cap} \leq A * \text{card } U\text{-loop}$ 
  by (rule scan-budget-from-loop-card[OF degree-factor progress])
  have insert:  $t * \text{cap} \leq A * \text{card } U\text{-loop}$ 
  proof -
    have  $t * \text{cap} \leq (A * k) * \text{cap}$ 
    using insert-factor by simp
    also have ... =  $A * (k * \text{cap})$ 
    by (simp add: algebra-simps)
    also have ...  $\leq A * \text{card } U\text{-loop}$ 
    using progress by simp
  finally show ?thesis .
  qed
  have  $k * \Delta * \text{cap} + t * \text{cap} \leq A * \text{card } U\text{-loop} + A * \text{card } U\text{-loop}$ 

```

using *scan insert* by *simp*
 also have $\dots = 2 * A * \text{card } U\text{-loop}$
 by *simp*
 finally show ?thesis .
 qed

lemma *loop-overhead-budget-from-progress*:
 assumes *factor*: $\text{Suc } t \leq A$
 and *progress*: $M * q \leq \text{card } U$
 shows $(\text{Suc } t) * M * q \leq A * \text{card } U$
proof –
 have $(\text{Suc } t) * M * q \leq A * (M * q)$
proof –
 have $(\text{Suc } t) * (M * q) \leq A * (M * q)$
 by (rule *mult-right-mono*[*OF factor*]) *simp*
 then show ?thesis
 by (*simp add: algebra-simps*)
 qed
 also have $\dots \leq A * \text{card } U$
 using *progress* by *simp*
 finally show ?thesis .
 qed

theorem *concrete-capped-step-core-cost-closes-level-bound-from-progress*:
 assumes *S-subset*: $S \subseteq V$
 and *degree*: *edge-outdegree-le* Δ
 and *S-cap*: $\text{card } S \leq \text{cap}$
 and *trace*: *concrete-partition-loop-trace*
 (*find-pivots-pivots-capped* $k \text{ cap } d \text{ } S \text{ } B$) $B \text{ } a \text{ } bs \text{ } d' \text{ } B' \text{ } Us \text{ } U\text{-loop}$
 and *child-costs*: *list-all2* $(\lambda c \text{ } X. c \leq \text{level-range-cost-bound } A \text{ } R \text{ } l \text{ } X) \text{ costs}$
 (*range-tree-chain-list* (*find-pivots-pivots-capped* $k \text{ cap } d \text{ } S \text{ } B$) $a \text{ } bs \text{ } B'$)
 and *degree-factor*: $\Delta \leq A$
 and *progress*: $k * \text{cap} \leq \text{card } U\text{-loop}$
 shows *concrete-capped-step-core-cost* $k \text{ cap } d \text{ } S \text{ } B \text{ costs} \leq$
level-range-cost-bound $A \text{ } R \text{ } (\text{Suc } l) \text{ } U\text{-loop}$
proof –
 have *scan-budget*: $k * \Delta * \text{cap} \leq A * \text{card } U\text{-loop}$
 by (rule *scan-budget-from-loop-card*[*OF degree-factor progress*])
 show ?thesis
 by (rule *concrete-capped-step-core-cost-closes-level-bound*
 [*OF S-subset degree S-cap trace child-costs scan-budget*])
 qed

theorem *concrete-capped-step-core-cost-le-loop-vertices*:
 assumes *S-subset*: $S \subseteq V$
 and *degree*: *edge-outdegree-le* Δ
 and *S-cap*: $\text{card } S \leq \text{cap}$
 and *trace*: *concrete-partition-loop-trace*
 (*find-pivots-pivots-capped* $k \text{ cap } d \text{ } S \text{ } B$) $B \text{ } a \text{ } bs \text{ } d' \text{ } B' \text{ } Us \text{ } U\text{-loop}$

and *child-costs*: *list-all2* ($\lambda c X. c \leq C * \text{card } X$) *costs*
 (*range-tree-chain-list* (*find-pivots-pivots-capped* $k \text{ cap } d S B$) $a \text{ bs } B'$)
shows *concrete-capped-step-core-cost* $k \text{ cap } d S B \text{ costs} \leq$
 $k * \Delta * \text{cap} + C * \text{card } U\text{-loop}$
proof –
let $?P = \text{find-pivots-pivots-capped } k \text{ cap } d S B$
have *children*:
list-all2 ($\lambda U X. U = X \wedge \text{complete-on } d' U$) Us
 (*range-tree-chain-list* $?P a \text{ bs } B'$)
using *trace unfolding concrete-partition-loop-trace-def* **by** *blast*
have *U-def*: $U\text{-loop} = \text{bound-tree } ?P (\text{Fin } a) \cup \bigcup (\text{set } Us)$
using *trace unfolding concrete-partition-loop-trace-def* **by** *blast*
have *Union-eq*:
 $\bigcup (\text{set } Us) = \bigcup (\text{set } (\text{range-tree-chain-list } ?P a \text{ bs } B'))$
using *children by (induction rule: list-all2-induct)* *auto*
have *U-eq-chain*:
 $U\text{-loop} = \text{bound-tree } ?P (\text{Fin } a) \cup \text{range-tree-chain } ?P a \text{ bs } B'$
using *U-def Union-eq Union-range-tree-chain-list[of ?P a bs B']* **by** *simp*
have *finite-U*: *finite* $U\text{-loop}$
using *U-eq-chain by simp*
have *chain-subset*: $\text{range-tree-chain } ?P a \text{ bs } B' \subseteq U\text{-loop}$
using *U-eq-chain by blast*
have *card-chain-le*: $\text{card } (\text{range-tree-chain } ?P a \text{ bs } B') \leq \text{card } U\text{-loop}$
by (*rule card-mono[OF finite-U chain-subset]*)
have *concrete-capped-step-core-cost* $k \text{ cap } d S B \text{ costs} \leq$
 $k * \Delta * \text{cap} + C * \text{card } (\text{range-tree-chain } ?P a \text{ bs } B')$
by (*rule concrete-capped-step-core-cost-le-range*
[OF S-subset degree S-cap trace child-costs])
also have $\dots \leq k * \Delta * \text{cap} + C * \text{card } U\text{-loop}$
using *card-chain-le by simp*
finally show *?thesis* .
qed

theorem *concrete-capped-step-accounted-cost-le*:

assumes *S-subset*: $S \subseteq V$
and *degree*: *edge-outdegree-le* Δ
and *S-cap*: $\text{card } S \leq \text{cap}$
and *trace*: *concrete-partition-loop-trace*
 (*find-pivots-pivots-capped* $k \text{ cap } d S B$) $B a \text{ bs } d' B' Us U\text{-loop}$
and *child-costs*: *list-all2* ($\lambda c X. c \leq C * \text{card } X$) *child-costs*
 (*range-tree-chain-list* (*find-pivots-pivots-capped* $k \text{ cap } d S B$) $a \text{ bs } B'$)
and *edge-costs*: *list-all2* ($\lambda c X. c \leq R * \text{card } (\text{outgoing-edges } X)$) *edge-costs*
 (*range-tree-chain-list* (*find-pivots-pivots-capped* $k \text{ cap } d S B$) $a \text{ bs } B'$)
shows *concrete-capped-step-accounted-cost* $k \text{ cap } d S B \text{ child-costs edge-costs} \leq$
 $k * \Delta * \text{cap} + C * \text{card } U\text{-loop} +$
 $R * \text{card } (\text{outgoing-edges}$
 (*range-tree-chain* (*find-pivots-pivots-capped* $k \text{ cap } d S B$) $a \text{ bs } B'))$

proof –

let $?P = \text{find-pivots-pivots-capped } k \text{ cap } d S B$

```

have mono: nondecreasing-from a bs
using trace unfolding concrete-partition-loop-trace-def by blast
have core:
  concrete-capped-step-core-cost k cap d S B child-costs ≤
    k * Δ * cap + C * card U-loop
by (rule concrete-capped-step-core-cost-le-loop-vertices
  [OF S-subset degree S-cap trace child-costs])
have core-unfolded:
  fp-iter-capped-scan-cost k cap d S S B + sum-list child-costs ≤
    k * Δ * cap + C * card U-loop
using core unfolding concrete-capped-step-core-cost-def .
have edges:
  sum-list edge-costs ≤ R * card (outgoing-edges (range-tree-chain ?P a bs B'))
using edge-costs-le-range-tree-chain-outgoing[OF mono edge-costs] .
show ?thesis
unfolding concrete-capped-step-accounted-cost-def
using core-unfolded edges by linarith
qed

theorem full-operational-partition-loop-state-trace-and-degree-cost:
assumes run:
  full-operational-partition-loop-state M t k cap l d P B d' D a betas bs B' Us U c
and sound: sound-label d
and pre: bmssp-pre-full d P B
and P-reaches:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
and degree: edge-outdegree-le Δ
obtains child-costs child-sets where
  concrete-partition-loop-trace P B a bs d' B' Us U
  length child-costs = length child-sets
   $\bigwedge X. X \in \text{set child-sets} \implies X \subseteq V$ 
   $c \leq \text{sum-list child-costs} + M * \text{length child-costs} +$ 
   $t * (\Delta * \text{sum-list} (\text{map card child-sets}) + M * \text{length child-costs})$ 
proof –
obtain child-costs child-sets where trace:
  concrete-partition-loop-trace P B a bs d' B' Us U
and len: length child-costs = length child-sets
and sets:  $\bigwedge X. X \in \text{set child-sets} \implies X \subseteq V$ 
and cost:
   $c \leq \text{sum-list child-costs} + M * \text{length child-costs} +$ 
   $t * (\text{sum-list} (\text{map} (\lambda X. \text{card} (\text{outgoing-edges } X)) \text{ child-sets}) +$ 
   $M * \text{length child-costs})$ 
using full-operational-partition-loop-state-trace-and-complete-edge-cost
  [OF run sound pre P-reaches] by blast
have edge-sum:
  sum-list (map ( $\lambda X. \text{card} (\text{outgoing-edges } X)$ ) child-sets) ≤
    Δ * sum-list (map card child-sets)
by (rule sum-card-outgoing-edges-le-degree[OF degree sets])
have cost-degree:
   $c \leq \text{sum-list child-costs} + M * \text{length child-costs} +$ 

```

```

      t * (Δ * sum-list (map card child-sets) + M * length child-costs)
proof -
  have sum-list child-costs + M * length child-costs +
    t * (sum-list (map (λX. card (outgoing-edges X)) child-sets) +
      M * length child-costs) ≤
    sum-list child-costs + M * length child-costs +
    t * (Δ * sum-list (map card child-sets) + M * length child-costs)
  using edge-sum by simp
  then show ?thesis
  using cost by linarith
qed
show thesis
  using that trace len sets cost-degree by blast
qed

```

definition *partition-initial-insert-cost-bound* **where**
partition-initial-insert-cost-bound $c\ t\ P \longleftrightarrow c \leq t * \text{card } P$

lemma *partition-initial-insert-cost-boundD*:
assumes *partition-initial-insert-cost-bound* $c\ t\ P$
shows $c \leq t * \text{card } P$
using *assms* **unfolding** *partition-initial-insert-cost-bound-def* .

lemma *partition-initial-insert-cost-bound-le*:
assumes *cost*: *partition-initial-insert-cost-bound* $c\ t\ P$
and *card-le*: $\text{card } P \leq N$
shows $c \leq t * N$
proof -
 have $c \leq t * \text{card } P$
 using *cost* **by** (rule *partition-initial-insert-cost-boundD*)
 also have $\dots \leq t * N$
 using *card-le* **by** *simp*
 finally **show** ?thesis .
qed

lemma *find-pivots-pivots-capped-subset*:
find-pivots-pivots-capped $k\ \text{cap } d\ S\ B \subseteq S$
unfolding *find-pivots-pivots-capped-def* **by** *auto*

lemma *find-pivots-pivots-capped-card-le*:
assumes *S-subset*: $S \subseteq V$
shows $\text{card } (\text{find-pivots-pivots-capped } k\ \text{cap } d\ S\ B) \leq \text{card } S$
proof -
 have *finite-S*: *finite* S
 using *finite-subset*[*OF* *S-subset* *finite-V*] .
 show ?thesis
by (rule *card-mono*[*OF* *finite-S* *find-pivots-pivots-capped-subset*])
qed

lemma *find-pivots-pivots-capped-scaled-card-le*:
assumes *S-subset*: $S \subseteq V$
and *S-k-cap*: $k * \text{card } S \leq \text{cap}$
shows $k * \text{card } (\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \leq \text{cap}$
proof –
have $\text{card } (\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \leq \text{card } S$
by (rule *find-pivots-pivots-capped-card-le*[OF *S-subset*])
then have $k * \text{card } (\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \leq$
 $k * \text{card } S$
by *simp*
then show *?thesis*
using *S-k-cap* **by** *linarith*
qed

lemma *find-pivots-pivots-capped-tree-antichain*:
assumes *anti*: *tree-antichain* *S*
shows *tree-antichain* (*find-pivots-pivots-capped* *k cap d S B*)
by (rule *tree-antichain-subset*[OF *anti find-pivots-pivots-capped-subset*])

lemma *partition-initial-insert-cost-bound-capped-pivots-le*:
assumes *cost*: *partition-initial-insert-cost-bound* *c t*
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B)$
and *S-subset*: $S \subseteq V$
and *S-cap*: $\text{card } S \leq \text{cap}$
shows $c \leq t * \text{cap}$
proof –
have *pivots-S*:
 $\text{card } (\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \leq \text{card } S$
by (rule *find-pivots-pivots-capped-card-le*[OF *S-subset*])
have *pivots-le*: $\text{card } (\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \leq \text{cap}$
by (rule *order-trans*[OF *pivots-S S-cap*])
show *?thesis*
by (rule *partition-initial-insert-cost-bound-le*[OF *cost pivots-le*])
qed

lemma *find-pivots-scan-and-initial-insert-cost-le*:
assumes *S-subset*: $S \subseteq V$
and *degree*: *edge-outdegree-le* Δ
and *S-cap*: $\text{card } S \leq \text{cap}$
and *insert*: *partition-initial-insert-cost-bound* *c-insert t*
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B)$
shows $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $k * \Delta * \text{cap} + t * \text{cap}$
proof –
have *scan*: $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B \leq k * \Delta * \text{cap}$
by (rule *find-pivots-capped-scan-cost-le*[OF *S-subset degree S-cap*])
have *insert-le*: $c\text{-insert} \leq t * \text{cap}$
by (rule *partition-initial-insert-cost-bound-capped-pivots-le*
 $[OF \text{ insert } S\text{-subset } S\text{-cap}]$)

```

show ?thesis
  using scan insert-le by simp
qed

lemma find-pivots-scan-and-initial-insert-cost-le-edge-count:
  assumes  $S\text{-subset}: S \subseteq V$ 
  and  $S\text{-cap}: \text{card } S \leq \text{cap}$ 
  and  $\text{insert}: \text{partition-initial-insert-cost-bound } c\text{-insert } t$ 
     $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B)$ 
  shows  $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$ 
     $k * \text{edge-count} * \text{cap} + t * \text{cap}$ 
  by (rule find-pivots-scan-and-initial-insert-cost-le
    [OF  $S\text{-subset}$  edge-outdegree-le-edge-count  $S\text{-cap}$  insert])

lemma find-pivots-scan-and-initial-insert-budget-from-progress:
  assumes  $S\text{-subset}: S \subseteq V$ 
  and  $\text{degree}: \text{edge-outdegree-le } \Delta$ 
  and  $S\text{-cap}: \text{card } S \leq \text{cap}$ 
  and  $\text{insert}: \text{partition-initial-insert-cost-bound } c\text{-insert } t$ 
     $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B)$ 
  and  $\text{degree-factor}: \Delta \leq A$ 
  and  $\text{insert-factor}: t \leq A * k$ 
  and  $\text{progress}: k * \text{cap} \leq \text{card } U$ 
  shows  $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$ 
     $2 * A * \text{card } U$ 
  proof -
    have local:
       $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$ 
         $k * \Delta * \text{cap} + t * \text{cap}$ 
      by (rule find-pivots-scan-and-initial-insert-cost-le
        [OF  $S\text{-subset}$  degree  $S\text{-cap}$  insert])
    have budget:  $k * \Delta * \text{cap} + t * \text{cap} \leq 2 * A * \text{card } U$ 
      by (rule scan-insert-budget-from-loop-card
        [OF degree-factor insert-factor progress])
    show ?thesis
      using local budget by linarith
  qed

lemma find-pivots-pivots-capped-card-le-seen-if-no-overflow:
  assumes  $S\text{-subset}: S \subseteq V$ 
  and  $\text{anti}: \text{tree-antichain } S$ 
  and  $k\text{-pos}: 0 < k$ 
  and  $\text{no-overflow}: \neg \text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B) > \text{cap}$ 
  shows  $\text{card } (\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \leq$ 
     $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B)$ 
  proof -
    let ?P =  $\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B$ 
    let ?W =  $\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B$ 

```


have cases:
 $k * \text{card } ?P \leq \text{card } ?W \vee (?P = S \wedge \text{card } ?W > \text{cap})$
by (rule *find-pivots-pivots-capped-card-times-le-seen-or-overflow*
 $[OF\ S\text{-subset}\ anti]$)
then have mult-le: $k * \text{card } ?P \leq \text{card } ?W$
using *no-overflow* **by** *blast*
have $\text{card } ?P \leq k * \text{card } ?P$
using *k-pos* **by** (cases *k*) *simp-all*
then show *?thesis*
using *mult-le* **by** *linarith*
qed

lemma *partition-initial-insert-cost-bound-capped-pivots-le-seen-scaled-from-mult:*
assumes *insert: partition-initial-insert-cost-bound c t P*
and *pivot-mult: k * card P ≤ card W*
and *insert-factor: t ≤ A * k*
shows $c \leq A * \text{card } W$
proof –
have $c \leq t * \text{card } P$
by (rule *partition-initial-insert-cost-boundD* $[OF\ insert]$)
also have $\dots \leq (A * k) * \text{card } P$
using *insert-factor* **by** *simp*
also have $\dots = A * (k * \text{card } P)$
by (*simp add: algebra-simps*)
also have $\dots \leq A * \text{card } W$
using *pivot-mult* **by** *simp*
finally show *?thesis* .
qed

lemma *find-pivots-pivots-capped-card-times-le-seen:*
assumes *S-subset: S ⊆ V*
and *anti: tree-antichain S*
and *S-k-cap: k * card S ≤ cap*
shows $k * \text{card } (\text{find-pivots-pivots-capped } k\ \text{cap } d\ S\ B) \leq$
 $\text{card } (\text{find-pivots-seen-capped } k\ \text{cap } d\ S\ B)$
proof –
let $?P = \text{find-pivots-pivots-capped } k\ \text{cap } d\ S\ B$
let $?W = \text{find-pivots-seen-capped } k\ \text{cap } d\ S\ B$
have cases: $k * \text{card } ?P \leq \text{card } ?W \vee (?P = S \wedge \text{card } ?W > \text{cap})$
by (rule *find-pivots-pivots-capped-card-times-le-seen-or-overflow*
 $[OF\ S\text{-subset}\ anti]$)
then show *?thesis*
proof
assume $k * \text{card } ?P \leq \text{card } ?W$
then show *?thesis* .
next
assume *overflow: ?P = S ∧ card ?W > cap*
then have $k * \text{card } ?P \leq \text{cap}$
using *S-k-cap* **by** *simp*

also have $\dots \leq \text{card } ?W$
 using *overflow* by *simp*
 finally show *?thesis* .
 qed
 qed

lemma *partition-initial-insert-cost-bound-capped-pivots-le-seen-scaled*:
 assumes *insert*: *partition-initial-insert-cost-bound* c t
 (*find-pivots-pivots-capped* k cap d S B)
 and *S-subset*: $S \subseteq V$
 and *anti*: *tree-antichain* S
 and *S-k-cap*: $k * \text{card } S \leq \text{cap}$
 and *insert-factor*: $t \leq A * k$
 shows $c \leq A * \text{card } (\text{find-pivots-seen-capped } k \text{ cap } d S B)$
proof –
 have *pivot-mult*:
 $k * \text{card } (\text{find-pivots-pivots-capped } k \text{ cap } d S B) \leq$
 $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d S B)$
 by (rule *find-pivots-pivots-capped-card-times-le-seen*
 [*OF S-subset anti S-k-cap*])
 show *?thesis*
 by (rule *partition-initial-insert-cost-bound-capped-pivots-le-seen-scaled-from-mult*
 [*OF insert pivot-mult insert-factor*])
 qed

lemma *partition-initial-insert-cost-bound-capped-pivots-le-seen-if-no-overflow*:
 assumes *insert*: *partition-initial-insert-cost-bound* c t
 (*find-pivots-pivots-capped* k cap d S B)
 and *S-subset*: $S \subseteq V$
 and *anti*: *tree-antichain* S
 and *k-pos*: $0 < k$
 and *no-overflow*:
 $\neg \text{card } (\text{find-pivots-seen-capped } k \text{ cap } d S B) > \text{cap}$
 shows $c \leq t * \text{card } (\text{find-pivots-seen-capped } k \text{ cap } d S B)$
proof –
 have *pivots-le*:
 $\text{card } (\text{find-pivots-pivots-capped } k \text{ cap } d S B) \leq$
 $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d S B)$
 by (rule *find-pivots-pivots-capped-card-le-seen-if-no-overflow*
 [*OF S-subset anti k-pos no-overflow*])
 show *?thesis*
 by (rule *partition-initial-insert-cost-bound-le*[*OF insert pivots-le*])
 qed

lemma *partition-initial-insert-cost-bound-capped-pivots-le-seen*:
 assumes *insert*: *partition-initial-insert-cost-bound* c t
 (*find-pivots-pivots-capped* k cap d S B)
 and *S-subset*: $S \subseteq V$
 and *S-cap*: $\text{card } S \leq \text{cap}$

and *anti*: *tree-antichain* S
and *k-pos*: $0 < k$
shows $c \leq t * \text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B)$
proof (*cases* $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B) > \text{cap}$)
case *True*
have *c-le-cap*: $c \leq t * \text{cap}$
by (*rule* *partition-initial-insert-cost-bound-capped-pivots-le*
 $[OF \text{ insert } S\text{-subset } S\text{-cap}]$)
have $t * \text{cap} \leq t * \text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B)$
using *True* **by** *simp*
then show *?thesis*
using *c-le-cap* **by** *linarith*
next
case *False*
show *?thesis*
by (*rule* *partition-initial-insert-cost-bound-capped-pivots-le-seen-if-no-overflow*
 $[OF \text{ insert } S\text{-subset } anti \text{ } k\text{-pos } False]$)
qed

lemma *find-pivots-scan-and-initial-insert-cost-le-seen-if-no-overflow*:

assumes *S-subset*: $S \subseteq V$
and *degree*: *edge-outdegree-le* Δ
and *anti*: *tree-antichain* S
and *k-pos*: $0 < k$
and *no-overflow*:
 $\neg \text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B) > \text{cap}$
and *insert*: *partition-initial-insert-cost-bound* *c-insert* t
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B)$
shows $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $(k * \Delta + t) * \text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B)$
proof –
let $?W = \text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B$
have *scan*: $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B \leq$
 $k * \Delta * \text{card } ?W$
by (*rule* *find-pivots-capped-scan-cost-le-seen* $[OF \text{ } S\text{-subset } degree]$)
have *insert-le*: $c\text{-insert} \leq t * \text{card } ?W$
by (*rule* *partition-initial-insert-cost-bound-capped-pivots-le-seen-if-no-overflow*
 $[OF \text{ insert } S\text{-subset } anti \text{ } k\text{-pos } no\text{-overflow}]$)
have $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $k * \Delta * \text{card } ?W + t * \text{card } ?W$
using *scan* *insert-le* **by** *simp*
also have $\dots = (k * \Delta + t) * \text{card } ?W$
by (*simp* *add*: *algebra-simps*)
finally show *?thesis* .
qed

lemma *find-pivots-scan-and-initial-insert-cost-le-seen*:

assumes *S-subset*: $S \subseteq V$
and *degree*: *edge-outdegree-le* Δ

and $S\text{-cap}$: $\text{card } S \leq \text{cap}$
and anti : $\text{tree-antichain } S$
and $k\text{-pos}$: $0 < k$
and insert : $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B)$
shows $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $(k * \Delta + t) * \text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B)$
proof –
let $?W = \text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B$
have scan : $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B \leq$
 $k * \Delta * \text{card } ?W$
by $(\text{rule find-pivots-capped-scan-cost-le-seen}[\text{OF } S\text{-subset degree}])$
have insert-le : $c\text{-insert} \leq t * \text{card } ?W$
by $(\text{rule partition-initial-insert-cost-bound-capped-pivots-le-seen}$
 $[\text{OF insert } S\text{-subset } S\text{-cap anti } k\text{-pos}])$
have $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $k * \Delta * \text{card } ?W + t * \text{card } ?W$
using scan insert-le **by** simp
also have $\dots = (k * \Delta + t) * \text{card } ?W$
by $(\text{simp add: algebra-simps})$
finally show $?thesis$.
qed

lemma $\text{find-pivots-scan-and-initial-insert-cost-le-seen-scaled}$:
assumes $S\text{-subset}$: $S \subseteq V$
and degree : $\text{edge-outdegree-le } \Delta$
and anti : $\text{tree-antichain } S$
and $S\text{-k-cap}$: $k * \text{card } S \leq \text{cap}$
and insert : $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B)$
and insert-factor : $t \leq A\text{-insert} * k$
shows $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $(k * \Delta + A\text{-insert}) * \text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B)$
proof –
let $?W = \text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B$
have scan : $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B \leq$
 $k * \Delta * \text{card } ?W$
by $(\text{rule find-pivots-capped-scan-cost-le-seen}[\text{OF } S\text{-subset degree}])$
have insert-le : $c\text{-insert} \leq A\text{-insert} * \text{card } ?W$
by $(\text{rule partition-initial-insert-cost-bound-capped-pivots-le-seen-scaled}$
 $[\text{OF insert } S\text{-subset anti } S\text{-k-cap insert-factor}])$
have $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $k * \Delta * \text{card } ?W + A\text{-insert} * \text{card } ?W$
using scan insert-le **by** simp
also have $\dots = (k * \Delta + A\text{-insert}) * \text{card } ?W$
by $(\text{simp add: algebra-simps})$
finally show $?thesis$.
qed

lemma *find-pivots-scan-and-initial-insert-budget-from-seen-scaled:*

assumes *S-subset*: $S \subseteq V$
and *degree*: *edge-outdegree-le* Δ
and *anti*: *tree-antichain* S
and *S-k-cap*: $k * \text{card } S \leq \text{cap}$
and *insert*: *partition-initial-insert-cost-bound* $c\text{-insert } t$
 (*find-pivots-pivots-capped* $k \text{ cap } d \text{ } S \text{ } B$)
and *insert-factor*: $t \leq A\text{-insert} * k$
and *factor*: $k * \Delta + A\text{-insert} \leq A$
and *seen-progress*:
 $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B) \leq \text{card } U$
shows *fp-iter-capped-scan-cost* $k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $A * \text{card } U$

proof –

let $?W = \text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B$
have *local*:
 $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $(k * \Delta + A\text{-insert}) * \text{card } ?W$
by (*rule find-pivots-scan-and-initial-insert-cost-le-seen-scaled*
 [*OF S-subset degree anti S-k-cap insert insert-factor*])
have $(k * \Delta + A\text{-insert}) * \text{card } ?W \leq A * \text{card } U$
by (*rule mult-mono*[*OF factor seen-progress*]) *simp-all*
then show *?thesis*
using *local* **by** *linarith*

qed

lemma *find-pivots-scan-and-initial-insert-budget-from-seen:*

assumes *S-subset*: $S \subseteq V$
and *degree*: *edge-outdegree-le* Δ
and *anti*: *tree-antichain* S
and *k-pos*: $0 < k$
and *no-overflow*:
 $\neg \text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B) > \text{cap}$
and *insert*: *partition-initial-insert-cost-bound* $c\text{-insert } t$
 (*find-pivots-pivots-capped* $k \text{ cap } d \text{ } S \text{ } B$)
and *factor*: $k * \Delta + t \leq A$
and *seen-progress*:
 $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B) \leq \text{card } U$
shows *fp-iter-capped-scan-cost* $k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $A * \text{card } U$

proof –

let $?W = \text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B$
have *local*:
 $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $(k * \Delta + t) * \text{card } ?W$
by (*rule find-pivots-scan-and-initial-insert-cost-le-seen-if-no-overflow*
 [*OF S-subset degree anti k-pos no-overflow insert*])
have $(k * \Delta + t) * \text{card } ?W \leq A * \text{card } U$
by (*rule mult-mono*[*OF factor seen-progress*]) *simp-all*

```

then show ?thesis
  using local by linarith
qed

lemma find-pivots-scan-and-initial-insert-budget-from-seen-total:
  assumes S-subset:  $S \subseteq V$ 
  and degree: edge-outdegree-le  $\Delta$ 
  and S-cap:  $\text{card } S \leq \text{cap}$ 
  and anti: tree-antichain S
  and k-pos:  $0 < k$ 
  and insert: partition-initial-insert-cost-bound c-insert t
    (find-pivots-pivots-capped k cap d S B)
  and factor:  $k * \Delta + t \leq A$ 
  and seen-progress:
     $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ S B}) \leq \text{card } U$ 
  shows fp-iter-capped-scan-cost k cap d S S B + c-insert  $\leq$ 
     $A * \text{card } U$ 
proof –
  let ?W = find-pivots-seen-capped k cap d S B
  have local:
    fp-iter-capped-scan-cost k cap d S S B + c-insert  $\leq$ 
       $(k * \Delta + t) * \text{card } ?W$ 
  by (rule find-pivots-scan-and-initial-insert-cost-le-seen
    [OF S-subset degree S-cap anti k-pos insert])
  have  $(k * \Delta + t) * \text{card } ?W \leq A * \text{card } U$ 
  by (rule mult-mono[OF factor seen-progress]) simp-all
  then show ?thesis
    using local by linarith
qed

```

The relation below is the first costed operational run in this file. It refines the full operational relation by adding a natural-number cost to each BMSSP call and each partition loop. The constructors charge base-case scans, FindPivots scans and initial inserts, pulls, batch prepends, recursive child calls, and the remaining tail of the loop.

This relation is intentionally generic in the partition-cost predicates. It does not know whether a sorted list, bucketed partition, or another data structure implements the primitive operations. Its job is to expose the exact cost skeleton that the later refinement layers and the final headline theorem instantiate.

```

inductive costed-full-operational-partition-loop-state
  and costed-full-operational-bmssp where
    Cost-State-Done:
      keys-of D = {}  $\implies$ 
      bound-le (Fin a) B  $\implies$ 
      complete-on d' (bound-tree P (Fin a))  $\implies$ 
      costed-full-operational-partition-loop-state  $\Delta$  M-of t k cap l d P B d' D a
      [] [] (Fin a) [range-tree P a (Fin a)]

```

$(\text{bound-tree } P \text{ (Fin } a) \cup \bigcup (\text{set } [\text{range-tree } P \text{ } a \text{ (Fin } a)])) \ 0$
 | *Cost-State-Step:*
 $\text{pull-separates } D \text{ (M-of } l) \text{ Bmax } S\text{-pull } \beta \text{ } D\text{-pull} \implies$
 $\text{bound-le (Fin } \beta) \text{ } B \implies$
 $\text{bmssp-pre-full } d \text{ (split-below } d \text{ } P \text{ } \beta) \text{ (Fin } \beta) \implies$
 $S\text{-pull} = \text{split-below } d \text{ } P \text{ } \beta \implies$
 $a \leq b \implies$
 $\text{complete-on } d' \text{ (bound-tree } P \text{ (Fin } a)) \implies$
 $\text{costed-full-operational-bmssp } \Delta \text{ M-of } t \text{ } k \text{ cap } l \text{ } d \text{ } S\text{-pull (Fin } \beta)$
 $d\text{-child (Fin } b) \text{ } U\text{-child } c\text{-child} \implies$
 $\text{complete-preserved } d\text{-child } d' \text{ } U\text{-child} \implies$
 $\text{batch} =$
 $\text{edge-relaxation-pairs-between } d\text{-child } U\text{-child } b \text{ } \beta \text{ } @$
 $\text{label-pairs-between } d \text{ } S\text{-pull } b \text{ } \beta \implies$
 $D\text{-next} = \text{batch-min-update } D\text{-pull } \text{batch} \implies$
 $\text{partition-pull-cost-bound } c\text{-pull } S\text{-pull} \implies$
 $\text{partition-batch-cost-bound } c\text{-batch } t \text{ } \text{batch} \implies$
 $\text{costed-full-operational-partition-loop-state } \Delta \text{ M-of } t \text{ } k \text{ cap } l \text{ } d \text{ } P \text{ } B \text{ } d'$
 $D\text{-next } b \text{ } \beta \text{ } \text{bs } B' \text{ } U\text{-tail } U\text{-tail } c\text{-tail} \implies$
 $c = c\text{-pull} + c\text{-batch} + c\text{-child} + c\text{-tail} \implies$
 $\text{costed-full-operational-partition-loop-state } \Delta \text{ M-of } t \text{ } k \text{ cap } l \text{ } d \text{ } P \text{ } B \text{ } d' \text{ } D \text{ } a$
 $(\beta \# \beta \text{ } \text{bs}) (b \# \text{bs}) B'$
 $(\text{range-tree } P \text{ } a \text{ (Fin } b) \# U\text{-tail})$
 $(\text{bound-tree } P \text{ (Fin } a) \cup$
 $\bigcup (\text{set } (\text{range-tree } P \text{ } a \text{ (Fin } b) \# U\text{-tail}))) \text{ } c$
 | *Cost-Base:*
 $S = \{x\} \implies$
 $\text{costed-full-operational-bmssp } \Delta \text{ M-of } t \text{ } k \text{ cap } 0 \text{ } d \text{ } S \text{ } B$
 $(\lambda v. \text{ if } v \in \text{base-case-vertices } k \text{ } x \text{ } B \text{ then dist } s \text{ } v \text{ else } d \text{ } v)$
 $(\text{base-case-bound } k \text{ } x \text{ } B)$
 $(\text{base-case-vertices } k \text{ } x \text{ } B)$
 $(\text{base-case-scan-cost } \Delta \text{ } k \text{ } x \text{ } B)$
 | *Cost-Step:*
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \implies$
 $\text{costed-full-operational-partition-loop-state } \Delta \text{ M-of } t \text{ } k \text{ cap } l$
 $(\text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \text{ } B \text{ } d' \text{ } D \text{ } a \text{ } \beta \text{ } \text{bs } B'$
 $U\text{ } U\text{-loop } c\text{-loop} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{ find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{ find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\} \implies$
 $c = \text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} + c\text{-loop} \implies$
 $\text{costed-full-operational-bmssp } \Delta \text{ M-of } t \text{ } k \text{ cap (Suc } l) \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$

theorem *costed-full-operational-partition-loop-state-refines:*
costed-full-operational-partition-loop-state Δ *M-of t k cap l d P B d' D a*
betas bs B' Us U c $\implies$
full-operational-partition-loop-state (*M-of l*) *t k cap l d P B d' D a*
betas bs B' Us U c

and *costed-full-operational-bmssp-refines:*
costed-full-operational-bmssp Δ *M-of t k cap l d S B d' B' U c* $\implies$
full-operational-bmssp *k cap l d S B d' B' U*

proof (*induction rule:*
costed-full-operational-partition-loop-state-costed-full-operational-bmssp.inducts)
case (*Cost-State-Done D a B d' P* Δ *M-of t k cap l d*)
show ?*case*
by (*rule full-operational-partition-loop-state.State-Done*
 $[OF$ *Cost-State-Done(1)* *Cost-State-Done(2)* *Cost-State-Done(3)])
next
case (*Cost-State-Step D M-of l Bmax S-pull beta D-pull B d P a b d'*
 Δ *t k cap d-child U-child c-child batch D-next c-pull c-batch*
betas bs B' Us-tail U-tail c-tail c)
have *child-run:*
full-operational-bmssp *k cap l d S-pull* (*Fin beta*)
d-child (*Fin b*) *U-child*
using *Cost-State-Step* **by** *blast*
have *tail-run:*
full-operational-partition-loop-state (*M-of l*) *t k cap l d P B d'*
D-next b betas bs B' Us-tail U-tail c-tail
using *Cost-State-Step* **by** *blast*
show ?*case*
proof (*rule full-operational-partition-loop-state.State-Step*)
show *pull-separates D* (*M-of l*) *Bmax S-pull beta D-pull*
using *Cost-State-Step* **by** *blast*
show *bound-le* (*Fin beta*) *B*
using *Cost-State-Step* **by** *blast*
show *bmssp-pre-full d* (*split-below d P beta*) (*Fin beta*)
using *Cost-State-Step* **by** *blast*
show *S-pull = split-below d P beta*
using *Cost-State-Step* **by** *blast*
show $a \leq b$
using *Cost-State-Step* **by** *blast*
show *complete-on d'* (*bound-tree P* (*Fin a*))
using *Cost-State-Step* **by** *blast*
show *full-operational-bmssp* *k cap l d S-pull* (*Fin beta*)
d-child (*Fin b*) *U-child*
using *child-run* .
show *complete-preserved d-child d' U-child*
using *Cost-State-Step* **by** *blast*
show *batch =*
edge-relaxation-pairs-between d-child U-child b beta @
label-pairs-between d S-pull b beta
using *Cost-State-Step* **by** *blast**


```

show  $D\text{-next} = \text{batch-min-update } D\text{-pull } \text{batch}$ 
  using Cost-State-Step by blast
show  $\text{partition-pull-cost-bound } c\text{-pull } S\text{-pull}$ 
  using Cost-State-Step by blast
show  $\text{partition-batch-cost-bound } c\text{-batch } t \text{ batch}$ 
  using Cost-State-Step by blast
show  $\text{full-operational-partition-loop-state } (M\text{-of } l) \ t \ k \ \text{cap } l \ d \ P \ B \ d'$ 
   $D\text{-next } b \ \text{betas } bs \ B' \ Us\text{-tail } U\text{-tail } c\text{-tail}$ 
  using tail-run .
show  $c = c\text{-pull} + c\text{-batch} + c\text{-child} + c\text{-tail}$ 
  using Cost-State-Step by blast
qed
next
case (Cost-Base  $S \ x \ \Delta \ M\text{-of } t \ k \ \text{cap } d \ B$ )
show ?case
  by (rule Full-Base[OF Cost-Base.hyps])
next
case (Cost-Step  $D \ k \ \text{cap } d \ S \ B \ c\text{-insert } t \ \Delta \ M\text{-of } l \ d' \ a \ \text{betas } bs \ B'$ 
   $Us \ U\text{-loop } c\text{-loop } U \ c$ )
have state:
   $\text{full-operational-partition-loop-state } (M\text{-of } l) \ t \ k \ \text{cap } l$ 
   $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$ 
   $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$ 
   $Us \ U\text{-loop } c\text{-loop}$ 
  using Cost-Step by blast
have loop:
   $\text{full-operational-partition-loop } k \ \text{cap } l$ 
   $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$ 
   $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ a \ \text{betas } bs \ B'$ 
   $Us \ U\text{-loop}$ 
  by (rule full-operational-partition-loop-state-refines[OF state])
show ?case
proof (rule Full-Step)
show  $\text{full-operational-partition-loop } k \ \text{cap } l$ 
   $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$ 
   $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ a \ \text{betas } bs \ B'$ 
   $Us \ U\text{-loop}$ 
  using loop .
show complete-on  $d'$ 
   $\{v \in \text{bound-tree } S \ B'. \$ 
   $\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\}$ 
  using Cost-Step by blast
show  $U = U\text{-loop} \cup$ 
   $\{v \in \text{bound-tree } S \ B'. \$ 
   $\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\}$ 
  using Cost-Step by blast
qed
qed

```

theorem *costed-full-operational-bmssp-correct:*

assumes *run:*

costed-full-operational-bmssp Δ *M-of t k cap l d S B d' B' U c*

and *sound:* *sound-label d*

and *pre:* *bmssp-pre-full d S B*

and *S-reaches:* $\bigwedge x. x \in S \implies \text{reachable } s \ x$

shows *bmssp-post-full d S B d' B' U*

proof –

have *run': full-operational-bmssp k cap l d S B d' B' U*

by (rule *costed-full-operational-bmssp-refines*[*OF run*])

show *?thesis*

by (rule *full-operational-bmssp-correct*[*OF run' sound pre S-reaches*])

qed

lemma *costed-full-operational-partition-loop-state-lengths:*

assumes *run:*

costed-full-operational-partition-loop-state Δ *M-of t k cap l d P B d' D a*

betas bs B' Us U c

shows *length betas = length bs*

and *length Us = Suc (length bs)*

proof –

have *state:*

full-operational-partition-loop-state (M-of l) t k cap l d P B d' D a

betas bs B' Us U c

by (rule *costed-full-operational-partition-loop-state-refines*[*OF run*])

show *length betas = length bs*

by (rule *full-operational-partition-loop-state-lengths(1)*[*OF state*])

show *length Us = Suc (length bs)*

by (rule *full-operational-partition-loop-state-lengths(2)*[*OF state*])

qed

theorem *finite-initial-label-costed-full-operational-top-level-correct:*

assumes *all-reachable:* $\bigwedge v. v \in V \implies \text{reachable } s \ v$

and *run:*

costed-full-operational-bmssp Δ *M-of t k cap l finite-initial-label {s}*

Infinity d' Infinity U c

shows *sssp-correct d'*

proof –

have *run': full-operational-bmssp k cap l finite-initial-label {s}*

Infinity d' Infinity U

by (rule *costed-full-operational-bmssp-refines*[*OF run*])

show *?thesis*

by (rule *finite-initial-label-full-operational-top-level-correct*
[*OF all-reachable run'*])

qed

theorem *costed-full-operational-partition-loop-state-trace-and-degree-cost:*

assumes *run:*

costed-full-operational-partition-loop-state Δ *M-of t k cap l d P B d' D a*

betas bs B' Us U c
 and sound: sound-label d
 and pre: bmssp-pre-full d P B
 and P-reaches: $\bigwedge x. x \in P \implies \text{reachable } s \ x$
 and degree: edge-outdegree-le Δ
 obtains child-costs child-sets where
 concrete-partition-loop-trace P B a bs d' B' Us U
 length child-costs = length child-sets
 $\bigwedge X. X \in \text{set child-sets} \implies X \subseteq V$
 $c \leq \text{sum-list child-costs} + (M\text{-of } l) * \text{length child-costs} +$
 $t * (\Delta * \text{sum-list (map card child-sets)} + (M\text{-of } l) * \text{length child-costs})$
 proof –
 have state:
 full-operational-partition-loop-state (M-of l) t k cap l d P B d' D a
 betas bs B' Us U c
 by (rule costed-full-operational-partition-loop-state-refines[OF run])
 show thesis
 by (rule full-operational-partition-loop-state-trace-and-degree-cost
 [OF state sound pre P-reaches degree that])
 qed

theorem costed-full-operational-partition-loop-state-child-call-data:
 costed-full-operational-partition-loop-state Δ M-of t k cap l d P B d' D a
 betas bs B' Us U c $\implies$
 sound-label d $\implies$
 bmssp-pre-full d P B $\implies$
 $(\bigwedge x. x \in P \implies \text{reachable } s \ x) \implies$
 $\exists \text{ child-costs child-sets.}$
 $\text{length child-costs} = \text{length child-sets} \wedge$
 $\text{length child-costs} = \text{length bs} \wedge$
 list-all2
 $(\lambda c\text{-child } U\text{-child. } \exists S\text{-child } B\text{-child } d\text{-child } B\text{-child'.$
 $\text{costed-full-operational-bmssp } \Delta \text{ M-of } t \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child' } U\text{-child } c\text{-child} \wedge$
 $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child} \wedge$
 $(\forall x \in S\text{-child. } \text{reachable } s \ x) \wedge$
 $\text{card } S\text{-child} \leq M\text{-of } l)$
 $\text{child-costs child-sets} \wedge$
 $(\forall X \in \text{set child-sets. } X \subseteq V) \wedge$
 $c \leq \text{sum-list child-costs} + (M\text{-of } l) * \text{length child-costs} +$
 $t * (\text{sum-list (map } (\lambda X. \text{card (outgoing-edges } X)) \text{ child-sets)} +$
 $(M\text{-of } l) * \text{length child-costs})$
 and costed-full-operational-bmssp-child-call-data-trivial:
 costed-full-operational-bmssp Δ M-of t k cap l d S B d' B' U c $\implies \text{True}$
 proof (induction rule:
 costed-full-operational-partition-loop-state-costed-full-operational-bmssp.inducts)
 case (Cost-State-Done D a B d' P Δ M-of t k cap l d)
 show ?case
 by (intro exI[of -] exI[of -]) simp

```

next
  case (Cost-State-Step D M-of l Bmax S-pull beta D-pull B d P a b d'
     $\Delta$  t k cap d-child U-child c-child batch D-next c-pull c-batch
    betas bs B' Us-tail U-tail c-tail c)
  obtain child-costs child-sets where len-tail:
    length child-costs = length child-sets
  and child-len-tail: length child-costs = length bs
  and calls-tail:
    list-all2
      ( $\lambda c\text{-child } U\text{-child}. \exists S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
        costed-full-operational-bmssp  $\Delta$  M-of t k cap l d S-child B-child
        d-child B-child' U-child c-child  $\wedge$ 
        bmssp-pre-full d S-child B-child  $\wedge$ 
        ( $\forall x \in S\text{-child}. \text{reachable } s \ x$ )  $\wedge$ 
        card S-child  $\leq$  M-of l)
      child-costs child-sets
  and sets-tail:  $\forall X \in \text{set } \text{child-sets}. X \subseteq V$ 
  and tail:
    c-tail  $\leq$  sum-list child-costs + M-of l * length child-costs +
      t * (sum-list (map ( $\lambda X. \text{card } (\text{outgoing-edges } X)$ ) child-sets) +
        M-of l * length child-costs)
  using Cost-State-Step.IH(8)
    [OF Cost-State-Step.premis(1) Cost-State-Step.premis(2) Cost-State-
Step.premis(3)]
  by blast
  have child-reaches:
     $\bigwedge x. x \in S\text{-pull} \implies \text{reachable } s \ x$ 
    using Cost-State-Step
    unfolding split-below-def by blast
  have child-pre: bmssp-pre-full d S-pull (Fin beta)
    using Cost-State-Step by blast
  have child-run:
    costed-full-operational-bmssp  $\Delta$  M-of t k cap l d S-pull (Fin beta)
    d-child (Fin b) U-child c-child
    using Cost-State-Step by blast
  have child-post:
    bmssp-post-full d S-pull (Fin beta) d-child (Fin b) U-child
    by (rule costed-full-operational-bmssp-correct
      [OF child-run Cost-State-Step.premis(1) child-pre child-reaches])
  have child-set: U-child  $\subseteq V$ 
    using child-post unfolding bmssp-post-full-def bound-tree-def by blast
  have P-subset: P  $\subseteq V$ 
    using Cost-State-Step unfolding bmssp-pre-full-def by blast
  have finite-P: finite P
    using finite-subset [OF P-subset finite-V] .
  have S-pull-subset: S-pull  $\subseteq P$ 
    using Cost-State-Step unfolding split-below-def by blast
  have finite-S-pull: finite S-pull
    using finite-subset [OF S-pull-subset finite-P] .

```

have *card-pull*: $\text{card } S\text{-pull} \leq M\text{-of } l$
using *Cost-State-Step* **unfolding** *pull-separates-def* **by** *blast*
have *pull-cost*: $c\text{-pull} \leq M\text{-of } l$
using *Cost-State-Step* *card-pull*
unfolding *partition-pull-cost-bound-def* **by** *linarith*
have *edge-len*:
 $\text{length } (\text{edge-relaxation-pairs-between } d\text{-child } U\text{-child } b \text{ beta}) \leq$
 $\text{card } (\text{outgoing-edges } U\text{-child})$
by (rule *edge-relaxation-pairs-between-length-le-outgoing*)
have *label-len*:
 $\text{length } (\text{label-pairs-between } d \text{ } S\text{-pull } b \text{ beta}) \leq \text{card } S\text{-pull}$
by (rule *label-pairs-between-length-le-card* [OF *finite-S-pull*])
have *batch-len*:
 $\text{length batch} \leq \text{card } (\text{outgoing-edges } U\text{-child}) + M\text{-of } l$
using *Cost-State-Step* *edge-len* *label-len* *card-pull* **by** *simp*
have *batch-cost*:
 $c\text{-batch} \leq t * (\text{card } (\text{outgoing-edges } U\text{-child}) + M\text{-of } l)$
proof –
have $c\text{-batch} \leq t * \text{length batch}$
using *Cost-State-Step*
unfolding *partition-batch-cost-bound-def* **by** *simp*
also have $\dots \leq t * (\text{card } (\text{outgoing-edges } U\text{-child}) + M\text{-of } l)$
using *batch-len* **by** *simp*
finally show *?thesis* .
qed
let *?child-costs* = $c\text{-child} \# \text{child-costs}$
let *?child-sets* = $U\text{-child} \# \text{child-sets}$
have *len*: $\text{length } ?\text{child-costs} = \text{length } ?\text{child-sets}$
using *len-tail* **by** *simp*
have *child-len*: $\text{length } ?\text{child-costs} = \text{length } (b \# bs)$
using *child-len-tail* **by** *simp*
have *call-head*:
 $\exists S\text{-child } B\text{-child } d\text{-child}' B\text{-child}'.$
 $\text{costed-full-operational-bmssp } \Delta \text{ } M\text{-of } t \text{ } k \text{ cap } l \text{ } d \text{ } S\text{-child } B\text{-child}$
 $d\text{-child}' B\text{-child}' U\text{-child } c\text{-child} \wedge$
 $\text{bmssp-pre-full } d \text{ } S\text{-child } B\text{-child} \wedge$
 $(\forall x \in S\text{-child}. \text{reachable } s \text{ } x) \wedge$
 $\text{card } S\text{-child} \leq M\text{-of } l$
using *child-run* *child-pre* *child-reaches* *card-pull* **by** *blast*
have *calls*:
list-all2
 $(\lambda c\text{-child } U\text{-child}. \exists S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\text{costed-full-operational-bmssp } \Delta \text{ } M\text{-of } t \text{ } k \text{ cap } l \text{ } d \text{ } S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' U\text{-child } c\text{-child} \wedge$
 $\text{bmssp-pre-full } d \text{ } S\text{-child } B\text{-child} \wedge$
 $(\forall x \in S\text{-child}. \text{reachable } s \text{ } x) \wedge$
 $\text{card } S\text{-child} \leq M\text{-of } l)$
 $?child\text{-costs } ?child\text{-sets}$
using *call-head* *calls-tail* **by** *simp*

```

have sets:  $\forall X \in \text{set } ?\text{child-sets}. X \subseteq V$ 
using child-set sets-tail by simp
have cost:
   $c \leq \text{sum-list } ?\text{child-costs} + M\text{-of } l * \text{length } ?\text{child-costs} +$ 
   $t * (\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) ?\text{child-sets}) +$ 
   $M\text{-of } l * \text{length } ?\text{child-costs})$ 
proof -
  have c-eq:  $c = c\text{-pull} + c\text{-batch} + c\text{-child} + c\text{-tail}$ 
  using Cost-State-Step by blast
  have c-pull + c-batch + c-child + c-tail  $\leq$ 
     $M\text{-of } l + t * (\text{card } (\text{outgoing-edges } U\text{-child}) + M\text{-of } l) +$ 
     $c\text{-child} +$ 
     $(\text{sum-list } \text{child-costs} + M\text{-of } l * \text{length } \text{child-costs} +$ 
     $t * (\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{child-sets}) +$ 
     $M\text{-of } l * \text{length } \text{child-costs}))$ 
  using pull-cost batch-cost tail by simp
  also have ... =
     $\text{sum-list } ?\text{child-costs} + M\text{-of } l * \text{length } ?\text{child-costs} +$ 
     $t * (\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) ?\text{child-sets}) +$ 
     $M\text{-of } l * \text{length } ?\text{child-costs})$ 
  by (simp add: algebra-simps)
  finally show ?thesis
  using c-eq by simp
qed
show ?case
proof (intro exI[of - ?child-costs] exI[of - ?child-sets] conjI)
  show length ?child-costs = length ?child-sets
  using len .
  show length ?child-costs = length (b # bs)
  using child-len .
  show list-all2
    ( $\lambda c\text{-child } U\text{-child}. \exists S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
     $\text{costed-full-operational-bmssp } \Delta M\text{-of } t k \text{ cap } l d S\text{-child } B\text{-child}$ 
     $d\text{-child } B\text{-child}' U\text{-child } c\text{-child} \wedge$ 
     $\text{bmssp-pre-full } d S\text{-child } B\text{-child} \wedge$ 
     $(\forall x \in S\text{-child}. \text{reachable } s x) \wedge$ 
     $\text{card } S\text{-child} \leq M\text{-of } l)$ 
    ?child-costs ?child-sets
  using calls .
  show  $\forall X \in \text{set } ?\text{child-sets}. X \subseteq V$ 
  using sets .
  show  $c \leq \text{sum-list } ?\text{child-costs} + M\text{-of } l * \text{length } ?\text{child-costs} +$ 
   $t * (\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) ?\text{child-sets}) +$ 
   $M\text{-of } l * \text{length } ?\text{child-costs})$ 
  using cost .
qed
next
case (Cost-Base  $S x \Delta M\text{-of } t k \text{ cap } d B$ )
show ?case

```

```

    by simp
next
  case (Cost-Step D k cap d S B c-insert t Δ M-of l d' a betas bs B'
        Us U-loop c-loop U c)
  show ?case
    by simp
qed

theorem costed-full-operational-partition-loop-state-child-calls-and-cost:
assumes run:
  costed-full-operational-partition-loop-state Δ M-of t k cap l d P B d' D a
  betas bs B' Us U c
and sound: sound-label d
and pre: bmssp-pre-full d P B
and P-reaches:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
obtains child-costs child-sets where
  concrete-partition-loop-trace P B a bs d' B' Us U
  length child-costs = length child-sets
  length child-costs = length bs
  list-all2
    ( $\lambda c\text{-child } U\text{-child}. \exists S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
      costed-full-operational-bmssp Δ M-of t k cap l d S-child B-child
      d-child B-child' U-child c-child ∧
      bmssp-pre-full d S-child B-child ∧
      ( $\forall x \in S\text{-child}. \text{reachable } s \ x$ ) ∧
      card S-child ≤ M-of l)
  child-costs child-sets
   $\bigwedge X. X \in \text{set } \text{child-sets} \implies X \subseteq V$ 
  c ≤ sum-list child-costs + (M-of l) * length child-costs +
    t * (sum-list (map ( $\lambda X. \text{card } (\text{outgoing-edges } X)$ ) child-sets) +
      (M-of l) * length child-costs)

proof –
have state:
  full-operational-partition-loop-state (M-of l) t k cap l d P B d' D a
  betas bs B' Us U c
by (rule costed-full-operational-partition-loop-state-refines[OF run])
have loop:
  full-operational-partition-loop k cap l d P B d' a betas bs B' Us U
by (rule full-operational-partition-loop-state-refines[OF state])
have trace: concrete-partition-loop-trace P B a bs d' B' Us U
by (rule full-operational-partition-loop-trace[OF loop sound pre P-reaches])
have data:
   $\exists \text{child-costs } \text{child-sets}.$ 
    length child-costs = length child-sets ∧
    length child-costs = length bs ∧
    list-all2
      ( $\lambda c\text{-child } U\text{-child}. \exists S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
        costed-full-operational-bmssp Δ M-of t k cap l d S-child B-child
        d-child B-child' U-child c-child ∧

```

```

    bmssp-pre-full d S-child B-child  $\wedge$ 
    ( $\forall x \in S\text{-child}. \text{reachable } s \ x$ )  $\wedge$ 
    card S-child  $\leq$  M-of l)
  child-costs child-sets  $\wedge$ 
  ( $\forall X \in \text{set child-sets}. X \subseteq V$ )  $\wedge$ 
  c  $\leq$  sum-list child-costs + (M-of l) * length child-costs +
  t * (sum-list (map ( $\lambda X. \text{card } (\text{outgoing-edges } X)$ ) child-sets) +
  (M-of l) * length child-costs)
  by (rule costed-full-operational-partition-loop-state-child-call-data
  [OF run sound pre P-reaches])
obtain child-costs child-sets where len:
  length child-costs = length child-sets
and child-len: length child-costs = length bs
and calls:
  list-all2
  ( $\lambda c\text{-child } U\text{-child}. \exists S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
  costed-full-operational-bmssp  $\Delta$  M-of t k cap l d S-child B-child
  d-child B-child' U-child c-child  $\wedge$ 
  bmssp-pre-full d S-child B-child  $\wedge$ 
  ( $\forall x \in S\text{-child}. \text{reachable } s \ x$ )  $\wedge$ 
  card S-child  $\leq$  M-of l)
  child-costs child-sets
and sets:  $\forall X \in \text{set child-sets}. X \subseteq V$ 
and cost:
  c  $\leq$  sum-list child-costs + (M-of l) * length child-costs +
  t * (sum-list (map ( $\lambda X. \text{card } (\text{outgoing-edges } X)$ ) child-sets) +
  (M-of l) * length child-costs)
using data by blast
show thesis
proof (rule that[OF trace len child-len calls - cost])
  fix X
  assume X  $\in$  set child-sets
  then show X  $\subseteq$  V
  using sets by blast
qed
qed

theorem costed-full-operational-partition-loop-state-cost-from-child-bound:
assumes run:
  costed-full-operational-partition-loop-state  $\Delta$  M-of t k cap l d P B d' D a
  betas bs B' Us U c
and sound: sound-label d
and pre: bmssp-pre-full d P B
and P-reaches:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
and degree: edge-outdegree-le  $\Delta$ 
and child-bound:
   $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
   $\llbracket \text{costed-full-operational-bmssp } \Delta \text{ M-of t k cap l d S-child B-child}$ 
   $d\text{-child B-child' U-child c-child};$ 

```


$bmssp\text{-}pre\text{-}full\ d\ S\text{-}child\ B\text{-}child;$
 $\bigwedge x. x \in S\text{-}child \implies reachable\ s\ x;$
 $card\ S\text{-}child \leq M\text{-}of\ l]$
 $\implies c\text{-}child \leq C * card\ U\text{-}child$

obtains *child-sets* **where**

$concrete\text{-}partition\text{-}loop\text{-}trace\ P\ B\ a\ bs\ d'\ B'\ Us\ U$
 $\bigwedge X. X \in set\ child\text{-}sets \implies X \subseteq V$
 $length\ child\text{-}sets = length\ bs$
 $c \leq (C + t * \Delta) * sum\text{-}list\ (map\ card\ child\text{-}sets) +$
 $(Suc\ t) * (M\text{-}of\ l) * length\ bs$

proof –

have *state*:

$full\text{-}operational\text{-}partition\text{-}loop\text{-}state\ (M\text{-}of\ l)\ t\ k\ cap\ l\ d\ P\ B\ d'\ D\ a$
 $betas\ bs\ B'\ Us\ U\ c$

by (*rule* *costed-full-operational-partition-loop-state-refines*[*OF run*])

have *loop*:

$full\text{-}operational\text{-}partition\text{-}loop\ k\ cap\ l\ d\ P\ B\ d'\ a\ betas\ bs\ B'\ Us\ U$

by (*rule* *full-operational-partition-loop-state-refines*[*OF state*])

have *trace*:

$concrete\text{-}partition\text{-}loop\text{-}trace\ P\ B\ a\ bs\ d'\ B'\ Us\ U$

by (*rule* *full-operational-partition-loop-trace*[*OF loop sound pre P-reaches*])

have *data*:

$\exists\ child\text{-}costs\ child\text{-}sets.$
 $length\ child\text{-}costs = length\ child\text{-}sets \wedge$
 $length\ child\text{-}costs = length\ bs \wedge$
 $list\text{-}all2$
 $(\lambda c\text{-}child\ U\text{-}child. \exists\ S\text{-}child\ B\text{-}child\ d\text{-}child\ B\text{-}child'.$
 $costed\text{-}full\text{-}operational\text{-}bmssp\ \Delta\ M\text{-}of\ t\ k\ cap\ l\ d\ S\text{-}child\ B\text{-}child$
 $d\text{-}child\ B\text{-}child'\ U\text{-}child\ c\text{-}child \wedge$
 $bmssp\text{-}pre\text{-}full\ d\ S\text{-}child\ B\text{-}child \wedge$
 $(\forall x \in S\text{-}child. reachable\ s\ x) \wedge$
 $card\ S\text{-}child \leq M\text{-}of\ l)$
 $child\text{-}costs\ child\text{-}sets \wedge$
 $(\forall X \in set\ child\text{-}sets. X \subseteq V) \wedge$
 $c \leq sum\text{-}list\ child\text{-}costs + (M\text{-}of\ l) * length\ child\text{-}costs +$
 $t * (sum\text{-}list\ (map\ (\lambda X. card\ (outgoing\text{-}edges\ X))\ child\text{-}sets) +$
 $(M\text{-}of\ l) * length\ child\text{-}costs)$

by (*rule* *costed-full-operational-partition-loop-state-child-call-data*
[*OF run sound pre P-reaches*])

obtain *child-costs child-sets* **where** *len*: $length\ child\text{-}costs = length\ child\text{-}sets$

and *child-len*: $length\ child\text{-}costs = length\ bs$

and *calls*:

$list\text{-}all2$
 $(\lambda c\text{-}child\ U\text{-}child. \exists\ S\text{-}child\ B\text{-}child\ d\text{-}child\ B\text{-}child'.$
 $costed\text{-}full\text{-}operational\text{-}bmssp\ \Delta\ M\text{-}of\ t\ k\ cap\ l\ d\ S\text{-}child\ B\text{-}child$
 $d\text{-}child\ B\text{-}child'\ U\text{-}child\ c\text{-}child \wedge$
 $bmssp\text{-}pre\text{-}full\ d\ S\text{-}child\ B\text{-}child \wedge$
 $(\forall x \in S\text{-}child. reachable\ s\ x) \wedge$
 $card\ S\text{-}child \leq M\text{-}of\ l)$

```

      child-costs child-sets
and sets:  $\bigwedge X. X \in \text{set child-sets} \implies X \subseteq V$ 
and cost:
   $c \leq \text{sum-list child-costs} + (M\text{-of } l) * \text{length child-costs} +$ 
     $t * (\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ child-sets}) +$ 
       $(M\text{-of } l) * \text{length child-costs})$ 
using data by blast
have child-sets-len:  $\text{length child-sets} = \text{length bs}$ 
using len child-len by simp
have child-cost-bounds:
  list-all2  $(\lambda c\text{-child } U\text{-child}. c\text{-child} \leq C * \text{card } U\text{-child})$ 
    child-costs child-sets
using calls
proof (induction rule: list-all2-induct)
  case Nil
  then show ?case by simp
next
  case (Cons c-child child-costs U-child child-sets)
  obtain S-child B-child d-child B-child' where child:
    costed-full-operational-bmssp  $\Delta M\text{-of } t \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$ 
    d-child B-child' U-child c-child
  and child-pre: bmssp-pre-full d S-child B-child
  and child-reaches:  $\forall x \in S\text{-child}. \text{reachable } s \ x$ 
  and child-card:  $\text{card } S\text{-child} \leq M\text{-of } l$ 
  using Cons.hyps by blast
  have head:  $c\text{-child} \leq C * \text{card } U\text{-child}$ 
  by (rule child-bound[OF child child-pre - child-card])
    (use child-reaches in blast)
  show ?case
  using head Cons.IH Cons.hyps by simp
qed
have child-sum:
   $\text{sum-list child-costs} \leq C * \text{sum-list } (\text{map card child-sets})$ 
proof -
  have sum-list child-costs  $\leq$ 
     $\text{sum-list } (\text{map } (\lambda X. C * \text{card } X) \text{ child-sets})$ 
  using sum-list-le-if-list-all2[OF child-cost-bounds] .
  also have  $\dots = C * \text{sum-list } (\text{map card child-sets})$ 
  by (induction child-sets) (simp-all add: algebra-simps)
  finally show ?thesis .
qed
have edge-sum:
   $\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ child-sets}) \leq$ 
     $\Delta * \text{sum-list } (\text{map card child-sets})$ 
  by (rule sum-card-outgoing-edges-le-degree[OF degree sets])
have c  $\leq$ 
   $C * \text{sum-list } (\text{map card child-sets}) + (M\text{-of } l) * \text{length child-sets} +$ 
   $t * (\Delta * \text{sum-list } (\text{map card child-sets}) +$ 
     $(M\text{-of } l) * \text{length child-sets})$ 

```

```

proof –
  have middle:
     $(M\text{-of } l) * \text{length } \text{child-costs} \leq (M\text{-of } l) * \text{length } \text{child-sets}$ 
    using len by simp
  have inside:
     $\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ child-sets}) +$ 
       $(M\text{-of } l) * \text{length } \text{child-costs} \leq$ 
       $\Delta * \text{sum-list } (\text{map } \text{card } \text{child-sets}) +$ 
       $(M\text{-of } l) * \text{length } \text{child-sets}$ 
    using edge-sum middle by linarith
  have tail-term:
     $t * (\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ child-sets}) +$ 
       $(M\text{-of } l) * \text{length } \text{child-costs}) \leq$ 
       $t * (\Delta * \text{sum-list } (\text{map } \text{card } \text{child-sets}) +$ 
       $(M\text{-of } l) * \text{length } \text{child-sets})$ 
    using inside by simp
  have  $\text{sum-list } \text{child-costs} + (M\text{-of } l) * \text{length } \text{child-costs} +$ 
     $t * (\text{sum-list } (\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) \text{ child-sets}) +$ 
     $(M\text{-of } l) * \text{length } \text{child-costs}) \leq$ 
     $C * \text{sum-list } (\text{map } \text{card } \text{child-sets}) + (M\text{-of } l) * \text{length } \text{child-sets} +$ 
     $t * (\Delta * \text{sum-list } (\text{map } \text{card } \text{child-sets}) +$ 
     $(M\text{-of } l) * \text{length } \text{child-sets})$ 
    using child-sum middle tail-term by linarith
  then show ?thesis
    using cost by linarith
qed
also have ... =
   $(C + t * \Delta) * \text{sum-list } (\text{map } \text{card } \text{child-sets}) +$ 
   $(\text{Suc } t) * (M\text{-of } l) * \text{length } \text{child-sets}$ 
  by (simp add: algebra-simps)
finally have final:
   $c \leq (C + t * \Delta) * \text{sum-list } (\text{map } \text{card } \text{child-sets}) +$ 
   $(\text{Suc } t) * (M\text{-of } l) * \text{length } \text{bs}$ 
  using child-sets-len by simp
show thesis
proof (rule that[OF trace - child-sets-len final])
  fix X
  assume  $X \in \text{set } \text{child-sets}$ 
  then show  $X \subseteq V$ 
    using sets by blast
qed
qed

end

end
theory BMSSP-Range-Costed
  imports BMSSP-Complexity
begin

```

30 Range-Synchronised Costed Runs

The algorithm's time proof charges the recursive calls in one BMSSP loop to the disjoint distance ranges cut out by the successive returned bounds. The earlier costed relation keeps enough information for correctness and local data-structure costs, but its child-call cost is attached to the raw child output. This theory strengthens the operational run with the range invariant: every child output is exactly the next range slice.

That strengthening is the point at which the informal amortised analysis in the paper becomes a reusable Isabelle recurrence. A child call is not charged against the whole parent tree. It is charged against a slice *range-tree* between the previous loop bound and the child output bound. The list version *range-tree-child-list* records all such slices for one parent loop. Since those slices are disjoint and lie inside the final loop output, their vertex and outgoing-edge sums can be bounded by the final output itself.

The exported cost shape is *level-range-cost-bound*. Its first term counts completed vertices multiplied by a level factor; its second term counts outgoing edges with the range cost coefficient. The main technical work below is to show that the local costs of pulling, batching, and recursive child calls close under this shape when moving from level l to *Suc* l .

There are two related relations. The first one uses a single batch cost parameter for the concatenation of edge relaxations and reinserted source labels. The split relation refines it by charging edge batches and source batches separately. That split form is useful for later progress arguments: edge batches are paid for by outgoing edges of the completed ranges, while source batches are paid for by the number of child sources exposed by the partition pulls.

context *unique-shortest-digraph*
begin

lemma *child-costs-le-level-range-child-list-bound*:

assumes *mono*: *nondecreasing-from a bs*

and *costs*: *list-all2* ($\lambda c X. c \leq \text{level-range-cost-bound } A \ R \ l \ X$) *costs*
(range-tree-child-list S a bs)

shows *sum-list costs* $\leq$

$A * l * \text{card } (\text{range-tree-chain } S \ a \ bs \ B) +$

$R * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } S \ a \ bs \ B))$

proof –

define *Xs* **where** *Xs* = *range-tree-child-list S a bs*

have *cost-sum*:

sum-list costs $\leq \text{sum-list } (\text{map } (\text{level-range-cost-bound } A \ R \ l) \ Xs)$

using *costs* **unfolding** *Xs-def*

proof (*induction rule: list-all2-induct*)

case *Nil*

then show *?case* **by** *simp*

next

```

    case (Cons c X costs Xs)
  then show ?case
    unfolding level-range-cost-bound-def by simp
qed
have split-sum:
  sum-list (map (level-range-cost-bound A R l) Xs) =
    A * l * sum-list (map card Xs) +
    R * sum-list (map ( $\lambda X$ . card (outgoing-edges X)) Xs)
proof (induction Xs)
  case Nil
  then show ?case
    unfolding level-range-cost-bound-def by simp
next
  case (Cons X Xs)
  then show ?case
    unfolding level-range-cost-bound-def by (simp add: algebra-simps)
qed
have card-sum:
  sum-list (map card Xs)  $\leq$  card (range-tree-chain S a bs B)
  unfolding Xs-def
  by (rule card-range-tree-child-list-le-chain[OF mono])
have edge-sum:
  sum-list (map ( $\lambda X$ . card (outgoing-edges X)) Xs)  $\leq$ 
    card (outgoing-edges (range-tree-chain S a bs B))
  unfolding Xs-def
  by (rule card-outgoing-edges-range-tree-child-list-le-chain[OF mono])
have sum-list (map (level-range-cost-bound A R l) Xs)  $\leq$ 
  A * l * card (range-tree-chain S a bs B) +
  R * card (outgoing-edges (range-tree-chain S a bs B))
proof -
  have left:
    A * l * sum-list (map card Xs)  $\leq$ 
    A * l * card (range-tree-chain S a bs B)
    using card-sum by simp
  have right:
    R * sum-list (map ( $\lambda X$ . card (outgoing-edges X)) Xs)  $\leq$ 
    R * card (outgoing-edges (range-tree-chain S a bs B))
    using edge-sum by simp
  show ?thesis
    using left right split-sum by linarith
qed
then show ?thesis
  using cost-sum by linarith
qed

inductive range-costed-partition-loop-state
and range-costed-bmssp where
  Range-State-Done:
    keys-of D = {}  $\implies$ 

```

$\text{bound-le } (\text{Fin } a) B \implies$
 $\text{complete-on } d' (\text{bound-tree } P B) \implies$
 $\text{range-costed-partition-loop-state } \Delta M\text{-of } t k \text{ cap } l d P B d' D a$
 $\square \square B [\text{range-tree } P a B]$
 $(\text{bound-tree } P (\text{Fin } a) \cup \bigcup (\text{set } [\text{range-tree } P a B])) 0 \square$

| *Range-State-Stop:*

$\text{bound-le } (\text{Fin } a) B \implies$
 $\text{complete-on } d' (\text{bound-tree } P (\text{Fin } a)) \implies$
 $k * \text{cap} \leq \text{card } (\text{bound-tree } P (\text{Fin } a)) \implies$
 $\text{range-costed-partition-loop-state } \Delta M\text{-of } t k \text{ cap } l d P B d' D a$
 $\square \square (\text{Fin } a) [\text{range-tree } P a (\text{Fin } a)]$
 $(\text{bound-tree } P (\text{Fin } a) \cup \bigcup (\text{set } [\text{range-tree } P a (\text{Fin } a)])) 0 \square$

| *Range-State-Step:*

$\text{pull-separates } D (M\text{-of } l) B_{\max} S\text{-pull } \beta D\text{-pull} \implies$
 $\text{bound-le } (\text{Fin } \beta) B \implies$
 $\text{bmssp-pre-full } d S\text{-pull } (\text{Fin } \beta) \implies$
 $(\forall x \in S\text{-pull}. \text{reachable } s x) \implies$
 $a \leq b \implies$
 $\text{bound-le } (\text{Fin } a) B' \implies$
 $\text{complete-on } d' (\text{bound-tree } P (\text{Fin } a)) \implies$
 $\text{card } (\text{bound-tree } P (\text{Fin } a)) < k * \text{cap} \implies$
 $\text{range-costed-bmssp } \Delta M\text{-of } t k \text{ cap } l d S\text{-pull } (\text{Fin } \beta)$
 $d\text{-child } (\text{Fin } b) U\text{-child } c\text{-child} \implies$
 $U\text{-child} = \text{range-tree } P a (\text{Fin } b) \implies$
 $\text{complete-preserved } d\text{-child } d' U\text{-child} \implies$
 $\text{batch} =$
 $\text{edge-relaxation-pairs-between } d\text{-child } U\text{-child } b \beta @$
 $\text{label-pairs-between } d S\text{-pull } b \beta \implies$
 $D\text{-next} = \text{batch-min-update } D\text{-pull } \text{batch} \implies$
 $\text{partition-pull-cost-bound } c\text{-pull } S\text{-pull} \implies$
 $\text{partition-batch-cost-bound } c\text{-batch } t \text{ batch} \implies$
 $\text{range-costed-partition-loop-state } \Delta M\text{-of } t k \text{ cap } l d P B d'$
 $D\text{-next } b \beta s b s B' U\text{-tail } U\text{-tail } c\text{-tail } \text{child-costs-tail} \implies$
 $c = c\text{-pull} + c\text{-batch} + c\text{-child} + c\text{-tail} \implies$
 $\text{range-costed-partition-loop-state } \Delta M\text{-of } t k \text{ cap } l d P B d' D a$
 $(\beta \# \beta s) (b \# b s) B'$
 $(\text{range-tree } P a (\text{Fin } b) \# U\text{-tail})$
 $(\text{bound-tree } P (\text{Fin } a) \cup$
 $\bigcup (\text{set } (\text{range-tree } P a (\text{Fin } b) \# U\text{-tail}))) c$
 $(c\text{-child} \# \text{child-costs-tail})$

| *Range-Cost-Base:*

$S = \{x\} \implies$
 $\text{range-costed-bmssp } \Delta M\text{-of } t k \text{ cap } 0 d S B$
 $(\lambda v. \text{if } v \in \text{base-case-vertices } k x B \text{ then } \text{dist } s v \text{ else } d v)$
 $(\text{base-case-bound } k x B)$
 $(\text{base-case-vertices } k x B)$
 $(\text{base-case-scan-cost } \Delta k x B)$

| *Range-Cost-Step:*

$D = \text{label-partition-view}$

$$\begin{aligned}
& (\text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B) \\
& (\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \implies \\
& \text{partition-initial-insert-cost-bound } c\text{-insert } t \\
& (\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \implies \\
& \text{range-costed-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } k \text{ cap } l \\
& (\text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B) \\
& (\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \text{ } B \text{ } d' \text{ } D \text{ } a \text{ } \text{betas } bs \text{ } B' \\
& Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs-loop} \implies \\
& \text{complete-on } d' \\
& \{v \in \text{bound-tree } S \text{ } B'. \text{ find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\} \implies \\
& U = U\text{-loop} \cup \\
& \{v \in \text{bound-tree } S \text{ } B'. \text{ find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\} \implies \\
& c = \text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} + c\text{-loop} \implies \\
& \text{range-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } k \text{ cap } (Suc \text{ } l) \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c
\end{aligned}$$

The mutually inductive relation above adds the accounting data that the operational theory intentionally omitted. A loop derivation carries the pull bounds *betas*, the child output bounds *bs*, the concrete range slices *Us*, a total cost *c*, and the list of recursive child costs. The step constructor synchronises these fields by requiring the returned child set to be exactly *range-tree* for the current interval.

The BMSSP branch keeps the usual split between the base case and the recursive step. In the recursive step, FindPivots pays the scan and initial insertion cost, then the range-synchronised loop pays all pulls, batches, and child calls. The correctness argument later forgets the costs and reuses the operational trace theorem; the runtime argument keeps the cost and child-cost lists.

inductive-cases *range-costed-bmssp-zeroE*:
range-costed-bmssp Δ *M-of* *t* *k* *cap* *0* *d* *S* *B* *d'* *B'* *U* *c*

inductive-cases *range-costed-bmssp-SucE*:
range-costed-bmssp Δ *M-of* *t* *k* *cap* *(Suc l)* *d* *S* *B* *d'* *B'* *U* *c*

lemma *range-costed-partition-loop-state-mono*:

assumes *run*:

range-costed-partition-loop-state Δ *M-of* *t* *k* *cap* *l* *d* *P* *B* *d'* *D* *a*
betas *bs* *B'* *Us* *U* *c* *child-costs*

shows *nondecreasing-from* *a* *bs*

using *run* **by** (*induction*) *auto*

theorem *range-costed-partition-loop-state-child-call-ranges*:

assumes *run*:

range-costed-partition-loop-state Δ *M-of* *t* *k* *cap* *l* *d* *P* *B* *d'* *D* *a*
betas *bs* *B'* *Us* *U* *c* *child-costs*

shows *length* *child-costs* = *length* *bs*

and *list-all2*

($\lambda c\text{-child}$ *U-child*. $\exists$ *S-child* *B-child* *d-child* *B-child'*).

range-costed-bmssp Δ *M-of* *t* *k* *cap* *l* *d* *S-child* *B-child*

```

      d-child B-child' U-child c-child  $\wedge$ 
      bmssp-pre-full d S-child B-child  $\wedge$ 
      ( $\forall x \in S\text{-child. reachable } s \ x$ )  $\wedge$ 
      card S-child  $\leq M\text{-of } l$ )
      child-costs (range-tree-child-list P a bs)
    using run
  by (induction) (auto simp: pull-separates-def)

lemma range-costed-pull-cost-le-M:
  assumes pull: pull-separates D M Bmax S beta D-pull
    and cost: partition-pull-cost-bound c S
  shows c  $\leq M$ 
proof -
  have card S  $\leq M$ 
    using pull unfolding pull-separates-def by blast
  then show ?thesis
    using cost unfolding partition-pull-cost-bound-def by linarith
qed

lemma range-costed-batch-cost-le:
  assumes pre: bmssp-pre-full d S (Fin beta)
    and batch:
      batch =
        edge-relaxation-pairs-between d-child U b beta @
        label-pairs-between d S b beta
    and cost: partition-batch-cost-bound c t batch
    and card-S: card S  $\leq M$ 
  shows c  $\leq t * (\text{card } (\text{outgoing-edges } U) + M)$ 
proof -
  have S-subset: S  $\subseteq V$ 
    using pre unfolding bmssp-pre-full-def by blast
  have finite-S: finite S
    using finite-subset[OF S-subset finite-V] .
  have edge-len:
    length (edge-relaxation-pairs-between d-child U b beta)  $\leq$ 
    card (outgoing-edges U)
    by (rule edge-relaxation-pairs-between-length-le-outgoing)
  have label-len:
    length (label-pairs-between d S b beta)  $\leq$  card S
    by (rule label-pairs-between-length-le-card[OF finite-S])
  have batch-len: length batch  $\leq$  card (outgoing-edges U) + M
    using batch edge-len label-len card-S by simp
  have c  $\leq t * \text{length batch}$ 
    using cost unfolding partition-batch-cost-bound-def by simp
  also have ...  $\leq t * (\text{card } (\text{outgoing-edges } U) + M)$ 
    using batch-len by simp
  finally show ?thesis .
qed

```


lemma *range-costed-split-batch-cost-le*:
assumes *pre*: *bmssp-pre-full* *d S* (*Fin* *beta*)
and *edge-cost*:
partition-batch-cost-bound *c-edges* *t*
(*edge-relaxation-pairs-between* *d-child* *U* *b* *beta*)
and *source-cost*:
partition-batch-cost-bound *c-sources* *h*
(*label-pairs-between* *d S* *b* *beta*)
and *c-def*: $c = c\text{-edges} + c\text{-sources}$
and *card-S*: $\text{card } S \leq M$
shows $c \leq t * \text{card } (\text{outgoing-edges } U) + h * M$
proof –
have *S-subset*: $S \subseteq V$
using *pre* **unfolding** *bmssp-pre-full-def* **by** *blast*
have *finite-S*: *finite* *S*
using *finite-subset*[*OF* *S-subset* *finite-V*] .
have *edge-len*:
length (*edge-relaxation-pairs-between* *d-child* *U* *b* *beta*) $\leq$
card (*outgoing-edges* *U*)
by (*rule* *edge-relaxation-pairs-between-length-le-outgoing*)
have *label-len*:
length (*label-pairs-between* *d S* *b* *beta*) $\leq$ *card* *S*
by (*rule* *label-pairs-between-length-le-card*[*OF* *finite-S*])
have *edge-part*: $c\text{-edges} \leq t * \text{card } (\text{outgoing-edges } U)$
proof –
have $c\text{-edges} \leq t * \text{length } (\text{edge-relaxation-pairs-between } d\text{-child } U \text{ } b \text{ } \beta)$
using *edge-cost* **unfolding** *partition-batch-cost-bound-def* **by** *simp*
also have $\dots \leq t * \text{card } (\text{outgoing-edges } U)$
using *edge-len* **by** *simp*
finally show *?thesis* .
qed
have *source-part*: $c\text{-sources} \leq h * M$
proof –
have $c\text{-sources} \leq h * \text{length } (\text{label-pairs-between } d \text{ } S \text{ } b \text{ } \beta)$
using *source-cost* **unfolding** *partition-batch-cost-bound-def* **by** *simp*
also have $\dots \leq h * \text{card } S$
using *label-len* **by** *simp*
also have $\dots \leq h * M$
using *card-S* **by** *simp*
finally show *?thesis* .
qed
show *?thesis*
using *edge-part* *source-part* *c-def* **by** *linarith*
qed

lemma *range-costed-split-batch-cost-le-card*:
assumes *pre*: *bmssp-pre-full* *d S* (*Fin* *beta*)
and *edge-cost*:
partition-batch-cost-bound *c-edges* *t*

```

      (edge-relaxation-pairs-between d-child U b beta)
    and source-cost:
      partition-batch-cost-bound c-sources h
      (label-pairs-between d S b beta)
    and c-def:  $c = c\text{-edges} + c\text{-sources}$ 
  shows  $c \leq t * \text{card}(\text{outgoing-edges } U) + h * \text{card } S$ 
proof -
  have S-subset:  $S \subseteq V$ 
  using pre unfolding bmssp-pre-full-def by blast
  have finite-S: finite S
  using finite-subset[OF S-subset finite-V] .
  have edge-len:
    length (edge-relaxation-pairs-between d-child U b beta)  $\leq$ 
      card (outgoing-edges U)
  by (rule edge-relaxation-pairs-between-length-le-outgoing)
  have label-len:
    length (label-pairs-between d S b beta)  $\leq$  card S
  by (rule label-pairs-between-length-le-card[OF finite-S])
  have edge-part:  $c\text{-edges} \leq t * \text{card}(\text{outgoing-edges } U)$ 
proof -
  have  $c\text{-edges} \leq t * \text{length}(\text{edge-relaxation-pairs-between d-child U b beta})$ 
  using edge-cost unfolding partition-batch-cost-bound-def by simp
  also have  $\dots \leq t * \text{card}(\text{outgoing-edges } U)$ 
  using edge-len by simp
  finally show ?thesis .
qed
have source-part:  $c\text{-sources} \leq h * \text{card } S$ 
proof -
  have  $c\text{-sources} \leq h * \text{length}(\text{label-pairs-between d S b beta})$ 
  using source-cost unfolding partition-batch-cost-bound-def by simp
  also have  $\dots \leq h * \text{card } S$ 
  using label-len by simp
  finally show ?thesis .
qed
show ?thesis
  using edge-part source-part c-def by linarith
qed

lemma range-costed-split-batch-cost-le-range-card:
  assumes pre: bmssp-pre-full d S (Fin beta)
  and edge-cost:
    partition-batch-cost-bound c-edges t
    (edge-relaxation-pairs-between d-child U b beta)
  and source-cost:
    partition-batch-cost-bound c-sources h
    (label-pairs-between d S b beta)
  and complete: complete-on d-child U
  and reaches:  $\bigwedge u. u \in U \implies \text{reachable } s u$ 
  and c-def:  $c = c\text{-edges} + c\text{-sources}$ 

```

```

shows  $c \leq t * \text{card} (\text{outgoing-edges-range } U \text{ } b \text{ } (Fin \text{ } beta)) + h * \text{card } S$ 
proof -
  have  $S\text{-subset}: S \subseteq V$ 
  using pre unfolding bmssp-pre-full-def by blast
  have  $\text{finite-}S: \text{finite } S$ 
  using finite-subset[OF  $S\text{-subset}$  finite- $V$ ] .
  have  $\text{edge-len}$ :
    length (edge-relaxation-pairs-between  $d$ -child  $U$   $b$   $beta$ )  $\leq$ 
      card (outgoing-edges-range  $U$   $b$   $(Fin \text{ } beta)$ )
  by (rule edge-relaxation-pairs-between-length-le-outgoing-edges-range
    [OF complete reaches])
  have  $\text{label-len}$ :
    length (label-pairs-between  $d$   $S$   $b$   $beta$ )  $\leq$  card  $S$ 
  by (rule label-pairs-between-length-le-card[OF finite- $S$ ])
  have  $\text{edge-part}$ :
     $c\text{-edges} \leq t * \text{card} (\text{outgoing-edges-range } U \text{ } b \text{ } (Fin \text{ } beta))$ 
  proof -
    have  $c\text{-edges} \leq t * \text{length} (\text{edge-relaxation-pairs-between } d\text{-child } U \text{ } b \text{ } beta)$ 
    using edge-cost unfolding partition-batch-cost-bound-def by simp
    also have  $\dots \leq t * \text{card} (\text{outgoing-edges-range } U \text{ } b \text{ } (Fin \text{ } beta))$ 
    using edge-len by simp
    finally show ?thesis .
  qed
  have  $\text{source-part}: c\text{-sources} \leq h * \text{card } S$ 
  proof -
    have  $c\text{-sources} \leq h * \text{length} (\text{label-pairs-between } d \text{ } S \text{ } b \text{ } beta)$ 
    using source-cost unfolding partition-batch-cost-bound-def by simp
    also have  $\dots \leq h * \text{card } S$ 
    using label-len by simp
    finally show ?thesis .
  qed
  show ?thesis
  using edge-part source-part c-def by linarith
qed

```

The single-batch relation is convenient, but the sharper progress lemmas need to distinguish where a batch entry came from. Edge entries are generated by *edge-relaxation-pairs-between* and are charged to outgoing edges of the just-completed range. Source entries are generated by *label-pairs-between* and are charged to the number of pulled child sources. The split relation below records both costs explicitly.

This is a refinement, not a different algorithm. Its proof to the single-batch relation uses the append rule for *partition-batch-cost-bound* and the elementary fact that a source-batch coefficient bounded by the edge coefficient can be absorbed into the combined batch budget.

inductive *split-range-costed-partition-loop-state*
and *split-range-costed-bmssp* **where**
Split-Range-State-Done:

$keys-of\ D = \{\} \implies$
 $bound-le\ (Fin\ a)\ B \implies$
 $complete-on\ d'\ (bound-tree\ P\ B) \implies$
 $split-range-costed-partition-loop-state\ \Delta\ M-of\ t\ h\ k\ cap\ l\ d\ P\ B\ d'\ D\ a$
 $\quad []\ []\ B\ [range-tree\ P\ a\ B]$
 $\quad (bound-tree\ P\ (Fin\ a) \cup \bigcup (set\ [range-tree\ P\ a\ B]))\ 0\ []$
| *Split-Range-State-Stop:*
 $bound-le\ (Fin\ a)\ B \implies$
 $complete-on\ d'\ (bound-tree\ P\ (Fin\ a)) \implies$
 $k * cap \leq card\ (bound-tree\ P\ (Fin\ a)) \implies$
 $split-range-costed-partition-loop-state\ \Delta\ M-of\ t\ h\ k\ cap\ l\ d\ P\ B\ d'\ D\ a$
 $\quad []\ []\ (Fin\ a)\ [range-tree\ P\ a\ (Fin\ a)]$
 $\quad (bound-tree\ P\ (Fin\ a) \cup \bigcup (set\ [range-tree\ P\ a\ (Fin\ a)]))\ 0\ []$
| *Split-Range-State-Step:*
 $pull-separates\ D\ (M-of\ l)\ Bmax\ S-pull\ beta\ D-pull \implies$
 $bound-le\ (Fin\ beta)\ B \implies$
 $bmssp-pre-full\ d\ S-pull\ (Fin\ beta) \implies$
 $(\forall x \in S-pull. \text{reachable } s\ x) \implies$
 $a \leq b \implies$
 $bound-le\ (Fin\ a)\ B' \implies$
 $complete-on\ d'\ (bound-tree\ P\ (Fin\ a)) \implies$
 $card\ (bound-tree\ P\ (Fin\ a)) < k * cap \implies$
 $split-range-costed-bmssp\ \Delta\ M-of\ t\ h\ k\ cap\ l\ d\ S-pull\ (Fin\ beta)$
 $\quad d-child\ (Fin\ b)\ U-child\ c-child \implies$
 $U-child = range-tree\ P\ a\ (Fin\ b) \implies$
 $complete-preserved\ d-child\ d'\ U-child \implies$
 $edge-batch = edge-relaxation-pairs-between\ d-child\ U-child\ b\ beta \implies$
 $source-batch = label-pairs-between\ d\ S-pull\ b\ beta \implies$
 $batch = edge-batch\ @\ source-batch \implies$
 $D-next = batch-min-update\ D-pull\ batch \implies$
 $partition-pull-cost-bound\ c-pull\ S-pull \implies$
 $partition-batch-cost-bound\ c-edges\ t\ edge-batch \implies$
 $partition-batch-cost-bound\ c-sources\ h\ source-batch \implies$
 $split-range-costed-partition-loop-state\ \Delta\ M-of\ t\ h\ k\ cap\ l\ d\ P\ B\ d'$
 $\quad D-next\ b\ betas\ bs\ B'\ Us-tail\ U-tail\ c-tail\ child-costs-tail \implies$
 $c = c-pull + c-edges + c-sources + c-child + c-tail \implies$
 $split-range-costed-partition-loop-state\ \Delta\ M-of\ t\ h\ k\ cap\ l\ d\ P\ B\ d'\ D\ a$
 $\quad (beta\ \# \ betas)\ (b\ \# \ bs)\ B'$
 $\quad (range-tree\ P\ a\ (Fin\ b)\ \# \ Us-tail)$
 $\quad (bound-tree\ P\ (Fin\ a) \cup$
 $\quad \quad \bigcup (set\ (range-tree\ P\ a\ (Fin\ b)\ \# \ Us-tail)))\ c$
 $\quad (c-child\ \# \ child-costs-tail)$
| *Split-Range-Cost-Base:*
 $S = \{x\} \implies$
 $split-range-costed-bmssp\ \Delta\ M-of\ t\ h\ k\ cap\ 0\ d\ S\ B$
 $(\lambda v. \text{if } v \in \text{base-case-vertices } k\ x\ B \text{ then } dist\ s\ v \text{ else } d\ v)$
 $(base-case-bound\ k\ x\ B)$
 $(base-case-vertices\ k\ x\ B)$
 $(base-case-scan-cost\ \Delta\ k\ x\ B)$

| *Split-Range-Cost-Step*:

$D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \text{ cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \implies$
 $\text{split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \text{ cap } l$
 $(\text{find-pivots-label-capped } k \text{ cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs-loop} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $c = \text{fp-iter-capped-scan-cost } k \text{ cap } d \ S \ S \ B + c\text{-insert} + c\text{-loop} \implies$
 $\text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } (Suc \ l) \ d \ S \ B \ d' \ B' \ U \ c$

inductive-cases *split-range-costed-bmssp-zeroE*:

$\text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } 0 \ d \ S \ B \ d' \ B' \ U \ c$

inductive-cases *split-range-costed-bmssp-SucE*:

$\text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } (Suc \ l) \ d \ S \ B \ d' \ B' \ U \ c$

theorem *split-range-costed-partition-loop-state-refines*:

shows

$\text{split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \text{ cap } l \ d \ P \ B \ d' \ D \ a$
 $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs} \implies$
 $h \leq t \implies$
 $\text{range-costed-partition-loop-state } \Delta \ M\text{-of } t \ k \text{ cap } l \ d \ P \ B \ d' \ D \ a$
 $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs}$

and

$\text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies$
 $h \leq t \implies$
 $\text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \text{ cap } l \ d \ S \ B \ d' \ B' \ U \ c$

proof (*induction rule*:

$\text{split-range-costed-partition-loop-state-split-range-costed-bmssp.inducts}$)

case (*Split-Range-State-Done* $D \ a \ B \ d' \ P \ \Delta \ M\text{-of } t \ h \ k \text{ cap } l \ d$)

show ?case

using *Split-Range-State-Done* **by** (*meson Range-State-Done*)

next

case (*Split-Range-State-Stop* $a \ B \ d' \ P \ k \text{ cap } \Delta \ M\text{-of } t \ h \ l \ d \ D$)

show ?case

using *Split-Range-State-Stop* **by** (*meson Range-State-Stop*)

next

case (*Split-Range-State-Step* $D \ M\text{-of } l \ B\text{max} \ S\text{-pull } \beta \ D\text{-pull } B \ d \ a \ b \ B'$
 $d' \ P \ k \text{ cap } \Delta \ t \ h \ d\text{-child } U\text{-child } c\text{-child } \text{edge-batch } \text{source-batch}$
 $\text{batch } D\text{-next } c\text{-pull } c\text{-edges } c\text{-sources } \text{betas } bs \ Us\text{-tail } U\text{-tail } c\text{-tail}$
 $\text{child-costs-tail } c$)

have $h\text{-le-}t$: $h \leq t$

```

using Split-Range-State-Step.prems by simp
have edge-cost':
  partition-batch-cost-bound c-edges t
    (edge-relaxation-pairs-between d-child U-child b beta)
using Split-Range-State-Step by simp
have source-cost':
  partition-batch-cost-bound c-sources h
    (label-pairs-between d S-pull b beta)
using Split-Range-State-Step by simp
have batch-cost:
  partition-batch-cost-bound
    (c-edges + c-sources) t
    (edge-relaxation-pairs-between d-child U-child b beta @
      label-pairs-between d S-pull b beta)
by (rule partition-batch-cost-bound-append[OF edge-cost' source-cost' h-le-t])
have c-eq:
  c = c-pull + (c-edges + c-sources) + c-child + c-tail
using Split-Range-State-Step by simp
show ?case
by (rule Range-State-Step[
  where batch =
    edge-relaxation-pairs-between d-child U-child b beta @
    label-pairs-between d S-pull b beta
    and c-batch = c-edges + c-sources])
  (use Split-Range-State-Step h-le-t batch-cost c-eq in auto)
next
case (Split-Range-Cost-Base S x  $\Delta$  M-of t h k cap d B)
show ?case
by (rule Range-Cost-Base[OF Split-Range-Cost-Base.hyps])
next
case Split-Range-Cost-Step
show ?case
by (rule Range-Cost-Step)
  (use Split-Range-Cost-Step in auto)
qed

```

The refinement theorem $\llbracket \text{split-range-costed-partition-loop-state } ?\Delta \text{ } ?M\text{-of } ?t \text{ } ?h \text{ } ?k \text{ } ?cap \text{ } ?l \text{ } ?d \text{ } ?P \text{ } ?B \text{ } ?d' \text{ } ?D \text{ } ?a \text{ } ?betas \text{ } ?bs \text{ } ?B' \text{ } ?Us \text{ } ?U \text{ } ?c \text{ } ?child\text{-costs}; ?h \leq ?t \rrbracket \implies \text{range-costed-partition-loop-state } ?\Delta \text{ } ?M\text{-of } ?t \text{ } ?k \text{ } ?cap \text{ } ?l \text{ } ?d \text{ } ?P \text{ } ?B \text{ } ?d' \text{ } ?D \text{ } ?a \text{ } ?betas \text{ } ?bs \text{ } ?B' \text{ } ?Us \text{ } ?U \text{ } ?c \text{ } ?child\text{-costs}$

$\llbracket \text{split-range-costed-bmssp } ?\Delta \text{ } ?M\text{-of } ?t \text{ } ?h \text{ } ?k \text{ } ?cap \text{ } ?l \text{ } ?d \text{ } ?S \text{ } ?B \text{ } ?d' \text{ } ?B' \text{ } ?U \text{ } ?c; ?h \leq ?t \rrbracket \implies \text{range-costed-bmssp } ?\Delta \text{ } ?M\text{-of } ?t \text{ } ?k \text{ } ?cap \text{ } ?l \text{ } ?d \text{ } ?S \text{ } ?B \text{ } ?d' \text{ } ?B' \text{ } ?U \text{ } ?c$ is the formal connection between the two presentations. Once it is proved, any semantic theorem established for the single-batch relation can be reused for the split relation. The remaining split-specific lemmas therefore focus on costs: extracting child sources, bounding edge and source batches separately, and then recombining those bounds into the level recurrence.

lemma *split-range-costed-partition-loop-state-mono*:

assumes *run*:
split-range-costed-partition-loop-state Δ *M-of t h k cap l d P B d' D a*
betas bs B' Us U c child-costs
shows *nondecreasing-from a bs*
using *run* **by** (*induction*) *auto*

theorem *split-range-costed-partition-loop-state-child-call-ranges*:
assumes *run*:
split-range-costed-partition-loop-state Δ *M-of t h k cap l d P B d' D a*
betas bs B' Us U c child-costs
shows *length child-costs = length bs*
and *list-all2*
 $(\lambda c\text{-child } U\text{-child}. \exists S\text{-child } B\text{-child } d\text{-child } B\text{-child'}. \\
\text{split-range-costed-bmssp } \Delta \text{ M-of t h k cap l d S-child B-child} \\
d\text{-child B-child' U-child c-child } \wedge \\
\text{bmssp-pre-full d S-child B-child } \wedge \\
(\forall x \in S\text{-child}. \text{reachable s x}) \wedge \\
\text{card S-child} \leq \text{M-of l}) \\
\text{child-costs (range-tree-child-list P a bs)})$
using *run*
by (*induction*) (*auto simp: pull-separates-def*)

theorem *split-range-costed-partition-loop-state-cost-bound-by-child-sources*:
split-range-costed-partition-loop-state Δ *M-of t h k cap l d P B d' D a*
betas bs B' Us U c child-costs $\implies$
 $\exists \text{child-sources}. \\
\text{length child-sources} = \text{length bs} \wedge \\
\text{list-all2} \\
(\lambda S\text{-child } U\text{-child}. \exists c\text{-child } B\text{-child } d\text{-child } B\text{-child'}. \\
\text{split-range-costed-bmssp } \Delta \text{ M-of t h k cap l d S-child B-child} \\
d\text{-child B-child' U-child c-child } \wedge \\
\text{bmssp-pre-full d S-child B-child } \wedge \\
(\forall x \in S\text{-child}. \text{reachable s x}) \wedge \\
\text{card S-child} \leq \text{M-of l}) \\
\text{child-sources (range-tree-child-list P a bs)} \wedge \\
c \leq \text{sum-list child-costs} + \text{sum-list (map card child-sources)} + \\
t * \text{sum-list} \\
(\text{map } (\lambda X. \text{card (outgoing-edges X)}) (\text{range-tree-child-list P a bs})) + \\
h * \text{sum-list (map card child-sources)})$
and *split-range-costed-bmssp-child-sources-trivial*:
split-range-costed-bmssp Δ *M-of t h k cap l d S B d' B' U c* $\implies$ *True*
proof (*induction rule*:
split-range-costed-partition-loop-state-split-range-costed-bmssp.inducts)
case (*Split-Range-State-Done D a B d' P* Δ *M-of t h k cap l d*)
show *?case*
by (*intro exI[of - []]*) *simp*
next
case (*Split-Range-State-Stop a B d' P k cap* Δ *M-of t h l d D*)
show *?case*

```

    by (intro exI[of - []]) simp
next
case (Split-Range-State-Step D M-of l Bmax S-pull beta D-pull B d a b B'
    d' P k cap Δ t h d-child U-child c-child edge-batch source-batch
    batch D-next c-pull c-edges c-sources betas bs Us-tail U-tail c-tail
    child-costs-tail c)
obtain child-sources-tail where len-tail:
    length child-sources-tail = length bs
and sources-tail:
    list-all2
    (λS-child U-child. ∃ c-child B-child d-child B-child'.
        split-range-costed-bmssp Δ M-of t h k cap l d S-child B-child
        d-child B-child' U-child c-child ∧
        bmssp-pre-full d S-child B-child ∧
        (∀ x ∈ S-child. reachable s x) ∧
        card S-child ≤ M-of l)
    child-sources-tail (range-tree-child-list P b bs)
and tail-cost:
    c-tail ≤
    sum-list child-costs-tail + sum-list (map card child-sources-tail) +
    t * sum-list
    (map (λX. card (outgoing-edges X)) (range-tree-child-list P b bs)) +
    h * sum-list (map card child-sources-tail)
using Split-Range-State-Step.IH by blast
have card-pull: card S-pull ≤ M-of l
using Split-Range-State-Step unfolding pull-separates-def by blast
have pull-cost: c-pull ≤ card S-pull
using Split-Range-State-Step
    unfolding partition-pull-cost-bound-def by blast
have batch-cost:
    c-edges + c-sources ≤
    t * card (outgoing-edges U-child) + h * card S-pull
by (rule range-costed-split-batch-cost-le-card[
    where d = d and S = S-pull and d-child = d-child and b = b
    and beta = beta])
    (use Split-Range-State-Step in auto)
let ?child-sources = S-pull # child-sources-tail
have len: length ?child-sources = length (b # bs)
using len-tail by simp
have head-source:
    ∃ c-child' B-child d-child' B-child'.
        split-range-costed-bmssp Δ M-of t h k cap l d S-pull B-child
        d-child' B-child' U-child c-child' ∧
        bmssp-pre-full d S-pull B-child ∧
        (∀ x ∈ S-pull. reachable s x) ∧
        card S-pull ≤ M-of l
using Split-Range-State-Step card-pull by blast
have sources:
    list-all2

```



```

(λS-child U-child. ∃ c-child B-child d-child B-child'.
  split-range-costed-bmssp Δ M-of t h k cap l d S-child B-child
    d-child B-child' U-child c-child ∧
  bmssp-pre-full d S-child B-child ∧
  (∀ x ∈ S-child. reachable s x) ∧
  card S-child ≤ M-of l)
?child-sources (range-tree-child-list P a (b # bs))
using head-source sources-tail Split-Range-State-Step by simp
have cost:
c ≤ sum-list (c-child # child-costs-tail) +
  sum-list (map card ?child-sources) +
  t * sum-list
    (map (λX. card (outgoing-edges X))
      (range-tree-child-list P a (b # bs))) +
  h * sum-list (map card ?child-sources)
proof -
have c-eq: c = c-pull + c-edges + c-sources + c-child + c-tail
using Split-Range-State-Step by blast
have c-pull + c-edges + c-sources + c-child + c-tail ≤
  card S-pull +
  (t * card (outgoing-edges U-child) + h * card S-pull) +
  c-child +
  (sum-list child-costs-tail + sum-list (map card child-sources-tail) +
    t * sum-list
      (map (λX. card (outgoing-edges X)) (range-tree-child-list P b bs))) +
  h * sum-list (map card child-sources-tail)
using pull-cost batch-cost tail-cost by linarith
also have ... =
  sum-list (c-child # child-costs-tail) +
  sum-list (map card ?child-sources) +
  t * (card (outgoing-edges U-child) +
    sum-list
      (map (λX. card (outgoing-edges X)) (range-tree-child-list P b bs))) +
  h * sum-list (map card ?child-sources)
by (simp add: algebra-simps)
also have ... =
  sum-list (c-child # child-costs-tail) +
  sum-list (map card ?child-sources) +
  t * sum-list
    (map (λX. card (outgoing-edges X))
      (range-tree-child-list P a (b # bs))) +
  h * sum-list (map card ?child-sources)
using Split-Range-State-Step by simp
finally show ?thesis
using c-eq by simp
qed
show ?case
by (intro exI[of - ?child-sources] conjI)
  (use len sources cost in simp-all)

```

```

next
  case (Split-Range-Cost-Base  $S\ x\ \Delta\ M\text{-of}\ t\ h\ k\ \text{cap}\ d\ B$ )
  show ?case
    by simp
next
  case (Split-Range-Cost-Step  $D\ k\ \text{cap}\ d\ S\ B\ c\text{-insert}\ t\ \Delta\ M\text{-of}\ h\ l\ d'\ a$ 
     $\text{betas}\ bs\ B'\ Us\ U\text{-loop}\ c\text{-loop}\ \text{child-costs-loop}\ U\ c$ )
  show ?case
    by simp
qed

theorem split-range-costed-partition-loop-state-cost-from-child-and-source-bounds:
  assumes run:
    split-range-costed-partition-loop-state  $\Delta\ M\text{-of}\ t\ h\ k\ \text{cap}\ l\ d\ P\ B\ d'\ D\ a$ 
     $\text{betas}\ bs\ B'\ Us\ U\ c\ \text{child-costs}$ 
  and child-cost-bounds:
    list-all2 ( $\lambda c\text{-child}\ U\text{-child}.$ 
       $c\text{-child} \leq \text{level-range-cost-bound}\ A\ R\ L\ U\text{-child}$ 
       $\text{child-costs}\ (\text{range-tree-child-list}\ P\ a\ bs)$ )
  and source-progress:
     $\bigwedge S\text{-child}\ B\text{-child}\ d\text{-child}\ B\text{-child}'\ U\text{-child}\ c\text{-child}.$ 
       $\llbracket \text{split-range-costed-bmssp}\ \Delta\ M\text{-of}\ t\ h\ k\ \text{cap}\ l\ d\ S\text{-child}\ B\text{-child}$ 
         $d\text{-child}\ B\text{-child}'\ U\text{-child}\ c\text{-child};$ 
         $\text{bmssp-pre-full}\ d\ S\text{-child}\ B\text{-child};$ 
         $\bigwedge x. x \in S\text{-child} \implies \text{reachable}\ s\ x;$ 
         $\text{card}\ S\text{-child} \leq M\text{-of}\ l \rrbracket$ 
       $\implies \text{card}\ S\text{-child} \leq \text{card}\ U\text{-child}$ 
  shows  $c \leq$ 
     $A * L * \text{card}\ (\text{range-tree-chain}\ P\ a\ bs\ B') +$ 
     $(R + t) * \text{card}\ (\text{outgoing-edges}\ (\text{range-tree-chain}\ P\ a\ bs\ B')) +$ 
     $\text{Suc}\ h * \text{card}\ (\text{range-tree-chain}\ P\ a\ bs\ B')$ 
proof –
  obtain child-sources where len-sources:
     $\text{length}\ \text{child-sources} = \text{length}\ bs$ 
  and sources:
    list-all2
      ( $\lambda S\text{-child}\ U\text{-child}.$   $\exists c\text{-child}\ B\text{-child}\ d\text{-child}\ B\text{-child}'.$ 
        split-range-costed-bmssp  $\Delta\ M\text{-of}\ t\ h\ k\ \text{cap}\ l\ d\ S\text{-child}\ B\text{-child}$ 
           $d\text{-child}\ B\text{-child}'\ U\text{-child}\ c\text{-child} \wedge$ 
           $\text{bmssp-pre-full}\ d\ S\text{-child}\ B\text{-child} \wedge$ 
           $(\forall x \in S\text{-child}. \text{reachable}\ s\ x) \wedge$ 
           $\text{card}\ S\text{-child} \leq M\text{-of}\ l$ )
      child-sources  $(\text{range-tree-child-list}\ P\ a\ bs)$ 
  and cost:
     $c \leq \text{sum-list}\ \text{child-costs} + \text{sum-list}\ (\text{map}\ \text{card}\ \text{child-sources}) +$ 
     $t * \text{sum-list}$ 
       $(\text{map}\ (\lambda X. \text{card}\ (\text{outgoing-edges}\ X))\ (\text{range-tree-child-list}\ P\ a\ bs)) +$ 
     $h * \text{sum-list}\ (\text{map}\ \text{card}\ \text{child-sources})$ 
using split-range-costed-partition-loop-state-cost-bound-by-child-sources[OF run]

```

```

  by blast
have mono: nondecreasing-from a bs
  by (rule split-range-costed-partition-loop-state-mono[OF run])
have source-dom:
  list-all2 ( $\lambda S\text{-child } U\text{-child}. \text{card } S\text{-child} \leq \text{card } U\text{-child}$ )
  child-sources (range-tree-child-list P a bs)
using sources
proof (induction rule: list-all2-induct)
  case Nil
  then show ?case by simp
next
  case (Cons S-child child-sources U-child child-sets)
  obtain c-child B-child d-child B-child' where child:
    split-range-costed-bmssp  $\Delta$  M-of t h k cap l d S-child B-child
    d-child B-child' U-child c-child
  and child-pre: bmssp-pre-full d S-child B-child
  and child-reaches:  $\forall x \in S\text{-child}. \text{reachable } s \ x$ 
  and child-card:  $\text{card } S\text{-child} \leq M\text{-of } l$ 
  using Cons.hyps by blast
  have head:  $\text{card } S\text{-child} \leq \text{card } U\text{-child}$ 
  by (rule source-progress[OF child child-pre - child-card])
  (use child-reaches in blast)
  show ?case
  using head Cons.IH Cons.hyps by simp
qed
have source-sum:
  sum-list (map card child-sources)  $\leq$ 
  card (range-tree-chain P a bs B')
  by (rule sum-card-dominated-by-range-tree-child-list-le-chain
    [OF mono source-dom])
have child-sum:
  sum-list child-costs  $\leq$ 
  A * L * card (range-tree-chain P a bs B') +
  R * card (outgoing-edges (range-tree-chain P a bs B'))
  by (rule child-costs-le-level-range-child-list-bound
    [OF mono child-cost-bounds])
have edge-sum:
  sum-list
    (map ( $\lambda X. \text{card } (\text{outgoing-edges } X)$ ) (range-tree-child-list P a bs))  $\leq$ 
  card (outgoing-edges (range-tree-chain P a bs B'))
  by (rule card-outgoing-edges-range-tree-child-list-le-chain[OF mono])
have c  $\leq$ 
  A * L * card (range-tree-chain P a bs B') +
  R * card (outgoing-edges (range-tree-chain P a bs B')) +
  sum-list (map card child-sources) +
  t * card (outgoing-edges (range-tree-chain P a bs B')) +
  h * sum-list (map card child-sources)
proof -
  have edge-part:

```

```

    t * sum-list
      (map (λX. card (outgoing-edges X)) (range-tree-child-list P a bs)) ≤
    t * card (outgoing-edges (range-tree-chain P a bs B'))
  using edge-sum by simp
  have sum-list child-costs + sum-list (map card child-sources) +
    t * sum-list
      (map (λX. card (outgoing-edges X)) (range-tree-child-list P a bs)) +
    h * sum-list (map card child-sources) ≤
    A * L * card (range-tree-chain P a bs B') +
    R * card (outgoing-edges (range-tree-chain P a bs B')) +
    sum-list (map card child-sources) +
    t * card (outgoing-edges (range-tree-chain P a bs B')) +
    h * sum-list (map card child-sources)
  using child-sum edge-part by linarith
  then show ?thesis
    using cost by linarith
qed
also have ... ≤
  A * L * card (range-tree-chain P a bs B') +
  (R + t) * card (outgoing-edges (range-tree-chain P a bs B')) +
  Suc h * card (range-tree-chain P a bs B')
proof -
  have source-part:
    sum-list (map card child-sources) +
    h * sum-list (map card child-sources) ≤
    Suc h * card (range-tree-chain P a bs B')
  proof -
    have h-part:
      h * sum-list (map card child-sources) ≤
      h * card (range-tree-chain P a bs B')
    using source-sum by simp
    have sum-list (map card child-sources) +
      h * sum-list (map card child-sources) ≤
      card (range-tree-chain P a bs B') +
      h * card (range-tree-chain P a bs B')
    using source-sum h-part by linarith
    then show ?thesis
      by (simp add: algebra-simps)
  qed
show ?thesis
  using source-part by (simp add: algebra-simps)
qed
finally show ?thesis .
qed

```

theorem *split-range-costed-partition-loop-state-child-cost-bounds:*
assumes *run:*
split-range-costed-partition-loop-state Δ M-of t h k cap l d P B d' D a
betas bs B' Us U c child-costs

and *child-bound*:
 $\wedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\llbracket \text{split-range-costed-bmssp } \Delta \text{ M-of } t \text{ h k cap } l \text{ d } S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \text{ U-child } c\text{-child};$
 $\text{bmssp-pre-full } d \text{ S-child } B\text{-child};$
 $\wedge x. x \in S\text{-child} \implies \text{reachable } s \text{ } x;$
 $\text{card } S\text{-child} \leq \text{M-of } l \rrbracket$
 $\implies c\text{-child} \leq \text{level-range-cost-bound } A \text{ R L } U\text{-child}$
shows *list-all2* ($\lambda c\text{-child } U\text{-child}.$
 $c\text{-child} \leq \text{level-range-cost-bound } A \text{ R L } U\text{-child}$)
 $\text{child-costs } (\text{range-tree-child-list } P \text{ a bs})$
proof –
have *calls*:
 list-all2
 $(\lambda c\text{-child } U\text{-child}. \exists S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\text{split-range-costed-bmssp } \Delta \text{ M-of } t \text{ h k cap } l \text{ d } S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \text{ U-child } c\text{-child} \wedge$
 $\text{bmssp-pre-full } d \text{ S-child } B\text{-child} \wedge$
 $(\forall x \in S\text{-child}. \text{reachable } s \text{ } x) \wedge$
 $\text{card } S\text{-child} \leq \text{M-of } l)$
 $\text{child-costs } (\text{range-tree-child-list } P \text{ a bs})$
by (*rule split-range-costed-partition-loop-state-child-call-ranges(2)*[*OF run*])
show *?thesis*
using *calls*
proof (*induction rule: list-all2-induct*)
case *Nil*
then show *?case* **by** *simp*
next
case (*Cons c-child child-costs U-child child-sets*)
obtain *S-child B-child d-child B-child'* **where** *child*:
 $\text{split-range-costed-bmssp } \Delta \text{ M-of } t \text{ h k cap } l \text{ d } S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \text{ U-child } c\text{-child}$
and *child-pre*: $\text{bmssp-pre-full } d \text{ S-child } B\text{-child}$
and *child-reaches*: $\forall x \in S\text{-child}. \text{reachable } s \text{ } x$
and *child-card*: $\text{card } S\text{-child} \leq \text{M-of } l$
using *Cons.hyps* **by** *blast*
have *head*:
 $c\text{-child} \leq \text{level-range-cost-bound } A \text{ R L } U\text{-child}$
by (*rule child-bound*[*OF child child-pre - child-card*])
 $(\text{use } \text{child-reaches} \text{ in } \text{blast})$
show *?case*
using *head Cons.IH Cons.hyps* **by** *simp*
qed
qed

theorem *split-range-costed-partition-loop-state-cost-from-child-bound*:
assumes *run*:
 $\text{split-range-costed-partition-loop-state } \Delta \text{ M-of } t \text{ h k cap } l \text{ d } P \text{ B } d' \text{ D a}$
 $\text{betas bs } B' \text{ Us } U \text{ c child-costs}$

and *child-bound*:
 $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\llbracket \text{split-range-costed-bmssp } \Delta \text{ M-of } t \text{ h k cap l d } S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \text{ U-child } c\text{-child};$
 $\text{bmssp-pre-full } d \text{ S-child } B\text{-child};$
 $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \text{ } x;$
 $\text{card } S\text{-child} \leq \text{M-of } l \rrbracket$
 $\implies c\text{-child} \leq \text{level-range-cost-bound } A \text{ R L } U\text{-child}$
and *source-progress*:
 $\bigwedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}' \text{ U-child } c\text{-child}.$
 $\llbracket \text{split-range-costed-bmssp } \Delta \text{ M-of } t \text{ h k cap l d } S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \text{ U-child } c\text{-child};$
 $\text{bmssp-pre-full } d \text{ S-child } B\text{-child};$
 $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \text{ } x;$
 $\text{card } S\text{-child} \leq \text{M-of } l \rrbracket$
 $\implies \text{card } S\text{-child} \leq \text{card } U\text{-child}$
shows $c \leq$
 $A * L * \text{card } (\text{range-tree-chain } P \text{ a bs } B') +$
 $(R + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \text{ a bs } B')) +$
 $\text{Suc } h * \text{card } (\text{range-tree-chain } P \text{ a bs } B')$
proof –
have *child-cost-bounds*:
 $\text{list-all2 } (\lambda c\text{-child } U\text{-child}.$
 $\quad c\text{-child} \leq \text{level-range-cost-bound } A \text{ R L } U\text{-child})$
 $\text{child-costs } (\text{range-tree-child-list } P \text{ a bs})$
by (rule *split-range-costed-partition-loop-state-child-cost-bounds*
 $[\text{OF run child-bound}])$
show *?thesis*
by (rule *split-range-costed-partition-loop-state-cost-from-child-and-source-bounds*
 $[\text{OF run child-cost-bounds source-progress}])$
qed

theorem *split-range-costed-partition-loop-state-trace*:
 $\text{split-range-costed-partition-loop-state } \Delta \text{ M-of } t \text{ h k cap l d } P \text{ B } d' \text{ D a}$
 $\quad \text{betas bs } B' \text{ Us } U \text{ c child-costs} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \text{ P B} \implies$
 $(\bigwedge x. x \in P \implies \text{reachable } s \text{ } x) \implies$
 $\text{concrete-partition-loop-trace } P \text{ B a bs } d' \text{ B' Us } U$
and *split-range-costed-bmssp-correct*:
 $\text{split-range-costed-bmssp } \Delta \text{ M-of } t \text{ h k cap l d } S \text{ B } d' \text{ B' U c} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \text{ S B} \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \text{ } x) \implies$
 $\text{bmssp-post-full } d \text{ S B } d' \text{ B' U}$
proof (*induction rule*):
 $\text{split-range-costed-partition-loop-state-split-range-costed-bmssp.inducts})$
case (*Split-Range-State-Done* $D \text{ a B } d' \text{ P } \Delta \text{ M-of } t \text{ h k cap l d}$)
have *a-le*: $\text{bound-le } (F \text{ in } a) \text{ B}$

```

    using Split-Range-State-Done by blast
  have complete-B: complete-on  $d'$  (bound-tree  $P$   $B$ )
    using Split-Range-State-Done by blast
  have lower: complete-on  $d'$  (bound-tree  $P$  (Fin  $a$ ))
    by (rule complete-on-subset[OF complete-B])
      (rule bound-tree-bound-mono[OF a-le])
  have range-complete: complete-on  $d'$  (range-tree  $P$   $a$   $B$ )
    by (rule complete-on-subset[OF complete-B])
      (rule range-tree-subset-bound-tree)
  have B-le: bound-le  $B$   $B$ 
    by (cases  $B$ ) simp-all
  show ?case
    unfolding concrete-partition-loop-trace-def
    using B-le a-le lower range-complete by simp
next
  case (Split-Range-State-Stop  $a$   $B$   $d'$   $P$   $k$  cap  $\Delta$  M-of  $t$   $h$   $l$   $d$   $D$ )
  then show ?case
    unfolding concrete-partition-loop-trace-def complete-on-def by simp
next
  case Split-Range-State-Step
  then show ?case
    unfolding concrete-partition-loop-trace-def bmssp-post-full-def
      complete-preserved-def
    by (auto split: bound.splits)
next
  case (Split-Range-Cost-Base  $S$   $x$   $\Delta$  M-of  $t$   $h$   $k$  cap  $d$   $B$ )
  have post:
    bmssp-post  $d$   $S$   $B$ 
    ( $\lambda v$ . if  $v \in$  base-case-vertices  $k$   $x$   $B$  then dist  $s$   $v$  else  $d$   $v$ )
    (base-case-bound  $k$   $x$   $B$ )
    (base-case-vertices  $k$   $x$   $B$ )
  using base-case-result-bmssp-post[OF Split-Range-Cost-Base.hyps,
    where  $k = k$  and  $B = B$  and  $d = d$ ]
  unfolding base-case-result-def by simp
  then show ?case
    by (rule bmssp-post-imp-post-full)
next
  case (Split-Range-Cost-Step  $D$   $k$  cap  $d$   $S$   $B$  c-insert  $t$   $\Delta$  M-of  $h$   $l$   $d'$   $a$ 
    betas  $bs$   $B'$   $Us$  U-loop c-loop child-costs-loop  $U$   $c$ )
  let ?d-fp = find-pivots-label-capped  $k$  cap  $d$   $S$   $B$ 
  let ? $P$  = find-pivots-pivots-capped  $k$  cap  $d$   $S$   $B$ 
  let ? $W$  =  $\{v \in$  bound-tree  $S$   $B'$ . ?d-fp  $v =$  dist  $s$   $v\}$ 
  have sound-fp: sound-label ?d-fp
    unfolding find-pivots-label-capped-def
    by (rule fp-iter-capped-label-sound[OF Split-Range-Cost-Step.prems(1)
      Split-Range-Cost-Step.prems(3)])
  have pivot-pre: bmssp-pre-full ?d-fp ? $P$   $B$ 
    using find-pivots-capped-establishes-pivot-pre-concrete
      [OF Split-Range-Cost-Step.prems(1) Split-Range-Cost-Step.prems(2)]

```

```

    Split-Range-Cost-Step.prems(3)] .
  have P-subset-S: ?P  $\subseteq$  S
    unfolding find-pivots-pivots-capped-def by auto
  have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
    using P-subset-S Split-Range-Cost-Step.prems(3) by blast
  have trace:
    concrete-partition-loop-trace ?P B a bs d' B' Us U-loop
    using Split-Range-Cost-Step.IH sound-fp pivot-pre P-reaches by blast
  have loop-post: bmssp-post-full ?d-fp ?P B d' B' U-loop
    by (rule concrete-partition-loop-trace-post[OF trace])
  have le: bound-le B' B
    using loop-post unfolding bmssp-post-full-def by blast
  have U-tree: U = bound-tree S B'
  proof -
    have U-loop  $\cup$  ?W = bound-tree S B'
      by (rule concrete-capped-find-pivots-final-tree-assembly
        [OF Split-Range-Cost-Step.prems(1) Split-Range-Cost-Step.prems(2)
          Split-Range-Cost-Step.prems(3) loop-post])
    then show ?thesis
      using Split-Range-Cost-Step(6) by simp
  qed
  have loop-complete: complete-on d' U-loop
    using loop-post unfolding bmssp-post-full-def by blast
  have U-complete: complete-on d' U
    using complete-on-Un[OF loop-complete Split-Range-Cost-Step(5)]
      Split-Range-Cost-Step(6) by simp
  show ?case
    using le U-tree U-complete unfolding bmssp-post-full-def by blast
  qed

```

The next block closes the level recurrence for the split relation. The input hypotheses have the same shape as the informal induction in the paper: each child call already satisfies a level- L bound on its own output slice, and every pulled child source set makes enough progress to be charged to that slice. The trace theorem supplies the geometric fact needed to sum these charges: the range slices form a chain inside the parent output.

Once the slice sums are bounded by the parent output and its outgoing edges, the arithmetic is straightforward. The child level factor contributes $A * L$, the source-progress term contributes one more A , and the edge batch cost contributes the extra t in the range coefficient. This is why the target has level $Suc\ L$ and coefficient $R + t$.

theorem *split-range-costed-partition-loop-state-closes-level-bound-from-child-cost-bounds-general*:

assumes *run*:

split-range-costed-partition-loop-state Δ *M-of* *t* *h* *k* *cap* *l* *d* *P* *B* *d'* *D* *a*
betas *bs* *B'* *Us* *U* *c* *child-costs*

and *sound*: *sound-label* *d*

and *pre*: *bmssp-pre-full* *d* *P* *B*

and P -reaches: $\bigwedge x. x \in P \implies \text{reachable } s \ x$
and child-cost-bounds :
 $\text{list-all2 } (\lambda c\text{-child } U\text{-child}.$
 $\quad c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child})$
 $\quad \text{child-costs } (\text{range-tree-child-list } P \ a \ bs)$
and source-progress :
 $\bigwedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}.$
 $\quad \llbracket \text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $\quad \quad d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$
 $\quad \text{bmssp-pre-full } d \ S\text{-child } B\text{-child};$
 $\quad \bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\quad \text{card } S\text{-child} \leq M\text{-of } l \rrbracket$
 $\implies \text{card } S\text{-child} \leq \text{card } U\text{-child}$
and source-factor : $\text{Suc } h \leq A$
shows $c \leq A * \text{Suc } L * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U)$
proof –
have trace : $\text{concrete-partition-loop-trace } P \ B \ a \ bs \ d' \ B' \ Us \ U$
by ($\text{rule } \text{split-range-costed-partition-loop-state-trace}$
 $\quad [OF \ \text{run sound pre } P\text{-reaches}])$
have children :
 $\text{list-all2 } (\lambda U \ X. \ U = X \wedge \text{complete-on } d' \ U) \ Us$
 $\quad (\text{range-tree-chain-list } P \ a \ bs \ B')$
using $\text{trace unfolding concrete-partition-loop-trace-def}$ **by** blast
have $U\text{-def}$: $U = \text{bound-tree } P \ (\text{Fin } a) \cup \bigcup (\text{set } Us)$
using $\text{trace unfolding concrete-partition-loop-trace-def}$ **by** blast
have $Union\text{-eq}$:
 $\bigcup (\text{set } Us) = \bigcup (\text{set } (\text{range-tree-chain-list } P \ a \ bs \ B'))$
using $\text{children by (induction rule: list-all2-induct) auto}$
have $U\text{-eq-chain}$:
 $U = \text{bound-tree } P \ (\text{Fin } a) \cup \text{range-tree-chain } P \ a \ bs \ B'$
using $U\text{-def } Union\text{-eq } Union\text{-range-tree-chain-list[of } P \ a \ bs \ B']$ **by** simp
have $\text{finite-}U$: $\text{finite } U$
using $U\text{-eq-chain by simp}$
have chain-subset : $\text{range-tree-chain } P \ a \ bs \ B' \subseteq U$
using $U\text{-eq-chain by blast}$
have card-chain-le : $\text{card } (\text{range-tree-chain } P \ a \ bs \ B') \leq \text{card } U$
by ($\text{rule } \text{card-mono}[OF \ \text{finite-}U \ \text{chain-subset}])$
have outgoing-subset :
 $\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B') \subseteq \text{outgoing-edges } U$
by ($\text{rule } \text{outgoing-edges-mono}[OF \ \text{chain-subset}])$
have $\text{finite-out-}U$: $\text{finite } (\text{outgoing-edges } U)$
by simp
have card-out-le :
 $\text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B')) \leq$
 $\quad \text{card } (\text{outgoing-edges } U)$
by ($\text{rule } \text{card-mono}[OF \ \text{finite-out-}U \ \text{outgoing-subset}])$
have loop :
 $c \leq$
 $A * L * \text{card } (\text{range-tree-chain } P \ a \ bs \ B') +$

```

      (R + t) * card (outgoing-edges (range-tree-chain P a bs B')) +
      Suc h * card (range-tree-chain P a bs B')
    by (rule split-range-costed-partition-loop-state-cost-from-child-and-source-bounds
        [OF run child-cost-bounds source-progress])
  have source-term:
    Suc h * card (range-tree-chain P a bs B') ≤ A * card U
  proof -
    have Suc h * card (range-tree-chain P a bs B') ≤
      A * card (range-tree-chain P a bs B')
      using source-factor by (rule mult-right-mono) simp
    also have ... ≤ A * card U
      using card-chain-le by simp
    finally show ?thesis .
  qed
  have c ≤
    A * L * card U + (R + t) * card (outgoing-edges U) + A * card U
  proof -
    have vterm:
      A * L * card (range-tree-chain P a bs B') ≤ A * L * card U
      using card-chain-le by simp
    have eterm:
      (R + t) * card (outgoing-edges (range-tree-chain P a bs B')) ≤
      (R + t) * card (outgoing-edges U)
      using card-out-le by simp
    have A * L * card (range-tree-chain P a bs B') +
      (R + t) * card (outgoing-edges (range-tree-chain P a bs B')) +
      Suc h * card (range-tree-chain P a bs B') ≤
      A * L * card U + (R + t) * card (outgoing-edges U) + A * card U
      using vterm eterm source-term by linarith
    then show ?thesis
      using loop by linarith
  qed
  also have ... =
    A * Suc L * card U + (R + t) * card (outgoing-edges U)
    by (simp add: algebra-simps)
  finally show ?thesis .
  qed

```

theorem *split-range-costed-partition-loop-state-closes-level-bound-from-source-progress-general:*

assumes *run:*

split-range-costed-partition-loop-state Δ *M-of t h k cap l d P B d' D a*
betas bs B' Us U c child-costs

and *sound:* *sound-label d*

and *pre:* *bmssp-pre-full d P B*

and *P-reaches:* $\bigwedge x. x \in P \implies \text{reachable } s \ x$

and *child-bound:*

$\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$

$\llbracket \text{split-range-costed-bmssp } \Delta \text{ M-of t h k cap l d S-child B-child}$

$d\text{-child } B\text{-child}' U\text{-child } c\text{-child};$
 $bmssp\text{-pre-full } d \text{ } S\text{-child } B\text{-child};$
 $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\text{card } S\text{-child} \leq M\text{-of } l$
 $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$
and *source-progress*:
 $\bigwedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}' U\text{-child } c\text{-child}.$
 $\llbracket \text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' U\text{-child } c\text{-child};$
 $bmssp\text{-pre-full } d \text{ } S\text{-child } B\text{-child};$
 $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\text{card } S\text{-child} \leq M\text{-of } l$
 $\implies \text{card } S\text{-child} \leq \text{card } U\text{-child}$
and *source-factor*: $\text{Suc } h \leq A$
shows $c \leq A * \text{Suc } L * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U)$
proof –
have *child-cost-bounds*:
 $\text{list-all2 } (\lambda c\text{-child } U\text{-child}.$
 $c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child})$
 $\text{child-costs } (\text{range-tree-child-list } P \ a \ bs)$
by (*rule split-range-costed-partition-loop-state-child-cost-bounds*
 $[OF \ \text{run } \text{child-bound}]$)
show *?thesis*
by (*rule*
 $\text{split-range-costed-partition-loop-state-closes-level-bound-from-child-cost-bounds-}$
 general
 $[OF \ \text{run } \text{sound } \text{pre } P\text{-reaches } \text{child-cost-bounds } \text{source-progress}$
 $\text{source-factor}]$)
qed

theorem *split-range-costed-nonbase-step-closes-level-bound-from-source-progress-general*:

assumes *loop*:
 $\text{split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 $\text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs}$
and *sound*: $\text{sound-label } d$
and *pre*: $bmssp\text{-pre-full } d \ P \ B$
and *P-reaches*: $\bigwedge x. x \in P \implies \text{reachable } s \ x$
and *child-bound*:
 $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\llbracket \text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' U\text{-child } c\text{-child};$
 $bmssp\text{-pre-full } d \text{ } S\text{-child } B\text{-child};$
 $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\text{card } S\text{-child} \leq M\text{-of } l$
 $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$
and *source-progress*:
 $\bigwedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}' U\text{-child } c\text{-child}.$
 $\llbracket \text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$

```

    d-child B-child' U-child c-child;
    bmssp-pre-full d S-child B-child;
     $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$ 
    card S-child  $\leq M\text{-of } l$ 
 $\implies \text{card } S\text{-child} \leq \text{card } U\text{-child}$ 
  and source-factor:  $\text{Suc } h \leq A$ 
  and U-def:  $U = U\text{-loop} \cup W$ 
  and finite-U: finite U
  and scan-insert:  $c\text{-scan-insert} \leq A * \text{card } U$ 
  and c-def:  $c = c\text{-scan-insert} + c\text{-loop}$ 
  shows  $c \leq A * \text{Suc } (L) * \text{card } U +$ 
     $(R + t) * \text{card } (\text{outgoing-edges } U)$ 
proof -
  have loop-bound:
     $c\text{-loop} \leq A * \text{Suc } L * \text{card } U\text{-loop} +$ 
     $(R + t) * \text{card } (\text{outgoing-edges } U\text{-loop})$ 
  by (rule split-range-costed-partition-loop-state-closes-level-bound-from-source-
progress-general
    [OF loop sound pre P-reaches child-bound source-progress source-factor])
  have U-loop-subset:  $U\text{-loop} \subseteq U$ 
  using U-def by blast
  have card-loop-le:  $\text{card } U\text{-loop} \leq \text{card } U$ 
  by (rule card-mono[OF finite-U U-loop-subset])
  have outgoing-subset:  $\text{outgoing-edges } U\text{-loop} \subseteq \text{outgoing-edges } U$ 
  by (rule outgoing-edges-mono[OF U-loop-subset])
  have finite-out-U: finite (outgoing-edges U)
  by simp
  have card-out-le:  $\text{card } (\text{outgoing-edges } U\text{-loop}) \leq \text{card } (\text{outgoing-edges } U)$ 
  by (rule card-mono[OF finite-out-U outgoing-subset])
  have  $c \leq$ 
     $A * \text{card } U +$ 
     $(A * \text{Suc } L * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U))$ 
proof -
  have loop-to-U:
     $c\text{-loop} \leq A * \text{Suc } L * \text{card } U +$ 
     $(R + t) * \text{card } (\text{outgoing-edges } U)$ 
  proof -
    have  $A * \text{Suc } L * \text{card } U\text{-loop} \leq A * \text{Suc } L * \text{card } U$ 
    using card-loop-le by simp
    moreover have  $(R + t) * \text{card } (\text{outgoing-edges } U\text{-loop}) \leq$ 
     $(R + t) * \text{card } (\text{outgoing-edges } U)$ 
    using card-out-le by simp
    ultimately show ?thesis
    using loop-bound by linarith
  qed
  show ?thesis
  using scan-insert loop-to-U c-def by linarith
qed
also have ... =

```

$A * \text{Suc} (\text{Suc } L) * \text{card } U + (R + t) * \text{card} (\text{outgoing-edges } U)$
 by (simp add: algebra-simps)
 finally show ?thesis .
 qed

lemma *split-range-costed-base-level-bound:*

assumes $R\text{-pos}: 0 < R$
 shows $\text{base-case-scan-cost } \Delta \ k \ x \ B \leq$
 $\text{level-range-cost-bound } A \ R \ (\text{Suc } 0) \ (\text{base-case-vertices } k \ x \ B)$
proof –
 have $\text{base-case-scan-cost } \Delta \ k \ x \ B \leq$
 $R * \text{card} (\text{outgoing-edges } (\text{base-case-vertices } k \ x \ B))$
 using $R\text{-pos}$ unfolding *base-case-scan-cost-def* by simp
 then show ?thesis
 unfolding *level-range-cost-bound-def* by linarith
 qed

theorem *split-range-costed-bmssp-level-bound-from-source-progress-and-local-budgets:*

assumes $\text{base-budget}: \Delta \leq A$
 and $R\text{-pos}: 0 < R$
 and $\text{source-factor}: \text{Suc } h \leq A$
 and source-progress :
 $\bigwedge l \ d \ S \ B \ d' \ B' \ U \ c.$
 $\text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq M\text{-of } l \implies$
 $\text{card } S \leq \text{card } U$
 and step-budget :
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
 $\text{child-costs } U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{fp-iter-capped-scan-cost } k \ \text{cap } d \ S \ S \ B + c\text{-insert} \leq A * \text{card } U$

```

and run:
  split-range-costed-bmssp  $\Delta$  M-of t h k cap l d S B d' B' U c
and sound: sound-label d
and pre: bmssp-pre-full d S B
and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
shows  $c \leq \text{level-range-cost-bound } A \ (R + l * t) \ (2 * l + 1) \ U$ 
using run sound pre S-reaches
proof (induction l arbitrary: d S B d' B' U c rule: less-induct)
  case (less l)
  show ?case
  proof (cases l)
    case 0
    have run0:
      split-range-costed-bmssp  $\Delta$  M-of t h k cap 0 d S B d' B' U c
      using less.premis(1) 0 by simp
    show ?thesis
    using run0 R-pos 0
    by (cases rule: split-range-costed-bmssp.cases)
    (simp-all add: split-range-costed-base-level-bound)
  next
  case (Suc l0)
  have run-suc:
    split-range-costed-bmssp  $\Delta$  M-of t h k cap (Suc l0) d S B d' B' U c
    using less.premis(1) Suc by simp
  show ?thesis
  proof (cases rule: split-range-costed-bmssp-SucE[OF run-suc])
    case (1 c-insert a betas bs Us U-loop c-loop child-costs-loop)
    let ?d-fp = find-pivots-label-capped k cap d S B
    let ?P = find-pivots-pivots-capped k cap d S B
    let ?W =  $\{v \in \text{bound-tree } S \ B'. \ ?d\text{-fp } v = \text{dist } s \ v\}$ 
    have sound-fp: sound-label ?d-fp
    unfolding find-pivots-label-capped-def
    by (rule fp-iter-capped-label-sound[OF less.premis(2) less.premis(4)])
    have pivot-pre: bmssp-pre-full ?d-fp ?P B
    using find-pivots-capped-establishes-pivot-pre-concrete
    [OF less.premis(2) less.premis(3) less.premis(4)] .
    have P-subset-S:  $?P \subseteq S$ 
    unfolding find-pivots-pivots-capped-def by auto
    have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
    using P-subset-S less.premis(4) by blast
    have child-bound:
       $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
       $\llbracket \text{split-range-costed-bmssp } \Delta \text{ M-of t h k cap } l0 \ ?d\text{-fp } S\text{-child } B\text{-child}$ 
       $\text{d-child } B\text{-child}' \ U\text{-child } c\text{-child};$ 
       $\text{bmssp-pre-full } ?d\text{-fp } S\text{-child } B\text{-child};$ 
       $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$ 
       $\text{card } S\text{-child} \leq \text{M-of } l0 \rrbracket$ 
       $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ (R + l0 * t)$ 
       $(2 * l0 + 1) \ U\text{-child}$ 

```

```

proof –
  fix c-child U-child S-child B-child d-child B-child'
  assume child:
    split-range-costed-bmssp  $\Delta$  M-of t h k cap l0 ?d-fp S-child B-child
    d-child B-child' U-child c-child
    and child-pre: bmssp-pre-full ?d-fp S-child B-child
    and child-reaches:  $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x$ 
  show c-child  $\leq$  level-range-cost-bound A (R + l0 * t)
    (2 * l0 + 1) U-child
    by (rule less.IH[of l0 ?d-fp S-child B-child d-child B-child'
      U-child c-child, OF - child sound-fp child-pre child-reaches])
      (simp add: Suc)
qed
have source-progress-loop:
   $\bigwedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}.$ 
   $\llbracket \text{split-range-costed-bmssp } \Delta \text{ M-of t h k cap l0 ?d-fp S-child B-child}$ 
    d-child B-child' U-child c-child;
    bmssp-pre-full ?d-fp S-child B-child;
     $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x$ ;
    card S-child  $\leq$  M-of l0  $\rrbracket$ 
     $\implies \text{card } S\text{-child} \leq \text{card } U\text{-child}$ 
  by (rule source-progress)
have scan-insert:
  fp-iter-capped-scan-cost k cap d S S B + c-insert  $\leq$ 
    A * card U
  by (rule step-budget[OF refl 1(3) 1(4) 1(5) 1(1)
    less.premis(2) less.premis(3) less.premis(4)])
have trace:
  concrete-partition-loop-trace ?P B a bs d' B' Us U-loop
  by (rule split-range-costed-partition-loop-state-trace
    [OF 1(4) sound-fp pivot-pre P-reaches])
have loop-post: bmssp-post-full ?d-fp ?P B d' B' U-loop
  by (rule concrete-partition-loop-trace-post[OF trace])
have finite-U-loop: finite U-loop
  using loop-post unfolding bmssp-post-full-def by simp
have finite-W: finite ?W
  by simp
have finite-U: finite U
  using 1(1) finite-U-loop finite-W by simp
have step:
  c  $\leq$  A * Suc (Suc (2 * l0 + 1)) * card U +
    (R + l0 * t + t) * card (outgoing-edges U)
  by (rule split-range-costed-nonbase-step-closes-level-bound-from-source-progress-
general
    [OF 1(4) sound-fp pivot-pre P-reaches child-bound
      source-progress-loop source-factor 1(1) finite-U scan-insert 1(2)])
show ?thesis
  using step Suc unfolding level-range-cost-bound-def
  by (simp add: algebra-simps)

```

qed
 qed
 qed

theorem *split-range-costed-bmssp-level-bound-from-source-progress-budgets:*

assumes *degree: edge-outdegree-le Δ*

and *degree-factor: $\Delta \leq A$*

and *R-pos: $0 < R$*

and *insert-factor: $t \leq A * k$*

and *source-factor: $Suc\ h \leq 2 * A$*

and *source-progress:*

$\bigwedge l\ d\ S\ B\ d'\ B'\ U\ c.$

split-range-costed-bmssp Δ M-of t h k cap l d S B d' B' U c $\implies$

bmssp-pre-full d S B $\implies$

($\bigwedge x. x \in S \implies reachable\ s\ x$) $\implies$

card S $\leq$ M-of l $\implies$

card S $\leq$ card U

and *step-progress:*

$\bigwedge l\ d\ S\ B\ D\ c\text{-insert}\ d'\ a\ betas\ bs\ B'\ Us\ U\text{-loop}\ c\text{-loop}$

child-costs U.

D = label-partition-view

(find-pivots-label-capped k cap d S B)

(find-pivots-pivots-capped k cap d S B) $\implies$

partition-initial-insert-cost-bound c-insert t

(find-pivots-pivots-capped k cap d S B) $\implies$

split-range-costed-partition-loop-state Δ M-of t h k cap l

(find-pivots-label-capped k cap d S B)

(find-pivots-pivots-capped k cap d S B) B d' D a betas bs B'

Us U-loop c-loop child-costs $\implies$

complete-on d'

{v $\in$ bound-tree S B'. find-pivots-label-capped k cap d S B v = dist s v} $\implies$

U = U-loop $\cup$

{v $\in$ bound-tree S B'. find-pivots-label-capped k cap d S B v = dist s v} $\implies$

sound-label d $\implies$

bmssp-pre-full d S B $\implies$

($\bigwedge x. x \in S \implies reachable\ s\ x$) $\implies$

*card S $\leq$ cap $\wedge$ k * cap $\leq$ card U*

and *run:*

split-range-costed-bmssp Δ M-of t h k cap l d S B d' B' U c

and *sound: sound-label d*

and *pre: bmssp-pre-full d S B*

and *S-reaches: $\bigwedge x. x \in S \implies reachable\ s\ x$*

shows *c $\leq$ level-range-cost-bound (2 * A) (R + l * t) (2 * l + 1) U*

proof (rule *split-range-costed-bmssp-level-bound-from-source-progress-and-local-budgets*

[**where** *A = 2 * A* **and** *R = R,*

OF - R-pos source-factor source-progress - run sound pre S-reaches])

show *$\Delta \leq 2 * A$*

using *degree-factor* **by** *simp*


```

next
  fix  $l\ d\ S\ B\ D\ c\text{-insert}\ d'\ a\ \text{betas}\ bs\ B'\ Us\ U\text{-loop}\ c\text{-loop}\ \text{child-costs}\ U$ 
  assume  $D\text{-def}$ :
     $D = \text{label-partition-view}$ 
     $(\text{find-pivots-label-capped}\ k\ \text{cap}\ d\ S\ B)$ 
     $(\text{find-pivots-pivots-capped}\ k\ \text{cap}\ d\ S\ B)$ 
  and  $\text{insert}$ :
     $\text{partition-initial-insert-cost-bound}\ c\text{-insert}\ t$ 
     $(\text{find-pivots-pivots-capped}\ k\ \text{cap}\ d\ S\ B)$ 
  and  $\text{loop}$ :
     $\text{split-range-costed-partition-loop-state}\ \Delta\ M\text{-of}\ t\ h\ k\ \text{cap}\ l$ 
     $(\text{find-pivots-label-capped}\ k\ \text{cap}\ d\ S\ B)$ 
     $(\text{find-pivots-pivots-capped}\ k\ \text{cap}\ d\ S\ B)\ B\ d'\ D\ a\ \text{betas}\ bs\ B'$ 
     $Us\ U\text{-loop}\ c\text{-loop}\ \text{child-costs}$ 
  and  $\text{complete}$ :
     $\text{complete-on}\ d'$ 
     $\{v \in \text{bound-tree}\ S\ B'.\ \text{find-pivots-label-capped}\ k\ \text{cap}\ d\ S\ B\ v = \text{dist}\ s\ v\}$ 
  and  $U\text{-def}$ :
     $U = U\text{-loop} \cup$ 
     $\{v \in \text{bound-tree}\ S\ B'.\ \text{find-pivots-label-capped}\ k\ \text{cap}\ d\ S\ B\ v = \text{dist}\ s\ v\}$ 
  and  $\text{sound-s}$ :  $\text{sound-label}\ d$ 
  and  $\text{pre-s}$ :  $\text{bmssp-pre-full}\ d\ S\ B$ 
  and  $\text{reaches-s}$ :  $\bigwedge x. x \in S \implies \text{reachable}\ s\ x$ 
  have  $S\text{-subset}$ :  $S \subseteq V$ 
  using  $\text{pre-s}\ \text{unfolding}\ \text{bmssp-pre-full-def}\ \text{by}\ \text{blast}$ 
  have  $\text{progress}$ :
     $\text{card}\ S \leq \text{cap} \wedge k * \text{cap} \leq \text{card}\ U$ 
  by  $(\text{rule}\ \text{step-progress}[\text{OF}\ D\text{-def}\ \text{insert}\ \text{loop}\ \text{complete}\ U\text{-def}\ \text{sound-s}\ \text{pre-s}\ \text{reaches-}$ 
 $s])$ 
  then have  $S\text{-cap}$ :  $\text{card}\ S \leq \text{cap}$ 
  by  $\text{blast}$ 
  from  $\text{progress}\ \text{have}\ U\text{-progress}$ :  $k * \text{cap} \leq \text{card}\ U$ 
  by  $\text{blast}$ 
  show  $\text{fp-iter-capped-scan-cost}\ k\ \text{cap}\ d\ S\ S\ B + c\text{-insert} \leq$ 
 $2 * A * \text{card}\ U$ 
  by  $(\text{rule}\ \text{find-pivots-scan-and-initial-insert-budget-from-progress}$ 
 $[\text{OF}\ S\text{-subset}\ \text{degree}\ S\text{-cap}\ \text{insert}\ \text{degree-factor}\ \text{insert-factor}\ U\text{-progress}])$ 
qed

theorem  $\text{finite-initial-label-split-range-costed-top-level-correct}$ :
  assumes  $\text{all-reachable}$ :  $\bigwedge v. v \in V \implies \text{reachable}\ s\ v$ 
  and  $\text{run}$ :
     $\text{split-range-costed-bmssp}\ \Delta\ M\text{-of}\ t\ h\ k\ \text{cap}\ l\ \text{finite-initial-label}\ \{s\}$ 
     $\text{Infinity}\ d'\ \text{Infinity}\ U\ c$ 
  shows  $\text{sssp-correct}\ d'$ 
proof –
  have  $\text{pre}$ :  $\text{bmssp-pre-full}\ \text{finite-initial-label}\ \{s\}\ \text{Infinity}$ 
  using  $\text{all-reachable}\ \text{finite-initial-label-source-complete}$ 
  by  $(\text{rule}\ \text{top-bmssp-pre-full})$ 

```

```

have sound: sound-label finite-initial-label
  using finite-initial-label-sound[OF all-reachable] .
have S-reaches:  $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$ 
  using all-reachable source-in-V by blast
have post-full:
  bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
  by (rule split-range-costed-bmssp-correct[OF run sound pre S-reaches])
have post: bmssp-post finite-initial-label {s} Infinity d' Infinity U
  using bmssp-post-full-imp-post[OF post-full] .
then have U-V:  $U = V$ 
  using bound-tree-source-infinity[OF all-reachable]
  unfolding bmssp-post-def by auto
have complete: complete-on d' U
  using post unfolding bmssp-post-def by auto
then show ?thesis
  using U-V unfolding complete-on-def sssp-correct-def by auto
qed

theorem finite-initial-label-split-range-costed-top-level-correct-and-local-budget-bound:
assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
and base-budget:  $\Delta \leq A$ 
and R-pos:  $0 < R$ 
and source-factor:  $\text{Suc } h \leq A$ 
and source-progress:
 $\bigwedge l \ d \ S \ B \ d' \ B' \ U \ c.$ 
  split-range-costed-bmssp  $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies$ 
  bmssp-pre-full  $d \ S \ B \implies$ 
   $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$ 
  card  $S \leq M\text{-of } l \implies$ 
  card  $S \leq \text{card } U$ 
and step-budget:
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$ 
  child-costs  $U.$ 
   $D = \text{label-partition-view}$ 
  (find-pivots-label-capped  $k \ \text{cap } d \ S \ B$ )
  (find-pivots-pivots-capped  $k \ \text{cap } d \ S \ B$ )  $\implies$ 
partition-initial-insert-cost-bound  $c\text{-insert } t$ 
  (find-pivots-pivots-capped  $k \ \text{cap } d \ S \ B$ )  $\implies$ 
split-range-costed-partition-loop-state  $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l$ 
  (find-pivots-label-capped  $k \ \text{cap } d \ S \ B$ )
  (find-pivots-pivots-capped  $k \ \text{cap } d \ S \ B$ )  $B \ d' \ D \ a \ \text{betas } bs \ B'$ 
   $Us \ U\text{-loop } c\text{-loop} \ \text{child-costs} \implies$ 
complete-on  $d'$ 
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$ 
 $U = U\text{-loop} \cup$ 
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$ 
sound-label  $d \implies$ 
bmssp-pre-full  $d \ S \ B \implies$ 

```

$(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{fp-iter-capped-scan-cost } k \text{ cap } d \ S \ S \ B + c\text{-insert} \leq A * \text{card } U$
and run:
 $\text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } l \ \text{finite-initial-label } \{s\}$
 $\text{Infinity } d' \ \text{Infinity } U \ c$
shows $\text{sssp-correct } d' \wedge$
 $c \leq \text{level-range-cost-bound } A \ (R + l * t) \ (2 * l + 1) \ U$
proof
show $\text{sssp-correct } d'$
by $(\text{rule } \text{finite-initial-label-split-range-costed-top-level-correct}$
 $[\text{OF } \text{all-reachable run}])$
next
have $\text{pre: } \text{bmssp-pre-full } \text{finite-initial-label } \{s\} \ \text{Infinity}$
using $\text{all-reachable } \text{finite-initial-label-source-complete}$
by $(\text{rule } \text{top-bmssp-pre-full})$
have $\text{sound: } \text{sound-label } \text{finite-initial-label}$
using $\text{finite-initial-label-sound}[\text{OF } \text{all-reachable}]$.
have $S\text{-reaches: } \bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$
using $\text{all-reachable source-in-} V \text{ by blast}$
show $c \leq \text{level-range-cost-bound } A \ (R + l * t) \ (2 * l + 1) \ U$
by $(\text{rule } \text{split-range-costed-bmssp-level-bound-from-source-progress-and-local-}$
 budgets
 $[\text{OF } \text{base-budget } R\text{-pos } \text{source-factor } \text{source-progress } \text{step-budget run sound pre}$
 $S\text{-reaches}])$
qed

theorem $\text{finite-initial-label-split-range-costed-top-level-correct-and-local-budget-}$
 graph-bound:
assumes $\text{all-reachable: } \bigwedge v. v \in V \implies \text{reachable } s \ v$
and $\text{base-budget: } \Delta \leq A$
and $R\text{-pos: } 0 < R$
and $\text{source-factor: } \text{Suc } h \leq A$
and source-progress:
 $\bigwedge l \ d \ S \ B \ d' \ B' \ U \ c.$
 $\text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq M\text{-of } l \implies$
 $\text{card } S \leq \text{card } U$
and step-budget:
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
 $\text{child-costs } U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \text{ cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \implies$
 $\text{split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \text{ cap } l$
 $(\text{find-pivots-label-capped } k \text{ cap } d \ S \ B)$

$(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \text{ } B \text{ } d' \text{ } D \text{ } a \text{ betas } bs \text{ } B'$
 $Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{ find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{ find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \text{ } S \text{ } B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \text{ } x) \implies$
 $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq A * \text{card } U$
and run:
 $\text{split-range-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } l \text{ finite-initial-label } \{s\}$
 $\text{Infinity } d' \text{ Infinity } U \text{ } c$
shows $\text{sssp-correct } d' \wedge$
 $c \leq A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$
proof –
have *combined:*
 $\text{sssp-correct } d' \wedge$
 $c \leq \text{level-range-cost-bound } A \text{ } (R + l * t) \text{ } (2 * l + 1) \text{ } U$
by (*rule finite-initial-label-split-range-costed-top-level-correct-and-local-budget-bound*
 $[OF \text{ all-reachable base-budget } R\text{-pos source-factor source-progress step-budget run}]$)
then have *correct:* $\text{sssp-correct } d'$
by *blast*
have *pre:* $\text{bmssp-pre-full finite-initial-label } \{s\} \text{ Infinity}$
using *all-reachable finite-initial-label-source-complete*
by (*rule top-bmssp-pre-full*)
have *sound:* $\text{sound-label finite-initial-label}$
using *finite-initial-label-sound* $[OF \text{ all-reachable}]$.
have *S-reaches:* $\bigwedge x. x \in \{s\} \implies \text{reachable } s \text{ } x$
using *all-reachable source-in-V* **by** *blast*
have *post-full:*
 $\text{bmssp-post-full finite-initial-label } \{s\} \text{ Infinity } d' \text{ Infinity } U$
by (*rule split-range-costed-bmssp-correct* $[OF \text{ run sound pre } S\text{-reaches}]$)
have *post:* $\text{bmssp-post finite-initial-label } \{s\} \text{ Infinity } d' \text{ Infinity } U$
using *bmssp-post-full-imp-post* $[OF \text{ post-full}]$.
have *U-V:* $U = V$
using *post bound-tree-source-infinity* $[OF \text{ all-reachable}]$
unfolding *bmssp-post-def* **by** *auto*
have *cost-U:*
 $c \leq \text{level-range-cost-bound } A \text{ } (R + l * t) \text{ } (2 * l + 1) \text{ } U$
using *combined* **by** *blast*
have *graph-bound:*
 $\text{level-range-cost-bound } A \text{ } (R + l * t) \text{ } (2 * l + 1) \text{ } U$
 $\leq A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$
proof –
have *U-card:* $\text{card } U = \text{vertex-count}$
unfolding *U-V vertex-count-def* **by** *simp*

```

have out-le:  $\text{card } (\text{outgoing-edges } U) \leq \text{edge-count}$ 
by (rule edge-count-outgoing-bound)
have out-term:
   $(R + l * t) * \text{card } (\text{outgoing-edges } U) \leq (R + l * t) * \text{edge-count}$ 
using out-le by simp
show ?thesis
unfolding level-range-cost-bound-def
using U-card out-term by (simp add: algebra-simps)
qed
show ?thesis
using correct cost-U graph-bound by linarith
qed

```

theorem finite-initial-label-split-range-costed-top-level-correct-and-source-progress-bound:

```

assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
and degree:  $\text{edge-outdegree-le } \Delta$ 
and degree-factor:  $\Delta \leq A$ 
and R-pos:  $0 < R$ 
and insert-factor:  $t \leq A * k$ 
and source-factor:  $\text{Suc } h \leq 2 * A$ 
and source-progress:
   $\bigwedge l \ d \ S \ B \ d' \ B' \ U \ c.$ 
    split-range-costed-bmssp  $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies$ 
    bmssp-pre-full  $d \ S \ B \implies$ 
     $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$ 
     $\text{card } S \leq M\text{-of } l \implies$ 
     $\text{card } S \leq \text{card } U$ 
and step-progress:
   $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$ 
    child-costs  $U.$ 
     $D = \text{label-partition-view}$ 
     $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$ 
     $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$ 
    partition-initial-insert-cost-bound  $c\text{-insert } t$ 
     $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$ 
    split-range-costed-partition-loop-state  $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l$ 
     $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$ 
     $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$ 
     $Us \ U\text{-loop } c\text{-loop} \ \text{child-costs} \implies$ 
    complete-on  $d'$ 
     $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$ 
     $U = U\text{-loop} \cup$ 
     $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$ 
    sound-label  $d \implies$ 
    bmssp-pre-full  $d \ S \ B \implies$ 
     $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$ 
     $\text{card } S \leq \text{cap} \wedge k * \text{cap} \leq \text{card } U$ 
and run:

```

split-range-costed-bmssp Δ *M-of t h k cap l finite-initial-label* $\{s\}$
Infinity d' Infinity U c
shows *sssp-correct d' $\wedge$*
*c $\leq$ level-range-cost-bound (2 * A) (R + l * t) (2 * l + 1) U*
proof
show *sssp-correct d'*
by (*rule finite-initial-label-split-range-costed-top-level-correct*
[OF all-reachable run])
next
have *pre: bmssp-pre-full finite-initial-label {s} Infinity*
using *all-reachable finite-initial-label-source-complete*
by (*rule top-bmssp-pre-full*)
have *sound: sound-label finite-initial-label*
using *finite-initial-label-sound[OF all-reachable]* .
have *S-reaches: $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$*
using *all-reachable source-in-V* **by** *blast*
show *c $\leq$ level-range-cost-bound (2 * A) (R + l * t) (2 * l + 1) U*
by (*rule split-range-costed-bmssp-level-bound-from-source-progress-budgets*
[OF degree degree-factor R-pos insert-factor source-factor source-progress
step-progress run sound pre S-reaches])
qed

theorem *finite-initial-label-split-range-costed-top-level-correct-and-graph-bound:*

assumes *all-reachable: $\bigwedge v. v \in V \implies \text{reachable } s \ v$*
and *degree: edge-outdegree-le Δ*
and *degree-factor: $\Delta \leq A$*
and *R-pos: $0 < R$*
and *insert-factor: $t \leq A * k$*
and *source-factor: $\text{Suc } h \leq 2 * A$*
and *source-progress:*
 $\bigwedge l \ d \ S \ B \ d' \ B' \ U \ c.$
split-range-costed-bmssp Δ M-of t h k cap l d S B d' B' U c $\implies$
bmssp-pre-full d S B $\implies$
($\bigwedge x. x \in S \implies \text{reachable } s \ x$) $\implies$
card S $\leq$ M-of l $\implies$
card S $\leq$ card U
and *step-progress:*
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
child-costs U.
D = label-partition-view
(find-pivots-label-capped k cap d S B)
(find-pivots-pivots-capped k cap d S B) $\implies$
partition-initial-insert-cost-bound c-insert t
(find-pivots-pivots-capped k cap d S B) $\implies$
split-range-costed-partition-loop-state Δ M-of t h k cap l
(find-pivots-label-capped k cap d S B)
(find-pivots-pivots-capped k cap d S B) B d' D a betas bs B'
Us U-loop c-loop child-costs $\implies$
complete-on d'

$\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq \text{cap} \wedge k * \text{cap} \leq \text{card } U$
and run:
 $\text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ \text{finite-initial-label } \{s\}$
 $\text{Infinity } d' \ \text{Infinity } U \ c$
shows $\text{sssp-correct } d' \wedge$
 $c \leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$
proof –
have *combined:*
 $\text{sssp-correct } d' \wedge$
 $c \leq \text{level-range-cost-bound } (2 * A) \ (R + l * t) \ (2 * l + 1) \ U$
by (rule *finite-initial-label-split-range-costed-top-level-correct-and-source-progress-bound*)
 $[OF \ \text{all-reachable degree degree-factor } R\text{-pos insert-factor source-factor}$
 $\text{source-progress step-progress run}]$
then have *correct:* $\text{sssp-correct } d'$
by *blast*
have *pre:* $\text{bmssp-pre-full finite-initial-label } \{s\} \ \text{Infinity}$
using *all-reachable finite-initial-label-source-complete*
by (rule *top-bmssp-pre-full*)
have *sound:* $\text{sound-label finite-initial-label}$
using *finite-initial-label-sound* $[OF \ \text{all-reachable}]$.
have *S-reaches:* $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$
using *all-reachable source-in-V* **by** *blast*
have *post-full:*
 $\text{bmssp-post-full finite-initial-label } \{s\} \ \text{Infinity } d' \ \text{Infinity } U$
by (rule *split-range-costed-bmssp-correct* $[OF \ \text{run sound pre } S\text{-reaches}]$)
have *post:* $\text{bmssp-post finite-initial-label } \{s\} \ \text{Infinity } d' \ \text{Infinity } U$
using *bmssp-post-full-imp-post* $[OF \ \text{post-full}]$.
have *U-V:* $U = V$
using *post bound-tree-source-infinity* $[OF \ \text{all-reachable}]$
unfolding *bmssp-post-def* **by** *auto*
have *cost-U:*
 $c \leq \text{level-range-cost-bound } (2 * A) \ (R + l * t) \ (2 * l + 1) \ U$
using *combined* **by** *blast*
have *graph-bound:*
 $\text{level-range-cost-bound } (2 * A) \ (R + l * t) \ (2 * l + 1) \ U$
 $\leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$
proof –
have *U-card:* $\text{card } U = \text{vertex-count}$
unfolding *U-V vertex-count-def* **by** *simp*
have *out-le:* $\text{card } (\text{outgoing-edges } U) \leq \text{edge-count}$
by (rule *edge-count-outgoing-bound*)
have *out-term:*

```

      (R + l * t) * card (outgoing-edges U) ≤ (R + l * t) * edge-count
    using out-le by simp
  show ?thesis
    unfolding level-range-cost-bound-def
    using U-card out-term by (simp add: algebra-simps)
qed
show ?thesis
  using correct cost-U graph-bound by linarith
qed

theorem split-range-costed-bmssp-source-card-le-if-sound-label-below-output:
  assumes run:
    split-range-costed-bmssp Δ M-of t h k cap l d S B d' B' U c
  and sound: sound-label d
  and pre: bmssp-pre-full d S B
  and S-reaches: ∧x. x ∈ S ⇒ reachable s x
  and below: ∧x. x ∈ S ⇒ below-bound (d x) B'
  shows card S ≤ card U
proof -
  have post: bmssp-post-full d S B d' B' U
  by (rule split-range-costed-bmssp-correct[OF run sound pre S-reaches])
  have U-eq: U = bound-tree S B'
  using post unfolding bmssp-post-full-def by blast
  have S-subset-U: S ⊆ U
  proof
    fix x
    assume xS: x ∈ S
    have S-subset: S ⊆ V
    using pre unfolding bmssp-pre-full-def by blast
    have xV: x ∈ V
    using S-subset xS by blast
    have reach-x: reachable s x
    by (rule S-reaches[OF xS])
    have dist-le: dist s x ≤ d x
    using sound xV reach-x unfolding sound-label-def by blast
    have below-dist: below-bound (dist s x) B'
    using below-bound-le-trans[OF dist-le below[OF xS]] .
    show x ∈ U
    unfolding U-eq
    by (rule source-in-own-bound-tree[OF xS reach-x below-dist])
  qed
  have finite-U: finite U
  unfolding U-eq bound-tree-def using finite-V by auto
  show ?thesis
  by (rule card-mono[OF finite-U S-subset-U])
qed

```

```

theorem split-range-costed-partition-loop-state-success-or-threshold:
  split-range-costed-partition-loop-state Δ M-of t h k cap l d P B d' D a

```


$\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ P \ B \implies$
 $(\bigwedge x. x \in P \implies \text{reachable } s \ x) \implies$
 $B' = B \vee k * \text{cap} \leq \text{card } U$
and *split-range-costed-bmssp-success-or-threshold-trivial*:
 $\text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies \text{True}$
proof (*induction rule: split-range-costed-partition-loop-state-split-range-costed-bmssp.inducts*)
case (*Split-Range-State-Done* $D \ a \ B \ d' \ P \ \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d$)
then show *?case*
by *simp*
next
case (*Split-Range-State-Stop* $a \ B \ d' \ P \ k \ \text{cap} \ \Delta \ M\text{-of } t \ h \ l \ d \ D$)
then show *?case*
by *simp*
next
case (*Split-Range-State-Step* $D \ M\text{-of } l \ B_{\text{max}} \ S\text{-pull } \text{beta} \ D\text{-pull } B \ d \ a \ b \ B\text{-out}$
 $d\text{-out } P \ k \ \text{cap} \ \Delta \ t \ h \ d\text{-child } U\text{-child } c\text{-child } \text{edge-batch } \text{source-batch}$
 $\text{batch } D\text{-next } c\text{-pull } c\text{-edges } c\text{-sources } \text{betas } bs \ Us\text{-tail } U\text{-tail } c\text{-tail}$
 $\text{child-costs-tail } c$)
have *tail*:
 $B\text{-out} = B \vee k * \text{cap} \leq \text{card } U\text{-tail}$
using *Split-Range-State-Step.IH Split-Range-State-Step.premis* **by** *blast*
then show *?case*
proof
assume $B\text{-out} = B$
then show *?thesis*
by *blast*
next
assume *threshold-tail*: $k * \text{cap} \leq \text{card } U\text{-tail}$
have *tail-trace*:
 $\text{concrete-partition-loop-trace } P \ B \ b \ bs \ d\text{-out } B\text{-out } Us\text{-tail } U\text{-tail}$
by (*rule split-range-costed-partition-loop-state-trace*)
(use Split-Range-State-Step in blast)+
have *U-tail-def*:
 $U\text{-tail} = \text{bound-tree } P \ (Fin \ b) \cup \bigcup (set \ Us\text{-tail})$
using *tail-trace unfolding concrete-partition-loop-trace-def* **by** *blast*
have *split-b*:
 $\text{bound-tree } P \ (Fin \ b) =$
 $\text{bound-tree } P \ (Fin \ a) \cup \text{range-tree } P \ a \ (Fin \ b)$
using *bound-tree-split-at[of a Fin b P] Split-Range-State-Step*
by *simp*
have *U-eq-tail*:
 $\text{bound-tree } P \ (Fin \ a) \cup$
 $\bigcup (set \ (\text{range-tree } P \ a \ (Fin \ b) \ \# \ Us\text{-tail})) =$
 $U\text{-tail}$
using *U-tail-def split-b* **by** *auto*
show *?thesis*

```

    using threshold-tail U-eq-tail by simp
qed
next
case (Split-Range-Cost-Base  $S\ x\ \Delta\ M\text{-of}\ t\ h\ k\ \text{cap}\ d\ B$ )
then show ?case
  by simp
next
case (Split-Range-Cost-Step  $D\ k\ \text{cap}\ d\ S\ B\ c\text{-insert}\ t\ \Delta\ M\text{-of}\ h\ l\ d'\ a$ 
   $\text{betas}\ bs\ B'\ Us\ U\text{-loop}\ c\text{-loop}\ \text{child-costs-loop}\ U\ c$ )
then show ?case
  by simp
qed

theorem split-range-costed-bmssp-Suc-success-or-threshold:
  assumes run:
     $\text{split-range-costed-bmssp}\ \Delta\ M\text{-of}\ t\ h\ k\ \text{cap}\ (\text{Suc}\ l)\ d\ S\ B\ d'\ B'\ U\ c$ 
  and sound: sound-label  $d$ 
  and pre: bmssp-pre-full  $d\ S\ B$ 
  and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable}\ s\ x$ 
  shows  $B' = B \vee k * \text{cap} \leq \text{card}\ U$ 
  using run
proof cases
case (Split-Range-Cost-Step  $D\ c\text{-insert}\ a\ \text{betas}\ bs\ Us\ U\text{-loop}\ c\text{-loop}\ \text{child-costs-loop}$ )
let ?d-fp = find-pivots-label-capped  $k\ \text{cap}\ d\ S\ B$ 
let ?P = find-pivots-pivots-capped  $k\ \text{cap}\ d\ S\ B$ 
let ?W =  $\{v \in \text{bound-tree}\ S\ B'.\ ?d\text{-fp}\ v = \text{dist}\ s\ v\}$ 
have sound-fp: sound-label ?d-fp
  unfolding find-pivots-label-capped-def
  by (rule fp-iter-capped-label-sound[OF sound S-reaches])
have pivot-pre: bmssp-pre-full ?d-fp ?P  $B$ 
  using find-pivots-capped-establishes-pivot-pre-concrete
  [OF sound pre S-reaches] .
have P-subset-S: ?P  $\subseteq S$ 
  unfolding find-pivots-pivots-capped-def by auto
have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable}\ s\ x$ 
  using P-subset-S S-reaches by blast
have loop-class:
 $B' = B \vee k * \text{cap} \leq \text{card}\ U\text{-loop}$ 
  by (rule split-range-costed-partition-loop-state-success-or-threshold
    [OF Split-Range-Cost-Step(3) sound-fp pivot-pre P-reaches])
have loop-trace:
concrete-partition-loop-trace ?P  $B\ a\ bs\ d'\ B'\ Us\ U\text{-loop}$ 
  by (rule split-range-costed-partition-loop-state-trace
    [OF Split-Range-Cost-Step(3) sound-fp pivot-pre P-reaches])
have loop-post: bmssp-post-full ?d-fp ?P  $B\ d'\ B'\ U\text{-loop}$ 
  by (rule concrete-partition-loop-trace-post[OF loop-trace])
have finite-U-loop: finite  $U\text{-loop}$ 
  using loop-post unfolding bmssp-post-full-def by simp
show ?thesis

```

```

    using loop-class
  proof
    assume  $B' = B$ 
    then show ?thesis
      by blast
  next
    assume threshold-loop:  $k * \text{cap} \leq \text{card } U\text{-loop}$ 
    have U-eq:  $U = U\text{-loop} \cup ?W$ 
      using Split-Range-Cost-Step(5) by simp
    have finite-U: finite U
      unfolding U-eq using finite-U-loop by simp
    have U-loop-subset:  $U\text{-loop} \subseteq U$ 
      unfolding U-eq by blast
    have  $\text{card } U\text{-loop} \leq \text{card } U$ 
      by (rule card-mono[OF finite-U U-loop-subset])
    then show ?thesis
      using threshold-loop by linarith
  qed
qed

theorem split-range-costed-bmssp-Suc-source-card-le-from-label-below:
  assumes run:
    split-range-costed-bmssp  $\Delta$  M-of t h k cap (Suc l) d S B d' B' U c
  and sound: sound-label d
  and pre: bmssp-pre-full d S B
  and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
  and below:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$ 
  and S-cap:  $\text{card } S \leq \text{cap}$ 
  and k-pos:  $0 < k$ 
  shows  $\text{card } S \leq \text{card } U$ 
proof -
  have call-class:  $B' = B \vee k * \text{cap} \leq \text{card } U$ 
    by (rule split-range-costed-bmssp-Suc-success-or-threshold
      [OF run sound pre S-reaches])
  then show ?thesis
  proof
    assume B'-eq:  $B' = B$ 
    show ?thesis
    proof (rule split-range-costed-bmssp-source-card-le-if-sound-label-below-output
      [OF run sound pre S-reaches])
      fix x
      assume  $x \in S$ 
      then show below-bound  $(d \ x) \ B'$ 
        using below B'-eq by simp
    qed
  next
    assume threshold:  $k * \text{cap} \leq \text{card } U$ 
    have  $\text{cap} \leq k * \text{cap}$ 
      using k-pos by (cases k) simp-all

```

```

    then show ?thesis
    using S-cap threshold by linarith
qed
qed

theorem split-range-costed-capped-step-threshold-if-not-success:
  assumes loop:
    split-range-costed-partition-loop-state  $\Delta$  M-of t h k cap l
    (find-pivots-label-capped k cap d S B)
    (find-pivots-pivots-capped k cap d S B) B d' D a betas bs B'
    Us U-loop c-loop child-costs
  and U-def:
     $U = U\text{-loop} \cup \{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\}$ 
  and sound: sound-label d
  and pre: bmssp-pre-full d S B
  and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
  and not-success:  $B' \neq B$ 
  shows  $k * \text{cap} \leq \text{card } U$ 
proof -
  let ?d-fp = find-pivots-label-capped k cap d S B
  let ?P = find-pivots-pivots-capped k cap d S B
  have sound-fp: sound-label ?d-fp
    unfolding find-pivots-label-capped-def
    by (rule fp-iter-capped-label-sound[OF sound S-reaches])
  have pivot-pre: bmssp-pre-full ?d-fp ?P B
    using find-pivots-capped-establishes-pivot-pre-concrete
    [OF sound pre S-reaches] .
  have P-subset-S:  $?P \subseteq S$ 
    unfolding find-pivots-pivots-capped-def by auto
  have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
    using P-subset-S S-reaches by blast
  have loop-class:
     $B' = B \vee k * \text{cap} \leq \text{card } U\text{-loop}$ 
    by (rule split-range-costed-partition-loop-state-success-or-threshold
    [OF loop sound-fp pivot-pre P-reaches])
  have threshold-loop:  $k * \text{cap} \leq \text{card } U\text{-loop}$ 
    using loop-class not-success by blast
  have trace: concrete-partition-loop-trace ?P B a bs d' B' Us U-loop
    by (rule split-range-costed-partition-loop-state-trace
    [OF loop sound-fp pivot-pre P-reaches])
  have loop-post: bmssp-post-full ?d-fp ?P B d' B' U-loop
    by (rule concrete-partition-loop-trace-post[OF trace])
  have finite-U-loop: finite U-loop
    using loop-post unfolding bmssp-post-full-def by simp
  have finite-U: finite U
    unfolding U-def using finite-U-loop by simp
  have U-loop-subset:  $U\text{-loop} \subseteq U$ 
    unfolding U-def by blast

```

have $\text{card } U\text{-loop} \leq \text{card } U$
by (rule *card-mono*[*OF finite-U U-loop-subset*])
then show *?thesis*
using *threshold-loop* **by** *linarith*
qed

theorem *split-range-costed-capped-step-scan-insert-budget-if-not-success*:
assumes *degree*: $\text{edge-outdegree-le } \Delta$
and *degree-factor*: $\Delta \leq A$
and *insert-factor*: $t \leq A * k$
and *insert*:
partition-initial-insert-cost-bound c-insert t
(find-pivots-pivots-capped k cap d S B)
and *loop*:
split-range-costed-partition-loop-state Δ M-of t h k cap l
(find-pivots-label-capped k cap d S B)
(find-pivots-pivots-capped k cap d S B) B d' D a betas bs B'
Us U-loop c-loop child-costs
and *U-def*:
 $U = U\text{-loop} \cup \{v \in \text{bound-tree } S B'. \text{find-pivots-label-capped } k \text{ cap } d S B v = \text{dist } s v\}$
and *sound*: *sound-label d*
and *pre*: *bmssp-pre-full d S B*
and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s x$
and *S-cap*: $\text{card } S \leq \text{cap}$
and *not-success*: $B' \neq B$
shows $\text{fp-iter-capped-scan-cost } k \text{ cap } d S S B + c\text{-insert} \leq 2 * A * \text{card } U$
proof –
have *S-subset*: $S \subseteq V$
using *pre unfolding bmssp-pre-full-def* **by** *blast*
have *progress*: $k * \text{cap} \leq \text{card } U$
by (rule *split-range-costed-capped-step-threshold-if-not-success*
[*OF loop U-def sound pre S-reaches not-success*])
show *?thesis*
by (rule *find-pivots-scan-and-initial-insert-budget-from-progress*
[*OF S-subset degree S-cap insert degree-factor insert-factor progress*])
qed

theorem *split-range-costed-capped-step-scan-insert-budget-from-seen-or-threshold*:
assumes *degree*: $\text{edge-outdegree-le } \Delta$
and *degree-factor*: $\Delta \leq A$
and *insert-factor*: $t \leq A * k$
and *seen-factor*: $k * \Delta + t \leq 2 * A$
and *insert*:
partition-initial-insert-cost-bound c-insert t
(find-pivots-pivots-capped k cap d S B)
and *loop*:
split-range-costed-partition-loop-state Δ M-of t h k cap l

$(\text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \text{ } B \text{ } d' \text{ } D \text{ } a \text{ } \text{betas } bs \text{ } B'$
 $Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs}$
and $U\text{-def}$:
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{ find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\}$
and sound : $\text{sound-label } d$
and pre : $\text{bmssp-pre-full } d \text{ } S \text{ } B$
and $S\text{-reaches}$: $\bigwedge x. x \in S \implies \text{reachable } s \text{ } x$
and $S\text{-cap}$: $\text{card } S \leq \text{cap}$
and anti : $\text{tree-antichain } S$
and $k\text{-pos}$: $0 < k$
and seen-success :
 $B' = B \implies$
 $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B) \leq \text{card } U$
shows $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $2 * A * \text{card } U$
proof ($\text{cases } B' = B$)
case True
have $S\text{-subset}$: $S \subseteq V$
using pre **unfolding** $\text{bmssp-pre-full-def}$ **by** blast
show $?thesis$
by ($\text{rule } \text{find-pivots-scan-and-initial-insert-budget-from-seen-total}$
 $[OF \text{ } S\text{-subset } \text{degree } S\text{-cap } \text{anti } k\text{-pos } \text{insert } \text{seen-factor}$
 $\text{seen-success}[OF \text{ True}]]$)
next
case False
show $?thesis$
by ($\text{rule } \text{split-range-costed-capped-step-scan-insert-budget-if-not-success}$
 $[OF \text{ } \text{degree } \text{degree-factor } \text{insert-factor } \text{insert } \text{loop } U\text{-def } \text{sound } \text{pre}$
 $S\text{-reaches } S\text{-cap } \text{False}])$
qed

theorem $\text{split-range-costed-capped-step-scan-insert-budget-from-scaled-seen-or-threshold}$:
assumes degree : $\text{edge-outdegree-le } \Delta$
and degree-factor : $\Delta \leq A$
and insert-factor : $t \leq A * k$
and $\text{insert-scaled-factor}$: $t \leq A\text{-insert} * k$
and $\text{seen-scaled-factor}$: $k * \Delta + A\text{-insert} \leq 2 * A$
and insert :
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B)$
and loop :
 $\text{split-range-costed-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } l$
 $(\text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \text{ } B \text{ } d' \text{ } D \text{ } a \text{ } \text{betas } bs \text{ } B'$
 $Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs}$
and $U\text{-def}$:

```

     $U = U\text{-loop} \cup \{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\}$ 
and sound: sound-label  $d$ 
and pre: bmssp-pre-full  $d \ S \ B$ 
and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
and S-k-cap:  $k * \text{card } S \leq \text{cap}$ 
and anti: tree-antichain  $S$ 
and k-pos:  $0 < k$ 
and seen-success:
     $B' = B \implies$ 
     $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \ S \ B) \leq \text{card } U$ 
shows fp-iter-capped-scan-cost  $k \text{ cap } d \ S \ S \ B + c\text{-insert} \leq$ 
     $2 * A * \text{card } U$ 
proof (cases  $B' = B$ )
  case True
    have S-subset:  $S \subseteq V$ 
    using pre unfolding bmssp-pre-full-def by blast
    show ?thesis
    by (rule find-pivots-scan-and-initial-insert-budget-from-seen-scaled
      [OF S-subset degree anti S-k-cap insert insert-scaled-factor
        seen-scaled-factor seen-success[OF True]])
  next
    case False
    have S-cap:  $\text{card } S \leq \text{cap}$ 
    proof –
      have  $\text{card } S \leq k * \text{card } S$ 
      using k-pos by (cases  $k$ ) simp-all
      then show ?thesis
      using S-k-cap by linarith
    qed
    show ?thesis
    by (rule split-range-costed-capped-step-scan-insert-budget-if-not-success
      [OF degree degree-factor insert-factor insert loop U-def sound pre
        S-reaches S-cap False])
  qed

theorem split-range-costed-bmssp-level-bound-from-seen-progress-budgets:
assumes degree: edge-outdegree-le  $\Delta$ 
and degree-factor:  $\Delta \leq A$ 
and R-pos:  $0 < R$ 
and insert-factor:  $t \leq A * k$ 
and seen-factor:  $k * \Delta + t \leq 2 * A$ 
and source-factor:  $\text{Suc } h \leq 2 * A$ 
and k-pos:  $0 < k$ 
and source-progress:
   $\bigwedge l \ d \ S \ B \ d' \ B' \ U \ c. \text{ split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies$ 
   $\text{bmssp-pre-full } d \ S \ B \implies$ 
   $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$ 

```

$\text{card } S \leq M\text{-of } l \implies$
 $\text{card } S \leq \text{card } U$
and *step-seen-progress*:
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
 $\text{child-costs } U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq \text{cap} \wedge \text{tree-antichain } S \wedge$
 $(B' = B \longrightarrow$
 $\text{card } (\text{find-pivots-seen-capped } k \ \text{cap } d \ S \ B) \leq \text{card } U)$
and *run*:
 $\text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c$
and *sound*: $\text{sound-label } d$
and *pre*: $\text{bmssp-pre-full } d \ S \ B$
and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
shows $c \leq \text{level-range-cost-bound } (2 * A) \ (R + l * t) \ (2 * l + 1) \ U$
proof (rule *split-range-costed-bmssp-level-bound-from-source-progress-and-local-budgets*
[**where** $A = 2 * A$ **and** $R = R,$
 $OF - R\text{-pos source-factor source-progress - run sound pre } S\text{-reaches}$])
show $\Delta \leq 2 * A$
using *degree-factor* **by** *simp*
next
fix $l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs } U$
assume *D-def*:
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B)$
and *insert*:
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B)$
and *loop*:
 $\text{split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$

Us U-loop c-loop child-costs
and complete:
complete-on d'
 $\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\}$
and U-def:
 $U = \text{U-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\}$
and sound-s: *sound-label d*
and pre-s: *bmssp-pre-full d S B*
and reaches-s: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
have progress:
 $\text{card } S \leq \text{cap} \wedge \text{tree-antichain } S \wedge$
 $(B' = B \longrightarrow$
 $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \ S \ B) \leq \text{card } U)$
by (*rule step-seen-progress*
 $[\text{OF } D\text{-def insert loop complete U-def sound-s pre-s reaches-s}])$
then have S-cap: $\text{card } S \leq \text{cap}$
by blast
from progress have anti: *tree-antichain S*
by blast
from progress have seen-success:
 $B' = B \implies$
 $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \ S \ B) \leq \text{card } U$
by blast
show fp-iter-capped-scan-cost k cap d S S B + c-insert $\leq$
 $2 * A * \text{card } U$
by (*rule split-range-costed-capped-step-scan-insert-budget-from-seen-or-threshold*
 $[\text{OF degree degree-factor insert-factor seen-factor insert loop U-def sound-s}$
 $\text{pre-s reaches-s S-cap anti k-pos seen-success}])$
qed

theorem split-range-costed-bmssp-level-bound-from-scaled-seen-progress-budgets:
assumes *degree: edge-outdegree-le Δ*
and degree-factor: $\Delta \leq A$
and R-pos: $0 < R$
and insert-factor: $t \leq A * k$
and insert-scaled-factor: $t \leq A\text{-insert} * k$
and seen-scaled-factor: $k * \Delta + A\text{-insert} \leq 2 * A$
and source-factor: $\text{Suc } h \leq 2 * A$
and k-pos: $0 < k$
and source-progress:
 $\bigwedge l \ d \ S \ B \ d' \ B' \ U \ c.$
 $\text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq M\text{-of } l \implies$
 $\text{card } S \leq \text{card } U$
and step-scaled-seen-progress:
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ \text{U-loop } c\text{-loop}$

child-costs U.
D = label-partition-view
(find-pivots-label-capped k cap d S B)
(find-pivots-pivots-capped k cap d S B) $\implies$
partition-initial-insert-cost-bound c-insert t
(find-pivots-pivots-capped k cap d S B) $\implies$
split-range-costed-partition-loop-state Δ M-of t h k cap l
(find-pivots-label-capped k cap d S B)
(find-pivots-pivots-capped k cap d S B) B d' D a betas bs B'
Us U-loop c-loop child-costs $\implies$
complete-on d'
 $\{v \in \text{bound-tree } S B'. \text{ find-pivots-label-capped } k \text{ cap } d S B v = \text{dist } s v\} \implies$
U = U-loop $\cup$
 $\{v \in \text{bound-tree } S B'. \text{ find-pivots-label-capped } k \text{ cap } d S B v = \text{dist } s v\} \implies$
sound-label d $\implies$
bmssp-pre-full d S B $\implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s x) \implies$
 *$k * \text{card } S \leq \text{cap} \wedge \text{tree-antichain } S \wedge$*
 $(B' = B \longrightarrow$
 $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d S B) \leq \text{card } U)$
and run:
split-range-costed-bmssp Δ M-of t h k cap l d S B d' B' U c
and sound: *sound-label d*
and pre: *bmssp-pre-full d S B*
and S-reaches: *$\bigwedge x. x \in S \implies \text{reachable } s x$*
shows *$c \leq \text{level-range-cost-bound } (2 * A) (R + l * t) (2 * l + 1) U$*
proof *(rule split-range-costed-bmssp-level-bound-from-source-progress-and-local-*
budgets
*[where $A = 2 * A$ and $R = R,$*
OF - R-pos source-factor source-progress - run sound pre S-reaches])
show *$\Delta \leq 2 * A$*
using *degree-factor by simp*
next
fix *l d S B D c-insert d' a betas bs B' Us U-loop c-loop child-costs U*
assume *D-def:*
D = label-partition-view
(find-pivots-label-capped k cap d S B)
(find-pivots-pivots-capped k cap d S B)
and insert:
partition-initial-insert-cost-bound c-insert t
(find-pivots-pivots-capped k cap d S B)
and loop:
split-range-costed-partition-loop-state Δ M-of t h k cap l
(find-pivots-label-capped k cap d S B)
(find-pivots-pivots-capped k cap d S B) B d' D a betas bs B'
Us U-loop c-loop child-costs
and complete:
complete-on d'
 $\{v \in \text{bound-tree } S B'. \text{ find-pivots-label-capped } k \text{ cap } d S B v = \text{dist } s v\}$

and *U-def*:
 $U = U\text{-loop} \cup \{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\}$
and *sound-s*: *sound-label* d
and *pre-s*: *bmssp-pre-full* $d \ S \ B$
and *reaches-s*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
have *progress*:
 $k * \text{card } S \leq \text{cap} \wedge \text{tree-antichain } S \wedge$
 $(B' = B \implies \text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \ S \ B) \leq \text{card } U)$
by (*rule step-scaled-seen-progress*
 $[OF \ D\text{-def insert loop complete } U\text{-def sound-s pre-s reaches-s])$
then have *S-k-cap*: $k * \text{card } S \leq \text{cap}$
by *blast*
from *progress* **have** *anti*: *tree-antichain* S
by *blast*
from *progress* **have** *seen-success*:
 $B' = B \implies \text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \ S \ B) \leq \text{card } U$
by *blast*
show *fp-iter-capped-scan-cost* $k \text{ cap } d \ S \ S \ B + c\text{-insert} \leq$
 $2 * A * \text{card } U$
by (*rule split-range-costed-capped-step-scan-insert-budget-from-scaled-seen-or-threshold*
 $[OF \ \text{degree degree-factor insert-factor insert-scaled-factor}$
 $\text{seen-scaled-factor insert loop } U\text{-def sound-s pre-s reaches-s}$
 $S\text{-k-cap anti k-pos seen-success}])$
qed

theorem *finite-initial-label-split-range-costed-top-level-correct-and-seen-progress-bound*:

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$
and *seen-factor*: $k * \Delta + t \leq 2 * A$
and *source-factor*: $\text{Suc } h \leq 2 * A$
and *k-pos*: $0 < k$
and *source-progress*:
 $\bigwedge l \ d \ S \ B \ d' \ B' \ U \ c. \text{ split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq M\text{-of } l \implies$
 $\text{card } S \leq \text{card } U$
and *step-seen-progress*:
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
 $\text{child-costs } U.$

$D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \text{ cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \implies$
 $\text{split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \text{ cap } l$
 $(\text{find-pivots-label-capped } k \text{ cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq \text{cap} \wedge \text{tree-antichain } S \wedge$
 $(B' = B \longrightarrow$
 $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \ S \ B) \leq \text{card } U)$
and run:
 $\text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } l \ \text{finite-initial-label } \{s\}$
 $\text{Infinity } d' \ \text{Infinity } U \ c$
shows $\text{sssp-correct } d' \wedge$
 $c \leq \text{level-range-cost-bound } (2 * A) \ (R + l * t) \ (2 * l + 1) \ U$
proof
show $\text{sssp-correct } d'$
by $(\text{rule } \text{finite-initial-label-split-range-costed-top-level-correct}$
 $[\text{OF all-reachable run}])$
next
have pre: $\text{bmssp-pre-full } \text{finite-initial-label } \{s\} \ \text{Infinity}$
using $\text{all-reachable } \text{finite-initial-label-source-complete}$
by $(\text{rule } \text{top-bmssp-pre-full})$
have sound: $\text{sound-label } \text{finite-initial-label}$
using $\text{finite-initial-label-sound}[\text{OF all-reachable}]$.
have $S\text{-reaches: } \bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$
using $\text{all-reachable source-in-} V \text{ by blast}$
show $c \leq \text{level-range-cost-bound } (2 * A) \ (R + l * t) \ (2 * l + 1) \ U$
by $(\text{rule } \text{split-range-costed-bmssp-level-bound-from-seen-progress-budgets}$
 $[\text{OF degree degree-factor } R\text{-pos } \text{insert-factor } \text{seen-factor } \text{source-factor } k\text{-pos}$
 $\text{source-progress step-seen-progress run sound pre } S\text{-reaches}])$
qed

theorem $\text{finite-initial-label-split-range-costed-top-level-correct-and-scaled-seen-}$
 progress-bound:
assumes $\text{all-reachable: } \bigwedge v. v \in V \implies \text{reachable } s \ v$
and $\text{degree: edge-outdegree-le } \Delta$
and $\text{degree-factor: } \Delta \leq A$
and $R\text{-pos: } 0 < R$
and $\text{insert-factor: } t \leq A * k$

and *insert-scaled-factor*: $t \leq A\text{-insert} * k$
and *seen-scaled-factor*: $k * \Delta + A\text{-insert} \leq 2 * A$
and *source-factor*: $\text{Suc } h \leq 2 * A$
and *k-pos*: $0 < k$
and *source-progress*:
 $\bigwedge l \, d \, S \, B \, d' \, B' \, U \, c.$
 $\text{split-range-costed-bmssp } \Delta \, M\text{-of } t \, h \, k \, \text{cap } l \, d \, S \, B \, d' \, B' \, U \, c \implies$
 $\text{bmssp-pre-full } d \, S \, B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \, x) \implies$
 $\text{card } S \leq M\text{-of } l \implies$
 $\text{card } S \leq \text{card } U$
and *step-scaled-seen-progress*:
 $\bigwedge l \, d \, S \, B \, D \, c\text{-insert } d' \, a \, \text{betas } bs \, B' \, Us \, U\text{-loop } c\text{-loop}$
 $\text{child-costs } U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \, \text{cap } d \, S \, B)$
 $(\text{find-pivots-pivots-capped } k \, \text{cap } d \, S \, B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \, \text{cap } d \, S \, B) \implies$
 $\text{split-range-costed-partition-loop-state } \Delta \, M\text{-of } t \, h \, k \, \text{cap } l$
 $(\text{find-pivots-label-capped } k \, \text{cap } d \, S \, B)$
 $(\text{find-pivots-pivots-capped } k \, \text{cap } d \, S \, B) \, B \, d' \, D \, a \, \text{betas } bs \, B'$
 $Us \, U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \, B'. \text{find-pivots-label-capped } k \, \text{cap } d \, S \, B \, v = \text{dist } s \, v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \, B'. \text{find-pivots-label-capped } k \, \text{cap } d \, S \, B \, v = \text{dist } s \, v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \, S \, B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \, x) \implies$
 $k * \text{card } S \leq \text{cap} \wedge \text{tree-antichain } S \wedge$
 $(B' = B \longrightarrow$
 $\text{card } (\text{find-pivots-seen-capped } k \, \text{cap } d \, S \, B) \leq \text{card } U)$
and *run*:
 $\text{split-range-costed-bmssp } \Delta \, M\text{-of } t \, h \, k \, \text{cap } l \, \text{finite-initial-label } \{s\}$
 $\text{Infinity } d' \, \text{Infinity } U \, c$
shows $\text{sssp-correct } d' \wedge$
 $c \leq \text{level-range-cost-bound } (2 * A) \, (R + l * t) \, (2 * l + 1) \, U$
proof
show $\text{sssp-correct } d'$
by $(\text{rule } \text{finite-initial-label-split-range-costed-top-level-correct}$
 $[\text{OF all-reachable run}])$
next
have *pre*: $\text{bmssp-pre-full } \text{finite-initial-label } \{s\} \, \text{Infinity}$
using *all-reachable finite-initial-label-source-complete*
by $(\text{rule } \text{top-bmssp-pre-full})$
have *sound*: $\text{sound-label } \text{finite-initial-label}$
using $\text{finite-initial-label-sound}[\text{OF all-reachable}]$.
have *S-reaches*: $\bigwedge x. x \in \{s\} \implies \text{reachable } s \, x$

using *all-reachable source-in-V* **by** *blast*
show $c \leq \text{level-range-cost-bound } (2 * A) (R + l * t) (2 * l + 1) U$
by (*rule split-range-costed-bmssp-level-bound-from-scaled-seen-progress-budgets*
[OF degree degree-factor R-pos insert-factor insert-scaled-factor
seen-scaled-factor source-factor k-pos source-progress
step-scaled-seen-progress run sound pre S-reaches]))
qed

theorem *finite-initial-label-split-range-costed-top-level-correct-and-seen-progress-graph-bound:*

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$
and *seen-factor*: $k * \Delta + t \leq 2 * A$
and *source-factor*: $\text{Suc } h \leq 2 * A$
and *k-pos*: $0 < k$
and *source-progress*:
 $\bigwedge l \ d \ S \ B \ d' \ B' \ U \ c.$
split-range-costed-bmssp $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies$
bmssp-pre-full $d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq M\text{-of } l \implies$
 $\text{card } S \leq \text{card } U$
and *step-seen-progress*:
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
child-costs $U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
partition-initial-insert-cost-bound $c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
split-range-costed-partition-loop-state $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$
complete-on d'
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
sound-label $d \implies$
bmssp-pre-full $d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq \text{cap} \wedge \text{tree-antichain } S \wedge$
 $(B' = B \longrightarrow$
 $\text{card } (\text{find-pivots-seen-capped } k \ \text{cap } d \ S \ B) \leq \text{card } U)$
and *run*:
split-range-costed-bmssp $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ \text{finite-initial-label } \{s\}$

```

      Infinity d' Infinity U c
shows sssp-correct d' ∧
  c ≤ 2 * A * (2 * l + 1) * vertex-count + (R + l * t) * edge-count
proof -
  have combined:
    sssp-correct d' ∧
    c ≤ level-range-cost-bound (2 * A) (R + l * t) (2 * l + 1) U
  by (rule finite-initial-label-split-range-costed-top-level-correct-and-seen-progress-bound
    [OF all-reachable degree degree-factor R-pos insert-factor seen-factor
      source-factor k-pos source-progress step-seen-progress run])
  then have correct: sssp-correct d'
  by blast
  have pre: bmssp-pre-full finite-initial-label {s} Infinity
    using all-reachable finite-initial-label-source-complete
  by (rule top-bmssp-pre-full)
  have sound: sound-label finite-initial-label
    using finite-initial-label-sound[OF all-reachable] .
  have S-reaches: ∧x. x ∈ {s} ⇒ reachable s x
    using all-reachable source-in-V by blast
  have post-full:
    bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
  by (rule split-range-costed-bmssp-correct[OF run sound pre S-reaches])
  have post: bmssp-post finite-initial-label {s} Infinity d' Infinity U
    using bmssp-post-full-imp-post[OF post-full] .
  have U-V: U = V
    using post bound-tree-source-infinity[OF all-reachable]
  unfolding bmssp-post-def by auto
  have cost-U:
    c ≤ level-range-cost-bound (2 * A) (R + l * t) (2 * l + 1) U
  using combined by blast
  have graph-bound:
    level-range-cost-bound (2 * A) (R + l * t) (2 * l + 1) U
    ≤ 2 * A * (2 * l + 1) * vertex-count + (R + l * t) * edge-count
proof -
  have U-card: card U = vertex-count
    unfolding U-V vertex-count-def by simp
  have out-le: card (outgoing-edges U) ≤ edge-count
    by (rule edge-count-outgoing-bound)
  have out-term:
    (R + l * t) * card (outgoing-edges U) ≤ (R + l * t) * edge-count
  using out-le by simp
  show ?thesis
    unfolding level-range-cost-bound-def
  using U-card out-term by (simp add: algebra-simps)
qed
show ?thesis
  using correct cost-U graph-bound by linarith
qed

```

theorem *finite-initial-label-split-range-costed-top-level-correct-and-scaled-seen-progress-graph-bound:*

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$
and *insert-scaled-factor*: $t \leq A\text{-insert} * k$
and *seen-scaled-factor*: $k * \Delta + A\text{-insert} \leq 2 * A$
and *source-factor*: $\text{Suc } h \leq 2 * A$
and *k-pos*: $0 < k$
and *source-progress*:
 $\bigwedge l \ d \ S \ B \ d' \ B' \ U \ c.$
split-range-costed-bmssp $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies$
bmssp-pre-full $d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq M\text{-of } l \implies$
 $\text{card } S \leq \text{card } U$
and *step-scaled-seen-progress*:
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
child-costs $U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
partition-initial-insert-cost-bound $c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
split-range-costed-partition-loop-state $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$
complete-on d'
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
sound-label $d \implies$
bmssp-pre-full $d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $k * \text{card } S \leq \text{cap} \wedge \text{tree-antichain } S \wedge$
 $(B' = B \longrightarrow$
 $\text{card } (\text{find-pivots-seen-capped } k \ \text{cap } d \ S \ B) \leq \text{card } U)$
and *run*:
split-range-costed-bmssp $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ \text{finite-initial-label } \{s\}$
Infinity $d' \ \text{Infinity } U \ c$
shows *sssp-correct* $d' \wedge$
 $c \leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$
proof –
have *combined*:
sssp-correct $d' \wedge$


```

    c ≤ level-range-cost-bound (2 * A) (R + l * t) (2 * l + 1) U
  by (rule finite-initial-label-split-range-costed-top-level-correct-and-scaled-seen-
progress-bound
    [OF all-reachable degree degree-factor R-pos insert-factor insert-scaled-factor
seen-scaled-factor source-factor k-pos source-progress
step-scaled-seen-progress run])
  then have correct: sssp-correct d'
    by blast
  have pre: bmssp-pre-full finite-initial-label {s} Infinity
    using all-reachable finite-initial-label-source-complete
  by (rule top-bmssp-pre-full)
  have sound: sound-label finite-initial-label
    using finite-initial-label-sound[OF all-reachable] .
  have S-reaches:  $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$ 
    using all-reachable source-in-V by blast
  have post-full:
    bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
  by (rule split-range-costed-bmssp-correct[OF run sound pre S-reaches])
  have post: bmssp-post finite-initial-label {s} Infinity d' Infinity U
    using bmssp-post-full-imp-post[OF post-full] .
  have U-V: U = V
    using post bound-tree-source-infinity[OF all-reachable]
  unfolding bmssp-post-def by auto
  have cost-U:
    c ≤ level-range-cost-bound (2 * A) (R + l * t) (2 * l + 1) U
  using combined by blast
  have graph-bound:
    level-range-cost-bound (2 * A) (R + l * t) (2 * l + 1) U
    ≤ 2 * A * (2 * l + 1) * vertex-count + (R + l * t) * edge-count
  proof -
    have U-card: card U = vertex-count
      unfolding U-V vertex-count-def by simp
    have out-le: card (outgoing-edges U) ≤ edge-count
      by (rule edge-count-outgoing-bound)
    have out-term:
      (R + l * t) * card (outgoing-edges U) ≤ (R + l * t) * edge-count
    using out-le by simp
    show ?thesis
      unfolding level-range-cost-bound-def
      using U-card out-term by (simp add: algebra-simps)
  qed
  show ?thesis
    using correct cost-U graph-bound by linarith
  qed

theorem range-costed-partition-loop-state-cost-bound:
  assumes run:
    range-costed-partition-loop-state  $\Delta$  M-of t k cap l d P B d' D a
    betas bs B' Us U c child-costs

```

shows $c \leq \text{sum-list child-costs} + (M\text{-of } l) * \text{length child-costs} +$
 $t * (\text{sum-list}$
 $(\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) (\text{range-tree-child-list } P \ a \ bs)) +$
 $(M\text{-of } l) * \text{length child-costs})$
using *run*
by (*induction*)
 $(\text{auto simp: pull-separates-def partition-pull-cost-bound-def}$
 $\text{partition-batch-cost-bound-def algebra-simps}$
 $\text{dest: range-costed-pull-cost-le-M range-costed-batch-cost-le})$

theorem *range-costed-partition-loop-state-cost-bound-by-child-sources:*
 $\text{range-costed-partition-loop-state } \Delta \ M\text{-of } t \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs} \implies$
 $\exists \text{ child-sources.}$
 $\text{length child-sources} = \text{length } bs \wedge$
 list-all2
 $(\lambda S\text{-child } U\text{-child. } \exists c\text{-child } B\text{-child } d\text{-child } B\text{-child'.$
 $\text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child' } U\text{-child } c\text{-child} \wedge$
 $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child} \wedge$
 $(\forall x \in S\text{-child. reachable } s \ x) \wedge$
 $\text{card } S\text{-child} \leq M\text{-of } l)$
 $\text{child-sources } (\text{range-tree-child-list } P \ a \ bs) \wedge$
 $c \leq \text{sum-list child-costs} + \text{sum-list } (\text{map card child-sources}) +$
 $t * (\text{sum-list}$
 $(\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) (\text{range-tree-child-list } P \ a \ bs)) +$
 $\text{sum-list } (\text{map card child-sources}))$

and *range-costed-bmssp-child-sources-trivial:*
 $\text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies \text{True}$
proof (*induction rule:*
 $\text{range-costed-partition-loop-state-range-costed-bmssp.inducts})$
case (*Range-State-Done* $D \ a \ B \ d' \ P \ \Delta \ M\text{-of } t \ k \ \text{cap } l \ d$)
show ?case
by (*intro exI[of -] simp*)
next
case (*Range-State-Stop* $a \ B \ d' \ P \ k \ \text{cap } \Delta \ M\text{-of } t \ l \ d \ D$)
show ?case
by (*intro exI[of -] simp*)
next
case (*Range-State-Step* $D \ M\text{-of } l \ B\text{max} \ S\text{-pull } \text{beta} \ D\text{-pull } B \ d \ a \ b \ B'$
 $d' \ P \ k \ \text{cap } \Delta \ t \ d\text{-child } U\text{-child } c\text{-child } \text{batch } D\text{-next } c\text{-pull}$
 $c\text{-batch } \text{betas } bs \ Us\text{-tail } U\text{-tail } c\text{-tail } \text{child-costs-tail } c$)
obtain *child-sources-tail* **where** *len-tail:*
 $\text{length child-sources-tail} = \text{length } bs$
and *sources-tail:*
 list-all2
 $(\lambda S\text{-child } U\text{-child. } \exists c\text{-child } B\text{-child } d\text{-child } B\text{-child'.$
 $\text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child' } U\text{-child } c\text{-child} \wedge$

```

    bmssp-pre-full d S-child B-child  $\wedge$ 
    ( $\forall x \in S\text{-child}. \text{reachable } s \ x$ )  $\wedge$ 
    card S-child  $\leq$  M-of l
    child-sources-tail (range-tree-child-list P b bs)
and tail-cost:
    c-tail  $\leq$ 
    sum-list child-costs-tail + sum-list (map card child-sources-tail) +
    t * (sum-list
    (map ( $\lambda X.$  card (outgoing-edges X)) (range-tree-child-list P b bs)) +
    sum-list (map card child-sources-tail))
using Range-State-Step.IH(11) by blast
have card-pull: card S-pull  $\leq$  M-of l
using Range-State-Step unfolding pull-separates-def by blast
have pull-cost: c-pull  $\leq$  card S-pull
using Range-State-Step
unfolding partition-pull-cost-bound-def by blast
have batch-cost:
    c-batch  $\leq$  t * (card (outgoing-edges U-child) + card S-pull)
by (rule range-costed-batch-cost-le
    [where d = d and S = S-pull and d-child = d-child and b = b
    and beta = beta and batch = batch and M = card S-pull])
    (use Range-State-Step in auto)
let ?child-sources = S-pull # child-sources-tail
have len: length ?child-sources = length (b # bs)
using len-tail by simp
have head-source:
     $\exists c\text{-child}' B\text{-child}' d\text{-child}' B\text{-child}'.$ 
    range-costed-bmssp  $\Delta$  M-of t k cap l d S-pull B-child
    d-child' B-child' U-child c-child'  $\wedge$ 
    bmssp-pre-full d S-pull B-child  $\wedge$ 
    ( $\forall x \in S\text{-pull}. \text{reachable } s \ x$ )  $\wedge$ 
    card S-pull  $\leq$  M-of l
using Range-State-Step card-pull by blast
have sources:
    list-all2
    ( $\lambda S\text{-child } U\text{-child}. \exists c\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
    range-costed-bmssp  $\Delta$  M-of t k cap l d S-child B-child
    d-child B-child' U-child c-child  $\wedge$ 
    bmssp-pre-full d S-child B-child  $\wedge$ 
    ( $\forall x \in S\text{-child}. \text{reachable } s \ x$ )  $\wedge$ 
    card S-child  $\leq$  M-of l
    ?child-sources (range-tree-child-list P a (b # bs))
using head-source sources-tail Range-State-Step by simp
have cost:
    c  $\leq$  sum-list (c-child # child-costs-tail) +
    sum-list (map card ?child-sources) +
    t * (sum-list
    (map ( $\lambda X.$  card (outgoing-edges X))
    (range-tree-child-list P a (b # bs)))) +

```

```

      sum-list (map card ?child-sources))
proof -
  have c-eq:  $c = c\text{-pull} + c\text{-batch} + c\text{-child} + c\text{-tail}$ 
  using Range-State-Step by blast
  have  $c\text{-pull} + c\text{-batch} + c\text{-child} + c\text{-tail} \leq$ 
    card  $S\text{-pull} + t * (\text{card } (\text{outgoing-edges } U\text{-child}) + \text{card } S\text{-pull}) +$ 
     $c\text{-child} +$ 
    (sum-list child-costs-tail + sum-list (map card child-sources-tail) +
     t * (sum-list
       (map ( $\lambda X.$  card (outgoing-edges  $X$ )) (range-tree-child-list  $P$   $b$   $bs$ )) +
       sum-list (map card child-sources-tail))))
  using pull-cost batch-cost tail-cost by simp
  also have ... =
    sum-list ( $c\text{-child} \# \text{child-costs-tail}$ ) +
    sum-list (map card ?child-sources) +
     $t * (\text{card } (\text{outgoing-edges } U\text{-child}) +$ 
    sum-list
      (map ( $\lambda X.$  card (outgoing-edges  $X$ )) (range-tree-child-list  $P$   $b$   $bs$ )) +
      sum-list (map card ?child-sources))
  by (simp add: algebra-simps)
  also have ... =
    sum-list ( $c\text{-child} \# \text{child-costs-tail}$ ) +
    sum-list (map card ?child-sources) +
     $t * (\text{sum-list}$ 
      (map ( $\lambda X.$  card (outgoing-edges  $X$ ))
        (range-tree-child-list  $P$   $a$  ( $b \# bs$ ))) +
      sum-list (map card ?child-sources))
  using Range-State-Step by simp
  finally show ?thesis
  using c-eq by simp
qed
show ?case
  by (intro exI[of - ?child-sources] conjI)
    (use len sources cost in simp-all)
next
case (Range-Cost-Base  $S$   $x$   $\Delta$   $M\text{-of } t$   $k$  cap  $d$   $B$ )
show ?case
  by simp
next
case (Range-Cost-Step  $D$   $k$  cap  $d$   $S$   $B$   $c\text{-insert } t$   $\Delta$   $M\text{-of } l$   $d'$   $a$  betas  $bs$   $B'$ 
   $Us$   $U\text{-loop } c\text{-loop } \text{child-costs-loop } U$   $c$ )
show ?case
  by simp
qed

theorem range-costed-partition-loop-state-cost-from-child-and-source-bounds:
  assumes run:
    range-costed-partition-loop-state  $\Delta$   $M\text{-of } t$   $k$  cap  $l$   $P$   $B$   $d'$   $D$   $a$ 
    betas  $bs$   $B'$   $Us$   $U$   $c$  child-costs

```

and *child-cost-bounds*:
 $list_all2\ (\lambda c_child\ U_child.$
 $\quad c_child \leq level_range_cost_bound\ A\ R\ L\ U_child)$
 $\quad child_costs\ (range_tree_child_list\ P\ a\ bs)$
and *source-progress*:
 $\wedge S_child\ B_child\ d_child\ B_child'\ U_child\ c_child.$
 $\quad \llbracket range_costed_bmssp\ \Delta\ M_of\ t\ k\ cap\ l\ d\ S_child\ B_child$
 $\quad\quad d_child\ B_child'\ U_child\ c_child;$
 $\quad\quad bmssp_pre_full\ d\ S_child\ B_child;$
 $\quad\quad \wedge x. x \in S_child \implies reachable\ s\ x;$
 $\quad\quad card\ S_child \leq M_of\ l \rrbracket$
 $\implies card\ S_child \leq card\ U_child$
shows $c \leq$
 $A * L * card\ (range_tree_chain\ P\ a\ bs\ B') +$
 $(R + t) * card\ (outgoing_edges\ (range_tree_chain\ P\ a\ bs\ B')) +$
 $Suc\ t * card\ (range_tree_chain\ P\ a\ bs\ B')$
proof –
obtain *child-sources* **where** *len-sources*:
 $length\ child_sources = length\ bs$
and *sources*:
 $list_all2$
 $\quad (\lambda S_child\ U_child. \exists c_child\ B_child\ d_child\ B_child'.$
 $\quad\quad range_costed_bmssp\ \Delta\ M_of\ t\ k\ cap\ l\ d\ S_child\ B_child$
 $\quad\quad d_child\ B_child'\ U_child\ c_child \wedge$
 $\quad\quad bmssp_pre_full\ d\ S_child\ B_child \wedge$
 $\quad\quad (\forall x \in S_child. reachable\ s\ x) \wedge$
 $\quad\quad card\ S_child \leq M_of\ l)$
 $\quad child_sources\ (range_tree_child_list\ P\ a\ bs)$
and *cost*:
 $c \leq sum_list\ child_costs + sum_list\ (map\ card\ child_sources) +$
 $t * (sum_list$
 $\quad (map\ (\lambda X. card\ (outgoing_edges\ X))\ (range_tree_child_list\ P\ a\ bs)) +$
 $\quad sum_list\ (map\ card\ child_sources))$
using *range-costed-partition-loop-state-cost-bound-by-child-sources*[*OF run*]
by *blast*
have *mono*: *nondecreasing-from* *a bs*
by (*rule* *range-costed-partition-loop-state-mono*[*OF run*])
have *source-dom*:
 $list_all2\ (\lambda S_child\ U_child. card\ S_child \leq card\ U_child)$
 $\quad child_sources\ (range_tree_child_list\ P\ a\ bs)$
using *sources*
proof (*induction rule*: *list-all2-induct*)
case *Nil*
then show *?case* **by** *simp*
next
case (*Cons* *S-child* *child-sources* *U-child* *child-sets*)
obtain *c-child* *B-child* *d-child* *B-child'* **where** *child*:
 $range_costed_bmssp\ \Delta\ M_of\ t\ k\ cap\ l\ d\ S_child\ B_child$
 $\quad d_child\ B_child'\ U_child\ c_child$

```

and child-pre: bmssp-pre-full d S-child B-child
and child-reaches:  $\forall x \in S\text{-child}. \text{reachable } s \ x$ 
and child-card:  $\text{card } S\text{-child} \leq M\text{-of } l$ 
using Cons.hyps by blast
have head:  $\text{card } S\text{-child} \leq \text{card } U\text{-child}$ 
by (rule source-progress[OF child child-pre - child-card])
  (use child-reaches in blast)
show ?case
using head Cons.IH Cons.hyps by simp
qed
have source-sum:
  sum-list (map card child-sources)  $\leq$ 
    card (range-tree-chain P a bs B')
by (rule sum-card-dominated-by-range-tree-child-list-le-chain
  [OF mono source-dom])
have child-sum:
  sum-list child-costs  $\leq$ 
    A * L * card (range-tree-chain P a bs B') +
    R * card (outgoing-edges (range-tree-chain P a bs B'))
by (rule child-costs-le-level-range-child-list-bound
  [OF mono child-cost-bounds])
have edge-sum:
  sum-list
    (map ( $\lambda X. \text{card } (\text{outgoing-edges } X)$ ) (range-tree-child-list P a bs))  $\leq$ 
    card (outgoing-edges (range-tree-chain P a bs B'))
by (rule card-outgoing-edges-range-tree-child-list-le-chain[OF mono])
have c  $\leq$ 
  A * L * card (range-tree-chain P a bs B') +
  R * card (outgoing-edges (range-tree-chain P a bs B')) +
  sum-list (map card child-sources) +
  t * (card (outgoing-edges (range-tree-chain P a bs B')) +
    sum-list (map card child-sources))
proof –
have tail:
  t * (sum-list
    (map ( $\lambda X. \text{card } (\text{outgoing-edges } X)$ ) (range-tree-child-list P a bs)) +
    sum-list (map card child-sources))  $\leq$ 
  t * (card (outgoing-edges (range-tree-chain P a bs B')) +
    sum-list (map card child-sources))
using edge-sum by simp
have sum-list child-costs + sum-list (map card child-sources) +
  t * (sum-list
    (map ( $\lambda X. \text{card } (\text{outgoing-edges } X)$ ) (range-tree-child-list P a bs)) +
    sum-list (map card child-sources))  $\leq$ 
  A * L * card (range-tree-chain P a bs B') +
  R * card (outgoing-edges (range-tree-chain P a bs B')) +
  sum-list (map card child-sources) +
  t * (card (outgoing-edges (range-tree-chain P a bs B')) +
    sum-list (map card child-sources))

```

```

    using child-sum tail by linarith
  then show ?thesis
    using cost by linarith
qed
also have ... ≤
  A * L * card (range-tree-chain P a bs B') +
  (R + t) * card (outgoing-edges (range-tree-chain P a bs B')) +
  Suc t * card (range-tree-chain P a bs B')
proof -
  have source-part:
    sum-list (map card child-sources) +
    t * sum-list (map card child-sources) ≤
    Suc t * card (range-tree-chain P a bs B')
  proof -
    have t-part:
      t * sum-list (map card child-sources) ≤
      t * card (range-tree-chain P a bs B')
    using source-sum by simp
    have sum-list (map card child-sources) +
      t * sum-list (map card child-sources) ≤
      card (range-tree-chain P a bs B') +
      t * card (range-tree-chain P a bs B')
    using source-sum t-part by linarith
    then show ?thesis
      by (simp add: algebra-simps)
  qed
show ?thesis
  using source-part by (simp add: algebra-simps)
qed
finally show ?thesis .
qed

theorem range-costed-partition-loop-state-cost-from-child-cost-bounds:
assumes run:
  range-costed-partition-loop-state Δ M-of t k cap l d P B d' D a
  betas bs B' Us U c child-costs
and child-cost-bounds:
  list-all2 (λc-child U-child.
    c-child ≤ level-range-cost-bound A R L U-child)
  child-costs (range-tree-child-list P a bs)
shows c ≤
  A * L * card (range-tree-chain P a bs B') +
  (R + t) * card (outgoing-edges (range-tree-chain P a bs B')) +
  (Suc t) * (M-of l) * length bs
proof -
  have child-len: length child-costs = length bs
  by (rule range-costed-partition-loop-state-child-call-ranges(1)[OF run])
  have cost:
    c ≤ sum-list child-costs + (M-of l) * length child-costs +

```

```

    t * (sum-list
      (map (λX. card (outgoing-edges X)) (range-tree-child-list P a bs)) +
      (M-of l) * length child-costs)
  by (rule range-costed-partition-loop-state-cost-bound[OF run])
have mono: nondecreasing-from a bs
  by (rule range-costed-partition-loop-state-mono[OF run])
have child-sum:
  sum-list child-costs ≤
    A * L * card (range-tree-chain P a bs B') +
    R * card (outgoing-edges (range-tree-chain P a bs B'))
  by (rule child-costs-le-level-range-child-list-bound
    [OF mono child-cost-bounds])
have edge-sum:
  sum-list
    (map (λX. card (outgoing-edges X)) (range-tree-child-list P a bs)) ≤
    card (outgoing-edges (range-tree-chain P a bs B'))
  by (rule card-outgoing-edges-range-tree-child-list-le-chain[OF mono])
have c ≤
  A * L * card (range-tree-chain P a bs B') +
  R * card (outgoing-edges (range-tree-chain P a bs B')) +
  (M-of l) * length bs +
  t * (card (outgoing-edges (range-tree-chain P a bs B')) +
    (M-of l) * length bs)
proof -
  have middle:
    (M-of l) * length child-costs = (M-of l) * length bs
    using child-len by simp
  have tail:
    t * (sum-list
      (map (λX. card (outgoing-edges X)) (range-tree-child-list P a bs)) +
      (M-of l) * length child-costs) ≤
    t * (card (outgoing-edges (range-tree-chain P a bs B')) +
      (M-of l) * length bs)
  proof -
    have sum-list
      (map (λX. card (outgoing-edges X)) (range-tree-child-list P a bs)) +
      (M-of l) * length child-costs ≤
      card (outgoing-edges (range-tree-chain P a bs B')) +
      (M-of l) * length bs
      using edge-sum middle by linarith
    then show ?thesis
      by simp
  qed
have sum-list child-costs + (M-of l) * length child-costs +
  t * (sum-list
    (map (λX. card (outgoing-edges X)) (range-tree-child-list P a bs)) +
    (M-of l) * length child-costs) ≤
  A * L * card (range-tree-chain P a bs B') +
  R * card (outgoing-edges (range-tree-chain P a bs B')) +

```



```

      (M-of l) * length bs +
      t * (card (outgoing-edges (range-tree-chain P a bs B')) +
      (M-of l) * length bs)
    using child-sum middle tail by linarith
  then show ?thesis
    using cost by linarith
qed
also have ... =
  A * L * card (range-tree-chain P a bs B') +
  (R + t) * card (outgoing-edges (range-tree-chain P a bs B')) +
  (Suc t) * (M-of l) * length bs
  by (simp add: algebra-simps)
finally show ?thesis .
qed

theorem range-costed-partition-loop-state-child-cost-bounds:
  assumes run:
    range-costed-partition-loop-state  $\Delta$  M-of t k cap l d P B d' D a
    betas bs B' Us U c child-costs
  and child-bound:
     $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
     $\llbracket \text{range-costed-bmssp } \Delta \text{ M-of t k cap l d } S\text{-child } B\text{-child}$ 
       $d\text{-child } B\text{-child}' \text{ } U\text{-child } c\text{-child};$ 
       $\text{bmssp-pre-full } d \text{ } S\text{-child } B\text{-child};$ 
       $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \text{ } x;$ 
       $\text{card } S\text{-child} \leq M\text{-of } l \rrbracket$ 
     $\implies c\text{-child} \leq \text{level-range-cost-bound } A \text{ } R \text{ } L \text{ } U\text{-child}$ 
  shows list-all2 ( $\lambda c\text{-child } U\text{-child}.$ 
     $c\text{-child} \leq \text{level-range-cost-bound } A \text{ } R \text{ } L \text{ } U\text{-child}$ )
    child-costs (range-tree-child-list P a bs)
proof –
  have calls:
    list-all2
      ( $\lambda c\text{-child } U\text{-child}.$   $\exists S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
        range-costed-bmssp  $\Delta$  M-of t k cap l d S-child B-child
          d-child B-child' U-child c-child  $\wedge$ 
          bmssp-pre-full d S-child B-child  $\wedge$ 
          ( $\forall x \in S\text{-child}.$  reachable s x)  $\wedge$ 
          card S-child  $\leq$  M-of l)
        child-costs (range-tree-child-list P a bs)
  by (rule range-costed-partition-loop-state-child-call-ranges(2))[OF run]
show ?thesis
  using calls
proof (induction rule: list-all2-induct)
  case Nil
  then show ?case by simp
next
  case (Cons c-child child-costs U-child child-sets)
  obtain S-child B-child d-child B-child' where child:

```

$\text{range-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } k \text{ } \text{cap } l \text{ } d \text{ } S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' \text{ } U\text{-child } c\text{-child}$
and $\text{child-pre: } \text{bmssp-pre-full } d \text{ } S\text{-child } B\text{-child}$
and $\text{child-reaches: } \forall x \in S\text{-child. } \text{reachable } s \text{ } x$
and $\text{child-card: } \text{card } S\text{-child} \leq M\text{-of } l$
using Cons.hyps **by** blast
have head:
 $c\text{-child} \leq \text{level-range-cost-bound } A \text{ } R \text{ } L \text{ } U\text{-child}$
by $(\text{rule } \text{child-bound}[\text{OF } \text{child } \text{child-pre} - \text{child-card}])$
 $(\text{use } \text{child-reaches} \text{ in } \text{blast})$
show $?case$
using $\text{head Cons.IH Cons.hyps}$ **by** simp
qed
qed

theorem $\text{range-costed-partition-loop-state-cost-from-child-bound:}$
assumes run:
 $\text{range-costed-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } k \text{ } \text{cap } l \text{ } d \text{ } P \text{ } B \text{ } d' \text{ } D \text{ } a$
 $\text{betas } bs \text{ } B' \text{ } Us \text{ } U \text{ } c \text{ } \text{child-costs}$
and child-bound:
 $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\llbracket \text{range-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } k \text{ } \text{cap } l \text{ } d \text{ } S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' \text{ } U\text{-child } c\text{-child};$
 $\text{bmssp-pre-full } d \text{ } S\text{-child } B\text{-child};$
 $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \text{ } x;$
 $\text{card } S\text{-child} \leq M\text{-of } l \rrbracket$
 $\implies c\text{-child} \leq \text{level-range-cost-bound } A \text{ } R \text{ } L \text{ } U\text{-child}$
shows $c \leq$
 $A * L * \text{card } (\text{range-tree-chain } P \text{ } a \text{ } bs \text{ } B') +$
 $(R + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \text{ } a \text{ } bs \text{ } B')) +$
 $(\text{Suc } t) * (M\text{-of } l) * \text{length } bs$

proof –
have $\text{child-cost-bounds:}$
 $\text{list-all2 } (\lambda c\text{-child } U\text{-child.}$
 $c\text{-child} \leq \text{level-range-cost-bound } A \text{ } R \text{ } L \text{ } U\text{-child})$
 $\text{child-costs } (\text{range-tree-child-list } P \text{ } a \text{ } bs)$
by $(\text{rule } \text{range-costed-partition-loop-state-child-cost-bounds}$
 $[\text{OF } \text{run } \text{child-bound}])$
show $?thesis$
by $(\text{rule } \text{range-costed-partition-loop-state-cost-from-child-cost-bounds}$
 $[\text{OF } \text{run } \text{child-cost-bounds}])$
qed

theorem $\text{range-costed-partition-loop-state-trace:}$
 $\text{range-costed-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } k \text{ } \text{cap } l \text{ } d \text{ } P \text{ } B \text{ } d' \text{ } D \text{ } a$
 $\text{betas } bs \text{ } B' \text{ } Us \text{ } U \text{ } c \text{ } \text{child-costs} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \text{ } P \text{ } B \implies$
 $(\bigwedge x. x \in P \implies \text{reachable } s \text{ } x) \implies$

$concrete-partition-loop-trace\ P\ B\ a\ bs\ d'\ B'\ Us\ U$
and $range-costed-bmssp-correct$:
 $range-costed-bmssp\ \Delta\ M-of\ t\ k\ cap\ l\ d\ S\ B\ d'\ B'\ U\ c \implies$
 $sound-label\ d \implies$
 $bmssp-pre-full\ d\ S\ B \implies$
 $(\bigwedge x. x \in S \implies reachable\ s\ x) \implies$
 $bmssp-post-full\ d\ S\ B\ d'\ B'\ U$
proof (induction rule:
 $range-costed-partition-loop-state-range-costed-bmssp.inducts$)
case ($Range-State-Done\ D\ a\ B\ d'\ P\ \Delta\ M-of\ t\ k\ cap\ l\ d$)
have $a-le$: $bound-le\ (Fin\ a)\ B$
using $Range-State-Done$ **by** $blast$
have $complete-B$: $complete-on\ d'\ (bound-tree\ P\ B)$
using $Range-State-Done$ **by** $blast$
have $lower$: $complete-on\ d'\ (bound-tree\ P\ (Fin\ a))$
by ($rule\ complete-on-subset[OF\ complete-B]$)
 $(rule\ bound-tree-bound-mono[OF\ a-le])$
have $range-complete$: $complete-on\ d'\ (range-tree\ P\ a\ B)$
by ($rule\ complete-on-subset[OF\ complete-B]$)
 $(rule\ range-tree-subset-bound-tree)$
have $B-le$: $bound-le\ B\ B$
by ($cases\ B$) $simp-all$
show $?case$
unfolding $concrete-partition-loop-trace-def$
using $B-le\ a-le\ lower\ range-complete$ **by** $simp$
next
case ($Range-State-Stop\ a\ B\ d'\ P\ k\ cap\ \Delta\ M-of\ t\ l\ d\ D$)
then show $?case$
unfolding $concrete-partition-loop-trace-def\ complete-on-def$ **by** $simp$
next
case $Range-State-Step$
then show $?case$
unfolding $concrete-partition-loop-trace-def\ bmssp-post-full-def$
 $complete-preserved-def$
by ($auto\ split$: $bound.splits$)
next
case ($Range-Cost-Base\ S\ x\ \Delta\ M-of\ t\ k\ cap\ d\ B$)
have $post$:
 $bmssp-post\ d\ S\ B$
 $(\lambda v. if\ v \in base-case-vertices\ k\ x\ B\ then\ dist\ s\ v\ else\ d\ v)$
 $(base-case-bound\ k\ x\ B)$
 $(base-case-vertices\ k\ x\ B)$
using $base-case-result-bmssp-post[OF\ Range-Cost-Base.hyps,$
 $where\ k = k\ and\ B = B\ and\ d = d]$
unfolding $base-case-result-def$ **by** $simp$
then show $?case$
by ($rule\ bmssp-post-imp-post-full$)
next
case ($Range-Cost-Step\ D\ k\ cap\ d\ S\ B\ c-insert\ t\ \Delta\ M-of\ l\ d'\ a$

```

    betas bs B' Us U-loop c-loop child-costs-loop U c)
let ?d-fp = find-pivots-label-capped k cap d S B
let ?P = find-pivots-pivots-capped k cap d S B
let ?W = {v ∈ bound-tree S B'. ?d-fp v = dist s v}
have sound-fp: sound-label ?d-fp
    unfolding find-pivots-label-capped-def
    by (rule fp-iter-capped-label-sound[OF Range-Cost-Step.premis(1)
        Range-Cost-Step.premis(3)])
have pivot-pre: bmssp-pre-full ?d-fp ?P B
    using find-pivots-capped-establishes-pivot-pre-concrete
        [OF Range-Cost-Step.premis(1) Range-Cost-Step.premis(2)
        Range-Cost-Step.premis(3)] .
have P-subset-S: ?P ⊆ S
    unfolding find-pivots-pivots-capped-def by auto
have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
    using P-subset-S Range-Cost-Step.premis(3) by blast
have trace:
    concrete-partition-loop-trace ?P B a bs d' B' Us U-loop
    using Range-Cost-Step.IH sound-fp pivot-pre P-reaches by blast
have loop-post: bmssp-post-full ?d-fp ?P B d' B' U-loop
    by (rule concrete-partition-loop-trace-post[OF trace])
have le: bound-le B' B
    using loop-post unfolding bmssp-post-full-def by blast
have U-tree: U = bound-tree S B'
proof –
    have U-loop ∪ ?W = bound-tree S B'
        by (rule concrete-capped-find-pivots-final-tree-assembly
            [OF Range-Cost-Step.premis(1) Range-Cost-Step.premis(2)
            Range-Cost-Step.premis(3) loop-post])
    then show ?thesis
        using Range-Cost-Step(6) by simp
qed
have loop-complete: complete-on d' U-loop
    using loop-post unfolding bmssp-post-full-def by blast
have U-complete: complete-on d' U
    using complete-on-Un[OF loop-complete Range-Cost-Step(5)]
        Range-Cost-Step(6) by simp
show ?case
    using le U-tree U-complete unfolding bmssp-post-full-def by blast
qed

```

theorem range-costed-bmssp-source-card-le-if-label-below-output:

assumes run:

```

    range-costed-bmssp Δ M-of t k cap l d S B d' B' U c
and sound: sound-label d
and pre: bmssp-pre-full d S B
and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
and complete: complete-on d S
and below:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B'$ 

```

shows $\text{card } S \leq \text{card } U$
proof –
 have $\text{post}: \text{bmssp-post-full } d \ S \ B \ d' \ B' \ U$
 by (rule $\text{range-costed-bmssp-correct}[OF \ \text{run} \ \text{sound} \ \text{pre} \ S\text{-reaches}]$)
show $?thesis$
 by (rule $\text{bmssp-post-full-source-card-le-if-label-below-output}$
 $[OF \ \text{post} \ \text{complete} \ S\text{-reaches} \ \text{below}]$)
qed

theorem $\text{range-costed-bmssp-source-card-le-if-sound-label-below-output}$:
 assumes run :
 $\text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c$
 and sound : $\text{sound-label } d$
 and pre : $\text{bmssp-pre-full } d \ S \ B$
 and $S\text{-reaches}$: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
 and below : $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B'$
shows $\text{card } S \leq \text{card } U$
proof –
 have $\text{post}: \text{bmssp-post-full } d \ S \ B \ d' \ B' \ U$
 by (rule $\text{range-costed-bmssp-correct}[OF \ \text{run} \ \text{sound} \ \text{pre} \ S\text{-reaches}]$)
 have $U\text{-eq}: U = \text{bound-tree } S \ B'$
 using post **unfolding** $\text{bmssp-post-full-def}$ **by** blast
 have $S\text{-subset-}U: S \subseteq U$
proof
 fix x
 assume $xS: x \in S$
 have $S\text{-subset}: S \subseteq V$
 using pre **unfolding** $\text{bmssp-pre-full-def}$ **by** blast
 have $xV: x \in V$
 using $S\text{-subset } xS$ **by** blast
 have $\text{reach-}x: \text{reachable } s \ x$
 by (rule $S\text{-reaches}[OF \ xS]$)
 have $\text{dist-le}: \text{dist } s \ x \leq d \ x$
 using $\text{sound } xV \ \text{reach-}x$ **unfolding** sound-label-def **by** blast
 have $\text{below-dist}: \text{below-bound } (d \ s \ x) \ B'$
 using $\text{below-bound-le-trans}[OF \ \text{dist-le} \ \text{below}[OF \ xS]]$.
show $x \in U$
unfolding $U\text{-eq}$
 by (rule $\text{source-in-own-bound-tree}[OF \ xS \ \text{reach-}x \ \text{below-dist}]$)
qed
 have $\text{finite-}U: \text{finite } U$
unfolding $U\text{-eq} \ \text{bound-tree-def}$ **using** $\text{finite-}V$ **by** auto
show $?thesis$
 by (rule $\text{card-mono}[OF \ \text{finite-}U \ S\text{-subset-}U]$)
qed

theorem $\text{range-costed-partition-loop-state-success-or-threshold}$:
 $\text{range-costed-partition-loop-state } \Delta \ M\text{-of } t \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs} \implies$

```

    sound-label  $d \implies$ 
    bmssp-pre-full  $d P B \implies$ 
    ( $\bigwedge x. x \in P \implies \text{reachable } s x$ )  $\implies$ 
     $B' = B \vee k * \text{cap} \leq \text{card } U$ 
and range-costed-bmssp-success-or-threshold-trivial:
    range-costed-bmssp  $\Delta M\text{-of } t k \text{ cap } l d S B d' B' U c \implies \text{True}$ 
proof (induction rule: range-costed-partition-loop-state-range-costed-bmssp.inducts)
    case (Range-State-Done  $D a B d' P \Delta M\text{-of } t k \text{ cap } l d$ )
    then show ?case
    by simp
next
    case (Range-State-Stop  $a B d' P k \text{ cap } \Delta M\text{-of } t l d D$ )
    then show ?case
    by simp
next
    case (Range-State-Step  $D M\text{-of } l B_{\max} S\text{-pull } \beta D\text{-pull } B d a b B\text{-out}$ 
     $d\text{-out } P k \text{ cap } \Delta t d\text{-child } U\text{-child } c\text{-child } \text{batch } D\text{-next } c\text{-pull}$ 
     $c\text{-batch } \beta_{\text{ts}} b_{\text{ts}} U\text{-tail } U\text{-tail } c\text{-tail } \text{child-costs-tail } c$ )
    have tail:
     $B\text{-out} = B \vee k * \text{cap} \leq \text{card } U\text{-tail}$ 
    using Range-State-Step.IH Range-State-Step.prem by blast
    then show ?case
    proof
    assume  $B\text{-out} = B$ 
    then show ?thesis
    by blast
    next
    assume threshold-tail:  $k * \text{cap} \leq \text{card } U\text{-tail}$ 
    have tail-trace:
    concrete-partition-loop-trace  $P B b b_{\text{ts}} d\text{-out } B\text{-out } U\text{-tail } U\text{-tail}$ 
    by (rule range-costed-partition-loop-state-trace)
    (use Range-State-Step in blast)+
    have U-tail-def:
     $U\text{-tail} = \text{bound-tree } P (\text{Fin } b) \cup \bigcup (\text{set } U\text{-tail})$ 
    using tail-trace unfolding concrete-partition-loop-trace-def by blast
    have split-b:
     $\text{bound-tree } P (\text{Fin } b) =$ 
     $\text{bound-tree } P (\text{Fin } a) \cup \text{range-tree } P a (\text{Fin } b)$ 
    using bound-tree-split-at[of  $a \text{ Fin } b P$ ] Range-State-Step
    by simp
    have U-eq-tail:
     $\text{bound-tree } P (\text{Fin } a) \cup$ 
     $\bigcup (\text{set } (\text{range-tree } P a (\text{Fin } b) \# U\text{-tail})) =$ 
     $U\text{-tail}$ 
    using U-tail-def split-b by auto
    show ?thesis
    using threshold-tail U-eq-tail by simp
qed
next

```

```

case (Range-Cost-Base  $S$   $x$   $\Delta$   $M$ -of  $t$   $k$  cap  $d$   $B$ )
then show ?case
  by simp
next
case (Range-Cost-Step  $D$   $k$  cap  $d$   $S$   $B$  c-insert  $t$   $\Delta$   $M$ -of  $l$   $d'$   $a$  betas  $bs$   $B'$ 
   $Us$  U-loop c-loop child-costs-loop  $U$   $c$ )
then show ?case
  by simp
qed

theorem range-costed-bmssp-Suc-success-or-threshold:
assumes run:
  range-costed-bmssp  $\Delta$   $M$ -of  $t$   $k$  cap (Suc  $l$ )  $d$   $S$   $B$   $d'$   $B'$   $U$   $c$ 
and sound: sound-label  $d$ 
and pre: bmssp-pre-full  $d$   $S$   $B$ 
and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
shows  $B' = B \vee k * \text{cap} \leq \text{card } U$ 
using run
proof cases
case (Range-Cost-Step  $D$  c-insert  $a$  betas  $bs$   $Us$  U-loop c-loop child-costs-loop)
let ?d-fp = find-pivots-label-capped  $k$  cap  $d$   $S$   $B$ 
let ?P = find-pivots-pivots-capped  $k$  cap  $d$   $S$   $B$ 
let ?W =  $\{v \in \text{bound-tree } S \ B'. \ ?d\text{-fp } v = \text{dist } s \ v\}$ 
have sound-fp: sound-label ?d-fp
  unfolding find-pivots-label-capped-def
  by (rule fp-iter-capped-label-sound[OF sound S-reaches])
have pivot-pre: bmssp-pre-full ?d-fp ?P  $B$ 
  using find-pivots-capped-establishes-pivot-pre-concrete
  [OF sound pre S-reaches] .
have P-subset-S: ?P  $\subseteq S$ 
  unfolding find-pivots-pivots-capped-def by auto
have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
  using P-subset-S S-reaches by blast
have loop-class:
   $B' = B \vee k * \text{cap} \leq \text{card } U\text{-loop}$ 
  by (rule range-costed-partition-loop-state-success-or-threshold
  [OF Range-Cost-Step(3) sound-fp pivot-pre P-reaches])
have loop-trace:
  concrete-partition-loop-trace ?P  $B$   $a$   $bs$   $d'$   $B'$   $Us$  U-loop
  by (rule range-costed-partition-loop-state-trace
  [OF Range-Cost-Step(3) sound-fp pivot-pre P-reaches])
have loop-post: bmssp-post-full ?d-fp ?P  $B$   $d'$   $B'$  U-loop
  by (rule concrete-partition-loop-trace-post[OF loop-trace])
have finite-U-loop: finite U-loop
  using loop-post unfolding bmssp-post-full-def by simp
show ?thesis
  using loop-class
proof
  assume  $B' = B$ 

```

```

    then show ?thesis
      by blast
  next
    assume threshold-loop:  $k * \text{cap} \leq \text{card } U\text{-loop}$ 
    have U-eq:  $U = U\text{-loop} \cup ?W$ 
      using Range-Cost-Step(5) by simp
    have finite-U:  $\text{finite } U$ 
      unfolding U-eq using finite-U-loop by simp
    have U-loop-subset:  $U\text{-loop} \subseteq U$ 
      unfolding U-eq by blast
    have card U-loop  $\leq \text{card } U$ 
      by (rule card-mono[OF finite-U U-loop-subset])
    then show ?thesis
      using threshold-loop by linarith
  qed
qed

theorem range-costed-bmssp-Suc-source-card-le-from-complete-sources:
  assumes run:
    range-costed-bmssp  $\Delta$  M-of t k cap (Suc l) d S B d' B' U c
  and sound: sound-label d
  and pre: bmssp-pre-full d S B
  and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
  and complete: complete-on d S
  and below:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$ 
  and S-cap:  $\text{card } S \leq \text{cap}$ 
  and k-pos:  $0 < k$ 
  shows  $\text{card } S \leq \text{card } U$ 
proof -
  have call-class:  $B' = B \vee k * \text{cap} \leq \text{card } U$ 
    by (rule range-costed-bmssp-Suc-success-or-threshold
      [OF run sound pre S-reaches])
  then show ?thesis
  proof
    assume B'-eq:  $B' = B$ 
    show ?thesis
    proof (rule range-costed-bmssp-source-card-le-if-label-below-output
      [OF run sound pre S-reaches complete])
      fix x
      assume  $x \in S$ 
      then show below-bound (d x) B'
        using below B'-eq by simp
    qed
  next
    assume threshold:  $k * \text{cap} \leq \text{card } U$ 
    have  $\text{cap} \leq k * \text{cap}$ 
      using k-pos by (cases k) simp-all
    then show ?thesis
      using S-cap threshold by linarith
  qed

```


qed
qed

theorem *range-costed-bmssp-Suc-source-card-le-from-label-below:*

assumes *run:*

range-costed-bmssp Δ *M-of t k cap (Suc l) d S B d' B' U c*

and *sound:* *sound-label d*

and *pre:* *bmssp-pre-full d S B*

and *S-reaches:* $\bigwedge x. x \in S \implies \text{reachable } s \ x$

and *below:* $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$

and *S-cap:* $\text{card } S \leq \text{cap}$

and *k-pos:* $0 < k$

shows *card S ≤ card U*

proof –

have *call-class:* $B' = B \vee k * \text{cap} \leq \text{card } U$

by (*rule range-costed-bmssp-Suc-success-or-threshold*
[*OF run sound pre S-reaches*])

then show *?thesis*

proof

assume *B'-eq:* $B' = B$

show *?thesis*

proof (*rule range-costed-bmssp-source-card-le-if-sound-label-below-output*
[*OF run sound pre S-reaches*])

fix *x*

assume $x \in S$

then show *below-bound (d x) B'*

using *below B'-eq by simp*

qed

next

assume *threshold:* $k * \text{cap} \leq \text{card } U$

have $\text{cap} \leq k * \text{cap}$

using *k-pos by (cases k) simp-all*

then show *?thesis*

using *S-cap threshold by linarith*

qed

qed

theorem *range-costed-capped-step-threshold-if-not-success:*

assumes *loop:*

range-costed-partition-loop-state Δ *M-of t k cap l*

(*find-pivots-label-capped k cap d S B*)

(*find-pivots-pivots-capped k cap d S B*) $B \ d' \ D \ a \ \text{betas } bs \ B'$

Us U-loop c-loop child-costs

and *U-def:*

$U = U\text{-loop} \cup$

$\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\}$

and *sound:* *sound-label d*

and *pre:* *bmssp-pre-full d S B*

and *S-reaches:* $\bigwedge x. x \in S \implies \text{reachable } s \ x$

and *not-success*: $B' \neq B$
shows $k * \text{cap} \leq \text{card } U$
proof –
let $?d\text{-fp} = \text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B$
let $?P = \text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B$
let $?W = \{v \in \text{bound-tree } S \text{ } B'. \text{ } ?d\text{-fp } v = \text{dist } s \text{ } v\}$
have *sound-fp*: *sound-label* $?d\text{-fp}$
unfolding *find-pivots-label-capped-def*
by (*rule fp-iter-capped-label-sound*[*OF* *sound* *S-reaches*])
have *pivot-pre*: *bmssp-pre-full* $?d\text{-fp} \text{ } ?P \text{ } B$
using *find-pivots-capped-establishes-pivot-pre-concrete*
 $[OF \text{ } \text{sound } \text{pre } S\text{-reaches}]$.
have *P-subset-S*: $?P \subseteq S$
unfolding *find-pivots-pivots-capped-def* **by** *auto*
have *P-reaches*: $\bigwedge x. x \in ?P \implies \text{reachable } s \text{ } x$
using *P-subset-S* *S-reaches* **by** *blast*
have *loop-class*:
 $B' = B \vee k * \text{cap} \leq \text{card } U\text{-loop}$
by (*rule range-costed-partition-loop-state-success-or-threshold*
 $[OF \text{ } \text{loop } \text{sound-fp } \text{pivot-pre } P\text{-reaches}]$)
have *threshold-loop*: $k * \text{cap} \leq \text{card } U\text{-loop}$
using *loop-class* *not-success* **by** *blast*
have *trace*: *concrete-partition-loop-trace* $?P \text{ } B \text{ } a \text{ } bs \text{ } d' \text{ } B' \text{ } Us \text{ } U\text{-loop}$
by (*rule range-costed-partition-loop-state-trace*
 $[OF \text{ } \text{loop } \text{sound-fp } \text{pivot-pre } P\text{-reaches}]$)
have *loop-post*: *bmssp-post-full* $?d\text{-fp} \text{ } ?P \text{ } B \text{ } d' \text{ } B' \text{ } U\text{-loop}$
by (*rule concrete-partition-loop-trace-post*[*OF* *trace*])
have *finite-U-loop*: *finite* $U\text{-loop}$
using *loop-post* **unfolding** *bmssp-post-full-def* **by** *simp*
have *finite-U*: *finite* U
unfolding *U-def* **using** *finite-U-loop* **by** *simp*
have *U-loop-subset*: $U\text{-loop} \subseteq U$
unfolding *U-def* **by** *blast*
have $\text{card } U\text{-loop} \leq \text{card } U$
by (*rule card-mono*[*OF* *finite-U* *U-loop-subset*])
then show *?thesis*
using *threshold-loop* **by** *linarith*
qed

theorem *range-costed-capped-step-scan-insert-budget-if-not-success*:
assumes *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *insert-factor*: $t \leq A * k$
and *insert*:
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B)$
and *loop*:
 $\text{range-costed-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } k \text{ cap } l$
 $(\text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B)$

$(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \text{ } B \text{ } d' \text{ } D \text{ } a \text{ } \text{betas } bs \text{ } B'$
 $Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs}$
and $U\text{-def}$:
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{ find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\}$
and sound : $\text{sound-label } d$
and pre : $\text{bmssp-pre-full } d \text{ } S \text{ } B$
and $S\text{-reaches}$: $\bigwedge x. x \in S \implies \text{reachable } s \text{ } x$
and $S\text{-cap}$: $\text{card } S \leq \text{cap}$
and not-success : $B' \neq B$
shows $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $2 * A * \text{card } U$
proof –
have $S\text{-subset}$: $S \subseteq V$
using pre **unfolding** $\text{bmssp-pre-full-def}$ **by** blast
have progress : $k * \text{cap} \leq \text{card } U$
by $(\text{rule range-costed-capped-step-threshold-if-not-success}$
 $[\text{OF loop } U\text{-def sound pre } S\text{-reaches not-success}])$
show $?thesis$
by $(\text{rule find-pivots-scan-and-initial-insert-budget-from-progress}$
 $[\text{OF } S\text{-subset degree } S\text{-cap insert degree-factor insert-factor progress}])$
qed

theorem $\text{range-costed-partition-loop-state-closes-level-bound-general}$:

assumes run :

$\text{range-costed-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } k \text{ cap } l \text{ } d \text{ } P \text{ } B \text{ } d' \text{ } D \text{ } a$
 $\text{betas } bs \text{ } B' \text{ } Us \text{ } U \text{ } c \text{ } \text{child-costs}$

and sound : $\text{sound-label } d$

and pre : $\text{bmssp-pre-full } d \text{ } P \text{ } B$

and $P\text{-reaches}$: $\bigwedge x. x \in P \implies \text{reachable } s \text{ } x$

and child-bound :

$\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$

$\llbracket \text{range-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } k \text{ cap } l \text{ } d \text{ } S\text{-child } B\text{-child}$

$d\text{-child } B\text{-child}' \text{ } U\text{-child } c\text{-child};$

$\text{bmssp-pre-full } d \text{ } S\text{-child } B\text{-child};$

$\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \text{ } x;$

$\text{card } S\text{-child} \leq M\text{-of } l \rrbracket$

$\implies c\text{-child} \leq \text{level-range-cost-bound } A \text{ } R \text{ } L \text{ } U\text{-child}$

and overhead : $(\text{Suc } t) * (M\text{-of } l) * \text{length } bs \leq A * \text{card } U$

shows $c \leq A * \text{Suc } L * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U)$

proof –

have trace : $\text{concrete-partition-loop-trace } P \text{ } B \text{ } a \text{ } bs \text{ } d' \text{ } B' \text{ } Us \text{ } U$

by $(\text{rule range-costed-partition-loop-state-trace}$

$[\text{OF run sound pre } P\text{-reaches}])$

have children :

$\text{list-all2 } (\lambda U \text{ } X. U = X \wedge \text{complete-on } d' \text{ } U) \text{ } Us$

$(\text{range-tree-chain-list } P \text{ } a \text{ } bs \text{ } B')$

using trace **unfolding** $\text{concrete-partition-loop-trace-def}$ **by** blast

have $U\text{-def}$: $U = \text{bound-tree } P \text{ } (Fin \text{ } a) \cup \bigcup (\text{set } Us)$

```

using trace unfolding concrete-partition-loop-trace-def by blast
have Union-eq:
   $\bigcup(\text{set } Us) = \bigcup(\text{set } (\text{range-tree-chain-list } P \ a \ bs \ B'))$ 
using children by (induction rule: list-all2-induct) auto
have U-eq-chain:
   $U = \text{bound-tree } P \ (\text{Fin } a) \cup \text{range-tree-chain } P \ a \ bs \ B'$ 
using U-def Union-eq Union-range-tree-chain-list[of  $P \ a \ bs \ B'$ ] by simp
have finite-U: finite U
using U-eq-chain by simp
have chain-subset:  $\text{range-tree-chain } P \ a \ bs \ B' \subseteq U$ 
using U-eq-chain by blast
have card-chain-le:  $\text{card } (\text{range-tree-chain } P \ a \ bs \ B') \leq \text{card } U$ 
by (rule card-mono[OF finite-U chain-subset])
have outgoing-subset:
   $\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B') \subseteq \text{outgoing-edges } U$ 
by (rule outgoing-edges-mono[OF chain-subset])
have finite-out-U: finite (outgoing-edges U)
by simp
have card-out-le:
   $\text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B')) \leq$ 
   $\text{card } (\text{outgoing-edges } U)$ 
by (rule card-mono[OF finite-out-U outgoing-subset])
have loop:
   $c \leq$ 
   $A * L * \text{card } (\text{range-tree-chain } P \ a \ bs \ B') +$ 
   $(R + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B')) +$ 
   $(\text{Suc } t) * (M\text{-of } l) * \text{length } bs$ 
by (rule range-costed-partition-loop-state-cost-from-child-bound
  [OF run child-bound])
have c ≤
   $A * L * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U) + A * \text{card } U$ 
proof –
have vterm:
   $A * L * \text{card } (\text{range-tree-chain } P \ a \ bs \ B') \leq A * L * \text{card } U$ 
using card-chain-le by simp
have eterm:
   $(R + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B')) \leq$ 
   $(R + t) * \text{card } (\text{outgoing-edges } U)$ 
using card-out-le by simp
have  $A * L * \text{card } (\text{range-tree-chain } P \ a \ bs \ B') +$ 
   $(R + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B')) +$ 
   $(\text{Suc } t) * (M\text{-of } l) * \text{length } bs \leq$ 
   $A * L * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U) + A * \text{card } U$ 
using vterm eterm overhead by linarith
then show ?thesis
using loop by linarith
qed
also have ... =
   $A * \text{Suc } L * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U)$ 

```

by (simp add: algebra-simps)
 finally show ?thesis .
 qed

theorem range-costed-partition-loop-state-closes-level-bound-from-source-progress-general:

assumes run:

range-costed-partition-loop-state Δ M-of t k cap l d P B d' D a
 betas bs B' Us U c child-costs

and sound: sound-label d

and pre: bmssp-pre-full d P B

and P-reaches: $\bigwedge x. x \in P \implies \text{reachable } s \ x$

and child-bound:

$\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\llbracket \text{range-costed-bmssp } \Delta \text{ M-of t k cap l d } S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$
 $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child};$
 $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\text{card } S\text{-child} \leq \text{M-of } l \rrbracket$
 $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$

and source-progress:

$\bigwedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}.$
 $\llbracket \text{range-costed-bmssp } \Delta \text{ M-of t k cap l d } S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$
 $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child};$
 $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\text{card } S\text{-child} \leq \text{M-of } l \rrbracket$
 $\implies \text{card } S\text{-child} \leq \text{card } U\text{-child}$

and source-factor: $\text{Suc } t \leq A$

shows $c \leq A * \text{Suc } L * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U)$

proof –

have trace: concrete-partition-loop-trace P B a bs d' B' Us U

by (rule range-costed-partition-loop-state-trace
 [OF run sound pre P-reaches])

have children:

list-all2 ($\lambda U \ X. U = X \wedge \text{complete-on } d' \ U$) Us
 (range-tree-chain-list P a bs B')

using trace unfolding concrete-partition-loop-trace-def by blast

have U-def: $U = \text{bound-tree } P \ (\text{Fin } a) \cup \bigcup (\text{set } Us)$

using trace unfolding concrete-partition-loop-trace-def by blast

have Union-eq:

$\bigcup (\text{set } Us) = \bigcup (\text{set } (\text{range-tree-chain-list } P \ a \ bs \ B'))$

using children by (induction rule: list-all2-induct) auto

have U-eq-chain:

$U = \text{bound-tree } P \ (\text{Fin } a) \cup \text{range-tree-chain } P \ a \ bs \ B'$

using U-def Union-eq Union-range-tree-chain-list[of P a bs B'] by simp

have finite-U: finite U

using U-eq-chain by simp

have chain-subset: range-tree-chain P a bs B' $\subseteq$ U

```

using U-eq-chain by blast
have card-chain-le:  $\text{card } (\text{range-tree-chain } P \ a \ \text{bs } B') \leq \text{card } U$ 
by (rule card-mono[OF finite-U chain-subset])
have outgoing-subset:
   $\text{outgoing-edges } (\text{range-tree-chain } P \ a \ \text{bs } B') \subseteq \text{outgoing-edges } U$ 
by (rule outgoing-edges-mono[OF chain-subset])
have finite-out-U:  $\text{finite } (\text{outgoing-edges } U)$ 
by simp
have card-out-le:
   $\text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ \text{bs } B')) \leq$ 
   $\text{card } (\text{outgoing-edges } U)$ 
by (rule card-mono[OF finite-out-U outgoing-subset])
have child-cost-bounds:
  list-all2 ( $\lambda c\text{-child } U\text{-child}.$ 
     $c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$ )
  child-costs ( $\text{range-tree-child-list } P \ a \ \text{bs}$ )
by (rule range-costed-partition-loop-state-child-cost-bounds
  [OF run child-bound])
have loop:
   $c \leq$ 
   $A * L * \text{card } (\text{range-tree-chain } P \ a \ \text{bs } B') +$ 
   $(R + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ \text{bs } B')) +$ 
   $\text{Suc } t * \text{card } (\text{range-tree-chain } P \ a \ \text{bs } B')$ 
by (rule range-costed-partition-loop-state-cost-from-child-and-source-bounds
  [OF run child-cost-bounds source-progress])
have  $c \leq$ 
   $A * L * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U) + A * \text{card } U$ 
proof –
  have vterm:
     $A * L * \text{card } (\text{range-tree-chain } P \ a \ \text{bs } B') \leq A * L * \text{card } U$ 
    using card-chain-le by simp
  have eterm:
     $(R + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ \text{bs } B')) \leq$ 
     $(R + t) * \text{card } (\text{outgoing-edges } U)$ 
    using card-out-le by simp
  have source-term:
     $\text{Suc } t * \text{card } (\text{range-tree-chain } P \ a \ \text{bs } B') \leq A * \text{card } U$ 
proof –
  have  $\text{Suc } t * \text{card } (\text{range-tree-chain } P \ a \ \text{bs } B') \leq$ 
   $A * \text{card } (\text{range-tree-chain } P \ a \ \text{bs } B')$ 
  by (rule mult-right-mono[OF source-factor]) simp
  also have  $\dots \leq A * \text{card } U$ 
  using card-chain-le by simp
  finally show ?thesis .
qed
have  $A * L * \text{card } (\text{range-tree-chain } P \ a \ \text{bs } B') +$ 
   $(R + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ \text{bs } B')) +$ 
   $\text{Suc } t * \text{card } (\text{range-tree-chain } P \ a \ \text{bs } B') \leq$ 
   $A * L * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U) + A * \text{card } U$ 

```

```

    using vterm eterm source-term by linarith
  then show ?thesis
    using loop by linarith
qed
also have ... =
  A * Suc L * card U + (R + t) * card (outgoing-edges U)
  by (simp add: algebra-simps)
finally show ?thesis .
qed

theorem range-costed-partition-loop-state-closes-level-bound-from-child-cost-bounds-
general:
  assumes run:
    range-costed-partition-loop-state  $\Delta$  M-of t k cap l d P B d' D a
    betas bs B' Us U c child-costs
  and sound: sound-label d
  and pre: bmssp-pre-full d P B
  and P-reaches:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
  and child-cost-bounds:
    list-all2 ( $\lambda c$ -child U-child.
      c-child  $\leq$  level-range-cost-bound A R L U-child)
    child-costs (range-tree-child-list P a bs)
  and overhead: (Suc t) * (M-of l) * length bs  $\leq$  A * card U
  shows c  $\leq$  A * Suc L * card U + (R + t) * card (outgoing-edges U)
proof -
  have trace: concrete-partition-loop-trace P B a bs d' B' Us U
  by (rule range-costed-partition-loop-state-trace
    [OF run sound pre P-reaches])
  have children:
    list-all2 ( $\lambda U \ X. U = X \wedge \text{complete-on } d' \ U$ ) Us
    (range-tree-chain-list P a bs B')
  using trace unfolding concrete-partition-loop-trace-def by blast
  have U-def:  $U = \text{bound-tree } P \ (\text{Fin } a) \cup \bigcup (\text{set } Us)$ 
  using trace unfolding concrete-partition-loop-trace-def by blast
  have Union-eq:
     $\bigcup (\text{set } Us) = \bigcup (\text{set } (\text{range-tree-chain-list } P \ a \ bs \ B'))$ 
  using children by (induction rule: list-all2-induct) auto
  have U-eq-chain:
     $U = \text{bound-tree } P \ (\text{Fin } a) \cup \text{range-tree-chain } P \ a \ bs \ B'$ 
  using U-def Union-eq Union-range-tree-chain-list[of P a bs B'] by simp
  have finite-U: finite U
  using U-eq-chain by simp
  have chain-subset: range-tree-chain P a bs B'  $\subseteq$  U
  using U-eq-chain by blast
  have card-chain-le: card (range-tree-chain P a bs B')  $\leq$  card U
  by (rule card-mono[OF finite-U chain-subset])
  have outgoing-subset:
    outgoing-edges (range-tree-chain P a bs B')  $\subseteq$  outgoing-edges U
  by (rule outgoing-edges-mono[OF chain-subset])

```

```

have finite-out-U: finite (outgoing-edges U)
  by simp
have card-out-le:
  card (outgoing-edges (range-tree-chain P a bs B')) ≤
    card (outgoing-edges U)
  by (rule card-mono[OF finite-out-U outgoing-subset])
have loop:
  c ≤
    A * L * card (range-tree-chain P a bs B') +
    (R + t) * card (outgoing-edges (range-tree-chain P a bs B')) +
    (Suc t) * (M-of l) * length bs
  by (rule range-costed-partition-loop-state-cost-from-child-cost-bounds
    [OF run child-cost-bounds])
have c ≤
  A * L * card U + (R + t) * card (outgoing-edges U) + A * card U
proof –
  have vterm:
    A * L * card (range-tree-chain P a bs B') ≤ A * L * card U
    using card-chain-le by simp
  have eterm:
    (R + t) * card (outgoing-edges (range-tree-chain P a bs B')) ≤
      (R + t) * card (outgoing-edges U)
    using card-out-le by simp
  have A * L * card (range-tree-chain P a bs B') +
    (R + t) * card (outgoing-edges (range-tree-chain P a bs B')) +
    (Suc t) * (M-of l) * length bs ≤
    A * L * card U + (R + t) * card (outgoing-edges U) + A * card U
    using vterm eterm overhead by linarith
  then show ?thesis
    using loop by linarith
qed
also have ... =
  A * Suc L * card U + (R + t) * card (outgoing-edges U)
  by (simp add: algebra-simps)
finally show ?thesis .
qed

```

```

theorem range-costed-partition-loop-state-closes-level-bound:
assumes run:
  range-costed-partition-loop-state  $\Delta$  M-of t k cap l d P B d' D a
  betas bs B' Us U c child-costs
and sound: sound-label d
and pre: bmssp-pre-full d P B
and P-reaches:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
and child-bound:
   $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
   $\llbracket \text{range-costed-bmssp } \Delta \text{ M-of } t \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$ 
    d-child B-child' U-child c-child;
    bmssp-pre-full d S-child B-child;

```


$\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\text{card } S\text{-child} \leq M\text{-of } l]$
 $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ l \ U\text{-child}$
and *overhead*: $(\text{Suc } t) * (M\text{-of } l) * \text{length } bs \leq A * \text{card } U$
shows $c \leq A * \text{Suc } l * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U)$
proof –
have *trace*: *concrete-partition-loop-trace* $P \ B \ a \ bs \ d' \ B' \ Us \ U$
by (*rule range-costed-partition-loop-state-trace*
 $[OF \ \text{run sound pre } P\text{-reaches}]$)
have *children*:
 $\text{list-all2 } (\lambda U \ X. U = X \wedge \text{complete-on } d' \ U) \ Us$
 $(\text{range-tree-chain-list } P \ a \ bs \ B')$
using *trace unfolding concrete-partition-loop-trace-def* **by** *blast*
have *U-def*: $U = \text{bound-tree } P \ (\text{Fin } a) \cup \bigcup (\text{set } Us)$
using *trace unfolding concrete-partition-loop-trace-def* **by** *blast*
have *Union-eq*:
 $\bigcup (\text{set } Us) = \bigcup (\text{set } (\text{range-tree-chain-list } P \ a \ bs \ B'))$
using *children by (induction rule: list-all2-induct) auto*
have *U-eq-chain*:
 $U = \text{bound-tree } P \ (\text{Fin } a) \cup \text{range-tree-chain } P \ a \ bs \ B'$
using *U-def Union-eq Union-range-tree-chain-list[of P a bs B']* **by** *simp*
have *finite-U*: *finite* U
using *U-eq-chain by simp*
have *chain-subset*: $\text{range-tree-chain } P \ a \ bs \ B' \subseteq U$
using *U-eq-chain by blast*
have *card-chain-le*: $\text{card } (\text{range-tree-chain } P \ a \ bs \ B') \leq \text{card } U$
by (*rule card-mono[OF finite-U chain-subset]*)
have *outgoing-subset*:
 $\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B') \subseteq \text{outgoing-edges } U$
by (*rule outgoing-edges-mono[OF chain-subset]*)
have *finite-out-U*: *finite* $(\text{outgoing-edges } U)$
by *simp*
have *card-out-le*:
 $\text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B')) \leq$
 $\text{card } (\text{outgoing-edges } U)$
by (*rule card-mono[OF finite-out-U outgoing-subset]*)
have *loop*:
 $c \leq$
 $A * l * \text{card } (\text{range-tree-chain } P \ a \ bs \ B') +$
 $(R + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B')) +$
 $(\text{Suc } t) * (M\text{-of } l) * \text{length } bs$
by (*rule range-costed-partition-loop-state-cost-from-child-bound*
 $[OF \ \text{run child-bound}]$)
have $c \leq$
 $A * l * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U) + A * \text{card } U$
proof –
have *vterm*:
 $A * l * \text{card } (\text{range-tree-chain } P \ a \ bs \ B') \leq A * l * \text{card } U$
using *card-chain-le by simp*

```

have eterm:
   $(R + t) * \text{card} (\text{outgoing-edges} (\text{range-tree-chain } P \ a \ bs \ B')) \leq$ 
   $(R + t) * \text{card} (\text{outgoing-edges } U)$ 
using card-out-le by simp
have  $A * l * \text{card} (\text{range-tree-chain } P \ a \ bs \ B') +$ 
   $(R + t) * \text{card} (\text{outgoing-edges} (\text{range-tree-chain } P \ a \ bs \ B')) +$ 
   $(\text{Suc } t) * (M\text{-of } l) * \text{length } bs \leq$ 
   $A * l * \text{card } U + (R + t) * \text{card} (\text{outgoing-edges } U) + A * \text{card } U$ 
using vterm eterm overhead by linarith
then show ?thesis
using loop by linarith
qed
also have  $\dots =$ 
   $A * \text{Suc } l * \text{card } U + (R + t) * \text{card} (\text{outgoing-edges } U)$ 
by (simp add: algebra-simps)
finally show ?thesis .
qed

theorem range-costed-nonbase-step-closes-level-bound-general:
assumes loop:
  range-costed-partition-loop-state  $\Delta \ M\text{-of } t \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$ 
  betas bs B' Us U-loop c-loop child-costs
and sound: sound-label d
and pre: bmssp-pre-full d P B
and P-reaches:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
and child-bound:
   $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
   $\llbracket \text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$ 
   $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$ 
  bmssp-pre-full d S-child B-child;
   $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$ 
   $\text{card } S\text{-child} \leq M\text{-of } l \rrbracket$ 
   $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$ 
and overhead:  $(\text{Suc } t) * (M\text{-of } l) * \text{length } bs \leq A * \text{card } U\text{-loop}$ 
and U-def:  $U = U\text{-loop} \cup W$ 
and finite-U: finite U
and scan-insert:  $c\text{-scan-insert} \leq A * \text{card } U$ 
and c-def:  $c = c\text{-scan-insert} + c\text{-loop}$ 
shows  $c \leq A * \text{Suc } (\text{Suc } L) * \text{card } U +$ 
   $(R + t) * \text{card} (\text{outgoing-edges } U)$ 
proof –
have loop-bound:
   $c\text{-loop} \leq A * \text{Suc } L * \text{card } U\text{-loop} +$ 
   $(R + t) * \text{card} (\text{outgoing-edges } U\text{-loop})$ 
by (rule range-costed-partition-loop-state-closes-level-bound-general
  [OF loop sound pre P-reaches child-bound overhead])
have U-loop-subset:  $U\text{-loop} \subseteq U$ 
using U-def by blast
have card-loop-le:  $\text{card } U\text{-loop} \leq \text{card } U$ 

```

```

    by (rule card-mono[OF finite-U U-loop-subset])
  have outgoing-subset: outgoing-edges U-loop  $\subseteq$  outgoing-edges U
    by (rule outgoing-edges-mono[OF U-loop-subset])
  have finite-out-U: finite (outgoing-edges U)
    by simp
  have card-out-le: card (outgoing-edges U-loop)  $\leq$  card (outgoing-edges U)
    by (rule card-mono[OF finite-out-U outgoing-subset])
  have c  $\leq$ 
    A * card U +
    (A * Suc L * card U + (R + t) * card (outgoing-edges U))
  proof -
    have loop-to-U:
      c-loop  $\leq$  A * Suc L * card U +
      (R + t) * card (outgoing-edges U)
    proof -
      have A * Suc L * card U-loop  $\leq$  A * Suc L * card U
        using card-loop-le by simp
      moreover have (R + t) * card (outgoing-edges U-loop)  $\leq$ 
        (R + t) * card (outgoing-edges U)
        using card-out-le by simp
      ultimately show ?thesis
        using loop-bound by linarith
    qed
  show ?thesis
    using scan-insert loop-to-U c-def by linarith
  qed
  also have ... =
    A * Suc (Suc L) * card U + (R + t) * card (outgoing-edges U)
    by (simp add: algebra-simps)
  finally show ?thesis .
  qed

```

theorem *range-costed-nonbase-step-closes-level-bound-from-source-progress-general:*

assumes *loop:*

range-costed-partition-loop-state Δ M-of t k cap l d P B d' D a

betas bs B' Us U-loop c-loop child-costs

and *sound: sound-label d*

and *pre: bmssp-pre-full d P B*

and *P-reaches: $\bigwedge x. x \in P \implies \text{reachable } s \ x$*

and *child-bound:*

$\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$

$\llbracket \text{range-costed-bmssp } \Delta \text{ M-of } t \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$

$d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$

$\text{bmssp-pre-full } d \ S\text{-child } B\text{-child};$

$\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$

$\text{card } S\text{-child} \leq \text{M-of } l \rrbracket$

$\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$

and *source-progress:*

$\bigwedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}.$

```

    [[range-costed-bmssp  $\Delta$  M-of t k cap l d S-child B-child
      d-child B-child' U-child c-child;
      bmssp-pre-full d S-child B-child;
       $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x$ ;
      card S-child  $\leq$  M-of l]]
     $\implies$  card S-child  $\leq$  card U-child
  and source-factor: Suc t  $\leq$  A
  and U-def: U = U-loop  $\cup$  W
  and finite-U: finite U
  and scan-insert: c-scan-insert  $\leq$  A * card U
  and c-def: c = c-scan-insert + c-loop
  shows c  $\leq$  A * Suc (Suc L) * card U +
    (R + t) * card (outgoing-edges U)
  proof -
    have loop-bound:
      c-loop  $\leq$  A * Suc L * card U-loop +
        (R + t) * card (outgoing-edges U-loop)
    by (rule range-costed-partition-loop-state-closes-level-bound-from-source-progress-
      general
        [OF loop sound pre P-reaches child-bound source-progress source-factor])
    have U-loop-subset: U-loop  $\subseteq$  U
    using U-def by blast
    have card-loop-le: card U-loop  $\leq$  card U
    by (rule card-mono[OF finite-U U-loop-subset])
    have outgoing-subset: outgoing-edges U-loop  $\subseteq$  outgoing-edges U
    by (rule outgoing-edges-mono[OF U-loop-subset])
    have finite-out-U: finite (outgoing-edges U)
    by simp
    have card-out-le: card (outgoing-edges U-loop)  $\leq$  card (outgoing-edges U)
    by (rule card-mono[OF finite-out-U outgoing-subset])
    have c  $\leq$ 
      A * card U +
      (A * Suc L * card U + (R + t) * card (outgoing-edges U))
    proof -
      have loop-to-U:
        c-loop  $\leq$  A * Suc L * card U +
          (R + t) * card (outgoing-edges U)
      proof -
        have A * Suc L * card U-loop  $\leq$  A * Suc L * card U
        using card-loop-le by simp
        moreover have (R + t) * card (outgoing-edges U-loop)  $\leq$ 
          (R + t) * card (outgoing-edges U)
        using card-out-le by simp
        ultimately show ?thesis
        using loop-bound by linarith
      qed
    qed
    show ?thesis
    using scan-insert loop-to-U c-def by linarith
  qed

```

also have ... =
 $A * \text{Suc} (\text{Suc } L) * \text{card } U + (R + t) * \text{card} (\text{outgoing-edges } U)$
by (*simp add: algebra-simps*)
finally show ?thesis .
qed

theorem *range-costed-nonbase-step-closes-level-bound-from-child-cost-bounds-general:*
assumes *loop:*
 $\text{range-costed-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } k \text{ } \text{cap } l \text{ } d \text{ } P \text{ } B \text{ } d' \text{ } D \text{ } a$
 $\text{betas } bs \text{ } B' \text{ } Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs}$
and *sound:* $\text{sound-label } d$
and *pre:* $\text{bmssp-pre-full } d \text{ } P \text{ } B$
and *P-reaches:* $\bigwedge x. x \in P \implies \text{reachable } s \text{ } x$
and *child-cost-bounds:*
 $\text{list-all2 } (\lambda c\text{-child } U\text{-child}.$
 $\quad c\text{-child} \leq \text{level-range-cost-bound } A \text{ } R \text{ } L \text{ } U\text{-child})$
 $\text{child-costs } (\text{range-tree-child-list } P \text{ } a \text{ } bs)$
and *overhead:* $(\text{Suc } t) * (M\text{-of } l) * \text{length } bs \leq A * \text{card } U\text{-loop}$
and *U-def:* $U = U\text{-loop} \cup W$
and *finite-U:* $\text{finite } U$
and *scan-insert:* $c\text{-scan-insert} \leq A * \text{card } U$
and *c-def:* $c = c\text{-scan-insert} + c\text{-loop}$
shows $c \leq A * \text{Suc} (\text{Suc } L) * \text{card } U +$
 $(R + t) * \text{card} (\text{outgoing-edges } U)$
proof –
have *loop-bound:*
 $c\text{-loop} \leq A * \text{Suc } L * \text{card } U\text{-loop} +$
 $(R + t) * \text{card} (\text{outgoing-edges } U\text{-loop})$
by (*rule*
 $\text{range-costed-partition-loop-state-closes-level-bound-from-child-cost-bounds-general}$
 $[\text{OF loop sound pre P-reaches child-cost-bounds overhead}])$
have *U-loop-subset:* $U\text{-loop} \subseteq U$
using *U-def by blast*
have *card-loop-le:* $\text{card } U\text{-loop} \leq \text{card } U$
by (*rule card-mono[OF finite-U U-loop-subset]*)
have *outgoing-subset:* $\text{outgoing-edges } U\text{-loop} \subseteq \text{outgoing-edges } U$
by (*rule outgoing-edges-mono[OF U-loop-subset]*)
have *finite-out-U:* $\text{finite} (\text{outgoing-edges } U)$
by *simp*
have *card-out-le:* $\text{card} (\text{outgoing-edges } U\text{-loop}) \leq \text{card} (\text{outgoing-edges } U)$
by (*rule card-mono[OF finite-out-U outgoing-subset]*)
have $c \leq$
 $A * \text{card } U +$
 $(A * \text{Suc } L * \text{card } U + (R + t) * \text{card} (\text{outgoing-edges } U))$
proof –
have *loop-to-U:*
 $c\text{-loop} \leq A * \text{Suc } L * \text{card } U +$

$(R + t) * \text{card } (\text{outgoing-edges } U)$
proof –
 have $A * \text{Suc } L * \text{card } U\text{-loop} \leq A * \text{Suc } L * \text{card } U$
 using *card-loop-le* **by** *simp*
 moreover have $(R + t) * \text{card } (\text{outgoing-edges } U\text{-loop}) \leq$
 $(R + t) * \text{card } (\text{outgoing-edges } U)$
 using *card-out-le* **by** *simp*
 ultimately show *?thesis*
 using *loop-bound* **by** *linarith*
qed
 show *?thesis*
 using *scan-insert loop-to-U c-def* **by** *linarith*
qed
 also have $\dots =$
 $A * \text{Suc } (\text{Suc } L) * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U)$
by (*simp add: algebra-simps*)
 finally show *?thesis* .
qed

theorem *range-costed-nonbase-step-closes-level-bound:*

assumes *loop:*

range-costed-partition-loop-state Δ *M-of* t k *cap* l d P B d' D a

betas bs B' Us *U-loop* *c-loop* *child-costs*

and *sound:* *sound-label* d

and *pre:* *bmssp-pre-full* d P B

and *P-reaches:* $\bigwedge x. x \in P \implies \text{reachable } s \ x$

and *child-bound:*

$\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$

$\llbracket \text{range-costed-bmssp } \Delta \text{ M-of } t \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$

$d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$

$\text{bmssp-pre-full } d \ S\text{-child } B\text{-child};$

$\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$

$\text{card } S\text{-child} \leq \text{M-of } l \rrbracket$

$\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ l \ U\text{-child}$

and *overhead:* $(\text{Suc } t) * (\text{M-of } l) * \text{length } bs \leq A * \text{card } U\text{-loop}$

and *U-def:* $U = U\text{-loop} \cup W$

and *finite-U:* *finite* U

and *scan-insert:* $c\text{-scan-insert} \leq A * \text{card } U$

and *c-def:* $c = c\text{-scan-insert} + c\text{-loop}$

shows $c \leq A * \text{Suc } (\text{Suc } l) * \text{card } U +$

$(R + t) * \text{card } (\text{outgoing-edges } U)$

proof –

have *loop-bound:*

$c\text{-loop} \leq A * \text{Suc } l * \text{card } U\text{-loop} +$

$(R + t) * \text{card } (\text{outgoing-edges } U\text{-loop})$

by (*rule range-costed-partition-loop-state-closes-level-bound*

$\llbracket \text{OF loop sound pre P-reaches child-bound overhead} \rrbracket$)

have *U-loop-subset:* $U\text{-loop} \subseteq U$

using *U-def* **by** *blast*

```

have card-loop-le:  $\text{card } U\text{-loop} \leq \text{card } U$ 
  by (rule card-mono[OF finite-U U-loop-subset])
have outgoing-subset:  $\text{outgoing-edges } U\text{-loop} \subseteq \text{outgoing-edges } U$ 
  by (rule outgoing-edges-mono[OF U-loop-subset])
have finite-out-U: finite (outgoing-edges U)
  by simp
have card-out-le:  $\text{card } (\text{outgoing-edges } U\text{-loop}) \leq \text{card } (\text{outgoing-edges } U)$ 
  by (rule card-mono[OF finite-out-U outgoing-subset])
have  $c \leq$ 
   $A * \text{card } U +$ 
   $(A * \text{Suc } l * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U))$ 
proof -
  have loop-to-U:
     $c\text{-loop} \leq A * \text{Suc } l * \text{card } U +$ 
     $(R + t) * \text{card } (\text{outgoing-edges } U)$ 
  proof -
    have  $A * \text{Suc } l * \text{card } U\text{-loop} \leq A * \text{Suc } l * \text{card } U$ 
      using card-loop-le by simp
    moreover have  $(R + t) * \text{card } (\text{outgoing-edges } U\text{-loop}) \leq$ 
       $(R + t) * \text{card } (\text{outgoing-edges } U)$ 
      using card-out-le by simp
    ultimately show ?thesis
      using loop-bound by linarith
  qed
show ?thesis
  using scan-insert loop-to-U c-def by linarith
qed
also have ... =
   $A * \text{Suc } (\text{Suc } l) * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U)$ 
  by (simp add: algebra-simps)
finally show ?thesis .
qed

```

```

theorem range-costed-base-level-bound:
  assumes R-pos:  $0 < R$ 
  shows base-case-scan-cost  $\Delta k x B \leq$ 
    level-range-cost-bound  $A R (\text{Suc } 0) (\text{base-case-vertices } k x B)$ 
proof -
  have base-case-scan-cost  $\Delta k x B \leq$ 
     $R * \text{card } (\text{outgoing-edges } (\text{base-case-vertices } k x B))$ 
    using R-pos unfolding base-case-scan-cost-def by simp
  then show ?thesis
    unfolding level-range-cost-bound-def by linarith
qed

```

The unsplit relation repeats the same recurrence in a form that is closer to the abstract cost interface. The base theorem $0 < ?R \implies \text{base-case-scan-cost } ?\Delta \ ?k \ ?x \ ?B \leq \text{level-range-cost-bound } ?A \ ?R \ (\text{Suc } 0) \ (\text{base-case-vertices } ?k \ ?x \ ?B)$ pays for the bounded base scan with the outgoing-edge component

of *level-range-cost-bound*. The main theorem below then performs induction on the recursion level: the induction hypothesis bounds each child call, while the supplied local-budget hypotheses pay for loop overhead and FindPivots overhead at the current level.

This theorem is intentionally parameterised by the constants A , R , t , and $M\text{-of}$. Later theories instantiate those parameters with the BMSSP schedule and with the bucketed partition costs, but the recurrence itself does not depend on how the partition primitive is implemented.

theorem *range-costed-bmssp-level-bound-from-local-budgets*:

assumes *base-budget*: $\Delta \leq A$

and *R-pos*: $0 < R$

and *loop-budget*:

$\bigwedge l \ d \ P \ B \ d' \ D \ a \ \text{betas} \ bs \ B' \ Us \ U \ c \ \text{child-costs}.$

range-costed-partition-loop-state $\Delta \ M\text{-of} \ t \ k \ \text{cap} \ l \ d \ P \ B \ d' \ D \ a$

betas $bs \ B' \ Us \ U \ c \ \text{child-costs} \implies$

sound-label $d \implies$

bmssp-pre-full $d \ P \ B \implies$

$(\bigwedge x. x \in P \implies \text{reachable } s \ x) \implies$

$(\text{Suc } t) * (M\text{-of } l) * \text{length } bs \leq A * \text{card } U$

and *step-budget*:

$\bigwedge l \ d \ S \ B \ D \ c\text{-insert} \ d' \ a \ \text{betas} \ bs \ B' \ Us \ U\text{-loop} \ c\text{-loop}$

child-costs $U.$

$D = \text{label-partition-view}$

$(\text{find-pivots-label-capped } k \ \text{cap} \ d \ S \ B)$

$(\text{find-pivots-pivots-capped } k \ \text{cap} \ d \ S \ B) \implies$

partition-initial-insert-cost-bound $c\text{-insert} \ t$

$(\text{find-pivots-pivots-capped } k \ \text{cap} \ d \ S \ B) \implies$

range-costed-partition-loop-state $\Delta \ M\text{-of} \ t \ k \ \text{cap} \ l$

$(\text{find-pivots-label-capped } k \ \text{cap} \ d \ S \ B)$

$(\text{find-pivots-pivots-capped } k \ \text{cap} \ d \ S \ B) \ B \ d' \ D \ a \ \text{betas} \ bs \ B'$

$Us \ U\text{-loop} \ c\text{-loop} \ \text{child-costs} \implies$

complete-on d'

$\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap} \ d \ S \ B \ v = \text{dist } s \ v\} \implies$

$U = U\text{-loop} \cup$

$\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap} \ d \ S \ B \ v = \text{dist } s \ v\} \implies$

sound-label $d \implies$

bmssp-pre-full $d \ S \ B \implies$

$(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$

$\text{fp-iter-capped-scan-cost } k \ \text{cap} \ d \ S \ S \ B + c\text{-insert} \leq A * \text{card } U$

and *run*:

range-costed-bmssp $\Delta \ M\text{-of} \ t \ k \ \text{cap} \ l \ d \ S \ B \ d' \ B' \ U \ c$

and *sound*: *sound-label* d

and *pre*: *bmssp-pre-full* $d \ S \ B$

and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$

shows $c \leq \text{level-range-cost-bound } A \ (R + l * t) \ (2 * l + 1) \ U$

using *run* *sound* *pre* *S-reaches*

proof (*induction* l *arbitrary*: $d \ S \ B \ d' \ B' \ U \ c$ *rule*: *less-induct*)

case (*less* l)


```

show ?case
proof (cases l)
  case 0
  have run0:
    range-costed-bmssp  $\Delta$  M-of t k cap 0 d S B d' B' U c
    using less.prem(1) 0 by simp
  show ?thesis
    using run0 R-pos 0
    by (cases rule: range-costed-bmssp.cases)
      (simp-all add: range-costed-base-level-bound)
next
  case (Suc l0)
  have run-suc:
    range-costed-bmssp  $\Delta$  M-of t k cap (Suc l0) d S B d' B' U c
    using less.prem(1) Suc by simp
  show ?thesis
proof (cases rule: range-costed-bmssp-SucE[OF run-suc])
  case (1 c-insert a betas bs Us U-loop c-loop child-costs-loop)
  let ?d-fp = find-pivots-label-capped k cap d S B
  let ?P = find-pivots-pivots-capped k cap d S B
  let ?W = {v  $\in$  bound-tree S B'. ?d-fp v = dist s v}
  have sound-fp: sound-label ?d-fp
    unfolding find-pivots-label-capped-def
    by (rule fp-iter-capped-label-sound[OF less.prem(2) less.prem(4)])
  have pivot-pre: bmssp-pre-full ?d-fp ?P B
    using find-pivots-capped-establishes-pivot-pre-concrete
      [OF less.prem(2) less.prem(3) less.prem(4)] .
  have P-subset-S: ?P  $\subseteq$  S
    unfolding find-pivots-pivots-capped-def by auto
  have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
    using P-subset-S less.prem(4) by blast
  have child-bound:
     $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
     $\llbracket \text{range-costed-bmssp } \Delta \text{ M-of } t \ k \ \text{cap } l0 \ ?d\text{-fp } S\text{-child } B\text{-child}$ 
     $\quad d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$ 
     $\text{bmssp-pre-full } ?d\text{-fp } S\text{-child } B\text{-child};$ 
     $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$ 
     $\text{card } S\text{-child} \leq \text{M-of } l0 \rrbracket$ 
     $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ (R + l0 * t)$ 
     $\quad (2 * l0 + 1) \ U\text{-child}$ 
  proof –
    fix c-child U-child S-child B-child d-child B-child'
    assume child:
      range-costed-bmssp  $\Delta$  M-of t k cap l0 ?d-fp S-child B-child
      d-child B-child' U-child c-child
    and child-pre: bmssp-pre-full ?d-fp S-child B-child
    and child-reaches:  $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x$ 
    show c-child  $\leq$  level-range-cost-bound A (R + l0 * t)
      (2 * l0 + 1) U-child

```

by (rule less.IH[of l0 ?d-fp S-child B-child d-child B-child'
 U-child c-child, OF - child sound-fp child-pre child-reaches])
 (simp add: Suc)
 qed
 have overhead:
 (Suc t) * (M-of l0) * length bs ≤ A * card U-loop
 by (rule loop-budget[OF 1(4) sound-fp pivot-pre P-reaches])
 have scan-insert:
 fp-iter-capped-scan-cost k cap d S S B + c-insert ≤
 A * card U
 by (rule step-budget[OF refl 1(3) 1(4) 1(5) 1(1)
 less.prem1(2) less.prem1(3) less.prem1(4)])
 have trace:
 concrete-partition-loop-trace ?P B a bs d' B' Us U-loop
 by (rule range-costed-partition-loop-state-trace
 [OF 1(4) sound-fp pivot-pre P-reaches])
 have loop-post: bmssp-post-full ?d-fp ?P B d' B' U-loop
 by (rule concrete-partition-loop-trace-post[OF trace])
 have finite-U-loop: finite U-loop
 using loop-post unfolding bmssp-post-full-def by simp
 have finite-W: finite ?W
 by simp
 have finite-U: finite U
 using 1(1) finite-U-loop finite-W by simp
 have step:
 c ≤ A * Suc (Suc (2 * l0 + 1)) * card U +
 (R + l0 * t + t) * card (outgoing-edges U)
 by (rule range-costed-nonbase-step-closes-level-bound-general
 [OF 1(4) sound-fp pivot-pre P-reaches child-bound overhead
 1(1) finite-U scan-insert 1(2)])
 show ?thesis
 using step Suc unfolding level-range-cost-bound-def
 by (simp add: algebra-simps)
 qed
 qed
 qed

theorem range-costed-bmssp-level-bound-from-source-progress-and-local-budgets:

assumes base-budget: $\Delta \leq A$
 and R-pos: $0 < R$
 and source-factor: $\text{Suc } t \leq A$
 and source-progress:
 $\bigwedge l \ d \ S \ B \ d' \ B' \ U \ c.$
 range-costed-bmssp Δ M-of t k cap l d S B d' B' U c $\implies$
 bmssp-pre-full d S B $\implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 card S ≤ M-of l $\implies$
 card S ≤ card U
 and step-budget:

```

 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$ 
 $\text{child-costs } U.$ 
 $D = \text{label-partition-view}$ 
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$ 
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$ 
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$ 
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$ 
 $\text{range-costed-partition-loop-state } \Delta \ M\text{-of } t \ k \ \text{cap } l$ 
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$ 
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$ 
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$ 
 $\text{complete-on } d'$ 
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$ 
 $U = U\text{-loop} \cup$ 
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$ 
 $\text{sound-label } d \implies$ 
 $\text{bmssp-pre-full } d \ S \ B \implies$ 
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$ 
 $\text{fp-iter-capped-scan-cost } k \ \text{cap } d \ S \ S \ B + c\text{-insert} \leq A * \text{card } U$ 
and run:
 $\text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c$ 
and sound:  $\text{sound-label } d$ 
and pre:  $\text{bmssp-pre-full } d \ S \ B$ 
and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
shows  $c \leq \text{level-range-cost-bound } A \ (R + l * t) \ (2 * l + 1) \ U$ 
using run sound pre S-reaches
proof (induction l arbitrary: d S B d' B' U c rule: less-induct)
case (less l)
show ?case
proof (cases l)
case 0
have run0:
 $\text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \ \text{cap } 0 \ d \ S \ B \ d' \ B' \ U \ c$ 
using less.prem(1) 0 by simp
show ?thesis
using run0 R-pos 0
by (cases rule: range-costed-bmssp.cases)
(simp-all add: range-costed-base-level-bound)
next
case (Suc l0)
have run-suc:
 $\text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \ \text{cap } (\text{Suc } l0) \ d \ S \ B \ d' \ B' \ U \ c$ 
using less.prem(1) Suc by simp
show ?thesis
proof (cases rule: range-costed-bmssp-SucE[OF run-suc])
case (1 c-insert a betas bs Us U-loop c-loop child-costs-loop)
let ?d-fp =  $\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B$ 
let ?P =  $\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B$ 
let ?W =  $\{v \in \text{bound-tree } S \ B'. \ ?d\text{-fp } v = \text{dist } s \ v\}$ 

```

```

have sound-fp: sound-label ?d-fp
  unfolding find-pivots-label-capped-def
  by (rule fp-iter-capped-label-sound[OF less.prems(2) less.prems(4)])
have pivot-pre: bmssp-pre-full ?d-fp ?P B
  using find-pivots-capped-establishes-pivot-pre-concrete
    [OF less.prems(2) less.prems(3) less.prems(4)] .
have P-subset-S: ?P  $\subseteq$  S
  unfolding find-pivots-pivots-capped-def by auto
have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
  using P-subset-S less.prems(4) by blast
have child-bound:
   $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
   $\llbracket \text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \ \text{cap } l0 \ ?d\text{-fp } S\text{-child } B\text{-child}$ 
     $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$ 
    bmssp-pre-full ?d-fp S-child B-child;
     $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$ 
     $\text{card } S\text{-child} \leq M\text{-of } l0 \rrbracket$ 
     $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ (R + l0 * t)$ 
     $(2 * l0 + 1) \ U\text{-child}$ 
proof –
  fix c-child U-child S-child B-child d-child B-child'
  assume child:
    range-costed-bmssp  $\Delta \ M\text{-of } t \ k \ \text{cap } l0 \ ?d\text{-fp } S\text{-child } B\text{-child}$ 
       $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}$ 
    and child-pre: bmssp-pre-full ?d-fp S-child B-child
    and child-reaches:  $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x$ 
  show  $c\text{-child} \leq \text{level-range-cost-bound } A \ (R + l0 * t)$ 
     $(2 * l0 + 1) \ U\text{-child}$ 
    by (rule less.IH[of l0 ?d-fp S-child B-child d-child B-child'
      U-child c-child, OF - child sound-fp child-pre child-reaches])
    (simp add: Suc)
qed
have source-progress-loop:
   $\bigwedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}.$ 
   $\llbracket \text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \ \text{cap } l0 \ ?d\text{-fp } S\text{-child } B\text{-child}$ 
     $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$ 
    bmssp-pre-full ?d-fp S-child B-child;
     $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$ 
     $\text{card } S\text{-child} \leq M\text{-of } l0 \rrbracket$ 
     $\implies \text{card } S\text{-child} \leq \text{card } U\text{-child}$ 
  by (rule source-progress)
have scan-insert:
  fp-iter-capped-scan-cost k cap d S S B + c-insert  $\leq$ 
     $A * \text{card } U$ 
  by (rule step-budget[OF refl 1(3) 1(4) 1(5) 1(1)
    less.prems(2) less.prems(3) less.prems(4)])
have trace:
  concrete-partition-loop-trace ?P B a bs d' B' Us U-loop
  by (rule range-costed-partition-loop-state-trace)

```

```

    [OF 1(4) sound-fp pivot-pre P-reaches])
  have loop-post: bmssp-post-full ?d-fp ?P B d' B' U-loop
    by (rule concrete-partition-loop-trace-post[OF trace])
  have finite-U-loop: finite U-loop
    using loop-post unfolding bmssp-post-full-def by simp
  have finite-W: finite ?W
    by simp
  have finite-U: finite U
    using 1(1) finite-U-loop finite-W by simp
  have step:
    c ≤ A * Suc (Suc (2 * l0 + 1)) * card U +
      (R + l0 * t + t) * card (outgoing-edges U)
    by (rule range-costed-nonbase-step-closes-level-bound-from-source-progress-
general
      [OF 1(4) sound-fp pivot-pre P-reaches child-bound
        source-progress-loop source-factor 1(1) finite-U scan-insert 1(2)])
  show ?thesis
    using step Suc unfolding level-range-cost-bound-def
    by (simp add: algebra-simps)
qed
qed
qed

```

theorem range-costed-bmssp-level-bound-from-source-progress-budgets:

```

  assumes degree: edge-outdegree-le Δ
    and degree-factor: Δ ≤ A
    and R-pos: 0 < R
    and insert-factor: t ≤ A * k
    and source-factor: Suc t ≤ 2 * A
    and source-progress:
      ∧ l d S B d' B' U c.
        range-costed-bmssp Δ M-of t k cap l d S B d' B' U c ⇒
        bmssp-pre-full d S B ⇒
        (∧ x. x ∈ S ⇒ reachable s x) ⇒
        card S ≤ M-of l ⇒
        card S ≤ card U
    and step-progress:
      ∧ l d S B D c-insert d' a betas bs B' Us U-loop c-loop
        child-costs U.
        D = label-partition-view
          (find-pivots-label-capped k cap d S B)
          (find-pivots-pivots-capped k cap d S B) ⇒
        partition-initial-insert-cost-bound c-insert t
          (find-pivots-pivots-capped k cap d S B) ⇒
        range-costed-partition-loop-state Δ M-of t k cap l
          (find-pivots-label-capped k cap d S B)
          (find-pivots-pivots-capped k cap d S B) B d' D a betas bs B'
          Us U-loop c-loop child-costs ⇒
        complete-on d'

```

$\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq \text{cap} \wedge k * \text{cap} \leq \text{card } U$
and *run*:
 $\text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \text{ cap } l \ d \ S \ B \ d' \ B' \ U \ c$
and *sound*: $\text{sound-label } d$
and *pre*: $\text{bmssp-pre-full } d \ S \ B$
and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
shows $c \leq \text{level-range-cost-bound } (2 * A) \ (R + l * t) \ (2 * l + 1) \ U$
proof (*rule range-costed-bmssp-level-bound-from-source-progress-and-local-budgets*
[**where** $A = 2 * A$ **and** $R = R$,
 $OF - R\text{-pos source-factor} - - \text{run sound pre } S\text{-reaches}$])
show $\Delta \leq 2 * A$
using *degree-factor* **by** *simp*
next
fix $l \ d \ S \ B \ d' \ B' \ U \ c$
assume *child*:
 $\text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \text{ cap } l \ d \ S \ B \ d' \ B' \ U \ c$
and *child-pre*: $\text{bmssp-pre-full } d \ S \ B$
and *child-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
and *child-card*: $\text{card } S \leq M\text{-of } l$
show $\text{card } S \leq \text{card } U$
by (*rule source-progress*[$OF \ \text{child } \text{child-pre } \text{child-reaches } \text{child-card}$])
next
fix $l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs } U$
assume *D-def*:
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \text{ cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B)$
and *insert*:
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B)$
and *loop*:
 $\text{range-costed-partition-loop-state } \Delta \ M\text{-of } t \ k \text{ cap } l$
 $(\text{find-pivots-label-capped } k \text{ cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs}$
and *complete*:
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\}$
and *U-def*:
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\}$
and *sound-s*: $\text{sound-label } d$
and *pre-s*: $\text{bmssp-pre-full } d \ S \ B$

and *reaches-s*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
have *S-subset*: $S \subseteq V$
using *pre-s unfolding* *bmssp-pre-full-def* **by** *blast*
have *progress*:
 $\text{card } S \leq \text{cap} \wedge k * \text{cap} \leq \text{card } U$
by (*rule step-progress*[*OF D-def insert loop complete U-def sound-s pre-s reaches-*
s])
then have *S-cap*: $\text{card } S \leq \text{cap}$
by *blast*
from *progress* **have** *U-progress*: $k * \text{cap} \leq \text{card } U$
by *blast*
show *fp-iter-capped-scan-cost* $k \ \text{cap} \ d \ S \ S \ B + c\text{-insert} \leq$
 $2 * A * \text{card } U$
by (*rule find-pivots-scan-and-initial-insert-budget-from-progress*
[*OF S-subset degree S-cap insert degree-factor insert-factor U-progress*])
qed

theorem *range-costed-bmssp-level-bound-from-progress-budgets*:

assumes *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$
and *loop-factor*: $\text{Suc } t \leq 2 * A$
and *loop-progress*:
 $\bigwedge l \ d \ P \ B \ d' \ D \ a \ \text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs}.$
 $\text{range-costed-partition-loop-state } \Delta \ M\text{-of } t \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ P \ B \implies$
 $(\bigwedge x. x \in P \implies \text{reachable } s \ x) \implies$
 $(M\text{-of } l) * \text{length } bs \leq \text{card } U$
and *step-progress*:
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
 $\text{child-costs } U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{range-costed-partition-loop-state } \Delta \ M\text{-of } t \ k \ \text{cap } l$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$

$(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq \text{cap} \wedge k * \text{cap} \leq \text{card } U$
and *run*:
 $\text{range-costed-bmssp } \Delta \text{ } M\text{-of } t \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c$
and *sound*: *sound-label* d
and *pre*: *bmssp-pre-full* $d \ S \ B$
and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
shows $c \leq \text{level-range-cost-bound } (2 * A) \ (R + l * t) \ (2 * l + 1) \ U$
proof (*rule range-costed-bmssp-level-bound-from-local-budgets*
[**where** $A = 2 * A$ **and** $R = R$,
 $OF - R\text{-pos} - - \text{run } \text{sound } \text{pre } S\text{-reaches}$])
show $\Delta \leq 2 * A$
using *degree-factor* **by** *simp*
next
fix $l \ d \ P \ B \ d' \ D \ a \ \text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs}$
assume *loop*:
 $\text{range-costed-partition-loop-state } \Delta \text{ } M\text{-of } t \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs}$
and *sound-l*: *sound-label* d
and *pre-l*: *bmssp-pre-full* $d \ P \ B$
and *reaches-l*: $\bigwedge x. x \in P \implies \text{reachable } s \ x$
have *progress*: $(M\text{-of } l) * \text{length } bs \leq \text{card } U$
by (*rule loop-progress*[$OF \ \text{loop } \text{sound-l } \text{pre-l } \text{reaches-l}$])
show $(\text{Suc } t) * (M\text{-of } l) * \text{length } bs \leq 2 * A * \text{card } U$
by (*rule loop-overhead-budget-from-progress*[$OF \ \text{loop-factor } \text{progress}$])
next
fix $l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs } U$
assume *D-def*:
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B)$
and *insert*:
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B)$
and *loop*:
 $\text{range-costed-partition-loop-state } \Delta \text{ } M\text{-of } t \ k \ \text{cap } l$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs}$
and *complete*:
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\}$
and *U-def*:
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\}$
and *sound-s*: *sound-label* d
and *pre-s*: *bmssp-pre-full* $d \ S \ B$
and *reaches-s*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
have *S-subset*: $S \subseteq V$

using *pre-s* **unfolding** *bmssp-pre-full-def* **by** *blast*
have *progress*:
 $\text{card } S \leq \text{cap} \wedge k * \text{cap} \leq \text{card } U$
by (*rule step-progress*[*OF D-def insert loop complete U-def sound-s pre-s reaches-*
s])
then have *S-cap*: $\text{card } S \leq \text{cap}$
by *blast*
from *progress* **have** *U-progress*: $k * \text{cap} \leq \text{card } U$
by *blast*
show *fp-iter-capped-scan-cost* $k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $2 * A * \text{card } U$
by (*rule find-pivots-scan-and-initial-insert-budget-from-progress*
[*OF S-subset degree S-cap insert degree-factor insert-factor U-progress*])
qed

theorem *finite-initial-label-range-costed-top-level-correct*:
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \text{ } v$
and *run*:
 $\text{range-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } k \text{ cap } l \text{ } \text{finite-initial-label } \{s\}$
 $\text{Infinity } d' \text{ Infinity } U \text{ } c$
shows *sssp-correct* d'
proof –
have *pre*: *bmssp-pre-full* *finite-initial-label* $\{s\}$ *Infinity*
using *all-reachable* *finite-initial-label-source-complete*
by (*rule top-bmssp-pre-full*)
have *sound*: *sound-label* *finite-initial-label*
using *finite-initial-label-sound*[*OF all-reachable*] .
have *S-reaches*: $\bigwedge x. x \in \{s\} \implies \text{reachable } s \text{ } x$
using *all-reachable* *source-in-V* **by** *blast*
have *post-full*:
 $\text{bmssp-post-full } \text{finite-initial-label } \{s\} \text{ Infinity } d' \text{ Infinity } U$
by (*rule range-costed-bmssp-correct*[*OF run sound pre S-reaches*])
have *post*: *bmssp-post* *finite-initial-label* $\{s\}$ *Infinity* d' *Infinity* U
using *bmssp-post-full-imp-post*[*OF post-full*] .
then have *U-V*: $U = V$
using *bound-tree-source-infinity*[*OF all-reachable*]
unfolding *bmssp-post-def* **by** *auto*
have *complete*: *complete-on* $d' \text{ } U$
using *post* **unfolding** *bmssp-post-def* **by** *auto*
then show *?thesis*
using *U-V* **unfolding** *complete-on-def* *sssp-correct-def* **by** *auto*
qed

theorem *finite-initial-label-range-costed-top-level-correct-and-bound*:
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \text{ } v$
and *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$

and *loop-factor*: $\text{Suc } t \leq 2 * A$
and *loop-progress*:
 $\bigwedge l \, d \, P \, B \, d' \, D \, a \, \text{betas } bs \, B' \, Us \, U \, c \, \text{child-costs}.$
 $\text{range-costed-partition-loop-state } \Delta \, M\text{-of } t \, k \, \text{cap } l \, d \, P \, B \, d' \, D \, a$
 $\text{betas } bs \, B' \, Us \, U \, c \, \text{child-costs} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \, P \, B \implies$
 $(\bigwedge x. x \in P \implies \text{reachable } s \, x) \implies$
 $(M\text{-of } l) * \text{length } bs \leq \text{card } U$
and *step-progress*:
 $\bigwedge l \, d \, S \, B \, D \, c\text{-insert } d' \, a \, \text{betas } bs \, B' \, Us \, U\text{-loop } c\text{-loop}$
 $\text{child-costs } U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \, \text{cap } d \, S \, B)$
 $(\text{find-pivots-pivots-capped } k \, \text{cap } d \, S \, B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \, \text{cap } d \, S \, B) \implies$
 $\text{range-costed-partition-loop-state } \Delta \, M\text{-of } t \, k \, \text{cap } l$
 $(\text{find-pivots-label-capped } k \, \text{cap } d \, S \, B)$
 $(\text{find-pivots-pivots-capped } k \, \text{cap } d \, S \, B) \, B \, d' \, D \, a \, \text{betas } bs \, B'$
 $Us \, U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \, B'. \text{find-pivots-label-capped } k \, \text{cap } d \, S \, B \, v = \text{dist } s \, v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \, B'. \text{find-pivots-label-capped } k \, \text{cap } d \, S \, B \, v = \text{dist } s \, v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \, S \, B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \, x) \implies$
 $\text{card } S \leq \text{cap} \wedge k * \text{cap} \leq \text{card } U$
and *run*:
 $\text{range-costed-bmssp } \Delta \, M\text{-of } t \, k \, \text{cap } l \, \text{finite-initial-label } \{s\}$
 $\text{Infinity } d' \, \text{Infinity } U \, c$
shows $\text{sssp-correct } d' \wedge$
 $c \leq \text{level-range-cost-bound } (2 * A) \, (R + l * t) \, (2 * l + 1) \, U$
proof
show $\text{sssp-correct } d'$
by $(\text{rule } \text{finite-initial-label-range-costed-top-level-correct}$
 $[\text{OF all-reachable run}])$
next
have *pre*: $\text{bmssp-pre-full } \text{finite-initial-label } \{s\} \, \text{Infinity}$
using $\text{all-reachable } \text{finite-initial-label-source-complete}$
by $(\text{rule } \text{top-bmssp-pre-full})$
have *sound*: $\text{sound-label } \text{finite-initial-label}$
using $\text{finite-initial-label-sound}[\text{OF all-reachable}]$.
have *S-reaches*: $\bigwedge x. x \in \{s\} \implies \text{reachable } s \, x$
using $\text{all-reachable source-in-} V$ **by** *blast*
show $c \leq \text{level-range-cost-bound } (2 * A) \, (R + l * t) \, (2 * l + 1) \, U$
by $(\text{rule } \text{range-costed-bmssp-level-bound-from-progress-budgets}$
 $[\text{OF degree degree-factor } R\text{-pos insert-factor loop-factor loop-progress}$

step-progress run sound pre S-reaches])
qed

theorem *finite-initial-label-range-costed-top-level-correct-and-source-progress-bound:*

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$
and *source-factor*: $\text{Suc } t \leq 2 * A$
and *source-progress*:
 $\bigwedge l \ d \ S \ B \ d' \ B' \ U \ c.$
 $\text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq M\text{-of } l \implies$
 $\text{card } S \leq \text{card } U$
and *step-progress*:
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
 $\text{child-costs } U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{range-costed-partition-loop-state } \Delta \ M\text{-of } t \ k \ \text{cap } l$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq \text{cap} \wedge k * \text{cap} \leq \text{card } U$
and *run*:
 $\text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \ \text{cap } l \ \text{finite-initial-label } \{s\}$
 $\text{Infinity } d' \ \text{Infinity } U \ c$
shows $\text{sssp-correct } d' \wedge$
 $c \leq \text{level-range-cost-bound } (2 * A) \ (R + l * t) \ (2 * l + 1) \ U$
proof
show $\text{sssp-correct } d'$
by (*rule finite-initial-label-range-costed-top-level-correct*
 $[\text{OF all-reachable run}]$)
next
have *pre*: $\text{bmssp-pre-full finite-initial-label } \{s\} \ \text{Infinity}$

using *all-reachable finite-initial-label-source-complete*
by (rule *top-bmssp-pre-full*)
have *sound*: *sound-label finite-initial-label*
using *finite-initial-label-sound*[*OF all-reachable*] .
have *S-reaches*: $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$
using *all-reachable source-in-V* **by** *blast*
show $c \leq \text{level-range-cost-bound } (2 * A) (R + l * t) (2 * l + 1) U$
by (rule *range-costed-bmssp-level-bound-from-source-progress-budgets*
[OF degree degree-factor R-pos insert-factor source-factor source-progress
step-progress run sound pre S-reaches])
qed

theorem *finite-initial-label-range-costed-top-level-correct-and-graph-bound*:

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$
and *source-factor*: $\text{Suc } t \leq 2 * A$
and *source-progress*:
 $\bigwedge l \ d \ S \ B \ d' \ B' \ U \ c.$
 $\text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq M\text{-of } l \implies$
 $\text{card } S \leq \text{card } U$
and *step-progress*:
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
 $\text{child-costs } U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{range-costed-partition-loop-state } \Delta \ M\text{-of } t \ k \ \text{cap } l$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $\text{card } S \leq \text{cap} \wedge k * \text{cap} \leq \text{card } U$
and *run*:
 $\text{range-costed-bmssp } \Delta \ M\text{-of } t \ k \ \text{cap } l \ \text{finite-initial-label } \{s\}$
 $\text{Infinity } d' \ \text{Infinity } U \ c$

```

shows sssp-correct  $d' \wedge$ 
   $c \leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$ 
proof –
  have combined:
    sssp-correct  $d' \wedge$ 
     $c \leq \text{level-range-cost-bound } (2 * A) (R + l * t) (2 * l + 1) U$ 
    by (rule finite-initial-label-range-costed-top-level-correct-and-source-progress-bound
      [OF all-reachable degree degree-factor R-pos insert-factor source-factor
        source-progress step-progress run])
  then have correct: sssp-correct  $d'$ 
    by blast
  have pre: bmssp-pre-full finite-initial-label  $\{s\}$  Infinity
    using all-reachable finite-initial-label-source-complete
    by (rule top-bmssp-pre-full)
  have sound: sound-label finite-initial-label
    using finite-initial-label-sound[OF all-reachable] .
  have S-reaches:  $\bigwedge x. x \in \{s\} \implies \text{reachable } s x$ 
    using all-reachable source-in-V by blast
  have post-full:
    bmssp-post-full finite-initial-label  $\{s\}$  Infinity  $d'$  Infinity  $U$ 
    by (rule range-costed-bmssp-correct[OF run sound pre S-reaches])
  have post: bmssp-post finite-initial-label  $\{s\}$  Infinity  $d'$  Infinity  $U$ 
    using bmssp-post-full-imp-post[OF post-full] .
  have  $U-V: U = V$ 
    using post bound-tree-source-infinity[OF all-reachable]
    unfolding bmssp-post-def by auto
  have cost-U:
     $c \leq \text{level-range-cost-bound } (2 * A) (R + l * t) (2 * l + 1) U$ 
    using combined by blast
  have graph-bound:
     $\text{level-range-cost-bound } (2 * A) (R + l * t) (2 * l + 1) U$ 
     $\leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$ 
proof –
    have U-card:  $\text{card } U = \text{vertex-count}$ 
      unfolding  $U-V$  vertex-count-def by simp
    have out-le:  $\text{card } (\text{outgoing-edges } U) \leq \text{edge-count}$ 
      by (rule edge-count-outgoing-bound)
    have out-term:
       $(R + l * t) * \text{card } (\text{outgoing-edges } U) \leq (R + l * t) * \text{edge-count}$ 
      using out-le by simp
    show ?thesis
      unfolding level-range-cost-bound-def
      using U-card out-term by (simp add: algebra-simps)
  qed
show ?thesis
  using correct cost-U graph-bound by linarith
qed

```

```

end

end
theory BMSSP-Exact-Range-Costed
  imports BMSSP-Range-Costed
begin

```

31 Exact Split Range-Costed Runs

The range-costed relation records the algorithm's range accounting, but leaves the pulled child source abstract. The exact variant below keeps the same cost and range information while additionally recording that each pull source is exactly the lower label split used by the operational algorithm. It refines the existing range-costed relation, so the established correctness and cost lemmas remain reusable.

This extra equality is small syntactically and important analytically. A pull operation returns a child source set and a child bound. For correctness it is enough to know that the child call satisfies the BMSSP precondition. For progress and runtime, however, the source set must be identified with the mathematical split of the parent sources below the child bound. Only then can the proof inherit the antichain and scaled-cardinality facts that were proved for lower splits.

The theory therefore sits between range synchronisation and direct insertion. It keeps the range slices, child-cost list, and edge/source batch split from the preceding layer, while adding enough exactness to discharge the source progress side conditions needed by the recurrence. Later theories refine this one by splitting edge batches further; they rely on the exact source information introduced here rather than reproving it.

No new algorithm is introduced. Each theorem either forgets the exactness to recover the split-range relation, or uses exactness to derive stronger cost premises for the same loop trace.

```

context unique-shortest-digraph
begin

```

```

inductive exact-split-range-costed-partition-loop-state
and exact-split-range-costed-bmssp where
  Exact-Split-Range-State-Done:
     $\text{keys-of } D = \{\} \implies$ 
     $\text{bound-le } (\text{Fin } a) \ B \implies$ 
     $\text{complete-on } d' \ (\text{bound-tree } P \ B) \implies$ 
     $\text{exact-split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$ 
     $\square \square \ B \ [\text{range-tree } P \ a \ B]$ 
     $(\text{bound-tree } P \ (\text{Fin } a) \cup \bigcup (\text{set } [\text{range-tree } P \ a \ B])) \ 0 \square$ 
| Exact-Split-Range-State-Stop:
   $\text{bound-le } (\text{Fin } a) \ B \implies$ 

```

$complete-on\ d' (bound-tree\ P\ (Fin\ a)) \implies$
 $k * cap \leq card\ (bound-tree\ P\ (Fin\ a)) \implies$
 $exact-split-range-costed-partition-loop-state\ \Delta\ M-of\ t\ h\ k\ cap\ l\ d\ P\ B\ d'\ D\ a$
 $\square\ \square\ (Fin\ a)\ [range-tree\ P\ a\ (Fin\ a)]$
 $(bound-tree\ P\ (Fin\ a) \cup \bigcup (set\ [range-tree\ P\ a\ (Fin\ a)]))\ 0\ \square$
| *Exact-Split-Range-State-Step:*
 $pull-separates\ D\ (M-of\ l)\ Bmax\ S-pull\ beta\ D-pull \implies$
 $bound-le\ (Fin\ beta)\ B \implies$
 $bmssp-pre-full\ d\ S-pull\ (Fin\ beta) \implies$
 $S-pull = split-below\ d\ P\ beta \implies$
 $(\forall x \in S-pull. reachable\ s\ x) \implies$
 $a \leq b \implies$
 $bound-le\ (Fin\ a)\ B' \implies$
 $complete-on\ d' (bound-tree\ P\ (Fin\ a)) \implies$
 $card\ (bound-tree\ P\ (Fin\ a)) < k * cap \implies$
 $exact-split-range-costed-bmssp\ \Delta\ M-of\ t\ h\ k\ cap\ l\ d\ S-pull\ (Fin\ beta)$
 $d-child\ (Fin\ b)\ U-child\ c-child \implies$
 $U-child = range-tree\ P\ a\ (Fin\ b) \implies$
 $complete-preserved\ d-child\ d'\ U-child \implies$
 $edge-batch = edge-relaxation-pairs-between\ d-child\ U-child\ b\ beta \implies$
 $source-batch = label-pairs-between\ d\ S-pull\ b\ beta \implies$
 $batch = edge-batch\ @\ source-batch \implies$
 $D-next = batch-min-update\ D-pull\ batch \implies$
 $partition-pull-cost-bound\ c-pull\ S-pull \implies$
 $partition-batch-cost-bound\ c-edges\ t\ edge-batch \implies$
 $partition-batch-cost-bound\ c-sources\ h\ source-batch \implies$
 $exact-split-range-costed-partition-loop-state\ \Delta\ M-of\ t\ h\ k\ cap\ l\ d\ P\ B\ d'$
 $D-next\ b\ betas\ bs\ B'\ Us-tail\ U-tail\ c-tail\ child-costs-tail \implies$
 $c = c-pull + c-edges + c-sources + c-child + c-tail \implies$
 $exact-split-range-costed-partition-loop-state\ \Delta\ M-of\ t\ h\ k\ cap\ l\ d\ P\ B\ d'\ D\ a$
 $(beta\ \# \ betas)\ (b\ \# \ bs)\ B'$
 $(range-tree\ P\ a\ (Fin\ b)\ \# \ Us-tail)$
 $(bound-tree\ P\ (Fin\ a) \cup$
 $\bigcup (set\ (range-tree\ P\ a\ (Fin\ b)\ \# \ Us-tail)))\ c$
 $(c-child\ \# \ child-costs-tail)$
| *Exact-Split-Range-Cost-Base:*
 $S = \{x\} \implies$
 $exact-split-range-costed-bmssp\ \Delta\ M-of\ t\ h\ k\ cap\ 0\ d\ S\ B$
 $(\lambda v. if\ v \in base-case-vertices\ k\ x\ B\ then\ dist\ s\ v\ else\ d\ v)$
 $(base-case-bound\ k\ x\ B)$
 $(base-case-vertices\ k\ x\ B)$
 $(base-case-scan-cost\ \Delta\ k\ x\ B)$
| *Exact-Split-Range-Cost-Step:*
 $D = label-partition-view$
 $(find-pivots-label-capped\ k\ cap\ d\ S\ B)$
 $(find-pivots-pivots-capped\ k\ cap\ d\ S\ B) \implies$
 $partition-initial-insert-cost-bound\ c-insert\ t$
 $(find-pivots-pivots-capped\ k\ cap\ d\ S\ B) \implies$
 $exact-split-range-costed-partition-loop-state\ \Delta\ M-of\ t\ h\ k\ cap\ l$

$(\text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \text{ } B \text{ } d' \text{ } D \text{ } a \text{ } \text{betas } bs \text{ } B'$
 $Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs-loop} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\} \implies$
 $c = \text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} + c\text{-loop} \implies$
 $\text{exact-split-range-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } (Suc \text{ } l) \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$

The defining difference from the split-range relation is the premise $S\text{-pull} = \text{split-below } d \text{ } P \text{ } beta$ in the loop step. This says that the implementation-level pull has not merely produced an admissible child set; it has produced exactly the semantic lower split of the parent problem. The rest of the constructor mirrors the earlier relation: edge and source batches are charged separately, the child output is the next *range-tree*, and the total cost is the sum of local and recursive costs.

The equality to *split-below* is deliberately stored at the point where the pull happens. Downstream proofs then extract it from the derivation tree rather than reconstructing it from the partition invariant. This keeps the cost proofs focused on summing costs and ranges.

inductive-cases $\text{exact-split-range-costed-bmssp-zeroE}$:
 $\text{exact-split-range-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } 0 \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$

inductive-cases $\text{exact-split-range-costed-bmssp-SucE}$:
 $\text{exact-split-range-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } (Suc \text{ } l) \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$

theorem $\text{exact-split-range-costed-refines-split-range-costed}$:

shows

$\text{exact-split-range-costed-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } l \text{ } d \text{ } P \text{ } B \text{ } d' \text{ } D \text{ } a$
 $\text{betas } bs \text{ } B' \text{ } Us \text{ } U \text{ } c \text{ } \text{child-costs} \implies$
 $\text{split-range-costed-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } l \text{ } d \text{ } P \text{ } B \text{ } d' \text{ } D \text{ } a$
 $\text{betas } bs \text{ } B' \text{ } Us \text{ } U \text{ } c \text{ } \text{child-costs}$

and

$\text{exact-split-range-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } l \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c \implies$
 $\text{split-range-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } l \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$

proof (*induction rule*:

$\text{exact-split-range-costed-partition-loop-state-exact-split-range-costed-bmssp.inducts}$)

case ($\text{Exact-Split-Range-State-Done } D \text{ } a \text{ } B \text{ } d' \text{ } P \text{ } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } l \text{ } d$)

show $?case$

by (*rule Split-Range-State-Done*)

(*use Exact-Split-Range-State-Done in auto*)

next

case ($\text{Exact-Split-Range-State-Stop } a \text{ } B \text{ } d' \text{ } P \text{ } k \text{ cap } \Delta \text{ } M\text{-of } t \text{ } h \text{ } l \text{ } d \text{ } D$)

show $?case$

by (*rule Split-Range-State-Stop*)

(*use Exact-Split-Range-State-Stop in auto*)


```

next
  case (Exact-Split-Range-State-Step  $D$   $M$ -of  $l$   $B_{\max}$   $S$ -pull  $\beta$   $D$ -pull  $B$   $d$ 
     $P$   $a$   $b$   $B'$   $d'$   $k$   $\text{cap}$   $\Delta$   $t$   $h$   $d$ -child  $U$ -child  $c$ -child edge-batch
    source-batch batch  $D$ -next  $c$ -pull  $c$ -edges  $c$ -sources  $\beta$   $s$   $U$ -tail
     $U$ -tail  $c$ -tail child-costs-tail  $c$ )
  show ?case
  by (rule Split-Range-State-Step)
    (use Exact-Split-Range-State-Step in auto)
next
  case (Exact-Split-Range-Cost-Base  $S$   $x$   $\Delta$   $M$ -of  $t$   $h$   $k$   $\text{cap}$   $d$   $B$ )
  show ?case
  by (rule Split-Range-Cost-Base)
    (use Exact-Split-Range-Cost-Base in auto)
next
  case (Exact-Split-Range-Cost-Step  $D$   $k$   $\text{cap}$   $d$   $S$   $B$   $c$ -insert  $t$   $\Delta$   $M$ -of  $h$   $l$ 
     $d'$   $a$   $\beta$   $s$   $B'$   $U$   $U$ -loop  $c$ -loop child-costs-loop  $U$   $c$ )
  show ?case
  by (rule Split-Range-Cost-Step)
    (use Exact-Split-Range-Cost-Step in auto)
qed

```

theorem *exact-split-range-costed-bmssp-correct*:

```

assumes run:
  exact-split-range-costed-bmssp  $\Delta$   $M$ -of  $t$   $h$   $k$   $\text{cap}$   $l$   $d$   $S$   $B$   $d'$   $B'$   $U$   $c$ 
and sound: sound-label  $d$ 
and pre: bmssp-pre-full  $d$   $S$   $B$ 
and  $S$ -reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
shows bmssp-post-full  $d$   $S$   $B$   $d'$   $B'$   $U$ 
by (rule split-range-costed-bmssp-correct
  [OF exact-split-range-costed-refines-split-range-costed(2)] [OF run]
  sound pre S-reaches)

```

theorem *finite-initial-label-exact-split-range-costed-top-level-correct*:

```

assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
and run:
  exact-split-range-costed-bmssp  $\Delta$   $M$ -of  $t$   $h$   $k$   $\text{cap}$   $l$ 
  finite-initial-label  $\{s\}$  Infinity  $d'$  Infinity  $U$   $c$ 
shows sssp-correct  $d'$ 
by (rule finite-initial-label-split-range-costed-top-level-correct
  [OF all-reachable exact-split-range-costed-refines-split-range-costed(2)]
  [OF run])

```

The refinement theorem *exact-split-range-costed-partition-loop-state* ? Δ ? M -of ? t ? h ? k ? cap ? l ? d ? P ? B ? d' ? D ? a ? β s ? bs ? B' ? Us ? U ? c ?*child-costs* $\implies$ *split-range-costed-partition-loop-state* ? Δ ? M -of ? t ? h ? k ? cap ? l ? d ? P ? B ? d' ? D ? a ? β s ? bs ? B' ? Us ? U ? c ?*child-costs* is the compatibility layer: forgetting the exact source equality gives a valid split-range run. Correctness and top-level SSSP correctness are then inherited immediately from the previous theory.

The remaining results use the exact source equality in the other direction. They expose the child calls together with the lower-split facts needed for the induction hypothesis: bounded cardinality, reachability, and labels below the child bound. Those facts are the source-progress side of the runtime proof.

theorem *exact-split-range-costed-partition-loop-state-child-cost-calls-below*:

exact-split-range-costed-partition-loop-state Δ *M-of t h k cap l d P B d' D a*

betas bs B' Us U c child-costs $\implies$

length child-costs = *length bs* $\wedge$

list-all2

(λc -child *U-child*. $\exists$ *S-child B-child d-child B-child'*.

exact-split-range-costed-bmssp Δ *M-of t h k cap l d S-child B-child*

d-child B-child' U-child c-child $\wedge$

bmssp-pre-full d S-child B-child $\wedge$

($\forall x \in S\text{-child. reachable } s \ x$) $\wedge$

card S-child $\leq$ *M-of l* $\wedge$

($\forall x \in S\text{-child. below-bound } (d \ x) \ B\text{-child}$))

child-costs (*range-tree-child-list P a bs*)

and *exact-split-range-costed-bmssp-child-cost-calls-below-trivial*:

exact-split-range-costed-bmssp Δ *M-of t h k cap l d S B d' B' U c* $\implies$ *True*

proof (*induction rule*:

exact-split-range-costed-partition-loop-state-exact-split-range-costed-bmssp.inducts)

case (*Exact-Split-Range-State-Done D a B d' P* Δ *M-of t h k cap l d*)

show ?*case*

by *simp*

next

case (*Exact-Split-Range-State-Stop a B d' P k cap* Δ *M-of t h l d D*)

show ?*case*

by *simp*

next

case (*Exact-Split-Range-State-Step D M-of l Bmax S-pull beta D-pull B d*

P a b B' d' k cap Δ *t h d-child U-child c-child edge-batch*

source-batch batch D-next c-pull c-edges c-sources betas bs Us-tail

U-tail c-tail child-costs-tail c)

have *tail*:

length child-costs-tail = *length bs* $\wedge$

list-all2

(λc -child *U-child*. $\exists$ *S-child B-child d-child B-child'*.

exact-split-range-costed-bmssp Δ *M-of t h k cap l d S-child B-child*

d-child B-child' U-child c-child $\wedge$

bmssp-pre-full d S-child B-child $\wedge$

($\forall x \in S\text{-child. reachable } s \ x$) $\wedge$

card S-child $\leq$ *M-of l* $\wedge$

($\forall x \in S\text{-child. below-bound } (d \ x) \ B\text{-child}$))

child-costs-tail (*range-tree-child-list P b bs*)

using *Exact-Split-Range-State-Step.IH* **by** *blast*

have *card-pull*: *card S-pull* $\leq$ *M-of l*

using *Exact-Split-Range-State-Step*

unfolding *pull-separates-def* **by** *blast*

```

have below-pull:  $\forall x \in S\text{-pull}. \text{below-bound } (d \ x) \ (Fin \ \text{beta})$ 
proof
  fix  $x$ 
  assume  $xS: x \in S\text{-pull}$ 
  have  $S\text{-pull} = \text{split-below } d \ P \ \text{beta}$ 
    using Exact-Split-Range-State-Step by blast
  then show  $\text{below-bound } (d \ x) \ (Fin \ \text{beta})$ 
    using  $xS$  unfolding split-below-def by auto
qed
have head:
   $\exists S\text{-child } B\text{-child } d\text{-child}' \ B\text{-child}'.$ 
   $\text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$ 
   $d\text{-child}' \ B\text{-child}' \ U\text{-child } c\text{-child} \wedge$ 
   $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child} \wedge$ 
   $(\forall x \in S\text{-child}. \text{reachable } s \ x) \wedge$ 
   $\text{card } S\text{-child} \leq M\text{-of } l \wedge$ 
   $(\forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child})$ 
  using Exact-Split-Range-State-Step card-pull below-pull by blast
show ?case
  using tail head Exact-Split-Range-State-Step by simp
next
  case (Exact-Split-Range-Cost-Base  $S \ x \ \Delta \ M\text{-of } t \ h \ k \ \text{cap } d \ B$ )
  show ?case
    by simp
next
  case (Exact-Split-Range-Cost-Step  $D \ k \ \text{cap } d \ S \ B \ c\text{-insert } t \ \Delta \ M\text{-of } h \ l$ 
     $d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs-loop } U \ c$ )
  show ?case
    by simp
qed

theorem exact-split-range-costed-partition-loop-state-child-cost-bounds:
assumes run:
   $\text{exact-split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$ 
   $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs}$ 
and child-bound:
   $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
   $\llbracket \text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$ 
   $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$ 
   $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child};$ 
   $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$ 
   $\text{card } S\text{-child} \leq M\text{-of } l;$ 
   $\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child} \rrbracket$ 
   $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$ 
shows list-all2  $(\lambda c\text{-child } U\text{-child}.$ 
   $c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child})$ 
   $\text{child-costs } (\text{range-tree-child-list } P \ a \ bs)$ 
proof –
  have calls:

```

list-all2
 $(\lambda c\text{-child } U\text{-child}. \exists S\text{-child } B\text{-child } d\text{-child } B\text{-child}'. \\
\text{exact-split-range-costed-bmssp } \Delta M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child} \\
d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child} \wedge \\
\text{bmssp-pre-full } d \ S\text{-child } B\text{-child} \wedge \\
(\forall x \in S\text{-child}. \text{reachable } s \ x) \wedge \\
\text{card } S\text{-child} \leq M\text{-of } l \wedge \\
(\forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child})) \\
\text{child-costs } (\text{range-tree-child-list } P \ a \ bs) \\
\text{using exact-split-range-costed-partition-loop-state-child-cost-calls-below} \\
[OF \text{ run}] \text{ by blast} \\
\text{then show ?thesis} \\
\text{proof (induction rule: list-all2-induct)} \\
\text{case Nil} \\
\text{then show ?case by simp} \\
\text{next} \\
\text{case (Cons c-child child-costs U-child child-ranges)} \\
\text{obtain } S\text{-child } B\text{-child } d\text{-child } B\text{-child}' \text{ where child:} \\
\text{exact-split-range-costed-bmssp } \Delta M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child} \\
d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child} \\
\text{and child-pre: bmssp-pre-full } d \ S\text{-child } B\text{-child} \\
\text{and child-reaches: } \forall x \in S\text{-child}. \text{reachable } s \ x \\
\text{and child-card: card } S\text{-child} \leq M\text{-of } l \\
\text{and child-below: } \forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child} \\
\text{using Cons.hyps by blast} \\
\text{have head:} \\
c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child} \\
\text{by (rule child-bound[OF child child-pre - child-card])} \\
(\text{use child-reaches child-below in blast})+ \\
\text{show ?case} \\
\text{using head Cons.IH Cons.hyps by simp} \\
\text{qed} \\
\text{qed}$

theorem *exact-split-range-costed-partition-loop-state-child-cost-bounds-with-invariants:*
 $\text{exact-split-range-costed-partition-loop-state } \Delta M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a \\
\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs} \implies \\
P \subseteq V \implies \\
k * \text{card } P \leq \text{cap} \implies \\
\text{tree-antichain } P \implies \\
(\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'. \\
\llbracket \text{exact-split-range-costed-bmssp } \Delta M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child} \\
d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}; \\
\text{bmssp-pre-full } d \ S\text{-child } B\text{-child}; \\
\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x; \\
\text{card } S\text{-child} \leq M\text{-of } l; \\
\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child}; \\
k * \text{card } S\text{-child} \leq \text{cap};
\rrbracket$

```

      tree-antichain S-child]]
     $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child} \implies$ 
    list-all2 ( $\lambda c\text{-child } U\text{-child}.$ 
       $c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$ )
    child-costs (range-tree-child-list P a bs)
  and exact-split-range-costed-bmssp-child-cost-bounds-with-invariants-trivial:
    exact-split-range-costed-bmssp  $\Delta$  M-of t h k cap l d S B d' B' U c  $\implies$  True
  proof (induction rule:
    exact-split-range-costed-partition-loop-state-exact-split-range-costed-
    bmssp.inducts)
    case (Exact-Split-Range-State-Done D a B d' P  $\Delta$  M-of t h k cap l d)
    then show ?case
      by simp
    next
    case (Exact-Split-Range-State-Stop a B d' P k cap  $\Delta$  M-of t h l d D)
    then show ?case
      by simp
    next
    case (Exact-Split-Range-State-Step D M-of l Bmax S-pull beta D-pull B d
      P a b B' d' k cap  $\Delta$  t h d-child U-child c-child edge-batch
      source-batch batch D-next c-pull c-edges c-sources betas bs Us-tail
      U-tail c-tail child-costs-tail c)
    have tail:
      list-all2 ( $\lambda c\text{-child } U\text{-child}.$ 
         $c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$ )
      child-costs-tail (range-tree-child-list P b bs)
    using Exact-Split-Range-State-Step.IH Exact-Split-Range-State-Step.prem
    by blast
    have card-pull: card S-pull  $\leq$  M-of l
    using Exact-Split-Range-State-Step
    unfolding pull-separates-def by blast
    have below-pull:  $\bigwedge x. x \in S\text{-pull} \implies \text{below-bound } (d \ x) \ (Fin \ beta)$ 
    proof -
      fix x
      assume xS:  $x \in S\text{-pull}$ 
      have S-pull = split-below d P beta
      using Exact-Split-Range-State-Step by blast
      then show below-bound (d x) (Fin beta)
      using xS unfolding split-below-def by auto
    qed
    have pull-k-cap:  $k * \text{card } S\text{-pull} \leq \text{cap}$ 
    proof -
      have S-pull = split-below d P beta
      using Exact-Split-Range-State-Step by blast
      then show ?thesis
      using split-below-scaled-card-le
      [OF Exact-Split-Range-State-Step.prem(1)
        Exact-Split-Range-State-Step.prem(2), of d beta]
      by simp

```

```

qed
have pull-anti: tree-antichain S-pull
proof -
  have S-pull = split-below d P beta
  using Exact-Split-Range-State-Step by blast
  then show ?thesis
  using split-below-tree-antichain
  [OF Exact-Split-Range-State-Step.prem3, of d beta]
  by simp
qed
have child-run:
  exact-split-range-costed-bmssp  $\Delta$  M-of t h k cap l d S-pull (Fin beta)
  d-child (Fin b) U-child c-child
  using Exact-Split-Range-State-Step by blast
have pre-pull: bmssp-pre-full d S-pull (Fin beta)
  using Exact-Split-Range-State-Step by blast
have reaches-pull:  $\bigwedge x. x \in S\text{-pull} \implies \text{reachable } s \ x$ 
  using Exact-Split-Range-State-Step by blast
have head:
  c-child  $\leq$  level-range-cost-bound A R L U-child
  by (rule Exact-Split-Range-State-Step.prem4)
  [OF child-run pre-pull reaches-pull card-pull below-pull
  pull-k-cap pull-anti])
show ?case
  using head tail Exact-Split-Range-State-Step by simp
next
case (Exact-Split-Range-Cost-Base S x  $\Delta$  M-of t h k cap d B)
then show ?case
  by simp
next
case (Exact-Split-Range-Cost-Step D k cap d S B c-insert t  $\Delta$  M-of h l
  d' a betas bs B' Us U-loop c-loop child-costs-loop U c)
then show ?case
  by simp
qed

```

The child-source extraction theorem packages one loop trace as a list of source sets aligned with *range-tree-child-list*. Alignment matters: each source set is charged against the same range slice as its recursive child call. Once this list exists, subsequent cost theorems can use ordinary list-wise reasoning instead of induction over the full operational relation.

theorem *exact-split-range-costed-partition-loop-state-child-sources-below:*
 $\text{exact-split-range-costed-partition-loop-state } \Delta \text{ M-of t h k cap l d P B d' D a}$
 $\text{betas bs B' Us U c child-costs} \implies$
 $\exists \text{ child-sources.}$
 $\text{length child-sources} = \text{length bs} \wedge$
 list-all2
 $(\lambda S\text{-child U-child. } \exists c\text{-child B-child d-child B-child'.$
 $\text{exact-split-range-costed-bmssp } \Delta \text{ M-of t h k cap l d S-child B-child}$

$d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child} \wedge$
 $bmssp\text{-pre-full } d \ S\text{-child } B\text{-child} \wedge$
 $(\forall x \in S\text{-child}. \text{reachable } s \ x) \wedge$
 $\text{card } S\text{-child} \leq M\text{-of } l \wedge$
 $(\forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child}))$
 $\text{child-sources } (\text{range-tree-child-list } P \ a \ bs)$
and $\text{exact-split-range-costed-bmssp-child-sources-below-trivial:}$
 $\text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies \text{True}$
proof (*induction rule:*
 $\text{exact-split-range-costed-partition-loop-state-exact-split-range-costed-}$
 $bmssp.inducts$)
case ($\text{Exact-Split-Range-State-Done } D \ a \ B \ d' \ P \ \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d$)
show $?case$
by ($\text{intro } exI[\text{of } - \ \square] \text{ simp}$)
next
case ($\text{Exact-Split-Range-State-Stop } a \ B \ d' \ P \ k \ \text{cap } \Delta \ M\text{-of } t \ h \ l \ d \ D$)
show $?case$
by ($\text{intro } exI[\text{of } - \ \square] \text{ simp}$)
next
case ($\text{Exact-Split-Range-State-Step } D \ M\text{-of } l \ B\text{max } S\text{-pull } \beta \ D\text{-pull } B \ d$
 $P \ a \ b \ B' \ d' \ k \ \text{cap } \Delta \ t \ h \ d\text{-child } U\text{-child } c\text{-child } \text{edge-batch}$
 $\text{source-batch } \text{batch } D\text{-next } c\text{-pull } c\text{-edges } c\text{-sources } \beta \ bs \ Us\text{-tail}$
 $U\text{-tail } c\text{-tail } \text{child-costs-tail } c$)
obtain $\text{child-sources-tail}$ **where** len-tail:
 $\text{length } \text{child-sources-tail} = \text{length } bs$
and tail:
 list-all2
 $(\lambda S\text{-child } U\text{-child}. \exists c\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child} \wedge$
 $bmssp\text{-pre-full } d \ S\text{-child } B\text{-child} \wedge$
 $(\forall x \in S\text{-child}. \text{reachable } s \ x) \wedge$
 $\text{card } S\text{-child} \leq M\text{-of } l \wedge$
 $(\forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child}))$
 $\text{child-sources-tail } (\text{range-tree-child-list } P \ b \ bs)$
using $\text{Exact-Split-Range-State-Step.IH}$ **by** blast
have $\text{card-pull: } \text{card } S\text{-pull} \leq M\text{-of } l$
using $\text{Exact-Split-Range-State-Step}$
unfolding $\text{pull-separates-def}$ **by** blast
have $\text{below-pull: } \forall x \in S\text{-pull}. \text{below-bound } (d \ x) \ (\text{Fin } \beta)$
proof
fix x
assume $xS: x \in S\text{-pull}$
have $S\text{-pull} = \text{split-below } d \ P \ \beta$
using $\text{Exact-Split-Range-State-Step}$ **by** blast
then show $\text{below-bound } (d \ x) \ (\text{Fin } \beta)$
using xS **unfolding** split-below-def **by** auto
qed
let $?child\text{-sources} = S\text{-pull} \ \# \ \text{child-sources-tail}$

```

have head:
   $\exists c\text{-child}' B\text{-child } d\text{-child}' B\text{-child}'.$ 
   $\text{exact-split-range-costed-bmssp } \Delta M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-pull } B\text{-child}$ 
   $d\text{-child}' B\text{-child}' U\text{-child } c\text{-child}' \wedge$ 
   $\text{bmssp-pre-full } d \ S\text{-pull } B\text{-child} \wedge$ 
   $(\forall x \in S\text{-pull}. \text{reachable } s \ x) \wedge$ 
   $\text{card } S\text{-pull} \leq M\text{-of } l \wedge$ 
   $(\forall x \in S\text{-pull}. \text{below-bound } (d \ x) \ B\text{-child})$ 
  using Exact-Split-Range-State-Step card-pull below-pull
  by blast
show ?case
  by (intro exI[of - ?child-sources] conjI)
  (use len-tail head tail Exact-Split-Range-State-Step in simp-all)
next
  case (Exact-Split-Range-Cost-Base  $S \ x \ \Delta M\text{-of } t \ h \ k \ \text{cap } d \ B$ )
  show ?case
  by simp
next
  case (Exact-Split-Range-Cost-Step  $D \ k \ \text{cap } d \ S \ B \ c\text{-insert } t \ \Delta M\text{-of } h \ l$ 
     $d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs-loop } U \ c$ )
  show ?case
  by simp
qed

```

The next cost bounds combine the extracted child sources with the explicit edge and source batch costs stored in the relation. First the local loop cost is bounded by the sum of child costs, source cardinalities, and outgoing-edge charges over the child ranges. Then the range-chain lemmas replace those list sums by the corresponding counts for the parent loop output.

theorem *exact-split-range-costed-partition-loop-state-cost-bound-by-child-sources-below:*

```

 $\text{exact-split-range-costed-partition-loop-state } \Delta M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$ 
 $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs} \implies$ 
 $\exists \text{child-sources}.$ 
 $\text{length child-sources} = \text{length } bs \wedge$ 
 $\text{list-all2}$ 
 $(\lambda S\text{-child } U\text{-child}. \exists c\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
   $\text{exact-split-range-costed-bmssp } \Delta M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$ 
   $d\text{-child } B\text{-child}' U\text{-child } c\text{-child} \wedge$ 
   $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child} \wedge$ 
   $(\forall x \in S\text{-child}. \text{reachable } s \ x) \wedge$ 
   $\text{card } S\text{-child} \leq M\text{-of } l \wedge$ 
   $(\forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child}))$ 
 $\text{child-sources } (\text{range-tree-child-list } P \ a \ bs) \wedge$ 
 $c \leq \text{sum-list child-costs} + \text{sum-list } (\text{map card child-sources}) +$ 
 $t * \text{sum-list}$ 
 $(\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) (\text{range-tree-child-list } P \ a \ bs)) +$ 
 $h * \text{sum-list } (\text{map card child-sources})$ 
and exact-split-range-costed-bmssp-cost-child-sources-below-trivial:

```


$exact-split-range-costed-bmssp \Delta M\text{-of } t \ h \ k \ cap \ l \ d \ S \ B \ d' \ B' \ U \ c \implies True$
proof (induction rule:
 $exact-split-range-costed-partition-loop-state-exact-split-range-costed-$
 $bmssp.inducts$)
case ($Exact-Split-Range-State-Done \ D \ a \ B \ d' \ P \ \Delta \ M\text{-of } t \ h \ k \ cap \ l \ d$)
show $?case$
by ($intro \ exI[of - \ \square] \ simp$)
next
case ($Exact-Split-Range-State-Stop \ a \ B \ d' \ P \ k \ cap \ \Delta \ M\text{-of } t \ h \ l \ d \ D$)
show $?case$
by ($intro \ exI[of - \ \square] \ simp$)
next
case ($Exact-Split-Range-State-Step \ D \ M\text{-of } l \ Bmax \ S\text{-pull} \ beta \ D\text{-pull} \ B \ d$
 $P \ a \ b \ B' \ d' \ k \ cap \ \Delta \ t \ h \ d\text{-child} \ U\text{-child} \ c\text{-child} \ edge\text{-batch}$
 $source\text{-batch} \ batch \ D\text{-next} \ c\text{-pull} \ c\text{-edges} \ c\text{-sources} \ betas \ bs \ Us\text{-tail}$
 $U\text{-tail} \ c\text{-tail} \ child\text{-costs}\text{-tail} \ c$)
obtain $child\text{-sources}\text{-tail}$ **where** $len\text{-tail}$:
 $length \ child\text{-sources}\text{-tail} = length \ bs$
and $sources\text{-tail}$:
 $list\text{-all2}$
 $(\lambda S\text{-child} \ U\text{-child}. \exists c\text{-child} \ B\text{-child} \ d\text{-child} \ B\text{-child}')$
 $exact-split-range-costed-bmssp \Delta M\text{-of } t \ h \ k \ cap \ l \ d \ S\text{-child} \ B\text{-child}$
 $d\text{-child} \ B\text{-child}' \ U\text{-child} \ c\text{-child} \wedge$
 $bmssp\text{-pre}\text{-full} \ d \ S\text{-child} \ B\text{-child} \wedge$
 $(\forall x \in S\text{-child}. reachable \ s \ x) \wedge$
 $card \ S\text{-child} \leq M\text{-of } l \wedge$
 $(\forall x \in S\text{-child}. below\text{-bound} \ (d \ x) \ B\text{-child}))$
 $child\text{-sources}\text{-tail} \ (range\text{-tree}\text{-child}\text{-list} \ P \ b \ bs)$
and $tail\text{-cost}$:
 $c\text{-tail} \leq$
 $sum\text{-list} \ child\text{-costs}\text{-tail} + sum\text{-list} \ (map \ card \ child\text{-sources}\text{-tail}) +$
 $t * sum\text{-list}$
 $(map \ (\lambda X. card \ (outgoing\text{-edges} \ X)) \ (range\text{-tree}\text{-child}\text{-list} \ P \ b \ bs)) +$
 $h * sum\text{-list} \ (map \ card \ child\text{-sources}\text{-tail})$
using $Exact-Split-Range-State-Step.IH$ **by** $blast$
have $card\text{-pull}$: $card \ S\text{-pull} \leq M\text{-of } l$
using $Exact-Split-Range-State-Step$
unfolding $pull\text{-separates}\text{-def}$ **by** $blast$
have $below\text{-pull}$: $\forall x \in S\text{-pull}. below\text{-bound} \ (d \ x) \ (Fin \ beta)$
proof
fix x
assume xS : $x \in S\text{-pull}$
have $S\text{-pull} = split\text{-below} \ d \ P \ beta$
using $Exact-Split-Range-State-Step$ **by** $blast$
then show $below\text{-bound} \ (d \ x) \ (Fin \ beta)$
using xS **unfolding** $split\text{-below}\text{-def}$ **by** $auto$
qed
have $pull\text{-cost}$: $c\text{-pull} \leq card \ S\text{-pull}$
using $Exact-Split-Range-State-Step$

unfolding *partition-pull-cost-bound-def* **by** *blast*
have *batch-cost*:
 $c\text{-edges} + c\text{-sources} \leq$
 $t * \text{card } (\text{outgoing-edges } U\text{-child}) + h * \text{card } S\text{-pull}$
by (*rule range-costed-split-batch-cost-le-card* [
where $d = d$ **and** $S = S\text{-pull}$ **and** $d\text{-child} = d\text{-child}$ **and** $b = b$
and $\text{beta} = \text{beta}$])
(use Exact-Split-Range-State-Step in auto)
let $?child\text{-sources} = S\text{-pull} \# child\text{-sources-tail}$
have $\text{len: length } ?child\text{-sources} = \text{length } (b \# bs)$
using *len-tail* **by** *simp*
have *head-source*:
 $\exists c\text{-child}' B\text{-child } d\text{-child}' B\text{-child}'.$
 $\text{exact-split-range-costed-bmssp } \Delta M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-pull } B\text{-child}$
 $d\text{-child}' B\text{-child}' U\text{-child } c\text{-child}' \wedge$
 $\text{bmssp-pre-full } d \ S\text{-pull } B\text{-child} \wedge$
 $(\forall x \in S\text{-pull. reachable } s \ x) \wedge$
 $\text{card } S\text{-pull} \leq M\text{-of } l \wedge$
 $(\forall x \in S\text{-pull. below-bound } (d \ x) \ B\text{-child})$
using *Exact-Split-Range-State-Step card-pull below-pull* **by** *blast*
have *sources*:
list-all2
 $(\lambda S\text{-child } U\text{-child. } \exists c\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\text{exact-split-range-costed-bmssp } \Delta M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' U\text{-child } c\text{-child} \wedge$
 $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child} \wedge$
 $(\forall x \in S\text{-child. reachable } s \ x) \wedge$
 $\text{card } S\text{-child} \leq M\text{-of } l \wedge$
 $(\forall x \in S\text{-child. below-bound } (d \ x) \ B\text{-child}))$
 $?child\text{-sources } (\text{range-tree-child-list } P \ a \ (b \# bs))$
using *head-source sources-tail Exact-Split-Range-State-Step* **by** *simp*
have *cost*:
 $c \leq \text{sum-list } (c\text{-child} \# child\text{-costs-tail}) +$
 $\text{sum-list } (\text{map card } ?child\text{-sources}) +$
 $t * \text{sum-list}$
 $(\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X))$
 $(\text{range-tree-child-list } P \ a \ (b \# bs))) +$
 $h * \text{sum-list } (\text{map card } ?child\text{-sources})$
proof –
have *c-eq*: $c = c\text{-pull} + c\text{-edges} + c\text{-sources} + c\text{-child} + c\text{-tail}$
using *Exact-Split-Range-State-Step* **by** *blast*
have $c\text{-pull} + c\text{-edges} + c\text{-sources} + c\text{-child} + c\text{-tail} \leq$
 $\text{card } S\text{-pull} +$
 $(t * \text{card } (\text{outgoing-edges } U\text{-child}) + h * \text{card } S\text{-pull}) +$
 $c\text{-child} +$
 $(\text{sum-list } child\text{-costs-tail} + \text{sum-list } (\text{map card } child\text{-sources-tail}) +$
 $t * \text{sum-list}$
 $(\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) (\text{range-tree-child-list } P \ b \ bs)) +$
 $h * \text{sum-list } (\text{map card } child\text{-sources-tail}))$

```

    using pull-cost batch-cost tail-cost by linarith
  also have ... =
    sum-list (c-child # child-costs-tail) +
    sum-list (map card ?child-sources) +
    t * (card (outgoing-edges U-child) +
    sum-list
      (map (λX. card (outgoing-edges X)) (range-tree-child-list P b bs))) +
    h * sum-list (map card ?child-sources)
  by (simp add: algebra-simps)
  also have ... =
    sum-list (c-child # child-costs-tail) +
    sum-list (map card ?child-sources) +
    t * sum-list
      (map (λX. card (outgoing-edges X))
        (range-tree-child-list P a (b # bs))) +
    h * sum-list (map card ?child-sources)
  using Exact-Split-Range-State-Step by simp
  finally show ?thesis
  using c-eq by simp
qed
show ?case
  by (intro exI[of - ?child-sources] conjI)
    (use len sources cost in simp-all)
next
  case (Exact-Split-Range-Cost-Base S x Δ M-of t h k cap d B)
  show ?case
    by simp
next
  case (Exact-Split-Range-Cost-Step D k cap d S B c-insert t Δ M-of h l
    d' a betas bs B' Us U-loop c-loop child-costs-loop U c)
  show ?case
    by simp
qed

theorem exact-split-range-costed-partition-loop-state-cost-bound-by-child-sources-
and-edge-ranges:
  exact-split-range-costed-partition-loop-state Δ M-of t h k cap l d P B d' D a
    betas bs B' Us U c child-costs  $\implies$ 
  sound-label d  $\implies$ 
   $\exists$  child-sources.
    length child-sources = length bs  $\wedge$ 
    list-all2
      (λS-child U-child.  $\exists$  c-child B-child d-child B-child'.
        exact-split-range-costed-bmssp Δ M-of t h k cap l d S-child B-child
          d-child B-child' U-child c-child  $\wedge$ 
        bmssp-pre-full d S-child B-child  $\wedge$ 
        ( $\forall x \in S\text{-child. reachable } s \ x$ )  $\wedge$ 
        card S-child  $\leq$  M-of l  $\wedge$ 
        ( $\forall x \in S\text{-child. below-bound } (d \ x) \ B\text{-child}$ ))

```

$child-sources \ (range-tree-child-list \ P \ a \ bs) \wedge$
 $c \leq sum-list \ child-costs + sum-list \ (map \ card \ child-sources) +$
 $t * sum-list$
 $(map \ card \ (range-tree-child-edge-range-list \ P \ a \ betas \ bs)) +$
 $h * sum-list \ (map \ card \ child-sources)$
and *exact-split-range-costed-bmssp-cost-child-sources-and-edge-ranges-trivial:*
 $exact-split-range-costed-bmssp \ \Delta \ M-of \ t \ h \ k \ cap \ l \ d \ S \ B \ d' \ B' \ U \ c \implies \ True$
proof (*induction rule:*
 $exact-split-range-costed-partition-loop-state-exact-split-range-costed-$
 $bmssp.inducts$)
case (*Exact-Split-Range-State-Done* $D \ a \ B \ d' \ P \ \Delta \ M-of \ t \ h \ k \ cap \ l \ d$)
show ?case
by (*intro* $exI[of - []]$) *simp*
next
case (*Exact-Split-Range-State-Stop* $a \ B \ d' \ P \ k \ cap \ \Delta \ M-of \ t \ h \ l \ d \ D$)
show ?case
by (*intro* $exI[of - []]$) *simp*
next
case (*Exact-Split-Range-State-Step* $D \ M-of \ l \ Bmax \ S-pull \ beta \ D-pull \ B \ d$
 $P \ a \ b \ B' \ d' \ k \ cap \ \Delta \ t \ h \ d-child \ U-child \ c-child \ edge-batch$
 $source-batch \ batch \ D-next \ c-pull \ c-edges \ c-sources \ betas \ bs \ Us-tail$
 $U-tail \ c-tail \ child-costs-tail \ c$)
obtain *child-sources-tail* **where** *len-tail:*
 $length \ child-sources-tail = length \ bs$
and *sources-tail:*
 $list-all2$
 $(\lambda S-child \ U-child. \ \exists \ c-child \ B-child \ d-child \ B-child'. \$
 $exact-split-range-costed-bmssp \ \Delta \ M-of \ t \ h \ k \ cap \ l \ d \ S-child \ B-child$
 $d-child \ B-child' \ U-child \ c-child \wedge$
 $bmssp-pre-full \ d \ S-child \ B-child \wedge$
 $(\forall x \in S-child. \ reachable \ s \ x) \wedge$
 $card \ S-child \leq M-of \ l \wedge$
 $(\forall x \in S-child. \ below-bound \ (d \ x) \ B-child))$
 $child-sources-tail \ (range-tree-child-list \ P \ b \ bs)$
and *tail-cost:*
 $c-tail \leq$
 $sum-list \ child-costs-tail + sum-list \ (map \ card \ child-sources-tail) +$
 $t * sum-list$
 $(map \ card \ (range-tree-child-edge-range-list \ P \ b \ betas \ bs)) +$
 $h * sum-list \ (map \ card \ child-sources-tail)$
using *Exact-Split-Range-State-Step.IH*
 $Exact-Split-Range-State-Step.premis$ **by** *blast*
have *card-pull:* $card \ S-pull \leq M-of \ l$
using *Exact-Split-Range-State-Step*
unfolding *pull-separates-def* **by** *blast*
have *below-pull:* $\forall x \in S-pull. \ below-bound \ (d \ x) \ (Fin \ beta)$
proof
fix x
assume $xS: x \in S-pull$

```

have  $S\text{-pull} = \text{split-below } d \ P \ \text{beta}$ 
  using Exact-Split-Range-State-Step by blast
then show  $\text{below-bound } (d \ x) \ (\text{Fin } \text{beta})$ 
  using  $xS$  unfolding split-below-def by auto
qed
have  $\text{pull-cost: } c\text{-pull} \leq \text{card } S\text{-pull}$ 
  using Exact-Split-Range-State-Step
  unfolding partition-pull-cost-bound-def by blast
have child-run:
   $\text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-pull} \ (\text{Fin } \text{beta})$ 
   $d\text{-child } (\text{Fin } b) \ U\text{-child } c\text{-child}$ 
  using Exact-Split-Range-State-Step by blast
have child-pre:  $\text{bmssp-pre-full } d \ S\text{-pull} \ (\text{Fin } \text{beta})$ 
  using Exact-Split-Range-State-Step by blast
have child-reaches:  $\bigwedge x. x \in S\text{-pull} \implies \text{reachable } s \ x$ 
  using Exact-Split-Range-State-Step by blast
have child-post:
   $\text{bmssp-post-full } d \ S\text{-pull} \ (\text{Fin } \text{beta}) \ d\text{-child } (\text{Fin } b) \ U\text{-child}$ 
  by (rule exact-split-range-costed-bmssp-correct
    [OF child-run Exact-Split-Range-State-Step.prems child-pre child-reaches])
have child-complete:  $\text{complete-on } d\text{-child } U\text{-child}$ 
  using child-post unfolding bmssp-post-full-def by blast
have U-child-eq:  $U\text{-child} = \text{range-tree } P \ a \ (\text{Fin } b)$ 
  using Exact-Split-Range-State-Step by blast
have U-child-reaches:  $\bigwedge u. u \in U\text{-child} \implies \text{reachable } s \ u$ 
  unfolding U-child-eq range-tree-def tree-set-def tree-path-def by blast
have batch-cost:
   $c\text{-edges} + c\text{-sources} \leq$ 
   $t * \text{card } (\text{outgoing-edges-range } U\text{-child } b \ (\text{Fin } \text{beta})) +$ 
   $h * \text{card } S\text{-pull}$ 
  by (rule range-costed-split-batch-cost-le-range-card
    where  $d = d$  and  $S = S\text{-pull}$  and  $d\text{-child} = d\text{-child}$  and  $b = b$ 
    and  $\text{beta} = \text{beta}$ )
  (use Exact-Split-Range-State-Step child-complete U-child-reaches in auto)
let  $?child\text{-sources} = S\text{-pull} \ \# \ \text{child-sources-tail}$ 
have len:  $\text{length } ?child\text{-sources} = \text{length } (b \ \# \ bs)$ 
  using len-tail by simp
have head-source:
   $\exists c\text{-child}' \ B\text{-child } d\text{-child}' \ B\text{-child}'.$ 
   $\text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-pull } B\text{-child}$ 
   $d\text{-child}' \ B\text{-child}' \ U\text{-child } c\text{-child}' \wedge$ 
   $\text{bmssp-pre-full } d \ S\text{-pull } B\text{-child} \wedge$ 
   $(\forall x \in S\text{-pull}. \text{reachable } s \ x) \wedge$ 
   $\text{card } S\text{-pull} \leq M\text{-of } l \wedge$ 
   $(\forall x \in S\text{-pull}. \text{below-bound } (d \ x) \ B\text{-child})$ 
  using Exact-Split-Range-State-Step card-pull below-pull by blast
have sources:
  list-all2
   $(\lambda S\text{-child } U\text{-child}. \exists c\text{-child } B\text{-child } d\text{-child } B\text{-child}')$ 

```

```

    exact-split-range-costed-bmssp  $\Delta$  M-of t h k cap l d S-child B-child
      d-child B-child' U-child c-child  $\wedge$ 
      bmssp-pre-full d S-child B-child  $\wedge$ 
      ( $\forall x \in S\text{-child. reachable } s \ x$ )  $\wedge$ 
      card S-child  $\leq$  M-of l  $\wedge$ 
      ( $\forall x \in S\text{-child. below-bound } (d \ x) \ B\text{-child}$ )
      ?child-sources (range-tree-child-list P a (b # bs))
  using head-source sources-tail Exact-Split-Range-State-Step by simp
  have cost:
    c  $\leq$  sum-list (c-child # child-costs-tail) +
      sum-list (map card ?child-sources) +
      t * sum-list
        (map card (range-tree-child-edge-range-list P a (beta # betas) (b # bs))) +
      h * sum-list (map card ?child-sources)
  proof -
    have c-eq: c = c-pull + c-edges + c-sources + c-child + c-tail
    using Exact-Split-Range-State-Step by blast
    have c-pull + c-edges + c-sources + c-child + c-tail  $\leq$ 
      card S-pull +
      (t * card (outgoing-edges-range U-child b (Fin beta)) +
       h * card S-pull) +
      c-child +
      (sum-list child-costs-tail + sum-list (map card child-sources-tail) +
       t * sum-list
         (map card (range-tree-child-edge-range-list P b betas bs)) +
        h * sum-list (map card child-sources-tail))
    using pull-cost batch-cost tail-cost by linarith
    also have ... =
      sum-list (c-child # child-costs-tail) +
      sum-list (map card ?child-sources) +
      t * (card (outgoing-edges-range U-child b (Fin beta)) +
        sum-list
          (map card (range-tree-child-edge-range-list P b betas bs))) +
      h * sum-list (map card ?child-sources)
    by (simp add: algebra-simps)
    also have ... =
      sum-list (c-child # child-costs-tail) +
      sum-list (map card ?child-sources) +
      t * sum-list
        (map card (range-tree-child-edge-range-list P a (beta # betas) (b # bs))) +
      h * sum-list (map card ?child-sources)
    using U-child-eq by simp
    finally show ?thesis
      using c-eq by simp
  qed
  show ?case
    by (intro exI[of - ?child-sources] conjI)
      (use len sources cost in simp-all)
next

```

```

case (Exact-Split-Range-Cost-Base  $S\ x\ \Delta\ M\text{-of}\ t\ h\ k\ \text{cap}\ d\ B$ )
show ?case
  by simp
next
case (Exact-Split-Range-Cost-Step  $D\ k\ \text{cap}\ d\ S\ B\ c\text{-insert}\ t\ \Delta\ M\text{-of}\ h\ l$ 
   $d'\ a\ \text{betas}\ bs\ B'\ Us\ U\text{-loop}\ c\text{-loop}\ \text{child-costs-loop}\ U\ c$ )
show ?case
  by simp
qed

```

theorem *exact-split-range-costed-partition-loop-state-cost-from-child-source-and-edge-range-bounds*:

assumes *run*:

exact-split-range-costed-partition-loop-state $\Delta\ M\text{-of}\ t\ h\ k\ \text{cap}\ l\ d\ P\ B\ d'\ D\ a$
betas $bs\ B'\ Us\ U\ c\ \text{child-costs}$

and *sound*: *sound-label* d

and *child-cost-bounds*:

list-all2 $(\lambda c\text{-child}\ U\text{-child}.$

$c\text{-child} \leq \text{level-range-cost-bound}\ A\ R\ L\ U\text{-child})$

$\text{child-costs}\ (\text{range-tree-child-list}\ P\ a\ bs)$

and *source-progress*:

$\bigwedge S\text{-child}\ B\text{-child}\ d\text{-child}\ B\text{-child}'\ U\text{-child}\ c\text{-child}.$

$\llbracket \text{exact-split-range-costed-bmssp}\ \Delta\ M\text{-of}\ t\ h\ k\ \text{cap}\ l\ d\ S\text{-child}\ B\text{-child}$

$d\text{-child}\ B\text{-child}'\ U\text{-child}\ c\text{-child};$

$\text{bmssp-pre-full}\ d\ S\text{-child}\ B\text{-child};$

$\bigwedge x. x \in S\text{-child} \implies \text{reachable}\ s\ x;$

$\text{card}\ S\text{-child} \leq M\text{-of}\ l;$

$\bigwedge x. x \in S\text{-child} \implies \text{below-bound}\ (d\ x)\ B\text{-child} \rrbracket$

$\implies \text{card}\ S\text{-child} \leq \text{card}\ U\text{-child}$

shows $c \leq$

$A * L * \text{card}\ (\text{range-tree-chain}\ P\ a\ bs\ B') +$

$R * \text{card}\ (\text{outgoing-edges}\ (\text{range-tree-chain}\ P\ a\ bs\ B')) +$

$t * \text{sum-list}$

$(\text{map}\ \text{card}\ (\text{range-tree-child-edge-range-list}\ P\ a\ \text{betas}\ bs)) +$

$(\text{Suc}\ h) * \text{card}\ (\text{range-tree-chain}\ P\ a\ bs\ B')$

proof –

obtain *child-sources* **where** *len-sources*:

$\text{length}\ \text{child-sources} = \text{length}\ bs$

and *sources*:

list-all2

$(\lambda S\text{-child}\ U\text{-child}. \exists c\text{-child}\ B\text{-child}\ d\text{-child}\ B\text{-child}'.$

$\text{exact-split-range-costed-bmssp}\ \Delta\ M\text{-of}\ t\ h\ k\ \text{cap}\ l\ d\ S\text{-child}\ B\text{-child}$

$d\text{-child}\ B\text{-child}'\ U\text{-child}\ c\text{-child} \wedge$

$\text{bmssp-pre-full}\ d\ S\text{-child}\ B\text{-child} \wedge$

$(\forall x \in S\text{-child}. \text{reachable}\ s\ x) \wedge$

$\text{card}\ S\text{-child} \leq M\text{-of}\ l \wedge$

$(\forall x \in S\text{-child}. \text{below-bound}\ (d\ x)\ B\text{-child}))$

$\text{child-sources}\ (\text{range-tree-child-list}\ P\ a\ bs)$

and *cost*:

```

     $c \leq \text{sum-list } \text{child-costs} + \text{sum-list } (\text{map } \text{card } \text{child-sources}) +$ 
     $t * \text{sum-list}$ 
     $(\text{map } \text{card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } \text{bs})) +$ 
     $h * \text{sum-list } (\text{map } \text{card } \text{child-sources})$ 
using exact-split-range-costed-partition-loop-state-cost-bound-by-child-sources-
and-edge-ranges
    [OF run sound] by blast
have mono: nondecreasing-from a bs
proof –
    have split-run:
    split-range-costed-partition-loop-state  $\Delta$  M-of t h k cap l d P B d' D a
    betas bs B' Us U c child-costs
    using exact-split-range-costed-refines-split-range-costed(1)[OF run] .
    then show ?thesis
    by (rule split-range-costed-partition-loop-state-mono)
qed
have source-dom:
    list-all2 ( $\lambda S\text{-child } U\text{-child}. \text{card } S\text{-child} \leq \text{card } U\text{-child}$ )
    child-sources (range-tree-child-list P a bs)
    using sources
proof (induction rule: list-all2-induct)
    case Nil
    then show ?case by simp
next
    case (Cons S-child child-sources U-child child-sets)
    obtain c-child B-child d-child B-child' where child:
    exact-split-range-costed-bmssp  $\Delta$  M-of t h k cap l d S-child B-child
    d-child B-child' U-child c-child
    and child-pre: bmssp-pre-full d S-child B-child
    and child-reaches:  $\forall x \in S\text{-child}. \text{reachable } s \ x$ 
    and child-card:  $\text{card } S\text{-child} \leq M\text{-of } l$ 
    and child-below:  $\forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child}$ 
    using Cons.hyps by blast
have head:  $\text{card } S\text{-child} \leq \text{card } U\text{-child}$ 
    by (rule source-progress[OF child child-pre - child-card])
    (use child-reaches child-below in blast)+
show ?case
    using head Cons.IH Cons.hyps by simp
qed
have source-sum:
    sum-list (map card child-sources)  $\leq$ 
    card (range-tree-chain P a bs B')
    by (rule sum-card-dominated-by-range-tree-child-list-le-chain
    [OF mono source-dom])
have child-sum:
    sum-list child-costs  $\leq$ 
    A * L * card (range-tree-chain P a bs B') +
    R * card (outgoing-edges (range-tree-chain P a bs B'))
    by (rule child-costs-le-level-range-child-list-bound

```



```

[OF mono child-cost-bounds])
have c ≤
  A * L * card (range-tree-chain P a bs B') +
  R * card (outgoing-edges (range-tree-chain P a bs B')) +
  t * sum-list
    (map card (range-tree-child-edge-range-list P a betas bs)) +
  sum-list (map card child-sources) +
  h * sum-list (map card child-sources)
using cost child-sum by linarith
also have ... ≤
  A * L * card (range-tree-chain P a bs B') +
  R * card (outgoing-edges (range-tree-chain P a bs B')) +
  t * sum-list
    (map card (range-tree-child-edge-range-list P a betas bs)) +
  (Suc h) * card (range-tree-chain P a bs B')
proof -
  have source-part:
    sum-list (map card child-sources) +
    h * sum-list (map card child-sources) ≤
    Suc h * card (range-tree-chain P a bs B')
  proof -
    have h-part:
      h * sum-list (map card child-sources) ≤
      h * card (range-tree-chain P a bs B')
    using source-sum by simp
  have sum-list (map card child-sources) +
    h * sum-list (map card child-sources) ≤
    card (range-tree-chain P a bs B') +
    h * card (range-tree-chain P a bs B')
  using source-sum h-part by linarith
  then show ?thesis
    by (simp add: algebra-simps)
qed
show ?thesis
  using source-part by linarith
qed
finally show ?thesis .
qed

corollary exact-split-range-costed-partition-loop-state-cost-from-child-source-and-
edge-range-graph-bound:
  assumes run:
    exact-split-range-costed-partition-loop-state Δ M-of t h k cap l d P B d' D a
    betas bs B' Us U c child-costs
  and sound: sound-label d
  and child-cost-bounds:
    list-all2 (λc-child U-child.
      c-child ≤ level-range-cost-bound A R L U-child)
    child-costs (range-tree-child-list P a bs)

```

```

and source-progress:
   $\wedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}' U\text{-child } c\text{-child}.$ 
   $\llbracket \text{exact-split-range-costed-bmssp } \Delta M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$ 
     $d\text{-child } B\text{-child}' U\text{-child } c\text{-child};$ 
     $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child};$ 
     $\wedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$ 
     $\text{card } S\text{-child} \leq M\text{-of } l;$ 
     $\wedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child} \rrbracket$ 
     $\implies \text{card } S\text{-child} \leq \text{card } U\text{-child}$ 
shows  $c \leq$ 
   $A * L * \text{card } (\text{range-tree-chain } P \ a \ bs \ B') +$ 
   $R * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B')) +$ 
   $t * \text{edge-count} +$ 
   $(\text{Suc } h) * \text{card } (\text{range-tree-chain } P \ a \ bs \ B')$ 
proof –
have split-run:
   $\text{split-range-costed-partition-loop-state } \Delta M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$ 
   $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs}$ 
using exact-split-range-costed-refines-split-range-costed(1)[OF run] .
have mono: nondecreasing-from  $a \ bs$ 
by (rule split-range-costed-partition-loop-state-mono[OF split-run])
have direct-edges:
   $\text{sum-list } (\text{map } \text{card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$ 
   $\leq \text{edge-count}$ 
by (rule sum-card-range-tree-child-edge-range-list-le-edge-count[OF mono])
have separated:
   $c \leq$ 
   $A * L * \text{card } (\text{range-tree-chain } P \ a \ bs \ B') +$ 
   $R * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B')) +$ 
   $t * \text{sum-list}$ 
   $(\text{map } \text{card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs)) +$ 
   $(\text{Suc } h) * \text{card } (\text{range-tree-chain } P \ a \ bs \ B')$ 
by (rule exact-split-range-costed-partition-loop-state-cost-from-child-source-and-
edge-range-bounds
  [OF run sound child-cost-bounds source-progress])
have  $t * \text{sum-list } (\text{map } \text{card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$ 
   $\leq t * \text{edge-count}$ 
using direct-edges by simp
then show ?thesis
using separated by linarith
qed

```

The final theorem in this theory is the reusable recurrence step for exact split ranges. Given child costs already bounded by *level-range-cost-bound* and a source-progress hypothesis, it proves the parent-loop bound with one additional level of vertex cost and one additional batch coefficient on outgoing edges. This is the shape consumed by the direct-insert and bucketed refinements.

theorem *exact-split-range-costed-partition-loop-state-cost-from-child-and-source-*

bounds:

assumes *run:*

exact-split-range-costed-partition-loop-state Δ *M-of t h k cap l d P B d' D a*
betas bs B' Us U c child-costs

and *child-cost-bounds:*

list-all2 ($\lambda c\text{-child } U\text{-child}.$
 $c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$)
child-costs (*range-tree-child-list* *P a bs*)

and *source-progress:*

$\bigwedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}.$
 $\llbracket \text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$
 $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child};$
 $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\text{card } S\text{-child} \leq M\text{-of } l;$
 $\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child} \rrbracket$
 $\implies \text{card } S\text{-child} \leq \text{card } U\text{-child}$

shows $c \leq$

$A * L * \text{card } (\text{range-tree-chain } P \ a \ bs \ B') +$
 $(R + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B')) +$
 $(\text{Suc } h) * \text{card } (\text{range-tree-chain } P \ a \ bs \ B')$

proof –

obtain *child-sources* **where** *len-sources:*

length child-sources = *length bs*

and *sources:*

list-all2
 $(\lambda S\text{-child } U\text{-child}. \exists c\text{-child } B\text{-child } d\text{-child } B\text{-child}').$
 $\text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child} \wedge$
 $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child} \wedge$
 $(\forall x \in S\text{-child}. \text{reachable } s \ x) \wedge$
 $\text{card } S\text{-child} \leq M\text{-of } l \wedge$
 $(\forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child}))$
 $\text{child-sources } (\text{range-tree-child-list } P \ a \ bs)$

and *cost:*

$c \leq \text{sum-list } \text{child-costs} + \text{sum-list } (\text{map } \text{card } \text{child-sources}) +$
 $t * \text{sum-list}$
 $(\text{map } (\lambda X. \text{card } (\text{outgoing-edges } X)) (\text{range-tree-child-list } P \ a \ bs)) +$
 $h * \text{sum-list } (\text{map } \text{card } \text{child-sources})$

using *exact-split-range-costed-partition-loop-state-cost-bound-by-child-sources-*
below

$[OF \ \text{run}]$ **by** *blast*

have *mono: nondecreasing-from a bs*

proof –

have *split-run:*

split-range-costed-partition-loop-state Δ *M-of t h k cap l d P B d' D a*
betas bs B' Us U c child-costs

using *exact-split-range-costed-refines-split-range-costed(1)[OF run]* .

then show *?thesis*

```

    by (rule split-range-costed-partition-loop-state-mono)
qed
have source-dom:
  list-all2 ( $\lambda S\text{-child } U\text{-child. card } S\text{-child} \leq \text{card } U\text{-child}$ )
  child-sources (range-tree-child-list  $P$   $a$   $bs$ )
using sources
proof (induction rule: list-all2-induct)
  case Nil
  then show ?case by simp
next
  case (Cons  $S\text{-child}$   $child\text{-sources}$   $U\text{-child}$   $child\text{-sets}$ )
  obtain  $c\text{-child}$   $B\text{-child}$   $d\text{-child}$   $B\text{-child}'$  where  $child$ :
    exact-split-range-costed-bmssp  $\Delta$   $M\text{-of } t$   $h$   $k$   $cap$   $l$   $d$   $S\text{-child}$   $B\text{-child}$ 
     $d\text{-child}$   $B\text{-child}'$   $U\text{-child}$   $c\text{-child}$ 
  and  $child\text{-pre}$ :  $bmssp\text{-pre-full } d$   $S\text{-child}$   $B\text{-child}$ 
  and  $child\text{-reaches}$ :  $\forall x \in S\text{-child. reachable } s$   $x$ 
  and  $child\text{-card}$ :  $\text{card } S\text{-child} \leq M\text{-of } l$ 
  and  $child\text{-below}$ :  $\forall x \in S\text{-child. below-bound } (d$   $x)$   $B\text{-child}$ 
  using  $Cons.hyps$  by blast
  have head:  $\text{card } S\text{-child} \leq \text{card } U\text{-child}$ 
  by (rule source-progress[OF  $child$   $child\text{-pre}$  -  $child\text{-card}$ ])
    (use  $child\text{-reaches}$   $child\text{-below}$  in blast)+
  show ?case
    using head  $Cons.IH$   $Cons.hyps$  by simp
qed
have source-sum:
  sum-list (map card  $child\text{-sources}$ )  $\leq$ 
  card (range-tree-chain  $P$   $a$   $bs$   $B'$ )
  by (rule sum-card-dominated-by-range-tree-child-list-le-chain
    [OF mono source-dom])
have child-sum:
  sum-list  $child\text{-costs} \leq$ 
   $A * L * \text{card } (range\text{-tree-chain } P$   $a$   $bs$   $B')$   $+$ 
   $R * \text{card } (outgoing\text{-edges } (range\text{-tree-chain } P$   $a$   $bs$   $B'))$ 
  by (rule child-costs-le-level-range-child-list-bound
    [OF mono child-cost-bounds])
have edge-sum:
  sum-list
    (map ( $\lambda X. \text{card } (outgoing\text{-edges } X)$ ) (range-tree-child-list  $P$   $a$   $bs$ ))  $\leq$ 
  card (outgoing-edges (range-tree-chain  $P$   $a$   $bs$   $B'$ ))
  by (rule card-outgoing-edges-range-tree-child-list-le-chain[OF mono])
have  $c \leq$ 
   $A * L * \text{card } (range\text{-tree-chain } P$   $a$   $bs$   $B')$   $+$ 
   $R * \text{card } (outgoing\text{-edges } (range\text{-tree-chain } P$   $a$   $bs$   $B'))$   $+$ 
  sum-list (map card  $child\text{-sources}$ )  $+$ 
   $t * \text{card } (outgoing\text{-edges } (range\text{-tree-chain } P$   $a$   $bs$   $B'))$   $+$ 
   $h * \text{sum-list } (map \text{card } child\text{-sources})$ 
proof -
  have edge-part:

```

```

    t * sum-list
      (map (λX. card (outgoing-edges X)) (range-tree-child-list P a bs))
    ≤ t * card (outgoing-edges (range-tree-chain P a bs B'))
  using edge-sum by simp
  have sum-list child-costs + sum-list (map card child-sources) +
    t * sum-list
      (map (λX. card (outgoing-edges X)) (range-tree-child-list P a bs)) +
    h * sum-list (map card child-sources) ≤
    A * L * card (range-tree-chain P a bs B') +
    R * card (outgoing-edges (range-tree-chain P a bs B')) +
    sum-list (map card child-sources) +
    t * card (outgoing-edges (range-tree-chain P a bs B')) +
    h * sum-list (map card child-sources)
  using child-sum edge-part by linarith
  then show ?thesis
    using cost by linarith
qed
also have ... ≤
  A * L * card (range-tree-chain P a bs B') +
  R * card (outgoing-edges (range-tree-chain P a bs B')) +
  t * card (outgoing-edges (range-tree-chain P a bs B')) +
  (Suc h) * card (range-tree-chain P a bs B')
proof -
  have source-part:
    sum-list (map card child-sources) +
    h * sum-list (map card child-sources) ≤
    Suc h * card (range-tree-chain P a bs B')
  proof -
    have h-part:
      h * sum-list (map card child-sources) ≤
      h * card (range-tree-chain P a bs B')
    using source-sum by simp
    have sum-list (map card child-sources) +
      h * sum-list (map card child-sources) ≤
      card (range-tree-chain P a bs B') +
      h * card (range-tree-chain P a bs B')
    using source-sum h-part by linarith
    then show ?thesis
      by (simp add: algebra-simps)
  qed
  show ?thesis
    using source-part by linarith
qed
also have ... =
  A * L * card (range-tree-chain P a bs B') +
  (R + t) * card (outgoing-edges (range-tree-chain P a bs B')) +
  (Suc h) * card (range-tree-chain P a bs B')
  by (simp add: algebra-simps)
finally show ?thesis .

```

```

qed

end

end
theory BMSSP-Direct-Insert-Costed
  imports BMSSP-Exact-Range-Costed
begin

```

32 Direct-Insert Costed Runs

This theory refines the range-costed BMSSP run with the final accounting distinction needed by the bucketed partition implementation. The earlier split-range relation separates edge-generated batch entries from source-label batch entries. Here the edge-generated entries are split again. Some relaxed edge values belong directly in the current parent range and are paid for by the ordinary insert cost parameter t . Other relaxed values fall below the next child bound and are prepended into the lower block; those are paid for by the batch parameter h .

The distinction mirrors the bucketed data structure. A direct insert is the operation that searches for a bucket position and may trigger the logarithmic bucket-splitting work. A lower-range prepend is cheaper in the abstract accounting because it is grouped with other items destined for the next pull. Recording the two lists separately lets the later amortised theorem keep the direct-insert term visible instead of hiding it inside one combined batch budget.

The semantic content is intentionally unchanged. The inductive run still produces the same completed range slices as the exact range-costed relation, and the same BMSSP postcondition is recovered by forgetting the extra accounting fields. What changes is the cost expression: each loop step now exposes a direct-edge batch, a lower-edge batch, a source batch, the child recursive cost, and the tail-loop cost.

This is the last local-cost layer before the global recurrence is instantiated. The closing theorems near the end of the file package the separated charges into two forms: an amortised bound that keeps direct range inserts explicit, and a level bound that can feed the headline top-level analysis.

```

context unique-shortest-digraph
begin

```

```

inductive direct-insert-costed-partition-loop-state
  and direct-insert-costed-bmssp where
    Direct-Insert-Costed-State-Done:
       $\text{keys-of } D = \{\} \implies$ 
       $\text{bound-le } (\text{Fin } a) \ B \implies$ 
       $\text{complete-on } d' \ (\text{bound-tree } P \ B) \implies$ 

```

$\text{direct-insert-costed-partition-loop-state } \Delta \text{ M-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 $\square \square \ B \ [\text{range-tree } P \ a \ B]$
 $(\text{bound-tree } P \ (\text{Fin } a) \cup \bigcup (\text{set } [\text{range-tree } P \ a \ B])) \ 0 \square$

| *Direct-Insert-Costed-State-Stop:*

$\text{bound-le } (\text{Fin } a) \ B \implies$
 $\text{complete-on } d' \ (\text{bound-tree } P \ (\text{Fin } a)) \implies$
 $k * \text{cap} \leq \text{card } (\text{bound-tree } P \ (\text{Fin } a)) \implies$
 $\text{direct-insert-costed-partition-loop-state } \Delta \text{ M-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 $\square \square \ (\text{Fin } a) \ [\text{range-tree } P \ a \ (\text{Fin } a)]$
 $(\text{bound-tree } P \ (\text{Fin } a) \cup \bigcup (\text{set } [\text{range-tree } P \ a \ (\text{Fin } a)])) \ 0 \square$

| *Direct-Insert-Costed-State-Step:*

$\text{pull-separates } D \ (\text{M-of } l) \ B_{\text{max}} \ S\text{-pull } \beta \ D\text{-pull} \implies$
 $\text{bound-le } (\text{Fin } \beta) \ B \implies$
 $\text{bmssp-pre-full } d \ S\text{-pull } (\text{Fin } \beta) \implies$
 $S\text{-pull} = \text{split-below } d \ P \ \beta \implies$
 $(\forall x \in S\text{-pull}. \text{reachable } s \ x) \implies$
 $a \leq b \implies$
 $\text{bound-le } (\text{Fin } a) \ B' \implies$
 $\text{complete-on } d' \ (\text{bound-tree } P \ (\text{Fin } a)) \implies$
 $\text{card } (\text{bound-tree } P \ (\text{Fin } a)) < k * \text{cap} \implies$
 $\text{direct-insert-costed-bmssp } \Delta \text{ M-of } t \ h \ k \ \text{cap } l \ d \ S\text{-pull } (\text{Fin } \beta)$
 $d\text{-child } (\text{Fin } b) \ U\text{-child } c\text{-child} \implies$
 $U\text{-child} = \text{range-tree } P \ a \ (\text{Fin } b) \implies$
 $\text{complete-preserved } d\text{-child } d' \ U\text{-child} \implies$
 $\text{direct-edge-batch} = \text{edge-relaxation-pairs-in-bound } d\text{-child } U\text{-child } \beta \ B \implies$
 $\text{lower-edge-batch} = \text{edge-relaxation-pairs-between } d\text{-child } U\text{-child } b \ \beta \implies$
 $\text{source-batch} = \text{label-pairs-between } d \ S\text{-pull } b \ \beta \implies$
 $\text{batch} = \text{direct-edge-batch} @ \text{lower-edge-batch} @ \text{source-batch} \implies$
 $D\text{-next} = \text{batch-min-update } D\text{-pull } \text{batch} \implies$
 $\text{partition-pull-cost-bound } c\text{-pull } S\text{-pull} \implies$
 $\text{partition-batch-cost-bound } c\text{-direct } t \ \text{direct-edge-batch} \implies$
 $\text{partition-batch-cost-bound } c\text{-lower } h \ \text{lower-edge-batch} \implies$
 $\text{partition-batch-cost-bound } c\text{-sources } h \ \text{source-batch} \implies$
 $\text{direct-insert-costed-partition-loop-state } \Delta \text{ M-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d'$
 $D\text{-next } b \ \beta \text{as } b \ B' \ U\text{-tail } U\text{-tail } c\text{-tail } \text{child-costs-tail} \implies$
 $c = c\text{-pull} + c\text{-direct} + c\text{-lower} + c\text{-sources} + c\text{-child} + c\text{-tail} \implies$
 $\text{direct-insert-costed-partition-loop-state } \Delta \text{ M-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 $(\beta \# \beta \text{as}) \ (b \# b \ B')$
 $(\text{range-tree } P \ a \ (\text{Fin } b) \# U\text{-tail})$
 $(\text{bound-tree } P \ (\text{Fin } a) \cup$
 $\bigcup (\text{set } (\text{range-tree } P \ a \ (\text{Fin } b) \# U\text{-tail}))) \ c$
 $(c\text{-child} \# \text{child-costs-tail})$

| *Direct-Insert-Costed-Base:*

$S = \{x\} \implies$
 $\text{direct-insert-costed-bmssp } \Delta \text{ M-of } t \ h \ k \ \text{cap } 0 \ d \ S \ B$
 $(\lambda v. \text{if } v \in \text{base-case-vertices } k \ x \ B \text{ then } \text{dist } s \ v \text{ else } d \ v)$
 $(\text{base-case-bound } k \ x \ B)$
 $(\text{base-case-vertices } k \ x \ B)$
 $(\text{base-case-scan-cost } \Delta \ k \ x \ B)$

| *Direct-Insert-Costed-Step:*

$D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \text{ cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \implies$
 $\text{direct-insert-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \text{ cap } l$
 $(\text{find-pivots-label-capped } k \text{ cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs-loop} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $c = \text{fp-iter-capped-scan-cost } k \text{ cap } d \ S \ S \ B + c\text{-insert} + c\text{-loop} \implies$
 $\text{direct-insert-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } (Suc \ l) \ d \ S \ B \ d' \ B' \ U \ c$

The constructors are the same operational skeleton as before, but the step constructor now names three batches. *edge-relaxation-pairs-in-bound* extracts direct edge relaxations in the parent range, *edge-relaxation-pairs-between* extracts lower edge relaxations between the child output bound and the pull bound, and *label-pairs-between* extracts old source labels in the same lower interval.

Keeping these lists in the derivation tree is useful because each has a different combinatorial owner. Direct edge entries are counted by the direct range lists; lower edge entries are counted by child edge-range lists; source entries are counted through child source sets and then by the progress lemma. The proof scripts that follow mostly say that this extra bookkeeping is conservative: it refines the earlier correctness trace without changing which vertices are completed.

inductive-cases *direct-insert-costed-bmssp-zeroE:*

$\text{direct-insert-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } 0 \ d \ S \ B \ d' \ B' \ U \ c$

inductive-cases *direct-insert-costed-bmssp-SucE:*

$\text{direct-insert-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } (Suc \ l) \ d \ S \ B \ d' \ B' \ U \ c$

lemma *direct-insert-costed-partition-loop-state-mono:*

assumes *run:*

$\text{direct-insert-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \text{ cap } l \ d \ P \ B \ d' \ D \ a$
 $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs}$

shows *nondecreasing-from a bs*

using *run by (induction) auto*

lemma *direct-insert-costed-partition-loop-state-beta-bounds:*

assumes *run:*

$\text{direct-insert-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \text{ cap } l \ d \ P \ B \ d' \ D \ a$
 $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs}$

shows *bounds-le B betas*


```

using run by (induction) auto

theorem direct-insert-costed-partition-loop-state-trace:
  direct-insert-costed-partition-loop-state  $\Delta$  M-of t h k cap l d P B d' D a
    betas bs B' Us U c child-costs  $\implies$ 
    sound-label d  $\implies$ 
    bmssp-pre-full d P B  $\implies$ 
     $(\bigwedge x. x \in P \implies \text{reachable } s \ x) \implies$ 
    concrete-partition-loop-trace P B a bs d' B' Us U
and direct-insert-costed-bmssp-correct:
  direct-insert-costed-bmssp  $\Delta$  M-of t h k cap l d S B d' B' U c  $\implies$ 
    sound-label d  $\implies$ 
    bmssp-pre-full d S B  $\implies$ 
     $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$ 
    bmssp-post-full d S B d' B' U
proof (induction rule):
  direct-insert-costed-partition-loop-state-direct-insert-costed-bmssp.inducts)
case (Direct-Insert-Costed-State-Done D a B d' P  $\Delta$  M-of t h k cap l d)
have a-le: bound-le (Fin a) B
  using Direct-Insert-Costed-State-Done by blast
have complete-B: complete-on d' (bound-tree P B)
  using Direct-Insert-Costed-State-Done by blast
have lower: complete-on d' (bound-tree P (Fin a))
  by (rule complete-on-subset[OF complete-B])
    (rule bound-tree-bound-mono[OF a-le])
have range-complete: complete-on d' (range-tree P a B)
  by (rule complete-on-subset[OF complete-B])
    (rule range-tree-subset-bound-tree)
have B-le: bound-le B B
  by (cases B) simp-all
show ?case
  unfolding concrete-partition-loop-trace-def
  using B-le a-le lower range-complete by simp
next
case (Direct-Insert-Costed-State-Stop a B d' P k cap  $\Delta$  M-of t h l d D)
then show ?case
  unfolding concrete-partition-loop-trace-def complete-on-def by simp
next
case Direct-Insert-Costed-State-Step
then show ?case
  unfolding concrete-partition-loop-trace-def bmssp-post-full-def
    complete-preserved-def
  by (auto split: bound.splits)
next
case (Direct-Insert-Costed-Base S x  $\Delta$  M-of t h k cap d B)
have post:
  bmssp-post d S B
    ( $\lambda v. \text{if } v \in \text{base-case-vertices } k \ x \ B \text{ then } \text{dist } s \ v \text{ else } d \ v$ )
    (base-case-bound k x B)

```

```

    (base-case-vertices  $k$   $x$   $B$ )
  using base-case-result-bmssp-post[OF Direct-Insert-Costed-Base.hyps,
    where  $k = k$  and  $B = B$  and  $d = d$ ]
  unfolding base-case-result-def by simp
  then show ?case
    by (rule bmssp-post-imp-post-full)
next
case (Direct-Insert-Costed-Step  $D$   $k$   $\text{cap } d$   $S$   $B$   $c\text{-insert } t$   $\Delta$   $M\text{-of } h$   $l$   $d'$   $a$ 
  betas  $bs$   $B'$   $Us$   $U\text{-loop}$   $c\text{-loop}$   $\text{child-costs-loop}$   $U$   $c$ )
let ?d-fp = find-pivots-label-capped  $k$   $\text{cap } d$   $S$   $B$ 
let ?P = find-pivots-pivots-capped  $k$   $\text{cap } d$   $S$   $B$ 
let ?W =  $\{v \in \text{bound-tree } S \ B'. \ ?d\text{-fp } v = \text{dist } s \ v\}$ 
have sound-fp: sound-label ?d-fp
  unfolding find-pivots-label-capped-def
  by (rule fp-iter-capped-label-sound[OF Direct-Insert-Costed-Step.prem(1)
    Direct-Insert-Costed-Step.prem(3)])
have pivot-pre: bmssp-pre-full ?d-fp ?P  $B$ 
  using find-pivots-capped-establishes-pivot-pre-concrete
    [OF Direct-Insert-Costed-Step.prem(1) Direct-Insert-Costed-Step.prem(2)
    Direct-Insert-Costed-Step.prem(3)] .
have P-subset-S: ?P  $\subseteq S$ 
  unfolding find-pivots-pivots-capped-def by auto
have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
  using P-subset-S Direct-Insert-Costed-Step.prem(3) by blast
have trace:
  concrete-partition-loop-trace ?P  $B$   $a$   $bs$   $d'$   $B'$   $Us$   $U\text{-loop}$ 
  using Direct-Insert-Costed-Step.IH sound-fp pivot-pre P-reaches by blast
have loop-post: bmssp-post-full ?d-fp ?P  $B$   $d'$   $B'$   $U\text{-loop}$ 
  by (rule concrete-partition-loop-trace-post[OF trace])
have le: bound-le  $B'$   $B$ 
  using loop-post unfolding bmssp-post-full-def by blast
have U-tree:  $U = \text{bound-tree } S \ B'$ 
proof -
  have U-loop  $\cup$  ?W = bound-tree  $S$   $B'$ 
  by (rule concrete-capped-find-pivots-final-tree-assembly
    [OF Direct-Insert-Costed-Step.prem(1) Direct-Insert-Costed-Step.prem(2)
    Direct-Insert-Costed-Step.prem(3) loop-post])
  then show ?thesis
    using Direct-Insert-Costed-Step(6) by simp
qed
have loop-complete: complete-on  $d'$   $U\text{-loop}$ 
  using loop-post unfolding bmssp-post-full-def by blast
have U-complete: complete-on  $d'$   $U$ 
  using complete-on-Un[OF loop-complete Direct-Insert-Costed-Step(5)
    Direct-Insert-Costed-Step(6)] by simp
show ?case
  using le U-tree U-complete unfolding bmssp-post-full-def by blast
qed

```

theorem *finite-initial-label-direct-insert-costed-top-level-correct:*
assumes *all-reachable:* $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *run:*
direct-insert-costed-bmssp Δ *M-of t h k cap l*
finite-initial-label $\{s\}$ *Infinity d' Infinity U c*
shows *sssp-correct d'*
proof –
have *pre:* *bmssp-pre-full finite-initial-label {s} Infinity*
using *all-reachable finite-initial-label-source-complete*
by (*rule top-bmssp-pre-full*)
have *sound:* *sound-label finite-initial-label*
using *finite-initial-label-sound[OF all-reachable]* .
have *S-reaches:* $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$
using *all-reachable source-in-V* **by** *blast*
have *post-full:*
bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
by (*rule direct-insert-costed-bmssp-correct[OF run sound pre S-reaches]*)
have *post:* *bmssp-post finite-initial-label {s} Infinity d' Infinity U*
using *bmssp-post-full-imp-post[OF post-full]* .
then have *U-V:* $U = V$
using *bound-tree-source-infinity[OF all-reachable]*
unfolding *bmssp-post-def* **by** *auto*
have *complete:* *complete-on d' U*
using *post unfolding bmssp-post-def* **by** *auto*
then show *?thesis*
using *U-V unfolding complete-on-def sssp-correct-def* **by** *auto*
qed

theorem *finite-initial-label-direct-insert-costed-top-level-reachable-correct:*
assumes *run:*
direct-insert-costed-bmssp Δ *M-of t h k cap l*
finite-initial-label $\{s\}$ *Infinity d' Infinity U c*
shows *sssp-correct d'*
proof –
have *pre:* *bmssp-pre-full finite-initial-label {s} Infinity*
using *finite-initial-label-source-complete*
by (*rule top-bmssp-pre-full-reachable*)
have *sound:* *sound-label finite-initial-label*
by (*rule finite-initial-label-sound-reachable*)
have *S-reaches:* $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$
using *reachable-refl[OF source-in-V]* **by** *blast*
have *post-full:*
bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
by (*rule direct-insert-costed-bmssp-correct[OF run sound pre S-reaches]*)
have *post:* *bmssp-post finite-initial-label {s} Infinity d' Infinity U*
using *bmssp-post-full-imp-post[OF post-full]* .
then show *?thesis*
by (*rule successful-root-bmssp-sssp-correct*)
qed

The correctness theorem $\llbracket \text{direct-insert-costed-bmssp } ?\Delta \text{ } ?M\text{-of } ?t \text{ } ?h \text{ } ?k \text{ } ?cap \text{ } ?l \text{ } ?d \text{ } ?S \text{ } ?B \text{ } ?d' \text{ } ?B' \text{ } ?U \text{ } ?c; \text{ sound-label } ?d; \text{ bmssp-pre-full } ?d \text{ } ?S \text{ } ?B; \bigwedge x. x \in ?S \implies \text{reachable } s \text{ } x \rrbracket \implies \text{bmssp-post-full } ?d \text{ } ?S \text{ } ?B \text{ } ?d' \text{ } ?B' \text{ } ?U$ is the semantic projection of the direct-insert relation. Once projected, the top-level corollaries are the familiar root BMSSP statements: from the finite initial label at the source and the infinite output bound, the returned label function is a complete SSSP solution.

The next group of lemmas supplies the progress side conditions used by the cost recurrence. They show that successful BMSSP calls either return the original bound or have accumulated enough completed vertices to meet the threshold, and that a source set whose labels are below the output bound is dominated by the output tree. These facts turn source-batch costs into vertex-count terms.

theorem *direct-insert-costed-bmssp-source-card-le-if-label-below-output:*

assumes *run:*

direct-insert-costed-bmssp Δ *M-of t h k cap l d S B d' B' U c*

and *sound:* *sound-label d*

and *pre:* *bmssp-pre-full d S B*

and *S-reaches:* $\bigwedge x. x \in S \implies \text{reachable } s \text{ } x$

and *complete:* *complete-on d S*

and *below:* $\bigwedge x. x \in S \implies \text{below-bound } (d \text{ } x) \text{ } B'$

shows *card S* $\leq$ *card U*

proof –

have *post:* *bmssp-post-full d S B d' B' U*

by (*rule direct-insert-costed-bmssp-correct*[*OF run sound pre S-reaches*])

show *?thesis*

by (*rule bmssp-post-full-source-card-le-if-label-below-output*
[*OF post complete S-reaches below*])

qed

theorem *direct-insert-costed-bmssp-source-card-le-if-sound-label-below-output:*

assumes *run:*

direct-insert-costed-bmssp Δ *M-of t h k cap l d S B d' B' U c*

and *sound:* *sound-label d*

and *pre:* *bmssp-pre-full d S B*

and *S-reaches:* $\bigwedge x. x \in S \implies \text{reachable } s \text{ } x$

and *below:* $\bigwedge x. x \in S \implies \text{below-bound } (d \text{ } x) \text{ } B'$

shows *card S* $\leq$ *card U*

proof –

have *post:* *bmssp-post-full d S B d' B' U*

by (*rule direct-insert-costed-bmssp-correct*[*OF run sound pre S-reaches*])

have *U-eq:* *U = bound-tree S B'*

using *post unfolding* *bmssp-post-full-def* **by** *blast*

have *S-subset-U:* *S* $\subseteq$ *U*

proof

fix *x*

assume *xs:* *x* $\in$ *S*

```

have  $S$ -subset:  $S \subseteq V$ 
  using pre unfolding bmssp-pre-full-def by blast
have  $xV$ :  $x \in V$ 
  using  $S$ -subset  $xS$  by blast
have reach- $x$ : reachable  $s$   $x$ 
  by (rule  $S$ -reaches[ $OF$   $xS$ ])
have dist-le:  $dist$   $s$   $x \leq d$   $x$ 
  using sound  $xV$  reach- $x$  unfolding sound-label-def by blast
have below-dist: below-bound ( $dist$   $s$   $x$ )  $B'$ 
  using below-bound-le-trans[ $OF$  dist-le below[ $OF$   $xS$ ]] .
show  $x \in U$ 
  unfolding  $U$ -eq
  by (rule source-in-own-bound-tree[ $OF$   $xS$  reach- $x$  below-dist])
qed
have finite- $U$ : finite  $U$ 
  unfolding  $U$ -eq bound-tree-def using finite- $V$  by auto
show ?thesis
  by (rule card-mono[ $OF$  finite- $U$   $S$ -subset- $U$ ])
qed

theorem direct-insert-costed-partition-loop-state-success-or-threshold:
  direct-insert-costed-partition-loop-state  $\Delta$   $M$ -of  $t$   $h$   $k$  cap  $l$   $d$   $P$   $B$   $d'$   $D$   $a$ 
  betas  $bs$   $B'$   $Us$   $U$   $c$  child-costs  $\implies$ 
  sound-label  $d \implies$ 
  bmssp-pre-full  $d$   $P$   $B \implies$ 
  ( $\bigwedge x. x \in P \implies$  reachable  $s$   $x$ )  $\implies$ 
   $B' = B \vee k * cap \leq card$   $U$ 
and direct-insert-costed-bmssp-success-or-threshold-trivial:
  direct-insert-costed-bmssp  $\Delta$   $M$ -of  $t$   $h$   $k$  cap  $l$   $d$   $S$   $B$   $d'$   $B'$   $U$   $c \implies$  True
proof (induction rule:
  direct-insert-costed-partition-loop-state-direct-insert-costed-bmssp.inducts)
case (Direct-Insert-Costed-State-Done  $D$   $a$   $B$   $d'$   $P$   $\Delta$   $M$ -of  $t$   $h$   $k$  cap  $l$   $d$ )
  then show ?case
    by simp
next
  case (Direct-Insert-Costed-State-Stop  $a$   $B$   $d'$   $P$   $k$  cap  $\Delta$   $M$ -of  $t$   $h$   $l$   $d$   $D$ )
  then show ?case
    by simp
next
  case (Direct-Insert-Costed-State-Step  $D$   $M$ -of  $l$   $B_{max}$   $S$ -pull beta  $D$ -pull  $B$   $d$ 
     $P$   $a$   $b$   $B$ -out  $d$ -out  $k$  cap  $\Delta$   $t$   $h$   $d$ -child  $U$ -child  $c$ -child
    direct-edge-batch lower-edge-batch source-batch batch  $D$ -next  $c$ -pull
     $c$ -direct  $c$ -lower  $c$ -sources betas  $bs$   $Us$ -tail  $U$ -tail  $c$ -tail
    child-costs-tail  $c$ )
    have tail:
       $B$ -out =  $B \vee k * cap \leq card$   $U$ -tail
      using Direct-Insert-Costed-State-Step.IH Direct-Insert-Costed-State-Step.premis
by blast
    then show ?case

```

```

proof
  assume  $B\text{-out} = B$ 
  then show ?thesis
    by blast
next
  assume threshold-tail:  $k * \text{cap} \leq \text{card } U\text{-tail}$ 
  have tail-trace:
    concrete-partition-loop-trace  $P \ B \ b \ bs \ d\text{-out} \ B\text{-out} \ Us\text{-tail} \ U\text{-tail}$ 
    by (rule direct-insert-costed-partition-loop-state-trace)
    (use Direct-Insert-Costed-State-Step in blast)+
  have U-tail-def:
     $U\text{-tail} = \text{bound-tree } P \ (\text{Fin } b) \cup \bigcup (\text{set } Us\text{-tail})$ 
    using tail-trace unfolding concrete-partition-loop-trace-def by blast
  have split-b:
     $\text{bound-tree } P \ (\text{Fin } b) =$ 
     $\text{bound-tree } P \ (\text{Fin } a) \cup \text{range-tree } P \ a \ (\text{Fin } b)$ 
    using bound-tree-split-at[of a Fin b P] Direct-Insert-Costed-State-Step
    by simp
  have U-eq-tail:
     $\text{bound-tree } P \ (\text{Fin } a) \cup$ 
     $\bigcup (\text{set } (\text{range-tree } P \ a \ (\text{Fin } b) \ \# \ Us\text{-tail})) =$ 
     $U\text{-tail}$ 
    using U-tail-def split-b by auto
  show ?thesis
    using threshold-tail U-eq-tail by simp
qed
next
  case (Direct-Insert-Costed-Base  $S \ x \ \Delta \ M\text{-of } t \ h \ k \ \text{cap} \ d \ B$ )
  then show ?case
    by simp
next
  case (Direct-Insert-Costed-Step  $D \ k \ \text{cap} \ d \ S \ B \ c\text{-insert } t \ \Delta \ M\text{-of } h \ l \ d' \ a$ 
     $\text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs-loop } U \ c$ )
  then show ?case
    by simp
qed

theorem direct-insert-costed-bmssp-Suc-success-or-threshold:
  assumes run:
    direct-insert-costed-bmssp  $\Delta \ M\text{-of } t \ h \ k \ \text{cap} \ (\text{Suc } l) \ d \ S \ B \ d' \ B' \ U \ c$ 
  and sound: sound-label  $d$ 
  and pre: bmssp-pre-full  $d \ S \ B$ 
  and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
  shows  $B' = B \vee k * \text{cap} \leq \text{card } U$ 
  using run
proof cases
  case (Direct-Insert-Costed-Step  $D \ c\text{-insert } a \ \text{betas } bs \ Us \ U\text{-loop } c\text{-loop } \text{child-costs-loop}$ )
  let ?d-fp = find-pivots-label-capped  $k \ \text{cap} \ d \ S \ B$ 

```

```

let ?P = find-pivots-pivots-capped k cap d S B
let ?W = {v ∈ bound-tree S B'. ?d-fp v = dist s v}
have sound-fp: sound-label ?d-fp
  unfolding find-pivots-label-capped-def
  by (rule fp-iter-capped-label-sound[OF sound S-reaches])
have pivot-pre: bmssp-pre-full ?d-fp ?P B
  using find-pivots-capped-establishes-pivot-pre-concrete
    [OF sound pre S-reaches] .
have P-subset-S: ?P ⊆ S
  unfolding find-pivots-pivots-capped-def by auto
have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
  using P-subset-S S-reaches by blast
have loop-class:
   $B' = B \vee k * \text{cap} \leq \text{card } U\text{-loop}$ 
  by (rule direct-insert-costed-partition-loop-state-success-or-threshold
    [OF Direct-Insert-Costed-Step(3) sound-fp pivot-pre P-reaches])
have loop-trace:
  concrete-partition-loop-trace ?P B a bs d' B' Us U-loop
  by (rule direct-insert-costed-partition-loop-state-trace
    [OF Direct-Insert-Costed-Step(3) sound-fp pivot-pre P-reaches])
have loop-post: bmssp-post-full ?d-fp ?P B d' B' U-loop
  by (rule concrete-partition-loop-trace-post[OF loop-trace])
have finite-U-loop: finite U-loop
  using loop-post unfolding bmssp-post-full-def by simp
show ?thesis
  using loop-class
proof
  assume  $B' = B$ 
  then show ?thesis
    by blast
next
  assume threshold-loop:  $k * \text{cap} \leq \text{card } U\text{-loop}$ 
  have U-eq:  $U = U\text{-loop} \cup ?W$ 
    using Direct-Insert-Costed-Step(5) by simp
  have finite-U: finite U
    unfolding U-eq using finite-U-loop by simp
  have U-loop-subset:  $U\text{-loop} \subseteq U$ 
    unfolding U-eq by blast
  have  $\text{card } U\text{-loop} \leq \text{card } U$ 
    by (rule card-mono[OF finite-U U-loop-subset])
  then show ?thesis
    using threshold-loop by linarith
qed
qed

theorem direct-insert-costed-bmssp-Suc-source-card-le-from-label-below:
assumes run:
   $\text{direct-insert-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap} \ (\text{Suc } l) \ d \ S \ B \ d' \ B' \ U \ c$ 
and sound: sound-label d

```

```

and pre: bmssp-pre-full d S B
and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
and below:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$ 
and S-cap:  $\text{card } S \leq \text{cap}$ 
and k-pos:  $0 < k$ 
shows  $\text{card } S \leq \text{card } U$ 
proof –
  have call-class:  $B' = B \vee k * \text{cap} \leq \text{card } U$ 
    by (rule direct-insert-costed-bmssp-Suc-success-or-threshold
      [OF run sound pre S-reaches])
  then show ?thesis
  proof
    assume B'-eq:  $B' = B$ 
    show ?thesis
    proof (rule direct-insert-costed-bmssp-source-card-le-if-sound-label-below-output
      [OF run sound pre S-reaches])
      fix x
      assume  $x \in S$ 
      then show  $\text{below-bound } (d \ x) \ B'$ 
        using below B'-eq by simp
      qed
    next
      assume threshold:  $k * \text{cap} \leq \text{card } U$ 
      have  $\text{cap} \leq k * \text{cap}$ 
      using k-pos by (cases k) simp-all
      then show ?thesis
        using S-cap threshold by linarith
      qed
    qed
  qed

```

The child-cost lemmas extract the recursive calls hidden inside a partition loop. Each child call is paired with the range slice it completed, together with the exact source set pulled for that child and the proof obligations needed to apply the induction hypothesis. The additional *below-bound* $(d \ x)$ *B-child* invariant is what makes source progress available without re-deriving the pull/split equality at every later use.

theorem *direct-insert-costed-partition-loop-state-child-cost-calls-below*:
 $\text{direct-insert-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs} \implies$
 $\text{length } \text{child-costs} = \text{length } bs \wedge$
 list-all2
 $(\lambda c\text{-child } U\text{-child}. \exists S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\text{direct-insert-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child} \wedge$
 $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child} \wedge$
 $(\forall x \in S\text{-child}. \text{reachable } s \ x) \wedge$
 $\text{card } S\text{-child} \leq M\text{-of } l \wedge$
 $(\forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child}))$
 $\text{child-costs } (\text{range-tree-child-list } P \ a \ bs)$

and *direct-insert-costed-bmssp-child-cost-calls-below-trivial*:
direct-insert-costed-bmssp Δ *M-of t h k cap l d S B d' B' U c* $\implies$ *True*
proof (*induction rule*):
direct-insert-costed-partition-loop-state-direct-insert-costed-bmssp.inducts)
case (*Direct-Insert-Costed-State-Done D a B d' P* Δ *M-of t h k cap l d*)
show *?case*
by *simp*
next
case (*Direct-Insert-Costed-State-Stop a B d' P k cap* Δ *M-of t h l d D*)
show *?case*
by *simp*
next
case (*Direct-Insert-Costed-State-Step D M-of l Bmax S-pull beta D-pull B d*
P a b B' d' k cap Δ *t h d-child U-child c-child*
direct-edge-batch lower-edge-batch source-batch batch D-next c-pull
c-direct c-lower c-sources betas bs Us-tail U-tail c-tail
child-costs-tail c)
have *tail*:
length child-costs-tail = length bs $\wedge$
list-all2
 $(\lambda c\text{-child } U\text{-child}. \exists S\text{-child } B\text{-child } d\text{-child } B\text{-child'}. \text{direct-insert-costed-bmssp } \Delta \text{ M-of t h k cap l d } S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child' } U\text{-child } c\text{-child} \wedge$
 $\text{bmssp-pre-full } d \text{ S-child } B\text{-child} \wedge$
 $(\forall x \in S\text{-child}. \text{reachable } s \ x) \wedge$
 $\text{card } S\text{-child} \leq \text{M-of l} \wedge$
 $(\forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child}))$
child-costs-tail (range-tree-child-list P b bs)
using *Direct-Insert-Costed-State-Step.IH* **by** *blast*
have *card-pull*: *card S-pull* $\leq$ *M-of l*
using *Direct-Insert-Costed-State-Step*
unfolding *pull-separates-def* **by** *blast*
have *below-pull*: $\forall x \in S\text{-pull}. \text{below-bound } (d \ x) \ (Fin \ beta)$
proof
fix *x*
assume *xs*: $x \in S\text{-pull}$
have *S-pull* = *split-below d P beta*
using *Direct-Insert-Costed-State-Step* **by** *blast*
then show *below-bound (d x) (Fin beta)*
using *xs* **unfolding** *split-below-def* **by** *auto*
qed
have *head*:
 $\exists S\text{-child } B\text{-child } d\text{-child' } B\text{-child'}. \text{direct-insert-costed-bmssp } \Delta \text{ M-of t h k cap l d } S\text{-child } B\text{-child}$
 $d\text{-child' } B\text{-child' } U\text{-child } c\text{-child} \wedge$
 $\text{bmssp-pre-full } d \text{ S-child } B\text{-child} \wedge$
 $(\forall x \in S\text{-child}. \text{reachable } s \ x) \wedge$
 $\text{card } S\text{-child} \leq \text{M-of l} \wedge$
 $(\forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child})$

```

    using Direct-Insert-Costed-State-Step card-pull below-pull by blast
  show ?case
    using tail head Direct-Insert-Costed-State-Step by simp
next
  case (Direct-Insert-Costed-Base S x Δ M-of t h k cap d B)
  show ?case
    by simp
next
  case (Direct-Insert-Costed-Step D k cap d S B c-insert t Δ M-of h l
    d' a betas bs B' Us U-loop c-loop child-costs-loop U c)
  show ?case
    by simp
qed

theorem direct-insert-costed-partition-loop-state-child-cost-calls-below-with-bounds:
  direct-insert-costed-partition-loop-state Δ M-of t h k cap l d P B d' D a
  betas bs B' Us U c child-costs ⇒
  list-all2
  ( $\lambda c\text{-child } UB. \text{ case } UB \text{ of } (U\text{-child}, B\text{-child}) \Rightarrow$ 
     $\exists S\text{-child } d\text{-child } B\text{-child}'.$ 
    direct-insert-costed-bmssp Δ M-of t h k cap l d S-child B-child
    d-child B-child' U-child c-child ∧
    bmssp-pre-full d S-child B-child ∧
     $(\forall x \in S\text{-child}. \text{reachable } s \ x) \wedge$ 
    card S-child ≤ M-of l ∧
     $(\forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child}))$ 
    child-costs (range-tree-child-bound-pair-list P a betas bs))
and direct-insert-costed-bmssp-child-cost-calls-below-with-bounds-trivial:
  direct-insert-costed-bmssp Δ M-of t h k cap l d S B d' B' U c ⇒ True
proof (induction rule:
  direct-insert-costed-partition-loop-state-direct-insert-costed-bmssp.inducts)
  case (Direct-Insert-Costed-State-Done D a B d' P Δ M-of t h k cap l d)
  show ?case
    by simp
next
  case (Direct-Insert-Costed-State-Stop a B d' P k cap Δ M-of t h l d D)
  show ?case
    by simp
next
  case (Direct-Insert-Costed-State-Step D M-of l Bmax S-pull beta D-pull B d
    P a b B' d' k cap Δ t h d-child U-child c-child
    direct-edge-batch lower-edge-batch source-batch batch D-next c-pull
    c-direct c-lower c-sources betas bs Us-tail U-tail c-tail
    child-costs-tail c)
  have tail:
    list-all2
    ( $\lambda c\text{-child } UB. \text{ case } UB \text{ of } (U\text{-child}, B\text{-child}) \Rightarrow$ 
       $\exists S\text{-child } d\text{-child } B\text{-child}'.$ 
      direct-insert-costed-bmssp Δ M-of t h k cap l d S-child B-child

```

```

      d-child B-child' U-child c-child ∧
      bmssp-pre-full d S-child B-child ∧
      (∀ x ∈ S-child. reachable s x) ∧
      card S-child ≤ M-of l ∧
      (∀ x ∈ S-child. below-bound (d x) B-child))
    child-costs-tail (range-tree-child-bound-pair-list P b betas bs)
  using Direct-Insert-Costed-State-Step.IH by blast
have card-pull: card S-pull ≤ M-of l
  using Direct-Insert-Costed-State-Step
  unfolding pull-separates-def by blast
have below-pull: ∀ x ∈ S-pull. below-bound (d x) (Fin beta)
proof
  fix x
  assume xS: x ∈ S-pull
  have S-pull = split-below d P beta
    using Direct-Insert-Costed-State-Step by blast
  then show below-bound (d x) (Fin beta)
    using xS unfolding split-below-def by auto
qed
have head:
  ∃ S-child d-child' B-child'.
    direct-insert-costed-bmssp Δ M-of t h k cap l d S-child (Fin beta)
    d-child' B-child' U-child c-child ∧
    bmssp-pre-full d S-child (Fin beta) ∧
    (∀ x ∈ S-child. reachable s x) ∧
    card S-child ≤ M-of l ∧
    (∀ x ∈ S-child. below-bound (d x) (Fin beta))
  using Direct-Insert-Costed-State-Step card-pull below-pull by blast
show ?case
  using tail head Direct-Insert-Costed-State-Step by simp
next
case (Direct-Insert-Costed-Base S x Δ M-of t h k cap d B)
show ?case
  by simp
next
case (Direct-Insert-Costed-Step D k cap d S B c-insert t Δ M-of h l
      d' a betas bs B' Us U-loop c-loop child-costs-loop U c)
show ?case
  by simp
qed

theorem direct-insert-costed-partition-loop-state-child-cost-bounds:
  assumes run:
    direct-insert-costed-partition-loop-state Δ M-of t h k cap l d P B d' D a
    betas bs B' Us U c child-costs
  and child-bound:
    ∧ c-child U-child S-child B-child d-child B-child'.
    [direct-insert-costed-bmssp Δ M-of t h k cap l d S-child B-child
      d-child B-child' U-child c-child;

```

$bmssp\text{-}pre\text{-}full\ d\ S\text{-}child\ B\text{-}child;$
 $\bigwedge x. x \in S\text{-}child \implies reachable\ s\ x;$
 $card\ S\text{-}child \leq M\text{-}of\ l;$
 $\bigwedge x. x \in S\text{-}child \implies below\text{-}bound\ (d\ x)\ B\text{-}child]$
 $\implies c\text{-}child \leq level\text{-}range\text{-}cost\text{-}bound\ A\ R\ L\ U\text{-}child$
shows $list\text{-}all2\ (\lambda c\text{-}child\ U\text{-}child.$
 $c\text{-}child \leq level\text{-}range\text{-}cost\text{-}bound\ A\ R\ L\ U\text{-}child)$
 $child\text{-}costs\ (range\text{-}tree\text{-}child\text{-}list\ P\ a\ bs)$
proof –
have $calls:$
 $list\text{-}all2$
 $(\lambda c\text{-}child\ U\text{-}child. \exists S\text{-}child\ B\text{-}child\ d\text{-}child\ B\text{-}child'.$
 $direct\text{-}insert\text{-}costed\text{-}bmssp\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ l\ d\ S\text{-}child\ B\text{-}child$
 $d\text{-}child\ B\text{-}child'\ U\text{-}child\ c\text{-}child \wedge$
 $bmssp\text{-}pre\text{-}full\ d\ S\text{-}child\ B\text{-}child \wedge$
 $(\forall x \in S\text{-}child. reachable\ s\ x) \wedge$
 $card\ S\text{-}child \leq M\text{-}of\ l \wedge$
 $(\forall x \in S\text{-}child. below\text{-}bound\ (d\ x)\ B\text{-}child))$
 $child\text{-}costs\ (range\text{-}tree\text{-}child\text{-}list\ P\ a\ bs)$
using $direct\text{-}insert\text{-}costed\text{-}partition\text{-}loop\text{-}state\text{-}child\text{-}cost\text{-}calls\text{-}below$
 $[OF\ run]$ **by** $blast$
then show $?thesis$
proof ($induction\ rule: list\text{-}all2\text{-}induct$)
case Nil
then show $?case$ **by** $simp$
next
case ($Cons\ c\text{-}child\ child\text{-}costs\ U\text{-}child\ child\text{-}ranges$)
obtain $S\text{-}child\ B\text{-}child\ d\text{-}child\ B\text{-}child'$ **where** $child:$
 $direct\text{-}insert\text{-}costed\text{-}bmssp\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ l\ d\ S\text{-}child\ B\text{-}child$
 $d\text{-}child\ B\text{-}child'\ U\text{-}child\ c\text{-}child$
and $child\text{-}pre:$ $bmssp\text{-}pre\text{-}full\ d\ S\text{-}child\ B\text{-}child$
and $child\text{-}reaches:$ $\forall x \in S\text{-}child. reachable\ s\ x$
and $child\text{-}card:$ $card\ S\text{-}child \leq M\text{-}of\ l$
and $child\text{-}below:$ $\forall x \in S\text{-}child. below\text{-}bound\ (d\ x)\ B\text{-}child$
using $Cons.hyps$ **by** $blast$
have $head:$
 $c\text{-}child \leq level\text{-}range\text{-}cost\text{-}bound\ A\ R\ L\ U\text{-}child$
by ($rule\ child\text{-}bound[OF\ child\ child\text{-}pre\ -\ child\text{-}card]$)
 $(use\ child\text{-}reaches\ child\text{-}below\ in\ blast)+$
show $?case$
using $head\ Cons.IH\ Cons.hyps$ **by** $simp$
qed
qed

theorem $direct\text{-}insert\text{-}costed\text{-}partition\text{-}loop\text{-}state\text{-}child\text{-}cost\text{-}bounds\text{-}with\text{-}invariants:$
 $direct\text{-}insert\text{-}costed\text{-}partition\text{-}loop\text{-}state\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ l\ d\ P\ B\ d'\ D\ a$
 $betas\ bs\ B'\ Us\ U\ c\ child\text{-}costs \implies$
 $P \subseteq V \implies$
 $k * card\ P \leq cap \implies$

$tree\text{-}antichain\ P \implies$
 $(\bigwedge c\text{-}child\ U\text{-}child\ S\text{-}child\ B\text{-}child\ d\text{-}child\ B\text{-}child'. \\
\llbracket direct\text{-}insert\text{-}costed\text{-}bmssp\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ l\ d\ S\text{-}child\ B\text{-}child \\
d\text{-}child\ B\text{-}child'\ U\text{-}child\ c\text{-}child; \\
bmssp\text{-}pre\text{-}full\ d\ S\text{-}child\ B\text{-}child; \\
\bigwedge x. x \in S\text{-}child \implies reachable\ s\ x; \\
card\ S\text{-}child \leq M\text{-}of\ l; \\
\bigwedge x. x \in S\text{-}child \implies below\text{-}bound\ (d\ x)\ B\text{-}child; \\
k * card\ S\text{-}child \leq cap; \\
tree\text{-}antichain\ S\text{-}child \rrbracket \\
\implies c\text{-}child \leq level\text{-}range\text{-}cost\text{-}bound\ A\ R\ L\ U\text{-}child) \implies \\
list\text{-}all2\ (\lambda c\text{-}child\ U\text{-}child. \\
c\text{-}child \leq level\text{-}range\text{-}cost\text{-}bound\ A\ R\ L\ U\text{-}child) \\
child\text{-}costs\ (range\text{-}tree\text{-}child\text{-}list\ P\ a\ bs) \\
and direct\text{-}insert\text{-}costed\text{-}bmssp\text{-}child\text{-}cost\text{-}bounds\text{-}with\text{-}invariants\text{-}trivial: \\
direct\text{-}insert\text{-}costed\text{-}bmssp\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ l\ d\ S\ B\ d'\ B'\ U\ c \implies True \\
proof (induction rule: \\
direct\text{-}insert\text{-}costed\text{-}partition\text{-}loop\text{-}state\text{-}direct\text{-}insert\text{-}costed\text{-}bmssp.inducts) \\
case (Direct\text{-}Insert\text{-}Costed\text{-}State\text{-}Done\ D\ a\ B\ d'\ P\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ l\ d) \\
then show ?case \\
by simp \\
next \\
case (Direct\text{-}Insert\text{-}Costed\text{-}State\text{-}Stop\ a\ B\ d'\ P\ k\ cap\ \Delta\ M\text{-}of\ t\ h\ l\ d\ D) \\
then show ?case \\
by simp \\
next \\
case (Direct\text{-}Insert\text{-}Costed\text{-}State\text{-}Step\ D\ M\text{-}of\ l\ Bmax\ S\text{-}pull\ beta\ D\text{-}pull\ B\ d \\
P\ a\ b\ B'\ d'\ k\ cap\ \Delta\ t\ h\ d\text{-}child\ U\text{-}child\ c\text{-}child \\
direct\text{-}edge\text{-}batch\ lower\text{-}edge\text{-}batch\ source\text{-}batch\ batch\ D\text{-}next\ c\text{-}pull \\
c\text{-}direct\ c\text{-}lower\ c\text{-}sources\ betas\ bs\ Us\text{-}tail\ U\text{-}tail\ c\text{-}tail \\
child\text{-}costs\text{-}tail\ c) \\
have tail: \\
list\text{-}all2\ (\lambda c\text{-}child\ U\text{-}child. \\
c\text{-}child \leq level\text{-}range\text{-}cost\text{-}bound\ A\ R\ L\ U\text{-}child) \\
child\text{-}costs\text{-}tail\ (range\text{-}tree\text{-}child\text{-}list\ P\ b\ bs) \\
using Direct\text{-}Insert\text{-}Costed\text{-}State\text{-}Step.IH Direct\text{-}Insert\text{-}Costed\text{-}State\text{-}Step.premis \\
by blast \\
have card\text{-}pull: card\ S\text{-}pull $\leq$ M\text{-}of\ l \\
using Direct\text{-}Insert\text{-}Costed\text{-}State\text{-}Step \\
unfolding pull\text{-}separates\text{-}def **by blast** \\
have below\text{-}pull: $\bigwedge x. x \in S\text{-}pull \implies below\text{-}bound\ (d\ x)\ (Fin\ beta)$ \\
proof – \\
fix x \\
assume xS: $x \in S\text{-}pull$ \\
have S\text{-}pull = split\text{-}below\ d\ P\ beta \\
using Direct\text{-}Insert\text{-}Costed\text{-}State\text{-}Step **by blast** \\
then show below\text{-}bound\ (d\ x)\ (Fin\ beta) \\
using xS **unfolding** split\text{-}below\text{-}def **by auto** \\
qed$

```

have pull-k-cap:  $k * \text{card } S\text{-pull} \leq \text{cap}$ 
proof –
  have S-pull = split-below d P beta
    using Direct-Insert-Costed-State-Step by blast
  then show ?thesis
    using split-below-scaled-card-le
      [OF Direct-Insert-Costed-State-Step.prems(1)
        Direct-Insert-Costed-State-Step.prems(2), of d beta]
    by simp
qed
have pull-anti: tree-antichain S-pull
proof –
  have S-pull = split-below d P beta
    using Direct-Insert-Costed-State-Step by blast
  then show ?thesis
    using split-below-tree-antichain
      [OF Direct-Insert-Costed-State-Step.prems(3), of d beta]
    by simp
qed
have child-run:
  direct-insert-costed-bmssp  $\Delta$  M-of t h k cap l d S-pull (Fin beta)
  d-child (Fin b) U-child c-child
    using Direct-Insert-Costed-State-Step by blast
have pre-pull: bmssp-pre-full d S-pull (Fin beta)
    using Direct-Insert-Costed-State-Step by blast
have reaches-pull:  $\bigwedge x. x \in S\text{-pull} \implies \text{reachable } s \ x$ 
    using Direct-Insert-Costed-State-Step by blast
have head:
  c-child  $\leq$  level-range-cost-bound A R L U-child
  by (rule Direct-Insert-Costed-State-Step.prems(4))
  [OF child-run pre-pull reaches-pull card-pull below-pull
    pull-k-cap pull-anti)
show ?case
  using head tail Direct-Insert-Costed-State-Step by simp
next
case (Direct-Insert-Costed-Base S x  $\Delta$  M-of t h k cap d B)
then show ?case
  by simp
next
case (Direct-Insert-Costed-Step D k cap d S B c-insert t  $\Delta$  M-of h l
  d' a betas bs B' Us U-loop c-loop child-costs-loop U c)
then show ?case
  by simp
qed

theorem direct-insert-costed-partition-loop-state-child-amortized-cost-bounds-with-
invariants:
  direct-insert-costed-partition-loop-state  $\Delta$  M-of t h k cap l d P B d' D a
  betas bs B' Us U c child-costs  $\implies$ 

```

$P \subseteq V \implies$
 $k * \text{card } P \leq \text{cap} \implies$
 $\text{tree-antichain } P \implies$
 $(\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}')$
 $\llbracket \text{direct-insert-costed-bmssp } \Delta M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$
 $\quad \text{bmssp-pre-full } d \ S\text{-child } B\text{-child};$
 $\quad \bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\quad \text{card } S\text{-child} \leq M\text{-of } l;$
 $\quad \bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child};$
 $\quad k * \text{card } S\text{-child} \leq \text{cap};$
 $\quad \text{tree-antichain } S\text{-child} \rrbracket$
 $\implies c\text{-child} \leq$
 $\quad \text{bmssp-amortized-cost-bound } A \ R \ h \ t \ q \ L \ B\text{-child } U\text{-child}) \implies$
 $\text{list-all2 } (\lambda c\text{-child } UB. \text{ case } UB \text{ of } (U\text{-child}, B\text{-child}) \Rightarrow$
 $\quad c\text{-child} \leq \text{bmssp-amortized-cost-bound } A \ R \ h \ t \ q \ L \ B\text{-child } U\text{-child})$
 $\text{child-costs } (\text{range-tree-child-bound-pair-list } P \ a \ \text{betas } bs)$
and $\text{direct-insert-costed-bmssp-child-amortized-cost-bounds-with-invariants-trivial:}$
 $\text{direct-insert-costed-bmssp } \Delta M\text{-of } t \ h \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c \implies \text{True}$
proof (*induction rule:*
 $\text{direct-insert-costed-partition-loop-state-direct-insert-costed-bmssp.inducts})$
case ($\text{Direct-Insert-Costed-State-Done } D \ a \ B \ d' \ P \ \Delta M\text{-of } t \ h \ k \ \text{cap } l \ d$)
then show $?case$
by simp
next
case ($\text{Direct-Insert-Costed-State-Stop } a \ B \ d' \ P \ k \ \text{cap } \Delta M\text{-of } t \ h \ l \ d \ D$)
then show $?case$
by simp
next
case ($\text{Direct-Insert-Costed-State-Step } D \ M\text{-of } l \ B\text{max } S\text{-pull } \text{beta } D\text{-pull } B \ d$
 $P \ a \ b \ B' \ d' \ k \ \text{cap } \Delta t \ h \ d\text{-child } U\text{-child } c\text{-child}$
 $\text{direct-edge-batch } \text{lower-edge-batch } \text{source-batch } \text{batch } D\text{-next } c\text{-pull}$
 $c\text{-direct } c\text{-lower } c\text{-sources } \text{betas } bs \ Us\text{-tail } U\text{-tail } c\text{-tail}$
 $\text{child-costs-tail } c)$
have tail:
 $\text{list-all2 } (\lambda c\text{-child } UB. \text{ case } UB \text{ of } (U\text{-child}, B\text{-child}) \Rightarrow$
 $\quad c\text{-child} \leq \text{bmssp-amortized-cost-bound } A \ R \ h \ t \ q \ L \ B\text{-child } U\text{-child})$
 $\text{child-costs-tail } (\text{range-tree-child-bound-pair-list } P \ b \ \text{betas } bs)$
using $\text{Direct-Insert-Costed-State-Step.IH } \text{Direct-Insert-Costed-State-Step.premis}$
by blast
have $\text{card-pull: } \text{card } S\text{-pull} \leq M\text{-of } l$
using $\text{Direct-Insert-Costed-State-Step}$
unfolding $\text{pull-separates-def}$ **by** blast
have $\text{below-pull: } \bigwedge x. x \in S\text{-pull} \implies \text{below-bound } (d \ x) \ (\text{Fin } \text{beta})$
proof –
fix x
assume $xS: x \in S\text{-pull}$
have $S\text{-pull} = \text{split-below } d \ P \ \text{beta}$
using $\text{Direct-Insert-Costed-State-Step}$ **by** blast

```

    then show below-bound (d x) (Fin beta)
      using xS unfolding split-below-def by auto
  qed
  have pull-k-cap: k * card S-pull ≤ cap
  proof -
    have S-pull = split-below d P beta
      using Direct-Insert-Costed-State-Step by blast
    then show ?thesis
      using split-below-scaled-card-le
        [OF Direct-Insert-Costed-State-Step.prem1(1)
          Direct-Insert-Costed-State-Step.prem1(2), of d beta]
      by simp
  qed
  have pull-anti: tree-antichain S-pull
  proof -
    have S-pull = split-below d P beta
      using Direct-Insert-Costed-State-Step by blast
    then show ?thesis
      using split-below-tree-antichain
        [OF Direct-Insert-Costed-State-Step.prem1(3), of d beta]
      by simp
  qed
  have child-run:
    direct-insert-costed-bmssp Δ M-of t h k cap l d S-pull (Fin beta)
    d-child (Fin b) U-child c-child
    using Direct-Insert-Costed-State-Step by blast
  have pre-pull: bmssp-pre-full d S-pull (Fin beta)
    using Direct-Insert-Costed-State-Step by blast
  have reaches-pull: ∧x. x ∈ S-pull ⇒ reachable s x
    using Direct-Insert-Costed-State-Step by blast
  have head:
    c-child ≤
      bmssp-amortized-cost-bound A R h t q L (Fin beta) U-child
    by (rule Direct-Insert-Costed-State-Step.prem1(4))
      [OF child-run pre-pull reaches-pull card-pull below-pull
        pull-k-cap pull-anti])
  show ?case
    using head tail Direct-Insert-Costed-State-Step by simp
  next
    case (Direct-Insert-Costed-Base S x Δ M-of t h k cap d B)
    then show ?case
      by simp
  next
    case (Direct-Insert-Costed-Step D k cap d S B c-insert t Δ M-of h l
      d' a betas bs B' Us U-loop c-loop child-costs-loop U c)
    then show ?case
      by simp
  qed

```

The central loop cost bound separates the four local charges that survive

after recursive child costs have been bounded: pulled child sources, direct edge-range inserts, lower edge-range batches, and source batches. The list functions over *range-tree-child-list* and its edge-range variants are the combinatorial map from one loop trace to disjoint graph regions. Their disjointness lemmas are what prevent the loop from paying for the same edge range repeatedly.

theorem *direct-insert-costed-partition-loop-state-cost-bound-by-child-sources-and-edge-ranges*:

```

  direct-insert-costed-partition-loop-state Δ M-of t h k cap l d P B d' D a
    betas bs B' Us U c child-costs ⇒
  sound-label d ⇒
  ∃ child-sources.
    length child-sources = length bs ∧
    list-all2
      (λ S-child U-child. ∃ c-child B-child d-child B-child'.
        direct-insert-costed-bmssp Δ M-of t h k cap l d S-child B-child
          d-child B-child' U-child c-child ∧
        bmssp-pre-full d S-child B-child ∧
        (∀ x ∈ S-child. reachable s x) ∧
        card S-child ≤ M-of l ∧
        (∀ x ∈ S-child. below-bound (d x) B-child))
        child-sources (range-tree-child-list P a bs) ∧
      c ≤ sum-list child-costs + sum-list (map card child-sources) +
      t * sum-list
        (map card (range-tree-child-direct-edge-range-list P B a betas bs)) +
      h * sum-list
        (map card (range-tree-child-edge-range-list P a betas bs)) +
      h * sum-list (map card child-sources)
and direct-insert-costed-bmssp-cost-child-sources-and-edge-ranges-trivial:
  direct-insert-costed-bmssp Δ M-of t h k cap l d S B d' B' U c ⇒ True
proof (induction rule:
  direct-insert-costed-partition-loop-state-direct-insert-costed-bmssp.inducts)
case (Direct-Insert-Costed-State-Done D a B d' P Δ M-of t h k cap l d)
show ?case
  by (intro exI[of - []]) simp
next
case (Direct-Insert-Costed-State-Stop a B d' P k cap Δ M-of t h l d D)
show ?case
  by (intro exI[of - []]) simp
next
case (Direct-Insert-Costed-State-Step D M-of l Bmax S-pull beta D-pull B d
  P a b B' d' k cap Δ t h d-child U-child c-child
  direct-edge-batch lower-edge-batch source-batch batch D-next c-pull
  c-direct c-lower c-sources betas bs Us-tail U-tail c-tail
  child-costs-tail c)
obtain child-sources-tail where len-tail:
  length child-sources-tail = length bs
and sources-tail:
  list-all2

```

```

    (λS-child U-child. ∃ c-child B-child d-child B-child'.
      direct-insert-costed-bmssp Δ M-of t h k cap l d S-child B-child
        d-child B-child' U-child c-child ∧
        bmssp-pre-full d S-child B-child ∧
        (∀ x ∈ S-child. reachable s x) ∧
        card S-child ≤ M-of l ∧
        (∀ x ∈ S-child. below-bound (d x) B-child))
      child-sources-tail (range-tree-child-list P b bs)
  and tail-cost:
    c-tail ≤
      sum-list child-costs-tail + sum-list (map card child-sources-tail) +
      t * sum-list
        (map card (range-tree-child-direct-edge-range-list P B b betas bs)) +
      h * sum-list
        (map card (range-tree-child-edge-range-list P b betas bs)) +
      h * sum-list (map card child-sources-tail)
  using Direct-Insert-Costed-State-Step.IH Direct-Insert-Costed-State-Step.prem
by blast
  have card-pull: card S-pull ≤ M-of l
  using Direct-Insert-Costed-State-Step
  unfolding pull-separates-def by blast
  have below-pull: ∀ x ∈ S-pull. below-bound (d x) (Fin beta)
  proof
    fix x
    assume xS: x ∈ S-pull
    have S-pull = split-below d P beta
    using Direct-Insert-Costed-State-Step by blast
    then show below-bound (d x) (Fin beta)
    using xS unfolding split-below-def by auto
  qed
  have pull-cost: c-pull ≤ card S-pull
  using Direct-Insert-Costed-State-Step
  unfolding partition-pull-cost-bound-def by blast
  have child-run:
    direct-insert-costed-bmssp Δ M-of t h k cap l d S-pull (Fin beta)
    d-child (Fin b) U-child c-child
  using Direct-Insert-Costed-State-Step by blast
  have child-pre: bmssp-pre-full d S-pull (Fin beta)
  using Direct-Insert-Costed-State-Step by blast
  have child-reaches: ∧ x. x ∈ S-pull ⇒ reachable s x
  using Direct-Insert-Costed-State-Step by blast
  have child-post:
    bmssp-post-full d S-pull (Fin beta) d-child (Fin b) U-child
  by (rule direct-insert-costed-bmssp-correct
    [OF child-run Direct-Insert-Costed-State-Step.prem child-pre child-reaches])
  have child-complete: complete-on d-child U-child
  using child-post unfolding bmssp-post-full-def by blast
  have U-child-eq: U-child = range-tree P a (Fin b)
  using Direct-Insert-Costed-State-Step by blast

```

have *U-child-reaches*: $\bigwedge u. u \in U\text{-child} \implies \text{reachable } s \ u$
unfolding *U-child-eq* *range-tree-def* *tree-set-def* *tree-path-def* **by** *blast*
have *direct-cost*:
 $c\text{-direct} \leq t * \text{card } (\text{outgoing-edges-range } U\text{-child } \text{beta } B)$
by (*rule partition-batch-cost-bound-edge-relaxation-pairs-in-bound*
 $[\text{where } d = d\text{-child}]$)
(use Direct-Insert-Costed-State-Step child-complete U-child-reaches in auto)
have *lower-source-cost*:
 $c\text{-lower} + c\text{-sources} \leq$
 $h * \text{card } (\text{outgoing-edges-range } U\text{-child } b \ (F\text{in } \text{beta})) +$
 $h * \text{card } S\text{-pull}$
by (*rule range-costed-split-batch-cost-le-range-card*
 $\text{where } d = d \text{ and } S = S\text{-pull and } d\text{-child} = d\text{-child and } b = b$
 $\text{and } \text{beta} = \text{beta}$)
(use Direct-Insert-Costed-State-Step child-complete U-child-reaches in auto)
let *?child-sources* = *S-pull* # *child-sources-tail*
have *len*: $\text{length } ?\text{child-sources} = \text{length } (b \# \text{bs})$
using *len-tail* **by** *simp*
have *head-source*:
 $\exists c\text{-child}' \ B\text{-child } d\text{-child}' \ B\text{-child}'.$
 $\text{direct-insert-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-pull } B\text{-child}$
 $d\text{-child}' \ B\text{-child}' \ U\text{-child } c\text{-child}' \wedge$
 $\text{bmssp-pre-full } d \ S\text{-pull } B\text{-child} \wedge$
 $(\forall x \in S\text{-pull}. \text{reachable } s \ x) \wedge$
 $\text{card } S\text{-pull} \leq M\text{-of } l \wedge$
 $(\forall x \in S\text{-pull}. \text{below-bound } (d \ x) \ B\text{-child})$
using *Direct-Insert-Costed-State-Step card-pull below-pull* **by** *blast*
have *sources*:
 list-all2
 $(\lambda S\text{-child } U\text{-child}. \exists c\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\text{direct-insert-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child} \wedge$
 $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child} \wedge$
 $(\forall x \in S\text{-child}. \text{reachable } s \ x) \wedge$
 $\text{card } S\text{-child} \leq M\text{-of } l \wedge$
 $(\forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child}))$
 $?child\text{-sources } (\text{range-tree-child-list } P \ a \ (b \# \text{bs}))$
using *head-source sources-tail Direct-Insert-Costed-State-Step* **by** *simp*
have *cost*:
 $c \leq \text{sum-list } (c\text{-child} \# \text{child-costs-tail}) +$
 $\text{sum-list } (\text{map card } ?\text{child-sources}) +$
 $t * \text{sum-list}$
 $(\text{map card}$
 $(\text{range-tree-child-direct-edge-range-list } P \ B \ a \ (\text{beta} \# \text{betas}) \ (b \# \text{bs}))) +$
 $h * \text{sum-list}$
 $(\text{map card}$
 $(\text{range-tree-child-edge-range-list } P \ a \ (\text{beta} \# \text{betas}) \ (b \# \text{bs}))) +$
 $h * \text{sum-list } (\text{map card } ?\text{child-sources})$
proof –

```

have c-eq:
  c = c-pull + c-direct + c-lower + c-sources + c-child + c-tail
  using Direct-Insert-Costed-State-Step by blast
have c-pull + c-direct + c-lower + c-sources + c-child + c-tail ≤
  card S-pull +
  t * card (outgoing-edges-range U-child beta B) +
  (h * card (outgoing-edges-range U-child b (Fin beta))) +
  h * card S-pull +
  c-child +
  (sum-list child-costs-tail + sum-list (map card child-sources-tail) +
  t * sum-list
    (map card (range-tree-child-direct-edge-range-list P B b betas bs)) +
  h * sum-list
    (map card (range-tree-child-edge-range-list P b betas bs)) +
  h * sum-list (map card child-sources-tail))
  using pull-cost direct-cost lower-source-cost tail-cost by linarith
also have ... =
  sum-list (c-child # child-costs-tail) +
  sum-list (map card ?child-sources) +
  t * (card (outgoing-edges-range U-child beta B) +
  sum-list
    (map card (range-tree-child-direct-edge-range-list P B b betas bs))) +
  h * (card (outgoing-edges-range U-child b (Fin beta)) +
  sum-list
    (map card (range-tree-child-edge-range-list P b betas bs))) +
  h * sum-list (map card ?child-sources)
  by (simp add: algebra-simps)
also have ... =
  sum-list (c-child # child-costs-tail) +
  sum-list (map card ?child-sources) +
  t * sum-list
    (map card
      (range-tree-child-direct-edge-range-list P B a (beta # betas) (b # bs))) +
  h * sum-list
    (map card
      (range-tree-child-edge-range-list P a (beta # betas) (b # bs))) +
  h * sum-list (map card ?child-sources)
  using U-child-eq by simp
finally show ?thesis
  using c-eq by simp
qed
show ?case
  by (intro exI [of - ?child-sources] conjI)
    (use len sources cost in simp-all)
next
  case (Direct-Insert-Costed-Base S x Δ M-of t h k cap d B)
  show ?case
    by simp
next

```

case (*Direct-Insert-Costed-Step* D k $\text{cap } d$ S B $c\text{-insert } t$ Δ $M\text{-of } h$ l
 d' a $\text{betas } bs$ B' Us $U\text{-loop } c\text{-loop } \text{child-costs-loop } U$ c)
show ?*case*
by *simp*
qed

theorem *direct-insert-costed-partition-loop-state-cost-from-child-source-and-edge-range-bounds*:

assumes *run*:

$\text{direct-insert-costed-partition-loop-state } \Delta$ $M\text{-of } t$ h k $\text{cap } l$ d P B d' D a
 $\text{betas } bs$ B' Us U c child-costs

and *sound*: $\text{sound-label } d$

and *child-cost-bounds*:

$\text{list-all2 } (\lambda c\text{-child } U\text{-child}.$

$c\text{-child} \leq \text{level-range-cost-bound } A$ R L $U\text{-child}$)

$\text{child-costs } (\text{range-tree-child-list } P$ a $bs)$

and *source-progress*:

$\bigwedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}'$ $U\text{-child } c\text{-child}.$

$\llbracket \text{direct-insert-costed-bmssp } \Delta$ $M\text{-of } t$ h k $\text{cap } l$ d $S\text{-child } B\text{-child}$

$d\text{-child } B\text{-child}'$ $U\text{-child } c\text{-child};$

$\text{bmssp-pre-full } d$ $S\text{-child } B\text{-child};$

$\bigwedge x. x \in S\text{-child} \implies \text{reachable } s$ $x;$

$\text{card } S\text{-child} \leq M\text{-of } l;$

$\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d$ $x)$ $B\text{-child} \rrbracket$

$\implies \text{card } S\text{-child} \leq \text{card } U\text{-child}$

shows $c \leq$

$A * L * \text{card } (\text{range-tree-chain } P$ a bs $B')$ $+$

$R * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P$ a bs $B'))$ $+$

$t * \text{sum-list}$

$(\text{map } \text{card } (\text{range-tree-child-direct-edge-range-list } P$ B a $\text{betas } bs))$ $+$

$h * \text{sum-list}$

$(\text{map } \text{card } (\text{range-tree-child-edge-range-list } P$ a $\text{betas } bs))$ $+$

$(\text{Suc } h) * \text{card } (\text{range-tree-chain } P$ a bs $B')$

proof –

obtain *child-sources* **where** *sources*:

list-all2

$(\lambda S\text{-child } U\text{-child}. \exists c\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$

$\text{direct-insert-costed-bmssp } \Delta$ $M\text{-of } t$ h k $\text{cap } l$ d $S\text{-child } B\text{-child}$

$d\text{-child } B\text{-child}'$ $U\text{-child } c\text{-child} \wedge$

$\text{bmssp-pre-full } d$ $S\text{-child } B\text{-child} \wedge$

$(\forall x \in S\text{-child}. \text{reachable } s$ $x) \wedge$

$\text{card } S\text{-child} \leq M\text{-of } l \wedge$

$(\forall x \in S\text{-child}. \text{below-bound } (d$ $x)$ $B\text{-child}))$

$\text{child-sources } (\text{range-tree-child-list } P$ a $bs)$

and *cost*:

$c \leq \text{sum-list } \text{child-costs} + \text{sum-list } (\text{map } \text{card } \text{child-sources}) +$

$t * \text{sum-list}$

$(\text{map } \text{card } (\text{range-tree-child-direct-edge-range-list } P$ B a $\text{betas } bs))$ $+$

$h * \text{sum-list}$

```

      (map card (range-tree-child-edge-range-list P a betas bs)) +
      h * sum-list (map card child-sources)
    using direct-insert-costed-partition-loop-state-cost-bound-by-child-sources-and-
edge-ranges
    [OF run sound] by blast
  have mono: nondecreasing-from a bs
  by (rule direct-insert-costed-partition-loop-state-mono[OF run])
  have source-dom:
    list-all2 ( $\lambda S\text{-child } U\text{-child. card } S\text{-child} \leq \text{card } U\text{-child}$ )
    child-sources (range-tree-child-list P a bs)
  using sources
  proof (induction rule: list-all2-induct)
    case Nil
    then show ?case by simp
  next
  case (Cons S-child child-sources U-child child-sets)
  obtain c-child B-child d-child B-child' where child:
    direct-insert-costed-bmssp  $\Delta$  M-of t h k cap l d S-child B-child
    d-child B-child' U-child c-child
  and child-pre: bmssp-pre-full d S-child B-child
  and child-reaches:  $\forall x \in S\text{-child. reachable } s \ x$ 
  and child-card:  $\text{card } S\text{-child} \leq M\text{-of } l$ 
  and child-below:  $\forall x \in S\text{-child. below-bound } (d \ x) \ B\text{-child}$ 
  using Cons.hyps by blast
  have head:  $\text{card } S\text{-child} \leq \text{card } U\text{-child}$ 
  by (rule source-progress[OF child child-pre - child-card])
    (use child-reaches child-below in blast)+
  show ?case
  using head Cons.IH Cons.hyps by simp
qed
  have source-sum:
    sum-list (map card child-sources)  $\leq$ 
    card (range-tree-chain P a bs B')
  by (rule sum-card-dominated-by-range-tree-child-list-le-chain
    [OF mono source-dom])
  have child-sum:
    sum-list child-costs  $\leq$ 
    A * L * card (range-tree-chain P a bs B') +
    R * card (outgoing-edges (range-tree-chain P a bs B'))
  by (rule child-costs-le-level-range-child-list-bound
    [OF mono child-cost-bounds])
  have c  $\leq$ 
    A * L * card (range-tree-chain P a bs B') +
    R * card (outgoing-edges (range-tree-chain P a bs B')) +
    t * sum-list
      (map card (range-tree-child-direct-edge-range-list P B a betas bs)) +
    h * sum-list
      (map card (range-tree-child-edge-range-list P a betas bs)) +
    sum-list (map card child-sources) +

```

```

    h * sum-list (map card child-sources)
  using cost child-sum by linarith
also have ... ≤
  A * L * card (range-tree-chain P a bs B') +
  R * card (outgoing-edges (range-tree-chain P a bs B')) +
  t * sum-list
    (map card (range-tree-child-direct-edge-range-list P B a betas bs)) +
  h * sum-list
    (map card (range-tree-child-edge-range-list P a betas bs)) +
  (Suc h) * card (range-tree-chain P a bs B')
proof -
  have source-part:
    sum-list (map card child-sources) +
    h * sum-list (map card child-sources) ≤
    Suc h * card (range-tree-chain P a bs B')
  proof -
    have h-part:
      h * sum-list (map card child-sources) ≤
      h * card (range-tree-chain P a bs B')
    using source-sum by simp
  have sum-list (map card child-sources) +
    h * sum-list (map card child-sources) ≤
    card (range-tree-chain P a bs B') +
    h * card (range-tree-chain P a bs B')
  using source-sum h-part by linarith
  then show ?thesis
    by (simp add: algebra-simps)
qed
show ?thesis
  using source-part by linarith
qed
finally show ?thesis .
qed

```

theorem *direct-insert-costed-partition-loop-state-cost-from-child-source-and-amortized-bounds:*

assumes *run:*

direct-insert-costed-partition-loop-state Δ *M-of t h k cap l d P B d' D a*
betas bs B' Us U c child-costs

and *sound:* *sound-label d*

and *child-cost-bounds:*

list-all2 (λc -child *UB*. *case UB of (U-child, B-child) $\Rightarrow$*
c-child $\leq$ *bmssp-amortized-cost-bound A R h t q L B-child U-child*)
child-costs (*range-tree-child-bound-pair-list P a betas bs*)

and *source-progress:*

$\bigwedge$ *S-child B-child d-child B-child' U-child c-child.*
 $\llbracket$ *direct-insert-costed-bmssp* Δ *M-of t h k cap l d S-child B-child*
d-child B-child' U-child c-child;
bmssp-pre-full d S-child B-child;

$\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\text{card } S\text{-child} \leq M\text{-of } l;$
 $\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child}]$
 $\implies \text{card } S\text{-child} \leq \text{card } U\text{-child}$

shows $c \leq$

$A * L * \text{card } (\text{range-tree-chain } P \ a \ bs \ B') +$
 $(R + q * h) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B')) +$
 $t * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-zero-edge-range-list } P \ a \ betas \ bs)) +$
 $t * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-direct-edge-range-list } P \ B \ a \ betas \ bs)) +$
 $h * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-edge-range-list } P \ a \ betas \ bs)) +$
 $(\text{Suc } h) * \text{card } (\text{range-tree-chain } P \ a \ bs \ B')$

proof –

obtain *child-sources* **where** *sources*:

list-all2

$(\lambda S\text{-child } U\text{-child}. \exists c\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\text{direct-insert-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child} \wedge$
 $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child} \wedge$
 $(\forall x \in S\text{-child}. \text{reachable } s \ x) \wedge$
 $\text{card } S\text{-child} \leq M\text{-of } l \wedge$
 $(\forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child}))$
 $\text{child-sources } (\text{range-tree-child-list } P \ a \ bs)$

and *cost*:

$c \leq \text{sum-list } \text{child-costs} + \text{sum-list } (\text{map card } \text{child-sources}) +$
 $t * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-direct-edge-range-list } P \ B \ a \ betas \ bs)) +$
 $h * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-edge-range-list } P \ a \ betas \ bs)) +$
 $h * \text{sum-list } (\text{map card } \text{child-sources})$

using *direct-insert-costed-partition-loop-state-cost-bound-by-child-sources-and-edge-ranges*

$[OF \ \text{run} \ \text{sound}]$ **by** *blast*

have *mono: nondecreasing-from a bs*

by $(\text{rule } \text{direct-insert-costed-partition-loop-state-mono}[OF \ \text{run}])$

have *source-dom*:

$\text{list-all2 } (\lambda S\text{-child } U\text{-child}. \text{card } S\text{-child} \leq \text{card } U\text{-child})$
 $\text{child-sources } (\text{range-tree-child-list } P \ a \ bs)$

using *sources*

proof $(\text{induction rule: list-all2-induct})$

case *Nil*

then show *?case* **by** *simp*

next

case $(\text{Cons } S\text{-child } \text{child-sources } U\text{-child } \text{child-sets})$

obtain *c-child B-child d-child B-child'* **where** *child*:

$\text{direct-insert-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}$


```

and child-pre: bmssp-pre-full d S-child B-child
and child-reaches:  $\forall x \in S\text{-child}. \text{reachable } s \ x$ 
and child-card:  $\text{card } S\text{-child} \leq M\text{-of } l$ 
and child-below:  $\forall x \in S\text{-child}. \text{below-bound } (d \ x) \ B\text{-child}$ 
using Cons.hyps by blast
have head:  $\text{card } S\text{-child} \leq \text{card } U\text{-child}$ 
by (rule source-progress[OF child child-pre - child-card])
    (use child-reaches child-below in blast)+
show ?case
using head Cons.IH Cons.hyps by simp
qed
have source-sum:
  sum-list (map card child-sources)  $\leq$ 
    card (range-tree-chain P a bs B')
by (rule sum-card-dominated-by-range-tree-child-list-le-chain
    [OF mono source-dom])
have child-sum-raw:
  sum-list child-costs  $\leq$ 
    A * L * sum-list (map card (range-tree-child-list P a bs)) +
    (R + q * h) *
    sum-list (map ( $\lambda U. \text{card } (\text{outgoing-edges } U)$ )
    (range-tree-child-list P a bs)) +
    t * sum-list
    (map card (range-tree-child-zero-edge-range-list P a betas bs))
by (rule sum-bmssp-amortized-child-bounds-le[OF child-cost-bounds])
have child-card-sum:
  sum-list (map card (range-tree-child-list P a bs))  $\leq$ 
    card (range-tree-chain P a bs B')
by (rule card-range-tree-child-list-le-chain[OF mono])
have child-out-sum:
  sum-list (map ( $\lambda U. \text{card } (\text{outgoing-edges } U)$ )
    (range-tree-child-list P a bs))  $\leq$ 
    card (outgoing-edges (range-tree-chain P a bs B'))
by (rule card-outgoing-edges-range-tree-child-list-le-chain[OF mono])
have child-sum:
  sum-list child-costs  $\leq$ 
    A * L * card (range-tree-chain P a bs B') +
    (R + q * h) * card (outgoing-edges (range-tree-chain P a bs B')) +
    t * sum-list
    (map card (range-tree-child-zero-edge-range-list P a betas bs))
proof -
have vterm:
  A * L * sum-list (map card (range-tree-child-list P a bs))  $\leq$ 
    A * L * card (range-tree-chain P a bs B')
using child-card-sum by simp
have eterm:
  (R + q * h) *
    sum-list (map ( $\lambda U. \text{card } (\text{outgoing-edges } U)$ )
    (range-tree-child-list P a bs))  $\leq$ 

```

```

      (R + q * h) * card (outgoing-edges (range-tree-chain P a bs B'))
    using child-out-sum by simp
  show ?thesis
    using child-sum-raw vterm eterm by linarith
qed
have c ≤
  A * L * card (range-tree-chain P a bs B') +
  (R + q * h) * card (outgoing-edges (range-tree-chain P a bs B')) +
  t * sum-list
    (map card (range-tree-child-zero-edge-range-list P a betas bs)) +
  t * sum-list
    (map card (range-tree-child-direct-edge-range-list P B a betas bs)) +
  h * sum-list
    (map card (range-tree-child-edge-range-list P a betas bs)) +
  sum-list (map card child-sources) +
  h * sum-list (map card child-sources)
  using cost child-sum by linarith
also have ... ≤
  A * L * card (range-tree-chain P a bs B') +
  (R + q * h) * card (outgoing-edges (range-tree-chain P a bs B')) +
  t * sum-list
    (map card (range-tree-child-zero-edge-range-list P a betas bs)) +
  t * sum-list
    (map card (range-tree-child-direct-edge-range-list P B a betas bs)) +
  h * sum-list
    (map card (range-tree-child-edge-range-list P a betas bs)) +
  (Suc h) * card (range-tree-chain P a bs B')
proof -
  have source-part:
    sum-list (map card child-sources) +
    h * sum-list (map card child-sources) ≤
    Suc h * card (range-tree-chain P a bs B')
  proof -
    have h-part:
      h * sum-list (map card child-sources) ≤
      h * card (range-tree-chain P a bs B')
    using source-sum by simp
  have sum-list (map card child-sources) +
    h * sum-list (map card child-sources) ≤
    card (range-tree-chain P a bs B') +
    h * card (range-tree-chain P a bs B')
  using source-sum h-part by linarith
  then show ?thesis
    by (simp add: algebra-simps)
qed
show ?thesis
  using source-part by linarith
qed
finally show ?thesis .

```

qed

The amortised closing theorem below keeps the direct-insert range term visible because the bucketed implementation has a sharper accounting story for it. The child calls contribute a lower-level amortised bound, the source-progress hypothesis absorbs the pulled source sets into completed child ranges, and the edge-range disjointness lemmas collapse the direct and lower edge lists into the outgoing-edge terms of the parent output.

theorem *direct-insert-costed-partition-loop-state-closes-amortized-bound-from-child-costs*:

assumes *run*:

direct-insert-costed-partition-loop-state Δ *M-of t h k cap l d P B d' D a*

betas bs B' Us U c child-costs

and *sound*: *sound-label d*

and *pre*: *bmssp-pre-full d P B*

and *P-reaches*: $\bigwedge x. x \in P \implies \text{reachable } s \ x$

and *child-cost-bounds*:

list-all2 ($\lambda c\text{-child } UB. \text{ case } UB \text{ of } (U\text{-child}, B\text{-child}) \implies$

c-child $\leq \text{bmssp-amortized-cost-bound } A \ R \ h \ t \ q \ L \ B\text{-child } U\text{-child}$)

child-costs (*range-tree-child-bound-pair-list P a betas bs*)

and *source-factor*: *Suc h* $\leq A$

and *source-progress*:

$\bigwedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}.$

$\llbracket \text{direct-insert-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$

d-child B-child' U-child c-child;

bmssp-pre-full d S-child B-child;

$\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x$;

card S-child $\leq M\text{-of } l$;

$\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child}$

$\implies \text{card } S\text{-child} \leq \text{card } U\text{-child}$

shows *c* $\leq \text{bmssp-amortized-cost-bound } A \ R \ h \ t \ (Suc \ q) \ (Suc \ L) \ B \ U$

proof –

have *trace*: *concrete-partition-loop-trace P B a bs d' B' Us U*

by (*rule direct-insert-costed-partition-loop-state-trace*
[*OF run sound pre P-reaches*])

have *children*:

list-all2 ($\lambda U \ X. U = X \wedge \text{complete-on } d' \ U$) *Us*

(*range-tree-chain-list P a bs B'*)

using *trace unfolding concrete-partition-loop-trace-def* **by** *blast*

have *U-def*: *U* = *bound-tree P (Fin a) $\cup \bigcup(\text{set } Us)$*

using *trace unfolding concrete-partition-loop-trace-def* **by** *blast*

have *Union-eq*:

$\bigcup(\text{set } Us) = \bigcup(\text{set } (\text{range-tree-chain-list } P \ a \ bs \ B'))$

using *children* **by** (*induction rule: list-all2-induct*) *auto*

have *U-eq-chain*:

U = *bound-tree P (Fin a) $\cup \text{range-tree-chain } P \ a \ bs \ B'$*

using *U-def Union-eq Union-range-tree-chain-list*[*of P a bs B'*] **by** *simp*

have *finite-U*: *finite U*

using *U-eq-chain* **by** *simp*

```

have chain-subset: range-tree-chain  $P$   $a$   $bs$   $B' \subseteq U$ 
  using U-eq-chain by blast
have card-chain-le: card (range-tree-chain  $P$   $a$   $bs$   $B'$ )  $\leq$  card  $U$ 
  by (rule card-mono[OF finite-U chain-subset])
have outgoing-subset:
  outgoing-edges (range-tree-chain  $P$   $a$   $bs$   $B'$ )  $\subseteq$  outgoing-edges  $U$ 
  by (rule outgoing-edges-mono[OF chain-subset])
have finite-out-U: finite (outgoing-edges  $U$ )
  by simp
have card-out-le:
  card (outgoing-edges (range-tree-chain  $P$   $a$   $bs$   $B'$ ))  $\leq$ 
    card (outgoing-edges  $U$ )
  by (rule card-mono[OF finite-out-U outgoing-subset])
have range-subset:
  outgoing-edges-range (range-tree-chain  $P$   $a$   $bs$   $B'$ )  $0$   $B \subseteq$ 
    outgoing-edges-range  $U$   $0$   $B$ 
  by (rule outgoing-edges-range-mono-sources[OF chain-subset])
have card-range-le:
  card (outgoing-edges-range (range-tree-chain  $P$   $a$   $bs$   $B'$ )  $0$   $B$ )  $\leq$ 
    card (outgoing-edges-range  $U$   $0$   $B$ )
  by (rule card-mono[OF finite-outgoing-edges-range range-subset])
have mono: nondecreasing-from  $a$   $bs$ 
  by (rule direct-insert-costed-partition-loop-state-mono[OF run])
have beta-bounds: bounds-le  $B$   $\beta$ 
  by (rule direct-insert-costed-partition-loop-state-beta-bounds[OF run])
have cost:
   $c \leq$ 
     $A * L * \text{card (range-tree-chain } P \text{ } a \text{ } bs \text{ } B') +$ 
     $(R + q * h) * \text{card (outgoing-edges (range-tree-chain } P \text{ } a \text{ } bs \text{ } B')) +$ 
     $t * \text{sum-list}$ 
     $(\text{map card (range-tree-child-zero-edge-range-list } P \text{ } a \text{ } \beta \text{ } bs)) +$ 
     $t * \text{sum-list}$ 
     $(\text{map card (range-tree-child-direct-edge-range-list } P \text{ } B \text{ } a \text{ } \beta \text{ } bs)) +$ 
     $h * \text{sum-list}$ 
     $(\text{map card (range-tree-child-edge-range-list } P \text{ } a \text{ } \beta \text{ } bs)) +$ 
     $(\text{Suc } h) * \text{card (range-tree-chain } P \text{ } a \text{ } bs \text{ } B')$ 
  by (rule direct-insert-costed-partition-loop-state-cost-from-child-source-and-
amortized-bounds
    [OF run sound child-cost-bounds source-progress])
have lower-sum:
  sum-list (map card (range-tree-child-edge-range-list  $P$   $a$   $\beta$   $bs$ ))
     $\leq$  card (outgoing-edges (range-tree-chain  $P$   $a$   $bs$   $B'$ ))
  by (rule sum-card-range-tree-child-edge-range-list-le-outgoing-edges-chain
    [OF mono])
have direct-sum:
  sum-list
    (map card (range-tree-child-zero-edge-range-list  $P$   $a$   $\beta$   $bs$ )) +
  sum-list
    (map card (range-tree-child-direct-edge-range-list  $P$   $B$   $a$   $\beta$   $bs$ ))

```

```

    ≤ card (outgoing-edges-range (range-tree-chain P a bs B') 0 B)
  by (rule sum-card-range-tree-child-zero-direct-edge-ranges-le-outgoing-edges-
range-chain
    [OF mono beta-bounds])
  have vertex-part:
    A * L * card (range-tree-chain P a bs B') +
    Suc h * card (range-tree-chain P a bs B') ≤
    A * Suc L * card U
  proof -
    have source-term:
      Suc h * card (range-tree-chain P a bs B') ≤
      A * card U
    proof -
      have Suc h * card (range-tree-chain P a bs B') ≤
        A * card (range-tree-chain P a bs B')
      by (rule mult-right-mono[OF source-factor]) simp
      also have ... ≤ A * card U
      using card-chain-le by simp
      finally show ?thesis .
    qed
    have child-term:
      A * L * card (range-tree-chain P a bs B') ≤
      A * L * card U
    using card-chain-le by simp
    have A * L * card U + A * card U = A * Suc L * card U
    by (simp add: algebra-simps)
    then show ?thesis
    using child-term source-term by linarith
  qed
  have edge-part:
    (R + q * h) * card (outgoing-edges (range-tree-chain P a bs B') +
    h * sum-list
      (map card (range-tree-child-edge-range-list P a betas bs)))
    ≤ (R + Suc q * h) * card (outgoing-edges U)
  proof -
    have lower-term:
      h * sum-list
        (map card (range-tree-child-edge-range-list P a betas bs))
      ≤ h * card (outgoing-edges (range-tree-chain P a bs B'))
    using lower-sum by simp
    have chain-term:
      (R + q * h) * card (outgoing-edges (range-tree-chain P a bs B')) +
      h * card (outgoing-edges (range-tree-chain P a bs B')) =
      (R + Suc q * h) *
      card (outgoing-edges (range-tree-chain P a bs B'))
    by (simp add: algebra-simps)
    have to-chain:
      (R + q * h) * card (outgoing-edges (range-tree-chain P a bs B')) +
      h * sum-list

```

```

      (map card (range-tree-child-edge-range-list P a betas bs))
    ≤ (R + Suc q * h) *
      card (outgoing-edges (range-tree-chain P a bs B'))
    using lower-term chain-term by linarith
  have to-U:
    (R + Suc q * h) *
      card (outgoing-edges (range-tree-chain P a bs B')) ≤
    (R + Suc q * h) * card (outgoing-edges U)
    using card-out-le by simp
  show ?thesis
    using to-chain to-U by linarith
qed
have direct-part:
  t * sum-list
    (map card (range-tree-child-zero-edge-range-list P a betas bs)) +
  t * sum-list
    (map card (range-tree-child-direct-edge-range-list P B a betas bs))
  ≤ t * card (outgoing-edges-range U 0 B)
proof -
  have t * (sum-list
    (map card (range-tree-child-zero-edge-range-list P a betas bs)) +
    sum-list
    (map card (range-tree-child-direct-edge-range-list P B a betas bs)))
  ≤ t * card (outgoing-edges-range (range-tree-chain P a bs B') 0 B)
    using direct-sum by simp
  also have ... ≤ t * card (outgoing-edges-range U 0 B)
    using card-range-le by simp
  finally show ?thesis
    by (simp add: algebra-simps)
qed
have c ≤
  A * Suc L * card U +
  (R + Suc q * h) * card (outgoing-edges U) +
  t * card (outgoing-edges-range U 0 B)
  using cost vertex-part edge-part direct-part by linarith
then show ?thesis
  unfolding bmssp-amortized-cost-bound-def .
qed

```

The final level-bound theorem is the non-amortised companion used by the abstract headline recurrence. It closes the same loop skeleton, but leaves the direct and lower edge-range sums explicit rather than folding them into *bmssp-amortized-cost-bound*. This gives later theories the freedom to plug in either an abstract partition budget or the bucketed amortised budget without changing the semantic development.

theorem *direct-insert-costed-partition-loop-state-closes-level-bound-from-child-costs-and-edge-ranges:*

assumes *run:*

direct-insert-costed-partition-loop-state Δ *M-of* *t h k cap l d P B d' D a*

betas bs B' Us U c child-costs
and sound: sound-label d
and pre: bmssp-pre-full d P B
and P-reaches: $\bigwedge x. x \in P \implies \text{reachable } s \ x$
and child-cost-bounds:
 list-all2 ($\lambda c\text{-child } U\text{-child}.$
 $c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$)
 child-costs (range-tree-child-list P a bs)
and source-factor: $\text{Suc } h \leq A$
and source-progress:
 $\bigwedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}.$
 $\llbracket \text{direct-insert-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$
 bmssp-pre-full d S-child B-child;
 $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\text{card } S\text{-child} \leq M\text{-of } l;$
 $\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child} \rrbracket$
 $\implies \text{card } S\text{-child} \leq \text{card } U\text{-child}$
shows $c \leq$
 $A * \text{Suc } L * \text{card } U + R * \text{card } (\text{outgoing-edges } U) +$
 $t * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-direct-edge-range-list } P \ B \ a \ \text{betas } bs)) +$
 $h * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$
proof –
have trace: concrete-partition-loop-trace P B a bs d' B' Us U
by (rule direct-insert-costed-partition-loop-state-trace
 [OF run sound pre P-reaches])
have children:
 list-all2 ($\lambda U \ X. U = X \wedge \text{complete-on } d' \ U$) Us
 (range-tree-chain-list P a bs B')
using trace **unfolding** concrete-partition-loop-trace-def **by** blast
have U-def: $U = \text{bound-tree } P \ (\text{Fin } a) \cup \bigcup (\text{set } Us)$
using trace **unfolding** concrete-partition-loop-trace-def **by** blast
have Union-eq:
 $\bigcup (\text{set } Us) = \bigcup (\text{set } (\text{range-tree-chain-list } P \ a \ \text{bs } B'))$
using children **by** (induction rule: list-all2-induct) auto
have U-eq-chain:
 $U = \text{bound-tree } P \ (\text{Fin } a) \cup \text{range-tree-chain } P \ a \ \text{bs } B'$
using U-def Union-eq Union-range-tree-chain-list[OF P a bs B'] **by** simp
have finite-U: finite U
using U-eq-chain **by** simp
have chain-subset: range-tree-chain P a bs B' $\subseteq U$
using U-eq-chain **by** blast
have card-chain-le: $\text{card } (\text{range-tree-chain } P \ a \ \text{bs } B') \leq \text{card } U$
by (rule card-mono[OF finite-U chain-subset])
have outgoing-subset:
 outgoing-edges (range-tree-chain P a bs B') $\subseteq \text{outgoing-edges } U$
by (rule outgoing-edges-mono[OF chain-subset])

```

have finite-out-U: finite (outgoing-edges U)
by simp
have card-out-le:
  card (outgoing-edges (range-tree-chain P a bs B')) ≤
    card (outgoing-edges U)
by (rule card-mono[OF finite-out-U outgoing-subset])
have loop:
  c ≤
    A * L * card (range-tree-chain P a bs B') +
    R * card (outgoing-edges (range-tree-chain P a bs B')) +
    t * sum-list
      (map card (range-tree-child-direct-edge-range-list P B a betas bs)) +
    h * sum-list
      (map card (range-tree-child-edge-range-list P a betas bs)) +
    Suc h * card (range-tree-chain P a bs B')
by (rule direct-insert-costed-partition-loop-state-cost-from-child-source-and-edge-
range-bounds
  [OF run sound child-cost-bounds source-progress])
have c ≤
  A * L * card U + R * card (outgoing-edges U) +
  t * sum-list
    (map card (range-tree-child-direct-edge-range-list P B a betas bs)) +
  h * sum-list
    (map card (range-tree-child-edge-range-list P a betas bs)) +
  A * card U
proof –
have vterm:
  A * L * card (range-tree-chain P a bs B') ≤ A * L * card U
using card-chain-le by simp
have eterm:
  R * card (outgoing-edges (range-tree-chain P a bs B')) ≤
    R * card (outgoing-edges U)
using card-out-le by simp
have source-term:
  Suc h * card (range-tree-chain P a bs B') ≤ A * card U
proof –
have Suc h * card (range-tree-chain P a bs B') ≤
  A * card (range-tree-chain P a bs B')
by (rule mult-right-mono[OF source-factor]) simp
also have ... ≤ A * card U
using card-chain-le by simp
finally show ?thesis .
qed
show ?thesis
using loop vterm eterm source-term by linarith
qed
also have ... =
  A * Suc L * card U + R * card (outgoing-edges U) +
  t * sum-list

```



```

      (map card (range-tree-child-direct-edge-range-list P B a betas bs)) +
      h * sum-list
      (map card (range-tree-child-edge-range-list P a betas bs))
    by (simp add: algebra-simps)
  finally show ?thesis .
qed

end

end
theory BMSSP-Strict-Tie-Breaking
  imports BMSSP-Direct-Insert-Costed
begin

```

33 Strict Tie-Breaking Consequences

Lexicographic tie-breaking removes distance ties by comparing extended path values. The existing development abstracts uniqueness of shortest paths, but source progress for the base case also needs strict growth along the selected shortest-path tree. This file records exactly that consequence, so the remaining complexity proof can be connected either to a formal extended-distance construction or to any graph model that already provides the same strict tree order.

The cost recurrence uses progress statements of the form "a source set is no larger than the output range that completed it". For non-base recursive calls this comes from the BMSSP threshold: either the call succeeds with the old bound, or it completes enough vertices. The base case is subtler. It takes the first $k + 1$ vertices in distance order from a singleton source. To know that the singleton source itself is included, the proof needs the source to be the strict first element of that order.

The strict tie-breaking locale supplies this missing monotonicity. Along the chosen shortest-path tree, descendants have strictly larger distance than ancestors. Consequently, in the singleton bound tree rooted at x , the root x is the unique minimum-distance vertex. This is the formal counterpart of the paper's assumption that ties can be perturbed away without changing the combinatorial structure of the algorithm.

The later half of the theory threads this progress fact through the exact range-costed and direct-insert costing relations. It closes the source-progress side condition required by the local-budget theorems, then packages the resulting level and amortised graph bounds for the top-level run.

```

context finite-weighted-digraph
begin

```

```

lemma walk-weight-pos-if-positive-edges:
  assumes positive:  $\bigwedge u v. (u, v) \in E \implies 0 < w \ u \ v$ 

```

```

    and walk-p: walk p
    and len: 1 < length p
    shows 0 < walk-weight p
  proof (cases p)
    case Nil
    then show ?thesis
    using len by simp
  next
    case (Cons x xs)
    then show ?thesis
    proof (cases xs)
      case Nil
      then show ?thesis
      using Cons len by simp
    next
      case (Cons y ys)
      have p-def: p = x # y # ys
      using ⟨p = x # xs⟩ Cons by simp
      have edge: (x, y) ∈ E
      using walk-p p-def by simp
      have tail-walk: walk (y # ys)
      using walk-p p-def by simp
      have 0 < w x y
      by (rule positive[OF edge])
      moreover have 0 ≤ walk-weight (y # ys)
      by (rule walk-weight-nonneg[OF tail-walk])
      ultimately show ?thesis
      using p-def by simp
    qed
  qed
end

context unique-shortest-digraph
begin

lemma tree-path-strict-dist-if-positive-edges:
  assumes positive:  $\bigwedge u v. (u, v) \in E \implies 0 < w\ u\ v$ 
  and tree: tree-path u v
  and neq: u ≠ v
  shows dist s u < dist s v
proof -
  let ?p = shortest-path-to v
  have reach-v: reachable s v and u-path: u ∈ set ?p
  using tree by (auto dest: tree-pathD)
  obtain i where i: i < length ?p ! i = u
  using u-path by (auto simp: in-set-conv-nth)
  have sp: shortest-walk s ?p v
  by (rule shortest-path-to-shortest[OF reach-v])

```

```

have simple-p: simple-walk-betw s ?p v
  using sp unfolding shortest-walk-def by blast
have weight-p: walk-weight ?p = dist s v
  using sp unfolding shortest-walk-def by blast
have p-ne: ?p ≠ [] and last-p: last ?p = v
  using sp unfolding shortest-walk-def simple-walk-betw-def walk-betw-def
  by blast+
have suc-i: Suc i < length ?p
proof (rule ccontr)
  assume ¬ Suc i < length ?p
  then have last-i: i = length ?p - 1
    using i(1) by simp
  have ?p ! i = last ?p
    using p-ne last-i by (simp add: last-conv-nth)
  then show False
    using i(2) last-p neq by simp
qed
let ?pref = take (Suc i) ?p
let ?suf = drop i ?p
have pref-short: shortest-walk s ?pref u
  using shortest-walk-prefix-shortest[OF sp i(1)] i(2) by simp
have pref-weight: walk-weight ?pref = dist s u
  using pref-short unfolding shortest-walk-def by blast
have pref-betw: walk-betw s ?pref u
  using pref-short unfolding shortest-walk-def simple-walk-betw-def by blast
have suf-betw0: walk-betw (?p ! i) ?suf v
  by (rule walk-suffix-from-index[OF simple-p i(1)])
have suf-betw: walk-betw u ?suf v
  using suf-betw0 i(2) by simp
have suf-walk: walk ?suf
  using suf-betw unfolding walk-betw-def by blast
have suf-len: 1 < length ?suf
  using suc-i by simp
have suf-pos: 0 < walk-weight ?suf
  by (rule walk-weight-pos-if-positive-edges[OF positive suf-walk suf-len])
have split-p: ?pref @ tl ?suf = ?p
  using take-Suc-append-tl-drop[OF i(1)] .
have walk-weight ?p = walk-weight ?pref + walk-weight ?suf
  using walk-weight-append-tl-betw[OF pref-betw suf-betw] split-p by simp
then show ?thesis
  using weight-p pref-weight suf-pos by linarith
qed

end

locale strict-tie-breaking-digraph = unique-shortest-digraph +
  assumes tree-path-strict-dist:
    [|tree-path u v; u ≠ v|] ⟹ dist s u < dist s v

```

locale *positive-unique-shortest-digraph* = *unique-shortest-digraph* +
assumes *positive-weight*: $(u, v) \in E \implies 0 < w\ u\ v$
begin

sublocale *strict-tie-breaking-digraph*

proof

show $\bigwedge u\ v. \llbracket \text{tree-path } u\ v; u \neq v \rrbracket \implies \text{dist } s\ u < \text{dist } s\ v$
by (*rule tree-path-strict-dist-if-positive-edges*[*OF positive-weight*])
qed

end

The first locale bridge shows one concrete way to obtain strict tree growth: positive edge weights imply every non-empty suffix of a shortest path has positive weight, and therefore every proper descendant in the selected shortest-path tree has strictly larger distance. The abstract *strict-tie-breaking-digraph* locale records only the consequence needed by the runtime proof, avoiding a commitment to any particular perturbation or tie-breaking construction.

context *strict-tie-breaking-digraph*
begin

lemma *bound-tree-singleton-tree-path*:

assumes $y \in \text{bound-tree } \{x\}\ B$
shows *tree-path* $x\ y$

proof –

have *reach-y*: *reachable* $s\ y$ **and** *through-y*: *through* $\{x\}\ y$
using *assms* **unfolding** *bound-tree-def* **by** *auto*
then show *?thesis*
using *through-iff-tree-path*[*OF reach-y*, *of* $\{x\}$] **by** *auto*
qed

lemma *bound-tree-singleton-root-strict-min*:

assumes $x\text{-bound}$: $x \in \text{bound-tree } \{x\}\ B$
and $y\text{-bound}$: $y \in \text{bound-tree } \{x\}\ B$
and neq : $y \neq x$
shows $\text{dist } s\ x < \text{dist } s\ y$

proof –

have *tree-path* $x\ y$
by (*rule bound-tree-singleton-tree-path*[*OF y-bound*])
then show *?thesis*
using *tree-path-strict-dist neq* **by** *blast*
qed

lemma *base-case-order-hd-root*:

assumes $x\text{-bound}$: $x \in \text{bound-tree } \{x\}\ B$
shows *base-case-order* $x\ B \neq [] \wedge \text{hd } (\text{base-case-order } x\ B) = x$

proof –

let $?xs = \text{base-case-order } x\ B$
have $\text{set } ?xs = \text{bound-tree } \{x\}\ B$

```

    by (rule base-case-order-set)
  have x-set:  $x \in \text{set } ?xs$ 
    using x-bound set-xs by simp
  then have xs-ne:  $?xs \neq []$ 
    by auto
  have hd-in:  $\text{hd } ?xs \in \text{bound-tree } \{x\} B$ 
    using xs-ne set-xs by (metis hd-in-set)
  have sorted: sorted-wrt  $(\lambda u v. \text{dist } s u \leq \text{dist } s v)$  ?xs
    by (rule base-case-order-sorted)
  have hd-eq:  $\text{hd } ?xs = x$ 
  proof (rule ccontr)
    assume hd-neq:  $\text{hd } ?xs \neq x$ 
    obtain i where i:  $i < \text{length } ?xs$   $?xs ! i = x$ 
      using x-set by (auto simp: in-set-conv-nth)
    have i-pos:  $0 < i$ 
    proof (rule ccontr)
      assume  $\neg 0 < i$ 
      then have  $i = 0$ 
        by simp
      then show False
        using i xs-ne hd-neq by (simp add: hd-conv-nth)
    qed
    have hd-nth:  $?xs ! 0 = \text{hd } ?xs$ 
      using xs-ne by (simp add: hd-conv-nth)
    have dist s (hd ?xs)  $\leq \text{dist } s x$ 
      using sorted-wrt-nth-less[OF sorted i-pos i(1)] i(2) hd-nth by simp
    moreover have  $\text{dist } s x < \text{dist } s (\text{hd } ?xs)$ 
      by (rule bound-tree-singleton-root-strict-min[OF x-bound hd-in hd-neq])
    ultimately show False
      by simp
  qed
  show ?thesis
    using xs-ne hd-eq by blast
qed

lemma source-in-base-case-vertices:
  assumes x-bound:  $x \in \text{bound-tree } \{x\} B$ 
    and k-pos:  $0 < k$ 
  shows  $x \in \text{base-case-vertices } k x B$ 
  proof (cases length (base-case-order x B)  $\leq k$ )
    case True
    then show ?thesis
      using x-bound base-case-success[of x B k] by simp
  next
    case False
    let ?xs = base-case-order x B
    have len:  $k < \text{length } ?xs$ 
      using False by simp
    have hd-root:  $\text{hd } ?xs = x$  and xs-ne:  $?xs \neq []$ 

```

```

    using base-case-order-hd-root[OF x-bound] by blast+
  have x-take:  $x \in \text{set } (\text{take } (\text{Suc } k) \text{ } ?xs)$ 
proof -
  have ?xs ! 0 = x
    using hd-root xs-ne by (simp add: hd-conv-nth)
  moreover have 0 < length (take (Suc k) ?xs)
    using len by simp
  ultimately show ?thesis
    using nth-mem[of 0 take (Suc k) ?xs] by simp
qed
have kth-bound:  $?xs ! k \in \text{bound-tree } \{x\} B$ 
  using len base-case-order-set[of x B] nth-mem by metis
have kth-neq:  $?xs ! k \neq x$ 
proof
  assume kth-eq:  $?xs ! k = x$ 
  have distinct: distinct ?xs
    by (rule base-case-order-distinct)
  have ?xs ! 0 = ?xs ! k
    using kth-eq hd-root xs-ne by (simp add: hd-conv-nth)
  moreover have 0 < length ?xs k < length ?xs 0  $\neq k$ 
    using xs-ne len k-pos by auto
  ultimately show False
    using distinct by (meson nth-eq-iff-index-eq)
qed
have dist-lt:  $\text{dist } s \ x < \text{dist } s \ (?xs ! k)$ 
  by (rule bound-tree-singleton-root-strict-min[OF x-bound kth-bound kth-neq])
have base-case-vertices k x B =
   $\{v \in \text{set } (\text{take } (\text{Suc } k) \text{ } ?xs). \text{dist } s \ v < \text{dist } s \ (?xs ! k)\}$ 
  using len unfolding base-case-vertices-def by (simp add: Let-def)
then show ?thesis
  using x-take dist-lt by simp
qed

```

The base-case lemmas turn strict tree growth into the concrete progress fact used by the recurrence. In a singleton problem, every vertex in *bound-tree* $\{x\} B$ lies below x in the selected shortest-path tree. Strictness makes x the first element of *base-case-order* $x B$. Hence the bounded base scan always includes its source whenever $k > 0$.

This small fact is what lets the base case participate in the same source accounting as the recursive cases: the source set has cardinality one and is contained in the returned base-case vertex set.

lemma *tree-path-tight-successor*:

```

  assumes tight: tight-edge-step u v
  shows tree-path u v
proof (cases u = v)
  case True
  have reach-u: reachable s u
    using tight unfolding tight-edge-step-def by blast
  have sp-u: shortest-walk s (shortest-path-to u) u

```

```

    by (rule shortest-path-to-shortest[OF reach-u])
  have u ∈ set (shortest-path-to u)
proof -
  have shortest-path-to u ≠ [] last (shortest-path-to u) = u
    using shortest-walk-hd[OF sp-u] by blast+
  then show ?thesis
    by (metis last-in-set)
qed
then show ?thesis
  using True reach-u unfolding tree-path-def by blast
next
case False
have edge: (u, v) ∈ E
  using tight unfolding tight-edge-step-def by blast
have reach-u: reachable s u
  using tight unfolding tight-edge-step-def by blast
have dist-v: dist s v = dist s u + w u v
  using tight unfolding tight-edge-step-def by blast
let ?p = shortest-path-to u
have sp-u: shortest-walk s ?p u
  by (rule shortest-path-to-shortest[OF reach-u])
have simple-p: simple-walk-betw s ?p u
  using sp-u unfolding shortest-walk-def by blast
have weight-p: walk-weight ?p = dist s u
  using sp-u unfolding shortest-walk-def by blast
have p-ne: ?p ≠ [] and last-p: last ?p = u
  using shortest-walk-hd[OF sp-u] by blast+
have v-notin: v ∉ set ?p
proof
  assume v-in: v ∈ set ?p
  have tree-vu: tree-path v u
    by (rule tree-pathI[OF reach-u v-in])
  have vu-neq: v ≠ u
    using False by simp
  have dist s v < dist s u
    by (rule tree-path-strict-dist[OF tree-vu vu-neq])
  moreover have dist s u ≤ dist s v
    using dist-v nonneg-weight[OF edge] by linarith
  ultimately show False
    by simp
qed
have simple-q: simple-walk-betw s (?p @ [v]) v
  by (rule simple-walk-snoc[OF simple-p edge v-notin])
have weight-q: walk-weight (?p @ [v]) = dist s v
  using weight-p p-ne last-p dist-v
  by (simp add: walk-weight-snoc)
have sp-q: shortest-walk s (?p @ [v]) v
  using simple-q weight-q unfolding shortest-walk-def by blast
have reach-v: reachable s v

```

```

    by (rule dist-triangle-edge(1)[OF edge reach-u])
  have sp-v: shortest-walk s (shortest-path-to v) v
    by (rule shortest-path-to-shortest[OF reach-v])
  have path-eq: shortest-path-to v = ?p @ [v]
    using unique-shortest-walk[OF sp-v sp-q] .
  have u ∈ set ?p
    using p-ne last-p by (metis last-in-set)
  then have u ∈ set (?p @ [v])
    by simp
  then have u ∈ set (shortest-path-to v)
    using path-eq by simp
  then show ?thesis
    by (rule tree-pathI[OF reach-v])
qed

```

lemma fp-next-bound-tree:

```

  assumes sound: sound-label d
    and pre: bmssp-pre-full d S B
    and F-bound: F ⊆ bound-tree S B
    and v-next: v ∈ fp-next d F B
  shows v ∈ bound-tree S B

```

proof –

```

  obtain u where uF: u ∈ F
    and edge: (u, v) ∈ E
    and relax: d u + w u v ≤ d v
    and below-cand: below-bound (d u + w u v) B
    using v-next unfolding fp-next-def by blast
  have u-bound: u ∈ bound-tree S B
    using F-bound uF by blast
  have reach-u: reachable s u
    using u-bound unfolding bound-tree-def by blast
  have reach-v: reachable s v
    by (rule dist-triangle-edge(1)[OF edge reach-u])
  have dist-triangle: dist s v ≤ dist s u + w u v
    by (rule dist-triangle-edge(2)[OF edge reach-u])
  have vV: v ∈ V
    using edge-in-V[OF edge] by simp
  have uV: u ∈ V
    using u-bound unfolding bound-tree-def by blast
  have dist-u-le: dist s u ≤ d u
    using sound uV reach-u unfolding sound-label-def by blast
  have dist-v-le-cand: dist s v ≤ d u + w u v
    using dist-triangle dist-u-le by linarith
  have below-v: below-bound (dist s v) B
    by (rule below-bound-le-trans[OF dist-v-le-cand below-cand])
  have through-v: through S v
  proof (cases d v = dist s v)
    case True
      have cand-le-dist: d u + w u v ≤ dist s v

```



```

    using relax True by simp
  have dist-v-eq:  $\text{dist } s \ v = \text{dist } s \ u + w \ u \ v$ 
    using dist-triangle dist-u-le cand-le-dist by linarith
  have tight-uv: tight-edge-step  $u \ v$ 
    using edge reach-u dist-v-eq unfolding tight-edge-step-def by blast
  have tree-uv: tree-path  $u \ v$ 
    by (rule tree-path-tight-successor[OF tight-uv])
  have through-u: through  $S \ u$ 
    using u-bound unfolding bound-tree-def by blast
  obtain  $x$  where  $xS$ :  $x \in S$  and tree-xu: tree-path  $x \ u$ 
    using through-iff-tree-path[OF reach-u, of  $S$ ] through-u by blast
  have tree-xv: tree-path  $x \ v$ 
    by (rule tree-path-trans[OF tree-uv tree-xu])
  show ?thesis
    using through-iff-tree-path[OF reach-v, of  $S$ ]  $xS$  tree-xv by blast
next
case False
have through-complete  $d \ S \ v$ 
  using pre  $vV$  reach-v below-v False unfolding bmssp-pre-full-def by blast
then show ?thesis
  unfolding through-complete-def through-def by blast
qed
show ?thesis
  using  $vV$  reach-v through-v below-v unfolding bound-tree-def by blast
qed

```

```

lemma fp-iter-capped-seen-subset-bound-tree:
  assumes sound: sound-label  $d$ 
    and pre: bmssp-pre-full  $d \ S \ B$ 
    and F-bound:  $F \subseteq \text{bound-tree } S \ B$ 
    and W-bound:  $W \subseteq \text{bound-tree } S \ B$ 
  shows fp-seen ( $\text{fp-iter-capped } n \ \text{cap } d \ F \ W \ B$ )  $\subseteq \text{bound-tree } S \ B$ 
  using assms
proof (induction  $n$  arbitrary:  $d \ F \ W$ )
case 0
then show ?case
  unfolding fp-seen-def by simp
next
case (Suc  $n$ )
let ? $d'$  = relax-frontier  $d \ F$ 
let ? $F'$  = fp-next  $d \ F \ B$ 
let ? $W'$  =  $W \cup ?F'$ 
have F-reaches:  $\bigwedge u. u \in F \implies \text{reachable } s \ u$ 
  using Suc.prem1(3) unfolding bound-tree-def by blast
have sound': sound-label ? $d'$ 
  by (rule relax-frontier-sound[OF Suc.prem1(1) F-reaches])
have pre': bmssp-pre-full ? $d' \ S \ B$ 
  by (rule relax-frontier-preserves-bmssp-pre-full
    [OF Suc.prem1(2) F-reaches])

```

```

have F'-bound: ?F'  $\subseteq$  bound-tree S B
proof
  fix v
  assume v  $\in$  ?F'
  then show v  $\in$  bound-tree S B
    by (rule fp-next-bound-tree[OF Suc.premis(1,2,3)])
qed
have W'-bound: ?W'  $\subseteq$  bound-tree S B
  using Suc.premis(4) F'-bound by blast
show ?case
proof (cases card ?W' > cap)
  case True
  then show ?thesis
    using W'-bound by (simp add: fp-seen-def Let-def)
next
  case False
  show ?thesis
    using Suc.IH[OF sound' pre' F'-bound W'-bound] False
    by (simp add: fp-seen-def Let-def)
qed
qed

lemma find-pivots-seen-capped-subset-bound-tree:
  assumes sound: sound-label d
  and pre: bmssp-pre-full d S B
  and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
  and below:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$ 
  shows find-pivots-seen-capped k cap d S B  $\subseteq$  bound-tree S B
proof -
  have S-subset: S  $\subseteq$  V
  using pre unfolding bmssp-pre-full-def by blast
  have below-dist:  $\bigwedge x. x \in S \implies \text{below-bound } (\text{dist } s \ x) \ B$ 
  proof -
    fix x
    assume xS: x  $\in$  S
    have dist s x  $\leq$  d x
    using sound S-subset xS S-reaches[OF xS]
    unfolding sound-label-def by blast
    then show below-bound (dist s x) B
      by (rule below-bound-le-trans[OF - below[OF xS]])
  qed
  have S-bound: S  $\subseteq$  bound-tree S B
  by (rule sources-subset-bound-tree[OF S-reaches below-dist])
  show ?thesis
    unfolding find-pivots-seen-capped-def
    by (rule fp-iter-capped-seen-subset-bound-tree
        [OF sound pre S-bound S-bound])
qed

```

The next block connects FindPivots' "seen" set to completed output

ranges. When a non-base call succeeds with the original bound, the final assembly theorem says that the loop output together with already-complete labelled vertices is exactly the parent bound tree. Since the seen set is contained in that tree, successful calls can charge the number of seen vertices to the returned output.

The same argument is stated for both exact split-range runs and direct-insert runs. The direct-insert version first forgets its extra accounting structure down to the exact range trace, then applies the same assembly reasoning. These lemmas are later paired with threshold lemmas for the unsuccessful branch.

theorem *exact-split-range-costed-step-seen-success:*

assumes *loop:*

exact-split-range-costed-partition-loop-state Δ *M-of t h k cap l*
(find-pivots-label-capped k cap d S B)
(find-pivots-pivots-capped k cap d S B) B d' D a betas bs B'
Us U-loop c-loop child-costs

and *U-def:*

$U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v =$
 $\text{dist } s \ v\}$

and *sound:* *sound-label d*

and *pre:* *bmssp-pre-full d S B*

and *S-reaches:* $\bigwedge x. x \in S \implies \text{reachable } s \ x$

and *below:* $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$

and *success:* $B' = B$

shows *card (find-pivots-seen-capped k cap d S B) $\leq$ card U*

proof –

let *?d-fp* = *find-pivots-label-capped k cap d S B*

let *?P* = *find-pivots-pivots-capped k cap d S B*

have *sound-fp:* *sound-label ?d-fp*

unfolding *find-pivots-label-capped-def*

by (*rule fp-iter-capped-label-sound*[*OF sound S-reaches*])

have *pivot-pre:* *bmssp-pre-full ?d-fp ?P B*

using *find-pivots-capped-establishes-pivot-pre-concrete*
 $[OF \text{ sound pre } S\text{-reaches}]$.

have *P-subset-S:* $?P \subseteq S$

unfolding *find-pivots-pivots-capped-def* **by** (*auto split: if-splits*)

have *P-reaches:* $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$

using *P-subset-S S-reaches* **by** *blast*

have *split-loop:*

split-range-costed-partition-loop-state Δ *M-of t h k cap l*

?d-fp ?P B d' D a betas bs B' Us U-loop c-loop child-costs

by (*rule exact-split-range-costed-refines-split-range-costed*(1)[*OF loop*])

have *trace:*

concrete-partition-loop-trace ?P B a bs d' B' Us U-loop

by (*rule split-range-costed-partition-loop-state-trace*

$[OF \text{ split-loop sound-fp pivot-pre } P\text{-reaches}]$)

have *loop-post:* *bmssp-post-full ?d-fp ?P B d' B' U-loop*

```

by (rule concrete-partition-loop-trace-post[OF trace])
have assembly:
  U-loop  $\cup$ 
   $\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v =$ 
     $\text{dist } s \ v\} =$ 
   $\text{bound-tree } S \ B'$ 
by (rule concrete-capped-find-pivots-final-tree-assembly
  [OF sound pre S-reaches loop-post])
have U-tree:  $U = \text{bound-tree } S \ B$ 
using U-def assembly success by simp
have seen-subset:
  find-pivots-seen-capped  $k \text{ cap } d \ S \ B \subseteq U$ 
using find-pivots-seen-capped-subset-bound-tree
  [OF sound pre S-reaches below] U-tree by simp
have finite-U: finite U
unfolding U-tree bound-tree-def using finite-V by auto
show ?thesis
by (rule card-mono[OF finite-U seen-subset])
qed

theorem direct-insert-costed-step-seen-success:
assumes loop:
  direct-insert-costed-partition-loop-state  $\Delta \ M\text{-of } t \ h \ k \text{ cap } l$ 
  (find-pivots-label-capped  $k \text{ cap } d \ S \ B$ )
  (find-pivots-pivots-capped  $k \text{ cap } d \ S \ B$ )  $B \ d' \ D \ a \ \text{betas } bs \ B'$ 
   $Us \ U\text{-loop } c\text{-loop } \text{child-costs}$ 
and U-def:
   $U = U\text{-loop} \cup$ 
   $\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v =$ 
     $\text{dist } s \ v\}$ 
and sound: sound-label d
and pre: bmssp-pre-full d S B
and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
and below:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$ 
and success:  $B' = B$ 
shows  $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \ S \ B) \leq \text{card } U$ 
proof –
let ?d-fp = find-pivots-label-capped  $k \text{ cap } d \ S \ B$ 
let ?P = find-pivots-pivots-capped  $k \text{ cap } d \ S \ B$ 
have sound-fp: sound-label ?d-fp
unfolding find-pivots-label-capped-def
by (rule fp-iter-capped-label-sound[OF sound S-reaches])
have pivot-pre: bmssp-pre-full ?d-fp ?P B
using find-pivots-capped-establishes-pivot-pre-concrete
  [OF sound pre S-reaches] .
have P-subset-S:  $?P \subseteq S$ 
unfolding find-pivots-pivots-capped-def by (auto split: if-splits)
have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
using P-subset-S S-reaches by blast

```

have *trace*:
 concrete-partition-loop-trace ?*P* *B* *a* *bs* *d'* *B'* *Us* *U-loop*
 by (*rule direct-insert-costed-partition-loop-state-trace*
 [*OF loop sound-fp pivot-pre P-reaches*])
have *loop-post*: *bmssp-post-full* ?*d-fp* ?*P* *B* *d'* *B'* *U-loop*
 by (*rule concrete-partition-loop-trace-post*[*OF trace*])
have *assembly*:
 U-loop $\cup$
 $\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v =$
 $\text{dist } s \ v\} =$
 $\text{bound-tree } S \ B'$
 by (*rule concrete-capped-find-pivots-final-tree-assembly*
 [*OF sound pre S-reaches loop-post*])
have *U-tree*: *U* = *bound-tree* *S* *B*
 using *U-def assembly success* **by** *simp*
have *seen-subset*:
 find-pivots-seen-capped *k* *cap* *d* *S* *B* $\subseteq$ *U*
 using *find-pivots-seen-capped-subset-bound-tree*
 [*OF sound pre S-reaches below*] *U-tree* **by** *simp*
have *finite-U*: *finite* *U*
 unfolding *U-tree bound-tree-def* **using** *finite-V* **by** *auto*
show ?*thesis*
 by (*rule card-mono*[*OF finite-U seen-subset*])
qed

theorem *direct-insert-costed-step-seen-success-subset*:

assumes *loop*:
 direct-insert-costed-partition-loop-state Δ *M-of t h k cap l*
 (*find-pivots-label-capped* *k* *cap* *d* *S* *B*)
 (*find-pivots-pivots-capped* *k* *cap* *d* *S* *B*) *B* *d'* *D* *a* *betas* *bs* *B'*
 Us *U-loop* *c-loop* *child-costs*
and *U-def*:
 U = *U-loop* $\cup$
 $\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v =$
 $\text{dist } s \ v\}$
and *sound*: *sound-label* *d*
and *pre*: *bmssp-pre-full* *d* *S* *B*
and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
and *below*: $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$
and *success*: *B'* = *B*
shows *find-pivots-seen-capped* *k* *cap* *d* *S* *B* $\subseteq$ *U*
proof –
let ?*d-fp* = *find-pivots-label-capped* *k* *cap* *d* *S* *B*
let ?*P* = *find-pivots-pivots-capped* *k* *cap* *d* *S* *B*
have *sound-fp*: *sound-label* ?*d-fp*
 unfolding *find-pivots-label-capped-def*
 by (*rule fp-iter-capped-label-sound*[*OF sound S-reaches*])
have *pivot-pre*: *bmssp-pre-full* ?*d-fp* ?*P* *B*
 using *find-pivots-capped-establishes-pivot-pre-concrete*

$[OF \text{ sound pre } S\text{-reaches}]$.
have $P\text{-subset-}S$: $?P \subseteq S$
unfolding $find\text{-pivots-pivots-capped-def}$ **by** (*auto split: if-splits*)
have $P\text{-reaches}$: $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$
using $P\text{-subset-}S$ $S\text{-reaches}$ **by** *blast*
have $trace$:
 $concrete\text{-partition-loop-trace } ?P \ B \ a \ bs \ d' \ B' \ Us \ U\text{-loop}$
by (*rule direct-insert-costed-partition-loop-state-trace*
 $[OF \text{ loop sound-fp pivot-pre } P\text{-reaches}]$)
have $loop\text{-post}$: $bmssp\text{-post-full } ?d\text{-fp } ?P \ B \ d' \ B' \ U\text{-loop}$
by (*rule concrete-partition-loop-trace-post* $[OF \text{ trace}]$)
have $assembly$:
 $U\text{-loop} \cup$
 $\{v \in bound\text{-tree } S \ B'. \ find\text{-pivots-label-capped } k \ cap \ d \ S \ B \ v =$
 $\text{dist } s \ v\} =$
 $bound\text{-tree } S \ B'$
by (*rule concrete-capped-find-pivots-final-tree-assembly*
 $[OF \text{ sound pre } S\text{-reaches } loop\text{-post}]$)
have $U\text{-tree}$: $U = bound\text{-tree } S \ B$
using $U\text{-def}$ $assembly \text{ success}$ **by** *simp*
show $?thesis$
using $find\text{-pivots-seen-capped-subset-bound-tree}$
 $[OF \text{ sound pre } S\text{-reaches below}] \ U\text{-tree}$ **by** *simp*
qed

theorem $direct\text{-insert-costed-capped-step-threshold-if-not-success}$:

assumes $loop$:

$direct\text{-insert-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ cap \ l$
 $(find\text{-pivots-label-capped } k \ cap \ d \ S \ B)$
 $(find\text{-pivots-pivots-capped } k \ cap \ d \ S \ B) \ B \ d' \ D \ a \ betas \ bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } child\text{-costs}$

and $U\text{-def}$:

$U = U\text{-loop} \cup$
 $\{v \in bound\text{-tree } S \ B'. \ find\text{-pivots-label-capped } k \ cap \ d \ S \ B \ v =$
 $\text{dist } s \ v\}$

and $sound$: $sound\text{-label } d$

and pre : $bmssp\text{-pre-full } d \ S \ B$

and $S\text{-reaches}$: $\bigwedge x. x \in S \implies \text{reachable } s \ x$

and $not\text{-success}$: $B' \neq B$

shows $k * cap \leq card \ U$

proof –

let $?d\text{-fp} = find\text{-pivots-label-capped } k \ cap \ d \ S \ B$

let $?P = find\text{-pivots-pivots-capped } k \ cap \ d \ S \ B$

have $sound\text{-fp}$: $sound\text{-label } ?d\text{-fp}$

unfolding $find\text{-pivots-label-capped-def}$

by (*rule fp-iter-capped-label-sound* $[OF \text{ sound } S\text{-reaches}]$)

have $pivot\text{-pre}$: $bmssp\text{-pre-full } ?d\text{-fp } ?P \ B$

using $find\text{-pivots-capped-establishes-pivot-pre-concrete}$
 $[OF \text{ sound pre } S\text{-reaches}]$.

have $P\text{-subset-}S$: $?P \subseteq S$
unfolding $\text{find-pivots-pivots-capped-def}$ **by** auto
have $P\text{-reaches}$: $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$
using $P\text{-subset-}S$ $S\text{-reaches}$ **by** blast
have loop-class :
 $B' = B \vee k * \text{cap} \leq \text{card } U\text{-loop}$
by ($\text{rule direct-insert-costed-partition-loop-state-success-or-threshold}$
 $[\text{OF loop sound-fp pivot-pre } P\text{-reaches}]$)
have threshold-loop : $k * \text{cap} \leq \text{card } U\text{-loop}$
using $\text{loop-class not-success}$ **by** blast
have trace : $\text{concrete-partition-loop-trace } ?P \ B \ a \ bs \ d' \ B' \ Us \ U\text{-loop}$
by ($\text{rule direct-insert-costed-partition-loop-state-trace}$
 $[\text{OF loop sound-fp pivot-pre } P\text{-reaches}]$)
have loop-post : $\text{bmssp-post-full } ?d\text{-fp } ?P \ B \ d' \ B' \ U\text{-loop}$
by ($\text{rule concrete-partition-loop-trace-post}[\text{OF trace}]$)
have $\text{finite-}U\text{-loop}$: $\text{finite } U\text{-loop}$
using $\text{loop-post unfolding bmssp-post-full-def}$ **by** simp
have $\text{finite-}U$: $\text{finite } U$
unfolding $U\text{-def}$ **using** $\text{finite-}U\text{-loop}$ **by** simp
have $U\text{-loop-subset}$: $U\text{-loop} \subseteq U$
unfolding $U\text{-def}$ **by** blast
have $\text{card } U\text{-loop} \leq \text{card } U$
by ($\text{rule card-mono}[\text{OF finite-}U \ U\text{-loop-subset}]$)
then show $?thesis$
using threshold-loop **by** linarith
qed

theorem $\text{direct-insert-costed-capped-step-scan-insert-budget-from-scaled-seen-or-threshold}$:

assumes degree : $\text{edge-outdegree-le } \Delta$
and degree-factor : $\Delta \leq A$
and insert-factor : $t \leq A * k$
and $\text{insert-scaled-factor}$: $t \leq A\text{-insert} * k$
and $\text{seen-scaled-factor}$: $k * \Delta + A\text{-insert} \leq 2 * A$
and insert :
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B)$
and loop :
 $\text{direct-insert-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs}$
and $U\text{-def}$:
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v =$
 $\text{dist } s \ v\}$
and sound : $\text{sound-label } d$
and pre : $\text{bmssp-pre-full } d \ S \ B$
and $S\text{-reaches}$: $\bigwedge x. x \in S \implies \text{reachable } s \ x$

```

and S-k-cap:  $k * \text{card } S \leq \text{cap}$ 
and anti: tree-antichain S
and k-pos:  $0 < k$ 
and seen-success:
   $B' = B \implies$ 
     $\text{card } (\text{find-pivots-seen-capped } k \text{ cap } d \text{ } S \text{ } B) \leq \text{card } U$ 
shows fp-iter-capped-scan-cost  $k \text{ cap } d \text{ } S \text{ } B + c\text{-insert} \leq$ 
   $2 * A * \text{card } U$ 
proof (cases  $B' = B$ )
  case True
  have S-subset:  $S \subseteq V$ 
  using pre unfolding bmssp-pre-full-def by blast
  show ?thesis
  by (rule find-pivots-scan-and-initial-insert-budget-from-seen-scaled
    [OF S-subset degree anti S-k-cap insert insert-scaled-factor
      seen-scaled-factor seen-success[OF True]])
next
  case False
  have S-subset:  $S \subseteq V$ 
  using pre unfolding bmssp-pre-full-def by blast
  have S-cap:  $\text{card } S \leq \text{cap}$ 
  proof –
  have  $\text{card } S \leq k * \text{card } S$ 
  using k-pos by (cases k) simp-all
  then show ?thesis
  using S-k-cap by linarith
qed
  have progress:  $k * \text{cap} \leq \text{card } U$ 
  by (rule direct-insert-costed-capped-step-threshold-if-not-success
    [OF loop U-def sound pre S-reaches False])
  show ?thesis
  by (rule find-pivots-scan-and-initial-insert-budget-from-progress
    [OF S-subset degree S-cap insert degree-factor insert-factor progress])
qed

theorem split-range-costed-bmssp-zero-source-card-le-from-label-below:
assumes run:
  split-range-costed-bmssp  $\Delta M\text{-of } t \text{ } h \text{ } k \text{ cap } 0 \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$ 
and sound: sound-label d
and pre: bmssp-pre-full  $d \text{ } S \text{ } B$ 
and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \text{ } x$ 
and below:  $\bigwedge x. x \in S \implies \text{below-bound } (d \text{ } x) \text{ } B$ 
and k-pos:  $0 < k$ 
shows  $\text{card } S \leq \text{card } U$ 
using run
proof cases
case (Split-Range-Cost-Base x)
have S-eq:  $S = \{x\}$ 
using Split-Range-Cost-Base by simp

```



```

have xS:  $x \in S$ 
  using S-eq by simp
have xV:  $x \in V$ 
  using pre xS unfolding bmssp-pre-full-def by blast
have reach-x: reachable s x
  by (rule S-reaches[OF xS])
have dist-le:  $\text{dist } s \ x \leq d \ x$ 
  using sound xV reach-x unfolding sound-label-def by blast
have below-dist: below-bound (dist s x) B
  using below-bound-le-trans[OF dist-le below[OF xS]] .
have x-bound:  $x \in \text{bound-tree } \{x\} \ B$ 
  by (rule source-in-own-bound-tree[OF - reach-x below-dist]) simp
have xU:  $x \in U$ 
  using source-in-base-case-vertices[OF x-bound k-pos]
  using Split-Range-Cost-Base by simp
have singleton-subset:  $\{x\} \subseteq U$ 
  using xU by blast
have finite-U: finite U
  using Split-Range-Cost-Base by simp
have card {x}  $\leq$  card U
  by (rule card-mono[OF finite-U singleton-subset])
then show ?thesis
  using S-eq by simp
qed

theorem split-range-costed-bmssp-source-card-le-from-label-below-all-levels:
  assumes run:
    split-range-costed-bmssp  $\Delta$  M-of t h k cap l d S B d' B' U c
  and sound: sound-label d
  and pre: bmssp-pre-full d S B
  and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
  and below:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$ 
  and S-cap:  $\text{card } S \leq \text{cap}$ 
  and k-pos:  $0 < k$ 
  shows  $\text{card } S \leq \text{card } U$ 
proof (cases l)
case 0
  have run0:
    split-range-costed-bmssp  $\Delta$  M-of t h k cap 0 d S B d' B' U c
  using run 0 by simp
  show ?thesis
    by (rule split-range-costed-bmssp-zero-source-card-le-from-label-below
      [OF run0 sound pre S-reaches below k-pos])
next
case (Suc l')
  have run-suc:
    split-range-costed-bmssp  $\Delta$  M-of t h k cap (Suc l') d S B d' B' U c
  using run Suc by simp
  show ?thesis

```

by (rule split-range-costed-bmssp-Suc-source-card-le-from-label-below
 [OF run-suc sound pre S-reaches below S-cap k-pos])

qed

theorem exact-split-range-costed-bmssp-source-card-le-from-label-below-all-levels:

assumes run:

exact-split-range-costed-bmssp Δ M-of t h k cap l d S B d' B' U c

and sound: sound-label d

and pre: bmssp-pre-full d S B

and S-reaches: $\bigwedge x. x \in S \implies \text{reachable } s \ x$

and below: $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$

and S-cap: $\text{card } S \leq \text{cap}$

and k-pos: $0 < k$

shows $\text{card } S \leq \text{card } U$

by (rule split-range-costed-bmssp-source-card-le-from-label-below-all-levels

[OF exact-split-range-costed-refines-split-range-costed(2)[OF run]

sound pre S-reaches below S-cap k-pos])

theorem direct-insert-costed-bmssp-zero-source-card-le-from-label-below:

assumes run:

direct-insert-costed-bmssp Δ M-of t h k cap 0 d S B d' B' U c

and sound: sound-label d

and pre: bmssp-pre-full d S B

and S-reaches: $\bigwedge x. x \in S \implies \text{reachable } s \ x$

and below: $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$

and k-pos: $0 < k$

shows $\text{card } S \leq \text{card } U$

using run

proof cases

case (Direct-Insert-Costed-Base x)

have S-eq: $S = \{x\}$

using Direct-Insert-Costed-Base by simp

have xS: $x \in S$

using S-eq by simp

have xV: $x \in V$

using pre xS unfolding bmssp-pre-full-def by blast

have reach-x: $\text{reachable } s \ x$

by (rule S-reaches[OF xS])

have dist-le: $\text{dist } s \ x \leq d \ x$

using sound xV reach-x unfolding sound-label-def by blast

have below-dist: $\text{below-bound } (d \ s \ x) \ B$

using below-bound-le-trans[OF dist-le below[OF xS]] .

have x-bound: $x \in \text{bound-tree } \{x\} \ B$

by (rule source-in-own-bound-tree[OF - reach-x below-dist]) simp

have xU: $x \in U$

using source-in-base-case-vertices[OF x-bound k-pos]

using Direct-Insert-Costed-Base by simp

have singleton-subset: $\{x\} \subseteq U$

using xU by blast

```

have finite-U: finite U
using Direct-Insert-Costed-Base by simp
have card {x} ≤ card U
by (rule card-mono[OF finite-U singleton-subset])
then show ?thesis
using S-eq by simp
qed

theorem direct-insert-costed-bmssp-source-card-le-from-label-below-all-levels:
assumes run:
  direct-insert-costed-bmssp Δ M-of t h k cap l d S B d' B' U c
and sound: sound-label d
and pre: bmssp-pre-full d S B
and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
and below:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$ 
and S-cap: card S ≤ cap
and k-pos:  $0 < k$ 
shows card S ≤ card U
proof (cases l)
case 0
have run0:
  direct-insert-costed-bmssp Δ M-of t h k cap 0 d S B d' B' U c
using run 0 by simp
show ?thesis
by (rule direct-insert-costed-bmssp-zero-source-card-le-from-label-below
  [OF run0 sound pre S-reaches below k-pos])
next
case (Suc l')
have run-suc:
  direct-insert-costed-bmssp Δ M-of t h k cap (Suc l') d S B d' B' U c
using run Suc by simp
show ?thesis
by (rule direct-insert-costed-bmssp-Suc-source-card-le-from-label-below
  [OF run-suc sound pre S-reaches below S-cap k-pos])
qed

```

From here the theory closes the recurrence under the strict progress assumptions. The source-progress hypothesis in the direct-insert cost layer is discharged by a case split on the recursion level. At level zero, the strict base-case order gives the source in the returned base set. At positive levels, the success-or-threshold lemmas and the seen-set bound show that a child call's sources are dominated by its output.

The resulting loop-closing theorems are wrappers around the direct-insert cost lemmas from the previous file. Their purpose is to remove the abstract source-progress assumption and replace it by explicit invariants: bounded child source size, tree antichains, and the scaled cardinality condition that the BMSSP schedule maintains.

theorem *direct-insert-costed-partition-loop-state-closes-level-bound-from-child-*

bound-with-invariants-and-edge-ranges:

assumes *run*:

direct-insert-costed-partition-loop-state Δ *M-of t h k cap l d P B d' D a*

betas bs B' Us U c child-costs

and *sound*: *sound-label d*

and *pre*: *bmssp-pre-full d P B*

and *P-reaches*: $\bigwedge x. x \in P \implies \text{reachable } s \ x$

and *P-k-cap*: $k * \text{card } P \leq \text{cap}$

and *P-anti*: *tree-antichain P*

and *child-bound*:

$\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$

$\llbracket \text{direct-insert-costed-bmssp } \Delta \text{ M-of t h k cap l d S-child B-child}$

d-child B-child' U-child c-child;

bmssp-pre-full d S-child B-child;

$\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$

card S-child $\leq$ *M-of l*;

$\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child};$

$k * \text{card } S\text{-child} \leq \text{cap};$

tree-antichain S-child $\rrbracket$

$\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$

and *M-cap*: *M-of l* $\leq$ *cap*

and *source-factor*: *Suc h* $\leq$ *A*

and *k-pos*: $0 < k$

shows *c* $\leq$

*A * Suc L * card U + R * card (outgoing-edges U) +*

*t * sum-list*

(map card (range-tree-child-direct-edge-range-list P B a betas bs)) +

*h * sum-list*

(map card (range-tree-child-edge-range-list P a betas bs))

proof –

have *P-subset*: $P \subseteq V$

using *pre* **unfolding** *bmssp-pre-full-def* **by** *blast*

have *child-cost-bounds*:

list-all2 $(\lambda c\text{-child } U\text{-child}.$

c-child $\leq$ *level-range-cost-bound A R L U-child*)

child-costs $(\text{range-tree-child-list } P \ a \ bs)$

by $(\text{rule } \text{direct-insert-costed-partition-loop-state-child-cost-bounds-with-invariants}$
 $[\text{OF run } P\text{-subset } P\text{-k-cap } P\text{-anti } \text{child-bound}])$

have *source-progress*:

$\bigwedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}.$

$\llbracket \text{direct-insert-costed-bmssp } \Delta \text{ M-of t h k cap l d S-child B-child}$

d-child B-child' U-child c-child;

bmssp-pre-full d S-child B-child;

$\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$

card S-child $\leq$ *M-of l*;

$\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child} \rrbracket$

$\implies \text{card } S\text{-child} \leq \text{card } U\text{-child}$

proof –

fix *S-child B-child d-child B-child' U-child c-child*

assume *child*:
 direct-insert-costed-bmssp Δ *M-of t h k cap l d S-child B-child*
 d-child B-child' U-child c-child
and *child-pre*: *bmssp-pre-full d S-child B-child*
and *child-reaches*: $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x$
and *child-card*: $\text{card } S\text{-child} \leq M\text{-of } l$
and *child-below*: $\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child}$
have *child-cap*: $\text{card } S\text{-child} \leq \text{cap}$
using *child-card M-cap* **by** *linarith*
show $\text{card } S\text{-child} \leq \text{card } U\text{-child}$
 by (*rule direct-insert-costed-bmssp-source-card-le-from-label-below-all-levels*
 [*OF child sound child-pre child-reaches child-below child-cap k-pos*])
qed
show *?thesis*
 by (*rule direct-insert-costed-partition-loop-state-closes-level-bound-from-child-*
costs-and-edge-ranges
 [*OF run sound pre P-reaches child-cost-bounds source-factor*
 source-progress])
qed

theorem *direct-insert-costed-partition-loop-state-closes-amortized-bound-from-child-*
bound-with-invariants:
assumes *run*:
 direct-insert-costed-partition-loop-state Δ *M-of t h k cap l d P B d' D a*
 betas bs B' Us U c child-costs
and *sound*: *sound-label d*
and *pre*: *bmssp-pre-full d P B*
and *P-reaches*: $\bigwedge x. x \in P \implies \text{reachable } s \ x$
and *P-k-cap*: $k * \text{card } P \leq \text{cap}$
and *P-anti*: *tree-antichain P*
and *child-bound*:
 $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\llbracket \text{direct-insert-costed-bmssp } \Delta \text{ M-of t h k cap l d S-child B-child}$
 d-child B-child' U-child c-child;
 bmssp-pre-full d S-child B-child;
 $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x$;
 $\text{card } S\text{-child} \leq M\text{-of } l$;
 $\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child}$;
 $k * \text{card } S\text{-child} \leq \text{cap}$;
 tree-antichain S-child $\rrbracket$
 $\implies c\text{-child} \leq$
 bmssp-amortized-cost-bound A R h t q L B-child U-child
and *M-cap*: $M\text{-of } l \leq \text{cap}$
and *source-factor*: $\text{Suc } h \leq A$
and *k-pos*: $0 < k$
shows $c \leq \text{bmssp-amortized-cost-bound } A \ R \ h \ t \ (\text{Suc } q) \ (\text{Suc } L) \ B \ U$
proof –
 have *P-subset*: $P \subseteq V$
 using *pre unfolding bmssp-pre-full-def* **by** *blast*

have *child-cost-bounds*:
list-all2 ($\lambda c\text{-child } UB. \text{ case } UB \text{ of } (U\text{-child}, B\text{-child}) \Rightarrow$
 $c\text{-child} \leq \text{bmssp-amortized-cost-bound } A \ R \ h \ t \ q \ L \ B\text{-child } U\text{-child}$)
 $\text{child-costs } (\text{range-tree-child-bound-pair-list } P \ a \ \text{betas } bs)$
by (*rule direct-insert-costed-partition-loop-state-child-amortized-cost-bounds-with-invariants*
 $[OF \text{ run } P\text{-subset } P\text{-k-cap } P\text{-anti } \text{child-bound}]])$
have *source-progress*:
 $\bigwedge S\text{-child } B\text{-child } d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}.$
 $\llbracket \text{direct-insert-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$
 $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child};$
 $\bigwedge x. x \in S\text{-child} \Rightarrow \text{reachable } s \ x;$
 $\text{card } S\text{-child} \leq M\text{-of } l;$
 $\bigwedge x. x \in S\text{-child} \Rightarrow \text{below-bound } (d \ x) \ B\text{-child} \rrbracket$
 $\Rightarrow \text{card } S\text{-child} \leq \text{card } U\text{-child}$
proof –
fix $S\text{-child } B\text{-child } d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}$
assume *child*:
 $\text{direct-insert-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}$
and *child-pre*: $\text{bmssp-pre-full } d \ S\text{-child } B\text{-child}$
and *child-reaches*: $\bigwedge x. x \in S\text{-child} \Rightarrow \text{reachable } s \ x$
and *child-card*: $\text{card } S\text{-child} \leq M\text{-of } l$
and *child-below*: $\bigwedge x. x \in S\text{-child} \Rightarrow \text{below-bound } (d \ x) \ B\text{-child}$
have *child-cap*: $\text{card } S\text{-child} \leq \text{cap}$
using *child-card* $M\text{-cap}$ **by** *linarith*
show $\text{card } S\text{-child} \leq \text{card } U\text{-child}$
by (*rule direct-insert-costed-bmssp-source-card-le-from-label-below-all-levels*
 $[OF \text{ child sound child-pre child-reaches child-below child-cap k-pos}]])$
qed
show *?thesis*
by (*rule direct-insert-costed-partition-loop-state-closes-amortized-bound-from-child-costs*
 $[OF \text{ run sound pre } P\text{-reaches } \text{child-cost-bounds } \text{source-factor}$
 $\text{source-progress}]])$
qed

theorem *direct-insert-costed-nonbase-step-closes-amortized-bound-from-child-bound-with-invariants*:

assumes *loop*:
 $\text{direct-insert-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 $\text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs}$
and *sound*: $\text{sound-label } d$
and *pre*: $\text{bmssp-pre-full } d \ P \ B$
and *P-reaches*: $\bigwedge x. x \in P \Rightarrow \text{reachable } s \ x$
and *P-k-cap*: $k * \text{card } P \leq \text{cap}$
and *P-anti*: $\text{tree-antichain } P$
and *child-bound*:

$\wedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'$.
 $\llbracket \text{direct-insert-costed-bmssp } \Delta \text{ M-of } t \text{ h } k \text{ cap } l \text{ d } S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \text{ } U\text{-child } c\text{-child};$
 $\quad \text{bmssp-pre-full } d \text{ } S\text{-child } B\text{-child};$
 $\quad \wedge x. x \in S\text{-child} \implies \text{reachable } s \text{ } x;$
 $\quad \text{card } S\text{-child} \leq \text{M-of } l;$
 $\quad \wedge x. x \in S\text{-child} \implies \text{below-bound } (d \text{ } x) \text{ } B\text{-child};$
 $\quad k * \text{card } S\text{-child} \leq \text{cap};$
 $\quad \text{tree-antichain } S\text{-child} \rrbracket$
 $\implies c\text{-child} \leq$
 $\quad \text{bmssp-amortized-cost-bound } A \text{ } R \text{ } h \text{ } t \text{ } q \text{ } L \text{ } B\text{-child } U\text{-child}$
and $M\text{-cap}: \text{M-of } l \leq \text{cap}$
and $\text{source-factor}: \text{Suc } h \leq A$
and $k\text{-pos}: 0 < k$
and $U\text{-def}: U = U\text{-loop} \cup W$
and $\text{finite-U}: \text{finite } U$
and $\text{scan-insert}: c\text{-scan-insert} \leq A * \text{card } U$
and $c\text{-def}: c = c\text{-scan-insert} + c\text{-loop}$
shows $c \leq$
 $\quad \text{bmssp-amortized-cost-bound } A \text{ } R \text{ } h \text{ } t \text{ } (\text{Suc } q) \text{ } (\text{Suc } (\text{Suc } L)) \text{ } B \text{ } U$
proof –
have $\text{loop-bound}:$
 $\quad c\text{-loop} \leq$
 $\quad \text{bmssp-amortized-cost-bound } A \text{ } R \text{ } h \text{ } t \text{ } (\text{Suc } q) \text{ } (\text{Suc } L) \text{ } B \text{ } U\text{-loop}$
by (rule $\text{direct-insert-costed-partition-loop-state-closes-amortized-bound-from-child-bound-with-invariants}$
 $\quad [\text{OF loop sound pre } P\text{-reaches } P\text{-k-cap } P\text{-anti child-bound } M\text{-cap}$
 $\quad \text{source-factor } k\text{-pos}])$
have $U\text{-loop-subset}: U\text{-loop} \subseteq U$
using $U\text{-def}$ **by** blast
have $\text{card-loop-le}: \text{card } U\text{-loop} \leq \text{card } U$
by (rule $\text{card-mono}[\text{OF finite-U } U\text{-loop-subset}])$
have $\text{outgoing-subset}: \text{outgoing-edges } U\text{-loop} \subseteq \text{outgoing-edges } U$
by (rule $\text{outgoing-edges-mono}[\text{OF } U\text{-loop-subset}])$
have $\text{finite-out-U}: \text{finite } (\text{outgoing-edges } U)$
by simp
have $\text{card-out-le}: \text{card } (\text{outgoing-edges } U\text{-loop}) \leq \text{card } (\text{outgoing-edges } U)$
by (rule $\text{card-mono}[\text{OF finite-out-U } \text{outgoing-subset}])$
have $\text{range-subset}:$
 $\quad \text{outgoing-edges-range } U\text{-loop } 0 \text{ } B \subseteq \text{outgoing-edges-range } U \text{ } 0 \text{ } B$
by (rule $\text{outgoing-edges-range-mono-sources}[\text{OF } U\text{-loop-subset}])$
have $\text{card-range-le}:$
 $\quad \text{card } (\text{outgoing-edges-range } U\text{-loop } 0 \text{ } B) \leq$
 $\quad \text{card } (\text{outgoing-edges-range } U \text{ } 0 \text{ } B)$
by (rule $\text{card-mono}[\text{OF finite-outgoing-edges-range range-subset}])$
have $\text{loop-to-U}:$
 $\quad c\text{-loop} \leq$
 $\quad A * \text{Suc } L * \text{card } U +$
 $\quad (R + \text{Suc } q * h) * \text{card } (\text{outgoing-edges } U) +$

```

    t * card (outgoing-edges-range U 0 B)
proof -
  have vterm:
    A * Suc L * card U-loop ≤ A * Suc L * card U
    using card-loop-le by simp
  have eterm:
    (R + Suc q * h) * card (outgoing-edges U-loop) ≤
    (R + Suc q * h) * card (outgoing-edges U)
    using card-out-le by simp
  have rterm:
    t * card (outgoing-edges-range U-loop 0 B) ≤
    t * card (outgoing-edges-range U 0 B)
    using card-range-le by simp
  show ?thesis
    using loop-bound vterm eterm rterm
    unfolding bmssp-amortized-cost-bound-def by linarith
qed
have c ≤
  A * card U +
  (A * Suc L * card U +
   (R + Suc q * h) * card (outgoing-edges U) +
   t * card (outgoing-edges-range U 0 B))
  using scan-insert loop-to-U c-def by linarith
also have ... =
  A * Suc (Suc L) * card U +
  (R + Suc q * h) * card (outgoing-edges U) +
  t * card (outgoing-edges-range U 0 B)
  by (simp add: algebra-simps)
finally show ?thesis
  unfolding bmssp-amortized-cost-bound-def .
qed

```

theorem *direct-insert-costed-bmssp-amortized-bound-from-local-budgets-with-invariants:*

```

assumes base-budget:  $\Delta \leq A$ 
and R-pos:  $0 < R$ 
and source-factor:  $\text{Suc } h \leq A$ 
and k-pos:  $0 < k$ 
and M-cap:  $\bigwedge i. i \leq l \implies M\text{-of } i \leq \text{cap}$ 
and step-budget:
   $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$ 
   child-costs U.
  D = label-partition-view
    (find-pivots-label-capped k cap d S B)
    (find-pivots-pivots-capped k cap d S B)  $\implies$ 
    partition-initial-insert-cost-bound c-insert t
    (find-pivots-pivots-capped k cap d S B)  $\implies$ 
    direct-insert-costed-partition-loop-state  $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l$ 
    (find-pivots-label-capped k cap d S B)

```


$(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \text{ } B \text{ } d' \text{ } D \text{ } a \text{ betas } bs \text{ } B'$
 $Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{ find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \text{ } B'.$
 $\text{ find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \text{ } S \text{ } B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \text{ } x) \implies$
 $(\bigwedge x. x \in S \implies \text{below-bound } (d \text{ } x) \text{ } B) \implies$
 $k * \text{card } S \leq \text{cap} \implies$
 $\text{tree-antichain } S \implies$
 $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq A * \text{card } U$
and run:
 $\text{direct-insert-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } l \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$
and sound: $\text{sound-label } d$
and pre: $\text{bmssp-pre-full } d \text{ } S \text{ } B$
and S-reaches: $\bigwedge x. x \in S \implies \text{reachable } s \text{ } x$
and below: $\bigwedge x. x \in S \implies \text{below-bound } (d \text{ } x) \text{ } B$
and S-k-cap: $k * \text{card } S \leq \text{cap}$
and S-anti: $\text{tree-antichain } S$
shows $c \leq \text{bmssp-amortized-cost-bound } A \text{ } R \text{ } h \text{ } t \text{ } l \text{ } (2 * l + 1) \text{ } B \text{ } U$
using $\text{run sound pre S-reaches below S-k-cap S-anti M-cap}$
proof ($\text{induction } l \text{ arbitrary: } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c \text{ rule: less-induct}$)
case ($\text{less } l$)
show $?case$
proof ($\text{cases } l$)
case 0
have run0:
 $\text{direct-insert-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } 0 \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$
using $\text{less.prem}(1) \text{ } 0 \text{ by simp}$
show $?thesis$
using run0
proof cases
case ($\text{Direct-Insert-Costed-Base } x$)
have scan:
 $\text{base-case-scan-cost } \Delta \text{ } k \text{ } x \text{ } B \leq$
 $R * \text{card } (\text{outgoing-edges } (\text{base-case-vertices } k \text{ } x \text{ } B))$
using $R\text{-pos unfolding base-case-scan-cost-def by simp}$
show $?thesis$
using $\text{Direct-Insert-Costed-Base } 0 \text{ scan}$
by ($\text{simp add: bmssp-amortized-cost-bound-def; linarith}$)
qed
next
case ($\text{Suc } l0$)
have run-suc:
 $\text{direct-insert-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } (\text{Suc } l0) \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$
using $\text{less.prem}(1) \text{ } \text{Suc by simp}$

show *?thesis*
proof (*cases rule: direct-insert-costed-bmssp-SucE[OF run-suc]*)
case (*1 c-insert a betas bs Us U-loop c-loop child-costs-loop*)
let *?d-fp = find-pivots-label-capped k cap d S B*
let *?P = find-pivots-pivots-capped k cap d S B*
let *?W = {v ∈ bound-tree S B'. ?d-fp v = dist s v}*
have *sound-fp: sound-label ?d-fp*
unfolding *find-pivots-label-capped-def*
by (*rule fp-iter-capped-label-sound[OF less.premis(2) less.premis(4)]*)
have *pivot-pre: bmssp-pre-full ?d-fp ?P B*
using *find-pivots-capped-establishes-pivot-pre-concrete*
[OF less.premis(2) less.premis(3) less.premis(4)] .
have *S-subset: S ⊆ V*
using *less.premis(3)* **unfolding** *bmssp-pre-full-def* **by** *blast*
have *P-subset-S: ?P ⊆ S*
unfolding *find-pivots-pivots-capped-def* **by** (*auto split: if-splits*)
have *P-reaches: ∧x. x ∈ ?P ⇒ reachable s x*
using *P-subset-S less.premis(4)* **by** *blast*
have *P-k-cap: k * card ?P ≤ cap*
by (*rule find-pivots-pivots-capped-scaled-card-le*
[OF S-subset less.premis(6)])
have *P-anti: tree-antichain ?P*
by (*rule find-pivots-pivots-capped-tree-antichain[OF less.premis(7)]*)
have *child-bound:*
 $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\llbracket \text{direct-insert-costed-bmssp } \Delta \text{ M-of t h k cap l0 ?d-fp } S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \text{ } U\text{-child } c\text{-child};$
 $\quad \text{bmssp-pre-full ?d-fp } S\text{-child } B\text{-child};$
 $\quad \bigwedge x. x \in S\text{-child} \Rightarrow \text{reachable s x};$
 $\quad \text{card } S\text{-child} \leq \text{M-of l0};$
 $\quad \bigwedge x. x \in S\text{-child} \Rightarrow \text{below-bound (?d-fp x) } B\text{-child};$
 $\quad k * \text{card } S\text{-child} \leq \text{cap};$
 $\quad \text{tree-antichain } S\text{-child} \rrbracket$
 $\Rightarrow c\text{-child} \leq$
 $\quad \text{bmssp-amortized-cost-bound A R h t l0 (2 * l0 + 1)}$
 $\quad B\text{-child } U\text{-child}$
proof –
fix *c-child U-child S-child B-child d-child B-child'*
assume *child:*
 $\text{direct-insert-costed-bmssp } \Delta \text{ M-of t h k cap l0 ?d-fp } S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \text{ } U\text{-child } c\text{-child}$
and *child-pre: bmssp-pre-full ?d-fp S-child B-child*
and *child-reaches: ∧x. x ∈ S-child ⇒ reachable s x*
and *child-below:*
 $\bigwedge x. x \in S\text{-child} \Rightarrow \text{below-bound (?d-fp x) } B\text{-child}$
and *child-k-cap: k * card S-child ≤ cap*
and *child-anti: tree-antichain S-child*
have *M-cap-child: ∧i. i ≤ l0 ⇒ M-of i ≤ cap*
using *less.premis(8) Suc by simp*

```

show  $c\text{-child} \leq$ 
   $\text{bmssp-amortized-cost-bound } A \ R \ h \ t \ l0 \ (2 * l0 + 1)$ 
   $B\text{-child } U\text{-child}$ 
  by (rule  $\text{less.IH}[\text{of } l0 \ ?d\text{-fp } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'$ 
     $U\text{-child } c\text{-child}, \text{OF} - \text{child sound-fp child-pre child-reaches}$ 
     $\text{child-below child-k-cap child-anti M-cap-child}]$ )
    (simp add: Suc)
qed
have  $\text{scan-insert:}$ 
   $\text{fp-iter-capped-scan-cost } k \ \text{cap } d \ S \ S \ B + c\text{-insert} \leq$ 
   $A * \text{card } U$ 
  by (rule  $\text{step-budget}[\text{OF refl } 1(3) \ 1(4) \ 1(5) \ 1(1)$ 
     $\text{less.prem}(2) \ \text{less.prem}(3) \ \text{less.prem}(4) \ \text{less.prem}(5)$ 
     $\text{less.prem}(6) \ \text{less.prem}(7)]$ )
have  $\text{trace:}$ 
   $\text{concrete-partition-loop-trace } ?P \ B \ a \ bs \ d' \ B' \ Us \ U\text{-loop}$ 
  by (rule  $\text{direct-insert-costed-partition-loop-state-trace}$ 
     $[\text{OF } 1(4) \ \text{sound-fp pivot-pre } P\text{-reaches}]$ )
have  $\text{loop-post: } \text{bmssp-post-full } ?d\text{-fp } ?P \ B \ d' \ B' \ U\text{-loop}$ 
  by (rule  $\text{concrete-partition-loop-trace-post}[\text{OF trace}]$ )
have  $\text{finite-U-loop: } \text{finite } U\text{-loop}$ 
  using  $\text{loop-post unfolding bmssp-post-full-def}$  by  $\text{simp}$ 
have  $\text{finite-W: } \text{finite } ?W$ 
  by  $\text{simp}$ 
have  $\text{finite-U: } \text{finite } U$ 
  using  $1(1) \ \text{finite-U-loop } \text{finite-W}$  by  $\text{simp}$ 
have  $\text{M-cap-l0: } M\text{-of } l0 \leq \text{cap}$ 
  using  $\text{less.prem}(8) \ \text{Suc}$  by  $\text{simp}$ 
have  $\text{step:}$ 
   $c \leq$ 
   $\text{bmssp-amortized-cost-bound } A \ R \ h \ t \ (\text{Suc } l0)$ 
   $(\text{Suc } (\text{Suc } (2 * l0 + 1))) \ B \ U$ 
  by (rule  $\text{direct-insert-costed-nonbase-step-closes-amortized-bound-from-child-}$ 
     $\text{bound-with-invariants}$ 
     $[\text{OF } 1(4) \ \text{sound-fp pivot-pre } P\text{-reaches } P\text{-k-cap } P\text{-anti child-bound}$ 
     $M\text{-cap-l0 source-factor } k\text{-pos } 1(1) \ \text{finite-U scan-insert } 1(2)]$ )
show  $?thesis$ 
  using  $\text{step Suc}$  by  $\text{simp}$ 
qed
qed
qed

theorem  $\text{direct-insert-costed-partition-loop-state-closes-level-bound-from-child-}$ 
 $\text{bound-with-invariants-and-edge-budget:}$ 
assumes  $\text{run:}$ 
   $\text{direct-insert-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$ 
   $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs}$ 
and  $\text{sound: } \text{sound-label } d$ 
and  $\text{pre: } \text{bmssp-pre-full } d \ P \ B$ 

```

and P -reaches: $\bigwedge x. x \in P \implies \text{reachable } s \ x$
and P - k -cap: $k * \text{card } P \leq \text{cap}$
and P -anti: $\text{tree-antichain } P$
and child-bound :
 $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\llbracket \text{direct-insert-costed-bmssp } \Delta \text{ } M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$
 $\quad \text{bmssp-pre-full } d \ S\text{-child } B\text{-child};$
 $\quad \bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\quad \text{card } S\text{-child} \leq M\text{-of } l;$
 $\quad \bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child};$
 $\quad k * \text{card } S\text{-child} \leq \text{cap};$
 $\quad \text{tree-antichain } S\text{-child} \rrbracket$
 $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$
and M -cap: $M\text{-of } l \leq \text{cap}$
and source-factor : $\text{Suc } h \leq A$
and k -pos: $0 < k$
and edge-budget :
 $t * \text{sum-list}$
 $\quad (\text{map card } (\text{range-tree-child-direct-edge-range-list } P \ B \ a \ \text{betas } bs)) +$
 $\quad h * \text{sum-list}$
 $\quad (\text{map card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$
 $\leq H * \text{card } (\text{outgoing-edges } U)$
shows $c \leq A * \text{Suc } L * \text{card } U + (R + H) * \text{card } (\text{outgoing-edges } U)$
proof –
have loop :
 $c \leq$
 $\quad A * \text{Suc } L * \text{card } U + R * \text{card } (\text{outgoing-edges } U) +$
 $\quad t * \text{sum-list}$
 $\quad (\text{map card } (\text{range-tree-child-direct-edge-range-list } P \ B \ a \ \text{betas } bs)) +$
 $\quad h * \text{sum-list}$
 $\quad (\text{map card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$
by (rule $\text{direct-insert-costed-partition-loop-state-closes-level-bound-from-child-bound-with-invariants-and-edge-ranges}$
 $\quad [OF \ \text{run sound pre } P\text{-reaches } P\text{-}k\text{-cap } P\text{-anti } \text{child-bound } M\text{-cap}$
 $\quad \text{source-factor } k\text{-pos}]$)
have $c \leq A * \text{Suc } L * \text{card } U + R * \text{card } (\text{outgoing-edges } U) +$
 $\quad H * \text{card } (\text{outgoing-edges } U)$
using loop edge-budget **by** linarith
also have $\dots = A * \text{Suc } L * \text{card } U + (R + H) * \text{card } (\text{outgoing-edges } U)$
by ($\text{simp add: algebra-simps}$)
finally show $?thesis$.
qed

theorem $\text{direct-insert-costed-nonbase-step-closes-level-bound-from-child-bound-with-invariants-and-edge-budget}$:

assumes loop :
 $\text{direct-insert-costed-partition-loop-state } \Delta \text{ } M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 $\text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs}$

and *sound*: *sound-label* *d*
and *pre*: *bmssp-pre-full* *d P B*
and *P-reaches*: $\bigwedge x. x \in P \implies \text{reachable } s \ x$
and *P-k-cap*: $k * \text{card } P \leq \text{cap}$
and *P-anti*: *tree-antichain* *P*
and *child-bound*:
 $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\llbracket \text{direct-insert-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$
 $\quad \text{bmssp-pre-full } d \ S\text{-child } B\text{-child};$
 $\quad \bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\quad \text{card } S\text{-child} \leq M\text{-of } l;$
 $\quad \bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child};$
 $\quad k * \text{card } S\text{-child} \leq \text{cap};$
 $\quad \text{tree-antichain } S\text{-child} \rrbracket$
 $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$
and *M-cap*: $M\text{-of } l \leq \text{cap}$
and *source-factor*: $\text{Suc } h \leq A$
and *k-pos*: $0 < k$
and *edge-budget*:
 $t * \text{sum-list}$
 $\quad (\text{map card } (\text{range-tree-child-direct-edge-range-list } P \ B \ a \ \text{betas } bs)) +$
 $\quad h * \text{sum-list}$
 $\quad (\text{map card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$
 $\leq H * \text{card } (\text{outgoing-edges } U\text{-loop})$
and *U-def*: $U = U\text{-loop} \cup W$
and *finite-U*: *finite* *U*
and *scan-insert*: $c\text{-scan-insert} \leq A * \text{card } U$
and *c-def*: $c = c\text{-scan-insert} + c\text{-loop}$
shows $c \leq A * \text{Suc } (\text{Suc } L) * \text{card } U +$
 $(R + H) * \text{card } (\text{outgoing-edges } U)$
proof –
have *loop-bound*:
 $c\text{-loop} \leq A * \text{Suc } L * \text{card } U\text{-loop} +$
 $(R + H) * \text{card } (\text{outgoing-edges } U\text{-loop})$
by (rule *direct-insert-costed-partition-loop-state-closes-level-bound-from-child-bound-with-invariants-and-edge-budget*
 $[\text{OF loop sound pre P-reaches P-k-cap P-anti child-bound M-cap}$
 $\text{source-factor k-pos edge-budget}]$)
have *U-loop-subset*: $U\text{-loop} \subseteq U$
using *U-def* **by** *blast*
have *card-loop-le*: $\text{card } U\text{-loop} \leq \text{card } U$
by (rule *card-mono*[*OF finite-U U-loop-subset*])
have *outgoing-subset*: $\text{outgoing-edges } U\text{-loop} \subseteq \text{outgoing-edges } U$
by (rule *outgoing-edges-mono*[*OF U-loop-subset*])
have *finite-out-U*: *finite* (*outgoing-edges* *U*)
by *simp*
have *card-out-le*:
 $\text{card } (\text{outgoing-edges } U\text{-loop}) \leq \text{card } (\text{outgoing-edges } U)$

by (rule card-mono[OF finite-out-U outgoing-subset])
 have $c \leq$
 $A * \text{card } U +$
 $(A * \text{Suc } L * \text{card } U + (R + H) * \text{card } (\text{outgoing-edges } U))$
 proof -
 have loop-to-U:
 $c\text{-loop} \leq A * \text{Suc } L * \text{card } U +$
 $(R + H) * \text{card } (\text{outgoing-edges } U)$
 proof -
 have $A * \text{Suc } L * \text{card } U\text{-loop} \leq A * \text{Suc } L * \text{card } U$
 using card-loop-le by simp
 moreover have $(R + H) * \text{card } (\text{outgoing-edges } U\text{-loop}) \leq$
 $(R + H) * \text{card } (\text{outgoing-edges } U)$
 using card-out-le by simp
 ultimately show ?thesis
 using loop-bound by linarith
 qed
 show ?thesis
 using scan-insert loop-to-U c-def by linarith
 qed
 also have ... =
 $A * \text{Suc } (\text{Suc } L) * \text{card } U + (R + H) * \text{card } (\text{outgoing-edges } U)$
 by (simp add: algebra-simps)
 finally show ?thesis .
 qed

theorem direct-insert-costed-bmssp-level-bound-from-local-budgets-with-invariants-and-edge-budget:

assumes base-budget: $\Delta \leq A$
 and R-pos: $0 < R$
 and source-factor: $\text{Suc } h \leq A$
 and k-pos: $0 < k$
 and M-cap: $\bigwedge i. i \leq l \implies M\text{-of } i \leq \text{cap}$
 and edge-budget:
 $\bigwedge l \ d \ P \ B \ d' \ D \ a \ \text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs.}$
 direct-insert-costed-partition-loop-state $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 betas $bs \ B' \ Us \ U \ c \ \text{child-costs} \implies$
 sound-label $d \implies$
 bmssp-pre-full $d \ P \ B \implies$
 $(\bigwedge x. x \in P \implies \text{reachable } s \ x) \implies$
 $t * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-direct-edge-range-list } P \ B \ a \ \text{betas } bs)) +$
 $h * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$
 $\leq H * \text{card } (\text{outgoing-edges } U)$
 and step-budget:
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
 child-costs $U.$
 $D = \text{label-partition-view}$

$(\text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \implies$
 $\text{direct-insert-costed-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } l$
 $(\text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \text{ } B \text{ } d' \text{ } D \text{ } a \text{ betas } bs \text{ } B'$
 $Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{ find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{ find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \text{ } S \text{ } B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \text{ } x) \implies$
 $(\bigwedge x. x \in S \implies \text{below-bound } (d \text{ } x) \text{ } B) \implies$
 $k * \text{card } S \leq \text{cap} \implies$
 $\text{tree-antichain } S \implies$
 $\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq A * \text{card } U$
and run:
 $\text{direct-insert-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } l \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$
and sound: $\text{sound-label } d$
and pre: $\text{bmssp-pre-full } d \text{ } S \text{ } B$
and S-reaches: $\bigwedge x. x \in S \implies \text{reachable } s \text{ } x$
and below: $\bigwedge x. x \in S \implies \text{below-bound } (d \text{ } x) \text{ } B$
and S-k-cap: $k * \text{card } S \leq \text{cap}$
and S-anti: $\text{tree-antichain } S$
shows $c \leq \text{level-range-cost-bound } A \text{ } (R + l * H) \text{ } (2 * l + 1) \text{ } U$
using $\text{run sound pre S-reaches below S-k-cap S-anti M-cap}$
proof ($\text{induction } l \text{ arbitrary: } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c \text{ rule: less-induct}$)
case ($\text{less } l$)
show $?case$
proof ($\text{cases } l$)
case 0
have run0:
 $\text{direct-insert-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } 0 \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$
using $\text{less.prem}(1) \text{ } 0 \text{ by simp}$
show $?thesis$
using $\text{run0 } R\text{-pos } 0$
by ($\text{cases rule: direct-insert-costed-bmssp.cases}$)
 $(\text{simp-all add: split-range-costed-base-level-bound})$
next
case ($\text{Suc } l0$)
have run-suc:
 $\text{direct-insert-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } (\text{Suc } l0) \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$
using $\text{less.prem}(1) \text{ } \text{Suc by simp}$
show $?thesis$
proof ($\text{cases rule: direct-insert-costed-bmssp-SucE[OF run-suc]}$)

```

case (1 c-insert a betas bs Us U-loop c-loop child-costs-loop)
let ?d-fp = find-pivots-label-capped k cap d S B
let ?P = find-pivots-pivots-capped k cap d S B
let ?W = {v ∈ bound-tree S B'. ?d-fp v = dist s v}
have sound-fp: sound-label ?d-fp
  unfolding find-pivots-label-capped-def
  by (rule fp-iter-capped-label-sound[OF less.prems(2) less.prems(4)])
have pivot-pre: bmssp-pre-full ?d-fp ?P B
  using find-pivots-capped-establishes-pivot-pre-concrete
    [OF less.prems(2) less.prems(3) less.prems(4)] .
have S-subset: S ⊆ V
  using less.prems(3) unfolding bmssp-pre-full-def by blast
have P-subset-S: ?P ⊆ S
  unfolding find-pivots-pivots-capped-def by (auto split: if-splits)
have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
  using P-subset-S less.prems(4) by blast
have P-k-cap: k * card ?P ≤ cap
  by (rule find-pivots-pivots-capped-scaled-card-le
    [OF S-subset less.prems(6)])
have P-anti: tree-antichain ?P
  by (rule find-pivots-pivots-capped-tree-antichain[OF less.prems(7)])
have child-bound:
   $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
   $\llbracket \text{direct-insert-costed-bmssp } \Delta \text{ M-of } t \ h \ k \ \text{cap } l0 \ ?d\text{-fp } S\text{-child } B\text{-child}$ 
   $\quad d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$ 
   $\quad \text{bmssp-pre-full } ?d\text{-fp } S\text{-child } B\text{-child};$ 
   $\quad \bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$ 
   $\quad \text{card } S\text{-child} \leq \text{M-of } l0;$ 
   $\quad \bigwedge x. x \in S\text{-child} \implies \text{below-bound } (?d\text{-fp } x) \ B\text{-child};$ 
   $\quad k * \text{card } S\text{-child} \leq \text{cap};$ 
   $\quad \text{tree-antichain } S\text{-child} \rrbracket$ 
   $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ (R + l0 * H)$ 
   $\quad (2 * l0 + 1) \ U\text{-child}$ 
proof –
  fix c-child U-child S-child B-child d-child B-child'
  assume child:
    direct-insert-costed-bmssp Δ M-of t h k cap l0 ?d-fp S-child B-child
    d-child B-child' U-child c-child
  and child-pre: bmssp-pre-full ?d-fp S-child B-child
  and child-reaches:  $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x$ 
  and child-below:
     $\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (?d\text{-fp } x) \ B\text{-child}$ 
  and child-k-cap: k * card S-child ≤ cap
  and child-anti: tree-antichain S-child
have M-cap-child:  $\bigwedge i. i \leq l0 \implies \text{M-of } i \leq \text{cap}$ 
  using less.prems(8) Suc by simp
show c-child ≤ level-range-cost-bound A (R + l0 * H)
   $\quad (2 * l0 + 1) \ U\text{-child}$ 
  by (rule less.IH[of l0 ?d-fp S-child B-child d-child B-child'

```



```

      U-child c-child, OF - child sound-fp child-pre child-reaches
      child-below child-k-cap child-anti M-cap-child])
    (simp add: Suc)
  qed
  have loop-edge-budget:
    t * sum-list
      (map card (range-tree-child-direct-edge-range-list ?P B a betas bs)) +
    h * sum-list
      (map card (range-tree-child-edge-range-list ?P a betas bs))
    ≤ H * card (outgoing-edges U-loop)
  by (rule edge-budget[OF 1(4) sound-fp pivot-pre P-reaches])
  have scan-insert:
    fp-iter-capped-scan-cost k cap d S S B + c-insert ≤
      A * card U
  by (rule step-budget[OF refl 1(3) 1(4) 1(5) 1(1)
    less.prem1(2) less.prem1(3) less.prem1(4) less.prem1(5)
    less.prem1(6) less.prem1(7)])
  have trace:
    concrete-partition-loop-trace ?P B a bs d' B' Us U-loop
  by (rule direct-insert-costed-partition-loop-state-trace
    [OF 1(4) sound-fp pivot-pre P-reaches])
  have loop-post: bmssp-post-full ?d-fp ?P B d' B' U-loop
  by (rule concrete-partition-loop-trace-post[OF trace])
  have finite-U-loop: finite U-loop
    using loop-post unfolding bmssp-post-full-def by simp
  have finite-W: finite ?W
    by simp
  have finite-U: finite U
    using 1(1) finite-U-loop finite-W by simp
  have M-cap-l0: M-of l0 ≤ cap
    using less.prem1(8) Suc by simp
  have step:
    c ≤ A * Suc (Suc (2 * l0 + 1)) * card U +
      (R + l0 * H + H) * card (outgoing-edges U)
  by (rule direct-insert-costed-nonbase-step-closes-level-bound-from-child-bound-
    with-invariants-and-edge-budget
      [OF 1(4) sound-fp pivot-pre P-reaches P-k-cap P-anti child-bound
        M-cap-l0 source-factor k-pos loop-edge-budget 1(1) finite-U
        scan-insert 1(2)])
  show ?thesis
    using step Suc unfolding level-range-cost-bound-def
    by (simp add: algebra-simps)
  qed
  qed
  qed

```

theorem *finite-initial-label-direct-insert-costed-top-level-correct-and-closed-refined-graph-bound-level-cap-from-local-budgets:*

assumes *all-reachable:* $\bigwedge v. v \in V \implies \text{reachable } s \ v$

and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *source-factor*: $\text{Suc } h \leq 2 * A$
and *k-pos*: $0 < k$
and *edge-budget*:
 $\bigwedge l' d P B d' D a \text{ betas } bs B' Us \text{ U-loop c-loop child-costs.}$
 $\text{direct-insert-costed-partition-loop-state } \Delta (\text{bmssp-level-cap } k t) t h k$
 $(\text{bmssp-level-cap } k t l) l' d P B d' D a \text{ betas } bs B'$
 $Us \text{ U-loop c-loop child-costs} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d P B \implies$
 $(\bigwedge x. x \in P \implies \text{reachable } s x) \implies$
 $t * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-direct-edge-range-list } P B a \text{ betas } bs)) +$
 $h * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-edge-range-list } P a \text{ betas } bs))$
 $\leq H * \text{card } (\text{outgoing-edges } U\text{-loop})$
and *step-budget*:
 $\bigwedge l' d S B D c\text{-insert } d' a \text{ betas } bs B' Us \text{ U-loop c-loop}$
 $\text{child-costs } U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k (\text{bmssp-level-cap } k t l) d S B)$
 $(\text{find-pivots-pivots-capped } k (\text{bmssp-level-cap } k t l) d S B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k (\text{bmssp-level-cap } k t l) d S B) \implies$
 $\text{direct-insert-costed-partition-loop-state } \Delta (\text{bmssp-level-cap } k t) t h k$
 $(\text{bmssp-level-cap } k t l) l'$
 $(\text{find-pivots-label-capped } k (\text{bmssp-level-cap } k t l) d S B)$
 $(\text{find-pivots-pivots-capped } k (\text{bmssp-level-cap } k t l) d S B) B d' D$
 $a \text{ betas } bs B' Us \text{ U-loop c-loop child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S B'.$
 $\text{find-pivots-label-capped } k (\text{bmssp-level-cap } k t l) d S B v =$
 $\text{dist } s v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S B'.$
 $\text{find-pivots-label-capped } k (\text{bmssp-level-cap } k t l) d S B v =$
 $\text{dist } s v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d S B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s x) \implies$
 $(\bigwedge x. x \in S \implies \text{below-bound } (d x) B) \implies$
 $k * \text{card } S \leq \text{bmssp-level-cap } k t l \implies$
 $\text{tree-antichain } S \implies$
 $\text{fp-iter-capped-scan-cost } k (\text{bmssp-level-cap } k t l) d S S B +$
 $c\text{-insert} \leq 2 * A * \text{card } U$
and *run*:
 $\text{direct-insert-costed-bmssp } \Delta (\text{bmssp-level-cap } k t) t h k$
 $(\text{bmssp-level-cap } k t l) l$

```

      finite-initial-label {s} Infinity d' Infinity U c
shows sssp-correct d' ∧
  c ≤ 2 * A * (2 * l + 1) * vertex-count + (R + l * H) * edge-count
proof
  show sssp-correct d'
    by (rule finite-initial-label-direct-insert-costed-top-level-correct
      [OF all-reachable run])
next
  have pre: bmssp-pre-full finite-initial-label {s} Infinity
    using all-reachable finite-initial-label-source-complete
    by (rule top-bmssp-pre-full)
  have sound: sound-label finite-initial-label
    using finite-initial-label-sound[OF all-reachable] .
  have S-reaches:  $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$ 
    using all-reachable source-in-V by blast
  have below:  $\bigwedge x. x \in \{s\} \implies$ 
    below-bound (finite-initial-label x) Infinity
    by simp
  have top-cap:  $k * \text{card } \{s\} \leq \text{bmssp-level-cap } k \ t \ l$ 
proof -
  have one-le:  $1 \leq \text{bmssp-level-width } t \ l$ 
    unfolding bmssp-level-width-def by simp
  have  $k * 1 \leq k * \text{bmssp-level-width } t \ l$ 
    by (rule mult-left-mono[OF one-le]) simp
  then show ?thesis
    unfolding bmssp-level-cap-def by simp
qed
  have top-anti: tree-antichain {s}
    by simp
  have range-bound:
    c ≤ level-range-cost-bound (2 * A) (R + l * H) (2 * l + 1) U
  proof (rule direct-insert-costed-bmssp-level-bound-from-local-budgets-with-
    invariants-and-edge-budget
    [where A = 2 * A and R = R and H = H,
      OF - R-pos source-factor k-pos - - - run sound pre S-reaches below
      top-cap top-anti])
    show  $\Delta \leq 2 * A$ 
      using degree-factor by simp
  next
    fix i
    assume i ≤ l
    then show bmssp-level-cap k t i ≤ bmssp-level-cap k t l
      by (rule bmssp-level-cap-mono)
  next
    fix l' d P B d' D a betas bs B' Us U-loop c-loop child-costs
    assume loop:
      direct-insert-costed-partition-loop-state Δ
      (bmssp-level-cap k t) t h k (bmssp-level-cap k t l) l'
      d P B d' D a betas bs B' Us U-loop c-loop child-costs

```

```

and sound-l: sound-label d
and pre-l: bmssp-pre-full d P B
and reaches-l:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
show t * sum-list
  (map card (range-tree-child-direct-edge-range-list P B a betas bs)) +
  h * sum-list (map card (range-tree-child-edge-range-list P a betas bs))
   $\leq H * \text{card}$  (outgoing-edges U-loop)
  by (rule edge-budget[OF loop sound-l pre-l reaches-l])
next
fix l' d S B D c-insert d' a betas bs B' Us U-loop c-loop child-costs U
assume D-def:
  D = label-partition-view
  (find-pivots-label-capped k (bmssp-level-cap k t l) d S B)
  (find-pivots-pivots-capped k (bmssp-level-cap k t l) d S B)
and insert:
  partition-initial-insert-cost-bound c-insert t
  (find-pivots-pivots-capped k (bmssp-level-cap k t l) d S B)
and loop:
  direct-insert-costed-partition-loop-state  $\Delta$ 
  (bmssp-level-cap k t) t h k (bmssp-level-cap k t l) l'
  (find-pivots-label-capped k (bmssp-level-cap k t l) d S B)
  (find-pivots-pivots-capped k (bmssp-level-cap k t l) d S B) B d' D
  a betas bs B' Us U-loop c-loop child-costs
and complete:
  complete-on d'
  {v  $\in$  bound-tree S B'.
   find-pivots-label-capped k (bmssp-level-cap k t l) d S B v =
   dist s v}
and U-def:
  U = U-loop  $\cup$ 
  {v  $\in$  bound-tree S B'.
   find-pivots-label-capped k (bmssp-level-cap k t l) d S B v =
   dist s v}
and sound-s: sound-label d
and pre-s: bmssp-pre-full d S B
and reaches-s:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
and below-s:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$ 
and S-k-cap: k * card S  $\leq$  bmssp-level-cap k t l
and anti: tree-antichain S
show fp-iter-capped-scan-cost k (bmssp-level-cap k t l) d S S B +
  c-insert  $\leq 2 * A * \text{card}$  U
  by (rule step-budget[OF D-def insert loop complete U-def sound-s pre-s
   reaches-s below-s S-k-cap anti])
qed
have post-full:
  bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
  by (rule direct-insert-costed-bmssp-correct[OF run sound pre S-reaches])
have post: bmssp-post finite-initial-label {s} Infinity d' Infinity U
  using bmssp-post-full-imp-post[OF post-full] .

```

```

have U-V:  $U = V$ 
  using post bound-tree-source-infinity[OF all-reachable]
  unfolding bmssp-post-def by auto
have graph-bound:
  level-range-cost-bound  $(2 * A) (R + l * H) (2 * l + 1) U$ 
     $\leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * H) * \text{edge-count}$ 
proof -
  have U-card:  $\text{card } U = \text{vertex-count}$ 
  unfolding U-V vertex-count-def by simp
  have out-le:  $\text{card } (\text{outgoing-edges } U) \leq \text{edge-count}$ 
  by (rule edge-count-outgoing-bound)
  have out-term:
     $(R + l * H) * \text{card } (\text{outgoing-edges } U) \leq (R + l * H) * \text{edge-count}$ 
  using out-le by simp
  show ?thesis
  unfolding level-range-cost-bound-def
  using U-card out-term by (simp add: algebra-simps)
qed
show  $c \leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * H) * \text{edge-count}$ 
  using range-bound graph-bound by linarith
qed

```

corollary *finite-initial-label-direct-insert-costed-top-level-correct-and-closed-bmssp-refined-graph-time-bound-level-cap-from-local-budgets:*

```

assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
and degree-factor:  $\Delta \leq A$ 
and R-pos:  $0 < R$ 
and source-factor:  $\text{Suc } h \leq 2 * A$ 
and k-pos:  $0 < k$ 
and edge-budget:
   $\bigwedge l' d P B d' D a \text{ betas } bs B' Us \text{ U-loop } c\text{-loop } \text{child-costs}.$ 
    direct-insert-costed-partition-loop-state  $\Delta (\text{bmssp-level-cap } k \ t) \ t \ h \ k$ 
       $(\text{bmssp-level-cap } k \ t \ l) \ l' d P B d' D a \text{ betas } bs B'$ 
       $Us \text{ U-loop } c\text{-loop } \text{child-costs} \implies$ 
    sound-label  $d \implies$ 
    bmssp-pre-full  $d P B \implies$ 
     $(\bigwedge x. x \in P \implies \text{reachable } s \ x) \implies$ 
     $t * \text{sum-list}$ 
       $(\text{map card } (\text{range-tree-child-direct-edge-range-list } P B a \text{ betas } bs)) +$ 
       $h * \text{sum-list}$ 
         $(\text{map card } (\text{range-tree-child-edge-range-list } P a \text{ betas } bs))$ 
         $\leq H * \text{card } (\text{outgoing-edges } U\text{-loop})$ 
and step-budget:
   $\bigwedge l' d S B D c\text{-insert } d' a \text{ betas } bs B' Us \text{ U-loop } c\text{-loop}$ 
    child-costs  $U.$ 
     $D = \text{label-partition-view}$ 
       $(\text{find-pivots-label-capped } k (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B)$ 
       $(\text{find-pivots-pivots-capped } k (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B) \implies$ 
    partition-initial-insert-cost-bound  $c\text{-insert } t$ 

```

$(\text{find-pivots-pivots-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B) \implies$
 $\text{direct-insert-costed-partition-loop-state } \Delta \text{ (bmssp-level-cap } k \text{ } t) \text{ } t \text{ } h \text{ } k$
 $\text{(bmssp-level-cap } k \text{ } t \text{ } l) \text{ } l'$
 $(\text{find-pivots-label-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B)$
 $(\text{find-pivots-pivots-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B) \text{ } B \text{ } d' \text{ } D$
 $a \text{ } \text{betas } bs \text{ } B' \text{ } Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \text{ } B'\}.$
 $\text{find-pivots-label-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B \text{ } v =$
 $\text{dist } s \text{ } v \} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \text{ } B'\}.$
 $\text{find-pivots-label-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B \text{ } v =$
 $\text{dist } s \text{ } v \} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \text{ } S \text{ } B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \text{ } x) \implies$
 $(\bigwedge x. x \in S \implies \text{below-bound } (d \text{ } x) \text{ } B) \implies$
 $k * \text{card } S \leq \text{bmssp-level-cap } k \text{ } t \text{ } l \implies$
 $\text{tree-antichain } S \implies$
 $\text{fp-iter-capped-scan-cost } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } S \text{ } B +$
 $c\text{-insert} \leq 2 * A * \text{card } U$
and run:
 $\text{direct-insert-costed-bmssp } \Delta \text{ (bmssp-level-cap } k \text{ } t) \text{ } t \text{ } h \text{ } k$
 $\text{(bmssp-level-cap } k \text{ } t \text{ } l) \text{ } l$
 $\text{finite-initial-label } \{s\} \text{ } \text{Infinity } d' \text{ } \text{Infinity } U \text{ } c$
shows $\text{sssp-correct } d' \wedge$
 $c \leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. H)$
 $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{ } \text{vertex-count}$
proof –
have graph:
 $\text{sssp-correct } d' \wedge$
 $c \leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * H) * \text{edge-count}$
by $(\text{rule } \text{finite-initial-label-direct-insert-costed-top-level-correct-and-closed-refined-}$
 $\text{graph-bound-level-cap-from-local-budgets}$
 $[\text{OF all-reachable degree-factor } R\text{-pos source-factor } k\text{-pos edge-budget}$
 $\text{step-budget run}])$
have bound:
 $2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * H) * \text{edge-count}$
 $\leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. H)$
 $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{ } \text{vertex-count}$
unfolding $\text{bmssp-refined-graph-time-bound-def}$ **by** simp
show $?thesis$
using $\text{graph bound by linarith}$
qed

theorem $\text{finite-initial-label-direct-insert-costed-top-level-correct-and-closed-refined-}$
 $\text{graph-bound-level-cap:}$
assumes $\text{all-reachable: } \bigwedge v. v \in V \implies \text{reachable } s \text{ } v$

and *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$
and *insert-scaled-factor*: $t \leq A\text{-insert} * k$
and *seen-scaled-factor*: $k * \Delta + A\text{-insert} \leq 2 * A$
and *source-factor*: $Suc\ h \leq 2 * A$
and *k-pos*: $0 < k$
and *edge-budget*:
 $\bigwedge l' d P B d' D a\ betas\ bs\ B' Us\ U\text{-loop}\ c\text{-loop}\ child\text{-costs}.$
direct-insert-costed-partition-loop-state $\Delta\ (bmssp\text{-level-cap}\ k\ t)\ t\ h\ k$
 $(bmssp\text{-level-cap}\ k\ t\ l)\ l' d P B d' D a\ betas\ bs\ B'$
 $Us\ U\text{-loop}\ c\text{-loop}\ child\text{-costs} \implies$
sound-label $d \implies$
bmssp-pre-full $d P B \implies$
 $(\bigwedge x. x \in P \implies reachable\ s\ x) \implies$
 $t * sum\text{-list}$
 $(map\ card\ (range\text{-tree-child-direct-edge-range-list}\ P\ B\ a\ betas\ bs)) +$
 $h * sum\text{-list}$
 $(map\ card\ (range\text{-tree-child-edge-range-list}\ P\ a\ betas\ bs))$
 $\leq H * card\ (outgoing\text{-edges}\ U\text{-loop})$
and *run*:
direct-insert-costed-bmssp $\Delta\ (bmssp\text{-level-cap}\ k\ t)\ t\ h\ k$
 $(bmssp\text{-level-cap}\ k\ t\ l)\ l$
finite-initial-label $\{s\}\ Infinity\ d'\ Infinity\ U\ c$
shows *sssp-correct* $d' \wedge$
 $c \leq 2 * A * (2 * l + 1) * vertex\text{-count} + (R + l * H) * edge\text{-count}$
proof (*rule* *finite-initial-label-direct-insert-costed-top-level-correct-and-closed-*
refined-graph-bound-level-cap-from-local-budgets
 $[OF\ all\text{-reachable}\ degree\text{-factor}\ R\text{-pos}\ source\text{-factor}\ k\text{-pos}\ edge\text{-budget} - run])$
fix $l' d S B D\ c\text{-insert}\ d' a\ betas\ bs\ B' Us\ U\text{-loop}\ c\text{-loop}$
 $child\text{-costs}\ U$
assume *D-def*:
 $D = label\text{-partition-view}$
 $(find\text{-pivots-label-capped}\ k\ (bmssp\text{-level-cap}\ k\ t\ l)\ d\ S\ B)$
 $(find\text{-pivots-pivots-capped}\ k\ (bmssp\text{-level-cap}\ k\ t\ l)\ d\ S\ B)$
and *insert*:
partition-initial-insert-cost-bound $c\text{-insert}\ t$
 $(find\text{-pivots-pivots-capped}\ k\ (bmssp\text{-level-cap}\ k\ t\ l)\ d\ S\ B)$
and *loop*:
direct-insert-costed-partition-loop-state $\Delta\ (bmssp\text{-level-cap}\ k\ t)\ t\ h\ k$
 $(bmssp\text{-level-cap}\ k\ t\ l)\ l'$
 $(find\text{-pivots-label-capped}\ k\ (bmssp\text{-level-cap}\ k\ t\ l)\ d\ S\ B)$
 $(find\text{-pivots-pivots-capped}\ k\ (bmssp\text{-level-cap}\ k\ t\ l)\ d\ S\ B)\ B\ d' D$
 $a\ betas\ bs\ B' Us\ U\text{-loop}\ c\text{-loop}\ child\text{-costs}$
and *complete*:
complete-on d'
 $\{v \in bound\text{-tree}\ S\ B'\}.$
 $find\text{-pivots-label-capped}\ k\ (bmssp\text{-level-cap}\ k\ t\ l)\ d\ S\ B\ v =$

$\text{dist } s \ v\}$
and *U-def*:
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'\}$
 $\text{find-pivots-label-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B \ v =$
 $\text{dist } s \ v\}$
and *sound-s*: *sound-label* d
and *pre-s*: *bmssp-pre-full* $d \ S \ B$
and *reaches-s*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
and *below-s*: $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$
and *S-k-cap*: $k * \text{card } S \leq \text{bmssp-level-cap } k \ t \ l$
and *anti*: *tree-antichain* S
have *seen-success*:
 $B' = B \implies$
 $\text{card } (\text{find-pivots-seen-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B)$
 $\leq \text{card } U$
by (*rule direct-insert-costed-step-seen-success*
 $[\text{OF loop } U\text{-def } \text{sound-s } \text{pre-s } \text{reaches-s } \text{below-s}]$)
show *fp-iter-capped-scan-cost* $k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ S \ B +$
 $c\text{-insert} \leq 2 * A * \text{card } U$
by (*rule direct-insert-costed-capped-step-scan-insert-budget-from-scaled-seen-or-*
threshold
 $[\text{OF degree degree-factor insert-factor insert-scaled-factor}$
 $\text{seen-scaled-factor insert loop } U\text{-def } \text{sound-s } \text{pre-s } \text{reaches-s}$
 $S\text{-k-cap anti k-pos seen-success}]$)
qed

corollary *finite-initial-label-direct-insert-costed-top-level-correct-and-closed-bmssp-*
refined-graph-time-bound-level-cap:

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$
and *insert-scaled-factor*: $t \leq A\text{-insert} * k$
and *seen-scaled-factor*: $k * \Delta + A\text{-insert} \leq 2 * A$
and *source-factor*: $\text{Suc } h \leq 2 * A$
and *k-pos*: $0 < k$
and *edge-budget*:
 $\bigwedge l' \ d \ P \ B \ d' \ D \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs}.$
 $\text{direct-insert-costed-partition-loop-state } \Delta \ (\text{bmssp-level-cap } k \ t) \ t \ h \ k$
 $(\text{bmssp-level-cap } k \ t \ l) \ l' \ d \ P \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ P \ B \implies$
 $(\bigwedge x. x \in P \implies \text{reachable } s \ x) \implies$
 $t * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-direct-edge-range-list } P \ B \ a \ \text{betas } bs)) +$
 $h * \text{sum-list}$

$(\text{map card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$
 $\leq H * \text{card } (\text{outgoing-edges } U\text{-loop})$
and *run*:
 $\text{direct-insert-costed-bmssp } \Delta \ (\text{bmssp-level-cap } k \ t) \ t \ h \ k$
 $(\text{bmssp-level-cap } k \ t \ l) \ l$
 $\text{finite-initial-label } \{s\} \ \text{Infinity } d' \ \text{Infinity } U \ c$
shows *sssp-correct* $d' \wedge$
 $c \leq \text{bmssp-refined-graph-time-bound } (\lambda\cdot. A) \ (\lambda\cdot. R) \ (\lambda\cdot. H)$
 $(\lambda\cdot. l) \ (\lambda\cdot. t) \ (\lambda\cdot. \text{edge-count}) \ \text{vertex-count}$
proof –
have *graph*:
 $\text{sssp-correct } d' \wedge$
 $c \leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * H) * \text{edge-count}$
by (rule *finite-initial-label-direct-insert-costed-top-level-correct-and-closed-refined-graph-bound-level-cap*)
 $[OF \text{ all-reachable degree degree-factor } R\text{-pos insert-factor insert-scaled-factor}$
 $\text{seen-scaled-factor source-factor } k\text{-pos edge-budget run}]$
have *bound*:
 $2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * H) * \text{edge-count}$
 $\leq \text{bmssp-refined-graph-time-bound } (\lambda\cdot. A) \ (\lambda\cdot. R) \ (\lambda\cdot. H)$
 $(\lambda\cdot. l) \ (\lambda\cdot. t) \ (\lambda\cdot. \text{edge-count}) \ \text{vertex-count}$
unfolding *bmssp-refined-graph-time-bound-def* **by** *simp*
show *?thesis*
using *graph bound by linarith*
qed

lemma *direct-insert-costed-partition-loop-state-trivial-edge-budget*:
assumes *run*:
 $\text{direct-insert-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs}$
and *sound*: *sound-label* d
and *pre*: *bmssp-pre-full* $d \ P \ B$
and *P-reaches*: $\bigwedge x. x \in P \implies \text{reachable } s \ x$
shows $t * \text{sum-list}$
 $(\text{map card } (\text{range-tree-child-direct-edge-range-list } P \ B \ a \ \text{betas } bs)) +$
 $h * \text{sum-list } (\text{map card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$
 $\leq (t + h) * \text{card } (\text{outgoing-edges } U)$
proof –
have *mono*: *nondecreasing-from* $a \ bs$
by (rule *direct-insert-costed-partition-loop-state-mono*[*OF run*])
have *trace*: *concrete-partition-loop-trace* $P \ B \ a \ bs \ d' \ B' \ Us \ U$
by (rule *direct-insert-costed-partition-loop-state-trace*
 $[OF \text{ run sound pre } P\text{-reaches}])$
have *children*:
 $\text{list-all2 } (\lambda U \ X. U = X \wedge \text{complete-on } d' \ U) \ Us$
 $(\text{range-tree-chain-list } P \ a \ bs \ B')$
using *trace unfolding concrete-partition-loop-trace-def* **by** *blast*
have *U-def*: $U = \text{bound-tree } P \ (Fin \ a) \cup \bigcup (\text{set } Us)$
using *trace unfolding concrete-partition-loop-trace-def* **by** *blast*

```

have Union-eq:
   $\bigcup(\text{set } Us) = \bigcup(\text{set } (\text{range-tree-chain-list } P \text{ a bs } B'))$ 
  using children by (induction rule: list-all2-induct) auto
have U-eq-chain:
   $U = \text{bound-tree } P \text{ (Fin a)} \cup \text{range-tree-chain } P \text{ a bs } B'$ 
  using U-def Union-eq Union-range-tree-chain-list[of  $P \text{ a bs } B'$ ] by simp
have chain-subset:  $\text{range-tree-chain } P \text{ a bs } B' \subseteq U$ 
  using U-eq-chain by blast
have outgoing-subset:
   $\text{outgoing-edges } (\text{range-tree-chain } P \text{ a bs } B') \subseteq \text{outgoing-edges } U$ 
  by (rule outgoing-edges-mono[OF chain-subset])
have card-out-le:
   $\text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \text{ a bs } B')) \leq$ 
   $\text{card } (\text{outgoing-edges } U)$ 
  by (rule card-mono) (simp-all add: outgoing-subset)
have sum-le-chain:
   $h * \text{sum-list } (\text{map card } (\text{range-tree-child-edge-range-list } P \text{ a betas bs})) +$ 
   $t * \text{sum-list}$ 
   $(\text{map card } (\text{range-tree-child-direct-edge-range-list } P \text{ B a betas bs}))$ 
   $\leq (h + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \text{ a bs } B'))$ 
  by (rule weighted-sum-child-lower-direct-edge-ranges-le-outgoing-edges-chain
  [OF mono])
have chain-to-U:
   $(h + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \text{ a bs } B')) \leq$ 
   $(h + t) * \text{card } (\text{outgoing-edges } U)$ 
  using card-out-le by simp
show ?thesis
  using sum-le-chain chain-to-U by (simp add: algebra-simps)
qed

```

corollary *finite-initial-label-direct-insert-costed-top-level-correct-and-closed-bmssp-refined-graph-time-bound-level-cap-trivial-edge-budget:*

```

assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
  and degree: edge-outdegree-le  $\Delta$ 
  and degree-factor:  $\Delta \leq A$ 
  and R-pos:  $0 < R$ 
  and insert-factor:  $t \leq A * k$ 
  and insert-scaled-factor:  $t \leq A\text{-insert} * k$ 
  and seen-scaled-factor:  $k * \Delta + A\text{-insert} \leq 2 * A$ 
  and source-factor:  $\text{Suc } h \leq 2 * A$ 
  and k-pos:  $0 < k$ 
  and run:
    direct-insert-costed-bmssp  $\Delta$  (bmssp-level-cap  $k \ t$ )  $t \ h \ k$ 
    (bmssp-level-cap  $k \ t \ l$ )  $l$ 
    finite-initial-label  $\{s\}$  Infinity  $d'$  Infinity  $U \ c$ 
shows sssp-correct  $d' \wedge$ 
   $c \leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. t + h)$ 
   $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{vertex-count}$ 
proof (rule finite-initial-label-direct-insert-costed-top-level-correct-and-closed-bmssp-

```

```

refined-graph-time-bound-level-cap
  [OF all-reachable degree degree-factor R-pos insert-factor insert-scaled-factor
    seen-scaled-factor source-factor k-pos - run])
fix l' d P B d' D a betas bs B' Us U-loop c-loop child-costs
assume loop:
  direct-insert-costed-partition-loop-state  $\Delta$ 
  (bmssp-level-cap k t) t h k (bmssp-level-cap k t l) l'
  d P B d' D a betas bs B' Us U-loop c-loop child-costs
and sound-l: sound-label d
and pre-l: bmssp-pre-full d P B
and reaches-l:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
show t * sum-list
  (map card (range-tree-child-direct-edge-range-list P B a betas bs)) +
  h * sum-list (map card (range-tree-child-edge-range-list P a betas bs))
   $\leq (t + h) * \text{card } (\text{outgoing-edges } U\text{-loop})$ 
by (rule direct-insert-costed-partition-loop-state-trivial-edge-budget
  [OF loop sound-l pre-l reaches-l])
qed

theorem finite-initial-label-direct-insert-costed-top-level-correct-and-closed-bmssp-
refined-graph-time-bound-level-cap-amortized:
assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
and degree: edge-outdegree-le  $\Delta$ 
and degree-factor:  $\Delta \leq A$ 
and R-pos:  $0 < R$ 
and insert-factor:  $t \leq A * k$ 
and insert-scaled-factor:  $t \leq A\text{-insert} * k$ 
and seen-scaled-factor:  $k * \Delta + A\text{-insert} \leq 2 * A$ 
and source-factor:  $\text{Suc } h \leq 2 * A$ 
and k-pos:  $0 < k$ 
and run:
  direct-insert-costed-bmssp  $\Delta$  (bmssp-level-cap k t) t h k
  (bmssp-level-cap k t l) l
  finite-initial-label {s} Infinity d' Infinity U c
shows sssp-correct d'  $\wedge$ 
  c  $\leq$  bmssp-refined-graph-time-bound ( $\lambda\cdot. A$ ) ( $\lambda\cdot. R$ ) ( $\lambda\cdot. h$ )
  ( $\lambda\cdot. l$ ) ( $\lambda\cdot. t$ ) ( $\lambda\cdot. \text{edge-count}$ ) vertex-count
proof
show sssp-correct d'
  by (rule finite-initial-label-direct-insert-costed-top-level-correct
  [OF all-reachable run])
next
have pre: bmssp-pre-full finite-initial-label {s} Infinity
  using all-reachable finite-initial-label-source-complete
  by (rule top-bmssp-pre-full)
have sound: sound-label finite-initial-label
  using finite-initial-label-sound [OF all-reachable] .
have S-reaches:  $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$ 
  using all-reachable source-in-V by blast

```

```

have below:  $\bigwedge x. x \in \{s\} \implies$ 
  below-bound (finite-initial-label x) Infinity
by simp
have top-cap:  $k * \text{card } \{s\} \leq \text{bmssp-level-cap } k \ t \ l$ 
proof -
  have one-le:  $1 \leq \text{bmssp-level-width } t \ l$ 
    unfolding bmssp-level-width-def by simp
  have  $k * 1 \leq k * \text{bmssp-level-width } t \ l$ 
    by (rule mult-left-mono[OF one-le]) simp
  then show ?thesis
    unfolding bmssp-level-cap-def by simp
qed
have top-anti: tree-antichain {s}
by simp
have amortized:
   $c \leq \text{bmssp-amortized-cost-bound } (2 * A) \ R \ h \ t \ l \ (2 * l + 1)$ 
  Infinity U
proof (rule direct-insert-costed-bmssp-amortized-bound-from-local-budgets-with-
invariants
  [where  $A = 2 * A$  and  $R = R$ ,
    OF - R-pos source-factor k-pos - - run sound pre S-reaches below
    top-cap top-anti])
  show  $\Delta \leq 2 * A$ 
    using degree-factor by simp
next
fix i
assume  $i \leq l$ 
then show  $\text{bmssp-level-cap } k \ t \ i \leq \text{bmssp-level-cap } k \ t \ l$ 
  by (rule bmssp-level-cap-mono)
next
fix l' d S B D c-insert d' a betas bs B' Us U-loop c-loop
  child-costs U
assume D-def:
  D = label-partition-view
  (find-pivots-label-capped k (bmssp-level-cap k t l) d S B)
  (find-pivots-pivots-capped k (bmssp-level-cap k t l) d S B)
and insert:
  partition-initial-insert-cost-bound c-insert t
  (find-pivots-pivots-capped k (bmssp-level-cap k t l) d S B)
and loop:
  direct-insert-costed-partition-loop-state  $\Delta$  (bmssp-level-cap k t) t h k
  (bmssp-level-cap k t l) l'
  (find-pivots-label-capped k (bmssp-level-cap k t l) d S B)
  (find-pivots-pivots-capped k (bmssp-level-cap k t l) d S B) B d' D
  a betas bs B' Us U-loop c-loop child-costs
and complete:
  complete-on d'
  {v  $\in$  bound-tree S B'.
    find-pivots-label-capped k (bmssp-level-cap k t l) d S B v =
```

```

      dist s v
and U-def:
   $U = U\text{-loop} \cup \{v \in \text{bound-tree } S \ B'\}$ 
   $\text{find-pivots-label-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B \ v =$ 
     $\text{dist } s \ v$ 
and sound-s: sound-label d
and pre-s: bmssp-pre-full d S B
and reaches-s:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
and below-s:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$ 
and S-k-cap:  $k * \text{card } S \leq \text{bmssp-level-cap } k \ t \ l$ 
and anti: tree-antichain S
have seen-success:
   $B' = B \implies$ 
     $\text{card } (\text{find-pivots-seen-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B)$ 
     $\leq \text{card } U$ 
  by (rule direct-insert-costed-step-seen-success
    [OF loop U-def sound-s pre-s reaches-s below-s])
show fp-iter-capped-scan-cost k (bmssp-level-cap k t l) d S S B +
  c-insert  $\leq 2 * A * \text{card } U$ 
  by (rule direct-insert-costed-capped-step-scan-insert-budget-from-scaled-seen-or-
threshold
    [OF degree degree-factor insert-factor insert-scaled-factor
      seen-scaled-factor insert loop U-def sound-s pre-s reaches-s
      S-k-cap anti k-pos seen-success])
qed
have post-full:
  bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
  by (rule direct-insert-costed-bmssp-correct[OF run sound pre S-reaches])
have post: bmssp-post finite-initial-label {s} Infinity d' Infinity U
  using bmssp-post-full-imp-post[OF post-full] .
have U-V:  $U = V$ 
  using post bound-tree-source-infinity[OF all-reachable]
  unfolding bmssp-post-def by auto
have graph-bound:
  bmssp-amortized-cost-bound (2 * A) R h t l (2 * l + 1) Infinity U
   $\leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. h)$ 
   $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{vertex-count}$ 
proof -
  have U-card:  $\text{card } U = \text{vertex-count}$ 
  unfolding U-V vertex-count-def by simp
  have out-le:  $\text{card } (\text{outgoing-edges } U) \leq \text{edge-count}$ 
  by (rule edge-count-outgoing-bound)
  have range-le:  $\text{card } (\text{outgoing-edges-range } U \ 0 \ \text{Infinity}) \leq \text{edge-count}$ 
  by (rule card-outgoing-edges-range-le-edge-count)
  have out-term:
     $(R + l * h) * \text{card } (\text{outgoing-edges } U) \leq$ 
     $(R + l * h) * \text{edge-count}$ 
  using out-le by simp

```

```

have range-term:
   $t * \text{card } (\text{outgoing-edges-range } U \ 0 \ \text{Infinity}) \leq t * \text{edge-count}$ 
using range-le by simp
show ?thesis
unfolding bmssp-amortized-cost-bound-def bmssp-refined-graph-time-bound-def
using U-card out-term range-term by (simp add: algebra-simps; linarith)
qed
show  $c \leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. h)$ 
   $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{ vertex-count}$ 
using amortized graph-bound by linarith
qed

```

The remaining exact split-range results repeat the same source-progress closure without the direct-insert edge split. They are kept because some clients of the development may want the simpler range-costed interface, while the bucketed headline uses the more refined direct-insert and amortised path. The proof shape is the same: extract child calls, use strict progress to dominate child sources by child outputs, and close the level recurrence.

Having both interfaces documented makes the proof stack easier to audit. The exact split-range family is the clean mathematical recurrence, and the direct-insert family is the implementation-aware recurrence that exposes the bucketed partition's local costs.

theorem *exact-split-range-costed-partition-loop-state-closes-level-bound-from-child-costs*:

```

assumes run:
   $\text{exact-split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$ 
   $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs}$ 
and sound: sound-label  $d$ 
and pre: bmssp-pre-full  $d \ P \ B$ 
and P-reaches:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
and child-cost-bounds:
   $\text{list-all2 } (\lambda c\text{-child } U\text{-child.}$ 
     $c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child})$ 
     $\text{child-costs } (\text{range-tree-child-list } P \ a \ bs)$ 
and M-cap:  $M\text{-of } l \leq \text{cap}$ 
and source-factor:  $\text{Suc } h \leq A$ 
and k-pos:  $0 < k$ 
shows  $c \leq A * \text{Suc } L * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U)$ 
proof –
have split-run:
   $\text{split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$ 
   $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs}$ 
using exact-split-range-costed-refines-split-range-costed(1)[OF run] .
have trace: concrete-partition-loop-trace  $P \ B \ a \ bs \ d' \ B' \ Us \ U$ 
by (rule split-range-costed-partition-loop-state-trace
  [OF split-run sound pre P-reaches])
have children:
   $\text{list-all2 } (\lambda U \ X. U = X \wedge \text{complete-on } d' \ U) \ Us$ 

```

```

    (range-tree-chain-list P a bs B')
  using trace unfolding concrete-partition-loop-trace-def by blast
have U-def: U = bound-tree P (Fin a)  $\cup$   $\bigcup$  (set Us)
  using trace unfolding concrete-partition-loop-trace-def by blast
have Union-eq:
   $\bigcup$  (set Us) =  $\bigcup$  (set (range-tree-chain-list P a bs B'))
  using children by (induction rule: list-all2-induct) auto
have U-eq-chain:
  U = bound-tree P (Fin a)  $\cup$  range-tree-chain P a bs B'
  using U-def Union-eq Union-range-tree-chain-list[of P a bs B'] by simp
have finite-U: finite U
  using U-eq-chain by simp
have chain-subset: range-tree-chain P a bs B'  $\subseteq$  U
  using U-eq-chain by blast
have card-chain-le: card (range-tree-chain P a bs B')  $\leq$  card U
  by (rule card-mono[OF finite-U chain-subset])
have outgoing-subset:
  outgoing-edges (range-tree-chain P a bs B')  $\subseteq$  outgoing-edges U
  by (rule outgoing-edges-mono[OF chain-subset])
have finite-out-U: finite (outgoing-edges U)
  by simp
have card-out-le:
  card (outgoing-edges (range-tree-chain P a bs B'))  $\leq$ 
    card (outgoing-edges U)
  by (rule card-mono[OF finite-out-U outgoing-subset])
have loop:
  c  $\leq$ 
    A * L * card (range-tree-chain P a bs B') +
    (R + t) * card (outgoing-edges (range-tree-chain P a bs B')) +
    Suc h * card (range-tree-chain P a bs B')
proof (rule exact-split-range-costed-partition-loop-state-cost-from-child-and-source-
bounds
  [OF run child-cost-bounds])
fix S-child B-child d-child B-child' U-child c-child
assume child:
  exact-split-range-costed-bmssp  $\Delta$  M-of t h k cap l d S-child B-child
  d-child B-child' U-child c-child
and child-pre: bmssp-pre-full d S-child B-child
and child-reaches:  $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x$ 
and child-card: card S-child  $\leq$  M-of l
and child-below:  $\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child}$ 
have child-cap: card S-child  $\leq$  cap
  using child-card M-cap by linarith
show card S-child  $\leq$  card U-child
  by (rule exact-split-range-costed-bmssp-source-card-le-from-label-below-all-levels
    [OF child sound child-pre child-reaches child-below child-cap k-pos])
qed
have c  $\leq$ 
  A * L * card U + (R + t) * card (outgoing-edges U) + A * card U

```

```

proof –
  have vterm:
     $A * L * \text{card } (\text{range-tree-chain } P \ a \ bs \ B') \leq A * L * \text{card } U$ 
    using card-chain-le by simp
  have eterm:
     $(R + t) * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P \ a \ bs \ B')) \leq$ 
     $(R + t) * \text{card } (\text{outgoing-edges } U)$ 
    using card-out-le by simp
  have source-term:
     $\text{Suc } h * \text{card } (\text{range-tree-chain } P \ a \ bs \ B') \leq A * \text{card } U$ 
  proof –
    have  $\text{Suc } h * \text{card } (\text{range-tree-chain } P \ a \ bs \ B') \leq$ 
     $A * \text{card } (\text{range-tree-chain } P \ a \ bs \ B')$ 
    by (rule mult-right-mono[OF source-factor]) simp
    also have  $\dots \leq A * \text{card } U$ 
    using card-chain-le by simp
    finally show ?thesis .
  qed
  show ?thesis
    using loop vterm eterm source-term by linarith
  qed
  also have  $\dots =$ 
     $A * \text{Suc } L * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U)$ 
    by (simp add: algebra-simps)
  finally show ?thesis .
qed

```

theorem *exact-split-range-costed-partition-loop-state-closes-level-bound-from-child-costs-and-edge-ranges*:

```

assumes run:
  exact-split-range-costed-partition-loop-state  $\Delta$  M-of t h k cap l d P B d' D a
  betas bs B' Us U c child-costs
and sound: sound-label d
and pre: bmssp-pre-full d P B
and P-reaches:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
and child-cost-bounds:
  list-all2 ( $\lambda c\text{-child } U\text{-child}.$ 
     $c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$ 
     $\text{child-costs } (\text{range-tree-child-list } P \ a \ bs)$ )
and M-cap:  $M\text{-of } l \leq \text{cap}$ 
and source-factor:  $\text{Suc } h \leq A$ 
and k-pos:  $0 < k$ 
shows  $c \leq$ 
   $A * \text{Suc } L * \text{card } U + R * \text{card } (\text{outgoing-edges } U) +$ 
   $t * \text{sum-list } (\text{map } \text{card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$ 
proof –
  have split-run:
    split-range-costed-partition-loop-state  $\Delta$  M-of t h k cap l d P B d' D a
    betas bs B' Us U c child-costs

```



```

using exact-split-range-costed-refines-split-range-costed(1)[OF run] .
have trace: concrete-partition-loop-trace P B a bs d' B' Us U
by (rule split-range-costed-partition-loop-state-trace
  [OF split-run sound pre P-reaches])
have children:
  list-all2 ( $\lambda U X. U = X \wedge \text{complete-on } d' U$ ) Us
  (range-tree-chain-list P a bs B')
using trace unfolding concrete-partition-loop-trace-def by blast
have U-def:  $U = \text{bound-tree } P (\text{Fin } a) \cup \bigcup (\text{set } Us)$ 
using trace unfolding concrete-partition-loop-trace-def by blast
have Union-eq:
 $\bigcup (\text{set } Us) = \bigcup (\text{set } (\text{range-tree-chain-list } P a bs B'))$ 
using children by (induction rule: list-all2-induct) auto
have U-eq-chain:
 $U = \text{bound-tree } P (\text{Fin } a) \cup \text{range-tree-chain } P a bs B'$ 
using U-def Union-eq Union-range-tree-chain-list[of P a bs B'] by simp
have finite-U: finite U
using U-eq-chain by simp
have chain-subset:  $\text{range-tree-chain } P a bs B' \subseteq U$ 
using U-eq-chain by blast
have card-chain-le:  $\text{card } (\text{range-tree-chain } P a bs B') \leq \text{card } U$ 
by (rule card-mono[OF finite-U chain-subset])
have outgoing-subset:
 $\text{outgoing-edges } (\text{range-tree-chain } P a bs B') \subseteq \text{outgoing-edges } U$ 
by (rule outgoing-edges-mono[OF chain-subset])
have finite-out-U: finite (outgoing-edges U)
by simp
have card-out-le:
 $\text{card } (\text{outgoing-edges } (\text{range-tree-chain } P a bs B')) \leq$ 
 $\text{card } (\text{outgoing-edges } U)$ 
by (rule card-mono[OF finite-out-U outgoing-subset])
have loop:
 $c \leq$ 
 $A * L * \text{card } (\text{range-tree-chain } P a bs B') +$ 
 $R * \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P a bs B')) +$ 
 $t * \text{sum-list } (\text{map } \text{card } (\text{range-tree-child-edge-range-list } P a bs B')) +$ 
 $\text{Suc } h * \text{card } (\text{range-tree-chain } P a bs B')$ 
proof (rule exact-split-range-costed-partition-loop-state-cost-from-child-source-
and-edge-range-bounds
  [OF run sound child-cost-bounds])
fix S-child B-child d-child B-child' U-child c-child
assume child:
  exact-split-range-costed-bmssp  $\Delta M\text{-of } t h k \text{ cap } l d S\text{-child } B\text{-child}$ 
  d-child B-child' U-child c-child
and child-pre: bmssp-pre-full d S-child B-child
and child-reaches:  $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s x$ 
and child-card:  $\text{card } S\text{-child} \leq M\text{-of } l$ 
and child-below:  $\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d x) B\text{-child}$ 
have child-cap:  $\text{card } S\text{-child} \leq \text{cap}$ 

```

```

    using child-card M-cap by linarith
  show card S-child ≤ card U-child
    by (rule exact-split-range-costed-bmssp-source-card-le-from-label-below-all-levels
        [OF child sound child-pre child-reaches child-below child-cap k-pos])
  qed
  have c ≤
    A * L * card U + R * card (outgoing-edges U) +
    t * sum-list (map card (range-tree-child-edge-range-list P a betas bs)) +
    A * card U
  proof -
    have vterm:
      A * L * card (range-tree-chain P a bs B') ≤ A * L * card U
      using card-chain-le by simp
    have eterm:
      R * card (outgoing-edges (range-tree-chain P a bs B')) ≤
      R * card (outgoing-edges U)
      using card-out-le by simp
    have source-term:
      Suc h * card (range-tree-chain P a bs B') ≤ A * card U
  proof -
    have Suc h * card (range-tree-chain P a bs B') ≤
      A * card (range-tree-chain P a bs B')
      by (rule mult-right-mono[OF source-factor]) simp
    also have ... ≤ A * card U
      using card-chain-le by simp
    finally show ?thesis .
  qed
  show ?thesis
    using loop vterm eterm source-term by linarith
  qed
  also have ... =
    A * Suc L * card U + R * card (outgoing-edges U) +
    t * sum-list (map card (range-tree-child-edge-range-list P a betas bs))
    by (simp add: algebra-simps)
  finally show ?thesis .
  qed

theorem exact-split-range-costed-partition-loop-state-closes-level-bound-from-child-
bound:
  assumes run:
    exact-split-range-costed-partition-loop-state Δ M-of t h k cap l d P B d' D a
    betas bs B' Us U c child-costs
  and sound: sound-label d
  and pre: bmssp-pre-full d P B
  and P-reaches:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
  and child-bound:
     $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
     $\llbracket \text{exact-split-range-costed-bmssp } \Delta \text{ M-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$ 
     $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$ 

```

$bmssp\text{-}pre\text{-}full\ d\ S\text{-}child\ B\text{-}child;$
 $\bigwedge x. x \in S\text{-}child \implies reachable\ s\ x;$
 $card\ S\text{-}child \leq M\text{-}of\ l;$
 $\bigwedge x. x \in S\text{-}child \implies below\text{-}bound\ (d\ x)\ B\text{-}child]$
 $\implies c\text{-}child \leq level\text{-}range\text{-}cost\text{-}bound\ A\ R\ L\ U\text{-}child$
and $M\text{-}cap: M\text{-}of\ l \leq cap$
and $source\text{-}factor: Suc\ h \leq A$
and $k\text{-}pos: 0 < k$
shows $c \leq A * Suc\ L * card\ U + (R + t) * card\ (outgoing\text{-}edges\ U)$
proof –
have $child\text{-}cost\text{-}bounds:$
 $list\text{-}all2\ (\lambda c\text{-}child\ U\text{-}child.$
 $c\text{-}child \leq level\text{-}range\text{-}cost\text{-}bound\ A\ R\ L\ U\text{-}child)$
 $child\text{-}costs\ (range\text{-}tree\text{-}child\text{-}list\ P\ a\ bs)$
by $(rule\ exact\text{-}split\text{-}range\text{-}costed\text{-}partition\text{-}loop\text{-}state\text{-}child\text{-}cost\text{-}bounds$
 $[OF\ run\ child\text{-}bound])$
show $?thesis$
by $(rule\ exact\text{-}split\text{-}range\text{-}costed\text{-}partition\text{-}loop\text{-}state\text{-}closes\text{-}level\text{-}bound\text{-}from\text{-}child\text{-}costs$
 $[OF\ run\ sound\ pre\ P\text{-}reaches\ child\text{-}cost\text{-}bounds\ M\text{-}cap\ source\text{-}factor\ k\text{-}pos])$
qed

theorem $exact\text{-}split\text{-}range\text{-}costed\text{-}partition\text{-}loop\text{-}state\text{-}closes\text{-}level\text{-}bound\text{-}from\text{-}child\text{-}bound\text{-}and\text{-}edge\text{-}ranges:$

assumes $run:$
 $exact\text{-}split\text{-}range\text{-}costed\text{-}partition\text{-}loop\text{-}state\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ l\ d\ P\ B\ d'\ D\ a$
 $betas\ bs\ B'\ Us\ U\ c\ child\text{-}costs$
and $sound: sound\text{-}label\ d$
and $pre: bmssp\text{-}pre\text{-}full\ d\ P\ B$
and $P\text{-}reaches: \bigwedge x. x \in P \implies reachable\ s\ x$
and $child\text{-}bound:$
 $\bigwedge c\text{-}child\ U\text{-}child\ S\text{-}child\ B\text{-}child\ d\text{-}child\ B\text{-}child'.$
 $\llbracket exact\text{-}split\text{-}range\text{-}costed\text{-}bmssp\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ l\ d\ S\text{-}child\ B\text{-}child$
 $d\text{-}child\ B\text{-}child'\ U\text{-}child\ c\text{-}child;$
 $bmssp\text{-}pre\text{-}full\ d\ S\text{-}child\ B\text{-}child;$
 $\bigwedge x. x \in S\text{-}child \implies reachable\ s\ x;$
 $card\ S\text{-}child \leq M\text{-}of\ l;$
 $\bigwedge x. x \in S\text{-}child \implies below\text{-}bound\ (d\ x)\ B\text{-}child]$
 $\implies c\text{-}child \leq level\text{-}range\text{-}cost\text{-}bound\ A\ R\ L\ U\text{-}child$
and $M\text{-}cap: M\text{-}of\ l \leq cap$
and $source\text{-}factor: Suc\ h \leq A$
and $k\text{-}pos: 0 < k$
shows $c \leq$
 $A * Suc\ L * card\ U + R * card\ (outgoing\text{-}edges\ U) +$
 $t * sum\text{-}list\ (map\ card\ (range\text{-}tree\text{-}child\text{-}edge\text{-}range\text{-}list\ P\ a\ betas\ bs))$

proof –
have $child\text{-}cost\text{-}bounds:$
 $list\text{-}all2\ (\lambda c\text{-}child\ U\text{-}child.$
 $c\text{-}child \leq level\text{-}range\text{-}cost\text{-}bound\ A\ R\ L\ U\text{-}child)$

child-costs (*range-tree-child-list* *P a bs*)
by (*rule exact-split-range-costed-partition-loop-state-child-cost-bounds*
 $[OF \text{ run } child-bound]$)
show *?thesis*
by (*rule exact-split-range-costed-partition-loop-state-closes-level-bound-from-child-costs-and-edge-ranges*
 $[OF \text{ run } sound \text{ pre } P-reaches \text{ child-cost-bounds } M-cap \text{ source-factor } k-pos]$)
qed

theorem *exact-split-range-costed-partition-loop-state-closes-level-bound-from-child-bound-with-invariants*:

assumes *run*:
exact-split-range-costed-partition-loop-state Δ *M-of t h k cap l d P B d' D a*
betas bs B' Us U c child-costs
and *sound*: *sound-label d*
and *pre*: *bmssp-pre-full d P B*
and *P-reaches*: $\bigwedge x. x \in P \implies reachable \ s \ x$
and *P-k-cap*: $k * card \ P \leq cap$
and *P-anti*: *tree-antichain P*
and *child-bound*:
 $\bigwedge c-child \ U-child \ S-child \ B-child \ d-child \ B-child'.$
 $\llbracket exact-split-range-costed-bmssp \ \Delta \ M-of \ t \ h \ k \ cap \ l \ d \ S-child \ B-child$
 $\quad d-child \ B-child' \ U-child \ c-child;$
 $\quad bmssp-pre-full \ d \ S-child \ B-child;$
 $\quad \bigwedge x. x \in S-child \implies reachable \ s \ x;$
 $\quad card \ S-child \leq M-of \ l;$
 $\quad \bigwedge x. x \in S-child \implies below-bound \ (d \ x) \ B-child;$
 $\quad k * card \ S-child \leq cap;$
 $\quad tree-antichain \ S-child \rrbracket$
 $\implies c-child \leq level-range-cost-bound \ A \ R \ L \ U-child$
and *M-cap*: $M-of \ l \leq cap$
and *source-factor*: $Suc \ h \leq A$
and *k-pos*: $0 < k$
shows $c \leq A * Suc \ L * card \ U + (R + t) * card \ (outgoing-edges \ U)$
proof –
have *P-subset*: $P \subseteq V$
using *pre* **unfolding** *bmssp-pre-full-def* **by** *blast*
have *child-cost-bounds*:
 $list-all2 \ (\lambda c-child \ U-child.$
 $\quad c-child \leq level-range-cost-bound \ A \ R \ L \ U-child)$
 $\quad child-costs \ (range-tree-child-list \ P \ a \ bs)$
by (*rule exact-split-range-costed-partition-loop-state-child-cost-bounds-with-invariants*
 $[OF \text{ run } P-subset \ P-k-cap \ P-anti \ child-bound]$)
show *?thesis*
by (*rule exact-split-range-costed-partition-loop-state-closes-level-bound-from-child-costs*
 $[OF \text{ run } sound \text{ pre } P-reaches \text{ child-cost-bounds } M-cap \text{ source-factor } k-pos]$)
qed

theorem *exact-split-range-costed-nonbase-step-closes-level-bound-from-child-bound:*

assumes *loop:*

exact-split-range-costed-partition-loop-state Δ *M-of t h k cap l d P B d' D a*
betas bs B' Us U-loop c-loop child-costs

and *sound:* *sound-label d*

and *pre:* *bmssp-pre-full d P B*

and *P-reaches:* $\bigwedge x. x \in P \implies \text{reachable } s \ x$

and *child-bound:*

$\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$

$\llbracket \text{exact-split-range-costed-bmssp } \Delta \text{ M-of t h k cap l d S-child B-child}$

d-child B-child' U-child c-child;

bmssp-pre-full d S-child B-child;

$\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$

card S-child $\leq$ *M-of l;*

$\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child} \rrbracket$

$\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$

and *M-cap:* *M-of l* $\leq$ *cap*

and *source-factor:* *Suc h* $\leq$ *A*

and *k-pos:* $0 < k$

and *U-def:* $U = U\text{-loop} \cup W$

and *finite-U:* *finite U*

and *scan-insert:* *c-scan-insert* $\leq A * \text{card } U$

and *c-def:* $c = c\text{-scan-insert} + c\text{-loop}$

shows $c \leq A * \text{Suc } (\text{Suc } L) * \text{card } U +$

$(R + t) * \text{card } (\text{outgoing-edges } U)$

proof –

have *loop-bound:*

$c\text{-loop} \leq A * \text{Suc } L * \text{card } U\text{-loop} +$

$(R + t) * \text{card } (\text{outgoing-edges } U\text{-loop})$

by (rule *exact-split-range-costed-partition-loop-state-closes-level-bound-from-child-bound*)

$[OF \text{ loop sound pre P-reaches child-bound M-cap source-factor k-pos}]$

have *U-loop-subset:* $U\text{-loop} \subseteq U$

using *U-def* **by** *blast*

have *card-loop-le:* $\text{card } U\text{-loop} \leq \text{card } U$

by (rule *card-mono*[*OF finite-U U-loop-subset*])

have *outgoing-subset:* $\text{outgoing-edges } U\text{-loop} \subseteq \text{outgoing-edges } U$

by (rule *outgoing-edges-mono*[*OF U-loop-subset*])

have *finite-out-U:* *finite (outgoing-edges U)*

by *simp*

have *card-out-le:* $\text{card } (\text{outgoing-edges } U\text{-loop}) \leq \text{card } (\text{outgoing-edges } U)$

by (rule *card-mono*[*OF finite-out-U outgoing-subset*])

have $c \leq$

$A * \text{card } U +$

$(A * \text{Suc } L * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U))$

proof –

have *loop-to-U:*

$c\text{-loop} \leq A * \text{Suc } L * \text{card } U +$

$(R + t) * \text{card } (\text{outgoing-edges } U)$
proof –
 have $A * \text{Suc } L * \text{card } U\text{-loop} \leq A * \text{Suc } L * \text{card } U$
 using *card-loop-le* **by** *simp*
 moreover have $(R + t) * \text{card } (\text{outgoing-edges } U\text{-loop}) \leq$
 $(R + t) * \text{card } (\text{outgoing-edges } U)$
 using *card-out-le* **by** *simp*
 ultimately show *?thesis*
 using *loop-bound* **by** *linarith*
qed
 show *?thesis*
 using *scan-insert loop-to-U c-def* **by** *linarith*
qed
 also have $\dots =$
 $A * \text{Suc } (\text{Suc } L) * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U)$
by (*simp add: algebra-simps*)
 finally show *?thesis* .
qed

theorem *exact-split-range-costed-nonbase-step-closes-level-bound-from-child-bound-and-edge-ranges:*

assumes *loop:*

exact-split-range-costed-partition-loop-state Δ *M-of t h k cap l d P B d' D a*
betas bs B' Us U-loop c-loop child-costs

and *sound:* *sound-label d*

and *pre:* *bmssp-pre-full d P B*

and *P-reaches:* $\bigwedge x. x \in P \implies \text{reachable } s x$

and *child-bound:*

$\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$

$\llbracket \text{exact-split-range-costed-bmssp } \Delta \text{ M-of t h k cap l d S-child B-child}$

$d\text{-child } B\text{-child}' U\text{-child } c\text{-child};$

$\text{bmssp-pre-full } d S\text{-child } B\text{-child};$

$\bigwedge x. x \in S\text{-child} \implies \text{reachable } s x;$

$\text{card } S\text{-child} \leq \text{M-of } l;$

$\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d x) B\text{-child} \rrbracket$

$\implies c\text{-child} \leq \text{level-range-cost-bound } A R L U\text{-child}$

and *M-cap:* $\text{M-of } l \leq \text{cap}$

and *source-factor:* $\text{Suc } h \leq A$

and *k-pos:* $0 < k$

and *U-def:* $U = U\text{-loop} \cup W$

and *finite-U:* *finite U*

and *scan-insert:* $c\text{-scan-insert} \leq A * \text{card } U$

and *c-def:* $c = c\text{-scan-insert} + c\text{-loop}$

shows $c \leq A * \text{Suc } (\text{Suc } L) * \text{card } U +$

$R * \text{card } (\text{outgoing-edges } U) +$

$t * \text{sum-list } (\text{map } \text{card } (\text{range-tree-child-edge-range-list } P a \text{ betas } bs))$

proof –

have *loop-bound:*

$c\text{-loop} \leq$

```

    A * Suc L * card U-loop + R * card (outgoing-edges U-loop) +
    t * sum-list (map card (range-tree-child-edge-range-list P a betas bs))
  by (rule exact-split-range-costed-partition-loop-state-closes-level-bound-from-child-
bound-and-edge-ranges
    [OF loop sound pre P-reaches child-bound M-cap source-factor k-pos])
  have U-loop-subset: U-loop  $\subseteq$  U
  using U-def by blast
  have card-loop-le: card U-loop  $\leq$  card U
  by (rule card-mono[OF finite-U U-loop-subset])
  have outgoing-subset: outgoing-edges U-loop  $\subseteq$  outgoing-edges U
  by (rule outgoing-edges-mono[OF U-loop-subset])
  have finite-out-U: finite (outgoing-edges U)
  by simp
  have card-out-le: card (outgoing-edges U-loop)  $\leq$  card (outgoing-edges U)
  by (rule card-mono[OF finite-out-U outgoing-subset])
  have c  $\leq$ 
    A * card U +
    (A * Suc L * card U + R * card (outgoing-edges U) +
    t * sum-list (map card (range-tree-child-edge-range-list P a betas bs)))
  proof -
    have loop-to-U:
      c-loop  $\leq$ 
      A * Suc L * card U + R * card (outgoing-edges U) +
      t * sum-list (map card (range-tree-child-edge-range-list P a betas bs))
    proof -
      have A * Suc L * card U-loop  $\leq$  A * Suc L * card U
      using card-loop-le by simp
      moreover have R * card (outgoing-edges U-loop)  $\leq$ 
        R * card (outgoing-edges U)
      using card-out-le by simp
      ultimately show ?thesis
      using loop-bound by linarith
    qed
  qed
  show ?thesis
  using scan-insert loop-to-U c-def by linarith
qed
also have ... =
  A * Suc (Suc L) * card U + R * card (outgoing-edges U) +
  t * sum-list (map card (range-tree-child-edge-range-list P a betas bs))
  by (simp add: algebra-simps)
  finally show ?thesis .
qed

```

theorem *exact-split-range-costed-nonbase-step-closes-level-bound-from-child-bound-with-invariants:*

assumes *loop:*

exact-split-range-costed-partition-loop-state Δ M-of t h k cap l d P B d' D a
betas bs B' Us U-loop c-loop child-costs

and *sound: sound-label d*

and *pre*: *bmssp-pre-full* *d P B*
and *P-reaches*: $\bigwedge x. x \in P \implies \text{reachable } s \ x$
and *P-k-cap*: $k * \text{card } P \leq \text{cap}$
and *P-anti*: *tree-antichain* *P*
and *child-bound*:
 $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\llbracket \text{exact-split-range-costed-bmssp } \Delta \text{ } M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$
 $\quad \text{bmssp-pre-full } d \ S\text{-child } B\text{-child};$
 $\quad \bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\quad \text{card } S\text{-child} \leq M\text{-of } l;$
 $\quad \bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child};$
 $\quad k * \text{card } S\text{-child} \leq \text{cap};$
 $\quad \text{tree-antichain } S\text{-child} \rrbracket$
 $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$
and *M-cap*: $M\text{-of } l \leq \text{cap}$
and *source-factor*: $\text{Suc } h \leq A$
and *k-pos*: $0 < k$
and *U-def*: $U = U\text{-loop} \cup W$
and *finite-U*: *finite* *U*
and *scan-insert*: $c\text{-scan-insert} \leq A * \text{card } U$
and *c-def*: $c = c\text{-scan-insert} + c\text{-loop}$
shows $c \leq A * \text{Suc } (\text{Suc } L) * \text{card } U +$
 $(R + t) * \text{card } (\text{outgoing-edges } U)$
proof –
have *loop-bound*:
 $c\text{-loop} \leq A * \text{Suc } L * \text{card } U\text{-loop} +$
 $(R + t) * \text{card } (\text{outgoing-edges } U\text{-loop})$
by (rule *exact-split-range-costed-partition-loop-state-closes-level-bound-from-child-bound-with-invariants*
 $\llbracket \text{OF loop sound pre } P\text{-reaches } P\text{-k-cap } P\text{-anti child-bound } M\text{-cap}$
 $\quad \text{source-factor } k\text{-pos} \rrbracket$)
have *U-loop-subset*: $U\text{-loop} \subseteq U$
using *U-def* **by** *blast*
have *card-loop-le*: $\text{card } U\text{-loop} \leq \text{card } U$
by (rule *card-mono*[*OF finite-U U-loop-subset*])
have *outgoing-subset*: $\text{outgoing-edges } U\text{-loop} \subseteq \text{outgoing-edges } U$
by (rule *outgoing-edges-mono*[*OF U-loop-subset*])
have *finite-out-U*: *finite* (*outgoing-edges* *U*)
by *simp*
have *card-out-le*: $\text{card } (\text{outgoing-edges } U\text{-loop}) \leq \text{card } (\text{outgoing-edges } U)$
by (rule *card-mono*[*OF finite-out-U outgoing-subset*])
have $c \leq$
 $A * \text{card } U +$
 $(A * \text{Suc } L * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U))$
proof –
have *loop-to-U*:
 $c\text{-loop} \leq A * \text{Suc } L * \text{card } U +$
 $(R + t) * \text{card } (\text{outgoing-edges } U)$

proof –
have $A * \text{Suc } L * \text{card } U\text{-loop} \leq A * \text{Suc } L * \text{card } U$
using *card-loop-le* **by** *simp*
moreover have $(R + t) * \text{card } (\text{outgoing-edges } U\text{-loop}) \leq$
 $(R + t) * \text{card } (\text{outgoing-edges } U)$
using *card-out-le* **by** *simp*
ultimately show *?thesis*
using *loop-bound* **by** *linarith*
qed
show *?thesis*
using *scan-insert loop-to-U c-def* **by** *linarith*
qed
also have $\dots =$
 $A * \text{Suc } (\text{Suc } L) * \text{card } U + (R + t) * \text{card } (\text{outgoing-edges } U)$
by (*simp add: algebra-simps*)
finally show *?thesis* .
qed

theorem *exact-split-range-costed-bmssp-level-bound-from-local-budgets:*
assumes *base-budget:* $\Delta \leq A$
and *R-pos:* $0 < R$
and *source-factor:* $\text{Suc } h \leq A$
and *k-pos:* $0 < k$
and *M-cap:* $\bigwedge i. i \leq l \implies M\text{-of } i \leq \text{cap}$
and *step-budget:*
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
 $\text{child-costs } U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{exact-split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $(\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B) \implies$
 $\text{fp-iter-capped-scan-cost } k \ \text{cap } d \ S \ S \ B + c\text{-insert} \leq A * \text{card } U$
and *run:*
 $\text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c$
and *sound:* $\text{sound-label } d$
and *pre:* $\text{bmssp-pre-full } d \ S \ B$

```

and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
and below:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$ 
shows  $c \leq \text{level-range-cost-bound } A \ (R + l * t) \ (2 * l + 1) \ U$ 
using run sound pre S-reaches below M-cap
proof (induction l arbitrary: d S B d' B' U c rule: less-induct)
  case (less l)
  show ?case
  proof (cases l)
    case 0
    have run0:
      exact-split-range-costed-bmssp  $\Delta$  M-of t h k cap 0 d S B d' B' U c
      using less.prem(1) 0 by simp
    have split-run0:
      split-range-costed-bmssp  $\Delta$  M-of t h k cap 0 d S B d' B' U c
      using exact-split-range-costed-refines-split-range-costed(2)[OF run0] .
    show ?thesis
    using split-run0 R-pos 0
    by (cases rule: split-range-costed-bmssp.cases)
      (simp-all add: split-range-costed-base-level-bound)
  next
  case (Suc l0)
  have run-suc:
    exact-split-range-costed-bmssp  $\Delta$  M-of t h k cap (Suc l0) d S B d' B' U c
    using less.prem(1) Suc by simp
  show ?thesis
  proof (cases rule: exact-split-range-costed-bmssp-SucE[OF run-suc])
    case (1 c-insert a betas bs Us U-loop c-loop child-costs-loop)
    let ?d-fp = find-pivots-label-capped k cap d S B
    let ?P = find-pivots-pivots-capped k cap d S B
    let ?W =  $\{v \in \text{bound-tree } S \ B'. \ ?d\text{-fp } v = \text{dist } s \ v\}$ 
    have sound-fp: sound-label ?d-fp
      unfolding find-pivots-label-capped-def
      by (rule fp-iter-capped-label-sound[OF less.prem(2) less.prem(4)])
    have pivot-pre: bmssp-pre-full ?d-fp ?P B
      using find-pivots-capped-establishes-pivot-pre-concrete
        [OF less.prem(2) less.prem(3) less.prem(4)] .
    have P-subset-S:  $?P \subseteq S$ 
      unfolding find-pivots-pivots-capped-def by (auto split: if-splits)
    have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
      using P-subset-S less.prem(4) by blast
    have child-bound:
       $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
       $\llbracket \text{exact-split-range-costed-bmssp } \Delta \text{ M-of t h k cap l0 ?d-fp } S\text{-child } B\text{-child}$ 
       $\text{d-child } B\text{-child}' \ U\text{-child } c\text{-child};$ 
       $\text{bmssp-pre-full ?d-fp } S\text{-child } B\text{-child};$ 
       $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$ 
       $\text{card } S\text{-child} \leq \text{M-of l0};$ 
       $\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (?d\text{-fp } x) \ B\text{-child} \rrbracket$ 
       $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ (R + l0 * t)$ 

```

$(2 * l0 + 1) \text{ } U\text{-child}$
proof –
fix $c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'$
assume $child$:
 $exact\text{-split-range-costed-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ } cap \text{ } l0 \text{ } ?d\text{-fp } S\text{-child } B\text{-child}$
 $d\text{-child } B\text{-child}' \text{ } U\text{-child } c\text{-child}$
and $child\text{-pre}$: $bmssp\text{-pre-full } ?d\text{-fp } S\text{-child } B\text{-child}$
and $child\text{-reaches}$: $\bigwedge x. x \in S\text{-child} \implies reachable \text{ } s \text{ } x$
and $child\text{-below}$: $\bigwedge x. x \in S\text{-child} \implies below\text{-bound } (?d\text{-fp } x) \text{ } B\text{-child}$
have $M\text{-cap-child}$: $\bigwedge i. i \leq l0 \implies M\text{-of } i \leq cap$
using $less.premis(6) \text{ } Suc$ **by** $simp$
show $c\text{-child} \leq level\text{-range-cost-bound } A \text{ } (R + l0 * t)$
 $(2 * l0 + 1) \text{ } U\text{-child}$
by ($rule \text{ } less.IH[of \text{ } l0 \text{ } ?d\text{-fp } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'$
 $U\text{-child } c\text{-child}, OF - child \text{ } sound\text{-fp } child\text{-pre } child\text{-reaches}$
 $child\text{-below } M\text{-cap-child}]$)
 $(simp \text{ } add: Suc)$
qed
have $scan\text{-insert}$:
 $fp\text{-iter-capped-scan-cost } k \text{ } cap \text{ } d \text{ } S \text{ } S \text{ } B + c\text{-insert} \leq$
 $A * card \text{ } U$
by ($rule \text{ } step\text{-budget}[OF \text{ } refl \text{ } 1(3) \text{ } 1(4) \text{ } 1(5) \text{ } 1(1)$
 $less.premis(2) \text{ } less.premis(3) \text{ } less.premis(4) \text{ } less.premis(5)]$)
have $trace$:
 $concrete\text{-partition-loop-trace } ?P \text{ } B \text{ } a \text{ } bs \text{ } d' \text{ } B' \text{ } Us \text{ } U\text{-loop}$
proof –
have $split\text{-loop}$:
 $split\text{-range-costed-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ } cap \text{ } l0$
 $?d\text{-fp } ?P \text{ } B \text{ } d'$
 $(label\text{-partition-view } ?d\text{-fp } ?P) \text{ } a \text{ } betas \text{ } bs \text{ } B' \text{ } Us \text{ } U\text{-loop } c\text{-loop}$
 $child\text{-costs-loop}$
using $exact\text{-split-range-costed-refines-split-range-costed}(1)[OF \text{ } 1(4)]$
by $simp$
show $?thesis$
by ($rule \text{ } split\text{-range-costed-partition-loop-state-trace}$
 $[OF \text{ } split\text{-loop } sound\text{-fp } pivot\text{-pre } P\text{-reaches}]$)
qed
have $loop\text{-post}$: $bmssp\text{-post-full } ?d\text{-fp } ?P \text{ } B \text{ } d' \text{ } B' \text{ } U\text{-loop}$
by ($rule \text{ } concrete\text{-partition-loop-trace-post}[OF \text{ } trace]$)
have $finite\text{-U-loop}$: $finite \text{ } U\text{-loop}$
using $loop\text{-post} \text{ } unfolding \text{ } bmssp\text{-post-full-def}$ **by** $simp$
have $finite\text{-W}$: $finite \text{ } ?W$
by $simp$
have $finite\text{-U}$: $finite \text{ } U$
using $1(1) \text{ } finite\text{-U-loop} \text{ } finite\text{-W}$ **by** $simp$
have $M\text{-cap-l0}$: $M\text{-of } l0 \leq cap$
using $less.premis(6) \text{ } Suc$ **by** $simp$
have $step$:
 $c \leq A * Suc \text{ } (Suc \text{ } (2 * l0 + 1)) * card \text{ } U +$

```

      (R + l0 * t + t) * card (outgoing-edges U)
    by (rule exact-split-range-costed-nonbase-step-closes-level-bound-from-child-
bound
      [OF 1(4) sound-fp pivot-pre P-reaches child-bound M-cap-l0
        source-factor k-pos 1(1) finite-U scan-insert 1(2)])
    show ?thesis
    using step Suc unfolding level-range-cost-bound-def
    by (simp add: algebra-simps)
  qed
qed
qed

```

theorem *exact-split-range-costed-bmssp-level-bound-from-local-budgets-with-invariants:*

```

assumes base-budget:  $\Delta \leq A$ 
and R-pos:  $0 < R$ 
and source-factor:  $\text{Suc } h \leq A$ 
and k-pos:  $0 < k$ 
and M-cap:  $\bigwedge i. i \leq l \implies M\text{-of } i \leq \text{cap}$ 
and step-budget:
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$ 
 $\text{child-costs } U.$ 
 $D = \text{label-partition-view}$ 
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$ 
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$ 
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$ 
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$ 
 $\text{exact-split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l$ 
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$ 
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$ 
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$ 
 $\text{complete-on } d'$ 
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$ 
 $U = U\text{-loop} \cup$ 
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$ 
 $\text{sound-label } d \implies$ 
 $\text{bmssp-pre-full } d \ S \ B \implies$ 
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$ 
 $(\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B) \implies$ 
 $k * \text{card } S \leq \text{cap} \implies$ 
 $\text{tree-antichain } S \implies$ 
 $\text{fp-iter-capped-scan-cost } k \ \text{cap } d \ S \ S \ B + c\text{-insert} \leq A * \text{card } U$ 
and run:
 $\text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c$ 
and sound:  $\text{sound-label } d$ 
and pre:  $\text{bmssp-pre-full } d \ S \ B$ 
and S-reaches:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
and below:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$ 
and S-k-cap:  $k * \text{card } S \leq \text{cap}$ 

```

```

  and S-anti: tree-antichain S
  shows  $c \leq \text{level-range-cost-bound } A \ (R + l * t) \ (2 * l + 1) \ U$ 
  using run sound pre S-reaches below S-k-cap S-anti M-cap
  proof (induction l arbitrary:  $d \ S \ B \ d' \ B' \ U \ c$  rule: less-induct)
  case (less l)
  show ?case
  proof (cases l)
  case 0
  have run0:
    exact-split-range-costed-bmssp  $\Delta$  M-of t h k cap 0 d S B d' B' U c
    using less.prems(1) 0 by simp
  have split-run0:
    split-range-costed-bmssp  $\Delta$  M-of t h k cap 0 d S B d' B' U c
    using exact-split-range-costed-refines-split-range-costed(2)[OF run0] .
  show ?thesis
  using split-run0 R-pos 0
  by (cases rule: split-range-costed-bmssp.cases)
    (simp-all add: split-range-costed-base-level-bound)
next
case (Suc l0)
have run-suc:
  exact-split-range-costed-bmssp  $\Delta$  M-of t h k cap (Suc l0) d S B d' B' U c
  using less.prems(1) Suc by simp
show ?thesis
proof (cases rule: exact-split-range-costed-bmssp-SucE[OF run-suc])
case (1 c-insert a betas bs Us U-loop c-loop child-costs-loop)
let ?d-fp = find-pivots-label-capped k cap d S B
let ?P = find-pivots-pivots-capped k cap d S B
let ?W =  $\{v \in \text{bound-tree } S \ B'. \ ?d\text{-fp } v = \text{dist } s \ v\}$ 
have sound-fp: sound-label ?d-fp
  unfolding find-pivots-label-capped-def
  by (rule fp-iter-capped-label-sound[OF less.prems(2) less.prems(4)])
have pivot-pre: bmssp-pre-full ?d-fp ?P B
  using find-pivots-capped-establishes-pivot-pre-concrete
    [OF less.prems(2) less.prems(3) less.prems(4)] .
have S-subset:  $S \subseteq V$ 
  using less.prems(3) unfolding bmssp-pre-full-def by blast
have P-subset-S:  $?P \subseteq S$ 
  unfolding find-pivots-pivots-capped-def by (auto split: if-splits)
have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
  using P-subset-S less.prems(4) by blast
have P-k-cap:  $k * \text{card } ?P \leq \text{cap}$ 
  by (rule find-pivots-pivots-capped-scaled-card-le
    [OF S-subset less.prems(6)])
have P-anti: tree-antichain ?P
  by (rule find-pivots-pivots-capped-tree-antichain[OF less.prems(7)])
have child-bound:
   $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
   $\llbracket \text{exact-split-range-costed-bmssp } \Delta \text{ } M\text{-of } t \ h \ k \ \text{cap } l0 \ ?d\text{-fp } S\text{-child } B\text{-child}$ 

```

```

    d-child B-child' U-child c-child;
    bmssp-pre-full ?d-fp S-child B-child;
     $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$ 
    card S-child  $\leq$  M-of l0;
     $\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (?d\text{-fp } x) \ B\text{-child};$ 
     $k * \text{card } S\text{-child} \leq \text{cap};$ 
    tree-antichain S-child]
 $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ (R + l0 * t)$ 
     $(2 * l0 + 1) \ U\text{-child}$ 
proof –
  fix c-child U-child S-child B-child d-child B-child'
  assume child:
    exact-split-range-costed-bmssp  $\Delta$  M-of t h k cap l0 ?d-fp S-child B-child
    d-child B-child' U-child c-child
  and child-pre: bmssp-pre-full ?d-fp S-child B-child
  and child-reaches:  $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x$ 
  and child-below:
     $\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (?d\text{-fp } x) \ B\text{-child}$ 
  and child-k-cap:  $k * \text{card } S\text{-child} \leq \text{cap}$ 
  and child-anti: tree-antichain S-child
  have M-cap-child:  $\bigwedge i. i \leq l0 \implies \text{M-of } i \leq \text{cap}$ 
  using less.premis(8) Suc by simp
  show c-child  $\leq \text{level-range-cost-bound } A \ (R + l0 * t)$ 
     $(2 * l0 + 1) \ U\text{-child}$ 
  by (rule less.IH[of l0 ?d-fp S-child B-child d-child B-child'
    U-child c-child, OF - child sound-fp child-pre child-reaches
    child-below child-k-cap child-anti M-cap-child])
    (simp add: Suc)
qed
have scan-insert:
  fp-iter-capped-scan-cost k cap d S S B + c-insert  $\leq$ 
    A * card U
  by (rule step-budget[OF refl 1(3) 1(4) 1(5) 1(1)
    less.premis(2) less.premis(3) less.premis(4) less.premis(5)
    less.premis(6) less.premis(7)])
have trace:
  concrete-partition-loop-trace ?P B a bs d' B' Us U-loop
proof –
  have split-loop:
    split-range-costed-partition-loop-state  $\Delta$  M-of t h k cap l0
    ?d-fp ?P B d'
    (label-partition-view ?d-fp ?P) a betas bs B' Us U-loop c-loop
    child-costs-loop
  using exact-split-range-costed-refines-split-range-costed(1)[OF 1(4)]
  by simp
  show ?thesis
  by (rule split-range-costed-partition-loop-state-trace
    [OF split-loop sound-fp pivot-pre P-reaches])
qed

```

```

have loop-post: bmssp-post-full ?d-fp ?P B d' B' U-loop
  by (rule concrete-partition-loop-trace-post[OF trace])
have finite-U-loop: finite U-loop
  using loop-post unfolding bmssp-post-full-def by simp
have finite-W: finite ?W
  by simp
have finite-U: finite U
  using 1(1) finite-U-loop finite-W by simp
have M-cap-l0: M-of l0 ≤ cap
  using less.prems(8) Suc by simp
have step:
  c ≤ A * Suc (Suc (2 * l0 + 1)) * card U +
    (R + l0 * t + t) * card (outgoing-edges U)
  by (rule exact-split-range-costed-nonbase-step-closes-level-bound-from-child-
bound-with-invariants
    [OF 1(4) sound-fp pivot-pre P-reaches P-k-cap P-anti child-bound
      M-cap-l0 source-factor k-pos 1(1) finite-U scan-insert 1(2)])
show ?thesis
  using step Suc unfolding level-range-cost-bound-def
  by (simp add: algebra-simps)
qed
qed
qed

```

theorem *finite-initial-label-exact-split-range-costed-top-level-correct-and-local-budget-bound:*

```

assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
and base-budget:  $\Delta \leq A$ 
and R-pos:  $0 < R$ 
and source-factor:  $\text{Suc } h \leq A$ 
and k-pos:  $0 < k$ 
and M-cap:  $\bigwedge i. i \leq l \implies M\text{-of } i \leq \text{cap}$ 
and step-budget:
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$ 
 $\text{child-costs } U.$ 
 $D = \text{label-partition-view}$ 
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$ 
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$ 
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$ 
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$ 
 $\text{exact-split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l$ 
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$ 
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$ 
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$ 
 $\text{complete-on } d'$ 
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$ 
 $U = U\text{-loop} \cup$ 
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$ 
 $\text{sound-label } d \implies$ 

```

$bmssp\text{-}pre\text{-}full\ d\ S\ B \implies$
 $(\bigwedge x. x \in S \implies reachable\ s\ x) \implies$
 $(\bigwedge x. x \in S \implies below\text{-}bound\ (d\ x)\ B) \implies$
 $fp\text{-}iter\text{-}capped\text{-}scan\text{-}cost\ k\ cap\ d\ S\ S\ B + c\text{-}insert \leq A * card\ U$
and *run*:
 $exact\text{-}split\text{-}range\text{-}costed\text{-}bmssp\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ l$
 $finite\text{-}initial\text{-}label\ \{s\}\ Infinity\ d'\ Infinity\ U\ c$
shows $sssp\text{-}correct\ d' \wedge$
 $c \leq level\text{-}range\text{-}cost\text{-}bound\ A\ (R + l * t)\ (2 * l + 1)\ U$
proof
show $sssp\text{-}correct\ d'$
by (*rule finite-initial-label-exact-split-range-costed-top-level-correct*
 $[OF\ all\text{-}reachable\ run]$)
next
have *pre*: $bmssp\text{-}pre\text{-}full\ finite\text{-}initial\text{-}label\ \{s\}\ Infinity$
using *all-reachable finite-initial-label-source-complete*
by (*rule top-bmssp-pre-full*)
have *sound*: $sound\text{-}label\ finite\text{-}initial\text{-}label$
using $finite\text{-}initial\text{-}label\text{-}sound\ [OF\ all\text{-}reachable]$.
have *S-reaches*: $\bigwedge x. x \in \{s\} \implies reachable\ s\ x$
using *all-reachable source-in-V* **by** *blast*
have *below*: $\bigwedge x. x \in \{s\} \implies below\text{-}bound\ (finite\text{-}initial\text{-}label\ x)\ Infinity$
by *simp*
show $c \leq level\text{-}range\text{-}cost\text{-}bound\ A\ (R + l * t)\ (2 * l + 1)\ U$
by (*rule exact-split-range-costed-bmssp-level-bound-from-local-budgets*
 $[OF\ base\text{-}budget\ R\text{-}pos\ source\text{-}factor\ k\text{-}pos\ M\text{-}cap\ step\text{-}budget\ run\ sound\ pre$
 $S\text{-}reaches\ below]$)
qed

theorem *finite-initial-label-exact-split-range-costed-top-level-correct-and-local-budget-graph-bound*:

assumes *all-reachable*: $\bigwedge v. v \in V \implies reachable\ s\ v$
and *base-budget*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *source-factor*: $Suc\ h \leq A$
and *k-pos*: $0 < k$
and *M-cap*: $\bigwedge i. i \leq l \implies M\text{-}of\ i \leq cap$
and *step-budget*:
 $\bigwedge l\ d\ S\ B\ D\ c\text{-}insert\ d'\ a\ betas\ bs\ B'\ Us\ U\text{-}loop\ c\text{-}loop$
 $child\text{-}costs\ U.$
 $D = label\text{-}partition\text{-}view$
 $(find\text{-}pivots\text{-}label\text{-}capped\ k\ cap\ d\ S\ B)$
 $(find\text{-}pivots\text{-}pivots\text{-}capped\ k\ cap\ d\ S\ B) \implies$
 $partition\text{-}initial\text{-}insert\text{-}cost\text{-}bound\ c\text{-}insert\ t$
 $(find\text{-}pivots\text{-}pivots\text{-}capped\ k\ cap\ d\ S\ B) \implies$
 $exact\text{-}split\text{-}range\text{-}costed\text{-}partition\text{-}loop\text{-}state\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ l$
 $(find\text{-}pivots\text{-}label\text{-}capped\ k\ cap\ d\ S\ B)$
 $(find\text{-}pivots\text{-}pivots\text{-}capped\ k\ cap\ d\ S\ B)\ B\ d'\ D\ a\ betas\ bs\ B'$
 $Us\ U\text{-}loop\ c\text{-}loop\ child\text{-}costs \implies$

complete-on d'
 $\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
sound-label d $\implies$
bmssp-pre-full d S B $\implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $(\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B) \implies$
 $\text{fp-iter-capped-scan-cost } k \text{ cap } d \ S \ S \ B + c\text{-insert} \leq A * \text{card } U$
and run:
exact-split-range-costed-bmssp Δ *M-of t h k cap l*
finite-initial-label {s} *Infinity d' Infinity U c*
shows *sssp-correct d' $\wedge$*
 $c \leq A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$
proof –
have *combined:*
sssp-correct d' $\wedge$
 $c \leq \text{level-range-cost-bound } A \ (R + l * t) \ (2 * l + 1) \ U$
by (*rule finite-initial-label-exact-split-range-costed-top-level-correct-and-local-budget-bound*
 $[OF \text{ all-reachable base-budget } R\text{-pos source-factor } k\text{-pos } M\text{-cap step-budget run}]$)
then have *correct: sssp-correct d'*
by *blast*
have *pre: bmssp-pre-full finite-initial-label {s} Infinity*
using *all-reachable finite-initial-label-source-complete*
by (*rule top-bmssp-pre-full*)
have *sound: sound-label finite-initial-label*
using *finite-initial-label-sound[OF all-reachable]* .
have *S-reaches: $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$*
using *all-reachable source-in-V* **by** *blast*
have *post-full:*
bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
by (*rule exact-split-range-costed-bmssp-correct[OF run sound pre S-reaches]*)
have *post: bmssp-post finite-initial-label {s} Infinity d' Infinity U*
using *bmssp-post-full-imp-post[OF post-full]* .
have *U-V: U = V*
using *post bound-tree-source-infinity[OF all-reachable]*
unfolding *bmssp-post-def* **by** *auto*
have *cost-U:*
 $c \leq \text{level-range-cost-bound } A \ (R + l * t) \ (2 * l + 1) \ U$
using *combined* **by** *blast*
have *graph-bound:*
 $\text{level-range-cost-bound } A \ (R + l * t) \ (2 * l + 1) \ U$
 $\leq A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$
proof –
have *U-card: card U = vertex-count*
unfolding *U-V vertex-count-def* **by** *simp*
have *out-le: card (outgoing-edges U) $\leq$ edge-count*
by (*rule edge-count-outgoing-bound*)

```

have out-term:
  (R + l * t) * card (outgoing-edges U) ≤ (R + l * t) * edge-count
  using out-le by simp
show ?thesis
  unfolding level-range-cost-bound-def
  using U-card out-term by (simp add: algebra-simps)
qed
show ?thesis
  using correct cost-U graph-bound by linarith
qed

theorem finite-initial-label-exact-split-range-costed-top-level-correct-and-scaled-seen-
progress-graph-bound:
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
  and degree: edge-outdegree-le  $\Delta$ 
  and degree-factor:  $\Delta \leq A$ 
  and R-pos:  $0 < R$ 
  and insert-factor:  $t \leq A * k$ 
  and insert-scaled-factor:  $t \leq A\text{-insert} * k$ 
  and seen-scaled-factor:  $k * \Delta + A\text{-insert} \leq 2 * A$ 
  and source-factor:  $\text{Suc } h \leq 2 * A$ 
  and k-pos:  $0 < k$ 
  and M-cap:  $\bigwedge i. i \leq l \implies M\text{-of } i \leq \text{cap}$ 
  and step-scaled-seen-progress:
     $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$ 
     $\text{child-costs } U.$ 
     $D = \text{label-partition-view}$ 
     $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$ 
     $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$ 
     $\text{partition-initial-insert-cost-bound } c\text{-insert } t$ 
     $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$ 
     $\text{exact-split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l$ 
     $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$ 
     $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$ 
     $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$ 
     $\text{complete-on } d'$ 
     $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$ 
     $U = U\text{-loop} \cup$ 
     $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$ 
     $\text{sound-label } d \implies$ 
     $\text{bmssp-pre-full } d \ S \ B \implies$ 
     $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$ 
     $(\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B) \implies$ 
     $k * \text{card } S \leq \text{cap} \wedge \text{tree-antichain } S \wedge$ 
     $(B' = B \longrightarrow$ 
     $\text{card } (\text{find-pivots-seen-capped } k \ \text{cap } d \ S \ B) \leq \text{card } U)$ 
  and run:
     $\text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l$ 
     $\text{finite-initial-label } \{s\} \ \text{Infinity } d' \ \text{Infinity } U \ c$ 

```

shows *sssp-correct* $d' \wedge$
 $c \leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$
proof (rule *finite-initial-label-exact-split-range-costed-top-level-correct-and-local-budget-graph-bound*
[where $A = 2 * A$ and $R = R$,
OF all-reachable - R-pos source-factor k-pos M-cap - run])
show $\Delta \leq 2 * A$
using *degree-factor* **by** *simp*
next
fix $l\ d\ S\ B\ D\ c\text{-insert}\ d'\ a\ \text{betas}\ bs\ B'\ Us\ U\text{-loop}\ c\text{-loop}\ \text{child-costs}\ U$
assume *D-def*:
 $D = \text{label-partition-view}$
(*find-pivots-label-capped* $k\ \text{cap}\ d\ S\ B$)
(*find-pivots-pivots-capped* $k\ \text{cap}\ d\ S\ B$)
and *insert*:
partition-initial-insert-cost-bound $c\text{-insert}\ t$
(*find-pivots-pivots-capped* $k\ \text{cap}\ d\ S\ B$)
and *loop*:
exact-split-range-costed-partition-loop-state $\Delta\ M\text{-of}\ t\ h\ k\ \text{cap}\ l$
(*find-pivots-label-capped* $k\ \text{cap}\ d\ S\ B$)
(*find-pivots-pivots-capped* $k\ \text{cap}\ d\ S\ B$) $B\ d'\ D\ a\ \text{betas}\ bs\ B'$
 $Us\ U\text{-loop}\ c\text{-loop}\ \text{child-costs}$
and *complete*:
complete-on d'
 $\{v \in \text{bound-tree}\ S\ B'.\ \text{find-pivots-label-capped}\ k\ \text{cap}\ d\ S\ B\ v = \text{dist}\ s\ v\}$
and *U-def*:
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree}\ S\ B'.\ \text{find-pivots-label-capped}\ k\ \text{cap}\ d\ S\ B\ v = \text{dist}\ s\ v\}$
and *sound-s*: *sound-label* d
and *pre-s*: *bmssp-pre-full* $d\ S\ B$
and *reaches-s*: $\bigwedge x. x \in S \implies \text{reachable}\ s\ x$
and *below-s*: $\bigwedge x. x \in S \implies \text{below-bound}\ (d\ x)\ B$
have *progress*:
 $k * \text{card}\ S \leq \text{cap} \wedge \text{tree-antichain}\ S \wedge$
 $(B' = B \implies$
 $\text{card}\ (\text{find-pivots-seen-capped}\ k\ \text{cap}\ d\ S\ B) \leq \text{card}\ U)$
by (rule *step-scaled-seen-progress*
[*OF D-def insert loop complete U-def sound-s pre-s reaches-s below-s*])
then **have** *S-k-cap*: $k * \text{card}\ S \leq \text{cap}$
by *blast*
from *progress* **have** *anti*: *tree-antichain* S
by *blast*
from *progress* **have** *seen-success*:
 $B' = B \implies$
 $\text{card}\ (\text{find-pivots-seen-capped}\ k\ \text{cap}\ d\ S\ B) \leq \text{card}\ U$
by *blast*
have *split-loop*:
split-range-costed-partition-loop-state $\Delta\ M\text{-of}\ t\ h\ k\ \text{cap}\ l$
(*find-pivots-label-capped* $k\ \text{cap}\ d\ S\ B$)

(find-pivots-pivots-capped k cap d S B) B d' D a betas bs B'
 Us U -loop c -loop child-costs
 by (rule exact-split-range-costed-refines-split-range-costed(1)[OF loop])
 show fp-iter-capped-scan-cost k cap d S S B + c -insert $\leq$
 $2 * A * \text{card } U$
 by (rule split-range-costed-capped-step-scan-insert-budget-from-scaled-seen-or-
 threshold
 [OF degree degree-factor insert-factor insert-scaled-factor
 seen-scaled-factor insert split-loop U-def sound-s pre-s reaches-s
 S-k-cap anti k-pos seen-success])
 qed

theorem finite-initial-label-exact-split-range-costed-top-level-correct-and-scaled-seen-
 progress-graph-bound-level-cap:

assumes all-reachable: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
 and degree: edge-outdegree-le Δ
 and degree-factor: $\Delta \leq A$
 and R-pos: $0 < R$
 and insert-factor: $t \leq A * k$
 and insert-scaled-factor: $t \leq A\text{-insert} * k$
 and seen-scaled-factor: $k * \Delta + A\text{-insert} \leq 2 * A$
 and source-factor: $\text{Suc } h \leq 2 * A$
 and k-pos: $0 < k$
 and step-scaled-seen-progress:
 $\bigwedge l' d S B D c\text{-insert } d' a \text{ betas } bs B' Us U\text{-loop } c\text{-loop}$
 child-costs U .
 $D = \text{label-partition-view}$
 (find-pivots-label-capped k (bmssp-level-cap k t l) d S B)
 (find-pivots-pivots-capped k (bmssp-level-cap k t l) d S B) $\implies$
 partition-initial-insert-cost-bound $c\text{-insert } t$
 (find-pivots-pivots-capped k (bmssp-level-cap k t l) d S B) $\implies$
 exact-split-range-costed-partition-loop-state Δ (bmssp-level-cap k t) t h k
 (bmssp-level-cap k t l) l'
 (find-pivots-label-capped k (bmssp-level-cap k t l) d S B)
 (find-pivots-pivots-capped k (bmssp-level-cap k t l) d S B) B d' D
 a betas bs B' Us U -loop c -loop child-costs $\implies$
 complete-on d'
 $\{v \in \text{bound-tree } S B'\}.$
 find-pivots-label-capped k (bmssp-level-cap k t l) d S B $v =$
 $\text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S B'\}.$
 find-pivots-label-capped k (bmssp-level-cap k t l) d S B $v =$
 $\text{dist } s \ v\} \implies$
 sound-label $d \implies$
 bmssp-pre-full d S $B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $(\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B) \implies$
 $k * \text{card } S \leq \text{bmssp-level-cap } k \ t \ l \wedge \text{tree-antichain } S \wedge$

$(B' = B \longrightarrow$
 $\text{card } (\text{find-pivots-seen-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B) \leq$
 $\text{card } U)$
and *run*:
 $\text{exact-split-range-costed-bmssp } \Delta \text{ (bmssp-level-cap } k \text{ } t) \text{ } t \text{ } h \text{ } k$
 $\text{(bmssp-level-cap } k \text{ } t \text{ } l) \text{ } l$
 $\text{finite-initial-label } \{s\} \text{ Infinity } d' \text{ Infinity } U \text{ } c$
shows *sssp-correct* $d' \wedge$
 $c \leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$
proof (*rule finite-initial-label-exact-split-range-costed-top-level-correct-and-scaled-*
seen-progress-graph-bound
 $[OF \text{ all-reachable } \text{degree } \text{degree-factor } R\text{-pos } \text{insert-factor } \text{insert-scaled-factor}$
 $\text{seen-scaled-factor } \text{source-factor } k\text{-pos } - - \text{run}]$)
fix i
assume $i \leq l$
then show $\text{bmssp-level-cap } k \text{ } t \text{ } i \leq \text{bmssp-level-cap } k \text{ } t \text{ } l$
by (*rule bmssp-level-cap-mono*)
next
fix $l' \text{ } d \text{ } S \text{ } B \text{ } D \text{ } c\text{-insert } d' \text{ } a \text{ } \text{betas } bs \text{ } B' \text{ } Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs } U$
assume $D\text{-def}$:
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B)$
 $(\text{find-pivots-pivots-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B)$
and *insert*:
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B)$
and *loop*:
 $\text{exact-split-range-costed-partition-loop-state } \Delta \text{ (bmssp-level-cap } k \text{ } t) \text{ } t \text{ } h \text{ } k$
 $\text{(bmssp-level-cap } k \text{ } t \text{ } l) \text{ } l'$
 $(\text{find-pivots-label-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B)$
 $(\text{find-pivots-pivots-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B) \text{ } B \text{ } d' \text{ } D$
 $a \text{ } \text{betas } bs \text{ } B' \text{ } Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs}$
and *complete*:
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{}$
 $\text{find-pivots-label-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B \text{ } v =$
 $\text{dist } s \text{ } v\}$
and $U\text{-def}$:
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{}$
 $\text{find-pivots-label-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B \text{ } v =$
 $\text{dist } s \text{ } v\}$
and *sound-s*: $\text{sound-label } d$
and *pre-s*: $\text{bmssp-pre-full } d \text{ } S \text{ } B$
and *reaches-s*: $\bigwedge x. x \in S \implies \text{reachable } s \text{ } x$
and *below-s*: $\bigwedge x. x \in S \implies \text{below-bound } (d \text{ } x) \text{ } B$
show $k * \text{card } S \leq \text{bmssp-level-cap } k \text{ } t \text{ } l \wedge \text{tree-antichain } S \wedge$
 $(B' = B \longrightarrow$
 $\text{card } (\text{find-pivots-seen-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B)$

$\leq \text{card } U$
by (*rule step-scaled-seen-progress*
 $[OF \ D\text{-def insert loop complete } U\text{-def sound-} s \text{ pre-} s \text{ reaches-} s \text{ below-} s])$
qed

theorem *finite-initial-label-exact-split-range-costed-top-level-correct-and-scaled-seen-success-graph-bound:*

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$
and *insert-scaled-factor*: $t \leq A\text{-insert} * k$
and *seen-scaled-factor*: $k * \Delta + A\text{-insert} \leq 2 * A$
and *source-factor*: $\text{Suc } h \leq 2 * A$
and *k-pos*: $0 < k$
and *top-cap*: $k \leq \text{cap}$
and *M-cap*: $\bigwedge i. i \leq l \implies M\text{-of } i \leq \text{cap}$
and *seen-success*:
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
 $\text{child-costs } U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{exact-split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \ \text{find-pivots-label-capped } k \ \text{cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $(\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B) \implies$
 $k * \text{card } S \leq \text{cap} \implies$
 $\text{tree-antichain } S \implies$
 $B' = B \implies$
 $\text{card } (\text{find-pivots-seen-capped } k \ \text{cap } d \ S \ B) \leq \text{card } U$
and *run*:
 $\text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l$
 $\text{finite-initial-label } \{s\} \ \text{Infinity } d' \ \text{Infinity } U \ c$
shows $\text{sssp-correct } d' \wedge$
 $c \leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$
proof –
have *correct*: $\text{sssp-correct } d'$

```

    by (rule finite-initial-label-exact-split-range-costed-top-level-correct
        [OF all-reachable run])
  have pre: bmssp-pre-full finite-initial-label {s} Infinity
    using all-reachable finite-initial-label-source-complete
    by (rule top-bmssp-pre-full)
  have sound: sound-label finite-initial-label
    using finite-initial-label-sound[OF all-reachable] .
  have S-reaches:  $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$ 
    using all-reachable source-in-V by blast
  have below:  $\bigwedge x. x \in \{s\} \implies$ 
    below-bound (finite-initial-label x) Infinity
    by simp
  have S-k-cap:  $k * \text{card } \{s\} \leq \text{cap}$ 
    using top-cap by simp
  have S-anti: tree-antichain {s}
    by simp
  have range-bound:
     $c \leq \text{level-range-cost-bound } (2 * A) (R + l * t) (2 * l + 1) U$ 
  proof (rule exact-split-range-costed-bmssp-level-bound-from-local-budgets-with-
    invariants
      [where A = 2 * A and R = R,
        OF - R-pos source-factor k-pos M-cap - run sound pre S-reaches below
        S-k-cap S-anti])
    show  $\Delta \leq 2 * A$ 
      using degree-factor by simp
  next
  fix l d S B D c-insert d' a betas bs B' Us U-loop c-loop child-costs U
  assume D-def:
    D = label-partition-view
      (find-pivots-label-capped k cap d S B)
      (find-pivots-pivots-capped k cap d S B)
  and insert:
    partition-initial-insert-cost-bound c-insert t
      (find-pivots-pivots-capped k cap d S B)
  and loop:
    exact-split-range-costed-partition-loop-state  $\Delta$  M-of t h k cap l
      (find-pivots-label-capped k cap d S B)
      (find-pivots-pivots-capped k cap d S B) B d' D a betas bs B'
      Us U-loop c-loop child-costs
  and complete:
    complete-on d'
      {v  $\in$  bound-tree S B'. find-pivots-label-capped k cap d S B v =
        dist s v}
  and U-def:
    U = U-loop  $\cup$ 
      {v  $\in$  bound-tree S B'. find-pivots-label-capped k cap d S B v =
        dist s v}
  and sound-s: sound-label d
  and pre-s: bmssp-pre-full d S B

```

```

and reaches-s:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
and below-s:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$ 
and S-k-cap:  $k * \text{card } S \leq \text{cap}$ 
and anti: tree-antichain S
have split-loop:
  split-range-costed-partition-loop-state  $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l$ 
    (find-pivots-label-capped  $k \ \text{cap } d \ S \ B$ )
    (find-pivots-pivots-capped  $k \ \text{cap } d \ S \ B$ )  $B \ d' \ D \ a \ \text{betas } bs \ B'$ 
     $Us \ U\text{-loop } c\text{-loop } \text{child-costs}$ 
  by (rule exact-split-range-costed-refines-split-range-costed(1)[OF loop])
have seen-success-step:
   $B' = B \implies$ 
     $\text{card } (\text{find-pivots-seen-capped } k \ \text{cap } d \ S \ B) \leq \text{card } U$ 
  by (rule seen-success
    [OF D-def insert loop complete U-def sound-s pre-s reaches-s below-s
      S-k-cap anti])
show fp-iter-capped-scan-cost  $k \ \text{cap } d \ S \ S \ B + c\text{-insert} \leq$ 
   $2 * A * \text{card } U$ 
  by (rule split-range-costed-capped-step-scan-insert-budget-from-scaled-seen-or-
threshold
    [OF degree degree-factor insert-factor insert-scaled-factor
      seen-scaled-factor insert split-loop U-def sound-s pre-s reaches-s
      S-k-cap anti k-pos seen-success-step])
qed
have post-full:
  bmssp-post-full finite-initial-label  $\{s\} \ \text{Infinity } d' \ \text{Infinity } U$ 
  by (rule exact-split-range-costed-bmssp-correct[OF run sound pre S-reaches])
have post: bmssp-post finite-initial-label  $\{s\} \ \text{Infinity } d' \ \text{Infinity } U$ 
  using bmssp-post-full-imp-post[OF post-full] .
have U-V:  $U = V$ 
  using post bound-tree-source-infinity[OF all-reachable]
  unfolding bmssp-post-def by auto
have graph-bound:
  level-range-cost-bound  $(2 * A) \ (R + l * t) \ (2 * l + 1) \ U$ 
   $\leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$ 
proof –
  have U-card:  $\text{card } U = \text{vertex-count}$ 
  unfolding U-V vertex-count-def by simp
have out-le:  $\text{card } (\text{outgoing-edges } U) \leq \text{edge-count}$ 
  by (rule edge-count-outgoing-bound)
have out-term:
   $(R + l * t) * \text{card } (\text{outgoing-edges } U) \leq (R + l * t) * \text{edge-count}$ 
  using out-le by simp
show ?thesis
  unfolding level-range-cost-bound-def
  using U-card out-term by (simp add: algebra-simps)
qed
show ?thesis
  using correct range-bound graph-bound by linarith

```


qed

theorem *finite-initial-label-exact-split-range-costed-top-level-correct-and-scaled-seen-success-graph-bound-level-cap*:

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$
and *insert-scaled-factor*: $t \leq A\text{-insert} * k$
and *seen-scaled-factor*: $k * \Delta + A\text{-insert} \leq 2 * A$
and *source-factor*: $\text{Suc } h \leq 2 * A$
and *k-pos*: $0 < k$
and *seen-success*:
 $\bigwedge l' \ d \ S \ B \ D \ c\text{-insert} \ d' \ a \ \text{betas} \ bs \ B' \ Us \ U\text{-loop} \ c\text{-loop}$
 $\text{child-costs } U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B) \implies$
 $\text{exact-split-range-costed-partition-loop-state } \Delta \ (\text{bmssp-level-cap } k \ t) \ t \ h \ k$
 $(\text{bmssp-level-cap } k \ t \ l) \ l'$
 $(\text{find-pivots-label-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B) \ B \ d' \ D$
 $a \ \text{betas} \ bs \ B' \ Us \ U\text{-loop} \ c\text{-loop} \ \text{child-costs} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \ B'\.$
 $\text{find-pivots-label-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B \ v =$
 $\text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'\.$
 $\text{find-pivots-label-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B \ v =$
 $\text{dist } s \ v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $(\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B) \implies$
 $k * \text{card } S \leq \text{bmssp-level-cap } k \ t \ l \implies$
 $\text{tree-antichain } S \implies$
 $B' = B \implies$
 $\text{card } (\text{find-pivots-seen-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B) \leq$
 $\text{card } U$
and *run*:
 $\text{exact-split-range-costed-bmssp } \Delta \ (\text{bmssp-level-cap } k \ t) \ t \ h \ k$
 $(\text{bmssp-level-cap } k \ t \ l) \ l$
 $\text{finite-initial-label } \{s\} \ \text{Infinity } d' \ \text{Infinity } U \ c$
shows *sssp-correct* $d' \wedge$
 $c \leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$

```

proof (rule finite-initial-label-exact-split-range-costed-top-level-correct-and-scaled-
seen-success-graph-bound
  [OF all-reachable degree degree-factor R-pos insert-factor insert-scaled-factor
    seen-scaled-factor source-factor k-pos - - - run])
show  $k \leq \text{bmssp-level-cap } k \ t \ l$ 
proof -
  have one-le:  $1 \leq \text{bmssp-level-width } t \ l$ 
  unfolding bmssp-level-width-def by simp
  have  $k * 1 \leq k * \text{bmssp-level-width } t \ l$ 
  by (rule mult-left-mono[OF one-le]) simp
  then show ?thesis
  unfolding bmssp-level-cap-def by simp
qed
next
fix  $i$ 
assume  $i \leq l$ 
then show  $\text{bmssp-level-cap } k \ t \ i \leq \text{bmssp-level-cap } k \ t \ l$ 
by (rule bmssp-level-cap-mono)
next
fix  $l' \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs } U$ 
assume D-def:
   $D = \text{label-partition-view}$ 
   $(\text{find-pivots-label-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B)$ 
   $(\text{find-pivots-pivots-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B)$ 
and insert:
   $\text{partition-initial-insert-cost-bound } c\text{-insert } t$ 
   $(\text{find-pivots-pivots-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B)$ 
and loop:
   $\text{exact-split-range-costed-partition-loop-state } \Delta \ (\text{bmssp-level-cap } k \ t) \ t \ h \ k$ 
   $(\text{bmssp-level-cap } k \ t \ l) \ l'$ 
   $(\text{find-pivots-label-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B)$ 
   $(\text{find-pivots-pivots-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B) \ B \ d' \ D$ 
   $a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs}$ 
and complete:
   $\text{complete-on } d'$ 
   $\{v \in \text{bound-tree } S \ B'. \$ 
   $\text{find-pivots-label-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B \ v =$ 
   $\text{dist } s \ v\}$ 
and U-def:
   $U = U\text{-loop} \cup$ 
   $\{v \in \text{bound-tree } S \ B'. \$ 
   $\text{find-pivots-label-capped } k \ (\text{bmssp-level-cap } k \ t \ l) \ d \ S \ B \ v =$ 
   $\text{dist } s \ v\}$ 
and sound-s: sound-label  $d$ 
and pre-s: bmssp-pre-full  $d \ S \ B$ 
and reaches-s:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
and below-s:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$ 
and S-k-cap:  $k * \text{card } S \leq \text{bmssp-level-cap } k \ t \ l$ 
and anti: tree-antichain  $S$ 

```

show $B' = B \implies$
 $\text{card } (\text{find-pivots-seen-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B)$
 $\leq \text{card } U$
by (rule seen-success
 $[OF \text{ } D\text{-def insert loop complete } U\text{-def sound-s pre-s reaches-s below-s}$
 $S\text{-k-cap anti}])$
qed

theorem *finite-initial-label-exact-split-range-costed-top-level-correct-and-closed-scaled-graph-bound-level-cap:*

assumes *all-reachable:* $\bigwedge v. v \in V \implies \text{reachable } s \text{ } v$
and *degree:* $\text{edge-outdegree-le } \Delta$
and *degree-factor:* $\Delta \leq A$
and *R-pos:* $0 < R$
and *insert-factor:* $t \leq A * k$
and *insert-scaled-factor:* $t \leq A\text{-insert} * k$
and *seen-scaled-factor:* $k * \Delta + A\text{-insert} \leq 2 * A$
and *source-factor:* $\text{Suc } h \leq 2 * A$
and *k-pos:* $0 < k$
and *run:*
 $\text{exact-split-range-costed-bmssp } \Delta \text{ (bmssp-level-cap } k \text{ } t) \text{ } t \text{ } h \text{ } k$
 $\text{(bmssp-level-cap } k \text{ } t \text{ } l) \text{ } l$
 $\text{finite-initial-label } \{s\} \text{ Infinity } d' \text{ Infinity } U \text{ } c$
shows $\text{sssp-correct } d' \wedge$
 $c \leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * t) * \text{edge-count}$
proof (rule *finite-initial-label-exact-split-range-costed-top-level-correct-and-scaled-seen-success-graph-bound-level-cap*
 $[OF \text{ } all\text{-reachable degree degree-factor } R\text{-pos insert-factor insert-scaled-factor}$
 $\text{seen-scaled-factor source-factor } k\text{-pos - run}])$
fix $l' \text{ } d \text{ } S \text{ } B \text{ } D \text{ } c\text{-insert } d' \text{ } a \text{ } \text{betas } bs \text{ } B' \text{ } Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs } U$
assume *loop:*
 $\text{exact-split-range-costed-partition-loop-state } \Delta \text{ (bmssp-level-cap } k \text{ } t) \text{ } t \text{ } h \text{ } k$
 $\text{(bmssp-level-cap } k \text{ } t \text{ } l) \text{ } l'$
 $\text{(find-pivots-label-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B)$
 $\text{(find-pivots-pivots-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B) \text{ } B \text{ } d' \text{ } D$
 $a \text{ } \text{betas } bs \text{ } B' \text{ } Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs}$
and *U-def:*
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{ } \text{find-pivots-label-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B \text{ } v =$
 $\text{dist } s \text{ } v\}$
and *sound-s:* $\text{sound-label } d$
and *pre-s:* $\text{bmssp-pre-full } d \text{ } S \text{ } B$
and *reaches-s:* $\bigwedge x. x \in S \implies \text{reachable } s \text{ } x$
and *below-s:* $\bigwedge x. x \in S \implies \text{below-bound } (d \text{ } x) \text{ } B$
and *success:* $B' = B$
show $\text{card } (\text{find-pivots-seen-capped } k \text{ (bmssp-level-cap } k \text{ } t \text{ } l) \text{ } d \text{ } S \text{ } B)$
 $\leq \text{card } U$
by (rule *exact-split-range-costed-step-seen-success*

[*OF loop U-def sound-s pre-s reaches-s below-s success*])

qed

corollary *finite-initial-label-exact-split-range-costed-top-level-correct-and-closed-bmssp-graph-time-bound-level-cap*:

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$
and *insert-scaled-factor*: $t \leq A\text{-insert} * k$
and *seen-scaled-factor*: $k * \Delta + A\text{-insert} \leq 2 * A$
and *source-factor*: $\text{Suc } h \leq 2 * A$
and *k-pos*: $0 < k$
and *run*:
exact-split-range-costed-bmssp Δ (*bmssp-level-cap* $k \ t$) $t \ h \ k$
(bmssp-level-cap $k \ t \ l$) l
finite-initial-label $\{s\}$ *Infinity* d' *Infinity* $U \ c$

shows *sssp-correct* $d' \wedge$
 $c \leq \text{bmssp-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. l) (\lambda-. t)$
 $(\lambda-. \text{edge-count}) \ \text{vertex-count}$

proof –
have *graph*:
sssp-correct $d' \wedge$
 $c \leq 2 * A * (2 * l + 1) * \text{vertex-count} +$
 $(R + l * t) * \text{edge-count}$
by (*rule finite-initial-label-exact-split-range-costed-top-level-correct-and-closed-scaled-graph-bound-level-cap*
[*OF all-reachable degree degree-factor R-pos insert-factor insert-scaled-factor*
seen-scaled-factor source-factor k-pos run])

then show *?thesis*
unfolding *bmssp-graph-time-bound-def* **by** *simp*

qed

lemma *exact-split-range-costed-partition-loop-state-trivial-edge-budget*:

assumes *run*:
exact-split-range-costed-partition-loop-state Δ *M-of* $t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
betas $bs \ B' \ Us \ U \ c \ \text{child-costs}$
and *sound*: *sound-label* d
and *pre*: *bmssp-pre-full* $d \ P \ B$
and *P-reaches*: $\bigwedge x. x \in P \implies \text{reachable } s \ x$

shows $t * \text{sum-list } (\text{map } \text{card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$
 $\leq t * \text{card } (\text{outgoing-edges } U)$

proof –
have *split-run*:
split-range-costed-partition-loop-state Δ *M-of* $t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
betas $bs \ B' \ Us \ U \ c \ \text{child-costs}$
using *exact-split-range-costed-refines-split-range-costed*(1)[*OF run*] .
have *mono*: *nondecreasing-from* $a \ bs$

by (rule *split-range-costed-partition-loop-state-mono*[*OF split-run*])
have *trace*: *concrete-partition-loop-trace* $P B a bs d' B' Us U$
by (rule *split-range-costed-partition-loop-state-trace*
[*OF split-run sound pre P-reaches*])
have *children*:
list-all2 ($\lambda U X. U = X \wedge \text{complete-on } d' U$) Us
(*range-tree-chain-list* $P a bs B'$)
using *trace unfolding concrete-partition-loop-trace-def* **by** *blast*
have *U-def*: $U = \text{bound-tree } P (Fin a) \cup \bigcup (set Us)$
using *trace unfolding concrete-partition-loop-trace-def* **by** *blast*
have *Union-eq*:
 $\bigcup (set Us) = \bigcup (set (\text{range-tree-chain-list } P a bs B'))$
using *children* **by** (*induction rule: list-all2-induct*) *auto*
have *U-eq-chain*:
 $U = \text{bound-tree } P (Fin a) \cup \text{range-tree-chain } P a bs B'$
using *U-def Union-eq Union-range-tree-chain-list*[*of P a bs B'*] **by** *simp*
have *chain-subset*: $\text{range-tree-chain } P a bs B' \subseteq U$
using *U-eq-chain* **by** *blast*
have *outgoing-subset*:
 $\text{outgoing-edges } (\text{range-tree-chain } P a bs B') \subseteq \text{outgoing-edges } U$
by (rule *outgoing-edges-mono*[*OF chain-subset*])
have *card-out-le*:
 $\text{card } (\text{outgoing-edges } (\text{range-tree-chain } P a bs B')) \leq$
 $\text{card } (\text{outgoing-edges } U)$
by (rule *card-mono*) (*simp-all add: outgoing-subset*)
have *sum-le-chain*:
 $\text{sum-list } (\text{map } \text{card } (\text{range-tree-child-edge-range-list } P a \text{ betas } bs))$
 $\leq \text{card } (\text{outgoing-edges } (\text{range-tree-chain } P a bs B'))$
by (rule *sum-card-range-tree-child-edge-range-list-le-outgoing-edges-chain*
[*OF mono*])
have *sum-le-U*:
 $\text{sum-list } (\text{map } \text{card } (\text{range-tree-child-edge-range-list } P a \text{ betas } bs))$
 $\leq \text{card } (\text{outgoing-edges } U)$
using *sum-le-chain card-out-le* **by** *linarith*
then show *?thesis*
by *simp*
qed

theorem *exact-split-range-costed-partition-loop-state-closes-level-bound-from-child-bound-with-invariants-and-edge-budget*:

assumes *run*:
exact-split-range-costed-partition-loop-state $\Delta M\text{-of } t h k \text{ cap } l d P B d' D a$
betas $bs B' Us U c \text{ child-costs}$
and *sound*: *sound-label* d
and *pre*: *bmssp-pre-full* $d P B$
and *P-reaches*: $\bigwedge x. x \in P \implies \text{reachable } s x$
and *P-k-cap*: $k * \text{card } P \leq \text{cap}$
and *P-anti*: *tree-antichain* P
and *child-bound*:

$\wedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\llbracket \text{exact-split-range-costed-bmssp } \Delta \text{ M-of } t \text{ h } k \text{ cap } l \text{ d } S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \text{ } U\text{-child } c\text{-child};$
 $\text{bmssp-pre-full } d \text{ } S\text{-child } B\text{-child};$
 $\wedge x. x \in S\text{-child} \implies \text{reachable } s \text{ } x;$
 $\text{card } S\text{-child} \leq \text{M-of } l;$
 $\wedge x. x \in S\text{-child} \implies \text{below-bound } (d \text{ } x) \text{ } B\text{-child};$
 $k * \text{card } S\text{-child} \leq \text{cap};$
 $\text{tree-antichain } S\text{-child} \rrbracket$
 $\implies c\text{-child} \leq \text{level-range-cost-bound } A \text{ } R \text{ } L \text{ } U\text{-child}$
and $M\text{-cap}: \text{M-of } l \leq \text{cap}$
and $\text{source-factor}: \text{Suc } h \leq A$
and $k\text{-pos}: 0 < k$
and $\text{edge-budget}:$
 $t * \text{sum-list } (\text{map } \text{card } (\text{range-tree-child-edge-range-list } P \text{ a } \text{betas } \text{bs})) \leq$
 $H * \text{card } (\text{outgoing-edges } U)$
shows $c \leq A * \text{Suc } L * \text{card } U + (R + H) * \text{card } (\text{outgoing-edges } U)$
proof –
have $P\text{-subset}: P \subseteq V$
using $\text{pre unfolding } \text{bmssp-pre-full-def}$ **by** blast
have $\text{child-cost-bounds}:$
 $\text{list-all2 } (\lambda c\text{-child } U\text{-child}.$
 $\quad c\text{-child} \leq \text{level-range-cost-bound } A \text{ } R \text{ } L \text{ } U\text{-child})$
 $\quad \text{child-costs } (\text{range-tree-child-list } P \text{ a } \text{bs})$
by $(\text{rule } \text{exact-split-range-costed-partition-loop-state-child-cost-bounds-with-}$
 invariants
 $\quad [\text{OF run } P\text{-subset } P\text{-k-cap } P\text{-anti } \text{child-bound}])$
have $\text{loop}:$
 $c \leq$
 $A * \text{Suc } L * \text{card } U + R * \text{card } (\text{outgoing-edges } U) +$
 $t * \text{sum-list } (\text{map } \text{card } (\text{range-tree-child-edge-range-list } P \text{ a } \text{betas } \text{bs}))$
by $(\text{rule } \text{exact-split-range-costed-partition-loop-state-closes-level-bound-from-child-}$
 $\text{costs-and-edge-ranges}$
 $\quad [\text{OF run } \text{sound pre } P\text{-reaches } \text{child-cost-bounds } M\text{-cap } \text{source-factor } k\text{-pos}])$
have $c \leq A * \text{Suc } L * \text{card } U + R * \text{card } (\text{outgoing-edges } U) +$
 $H * \text{card } (\text{outgoing-edges } U)$
using loop edge-budget **by** linarith
also have $\dots = A * \text{Suc } L * \text{card } U + (R + H) * \text{card } (\text{outgoing-edges } U)$
by $(\text{simp add: algebra-simps})$
finally show $?thesis .$
qed

theorem $\text{exact-split-range-costed-nonbase-step-closes-level-bound-from-child-bound-}$
 $\text{with-invariants-and-edge-budget}:$

assumes $\text{loop}:$
 $\text{exact-split-range-costed-partition-loop-state } \Delta \text{ M-of } t \text{ h } k \text{ cap } l \text{ d } P \text{ } B \text{ } d' \text{ } D \text{ a}$
 $\text{betas } \text{bs } B' \text{ } U\text{s } U\text{-loop } c\text{-loop } \text{child-costs}$
and $\text{sound}: \text{sound-label } d$
and $\text{pre}: \text{bmssp-pre-full } d \text{ } P \text{ } B$

and P -reaches: $\bigwedge x. x \in P \implies \text{reachable } s \ x$
and P - k -cap: $k * \text{card } P \leq \text{cap}$
and P -anti: $\text{tree-antichain } P$
and child-bound :
 $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$
 $\llbracket \text{exact-split-range-costed-bmssp } \Delta \text{ } M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-child } B\text{-child}$
 $\quad d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$
 $\quad \text{bmssp-pre-full } d \ S\text{-child } B\text{-child};$
 $\quad \bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$
 $\quad \text{card } S\text{-child} \leq M\text{-of } l;$
 $\quad \bigwedge x. x \in S\text{-child} \implies \text{below-bound } (d \ x) \ B\text{-child};$
 $\quad k * \text{card } S\text{-child} \leq \text{cap};$
 $\quad \text{tree-antichain } S\text{-child} \rrbracket$
 $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ R \ L \ U\text{-child}$
and M -cap: $M\text{-of } l \leq \text{cap}$
and source-factor : $\text{Suc } h \leq A$
and k -pos: $0 < k$
and edge-budget :
 $t * \text{sum-list } (\text{map } \text{card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs)) \leq$
 $\quad H * \text{card } (\text{outgoing-edges } U\text{-loop})$
and U -def: $U = U\text{-loop} \cup W$
and $\text{finite-}U$: $\text{finite } U$
and scan-insert : $c\text{-scan-insert} \leq A * \text{card } U$
and c -def: $c = c\text{-scan-insert} + c\text{-loop}$
shows $c \leq A * \text{Suc } (L * \text{card } U + (R + H) * \text{card } (\text{outgoing-edges } U))$
proof –
have loop-bound :
 $c\text{-loop} \leq A * \text{Suc } L * \text{card } U\text{-loop} +$
 $\quad (R + H) * \text{card } (\text{outgoing-edges } U\text{-loop})$
by (rule $\text{exact-split-range-costed-partition-loop-state-closes-level-bound-from-child-bound-with-invariants-and-edge-budget}$
 $\quad [\text{OF loop sound pre } P\text{-reaches } P\text{-}k\text{-cap } P\text{-anti child-bound } M\text{-cap}$
 $\quad \text{source-factor } k\text{-pos edge-budget}]$)
have $U\text{-loop-subset}$: $U\text{-loop} \subseteq U$
using U -def **by** blast
have card-loop-le : $\text{card } U\text{-loop} \leq \text{card } U$
by (rule $\text{card-mono}[\text{OF finite-}U \ U\text{-loop-subset}]$)
have outgoing-subset : $\text{outgoing-edges } U\text{-loop} \subseteq \text{outgoing-edges } U$
by (rule $\text{outgoing-edges-mono}[\text{OF } U\text{-loop-subset}]$)
have $\text{finite-out-}U$: $\text{finite } (\text{outgoing-edges } U)$
by simp
have card-out-le :
 $\text{card } (\text{outgoing-edges } U\text{-loop}) \leq \text{card } (\text{outgoing-edges } U)$
by (rule $\text{card-mono}[\text{OF finite-out-}U \ \text{outgoing-subset}]$)
have $c \leq$
 $A * \text{card } U +$
 $\quad (A * \text{Suc } L * \text{card } U + (R + H) * \text{card } (\text{outgoing-edges } U))$
proof –
have $\text{loop-to-}U$:

$c\text{-loop} \leq A * \text{Suc } L * \text{card } U +$
 $(R + H) * \text{card } (\text{outgoing-edges } U)$
proof –
have $A * \text{Suc } L * \text{card } U\text{-loop} \leq A * \text{Suc } L * \text{card } U$
using *card-loop-le* **by** *simp*
moreover have $(R + H) * \text{card } (\text{outgoing-edges } U\text{-loop}) \leq$
 $(R + H) * \text{card } (\text{outgoing-edges } U)$
using *card-out-le* **by** *simp*
ultimately show *?thesis*
using *loop-bound* **by** *linarith*
qed
show *?thesis*
using *scan-insert loop-to-U c-def* **by** *linarith*
qed
also have $\dots =$
 $A * \text{Suc } (\text{Suc } L) * \text{card } U + (R + H) * \text{card } (\text{outgoing-edges } U)$
by (*simp add: algebra-simps*)
finally show *?thesis* .
qed

theorem *exact-split-range-costed-bmssp-level-bound-from-local-budgets-with-invariants-and-edge-budget:*
assumes *base-budget: $\Delta \leq A$*
and *R-pos: $0 < R$*
and *source-factor: $\text{Suc } h \leq A$*
and *k-pos: $0 < k$*
and *M-cap: $\bigwedge i. i \leq l \implies M\text{-of } i \leq \text{cap}$*
and *edge-budget:*
 $\bigwedge l \ d \ P \ B \ d' \ D \ a \ \text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs.}$
 $\text{exact-split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 $\text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ P \ B \implies$
 $(\bigwedge x. x \in P \implies \text{reachable } s \ x) \implies$
 $t * \text{sum-list } (\text{map } \text{card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$
 $\leq H * \text{card } (\text{outgoing-edges } U)$
and *step-budget:*
 $\bigwedge l \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
 $\text{child-costs } U.$
 $D = \text{label-partition-view}$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{partition-initial-insert-cost-bound } c\text{-insert } t$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \implies$
 $\text{exact-split-range-costed-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l$
 $(\text{find-pivots-label-capped } k \ \text{cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \ \text{cap } d \ S \ B) \ B \ d' \ D \ a \ \text{betas } bs \ B'$
 $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$
 $\text{complete-on } d'$

$\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \ B'. \text{ find-pivots-label-capped } k \text{ cap } d \ S \ B \ v = \text{dist } s \ v\} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ S \ B \implies$
 $(\bigwedge x. x \in S \implies \text{reachable } s \ x) \implies$
 $(\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B) \implies$
 $k * \text{card } S \leq \text{cap} \implies$
 $\text{tree-antichain } S \implies$
 $\text{fp-iter-capped-scan-cost } k \text{ cap } d \ S \ S \ B + c\text{-insert} \leq A * \text{card } U$
and run:
 $\text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } l \ d \ S \ B \ d' \ B' \ U \ c$
and sound: $\text{sound-label } d$
and pre: $\text{bmssp-pre-full } d \ S \ B$
and S-reaches: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
and below: $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$
and S-k-cap: $k * \text{card } S \leq \text{cap}$
and S-anti: $\text{tree-antichain } S$
shows $c \leq \text{level-range-cost-bound } A \ (R + l * H) \ (2 * l + 1) \ U$
using $\text{run sound pre S-reaches below S-k-cap S-anti M-cap}$
proof ($\text{induction } l \text{ arbitrary: } d \ S \ B \ d' \ B' \ U \ c \text{ rule: less-induct}$)
case ($\text{less } l$)
show $?case$
proof ($\text{cases } l$)
case 0
have run0:
 $\text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } 0 \ d \ S \ B \ d' \ B' \ U \ c$
using $\text{less.prem}(1) \ 0 \text{ by simp}$
have split-run0:
 $\text{split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } 0 \ d \ S \ B \ d' \ B' \ U \ c$
using $\text{exact-split-range-costed-refines-split-range-costed}(2)[OF \text{ run0}]$.
show $?thesis$
using $\text{split-run0 R-pos } 0$
by ($\text{cases rule: split-range-costed-bmssp.cases}$)
 $(\text{simp-all add: split-range-costed-base-level-bound})$
next
case ($\text{Suc } l0$)
have run-suc:
 $\text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } (\text{Suc } l0) \ d \ S \ B \ d' \ B' \ U \ c$
using $\text{less.prem}(1) \ \text{Suc} \text{ by simp}$
show $?thesis$
proof ($\text{cases rule: exact-split-range-costed-bmssp-SucE}[OF \text{ run-suc}]$)
case ($1 \ c\text{-insert } a \ \text{betas } bs \ Us \ U\text{-loop } c\text{-loop } \text{child-costs-loop}$)
let $?d\text{-fp} = \text{find-pivots-label-capped } k \text{ cap } d \ S \ B$
let $?P = \text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B$
let $?W = \{v \in \text{bound-tree } S \ B'. \ ?d\text{-fp } v = \text{dist } s \ v\}$
have $\text{sound-fp: sound-label } ?d\text{-fp}$
unfolding $\text{find-pivots-label-capped-def}$
by ($\text{rule fp-iter-capped-label-sound}[OF \text{ less.prem}(2) \ \text{less.prem}(4)]$)

```

have pivot-pre: bmssp-pre-full ?d-fp ?P B
  using find-pivots-capped-establishes-pivot-pre-concrete
    [OF less.premis(2) less.premis(3) less.premis(4)] .
have S-subset:  $S \subseteq V$ 
  using less.premis(3) unfolding bmssp-pre-full-def by blast
have P-subset-S:  $?P \subseteq S$ 
  unfolding find-pivots-pivots-capped-def by (auto split: if-splits)
have P-reaches:  $\bigwedge x. x \in ?P \implies \text{reachable } s \ x$ 
  using P-subset-S less.premis(4) by blast
have P-k-cap:  $k * \text{card } ?P \leq \text{cap}$ 
  by (rule find-pivots-pivots-capped-scaled-card-le
    [OF S-subset less.premis(6)])
have P-anti: tree-antichain ?P
  by (rule find-pivots-pivots-capped-tree-antichain[OF less.premis(7)])
have child-bound:
   $\bigwedge c\text{-child } U\text{-child } S\text{-child } B\text{-child } d\text{-child } B\text{-child}'.$ 
   $\llbracket \text{exact-split-range-costed-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l0 \ ?d\text{-fp } S\text{-child } B\text{-child}$ 
     $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child};$ 
     $\text{bmssp-pre-full } ?d\text{-fp } S\text{-child } B\text{-child};$ 
     $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x;$ 
     $\text{card } S\text{-child} \leq M\text{-of } l0;$ 
     $\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (?d\text{-fp } x) \ B\text{-child};$ 
     $k * \text{card } S\text{-child} \leq \text{cap};$ 
     $\text{tree-antichain } S\text{-child} \rrbracket$ 
     $\implies c\text{-child} \leq \text{level-range-cost-bound } A \ (R + l0 * H)$ 
     $(2 * l0 + 1) \ U\text{-child}$ 
proof –
  fix c-child U-child S-child B-child d-child B-child'
  assume child:
    exact-split-range-costed-bmssp  $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l0 \ ?d\text{-fp } S\text{-child } B\text{-child}$ 
     $d\text{-child } B\text{-child}' \ U\text{-child } c\text{-child}$ 
  and child-pre: bmssp-pre-full ?d-fp S-child B-child
  and child-reaches:  $\bigwedge x. x \in S\text{-child} \implies \text{reachable } s \ x$ 
  and child-below:
     $\bigwedge x. x \in S\text{-child} \implies \text{below-bound } (?d\text{-fp } x) \ B\text{-child}$ 
  and child-k-cap:  $k * \text{card } S\text{-child} \leq \text{cap}$ 
  and child-anti: tree-antichain S-child
have M-cap-child:  $\bigwedge i. i \leq l0 \implies M\text{-of } i \leq \text{cap}$ 
  using less.premis(8) Suc by simp
show c-child  $\leq \text{level-range-cost-bound } A \ (R + l0 * H)$ 
   $(2 * l0 + 1) \ U\text{-child}$ 
  by (rule less.IH[of l0 ?d-fp S-child B-child d-child B-child'
    U-child c-child, OF - child sound-fp child-pre child-reaches
    child-below child-k-cap child-anti M-cap-child])
    (simp add: Suc)
qed
have loop-edge-budget:
   $t * \text{sum-list } (\text{map } \text{card } (\text{range-tree-child-edge-range-list } ?P \ a \ \text{betas } bs))$ 
   $\leq H * \text{card } (\text{outgoing-edges } U\text{-loop})$ 

```

```

    by (rule edge-budget[OF 1(4) sound-fp pivot-pre P-reaches])
  have scan-insert:
    fp-iter-capped-scan-cost k cap d S S B + c-insert ≤
      A * card U
  by (rule step-budget[OF refl 1(3) 1(4) 1(5) 1(1)
    less.premis(2) less.premis(3) less.premis(4) less.premis(5)
    less.premis(6) less.premis(7)])
  have trace:
    concrete-partition-loop-trace ?P B a bs d' B' Us U-loop
  proof -
    have split-loop:
      split-range-costed-partition-loop-state Δ M-of t h k cap l0
      ?d-fp ?P B d'
      (label-partition-view ?d-fp ?P) a betas bs B' Us U-loop c-loop
      child-costs-loop
    using exact-split-range-costed-refines-split-range-costed(1)[OF 1(4)]
    by simp
  show ?thesis
    by (rule split-range-costed-partition-loop-state-trace
      [OF split-loop sound-fp pivot-pre P-reaches])
  qed
  have loop-post: bmssp-post-full ?d-fp ?P B d' B' U-loop
    by (rule concrete-partition-loop-trace-post[OF trace])
  have finite-U-loop: finite U-loop
    using loop-post unfolding bmssp-post-full-def by simp
  have finite-W: finite ?W
    by simp
  have finite-U: finite U
    using 1(1) finite-U-loop finite-W by simp
  have M-cap-l0: M-of l0 ≤ cap
    using less.premis(8) Suc by simp
  have step:
    c ≤ A * Suc (Suc (2 * l0 + 1)) * card U +
      (R + l0 * H + H) * card (outgoing-edges U)
  by (rule exact-split-range-costed-nonbase-step-closes-level-bound-from-child-
    bound-with-invariants-and-edge-budget
      [OF 1(4) sound-fp pivot-pre P-reaches P-k-cap P-anti child-bound
        M-cap-l0 source-factor k-pos loop-edge-budget 1(1) finite-U
        scan-insert 1(2)])
  show ?thesis
    using step Suc unfolding level-range-cost-bound-def
    by (simp add: algebra-simps)
  qed
qed
qed

```

theorem *finite-initial-label-exact-split-range-costed-top-level-correct-and-closed-refined-graph-bound-level-cap:*

assumes *all-reachable:* $\bigwedge v. v \in V \implies \text{reachable } s \ v$

```

and degree: edge-outdegree-le  $\Delta$ 
and degree-factor:  $\Delta \leq A$ 
and R-pos:  $0 < R$ 
and insert-factor:  $t \leq A * k$ 
and insert-scaled-factor:  $t \leq A\text{-insert} * k$ 
and seen-scaled-factor:  $k * \Delta + A\text{-insert} \leq 2 * A$ 
and source-factor:  $Suc\ h \leq 2 * A$ 
and k-pos:  $0 < k$ 
and edge-budget:
   $\bigwedge l' d P B d' D a\ betas\ bs\ B'\ Us\ U\text{-loop}\ c\text{-loop}\ child\text{-costs}.$ 
    exact-split-range-costed-partition-loop-state  $\Delta\ (bmssp\text{-level-cap}\ k\ t)\ t\ h\ k$ 
       $(bmssp\text{-level-cap}\ k\ t\ l)\ l'\ d\ P\ B\ d'\ D\ a\ betas\ bs\ B'$ 
       $Us\ U\text{-loop}\ c\text{-loop}\ child\text{-costs} \implies$ 
      sound-label  $d \implies$ 
      bmssp-pre-full  $d\ P\ B \implies$ 
       $(\bigwedge x. x \in P \implies reachable\ s\ x) \implies$ 
       $t * sum\text{-list}\ (map\ card\ (range\text{-tree-child-edge-range-list}\ P\ a\ betas\ bs))$ 
       $\leq H * card\ (outgoing\text{-edges}\ U\text{-loop})$ 
and run:
  exact-split-range-costed-bmssp  $\Delta\ (bmssp\text{-level-cap}\ k\ t)\ t\ h\ k$ 
     $(bmssp\text{-level-cap}\ k\ t\ l)\ l$ 
    finite-initial-label  $\{s\}\ Infinity\ d'\ Infinity\ U\ c$ 
shows sssp-correct  $d' \wedge$ 
   $c \leq 2 * A * (2 * l + 1) * vertex\text{-count} + (R + l * H) * edge\text{-count}$ 
proof
  show sssp-correct  $d'$ 
    by (rule finite-initial-label-exact-split-range-costed-top-level-correct
      [OF all-reachable run])
next
  have pre: bmssp-pre-full finite-initial-label  $\{s\}\ Infinity$ 
    using all-reachable finite-initial-label-source-complete
    by (rule top-bmssp-pre-full)
  have sound: sound-label finite-initial-label
    using finite-initial-label-sound[OF all-reachable] .
  have S-reaches:  $\bigwedge x. x \in \{s\} \implies reachable\ s\ x$ 
    using all-reachable source-in-V by blast
  have below:  $\bigwedge x. x \in \{s\} \implies$ 
    below-bound (finite-initial-label  $x\ Infinity$ )
    by simp
  have top-cap:  $k * card\ \{s\} \leq bmssp\text{-level-cap}\ k\ t\ l$ 
proof –
  have one-le:  $1 \leq bmssp\text{-level-width}\ t\ l$ 
    unfolding bmssp-level-width-def by simp
  have  $k * 1 \leq k * bmssp\text{-level-width}\ t\ l$ 
    by (rule mult-left-mono[OF one-le]) simp
  then show ?thesis
    unfolding bmssp-level-cap-def by simp
qed
have top-anti: tree-antichain  $\{s\}$ 

```

```

    by simp
  have range-bound:
     $c \leq \text{level-range-cost-bound } (2 * A) (R + l * H) (2 * l + 1) U$ 
  proof (rule exact-split-range-costed-bmssp-level-bound-from-local-budgets-with-
    invariants-and-edge-budget
    [where  $A = 2 * A$  and  $R = R$  and  $H = H$ ,
      OF - R-pos source-factor k-pos - - run sound pre S-reaches below
      top-cap top-anti])
    show  $\Delta \leq 2 * A$ 
    using degree-factor by simp
  next
  fix i
  assume  $i \leq l$ 
  then show  $\text{bmssp-level-cap } k \ t \ i \leq \text{bmssp-level-cap } k \ t \ l$ 
    by (rule bmssp-level-cap-mono)
  next
  fix  $l' \ d \ P \ B \ d' \ D \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs}$ 
  assume loop:
    exact-split-range-costed-partition-loop-state  $\Delta$ 
    ( $\text{bmssp-level-cap } k \ t$ )  $t \ h \ k$  ( $\text{bmssp-level-cap } k \ t \ l$ )  $l'$ 
     $d \ P \ B \ d' \ D \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs}$ 
  and sound-l: sound-label  $d$ 
  and pre-l:  $\text{bmssp-pre-full } d \ P \ B$ 
  and reaches-l:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
  show  $t * \text{sum-list } (\text{map card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$ 
     $\leq H * \text{card } (\text{outgoing-edges } U\text{-loop})$ 
    by (rule edge-budget[OF loop sound-l pre-l reaches-l])
  next
  fix  $l' \ d \ S \ B \ D \ c\text{-insert } d' \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs } U$ 
  assume D-def:
     $D = \text{label-partition-view}$ 
    ( $\text{find-pivots-label-capped } k$  ( $\text{bmssp-level-cap } k \ t \ l$ )  $d \ S \ B$ )
    ( $\text{find-pivots-pivots-capped } k$  ( $\text{bmssp-level-cap } k \ t \ l$ )  $d \ S \ B$ )
  and insert:
    partition-initial-insert-cost-bound  $c\text{-insert } t$ 
    ( $\text{find-pivots-pivots-capped } k$  ( $\text{bmssp-level-cap } k \ t \ l$ )  $d \ S \ B$ )
  and loop:
    exact-split-range-costed-partition-loop-state  $\Delta$ 
    ( $\text{bmssp-level-cap } k \ t$ )  $t \ h \ k$  ( $\text{bmssp-level-cap } k \ t \ l$ )  $l'$ 
    ( $\text{find-pivots-label-capped } k$  ( $\text{bmssp-level-cap } k \ t \ l$ )  $d \ S \ B$ )
    ( $\text{find-pivots-pivots-capped } k$  ( $\text{bmssp-level-cap } k \ t \ l$ )  $d \ S \ B$ )  $B \ d' \ D$ 
     $a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs}$ 
  and complete:
    complete-on  $d'$ 
     $\{v \in \text{bound-tree } S \ B'\}$ 
    ( $\text{find-pivots-label-capped } k$  ( $\text{bmssp-level-cap } k \ t \ l$ )  $d \ S \ B \ v =$ 
       $\text{dist } s \ v$ )
  and U-def:
     $U = U\text{-loop} \cup$ 

```

```

    {v ∈ bound-tree S B'.
      find-pivots-label-capped k (bmssp-level-cap k t l) d S B v =
        dist s v}
  and sound-s: sound-label d
  and pre-s: bmssp-pre-full d S B
  and reaches-s:  $\bigwedge x. x \in S \implies \text{reachable } s \ x$ 
  and below-s:  $\bigwedge x. x \in S \implies \text{below-bound } (d \ x) \ B$ 
  and S-k-cap:  $k * \text{card } S \leq \text{bmssp-level-cap } k \ t \ l$ 
  and anti: tree-antichain S
have split-loop:
  split-range-costed-partition-loop-state  $\Delta$  (bmssp-level-cap k t) t h k
    (bmssp-level-cap k t l) l'
    (find-pivots-label-capped k (bmssp-level-cap k t l) d S B)
    (find-pivots-pivots-capped k (bmssp-level-cap k t l) d S B) B d' D
    a betas bs B' Us U-loop c-loop child-costs
  by (rule exact-split-range-costed-refines-split-range-costed(1)[OF loop])
have seen-success:
  B' = B  $\implies$ 
    card (find-pivots-seen-capped k (bmssp-level-cap k t l) d S B)
       $\leq \text{card } U$ 
  by (rule exact-split-range-costed-step-seen-success
    [OF loop U-def sound-s pre-s reaches-s below-s])
show fp-iter-capped-scan-cost k (bmssp-level-cap k t l) d S S B +
  c-insert  $\leq 2 * A * \text{card } U$ 
  by (rule split-range-costed-capped-step-scan-insert-budget-from-scaled-seen-or-
threshold
    [OF degree degree-factor insert-factor insert-scaled-factor
      seen-scaled-factor insert split-loop U-def sound-s pre-s reaches-s
      S-k-cap anti k-pos seen-success])
qed
have post-full:
  bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
  by (rule exact-split-range-costed-bmssp-correct[OF run sound pre S-reaches])
have post: bmssp-post finite-initial-label {s} Infinity d' Infinity U
  using bmssp-post-full-imp-post[OF post-full] .
have U-V: U = V
  using post bound-tree-source-infinity[OF all-reachable]
  unfolding bmssp-post-def by auto
have graph-bound:
  level-range-cost-bound (2 * A) (R + l * H) (2 * l + 1) U
     $\leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * H) * \text{edge-count}$ 
proof -
  have U-card: card U = vertex-count
  unfolding U-V vertex-count-def by simp
  have out-le: card (outgoing-edges U)  $\leq \text{edge-count}$ 
  by (rule edge-count-outgoing-bound)
  have out-term:
    (R + l * H) * card (outgoing-edges U)  $\leq (R + l * H) * \text{edge-count}$ 
  using out-le by simp

```

```

show ?thesis
  unfolding level-range-cost-bound-def
  using U-card out-term by (simp add: algebra-simps)
qed
show  $c \leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * H) * \text{edge-count}$ 
  using range-bound graph-bound by linarith
qed

corollary finite-initial-label-exact-split-range-costed-top-level-correct-and-closed-
bmssp-refined-graph-time-bound-level-cap:
assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
and degree: edge-outdegree-le  $\Delta$ 
and degree-factor:  $\Delta \leq A$ 
and R-pos:  $0 < R$ 
and insert-factor:  $t \leq A * k$ 
and insert-scaled-factor:  $t \leq A\text{-insert} * k$ 
and seen-scaled-factor:  $k * \Delta + A\text{-insert} \leq 2 * A$ 
and source-factor:  $\text{Suc } h \leq 2 * A$ 
and k-pos:  $0 < k$ 
and edge-budget:
 $\bigwedge l' \ d \ P \ B \ d' \ D \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs}.$ 
  exact-split-range-costed-partition-loop-state  $\Delta \ (\text{bmssp-level-cap } k \ t) \ t \ h \ k$ 
   $(\text{bmssp-level-cap } k \ t \ l) \ l' \ d \ P \ B \ d' \ D \ a \ \text{betas } bs \ B'$ 
   $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$ 
  sound-label  $d \implies$ 
  bmssp-pre-full  $d \ P \ B \implies$ 
   $(\bigwedge x. x \in P \implies \text{reachable } s \ x) \implies$ 
   $t * \text{sum-list } (\text{map } \text{card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$ 
   $\leq H * \text{card } (\text{outgoing-edges } U\text{-loop})$ 
and run:
  exact-split-range-costed-bmssp  $\Delta \ (\text{bmssp-level-cap } k \ t) \ t \ h \ k$ 
   $(\text{bmssp-level-cap } k \ t \ l) \ l$ 
  finite-initial-label  $\{s\} \ \text{Infinity } d' \ \text{Infinity } U \ c$ 
shows sssp-correct  $d' \wedge$ 
 $c \leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) \ (\lambda-. R) \ (\lambda-. H)$ 
 $(\lambda-. l) \ (\lambda-. t) \ (\lambda-. \text{edge-count}) \ \text{vertex-count}$ 
proof –
have graph:
  sssp-correct  $d' \wedge$ 
 $c \leq 2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * H) * \text{edge-count}$ 
by (rule finite-initial-label-exact-split-range-costed-top-level-correct-and-closed-
refined-graph-bound-level-cap
  [OF all-reachable degree degree-factor R-pos insert-factor insert-scaled-factor
  seen-scaled-factor source-factor k-pos edge-budget run])
have bound:
 $2 * A * (2 * l + 1) * \text{vertex-count} + (R + l * H) * \text{edge-count}$ 
 $\leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) \ (\lambda-. R) \ (\lambda-. H)$ 
 $(\lambda-. l) \ (\lambda-. t) \ (\lambda-. \text{edge-count}) \ \text{vertex-count}$ 
unfolding bmssp-refined-graph-time-bound-def by simp

```

```

show ?thesis
  using graph bound by linarith
qed

corollary finite-initial-label-exact-split-range-costed-top-level-correct-and-closed-
bmssp-refined-graph-time-bound-level-cap-trivial-edge-budget:
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
  and degree: edge-outdegree-le  $\Delta$ 
  and degree-factor:  $\Delta \leq A$ 
  and R-pos:  $0 < R$ 
  and insert-factor:  $t \leq A * k$ 
  and insert-scaled-factor:  $t \leq A\text{-insert} * k$ 
  and seen-scaled-factor:  $k * \Delta + A\text{-insert} \leq 2 * A$ 
  and source-factor:  $\text{Suc } h \leq 2 * A$ 
  and k-pos:  $0 < k$ 
  and run:
    exact-split-range-costed-bmssp  $\Delta$  (bmssp-level-cap k t) t h k
    (bmssp-level-cap k t l) l
    finite-initial-label {s} Infinity d' Infinity U c
  shows sssp-correct d'  $\wedge$ 
    c  $\leq$  bmssp-refined-graph-time-bound ( $\lambda\cdot. A$ ) ( $\lambda\cdot. R$ ) ( $\lambda\cdot. t$ )
    ( $\lambda\cdot. l$ ) ( $\lambda\cdot. t$ ) ( $\lambda\cdot. \text{edge-count}$ ) vertex-count
proof (rule finite-initial-label-exact-split-range-costed-top-level-correct-and-closed-
bmssp-refined-graph-time-bound-level-cap
  [OF all-reachable degree degree-factor R-pos insert-factor insert-scaled-factor
  seen-scaled-factor source-factor k-pos - run])
fix l' d P B d' D a betas bs B' Us U-loop c-loop child-costs
assume loop:
  exact-split-range-costed-partition-loop-state  $\Delta$ 
  (bmssp-level-cap k t) t h k (bmssp-level-cap k t l) l'
  d P B d' D a betas bs B' Us U-loop c-loop child-costs
and sound-l: sound-label d
and pre-l: bmssp-pre-full d P B
and reaches-l:  $\bigwedge x. x \in P \implies \text{reachable } s \ x$ 
show t * sum-list (map card (range-tree-child-edge-range-list P a betas bs))
   $\leq$  t * card (outgoing-edges U-loop)
by (rule exact-split-range-costed-partition-loop-state-trivial-edge-budget
  [OF loop sound-l pre-l reaches-l])
qed

end

end

theory BMSSP-Partition-Data-Structure
  imports BMSSP-Partition-Pull-Bridge
begin

```


34 Concrete Partition Data Structure

The abstract partition interface uses a set of active keys together with a value function. A concrete implementation must be careful around *Pull*: the abstract contract removes keys but leaves the value function unchanged. The state below therefore stores a sorted active list and a value memory. Removing a key only removes it from the active list; the remembered value is retained for the abstraction relation.

record *'k partition-state* =
entries-of :: ('k × real) list
values-of :: 'k ⇒ real

definition *entry-keys* :: ('k × real) list ⇒ 'k set **where**
entry-keys xs = fst ' set xs

definition *partition-state-invar* :: 'k partition-state ⇒ bool **where**
partition-state-invar P ⇔
distinct (map fst (entries-of P)) ∧
sorted-wrt (λp q. snd p ≤ snd q) (entries-of P) ∧
(∀ (x, b) ∈ set (entries-of P). values-of P x = b)

definition *partition-state-view* :: 'k partition-state ⇒ 'k partition-view **where**
partition-state-view P =
(⟦ keys-of = entry-keys (entries-of P), value-of = values-of P ⟧)

definition *partition-state-refines* ::
'k partition-state ⇒ 'k partition-view ⇒ bool **where**
partition-state-refines P D ⇔
partition-state-invar P ∧ *partition-state-view* P = D

definition *empty-partition-state* :: ('k ⇒ real) ⇒ 'k partition-state **where**
empty-partition-state v = (⟦ entries-of = [], values-of = v ⟧)

definition *partition-state-from-keys* ::
('k ⇒ real) ⇒ 'k list ⇒ 'k partition-state **where**
partition-state-from-keys d xs =
(⟦ entries-of = sort-key snd (map (λx. (x, d x)) xs),
values-of = d ⟧)

definition *partition-state-insert* ::
'k ⇒ real ⇒ 'k partition-state ⇒ 'k partition-state **where**
partition-state-insert x b P =
(let b' = (if x ∈ entry-keys (entries-of P)
then min (values-of P x) b else b);
rest = filter (λp. fst p ≠ x) (entries-of P)
in (⟦ entries-of = sort-key snd ((x, b') # rest),
values-of = (values-of P)(x := b') ⟧)

definition *partition-state-batch-prepend* ::
 $('k \times \text{real}) \text{ list} \Rightarrow 'k \text{ partition-state} \Rightarrow 'k \text{ partition-state}$ **where**
partition-state-batch-prepend $xs\ P =$
 $\text{fold } (\lambda(x, b)\ P'. \text{partition-state-insert } x\ b\ P')\ xs\ P$

definition *partition-state-pull-prefix* ::
 $\text{nat} \Rightarrow 'k \text{ partition-state} \Rightarrow ('k \times \text{real}) \text{ list}$ **where**
partition-state-pull-prefix $M\ P =$
 $(\text{let } xs = \text{entries-of } P \text{ in}$
 $\text{if length } xs \leq M \text{ then } xs$
 $\text{else takeWhile } (\lambda p. \text{snd } p < \text{snd } (xs\ !\ M))\ xs)$

definition *partition-state-pull-set* ::
 $\text{nat} \Rightarrow 'k \text{ partition-state} \Rightarrow 'k \text{ set}$ **where**
partition-state-pull-set $M\ P =$
 $\text{entry-keys } (\text{partition-state-pull-prefix } M\ P)$

definition *partition-state-pull-bound* ::
 $\text{nat} \Rightarrow \text{real} \Rightarrow 'k \text{ partition-state} \Rightarrow \text{real}$ **where**
partition-state-pull-bound $M\ B\ P =$
 $(\text{let } xs = \text{entries-of } P \text{ in if length } xs \leq M \text{ then } B \text{ else } \text{snd } (xs\ !\ M))$

definition *partition-state-delete-keys* ::
 $'k \text{ set} \Rightarrow 'k \text{ partition-state} \Rightarrow 'k \text{ partition-state}$ **where**
partition-state-delete-keys $S\ P =$
 $(\text{entries-of} = \text{filter } (\lambda p. \text{fst } p \notin S)\ (\text{entries-of } P),$
 $\text{values-of} = \text{values-of } P)$

definition *partition-state-pull* ::
 $\text{nat} \Rightarrow \text{real} \Rightarrow 'k \text{ partition-state} \Rightarrow$
 $'k \text{ set} \times \text{real} \times 'k \text{ partition-state}$ **where**
partition-state-pull $M\ B\ P =$
 $(\text{let } S = \text{partition-state-pull-set } M\ P;$
 $\text{beta} = \text{partition-state-pull-bound } M\ B\ P$
 $\text{in } (S, \text{beta}, \text{partition-state-delete-keys } S\ P))$

definition *costed-partition-state-insert* ::
 $\text{nat} \Rightarrow 'k \text{ partition-state} \Rightarrow 'k \Rightarrow \text{real} \Rightarrow \text{nat} \Rightarrow$
 $'k \text{ partition-state} \Rightarrow \text{bool}$ **where**
costed-partition-state-insert $t\ P\ x\ b\ c\ P' \longleftrightarrow$
 $P' = \text{partition-state-insert } x\ b\ P \wedge c = t$

definition *costed-partition-state-batch-prepend* ::
 $\text{nat} \Rightarrow 'k \text{ partition-state} \Rightarrow ('k \times \text{real}) \text{ list} \Rightarrow \text{nat} \Rightarrow$
 $'k \text{ partition-state} \Rightarrow \text{bool}$ **where**
costed-partition-state-batch-prepend $t\ P\ xs\ c\ P' \longleftrightarrow$
 $P' = \text{partition-state-batch-prepend } xs\ P \wedge c = t * \text{length } xs$

definition *costed-partition-state-pull* ::

$\text{nat} \Rightarrow \text{real} \Rightarrow 'k \text{ partition-state} \Rightarrow 'k \text{ set} \Rightarrow \text{real} \Rightarrow$
 $'k \text{ partition-state} \Rightarrow \text{nat} \Rightarrow \text{bool}$ **where**
 $\text{costed-partition-state-pull } M \ B \ P \ S \ \text{beta } P' \ c \longleftrightarrow$
 $\text{partition-state-pull } M \ B \ P = (S, \text{beta}, P') \wedge c = \text{card } S$

lemma *costed-partition-state-insert-exists*:
 $\exists c \ P'. \text{costed-partition-state-insert } t \ P \ x \ b \ c \ P'$
unfolding *costed-partition-state-insert-def* **by** *blast*

lemma *costed-partition-state-insert-deterministic*:
assumes *costed-partition-state-insert* $t \ P \ x \ b \ c \ P'$
and *costed-partition-state-insert* $t \ P \ x \ b \ c' \ P''$
shows $c = c' \wedge P' = P''$
using *assms* **unfolding** *costed-partition-state-insert-def* **by** *blast*

lemma *costed-partition-state-batch-prepend-exists*:
 $\exists c \ P'. \text{costed-partition-state-batch-prepend } t \ P \ xs \ c \ P'$
unfolding *costed-partition-state-batch-prepend-def* **by** *blast*

lemma *costed-partition-state-batch-prepend-deterministic*:
assumes *costed-partition-state-batch-prepend* $t \ P \ xs \ c \ P'$
and *costed-partition-state-batch-prepend* $t \ P \ xs \ c' \ P''$
shows $c = c' \wedge P' = P''$
using *assms* **unfolding** *costed-partition-state-batch-prepend-def* **by** *blast*

lemma *costed-partition-state-pull-exists*:
 $\exists S \ \text{beta } P' \ c. \text{costed-partition-state-pull } M \ B \ P \ S \ \text{beta } P' \ c$
unfolding *costed-partition-state-pull-def*
by (*metis prod-cases3*)

lemma *costed-partition-state-pull-deterministic*:
assumes *costed-partition-state-pull* $M \ B \ P \ S \ \text{beta } P' \ c$
and *costed-partition-state-pull* $M \ B \ P \ S' \ \text{beta}' \ P'' \ c'$
shows $S = S' \wedge \text{beta} = \text{beta}' \wedge P' = P'' \wedge c = c'$
using *assms* **unfolding** *costed-partition-state-pull-def* **by** *auto*

lemma *entry-keys-simps* [*simp*]:
 $\text{entry-keys } [] = \{\}$
 $\text{entry-keys } (x \# xs) = \text{insert } (\text{fst } x) (\text{entry-keys } xs)$
unfolding *entry-keys-def* **by** *auto*

lemma *entry-keys-filter-notin*:
 $\text{entry-keys } (\text{filter } (\lambda p. \text{fst } p \notin S) \ xs) = \text{entry-keys } xs - S$
unfolding *entry-keys-def* **by** *auto*

lemma *entry-keys-filter-neq*:
 $\text{entry-keys } (\text{filter } (\lambda p. \text{fst } p \neq x) \ xs) = \text{entry-keys } xs - \{x\}$
unfolding *entry-keys-def* **by** *auto*

```

lemma entry-keys-sort-key [simp]:
  entry-keys (sort-key f xs) = entry-keys xs
  unfolding entry-keys-def by simp

lemma entry-keys-insort-key [simp]:
  entry-keys (insort-key f x xs) = insert (fst x) (entry-keys xs)
  unfolding entry-keys-def by (simp add: set-insort-key)

lemma sorted-wrt-snd-sort-key [simp]:
  sorted-wrt ( $\lambda p\ q.\ \text{snd } p \leq \text{snd } q$ ) (sort-key snd xs)
proof –
  have sorted (map snd (sort-key snd xs))
    by (rule sorted-sort-key)
  then show ?thesis
    by (simp add: sorted-map)
qed

lemma distinct-map-fst-sort-key:
  assumes distinct (map fst xs)
  shows distinct (map fst (sort-key f xs))
proof –
  have distinct-sort: distinct (sort-key f xs)
    using assms unfolding distinct-map by simp
  have inj: inj-on fst (set (sort-key f xs))
    using assms unfolding distinct-map by simp
  show ?thesis
    unfolding distinct-map using distinct-sort inj by simp
qed

lemma empty-partition-state-invar [simp]:
  partition-state-invar (empty-partition-state v)
  unfolding empty-partition-state-def partition-state-invar-def by simp

lemma empty-partition-state-view [simp]:
  partition-state-view (empty-partition-state v) =
    ( $\emptyset$  keys-of =  $\{\}$ , value-of = v  $\emptyset$ )
  unfolding empty-partition-state-def partition-state-view-def by simp

lemma partition-state-from-keys-invar:
  assumes distinct xs
  shows partition-state-invar (partition-state-from-keys d xs)
proof –
  let ?ps = map ( $\lambda x.\ (x,\ d\ x)$ ) xs
  have base-distinct: distinct (map fst ?ps)
    using assms by (simp add: o-def)
  have distinct-ps: distinct (map fst (sort-key snd ?ps))
    by (rule distinct-map-fst-sort-key[OF base-distinct])
  have values-consistent:
     $\bigwedge x\ b.\ (x,\ b) \in \text{set } (\text{sort-key } \text{snd } ?ps) \implies d\ x = b$ 

```

```

    by auto
  have sorted-ps:
    sorted-wrt ( $\lambda p q. \text{snd } p \leq \text{snd } q$ ) (sort-key snd ?ps)
  by simp
  have consistent-ps:
     $\forall (x, b) \in \text{set } (\text{sort-key snd } ?ps). d \ x = b$ 
  using values-consistent by blast
  show ?thesis
    unfolding partition-state-from-keys-def partition-state-invar-def
    using distinct-ps sorted-ps consistent-ps by simp
qed

lemma partition-state-from-keys-view:
  partition-state-view (partition-state-from-keys d xs) =
    ( $\mid \text{keys-of} = \text{set } xs, \text{value-of} = d \mid$ )
  unfolding partition-state-from-keys-def partition-state-view-def entry-keys-def
  by (simp add: image-image)

lemma partition-state-from-keys-refines:
  assumes distinct xs
  shows partition-state-refines (partition-state-from-keys d xs)
    ( $\mid \text{keys-of} = \text{set } xs, \text{value-of} = d \mid$ )
  using partition-state-from-keys-invar[OF assms] partition-state-from-keys-view
  unfolding partition-state-refines-def by blast

lemma partition-state-insert-invar:
  assumes inv: partition-state-invar P
  shows partition-state-invar (partition-state-insert x b P)
proof -
  let ?b = if  $x \in \text{entry-keys } (\text{entries-of } P)$  then  $\min (\text{values-of } P \ x) \ b$  else  $b$ 
  let ?rest = filter ( $\lambda p. \text{fst } p \neq x$ ) (entries-of P)
  have distinct-rest: distinct (map fst ?rest)
    using inv unfolding partition-state-invar-def by (simp add: distinct-map-filter)
  have x-not-rest:  $x \notin \text{fst ' set } ?rest$ 
    by auto
  have distinct-new: distinct (map fst ((x, ?b) # ?rest))
    using distinct-rest x-not-rest by simp
  have values-consistent:
     $\bigwedge y c. (y, c) \in \text{set } (\text{sort-key snd } ((x, ?b) \# ?rest)) \implies$ 
     $((\text{values-of } P)(x := ?b)) \ y = c$ 
  proof -
    fix y c
    assume yc:  $(y, c) \in \text{set } (\text{sort-key snd } ((x, ?b) \# ?rest))$ 
    then have yc-set:  $(y, c) \in \text{set } ((x, ?b) \# ?rest)$ 
      by (metis set-sort)
    show  $((\text{values-of } P)(x := ?b)) \ y = c$ 
    proof (cases  $y = x$ )
    case True
    then have  $c = ?b$ 

```

```

    using yc-set x-not-rest by auto
  then show ?thesis
    using True by simp
next
  case False
  then have  $(y, c) \in \text{set } (\text{entries-of } P)$ 
    using yc-set by auto
  then have  $\text{values-of } P \ y = c$ 
    using inv unfolding partition-state-invar-def by auto
  then show ?thesis
    using False by simp
qed
qed
have distinct-sorted:
  distinct (map fst (sort-key snd ((x, ?b) # ?rest)))
  by (rule distinct-map-fst-sort-key[OF distinct-new])
have sorted-new:
  sorted-wrt  $(\lambda p \ q. \text{snd } p \leq \text{snd } q) \ (\text{sort-key snd } ((x, ?b) \# ?rest))$ 
proof -
  have sorted (map snd (sort-key snd ((x, ?b) # ?rest)))
    by (rule sorted-sort-key)
  then show ?thesis
    by (simp add: sorted-map)
qed
have consistent-new:
   $\forall (y, c) \in \text{set } (\text{sort-key snd } ((x, ?b) \# ?rest)).$ 
   $((\text{values-of } P)(x := ?b)) \ y = c$ 
  using values-consistent by blast
show ?thesis
  unfolding partition-state-insert-def partition-state-invar-def Let-def
  using distinct-sorted sorted-new consistent-new by simp
qed

theorem partition-state-insert-refines-min-update:
  assumes partition-state-invar P
  shows partition-state-view (partition-state-insert x b P) =
    min-update (partition-state-view P) x b
  unfolding partition-state-insert-def partition-state-view-def min-update-def Let-def
  by (simp add: entry-keys-filter-neq)

theorem partition-state-insert-refines-insert-spec:
  assumes inv: partition-state-invar P
  shows insert-spec (partition-state-view P) x b
    (partition-state-view (partition-state-insert x b P))
  unfolding partition-state-insert-refines-min-update[OF inv]
  by (rule min-update-insert-spec)

theorem partition-state-insert-refines-view:
  assumes ref: partition-state-refines P D

```

shows *partition-state-refines* (*partition-state-insert* x b P)
 (*min-update* D x b)
proof –
have *inv*: *partition-state-invar* P
and *view*: *partition-state-view* $P = D$
using *ref unfolding* *partition-state-refines-def* **by** *blast+*
have *inv'*: *partition-state-invar* (*partition-state-insert* x b P)
by (*rule* *partition-state-insert-invar*[*OF inv*])
have *view'*: *partition-state-view* (*partition-state-insert* x b P) =
min-update D x b
using *partition-state-insert-refines-min-update*[*OF inv*, *of x b*]
unfolding *view* .
show ?thesis
using *inv' view' unfolding* *partition-state-refines-def* **by** *blast*
qed

lemma *partition-state-batch-prepend-invar*:
assumes *partition-state-invar* P
shows *partition-state-invar* (*partition-state-batch-prepend* xs P)
using *assms*
proof (*induction xs arbitrary: P*)
case *Nil*
then show ?case
unfolding *partition-state-batch-prepend-def* **by** *simp*
next
case (*Cons* xb xs)
obtain x b **where** $xb = (x, b)$
by *force*
have *step*: *partition-state-invar* (*partition-state-insert* x b P)
by (*rule* *partition-state-insert-invar*[*OF Cons.prem*])
have *unfold-step*:
partition-state-batch-prepend ($xb \# xs$) $P =$
partition-state-batch-prepend xs (*partition-state-insert* x b P)
unfolding *partition-state-batch-prepend-def* xb **by** *simp*
show ?case
unfolding *unfold-step*
by (*rule* *Cons.IH*[*OF step*])
qed

theorem *partition-state-batch-prepend-refines-batch-min-update*:
assumes *partition-state-invar* P
shows *partition-state-view* (*partition-state-batch-prepend* xs P) =
batch-min-update (*partition-state-view* P) xs
using *assms*
proof (*induction xs arbitrary: P*)
case *Nil*
then show ?case
unfolding *partition-state-batch-prepend-def* *batch-min-update-def* **by** *simp*
next

```

case (Cons xb xs)
obtain x b where xb: xb = (x, b)
  by force
have step-inv: partition-state-invar (partition-state-insert x b P)
  by (rule partition-state-insert-invar[OF Cons.prems])
have step-view:
  partition-state-view (partition-state-insert x b P) =
    min-update (partition-state-view P) x b
  by (rule partition-state-insert-refines-min-update[OF Cons.prems])
have unfold-state:
  partition-state-batch-prepend (xb # xs) P =
    partition-state-batch-prepend xs (partition-state-insert x b P)
  unfolding partition-state-batch-prepend-def xb by simp
have unfold-view:
  batch-min-update (partition-state-view P) (xb # xs) =
    batch-min-update (min-update (partition-state-view P) x b) xs
  unfolding batch-min-update-def xb by simp
show ?case
  unfolding unfold-state unfold-view
  using Cons.IH[OF step-inv] step-view by simp
qed

theorem partition-state-batch-prepend-refines-view:
  assumes ref: partition-state-refines P D
  shows partition-state-refines (partition-state-batch-prepend xs P)
    (batch-min-update D xs)
proof –
  have inv: partition-state-invar P
  and view: partition-state-view P = D
  using ref unfolding partition-state-refines-def by blast
  have inv': partition-state-invar (partition-state-batch-prepend xs P)
  by (rule partition-state-batch-prepend-invar[OF inv])
  have view': partition-state-view (partition-state-batch-prepend xs P) =
    batch-min-update D xs
  using partition-state-batch-prepend-refines-batch-min-update[OF inv, of xs]
  unfolding view .
  show ?thesis
  using inv' view' unfolding partition-state-refines-def by blast
qed

lemma partition-state-delete-keys-invar:
  assumes partition-state-invar P
  shows partition-state-invar (partition-state-delete-keys S P)
  using assms
  unfolding partition-state-delete-keys-def partition-state-invar-def
  by (auto simp: distinct-map-filter sorted-wrt-filter)

lemma partition-state-delete-keys-view:
  partition-state-view (partition-state-delete-keys S P) =

```



```

    (| keys-of = keys-of (partition-state-view P) - S,
      value-of = value-of (partition-state-view P) |)
  unfolding partition-state-delete-keys-def partition-state-view-def
  by (auto simp: entry-keys-def)

lemma partition-state-pull-prefix-subset:
  set (partition-state-pull-prefix M P) ⊆ set (entries-of P)
  unfolding partition-state-pull-prefix-def by (auto simp: Let-def dest: set-
takeWhileD)

lemma partition-state-pull-set-subset:
  partition-state-pull-set M P ⊆ keys-of (partition-state-view P)
  using partition-state-pull-prefix-subset[of M P]
  unfolding partition-state-pull-set-def partition-state-view-def entry-keys-def by
auto

lemma partition-state-pull-prefix-card-le:
  assumes inv: partition-state-invar P
  shows card (partition-state-pull-set M P) ≤ M
proof (cases length (entries-of P) ≤ M)
case True
  have card (partition-state-pull-set M P) =
    card (set (map fst (entries-of P)))
  unfolding partition-state-pull-set-def partition-state-pull-prefix-def
    entry-keys-def using True by (simp add: Let-def)
  also have ... = length (entries-of P)
  proof -
    have distinct (map fst (entries-of P))
      using inv unfolding partition-state-invar-def by blast
    then have card (set (map fst (entries-of P))) =
      length (map fst (entries-of P))
    by (rule distinct-card)
    then show ?thesis
    by simp
  qed
  finally show ?thesis
  using True by simp
next
case False
  let ?xs = entries-of P
  let ?beta = snd (?xs ! M)
  let ?pref = takeWhile (λp. snd p < ?beta) ?xs
  have M-lt: M < length ?xs
    using False by simp
  have len-pref: length ?pref ≤ M
  proof (rule ccontr)
    assume ¬ length ?pref ≤ M
    then have M-pref: M < length ?pref
      by simp

```

```

have ?pref ! M = ?xs ! M
  by (rule takeWhile-nth[OF M-pref])
moreover have ?pref ! M ∈ set ?pref
  using M-pref by simp
ultimately show False
  using set-takeWhileD[of ?pref ! M λp. snd p < ?beta ?xs]
  by simp
qed
have card (partition-state-pull-set M P) ≤ length ?pref
proof -
  have card (fst ' set ?pref) ≤ card (set ?pref)
    by (rule card-image-le) simp
  also have ... ≤ length ?pref
    by (rule card-length)
  finally show ?thesis
    unfolding partition-state-pull-set-def partition-state-pull-prefix-def
    entry-keys-def using False by (simp add: Let-def)
qed
then show ?thesis
  using len-pref by linarith
qed

lemma sorted-wrt-takeWhile-less-eq-filter:
  fixes f :: 'a ⇒ 'b::linorder
  assumes sorted: sorted-wrt (λx y. f x ≤ f y) xs
  shows takeWhile (λx. f x < t) xs = filter (λx. f x < t) xs
proof (rule takeWhile-eq-filter)
  fix x
  assume x-drop: x ∈ set (dropWhile (λx. f x < t) xs)
  let ?pref = takeWhile (λx. f x < t) xs
  let ?drop = dropWhile (λx. f x < t) xs
  obtain i where i: i < length ?drop x = ?drop ! i
    using x-drop by (auto simp: in-set-conv-nth)
  have xs-split: ?pref @ ?drop = xs
    by simp
  have len-xs: length xs = length ?pref + length ?drop
    by (metis length-append xs-split)
  then have idx: length ?pref + i < length xs
    using i len-xs by simp
  have pref-len: length ?pref < length xs
    using idx by simp
  have not-first: ¬ f (xs ! length ?pref) < t
    using nth-length-takeWhile[of λx. f x < t xs] pref-len by simp
  have f (xs ! length ?pref) ≤ f (xs ! (length ?pref + i))
  proof (cases i)
    case 0
    then show ?thesis by simp
  next
    case (Suc j)

```

```

have length ?pref < length ?pref + i
  using Suc by simp
then show ?thesis
  by (rule sorted-wrt-nth-less[OF sorted]) (use pref-len idx in simp-all)
qed
moreover have x = xs ! (length ?pref + i)
proof -
  have ?drop ! i = (?pref @ ?drop) ! (length ?pref + i)
    by (simp only: nth-append-length-plus)
  also have ... = xs ! (length ?pref + i)
    using xs-split by simp
  finally show ?thesis
    using i by simp
qed
ultimately show ¬ f x < t
  using not-first by auto
qed

lemma partition-state-pull-set-exact:
  assumes inv: partition-state-invar P
  and upper:  $\bigwedge u. u \in \text{keys-of } (\text{partition-state-view } P) \implies \text{value-of } (\text{partition-state-view } P) \ u < B$ 
  shows partition-state-pull-set M P =
    {u ∈ keys-of (partition-state-view P).
     value-of (partition-state-view P) u < partition-state-pull-bound M B P}
proof (cases length (entries-of P) ≤ M)
case True
  then have pull-all:
    partition-state-pull-set M P = keys-of (partition-state-view P)
  unfolding partition-state-pull-set-def partition-state-pull-prefix-def
    partition-state-view-def by (simp add: Let-def)
  moreover have partition-state-pull-bound M B P = B
    using True unfolding partition-state-pull-bound-def by (simp add: Let-def)
  ultimately show ?thesis
    using upper by auto
next
case False
  let ?xs = entries-of P
  let ?beta = snd (?xs ! M)
  have beta: partition-state-pull-bound M B P = ?beta
    using False unfolding partition-state-pull-bound-def by (simp add: Let-def)
  have sorted: sorted-wrt (λp q. snd p ≤ snd q) ?xs
    using inv unfolding partition-state-invar-def by blast
  have filter-eq:
    takeWhile (λp. snd p < ?beta) ?xs =
      filter (λp. snd p < ?beta) ?xs
  by (rule sorted-wrt-takeWhile-less-eq-filter[OF sorted])
  have consistent:
     $\bigwedge x \ b. (x, b) \in \text{set } ?xs \implies \text{values-of } P \ x = b$ 

```

```

    using inv unfolding partition-state-invar-def by auto
  show ?thesis
    unfolding partition-state-pull-set-def partition-state-pull-prefix-def
      partition-state-view-def entry-keys-def beta using False filter-eq consistent
    by (auto simp: Let-def)
qed

theorem partition-state-pull-refines-pull-separates:
  assumes inv: partition-state-invar P
    and upper:  $\bigwedge u. u \in \text{keys-of } (\text{partition-state-view } P) \implies$ 
      value-of (partition-state-view P) u < B
    and pull: partition-state-pull M B P = (S, beta, P')
  shows pull-separates (partition-state-view P) M B S beta
    (partition-state-view P')
proof -
  have S-def: S = partition-state-pull-set M P
    and beta-def: beta = partition-state-pull-bound M B P
    and P'-def: P' = partition-state-delete-keys S P
    using pull unfolding partition-state-pull-def by (auto simp: Let-def)
  have S-subset:
    S  $\subseteq$  keys-of (partition-state-view P)
    unfolding S-def by (rule partition-state-pull-set-subset)
  have card-S: card S  $\leq$  M
    unfolding S-def by (rule partition-state-pull-prefix-card-le[OF inv])
  have P'-keys:
    keys-of (partition-state-view P') = keys-of (partition-state-view P) - S
    unfolding P'-def partition-state-delete-keys-view by simp
  have P'-values:
    value-of (partition-state-view P') = value-of (partition-state-view P)
    unfolding P'-def partition-state-delete-keys-view by simp
  have exact:
    S = {u  $\in$  keys-of (partition-state-view P).
      value-of (partition-state-view P) u < beta}
    unfolding S-def beta-def
    by (rule partition-state-pull-set-exact[OF inv upper])
  have pulled-before-remaining:
     $\bigwedge u v. \llbracket u \in S; v \in \text{keys-of } (\text{partition-state-view } P') \rrbracket \implies$ 
      value-of (partition-state-view P) u  $\leq$ 
      value-of (partition-state-view P') v
  proof -
    fix u v
    assume u: u  $\in$  S and v: v  $\in$  keys-of (partition-state-view P')
    have value-of (partition-state-view P) u < beta
      using u exact by blast
    moreover have  $\neg$  value-of (partition-state-view P) v < beta
      using v P'-keys exact by blast
    ultimately show
      value-of (partition-state-view P) u  $\leq$ 
      value-of (partition-state-view P') v
  end
end

```

```

    using P'-values by simp
qed
show ?thesis
proof (cases keys-of (partition-state-view P') = {})
  case True
  have beta-B: beta = B
  proof (cases length (entries-of P) ≤ M)
    case True
    then show ?thesis
      using beta-def unfolding partition-state-pull-bound-def by (simp add:
Let-def)
  next
  case False
  let ?xs = entries-of P
  have M-lt: M < length ?xs
    using False by simp
  have nth-key: fst (?xs ! M) ∈ keys-of (partition-state-view P)
    unfolding partition-state-view-def entry-keys-def using nth-mem[OF M-lt]
  by auto
  have nth-not-S: fst (?xs ! M) ∉ S
  proof -
    have beta-eq: beta = snd (?xs ! M)
      using beta-def False unfolding partition-state-pull-bound-def by (simp
add: Let-def)
    have value-eq:
      value-of (partition-state-view P) (fst (?xs ! M)) = snd (?xs ! M)
      using inv nth-mem[OF M-lt]
      unfolding partition-state-invar-def partition-state-view-def by auto
    show ?thesis
      using exact beta-eq value-eq by auto
  qed
  then have fst (?xs ! M) ∈ keys-of (partition-state-view P')
    using nth-key P'-keys by blast
  then show ?thesis
    using True by blast
qed
show ?thesis
  unfolding pull-separates-def
  using S-subset card-S P'-keys P'-values True beta-B
  pulled-before-remaining by auto
next
case False
have upper-pulled:
   $\bigwedge u. u \in S \implies \text{value-of } (\text{partition-state-view } P) \ u < \text{beta}$ 
  using exact by blast
have lower-remaining:
   $\bigwedge v. v \in \text{keys-of } (\text{partition-state-view } P') \implies$ 
   $\text{beta} \leq \text{value-of } (\text{partition-state-view } P') \ v$ 
  using exact P'-keys P'-values by fastforce

```

```

    show ?thesis
    unfolding pull-separates-def
    using S-subset card-S P'-keys P'-values False pulled-before-remaining
      upper-pulled lower-remaining by auto
  qed
qed

theorem partition-state-pull-refines-view:
  assumes ref: partition-state-refines P D
    and upper:  $\bigwedge u. u \in \text{keys-of } D \implies \text{value-of } D \ u < B$ 
    and pull: partition-state-pull M B P = (S, beta, P')
  defines D'  $\equiv$  ( $\lfloor \text{keys-of} = \text{keys-of } D - S, \text{value-of} = \text{value-of } D \rfloor$ )
  shows partition-state-refines P' D'  $\wedge$  pull-separates D M B S beta D'
proof -
  have inv: partition-state-invar P
    and view: partition-state-view P = D
    using ref unfolding partition-state-refines-def by blast+
  have S-def: S = partition-state-pull-set M P
    and P'-def: P' = partition-state-delete-keys S P
    using pull unfolding partition-state-pull-def by (auto simp: Let-def)
  have inv': partition-state-invar P'
    unfolding P'-def by (rule partition-state-delete-keys-invar[OF inv])
  have sep:
    pull-separates (partition-state-view P) M B S beta
      (partition-state-view P')
  proof (rule partition-state-pull-refines-pull-separates[OF inv - pull])
    fix u
    assume u  $\in$  keys-of (partition-state-view P)
    then show value-of (partition-state-view P) u < B
      using upper unfolding view by simp
  qed
  have view': partition-state-view P' = D'
  proof -
    show ?thesis
    unfolding P'-def D'-def partition-state-delete-keys-view view by simp
  qed
  show ?thesis
    using inv' view' sep unfolding partition-state-refines-def view D'-def by simp
qed

theorem partition-state-pull-invar:
  assumes inv: partition-state-invar P
    and pull: partition-state-pull M B P = (S, beta, P')
  shows partition-state-invar P'
  using pull partition-state-delete-keys-invar[OF inv]
  unfolding partition-state-pull-def by (auto simp: Let-def)

theorem costed-partition-state-insert-refines:
  assumes inv: partition-state-invar P

```

and *op*: *costed-partition-state-insert* *t P x b c P'*
shows *partition-state-invar P' ∧*
insert-spec (partition-state-view P) x b (partition-state-view P') ∧
partition-insert-cost-bound c t
proof –
have *P'-def*: *P' = partition-state-insert x b P*
and *c-def*: *c = t*
using *op* **unfolding** *costed-partition-state-insert-def* **by** *blast+*
have *inv'*: *partition-state-invar P'*
unfolding *P'-def* **by** (rule *partition-state-insert-invar[OF inv]*)
have *spec*:
insert-spec (partition-state-view P) x b (partition-state-view P')
unfolding *P'-def* **by** (rule *partition-state-insert-refines-insert-spec[OF inv]*)
have *cost*: *partition-insert-cost-bound c t*
unfolding *c-def* *partition-insert-cost-bound-def* **by** (rule *order-refl*)
show *?thesis*
using *inv' spec cost* **by** *blast*
qed

theorem *costed-partition-state-batch-prepend-refines*:
assumes *inv*: *partition-state-invar P*
and *op*: *costed-partition-state-batch-prepend t P xs c P'*
shows *partition-state-invar P' ∧*
partition-state-view P' = batch-min-update (partition-state-view P) xs ∧
partition-batch-cost-bound c t xs
proof –
have *P'-def*: *P' = partition-state-batch-prepend xs P*
and *c-def*: *c = t * length xs*
using *op* **unfolding** *costed-partition-state-batch-prepend-def* **by** *blast+*
have *inv'*: *partition-state-invar P'*
unfolding *P'-def* **by** (rule *partition-state-batch-prepend-invar[OF inv]*)
have *view*:
partition-state-view P' = batch-min-update (partition-state-view P) xs
unfolding *P'-def*
by (rule *partition-state-batch-prepend-refines-batch-min-update[OF inv]*)
have *cost*: *partition-batch-cost-bound c t xs*
unfolding *c-def* *partition-batch-cost-bound-def* **by** (rule *order-refl*)
show *?thesis*
using *inv' view cost* **by** *blast*
qed

theorem *costed-partition-state-pull-refines*:
assumes *inv*: *partition-state-invar P*
and *upper*: $\bigwedge u. u \in \text{keys-of } (\text{partition-state-view } P) \implies$
value-of (partition-state-view P) u < B
and *op*: *costed-partition-state-pull M B P S beta P' c*
shows *partition-state-invar P' ∧*
pull-separates (partition-state-view P) M B S beta
(partition-state-view P') ∧

partition-pull-cost-bound c S
proof –
 have *pull*: *partition-state-pull* M B $P = (S, \text{beta}, P')$
 and *c-def*: $c = \text{card } S$
 using *op unfolding* *costed-partition-state-pull-def* **by** *blast+*
 have *inv'*: *partition-state-invar* P'
 by (rule *partition-state-pull-invar*[*OF inv pull*])
 have *sep*:
pull-separates (*partition-state-view* P) M B S beta
 (*partition-state-view* P')
 by (rule *partition-state-pull-refines-pull-separates*[*OF inv upper pull*])
 have *cost*: *partition-pull-cost-bound* c S
 unfolding *c-def* *partition-pull-cost-bound-def* **by** (rule *order-refl*)
 show ?thesis
 using *inv' sep cost* **by** *blast*
qed

context *unique-shortest-digraph*
begin

lemma *partition-state-from-keys-label-view*:
partition-state-view (*partition-state-from-keys* d xs) =
label-partition-view d (*set* xs)
 unfolding *partition-state-from-keys-view* *label-partition-view-def* **by** *simp*

theorem *partition-state-pull-label-set-eq-split-below*:
 assumes *inv*: *partition-state-invar* P_D
 and *view*: *partition-state-view* $P_D = \text{label-partition-view } d$ S
 and *upper*: $\bigwedge u. u \in S \implies d\ u < B$
 and *pull*: *partition-state-pull* M B $P_D = (S\text{-pull}, \text{beta}, P_D')$
 shows $S\text{-pull} = \text{split-below } d$ S beta
proof –
 have *pull-sep*:
pull-separates (*partition-state-view* P_D) M B $S\text{-pull}$ beta
 (*partition-state-view* P_D')
proof (rule *partition-state-pull-refines-pull-separates*[*OF inv - pull*])
 fix u
 assume $u \in \text{keys-of } (\text{partition-state-view } P_D)$
 then show *value-of* (*partition-state-view* P_D) $u < B$
 using *upper* unfolding *view* **by** *simp*
qed
 show ?thesis
 using *pull-separates-label-set-eq-split-below*[*of d S M B S-pull beta*
partition-state-view P_D'] *pull-sep upper*
 unfolding *view* .
qed

theorem *partition-state-pull-establishes-lower-pre*:
 assumes *inv*: *partition-state-invar* P_D


```

    and view: partition-state-view  $P_D = \text{label-partition-view } d \ S$ 
    and pre: bmssp-pre-full  $d \ S \ (\text{Fin } B)$ 
    and upper:  $\bigwedge u. u \in S \implies d \ u < B$ 
    and pull: partition-state-pull  $M \ B \ P_D = (S\text{-pull}, \text{beta}, P_D')$ 
  shows bmssp-pre-full  $d \ S\text{-pull} \ (\text{Fin } \text{beta})$ 
proof -
  have pull-eq:  $S\text{-pull} = \text{split-below } d \ S \ \text{beta}$ 
    by (rule partition-state-pull-label-set-eq-split-below[OF inv view upper pull])
  have pull-sep:
    pull-separates (partition-state-view  $P_D$ )  $M \ B \ S\text{-pull} \ \text{beta}$ 
    (partition-state-view  $P_D'$ )
  proof (rule partition-state-pull-refines-pull-separates[OF inv - pull])
    fix u
    assume  $u \in \text{keys-of } (\text{partition-state-view } P_D)$ 
    then show value-of (partition-state-view  $P_D$ )  $u < B$ 
      using upper unfolding view by simp
    qed
  show ?thesis
    unfolding pull-eq
  proof (cases keys-of (partition-state-view  $P_D'$ ) = {})
    case True
    have  $\text{beta} = B$ 
      using pull-separates-empty-bound[OF pull-sep True] .
    moreover have  $\text{split-below } d \ S \ B = S$ 
      using upper unfolding split-below-def by auto
    then show bmssp-pre-full  $d \ (\text{split-below } d \ S \ \text{beta}) \ (\text{Fin } \text{beta})$ 
      using pre calculation by simp
  next
    case False
    obtain v where  $v: v \in \text{keys-of } (\text{partition-state-view } P_D')$ 
      using False by blast
    have beta-le-B:  $\text{beta} < B$ 
    proof -
      have beta-le:  $\text{beta} \leq \text{value-of } (\text{partition-state-view } P_D') \ v$ 
        by (rule pull-separates-nonempty-bound[OF pull-sep False v])
      have v-old:  $v \in S$ 
        using v pull-sep unfolding pull-separates-def view by auto
      have val-eq:  $\text{value-of } (\text{partition-state-view } P_D') \ v = d \ v$ 
        using pull-sep unfolding pull-separates-def view by auto
      show ?thesis
        using beta-le upper[OF v-old] val-eq by linarith
    qed
    show bmssp-pre-full  $d \ (\text{split-below } d \ S \ \text{beta}) \ (\text{Fin } \text{beta})$ 
      by (rule pull-minimum-pre-for-lower-split[OF pre]) (simp add: beta-le-B)
  qed
qed
end

```

```

end
theory BMSSP-Exact-Concrete-Cost
  imports BMSSP-Direct-Insert-Costed BMSSP-Partition-Data-Structure
begin

```

35 Exact Concrete Cost Runs

The existing direct-insert costed run relation records data-structure costs by upper-bound predicates. The relation below is a concrete-cost refinement layer: pulls and batch-prepends are performed by the concrete partition state operations, whose costs are exact. Its main theorem shows that every exact concrete run is accepted by the existing direct-insert costed relation, so all already-proved graph-time bounds apply to such runs.

Initial insertion is kept as a separate exact initial-state predicate. It records an exact insertion trace and the resulting abstract view; future totality work can instantiate it with a concrete enumeration of the pivot set.

This theory is not the final executable bucketed structure. It is the bridge from abstract cost predicates to concrete partition-state operations. The direct-insert relation says "there exists a pull cost satisfying the pull predicate" and similarly for batch costs. Here those costs are the values returned by *costed-partition-state-pull* and *costed-partition-state-batch-prepend*. The refinement theorem then proves that these exact operations satisfy the abstract cost relation.

The file also contains witness lemmas. They show that an initial partition can be built by inserting the pivot set, that terminal loop cases can be produced directly, and that nonterminal loop cases can be assembled from one concrete pull, one recursive child call, and three concrete batch prepends. These witnesses make the concrete-cost layer a useful target for later totality or executable-search work, even though the main Track 2 deliverable uses the bucketed amortised bounds.

```

inductive exact-partition-state-insert-list ::
  nat  $\Rightarrow$  'k partition-state  $\Rightarrow$  ('k  $\times$  real) list  $\Rightarrow$  nat  $\Rightarrow$ 
  'k partition-state  $\Rightarrow$  bool where
  Exact-Insert-List-Nil:
    exact-partition-state-insert-list t P [] 0 P
| Exact-Insert-List-Cons:
  costed-partition-state-insert t P x b c-x P1  $\Longrightarrow$ 
  exact-partition-state-insert-list t P1 xs c-tail P'  $\Longrightarrow$ 
  c = c-x + c-tail  $\Longrightarrow$ 
  exact-partition-state-insert-list t P ((x, b) # xs) c P'

```

Exact initial insertion is represented as a list trace. The trace is small enough to reason about directly: each element costs exactly t , preserves the partition invariant, and refines to a minimum-update on the abstract partition view. Building an initial partition from a distinct enumeration of

the pivot set therefore gives the abstract label partition view expected by the BMSSP loop.

lemma *batch-min-update-append* [simp]:
 $\text{batch-min-update } (\text{batch-min-update } D \text{ } xs) \text{ } ys = \text{batch-min-update } D \text{ } (xs @ ys)$
unfolding *batch-min-update-def* **by** *simp*

lemma *exact-partition-state-insert-list-cost*:
assumes *exact-partition-state-insert-list* $t \ P \ xs \ c \ P'$
shows $c = t * \text{length } xs$
using *assms*
proof (*induction*)
case *Exact-Insert-List-Nil*
then show ?case
by *simp*
next
case (*Exact-Insert-List-Cons* $t \ P \ x \ b \ c\text{-}x \ P_1 \ xs \ c\text{-tail } P' \ c$)
then show ?case
unfolding *costed-partition-state-insert-def* **by** *simp*
qed

lemma *exact-partition-state-insert-list-invar*:
assumes *inv*: *partition-state-invar* P
and *inserts*: *exact-partition-state-insert-list* $t \ P \ xs \ c \ P'$
shows *partition-state-invar* P'
using *inserts inv*
proof (*induction*)
case (*Exact-Insert-List-Nil* $t \ P$)
then show ?case
by *simp*
next
case (*Exact-Insert-List-Cons* $t \ P \ x \ b \ c\text{-}x \ P_1 \ xs \ c\text{-tail } P' \ c$)
have *inv1*: *partition-state-invar* P_1
using *costed-partition-state-insert-refines*
 $[OF \ \text{Exact-Insert-List-Cons.premis} \ \text{Exact-Insert-List-Cons.hyps}(1)]$
by *blast*
show ?case
by (*rule* *Exact-Insert-List-Cons.IH* [*OF inv1*])
qed

lemma *exact-partition-state-insert-list-batch-prepend*:
 $\text{exact-partition-state-insert-list } t \ P \ xs \ (t * \text{length } xs)$
 $(\text{partition-state-batch-prepend } xs \ P)$
proof (*induction xs arbitrary: P*)
case *Nil*
then show ?case
unfolding *partition-state-batch-prepend-def* **by** (*simp add: Exact-Insert-List-Nil*)
next
case (*Cons* $xb \ xs$)
obtain $x \ b$ **where** $xb: xb = (x, b)$

```

    by force
  have step:
    costed-partition-state-insert t P x b t (partition-state-insert x b P)
  unfolding costed-partition-state-insert-def by simp
  have tail:
    exact-partition-state-insert-list t (partition-state-insert x b P) xs
      (t * length xs)
      (partition-state-batch-prepend xs (partition-state-insert x b P))
  by (rule Cons.IH)
  have run:
    exact-partition-state-insert-list t P (xb # xs)
      (t + t * length xs)
      (partition-state-batch-prepend xs (partition-state-insert x b P))
  unfolding xb by (rule Exact-Insert-List-Cons[OF step tail refl])
  then show ?case
    unfolding xb partition-state-batch-prepend-def by simp
qed

context unique-shortest-digraph
begin

lemma batch-min-update-label-pairs-value-disjoint:
  assumes distinct: distinct xs
    and disjoint: set xs ∩ keys-of D = {}
  shows value-of (batch-min-update D (map (λx. (x, d x)) xs)) y =
    (if y ∈ set xs then d y else value-of D y)
  using distinct disjoint
proof (induction xs arbitrary: D y)
  case Nil
  then show ?case
    unfolding batch-min-update-def by simp
next
  case (Cons x xs)
  have x-not-D: x ∉ keys-of D
  using Cons.prem by auto
  have xs-disjoint:
    set xs ∩ keys-of (min-update D x (d x)) = {}
  using Cons.prem unfolding min-update-def by auto
  have tail:
    value-of
      (batch-min-update (min-update D x (d x))
        (map (λx. (x, d x)) xs)) y =
      (if y ∈ set xs then d y else value-of (min-update D x (d x)) y)
  by (rule Cons.IH) (use Cons.prem xs-disjoint in auto)
  have unfold:
    batch-min-update D (map (λx. (x, d x)) (x # xs)) =
      batch-min-update (min-update D x (d x)) (map (λx. (x, d x)) xs)
  unfolding batch-min-update-def by simp
  show ?case

```

```

proof (cases y = x)
  case True
  then have y  $\notin$  set xs
    using Cons.prem1 by simp
  then show ?thesis
    using tail x-not-D True unfolding unfold by (simp add: min-update-def)
next
  case False
  then show ?thesis
    using tail unfolding unfold by (simp add: min-update-def)
qed
qed

```

```

lemma min-update-label-partition-view-new:
  assumes x  $\notin$  S
  shows min-update (label-partition-view d S) x (d x) =
    label-partition-view d (insert x S)
  using assms unfolding min-update-def label-partition-view-def by simp

```

```

lemma batch-min-update-label-partition-view-disjoint:
  assumes distinct: distinct xs
  and disjoint: set xs  $\cap$  S = {}
  shows batch-min-update (label-partition-view d S)
    (map ( $\lambda x. (x, d x)$ ) xs) =
    label-partition-view d (S  $\cup$  set xs)
  using distinct disjoint
proof (induction xs arbitrary: S)
  case Nil
  then show ?case
    unfolding batch-min-update-def label-partition-view-def by simp
next
  case (Cons x xs)
  have x-not-S: x  $\notin$  S
    using Cons.prem1 by auto
  have xs-disjoint: set xs  $\cap$  insert x S = {}
    using Cons.prem1 by auto
  have step:
    min-update (label-partition-view d S) x (d x) =
      label-partition-view d (insert x S)
  by (rule min-update-label-partition-view-new[OF x-not-S])
  have tail:
    batch-min-update (label-partition-view d (insert x S))
      (map ( $\lambda x. (x, d x)$ ) xs) =
      label-partition-view d (insert x S  $\cup$  set xs)
  by (rule Cons.IH) (use Cons.prem1 xs-disjoint in auto)
  show ?case
    unfolding batch-min-update-def
    using step tail[unfolded batch-min-update-def] by auto
qed

```

lemma *batch-min-update-empty-label-view*:
assumes *distinct*: *distinct xs*
shows *batch-min-update* (*partition-state-view* (*empty-partition-state d*))
 $(\text{map } (\lambda x. (x, d \ x)) \ xs) =$
 $\text{label-partition-view } d \ (\text{set } xs)$
proof –
have *empty-view*:
 $\text{partition-state-view } (\text{empty-partition-state } d) = \text{label-partition-view } d \ \{\}$
unfolding *label-partition-view-def* **by** *simp*
have *batch-min-update* (*partition-state-view* (*empty-partition-state d*))
 $(\text{map } (\lambda x. (x, d \ x)) \ xs) =$
 $\text{batch-min-update } (\text{label-partition-view } d \ \{\}) \ (\text{map } (\lambda x. (x, d \ x)) \ xs)$
unfolding *empty-view* **by** *simp*
also have $\dots = \text{label-partition-view } d \ (\{\} \cup \text{set } xs)$
by (*rule batch-min-update-label-partition-view-disjoint[OF distinct]*) *simp*
finally show *?thesis*
by *simp*
qed

definition *exact-partition-initial-state* **where**
 $\text{exact-partition-initial-state } t \ d \ P \ P_D \ c \longleftrightarrow$
 $\text{partition-state-invar } P_D \wedge$
 $\text{partition-state-view } P_D = \text{label-partition-view } d \ P \wedge$
 $c = t * \text{card } P \wedge$
 $(\exists xs. \text{distinct } xs \wedge \text{set } xs = P \wedge$
 $\text{exact-partition-state-insert-list } t \ (\text{empty-partition-state } d)$
 $(\text{map } (\lambda x. (x, d \ x)) \ xs) \ c \ P_D)$

The predicate *exact-partition-initial-state* packages the initial build phase for a non-base BMSSP call. It records three facts at once: the concrete partition state is well formed, its abstract view is exactly the label view of the pivot set, and the build cost is exactly $t * \text{card } P$. The existence lemmas below use finite pivot sets to construct such a state by a batch of concrete inserts.

lemma *exact-partition-initial-state-invar*:
assumes *exact-partition-initial-state* $t \ d \ P \ P_D \ c$
shows *partition-state-invar* P_D
using *assms* **unfolding** *exact-partition-initial-state-def* **by** *blast*

lemma *exact-partition-initial-state-view*:
assumes *exact-partition-initial-state* $t \ d \ P \ P_D \ c$
shows $\text{partition-state-view } P_D = \text{label-partition-view } d \ P$
using *assms* **unfolding** *exact-partition-initial-state-def* **by** *blast*

lemma *exact-partition-initial-state-cost-bound*:
assumes *exact-partition-initial-state* $t \ d \ P \ P_D \ c$
shows *partition-initial-insert-cost-bound* $c \ t \ P$
using *assms* **unfolding** *exact-partition-initial-state-def*

partition-initial-insert-cost-bound-def **by** *simp*

lemma *exact-partition-initial-state-cost*:
assumes *exact-partition-initial-state* $t\ d\ P\ P_D\ c$
shows $c = t * \text{card } P$
using *assms* **unfolding** *exact-partition-initial-state-def* **by** *blast*

lemma *exact-partition-initial-state-exists*:
assumes *finite-P*: *finite* P
shows $\exists P_D\ c. \text{exact-partition-initial-state } t\ d\ P\ P_D\ c$
proof –
obtain xs **where** xs : *set* $xs = P$ *distinct* xs
using *finite-distinct-list*[*OF* *finite-P*] **by** *blast*
let $?pairs = \text{map } (\lambda x. (x, d\ x))\ xs$
let $?P_D = \text{partition-state-batch-prepend } ?pairs\ (\text{empty-partition-state } d)$
let $?c = t * \text{card } P$
have *run*:
 $\text{exact-partition-state-insert-list } t\ (\text{empty-partition-state } d)\ ?pairs$
 $(t * \text{length } ?pairs)\ ?P_D$
by (*rule* *exact-partition-state-insert-list-batch-prepend*)
have *len*: $\text{length } xs = \text{card } P$
using xs *finite-P* *distinct-card* **by** *fastforce*
have *run-c*:
 $\text{exact-partition-state-insert-list } t\ (\text{empty-partition-state } d)\ ?pairs$
 $?c\ ?P_D$
using *run* *len* **by** *simp*
have *inv*: *partition-state-invar* $?P_D$
by (*rule* *exact-partition-state-insert-list-invar*[*OF* *empty-partition-state-invar*
run])
have *view*: *partition-state-view* $?P_D = \text{label-partition-view } d\ P$
proof –
have *partition-state-view* $?P_D =$
 $\text{batch-min-update } (\text{partition-state-view } (\text{empty-partition-state } d))\ ?pairs$
by (*rule* *partition-state-batch-prepend-refines-batch-min-update*)
(*rule* *empty-partition-state-invar*)
also have $\dots = \text{label-partition-view } d\ (\text{set } xs)$
by (*rule* *batch-min-update-empty-label-view*[*OF* $xs(2)$])
also have $\dots = \text{label-partition-view } d\ P$
using $xs(1)$ **by** *simp*
finally show *?thesis* .
qed
have *exact-partition-initial-state* $t\ d\ P\ ?P_D\ ?c$
unfolding *exact-partition-initial-state-def*
using *inv* *view* *run-c* xs **by** *blast*
then show *?thesis*
by *blast*
qed

lemma *exact-partition-initial-state-exists-for-capped-pivots*:

assumes $S\text{-subset}: S \subseteq V$
shows $\exists P_D c. \text{exact-partition-initial-state } t$
 $(\text{find-pivots-label-capped } k \text{ cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \ P_D \ c$
proof –
have $\text{pivots-subset-}V$:
 $\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B \subseteq V$
using $\text{find-pivots-pivots-capped-subset } S\text{-subset}$ **by** blast
have $\text{finite-pivots}: \text{finite } (\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B)$
by $(\text{rule finite-subset}[OF \ \text{pivots-subset-}V \ \text{finite-}V])$
show $?thesis$
by $(\text{rule exact-partition-initial-state-exists}[OF \ \text{finite-pivots}])$
qed

lemma $\text{exact-partition-initial-state-exists-for-capped-pivots-with-cost}$:
assumes $S\text{-subset}: S \subseteq V$
shows $\exists P_D. \text{exact-partition-initial-state } t$
 $(\text{find-pivots-label-capped } k \text{ cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \ P_D$
 $(t * \text{card } (\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B))$
proof –
have $\text{pivots-subset-}V$:
 $\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B \subseteq V$
using $\text{find-pivots-pivots-capped-subset } S\text{-subset}$ **by** blast
have $\text{finite-pivots}: \text{finite } (\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B)$
by $(\text{rule finite-subset}[OF \ \text{pivots-subset-}V \ \text{finite-}V])$
obtain $P_D \ c$ **where** $\text{init}: \text{exact-partition-initial-state } t$
 $(\text{find-pivots-label-capped } k \text{ cap } d \ S \ B)$
 $(\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B) \ P_D \ c$
using $\text{exact-partition-initial-state-exists}[OF \ \text{finite-pivots}]$ **by** blast
have $c = t * \text{card } (\text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B)$
by $(\text{rule exact-partition-initial-state-cost}[OF \ \text{init}])$
then show $?thesis$
using init **by** blast
qed

lemma $\text{find-pivots-pivots-capped-reaches}$:
assumes $\text{reaches}: \forall x \in S. \text{reachable } s \ x$
shows $\forall x \in \text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B. \text{reachable } s \ x$
using $\text{reaches find-pivots-pivots-capped-subset}$ **by** blast

lemma $\text{find-pivots-pivots-capped-label-less-finite}$:
assumes $\text{below}: \bigwedge y. y \in S \implies d \ y < B\text{max}$
and $xP: x \in \text{find-pivots-pivots-capped } k \text{ cap } d \ S \ (Fin \ B\text{max})$
shows $\text{find-pivots-label-capped } k \text{ cap } d \ S \ (Fin \ B\text{max}) \ x < B\text{max}$
proof –
have $\text{below-bound}: \bigwedge y. y \in S \implies \text{below-bound } (d \ y) \ (Fin \ B\text{max})$
using below **by** simp
have $\text{below-bound } (\text{find-pivots-label-capped } k \text{ cap } d \ S \ (Fin \ B\text{max}) \ x)$


```

  (Fin Bmax)
  by (rule find-pivots-pivots-capped-label-below[OF below-bound xP])
  then show ?thesis
  by simp
qed

```

The main exact concrete run relation follows the same tree as the direct-insert costed relation, but it keeps concrete partition states in the loop. A step performs a concrete pull, recursively solves the pulled lower split, then performs three concrete batch prepends: direct edge relaxations, lower edge relaxations, and source labels. The accumulated cost is not merely bounded by predicates; it is the exact sum of the concrete operation costs, the child cost, and the tail-loop cost.

This exactness is useful for refinement. The direct-insert relation is the mature cost interface used by the graph-time theorems. By proving that every exact concrete run refines to that relation, all existing correctness and runtime results become available for the concrete partition-state execution.

inductive *exact-concrete-partition-loop-state*

and *exact-concrete-bmssp* **where**

Exact-Concrete-State-Done:

```

  keys-of (partition-state-view D) = {}  $\implies$ 
  bound-le (Fin a) B  $\implies$ 
  complete-on d' (bound-tree P B)  $\implies$ 
  exact-concrete-partition-loop-state  $\Delta$  M-of t h k cap l d P B d' D a
  [] [] B [range-tree P a B]
  (bound-tree P (Fin a)  $\cup \bigcup$  (set [range-tree P a B])) 0 []

```

| *Exact-Concrete-State-Stop:*

```

  bound-le (Fin a) B  $\implies$ 
  complete-on d' (bound-tree P (Fin a))  $\implies$ 
  k * cap  $\leq$  card (bound-tree P (Fin a))  $\implies$ 
  exact-concrete-partition-loop-state  $\Delta$  M-of t h k cap l d P B d' D a
  [] [] (Fin a) [range-tree P a (Fin a)]
  (bound-tree P (Fin a)  $\cup \bigcup$  (set [range-tree P a (Fin a)])) 0 []

```

| *Exact-Concrete-State-Step:*

```

  partition-state-invar D  $\implies$ 
  partition-upper-bound (partition-state-view D) Bmax  $\implies$ 
  costed-partition-state-pull (M-of l) Bmax D S-pull beta D-pull c-pull  $\implies$ 
  bound-le (Fin beta) B  $\implies$ 
  bmssp-pre-full d S-pull (Fin beta)  $\implies$ 
  S-pull = split-below d P beta  $\implies$ 
  ( $\forall x \in S\text{-pull. reachable } s \ x$ )  $\implies$ 
  a  $\leq$  b  $\implies$ 
  bound-le (Fin a) B'  $\implies$ 
  complete-on d' (bound-tree P (Fin a))  $\implies$ 
  card (bound-tree P (Fin a)) < k * cap  $\implies$ 
  exact-concrete-bmssp  $\Delta$  M-of t h k cap l d S-pull (Fin beta)
  d-child (Fin b) U-child c-child  $\implies$ 
  U-child = range-tree P a (Fin b)  $\implies$ 

```

$\text{complete-preserved } d\text{-child } d' \text{ } U\text{-child} \implies$
 $\text{direct-edge-batch} = \text{edge-relaxation-pairs-in-bound } d\text{-child } U\text{-child } \text{beta } B \implies$
 $\text{lower-edge-batch} = \text{edge-relaxation-pairs-between } d\text{-child } U\text{-child } b \text{ } \text{beta} \implies$
 $\text{source-batch} = \text{label-pairs-between } d \text{ } S\text{-pull } b \text{ } \text{beta} \implies$
 $\text{costed-partition-state-batch-prepend } t \text{ } D\text{-pull } \text{direct-edge-batch } c\text{-direct } D\text{-direct}$
 $\implies$
 $\text{costed-partition-state-batch-prepend } h \text{ } D\text{-direct } \text{lower-edge-batch } c\text{-lower } D\text{-lower}$
 $\implies$
 $\text{costed-partition-state-batch-prepend } h \text{ } D\text{-lower } \text{source-batch } c\text{-sources } D\text{-next}$
 $\implies$
 $\text{exact-concrete-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ } \text{cap } l \text{ } d \text{ } P \text{ } B \text{ } d'$
 $D\text{-next } b \text{ } \text{betas } bs \text{ } B' \text{ } Us\text{-tail } U\text{-tail } c\text{-tail } \text{child-costs-tail} \implies$
 $c = c\text{-pull} + c\text{-direct} + c\text{-lower} + c\text{-sources} + c\text{-child} + c\text{-tail} \implies$
 $\text{exact-concrete-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ } \text{cap } l \text{ } d \text{ } P \text{ } B \text{ } d' \text{ } D \text{ } a$
 $(\text{beta} \# \text{betas}) (b \# bs) B'$
 $(\text{range-tree } P \text{ } a \text{ } (Fin \text{ } b) \# Us\text{-tail})$
 $(\text{bound-tree } P \text{ } (Fin \text{ } a) \cup$
 $\quad \cup (\text{set } (\text{range-tree } P \text{ } a \text{ } (Fin \text{ } b) \# Us\text{-tail}))) c$
 $(c\text{-child} \# \text{child-costs-tail})$
| *Exact-Concrete-Base:*
 $S = \{x\} \implies$
 $\text{exact-concrete-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ } \text{cap } 0 \text{ } d \text{ } S \text{ } B$
 $(\lambda v. \text{ if } v \in \text{base-case-vertices } k \text{ } x \text{ } B \text{ then } \text{dist } s \text{ } v \text{ else } d \text{ } v)$
 $(\text{base-case-bound } k \text{ } x \text{ } B)$
 $(\text{base-case-vertices } k \text{ } x \text{ } B)$
 $(\text{base-case-scan-cost } \Delta \text{ } k \text{ } x \text{ } B)$
| *Exact-Concrete-Step:*
 $\text{exact-partition-initial-state } t$
 $(\text{find-pivots-label-capped } k \text{ } \text{cap } d \text{ } S \text{ } B)$
 $(\text{find-pivots-pivots-capped } k \text{ } \text{cap } d \text{ } S \text{ } B) \text{ } D \text{ } c\text{-insert} \implies$
 $\text{exact-concrete-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ } \text{cap } l$
 $(\text{find-pivots-label-capped } k \text{ } \text{cap } d \text{ } S \text{ } B)$
 $(\text{find-pivots-pivots-capped } k \text{ } \text{cap } d \text{ } S \text{ } B) \text{ } B \text{ } d' \text{ } D \text{ } a$
 $\text{betas } bs \text{ } B' \text{ } Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs-loop} \implies$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{ find-pivots-label-capped } k \text{ } \text{cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\} \implies$
 $U = U\text{-loop} \cup$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{ find-pivots-label-capped } k \text{ } \text{cap } d \text{ } S \text{ } B \text{ } v = \text{dist } s \text{ } v\} \implies$
 $c = \text{fp-iter-capped-scan-cost } k \text{ } \text{cap } d \text{ } S \text{ } S \text{ } B + c\text{-insert} + c\text{-loop} \implies$
 $\text{exact-concrete-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ } \text{cap } (\text{Suc } l) \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$

inductive-cases *exact-concrete-bmssp-zeroE:*

$\text{exact-concrete-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ } \text{cap } 0 \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$

inductive-cases *exact-concrete-bmssp-SucE:*

$\text{exact-concrete-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ } \text{cap } (\text{Suc } l) \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$

lemma *exact-partition-initial-state-cost-unique:*

assumes $\text{exact-partition-initial-state } t \text{ } d \text{ } P \text{ } P_D \text{ } c$

and *exact-partition-initial-state* $t \ d \ P \ P_D' \ c'$
shows $c = c'$
using *exact-partition-initial-state-cost*[*OF* *assms*(1)]
exact-partition-initial-state-cost[*OF* *assms*(2)]
by *simp*

lemma *exact-partition-initial-state-upper-bound*:
assumes *init*: *exact-partition-initial-state* $t \ d \ P \ P_D \ c$
and *upper*: $\bigwedge u. u \in P \implies d \ u < B$
shows *partition-upper-bound* (*partition-state-view* P_D) B
using *exact-partition-initial-state-view*[*OF* *init*] *upper*
unfolding *partition-upper-bound-def* **by** *simp*

lemma *exact-partition-initial-state-pull-split*:
assumes *init*: *exact-partition-initial-state* $t \ d \ P \ P_D \ c$
and *upper*: $\bigwedge u. u \in P \implies d \ u < B$
and *pull*: *partition-state-pull* $M \ B \ P_D = (S\text{-pull}, \text{beta}, P_D')$
shows $S\text{-pull} = \text{split-below } d \ P \ \text{beta}$
proof –
have *inv*: *partition-state-invar* P_D
by (*rule exact-partition-initial-state-invar*[*OF* *init*])
have *view*: *partition-state-view* $P_D = \text{label-partition-view } d \ P$
by (*rule exact-partition-initial-state-view*[*OF* *init*])
show ?thesis
by (*rule partition-state-pull-label-set-eq-split-below*
[*OF* *inv* *view* *upper* *pull*])
qed

lemma *exact-partition-initial-state-pull-exact-data*:
assumes *init*: *exact-partition-initial-state* $t \ d \ P \ P_D \ c$
and *pre*: *bmssp-pre-full* $d \ P \ (\text{Fin } B)$
and *upper*: $\bigwedge u. u \in P \implies d \ u < B$
and *pull*: *partition-state-pull* $M \ B \ P_D = (S\text{-pull}, \text{beta}, P_D')$
shows *costed-partition-state-pull* $M \ B \ P_D \ S\text{-pull} \ \text{beta} \ P_D' \ (\text{card } S\text{-pull}) \wedge$
 $S\text{-pull} = \text{split-below } d \ P \ \text{beta} \wedge$
bmssp-pre-full $d \ S\text{-pull} \ (\text{Fin } \text{beta}) \wedge$
partition-state-invar P_D'
proof –
have *inv*: *partition-state-invar* P_D
by (*rule exact-partition-initial-state-invar*[*OF* *init*])
have *view*: *partition-state-view* $P_D = \text{label-partition-view } d \ P$
by (*rule exact-partition-initial-state-view*[*OF* *init*])
have *costed*:
costed-partition-state-pull $M \ B \ P_D \ S\text{-pull} \ \text{beta} \ P_D' \ (\text{card } S\text{-pull})$
using *pull* **unfolding** *costed-partition-state-pull-def* **by** *simp*
have *split*: $S\text{-pull} = \text{split-below } d \ P \ \text{beta}$
by (*rule partition-state-pull-label-set-eq-split-below*
[*OF* *inv* *view* *upper* *pull*])
have *lower-pre*: *bmssp-pre-full* $d \ S\text{-pull} \ (\text{Fin } \text{beta})$

```

    by (rule partition-state-pull-establishes-lower-pre
        [OF inv view pre upper pull])
    have inv': partition-state-invar  $P_D'$ 
    by (rule partition-state-pull-invar[OF inv pull])
    show ?thesis
    using costed split lower-pre inv' by blast
qed

lemma exact-partition-initial-state-pull-reaches:
  assumes init: exact-partition-initial-state  $t d P P_D c$ 
    and upper:  $\bigwedge u. u \in P \implies d u < B$ 
    and pull: partition-state-pull  $M B P_D = (S\text{-pull}, \text{beta}, P_D')$ 
    and P-reaches:  $\forall x \in P. \text{reachable } s x$ 
  shows  $\forall x \in S\text{-pull}. \text{reachable } s x$ 
proof -
  have S-pull = split-below  $d P \text{beta}$ 
  by (rule exact-partition-initial-state-pull-split[OF init upper pull])
  then show ?thesis
  using P-reaches unfolding split-below-def by blast
qed

lemma exact-partition-initial-state-pull-beta-bound:
  assumes init: exact-partition-initial-state  $t d P P_D c$ 
    and upper:  $\bigwedge u. u \in P \implies d u < B$ 
    and pull: partition-state-pull  $M B P_D = (S\text{-pull}, \text{beta}, P_D')$ 
  shows bound-le ( $\text{Fin beta}$ ) ( $\text{Fin B}$ )
proof -
  have inv: partition-state-invar  $P_D$ 
  by (rule exact-partition-initial-state-invar[OF init])
  have view: partition-state-view  $P_D = \text{label-partition-view } d P$ 
  by (rule exact-partition-initial-state-view[OF init])
  have upper-D: partition-upper-bound (partition-state-view  $P_D$ )  $B$ 
  by (rule exact-partition-initial-state-upper-bound[OF init upper])
  have pull-sep:
    pull-separates (partition-state-view  $P_D$ )  $M B S\text{-pull beta}$ 
    (partition-state-view  $P_D'$ )
  proof (rule partition-state-pull-refines-pull-separates[OF inv - pull])
    fix u
    assume  $u \in \text{keys-of } (\text{partition-state-view } P_D)$ 
    then show value-of (partition-state-view  $P_D$ )  $u < B$ 
    using upper unfolding view by simp
  qed
  have beta  $\leq B$ 
  by (rule pull-separates-bound-le[OF pull-sep upper-D])
  then show ?thesis
  by simp
qed

```

The pull lemmas are the local bridge from a concrete partition state to the BMSSP child problem. Starting from an exact initial state, a concrete pull

refines to *pull-separates* on the abstract label partition view. Therefore the pulled source set is exactly *split-below* $d P$ *beta*, it inherits the child BMSSP precondition, and its vertices remain reachable. These are the premises required by the recursive exact concrete step.

lemma *exact-partition-initial-state-exists-with-cost*:

assumes *finite-P*: *finite P*

shows $\exists P_D. \text{exact-partition-initial-state } t \ d \ P \ P_D \ (t * \text{card } P)$

proof –

obtain $P_D \ c$ **where** *init*: *exact-partition-initial-state* $t \ d \ P \ P_D \ c$

using *exact-partition-initial-state-exists*[*OF finite-P*] **by** *blast*

have $c = t * \text{card } P$

by (*rule exact-partition-initial-state-cost*[*OF init*])

then show *?thesis*

using *init* **by** *blast*

qed

lemma *exact-concrete-bmssp-base-exists*:

assumes $S = \{x\}$

shows *exact-concrete-bmssp* $\Delta \ M\text{-of } t \ h \ k \ \text{cap } 0 \ d \ S \ B$

($\lambda v. \text{if } v \in \text{base-case-vertices } k \ x \ B \text{ then } \text{dist } s \ v \text{ else } d \ v$)

(*base-case-bound* $k \ x \ B$)

(*base-case-vertices* $k \ x \ B$)

(*base-case-scan-cost* $\Delta \ k \ x \ B$)

by (*rule Exact-Concrete-Base*[*OF assms*])

lemma *exact-concrete-bmssp-zero-singletonD*:

assumes *run*: *exact-concrete-bmssp* $\Delta \ M\text{-of } t \ h \ k \ \text{cap } 0 \ d \ \{x\} \ B \ d' \ B' \ U \ c$

shows $d' =$

($\lambda v. \text{if } v \in \text{base-case-vertices } k \ x \ B \text{ then } \text{dist } s \ v \text{ else } d \ v$) $\wedge$

$B' = \text{base-case-bound } k \ x \ B \wedge$

$U = \text{base-case-vertices } k \ x \ B \wedge$

$c = \text{base-case-scan-cost } \Delta \ k \ x \ B$

using *run* **by** *cases auto*

lemma *exact-concrete-bmssp-zero-singleton-cost-unique*:

assumes *run*: *exact-concrete-bmssp* $\Delta \ M\text{-of } t \ h \ k \ \text{cap } 0 \ d \ \{x\} \ B \ d' \ B' \ U \ c$

and *run'*: *exact-concrete-bmssp* $\Delta \ M\text{-of } t \ h \ k \ \text{cap } 0 \ d \ \{x\} \ B \ d'' \ B'' \ U' \ c'$

shows $c = c'$

using *exact-concrete-bmssp-zero-singletonD*[*OF run*]

exact-concrete-bmssp-zero-singletonD[*OF run'*]

by *simp*

lemma *exact-concrete-bmssp-zeroD*:

assumes *run*: *exact-concrete-bmssp* $\Delta \ M\text{-of } t \ h \ k \ \text{cap } 0 \ d \ S \ B \ d' \ B' \ U \ c$

shows $\exists x. S = \{x\} \wedge$

$d' = (\lambda v. \text{if } v \in \text{base-case-vertices } k \ x \ B \text{ then } \text{dist } s \ v \text{ else } d \ v) \wedge$

$B' = \text{base-case-bound } k \ x \ B \wedge$

$U = \text{base-case-vertices } k \ x \ B \wedge$

$c = \text{base-case-scan-cost } \Delta \ k \ x \ B$

```

using run
proof cases
  case (Exact-Concrete-Base x)
  then show ?thesis
    by blast
qed

lemma exact-concrete-bmssp-zero-deterministic:
  assumes run: exact-concrete-bmssp  $\Delta$  M-of t h k cap 0 d S B d' B' U c
  and run': exact-concrete-bmssp  $\Delta$  M-of t h k cap 0 d S B d'' B'' U' c'
  shows  $d' = d'' \wedge B' = B'' \wedge U = U' \wedge c = c'$ 
proof –
  obtain x where x:
     $S = \{x\}$ 
     $d' = (\lambda v. \text{if } v \in \text{base-case-vertices } k \ x \ B \ \text{then } \text{dist } s \ v \ \text{else } d \ v)$ 
     $B' = \text{base-case-bound } k \ x \ B$ 
     $U = \text{base-case-vertices } k \ x \ B$ 
     $c = \text{base-case-scan-cost } \Delta \ k \ x \ B$ 
    using exact-concrete-bmssp-zeroD[OF run] by blast
  obtain y where y:
     $S = \{y\}$ 
     $d'' = (\lambda v. \text{if } v \in \text{base-case-vertices } k \ y \ B \ \text{then } \text{dist } s \ v \ \text{else } d \ v)$ 
     $B'' = \text{base-case-bound } k \ y \ B$ 
     $U' = \text{base-case-vertices } k \ y \ B$ 
     $c' = \text{base-case-scan-cost } \Delta \ k \ y \ B$ 
    using exact-concrete-bmssp-zeroD[OF run'] by blast
  have  $x = y$ 
  using  $x(1) \ y(1)$  by simp
  then show ?thesis
    using  $x \ y$  by simp
qed

lemma exact-concrete-bmssp-zero-exists-iff-singleton:
   $(\exists d' B' U c. \text{exact-concrete-bmssp } \Delta \text{ M-of t h k cap 0 d S B d' B' U c})$ 
   $\longleftrightarrow (\exists x. S = \{x\})$ 
proof
  assume  $\exists d' B' U c.$ 
    exact-concrete-bmssp  $\Delta$  M-of t h k cap 0 d S B d' B' U c
  then obtain  $d' B' U c$  where
    run: exact-concrete-bmssp  $\Delta$  M-of t h k cap 0 d S B d' B' U c
  by blast
  obtain x where  $S = \{x\}$ 
  using exact-concrete-bmssp-zeroD[OF run] by blast
  then show  $\exists x. S = \{x\}$ 
    by blast
next
  assume  $\exists x. S = \{x\}$ 
  then obtain x where  $S = \{x\}$ 
    by blast

```

then show $\exists d' B' U c.$
exact-concrete-bmssp Δ *M-of t h k cap 0 d S B d' B' U c*
by (*intro exI*) (*rule exact-concrete-bmssp-base-exists*)
qed

lemma *complete-on-dist*:
complete-on (*dist s*) *U*
unfolding *complete-on-def* **by** *simp*

lemma *exact-concrete-partition-loop-state-done-witness*:
assumes *empty*: *keys-of* (*partition-state-view D*) = {}
and *a-bound*: *bound-le* (*Fin a*) *B*
and *complete*: *complete-on* *d'* (*bound-tree P B*)
shows *exact-concrete-partition-loop-state* Δ *M-of t h k cap l d P B d'*
D a [] [] B [range-tree P a B]
(bound-tree P (Fin a) $\cup \bigcup$ (set [range-tree P a B])) 0 []
by (*rule Exact-Concrete-State-Done[OF empty a-bound complete]*)

lemma *exact-concrete-partition-loop-state-stop-witness*:
assumes *a-bound*: *bound-le* (*Fin a*) *B*
and *complete*: *complete-on* *d'* (*bound-tree P (Fin a)*)
and *threshold*: $k * \text{cap} \leq \text{card } (\text{bound-tree } P \text{ (Fin a)})$
shows *exact-concrete-partition-loop-state* Δ *M-of t h k cap l d P B d'*
D a [] [] (Fin a) [range-tree P a (Fin a)]
(bound-tree P (Fin a) $\cup \bigcup$ (set [range-tree P a (Fin a)])) 0 []
by (*rule Exact-Concrete-State-Stop[OF a-bound complete threshold]*)

lemma *exact-concrete-partition-loop-state-done-exists*:
assumes *empty*: *keys-of* (*partition-state-view D*) = {}
and *a-bound*: *bound-le* (*Fin a*) *B*
and *complete*: *complete-on* *d'* (*bound-tree P B*)
shows $\exists \text{betas } bs B' Us U c \text{ child-costs.}$
exact-concrete-partition-loop-state Δ *M-of t h k cap l d P B d'*
D a betas bs B' Us U c child-costs

proof –
have *run*:
exact-concrete-partition-loop-state Δ *M-of t h k cap l d P B d'*
D a [] [] B [range-tree P a B]
(bound-tree P (Fin a) $\cup \bigcup$ (set [range-tree P a B])) 0 []
by (*rule exact-concrete-partition-loop-state-done-witness*
[OF empty a-bound complete])
then show *?thesis*
by *blast*
qed

lemma *exact-concrete-partition-loop-state-stop-exists*:
assumes *a-bound*: *bound-le* (*Fin a*) *B*
and *complete*: *complete-on* *d'* (*bound-tree P (Fin a)*)
and *threshold*: $k * \text{cap} \leq \text{card } (\text{bound-tree } P \text{ (Fin a)})$

shows $\exists \text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs}.$
exact-concrete-partition-loop-state $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d'$
D a betas bs B' Us U c child-costs
proof –
have *run*:
exact-concrete-partition-loop-state $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d'$
D a [] [] (Fin a) [range-tree P a (Fin a)]
(bound-tree P (Fin a) $\cup \bigcup (\text{set } [range-tree P a (Fin a)])$) 0 []
by (*rule exact-concrete-partition-loop-state-stop-witness*
[OF a-bound complete threshold])
then show *?thesis*
by *blast*
qed

lemma *exact-concrete-partition-loop-state-step-from-concrete-ops*:
assumes *inv-D: partition-state-invar D*
and *upper-D: partition-upper-bound (partition-state-view D) Bmax*
and *pull*:
partition-state-pull (M-of l) Bmax D = (S-pull, beta, D-pull)
and *beta-bound: bound-le (Fin beta) B*
and *child-pre: bmssp-pre-full d S-pull (Fin beta)*
and *S-pull-def: S-pull = split-below d P beta*
and *S-pull-reaches: $\forall x \in S\text{-pull}. \text{reachable } s \ x$*
and *a-le-b: $a \leq b$*
and *a-bound: bound-le (Fin a) B'*
and *complete-prefix: complete-on d' (bound-tree P (Fin a))*
and *below-threshold: card (bound-tree P (Fin a)) < k * cap*
and *child*:
exact-concrete-bmssp $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-pull (Fin beta)}$
d-child (Fin b) U-child c-child
and *U-child: U-child = range-tree P a (Fin b)*
and *preserved: complete-preserved d-child d' U-child*
and *tail*:
exact-concrete-partition-loop-state $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d'$
(partition-state-batch-prepend (label-pairs-between d S-pull b beta)
(partition-state-batch-prepend
(edge-relaxation-pairs-between d-child U-child b beta)
(partition-state-batch-prepend
(edge-relaxation-pairs-in-bound d-child U-child beta B)
D-pull)))
b betas bs B' Us-tail U-tail c-tail child-costs-tail
shows *exact-concrete-partition-loop-state $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$*
(beta # betas) (b # bs) B'
(range-tree P a (Fin b) # Us-tail)
(bound-tree P (Fin a) $\cup$
 $\bigcup (\text{set } (range-tree P a (Fin b) \# Us\text{-tail}))$)
(card S-pull +
 *$t * \text{length } (\text{edge-relaxation-pairs-in-bound } d\text{-child } U\text{-child } beta \ B)$ +*
 *$h * \text{length } (\text{edge-relaxation-pairs-between } d\text{-child } U\text{-child } b \ beta)$ +*


```

      h * length (label-pairs-between d S-pull b beta) +
      c-child + c-tail)
    (c-child # child-costs-tail)
proof –
  let ?direct = edge-relaxation-pairs-in-bound d-child U-child beta B
  let ?lower = edge-relaxation-pairs-between d-child U-child b beta
  let ?sources = label-pairs-between d S-pull b beta
  let ?D-direct = partition-state-batch-prepend ?direct D-pull
  let ?D-lower = partition-state-batch-prepend ?lower ?D-direct
  let ?D-next = partition-state-batch-prepend ?sources ?D-lower
  have pull-op:
    costed-partition-state-pull (M-of l) Bmax D S-pull beta D-pull
    (card S-pull)
  using pull unfolding costed-partition-state-pull-def by simp
  have direct-op:
    costed-partition-state-batch-prepend t D-pull ?direct
    (t * length ?direct) ?D-direct
  unfolding costed-partition-state-batch-prepend-def by simp
  have lower-op:
    costed-partition-state-batch-prepend h ?D-direct ?lower
    (h * length ?lower) ?D-lower
  unfolding costed-partition-state-batch-prepend-def by simp
  have source-op:
    costed-partition-state-batch-prepend h ?D-lower ?sources
    (h * length ?sources) ?D-next
  unfolding costed-partition-state-batch-prepend-def by simp
  show ?thesis
  by (rule Exact-Concrete-State-Step
    [OF inv-D upper-D pull-op beta-bound child-pre S-pull-def
      S-pull-reaches a-le-b a-bound complete-prefix below-threshold child
      U-child preserved refl refl refl direct-op lower-op source-op tail])
    simp
qed

```

The witness lemmas turn concrete operations into one exact loop step. The most explicit version assumes the concrete pull and the concrete tail state are already known, then constructs the exact step and computes its cost. The initialized version derives the pull split and child precondition from *exact-partition-initial-state*. Later existence lemmas use these builders to cover the terminal, threshold, and recursive cases of a non-base call.

lemma *exact-concrete-initialized-partition-loop-state-step*:
assumes *init*: *exact-partition-initial-state* t d P D c-insert
and *pre*: *bmssp-pre-full* d P (*Fin* Bmax)
and *upper*: $\bigwedge u. u \in P \implies d\ u < Bmax$
and *pull*: *partition-state-pull* (M-of l) Bmax D = (S-pull, beta, D-pull)
and *beta-bound*: *bound-le* (*Fin* beta) B
and *S-pull-reaches*: $\forall x \in S\text{-pull}. \text{reachable } s\ x$
and *a-le-b*: $a \leq b$
and *a-bound*: *bound-le* (*Fin* a) B'

and *complete-prefix*: *complete-on* d' (*bound-tree* P (*Fin* a))
and *below-threshold*: *card* (*bound-tree* P (*Fin* a)) $< k * cap$
and *child*:
 exact-concrete-bmssp Δ *M-of* $t h k cap l d S$ -pull (*Fin* β)
 d-child (*Fin* b) *U-child* c -child
and *U-child*: *U-child* = *range-tree* $P a$ (*Fin* b)
and *preserved*: *complete-preserved* *d-child* d' *U-child*
and *tail*:
 exact-concrete-partition-loop-state Δ *M-of* $t h k cap l d P B d'$
 (*partition-state-batch-prepend* (*label-pairs-between* $d S$ -pull $b \beta$)
 (*partition-state-batch-prepend*
 (*edge-relaxation-pairs-between* *d-child* *U-child* $b \beta$)
 (*partition-state-batch-prepend*
 (*edge-relaxation-pairs-in-bound* *d-child* *U-child* βB)
 D-pull)))
 $b \beta$ β $bs B' Us$ -tail *U-tail* c -tail *child-costs-tail*
shows *exact-concrete-partition-loop-state* Δ *M-of* $t h k cap l d P B d' D a$
 ($\beta \# \beta$) ($b \# bs$) B'
 (*range-tree* $P a$ (*Fin* b) $\# Us$ -tail)
 (*bound-tree* P (*Fin* a) $\cup$
 $\bigcup$ (*set* (*range-tree* $P a$ (*Fin* b) $\# Us$ -tail)))
 (*card* S -pull +
 $t * length$ (*edge-relaxation-pairs-in-bound* *d-child* *U-child* βB) +
 $h * length$ (*edge-relaxation-pairs-between* *d-child* *U-child* $b \beta$) +
 $h * length$ (*label-pairs-between* $d S$ -pull $b \beta$) +
 c -child + c -tail)
 (c -child $\# child$ -costs-tail)
proof –
have *inv-D*: *partition-state-invar* D
 by (*rule exact-partition-initial-state-invar*[*OF* *init*])
have *upper-D*: *partition-upper-bound* (*partition-state-view* D) B_{max}
 by (*rule exact-partition-initial-state-upper-bound*[*OF* *init upper*])
have *pull-data*:
 costed-partition-state-pull (*M-of* l) $B_{max} D S$ -pull βD -pull
 (*card* S -pull) $\wedge$
 S -pull = *split-below* $d P \beta$ $\wedge$
 bmssp-pre-full $d S$ -pull (*Fin* β) $\wedge$
 partition-state-invar D -pull
 by (*rule exact-partition-initial-state-pull-exact-data*
 [*OF* *init pre upper pull*])
have *child-pre*: *bmssp-pre-full* $d S$ -pull (*Fin* β)
 using *pull-data* **by** *blast*
have *S-pull-def*: S -pull = *split-below* $d P \beta$
 using *pull-data* **by** *blast*
show *?thesis*
 by (*rule exact-concrete-partition-loop-state-step-from-concrete-ops*
 [*OF* *inv-D upper-D pull* β -bound *child-pre* S -pull-def
 S -pull-reaches a -le- b a-bound *complete-prefix* *below-threshold* *child*
 U-child *preserved* *tail*])

qed

lemma *partition-state-pull-three-batch-prepend-invar:*
assumes *inv: partition-state-invar D*
and *pull: partition-state-pull M B D = (S-pull, beta, D-pull)*
shows *partition-state-invar*
 (*partition-state-batch-prepend zs*
 (*partition-state-batch-prepend ys*
 (*partition-state-batch-prepend xs D-pull*)))
proof –
have *inv-pull: partition-state-invar D-pull*
by (*rule partition-state-pull-invar[OF inv pull]*)
have *inv-xs: partition-state-invar (partition-state-batch-prepend xs D-pull)*
by (*rule partition-state-batch-prepend-invar[OF inv-pull]*)
have *inv-ys:*
 partition-state-invar
 (*partition-state-batch-prepend ys*
 (*partition-state-batch-prepend xs D-pull*))
by (*rule partition-state-batch-prepend-invar[OF inv-xs]*)
show *?thesis*
by (*rule partition-state-batch-prepend-invar[OF inv-ys]*)
qed

lemma *exact-concrete-initialized-partition-loop-state-step-exists:*
assumes *init: exact-partition-initial-state t d P D c-insert*
and *pre: bmssp-pre-full d P (Fin Bmax)*
and *upper: $\bigwedge u. u \in P \implies d\ u < Bmax$*
and *pull: partition-state-pull (M-of l) Bmax D = (S-pull, beta, D-pull)*
and *beta-bound: bound-le (Fin beta) B*
and *S-pull-reaches: $\forall x \in S\text{-pull}. \text{reachable } s\ x$*
and *a-le-b: $a \leq b$*
and *a-bound: bound-le (Fin a) B'*
and *complete-prefix: complete-on d' (bound-tree P (Fin a))*
and *below-threshold: $\text{card } (\text{bound-tree } P\ (\text{Fin } a)) < k * \text{cap}$*
and *child:*
 exact-concrete-bmssp Δ M-of t h k cap l d S-pull (Fin beta)
 d-child (Fin b) U-child c-child
and *U-child: U-child = range-tree P a (Fin b)*
and *preserved: complete-preserved d-child d' U-child*
and *tail:*
 exact-concrete-partition-loop-state Δ M-of t h k cap l d P B d'
 (*partition-state-batch-prepend (label-pairs-between d S-pull b beta)*
 (*partition-state-batch-prepend*
 (*edge-relaxation-pairs-between d-child U-child b beta*)
 (*partition-state-batch-prepend*
 (*edge-relaxation-pairs-in-bound d-child U-child beta B*)
 D-pull)))
 b betas bs B' Us-tail U-tail c-tail child-costs-tail
shows $\exists c\ \text{child-costs}.$

exact-concrete-partition-loop-state Δ *M-of t h k cap l d P B d' D a*
 (beta # betas) (b # bs) B'
 (range-tree P a (Fin b) # Us-tail)
 (bound-tree P (Fin a) $\cup$
 $\cup$ (set (range-tree P a (Fin b) # Us-tail))) c child-costs
proof –
have run:
exact-concrete-partition-loop-state Δ *M-of t h k cap l d P B d' D a*
 (beta # betas) (b # bs) B'
 (range-tree P a (Fin b) # Us-tail)
 (bound-tree P (Fin a) $\cup$
 $\cup$ (set (range-tree P a (Fin b) # Us-tail)))
 (card S-pull +
 t * length (edge-relaxation-pairs-in-bound d-child U-child beta B) +
 h * length (edge-relaxation-pairs-between d-child U-child b beta) +
 h * length (label-pairs-between d S-pull b beta) +
 c-child + c-tail)
 (c-child # child-costs-tail)
by (rule exact-concrete-initialized-partition-loop-state-step
 [OF init pre upper pull beta-bound S-pull-reaches a-le-b a-bound
 complete-prefix below-threshold child U-child preserved tail])
show ?thesis
using run by blast
qed

lemma *exact-concrete-initialized-partition-loop-state-step-exists-finite-bound:*

assumes init: *exact-partition-initial-state t d P D c-insert*
and pre: *bmssp-pre-full d P (Fin Bmax)*
and upper: $\bigwedge u. u \in P \implies d u < Bmax$
and pull: *partition-state-pull (M-of l) Bmax D = (S-pull, beta, D-pull)*
and P-reaches: $\forall x \in P. \text{reachable } s x$
and a-le-b: $a \leq b$
and a-bound: *bound-le (Fin a) B'*
and complete-prefix: *complete-on d' (bound-tree P (Fin a))*
and below-threshold: *card (bound-tree P (Fin a)) < k * cap*
and child:
exact-concrete-bmssp Δ *M-of t h k cap l d S-pull (Fin beta)*
d-child (Fin b) (range-tree P a (Fin b)) c-child
and preserved:
complete-preserved d-child d' (range-tree P a (Fin b))
and tail:
exact-concrete-partition-loop-state Δ *M-of t h k cap l d P*
 (Fin Bmax) d'
 (partition-state-batch-prepend (label-pairs-between d S-pull b beta)
 (partition-state-batch-prepend
 (edge-relaxation-pairs-between d-child
 (range-tree P a (Fin b)) b beta)
 (partition-state-batch-prepend
 (edge-relaxation-pairs-in-bound d-child

(range-tree P a (Fin b)) beta (Fin $B_{\max}$))
 D-pull)))
 b betas bs B' Us -tail U -tail c -tail child-costs-tail
shows $\exists c$ child-costs.
 exact-concrete-partition-loop-state Δ M -of t h k cap l d P
 (Fin $B_{\max}$) d' D a (beta # betas) (b # bs) B'
 (range-tree P a (Fin b) # Us -tail)
 (bound-tree P (Fin a) $\cup$
 $\bigcup$ (set (range-tree P a (Fin b) # Us -tail))) c child-costs
proof –
 have beta-bound: bound-le (Fin beta) (Fin $B_{\max}$)
 by (rule exact-partition-initial-state-pull-beta-bound[OF init upper pull])
 have S -pull-reaches: $\forall x \in S$ -pull. reachable s x
 by (rule exact-partition-initial-state-pull-reaches
 [OF init upper pull P -reaches])
 show ?thesis
 by (rule exact-concrete-initialized-partition-loop-state-step-exists
 [OF init pre upper pull beta-bound S -pull-reaches a -le- b a -bound
 complete-prefix below-threshold child refl preserved tail])
qed

lemma exact-concrete-initialized-partition-loop-state-nonterminal-exists:
assumes init: exact-partition-initial-state t d P D c -insert
and pre: bmssp-pre-full d P (Fin $B_{\max}$)
and upper: $\bigwedge u. u \in P \implies d$ u < $B_{\max}$
and pull: partition-state-pull (M -of l) $B_{\max}$ D = (S -pull, beta, D -pull)
and beta-bound: bound-le (Fin beta) B
and P -reaches: $\forall x \in P$. reachable s x
and complete-prefix: complete-on d' (bound-tree P (Fin a))
and below-threshold: card (bound-tree P (Fin a)) < $k * cap$
and step:
 $\exists d$ -child b c -child betas bs B' Us -tail U -tail c -tail
 child-costs-tail.
 $a \leq b \wedge$
 bound-le (Fin a) $B' \wedge$
 exact-concrete-bmssp Δ M -of t h k cap l d S -pull (Fin beta)
 d -child (Fin b) (range-tree P a (Fin b)) c -child $\wedge$
 complete-preserved d -child d' (range-tree P a (Fin b)) $\wedge$
 exact-concrete-partition-loop-state Δ M -of t h k cap l d P B d'
 (partition-state-batch-prepend (label-pairs-between d S -pull b beta)
 (partition-state-batch-prepend
 (edge-relaxation-pairs-between d -child
 (range-tree P a (Fin b)) b beta)
 (partition-state-batch-prepend
 (edge-relaxation-pairs-in-bound d -child
 (range-tree P a (Fin b)) beta B)
 D -pull)))
 b betas bs B' Us -tail U -tail c -tail child-costs-tail
shows $\exists$ betas bs B' Us U c child-costs.

exact-concrete-partition-loop-state Δ *M-of t h k cap l d P B d'*
D a betas bs B' Us U c child-costs
proof –
have *S-pull-reaches*: $\forall x \in S\text{-pull}. \text{reachable } s \ x$
by (*rule exact-partition-initial-state-pull-reaches*
[OF init upper pull P-reaches])
obtain *d-child b c-child betas bs B' Us-tail U-tail c-tail*
child-costs-tail **where**
a-le-b: $a \leq b$
and *a-bound*: *bound-le (Fin a) B'*
and *child*:
exact-concrete-bmssp Δ *M-of t h k cap l d S-pull (Fin beta)*
d-child (Fin b) (range-tree P a (Fin b)) c-child
and *preserved*:
complete-preserved d-child d' (range-tree P a (Fin b))
and *tail*:
exact-concrete-partition-loop-state Δ *M-of t h k cap l d P B d'*
(partition-state-batch-prepend (label-pairs-between d S-pull b beta)
(partition-state-batch-prepend
(edge-relaxation-pairs-between d-child
(range-tree P a (Fin b)) b beta)
(partition-state-batch-prepend
(edge-relaxation-pairs-in-bound d-child
(range-tree P a (Fin b)) beta B)
D-pull)))
b betas bs B' Us-tail U-tail c-tail child-costs-tail
using step by blast
have *run*:
 $\exists c \text{ child-costs}.$
exact-concrete-partition-loop-state Δ *M-of t h k cap l d P B d'*
D a (beta \# betas) (b \# bs) B'
(range-tree P a (Fin b) \# Us-tail)
(bound-tree P (Fin a) \cup
 $\bigcup (\text{set } (\text{range-tree } P \ a \ (Fin \ b) \ \# \ Us\text{-tail}))) \ c \text{ child-costs}$
by (*rule exact-concrete-initialized-partition-loop-state-step-exists*
[OF init pre upper pull beta-bound S-pull-reaches a-le-b a-bound
complete-prefix below-threshold child refl preserved tail])
then show *?thesis*
by *blast*
qed

lemma *exact-concrete-initialized-partition-loop-state-nonterminal-exists-finite-bound*:

assumes *init*: *exact-partition-initial-state t d P D c-insert*
and *pre*: *bmssp-pre-full d P (Fin Bmax)*
and *upper*: $\bigwedge u. u \in P \implies d \ u < Bmax$
and *pull*: *partition-state-pull (M-of l) Bmax D = (S-pull, beta, D-pull)*
and *P-reaches*: $\forall x \in P. \text{reachable } s \ x$
and *complete-prefix*: *complete-on d' (bound-tree P (Fin a))*

and *below-threshold*: $\text{card } (\text{bound-tree } P \text{ (Fin } a)) < k * \text{cap}$
and *step*:
 $\exists d\text{-child } b \text{ } c\text{-child } \text{betas } bs \text{ } B' \text{ } Us\text{-tail } U\text{-tail } c\text{-tail}$
 child-costs-tail.
 $a \leq b \wedge$
 $\text{bound-le } (\text{Fin } a) \text{ } B' \wedge$
 $\text{exact-concrete-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ } \text{cap } l \text{ } d \text{ } S\text{-pull } (\text{Fin } \text{beta})$
 $d\text{-child } (\text{Fin } b) \text{ } (\text{range-tree } P \text{ } a \text{ } (\text{Fin } b)) \text{ } c\text{-child } \wedge$
 $\text{complete-preserved } d\text{-child } d' \text{ } (\text{range-tree } P \text{ } a \text{ } (\text{Fin } b)) \wedge$
 $\text{exact-concrete-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ } \text{cap } l \text{ } d \text{ } P$
 $(\text{Fin } B\text{max}) \text{ } d'$
 $(\text{partition-state-batch-prepend } (\text{label-pairs-between } d \text{ } S\text{-pull } b \text{ } \text{beta})$
 $(\text{partition-state-batch-prepend}$
 $(\text{edge-relaxation-pairs-between } d\text{-child}$
 $(\text{range-tree } P \text{ } a \text{ } (\text{Fin } b)) \text{ } b \text{ } \text{beta})$
 $(\text{partition-state-batch-prepend}$
 $(\text{edge-relaxation-pairs-in-bound } d\text{-child}$
 $(\text{range-tree } P \text{ } a \text{ } (\text{Fin } b)) \text{ } \text{beta } (\text{Fin } B\text{max}))$
 $D\text{-pull})))$
 $b \text{ } \text{betas } bs \text{ } B' \text{ } Us\text{-tail } U\text{-tail } c\text{-tail } \text{child-costs-tail}$
shows $\exists \text{betas } bs \text{ } B' \text{ } Us \text{ } U \text{ } c \text{ } \text{child-costs.}$
 $\text{exact-concrete-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ } \text{cap } l \text{ } d \text{ } P$
 $(\text{Fin } B\text{max}) \text{ } d' \text{ } D \text{ } a \text{ } \text{betas } bs \text{ } B' \text{ } Us \text{ } U \text{ } c \text{ } \text{child-costs}$
proof $(\text{rule exact-concrete-initialized-partition-loop-state-nonterminal-exists}$
 $[\text{OF init pre upper pull - } P\text{-reaches complete-prefix below-threshold step}])$
show $\text{bound-le } (\text{Fin } \text{beta}) \text{ } (\text{Fin } B\text{max})$
by $(\text{rule exact-partition-initial-state-pull-beta-bound}[\text{OF init upper pull}])$
qed

lemma *exact-concrete-initialized-partition-loop-state-cases-exists-finite-bound*:

assumes *init*: $\text{exact-partition-initial-state } t \text{ } d \text{ } P \text{ } D \text{ } c\text{-insert}$

and *pre*: $\text{bmssp-pre-full } d \text{ } P \text{ } (\text{Fin } B\text{max})$

and *upper*: $\bigwedge u. u \in P \implies d \text{ } u < B\text{max}$

and *P-reaches*: $\forall x \in P. \text{reachable } s \text{ } x$

and *cases*:

$(\text{keys-of } (\text{partition-state-view } D) = \{\}) \wedge$
 $\text{bound-le } (\text{Fin } a) \text{ } (\text{Fin } B\text{max}) \wedge$
 $\text{complete-on } d' \text{ } (\text{bound-tree } P \text{ } (\text{Fin } B\text{max}))) \vee$
 $(\text{bound-le } (\text{Fin } a) \text{ } (\text{Fin } B\text{max}) \wedge$
 $\text{complete-on } d' \text{ } (\text{bound-tree } P \text{ } (\text{Fin } a)) \wedge$
 $k * \text{cap} \leq \text{card } (\text{bound-tree } P \text{ } (\text{Fin } a))) \vee$
 $(\exists S\text{-pull } \text{beta } D\text{-pull.}$
 $\text{partition-state-pull } (M\text{-of } l) \text{ } B\text{max } D = (S\text{-pull, beta, } D\text{-pull}) \wedge$
 $\text{complete-on } d' \text{ } (\text{bound-tree } P \text{ } (\text{Fin } a)) \wedge$
 $\text{card } (\text{bound-tree } P \text{ } (\text{Fin } a)) < k * \text{cap} \wedge$
 $(\exists d\text{-child } b \text{ } c\text{-child } \text{betas } bs \text{ } B' \text{ } Us\text{-tail } U\text{-tail } c\text{-tail}$
 child-costs-tail.
 $a \leq b \wedge$
 $\text{bound-le } (\text{Fin } a) \text{ } B' \wedge$

exact-concrete-bmssp Δ *M-of t h k cap l d S-pull (Fin beta)*
d-child (Fin b) (range-tree P a (Fin b)) c-child $\wedge$
complete-preserved d-child d' (range-tree P a (Fin b)) $\wedge$
exact-concrete-partition-loop-state Δ *M-of t h k cap l d P*
(Fin Bmax) d'
(partition-state-batch-prepend
(label-pairs-between d S-pull b beta)
(partition-state-batch-prepend
(edge-relaxation-pairs-between d-child
(range-tree P a (Fin b)) b beta)
(partition-state-batch-prepend
(edge-relaxation-pairs-in-bound d-child
(range-tree P a (Fin b)) beta (Fin Bmax))
D-pull)))
b betas bs B' Us-tail U-tail c-tail child-costs-tail))
shows $\exists$ *betas bs B' Us U c child-costs.*
exact-concrete-partition-loop-state Δ *M-of t h k cap l d P*
(Fin Bmax) d' D a betas bs B' Us U c child-costs
proof –
consider
(Done-Case) keys-of (partition-state-view D) = {}
bound-le (Fin a) (Fin Bmax)
complete-on d' (bound-tree P (Fin Bmax))
| *(Stop-Case) bound-le (Fin a) (Fin Bmax)*
complete-on d' (bound-tree P (Fin a))
*k * cap $\leq$ card (bound-tree P (Fin a))*
| *(Step-Case) S-pull beta D-pull where*
partition-state-pull (M-of l) Bmax D = (S-pull, beta, D-pull)
complete-on d' (bound-tree P (Fin a))
*card (bound-tree P (Fin a)) < k * cap*
 $\exists$ *d-child b c-child betas bs B' Us-tail U-tail c-tail*
child-costs-tail.
a $\leq$ b $\wedge$
bound-le (Fin a) B' $\wedge$
exact-concrete-bmssp Δ *M-of t h k cap l d S-pull (Fin beta)*
d-child (Fin b) (range-tree P a (Fin b)) c-child $\wedge$
complete-preserved d-child d' (range-tree P a (Fin b)) $\wedge$
exact-concrete-partition-loop-state Δ *M-of t h k cap l d P*
(Fin Bmax) d'
(partition-state-batch-prepend (label-pairs-between d S-pull b beta)
(partition-state-batch-prepend
(edge-relaxation-pairs-between d-child
(range-tree P a (Fin b)) b beta)
(partition-state-batch-prepend
(edge-relaxation-pairs-in-bound d-child
(range-tree P a (Fin b)) beta (Fin Bmax))
D-pull)))
b betas bs B' Us-tail U-tail c-tail child-costs-tail
using cases by blast


```

then show ?thesis
proof cases
  case Done-Case
  show ?thesis
    by (rule exact-concrete-partition-loop-state-done-exists
      [OF Done-Case])
  next
  case Stop-Case
  show ?thesis
    by (rule exact-concrete-partition-loop-state-stop-exists
      [OF Stop-Case])
  next
  case (Step-Case S-pull beta D-pull)
  show ?thesis
    by (rule exact-concrete-initialized-partition-loop-state-nonterminal-exists-finite-
bound
      [OF init pre upper Step-Case(1) P-reaches Step-Case(2)
        Step-Case(3) Step-Case(4)])
qed
qed

```

lemma exact-concrete-initialized-partition-loop-state-cases-exists-finite-bound-with-complete:

```

fixes W :: bound  $\Rightarrow$  'a set
assumes init: exact-partition-initial-state t d P D c-insert
and pre: bmssp-pre-full d P (Fin Bmax)
and upper:  $\bigwedge u. u \in P \implies d\ u < Bmax$ 
and P-reaches:  $\forall x \in P. \text{reachable } s\ x$ 
and cases:
  (keys-of (partition-state-view D) = {}  $\wedge$ 
    bound-le (Fin a) (Fin Bmax)  $\wedge$ 
    complete-on d' (bound-tree P (Fin Bmax))  $\wedge$ 
    complete-on d' (W (Fin Bmax)))  $\vee$ 
  (bound-le (Fin a) (Fin Bmax)  $\wedge$ 
    complete-on d' (bound-tree P (Fin a))  $\wedge$ 
    k * cap  $\leq$  card (bound-tree P (Fin a))  $\wedge$ 
    complete-on d' (W (Fin a)))  $\vee$ 
  ( $\exists$  S-pull beta D-pull B'.
    partition-state-pull (M-of l) Bmax D = (S-pull, beta, D-pull)  $\wedge$ 
    complete-on d' (bound-tree P (Fin a))  $\wedge$ 
    card (bound-tree P (Fin a)) < k * cap  $\wedge$ 
    complete-on d' (W B')  $\wedge$ 
    ( $\exists$  d-child b c-child betas bs Us-tail U-tail c-tail
      child-costs-tail.
        a  $\leq$  b  $\wedge$ 
        bound-le (Fin a) B'  $\wedge$ 
        exact-concrete-bmssp  $\Delta$  M-of t h k cap l d S-pull (Fin beta)
          d-child (Fin b) (range-tree P a (Fin b)) c-child  $\wedge$ 
        complete-preserved d-child d' (range-tree P a (Fin b))  $\wedge$ 

```

exact-concrete-partition-loop-state Δ *M-of t h k cap l d P*
(Fin Bmax) d'
(partition-state-batch-prepend
(label-pairs-between d S-pull b beta)
(partition-state-batch-prepend
(edge-relaxation-pairs-between d-child
(range-tree P a (Fin b)) b beta)
(partition-state-batch-prepend
(edge-relaxation-pairs-in-bound d-child
(range-tree P a (Fin b)) beta (Fin Bmax))
D-pull)))
b betas bs B' Us-tail U-tail c-tail child-costs-tail))
shows $\exists$ *betas bs B' Us U c child-costs.*
exact-concrete-partition-loop-state Δ *M-of t h k cap l d P*
(Fin Bmax) d' D a betas bs B' Us U c child-costs $\wedge$
complete-on d' (W B')
proof –
consider
(Done-Case) keys-of (partition-state-view D) = {}
bound-le (Fin a) (Fin Bmax)
complete-on d' (bound-tree P (Fin Bmax))
complete-on d' (W (Fin Bmax))
| *(Stop-Case) bound-le (Fin a) (Fin Bmax)*
complete-on d' (bound-tree P (Fin a))
*k * cap $\leq$ card (bound-tree P (Fin a))*
complete-on d' (W (Fin a))
| *(Step-Case) S-pull beta D-pull B' where*
partition-state-pull (M-of l) Bmax D = (S-pull, beta, D-pull)
complete-on d' (bound-tree P (Fin a))
*card (bound-tree P (Fin a)) < k * cap*
complete-on d' (W B')
 $\exists$ *d-child b c-child betas bs Us-tail U-tail c-tail*
child-costs-tail.
a $\leq$ b $\wedge$
bound-le (Fin a) B' $\wedge$
exact-concrete-bmssp Δ *M-of t h k cap l d S-pull (Fin beta)*
d-child (Fin b) (range-tree P a (Fin b)) c-child $\wedge$
complete-preserved d-child d' (range-tree P a (Fin b)) $\wedge$
exact-concrete-partition-loop-state Δ *M-of t h k cap l d P*
(Fin Bmax) d'
(partition-state-batch-prepend (label-pairs-between d S-pull b beta)
(partition-state-batch-prepend
(edge-relaxation-pairs-between d-child
(range-tree P a (Fin b)) b beta)
(partition-state-batch-prepend
(edge-relaxation-pairs-in-bound d-child
(range-tree P a (Fin b)) beta (Fin Bmax))
D-pull)))
b betas bs B' Us-tail U-tail c-tail child-costs-tail

```

using cases by blast
then show ?thesis
proof cases
  case Done-Case
  have run:
    exact-concrete-partition-loop-state  $\Delta$  M-of t h k cap l d P
      (Fin Bmax) d' D a [] [] (Fin Bmax)
      [range-tree P a (Fin Bmax)]
      (bound-tree P (Fin a)  $\cup$ 
         $\bigcup$  (set [range-tree P a (Fin Bmax)])) 0 []
  by (rule exact-concrete-partition-loop-state-done-witness
    [OF Done-Case(1) Done-Case(2) Done-Case(3)])
  then show ?thesis
    using Done-Case(4) by blast
next
  case Stop-Case
  have run:
    exact-concrete-partition-loop-state  $\Delta$  M-of t h k cap l d P
      (Fin Bmax) d' D a [] [] (Fin a)
      [range-tree P a (Fin a)]
      (bound-tree P (Fin a)  $\cup$ 
         $\bigcup$  (set [range-tree P a (Fin a)])) 0 []
  by (rule exact-concrete-partition-loop-state-stop-witness
    [OF Stop-Case(1) Stop-Case(2) Stop-Case(3)])
  then show ?thesis
    using Stop-Case(4) by blast
next
  case (Step-Case S-pull beta D-pull B')
  then obtain d-child b c-child betas bs Us-tail U-tail c-tail
    child-costs-tail where
    a-le-b: a  $\leq$  b
    and a-bound: bound-le (Fin a) B'
    and child:
      exact-concrete-bmssp  $\Delta$  M-of t h k cap l d S-pull (Fin beta)
        d-child (Fin b) (range-tree P a (Fin b)) c-child
    and preserved:
      complete-preserved d-child d' (range-tree P a (Fin b))
    and tail:
      exact-concrete-partition-loop-state  $\Delta$  M-of t h k cap l d P
        (Fin Bmax) d'
        (partition-state-batch-prepend (label-pairs-between d S-pull b beta)
          (partition-state-batch-prepend
            (edge-relaxation-pairs-between d-child
              (range-tree P a (Fin b)) b beta)
            (partition-state-batch-prepend
              (edge-relaxation-pairs-in-bound d-child
                (range-tree P a (Fin b)) beta (Fin Bmax))
              D-pull)))
        b betas bs B' Us-tail U-tail c-tail child-costs-tail

```

by blast
 have loop:
 $\exists c$ child-costs.
 exact-concrete-partition-loop-state Δ M-of t h k cap l d P
 (Fin Bmax) d' D a (beta # betas) (b # bs) B'
 (range-tree P a (Fin b) # Us-tail)
 (bound-tree P (Fin a) $\cup$
 $\cup$ (set (range-tree P a (Fin b) # Us-tail))) c child-costs
 by (rule exact-concrete-initialized-partition-loop-state-step-exists-finite-bound
 [OF init pre upper Step-Case(1) P-reaches a-le-b a-bound
 Step-Case(2) Step-Case(3) child preserved tail])
 then show ?thesis
 using Step-Case(4) by blast
 qed
 qed

lemma exact-concrete-initialized-partition-loop-state-cases-exists-with-complete:

fixes W :: bound $\Rightarrow$ 'a set
 assumes init: exact-partition-initial-state t d P D c-insert
 and pre: bmssp-pre-full d P (Fin Bmax)
 and upper: $\bigwedge u. u \in P \implies d\ u < Bmax$
 and P-reaches: $\forall x \in P. \text{reachable } s\ x$
 and cases:
 (keys-of (partition-state-view D) = $\{\}$) $\wedge$
 bound-le (Fin a) B $\wedge$
 complete-on d' (bound-tree P B) $\wedge$
 complete-on d' (W B)) $\vee$
 (bound-le (Fin a) B $\wedge$
 complete-on d' (bound-tree P (Fin a)) $\wedge$
 $k * \text{cap} \leq \text{card } (\text{bound-tree } P\ (\text{Fin } a)) \wedge$
 complete-on d' (W (Fin a))) $\vee$
 ($\exists$ S-pull beta D-pull B'.
 partition-state-pull (M-of l) Bmax D = (S-pull, beta, D-pull) $\wedge$
 bound-le (Fin beta) B $\wedge$
 complete-on d' (bound-tree P (Fin a)) $\wedge$
 $\text{card } (\text{bound-tree } P\ (\text{Fin } a)) < k * \text{cap} \wedge$
 complete-on d' (W B') $\wedge$
 ($\exists$ d-child b c-child betas bs Us-tail U-tail c-tail
 child-costs-tail.
 $a \leq b \wedge$
 bound-le (Fin a) B' $\wedge$
 exact-concrete-bmssp Δ M-of t h k cap l d S-pull (Fin beta)
 d-child (Fin b) (range-tree P a (Fin b)) c-child $\wedge$
 complete-preserved d-child d' (range-tree P a (Fin b)) $\wedge$
 exact-concrete-partition-loop-state Δ M-of t h k cap l d P
 B d'
 (partition-state-batch-prepend
 (label-pairs-between d S-pull b beta)
 (partition-state-batch-prepend

(edge-relaxation-pairs-between *d-child*
 (range-tree *P a (Fin b)*) *b beta*)
 (partition-state-batch-prepend
 (edge-relaxation-pairs-in-bound *d-child*
 (range-tree *P a (Fin b)*) *beta B*)
D-pull)))
b betas bs B' Us-tail U-tail c-tail child-costs-tail)
shows $\exists \text{betas } bs \ B' \ Us \ U \ c \ \text{child-costs}.$
exact-concrete-partition-loop-state $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P$
B d' D a betas bs B' Us U c child-costs $\wedge$
complete-on d' (W B')
proof –
consider
 (Done-Case) *keys-of (partition-state-view D) = {}*
bound-le (Fin a) B
complete-on d' (bound-tree P B)
complete-on d' (W B)
 | (Stop-Case) *bound-le (Fin a) B*
complete-on d' (bound-tree P (Fin a))
*k * cap ≤ card (bound-tree P (Fin a))*
complete-on d' (W (Fin a))
 | (Step-Case) *S-pull beta D-pull B' where*
partition-state-pull (M-of l) Bmax D = (S-pull, beta, D-pull)
bound-le (Fin beta) B
complete-on d' (bound-tree P (Fin a))
*card (bound-tree P (Fin a)) < k * cap*
complete-on d' (W B')
 $\exists \text{d-child } b \ \text{c-child } betas \ bs \ Us\text{-tail } U\text{-tail } c\text{-tail}$
child-costs-tail.
 $a \leq b \wedge$
bound-le (Fin a) B' ∧
exact-concrete-bmssp $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-pull } (Fin \ beta)$
d-child (Fin b) (range-tree P a (Fin b)) c-child $\wedge$
complete-preserved d-child d' (range-tree P a (Fin b)) $\wedge$
exact-concrete-partition-loop-state $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P$
B d'
 (partition-state-batch-prepend (label-pairs-between *d S-pull b beta*)
 (partition-state-batch-prepend
 (edge-relaxation-pairs-between *d-child*
 (range-tree *P a (Fin b)*) *b beta*)
 (partition-state-batch-prepend
 (edge-relaxation-pairs-in-bound *d-child*
 (range-tree *P a (Fin b)*) *beta B*)
D-pull)))
b betas bs B' Us-tail U-tail c-tail child-costs-tail
using cases by blast
then show *?thesis*
proof cases
case Done-Case

```

have run:
  exact-concrete-partition-loop-state  $\Delta$  M-of t h k cap l d P
    B d' D a [] [] B [range-tree P a B]
    (bound-tree P (Fin a)  $\cup$ 
       $\bigcup$  (set [range-tree P a B])) 0 []
  by (rule exact-concrete-partition-loop-state-done-witness
    [OF Done-Case(1) Done-Case(2) Done-Case(3)])
then show ?thesis
  using Done-Case(4) by blast
next
case Stop-Case
have run:
  exact-concrete-partition-loop-state  $\Delta$  M-of t h k cap l d P
    B d' D a [] [] (Fin a)
    [range-tree P a (Fin a)]
    (bound-tree P (Fin a)  $\cup$ 
       $\bigcup$  (set [range-tree P a (Fin a)])) 0 []
  by (rule exact-concrete-partition-loop-state-stop-witness
    [OF Stop-Case(1) Stop-Case(2) Stop-Case(3)])
then show ?thesis
  using Stop-Case(4) by blast
next
case (Step-Case S-pull beta D-pull B')
then obtain d-child b c-child betas bs Us-tail U-tail c-tail
  child-costs-tail where
    a-le-b:  $a \leq b$ 
    and a-bound: bound-le (Fin a) B'
    and child:
      exact-concrete-bmssp  $\Delta$  M-of t h k cap l d S-pull (Fin beta)
      d-child (Fin b) (range-tree P a (Fin b)) c-child
    and preserved:
      complete-preserved d-child d' (range-tree P a (Fin b))
    and tail:
      exact-concrete-partition-loop-state  $\Delta$  M-of t h k cap l d P
      B d'
      (partition-state-batch-prepend (label-pairs-between d S-pull b beta)
        (partition-state-batch-prepend
          (edge-relaxation-pairs-between d-child
            (range-tree P a (Fin b)) b beta)
          (partition-state-batch-prepend
            (edge-relaxation-pairs-in-bound d-child
              (range-tree P a (Fin b)) beta B)
            D-pull)))
      b betas bs B' Us-tail U-tail c-tail child-costs-tail
    by blast
have S-pull-reaches:  $\forall x \in S\text{-pull. reachable } s \ x$ 
  by (rule exact-partition-initial-state-pull-reaches
    [OF init upper Step-Case(1) P-reaches])
have loop:

```

$\exists c$ *child-costs*.
exact-concrete-partition-loop-state Δ *M-of t h k cap l d P*
 $B \ d' \ D \ a \ (\text{beta} \ \# \ \text{betas}) \ (b \ \# \ bs) \ B'$
 $(\text{range-tree } P \ a \ (\text{Fin } b) \ \# \ Us\text{-tail})$
 $(\text{bound-tree } P \ (\text{Fin } a) \cup$
 $\quad \cup (\text{set } (\text{range-tree } P \ a \ (\text{Fin } b) \ \# \ Us\text{-tail}))) \ c \ \text{child-costs}$
by (*rule exact-concrete-initialized-partition-loop-state-step-exists*
 $[OF \ \text{init} \ \text{pre} \ \text{upper} \ \text{Step-Case}(1) \ \text{Step-Case}(2) \ S\text{-pull-reaches}$
 $a\text{-le-}b \ a\text{-bound} \ \text{Step-Case}(3) \ \text{Step-Case}(4) \ \text{child} \ \text{refl}$
 $\text{preserved tail}]$)
then show *?thesis*
using *Step-Case(5)* **by** *blast*
qed
qed

lemma *exact-concrete-partition-loop-state-step-exists-from-concrete-ops*:
assumes *inv-D: partition-state-invar D*
and *upper-D: partition-upper-bound (partition-state-view D) Bmax*
and *pull*:
 $\text{partition-state-pull } (M\text{-of } l) \ Bmax \ D = (S\text{-pull}, \text{beta}, D\text{-pull})$
and *beta-bound*: $\text{bound-le } (\text{Fin } \text{beta}) \ B$
and *child-pre*: $\text{bmssp-pre-full } d \ S\text{-pull } (\text{Fin } \text{beta})$
and *S-pull-def*: $S\text{-pull} = \text{split-below } d \ P \ \text{beta}$
and *S-pull-reaches*: $\forall x \in S\text{-pull}. \text{reachable } s \ x$
and *a-le-b*: $a \leq b$
and *a-bound*: $\text{bound-le } (\text{Fin } a) \ B'$
and *complete-prefix*: $\text{complete-on } d' \ (\text{bound-tree } P \ (\text{Fin } a))$
and *below-threshold*: $\text{card } (\text{bound-tree } P \ (\text{Fin } a)) < k * \text{cap}$
and *child*:
 $\text{exact-concrete-bmssp } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S\text{-pull } (\text{Fin } \text{beta})$
 $d\text{-child } (\text{Fin } b) \ U\text{-child } c\text{-child}$
and *U-child*: $U\text{-child} = \text{range-tree } P \ a \ (\text{Fin } b)$
and *preserved*: $\text{complete-preserved } d\text{-child } d' \ U\text{-child}$
and *tail*:
 $\text{exact-concrete-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d'$
 $(\text{partition-state-batch-prepend } (\text{label-pairs-between } d \ S\text{-pull } b \ \text{beta})$
 $(\text{partition-state-batch-prepend}$
 $(\text{edge-relaxation-pairs-between } d\text{-child } U\text{-child } b \ \text{beta})$
 $(\text{partition-state-batch-prepend}$
 $(\text{edge-relaxation-pairs-in-bound } d\text{-child } U\text{-child } \text{beta } B)$
 $D\text{-pull})))$
 $b \ \text{betas } bs \ B' \ Us\text{-tail } U\text{-tail } c\text{-tail } \text{child-costs-tail}$
shows $\exists c \ \text{child-costs}$.
 $\text{exact-concrete-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ P \ B \ d' \ D \ a$
 $(\text{beta} \ \# \ \text{betas}) \ (b \ \# \ bs) \ B'$
 $(\text{range-tree } P \ a \ (\text{Fin } b) \ \# \ Us\text{-tail})$
 $(\text{bound-tree } P \ (\text{Fin } a) \cup$
 $\quad \cup (\text{set } (\text{range-tree } P \ a \ (\text{Fin } b) \ \# \ Us\text{-tail}))) \ c \ \text{child-costs}$
proof –

have *run*:
exact-concrete-partition-loop-state Δ *M-of t h k cap l d P B d' D a*
(beta # betas) (b # bs) B'
(range-tree P a (Fin b) # Us-tail)
(bound-tree P (Fin a) $\cup$
$\bigcup$ (set (range-tree P a (Fin b) # Us-tail)))
(card S-pull +
*t * length (edge-relaxation-pairs-in-bound d-child U-child beta B) +*
*h * length (edge-relaxation-pairs-between d-child U-child b beta) +*
*h * length (label-pairs-between d S-pull b beta) +*
c-child + c-tail)
(c-child # child-costs-tail)
by (*rule exact-concrete-partition-loop-state-step-from-concrete-ops*
[OF inv-D upper-D pull beta-bound child-pre S-pull-def
S-pull-reaches a-le-b a-bound complete-prefix below-threshold child
U-child preserved tail])
show *?thesis*
using *run by blast*
qed

lemma *exact-concrete-bmssp-step-with-exact-insert-cost*:
assumes *init: exact-partition-initial-state t*
(find-pivots-label-capped k cap d S B)
(find-pivots-pivots-capped k cap d S B) D c-insert
and *loop*:
exact-concrete-partition-loop-state Δ *M-of t h k cap l*
(find-pivots-label-capped k cap d S B)
(find-pivots-pivots-capped k cap d S B) B d' D a
betas bs B' Us U-loop c-loop child-costs-loop
and *complete*:
complete-on d'
{v $\in$ bound-tree S B'. find-pivots-label-capped k cap d S B v =
dist s v}
and *U*:
U = U-loop $\cup$
{v $\in$ bound-tree S B'. find-pivots-label-capped k cap d S B v =
dist s v}
shows *exact-concrete-bmssp* Δ *M-of t h k cap (Suc l) d S B d' B' U*
(fp-iter-capped-scan-cost k cap d S S B +
*t * card (find-pivots-pivots-capped k cap d S B) + c-loop)*
proof –
have *c-insert*:
*c-insert = t * card (find-pivots-pivots-capped k cap d S B)*
by (*rule exact-partition-initial-state-cost[OF init]*)
show *?thesis*
by (*rule Exact-Concrete-Step[OF init loop complete U]*)
(simp add: c-insert)
qed

lemma *exact-concrete-bmssp-Suc-exists-from-initial-loop:*
assumes *S-subset:* $S \subseteq V$
and *loop:*
 $\wedge D.$ *exact-partition-initial-state* t
(find-pivots-label-capped k *cap* d S B)
(find-pivots-pivots-capped k *cap* d S B) D
 $(t * \text{card } (\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B)) \implies$
 $\exists d' a \text{ betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs-loop}.$
exact-concrete-partition-loop-state Δ *M-of* t h k *cap* l
(find-pivots-label-capped k *cap* d S B)
(find-pivots-pivots-capped k *cap* d S B) B d' D a
betas bs B' Us $U\text{-loop } c\text{-loop } \text{child-costs-loop} \wedge$
complete-on d'
 $\{v \in \text{bound-tree } S \ B'. \}$
find-pivots-label-capped k *cap* d S B $v = \text{dist } s \ v\}$
shows $\exists d' \ B' \ U \ c.$
exact-concrete-bmssp Δ *M-of* t h k *cap* $(\text{Suc } l)$ d S B d' B' U c
proof –
let $?d\text{-fp} = \text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B$
let $?P = \text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B$
obtain D **where** *init:*
exact-partition-initial-state t $?d\text{-fp}$ $?P$ D $(t * \text{card } ?P)$
using *exact-partition-initial-state-exists-for-capped-pivots-with-cost*
 $[OF \ S\text{-subset}]$ **by** *blast*
obtain $d' a \text{ betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs-loop}$ **where**
loop-run:
exact-concrete-partition-loop-state Δ *M-of* t h k *cap* l
 $?d\text{-fp}$ $?P$ B d' D $a \text{ betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs-loop}$
and *complete:*
complete-on d'
 $\{v \in \text{bound-tree } S \ B'. \ ?d\text{-fp } v = \text{dist } s \ v\}$
using *loop[OF init]* **by** *blast*
let $?U = U\text{-loop} \cup \{v \in \text{bound-tree } S \ B'. \ ?d\text{-fp } v = \text{dist } s \ v\}$
have *run:*
exact-concrete-bmssp Δ *M-of* t h k *cap* $(\text{Suc } l)$ d S B d' B' $?U$
 $(\text{fp-iter-capped-scan-cost } k \text{ cap } d \text{ } S \text{ } S \text{ } B + t * \text{card } ?P + c\text{-loop})$
by *(rule exact-concrete-bmssp-step-with-exact-insert-cost*
 $[OF \ \text{init loop-run complete refl}]$)
show *?thesis*
using *run* **by** *blast*
qed

The non-base existence lemmas are phrased as case combinators. They do not decide which branch an implementation should take; instead they say that if a client can provide one of the expected witnesses (empty partition, threshold, or a recursive child plus tail loop), then a full exact concrete BMSSP step exists. This keeps totality concerns separate from the correctness and cost refinement already proved for any exact run.

lemma *exact-concrete-bmssp-Suc-exists-from-initial-loop-cases:*

```

fixes  $d :: 'a \Rightarrow \text{real}$ 
and  $S :: 'a \text{ set}$ 
and  $B :: \text{bound}$ 
and  $Bmax :: \text{real}$ 
and  $k \text{ cap} :: \text{nat}$ 
and  $d\text{-fp} :: 'a \Rightarrow \text{real}$ 
and  $P :: 'a \text{ set}$ 
defines  $d\text{-fp-def}$ :
   $d\text{-fp} \equiv \text{find-pivots-label-capped } k \text{ cap } d \ S \ B$ 
and  $P\text{-def}$ :
   $P \equiv \text{find-pivots-pivots-capped } k \text{ cap } d \ S \ B$ 
assumes  $S\text{-subset}$ :  $S \subseteq V$ 
and  $\text{pre-fp}$ :  $\text{bmssp-pre-full } d\text{-fp } P \ (\text{Fin } Bmax)$ 
and  $\text{upper}$ :  $\bigwedge x. x \in P \implies d\text{-fp } x < Bmax$ 
and  $P\text{-reaches}$ :  $\forall x \in P. \text{reachable } s \ x$ 
and  $\text{cases}$ :
   $\bigwedge D. \text{exact-partition-initial-state } t \ d\text{-fp } P \ D \ (t * \text{card } P) \implies$ 
     $\exists d' a.$ 
       $(\text{keys-of } (\text{partition-state-view } D) = \{\}) \wedge$ 
       $\text{bound-le } (\text{Fin } a) \ B \wedge$ 
       $\text{complete-on } d' \ (\text{bound-tree } P \ B) \wedge$ 
       $\text{complete-on } d'$ 
       $\{v \in \text{bound-tree } S \ B. d\text{-fp } v = \text{dist } s \ v\} \vee$ 
       $(\text{bound-le } (\text{Fin } a) \ B \wedge$ 
       $\text{complete-on } d' \ (\text{bound-tree } P \ (\text{Fin } a)) \wedge$ 
       $k * \text{cap} \leq \text{card } (\text{bound-tree } P \ (\text{Fin } a)) \wedge$ 
       $\text{complete-on } d'$ 
       $\{v \in \text{bound-tree } S \ (\text{Fin } a). d\text{-fp } v = \text{dist } s \ v\}) \vee$ 
       $(\exists S\text{-pull } \text{beta } D\text{-pull } B'.$ 
       $\text{partition-state-pull } (M\text{-of } l) \ Bmax \ D = (S\text{-pull}, \text{beta}, D\text{-pull}) \wedge$ 
       $\text{bound-le } (\text{Fin } \text{beta}) \ B \wedge$ 
       $\text{complete-on } d' \ (\text{bound-tree } P \ (\text{Fin } a)) \wedge$ 
       $\text{card } (\text{bound-tree } P \ (\text{Fin } a)) < k * \text{cap} \wedge$ 
       $\text{complete-on } d'$ 
       $\{v \in \text{bound-tree } S \ B'. d\text{-fp } v = \text{dist } s \ v\} \wedge$ 
       $(\exists d\text{-child } b \ c\text{-child } \text{betas } bs \ Us\text{-tail } U\text{-tail } c\text{-tail}$ 
       $\text{child-costs-tail.}$ 
       $a \leq b \wedge$ 
       $\text{bound-le } (\text{Fin } a) \ B' \wedge$ 
       $\text{exact-concrete-bmssp } \Delta \ M\text{-of } t \ h \ k \text{ cap } l \ d\text{-fp } S\text{-pull}$ 
       $(\text{Fin } \text{beta}) \ d\text{-child } (\text{Fin } b) \ (\text{range-tree } P \ a \ (\text{Fin } b))$ 
       $c\text{-child} \wedge$ 
       $\text{complete-preserved } d\text{-child } d' \ (\text{range-tree } P \ a \ (\text{Fin } b)) \wedge$ 
       $\text{exact-concrete-partition-loop-state } \Delta \ M\text{-of } t \ h \ k \text{ cap } l$ 
       $d\text{-fp } P \ B \ d'$ 
       $(\text{partition-state-batch-prepend}$ 
       $(\text{label-pairs-between } d\text{-fp } S\text{-pull } b \ \text{beta})$ 
       $(\text{partition-state-batch-prepend}$ 
       $(\text{edge-relaxation-pairs-between } d\text{-child}$ 

```

(range-tree P a (Fin b)) b β)
 (partition-state-batch-prepend
 (edge-relaxation-pairs-in-bound d -child
 (range-tree P a (Fin b)) β) B)
 D -pull)))
 b β s bs B' Us -tail U -tail c -tail $child$ -costs-tail))
shows $\exists d' B' U c$.
exact-concrete-bmssp Δ M -of t h k cap (Suc l) d S B d' B' U c
proof –
obtain D **where** *init*:
exact-partition-initial-state t d -fp P D ($t * card$ P)
using *exact-partition-initial-state-exists-for-capped-pivots-with-cost*
 [*OF* S -subset]
unfolding d -fp-def P -def **by** *blast*
obtain d' a **where** *case-data*:
 (keys-of (partition-state-view D) = $\{\}$) $\wedge$
 bound-le (Fin a) B $\wedge$
 complete-on d' (bound-tree P B) $\wedge$
 complete-on d'
 { $v \in$ bound-tree S B . d -fp $v = dist$ s v } $\vee$
 (bound-le (Fin a) B $\wedge$
 complete-on d' (bound-tree P (Fin a)) $\wedge$
 $k * cap \leq card$ (bound-tree P (Fin a)) $\wedge$
 complete-on d'
 { $v \in$ bound-tree S (Fin a). d -fp $v = dist$ s v } $\vee$
 ($\exists S$ -pull β D -pull B').
partition-state-pull (M -of l) $Bmax$ $D = (S$ -pull, β , D -pull) $\wedge$
 bound-le (Fin β) B $\wedge$
 complete-on d' (bound-tree P (Fin a)) $\wedge$
 $card$ (bound-tree P (Fin a)) $< k * cap$ $\wedge$
 complete-on d'
 { $v \in$ bound-tree S B' . d -fp $v = dist$ s v } $\wedge$
 ($\exists d$ -child b c -child β s bs Us -tail U -tail c -tail
 $child$ -costs-tail.
 $a \leq b$ $\wedge$
 bound-le (Fin a) B' $\wedge$
exact-concrete-bmssp Δ M -of t h k cap l d -fp S -pull
 (Fin β) d -child (Fin b) (range-tree P a (Fin b))
 c -child $\wedge$
complete-preserved d -child d' (range-tree P a (Fin b)) $\wedge$
exact-concrete-partition-loop-state Δ M -of t h k cap l
 d -fp P B d'
 (partition-state-batch-prepend
 (label-pairs-between d -fp S -pull b β)
 (partition-state-batch-prepend
 (edge-relaxation-pairs-between d -child
 (range-tree P a (Fin b)) b β)
 (partition-state-batch-prepend
 (edge-relaxation-pairs-in-bound d -child

```

      (range-tree P a (Fin b)) beta B)
    D-pull)))
  b betas bs B' Us-tail U-tail c-tail child-costs-tail))
using cases[OF init] by blast
obtain betas bs B' Us U-loop c-loop child-costs-loop where
  loop:
    exact-concrete-partition-loop-state  $\Delta$  M-of t h k cap l d-fp P
    B d' D a betas bs B' Us U-loop c-loop child-costs-loop
and complete:
  complete-on d'
    {v  $\in$  bound-tree S B'. d-fp v = dist s v}
using exact-concrete-initialized-partition-loop-state-cases-exists-with-complete
  [where W =  $\lambda B'. \{v \in \text{bound-tree } S \ B'. \text{d-fp } v = \text{dist } s \ v\}$ ,
    OF init pre-fp upper P-reaches case-data]
by blast
have init-actual:
  exact-partition-initial-state t
    (find-pivots-label-capped k cap d S B)
    (find-pivots-pivots-capped k cap d S B) D
    (t * card (find-pivots-pivots-capped k cap d S B))
using init unfolding d-fp-def P-def .
have loop-actual:
  exact-concrete-partition-loop-state  $\Delta$  M-of t h k cap l
    (find-pivots-label-capped k cap d S B)
    (find-pivots-pivots-capped k cap d S B)
    B d' D a betas bs B' Us U-loop c-loop child-costs-loop
using loop unfolding d-fp-def P-def .
have complete-actual:
  complete-on d'
    {v  $\in$  bound-tree S B'.
      find-pivots-label-capped k cap d S B v = dist s v}
using complete unfolding d-fp-def .
let ?U =
  U-loop  $\cup$ 
    {v  $\in$  bound-tree S B'.
      find-pivots-label-capped k cap d S B v = dist s v}
have run:
  exact-concrete-bmssp  $\Delta$  M-of t h k cap (Suc l) d S B d' B' ?U
    (fp-iter-capped-scan-cost k cap d S S B +
      t * card (find-pivots-pivots-capped k cap d S B) + c-loop)
by (rule exact-concrete-bmssp-step-with-exact-insert-cost
    [OF init-actual loop-actual complete-actual refl])
then show ?thesis
by blast
qed

lemma exact-concrete-bmssp-Suc-exists-from-finite-initial-loop-cases:
  fixes d :: 'a  $\Rightarrow$  real
  and S :: 'a set

```

and $Bmax :: real$
and $k\ cap :: nat$
and $d\text{-}fp :: 'a \Rightarrow real$
and $P :: 'a\ set$
defines $d\text{-}fp\text{-}def$:
 $d\text{-}fp \equiv find\text{-}pivots\text{-}label\text{-}capped\ k\ cap\ d\ S\ (Fin\ Bmax)$
and $P\text{-}def$:
 $P \equiv find\text{-}pivots\text{-}pivots\text{-}capped\ k\ cap\ d\ S\ (Fin\ Bmax)$
assumes $sound$: $sound\text{-}label\ d$
and pre : $bmssp\text{-}pre\text{-}full\ d\ S\ (Fin\ Bmax)$
and $S\text{-}reaches$: $\forall x \in S. reachable\ s\ x$
and $below$: $\bigwedge x. x \in S \implies d\ x < Bmax$
and $cases$:
 $\bigwedge D. exact\text{-}partition\text{-}initial\text{-}state\ t\ d\text{-}fp\ P\ D\ (t * card\ P) \implies$
 $\exists d' a.$
 $(keys\text{-}of\ (partition\text{-}state\text{-}view\ D) = \{\}) \wedge$
 $bound\text{-}le\ (Fin\ a)\ (Fin\ Bmax) \wedge$
 $complete\text{-}on\ d'\ (bound\text{-}tree\ P\ (Fin\ Bmax)) \wedge$
 $complete\text{-}on\ d'$
 $\{v \in bound\text{-}tree\ S\ (Fin\ Bmax). d\text{-}fp\ v = dist\ s\ v\} \vee$
 $(bound\text{-}le\ (Fin\ a)\ (Fin\ Bmax) \wedge$
 $complete\text{-}on\ d'\ (bound\text{-}tree\ P\ (Fin\ a)) \wedge$
 $k * cap \leq card\ (bound\text{-}tree\ P\ (Fin\ a)) \wedge$
 $complete\text{-}on\ d'$
 $\{v \in bound\text{-}tree\ S\ (Fin\ a). d\text{-}fp\ v = dist\ s\ v\} \vee$
 $(\exists S\text{-}pull\ beta\ D\text{-}pull\ B'.$
 $partition\text{-}state\text{-}pull\ (M\text{-}of\ l)\ Bmax\ D = (S\text{-}pull, beta, D\text{-}pull) \wedge$
 $complete\text{-}on\ d'\ (bound\text{-}tree\ P\ (Fin\ a)) \wedge$
 $card\ (bound\text{-}tree\ P\ (Fin\ a)) < k * cap \wedge$
 $complete\text{-}on\ d'$
 $\{v \in bound\text{-}tree\ S\ B'. d\text{-}fp\ v = dist\ s\ v\} \wedge$
 $(\exists d\text{-}child\ b\ c\text{-}child\ betas\ bs\ Us\text{-}tail\ U\text{-}tail\ c\text{-}tail$
 $child\text{-}costs\text{-}tail.$
 $a \leq b \wedge$
 $bound\text{-}le\ (Fin\ a)\ B' \wedge$
 $exact\text{-}concrete\text{-}bmssp\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ l\ d\text{-}fp\ S\text{-}pull$
 $(Fin\ beta)\ d\text{-}child\ (Fin\ b)\ (range\text{-}tree\ P\ a\ (Fin\ b))$
 $c\text{-}child \wedge$
 $complete\text{-}preserved\ d\text{-}child\ d'\ (range\text{-}tree\ P\ a\ (Fin\ b)) \wedge$
 $exact\text{-}concrete\text{-}partition\text{-}loop\text{-}state\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ l$
 $d\text{-}fp\ P\ (Fin\ Bmax)\ d'$
 $(partition\text{-}state\text{-}batch\text{-}prepend$
 $(label\text{-}pairs\text{-}between\ d\text{-}fp\ S\text{-}pull\ b\ beta)$
 $(partition\text{-}state\text{-}batch\text{-}prepend$
 $(edge\text{-}relaxation\text{-}pairs\text{-}between\ d\text{-}child$
 $(range\text{-}tree\ P\ a\ (Fin\ b))\ b\ beta)$
 $(partition\text{-}state\text{-}batch\text{-}prepend$
 $(edge\text{-}relaxation\text{-}pairs\text{-}in\text{-}bound\ d\text{-}child$
 $(range\text{-}tree\ P\ a\ (Fin\ b))\ beta\ (Fin\ Bmax))$

$D\text{-pull}))$
 $b \text{ betas } bs \ B' \ Us\text{-tail } U\text{-tail } c\text{-tail } child\text{-costs-tail}))$

shows $\exists d' \ B' \ U \ c.$
 $exact\text{-concrete-bmssp} \ \Delta \ M\text{-of } t \ h \ k \ cap \ (Suc \ l) \ d \ S \ (Fin \ Bmax)$
 $d' \ B' \ U \ c$

proof –
have $S\text{-subset}: S \subseteq V$
using $pre \ unfolding \ bmssp\text{-pre-full-def} \ by \ blast$
obtain D **where** $init:$
 $exact\text{-partition-initial-state } t \ d\text{-fp } P \ D \ (t * card \ P)$
using $exact\text{-partition-initial-state-exists-for-capped-pivots-with-cost}$
 $[OF \ S\text{-subset}]$
unfolding $d\text{-fp-def } P\text{-def} \ by \ blast$
have $pre\text{-fp}: bmssp\text{-pre-full } d\text{-fp } P \ (Fin \ Bmax)$
unfolding $d\text{-fp-def } P\text{-def}$
by $(rule \ find\text{-pivots-capped-establishes-pivot-pre-concrete}$
 $[OF \ sound \ pre]) \ (use \ S\text{-reaches} \ in \ blast)$
have $upper: \bigwedge u. u \in P \implies d\text{-fp } u < Bmax$
unfolding $d\text{-fp-def } P\text{-def}$
by $(rule \ find\text{-pivots-pivots-capped-label-less-finite} [OF \ below])$
have $P\text{-reaches}: \forall x \in P. \text{reachable } s \ x$
unfolding $P\text{-def}$
by $(rule \ find\text{-pivots-pivots-capped-reaches} [OF \ S\text{-reaches}])$
obtain $d' \ a$ **where** $case\text{-data}:$
 $(keys\text{-of } (partition\text{-state-view } D) = \{\}) \wedge$
 $bound\text{-le } (Fin \ a) \ (Fin \ Bmax) \wedge$
 $complete\text{-on } d' \ (bound\text{-tree } P \ (Fin \ Bmax)) \wedge$
 $complete\text{-on } d'$
 $\{v \in bound\text{-tree } S \ (Fin \ Bmax). \ d\text{-fp } v = dist \ s \ v\} \vee$
 $(bound\text{-le } (Fin \ a) \ (Fin \ Bmax) \wedge$
 $complete\text{-on } d' \ (bound\text{-tree } P \ (Fin \ a)) \wedge$
 $k * cap \leq card \ (bound\text{-tree } P \ (Fin \ a)) \wedge$
 $complete\text{-on } d'$
 $\{v \in bound\text{-tree } S \ (Fin \ a). \ d\text{-fp } v = dist \ s \ v\} \vee$
 $(\exists S\text{-pull } beta \ D\text{-pull } B').$
 $partition\text{-state-pull } (M\text{-of } l) \ Bmax \ D = (S\text{-pull}, beta, D\text{-pull}) \wedge$
 $complete\text{-on } d' \ (bound\text{-tree } P \ (Fin \ a)) \wedge$
 $card \ (bound\text{-tree } P \ (Fin \ a)) < k * cap \wedge$
 $complete\text{-on } d'$
 $\{v \in bound\text{-tree } S \ B'. \ d\text{-fp } v = dist \ s \ v\} \wedge$
 $(\exists d\text{-child } b \ c\text{-child } betas \ bs \ Us\text{-tail } U\text{-tail } c\text{-tail}$
 $child\text{-costs-tail}.$
 $a \leq b \wedge$
 $bound\text{-le } (Fin \ a) \ B' \wedge$
 $exact\text{-concrete-bmssp} \ \Delta \ M\text{-of } t \ h \ k \ cap \ l \ d\text{-fp } S\text{-pull}$
 $(Fin \ beta) \ d\text{-child } (Fin \ b) \ (range\text{-tree } P \ a \ (Fin \ b))$
 $c\text{-child} \wedge$
 $complete\text{-preserved } d\text{-child } d' \ (range\text{-tree } P \ a \ (Fin \ b)) \wedge$
 $exact\text{-concrete-partition-loop-state} \ \Delta \ M\text{-of } t \ h \ k \ cap \ l$

```

    d-fp P (Fin Bmax) d'
    (partition-state-batch-prepend
     (label-pairs-between d-fp S-pull b beta)
     (partition-state-batch-prepend
      (edge-relaxation-pairs-between d-child
       (range-tree P a (Fin b)) b beta)
      (partition-state-batch-prepend
       (edge-relaxation-pairs-in-bound d-child
        (range-tree P a (Fin b)) beta (Fin Bmax))
       D-pull)))
    b betas bs B' Us-tail U-tail c-tail child-costs-tail))
using cases[OF init] by blast
obtain betas bs B' Us U-loop c-loop child-costs-loop where
loop:
  exact-concrete-partition-loop-state  $\Delta$  M-of t h k cap l d-fp P
  (Fin Bmax) d' D a betas bs B' Us U-loop c-loop
  child-costs-loop
and complete:
  complete-on d'
  {v  $\in$  bound-tree S B'. d-fp v = dist s v}
using exact-concrete-initialized-partition-loop-state-cases-exists-finite-bound-with-
complete
  [where W =  $\lambda B'. \{v \in \text{bound-tree } S \ B'. \text{d-fp } v = \text{dist } s \ v\}$ ,
   OF init pre-fp upper P-reaches case-data]
by blast
have init-actual:
  exact-partition-initial-state t
  (find-pivots-label-capped k cap d S (Fin Bmax))
  (find-pivots-pivots-capped k cap d S (Fin Bmax)) D
  (t * card (find-pivots-pivots-capped k cap d S (Fin Bmax)))
using init unfolding d-fp-def P-def .
have loop-actual:
  exact-concrete-partition-loop-state  $\Delta$  M-of t h k cap l
  (find-pivots-label-capped k cap d S (Fin Bmax))
  (find-pivots-pivots-capped k cap d S (Fin Bmax))
  (Fin Bmax) d' D a betas bs B' Us U-loop c-loop
  child-costs-loop
using loop unfolding d-fp-def P-def .
have complete-actual:
  complete-on d'
  {v  $\in$  bound-tree S B'.
   find-pivots-label-capped k cap d S (Fin Bmax) v = dist s v}
using complete unfolding d-fp-def .
let ?U =
  U-loop  $\cup$ 
  {v  $\in$  bound-tree S B'.
   find-pivots-label-capped k cap d S (Fin Bmax) v = dist s v}
have run:
  exact-concrete-bmssp  $\Delta$  M-of t h k cap (Suc l) d S (Fin Bmax)

```

$d' B' ?U$
 $(fp\text{-}iter\text{-}capped\text{-}scan\text{-}cost\ k\ cap\ d\ S\ S\ (Fin\ Bmax) +$
 $t * card\ (find\text{-}pivots\text{-}pivots\text{-}capped\ k\ cap\ d\ S\ (Fin\ Bmax)) +$
 $c\text{-}loop)$
by (rule exact-concrete-bmssp-step-with-exact-insert-cost
 $[OF\ init\text{-}actual\ loop\text{-}actual\ complete\text{-}actual\ refl])$
then show *?thesis*
by blast
qed

lemma exact-concrete-bmssp-Suc-exists-if-pivots-empty:
assumes $S\text{-}subset: S \subseteq V$
and $a\text{-}bound: bound\text{-}le\ (Fin\ a)\ B$
and $pivots\text{-}empty:$
 $find\text{-}pivots\text{-}pivots\text{-}capped\ k\ cap\ d\ S\ B = \{\}$
shows $\exists d' B' U c.$
 $exact\text{-}concrete\text{-}bmssp\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ (Suc\ l)\ d\ S\ B\ d'\ B'\ U\ c$
proof (rule exact-concrete-bmssp-Suc-exists-from-initial-loop[$OF\ S\text{-}subset$])
let $?d\text{-}fp = find\text{-}pivots\text{-}label\text{-}capped\ k\ cap\ d\ S\ B$
let $?P = find\text{-}pivots\text{-}pivots\text{-}capped\ k\ cap\ d\ S\ B$
fix D
assume $init: exact\text{-}partition\text{-}initial\text{-}state\ t\ ?d\text{-}fp\ ?P\ D\ (t * card\ ?P)$
have $view: partition\text{-}state\text{-}view\ D = label\text{-}partition\text{-}view\ ?d\text{-}fp\ ?P$
by (rule exact-partition-initial-state-view[$OF\ init$])
have $empty: keys\text{-}of\ (partition\text{-}state\text{-}view\ D) = \{\}$
using $view\ pivots\text{-}empty\ unfolding\ label\text{-}partition\text{-}view\text{-}def$ **by** simp
have $loop:$
 $exact\text{-}concrete\text{-}partition\text{-}loop\text{-}state\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ l$
 $?d\text{-}fp\ ?P\ B\ (dist\ s)\ D\ a\ []\ []\ B\ [range\text{-}tree\ ?P\ a\ B]$
 $(bound\text{-}tree\ ?P\ (Fin\ a) \cup \bigcup (set\ [range\text{-}tree\ ?P\ a\ B]))\ 0\ []$
by (rule exact-concrete-partition-loop-state-done-witness
 $[OF\ empty\ a\text{-}bound\ complete\text{-}on\text{-}dist])$
have $complete:$
 $complete\text{-}on\ (dist\ s)$
 $\{v \in bound\text{-}tree\ S\ B. ?d\text{-}fp\ v = dist\ s\ v\}$
by (rule complete-on-dist)
show $\exists d' a\ betas\ bs\ B'\ Us\ U\text{-}loop\ c\text{-}loop\ child\text{-}costs\text{-}loop.$
 $exact\text{-}concrete\text{-}partition\text{-}loop\text{-}state\ \Delta\ M\text{-}of\ t\ h\ k\ cap\ l$
 $?d\text{-}fp\ ?P\ B\ d'\ D\ a\ betas\ bs\ B'\ Us\ U\text{-}loop\ c\text{-}loop$
 $child\text{-}costs\text{-}loop \wedge$
 $complete\text{-}on\ d'$
 $\{v \in bound\text{-}tree\ S\ B'. ?d\text{-}fp\ v = dist\ s\ v\}$
using $loop\ complete$ **by** blast
qed

lemma exact-concrete-bmssp-Suc-exists-if-initial-threshold:
assumes $S\text{-}subset: S \subseteq V$
and $a\text{-}bound: bound\text{-}le\ (Fin\ a)\ B$
and $threshold:$

$k * \text{cap} \leq$
 $\text{card } (\text{bound-tree } (\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B) \text{ } (\text{Fin } a))$
shows $\exists d' B' U c.$
 $\text{exact-concrete-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } (\text{Suc } l) \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B' \text{ } U \text{ } c$
proof (rule *exact-concrete-bmssp-Suc-exists-from-initial-loop*[*OF S-subset*])
let $?d\text{-fp} = \text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B$
let $?P = \text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B$
fix D
assume *init*: *exact-partition-initial-state* $t \text{ } ?d\text{-fp} \text{ } ?P \text{ } D \text{ } (t * \text{card } ?P)$
have *loop*:
 $\text{exact-concrete-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } l$
 $?d\text{-fp} \text{ } ?P \text{ } B \text{ } (\text{dist } s) \text{ } D \text{ } a \text{ } [] \text{ } [] \text{ } (\text{Fin } a)$
 $[\text{range-tree } ?P \text{ } a \text{ } (\text{Fin } a)]$
 $(\text{bound-tree } ?P \text{ } (\text{Fin } a) \cup \bigcup (\text{set } [\text{range-tree } ?P \text{ } a \text{ } (\text{Fin } a)]))$
 $0 \text{ } []$
by (rule *exact-concrete-partition-loop-state-stop-witness*
 $[\text{OF } a\text{-bound complete-on-dist threshold}])$
have *complete*:
 $\text{complete-on } (\text{dist } s)$
 $\{v \in \text{bound-tree } S \text{ } (\text{Fin } a). \text{ } ?d\text{-fp } v = \text{dist } s \text{ } v\}$
by (rule *complete-on-dist*)
show $\exists d' a \text{ betas } bs \text{ } B' \text{ } Us \text{ } U\text{-loop } c\text{-loop } \text{child-costs-loop}.$
 $\text{exact-concrete-partition-loop-state } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } l$
 $?d\text{-fp} \text{ } ?P \text{ } B \text{ } d' \text{ } D \text{ } a \text{ betas } bs \text{ } B' \text{ } Us \text{ } U\text{-loop } c\text{-loop}$
 $\text{child-costs-loop} \wedge$
 $\text{complete-on } d'$
 $\{v \in \text{bound-tree } S \text{ } B'. \text{ } ?d\text{-fp } v = \text{dist } s \text{ } v\}$
using *loop complete* **by** *blast*
qed

lemma *exact-concrete-bmssp-Suc-exists-if-pivots-empty-same-bound*:

assumes $S\text{-subset}: S \subseteq V$

and $a\text{-bound}: \text{bound-le } (\text{Fin } a) \text{ } B$

and *pivots-empty*:

$\text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B = \{\}$

shows $\exists d' U c.$

$\text{exact-concrete-bmssp } \Delta \text{ } M\text{-of } t \text{ } h \text{ } k \text{ cap } (\text{Suc } l) \text{ } d \text{ } S \text{ } B \text{ } d' \text{ } B \text{ } U \text{ } c$

proof –

let $?d\text{-fp} = \text{find-pivots-label-capped } k \text{ cap } d \text{ } S \text{ } B$

let $?P = \text{find-pivots-pivots-capped } k \text{ cap } d \text{ } S \text{ } B$

let $?U\text{-loop} =$

$\text{bound-tree } ?P \text{ } (\text{Fin } a) \cup \bigcup (\text{set } [\text{range-tree } ?P \text{ } a \text{ } B])$

obtain D **where** *init*:

$\text{exact-partition-initial-state } t \text{ } ?d\text{-fp} \text{ } ?P \text{ } D \text{ } (t * \text{card } ?P)$

using *exact-partition-initial-state-exists-for-capped-pivots-with-cost*
 $[\text{OF } S\text{-subset}]$ **by** *blast*

have *view*: *partition-state-view* $D = \text{label-partition-view } ?d\text{-fp} \text{ } ?P$

by (rule *exact-partition-initial-state-view*[*OF init*])

have *empty*: *keys-of* (*partition-state-view* D) = $\{\}$

using *view pivots-empty unfolding label-partition-view-def* **by** *simp*
have *loop*:
exact-concrete-partition-loop-state Δ *M-of t h k cap l*
?d-fp ?P B (dist s) D a [] [] B [range-tree ?P a B] ?U-loop
0 []
by (*rule exact-concrete-partition-loop-state-done-witness*
[OF empty a-bound complete-on-dist])
have *complete*:
complete-on (dist s)
{v ∈ bound-tree S B. ?d-fp v = dist s v}
by (*rule complete-on-dist*)
let *?U* = *?U-loop* $\cup$ *{v ∈ bound-tree S B. ?d-fp v = dist s v}*
have *run*:
exact-concrete-bmssp Δ *M-of t h k cap (Suc l) d S B*
(dist s) B ?U
*(fp-iter-capped-scan-cost k cap d S S B + t * card ?P + 0)*
by (*rule exact-concrete-bmssp-step-with-exact-insert-cost*
[OF init loop complete refl])
show *?thesis*
using *run* **by** *blast*
qed

lemma *exact-concrete-bmssp-Suc-exists-if-initial-threshold-bound*:
assumes *S-subset*: $S \subseteq V$
and *a-bound*: *bound-le (Fin a) B*
and *threshold*:
 $k * \text{cap} \leq$
 $\text{card} (\text{bound-tree} (\text{find-pivots-pivots-capped } k \text{ cap } d \text{ S B}) (\text{Fin a}))$
shows $\exists d' U c.$
exact-concrete-bmssp Δ *M-of t h k cap (Suc l) d S B*
d' (Fin a) U c
proof –
let *?d-fp* = *find-pivots-label-capped k cap d S B*
let *?P* = *find-pivots-pivots-capped k cap d S B*
let *?U-loop* =
bound-tree ?P (Fin a) $\cup \bigcup (\text{set } [\text{range-tree ?P a (Fin a)}])$
obtain *D* **where** *init*:
*exact-partition-initial-state t ?d-fp ?P D (t * card ?P)*
using *exact-partition-initial-state-exists-for-capped-pivots-with-cost*
[OF S-subset] **by** *blast*
have *loop*:
exact-concrete-partition-loop-state Δ *M-of t h k cap l*
?d-fp ?P B (dist s) D a [] [] (Fin a)
[range-tree ?P a (Fin a)] ?U-loop 0 []
by (*rule exact-concrete-partition-loop-state-stop-witness*
[OF a-bound complete-on-dist threshold])
have *complete*:
complete-on (dist s)
{v ∈ bound-tree S (Fin a). ?d-fp v = dist s v}

```

  by (rule complete-on-dist)
let ?U =
  ?U-loop  $\cup$  { $v \in \text{bound-tree } S \text{ (Fin } a). \text{ ?d-fp } v = \text{dist } s \text{ } v$ }
have run:
  exact-concrete-bmssp  $\Delta$  M-of t h k cap (Suc l) d S B
  (dist s) (Fin a) ?U
  (fp-iter-capped-scan-cost k cap d S S B + t * card ?P + 0)
  by (rule exact-concrete-bmssp-step-with-exact-insert-cost
    [OF init loop complete refl])
show ?thesis
  using run by blast
qed

```

The central refinement theorem forgets the concrete partition state and keeps only its abstract view. Concrete pulls refine to *pull-separates* and concrete batch prepends refine to *batch-min-update* with the expected cost predicates. After this projection, the exact run is a direct-insert costed run with the same labels, bounds, output sets, and total cost.

The final two theorems are immediate consequences of that projection: exact concrete BMSSP runs inherit the success-or-threshold property and the full BMSSP correctness theorem from the direct-insert layer.

theorem *exact-concrete-refines-direct-insert:*

```

  exact-concrete-partition-loop-state  $\Delta$  M-of t h k cap l d P B d' D a
  betas bs B' Us U c child-costs  $\implies$ 
  direct-insert-costed-partition-loop-state  $\Delta$  M-of t h k cap l d P B d'
  (partition-state-view D) a betas bs B' Us U c child-costs

```

and *exact-concrete-bmssp-refines-direct-insert:*

```

  exact-concrete-bmssp  $\Delta$  M-of t h k cap l d S B d' B' U c  $\implies$ 
  direct-insert-costed-bmssp  $\Delta$  M-of t h k cap l d S B d' B' U c

```

proof (induction rule: *exact-concrete-partition-loop-state-exact-concrete-bmssp.inducts*)

```

  case (Exact-Concrete-State-Done D a B d' P  $\Delta$  M-of t h k cap l d)
  then show ?case
    by (rule Direct-Insert-Costed-State-Done)

```

next

```

  case (Exact-Concrete-State-Stop D a B d' P k cap  $\Delta$  M-of t h l d)
  then show ?case
    by (rule Direct-Insert-Costed-State-Stop)

```

next

```

  case (Exact-Concrete-State-Step D Bmax M-of l S-pull beta D-pull c-pull B
    d P a b B' d' k cap  $\Delta$  t h d-child U-child c-child
    direct-edge-batch lower-edge-batch source-batch c-direct D-direct c-lower
    D-lower c-sources D-next betas bs Us-tail U-tail c-tail
    child-costs-tail c)

```

have *inv-D: partition-state-invar D*

using *Exact-Concrete-State-Step* **by** *blast*

have *upper-D: partition-upper-bound (partition-state-view D) Bmax*

using *Exact-Concrete-State-Step* **by** *blast*

have *pull-op:*

```

costed-partition-state-pull (M-of l) Bmax D S-pull beta D-pull c-pull
using Exact-Concrete-State-Step by blast
have pull:
  pull-separates (partition-state-view D) (M-of l) Bmax S-pull beta
    (partition-state-view D-pull)
  and pull-cost: partition-pull-cost-bound c-pull S-pull
  and inv-pull: partition-state-invar D-pull
  using costed-partition-state-pull-refines
    [OF inv-D - pull-op] upper-D
  unfolding partition-upper-bound-def by blast+
have direct-op:
  costed-partition-state-batch-prepend t D-pull direct-edge-batch c-direct D-direct
  using Exact-Concrete-State-Step by blast
have direct-ref:
  partition-state-invar D-direct  $\wedge$ 
  partition-state-view D-direct =
    batch-min-update (partition-state-view D-pull) direct-edge-batch  $\wedge$ 
    partition-batch-cost-bound c-direct t direct-edge-batch
  by (rule costed-partition-state-batch-prepend-refines
    [OF inv-pull direct-op])
have inv-direct: partition-state-invar D-direct
  using direct-ref by blast
have direct-view:
  partition-state-view D-direct =
    batch-min-update (partition-state-view D-pull) direct-edge-batch
  using direct-ref by blast
have direct-cost:
  partition-batch-cost-bound c-direct t direct-edge-batch
  using direct-ref by blast
have lower-op:
  costed-partition-state-batch-prepend h D-direct lower-edge-batch c-lower D-lower
  using Exact-Concrete-State-Step by blast
have lower-ref:
  partition-state-invar D-lower  $\wedge$ 
  partition-state-view D-lower =
    batch-min-update (partition-state-view D-direct) lower-edge-batch  $\wedge$ 
    partition-batch-cost-bound c-lower h lower-edge-batch
  by (rule costed-partition-state-batch-prepend-refines
    [OF inv-direct lower-op])
have inv-lower: partition-state-invar D-lower
  using lower-ref by blast
have lower-view:
  partition-state-view D-lower =
    batch-min-update (partition-state-view D-direct) lower-edge-batch
  using lower-ref by blast
have lower-cost:
  partition-batch-cost-bound c-lower h lower-edge-batch
  using lower-ref by blast
have source-op:

```

```

    costed-partition-state-batch-prepend  $h$   $D$ -lower source-batch  $c$ -sources  $D$ -next
using Exact-Concrete-State-Step by blast
have source-ref:
    partition-state-invar  $D$ -next  $\wedge$ 
    partition-state-view  $D$ -next =
        batch-min-update (partition-state-view  $D$ -lower) source-batch  $\wedge$ 
    partition-batch-cost-bound  $c$ -sources  $h$  source-batch
by (rule costed-partition-state-batch-prepend-refines
    [OF inv-lower source-op])
have source-view:
    partition-state-view  $D$ -next =
        batch-min-update (partition-state-view  $D$ -lower) source-batch
using source-ref by blast
have source-cost:
    partition-batch-cost-bound  $c$ -sources  $h$  source-batch
using source-ref by blast
have  $D$ -next-view:
    partition-state-view  $D$ -next =
        batch-min-update (partition-state-view  $D$ -pull)
        (direct-edge-batch @ lower-edge-batch @ source-batch)
using direct-view lower-view source-view by simp
have child-ref:
    direct-insert-costed-bmssp  $\Delta$   $M$ -of  $t$   $h$   $k$  cap  $l$   $d$   $S$ -pull ( $Fin$   $\beta$ )
     $d$ -child ( $Fin$   $b$ )  $U$ -child  $c$ -child
using Exact-Concrete-State-Step by blast
have tail-ref:
    direct-insert-costed-partition-loop-state  $\Delta$   $M$ -of  $t$   $h$   $k$  cap  $l$   $d$   $P$   $B$   $d'$ 
    (partition-state-view  $D$ -next)  $b$   $\beta$ s  $bs$   $B'$   $Us$ -tail  $U$ -tail  $c$ -tail
    child-costs-tail
using Exact-Concrete-State-Step by blast
show ?case
by (rule Direct-Insert-Costed-State-Step
    [where batch = direct-edge-batch @ lower-edge-batch @ source-batch
    and  $D$ -next = partition-state-view  $D$ -next])
    (use Exact-Concrete-State-Step pull child-ref D-next-view pull-cost
    direct-cost lower-cost source-cost tail-ref in auto)
next
case (Exact-Concrete-Base  $S$   $x$   $\Delta$   $M$ -of  $t$   $h$   $k$  cap  $d$   $B$ )
then show ?case
by (rule Direct-Insert-Costed-Base)
next
case (Exact-Concrete-Step  $t$   $k$  cap  $d$   $S$   $B$   $D$   $c$ -insert  $\Delta$   $M$ -of  $h$   $l$   $d'$   $a$ 
     $\beta$ s  $bs$   $B'$   $Us$   $U$ -loop  $c$ -loop child-costs-loop  $U$   $c$ )
let ? $d$ -fp = find-pivots-label-capped  $k$  cap  $d$   $S$   $B$ 
let ? $P$  = find-pivots-pivots-capped  $k$  cap  $d$   $S$   $B$ 
have init: exact-partition-initial-state  $t$  ? $d$ -fp ? $P$   $D$   $c$ -insert
and loop:
    exact-concrete-partition-loop-state  $\Delta$   $M$ -of  $t$   $h$   $k$  cap  $l$  ? $d$ -fp ? $P$   $B$   $d'$   $D$   $a$ 
     $\beta$ s  $bs$   $B'$   $Us$   $U$ -loop  $c$ -loop child-costs-loop

```

and *complete*:
complete-on $d' \{v \in \text{bound-tree } S \ B'. \ ?d\text{-fp } v = \text{dist } s \ v\}$
and *U-def*:
 $U = U\text{-loop} \cup \{v \in \text{bound-tree } S \ B'. \ ?d\text{-fp } v = \text{dist } s \ v\}$
and *c-def*:
 $c = \text{fp-iter-capped-scan-cost } k \ \text{cap } d \ S \ S \ B + c\text{-insert} + c\text{-loop}$
using *Exact-Concrete-Step* **by** *blast+*
have *view*: *partition-state-view* $D = \text{label-partition-view } ?d\text{-fp } ?P$
using *exact-partition-initial-state-view* $[OF \ \text{init}]$.
have *insert-cost*: *partition-initial-insert-cost-bound* $c\text{-insert } t \ ?P$
using *exact-partition-initial-state-cost-bound* $[OF \ \text{init}]$.
have *loop-ref*:
direct-insert-costed-partition-loop-state $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ ?d\text{-fp}$
 $?P \ B \ d' \ (\text{partition-state-view } D) \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop}$
child-costs-loop
using *Exact-Concrete-Step* **by** *blast*
show $?case$
by $(\text{rule } \text{Direct-Insert-Costed-Step}$
 $[OF \ \text{view } \text{insert-cost } \text{loop-ref } \text{complete } U\text{-def } c\text{-def}])$
qed

theorem *exact-concrete-bmssp-Suc-success-or-threshold*:
assumes *run*:
exact-concrete-bmssp $\Delta \ M\text{-of } t \ h \ k \ \text{cap } (Suc \ l) \ d \ S \ B \ d' \ B' \ U \ c$
and *sound*: *sound-label* d
and *pre*: *bmssp-pre-full* $d \ S \ B$
and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
shows $B' = B \vee k * \text{cap} \leq \text{card } U$
by $(\text{rule } \text{direct-insert-costed-bmssp-Suc-success-or-threshold}$
 $[OF \ \text{exact-concrete-bmssp-refines-direct-insert} [OF \ \text{run}] \ \text{sound } \text{pre}$
 $S\text{-reaches}])$

theorem *exact-concrete-bmssp-correct*:
assumes *run*:
exact-concrete-bmssp $\Delta \ M\text{-of } t \ h \ k \ \text{cap } l \ d \ S \ B \ d' \ B' \ U \ c$
and *sound*: *sound-label* d
and *pre*: *bmssp-pre-full* $d \ S \ B$
and *S-reaches*: $\bigwedge x. x \in S \implies \text{reachable } s \ x$
shows *bmssp-post-full* $d \ S \ B \ d' \ B' \ U$
by $(\text{rule } \text{direct-insert-costed-bmssp-correct}$
 $[OF \ \text{exact-concrete-bmssp-refines-direct-insert} [OF \ \text{run}] \ \text{sound } \text{pre } S\text{-reaches}])$

end

end

theory *BMSSP-Top-Level-Bounds*

imports *BMSSP-Strict-Tie-Breaking* *BMSSP-Exact-Concrete-Cost*
begin

36 Top-Level BMSSP Theorems

This theory is the entry point for the checked correctness result for the top-level BMSSP algorithm modelled in this development. Everything before this point proves local correctness and local cost statements: FindPivots preserves the complete-source invariant, the partition loop realizes the recursive BMSSP step, and the concrete partition operations account for Insert, BatchPrepend, and Pull. This file is where those ingredients are closed over a root call and converted into the single-source shortest-path headline.

The algorithmic run relations used here are the strongest ones in the project. They record the capped FindPivots step, the exact pulled child source, the range-synchronised recursive calls, and the separated edge and source batch costs. The early theorems therefore look a little verbose: their purpose is to expose a precise bridge from the executable recurrence to the abstract correctness theorem while retaining enough cost structure for the asymptotic proof.

The locale assumption is the paper's tie-breaking consequence: shortest paths form a strict tree order. Under that assumption, the proof instantiates the BMSSP parameters with *sssp-log-one-third-param* and *sssp-log-two-thirds-param*. These correspond to the paper's $\log^{\sim\{1/3\}} n$ and $\log^{\sim\{2/3\}} n$ scales. The level capacity is computed by *bmssp-level-cap*; the point of the schedule is that the recursive tree depth, pivot growth, and bucketed partition cost telescope into the target $O(m * \log^{\sim\{2/3\}} n)$ envelope.

The file proceeds in four stages. First, it proves root-level correctness for operational and costed runs. Second, it specializes the refined graph bound to the logarithmic parameter schedule. Third, it packages the least top-level execution cost as a function and proves Big-O bounds for it. Finally, it exposes locale-level headline theorems that downstream theories can cite without reopening the cost recurrence.

context *strict-tie-breaking-digraph*
begin

theorem *operational-range-split-algorithm-correct*:

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$

and *run*:

full-operational-bmssp *k* *cap* *l*

finite-initial-label $\{s\}$ *Infinity* *d'* *Infinity* *U*

shows $U = V \wedge \text{sssp-correct } d'$

proof

have *pre*: *bmssp-pre-full* *finite-initial-label* $\{s\}$ *Infinity*

using *all-reachable* *finite-initial-label-source-complete*

by (*rule top-bmssp-pre-full*)

have *sound*: *sound-label* *finite-initial-label*

using *finite-initial-label-sound*[*OF all-reachable*] .

have *S-reaches*: $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$

using *all-reachable source-in-V* by *blast*
 have *post-full*:
 bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
 by (rule *full-operational-bmssp-correct[OF run sound pre S-reaches]*)
 have *post*: *bmssp-post finite-initial-label {s} Infinity d' Infinity U*
 using *bmssp-post-full-imp-post[OF post-full]* .
 show $U = V$
 using *post bound-tree-source-infinitiy[OF all-reachable]*
 unfolding *bmssp-post-def* by *auto*
 show *sssp-correct d'*
 using *finite-initial-label-full-operational-top-level-correct*
 [OF all-reachable run] .
 qed

theorem *operational-range-split-algorithm-reachable-correct*:

assumes *run*:
 full-operational-bmssp k cap l
 finite-initial-label {s} Infinity d' Infinity U
 shows *sssp-correct d'*
 proof –
 have *pre*: *bmssp-pre-full finite-initial-label {s} Infinity*
 using *finite-initial-label-source-complete*
 by (rule *top-bmssp-pre-full-reachable*)
 have *sound*: *sound-label finite-initial-label*
 by (rule *finite-initial-label-sound-reachable*)
 have *S-reaches*: $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$
 using *reachable-refl[OF source-in-V]* by *blast*
 have *post-full*:
 bmssp-post-full finite-initial-label {s} Infinity d' Infinity U
 by (rule *full-operational-bmssp-correct[OF run sound pre S-reaches]*)
 have *post*: *bmssp-post finite-initial-label {s} Infinity d' Infinity U*
 using *bmssp-post-full-imp-post[OF post-full]* .
 then show *?thesis*
 by (rule *successful-root-bmssp-sssp-correct*)
 qed

theorem *range-split-algorithm-correct*:

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
 and *run*:
 exact-split-range-costed-bmssp Δ (bmssp-level-cap k t) t h k
 (bmssp-level-cap k t l) l
 finite-initial-label {s} Infinity d' Infinity U c
 shows $U = V \wedge \text{sssp-correct } d'$
 proof
 have *pre*: *bmssp-pre-full finite-initial-label {s} Infinity*
 using *all-reachable finite-initial-label-source-complete*
 by (rule *top-bmssp-pre-full*)
 have *sound*: *sound-label finite-initial-label*
 using *finite-initial-label-sound[OF all-reachable]* .

have $S\text{-reaches}$: $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$
using $\text{all-reachable source-in-} V$ **by** blast
have post-full :
 $\text{bmssp-post-full finite-initial-label } \{s\} \text{ Infinity } d' \text{ Infinity } U$
by (rule $\text{exact-split-range-costed-bmssp-correct}$
 $[OF \text{ run sound pre } S\text{-reaches}]$)
have post : $\text{bmssp-post finite-initial-label } \{s\} \text{ Infinity } d' \text{ Infinity } U$
using $\text{bmssp-post-full-imp-post}[OF \text{ post-full}]$.
show $U = V$
using $\text{post bound-tree-source-infinitiy}[OF \text{ all-reachable}]$
unfolding bmssp-post-def **by** auto
show $\text{sssp-correct } d'$
by (rule $\text{finite-initial-label-exact-split-range-costed-top-level-correct}$
 $[OF \text{ all-reachable run}]$)
qed

theorem $\text{range-split-algorithm-reachable-correct}$:

assumes run :

$\text{exact-split-range-costed-bmssp } \Delta (\text{bmssp-level-cap } k \ t) \ t \ h \ k$
 $(\text{bmssp-level-cap } k \ t \ l) \ l$
 $\text{finite-initial-label } \{s\} \text{ Infinity } d' \text{ Infinity } U \ c$

shows $\text{sssp-correct } d'$

proof –

have pre : $\text{bmssp-pre-full finite-initial-label } \{s\} \text{ Infinity}$
using $\text{finite-initial-label-source-complete}$
by (rule $\text{top-bmssp-pre-full-reachable}$)
have sound : $\text{sound-label finite-initial-label}$
by (rule $\text{finite-initial-label-sound-reachable}$)
have $S\text{-reaches}$: $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$
using $\text{reachable-refl}[OF \text{ source-in-} V]$ **by** blast
have post-full :
 $\text{bmssp-post-full finite-initial-label } \{s\} \text{ Infinity } d' \text{ Infinity } U$
by (rule $\text{exact-split-range-costed-bmssp-correct}$
 $[OF \text{ run sound pre } S\text{-reaches}]$)
have post : $\text{bmssp-post finite-initial-label } \{s\} \text{ Infinity } d' \text{ Infinity } U$
using $\text{bmssp-post-full-imp-post}[OF \text{ post-full}]$.
then show $?thesis$
by (rule $\text{successful-root-bmssp-sssp-correct}$)
qed

theorem $\text{direct-insert-algorithm-correct}$:

assumes all-reachable : $\bigwedge v. v \in V \implies \text{reachable } s \ v$

and run :

$\text{direct-insert-costed-bmssp } \Delta (\text{bmssp-level-cap } k \ t) \ t \ h \ k$
 $(\text{bmssp-level-cap } k \ t \ l) \ l$
 $\text{finite-initial-label } \{s\} \text{ Infinity } d' \text{ Infinity } U \ c$

shows $U = V \wedge \text{sssp-correct } d'$

proof

have pre : $\text{bmssp-pre-full finite-initial-label } \{s\} \text{ Infinity}$

```

    using all-reachable finite-initial-label-source-complete
    by (rule top-bmssp-pre-full)
  have sound: sound-label finite-initial-label
    using finite-initial-label-sound[OF all-reachable] .
  have S-reaches:  $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$ 
    using all-reachable source-in-V by blast
  have post-full:
    bmssp-post-full finite-initial-label  $\{s\}$  Infinity  $d'$  Infinity  $U$ 
    by (rule direct-insert-costed-bmssp-correct[OF run sound pre S-reaches])
  have post: bmssp-post finite-initial-label  $\{s\}$  Infinity  $d'$  Infinity  $U$ 
    using bmssp-post-full-imp-post[OF post-full] .
  show  $U = V$ 
    using post bound-tree-source-infinity[OF all-reachable]
    unfolding bmssp-post-def by auto
  show sssp-correct  $d'$ 
    by (rule finite-initial-label-direct-insert-costed-top-level-correct
        [OF all-reachable run])
qed

theorem direct-insert-algorithm-reachable-correct:
  assumes run:
    direct-insert-costed-bmssp  $\Delta$  (bmssp-level-cap  $k \ t$ )  $t \ h \ k$ 
    (bmssp-level-cap  $k \ t \ l$ )  $l$ 
    finite-initial-label  $\{s\}$  Infinity  $d'$  Infinity  $U \ c$ 
  shows sssp-correct  $d'$ 
  by (rule finite-initial-label-direct-insert-costed-top-level-reachable-correct[OF run])

theorem direct-insert-algorithm-correct-and-refined-graph-bound-under-edge-budget:
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
    and degree: edge-outdegree-le  $\Delta$ 
    and degree-factor:  $\Delta \leq A$ 
    and R-pos:  $0 < R$ 
    and insert-factor:  $t \leq A * k$ 
    and insert-scaled-factor:  $t \leq A\text{-insert} * k$ 
    and seen-scaled-factor:  $k * \Delta + A\text{-insert} \leq 2 * A$ 
    and source-factor:  $\text{Suc } h \leq 2 * A$ 
    and k-pos:  $0 < k$ 
  and edge-budget:
     $\bigwedge l' \ d \ P \ B \ d' \ D \ a \ \text{betas } bs \ B' \ Us \ U\text{-loop } c\text{-loop } \text{child-costs}.$ 
    direct-insert-costed-partition-loop-state  $\Delta$  (bmssp-level-cap  $k \ t$ )  $t \ h \ k$ 
    (bmssp-level-cap  $k \ t \ l$ )  $l' \ d \ P \ B \ d' \ D \ a \ \text{betas } bs \ B'$ 
     $Us \ U\text{-loop } c\text{-loop } \text{child-costs} \implies$ 
    sound-label  $d \implies$ 
    bmssp-pre-full  $d \ P \ B \implies$ 
     $(\bigwedge x. x \in P \implies \text{reachable } s \ x) \implies$ 
     $t * \text{sum-list}$ 
     $(\text{map card } (\text{range-tree-child-direct-edge-range-list } P \ B \ a \ \text{betas } bs)) +$ 
     $h * \text{sum-list}$ 
     $(\text{map card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$ 

```

$\leq H * \text{card}(\text{outgoing-edges } U\text{-loop})$
and run:
direct-insert-costed-bmssp Δ (*bmssp-level-cap* k t) t h k
(bmssp-level-cap k t l) l
finite-initial-label $\{s\}$ *Infinity* d' *Infinity* U c
shows $U = V \wedge \text{sssp-correct } d' \wedge$
 $c \leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. H)$
 $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{ vertex-count}$
proof –
have *UV-correct*: $U = V \wedge \text{sssp-correct } d'$
by (*rule direct-insert-algorithm-correct*[*OF all-reachable run*])
have *cost*:
 $\text{sssp-correct } d' \wedge$
 $c \leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. H)$
 $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{ vertex-count}$
by (*rule finite-initial-label-direct-insert-costed-top-level-correct-and-closed-bmssp-refined-graph-time-bound-level-cap*
 $[OF \text{ all-reachable degree degree-factor } R\text{-pos insert-factor insert-scaled-factor}$
 $\text{seen-scaled-factor source-factor } k\text{-pos edge-budget run}]$)
show *?thesis*
using *UV-correct cost by blast*
qed

theorem *direct-insert-algorithm-correct-and-refined-graph-bound-trivial-edge-budget:*

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$
and *insert-scaled-factor*: $t \leq A\text{-insert} * k$
and *seen-scaled-factor*: $k * \Delta + A\text{-insert} \leq 2 * A$
and *source-factor*: $\text{Suc } h \leq 2 * A$
and *k-pos*: $0 < k$
and run:
direct-insert-costed-bmssp Δ (*bmssp-level-cap* k t) t h k
(bmssp-level-cap k t l) l
finite-initial-label $\{s\}$ *Infinity* d' *Infinity* U c
shows $U = V \wedge \text{sssp-correct } d' \wedge$
 $c \leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. t + h)$
 $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{ vertex-count}$
proof –
have *UV-correct*: $U = V \wedge \text{sssp-correct } d'$
by (*rule direct-insert-algorithm-correct*[*OF all-reachable run*])
have *cost*:
 $\text{sssp-correct } d' \wedge$
 $c \leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. t + h)$
 $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{ vertex-count}$
by (*rule finite-initial-label-direct-insert-costed-top-level-correct-and-closed-bmssp-*

```

refined-graph-time-bound-level-cap-trivial-edge-budget
  [OF all-reachable degree degree-factor R-pos insert-factor insert-scaled-factor
    seen-scaled-factor source-factor k-pos run])
show ?thesis
  using UV-correct cost by blast
qed

theorem direct-insert-algorithm-correct-and-refined-graph-bound-amortized:
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
  and degree: edge-outdegree-le  $\Delta$ 
  and degree-factor:  $\Delta \leq A$ 
  and R-pos:  $0 < R$ 
  and insert-factor:  $t \leq A * k$ 
  and insert-scaled-factor:  $t \leq A\text{-insert} * k$ 
  and seen-scaled-factor:  $k * \Delta + A\text{-insert} \leq 2 * A$ 
  and source-factor:  $\text{Suc } h \leq 2 * A$ 
  and k-pos:  $0 < k$ 
  and run:
    direct-insert-costed-bmssp  $\Delta$  (bmssp-level-cap k t) t h k
    (bmssp-level-cap k t l) l
    finite-initial-label {s} Infinity d' Infinity U c
  shows  $U = V \wedge \text{sssp-correct } d' \wedge$ 
     $c \leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. h)$ 
     $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{ vertex-count}$ 
proof -
  have UV-correct:  $U = V \wedge \text{sssp-correct } d'$ 
  by (rule direct-insert-algorithm-correct[OF all-reachable run])
  have cost:
    sssp-correct  $d' \wedge$ 
     $c \leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. h)$ 
     $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{ vertex-count}$ 
  by (rule finite-initial-label-direct-insert-costed-top-level-correct-and-closed-bmssp-
    refined-graph-time-bound-level-cap-amortized
    [OF all-reachable degree degree-factor R-pos insert-factor insert-scaled-factor
      seen-scaled-factor source-factor k-pos run])
  show ?thesis
  using UV-correct cost by blast
qed

```

The preceding theorems keep the parameters abstract. They are useful because they show exactly which inequalities are required of a costed BMSSP run: outdegree must be bounded, Insert must be charged at the chosen parameter scale, and the source and seen-set terms must fit into the refined graph bound *bmssp-refined-graph-time-bound*. The amortized variant is the one that matches the bucketed partition analysis: the split work is not charged to each operation eagerly, but through the aggregate accounting used by the partition interface.

The next group substitutes the logarithmic schedule. The one-third parameter controls the number of levels and source growth, while the two-

thirds parameter controls the Insert scale supplied to the partition layer. The simple arithmetic lemmas in *sssp-log-one-third-param* and *sssp-log-two-thirds-param* prove that these choices satisfy the abstract side conditions.

theorem *direct-insert-algorithm-correct-and-refined-graph-bound-log-params-bounded-degree:*

```

fixes  $N\ p\ q :: \text{nat}$ 
defines  $p\text{-def}: p \equiv \text{sssp-log-one-third-param } N$ 
and  $q\text{-def}: q \equiv \text{sssp-log-two-thirds-param } N$ 
assumes  $\text{all-reachable}: \bigwedge v. v \in V \implies \text{reachable } s\ v$ 
and  $\text{degree}: \text{edge-outdegree-le } \Delta$ 
and  $\text{run}: \text{direct-insert-costed-bmssp } \Delta\ (\text{bmssp-level-cap } p\ q)\ q\ p\ p$ 
 $(\text{bmssp-level-cap } p\ q\ p)\ p$ 
 $\text{finite-initial-label } \{s\}\ \text{Infinity } d'\ \text{Infinity } U\ c$ 
shows  $U = V \wedge \text{sssp-correct } d' \wedge$ 
 $c \leq \text{bmssp-refined-graph-time-bound}$ 
 $(\lambda-. \text{Suc } \Delta * p)\ (\lambda-. q)\ (\lambda-. p)$ 
 $(\lambda-. p)\ (\lambda-. q)\ (\lambda-. \text{edge-count})\ \text{vertex-count}$ 

```

proof –

```

have  $p\text{-pos}: 0 < p$ 
unfolding  $p\text{-def}$  by  $\text{simp}$ 
have  $q\text{-pos}: 0 < q$ 
unfolding  $q\text{-def}$  by  $\text{simp}$ 
have  $q\text{-le-square}: q \leq p * p$ 
unfolding  $p\text{-def } q\text{-def}$ 
by  $(\text{rule } \text{sssp-log-two-thirds-param-le-one-third-square})$ 
have  $\text{degree-factor}: \Delta \leq \text{Suc } \Delta * p$ 
using  $p\text{-pos}$  by  $(\text{cases } p)\ \text{simp-all}$ 
have  $\text{insert-factor}: q \leq (\text{Suc } \Delta * p) * p$ 

```

proof –

```

have  $p * p \leq (\text{Suc } \Delta * p) * p$ 
by  $\text{simp}$ 
then show  $?thesis$ 
using  $q\text{-le-square}$  by  $\text{linarith}$ 

```

qed

```

have  $\text{insert-scaled-factor}: q \leq p * p$ 
using  $q\text{-le-square}$  .
have  $\text{seen-scaled-factor}: p * \Delta + p \leq 2 * (\text{Suc } \Delta * p)$ 

```

proof –

```

have  $p * \Delta + p = \text{Suc } \Delta * p$ 
by  $(\text{simp add: algebra-simps})$ 
also have  $\dots \leq 2 * (\text{Suc } \Delta * p)$ 
by  $\text{simp}$ 
finally show  $?thesis$  .

```

qed

```

have  $\text{source-factor}: \text{Suc } p \leq 2 * (\text{Suc } \Delta * p)$ 

```

proof –

```

have  $\text{Suc } p \leq 2 * p$ 
using  $p\text{-pos}$  by  $\text{simp}$ 

```

```

    also have ...  $\leq 2 * (Suc \Delta * p)$ 
      by simp
    finally show ?thesis .
  qed
show ?thesis
  by (rule direct-insert-algorithm-correct-and-refined-graph-bound-amortized
    [OF all-reachable degree degree-factor q-pos insert-factor
      insert-scaled-factor seen-scaled-factor source-factor p-pos run])
qed

theorem direct-insert-algorithm-correct-and-refined-graph-bound-log-params-fixed-degree:
  fixes  $N D p q :: nat$ 
  defines  $p\text{-def}: p \equiv sssp\text{-log-one-third-param } N$ 
    and  $q\text{-def}: q \equiv sssp\text{-log-two-thirds-param } N$ 
  assumes  $all\text{-reachable}: \bigwedge v. v \in V \implies reachable\ s\ v$ 
    and  $degree: edge\text{-outdegree-le } D$ 
    and  $run:$ 
     $direct\text{-insert-costed-bmssp } D\ (bmssp\text{-level-cap } p\ q)\ q\ p\ p$ 
     $(bmssp\text{-level-cap } p\ q\ p)\ p$ 
     $finite\text{-initial-label } \{s\}\ Infinity\ d'\ Infinity\ U\ c$ 
  shows  $U = V \wedge sssp\text{-correct } d' \wedge$ 
     $c \leq bmssp\text{-refined-graph-time-bound}$ 
     $(\lambda-. Suc\ D * p)\ (\lambda-. q)\ (\lambda-. p)$ 
     $(\lambda-. p)\ (\lambda-. q)\ (\lambda-. edge\text{-count})\ vertex\text{-count}$ 
proof –
  have  $run'$ :
     $direct\text{-insert-costed-bmssp } D$ 
     $(bmssp\text{-level-cap } (sssp\text{-log-one-third-param } N)\ (sssp\text{-log-two-thirds-param } N))$ 
     $(sssp\text{-log-two-thirds-param } N)\ (sssp\text{-log-one-third-param } N)$ 
     $(sssp\text{-log-one-third-param } N)$ 
     $(bmssp\text{-level-cap } (sssp\text{-log-one-third-param } N)\ (sssp\text{-log-two-thirds-param } N))$ 
     $(sssp\text{-log-one-third-param } N))$ 
     $(sssp\text{-log-one-third-param } N)$ 
     $finite\text{-initial-label } \{s\}\ Infinity\ d'\ Infinity\ U\ c$ 
  using  $run$  unfolding  $p\text{-def } q\text{-def}$  by  $simp$ 
show ?thesis
  unfolding  $p\text{-def } q\text{-def}$ 
  by (rule direct-insert-algorithm-correct-and-refined-graph-bound-log-params-bounded-degree
    [OF  $all\text{-reachable } degree\ run'$ ])
qed

theorem exact-concrete-algorithm-correct-and-refined-graph-bound-log-params-fixed-degree:
  fixes  $N D p q :: nat$ 
  defines  $p\text{-def}: p \equiv sssp\text{-log-one-third-param } N$ 
    and  $q\text{-def}: q \equiv sssp\text{-log-two-thirds-param } N$ 
  assumes  $all\text{-reachable}: \bigwedge v. v \in V \implies reachable\ s\ v$ 

```

```

and degree: edge-outdegree-le D
and run:
  exact-concrete-bmssp D (bmssp-level-cap p q) q p p
    (bmssp-level-cap p q p) p
    finite-initial-label {s} Infinity d' Infinity U c
shows U = V ∧ sssp-correct d' ∧
  c ≤ bmssp-refined-graph-time-bound
    (λ-. Suc D * p) (λ-. q) (λ-. p)
    (λ-. p) (λ-. q) (λ-. edge-count) vertex-count
proof –
  have direct-run:
    direct-insert-costed-bmssp D (bmssp-level-cap p q) q p p
      (bmssp-level-cap p q p) p
      finite-initial-label {s} Infinity d' Infinity U c
  by (rule exact-concrete-bmssp-refines-direct-insert[OF run])
  have direct-run':
    direct-insert-costed-bmssp D
      (bmssp-level-cap (sssp-log-one-third-param N) (sssp-log-two-thirds-param N))
      (sssp-log-two-thirds-param N) (sssp-log-one-third-param N)
      (sssp-log-one-third-param N)
      (bmssp-level-cap (sssp-log-one-third-param N) (sssp-log-two-thirds-param N)
        (sssp-log-one-third-param N))
      (sssp-log-one-third-param N)
      finite-initial-label {s} Infinity d' Infinity U c
  using direct-run unfolding p-def q-def by simp
  show ?thesis
  unfolding p-def q-def
  by (rule direct-insert-algorithm-correct-and-refined-graph-bound-log-params-fixed-
    degree
    [OF all-reachable degree direct-run'])
qed

```

The definitions below package the root execution as an ordinary total cost predicate. *exact-concrete-top-level-run* fixes the root source set to $\{s\}$ and the input bound to *Infinity*; the auxiliary *exact-concrete-root-run* keeps the returned bound visible for existence proofs that reason about threshold cases. The least-cost wrapper *exact-concrete-top-level-time* is not an algorithm in its own right; it is a mathematical way to speak about the cost of any concrete run satisfying the checked run relation.

definition *exact-concrete-top-level-run* ::
 $\text{nat} \Rightarrow \text{nat} \Rightarrow ('a \Rightarrow \text{real}) \Rightarrow 'a \text{ set} \Rightarrow \text{nat} \Rightarrow \text{bool}$ **where**
exact-concrete-top-level-run D N d' U c $\longleftrightarrow$
 (let p = sssp-log-one-third-param N;
 q = sssp-log-two-thirds-param N
 in *exact-concrete-bmssp* D (bmssp-level-cap p q) q p p
 (bmssp-level-cap p q p) p
 finite-initial-label {s} Infinity d' Infinity U c)

definition *exact-concrete-root-run* ::

$\text{nat} \Rightarrow \text{nat} \Rightarrow ('a \Rightarrow \text{real}) \Rightarrow \text{bound} \Rightarrow 'a \text{ set} \Rightarrow \text{nat} \Rightarrow \text{bool}$ **where**
 $\text{exact-concrete-root-run } D \ N \ d' \ B' \ U \ c \longleftrightarrow$
 $(\text{let } p = \text{sssp-log-one-third-param } N;$
 $\quad q = \text{sssp-log-two-thirds-param } N$
 $\text{in exact-concrete-bmssp } D \ (\text{bmssp-level-cap } p \ q) \ q \ p \ p$
 $\quad (\text{bmssp-level-cap } p \ q \ p) \ p$
 $\text{finite-initial-label } \{s\} \ \text{Infinity } d' \ B' \ U \ c)$

lemma $\text{exact-concrete-top-level-run-root-run-iff}$:
 $\text{exact-concrete-top-level-run } D \ N \ d' \ U \ c \longleftrightarrow$
 $\text{exact-concrete-root-run } D \ N \ d' \ \text{Infinity } U \ c$
unfolding $\text{exact-concrete-top-level-run-def}$ $\text{exact-concrete-root-run-def}$
by (simp add: Let-def)

definition $\text{exact-concrete-top-level-cost} :: \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow \text{bool}$ **where**
 $\text{exact-concrete-top-level-cost } D \ N \ c \longleftrightarrow$
 $(\exists d' \ U. \text{exact-concrete-top-level-run } D \ N \ d' \ U \ c)$

definition $\text{exact-concrete-top-level-time} :: \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat}$ **where**
 $\text{exact-concrete-top-level-time } D \ N =$
 $(\text{LEAST } c. \text{exact-concrete-top-level-cost } D \ N \ c)$

lemma $\text{exact-concrete-top-level-cost-exists-if-top-pivots-empty}$:

assumes pivots-empty :

$\text{find-pivots-pivots-capped } (\text{sssp-log-one-third-param } N)$
 $(\text{bmssp-level-cap } (\text{sssp-log-one-third-param } N))$
 $(\text{sssp-log-two-thirds-param } N) \ (\text{sssp-log-one-third-param } N))$
 $\text{finite-initial-label } \{s\} \ \text{Infinity} = \{\}$

shows $\exists c. \text{exact-concrete-top-level-cost } D \ N \ c$

proof –

let $?p = \text{sssp-log-one-third-param } N$

let $?q = \text{sssp-log-two-thirds-param } N$

have $p\text{-eq}: ?p = \text{Suc } (?p - 1)$

using $\text{sssp-log-one-third-param-pos[of } N]$ **by** simp

have $\text{source-subset}: \{s\} \subseteq V$

using $\text{source-in-}V$ **by** simp

have run :

$\exists d' \ U \ c.$

$\text{exact-concrete-bmssp } D \ (\text{bmssp-level-cap } ?p \ ?q) \ ?q \ ?p \ ?p$
 $(\text{bmssp-level-cap } ?p \ ?q \ ?p) \ (\text{Suc } (?p - 1))$
 $\text{finite-initial-label } \{s\} \ \text{Infinity } d' \ \text{Infinity } U \ c$

by ($\text{rule exact-concrete-bmssp-Suc-exists-if-pivots-empty-same-bound}$
 $[\text{OF source-subset - pivots-empty}]) \ \text{simp}$

then obtain $d' \ U \ c$ **where**

$\text{exact-concrete-bmssp } D \ (\text{bmssp-level-cap } ?p \ ?q) \ ?q \ ?p \ ?p$
 $(\text{bmssp-level-cap } ?p \ ?q \ ?p) \ (\text{Suc } (?p - 1))$
 $\text{finite-initial-label } \{s\} \ \text{Infinity } d' \ \text{Infinity } U \ c$

by blast

then have root-run :


```

    exact-concrete-bmssp D (bmssp-level-cap ?p ?q) ?q ?p ?p
      (bmssp-level-cap ?p ?q ?p) ?p
    finite-initial-label {s} Infinity d' Infinity U c
  using p-eq by simp
  have exact-concrete-top-level-run D N d' U c
    unfolding exact-concrete-top-level-run-def
    using root-run by (simp add: Let-def)
  then have exact-concrete-top-level-cost D N c
    unfolding exact-concrete-top-level-cost-def by blast
  then show ?thesis
    by blast
qed

```

```

lemma exact-concrete-top-level-cost-exists-from-initial-loop:
  fixes D N p q cap l :: nat
  and d-fp :: 'a ⇒ real
  and P :: 'a set
  defines p-def: p ≡ sssp-log-one-third-param N
  and q-def: q ≡ sssp-log-two-thirds-param N
  and cap-def: cap ≡ bmssp-level-cap p q p
  and l-def: l ≡ p - 1
  and d-fp-def:
    d-fp ≡ find-pivots-label-capped p cap finite-initial-label {s}
      Infinity
  and P-def:
    P ≡ find-pivots-pivots-capped p cap finite-initial-label {s}
      Infinity
  assumes loop:
    ∧ D-part. exact-partition-initial-state q d-fp P D-part
      (q * card P) ⇒
      ∃ d' a betas bs Us U-loop c-loop child-costs-loop.
        exact-concrete-partition-loop-state D (bmssp-level-cap p q)
          q p p cap l d-fp P Infinity d' D-part a betas bs Infinity
          Us U-loop c-loop child-costs-loop ∧
          complete-on d'
          {v ∈ bound-tree {s} Infinity. d-fp v = dist s v}
  shows ∃ c. exact-concrete-top-level-cost D N c
proof -
  have p-eq: p = Suc l
    using sssp-log-one-third-param-pos[of N]
    unfolding p-def l-def by simp
  have source-subset: {s} ⊆ V
    using source-in-V by simp
  obtain D-part where init:
    exact-partition-initial-state q d-fp P D-part (q * card P)
  using exact-partition-initial-state-exists-for-capped-pivots-with-cost
    [OF source-subset]
  unfolding d-fp-def P-def by blast
  obtain d' a betas bs Us U-loop c-loop child-costs-loop where

```

```

loop-run:
  exact-concrete-partition-loop-state  $D$  (bmssp-level-cap  $p$   $q$ )
     $q$   $p$   $p$  cap  $l$  d-fp  $P$  Infinity  $d'$   $D$ -part  $a$  betas  $bs$  Infinity
     $Us$   $U$ -loop  $c$ -loop child-costs-loop
and complete:
  complete-on  $d'$ 
     $\{v \in \text{bound-tree } \{s\} \text{ Infinity. } d\text{-fp } v = \text{dist } s \ v\}$ 
using loop[ $OF$  init] by blast
have init-actual:
  exact-partition-initial-state  $q$ 
    (find-pivots-label-capped  $p$  cap finite-initial-label  $\{s\}$  Infinity)
    (find-pivots-pivots-capped  $p$  cap finite-initial-label  $\{s\}$  Infinity)
     $D$ -part
    ( $q * \text{card}$ 
      (find-pivots-pivots-capped  $p$  cap finite-initial-label  $\{s\}$ 
        Infinity))
using init unfolding d-fp-def  $P$ -def .
have loop-actual:
  exact-concrete-partition-loop-state  $D$  (bmssp-level-cap  $p$   $q$ )
     $q$   $p$   $p$  cap  $l$ 
    (find-pivots-label-capped  $p$  cap finite-initial-label  $\{s\}$  Infinity)
    (find-pivots-pivots-capped  $p$  cap finite-initial-label  $\{s\}$  Infinity)
    Infinity  $d'$   $D$ -part  $a$  betas  $bs$  Infinity  $Us$   $U$ -loop  $c$ -loop
    child-costs-loop
using loop-run unfolding d-fp-def  $P$ -def .
have complete-actual:
  complete-on  $d'$ 
     $\{v \in \text{bound-tree } \{s\} \text{ Infinity.}$ 
      find-pivots-label-capped  $p$  cap finite-initial-label  $\{s\}$ 
      Infinity  $v = \text{dist } s \ v\}$ 
using complete unfolding d-fp-def .
let ? $U$  =
   $U$ -loop  $\cup$ 
   $\{v \in \text{bound-tree } \{s\} \text{ Infinity.}$ 
    find-pivots-label-capped  $p$  cap finite-initial-label  $\{s\}$ 
    Infinity  $v = \text{dist } s \ v\}$ 
have run-root-suc:
  exact-concrete-bmssp  $D$  (bmssp-level-cap  $p$   $q$ )  $q$   $p$   $p$  cap ( $Suc$   $l$ )
  finite-initial-label  $\{s\}$  Infinity  $d'$  Infinity ? $U$ 
  (fp-iter-capped-scan-cost  $p$  cap finite-initial-label  $\{s\}$   $\{s\}$  Infinity +
     $q * \text{card}$ 
    (find-pivots-pivots-capped  $p$  cap finite-initial-label  $\{s\}$ 
      Infinity) +  $c$ -loop)
by (rule exact-concrete-bmssp-step-with-exact-insert-cost
  [ $OF$  init-actual loop-actual complete-actual refl])
let ? $c$  =
  fp-iter-capped-scan-cost  $p$  cap finite-initial-label  $\{s\}$   $\{s\}$  Infinity +
   $q * \text{card}$ 
  (find-pivots-pivots-capped  $p$  cap finite-initial-label  $\{s\}$ 
    Infinity)

```

```

      Infinity) + c-loop
have run-root:
  exact-concrete-bmssp D (bmssp-level-cap p q) q p p cap p
  finite-initial-label {s} Infinity d' Infinity ?U ?c
  using run-root-suc p-eq by simp
have exact-concrete-top-level-run D N d' ?U ?c
  unfolding exact-concrete-top-level-run-def
  using run-root
  by (simp add: Let-def p-def q-def cap-def)
then have exact-concrete-top-level-cost D N ?c
  unfolding exact-concrete-top-level-cost-def by blast
then show ?thesis
  by blast
qed

lemma exact-concrete-top-level-cost-exists-from-root-run-below-threshold:
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
  and below-threshold:
    vertex-count <
    sssp-log-one-third-param N *
    bmssp-level-cap (sssp-log-one-third-param N)
    (sssp-log-two-thirds-param N) (sssp-log-one-third-param N)
  and run: exact-concrete-root-run D N d' B' U c
  shows  $\exists c. \text{exact-concrete-top-level-cost } D \ N \ c$ 
proof -
  let ?p = sssp-log-one-third-param N
  let ?q = sssp-log-two-thirds-param N
  let ?cap = bmssp-level-cap ?p ?q ?p
  have p-pos:  $0 < ?p$ 
  by simp
  have run-bmssp:
    exact-concrete-bmssp D (bmssp-level-cap ?p ?q) ?q ?p ?p ?cap ?p
    finite-initial-label {s} Infinity d' B' U c
    using run unfolding exact-concrete-root-run-def by (simp add: Let-def)
  have run-suc:
    exact-concrete-bmssp D (bmssp-level-cap ?p ?q) ?q ?p ?p ?cap
    (Suc (?p - 1)) finite-initial-label {s} Infinity d' B' U c
    using run-bmssp p-pos by simp
  have pre: bmssp-pre-full finite-initial-label {s} Infinity
    using all-reachable finite-initial-label-source-complete
    by (rule top-bmssp-pre-full)
  have sound: sound-label finite-initial-label
    using finite-initial-label-sound[OF all-reachable] .
  have source-reaches:
     $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$ 
    using all-reachable source-in-V by blast
  have success-or-threshold:  $B' = \text{Infinity} \vee ?p * ?cap \leq \text{card } U$ 
  by (rule exact-concrete-bmssp-Suc-success-or-threshold
    [OF run-suc sound pre source-reaches])

```

have *post*: *bmssp-post-full* *finite-initial-label* $\{s\}$ *Infinity* $d' B' U$
by (*rule exact-concrete-bmssp-correct*
 $[OF \text{ run-bmssp sound pre source-reaches}]$)
have *U-subset*: $U \subseteq V$
using *post unfolding* *bmssp-post-full-def* *bound-tree-def* **by** *blast*
have *card-U-le*: $\text{card } U \leq \text{vertex-count}$
by (*rule card-subset-vertex-count* $[OF \text{ U-subset}]$)
have *not-threshold*: $\neg ?p * ?cap \leq \text{card } U$
using *below-threshold* *card-U-le* **by** *linarith*
have *B'-eq*: $B' = \text{Infinity}$
using *success-or-threshold* *not-threshold* **by** *blast*
have *top-run*: *exact-concrete-top-level-run* $D N d' U c$
using *run* *B'-eq*
unfolding *exact-concrete-top-level-run-root-run-iff*
 $\text{exact-concrete-root-run-def}$
by *simp*
then have *exact-concrete-top-level-cost* $D N c$
unfolding *exact-concrete-top-level-cost-def* **by** *blast*
then show *?thesis*
by *blast*
qed

lemma *eventually-exact-concrete-top-level-cost-if-root-runs-exist-below-threshold*:

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s v$
and *below-threshold*:
 eventually
 $(\lambda N. \text{vertex-count} <$
 $\text{sssp-log-one-third-param } N *$
 $\text{bmssp-level-cap } (\text{sssp-log-one-third-param } N)$
 $(\text{sssp-log-two-thirds-param } N) (\text{sssp-log-one-third-param } N))$
 at-top
and *root-runs*:
 eventually
 $(\lambda N. \exists d' B' U c. \text{exact-concrete-root-run } D N d' B' U c)$
 at-top
shows $\text{eventually } (\lambda N. \exists c. \text{exact-concrete-top-level-cost } D N c) \text{ at-top}$
using *below-threshold* *root-runs*
proof *eventually-elim*
case (*elim* N)
then obtain $d' B' U c$ **where** *run*:
 $\text{exact-concrete-root-run } D N d' B' U c$
by *blast*
show *?case*
by (*rule exact-concrete-top-level-cost-exists-from-root-run-below-threshold*
 $[OF \text{ all-reachable elim}(1) \text{ run}]$)
qed

lemma *eventually-exact-concrete-top-level-cost-from-empty-top-pivots*:

$\text{eventually } (\lambda N. \exists c. \text{exact-concrete-top-level-cost } D N c) \text{ at-top}$

using *eventually-top-level-pivots-empty*
 [of *finite-initial-label Infinity*]
proof *eventually-elim*
case (*elim N*)
show ?*case*
by (*rule exact-concrete-top-level-cost-exists-if-top-pivots-empty*
 [*OF elim*])
qed

lemma *exact-concrete-top-level-time-witness*:
assumes *exact-concrete-top-level-cost D N c*
shows *exact-concrete-top-level-cost D N*
 (*exact-concrete-top-level-time D N*)
unfolding *exact-concrete-top-level-time-def*
by (*rule LeastI-ex*) (*use assms in blast*)

lemma *exact-concrete-top-level-time-le*:
assumes *exact-concrete-top-level-cost D N c*
shows *exact-concrete-top-level-time D N ≤ c*
unfolding *exact-concrete-top-level-time-def*
by (*rule Least-le*) (*rule assms*)

lemma *eventually-exact-concrete-top-level-cost-if-all*:
assumes $\bigwedge N. \exists c. \text{exact-concrete-top-level-cost } D \ N \ c$
shows *eventually* ($\lambda N. \exists c. \text{exact-concrete-top-level-cost } D \ N \ c$) *at-top*
by (*rule eventuallyI*) (*rule assms*)

lemma *eventually-exact-concrete-top-level-cost-square-if-all*:
assumes $\bigwedge N. \exists c. \text{exact-concrete-top-level-cost } D \ N \ c$
shows *eventually*
 ($\lambda N. \exists c. \text{exact-concrete-top-level-cost } D \ (N * N) \ c$) *at-top*
by (*rule eventuallyI*) (*rule assms*)

theorem *exact-concrete-top-level-run-correct-and-refined-bound-log-params-fixed-degree*:
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le D*
and *run*: *exact-concrete-top-level-run D N d' U c*
shows $U = V \wedge \text{sssp-correct } d' \wedge$
 $c \leq \text{bmssp-refined-graph-time-bound}$
 ($\lambda-. \text{Suc } D * \text{sssp-log-one-third-param } N$)
 ($\lambda-. \text{sssp-log-two-thirds-param } N$)
 ($\lambda-. \text{sssp-log-one-third-param } N$)
 ($\lambda-. \text{sssp-log-one-third-param } N$)
 ($\lambda-. \text{sssp-log-two-thirds-param } N$)
 ($\lambda-. \text{edge-count}$) *vertex-count*
proof –
let ?*p* = *sssp-log-one-third-param N*
let ?*q* = *sssp-log-two-thirds-param N*

have *run'*:
exact-concrete-bmssp *D* (*bmssp-level-cap* ?*p* ?*q*) ?*q* ?*p* ?*p*
(*bmssp-level-cap* ?*p* ?*q* ?*p*) ?*p*
finite-initial-label {*s*} *Infinity* *d'* *Infinity* *U* *c*
using *run* **unfolding** *exact-concrete-top-level-run-def* **by** (*simp* *add*: *Let-def*)
show ?*thesis*
by (rule *exact-concrete-algorithm-correct-and-refined-graph-bound-log-params-fixed-degree*
[*OF all-reachable degree run*])
qed

theorem *exact-concrete-top-level-cost-refined-bound-log-params-fixed-degree*:
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* *D*
and *cost*: *exact-concrete-top-level-cost* *D* *N* *c*
shows *c* $\leq$ *bmssp-refined-graph-time-bound*
(λ -. *Suc* *D* * *sssp-log-one-third-param* *N*)
(λ -. *sssp-log-two-thirds-param* *N*)
(λ -. *sssp-log-one-third-param* *N*)
(λ -. *sssp-log-one-third-param* *N*)
(λ -. *sssp-log-two-thirds-param* *N*)
(λ -. *edge-count*) *vertex-count*
proof –
obtain *d' U* **where** *run*: *exact-concrete-top-level-run* *D* *N* *d' U* *c*
using *cost* **unfolding** *exact-concrete-top-level-cost-def* **by** *blast*
show ?*thesis*
using *exact-concrete-top-level-run-correct-and-refined-bound-log-params-fixed-degree*
[*OF all-reachable degree run*]
by *blast*
qed

theorem *exact-concrete-top-level-time-refined-bound-log-params-fixed-degree*:
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* *D*
and *run-exists*: *exact-concrete-top-level-cost* *D* *N* *c*
shows *exact-concrete-top-level-time* *D* *N* $\leq$
bmssp-refined-graph-time-bound
(λ -. *Suc* *D* * *sssp-log-one-third-param* *N*)
(λ -. *sssp-log-two-thirds-param* *N*)
(λ -. *sssp-log-one-third-param* *N*)
(λ -. *sssp-log-one-third-param* *N*)
(λ -. *sssp-log-two-thirds-param* *N*)
(λ -. *edge-count*) *vertex-count*
proof –
have *cost*:
exact-concrete-top-level-cost *D* *N* (*exact-concrete-top-level-time* *D* *N*)
by (rule *exact-concrete-top-level-time-witness*[*OF run-exists*])
show ?*thesis*

by (rule exact-concrete-top-level-cost-refined-bound-log-params-fixed-degree
[OF all-reachable degree cost])

qed

From this point onward, the file is purely asymptotic. The function $T\text{-bmssp}$ abbreviates the least top-level concrete cost, and the following theorems compare it with increasingly convenient targets. The central comparison target is sssp-time-target , which expands to an edge-count term multiplied by the two-thirds logarithmic factor. The proofs reuse the complexity lemmas for $\text{bmssp-refined-graph-time-bound}$; no algorithmic invariant is reproved in this section.

definition $T\text{-bmssp} :: \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat}$ **where**

$T\text{-bmssp } D \ N = \text{exact-concrete-top-level-time } D \ N$

theorem $T\text{-bmssp-refined-bound-log-params-fixed-degree}$:

assumes all-reachable : $\bigwedge v. v \in V \implies \text{reachable } s \ v$

and degree : $\text{edge-outdegree-le } D$

and run-exists : $\text{exact-concrete-top-level-cost } D \ N \ c$

shows $T\text{-bmssp } D \ N \leq$

$\text{bmssp-refined-graph-time-bound}$

$(\lambda-. \text{Suc } D * \text{sssp-log-one-third-param } N)$

$(\lambda-. \text{sssp-log-two-thirds-param } N)$

$(\lambda-. \text{sssp-log-one-third-param } N)$

$(\lambda-. \text{sssp-log-one-third-param } N)$

$(\lambda-. \text{sssp-log-two-thirds-param } N)$

$(\lambda-. \text{edge-count}) \ \text{vertex-count}$

unfolding $T\text{-bmssp-def}$

by (rule exact-concrete-top-level-time-refined-bound-log-params-fixed-degree

[OF all-reachable degree run-exists])

theorem $T\text{-bmssp-bigo-sssp-time-target-log-params-fixed-degree-if-exact-runs-exist}$:

fixes $m :: \text{nat} \Rightarrow \text{nat}$

and $Cn \ Cm :: \text{real}$

assumes all-reachable : $\bigwedge v. v \in V \implies \text{reachable } s \ v$

and degree : $\text{edge-outdegree-le } D$

and $Cn\text{-pos}$: $0 < Cn$

and $Cm\text{-pos}$: $0 < Cm$

and $\text{vertex-count-dominated}$:

$\text{eventually } (\lambda n. \text{real vertex-count} \leq Cn * \text{real } (m \ n)) \ \text{at-top}$

and $\text{edge-count-dominated}$:

$\text{eventually } (\lambda n. \text{real edge-count} \leq Cm * \text{real } (m \ n)) \ \text{at-top}$

and exact-runs :

$\text{eventually } (\lambda n. \exists c. \text{exact-concrete-top-level-cost } D \ n \ c) \ \text{at-top}$

shows $(\lambda n. \text{real } (T\text{-bmssp } D \ n)) \in O(\lambda n. \text{sssp-time-target } m \ n)$

proof (rule $\text{bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-degree-slack}$

[where $D = D$ and $Cn = Cn$ and $Cm = Cm$

and $v = \lambda-. \text{vertex-count}$ and $m' = \lambda-. \text{edge-count}$])

show $0 < Cn$

```

    by (rule Cn-pos)
  show 0 < Cm
    by (rule Cm-pos)
  show eventually (λn. real ((λ-. vertex-count) n) ≤
    Cn * real (m n)) at-top
    using vertex-count-dominated by simp
  show eventually (λn. real ((λ-. edge-count) n) ≤
    Cm * real (m n)) at-top
    using edge-count-dominated by simp
  show eventually
    (λn. T-bmssp D n ≤
      bmssp-refined-graph-time-bound
        (λ-. Suc D * sssp-log-one-third-param n)
        (λ-. sssp-log-two-thirds-param n)
        (λ-. sssp-log-one-third-param n)
        (λ-. sssp-log-one-third-param n)
        (λ-. sssp-log-two-thirds-param n)
        (λ-. (λ-. edge-count) n) ((λ-. vertex-count) n)) at-top
    using exact-runs
  proof eventually-elim
    case (elim n)
    then obtain c where cost: exact-concrete-top-level-cost D n c
      by blast
    show ?case
      by (rule T-bmssp-refined-bound-log-params-fixed-degree
        [OF all-reachable degree cost])
  qed
qed

```

theorem *T-bmssp-bigo-sssp-time-target-square-size-params-fixed-degree-if-exact-runs-exist:*

```

  fixes m :: nat ⇒ nat
    and Cn Cm :: real
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
    and degree: edge-outdegree-le D
    and Cn-pos: 0 < Cn
    and Cm-pos: 0 < Cm
    and vertex-count-dominated:
      eventually (λn. real vertex-count ≤ Cn * real (m n)) at-top
    and edge-count-dominated:
      eventually (λn. real edge-count ≤ Cm * real (m n)) at-top
    and exact-runs:
      eventually (λn.  $\exists c. \text{exact-concrete-top-level-cost } D \ (n * n) \ c$ ) at-top
  shows (λn. real (T-bmssp D (n * n))) ∈
    O(λn. sssp-time-target m n)
  proof (rule bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-
    degree-square-arg-slack
    [where D = D and Cn = Cn and Cm = Cm
      and v = λ-. vertex-count and m' = λ-. edge-count])

```



```

show 0 < Cn
  by (rule Cn-pos)
show 0 < Cm
  by (rule Cm-pos)
show eventually (λn. real ((λ-. vertex-count) n) ≤
  Cn * real (m n)) at-top
  using vertex-count-dominated by simp
show eventually (λn. real ((λ-. edge-count) n) ≤
  Cm * real (m n)) at-top
  using edge-count-dominated by simp
show eventually
  (λn. T-bmssp D (n * n) ≤
    bmssp-refined-graph-time-bound
      (λ-. Suc D * sssp-log-one-third-param (n * n))
      (λ-. sssp-log-two-thirds-param (n * n))
      (λ-. sssp-log-one-third-param (n * n))
      (λ-. sssp-log-one-third-param (n * n))
      (λ-. sssp-log-two-thirds-param (n * n))
      (λ-. (λ-. edge-count) n) ((λ-. vertex-count) n)) at-top
  using exact-runs
proof eventually-elim
  case (elim n)
  then obtain c where cost: exact-concrete-top-level-cost D (n * n) c
  by blast
  show ?case
  by (rule T-bmssp-refined-bound-log-params-fixed-degree
    [OF all-reachable degree cost])
qed
qed

theorem T-bmssp-bigo-edge-count-target-log-params-fixed-degree-if-exact-runs-exist:
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
  and degree: edge-outdegree-le D
  and edge-count-pos: 0 < edge-count
  and exact-runs:
    eventually (λn.  $\exists c. \text{exact-concrete-top-level-cost } D \ n \ c$ ) at-top
  shows (λn. real (T-bmssp D n)) ∈
    O(λn. sssp-time-target (λ-. edge-count) n)
proof (rule T-bmssp-bigo-sssp-time-target-log-params-fixed-degree-if-exact-runs-
  exist
    [OF all-reachable degree, where Cn = 2 and Cm = 1])
  show 0 < (2::real)
  by simp
  show 0 < (1::real)
  by simp
  have vertex-le: vertex-count ≤ 2 * edge-count
  by (rule vertex-count-le-twice-edge-count-if-all-reachable
    [OF all-reachable edge-count-pos])
  show eventually

```

$(\lambda n. \text{real vertex-count} \leq 2 * \text{real} ((\lambda -. \text{edge-count}) n)) \text{ at-top}$
using *vertex-le* **by** *simp*
show *eventually*
 $(\lambda n. \text{real edge-count} \leq 1 * \text{real} ((\lambda -. \text{edge-count}) n)) \text{ at-top}$
by *simp*
show *eventually* $(\lambda n. \exists c. \text{exact-concrete-top-level-cost } D \ n \ c) \text{ at-top}$
by (rule *exact-runs*)
qed

theorem *T-bmssp-bigo-edge-count-target-log-params-fixed-degree:*
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* *D*
and *edge-count-pos*: $0 < \text{edge-count}$
shows $(\lambda n. \text{real} (T\text{-bmssp } D \ n)) \in$
 $O(\lambda n. \text{sssp-time-target } (\lambda -. \text{edge-count}) \ n)$
by (rule *T-bmssp-bigo-edge-count-target-log-params-fixed-degree-if-exact-runs-exist*
 $[OF \text{ all-reachable } \text{degree } \text{edge-count-pos}$
 $\text{eventually-exact-concrete-top-level-cost-from-empty-top-pivots}]$)

theorem *T-bmssp-bigo-size-target-log-params-fixed-degree:*
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* *D*
shows $(\lambda n. \text{real} (T\text{-bmssp } D \ n)) \in$
 $O(\lambda n. \text{sssp-time-target } (\lambda -. \text{Suc edge-count}) \ n)$
proof (rule *T-bmssp-bigo-sssp-time-target-log-params-fixed-degree-if-exact-runs-exist*
 $[OF \text{ all-reachable } \text{degree, where } Cn = 1 \text{ and } Cm = 1])$
show $0 < (1::\text{real})$
by *simp*
show $0 < (1::\text{real})$
by *simp*
have *vertex-le*: *vertex-count* $\leq \text{Suc edge-count}$
by (rule *vertex-count-le-Suc-edge-count-if-all-reachable* $[OF \text{ all-reachable}]$)
show *eventually*
 $(\lambda n. \text{real vertex-count} \leq 1 * \text{real} ((\lambda -. \text{Suc edge-count}) n)) \text{ at-top}$
using *vertex-le* **by** *simp*
show *eventually*
 $(\lambda n. \text{real edge-count} \leq 1 * \text{real} ((\lambda -. \text{Suc edge-count}) n)) \text{ at-top}$
by *simp*
show *eventually* $(\lambda n. \exists c. \text{exact-concrete-top-level-cost } D \ n \ c) \text{ at-top}$
by (rule *eventually-exact-concrete-top-level-cost-from-empty-top-pivots*)
qed

theorem *T-bmssp-bigo-target-log-params-fixed-degree-if-exact-runs-exist:*
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* *D*
and *edge-count-pos*: $0 < \text{edge-count}$
and *exact-runs*:
 $\text{eventually } (\lambda n. \exists c. \text{exact-concrete-top-level-cost } D \ n \ c) \text{ at-top}$

shows $(\lambda n. \text{real } (T\text{-bmssp } D \ n)) \in$
 $O(\lambda n. \text{real edge-count} * (\ln (\text{real } n + 2)) \text{ powr } (2 / 3))$
using *T-bmssp-bigo-edge-count-target-log-params-fixed-degree-if-exact-runs-exist*
 $[OF \text{ all-reachable degree edge-count-pos exact-runs}]$
unfolding *sssp-time-target-def sssp-log-factor-def* **by** *simp*

theorem *T-bmssp-bigo-target-log-params-fixed-degree-if-root-runs-exist-below-threshold:*
assumes *all-reachable: $\bigwedge v. v \in V \implies \text{reachable } s \ v$*
and *degree: edge-outdegree-le D*
and *edge-count-pos: $0 < \text{edge-count}$*
and *below-threshold:*
eventually
 $(\lambda N. \text{vertex-count} <$
 $\text{sssp-log-one-third-param } N *$
 $\text{bmssp-level-cap } (\text{sssp-log-one-third-param } N)$
 $(\text{sssp-log-two-thirds-param } N) (\text{sssp-log-one-third-param } N))$
at-top
and *root-runs:*
eventually
 $(\lambda N. \exists d' B' U c. \text{exact-concrete-root-run } D \ N \ d' \ B' \ U \ c)$
at-top
shows $(\lambda n. \text{real } (T\text{-bmssp } D \ n)) \in$
 $O(\lambda n. \text{real edge-count} * (\ln (\text{real } n + 2)) \text{ powr } (2 / 3))$
proof (rule *T-bmssp-bigo-target-log-params-fixed-degree-if-exact-runs-exist*
 $[OF \text{ all-reachable degree edge-count-pos}]$)
show *eventually* $(\lambda N. \exists c. \text{exact-concrete-top-level-cost } D \ N \ c)$ *at-top*
by (rule *eventually-exact-concrete-top-level-cost-if-root-runs-exist-below-threshold*
 $[OF \text{ all-reachable below-threshold root-runs}]$)
qed

theorem *T-bmssp-bigo-target-log-params-fixed-degree-if-root-runs-exist:*
assumes *all-reachable: $\bigwedge v. v \in V \implies \text{reachable } s \ v$*
and *degree: edge-outdegree-le D*
and *edge-count-pos: $0 < \text{edge-count}$*
and *root-runs:*
eventually
 $(\lambda N. \exists d' B' U c. \text{exact-concrete-root-run } D \ N \ d' \ B' \ U \ c)$
at-top
shows $(\lambda n. \text{real } (T\text{-bmssp } D \ n)) \in$
 $O(\lambda n. \text{real edge-count} * (\ln (\text{real } n + 2)) \text{ powr } (2 / 3))$
proof (rule *T-bmssp-bigo-target-log-params-fixed-degree-if-root-runs-exist-below-threshold*
 $[OF \text{ all-reachable degree edge-count-pos - root-runs}]$)
show *eventually*
 $(\lambda N. \text{vertex-count} <$
 $\text{sssp-log-one-third-param } N *$
 $\text{bmssp-level-cap } (\text{sssp-log-one-third-param } N)$
 $(\text{sssp-log-two-thirds-param } N) (\text{sssp-log-one-third-param } N))$

at-top
by (rule *eventually-top-level-threshold-exceeds-vertex-count*)
qed

theorem *T-bmssp-bigo-target-log-params-fixed-degree:*
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* *D*
and *edge-count-pos*: $0 < \text{edge-count}$
shows $(\lambda n. \text{real } (T\text{-bmssp } D \ n)) \in$
 $O(\lambda n. \text{real } \text{edge-count} * (\ln (\text{real } n + 2))) \text{ powr } (2 / 3))$
using *T-bmssp-bigo-edge-count-target-log-params-fixed-degree*
 $[OF \text{ all-reachable } \text{degree } \text{edge-count-pos}]$
unfolding *sssp-time-target-def sssp-log-factor-def* **by** *simp*

theorem *bmssp-runtime-bigo-target:*
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* *D*
and *edge-count-pos*: $0 < \text{edge-count}$
shows $(\lambda n. \text{real } (T\text{-bmssp } D \ n)) \in$
 $O(\lambda n. \text{real } \text{edge-count} * (\ln (\text{real } n + 2))) \text{ powr } (2 / 3))$
by (rule *T-bmssp-bigo-target-log-params-fixed-degree*
 $[OF \text{ all-reachable } \text{degree } \text{edge-count-pos}]$)

theorem *T-bmssp-bigo-target-log-params-fixed-degree-if-total:*
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* *D*
and *edge-count-pos*: $0 < \text{edge-count}$
and *total*: $\bigwedge N. \exists c. \text{exact-concrete-top-level-cost } D \ N \ c$
shows $(\lambda n. \text{real } (T\text{-bmssp } D \ n)) \in$
 $O(\lambda n. \text{real } \text{edge-count} * (\ln (\text{real } n + 2))) \text{ powr } (2 / 3))$
by (rule *T-bmssp-bigo-target-log-params-fixed-degree-if-exact-runs-exist*
 $[OF \text{ all-reachable } \text{degree } \text{edge-count-pos}$
 $\text{eventually-exact-concrete-top-level-cost-if-all}[OF \text{ total}]]$)

theorem *T-bmssp-bigo-edge-count-target-square-size-params-fixed-degree-if-exact-runs-exist:*
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* *D*
and *edge-count-pos*: $0 < \text{edge-count}$
and *exact-runs*:
 $\text{eventually } (\lambda n. \exists c. \text{exact-concrete-top-level-cost } D \ (n * n) \ c) \text{ at-top}$
shows $(\lambda n. \text{real } (T\text{-bmssp } D \ (n * n))) \in$
 $O(\lambda n. \text{sssp-time-target } (\lambda -. \text{edge-count}) \ n)$
proof (rule *T-bmssp-bigo-sssp-time-target-square-size-params-fixed-degree-if-exact-runs-exist*
 $[OF \text{ all-reachable } \text{degree, where } Cn = 2 \text{ and } Cm = 1])$
show $0 < (2::\text{real})$
by *simp*
show $0 < (1::\text{real})$

by *simp*
 have *vertex-le*: $\text{vertex-count} \leq 2 * \text{edge-count}$
 by (rule *vertex-count-le-twice-edge-count-if-all-reachable*
 [OF *all-reachable edge-count-pos*])
 show *eventually*
 ($\lambda n. \text{real vertex-count} \leq 2 * \text{real } ((\lambda-. \text{edge-count}) n)$) *at-top*
 using *vertex-le* by *simp*
 show *eventually*
 ($\lambda n. \text{real edge-count} \leq 1 * \text{real } ((\lambda-. \text{edge-count}) n)$) *at-top*
 by *simp*
 show *eventually*
 ($\lambda n. \exists c. \text{exact-concrete-top-level-cost } D (n * n) c$) *at-top*
 by (rule *exact-runs*)
 qed

theorem *T-bmssp-bigo-target-square-size-params-fixed-degree-if-exact-runs-exist*:
 assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s v$
 and *degree*: *edge-outdegree-le* *D*
 and *edge-count-pos*: $0 < \text{edge-count}$
 and *exact-runs*:
eventually ($\lambda n. \exists c. \text{exact-concrete-top-level-cost } D (n * n) c$) *at-top*
 shows ($\lambda n. \text{real } (T\text{-bmssp } D (n * n))$) $\in$
 $O(\lambda n. \text{real edge-count} * (\ln (\text{real } n + 2))) \text{ powr } (2 / 3)$
 using *T-bmssp-bigo-edge-count-target-square-size-params-fixed-degree-if-exact-runs-exist*
 [OF *all-reachable degree edge-count-pos exact-runs*]
 unfolding *sssp-time-target-def sssp-log-factor-def* by *simp*

theorem *T-bmssp-bigo-target-square-size-params-fixed-degree-if-total*:
 assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s v$
 and *degree*: *edge-outdegree-le* *D*
 and *edge-count-pos*: $0 < \text{edge-count}$
 and *total*: $\bigwedge N. \exists c. \text{exact-concrete-top-level-cost } D N c$
 shows ($\lambda n. \text{real } (T\text{-bmssp } D (n * n))$) $\in$
 $O(\lambda n. \text{real edge-count} * (\ln (\text{real } n + 2))) \text{ powr } (2 / 3)$
 by (rule *T-bmssp-bigo-target-square-size-params-fixed-degree-if-exact-runs-exist*
 [OF *all-reachable degree edge-count-pos*
eventually-exact-concrete-top-level-cost-square-if-all[OF *total*]])

theorem *range-split-algorithm-correct-and-closed-graph-bound*:
 assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s v$
 and *degree*: *edge-outdegree-le* Δ
 and *degree-factor*: $\Delta \leq A$
 and *R-pos*: $0 < R$
 and *insert-factor*: $t \leq A * k$
 and *insert-scaled-factor*: $t \leq A\text{-insert} * k$
 and *seen-scaled-factor*: $k * \Delta + A\text{-insert} \leq 2 * A$
 and *source-factor*: $\text{Suc } h \leq 2 * A$
 and *k-pos*: $0 < k$

and run:
exact-split-range-costed-bmssp Δ (*bmssp-level-cap* k t) t h k
(bmssp-level-cap k t l) l
finite-initial-label $\{s\}$ *Infinity* d' *Infinity* U c
shows $U = V \wedge \text{sssp-correct } d' \wedge$
 $c \leq \text{bmssp-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. l) (\lambda-. t)$
 $(\lambda-. \text{edge-count}) \text{ vertex-count}$
proof –
have *UV-correct*: $U = V \wedge \text{sssp-correct } d'$
by (*rule range-split-algorithm-correct*[*OF all-reachable run*])
have *cost*:
 $\text{sssp-correct } d' \wedge$
 $c \leq \text{bmssp-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. l) (\lambda-. t)$
 $(\lambda-. \text{edge-count}) \text{ vertex-count}$
by (*rule finite-initial-label-exact-split-range-costed-top-level-correct-and-closed-*
bmssp-graph-time-bound-level-cap
 $[OF \text{ all-reachable degree degree-factor } R\text{-pos insert-factor insert-scaled-factor}$
 $\text{seen-scaled-factor source-factor } k\text{-pos run}]$)
show *?thesis*
using *UV-correct cost by blast*
qed

theorem *range-split-algorithm-correct-and-refined-graph-bound-under-edge-budget:*

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: *edge-outdegree-le* Δ
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$
and *insert-scaled-factor*: $t \leq A\text{-insert} * k$
and *seen-scaled-factor*: $k * \Delta + A\text{-insert} \leq 2 * A$
and *source-factor*: $\text{Suc } h \leq 2 * A$
and *k-pos*: $0 < k$
and *edge-budget*:
 $\bigwedge l' d P B d' D a \text{ betas } bs B' Us \text{ U-loop c-loop child-costs.}$
exact-split-range-costed-partition-loop-state Δ (*bmssp-level-cap* k t) t h k
(bmssp-level-cap k t l) l' d P B d' D a *betas* bs B'
 $Us \text{ U-loop c-loop child-costs} \implies$
 $\text{sound-label } d \implies$
 $\text{bmssp-pre-full } d \ P \ B \implies$
 $(\bigwedge x. x \in P \implies \text{reachable } s \ x) \implies$
 $t * \text{sum-list } (\text{map card } (\text{range-tree-child-edge-range-list } P \ a \ \text{betas } bs))$
 $\leq H * \text{card } (\text{outgoing-edges } U\text{-loop})$
and run:
exact-split-range-costed-bmssp Δ (*bmssp-level-cap* k t) t h k
(bmssp-level-cap k t l) l
finite-initial-label $\{s\}$ *Infinity* d' *Infinity* U c
shows $U = V \wedge \text{sssp-correct } d' \wedge$
 $c \leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. H)$
 $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{ vertex-count}$

proof –
have *UV-correct*: $U = V \wedge \text{sssp-correct } d'$
by (rule *range-split-algorithm-correct*[*OF all-reachable run*])
have *cost*:
 $\text{sssp-correct } d' \wedge$
 $c \leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. H)$
 $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{ vertex-count}$
by (rule *finite-initial-label-exact-split-range-costed-top-level-correct-and-closed-bmssp-refined-graph-time-bound-level-cap*
[*OF all-reachable degree degree-factor R-pos insert-factor insert-scaled-factor*
seen-scaled-factor source-factor k-pos edge-budget run])
show ?thesis
using *UV-correct cost* **by** *blast*
qed

theorem *range-split-algorithm-correct-and-refined-graph-bound-trivial-edge-budget*:
assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$
and *degree*: $\text{edge-outdegree-le } \Delta$
and *degree-factor*: $\Delta \leq A$
and *R-pos*: $0 < R$
and *insert-factor*: $t \leq A * k$
and *insert-scaled-factor*: $t \leq A\text{-insert} * k$
and *seen-scaled-factor*: $k * \Delta + A\text{-insert} \leq 2 * A$
and *source-factor*: $\text{Suc } h \leq 2 * A$
and *k-pos*: $0 < k$
and *run*:
 $\text{exact-split-range-costed-bmssp } \Delta (\text{bmssp-level-cap } k \ t) \ t \ h \ k$
 $(\text{bmssp-level-cap } k \ t \ l) \ l$
 $\text{finite-initial-label } \{s\} \ \text{Infinity } d' \ \text{Infinity } U \ c$
shows $U = V \wedge \text{sssp-correct } d' \wedge$
 $c \leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. t)$
 $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{ vertex-count}$

proof –
have *UV-correct*: $U = V \wedge \text{sssp-correct } d'$
by (rule *range-split-algorithm-correct*[*OF all-reachable run*])
have *cost*:
 $\text{sssp-correct } d' \wedge$
 $c \leq \text{bmssp-refined-graph-time-bound } (\lambda-. A) (\lambda-. R) (\lambda-. t)$
 $(\lambda-. l) (\lambda-. t) (\lambda-. \text{edge-count}) \text{ vertex-count}$
by (rule *finite-initial-label-exact-split-range-costed-top-level-correct-and-closed-bmssp-refined-graph-time-bound-level-cap-trivial-edge-budget*
[*OF all-reachable degree degree-factor R-pos insert-factor insert-scaled-factor*
seen-scaled-factor source-factor k-pos run])
show ?thesis
using *UV-correct cost* **by** *blast*
qed

end

context *positive-unique-shortest-digraph*
begin

theorem *positive-weight-direct-insert-algorithm-correct-and-refined-graph-bound-log-params-fixed-degree*:

fixes $N D p q :: \text{nat}$
defines $p\text{-def}: p \equiv \text{sssp-log-one-third-param } N$
and $q\text{-def}: q \equiv \text{sssp-log-two-thirds-param } N$
assumes $\text{all-reachable}: \bigwedge v. v \in V \implies \text{reachable } s \ v$
and $\text{degree}: \text{edge-outdegree-le } D$
and run :
 $\text{direct-insert-costed-bmssp } D \ (\text{bmssp-level-cap } p \ q) \ q \ p \ p$
 $(\text{bmssp-level-cap } p \ q \ p) \ p$
 $\text{finite-initial-label } \{s\} \ \text{Infinity } d' \ \text{Infinity } U \ c$
shows $U = V \wedge \text{sssp-correct } d' \wedge$
 $c \leq \text{bmssp-refined-graph-time-bound}$
 $(\lambda-. \text{Suc } D * p) \ (\lambda-. q) \ (\lambda-. p)$
 $(\lambda-. p) \ (\lambda-. q) \ (\lambda-. \text{edge-count}) \ \text{vertex-count}$
proof –
have run' :
 $\text{direct-insert-costed-bmssp } D$
 $(\text{bmssp-level-cap } (\text{sssp-log-one-third-param } N) \ (\text{sssp-log-two-thirds-param } N))$
 $(\text{sssp-log-two-thirds-param } N) \ (\text{sssp-log-one-third-param } N)$
 $(\text{sssp-log-one-third-param } N)$
 $(\text{bmssp-level-cap } (\text{sssp-log-one-third-param } N) \ (\text{sssp-log-two-thirds-param } N))$
 $(\text{sssp-log-one-third-param } N))$
 $(\text{sssp-log-one-third-param } N)$
 $\text{finite-initial-label } \{s\} \ \text{Infinity } d' \ \text{Infinity } U \ c$
using run **unfolding** $p\text{-def } q\text{-def}$ **by** simp
show $?thesis$
unfolding $p\text{-def } q\text{-def}$
by $(\text{rule } \text{direct-insert-algorithm-correct-and-refined-graph-bound-log-params-fixed-degree})$
 $[\text{OF all-reachable degree run}']$
qed

theorem *positive-weight-exact-concrete-algorithm-correct-and-refined-graph-bound-log-params-fixed-degree*:

fixes $N D p q :: \text{nat}$
defines $p\text{-def}: p \equiv \text{sssp-log-one-third-param } N$
and $q\text{-def}: q \equiv \text{sssp-log-two-thirds-param } N$
assumes $\text{all-reachable}: \bigwedge v. v \in V \implies \text{reachable } s \ v$
and $\text{degree}: \text{edge-outdegree-le } D$
and run :
 $\text{exact-concrete-bmssp } D \ (\text{bmssp-level-cap } p \ q) \ q \ p \ p$
 $(\text{bmssp-level-cap } p \ q \ p) \ p$
 $\text{finite-initial-label } \{s\} \ \text{Infinity } d' \ \text{Infinity } U \ c$
shows $U = V \wedge \text{sssp-correct } d' \wedge$
 $c \leq \text{bmssp-refined-graph-time-bound}$


```

      (λ-. Suc D * p) (λ-. q) (λ-. p)
      (λ-. p) (λ-. q) (λ-. edge-count) vertex-count
proof –
  have direct-run:
    direct-insert-costed-bmssp D (bmssp-level-cap p q) q p p
    (bmssp-level-cap p q p) p
    finite-initial-label {s} Infinity d' Infinity U c
  by (rule exact-concrete-bmssp-refines-direct-insert[OF run])
  have direct-run':
    direct-insert-costed-bmssp D
    (bmssp-level-cap (sssp-log-one-third-param N) (sssp-log-two-thirds-param N))
    (sssp-log-two-thirds-param N) (sssp-log-one-third-param N)
    (sssp-log-one-third-param N)
    (bmssp-level-cap (sssp-log-one-third-param N) (sssp-log-two-thirds-param N)
      (sssp-log-one-third-param N))
    (sssp-log-one-third-param N)
    finite-initial-label {s} Infinity d' Infinity U c
  using direct-run unfolding p-def q-def by simp
  show ?thesis
  unfolding p-def q-def
  by (rule positive-weight-direct-insert-algorithm-correct-and-refined-graph-bound-
log-params-fixed-degree
    [OF all-reachable degree direct-run'])
qed

end

locale bounded-reduced-positive-instance = positive-unique-shortest-digraph +
  fixes D :: nat
  assumes all-vertices-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
  and bounded-edge-outdegree: edge-outdegree-le D
begin

theorem bounded-reduced-direct-insert-algorithm-correct-and-refined-graph-bound-
log-params:
  fixes N p q :: nat
  defines p-def:  $p \equiv \text{sssp-log-one-third-param } N$ 
  and q-def:  $q \equiv \text{sssp-log-two-thirds-param } N$ 
  assumes run:
    direct-insert-costed-bmssp D (bmssp-level-cap p q) q p p
    (bmssp-level-cap p q p) p
    finite-initial-label {s} Infinity d' Infinity U c
  shows  $U = V \wedge \text{sssp-correct } d' \wedge$ 
     $c \leq \text{bmssp-refined-graph-time-bound}$ 
    (λ-. Suc D * p) (λ-. q) (λ-. p)
    (λ-. p) (λ-. q) (λ-. edge-count) vertex-count
proof –
  have run':
    direct-insert-costed-bmssp D

```

```

    (bmssp-level-cap (sssp-log-one-third-param N) (sssp-log-two-thirds-param N))
    (sssp-log-two-thirds-param N) (sssp-log-one-third-param N)
    (sssp-log-one-third-param N)
    (bmssp-level-cap (sssp-log-one-third-param N) (sssp-log-two-thirds-param N)
      (sssp-log-one-third-param N))
    (sssp-log-one-third-param N)
    finite-initial-label {s} Infinity d' Infinity U c
  using run unfolding p-def q-def by simp
show ?thesis
  unfolding p-def q-def
  by (rule positive-weight-direct-insert-algorithm-correct-and-refined-graph-bound-
log-params-fixed-degree
    [OF all-vertices-reachable bounded-edge-outdegree run])
qed

```

theorem *bounded-reduced-exact-concrete-algorithm-correct-and-refined-graph-bound-log-params:*

```

  fixes N p q :: nat
  defines p-def: p ≡ sssp-log-one-third-param N
    and q-def: q ≡ sssp-log-two-thirds-param N
  assumes run:
    exact-concrete-bmssp D (bmssp-level-cap p q) q p p
    (bmssp-level-cap p q p) p
    finite-initial-label {s} Infinity d' Infinity U c
  shows U = V ∧ sssp-correct d' ∧
    c ≤ bmssp-refined-graph-time-bound
    (λ-. Suc D * p) (λ-. q) (λ-. p)
    (λ-. p) (λ-. q) (λ-. edge-count) vertex-count
proof –
  have direct-run:
    direct-insert-costed-bmssp D (bmssp-level-cap p q) q p p
    (bmssp-level-cap p q p) p
    finite-initial-label {s} Infinity d' Infinity U c
  by (rule exact-concrete-bmssp-refines-direct-insert[OF run])
  have direct-run':
    direct-insert-costed-bmssp D
    (bmssp-level-cap (sssp-log-one-third-param N) (sssp-log-two-thirds-param N))
    (sssp-log-two-thirds-param N) (sssp-log-one-third-param N)
    (sssp-log-one-third-param N)
    (bmssp-level-cap (sssp-log-one-third-param N) (sssp-log-two-thirds-param N)
      (sssp-log-one-third-param N))
    (sssp-log-one-third-param N)
    finite-initial-label {s} Infinity d' Infinity U c
  using direct-run unfolding p-def q-def by simp
show ?thesis
  unfolding p-def q-def
  by (rule bounded-reduced-direct-insert-algorithm-correct-and-refined-graph-bound-
log-params
    [OF direct-run])

```

qed

theorem *bounded-reduced-bmssp-runtime-bigo-target:*

assumes *edge-count-pos:* $0 < \text{edge-count}$

shows $(\lambda n. \text{real } (T\text{-bmssp } D \ n)) \in$

$O(\lambda n. \text{real } \text{edge-count} * (\ln (\text{real } n + 2)) \text{ powr } (2 / 3))$

by (*rule* *T-bmssp-bigo-target-log-params-fixed-degree*

[*OF all-vertices-reachable bounded-edge-outdegree edge-count-pos*])

end

locale *bmssp-runtime-headline-instance* =

bounded: *bounded-reduced-positive-instance* *V E w s D*

for *V* :: *'v* *set*

and *E* :: (*'v* × *'v*) *set*

and *w* :: *'v* $\Rightarrow$ *'v* $\Rightarrow$ *real*

and *s* :: *'v*

and *D* :: *nat*

begin

This locale is the public asymptotic headline for the top-level theory. It hides the reduced positive instance, the fixed outdegree bound, and the least-cost definition behind two simple functions, *T-bmssp* for the checked BMSSP running time and *m* for the edge-count scale. The theorem below is deliberately assumption-free inside the locale: all graph hypotheses are already part of the locale interpretation.

definition *T-bmssp* :: *nat* $\Rightarrow$ *nat* **where**

T-bmssp *N* = *bounded.T-bmssp* *D* *N*

definition *m* :: *nat* $\Rightarrow$ *nat* **where**

m *N* = *Suc* *bounded.edge-count*

theorem *bmssp-runtime-bigo-target:*

shows $(\lambda n. \text{real } (T\text{-bmssp } n)) \in$

$O(\lambda n. \text{real } (m \ n) * (\ln (\text{real } n + 2)) \text{ powr } (2 / 3))$

using *bounded.T-bmssp-bigo-size-target-log-params-fixed-degree*

[*OF* *bounded.all-vertices-reachable* *bounded.bounded-edge-outdegree*]

unfolding *T-bmssp-def* *m-def* *sssp-time-target-def* *sssp-log-factor-def* **by** *simp*

The theorem $(\lambda n. \text{real } (T\text{-bmssp } n)) \in O(\lambda n. \text{real } (m \ n) * \ln (\text{real } n + 2) \text{ powr } (2 / 3))$ is the headline statement used by the rest of the AFP entry. Its right-hand side is the deterministic BMSSP target: graph-size work times $\log^{\{2/3\}}$. The proof is short here because the recurrence has already been solved in the preceding fixed-degree theorem and the locale definitions only rename the time and size functions.

end

sublocale *bounded-reduced-positive-instance* <

```

runtime-headline: bmssp-runtime-headline-instance V E w s D
by unfold-locales

context bounded-reduced-positive-instance
begin

theorem bounded-reduced-runtime-headline-bmssp-runtime-bigo-target:
  shows ( $\lambda n.$  real (runtime-headline.T-bmssp n))  $\in$ 
     $O(\lambda n.$  real (runtime-headline.m n) * (ln (real n + 2)) powr (2 / 3))
  by (rule runtime-headline.bmssp-runtime-bigo-target)

end

locale reduction-correctness-contract =
  original: finite-weighted-digraph V E w s +
  reduced: bounded-reduced-positive-instance V' E' w' s' D
  for V :: 'v set
    and E :: ('v  $\times$  'v) set
    and w :: 'v  $\Rightarrow$  'v  $\Rightarrow$  real
    and s :: 'v
    and V' :: 'r set
    and E' :: ('r  $\times$  'r) set
    and w' :: 'r  $\Rightarrow$  'r  $\Rightarrow$  real
    and s' :: 'r
    and D :: nat +
  fixes decode-label :: ('r  $\Rightarrow$  real)  $\Rightarrow$  'v  $\Rightarrow$  real
  assumes transfer-sssp-correct:
    reduced.sssp-correct d'  $\Longrightarrow$  original.sssp-correct (decode-label d')
  begin

theorem reduced-direct-insert-run-transfers-correctness-and-refined-bound:
  fixes N p q :: nat
  defines p-def: p  $\equiv$  sssp-log-one-third-param N
    and q-def: q  $\equiv$  sssp-log-two-thirds-param N
  assumes run:
    reduced.direct-insert-costed-bmssp D (bmssp-level-cap p q) q p p
    (bmssp-level-cap p q p) p
    reduced.finite-initial-label {s'} Infinity d' Infinity U c
  shows original.sssp-correct (decode-label d')  $\wedge$ 
    c  $\leq$  bmssp-refined-graph-time-bound
    ( $\lambda.$  Suc D * p) ( $\lambda.$  q) ( $\lambda.$  p)
    ( $\lambda.$  p) ( $\lambda.$  q) ( $\lambda.$  reduced.edge-count)
    reduced.vertex-count
  proof -
  have run':
    reduced.direct-insert-costed-bmssp D
    (bmssp-level-cap (sssp-log-one-third-param N) (sssp-log-two-thirds-param N))
    (sssp-log-two-thirds-param N) (sssp-log-one-third-param N)
    (sssp-log-one-third-param N)

```

```

    (bmssp-level-cap (sssp-log-one-third-param N) (sssp-log-two-thirds-param N)
      (sssp-log-one-third-param N))
    (sssp-log-one-third-param N)
    reduced.finite-initial-label {s'} Infinity d' Infinity U c
  using run unfolding p-def q-def by simp
  have reduced-result:
    U = V' ∧ reduced.sssp-correct d' ∧
    c ≤ bmssp-refined-graph-time-bound
      (λ-. Suc D * sssp-log-one-third-param N)
      (λ-. sssp-log-two-thirds-param N)
      (λ-. sssp-log-one-third-param N)
      (λ-. sssp-log-one-third-param N)
      (λ-. sssp-log-two-thirds-param N)
      (λ-. reduced.edge-count) reduced.vertex-count
  by (rule reduced.bounded-reduced-direct-insert-algorithm-correct-and-refined-
    graph-bound-log-params
      [OF run])
  then show ?thesis
    unfolding p-def q-def
    using transfer-sssp-correct by blast
qed

```

theorem *reduced-exact-concrete-run-transfers-correctness-and-refined-bound:*

```

  fixes N p q :: nat
  defines p-def: p ≡ sssp-log-one-third-param N
    and q-def: q ≡ sssp-log-two-thirds-param N
  assumes run:
    reduced.exact-concrete-bmssp D (bmssp-level-cap p q) q p p
      (bmssp-level-cap p q p) p
    reduced.finite-initial-label {s'} Infinity d' Infinity U c
  shows original.sssp-correct (decode-label d') ∧
    c ≤ bmssp-refined-graph-time-bound
      (λ-. Suc D * p) (λ-. q) (λ-. p)
      (λ-. p) (λ-. q) (λ-. reduced.edge-count)
      reduced.vertex-count

```

proof –

```

  have direct-run:
    reduced.direct-insert-costed-bmssp D (bmssp-level-cap p q) q p p
      (bmssp-level-cap p q p) p
    reduced.finite-initial-label {s'} Infinity d' Infinity U c
  by (rule reduced.exact-concrete-bmssp-refines-direct-insert[OF run])
  have direct-run':
    reduced.direct-insert-costed-bmssp D
      (bmssp-level-cap (sssp-log-one-third-param N) (sssp-log-two-thirds-param N))
      (sssp-log-two-thirds-param N) (sssp-log-one-third-param N)
      (sssp-log-one-third-param N)
      (bmssp-level-cap (sssp-log-one-third-param N) (sssp-log-two-thirds-param N)
        (sssp-log-one-third-param N))
      (sssp-log-one-third-param N)

```

```

    reduced.finite-initial-label {s'} Infinity d' Infinity U c
  using direct-run unfolding p-def q-def by simp
show ?thesis
  unfolding p-def q-def
  by (rule reduced-direct-insert-run-transfers-correctness-and-refined-bound
      [OF direct-run])
qed

theorem reduced-exact-concrete-top-level-time-transfers-correctness-and-refined-
bound:
  assumes run-exists: reduced.exact-concrete-top-level-cost D N c
  shows  $\exists d' U. \text{reduced.exact-concrete-top-level-run } D N d' U$ 
    (reduced.exact-concrete-top-level-time D N)  $\wedge$ 
    original.sssp-correct (decode-label d')  $\wedge$ 
    reduced.exact-concrete-top-level-time D N  $\leq$ 
    bmssp-refined-graph-time-bound
    ( $\lambda-. \text{Suc } D * \text{sssp-log-one-third-param } N$ )
    ( $\lambda-. \text{sssp-log-two-thirds-param } N$ )
    ( $\lambda-. \text{sssp-log-one-third-param } N$ )
    ( $\lambda-. \text{sssp-log-one-third-param } N$ )
    ( $\lambda-. \text{sssp-log-two-thirds-param } N$ )
    ( $\lambda-. \text{reduced.edge-count}$ ) reduced.vertex-count
proof –
  have cost:
    reduced.exact-concrete-top-level-cost D N
    (reduced.exact-concrete-top-level-time D N)
  by (rule reduced.exact-concrete-top-level-time-witness[OF run-exists])
obtain d' U where run:
    reduced.exact-concrete-top-level-run D N d' U
    (reduced.exact-concrete-top-level-time D N)
  using cost unfolding reduced.exact-concrete-top-level-cost-def by blast
have reduced-result:
    U = V'  $\wedge$  reduced.sssp-correct d'  $\wedge$ 
    reduced.exact-concrete-top-level-time D N  $\leq$ 
    bmssp-refined-graph-time-bound
    ( $\lambda-. \text{Suc } D * \text{sssp-log-one-third-param } N$ )
    ( $\lambda-. \text{sssp-log-two-thirds-param } N$ )
    ( $\lambda-. \text{sssp-log-one-third-param } N$ )
    ( $\lambda-. \text{sssp-log-one-third-param } N$ )
    ( $\lambda-. \text{sssp-log-two-thirds-param } N$ )
    ( $\lambda-. \text{reduced.edge-count}$ ) reduced.vertex-count
  by (rule reduced.exact-concrete-top-level-run-correct-and-refined-bound-log-
params-fixed-degree
    [OF reduced.all-vertices-reachable reduced.bounded-edge-outdegree run])
  have original-correct: original.sssp-correct (decode-label d')
  by (rule transfer-sssp-correct) (use reduced-result in blast)
show ?thesis
  using run reduced-result original-correct by blast
qed

```

theorem *reduced-runtime-headline-bmssp-runtime-bigo-target:*
shows $(\lambda n. \text{real } (\text{reduced.runtime-headline.T-bmssp } n)) \in$
 $O(\lambda n. \text{real } (\text{reduced.runtime-headline.m } n) * (\ln (\text{real } n + 2)) \text{ powr } (2 / 3))$
by (rule *reduced.runtime-headline.bmssp-runtime-bigo-target*)
end

37 Reduction Contract for the Asymptotic Bound

The theorem below is the checked asymptotic contract needed from the graph reduction layer. Once a family of reduced instances supplies a fixed outdegree bound D , a linear edge-count domination for the vertex term, and the per-instance cost bound produced by the locale theorem above, the final SSSP target bound follows.

theorem *reduction-contract-bigo-sssp-time-target:*
fixes $D :: \text{nat}$
assumes $Cn\text{-pos}: 0 < Cn$
and *vertex-count-dominated:*
 $\text{eventually } (\lambda n. \text{real } n \leq Cn * \text{real } (m \ n)) \text{ at-top}$
and *reduced-instance-cost-bound:*
 $\text{eventually } (\lambda n. T \ n \leq \text{bmssp-refined-graph-time-bound } (\lambda n. \text{Suc } D * \text{sssp-log-one-third-param } n) \text{sssp-log-two-thirds-param } \text{sssp-log-one-third-param } \text{sssp-log-one-third-param } \text{sssp-log-two-thirds-param } m \ n) \text{ at-top}$
shows $(\lambda n. \text{real } (T \ n)) \in O(\lambda n. \text{sssp-time-target } m \ n)$
by (rule *bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-degree* [OF $Cn\text{-pos}$ *vertex-count-dominated* *reduced-instance-cost-bound*])

theorem *reduction-contract-bigo-sssp-time-target-with-constant-slack:*
fixes $D :: \text{nat}$
assumes $Cn\text{-pos}: 0 < Cn$
and $Ccost\text{-pos}: 0 < Ccost$
and *vertex-count-dominated:*
 $\text{eventually } (\lambda n. \text{real } n \leq Cn * \text{real } (m \ n)) \text{ at-top}$
and *reduced-instance-cost-bound:*
 $\text{eventually } (\lambda n. \text{real } (T \ n) \leq Ccost * \text{real } (\text{bmssp-refined-graph-time-bound } (\lambda n. \text{Suc } D * \text{sssp-log-one-third-param } n) \text{sssp-log-two-thirds-param } \text{sssp-log-one-third-param } \text{sssp-log-one-third-param } \text{sssp-log-two-thirds-param } m \ n)) \text{ at-top}$
shows $(\lambda n. \text{real } (T \ n)) \in O(\lambda n. \text{sssp-time-target } m \ n)$
by (rule *bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-degree-real-slack*)

[OF Cn-pos Ccost-pos vertex-count-dominated reduced-instance-cost-bound])

theorem *reduction-contract-bigo-sssp-time-target-square-size-params:*
fixes $D :: \text{nat}$
assumes $Cn\text{-pos}: 0 < Cn$
and *vertex-count-dominated:*
eventually $(\lambda n. \text{real } n \leq Cn * \text{real } (m \ n)) \text{ at-top}$
and *reduced-instance-cost-bound:*
eventually
 $(\lambda n. T \ n \leq$
 $\text{bmssp-refined-graph-time-bound}$
 $(\lambda n. \text{Suc } D * \text{sssp-log-one-third-param } (n * n))$
 $(\lambda n. \text{sssp-log-two-thirds-param } (n * n))$
 $(\lambda n. \text{sssp-log-one-third-param } (n * n))$
 $(\lambda n. \text{sssp-log-one-third-param } (n * n))$
 $(\lambda n. \text{sssp-log-two-thirds-param } (n * n)) \ m \ n) \text{ at-top}$
shows $(\lambda n. \text{real } (T \ n)) \in O(\lambda n. \text{sssp-time-target } m \ n)$
by (rule *bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-degree-square-arg*
[OF Cn-pos vertex-count-dominated reduced-instance-cost-bound])

theorem *reduction-contract-bigo-sssp-time-target-square-size-params-with-constant-slack:*
fixes $D :: \text{nat}$
assumes $Cn\text{-pos}: 0 < Cn$
and $Ccost\text{-pos}: 0 < Ccost$
and *vertex-count-dominated:*
eventually $(\lambda n. \text{real } n \leq Cn * \text{real } (m \ n)) \text{ at-top}$
and *reduced-instance-cost-bound:*
eventually
 $(\lambda n. \text{real } (T \ n) \leq Ccost * \text{real } (\text{bmssp-refined-graph-time-bound}$
 $(\lambda n. \text{Suc } D * \text{sssp-log-one-third-param } (n * n))$
 $(\lambda n. \text{sssp-log-two-thirds-param } (n * n))$
 $(\lambda n. \text{sssp-log-one-third-param } (n * n))$
 $(\lambda n. \text{sssp-log-one-third-param } (n * n))$
 $(\lambda n. \text{sssp-log-two-thirds-param } (n * n)) \ m \ n) \text{ at-top}$
shows $(\lambda n. \text{real } (T \ n)) \in O(\lambda n. \text{sssp-time-target } m \ n)$
by (rule *bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-degree-square-arg-real-slack*
[OF Cn-pos Ccost-pos vertex-count-dominated reduced-instance-cost-bound])

end
theory *BMSSP-Executable-Refinement-Internal*
imports *BMSSP-Executable-Base-Case BMSSP-Top-Level-Bounds*
begin

38 Executable Graph Refinement Bridge

This theory starts the refinement bridge between the concrete executable entry point *bmssp-distances* and the locale-based BMSSP correctness stack. The executable graph is a list of natural-number edge triples; the abstract semantics expects a finite weighted digraph locale. The definitions below name the carrier, edge set, and weight function induced by the list representation, then package the assumptions needed before the executable loop can be related to the direct-insert BMSSP relation.

fun *nat-edge-source* :: *nat-edge* $\Rightarrow$ *nat* **where**
nat-edge-source (*u*, *v*, *w*) = *u*

fun *nat-edge-target* :: *nat-edge* $\Rightarrow$ *nat* **where**
nat-edge-target (*u*, *v*, *w*) = *v*

fun *nat-edge-weight* :: *nat-edge* $\Rightarrow$ *nat* **where**
nat-edge-weight (*u*, *v*, *w*) = *w*

definition *nat-graph-edge-list* :: *nat-graph* $\Rightarrow$ (*nat* $\times$ *nat*) *list* **where**
nat-graph-edge-list *G* = *map* ($\lambda e.$ (*nat-edge-source* *e*, *nat-edge-target* *e*)) *G*

definition *nat-graph-edges* :: *nat-graph* $\Rightarrow$ (*nat* $\times$ *nat*) *set* **where**
nat-graph-edges *G* = *set* (*nat-graph-edge-list* *G*)

definition *nat-graph-vertices* :: *nat-graph* $\Rightarrow$ *nat set* **where**
nat-graph-vertices *G* = *set* (*map nat-edge-source* *G*) $\cup$ *set* (*map nat-edge-target* *G*)

definition *nat-graph-vertex-list* :: *nat-graph* $\Rightarrow$ *nat list* **where**
nat-graph-vertex-list *G* =
sort (*remdups* (*map nat-edge-source* *G* @ *map nat-edge-target* *G*))

definition *nat-graph-weight* :: *nat-graph* $\Rightarrow$ *nat* $\Rightarrow$ *nat* $\Rightarrow$ *real* **where**
nat-graph-weight *G* *u* *v* =
(*case map-of*
(*map* ($\lambda e.$ ((*nat-edge-source* *e*, *nat-edge-target* *e*),
real (*nat-edge-weight* *e*))) *G*) (*u*, *v*) of

None $\Rightarrow$ 0
| *Some* *c* $\Rightarrow$ *c*)

definition *nat-graph-total-weight* :: *nat-graph* $\Rightarrow$ *nat* **where**
nat-graph-total-weight *G* = *sum-list* (*map nat-edge-weight* *G*)

definition *nat-graph-well-formed* :: *nat-graph* $\Rightarrow$ *bool* **where**
nat-graph-well-formed *G* $\longleftrightarrow$
distinct (*nat-graph-edge-list* *G*) $\wedge$
nat-graph-total-weight *G* < *bmssp-infinity*

lemma *nat-graph-well-formed-distinct-edge-list*:
assumes *nat-graph-well-formed* *G*
shows *distinct* (*nat-graph-edge-list* *G*)
using *assms* **unfolding** *nat-graph-well-formed-def* **by** *blast*

definition *nat-graph-reachable* :: *nat-graph* $\Rightarrow$ *nat* $\Rightarrow$ *nat* $\Rightarrow$ *bool* **where**
nat-graph-reachable *G* *a* *b* $\longleftrightarrow$
finite-weighted-digraph.reachable
(*nat-graph-vertices* *G*) (*nat-graph-edges* *G*) *a* *b*

definition *nat-graph-dist* :: *nat-graph* $\Rightarrow$ *nat* $\Rightarrow$ *nat* $\Rightarrow$ *real* **where**
nat-graph-dist *G* *a* *b* =
finite-weighted-digraph.dist
(*nat-graph-vertices* *G*) (*nat-graph-edges* *G*) (*nat-graph-weight* *G*) *a* *b*

definition *executable-label-of* :: *nat-dist* $\Rightarrow$ *nat* $\Rightarrow$ *real* **where**
executable-label-of *ds* *v* =
(case *bmssp-lookup-dist* *ds* *v* of
 None $\Rightarrow$ *bmssp-bound*
 | *Some* *d* $\Rightarrow$ *real* *d*)

lemma *bmssp-lookup-dist-mem-key*:
assumes *bmssp-lookup-dist* *ds* *v* = *Some* *d*
shows *v* $\in$ *set* (*map fst ds*)
using *assms*
proof (*induction ds*)
 case *Nil*
 then show ?*case*
 by *simp*
next
 case (*Cons* *p* *ds*)
 obtain *x* *e* **where** *p*: *p* = (*x*, *e*)
 by (*cases p*)
 show ?*case*
 proof (*cases v* = *x*)
 case *True*
 then show ?*thesis*
 using *p* **by** *simp*
 next
 case *False*
 then have *bmssp-lookup-dist* *ds* *v* = *Some* *d*
 using *Cons.prems* *p* **by** *simp*
 then have *v* $\in$ *set* (*map fst ds*)
 by (*rule Cons.IH*)
 then show ?*thesis*
 using *p* **by** *simp*
qed
qed

```

lemma bmssp-lookup-dist-Some-pair-mem:
  assumes bmssp-lookup-dist ds v = Some d
  shows  $(v, d) \in \text{set } ds$ 
  using assms
proof (induction ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons p ds)
  obtain x e where p: p = (x, e)
    by (cases p)
  show ?case
  proof (cases v = x)
    case True
    then show ?thesis
      using Cons.prems unfolding p by simp
    next
    case False
    then have bmssp-lookup-dist ds v = Some d
      using Cons.prems unfolding p by simp
    then have  $(v, d) \in \text{set } ds$ 
      by (rule Cons.IH)
    then show ?thesis
      unfolding p by simp
  qed
qed

lemma bmssp-lookup-dist-Some-if-distinct-mem:
  assumes distinct: distinct (map fst ds)
  and mem: (v, d) ∈ set ds
  shows bmssp-lookup-dist ds v = Some d
  using distinct mem
proof (induction ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons p ds)
  obtain x e where p: p = (x, e)
    by (cases p)
  show ?case
  proof (cases v = x)
    case True
    have d-eq: d = e
    proof –
      have mem-cases: (v, d) = p ∨ (v, d) ∈ set ds
        using Cons.prems p by simp
      then show ?thesis

```

```

proof
  assume  $(v, d) = p$ 
  then show ?thesis
    using True p by simp
next
  assume  $(v, d) \in \text{set } ds$ 
  then have  $x \in \text{set } (\text{map fst } ds)$ 
    using True by force
  then show ?thesis
    using Cons.prem s p by simp
qed
qed
then show ?thesis
  using True p by simp
next
case False
then have mem-tail:  $(v, d) \in \text{set } ds$ 
  using Cons.prem s p by auto
have distinct-tail: distinct  $(\text{map fst } ds)$ 
  using Cons.prem s p by simp
show ?thesis
  using False p Cons.IH[OF distinct-tail mem-tail] by simp
qed
qed

lemma bmssp-lookup-dist-None-if-distinct-not-mem:
  assumes distinct: distinct  $(\text{map fst } ds)$ 
  and not-mem:  $v \notin \text{set } (\text{map fst } ds)$ 
  shows bmssp-lookup-dist  $ds\ v = \text{None}$ 
  using distinct not-mem by (induction ds) auto

lemma bmssp-set-dist-keys:
   $\text{set } (\text{map fst } (\text{bmssp-set-dist } v\ d\ ds)) = \text{insert } v\ (\text{set } (\text{map fst } ds))$ 
proof (induction ds)
case Nil
then show ?case
  by simp
next
case  $(\text{Cons } p\ ds)$ 
obtain  $x\ e$  where  $p = (x, e)$ 
  by (cases p)
show ?case
proof  $(\text{cases } v = x)$ 
case True
then show ?thesis
  unfolding  $p$  by simp
next
case False
then show ?thesis

```

```

    using Cons.IH unfolding p by auto
  qed
qed

lemma bmssp-set-dist-fst-image [simp]:
  fst ' set (bmssp-set-dist v d ds) = insert v (fst ' set ds)
  using bmssp-set-dist-keys[of v d ds] by simp

lemma bmssp-set-dist-preserves-distinct:
  assumes distinct (map fst ds)
  shows distinct (map fst (bmssp-set-dist v d ds))
  using assms
proof (induction ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons p ds)
  obtain x e where p: p = (x, e)
    by (cases p)
  have distinct-tail: distinct (map fst ds)
    using Cons.prem1 p by simp
  show ?case
  proof (cases v = x)
    case True
    then show ?thesis
      using Cons.prem1 p by simp
  next
    case False
    have x-not-tail: x ∉ set (map fst ds)
      using Cons.prem1 p by simp
    have x-not-updated: x ∉ set (map fst (bmssp-set-dist v d ds))
      using x-not-tail False unfolding bmssp-set-dist-keys by simp
    have updated-distinct: distinct (map fst (bmssp-set-dist v d ds))
      by (rule Cons.IH[OF distinct-tail])
    show ?thesis
      using False p x-not-updated updated-distinct by simp
  qed
qed

lemma bmssp-lookup-dist-set-dist-other:
  assumes v ≠ x
  shows bmssp-lookup-dist (bmssp-set-dist x dx ds) v =
    bmssp-lookup-dist ds v
  using assms
proof (induction ds)
  case Nil
  then show ?case
    by simp

```

```

next
  case (Cons p ds)
  obtain y e where p: p = (y, e)
  by (cases p)
  then show ?case
  using Cons by auto
qed

lemma bmssp-lookup-dist-set-dist-same:
  bmssp-lookup-dist (bmssp-set-dist x dx ds) x = Some dx
  by (induction ds) auto

lemma bmssp-normalize-dist-key-set [simp]:
  set (map fst (bmssp-normalize-dist ds)) = set (map fst ds)
  unfolding bmssp-normalize-dist-def by (metis set-map set-sort)

lemma bmssp-normalize-dist-fst-image [simp]:
  fst ' set (bmssp-normalize-dist ds) = fst ' set ds
  unfolding bmssp-normalize-dist-def by simp

lemma bmssp-lookup-dist-None-iff-not-key:
  assumes distinct: distinct (map fst ds)
  shows bmssp-lookup-dist ds v = None  $\longleftrightarrow$  v  $\notin$  set (map fst ds)
proof
  assume none: bmssp-lookup-dist ds v = None
  show v  $\notin$  set (map fst ds)
  proof
    assume v  $\in$  set (map fst ds)
    then obtain d where (v, d)  $\in$  set ds
    by force
    then have bmssp-lookup-dist ds v = Some d
    by (rule bmssp-lookup-dist-Some-if-distinct-mem[OF distinct])
    then show False
    using none by simp
  qed
qed
next
  assume v  $\notin$  set (map fst ds)
  then show bmssp-lookup-dist ds v = None
  by (rule bmssp-lookup-dist-None-if-distinct-not-mem[OF distinct])
qed

lemma bmssp-lookup-dist-normalize-dist:
  assumes distinct: distinct (map fst ds)
  shows bmssp-lookup-dist (bmssp-normalize-dist ds) v =
    bmssp-lookup-dist ds v
proof (cases bmssp-lookup-dist ds v)
  case None
  have distinct-norm:
    distinct (map fst (bmssp-normalize-dist ds))

```

```

    using distinct unfolding bmssp-normalize-dist-def
    by (rule distinct-map-fst-sort-key)
  have  $v \notin \text{set } (\text{map fst } (\text{bmssp-normalize-dist } ds))$ 
    using None distinct
    unfolding bmssp-lookup-dist-None-iff-not-key[OF distinct] by simp
  then show ?thesis
    using None bmssp-lookup-dist-None-if-distinct-not-mem[OF distinct-norm]
    by simp
next
case (Some d)
have mem:  $(v, d) \in \text{set } (\text{bmssp-normalize-dist } ds)$ 
  using bmssp-lookup-dist-Some-pair-mem[OF Some]
  unfolding bmssp-normalize-dist-def by simp
have distinct-norm:
  distinct (map fst (bmssp-normalize-dist ds))
  using distinct unfolding bmssp-normalize-dist-def
  by (rule distinct-map-fst-sort-key)
have bmssp-lookup-dist (bmssp-normalize-dist ds)  $v = \text{Some } d$ 
  by (rule bmssp-lookup-dist-Some-if-distinct-mem[OF distinct-norm mem])
then show ?thesis
  using Some by simp
qed

lemma bmssp-relax-edges-preserves-distinct-dist:
  assumes distinct: distinct (map fst ds)
  shows distinct (map fst (snd (bmssp-relax-edges G settled u du ds)))
  using distinct
proof (induction G arbitrary: ds)
  case Nil
  then show ?case
    by simp
next
case (Cons e es)
  obtain a b w where  $e = (a, b, w)$ 
    by (cases e)
  obtain updates ds1 where rec:
    bmssp-relax-edges es settled u du ds = (updates, ds1)
    by force
  have distinct-ds1: distinct (map fst ds1)
    using Cons.IH[OF Cons.prem] rec by simp
  show ?case
  proof (cases  $a = u \wedge b \notin \text{set settled}$ )
    case False
    then show ?thesis
      using distinct-ds1 rec unfolding e
      by (auto simp: Let-def split: option.splits if-splits)
  next
  case True
  note active = True

```

```

show ?thesis
proof (cases bmssp-lookup-dist ds1 b)
  case None
  then show ?thesis
    using active rec bmssp-set-dist-preserves-distinct[OF distinct-ds1]
    unfolding e by (simp add: Let-def)
next
  case (Some old)
  show ?thesis
  proof (cases du + w < old)
    case True
    then show ?thesis
      using active rec Some
      bmssp-set-dist-preserves-distinct[OF distinct-ds1]
      unfolding e by (simp add: Let-def)
  next
    case False
    then show ?thesis
      using active rec Some distinct-ds1 unfolding e by (simp add: Let-def)
  qed
qed
qed
qed

```

```

lemma bmssp-relax-vertices-preserves-distinct-dist:
  assumes distinct: distinct (map fst ds)
  shows distinct (map fst (snd (bmssp-relax-vertices G settled xs ds)))
  using distinct
proof (induction xs arbitrary: ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons u us)
  show ?case
  proof (cases bmssp-lookup-dist ds u)
    case None
    then show ?thesis
      using Cons.IH[OF Cons.prem] by simp
  next
    case (Some du)
    obtain updates ds1 where edge-rec:
      bmssp-relax-edges G settled u du ds = (updates, ds1)
    by force
    have distinct-ds1: distinct (map fst ds1)
      using bmssp-relax-edges-preserves-distinct-dist[OF Cons.prem,
        of G settled u du] edge-rec by simp
    obtain updates-us ds2 where vertices-rec:
      bmssp-relax-vertices G settled us ds1 = (updates-us, ds2)

```



```

    by force
  have distinct-ds2: distinct (map fst ds2)
    using Cons.IH[OF distinct-ds1] vertices-rec by simp
  show ?thesis
    using Some edge-rec vertices-rec distinct-ds2 by simp
qed
qed

lemma bmssp-relax-edges-dist-keys-subset:
  assumes keys: set (map fst ds)  $\subseteq$  set vertices
  and uV: u  $\in$  set vertices
  and edge-targets:
     $\bigwedge a\ b\ w. (a, b, w) \in \text{set } G \implies b \in \text{set vertices}$ 
  shows set (map fst (snd (bmssp-relax-edges G settled u du ds)))
     $\subseteq$  set vertices
  using keys edge-targets
proof (induction G arbitrary: ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons e es)
  obtain a b w where e: e = (a, b, w)
    by (cases e)
  obtain updates ds1 where rec:
    bmssp-relax-edges es settled u du ds = (updates, ds1)
  by force
  have tail-subset: set (map fst ds1)  $\subseteq$  set vertices
  proof -
    have targets-tail:
       $\bigwedge x\ y\ z. (x, y, z) \in \text{set } es \implies y \in \text{set vertices}$ 
    proof -
      fix x y z
      assume xyz: (x, y, z)  $\in$  set es
      have (x, y, z)  $\in$  set (e # es)
        using xyz by simp
      then show y  $\in$  set vertices
        by (rule Cons.prem(2))
    qed
    show ?thesis
      using Cons.IH[OF Cons.prem(1) targets-tail] rec by simp
  qed
  have bV: b  $\in$  set vertices
    using Cons.prem(2)[of a b w] unfolding e by simp
  show ?case
  proof (cases a = u  $\wedge$  b  $\notin$  set settled)
    case False
    then show ?thesis
      using tail-subset rec unfolding e

```

```

    by (auto simp: Let-def split: option.splits if-splits)
next
  case True
  note active = True
  let ?nd = du + w
  show ?thesis
  proof (cases bmssp-lookup-dist ds1 b)
    case None
    then show ?thesis
      using active rec tail-subset bV
      unfolding e by (simp add: Let-def bmssp-set-dist-keys)
  next
    case (Some old)
    show ?thesis
    proof (cases ?nd < old)
      case True
      then show ?thesis
        using active rec Some tail-subset bV
        unfolding e by (simp add: Let-def bmssp-set-dist-keys)
    next
      case False
      then show ?thesis
        using active rec Some tail-subset unfolding e by (simp add: Let-def)
    qed
  qed
qed
qed
qed

lemma bmssp-relax-vertices-dist-keys-subset:
  assumes keys: set (map fst ds)  $\subseteq$  set vertices
  and xs-subset: set xs  $\subseteq$  set vertices
  and edge-targets:
     $\bigwedge a b w. (a, b, w) \in \text{set } G \implies b \in \text{set vertices}$ 
  shows set (map fst (snd (bmssp-relax-vertices G settled xs ds)))
     $\subseteq$  set vertices
  using keys xs-subset
proof (induction xs arbitrary: ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons u us)
  have uV: u  $\in$  set vertices
  using Cons.prem by simp
  have us-subset: set us  $\subseteq$  set vertices
  using Cons.prem by simp
  show ?case
  proof (cases bmssp-lookup-dist ds u)
    case None

```

```

    then show ?thesis
    using Cons.IH[OF Cons.prem1 us-subset] by simp
next
case (Some du)
obtain updates ds1 where edge-rec:
  bmssp-relax-edges G settled u du ds = (updates, ds1)
by force
have keys1: set (map fst ds1)  $\subseteq$  set vertices
proof -
  have set (map fst (snd (bmssp-relax-edges G settled u du ds)))
     $\subseteq$  set vertices
  by (rule bmssp-relax-edges-dist-keys-subset
    [OF Cons.prem1 uV edge-targets])
  then show ?thesis
  using edge-rec by simp
qed
obtain updates-us ds2 where vertices-rec:
  bmssp-relax-vertices G settled us ds1 = (updates-us, ds2)
by force
have keys2: set (map fst ds2)  $\subseteq$  set vertices
using Cons.IH[OF keys1 us-subset] vertices-rec by simp
show ?thesis
using Some edge-rec vertices-rec keys2 by simp
qed
qed

```

definition *real-label-integral-on* :: *nat set* $\Rightarrow$ (*nat* $\Rightarrow$ *real*) $\Rightarrow$ *bool* **where**
real-label-integral-on *U d* $\longleftrightarrow$
 $(\forall v \in U. \text{real} (\text{nat} (\text{floor} (d \ v))) = d \ v)$

definition *encode-dist-assoc-list* :: *nat list* $\Rightarrow$ (*nat* $\Rightarrow$ *real*) $\Rightarrow$ *nat-dist* **where**
encode-dist-assoc-list *xs d* =
 $\text{map } (\lambda v. (v, \text{nat} (\text{floor} (d \ v)))) (\text{sort} (\text{remdups } xs))$

lemma *encode-dist-assoc-list-memI*:
assumes *v* $\in$ *set xs*
shows $\exists d. (v, d) \in \text{set} (\text{encode-dist-assoc-list } xs \ f)$
using *assms* **unfolding** *encode-dist-assoc-list-def* **by** *auto*

lemma *encode-dist-assoc-list-memD*:
assumes $(v, d) \in \text{set} (\text{encode-dist-assoc-list } xs \ f)$
shows *v* $\in$ *set xs*
using *assms* **unfolding** *encode-dist-assoc-list-def* **by** *auto*

lemma *sort-remdups-eq-if-set-eq*:
assumes *set xs* = *set ys*
shows $\text{sort} (\text{remdups } xs) = \text{sort} (\text{remdups } ys)$
proof -
 have *set-eq*: $\text{set} (\text{sort} (\text{remdups } xs)) = \text{set} (\text{sort} (\text{remdups } ys))$

```

    using assms by simp
  have distinct-x: distinct (sort (remdups xs))
    by simp
  have distinct-y: distinct (sort (remdups ys))
    by simp
  have sorted-x: sorted (sort (remdups xs))
    by simp
  have sorted-y: sorted (sort (remdups ys))
    by simp
  have mset-eq:
    mset (sort (remdups xs)) = mset (sort (remdups ys))
    using distinct-x distinct-y set-eq
    by (metis set-eq-iff-mset-eq-distinct)
  have sort (sort (remdups ys)) = sort (remdups xs)
  proof (rule properties-for-sort)
    show mset (sort (remdups xs)) = mset (sort (remdups ys))
      by (rule mset-eq)
    show sorted (sort (remdups xs))
      by (rule sorted-x)
  qed
  moreover have sort (sort (remdups ys)) = sort (remdups ys)
  proof (rule sort-key-id-if-sorted)
    show sorted (map (λx. x) (sort (remdups ys)))
      using sorted-y by simp
  qed
  ultimately show ?thesis
    by simp
qed

```

```

lemma encode-dist-assoc-list-cong-set:
  assumes set xs = set ys
  shows encode-dist-assoc-list xs f = encode-dist-assoc-list ys f
  unfolding encode-dist-assoc-list-def
  using sort-remdups-eq-if-set-eq[OF assms] by simp

```

```

lemma encode-dist-assoc-list-cong-set-floor:
  assumes set-eq: set xs = set ys
  and floors:
     $\bigwedge v. v \in \text{set } xs \implies$ 
       $\text{nat } (\text{floor } (f \ v)) = \text{nat } (\text{floor } (g \ v))$ 
  shows encode-dist-assoc-list xs f = encode-dist-assoc-list ys g
  proof -
    have order-eq: sort (remdups xs) = sort (remdups ys)
      by (rule sort-remdups-eq-if-set-eq[OF set-eq])
    have map-eq:
      map (λv. (v, nat (floor (f v)))) (sort (remdups xs)) =
        map (λv. (v, nat (floor (g v)))) (sort (remdups xs))
      by (rule map-cong[OF refl]) (use floors in auto)
    show ?thesis

```

unfolding *encode-dist-assoc-list-def* **using** *order-eq map-eq* **by** *simp*
qed

lemma *encode-dist-assoc-list-partition-key*:
 $\text{encode-dist-assoc-list } xs \ (\lambda v. \text{bmssp-partition-key } v \ (f \ v)) =$
 $\text{map } (\lambda v. \ (v, f \ v)) \ (\text{sort } (\text{remdups } xs))$
unfolding *encode-dist-assoc-list-def*
by (*simp add: bmssp-partition-key-floor*)

lemma *map-fst-pair-map* [*simp*]:
 $\text{map fst } (\text{map } (\lambda x. \ (x, f \ x)) \ xs) = xs$
by (*induction xs*) *simp-all*

lemma *bmssp-normalize-dist-encode*:
assumes *distinct: distinct (map fst ds)*
shows *bmssp-normalize-dist ds =*
 $\text{encode-dist-assoc-list } (\text{map fst } ds) \ (\text{executable-label-of } ds)$

proof –
let *?lhs = bmssp-normalize-dist ds*
let *?rhs = encode-dist-assoc-list (map fst ds) (executable-label-of ds)*
have *lhs-sorted: sorted (map fst ?lhs)*
unfolding *bmssp-normalize-dist-def* **by** *simp*
have *rhs-sorted: sorted (map fst ?rhs)*
proof –
have $\text{map fst } ?rhs = \text{sort } (\text{remdups } (\text{map fst } ds))$
unfolding *encode-dist-assoc-list-def* **by** (*simp add: o-def*)
then show *?thesis*
by *simp*

qed
have *lhs-distinct: distinct (map fst ?lhs)*
unfolding *bmssp-normalize-dist-def*
by (*rule distinct-map-fst-sort-key[OF distinct]*)
have *rhs-distinct: distinct (map fst ?rhs)*
proof –
have $\text{map fst } ?rhs = \text{sort } (\text{remdups } (\text{map fst } ds))$
unfolding *encode-dist-assoc-list-def* **by** (*simp add: o-def*)
then show *?thesis*
by *simp*

qed
have *lhs-set: set ?lhs = set ds*
unfolding *bmssp-normalize-dist-def* **by** *simp*
have *rhs-set: set ?rhs = set ds*
proof
show $\text{set } ?rhs \subseteq \text{set } ds$
proof
fix *p*
assume *p: p ∈ set ?rhs*
then obtain *v where v:*
 $v \in \text{set } (\text{map fst } ds)$

```

      p = (v, nat (floor (executable-label-of ds v)))
    unfolding encode-dist-assoc-list-def by auto
  then obtain d where mem: (v, d) ∈ set ds
    by force
  have lookup: bmssp-lookup-dist ds v = Some d
    by (rule bmssp-lookup-dist-Some-if-distinct-mem[OF distinct mem])
  then have nat (floor (executable-label-of ds v)) = d
    unfolding executable-label-of-def by simp
  then show p ∈ set ds
    using v mem by simp
qed
next
show set ds ⊆ set ?rhs
proof
  fix p
  assume p: p ∈ set ds
  obtain v d where p-eq: p = (v, d)
    by (cases p)
  then have mem: (v, d) ∈ set ds
    using p by simp
  have lookup: bmssp-lookup-dist ds v = Some d
    by (rule bmssp-lookup-dist-Some-if-distinct-mem[OF distinct mem])
  then have encoded-d: nat (floor (executable-label-of ds v)) = d
    unfolding executable-label-of-def by simp
  have v ∈ set (map fst ds)
    using mem by force
  then have (v, nat (floor (executable-label-of ds v))) ∈ set ?rhs
    unfolding encode-dist-assoc-list-def by simp
  then show p ∈ set ?rhs
    using p-eq encoded-d by simp
qed
qed
have inj: inj-on fst (set ?lhs ∪ set ?rhs)
proof -
  have inj-on fst (set ds)
    using distinct by (simp add: distinct-map inj-on-def)
  then show ?thesis
    using lhs-set rhs-set by simp
qed
show ?thesis
  by (rule map-sorted-distinct-set-unique
    [OF inj lhs-sorted lhs-distinct rhs-sorted rhs-distinct])
    (simp add: lhs-set rhs-set)
qed

lemma bmssp-loop-zero-encode:
  assumes distinct (map fst ds)
  shows bmssp-loop 0 G vertices settled ds P =
    encode-dist-assoc-list (map fst ds) (executable-label-of ds)

```

using *bmssp-normalize-dist-encode*[*OF assms*] **by** *simp*

lemma *bmssp-loop-empty-pull-encode*:
assumes *pull*: *bp-pull bmssp-block-size bmssp-bound* $P = (S, \text{beta}, P1)$
and *empty*: *filter* $(\lambda x. x \in S \wedge x \notin \text{set settled})$ *vertices* = []
and *distinct*: *distinct* $(\text{map fst } ds)$
shows *bmssp-loop* $(\text{Suc fuel})\ G\ \text{vertices settled } ds\ P =$
encode-dist-assoc-list $(\text{map fst } ds)\ (\text{executable-label-of } ds)$
using *pull empty bmssp-normalize-dist-encode*[*OF distinct*] **by** *simp*

lemma *bmssp-loop-preserves-distinct-output*:
assumes *distinct*: *distinct* $(\text{map fst } ds)$
shows *distinct* $(\text{map fst } (\text{bmssp-loop fuel } G\ \text{vertices settled } ds\ P))$
using *distinct*
by $(\text{induction fuel arbitrary: settled } ds\ P)$
 $(\text{auto simp: Let-def bmssp-normalize-dist-def}$
split: prod.splits
intro: distinct-map-fst-sort-key
dest: bmssp-relax-vertices-preserves-distinct-dist)

lemma *bmssp-loop-output-sorted-keys*:
sorted $(\text{map fst } (\text{bmssp-loop fuel } G\ \text{vertices settled } ds\ P))$
by $(\text{induction fuel arbitrary: settled } ds\ P)$
 $(\text{auto simp: Let-def bmssp-normalize-dist-def split: prod.splits})$

lemma *distinct-sorted-dist-encode*:
assumes *distinct*: *distinct* $(\text{map fst } ds)$
and *sorted*: *sorted* $(\text{map fst } ds)$
shows $ds = \text{encode-dist-assoc-list } (\text{map fst } ds)\ (\text{executable-label-of } ds)$
proof –
have *bmssp-normalize-dist* $ds = ds$
using *sorted unfolding bmssp-normalize-dist-def*
by $(\text{simp add: sort-key-id-if-sorted})$
then show *?thesis*
using *bmssp-normalize-dist-encode*[*OF distinct*] **by** *simp*
qed

lemma *bmssp-loop-output-encode*:
assumes *distinct*: *distinct* $(\text{map fst } ds)$
shows *bmssp-loop fuel* $G\ \text{vertices settled } ds\ P =$
encode-dist-assoc-list
 $(\text{map fst } (\text{bmssp-loop fuel } G\ \text{vertices settled } ds\ P))$
 $(\text{executable-label-of } (\text{bmssp-loop fuel } G\ \text{vertices settled } ds\ P))$
proof –
have *distinct-out*:
distinct $(\text{map fst } (\text{bmssp-loop fuel } G\ \text{vertices settled } ds\ P))$
by $(\text{rule bmssp-loop-preserves-distinct-output}[OF distinct])$
have *sorted-out*:
sorted $(\text{map fst } (\text{bmssp-loop fuel } G\ \text{vertices settled } ds\ P))$

```

    by (rule bmssp-loop-output-sorted-keys)
  show ?thesis
    by (rule distinct-sorted-dist-encode[OF distinct-out sorted-out])
qed

lemma bmssp-loop-output-keys-subset-vertices:
  assumes keys: set (map fst ds)  $\subseteq$  set vertices
  and edge-targets:
     $\bigwedge a\ b\ w. (a, b, w) \in \text{set } G \implies b \in \text{set vertices}$ 
  shows set (map fst (bmssp-loop fuel G vertices settled ds P))
     $\subseteq$  set vertices
  using keys
proof (induction fuel arbitrary: settled ds P)
  case 0
  then show ?case
    unfolding bmssp-normalize-dist-def by simp
next
  case (Suc fuel)
  obtain S beta P1 where pull:
    bp-pull bmssp-block-size bmssp-bound P = (S, beta, P1)
  by (cases bp-pull bmssp-block-size bmssp-bound P) auto
  let ?pulled = filter ( $\lambda x. x \in S \wedge x \notin \text{set settled}$ ) vertices
  show ?case
proof (cases ?pulled = [])
  case True
  then show ?thesis
    using pull Suc.premis unfolding bmssp-normalize-dist-def by simp
next
  case False
  let ?settled' = ?pulled @ settled
  obtain updates ds' where relaxed:
    bmssp-relax-vertices G ?settled' ?pulled ds = (updates, ds')
  by force
  have pulled-subset: set ?pulled  $\subseteq$  set vertices
  by simp
  have keys-ds': set (map fst ds')  $\subseteq$  set vertices
proof -
  have set (map fst
    (snd (bmssp-relax-vertices G ?settled' ?pulled ds)))
     $\subseteq$  set vertices
  by (rule bmssp-relax-vertices-dist-keys-subset
    [OF Suc.premis pulled-subset edge-targets])
  then show ?thesis
    using relaxed by simp
qed
  have rec:
    set (map fst
      (bmssp-loop fuel G vertices ?settled' ds'
        (bmssp-apply-updates updates P1)))  $\subseteq$  set vertices

```



```

    by (rule Suc.IH[OF keys-ds'])
  show ?thesis
    using pull False relaxed rec by (simp add: Let-def)
qed
qed

lemma executable-label-integral-on-keys:
  assumes distinct: distinct (map fst ds)
  shows real-label-integral-on (set (map fst ds)) (executable-label-of ds)
  unfolding real-label-integral-on-def
proof
  fix v
  assume v ∈ set (map fst ds)
  then obtain d where mem: (v, d) ∈ set ds
    by force
  have lookup: bmssp-lookup-dist ds v = Some d
    by (rule bmssp-lookup-dist-Some-if-distinct-mem[OF distinct mem])
  then show real (nat (floor (executable-label-of ds v))) =
    executable-label-of ds v
    unfolding executable-label-of-def by simp
qed

lemma nat-graph-vertex-list-set [simp]:
  set (nat-graph-vertex-list G) = nat-graph-vertices G
  unfolding nat-graph-vertex-list-def nat-graph-vertices-def by simp

lemma nat-graph-edge-list-set [simp]:
  set (nat-graph-edge-list G) = nat-graph-edges G
  unfolding nat-graph-edge-list-def nat-graph-edges-def by simp

lemma bmssp-relax-edges-update-edge:
  assumes (v, b) ∈ set (fst (bmssp-relax-edges G settled u du ds))
  shows (u, v) ∈ set (nat-graph-edge-list G)
  using assms
proof (induction G arbitrary: ds v b)
  case Nil
  then show ?case
    by simp
next
  case (Cons e es)
  obtain a c w where e: e = (a, c, w)
    by (cases e)
  obtain updates ds1 where rec:
    bmssp-relax-edges es settled u du ds = (updates, ds1)
    by force
  have tail:
     $\bigwedge v b. (v, b) \in \text{set updates} \implies$ 
    (u, v) ∈ set (nat-graph-edge-list es)
  proof -

```

```

fix v b
assume update: (v, b) ∈ set updates
have (v, b) ∈ set (fst (bmssp-relax-edges es settled u du ds))
  using update rec by simp
then show (u, v) ∈ set (nat-graph-edge-list es)
  by (rule Cons.IH)
qed
show ?case
proof (cases a = u ∧ c ∉ set settled)
  case False
  then have (v, b) ∈ set updates
    using Cons.prem by (auto simp: e rec split: option.splits if-splits)
  then have (u, v) ∈ set (nat-graph-edge-list es)
    by (rule tail)
  then show ?thesis
    unfolding e nat-graph-edge-list-def by simp
next
  case True
  note active = True
  let ?nd = du + w
  show ?thesis
  proof (cases bmssp-lookup-dist ds1 c)
    case None
    then have update-or-tail:
      (v, b) = (c, bmssp-partition-key c ?nd) ∨
      (v, b) ∈ set updates
      using Cons.prem active unfolding e by (simp add: rec Let-def)
    then show ?thesis
    proof
      assume (v, b) = (c, bmssp-partition-key c ?nd)
      then show ?thesis
        using True unfolding e nat-graph-edge-list-def by simp
    next
      assume (v, b) ∈ set updates
      then have (u, v) ∈ set (nat-graph-edge-list es)
        by (rule tail)
      then show ?thesis
        unfolding e nat-graph-edge-list-def by simp
    qed
  next
    case (Some old)
    show ?thesis
    proof (cases ?nd < old)
      case False
      then have (v, b) ∈ set updates
        using Cons.prem active Some unfolding e by (simp add: rec Let-def)
      then have (u, v) ∈ set (nat-graph-edge-list es)
        by (rule tail)
      then show ?thesis
    next
      case True
      then have (v, b) ∈ set updates
        using Cons.prem active True unfolding e by (simp add: rec Let-def)
      then have (u, v) ∈ set (nat-graph-edge-list es)
        by (rule tail)
      then show ?thesis
    qed
  qed

```

```

      unfolding e nat-graph-edge-list-def by simp
    next
      case True
      then have update-or-tail:
         $(v, b) = (c, \text{bmssp-partition-key } c \text{ ?nd}) \vee$ 
         $(v, b) \in \text{set updates}$ 
      using Cons.premis active Some unfolding e by (simp add: rec Let-def)
      then show ?thesis
      proof
        assume  $(v, b) = (c, \text{bmssp-partition-key } c \text{ ?nd})$ 
        then show ?thesis
          using active unfolding e nat-graph-edge-list-def by simp
      next
        assume  $(v, b) \in \text{set updates}$ 
        then have  $(u, v) \in \text{set (nat-graph-edge-list es)}$ 
          by (rule tail)
        then show ?thesis
          unfolding e nat-graph-edge-list-def by simp
      qed
    qed
  qed
qed

```

lemma *bmssp-relax-edges-update-partition-key:*
assumes $(v, b) \in \text{set (fst (bmssp-relax-edges } G \text{ settled } u \text{ } du \text{ } ds))}$
shows $\exists d. b = \text{bmssp-partition-key } v \text{ } d$
using *assms*
proof (*induction* *G* *arbitrary: ds v b*)
 case Nil
 then show ?case
 by simp
next
 case (Cons e es)
 obtain a c w where $e = (a, c, w)$
 by (cases e)
 obtain updates ds1 where rec:
 $\text{bmssp-relax-edges es settled } u \text{ } du \text{ } ds = (\text{updates}, ds1)$
 by force
 have tail:
 $\bigwedge v b. (v, b) \in \text{set updates} \implies$
 $\exists d. b = \text{bmssp-partition-key } v \text{ } d$
proof –
 fix v b
 assume update: $(v, b) \in \text{set updates}$
 have $(v, b) \in \text{set (fst (bmssp-relax-edges es settled } u \text{ } du \text{ } ds))}$
 using update rec by simp
 then show $\exists d. b = \text{bmssp-partition-key } v \text{ } d$
 by (rule Cons.IH)

```

qed
show ?case
proof (cases  $a = u \wedge c \notin \text{set settled}$ )
  case False
  then have  $(v, b) \in \text{set updates}$ 
    using Cons.premis by (auto simp: e rec split: option.splits if-splits)
  then show ?thesis
    by (rule tail)
next
case True
note active = True
let ?nd =  $du + w$ 
show ?thesis
proof (cases bmssp-lookup-dist ds1 c)
  case None
  then have update-or-tail:
     $(v, b) = (c, \text{bmssp-partition-key } c \text{ ?nd}) \vee$ 
     $(v, b) \in \text{set updates}$ 
    using Cons.premis active unfolding e by (simp add: rec Let-def)
  then show ?thesis
  proof
    assume  $(v, b) = (c, \text{bmssp-partition-key } c \text{ ?nd})$ 
    then show ?thesis
      by auto
  next
    assume  $(v, b) \in \text{set updates}$ 
    then show ?thesis
      by (rule tail)
  qed
next
case (Some old)
show ?thesis
proof (cases ?nd < old)
  case False
  then have  $(v, b) \in \text{set updates}$ 
    using Cons.premis active Some unfolding e by (simp add: rec Let-def)
  then show ?thesis
    by (rule tail)
next
case True
case True
then have update-or-tail:
   $(v, b) = (c, \text{bmssp-partition-key } c \text{ ?nd}) \vee$ 
   $(v, b) \in \text{set updates}$ 
  using Cons.premis active Some unfolding e by (simp add: rec Let-def)
then show ?thesis
proof
  assume  $(v, b) = (c, \text{bmssp-partition-key } c \text{ ?nd})$ 
  then show ?thesis
    by auto

```

```

    next
    assume  $(v, b) \in \text{set updates}$ 
    then show ?thesis
    by (rule tail)
  qed
qed
qed
qed
qed

lemma bmssp-relax-vertices-update-partition-key:
  assumes  $(v, b) \in \text{set (fst (bmssp-relax-vertices } G \text{ settled } xs \text{ } ds))$ 
  shows  $\exists d. b = \text{bmssp-partition-key } v \text{ } d$ 
  using assms
proof (induction xs arbitrary: ds)
  case Nil
  then show ?case
  by simp
next
  case (Cons u us)
  show ?case
  proof (cases bmssp-lookup-dist ds u)
    case None
    then have  $(v, b) \in \text{set (fst (bmssp-relax-vertices } G \text{ settled } us \text{ } ds))$ 
    using Cons.prem1 by simp
    then show ?thesis
    by (rule Cons.IH)
  next
    case (Some du)
    obtain updates-u ds1 where edge-rec:
       $\text{bmssp-relax-edges } G \text{ settled } u \text{ } du \text{ } ds = (\text{updates-}u, ds1)$ 
    by force
    obtain updates-us ds2 where vertices-rec:
       $\text{bmssp-relax-vertices } G \text{ settled } us \text{ } ds1 = (\text{updates-us}, ds2)$ 
    by force
    have update-cases:
       $(v, b) \in \text{set updates-}u \vee (v, b) \in \text{set updates-us}$ 
    using Cons.prem1 Some edge-rec vertices-rec by auto
    then show ?thesis
    proof
      assume  $(v, b) \in \text{set updates-}u$ 
      then have  $(v, b) \in \text{set (fst (bmssp-relax-edges } G \text{ settled } u \text{ } du \text{ } ds))$ 
      using edge-rec by simp
      then show ?thesis
      by (rule bmssp-relax-edges-update-partition-key)
    next
      assume  $(v, b) \in \text{set updates-us}$ 
      then have  $(v, b) \in \text{set (fst (bmssp-relax-vertices } G \text{ settled } us \text{ } ds1))$ 
      using vertices-rec by simp
    qed
  qed

```

```

    then show ?thesis
    by (rule Cons.IH)
  qed
qed
qed

lemma bmssp-relax-vertices-update-floor:
  assumes  $(v, b) \in \text{set } (\text{fst } (\text{bmssp-relax-vertices } G \text{ settled } xs \ ds))$ 
  shows  $\exists d. b = \text{bmssp-partition-key } v \ d \wedge \text{nat } (\text{floor } b) = d$ 
proof -
  obtain d where  $d: b = \text{bmssp-partition-key } v \ d$ 
  using bmssp-relax-vertices-update-partition-key[OF assms] by blast
  then have  $\text{nat } (\text{floor } b) = d$ 
  by (simp add: bmssp-partition-key-floor)
  then show ?thesis
  using d by blast
qed

lemma bmssp-relax-edges-update-floor:
  assumes  $(v, b) \in \text{set } (\text{fst } (\text{bmssp-relax-edges } G \text{ settled } u \ du \ ds))$ 
  shows  $\exists d. b = \text{bmssp-partition-key } v \ d \wedge \text{nat } (\text{floor } b) = d$ 
proof -
  obtain d where  $d: b = \text{bmssp-partition-key } v \ d$ 
  using bmssp-relax-edges-update-partition-key[OF assms] by blast
  then have  $\text{nat } (\text{floor } b) = d$ 
  by (simp add: bmssp-partition-key-floor)
  then show ?thesis
  using d by blast
qed

lemma bmssp-relax-edges-update-not-settled:
  assumes  $(v, b) \in \text{set } (\text{fst } (\text{bmssp-relax-edges } G \text{ settled } u \ du \ ds))$ 
  shows  $v \notin \text{set settled}$ 
  using assms
proof (induction G arbitrary: ds v b)
  case Nil
  then show ?case
  by simp
next
  case (Cons e es)
  obtain a c w where  $e: e = (a, c, w)$ 
  by (cases e)
  obtain updates ds1 where rec:
     $\text{bmssp-relax-edges } es \text{ settled } u \ du \ ds = (\text{updates}, ds1)$ 
  by force
  have tail:
     $\bigwedge v b. (v, b) \in \text{set updates} \implies v \notin \text{set settled}$ 
  proof -
    fix v b

```

```

    assume update: (v, b) ∈ set updates
    have (v, b) ∈ set (fst (bmssp-relax-edges es settled u du ds))
      using update rec by simp
    then show v ∉ set settled
      by (rule Cons.IH)
  qed
show ?case
proof (cases a = u ∧ c ∉ set settled)
  case False
  then have (v, b) ∈ set updates
    using Cons.premis rec unfolding e
    by (auto simp: Let-def split: option.splits if-splits)
  then show ?thesis
    by (rule tail)
next
  case True
  note active = True
  let ?nd = du + w
  show ?thesis
  proof (cases bmssp-lookup-dist ds1 c)
    case None
    then have update-or-tail:
      (v, b) = (c, bmssp-partition-key c ?nd) ∨
      (v, b) ∈ set updates
      using Cons.premis active rec unfolding e by (simp add: Let-def)
    then show ?thesis
    proof
      assume (v, b) = (c, bmssp-partition-key c ?nd)
      then show ?thesis
        using active by simp
    next
      assume (v, b) ∈ set updates
      then show ?thesis
        by (rule tail)
    qed
  next
    case (Some old)
    show ?thesis
    proof (cases ?nd < old)
      case False
      then have (v, b) ∈ set updates
        using Cons.premis active Some rec unfolding e by (simp add: Let-def)
      then show ?thesis
        by (rule tail)
    next
      case True
      then have update-or-tail:
        (v, b) = (c, bmssp-partition-key c ?nd) ∨
        (v, b) ∈ set updates

```

```

    using Cons.premis active Some rec unfolding e by (simp add: Let-def)
  then show ?thesis
proof
  assume (v, b) = (c, bmssp-partition-key c ?nd)
  then show ?thesis
    using active by simp
next
  assume (v, b) ∈ set updates
  then show ?thesis
    by (rule tail)
qed
qed
qed
qed
qed

lemma bmssp-relax-vertices-update-not-settled:
  assumes (v, b) ∈ set (fst (bmssp-relax-vertices G settled xs ds))
  shows v ∉ set settled
  using assms
proof (induction xs arbitrary: ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons u us)
  show ?case
  proof (cases bmssp-lookup-dist ds u)
    case None
    then have (v, b) ∈ set (fst (bmssp-relax-vertices G settled us ds))
      using Cons.premis by simp
    then show ?thesis
      by (rule Cons.IH)
  next
    case (Some du)
    obtain updates-u ds1 where edge-rec:
      bmssp-relax-edges G settled u du ds = (updates-u, ds1)
      by force
    obtain updates-us ds2 where vertices-rec:
      bmssp-relax-vertices G settled us ds1 = (updates-us, ds2)
      by force
    have update-cases:
      (v, b) ∈ set updates-u ∨ (v, b) ∈ set updates-us
      using Cons.premis Some edge-rec vertices-rec by auto
    then show ?thesis
  proof
    assume (v, b) ∈ set updates-u
    then have (v, b) ∈ set (fst (bmssp-relax-edges G settled u du ds))
      using edge-rec by simp

```



```

    then show ?thesis
      by (rule bmssp-relax-edges-update-not-settled)
  next
    assume  $(v, b) \in \text{set updates-us}$ 
    then have  $(v, b) \in \text{set (fst (bmssp-relax-vertices } G \text{ settled us } ds1))$ 
      using vertices-rec by simp
    then show ?thesis
      by (rule Cons.IH)
  qed
qed
qed

lemma bmssp-relax-edges-dist-keys:
   $\text{set (map fst (snd (bmssp-relax-edges } G \text{ settled u du ds)))} =$ 
   $\text{set (map fst ds)} \cup \text{fst ' set (fst (bmssp-relax-edges } G \text{ settled u du ds))}$ 
proof (induction  $G$  arbitrary:  $ds$ )
  case Nil
  then show ?case
    by simp
next
  case (Cons  $e$   $es$ )
  obtain  $a$   $c$   $w$  where  $e = (a, c, w)$ 
    by (cases  $e$ )
  obtain updates  $ds1$  where rec:
     $\text{bmssp-relax-edges } es \text{ settled u du ds} = (\text{updates}, ds1)$ 
    by force
  have tail:
     $\text{set (map fst } ds1) = \text{set (map fst ds)} \cup \text{fst ' set updates}$ 
    using Cons.IH[of  $ds$ ] rec by simp
  show ?case
  proof (cases  $a = u \wedge c \notin \text{set settled}$ )
    case False
    then show ?thesis
      using rec tail unfolding  $e$ 
      by (auto simp: Let-def split: option.splits if-splits)
  next
    case True
    note active = True
    let ?nd =  $du + w$ 
    show ?thesis
    proof (cases bmssp-lookup-dist  $ds1$   $c$ )
      case None
      then show ?thesis
        using active rec tail unfolding  $e$ 
        by (auto simp: Let-def bmssp-set-dist-keys)
    next
      case (Some old)
      show ?thesis
      proof (cases ?nd < old)

```

```

      case False
      then show ?thesis
        using active rec Some tail unfolding e by (auto simp: Let-def)
    next
      case True
      then show ?thesis
        using active rec Some tail unfolding e
        by (auto simp: Let-def bmssp-set-dist-keys)
    qed
  qed
qed

```

```

lemma bmssp-relax-vertices-dist-keys:
  set (map fst (snd (bmssp-relax-vertices G settled xs ds))) =
    set (map fst ds) ∪ fst ' set (fst (bmssp-relax-vertices G settled xs ds))
proof (induction xs arbitrary: ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons u us)
  show ?case
  proof (cases bmssp-lookup-dist ds u)
    case None
    then show ?thesis
      using Cons.IH[of ds] by simp
  next
    case (Some du)
    obtain updates-u ds1 where edge-rec:
      bmssp-relax-edges G settled u du ds = (updates-u, ds1)
    by force
    obtain updates-us ds2 where vertices-rec:
      bmssp-relax-vertices G settled us ds1 = (updates-us, ds2)
    by force
    have edge-keys:
      set (map fst ds1) = set (map fst ds) ∪ fst ' set updates-u
    using bmssp-relax-edges-dist-keys[of G settled u du ds] edge-rec
    by simp
    have tail-keys:
      set (map fst ds2) = set (map fst ds1) ∪ fst ' set updates-us
    using Cons.IH[of ds1] vertices-rec by simp
    show ?thesis
      using Some edge-rec vertices-rec edge-keys tail-keys by auto
  qed
qed

```

```

lemma bmssp-relax-edges-lookup-not-updated:
  assumes v ∉ fst ' set (fst (bmssp-relax-edges G settled u du ds))

```

```

shows bmssp-lookup-dist (snd (bmssp-relax-edges G settled u du ds)) v =
  bmssp-lookup-dist ds v
using assms
proof (induction G arbitrary: ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons e es)
  obtain a c w where e: e = (a, c, w)
    by (cases e)
  obtain updates ds1 where rec:
    bmssp-relax-edges es settled u du ds = (updates, ds1)
    by force
  have v-not-tail: v  $\notin$  fst ' set updates
    using Cons.prems rec unfolding e
    by (auto simp: Let-def split: option.splits if-splits)
  have tail-lookup: bmssp-lookup-dist ds1 v = bmssp-lookup-dist ds v
    using Cons.IH[of ds] v-not-tail rec by simp
  show ?case
  proof (cases a = u  $\wedge$  c  $\notin$  set settled)
    case False
    then show ?thesis
      using rec tail-lookup unfolding e
      by (auto simp: Let-def split: option.splits if-splits)
  next
  case True
  note active = True
  let ?nd = du + w
  show ?thesis
  proof (cases bmssp-lookup-dist ds1 c)
    case None
    then have v-ne-c: v  $\neq$  c
      using Cons.prems active rec unfolding e by (auto simp: Let-def)
    show ?thesis
      using active rec None tail-lookup v-ne-c unfolding e
      by (simp add: Let-def bmssp-lookup-dist-set-dist-other)
  next
  case (Some old)
  show ?thesis
  proof (cases ?nd < old)
    case False
    then show ?thesis
      using active rec Some tail-lookup unfolding e by (simp add: Let-def)
  next
  case True
  then have v-ne-c: v  $\neq$  c
    using Cons.prems active rec Some unfolding e by (auto simp: Let-def)
  show ?thesis

```

```

      using active rec Some True tail-lookup v-ne-c unfolding e
      by (simp add: Let-def bmssp-lookup-dist-set-dist-other)
    qed
  qed
  qed
  qed

lemma bmssp-relax-vertices-lookup-not-updated:
  assumes  $v \notin \text{fst } \text{'set (fst (bmssp-relax-vertices } G \text{ settled } xs \text{ } ds))}$ 
  shows  $\text{bmssp-lookup-dist (snd (bmssp-relax-vertices } G \text{ settled } xs \text{ } ds)) } v =$ 
     $\text{bmssp-lookup-dist } ds \text{ } v$ 
  using assms
proof (induction xs arbitrary: ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons u us)
  show ?case
  proof (cases  $\text{bmssp-lookup-dist } ds \text{ } u$ )
    case None
    then have v-not-tail:
       $v \notin \text{fst } \text{'set (fst (bmssp-relax-vertices } G \text{ settled } us \text{ } ds))}$ 
    using Cons.prems by simp
    show ?thesis
      using None Cons.IH[OF v-not-tail] by simp
  next
    case (Some du)
    obtain updates-u ds1 where edge-rec:
       $\text{bmssp-relax-edges } G \text{ settled } u \text{ } du \text{ } ds = (\text{updates-}u, ds1)$ 
    by force
    obtain updates-us ds2 where vertices-rec:
       $\text{bmssp-relax-vertices } G \text{ settled } us \text{ } ds1 = (\text{updates-}us, ds2)$ 
    by force
    have v-not-u:  $v \notin \text{fst } \text{'set updates-}u$ 
    using Cons.prems Some edge-rec vertices-rec by auto
    have v-not-us:  $v \notin \text{fst } \text{'set updates-}us$ 
    using Cons.prems Some edge-rec vertices-rec by auto
    have lookup-u:  $\text{bmssp-lookup-dist } ds1 \text{ } v = \text{bmssp-lookup-dist } ds \text{ } v$ 
    using bmssp-relax-edges-lookup-not-updated[of v G settled u du ds]
      v-not-u edge-rec by simp
    have lookup-us:  $\text{bmssp-lookup-dist } ds2 \text{ } v = \text{bmssp-lookup-dist } ds1 \text{ } v$ 
    using Cons.IH[of ds1] v-not-us vertices-rec by simp
    show ?thesis
      using Some edge-rec vertices-rec lookup-u lookup-us by simp
  qed
qed

```

lemma *bmssp-relax-edges-update-lookup-floor*:

```

assumes distinct-updates:
  distinct (map fst (fst (bmssp-relax-edges G settled u du ds)))
and update:  $(v, b) \in \text{set } (\text{fst } (\text{bmssp-relax-edges } G \text{ settled } u \text{ du } ds))$ 
shows bmssp-lookup-dist (snd (bmssp-relax-edges G settled u du ds))  $v =$ 
  Some (nat (floor b))
using distinct-updates update
proof (induction G arbitrary: ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons e es)
  obtain a c w where  $e = (a, c, w)$ 
    by (cases e)
  obtain updates ds1 where rec:
    bmssp-relax-edges es settled u du ds = (updates, ds1)
    by force
  show ?case
  proof (cases  $a = u \wedge c \notin \text{set settled}$ )
    case False
    have distinct-tail: distinct (map fst updates)
      using Cons.prem1 False rec unfolding e
      by (auto simp: Let-def split: option.splits if-splits)
    have update-tail:  $(v, b) \in \text{set } \text{updates}$ 
      using Cons.prem2 False rec unfolding e
      by (auto simp: Let-def split: option.splits if-splits)
    have tail-lookup:
      bmssp-lookup-dist ds1 v = Some (nat (floor b))
      using Cons.IH[of ds] distinct-tail update-tail rec by simp
    then show ?thesis
      using False rec unfolding e
      by (auto simp: Let-def split: option.splits if-splits)
  next
  case True
  note active = True
  let ?nd = du + w
  show ?thesis
  proof (cases bmssp-lookup-dist ds1 c)
    case None
    let ?new = (c, bmssp-partition-key c ?nd)
    have all-updates:
      fst (bmssp-relax-edges (e # es) settled u du ds) = ?new # updates
      using active None rec unfolding e by (simp add: Let-def)
    have all-ds:
      snd (bmssp-relax-edges (e # es) settled u du ds) =
        bmssp-set-dist c ?nd ds1
      using active None rec unfolding e by (simp add: Let-def)
    have distinct-tail: distinct (map fst updates)
      using Cons.prem1 all-updates by simp

```

```

have c-not-tail:  $c \notin \text{fst } \text{'set updates}$ 
  using Cons.premis(1) all-updates by simp
show ?thesis
proof (cases  $(v, b) = ?new$ )
  case True
  then show ?thesis
    using all-ds by (simp add: bmssp-partition-key-floor
      bmssp-lookup-dist-set-dist-same)
next
  case False
  then have update-tail:  $(v, b) \in \text{set updates}$ 
    using Cons.premis(2) all-updates by auto
  have v-ne-c:  $v \neq c$ 
    using update-tail c-not-tail by force
  have tail-lookup:
    bmssp-lookup-dist ds1 v = Some (nat (floor b))
    using Cons.IH[of ds] distinct-tail update-tail rec by simp
  show ?thesis
    using all-ds v-ne-c tail-lookup
    by (simp add: bmssp-lookup-dist-set-dist-other)
qed
next
case (Some old)
show ?thesis
proof (cases  $?nd < old$ )
  case False
  have distinct-tail: distinct (map fst updates)
    using Cons.premis(1) active Some False rec unfolding e
    by (simp add: Let-def)
  have update-tail:  $(v, b) \in \text{set updates}$ 
    using Cons.premis(2) active Some False rec unfolding e
    by (simp add: Let-def)
  have tail-lookup:
    bmssp-lookup-dist ds1 v = Some (nat (floor b))
    using Cons.IH[of ds] distinct-tail update-tail rec by simp
  then show ?thesis
    using active Some False rec unfolding e by (simp add: Let-def)
next
  case True
  let ?new = (c, bmssp-partition-key c ?nd)
  have all-updates:
    fst (bmssp-relax-edges (e # es) settled u du ds) = ?new # updates
    using active Some True rec unfolding e by (simp add: Let-def)
  have all-ds:
    snd (bmssp-relax-edges (e # es) settled u du ds) =
      bmssp-set-dist c ?nd ds1
    using active Some True rec unfolding e by (simp add: Let-def)
  have distinct-tail: distinct (map fst updates)
    using Cons.premis(1) all-updates by simp

```

```

have c-not-tail:  $c \notin \text{fst } \text{'set updates}$ 
  using Cons.premis(1) all-updates by simp
show ?thesis
proof (cases (v, b) = ?new)
  case True
  then show ?thesis
    using all-ds by (simp add: bmssp-partition-key-floor
      bmssp-lookup-dist-set-dist-same)
next
  case False
  then have update-tail:  $(v, b) \in \text{set updates}$ 
    using Cons.premis(2) all-updates by auto
  have v-ne-c:  $v \neq c$ 
    using update-tail c-not-tail by force
  have tail-lookup:
    bmssp-lookup-dist ds1 v = Some (nat (floor b))
    using Cons.IH[of ds] distinct-tail update-tail rec by simp
  show ?thesis
    using all-ds v-ne-c tail-lookup
    by (simp add: bmssp-lookup-dist-set-dist-other)
qed
qed
qed
qed
qed

```

```

lemma bmssp-relax-vertices-update-lookup-floor:
  assumes distinct-updates:
    distinct (map fst (fst (bmssp-relax-vertices G settled xs ds)))
  and update:  $(v, b) \in \text{set (fst (bmssp-relax-vertices G settled xs ds))}$ 
  shows bmssp-lookup-dist (snd (bmssp-relax-vertices G settled xs ds)) v =
    Some (nat (floor b))
  using distinct-updates update
proof (induction xs arbitrary: ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons u us)
  show ?case
  proof (cases bmssp-lookup-dist ds u)
    case None
    have distinct-tail:
      distinct (map fst (fst (bmssp-relax-vertices G settled us ds)))
      using Cons.premis(1) None by simp
    have update-tail:
       $(v, b) \in \text{set (fst (bmssp-relax-vertices G settled us ds))}$ 
      using Cons.premis(2) None by simp
    show ?thesis

```

```

    using None Cons.IH[OF distinct-tail update-tail] by simp
next
case (Some du)
obtain updates-u ds1 where edge-rec:
  bmssp-relax-edges G settled u du ds = (updates-u, ds1)
  by force
obtain updates-us ds2 where vertices-rec:
  bmssp-relax-vertices G settled us ds1 = (updates-us, ds2)
  by force
have updates-eq:
  fst (bmssp-relax-vertices G settled (u # us) ds) =
    updates-u @ updates-us
  using Some edge-rec vertices-rec by simp
have ds-eq:
  snd (bmssp-relax-vertices G settled (u # us) ds) = ds2
  using Some edge-rec vertices-rec by simp
have distinct-append: distinct (map fst (updates-u @ updates-us))
  using Cons.prems(1) updates-eq by simp
show ?thesis
proof (cases (v, b) ∈ set updates-u)
case True
have distinct-u: distinct (map fst updates-u)
  using distinct-append by simp
have lookup-u: bmssp-lookup-dist ds1 v = Some (nat (floor b))
  using bmssp-relax-edges-update-lookup-floor[of G settled u du ds v b]
  distinct-u True edge-rec by simp
have v-not-us: v ∉ fst ` set updates-us
  using distinct-append True by force
have lookup-us: bmssp-lookup-dist ds2 v = bmssp-lookup-dist ds1 v
  using bmssp-relax-vertices-lookup-not-updated[of v G settled us ds1]
  v-not-us vertices-rec by simp
show ?thesis
  using ds-eq lookup-u lookup-us by simp
next
case False
have update-us: (v, b) ∈ set updates-us
  using Cons.prems(2) updates-eq False by auto
have distinct-us: distinct (map fst updates-us)
  using distinct-append by simp
have lookup-us: bmssp-lookup-dist ds2 v = Some (nat (floor b))
  using Cons.IH[of ds1] distinct-us update-us vertices-rec by simp
show ?thesis
  using ds-eq lookup-us by simp
qed
qed
qed

```

lemma *bmssp-relax-edges-lookup-Some-preserved:*
assumes distinct: distinct (map fst ds)

and lookup: *bmssp-lookup-dist* $ds\ v = \text{Some } d$
shows $\exists d'. \text{bmssp-lookup-dist}$
 $(\text{snd } (\text{bmssp-relax-edges } G\ \text{settled } u\ du\ ds))\ v = \text{Some } d'$
proof –
have *v-key:* $v \in \text{set } (\text{map } \text{fst } ds)$
using *lookup* **by** (rule *bmssp-lookup-dist-mem-key*)
have *keys:*
 $\text{set } (\text{map } \text{fst } (\text{snd } (\text{bmssp-relax-edges } G\ \text{settled } u\ du\ ds)))) =$
 $\text{set } (\text{map } \text{fst } ds) \cup \text{fst } ' \text{set } (\text{fst } (\text{bmssp-relax-edges } G\ \text{settled } u\ du\ ds))$
by (rule *bmssp-relax-edges-dist-keys*)
have *distinct':*
 $\text{distinct } (\text{map } \text{fst } (\text{snd } (\text{bmssp-relax-edges } G\ \text{settled } u\ du\ ds))))$
by (rule *bmssp-relax-edges-preserves-distinct-dist*[*OF distinct*])
have $v \in \text{set } (\text{map } \text{fst } (\text{snd } (\text{bmssp-relax-edges } G\ \text{settled } u\ du\ ds))))$
using *v-key* *keys* **by** *blast*
then obtain d' **where** $(v, d') \in$
 $\text{set } (\text{snd } (\text{bmssp-relax-edges } G\ \text{settled } u\ du\ ds))$
by *force*
then have *bmssp-lookup-dist*
 $(\text{snd } (\text{bmssp-relax-edges } G\ \text{settled } u\ du\ ds))\ v = \text{Some } d'$
by (rule *bmssp-lookup-dist-Some-if-distinct-mem*[*OF distinct'*])
then show *?thesis*
by *blast*
qed

lemma *bmssp-relax-vertices-lookup-Some-preserved:*
assumes *distinct:* $\text{distinct } (\text{map } \text{fst } ds)$
and lookup: *bmssp-lookup-dist* $ds\ v = \text{Some } d$
shows $\exists d'. \text{bmssp-lookup-dist}$
 $(\text{snd } (\text{bmssp-relax-vertices } G\ \text{settled } xs\ ds))\ v = \text{Some } d'$
proof –
have *v-key:* $v \in \text{set } (\text{map } \text{fst } ds)$
using *lookup* **by** (rule *bmssp-lookup-dist-mem-key*)
have *keys:*
 $\text{set } (\text{map } \text{fst } (\text{snd } (\text{bmssp-relax-vertices } G\ \text{settled } xs\ ds)))) =$
 $\text{set } (\text{map } \text{fst } ds) \cup$
 $\text{fst } ' \text{set } (\text{fst } (\text{bmssp-relax-vertices } G\ \text{settled } xs\ ds))$
by (rule *bmssp-relax-vertices-dist-keys*)
have *distinct':*
 $\text{distinct } (\text{map } \text{fst } (\text{snd } (\text{bmssp-relax-vertices } G\ \text{settled } xs\ ds))))$
by (rule *bmssp-relax-vertices-preserves-distinct-dist*[*OF distinct*])
have $v \in \text{set } (\text{map } \text{fst } (\text{snd } (\text{bmssp-relax-vertices } G\ \text{settled } xs\ ds))))$
using *v-key* *keys* **by** *blast*
then obtain d' **where** $(v, d') \in$
 $\text{set } (\text{snd } (\text{bmssp-relax-vertices } G\ \text{settled } xs\ ds))$
by *force*
then have *bmssp-lookup-dist*
 $(\text{snd } (\text{bmssp-relax-vertices } G\ \text{settled } xs\ ds))\ v = \text{Some } d'$
by (rule *bmssp-lookup-dist-Some-if-distinct-mem*[*OF distinct'*])

```

    then show ?thesis
      by blast
qed

lemma bmssp-relax-edges-lookup-le:
  assumes distinct: distinct (map fst ds)
  and old: bmssp-lookup-dist ds v = Some old
  and new:
    bmssp-lookup-dist (snd (bmssp-relax-edges G settled u du ds)) v =
      Some new
  shows new ≤ old
  using distinct old new
proof (induction G arbitrary: ds v old new)
  case Nil
  then show ?case
    by simp
next
  case (Cons e es)
  obtain a c w where e: e = (a, c, w)
  by (cases e)
  obtain updates ds1 where rec:
    bmssp-relax-edges es settled u du ds = (updates, ds1)
  by force
  have tail-distinct: distinct (map fst ds1)
  using bmssp-relax-edges-preserves-distinct-dist[OF Cons.prem1],
    of es settled u du] rec by simp
  show ?case
proof (cases a = u ∧ c ∉ set settled)
  case False
  have tail-new:
    bmssp-lookup-dist (snd (bmssp-relax-edges es settled u du ds)) v =
      Some new
  using Cons.prem3 False rec unfolding e
  by (auto simp: Let-def split: option.splits if-splits)
  show ?thesis
  by (rule Cons.IH[OF Cons.prem1 Cons.prem2 tail-new])
next
  case True
  note active = True
  let ?nd = du + w
  show ?thesis
proof (cases bmssp-lookup-dist ds1 c)
  case None
  show ?thesis
proof (cases v = c)
  case True
  have old-c: bmssp-lookup-dist ds c = Some old
  using Cons.prem2 True by simp
  have ∃ d'. bmssp-lookup-dist

```

```

      (snd (bmssp-relax-edges es settled u du ds)) c = Some d'
    by (rule bmssp-relax-edges-lookup-Some-preserved
        [OF Cons.prem1 old-c])
  then show ?thesis
    using None rec by simp
next
case False
have tail-new:
  bmssp-lookup-dist
  (snd (bmssp-relax-edges es settled u du ds)) v = Some new
  using Cons.prem3 active None False rec unfolding e
  by (simp add: Let-def bmssp-lookup-dist-set-dist-other)
show ?thesis
  by (rule Cons.IH[OF Cons.prem1 Cons.prem2 tail-new])
qed
next
case (Some cur)
show ?thesis
proof (cases ?nd < cur)
case False
have tail-new:
  bmssp-lookup-dist
  (snd (bmssp-relax-edges es settled u du ds)) v = Some new
  using Cons.prem3 active Some False rec unfolding e
  by (simp add: Let-def)
show ?thesis
  by (rule Cons.IH[OF Cons.prem1 Cons.prem2 tail-new])
next
case True
note improves = True
show ?thesis
proof (cases v = c)
case True
have old-c: bmssp-lookup-dist ds c = Some old
  using Cons.prem2 True by simp
have tail-cur:
  bmssp-lookup-dist
  (snd (bmssp-relax-edges es settled u du ds)) c = Some cur
  using Some rec by simp
have cur-le-old: cur ≤ old
  by (rule Cons.IH[OF Cons.prem1 old-c tail-cur])
have new-eq: new = ?nd
  using Cons.prem3 active Some improves ⟨v = c⟩ rec unfolding e
  by (simp add: Let-def bmssp-lookup-dist-set-dist-same)
then show ?thesis
  using improves cur-le-old by linarith
next
case False
have tail-new:

```

```

      bmssp-lookup-dist
      (snd (bmssp-relax-edges es settled u du ds)) v = Some new
    using Cons.prem3 active Some True False rec unfolding e
    by (simp add: Let-def bmssp-lookup-dist-set-dist-other)
  show ?thesis
    by (rule Cons.IH[OF Cons.prem1 Cons.prem2 tail-new])
qed
qed
qed
qed
qed

lemma bmssp-relax-vertices-lookup-le:
  assumes distinct: distinct (map fst ds)
  and old: bmssp-lookup-dist ds v = Some old
  and new:
    bmssp-lookup-dist (snd (bmssp-relax-vertices G settled xs ds)) v =
      Some new
  shows new ≤ old
  using distinct old new
proof (induction xs arbitrary: ds v old new)
  case Nil
  then show ?case
    by simp
next
  case (Cons u us)
  show ?case
  proof (cases bmssp-lookup-dist ds u)
  case None
  have tail-new:
    bmssp-lookup-dist (snd (bmssp-relax-vertices G settled us ds)) v =
      Some new
  using Cons.prem3 None by simp
  show ?thesis
    by (rule Cons.IH[OF Cons.prem1 Cons.prem2 tail-new])
next
  case (Some du)
  obtain updates-u ds1 where edge-rec:
    bmssp-relax-edges G settled u du ds = (updates-u, ds1)
  by force
  obtain updates-us ds2 where vertices-rec:
    bmssp-relax-vertices G settled us ds1 = (updates-us, ds2)
  by force
  have distinct-ds1: distinct (map fst ds1)
  using bmssp-relax-edges-preserves-distinct-dist[OF Cons.prem1,
    of G settled u du] edge-rec by simp
  obtain mid where mid: bmssp-lookup-dist ds1 v = Some mid
  proof -
    have ∃ d'. bmssp-lookup-dist

```

```

      (snd (bmssp-relax-edges G settled u du ds)) v = Some d'
    by (rule bmssp-relax-edges-lookup-Some-preserved
        [OF Cons.prem1 Cons.prem2])
  then obtain d' where bmssp-lookup-dist
    (snd (bmssp-relax-edges G settled u du ds)) v = Some d'
  by blast
  then have bmssp-lookup-dist ds1 v = Some d'
  using edge-rec by simp
  then show ?thesis
  by (rule that)
qed
have mid-le-old: mid ≤ old
proof -
  have bmssp-lookup-dist
    (snd (bmssp-relax-edges G settled u du ds)) v = Some mid
  using mid edge-rec by simp
  then show ?thesis
  by (rule bmssp-relax-edges-lookup-le[OF Cons.prem1 Cons.prem2])
qed
have final-new:
  bmssp-lookup-dist (snd (bmssp-relax-vertices G settled us ds1)) v =
    Some new
  using Cons.prem3 Some edge-rec vertices-rec by simp
have new-le-mid: new ≤ mid
  by (rule Cons.IH[OF distinct-ds1 mid final-new])
then show ?thesis
  using mid-le-old by linarith
qed
qed

lemma bmssp-relax-edges-edge-lookup-le-candidate:
  assumes edge: (u, v, w) ∈ set G
  and v-unsettled: v ∉ set settled
  shows ∃ dv.
    bmssp-lookup-dist (snd (bmssp-relax-edges G settled u du ds)) v =
      Some dv ∧
      dv ≤ du + w
  using edge
proof (induction G arbitrary: ds)
  case Nil
  then show ?case
  by simp
next
  case (Cons e es)
  obtain a b c where e: e = (a, b, c)
  by (cases e)
  obtain updates ds1 where rec:
    bmssp-relax-edges es settled u du ds = (updates, ds1)
  by force

```

```

show ?case
proof (cases (u, v, w) ∈ set es)
  case True
  then obtain dv where tail-lookup:
    bmssp-lookup-dist
    (snd (bmssp-relax-edges es settled u du ds)) v = Some dv
  and tail-le: dv ≤ du + w
  using Cons.IH by blast
have ds1-lookup: bmssp-lookup-dist ds1 v = Some dv
  using tail-lookup rec by simp
show ?thesis
proof (cases a = u ∧ b ∉ set settled)
  case False
  then have final-lookup:
    bmssp-lookup-dist
    (snd (bmssp-relax-edges (e # es) settled u du ds)) v =
    Some dv
  using rec ds1-lookup unfolding e
  by (auto simp: Let-def split: option.splits if-splits)
show ?thesis
  using final-lookup tail-le by blast
next
case True
note active = True
let ?nd = du + c
show ?thesis
proof (cases bmssp-lookup-dist ds1 b)
  case None
  show ?thesis
  proof (cases v = b)
    case True
    note v-eq-b = True
    then have final-lookup:
      bmssp-lookup-dist
      (snd (bmssp-relax-edges (e # es) settled u du ds)) v =
      Some ?nd
    using active v-eq-b None rec unfolding e
    by (simp add: Let-def bmssp-lookup-dist-set-dist-same)
  have ?nd ≤ du + w
    using v-eq-b None ds1-lookup by simp
  then show ?thesis
    using final-lookup by blast
  next
  case False
  note v-ne-b = False
  then have final-lookup:
    bmssp-lookup-dist
    (snd (bmssp-relax-edges (e # es) settled u du ds)) v =
    Some dv

```

```

    using active v-ne-b None rec ds1-lookup unfolding e
    by (simp add: Let-def bmssp-lookup-dist-set-dist-other)
  show ?thesis
    using final-lookup tail-le by blast
qed
next
case (Some old)
show ?thesis
proof (cases ?nd < old)
  case False
  then have final-lookup:
    bmssp-lookup-dist
    (snd (bmssp-relax-edges (e # es) settled u du ds)) v =
    Some dv
    using active Some rec ds1-lookup unfolding e by (simp add: Let-def)
  show ?thesis
    using final-lookup tail-le by blast
next
case True
note improves = True
show ?thesis
proof (cases v = b)
  case True
  note v-eq-b = True
  then have final-lookup:
    bmssp-lookup-dist
    (snd (bmssp-relax-edges (e # es) settled u du ds)) v =
    Some ?nd
    using active Some improves rec v-eq-b
    unfolding e
    by (simp add: Let-def bmssp-lookup-dist-set-dist-same)
  have ?nd ≤ du + w
  proof -
    have old-eq: old = dv
    using v-eq-b Some ds1-lookup by simp
    show ?thesis
    using improves old-eq tail-le by linarith
  qed
  then show ?thesis
    using final-lookup by blast
next
case False
note v-ne-b = False
then have final-lookup:
  bmssp-lookup-dist
  (snd (bmssp-relax-edges (e # es) settled u du ds)) v =
  Some dv
  using active Some improves rec ds1-lookup v-ne-b
  unfolding e

```

```

      by (simp add: Let-def bmssp-lookup-dist-set-dist-other)
    show ?thesis
      using final-lookup tail-le by blast
  qed
qed
qed
qed
next
case False
then have head:  $e = (u, v, w)$ 
  using Cons.premis by simp
have active:  $a = u \wedge b \notin \text{set settled}$ 
  using head v-unsettled unfolding e by simp
let ?nd =  $du + w$ 
show ?thesis
proof (cases bmssp-lookup-dist ds1 v)
case None
then have final-lookup:
  bmssp-lookup-dist
  ( $\text{snd } (\text{bmssp-relax-edges } (e \# \text{es}) \text{ settled } u \text{ du } ds)) \text{ } v =$ 
  Some ?nd
  using active rec head unfolding e
  by (simp add: Let-def bmssp-lookup-dist-set-dist-same)
show ?thesis
  using final-lookup by blast
next
case (Some old)
show ?thesis
proof (cases ?nd < old)
case True
then have final-lookup:
  bmssp-lookup-dist
  ( $\text{snd } (\text{bmssp-relax-edges } (e \# \text{es}) \text{ settled } u \text{ du } ds)) \text{ } v =$ 
  Some ?nd
  using active Some rec head unfolding e
  by (simp add: Let-def bmssp-lookup-dist-set-dist-same)
show ?thesis
  using final-lookup by blast
next
case False
then have final-lookup:
  bmssp-lookup-dist
  ( $\text{snd } (\text{bmssp-relax-edges } (e \# \text{es}) \text{ settled } u \text{ du } ds)) \text{ } v =$ 
  Some old
  using active Some rec head unfolding e by (simp add: Let-def)
have old  $\leq$  ?nd
  using False by simp
then show ?thesis
  using final-lookup by blast

```


qed
 qed
 qed
 qed

lemma *bmssp-relax-edges-update-improves-lookup*:
assumes *distinct*: *distinct* (map fst ds)
and *update*: $(v, b) \in \text{set } (\text{fst } (\text{bmssp-relax-edges } G \text{ settled } u \text{ du } ds))$
and *old*: *bmssp-lookup-dist* ds v = *Some old*
shows *nat* (floor b) < *old*
using *distinct update old*
proof (*induction* G *arbitrary*: ds v b *old*)
 case Nil
 then show ?case
 by simp
next
 case (Cons e es)
 obtain a c w **where** $e = (a, c, w)$
 by (cases e)
 obtain updates ds1 **where** rec:
 bmssp-relax-edges es settled u du ds = (updates, ds1)
 by force
 show ?case
proof (cases $a = u \wedge c \notin \text{set settled}$)
 case False
 have *update-tail*:
 $(v, b) \in \text{set } (\text{fst } (\text{bmssp-relax-edges } es \text{ settled } u \text{ du } ds))$
 using Cons.prem1(2) False rec **unfolding** e
 by (auto simp: Let-def split: option.splits if-splits)
 show ?thesis
 by (rule Cons.IH[OF Cons.prem1(1) *update-tail* Cons.prem1(3)])
next
 case True
 note active = True
 let ?nd = du + w
 show ?thesis
proof (cases *bmssp-lookup-dist* ds1 c)
 case None
 have *update-or-tail*:
 $(v, b) = (c, \text{bmssp-partition-key } c \text{ ?nd}) \vee$
 $(v, b) \in \text{set updates}$
 using Cons.prem1(2) active None rec **unfolding** e **by** (simp add: Let-def)
 then show ?thesis
proof
 assume *current*: $(v, b) = (c, \text{bmssp-partition-key } c \text{ ?nd})$
 have *old-c*: *bmssp-lookup-dist* ds c = *Some old*
 using Cons.prem1(3) *current* **by** simp
 have $\exists d'. \text{bmssp-lookup-dist}$
 $(\text{snd } (\text{bmssp-relax-edges } es \text{ settled } u \text{ du } ds)) \text{ } c = \text{Some } d'$

```

    by (rule bmssp-relax-edges-lookup-Some-preserved
        [OF Cons.prem(1) old-c])
  then show ?thesis
    using None rec by simp
next
  assume (v, b) ∈ set updates
  then have update-tail:
    (v, b) ∈ set (fst (bmssp-relax-edges es settled u du ds))
    using rec by simp
  show ?thesis
    by (rule Cons.IH[OF Cons.prem(1) update-tail Cons.prem(3)])
qed
next
case (Some cur)
show ?thesis
proof (cases ?nd < cur)
  case False
  have update-tail:
    (v, b) ∈ set (fst (bmssp-relax-edges es settled u du ds))
    using Cons.prem(2) active Some False rec unfolding e
    by (simp add: Let-def)
  show ?thesis
    by (rule Cons.IH[OF Cons.prem(1) update-tail Cons.prem(3)])
next
case True
  have update-or-tail:
    (v, b) = (c, bmssp-partition-key c ?nd) ∨
    (v, b) ∈ set updates
    using Cons.prem(2) active Some True rec unfolding e
    by (simp add: Let-def)
  then show ?thesis
  proof
    assume current: (v, b) = (c, bmssp-partition-key c ?nd)
    have old-c: bmssp-lookup-dist ds c = Some old
      using Cons.prem(3) current by simp
    have tail-cur:
      bmssp-lookup-dist
      (snd (bmssp-relax-edges es settled u du ds)) c = Some cur
      using Some rec by simp
    have cur-le-old: cur ≤ old
      by (rule bmssp-relax-edges-lookup-le
          [OF Cons.prem(1) old-c tail-cur])
    have nat (floor b) = ?nd
      using current by (simp add: bmssp-partition-key-floor)
    then show ?thesis
      using True cur-le-old by linarith
  next
    assume (v, b) ∈ set updates
    then have update-tail:

```

```

      (v, b) ∈ set (fst (bmssp-relax-edges es settled u du ds))
      using rec by simp
    show ?thesis
      by (rule Cons.IH[OF Cons.prem(1) update-tail Cons.prem(3)])
  qed
qed
qed
qed
qed
qed

lemma bmssp-relax-vertices-update-improves-lookup:
  assumes distinct: distinct (map fst ds)
  and update: (v, b) ∈ set (fst (bmssp-relax-vertices G settled xs ds))
  and old: bmssp-lookup-dist ds v = Some old
  shows nat (floor b) < old
  using distinct update old
proof (induction xs arbitrary: ds v b old)
  case Nil
  then show ?case
    by simp
next
  case (Cons u us)
  show ?case
  proof (cases bmssp-lookup-dist ds u)
    case None
    have update-tail:
      (v, b) ∈ set (fst (bmssp-relax-vertices G settled us ds))
      using Cons.prem(2) None by simp
    show ?thesis
      by (rule Cons.IH[OF Cons.prem(1) update-tail Cons.prem(3)])
  next
    case (Some du)
    obtain updates-u ds1 where edge-rec:
      bmssp-relax-edges G settled u du ds = (updates-u, ds1)
      by force
    obtain updates-us ds2 where vertices-rec:
      bmssp-relax-vertices G settled us ds1 = (updates-us, ds2)
      by force
    have update-cases: (v, b) ∈ set updates-u ∨ (v, b) ∈ set updates-us
      using Cons.prem(2) Some edge-rec vertices-rec by auto
    then show ?thesis
  proof
    assume (v, b) ∈ set updates-u
    then have edge-update:
      (v, b) ∈ set (fst (bmssp-relax-edges G settled u du ds))
      using edge-rec by simp
    show ?thesis
      by (rule bmssp-relax-edges-update-improves-lookup
        [OF Cons.prem(1) edge-update Cons.prem(3)])
  end
  end
end

```

```

next
  assume update-us: (v, b) ∈ set updates-us
  have distinct-ds1: distinct (map fst ds1)
    using bmssp-relax-edges-preserves-distinct-dist[OF Cons.prem1],
    of G settled u du] edge-rec by simp
  obtain mid where mid: bmssp-lookup-dist ds1 v = Some mid
  proof -
    have ∃ d'. bmssp-lookup-dist
      (snd (bmssp-relax-edges G settled u du ds)) v = Some d'
    by (rule bmssp-relax-edges-lookup-Some-preserved
      [OF Cons.prem1 Cons.prem3])
    then obtain d' where bmssp-lookup-dist
      (snd (bmssp-relax-edges G settled u du ds)) v = Some d'
    by blast
    then have bmssp-lookup-dist ds1 v = Some d'
      using edge-rec by simp
    then show ?thesis
      by (rule that)
  qed
  have mid-le-old: mid ≤ old
  proof -
    have bmssp-lookup-dist
      (snd (bmssp-relax-edges G settled u du ds)) v = Some mid
    using mid edge-rec by simp
    then show ?thesis
      by (rule bmssp-relax-edges-lookup-le
        [OF Cons.prem1 Cons.prem3])
  qed
  have update-tail:
    (v, b) ∈ set (fst (bmssp-relax-vertices G settled us ds1))
  using update-us vertices-rec by simp
  have nat (floor b) < mid
  by (rule Cons.IH[OF distinct-ds1 update-tail mid])
  then show ?thesis
    using mid-le-old by linarith
  qed
qed
qed
qed

lemma bmssp-relax-edges-updates-distinct:
  assumes distinct-edges: distinct (nat-graph-edge-list G)
  shows distinct (map fst (fst (bmssp-relax-edges G settled u du ds)))
  using distinct-edges
proof (induction G arbitrary: ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons e es)

```

```

obtain  $a\ c\ w$  where  $e: e = (a, c, w)$ 
  by (cases  $e$ )
obtain  $updates\ ds1$  where  $rec:$ 
   $bmssp-relax-edges\ es\ settled\ u\ du\ ds = (updates, ds1)$ 
  by force
have  $tail-edges: distinct\ (nat-graph-edge-list\ es)$ 
  using  $Cons.premis\ unfolding\ e\ nat-graph-edge-list-def$  by simp
have  $tail-distinct: distinct\ (map\ fst\ updates)$ 
  using  $Cons.IH[OF\ tail-edges, of\ ds]\ rec$  by simp
have  $c-notin-tail:$ 
   $a = u \implies c \notin fst\ 'set\ updates$ 
proof
  assume  $a-u: a = u$  and  $c-tail: c \in fst\ 'set\ updates$ 
  then obtain  $b$  where  $update: (c, b) \in set\ updates$ 
  by force
  have  $update-rec:$ 
     $(c, b) \in set\ (fst\ (bmssp-relax-edges\ es\ settled\ u\ du\ ds))$ 
    using  $update\ rec$  by simp
  have  $(u, c) \in set\ (nat-graph-edge-list\ es)$ 
    by (rule  $bmssp-relax-edges-update-edge[OF\ update-rec]$ )
  then show False
    using  $Cons.premis\ a-u\ unfolding\ e\ nat-graph-edge-list-def$  by simp
qed
show ?case
proof (cases  $a = u \wedge c \notin set\ settled$ )
  case False
    then show ?thesis
      using  $tail-distinct$  by (auto simp: e rec split: option.splits if-splits)
  next
    case True
    note  $active = True$ 
    let  $?nd = du + w$ 
    have  $c-notin: c \notin fst\ 'set\ updates$ 
      using  $c-notin-tail\ active$  by blast
    show ?thesis
    proof (cases  $bmssp-lookup-dist\ ds1\ c$ )
      case None
        then show ?thesis
          using  $tail-distinct\ c-notin\ active\ unfolding\ e$ 
          by (simp add: rec Let-def)
      next
        case (Some  $old$ )
        show ?thesis
        proof (cases  $?nd < old$ )
          case False
            then show ?thesis
              using  $tail-distinct\ active\ Some\ unfolding\ e$  by (simp add: rec Let-def)
          next
            case True

```

```

    then show ?thesis
      using tail-distinct c-notin active Some unfolding e
      by (simp add: rec Let-def)
  qed
qed
qed
qed

lemma bmssp-relax-vertices-singleton-updates-distinct:
  assumes distinct-edges: distinct (nat-graph-edge-list G)
  shows distinct
    (map fst (fst (bmssp-relax-vertices G settled [u] ds)))
  using bmssp-relax-edges-updates-distinct[OF distinct-edges]
  by (cases bmssp-lookup-dist ds u) (auto split: prod.splits)

lemma bmssp-relax-vertices-updates-distinct-if-length-le-one:
  assumes distinct-edges: distinct (nat-graph-edge-list G)
  and len: length xs ≤ 1
  shows distinct (map fst (fst (bmssp-relax-vertices G settled xs ds)))
proof (cases xs)
  case Nil
  then show ?thesis
    by simp
next
  case (Cons u us)
  then have us = []
    using len by (cases us) simp-all
  then show ?thesis
    using Cons bmssp-relax-vertices-singleton-updates-distinct[OF distinct-edges]
    by simp
qed

lemma bmssp-relax-vertices-pulled-updates-distinct:
  assumes wf: nat-graph-well-formed G
  and distinct-vertices: distinct vertices
  and pull: pull-separates (bp-view P) bmssp-block-size B S beta D'
  shows distinct
    (map fst
      (fst (bmssp-relax-vertices G settled
        (filter (λx. x ∈ S ∧ x ∉ set old-settled) vertices) ds)))
proof -
  have len:
    length (filter (λx. x ∈ S ∧ x ∉ set old-settled) vertices)
      ≤ 1
  by (rule bmssp-pulled-length-le-one[OF distinct-vertices pull])
  have distinct-edges: distinct (nat-graph-edge-list G)
  by (rule nat-graph-well-formed-distinct-edge-list[OF wf])
  show ?thesis
  by (rule bmssp-relax-vertices-updates-distinct-if-length-le-one

```

[OF distinct-edges len])

qed

lemma *bmssp-apply-updates-ordered-after-pull*:
fixes *old-settled settled* :: nat list
assumes *wf*: nat-graph-well-formed *G*
and *distinct-vertices*: distinct vertices
and *ord*: bp-ordered-invariant *P*
and *upper*: partition-upper-bound (bp-view *P*) *B*
and *pull*: bp-pull bmssp-block-size *B P* = (*S*, beta, *P1*)
shows bp-ordered-invariant
 (bmssp-apply-updates
 (fst (bmssp-relax-vertices *G settled*
 (filter (λ*x*. *x* ∈ *S* ∧ *x* ∉ set old-settled) vertices) *ds*))
P1)

proof –
let *?pulled* =
 filter (λ*x*. *x* ∈ *S* ∧ *x* ∉ set old-settled) vertices
let *?updates* = fst (bmssp-relax-vertices *G settled ?pulled ds*)
have *sep*: pull-separates (bp-view *P*) bmssp-block-size *B S* beta (bp-view *P1*)
by (rule bmssp-pull-refines-pull-separates[OF ord upper pull])
have *P1-ord*: bp-ordered-invariant *P1*
by (rule bmssp-pull-ordered-invariant[OF ord pull])
have *distinct-updates*: distinct (map fst *?updates*)
by (rule bmssp-relax-vertices-pulled-updates-distinct
 [OF wf distinct-vertices sep])
show *?thesis*
by (rule bmssp-apply-updates-ordered-invariant-from-ordered
 [OF *P1-ord distinct-updates*])

qed

lemma *bmssp-partition-key-zero-lt-bound* [simp]:
 bmssp-partition-key *s* 0 < bmssp-bound
proof –
have bmssp-partition-key *s* 0 < real (Suc 0)
by (rule bmssp-partition-key-lt-suc-distance)
then show *?thesis*
 unfolding bmssp-bound-def bmssp-infinity-def **by** simp

qed

lemma *bmssp-partition-key-lt-bound-if-distance-lt*:
assumes *d* < bmssp-infinity
shows bmssp-partition-key *v d* < bmssp-bound
proof –
have bmssp-partition-key *v d* < real (Suc *d*)
by (rule bmssp-partition-key-lt-suc-distance)
also have ... ≤ real bmssp-infinity
using *assms* **by** simp
finally show *?thesis*

```

    unfolding bmssp-bound-def .
qed

lemma bmssp-initial-partition-bridge:
  shows bp-ordered-invariant
    (bp-result-of
      (c-bp-regularized-local-insert s (bmssp-partition-key s 0)
        (bp-empty bmssp-block-size bmssp-bound)))
  and partition-upper-bound
    (bp-view
      (bp-result-of
        (c-bp-regularized-local-insert s (bmssp-partition-key s 0)
          (bp-empty bmssp-block-size bmssp-bound))))
    bmssp-bound
  and bp-view
    (bp-result-of
      (c-bp-regularized-local-insert s (bmssp-partition-key s 0)
        (bp-empty bmssp-block-size bmssp-bound))) =
    min-update (bp-view (bp-empty bmssp-block-size bmssp-bound))
      s (bmssp-partition-key s 0)
proof -
  let ?P0 = bp-empty bmssp-block-size bmssp-bound
  let ?P1 =
    bp-result-of
      (c-bp-regularized-local-insert s (bmssp-partition-key s 0) ?P0)
  have block-pos: 0 < bmssp-block-size
    unfolding bmssp-block-size-def by simp
  have reg0: bp-regular-invariant ?P0
    by (rule bp-empty-regular-invariant[OF block-pos])
  have reg1: bp-regular-invariant ?P1
    by (rule c-bp-regularized-local-insert-regular-invariant[OF reg0])
  show bp-ordered-invariant ?P1
    by (rule bp-regular-invariant-ordered-invariant[OF reg1])
  have view:
    bp-view ?P1 = min-update (bp-view ?P0) s (bmssp-partition-key s 0)
    by (rule c-bp-regularized-local-insert-refines-min-update[OF reg0])
  show bp-view ?P1 =
    min-update (bp-view ?P0) s (bmssp-partition-key s 0)
    by (rule view)
  have upper0: partition-upper-bound (bp-view ?P0) bmssp-bound
    unfolding partition-upper-bound-def by simp
  have partition-upper-bound
    (min-update (bp-view ?P0) s (bmssp-partition-key s 0)) bmssp-bound
    by (rule min-update-preserves-upper-bound[OF upper0])
    (rule bmssp-partition-key-zero-lt-bound)
  then show partition-upper-bound (bp-view ?P1) bmssp-bound
    unfolding view .
qed

```


lemma *bmssp-loop-partition-step-bridge*:

fixes *old-settled settled vertices* :: nat list

and *pulled* :: nat list

and *updates* :: (nat $\times$ real) list

assumes *pulled-def*:

pulled =

filter ($\lambda x. x \in S \wedge x \notin \text{set old-settled}$) *vertices*

and *updates-def*: *updates* = *fst* (*bmssp-relax-vertices* *G* *settled* *pulled* *ds*)

and *P2-def*: *P2* = *bmssp-apply-updates* *updates* *P1*

and *wf*: *nat-graph-well-formed* *G*

and *distinct-vertices*: *distinct* *vertices*

and *ord*: *bp-ordered-invariant* *P*

and *upper*: *partition-upper-bound* (*bp-view* *P*) *B*

and *pull*: *bp-pull* *bmssp-block-size* *B* *P* = (*S*, *beta*, *P1*)

shows *pull-separates* (*bp-view* *P*) *bmssp-block-size* *B* *S* *beta* (*bp-view* *P1*)

and *bp-ordered-invariant* *P1*

and *partition-upper-bound* (*bp-view* *P1*) *B*

and *length* *pulled* ≤ 1

and *distinct* (*map* *fst* *updates*)

and ($\bigwedge x b. (x, b) \in \text{set updates} \implies$

$\exists d. b = \text{bmssp-partition-key } x \ d \wedge \text{nat } (\text{floor } b) = d$)

and *bp-view* *P2* = *batch-min-update* (*bp-view* *P1*) *updates*

and *bp-ordered-invariant* *P2*

and ($\bigwedge x b. (x, b) \in \text{set updates} \implies b < B$) $\implies$

partition-upper-bound (*bp-view* *P2*) *B*

proof –

have *sep*:

pull-separates (*bp-view* *P*) *bmssp-block-size* *B* *S* *beta* (*bp-view* *P1*)

by (*rule* *bmssp-pull-refines-pull-separates*[*OF* *ord* *upper* *pull*])

have *P1-ord*: *bp-ordered-invariant* *P1*

by (*rule* *bmssp-pull-ordered-invariant*[*OF* *ord* *pull*])

have *P1-upper*: *partition-upper-bound* (*bp-view* *P1*) *B*

by (*rule* *bmssp-pull-preserves-upper-bound*[*OF* *ord* *upper* *pull*])

have *pulled-len*: *length* *pulled* ≤ 1

unfolding *pulled-def*

by (*rule* *bmssp-pulled-length-le-one*[*OF* *distinct-vertices* *sep*])

have *updates-distinct*: *distinct* (*map* *fst* *updates*)

unfolding *updates-def* *pulled-def*

by (*rule* *bmssp-relax-vertices-pulled-updates-distinct*

[*OF* *wf* *distinct-vertices* *sep*])

have *updates-floor*:

$\bigwedge x b. (x, b) \in \text{set updates} \implies$

$\exists d. b = \text{bmssp-partition-key } x \ d \wedge \text{nat } (\text{floor } b) = d$

proof –

fix *x b*

assume $(x, b) \in \text{set updates}$

then have $(x, b) \in$

set (*fst* (*bmssp-relax-vertices* *G* *settled* *pulled* *ds*))

unfolding *updates-def* .

then show $\exists d. b = \text{bmssp-partition-key } x \ d \wedge \text{nat } (\text{floor } b) = d$
by (rule *bmssp-relax-vertices-update-floor*)
qed
have *view*:
 $\text{bp-view } P2 = \text{batch-min-update } (\text{bp-view } P1) \ \text{updates}$
unfolding *P2-def*
by (rule *bmssp-apply-updates-refines-batch-min-update-from-ordered*
 $[\text{OF } P1\text{-ord updates-distinct}]$)
have *P2-ord*: *bp-ordered-invariant P2*
unfolding *P2-def*
by (rule *bmssp-apply-updates-ordered-invariant-from-ordered*
 $[\text{OF } P1\text{-ord updates-distinct}]$)
show *pull-separates* (*bp-view P*) *bmssp-block-size B S beta* (*bp-view P1*)
by (rule *sep*)
show *bp-ordered-invariant P1*
by (rule *P1-ord*)
show *partition-upper-bound* (*bp-view P1*) *B*
by (rule *P1-upper*)
show *length pulled* ≤ 1
by (rule *pulled-len*)
show *distinct* (*map fst updates*)
by (rule *updates-distinct*)
show $\bigwedge x \ b. (x, b) \in \text{set updates} \implies$
 $\exists d. b = \text{bmssp-partition-key } x \ d \wedge \text{nat } (\text{floor } b) = d$
by (rule *updates-floor*)
show $\text{bp-view } P2 = \text{batch-min-update } (\text{bp-view } P1) \ \text{updates}$
by (rule *view*)
show *bp-ordered-invariant P2*
by (rule *P2-ord*)
show $(\bigwedge x \ b. (x, b) \in \text{set updates} \implies b < B) \implies$
 $\text{partition-upper-bound } (\text{bp-view } P2) \ B$
unfolding *P2-def*
by (rule *bmssp-apply-updates-preserves-upper-bound-from-ordered*
 $[\text{OF } P1\text{-ord } P1\text{-upper updates-distinct}]$)
qed

definition *bmssp-partition-keys-match* ::
 $\text{nat list} \Rightarrow \text{nat-dist} \Rightarrow \text{nat bucketed-partition} \Rightarrow \text{bool}$ **where**
 $\text{bmssp-partition-keys-match settled ds } P \longleftrightarrow$
 $\text{keys-of } (\text{bp-view } P) = \text{set } (\text{map fst ds}) - \text{set settled}$

lemma *bmssp-partition-keys-match-initial*:
 $\text{bmssp-partition-keys-match } [] \ [(\text{src}, 0)]$
 $(\text{bp-result-of}$
 $\ (\text{c-bp-regularized-local-insert src } (\text{bmssp-partition-key src } 0)$
 $\ (\text{bp-empty bmssp-block-size bmssp-bound})))$

proof –

have *view*:
 bp-view

```

    (bp-result-of
      (c-bp-regularized-local-insert src (bmssp-partition-key src 0)
        (bp-empty bmssp-block-size bmssp-bound))) =
    min-update (bp-view (bp-empty bmssp-block-size bmssp-bound))
      src (bmssp-partition-key src 0)
  by (rule bmssp-initial-partition-bridge(3))
show ?thesis
  unfolding bmssp-partition-keys-match-def view by simp
qed

lemma bmssp-filter-pulled-set-eq:
  assumes  $S$ -vertices:  $S \subseteq \text{set vertices}$ 
  and  $S$ -unsettled:  $S \cap \text{set settled} = \{\}$ 
  shows  $\text{set (filter } (\lambda x. x \in S \wedge x \notin \text{set settled}) \text{ vertices})} = S$ 
proof
  show  $\text{set (filter } (\lambda x. x \in S \wedge x \notin \text{set settled}) \text{ vertices})} \subseteq S$ 
  by auto
next
  show  $S \subseteq \text{set (filter } (\lambda x. x \in S \wedge x \notin \text{set settled}) \text{ vertices})}$ 
proof
  fix  $x$ 
  assume  $xS$ :  $x \in S$ 
  then have  $x \in \text{set vertices}$ 
  using  $S$ -vertices by blast
  moreover have  $x \notin \text{set settled}$ 
  using  $S$ -unsettled  $xS$  by blast
  ultimately show  $x \in \text{set (filter } (\lambda x. x \in S \wedge x \notin \text{set settled}) \text{ vertices})}$ 
  using  $xS$  by simp
qed
qed

lemma bmssp-partition-keys-match-step:
  fixes old-settled settled vertices :: nat list
  and pulled :: nat list
  and updates :: (nat  $\times$  real) list
  assumes match: bmssp-partition-keys-match old-settled ds  $P$ 
  and keys-vertices: keys-of (bp-view  $P$ )  $\subseteq \text{set vertices}$ 
  and pulled-def:
    pulled =
      filter ( $\lambda x. x \in S \wedge x \notin \text{set old-settled}$ ) vertices
  and settled-def: settled = pulled @ old-settled
  and relaxed: bmssp-relax-vertices  $G$  settled pulled ds = (updates, ds')
  and pull: pull-separates (bp-view  $P$ ) bmssp-block-size  $B$   $S$  beta
    (bp-view  $P1$ )
  and view: bp-view  $P2$  = batch-min-update (bp-view  $P1$ ) updates
  shows bmssp-partition-keys-match settled ds'  $P2$ 

```

```

proof –
  have  $S\text{-subset-keys}$ :  $S \subseteq \text{keys-of } (bp\text{-view } P)$ 
    by (rule pull-separates-subset[OF pull])
  have  $S\text{-vertices}$ :  $S \subseteq \text{set vertices}$ 
    using  $S\text{-subset-keys}$   $\text{keys-vertices}$  by blast
  have  $S\text{-unsettled}$ :  $S \cap \text{set old-settled} = \{\}$ 
    using  $S\text{-subset-keys}$   $\text{match unfolding bmssp-partition-keys-match-def}$  by auto
  have  $\text{pulled-set}$ :  $\text{set pulled} = S$ 
    unfolding  $\text{pulled-def}$ 
    by (rule bmssp-filter-pulled-set-eq[OF  $S\text{-vertices}$   $S\text{-unsettled}$ ])
  have  $\text{settled-set}$ :  $\text{set settled} = S \cup \text{set old-settled}$ 
    unfolding  $\text{settled-def}$  using  $\text{pulled-set}$  by auto
  have  $P1\text{-keys}$ :  $\text{keys-of } (bp\text{-view } P1) = \text{keys-of } (bp\text{-view } P) - S$ 
    by (rule pull-separates-remaining-keys[OF pull])
  have  $P2\text{-keys}$ :
     $\text{keys-of } (bp\text{-view } P2) =$ 
       $(\text{set } (\text{map fst } ds) - \text{set old-settled} - S) \cup \text{fst ' set updates}$ 
    using  $\text{match view } P1\text{-keys}$ 
    unfolding  $\text{bmssp-partition-keys-match-def}$ 
    by (auto simp: batch-min-update-keys)
  have  $ds'\text{-keys}$ :
     $\text{set } (\text{map fst } ds') = \text{set } (\text{map fst } ds) \cup \text{fst ' set updates}$ 
    using  $\text{bmssp-relax-vertices-dist-keys[of } G \text{ settled pulled } ds] \text{ relaxed}$ 
    by simp
  have  $\text{updates-unsettled}$ :
     $\text{fst ' set updates} \cap \text{set settled} = \{\}$ 
proof
  show  $\text{fst ' set updates} \cap \text{set settled} \subseteq \{\}$ 
proof
  fix  $x$ 
  assume  $x: x \in \text{fst ' set updates} \cap \text{set settled}$ 
  then obtain  $b$  where  $xb: (x, b) \in \text{set updates}$ 
    by auto
  have  $(x, b) \in \text{set } (\text{fst } (\text{bmssp-relax-vertices } G \text{ settled pulled } ds))$ 
    using  $xb \text{ relaxed}$  by simp
  then have  $x \notin \text{set settled}$ 
    by (rule bmssp-relax-vertices-update-not-settled)
  then show  $x \in \{\}$ 
    using  $x$  by simp
qed
  show  $\{\} \subseteq \text{fst ' set updates} \cap \text{set settled}$ 
    by simp
qed
  have  $\text{set } (\text{map fst } ds') - \text{set settled} =$ 
     $(\text{set } (\text{map fst } ds) - \text{set old-settled} - S) \cup \text{fst ' set updates}$ 
    using  $ds'\text{-keys}$   $\text{settled-set updates-unsettled}$  by auto
  then show ?thesis
    using  $P2\text{-keys}$  unfolding  $\text{bmssp-partition-keys-match-def}$  by simp
qed

```

lemma *bmssp-partition-keys-match-loop-step-bridge*:
fixes *old-settled settled vertices* :: nat list
and *pulled* :: nat list
and *updates* :: (nat × real) list
assumes *match*: *bmssp-partition-keys-match old-settled ds P*
and *keys-vertices*: *keys-of (bp-view P) ⊆ set vertices*
and *pulled-def*:
 pulled =
 filter (λx. x ∈ S ∧ x ∉ set old-settled) vertices
and *settled-def*: *settled* = *pulled @ old-settled*
and *relaxed*: *bmssp-relax-vertices G settled pulled ds = (updates, ds')*
and *P2-def*: *P2 = bmssp-apply-updates updates P1*
and *wf*: *nat-graph-well-formed G*
and *distinct-vertices*: *distinct vertices*
and *ord*: *bp-ordered-invariant P*
and *upper*: *partition-upper-bound (bp-view P) B*
and *pull*: *bp-pull bmssp-block-size B P = (S, beta, P1)*
shows *bmssp-partition-keys-match settled ds' P2*
proof –
 have *updates-def*: *updates = fst (bmssp-relax-vertices G settled pulled ds)*
 using *relaxed* **by** *simp*
 have *sep*:
 pull-separates (bp-view P) bmssp-block-size B S beta (bp-view P1)
 by (*rule bmssp-loop-partition-step-bridge(1)*
 [*OF pulled-def updates-def P2-def wf distinct-vertices ord upper pull*])
 have *view*: *bp-view P2 = batch-min-update (bp-view P1) updates*
 by (*rule bmssp-loop-partition-step-bridge(7)*
 [*OF pulled-def updates-def P2-def wf distinct-vertices ord upper pull*])
 show *?thesis*
 by (*rule bmssp-partition-keys-match-step*
 [*OF match keys-vertices pulled-def settled-def relaxed sep view*])
qed

lemma *batch-min-update-value-not-updated*:
assumes *x ∉ fst 'set xs*
shows *value-of (batch-min-update D xs) x = value-of D x*
using *assms*
proof (*induction xs arbitrary: D*)
 case *Nil*
 then show *?case*
 unfolding *batch-min-update-def* **by** *simp*
next
 case (*Cons xb xs*)
 obtain *y b* **where** *xb: xb = (y, b)*
 by *force*
 have *x-ne-y*: *x ≠ y*
 using *Cons.prem*s **unfolding** *xb* **by** *simp*
 have *x-not-tail*: *x ∉ fst 'set xs*

```

    using Cons.premis unfolding xb by simp
  have tail:
    value-of (batch-min-update (min-update D y b) xs) x =
      value-of (min-update D y b) x
    by (rule Cons.IH[OF x-not-tail])
  have step: value-of (min-update D y b) x = value-of D x
    using min-update-value-same[OF x-ne-y, of D b] .
  show ?case
  proof -
    have value-of (batch-min-update D (xb # xs)) x =
      value-of (batch-min-update (min-update D y b) xs) x
      unfolding batch-min-update-def xb by simp
    also have ... = value-of (min-update D y b) x
      by (rule tail)
    also have ... = value-of D x
      by (rule step)
    finally show ?case .
  qed
qed

lemma batch-min-update-value-pair-less-old:
  assumes distinct: distinct (map fst xs)
    and pair: (x, b) ∈ set xs
    and better: x ∈ keys-of D ⇒ b < value-of D x
  shows value-of (batch-min-update D xs) x = b
    using distinct pair better
  proof (induction xs arbitrary: D)
    case Nil
    then show ?case
      by simp
  next
    case (Cons xb xs)
    obtain y c where xb: xb = (y, c)
      by force
    have distinct-tail: distinct (map fst xs)
      using Cons.premis(1) unfolding xb by simp
    show ?case
    proof (cases x = y)
      case True
      have x-not-tail: x ∉ fst ` set xs
        using Cons.premis(1) True unfolding xb by simp
      have not-pair-tail: (x, b) ∉ set xs
        using x-not-tail by force
      have c-eq: c = b
      proof -
        have (x, b) = (y, c)
          using Cons.premis(2) not-pair-tail unfolding xb by auto
        then show ?thesis
          using True by simp
      qed
    qed
  qed

```

```

qed
have step: value-of (min-update D y c) x = b
proof (cases x ∈ keys-of D)
  case True
  then have c < value-of D x
    using Cons.prem3 c-eq by simp
  then show ?thesis
    using True ⟨x = y⟩ c-eq min-update-value-key-old[of x D c]
    by simp
next
  case False
  then show ?thesis
    using ⟨x = y⟩ c-eq min-update-value-key-new[of y D c]
    by simp
qed
have tail:
  value-of (batch-min-update (min-update D y c) xs) x =
  value-of (min-update D y c) x
  by (rule batch-min-update-value-not-updated[OF x-not-tail])
show ?thesis
proof -
  have value-of (batch-min-update D (xb # xs)) x =
    value-of (batch-min-update (min-update D y c) xs) x
    unfolding batch-min-update-def xb by simp
  also have ... = value-of (min-update D y c) x
    by (rule tail)
  also have ... = b
    by (rule step)
  finally show ?thesis .
qed
next
  case False
  have pair-tail: (x, b) ∈ set xs
    using Cons.prem2 False unfolding xb by auto
  have better-tail:
    x ∈ keys-of (min-update D y c) ⟹
    b < value-of (min-update D y c) x
  proof -
    assume x-in: x ∈ keys-of (min-update D y c)
    then have x ∈ keys-of D
      using False by simp
    then have b < value-of D x
      by (rule Cons.prem3)
    moreover have value-of (min-update D y c) x = value-of D x
      using min-update-value-same[OF False, of D c] .
    ultimately show b < value-of (min-update D y c) x
      by simp
  qed
  have tail:

```

```

    value-of (batch-min-update (min-update D y c) xs) x = b
  by (rule Cons.IH[OF distinct-tail pair-tail better-tail])
show ?thesis
proof -
  have value-of (batch-min-update D (xb # xs)) x =
    value-of (batch-min-update (min-update D y c) xs) x
  unfolding batch-min-update-def xb by simp
  also have ... = b
  by (rule tail)
  finally show ?thesis .
qed
qed
qed

```

definition *bmssp-partition-values-match* ::
 $\text{nat list} \Rightarrow \text{nat-dist} \Rightarrow \text{nat partition-view} \Rightarrow \text{bool}$ **where**
bmssp-partition-values-match settled ds D $\longleftrightarrow$
 $(\forall v d. \text{bmssp-lookup-dist ds } v = \text{Some } d \longrightarrow$
 $v \notin \text{set settled} \longrightarrow \text{value-of } D \ v = \text{bmssp-partition-key } v \ d)$

lemma *bmssp-partition-values-match-initial*:
bmssp-partition-values-match [] [(src, 0)]
 (bp-view
 (bp-result-of
 (c-bp-regularized-local-insert src (bmssp-partition-key src 0)
 (bp-empty bmssp-block-size bmssp-bound))))

proof –
 have view:
 bp-view
 (bp-result-of
 (c-bp-regularized-local-insert src (bmssp-partition-key src 0)
 (bp-empty bmssp-block-size bmssp-bound))) =
 min-update (bp-view (bp-empty bmssp-block-size bmssp-bound))
 src (bmssp-partition-key src 0)
 by (rule bmssp-initial-partition-bridge(3))
 show ?thesis
 unfolding bmssp-partition-values-match-def view
 by (auto simp: min-update-def)
 qed

lemma *bmssp-partition-values-match-pull*:
bmssp-partition-values-match settled ds D $\implies$
 value-of Dp = value-of D $\implies$
bmssp-partition-values-match settled ds Dp
 unfolding bmssp-partition-values-match-def by simp

lemma *bmssp-partition-values-match-step*:
 fixes old-settled settled vertices :: nat list
 and pulled :: nat list


```

    and updates :: (nat × real) list
  assumes vals: bmssp-partition-values-match old-settled ds (bp-view P)
    and keys: bmssp-partition-keys-match old-settled ds P
    and distinct-ds: distinct (map fst ds)
    and pulled-def:
      pulled =
        filter (λx. x ∈ S ∧ x ∉ set old-settled) vertices
    and settled-def: settled = pulled @ old-settled
    and relaxed: bmssp-relax-vertices G settled pulled ds = (updates, ds')
    and distinct-updates: distinct (map fst updates)
    and pull: pull-separates (bp-view P) bmssp-block-size B S beta
      (bp-view P1)
    and view: bp-view P2 = batch-min-update (bp-view P1) updates
  shows bmssp-partition-values-match settled ds' (bp-view P2)
proof -
  have pull-values: value-of (bp-view P1) = value-of (bp-view P)
    using pull unfolding pull-separates-def by simp
  show ?thesis
    unfolding bmssp-partition-values-match-def
  proof (intro allI impI)
    fix v d
    assume lookup': bmssp-lookup-dist ds' v = Some d
    assume v-unsettled: v ∉ set settled
    show value-of (bp-view P2) v = bmssp-partition-key v d
    proof (cases v ∈ fst 'set updates')
      case True
      then obtain b where update: (v, b) ∈ set updates
        by auto
      have update-rel:
        (v, b) ∈ set (fst (bmssp-relax-vertices G settled pulled ds))
        using update relaxed by simp
      have distinct-rel:
        distinct (map fst (fst (bmssp-relax-vertices G settled pulled ds)))
        using distinct-updates relaxed by simp
      have lookup-floor:
        bmssp-lookup-dist ds' v = Some (nat (floor b))
        using bmssp-relax-vertices-update-lookup-floor
          [OF distinct-rel update-rel] relaxed by simp
      have d-floor: d = nat (floor b)
        using lookup' lookup-floor by simp
      obtain du where b-key: b = bmssp-partition-key v du
        and du-floor: nat (floor b) = du
        using bmssp-relax-vertices-update-floor[OF update-rel] by blast
      have b-d: b = bmssp-partition-key v d
        using b-key du-floor d-floor by simp
      have better:
        v ∈ keys-of (bp-view P1) ⇒ b < value-of (bp-view P1) v
    proof -
      assume vP1: v ∈ keys-of (bp-view P1)

```

```

have P1-keys: keys-of (bp-view P1) = keys-of (bp-view P) - S
  by (rule pull-separates-remaining-keys[OF pull])
have vP: v ∈ keys-of (bp-view P)
  using vP1 P1-keys by blast
have v-key-ds: v ∈ set (map fst ds)
  using keys vP unfolding bmssp-partition-keys-match-def by blast
have v-not-old: v ∉ set old-settled
  using keys vP unfolding bmssp-partition-keys-match-def by blast
obtain old where old-pair: (v, old) ∈ set ds
  using v-key-ds by force
have old-lookup: bmssp-lookup-dist ds v = Some old
  by (rule bmssp-lookup-dist-Some-if-distinct-mem
    [OF distinct-ds old-pair])
have old-value:
  value-of (bp-view P) v = bmssp-partition-key v old
  using vals old-lookup v-not-old
  unfolding bmssp-partition-values-match-def by blast
have du-lt-old: du < old
  using bmssp-relax-vertices-update-improves-lookup
    [OF distinct-ds update-rel old-lookup] du-floor by simp
have bmssp-partition-key v du < bmssp-partition-key v old
  by (rule bmssp-partition-key-strict-mono-distance[OF du-lt-old])
then have b < value-of (bp-view P) v
  using b-key old-value by simp
then show b < value-of (bp-view P1) v
  using pull-values by simp
qed
have value-of (bp-view P2) v = b
  unfolding view
  by (rule batch-min-update-value-pair-less-old
    [OF distinct-updates update better])
then show ?thesis
  using b-d by simp
next
case False
have lookup-same: bmssp-lookup-dist ds' v = bmssp-lookup-dist ds v
  using bmssp-relax-vertices-lookup-not-updated[of v G settled pulled ds]
    False relaxed by simp
have lookup-old: bmssp-lookup-dist ds v = Some d
  using lookup' lookup-same by simp
have v-not-old: v ∉ set old-settled
  using v-unsettled settled-def by auto
have old-value:
  value-of (bp-view P) v = bmssp-partition-key v d
  using vals lookup-old v-not-old
  unfolding bmssp-partition-values-match-def by blast
have p2-p1: value-of (bp-view P2) v = value-of (bp-view P1) v
  unfolding view
  by (rule batch-min-update-value-not-updated[OF False])

```

```

    show ?thesis
    using p2-p1 pull-values old-value by simp
  qed
qed
qed

lemma bmssp-partition-values-match-loop-step-bridge:
  fixes old-settled settled vertices :: nat list
  and pulled :: nat list
  and updates :: (nat × real) list
  assumes vals: bmssp-partition-values-match old-settled ds (bp-view P)
  and keys: bmssp-partition-keys-match old-settled ds P
  and distinct-ds: distinct (map fst ds)
  and pulled-def:
    pulled =
      filter (λx. x ∈ S ∧ x ∉ set old-settled) vertices
  and settled-def: settled = pulled @ old-settled
  and relaxed: bmssp-relax-vertices G settled pulled ds = (updates, ds')
  and P2-def: P2 = bmssp-apply-updates updates P1
  and wf: nat-graph-well-formed G
  and distinct-vertices: distinct vertices
  and ord: bp-ordered-invariant P
  and upper: partition-upper-bound (bp-view P) B
  and pull: bp-pull bmssp-block-size B P = (S, beta, P1)
  shows bmssp-partition-values-match settled ds' (bp-view P2)
proof -
  have updates-def: updates = fst (bmssp-relax-vertices G settled pulled ds)
  using relaxed by simp
  have sep:
    pull-separates (bp-view P) bmssp-block-size B S beta (bp-view P1)
  by (rule bmssp-loop-partition-step-bridge(1)
    [OF pulled-def updates-def P2-def wf distinct-vertices ord upper pull])
  have distinct-updates: distinct (map fst updates)
  by (rule bmssp-loop-partition-step-bridge(5)
    [OF pulled-def updates-def P2-def wf distinct-vertices ord upper pull])
  have view: bp-view P2 = batch-min-update (bp-view P1) updates
  by (rule bmssp-loop-partition-step-bridge(7)
    [OF pulled-def updates-def P2-def wf distinct-vertices ord upper pull])
  show ?thesis
  by (rule bmssp-partition-values-match-step
    [OF vals keys distinct-ds pulled-def settled-def relaxed
      distinct-updates sep view])
qed

lemma bmssp-partition-key-le-imp-distance-le:
  assumes bmssp-partition-key u du ≤ bmssp-partition-key v dv
  shows du ≤ dv
proof (rule ccontr)
  assume ¬ du ≤ dv

```

```

then have  $dv < du$ 
  by simp
then have  $\text{bmssp-partition-key } v \text{ } dv < \text{bmssp-partition-key } u \text{ } du$ 
  by (rule  $\text{bmssp-partition-key-strict-mono-distance}$ )
then show False
  using assms by linarith
qed

lemma  $\text{bmssp-pull-residual-label-le}$ :
  assumes vals:  $\text{bmssp-partition-values-match old-settled ds (bp-view } P)$ 
    and keys:  $\text{bmssp-partition-keys-match old-settled ds } P$ 
    and pull:  $\text{pull-separates (bp-view } P) \text{ bmssp-block-size } B \text{ } S \text{ beta}$ 
       $(\text{bp-view } P1)$ 
    and uS:  $u \in S$ 
    and u-lookup:  $\text{bmssp-lookup-dist ds } u = \text{Some } du$ 
    and v-lookup:  $\text{bmssp-lookup-dist ds } v = \text{Some } dv$ 
    and v-unsettled:  $v \notin \text{set old-settled}$ 
    and v-not-S:  $v \notin S$ 
  shows  $du \leq dv$ 
proof -
  have S-keys:  $S \subseteq \text{keys-of (bp-view } P)$ 
    by (rule  $\text{pull-separates-subset[OF pull]}$ )
  have u-unsettled:  $u \notin \text{set old-settled}$ 
    using keys S-keys uS unfolding  $\text{bmssp-partition-keys-match-def}$  by blast
  have u-value:  $\text{value-of (bp-view } P) \text{ } u = \text{bmssp-partition-key } u \text{ } du$ 
    using vals u-lookup u-unsettled
    unfolding  $\text{bmssp-partition-values-match-def}$  by blast
  have P1-keys:  $\text{keys-of (bp-view } P1) = \text{keys-of (bp-view } P) - S$ 
    by (rule  $\text{pull-separates-remaining-keys[OF pull]}$ )
  have v-key:  $v \in \text{set (map fst ds)}$ 
    using v-lookup by (rule  $\text{bmssp-lookup-dist-mem-key}$ )
  have vP:  $v \in \text{keys-of (bp-view } P)$ 
    using keys v-key v-unsettled unfolding  $\text{bmssp-partition-keys-match-def}$ 
    by blast
  have vP1:  $v \in \text{keys-of (bp-view } P1)$ 
    using P1-keys vP v-not-S by blast
  have v-value:  $\text{value-of (bp-view } P) \text{ } v = \text{bmssp-partition-key } v \text{ } dv$ 
    using vals v-lookup v-unsettled
    unfolding  $\text{bmssp-partition-values-match-def}$  by blast
  have pull-values:  $\text{value-of (bp-view } P1) = \text{value-of (bp-view } P)$ 
    using pull unfolding  $\text{pull-separates-def}$  by simp
  have  $\text{value-of (bp-view } P) \text{ } u \leq \text{value-of (bp-view } P1) \text{ } v$ 
    by (rule  $\text{pull-separates-pulled-smallest[OF pull uS vP1]}$ )
  then have  $\text{bmssp-partition-key } u \text{ } du \leq \text{bmssp-partition-key } v \text{ } dv$ 
    using u-value v-value pull-values by simp
  then show ?thesis
    by (rule  $\text{bmssp-partition-key-le-imp-distance-le}$ )
qed

```

definition *bmssp-partition-state-match* ::
 $\text{nat list} \Rightarrow \text{nat list} \Rightarrow \text{nat-dist} \Rightarrow \text{nat bucketed-partition} \Rightarrow \text{bool}$ **where**
bmssp-partition-state-match *vertices settled ds P* $\longleftrightarrow$
 $\text{distinct } (\text{map fst } ds) \wedge$
 $\text{set } (\text{map fst } ds) \subseteq \text{set vertices} \wedge$
bmssp-partition-keys-match *settled ds P* $\wedge$
bmssp-partition-values-match *settled ds (bp-view P)*

lemma *bmssp-partition-state-match-initial*:
assumes *src-vertices*: $\text{src} \in \text{set vertices}$
shows *bmssp-partition-state-match* *vertices* $\square [(src, 0)]$
(bp-result-of
(c-bp-regularized-local-insert src (bmssp-partition-key src 0)
(bp-empty bmssp-block-size bmssp-bound)))

proof –
have *distinct*: $\text{distinct } (\text{map fst } [(src, 0)])$
by *simp*
have *keys-subset*: $\text{set } (\text{map fst } [(src, 0)]) \subseteq \text{set vertices}$
using *src-vertices* **by** *simp*
have *keys*:
bmssp-partition-keys-match $\square [(src, 0)]$
(bp-result-of
(c-bp-regularized-local-insert src (bmssp-partition-key src 0)
(bp-empty bmssp-block-size bmssp-bound)))
by *(rule bmssp-partition-keys-match-initial)*
have *vals*:
bmssp-partition-values-match $\square [(src, 0)]$
(bp-view
(bp-result-of
(c-bp-regularized-local-insert src (bmssp-partition-key src 0)
(bp-empty bmssp-block-size bmssp-bound)))
by *(rule bmssp-partition-values-match-initial)*
show *?thesis*
unfolding *bmssp-partition-state-match-def*
using *distinct keys-subset keys vals* **by** *simp*
qed

lemma *bmssp-partition-state-match-keys-subset*:
assumes *bmssp-partition-state-match* *vertices settled ds P*
shows *keys-of (bp-view P)* $\subseteq \text{set vertices}$
proof –
have *keys*: *bmssp-partition-keys-match* *settled ds P*
and *ds-keys*: $\text{set } (\text{map fst } ds) \subseteq \text{set vertices}$
using *assms* **unfolding** *bmssp-partition-state-match-def* **by** *auto*
show *?thesis*
using *keys ds-keys* **unfolding** *bmssp-partition-keys-match-def* **by** *blast*
qed

lemma *bmssp-partition-state-match-step-bridge*:

```

fixes old-settled settled vertices :: nat list
and pulled :: nat list
and updates :: (nat × real) list
assumes match: bmssp-partition-state-match vertices old-settled ds P
and edge-targets:
   $\bigwedge a\ b\ w. (a, b, w) \in \text{set } G \implies b \in \text{set vertices}$ 
and pulled-def:
  pulled =
    filter ( $\lambda x. x \in S \wedge x \notin \text{set old-settled}$ ) vertices
and settled-def: settled = pulled @ old-settled
and relaxed: bmssp-relax-vertices G settled pulled ds = (updates, ds')
and P2-def: P2 = bmssp-apply-updates updates P1
and wf: nat-graph-well-formed G
and distinct-vertices: distinct vertices
and ord: bp-ordered-invariant P
and upper: partition-upper-bound (bp-view P) B
and pull: bp-pull bmssp-block-size B P = (S, beta, P1)
shows bmssp-partition-state-match vertices settled ds' P2
proof –
  have distinct-ds: distinct (map fst ds)
    and ds-keys: set (map fst ds) ⊆ set vertices
    and keys: bmssp-partition-keys-match old-settled ds P
    and vals: bmssp-partition-values-match old-settled ds (bp-view P)
    using match unfolding bmssp-partition-state-match-def by auto
  have pulled-subset: set pulled ⊆ set vertices
    unfolding pulled-def by auto
  have distinct-ds': distinct (map fst ds')
    using bmssp-relax-vertices-preserves-distinct-dist
    [OF distinct-ds, of G settled pulled] relaxed by simp
  have ds'-keys: set (map fst ds') ⊆ set vertices
  proof –
    have set (map fst (snd (bmssp-relax-vertices G settled pulled ds)))
       $\subseteq \text{set vertices}$ 
      by (rule bmssp-relax-vertices-dist-keys-subset
        [OF ds-keys pulled-subset edge-targets])
    then show ?thesis
      using relaxed by simp
  qed
  have keys-vertices: keys-of (bp-view P) ⊆ set vertices
    by (rule bmssp-partition-state-match-keys-subset [OF match])
  have keys':
    bmssp-partition-keys-match settled ds' P2
    by (rule bmssp-partition-keys-match-loop-step-bridge
      [OF keys keys-vertices pulled-def settled-def relaxed P2-def
        wf distinct-vertices ord upper pull])
  have vals':
    bmssp-partition-values-match settled ds' (bp-view P2)
    by (rule bmssp-partition-values-match-loop-step-bridge
      [OF vals keys distinct-ds pulled-def settled-def relaxed P2-def

```

```

      wf distinct-vertices ord upper pull])
show ?thesis
  unfolding bmssp-partition-state-match-def
  using distinct-ds' ds'-keys keys' vals' by simp
qed

lemma bmssp-partition-state-pull-residual-label-le:
  assumes match: bmssp-partition-state-match vertices old-settled ds P
    and pull: pull-separates (bp-view P) bmssp-block-size B S beta
      (bp-view P1)
    and uS: u ∈ S
    and u-lookup: bmssp-lookup-dist ds u = Some du
    and v-lookup: bmssp-lookup-dist ds v = Some dv
    and v-unsettled: v ∉ set old-settled
    and v-not-S: v ∉ S
  shows du ≤ dv
proof -
  have vals: bmssp-partition-values-match old-settled ds (bp-view P)
    and keys: bmssp-partition-keys-match old-settled ds P
    using match unfolding bmssp-partition-state-match-def by auto
  show ?thesis
    by (rule bmssp-pull-residual-label-le
      [OF vals keys pull uS u-lookup v-lookup v-unsettled v-not-S])
qed

lemma bmssp-partition-state-pulled-not-settled:
  assumes match: bmssp-partition-state-match vertices settled ds P
    and pull: pull-separates (bp-view P) bmssp-block-size B S beta D'
    and uS: u ∈ S
  shows u ∉ set settled
proof -
  have keys: bmssp-partition-keys-match settled ds P
    using match unfolding bmssp-partition-state-match-def by blast
  have S-keys: S ⊆ keys-of (bp-view P)
    by (rule pull-separates-subset[OF pull])
  show ?thesis
    using keys S-keys uS unfolding bmssp-partition-keys-match-def by blast
qed

lemma bmssp-partition-state-pulled-lookup:
  assumes match: bmssp-partition-state-match vertices settled ds P
    and pull: pull-separates (bp-view P) bmssp-block-size B S beta D'
    and uS: u ∈ S
  obtains d where bmssp-lookup-dist ds u = Some d
proof -
  have distinct-ds: distinct (map fst ds)
    and keys: bmssp-partition-keys-match settled ds P
    using match unfolding bmssp-partition-state-match-def by auto
  have S-keys: S ⊆ keys-of (bp-view P)

```

by (rule pull-separates-subset[OF pull])
 have u-key: $u \in \text{set } (\text{map } \text{fst } ds)$
 using keys S-keys uS unfolding bmssp-partition-keys-match-def by blast
 then obtain d where mem: $(u, d) \in \text{set } ds$
 by force
 have bmssp-lookup-dist ds u = Some d
 by (rule bmssp-lookup-dist-Some-if-distinct-mem[OF distinct-ds mem])
 then show ?thesis
 by (rule that)
 qed

lemma bmssp-partition-state-pulled-list-lookup:
 assumes match: bmssp-partition-state-match vertices old-settled ds P
 and pull: pull-separates (bp-view P) bmssp-block-size B S beta D'
 and pulled-def:
 pulled =
 filter ($\lambda x. x \in S \wedge x \notin \text{set } \text{old-settled}$) vertices
 and u-pulled: $u \in \text{set } \text{pulled}$
 obtains d where bmssp-lookup-dist ds u = Some d
proof –
 have uS: $u \in S$
 using u-pulled unfolding pulled-def by auto
 show ?thesis
 by (rule bmssp-partition-state-pulled-lookup
 [OF match pull uS that])
 qed

lemma nat-graph-edge-in-vertices:
 assumes $(u, v) \in \text{nat-graph-edges } G$
 shows $u \in \text{nat-graph-vertices } G \ v \in \text{nat-graph-vertices } G$
 using assms
 unfolding nat-graph-edges-def nat-graph-edge-list-def nat-graph-vertices-def
 by auto

lemma nat-graph-weight-nonneg [simp]:
 $0 \leq \text{nat-graph-weight } G \ u \ v$
proof (cases map-of
 ($\text{map } (\lambda e. ((\text{nat-edge-source } e, \text{nat-edge-target } e),$
 $\text{real } (\text{nat-edge-weight } e))) \ G$) (u, v))
 case None
 then show ?thesis
 unfolding nat-graph-weight-def by simp
next
 case (Some c)
 then have $((u, v), c) \in \text{set}$
 ($\text{map } (\lambda e. ((\text{nat-edge-source } e, \text{nat-edge-target } e),$
 $\text{real } (\text{nat-edge-weight } e))) \ G$)
 by (rule map-of-SomeD)
 then show ?thesis

using *Some* unfolding nat-graph-weight-def by auto
qed

lemma *bmssp-edge-vertices-set*:
 $\text{set } (\text{concat } (\text{map } \text{bmssp-edge-vertices } G)) = \text{nat-graph-vertices } G$
 unfolding nat-graph-vertices-def
 by (induction *G*) (auto split: prod.splits)

lemma *bmssp-vertices-set*:
 $\text{set } (\text{bmssp-vertices } G \text{ src}) = \text{insert src } (\text{nat-graph-vertices } G)$
 unfolding bmssp-vertices-def using bmssp-edge-vertices-set[of *G*] by auto

lemma *bmssp-vertices-set-if-source*:
 assumes $\text{src} \in \text{nat-graph-vertices } G$
 shows $\text{set } (\text{bmssp-vertices } G \text{ src}) = \text{nat-graph-vertices } G$
 using bmssp-vertices-set[of *G* src] assms by auto

lemma *bmssp-edge-target-in-vertices*:
 assumes $(a, b, w) \in \text{set } G$
 shows $b \in \text{set } (\text{bmssp-vertices } G \text{ src})$
proof –
 have $b \in \text{nat-graph-vertices } G$
 using assms unfolding nat-graph-vertices-def by force
 then show ?thesis
 using bmssp-vertices-set[of *G* src] by auto
 qed

lemma *bmssp-lookup-dist-Some-mem*:
 assumes $\text{bmssp-lookup-dist } ds \text{ } v = \text{Some } d$
 shows $(v, d) \in \text{set } ds$
 using assms
proof (induction *ds*)
 case Nil
 then show ?case
 by simp
next
 case (Cons *p* *ds*)
 obtain *x e* where $p = (x, e)$
 by (cases *p*)
 show ?case
proof (cases $v = x$)
 case True
 then show ?thesis
 using Cons.prem1 unfolding *p* by simp
next
 case False
 then have $\text{bmssp-lookup-dist } ds \text{ } v = \text{Some } d$
 using Cons.prem1 unfolding *p* by simp
 then have $(v, d) \in \text{set } ds$

```

    by (rule Cons.IH)
  then show ?thesis
    unfolding p by simp
qed
qed

lemma bmssp-set-dist-mem-cases:
  assumes (v, d) ∈ set (bmssp-set-dist x dx ds)
  shows (v = x ∧ d = dx) ∨ (v, d) ∈ set ds
  using assms
proof (induction ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons p ds)
  obtain y e where p: p = (y, e)
    by (cases p)
  show ?case
  proof (cases x = y)
    case True
    then show ?thesis
      using Cons.prem unfolding p by auto
  next
    case False
    then have mem:
      (v, d) = (y, e) ∨ (v, d) ∈ set (bmssp-set-dist x dx ds)
      using Cons.prem unfolding p by auto
    then show ?thesis
    proof
      assume (v, d) = (y, e)
      then show ?thesis
        using False unfolding p by auto
    next
      assume (v, d) ∈ set (bmssp-set-dist x dx ds)
      then show ?thesis
        using Cons.IH by auto
    qed
  qed
qed
qed

lemma nat-graph-weight-of-edge:
  assumes wf: nat-graph-well-formed G
  and edge: (u, v, w) ∈ set G
  shows nat-graph-weight G u v = real w
proof -
  let ?xs =
    map (λe. ((nat-edge-source e, nat-edge-target e),
      real (nat-edge-weight e))) G

```

```

have distinct-edges: distinct (nat-graph-edge-list G)
  by (rule nat-graph-well-formed-distinct-edge-list[OF wf])
have fst-xs: map fst ?xs = nat-graph-edge-list G
  unfolding nat-graph-edge-list-def by (induction G) auto
have distinct-xs: distinct (map fst ?xs)
  using distinct-edges unfolding fst-xs .
have mem: ((u, v), real w) ∈ set ?xs
  using edge by force
have map-of ?xs (u, v) = Some (real w)
  using map-of-eq-Some-iff[OF distinct-xs] mem by blast
then show ?thesis
  unfolding nat-graph-weight-def by simp
qed

lemma exec-walk-append-edge:
  assumes walk: exec-walk vs es p
  and p-ne: p ≠ []
  and last-p: last p = u
  and vV: v ∈ set vs
  and edge: (u, v) ∈ set es
  shows exec-walk vs es (p @ [v])
proof -
  obtain x xs where p-def: p = x # xs
  using p-ne by (cases p) auto
  have aux:
    ∧ x u. [exec-walk vs es (x # xs); last (x # xs) = u;
      (u, v) ∈ set es] ⇒
      exec-walk vs es ((x # xs) @ [v])
  proof (induction xs)
    case Nil
    then show ?case
      using vV by simp
  next
    case (Cons y ys)
    have tail-walk: exec-walk vs es (y # ys)
      using Cons.prems by simp
    have tail-last: last (y # ys) = u
      using Cons.prems by simp
    have tail-app: exec-walk vs es ((y # ys) @ [v])
      by (rule Cons.IH[OF tail-walk tail-last Cons.prems(3)])
    show ?case
      using Cons.prems tail-app by simp
  qed
  show ?thesis
    unfolding p-def
    by (rule aux[OF - - edge]) (use walk last-p p-def in simp-all)
qed

lemma exec-walk-weight-append-edge:

```

```

    assumes  $p \neq []$ 
    shows  $\text{exec-walk-weight } W (p @ [v]) =$ 
       $\text{exec-walk-weight } W p + W (\text{last } p) v$ 
    using assms
  proof (induction  $W p$  rule: exec-walk-weight.induct)
    case (1  $W$ )
    then show ?case
      by simp
  next
    case (2  $W x$ )
    then show ?case
      by simp
  next
    case (3  $W x y xs$ )
    have  $\text{tail: exec-walk-weight } W ((y \# xs) @ [v]) =$ 
       $\text{exec-walk-weight } W (y \# xs) + W (\text{last } (y \# xs)) v$ 
    by (rule 3.IH) simp
    then show ?case
      by simp
  qed

  fun exec-path-edges where
     $\text{exec-path-edges } [] = []$ 
  |  $\text{exec-path-edges } [x] = []$ 
  |  $\text{exec-path-edges } (x \# y \# xs) = (x, y) \# \text{exec-path-edges } (y \# xs)$ 

  lemma exec-walk-path-edges-subset:
    assumes  $\text{exec-walk } vs \text{ es } p$ 
    shows  $\text{set } (\text{exec-path-edges } p) \subseteq \text{set } es$ 
    using assms by (induction  $p$  rule: exec-path-edges.induct) auto

  lemma exec-path-edges-vertices:
    assumes  $e \in \text{set } (\text{exec-path-edges } p)$ 
    shows  $\text{fst } e \in \text{set } p$  and  $e \in \text{set } p$ 
    using assms by (induction  $p$  rule: exec-path-edges.induct) auto

  lemma exec-path-edges-distinct-if-distinct:
    assumes distinct  $p$ 
    shows distinct  $(\text{exec-path-edges } p)$ 
    using assms
  proof (induction  $p$  rule: exec-path-edges.induct)
    case 1
    then show ?case
      by simp
  next
    case (2  $x$ )
    then show ?case
      by simp
  next

```

```

case ( $\exists x y xs$ )
have  $(x, y) \notin \text{set } (\text{exec-path-edges } (y \# xs))$ 
proof
  assume  $\text{edge}: (x, y) \in \text{set } (\text{exec-path-edges } (y \# xs))$ 
  then have  $x \in \text{set } (y \# xs)$ 
  using  $\text{exec-path-edges-vertices}(1)[OF \text{ edge}]$  by simp
  then show False
  using  $\exists.\text{prems}$  by simp
qed
moreover have  $\text{distinct } (\text{exec-path-edges } (y \# xs))$ 
  using  $\exists.IH \exists.\text{prems}$  by simp
ultimately show ?case
  by simp
qed

```

lemma *exec-walk-weight-sum-path-edges*:

```

   $\text{exec-walk-weight } W p =$ 
     $\text{sum-list } (\text{map } (\lambda e. W (\text{fst } e) (\text{snd } e)) (\text{exec-path-edges } p))$ 
by (induction p rule: exec-path-edges.induct) simp-all

```

lemma *nat-graph-weight-edge-list-sum*:

```

assumes wf:  $\text{nat-graph-well-formed } G$ 
shows  $\text{sum-list}$ 
   $(\text{map } (\lambda e. \text{nat-graph-weight } G (\text{fst } e) (\text{snd } e))$ 
     $(\text{nat-graph-edge-list } G)) =$ 
   $\text{real } (\text{nat-graph-total-weight } G)$ 
proof –
  have weights:
     $\bigwedge e. e \in \text{set } G \implies$ 
     $\text{nat-graph-weight } G (\text{nat-edge-source } e) (\text{nat-edge-target } e) =$ 
     $\text{real } (\text{nat-edge-weight } e)$ 
proof –
  fix e
  assume  $e: e \in \text{set } G$ 
  obtain  $u v w$  where  $e\text{-def}: e = (u, v, w)$ 
  by (cases e)
  show  $\text{nat-graph-weight } G (\text{nat-edge-source } e) (\text{nat-edge-target } e) =$ 
     $\text{real } (\text{nat-edge-weight } e)$ 
  unfolding  $e\text{-def}$ 
  by (rule nat-graph-weight-of-edge[OF wf]) (use e e-def in simp)
qed
have  $\text{sum-list}$ 
   $(\text{map } (\lambda e. \text{nat-graph-weight } G (\text{fst } e) (\text{snd } e))$ 
     $(\text{nat-graph-edge-list } G)) =$ 
   $\text{sum-list}$ 
   $(\text{map } (\lambda e. \text{nat-graph-weight } G (\text{nat-edge-source } e)$ 
     $(\text{nat-edge-target } e)) G)$ 
  unfolding  $\text{nat-graph-edge-list-def}$  by (simp add: o-def)
also have  $\dots = \text{sum-list } (\text{map } (\lambda e. \text{real } (\text{nat-edge-weight } e)) G)$ 

```

```

proof –
  have aux:  $\bigwedge xs. \text{set } xs \subseteq \text{set } G \implies$ 
     $\text{sum-list } (\text{map } (\lambda e. \text{nat-graph-weight } G \text{ (nat-edge-source } e) \text{ (nat-edge-target } e)) \text{ } xs) =$ 
     $\text{sum-list } (\text{map } (\lambda e. \text{real } (\text{nat-edge-weight } e)) \text{ } xs)$ 
  proof –
    fix xs
    assume  $\text{set } xs \subseteq \text{set } G$ 
    then show  $\text{sum-list } (\text{map } (\lambda e. \text{nat-graph-weight } G \text{ (nat-edge-source } e) \text{ (nat-edge-target } e)) \text{ } xs) =$ 
     $\text{sum-list } (\text{map } (\lambda e. \text{real } (\text{nat-edge-weight } e)) \text{ } xs)$ 
    by (induction xs) (auto simp: weights)
  qed
  show ?thesis
  by (rule aux) simp
qed
also have  $\dots = \text{real } (\text{sum-list } (\text{map } \text{nat-edge-weight } G))$ 
by (induction G) auto
finally show ?thesis
unfolding nat-graph-total-weight-def .
qed

```

```

lemma nat-graph-weight-nonnegative:
   $0 \leq \text{nat-graph-weight } G \text{ } u \text{ } v$ 
proof (cases map-of
  ( $\text{map } (\lambda e. ((\text{nat-edge-source } e, \text{nat-edge-target } e), \text{real } (\text{nat-edge-weight } e))) \text{ } G) \text{ } (u, v)$ )
  case None
  then show ?thesis
  unfolding nat-graph-weight-def by simp
next
  case (Some c)
  then have  $((u, v), c) \in \text{set}$ 
     $(\text{map } (\lambda e. ((\text{nat-edge-source } e, \text{nat-edge-target } e), \text{real } (\text{nat-edge-weight } e))) \text{ } G)$ 
  by (rule map-of-SomeD)
  then obtain e where  $e \in \text{set } G$ 
  and c:  $c = \text{real } (\text{nat-edge-weight } e)$ 
  by auto
  then show ?thesis
  unfolding nat-graph-weight-def using Some by simp
qed

```

```

lemma exec-walk-weight-nat-graph-le-total:
  assumes wf: nat-graph-well-formed G
  and walk: exec-walk vs (nat-graph-edge-list G) p
  and distinct: distinct p
  shows  $\text{exec-walk-weight } (\text{nat-graph-weight } G) \text{ } p \leq$ 
   $\text{real } (\text{nat-graph-total-weight } G)$ 

```

proof –
let ?w = λe. nat-graph-weight G (fst e) (snd e)
have path-subset: set (exec-path-edges p) ⊆ set (nat-graph-edge-list G)
by (rule exec-walk-path-edges-subset[OF walk])
have path-distinct: distinct (exec-path-edges p)
by (rule exec-path-edges-distinct-if-distinct[OF distinct])
have graph-distinct: distinct (nat-graph-edge-list G)
by (rule nat-graph-well-formed-distinct-edge-list[OF wf])
have path-sum:
sum-list (map ?w (exec-path-edges p)) =
sum ?w (set (exec-path-edges p))
by (rule sum-list-distinct-conv-sum-set[OF path-distinct])
have graph-sum:
sum-list (map ?w (nat-graph-edge-list G)) =
sum ?w (set (nat-graph-edge-list G))
by (rule sum-list-distinct-conv-sum-set[OF graph-distinct])
have finite-graph: finite (set (nat-graph-edge-list G))
by (rule finite-set)
have sum ?w (set (exec-path-edges p)) ≤
sum ?w (set (nat-graph-edge-list G))
by (rule sum-mono2[OF finite-graph path-subset])
(simp add: nat-graph-weight-nonnegative)
also have ... = real (nat-graph-total-weight G)
using graph-sum nat-graph-weight-edge-list-sum[OF wf] **by** simp
finally have sum-list (map ?w (exec-path-edges p)) ≤
real (nat-graph-total-weight G)
using path-sum **by** simp
then show ?thesis
unfolding exec-walk-weight-sum-path-edges .
qed

definition bmssp-label-witnesses ::
nat-graph ⇒ nat ⇒ nat list ⇒ nat-dist ⇒ bool **where**
bmssp-label-witnesses G src settled ds ⟷
(∀ v d. (v, d) ∈ set ds ⟶
(∃ p. p ≠ [] ∧ hd p = src ∧ last p = v ∧ distinct p ∧
exec-walk (bmssp-vertices G src) (nat-graph-edge-list G) p ∧
exec-walk-weight (nat-graph-weight G) p = real d ∧
set (butlast p) ⊆ set settled))

lemma bmssp-label-witnesses-mono-settled:
assumes witnesses: bmssp-label-witnesses G src settled ds
and subset: set settled ⊆ set settled'
shows bmssp-label-witnesses G src settled' ds
unfolding bmssp-label-witnesses-def
proof (intro allI impI)
fix v d
assume (v, d) ∈ set ds
then obtain p **where** p:

```

    p ≠ [] hd p = src last p = v distinct p
    exec-walk (bmssp-vertices G src) (nat-graph-edge-list G) p
    exec-walk-weight (nat-graph-weight G) p = real d
    set (butlast p) ⊆ set settled
    using witnesses unfolding bmssp-label-witnesses-def by blast
  have set (butlast p) ⊆ set settled'
    using p(7) subset by blast
  then show ∃ p. p ≠ [] ∧ hd p = src ∧ last p = v ∧ distinct p ∧
    exec-walk (bmssp-vertices G src) (nat-graph-edge-list G) p ∧
    exec-walk-weight (nat-graph-weight G) p = real d ∧
    set (butlast p) ⊆ set settled'
    using p(1-6) by blast
qed

lemma bmssp-label-witnesses-initial:
  bmssp-label-witnesses G src [] [(src, 0)]
  unfolding bmssp-label-witnesses-def
  by (intro allI impI, auto simp: bmssp-vertices-set intro!: exI[of - [src]])

lemma bmssp-label-witnesses-set-dist:
  assumes witnesses: bmssp-label-witnesses G src settled ds
  and p: p ≠ [] hd p = src last p = x distinct p
    exec-walk (bmssp-vertices G src) (nat-graph-edge-list G) p
    exec-walk-weight (nat-graph-weight G) p = real dx
    set (butlast p) ⊆ set settled
  shows bmssp-label-witnesses G src settled (bmssp-set-dist x dx ds)
  unfolding bmssp-label-witnesses-def
proof (intro allI impI)
  fix v d
  assume mem: (v, d) ∈ set (bmssp-set-dist x dx ds)
  consider (new) v = x d = dx | (old) (v, d) ∈ set ds
  using bmssp-set-dist-mem-cases[OF mem] by blast
  then show ∃ p. p ≠ [] ∧ hd p = src ∧ last p = v ∧ distinct p ∧
    exec-walk (bmssp-vertices G src) (nat-graph-edge-list G) p ∧
    exec-walk-weight (nat-graph-weight G) p = real d ∧
    set (butlast p) ⊆ set settled
  proof cases
    case new
    then show ?thesis
      using p by blast
  next
    case old
    then show ?thesis
      using witnesses unfolding bmssp-label-witnesses-def by blast
  qed
qed

```

lemma *bmssp-relax-edges-lookup-settled*:

```

  assumes v ∈ set settled

```



```

shows bmssp-lookup-dist (snd (bmssp-relax-edges G settled u du ds)) v =
  bmssp-lookup-dist ds v
using assms
proof (induction G arbitrary: ds)
  case Nil
  then show ?case
    by simp
next
  case (Cons e es)
  obtain a b w where e: e = (a, b, w)
    by (cases e)
  obtain updates ds1 where rec:
    bmssp-relax-edges es settled u du ds = (updates, ds1)
    by force
  have lookup-tail: bmssp-lookup-dist ds1 v = bmssp-lookup-dist ds v
    using Cons.IH[OF Cons.prem, of ds] rec by simp
  show ?case
  proof (cases a = u  $\wedge$  b  $\notin$  set settled)
    case False
    then show ?thesis
      using rec lookup-tail unfolding e
      by (auto simp: Let-def split: option.splits if-splits)
  next
    case True
    then have b-ne-v: b  $\neq$  v
      using Cons.prem by auto
    show ?thesis
    proof (cases bmssp-lookup-dist ds1 b)
      case None
      then show ?thesis
        using True rec lookup-tail b-ne-v
        unfolding e by (simp add: Let-def bmssp-lookup-dist-set-dist-other)
    next
      case (Some old)
      show ?thesis
      proof (cases du + w < old)
        case True
        then show ?thesis
          using  $\langle a = u \wedge b \notin \text{set settled} \rangle$  rec Some lookup-tail b-ne-v
          unfolding e by (simp add: Let-def bmssp-lookup-dist-set-dist-other)
      next
        case False
        then show ?thesis
          using  $\langle a = u \wedge b \notin \text{set settled} \rangle$  rec Some lookup-tail
          unfolding e by (simp add: Let-def)
      qed
    qed
  qed
qed

```

lemma *bmssp-relax-vertices-lookup-settled*:
assumes $v \in \text{set settled}$
shows $\text{bmssp-lookup-dist } (\text{snd } (\text{bmssp-relax-vertices } G \text{ settled } xs \text{ } ds)) \text{ } v =$
 $\text{bmssp-lookup-dist } ds \text{ } v$
proof (*induction xs arbitrary: ds*)
case *Nil*
then show *?case*
by *simp*
next
case (*Cons u us*)
show *?case*
proof (*cases bmssp-lookup-dist ds u*)
case *None*
then show *?thesis*
using *Cons.IH* **by** *simp*
next
case (*Some du*)
obtain *updates-u ds1* **where** *edge-rec*:
 $\text{bmssp-relax-edges } G \text{ settled } u \text{ } du \text{ } ds = (\text{updates-u}, ds1)$
by *force*
have *lookup-ds1*:
 $\text{bmssp-lookup-dist } ds1 \text{ } v = \text{bmssp-lookup-dist } ds \text{ } v$
using *bmssp-relax-edges-lookup-settled* [*OF assms, of G u du ds*]
edge-rec **by** *simp*
obtain *updates-us ds2* **where** *vertices-rec*:
 $\text{bmssp-relax-vertices } G \text{ settled } us \text{ } ds1 = (\text{updates-us}, ds2)$
by *force*
have *lookup-ds2*:
 $\text{bmssp-lookup-dist } ds2 \text{ } v = \text{bmssp-lookup-dist } ds1 \text{ } v$
using *Cons.IH* [*of ds1*] *vertices-rec* **by** *simp*
show *?thesis*
using *Some edge-rec vertices-rec lookup-ds1 lookup-ds2* **by** *simp*
qed
qed

lemma *bmssp-relax-vertices-edge-lookup-le-candidate*:
assumes *distinct*: $\text{distinct } (\text{map fst } ds)$
and *xs-subset*: $\text{set } xs \subseteq \text{set settled}$
and *u-xs*: $u \in \text{set } xs$
and *lookup-u*: $\text{bmssp-lookup-dist } ds \text{ } u = \text{Some } du$
and *edge*: $(u, v, w) \in \text{set } G$
and *v-unsettled*: $v \notin \text{set settled}$
shows $\exists dv.$
 $\text{bmssp-lookup-dist } (\text{snd } (\text{bmssp-relax-vertices } G \text{ settled } xs \text{ } ds)) \text{ } v =$
 $\text{Some } dv \wedge$
 $dv \leq du + w$
using *distinct xs-subset u-xs lookup-u*
proof (*induction xs arbitrary: ds du*)

```

case Nil
then show ?case
  by simp
next
case (Cons x xs)
show ?case
proof (cases bmssp-lookup-dist ds x)
  case None
  have u-ne-x:  $u \neq x$ 
  using None Cons.premis(4) by auto
  then have u-tail:  $u \in \text{set } xs$ 
  using Cons.premis(3) by simp
  obtain dv where tail-lookup:
    bmssp-lookup-dist
    (snd (bmssp-relax-vertices G settled xs ds)) v = Some dv
  and tail-le:  $dv \leq du + w$ 
  using Cons.IH[OF Cons.premis(1) - u-tail Cons.premis(4)]
  using Cons.premis(2) by auto
  have final-lookup:
    bmssp-lookup-dist
    (snd (bmssp-relax-vertices G settled (x # xs) ds)) v = Some dv
  using None tail-lookup by simp
  show ?thesis
  using final-lookup tail-le by blast
next
case (Some dx)
obtain updates-x ds1 where edge-rec:
  bmssp-relax-edges G settled x dx ds = (updates-x, ds1)
  by force
have distinct-ds1:  $\text{distinct } (\text{map fst } ds1)$ 
  using bmssp-relax-edges-preserves-distinct-dist[OF Cons.premis(1),
    of G settled x dx] edge-rec by simp
have xs-subset:  $\text{set } xs \subseteq \text{set settled}$ 
  using Cons.premis(2) by simp
obtain updates-xs ds2 where vertices-rec:
  bmssp-relax-vertices G settled xs ds1 = (updates-xs, ds2)
  by force
show ?thesis
proof (cases  $u = x$ )
  case True
  then have du-eq:  $du = dx$ 
  using Some Cons.premis(4) by simp
  have edge-x:  $(x, v, w) \in \text{set } G$ 
  using edge True by simp
  obtain mid where mid-lookup:
    bmssp-lookup-dist (snd (bmssp-relax-edges G settled x dx ds)) v =
    Some mid
  and mid-le:  $mid \leq dx + w$ 
  using bmssp-relax-edges-edge-lookup-le-candidate

```

```

      [OF edge-x v-unsettled, of dx ds]
    by blast
  have mid-ds1: bmssp-lookup-dist ds1 v = Some mid
    using mid-lookup edge-rec by simp
  obtain dv where final-lookup:
    bmssp-lookup-dist
      (snd (bmssp-relax-vertices G settled xs ds1)) v = Some dv
    using bmssp-relax-vertices-lookup-Some-preserved
      [OF distinct-ds1 mid-ds1]
    by blast
  have dv-le-mid:
    dv ≤ mid
  by (rule bmssp-relax-vertices-lookup-le
      [OF distinct-ds1 mid-ds1 final-lookup])
  have dv ≤ du + w
    using dv-le-mid mid-le du-eq by linarith
  then show ?thesis
    using Some edge-rec vertices-rec final-lookup by auto
next
case False
have u-tail: u ∈ set xs
  using Cons.premis(3) False by simp
have u-settled: u ∈ set settled
  using xs-subset u-tail by blast
have lookup-u-ds1: bmssp-lookup-dist ds1 u = Some du
  using bmssp-relax-edges-lookup-settled[OF u-settled, of G x dx ds]
    Cons.premis(4) edge-rec by simp
obtain dv where final-lookup:
  bmssp-lookup-dist
    (snd (bmssp-relax-vertices G settled xs ds1)) v = Some dv
  and final-le: dv ≤ du + w
  using Cons.IH[OF distinct-ds1 xs-subset u-tail lookup-u-ds1]
  by blast
show ?thesis
  using Some edge-rec vertices-rec final-lookup final-le by auto
qed
qed
qed

```

definition *bmssp-settled-exact* ::
 $\text{nat-graph} \Rightarrow \text{nat} \Rightarrow \text{nat list} \Rightarrow \text{nat-dist} \Rightarrow \text{bool}$ **where**
bmssp-settled-exact G src settled ds $\longleftrightarrow$
 $(\forall v d. v \in \text{set settled} \longrightarrow$
 $\text{bmssp-lookup-dist ds } v = \text{Some } d \longrightarrow$
 $\text{real } d = \text{nat-graph-dist } G \text{ src } v)$

lemma *bmssp-settled-exact-initial*:
bmssp-settled-exact G src [] ds
unfolding *bmssp-settled-exact-def* **by** simp

lemma *bmssp-settled-exact-step*:
assumes *exact-old*: *bmssp-settled-exact* *G* *src* *old-settled* *ds*
and *pulled-exact*:
 $\bigwedge v d. \llbracket v \in \text{set pulled}; \text{bmssp-lookup-dist } ds \ v = \text{Some } d \rrbracket$
 $\implies \text{real } d = \text{nat-graph-dist } G \ \text{src } v$
and *settled-def*: *settled* = *pulled* @ *old-settled*
and *relaxed*: *bmssp-relax-vertices* *G* *settled* *pulled* *ds* = (*updates*, *ds'*)
shows *bmssp-settled-exact* *G* *src* *settled* *ds'*
unfolding *bmssp-settled-exact-def*
proof (*intro allI impI*)
fix *v d*
assume *v-settled*: *v* $\in$ *set settled*
and *lookup'*: *bmssp-lookup-dist* *ds'* *v* = *Some d*
have *lookup-old*: *bmssp-lookup-dist* *ds* *v* = *Some d*
using *bmssp-relax-vertices-lookup-settled*[*OF v-settled*, *of G pulled ds*]
relaxed lookup' **by** *simp*
have *v* $\in$ *set pulled* $\vee$ *v* $\in$ *set old-settled*
using *v-settled settled-def* **by** *auto*
then show *real d* = *nat-graph-dist* *G* *src* *v*
proof
assume *v* $\in$ *set pulled*
then show ?thesis
by (*rule pulled-exact*[*OF - lookup-old*])
next
assume *v* $\in$ *set old-settled*
then show ?thesis
using *exact-old lookup-old* **unfolding** *bmssp-settled-exact-def* **by** *blast*
qed
qed

definition *bmssp-frontier-relaxed* ::
 $\text{nat-graph} \Rightarrow \text{nat} \Rightarrow \text{nat list} \Rightarrow \text{nat-dist} \Rightarrow \text{bool}$ **where**
bmssp-frontier-relaxed *G* *src* *settled* *ds* $\longleftrightarrow$
 $(\forall u \ v \ w \ du. (u, v, w) \in \text{set } G \longrightarrow$
 $u \in \text{set settled} \longrightarrow$
 $v \notin \text{set settled} \longrightarrow$
 $\text{bmssp-lookup-dist } ds \ u = \text{Some } du \longrightarrow$
 $\text{real } du = \text{nat-graph-dist } G \ \text{src } u \longrightarrow$
 $(\exists dv. \text{bmssp-lookup-dist } ds \ v = \text{Some } dv \wedge$
 $\text{real } dv \leq \text{real } du + \text{real } w))$

lemma *bmssp-frontier-relaxed-initial*:
bmssp-frontier-relaxed *G* *src* [] *ds*
unfolding *bmssp-frontier-relaxed-def* **by** *simp*

lemma *bmssp-frontier-relaxed-step*:
assumes *frontier-old*: *bmssp-frontier-relaxed* *G* *src* *old-settled* *ds*
and *distinct*: *distinct* (*map fst ds*)

```

    and settled-def: settled = pulled @ old-settled
    and relaxed: bmssp-relax-vertices G settled pulled ds = (updates, ds')
shows bmssp-frontier-relaxed G src settled ds'
unfolding bmssp-frontier-relaxed-def
proof (intro allI impI)
  fix u v w du
  assume edge: (u, v, w) ∈ set G
  and u-settled: u ∈ set settled
  and v-unsettled: v ∉ set settled
  and lookup-u': bmssp-lookup-dist ds' u = Some du
  and exact-u: real du = nat-graph-dist G src u
  have lookup-u: bmssp-lookup-dist ds u = Some du
  using bmssp-relax-vertices-lookup-settled[OF u-settled, of G pulled ds]
    relaxed lookup-u' by simp
  have u-cases: u ∈ set pulled ∨ u ∈ set old-settled
  using u-settled settled-def by auto
  show ∃ dv. bmssp-lookup-dist ds' v = Some dv ∧
    real dv ≤ real du + real w
  proof (cases u ∈ set old-settled)
    case True
    have v-old-unsettled: v ∉ set old-settled
    using v-unsettled settled-def by auto
    obtain old-dv where old-lookup:
      bmssp-lookup-dist ds v = Some old-dv
    and old-le: real old-dv ≤ real du + real w
    using frontier-old edge True v-old-unsettled lookup-u exact-u
    unfolding bmssp-frontier-relaxed-def by blast
    obtain dv where final-lookup:
      bmssp-lookup-dist
        (snd (bmssp-relax-vertices G settled pulled ds)) v = Some dv
    using bmssp-relax-vertices-lookup-Some-preserved
      [OF distinct old-lookup]
    by blast
    have dv-le-old: dv ≤ old-dv
    by (rule bmssp-relax-vertices-lookup-le
      [OF distinct old-lookup final-lookup])
    have real dv ≤ real du + real w
    using dv-le-old old-le by linarith
    then show ?thesis
    using final-lookup relaxed by auto
  next
  case False
  then have u-pulled: u ∈ set pulled
  using u-cases by blast
  have pulled-subset: set pulled ⊆ set settled
  using settled-def by auto
  obtain dv where lookup:
    bmssp-lookup-dist
      (snd (bmssp-relax-vertices G settled pulled ds)) v = Some dv

```

```

    and dv-le:  $dv \leq du + w$ 
    using bmssp-relax-vertices-edge-lookup-le-candidate
    [OF distinct pulled-subset u-pulled lookup-u edge v-unsettled]
    by blast
    have real dv  $\leq$  real du + real w
    using dv-le by simp
    then show ?thesis
    using lookup relaxed by auto
  qed
qed

definition bmssp-source-zero :: nat  $\Rightarrow$  nat-dist  $\Rightarrow$  bool where
  bmssp-source-zero src ds  $\longleftrightarrow$  bmssp-lookup-dist ds src = Some 0

lemma bmssp-source-zero-initial:
  bmssp-source-zero src [(src, 0)]
  unfolding bmssp-source-zero-def by simp

lemma bmssp-source-zero-step:
  assumes zero: bmssp-source-zero src ds
  and distinct: distinct (map fst ds)
  and relaxed: bmssp-relax-vertices G settled pulled ds = (updates, ds')
  shows bmssp-source-zero src ds'
proof -
  have lookup0: bmssp-lookup-dist ds src = Some 0
  using zero unfolding bmssp-source-zero-def .
  obtain d' where lookup':
    bmssp-lookup-dist
      (snd (bmssp-relax-vertices G settled pulled ds)) src = Some d'
  using bmssp-relax-vertices-lookup-Some-preserved[OF distinct lookup0]
  by blast
  have d'  $\leq$  0
  by (rule bmssp-relax-vertices-lookup-le[OF distinct lookup0 lookup'])
  then have d' = 0
  by simp
  then show ?thesis
  using lookup' relaxed unfolding bmssp-source-zero-def by simp
qed

definition bmssp-settled-have-lookups ::
  nat list  $\Rightarrow$  nat-dist  $\Rightarrow$  bool where
  bmssp-settled-have-lookups settled ds  $\longleftrightarrow$ 
    ( $\forall v \in \text{set settled. } \exists d. \text{bmssp-lookup-dist ds v} = \text{Some } d$ )

lemma bmssp-settled-have-lookups-initial:
  bmssp-settled-have-lookups [] ds
  unfolding bmssp-settled-have-lookups-def by simp

lemma bmssp-settled-have-lookups-step:

```

```

assumes have-old: bmssp-settled-have-lookups old-settled ds
and match: bmssp-partition-state-match vertices old-settled ds P
and pull: pull-separates (bp-view P) bmssp-block-size B S beta D'
and pulled-def:
  pulled = filter (λx. x ∈ S ∧ x ∉ set old-settled) vertices
and settled-def: settled = pulled @ old-settled
and relaxed: bmssp-relax-vertices G settled pulled ds = (updates, ds')
shows bmssp-settled-have-lookups settled ds'
unfolding bmssp-settled-have-lookups-def
proof
  fix v
  assume v-settled: v ∈ set settled
  have v-cases: v ∈ set pulled ∨ v ∈ set old-settled
    using v-settled settled-def by auto
  then obtain d where lookup: bmssp-lookup-dist ds v = Some d
  proof
    assume v-pulled: v ∈ set pulled
    obtain d where bmssp-lookup-dist ds v = Some d
      by (rule bmssp-partition-state-pulled-list-lookup
        [OF match pull pulled-def v-pulled])
    then show ?thesis ..
  next
    assume v ∈ set old-settled
    then obtain d where bmssp-lookup-dist ds v = Some d
      using have-old unfolding bmssp-settled-have-lookups-def by blast
    then show ?thesis ..
  qed
  have lookup':
    bmssp-lookup-dist
      (snd (bmssp-relax-vertices G settled pulled ds)) v =
      bmssp-lookup-dist ds v
    by (rule bmssp-relax-vertices-lookup-settled [OF v-settled])
  show  $\exists d. \text{bmssp-lookup-dist } ds' v = \text{Some } d$ 
    using lookup lookup' relaxed by simp
  qed

lemma finite-card-le-one-member-eq:
  assumes finite: finite A
  and card: card A ≤ 1
  and x: x ∈ A
  and y: y ∈ A
  shows x = y
proof (rule ccontr)
  assume x ≠ y
  then have card {x, y} = 2
    by simp
  moreover have {x, y} ⊆ A
    using x y by blast
  ultimately have  $2 \leq \text{card } A$ 

```



```

    using card-mono[OF finite, of {x, y}] by simp
  then show False
    using card by simp
qed

```

```

definition bmssp-dijkstra-state ::
  nat-graph  $\Rightarrow$  nat  $\Rightarrow$  nat list  $\Rightarrow$  nat list  $\Rightarrow$ 
  nat-dist  $\Rightarrow$  nat bucketed-partition  $\Rightarrow$  bool where
  bmssp-dijkstra-state G src vertices settled ds P  $\longleftrightarrow$ 
    bmssp-partition-state-match vertices settled ds P  $\wedge$ 
    bmssp-source-zero src ds  $\wedge$ 
    bmssp-settled-have-lookups settled ds  $\wedge$ 
    bmssp-label-witnesses G src settled ds  $\wedge$ 
    bmssp-settled-exact G src settled ds  $\wedge$ 
    bmssp-frontier-relaxed G src settled ds

```

```

lemma bmssp-dijkstra-state-initial:
  assumes src-vertices: src  $\in$  set vertices
  shows bmssp-dijkstra-state G src vertices [] [(src, 0)]
    (bp-result-of
      (c-bp-regularized-local-insert src (bmssp-partition-key src 0)
        (bp-empty bmssp-block-size bmssp-bound)))
proof -
  have match:
    bmssp-partition-state-match vertices [] [(src, 0)]
    (bp-result-of
      (c-bp-regularized-local-insert src (bmssp-partition-key src 0)
        (bp-empty bmssp-block-size bmssp-bound)))
  by (rule bmssp-partition-state-match-initial[OF src-vertices])
  show ?thesis
    unfolding bmssp-dijkstra-state-def
    using match bmssp-source-zero-initial
      bmssp-settled-have-lookups-initial bmssp-label-witnesses-initial
      bmssp-settled-exact-initial bmssp-frontier-relaxed-initial
    by simp
qed

```

```

lemma bmssp-relax-edges-preserves-label-witnesses-aux:
  assumes wf: nat-graph-well-formed Gfull
    and edge-subset: set Erel  $\subseteq$  set Gfull
    and witnesses: bmssp-label-witnesses Gfull src settled ds
    and u-settled: u  $\in$  set settled
    and lookup-u: bmssp-lookup-dist ds u = Some du
  shows bmssp-label-witnesses Gfull src settled
    (snd (bmssp-relax-edges Erel settled u du ds))
  using edge-subset witnesses lookup-u
proof (induction Erel arbitrary: ds)
  case Nil
  then show ?case

```

```

    by simp
next
case (Cons e es)
obtain a b w where e-def: e = (a, b, w)
  by (cases e)
obtain updates ds1 where rec:
  bmssp-relax-edges es settled u du ds = (updates, ds1)
  by force
have es-subset: set es  $\subseteq$  set Gfull
  using Cons.prem1 by simp
have tail-witnesses: bmssp-label-witnesses Gfull src settled ds1
  using Cons.IH[OF es-subset Cons.prem2 Cons.prem3] rec by simp
have lookup-u-ds1: bmssp-lookup-dist ds1 u = Some du
  using bmssp-relax-edges-lookup-settled[OF u-settled, of es u du ds]
  Cons.prem3 rec by simp
show ?case
proof (cases a = u  $\wedge$  b  $\notin$  set settled)
  case False
  then show ?thesis
    using rec tail-witnesses unfolding e-def
    by (auto simp: Let-def split: option.splits if-splits)
next
case True
note active = True
let ?nd = du + w
have edge-rel: (u, b, w)  $\in$  set (e # es)
  using active unfolding e-def by simp
have edge-mem: (u, b, w)  $\in$  set Gfull
  using Cons.prem1 edge-rel by blast
have edge-pair: (u, b)  $\in$  set (nat-graph-edge-list Gfull)
  unfolding nat-graph-edge-list-def using edge-mem by force
have b-vertex: b  $\in$  set (bmssp-vertices Gfull src)
  by (rule bmssp-edge-target-in-vertices[OF edge-mem])
have weight-b: nat-graph-weight Gfull u b = real w
  by (rule nat-graph-weight-of-edge[OF wf edge-mem])
have u-mem: (u, du)  $\in$  set ds1
  by (rule bmssp-lookup-dist-Some-mem[OF lookup-u-ds1])
obtain p where p:
  p  $\neq$  [] hd p = src last p = u distinct p
  exec-walk (bmssp-vertices Gfull src)
    (nat-graph-edge-list Gfull) p
  exec-walk-weight (nat-graph-weight Gfull) p = real du
  set (butlast p)  $\subseteq$  set settled
  using tail-witnesses u-mem unfolding bmssp-label-witnesses-def by blast
have set-p: set p  $\subseteq$  set settled
proof
  fix x
  assume x: x  $\in$  set p
  have p = butlast p @ [last p]

```

```

    using p(1) by simp
  then have  $x \in \text{set } (\text{butlast } p @ [\text{last } p])$ 
    using x by simp
  then have  $x \in \text{set } (\text{butlast } p) \cup \{\text{last } p\}$ 
    by (simp only: set-append set-simps)
  then have  $x \in \text{set } (\text{butlast } p) \vee x = \text{last } p$ 
    by blast
  then show  $x \in \text{set settled}$ 
proof
  assume  $x \in \text{set } (\text{butlast } p)$ 
  then show ?thesis
    using p(7) by blast
next
  assume  $x = \text{last } p$ 
  then show ?thesis
    using p(3) u-settled by simp
qed
qed
have b-not-p:  $b \notin \text{set } p$ 
  using active set-p by auto
have walk-pb:
  exec-walk (bmssp-vertices Gfull src)
    (nat-graph-edge-list Gfull) (p @ [b])
  by (rule exec-walk-append-edge[OF p(5) p(1) p(3) b-vertex edge-pair])
have distinct-pb:  $\text{distinct } (p @ [b])$ 
  using p(4) b-not-p by simp
have weight-pb:
  exec-walk-weight (nat-graph-weight Gfull) (p @ [b]) = real ?nd
  using exec-walk-weight-append-edge[OF p(1), of nat-graph-weight Gfull b]
    p(3,6) weight-b by simp
have butlast-pb:  $\text{set } (\text{butlast } (p @ [b])) \subseteq \text{set settled}$ 
  using p(1) set-p by simp
have new-witnesses:
  bmssp-label-witnesses Gfull src settled
    (bmssp-set-dist b ?nd ds1)
  by (rule bmssp-label-witnesses-set-dist
    [OF tail-witnesses, of p @ [b] b ?nd])
    (use p(1,2) active walk-pb distinct-pb weight-pb butlast-pb in auto)
show ?thesis
proof (cases bmssp-lookup-dist ds1 b)
  case None
  then show ?thesis
    using active rec new-witnesses unfolding e-def by (simp add: Let-def)
next
  case (Some old)
  show ?thesis
  proof (cases ?nd < old)
    case True
    then show ?thesis

```

```

      using active rec Some new-witnesses unfolding e-def
      by (simp add: Let-def)
    next
      case False
      then show ?thesis
        using active rec Some tail-witnesses unfolding e-def
        by (simp add: Let-def)
    qed
  qed
qed
qed

```

lemma *bmssp-relax-edges-preserves-label-witnesses:*
assumes *wf: nat-graph-well-formed G*
and *witnesses: bmssp-label-witnesses G src settled ds*
and *u-settled: u ∈ set settled*
and *lookup-u: bmssp-lookup-dist ds u = Some du*
shows *bmssp-label-witnesses G src settled*
(snd (bmssp-relax-edges G settled u du ds))
by (*rule bmssp-relax-edges-preserves-label-witnesses-aux*
[OF wf - witnesses u-settled lookup-u]) *simp*

lemma *bmssp-relax-vertices-preserves-label-witnesses:*
assumes *wf: nat-graph-well-formed G*
and *witnesses: bmssp-label-witnesses G src settled ds*
and *xs-subset: set xs ⊆ set settled*
shows *bmssp-label-witnesses G src settled*
(snd (bmssp-relax-vertices G settled xs ds))
using *witnesses xs-subset*
proof (*induction xs arbitrary: ds*)
 case Nil
 then show ?case
 by *simp*
 next
 case (Cons u us)
 show ?case
proof (*cases bmssp-lookup-dist ds u*)
 case None
 have *us-subset: set us ⊆ set settled*
 using *Cons.prem(2)* **by** *simp*
 show ?thesis
 using *None Cons.IH[OF Cons.prem(1) us-subset]* **by** *simp*
 next
 case (Some du)
 have *u-settled: u ∈ set settled*
 using *Cons.prem(2)* **by** *simp*
obtain *updates-u ds1* **where** *edge-rec:*
bmssp-relax-edges G settled u du ds = (updates-u, ds1)
by *force*

```

have witnesses1: bmssp-label-witnesses G src settled ds1
  using bmssp-relax-edges-preserves-label-witnesses
  [OF wf Cons.prem1(1) u-settled Some] edge-rec by simp
obtain updates-us ds2 where vertices-rec:
  bmssp-relax-vertices G settled us ds1 = (updates-us, ds2)
  by force
have us-subset: set us  $\subseteq$  set settled
  using Cons.prem1(2) by simp
have witnesses2: bmssp-label-witnesses G src settled ds2
  using Cons.IH[OF witnesses1 us-subset] vertices-rec by simp
show ?thesis
  using Some edge-rec vertices-rec witnesses2 by simp
qed
qed

```

```

lemma bmssp-label-witnesses-lookup-lt-infinity:
  assumes wf: nat-graph-well-formed G
    and witnesses: bmssp-label-witnesses G src settled ds
    and lookup: bmssp-lookup-dist ds v = Some d
  shows d < bmssp-infinity
proof -
  have mem: (v, d)  $\in$  set ds
    by (rule bmssp-lookup-dist-Some-mem[OF lookup])
  then obtain p where p:
    distinct p
    exec-walk (bmssp-vertices G src) (nat-graph-edge-list G) p
    exec-walk-weight (nat-graph-weight G) p = real d
    using witnesses unfolding bmssp-label-witnesses-def by blast
  have d-le-total:
    real d  $\leq$  real (nat-graph-total-weight G)
    using exec-walk-weight-nat-graph-le-total[OF wf p(2) p(1)] p(3)
    by simp
  have total-lt: nat-graph-total-weight G < bmssp-infinity
    using wf unfolding nat-graph-well-formed-def by simp
  show ?thesis
    using d-le-total total-lt by linarith
qed

```

```

lemma bmssp-relax-vertices-update-lt-bound:
  assumes wf: nat-graph-well-formed G
    and witnesses: bmssp-label-witnesses G src settled ds
    and xs-subset: set xs  $\subseteq$  set settled
    and distinct-updates:
      distinct (map fst (fst (bmssp-relax-vertices G settled xs ds)))
    and update: (v, b)  $\in$  set (fst (bmssp-relax-vertices G settled xs ds))
  shows b < bmssp-bound
proof -
  have witnesses':
    bmssp-label-witnesses G src settled

```

```

    (snd (bmssp-relax-vertices G settled xs ds))
  by (rule bmssp-relax-vertices-preserves-label-witnesses
      [OF wf witnesses xs-subset])
have lookup:
  bmssp-lookup-dist (snd (bmssp-relax-vertices G settled xs ds)) v =
    Some (nat (floor b))
  by (rule bmssp-relax-vertices-update-lookup-floor
      [OF distinct-updates update])
have d-lt: nat (floor b) < bmssp-infinity
  by (rule bmssp-label-witnesses-lookup-lt-infinity
      [OF wf witnesses' lookup])
obtain d where b-key: b = bmssp-partition-key v d
  and d-floor: nat (floor b) = d
  using bmssp-relax-vertices-update-floor[OF update] by blast
have key-lt: bmssp-partition-key v d < bmssp-bound
  by (rule bmssp-partition-key-lt-bound-if-distance-lt)
  (use d-lt d-floor in simp)
show ?thesis
  using b-key key-lt by simp
qed

lemma bmssp-loop-lookup-settled:
  assumes distinct: distinct (map fst ds)
  and v-settled: v ∈ set settled
  shows bmssp-lookup-dist (bmssp-loop fuel G vertices settled ds P) v =
    bmssp-lookup-dist ds v
  using distinct v-settled
proof (induction fuel arbitrary: settled ds P)
  case 0
  then show ?case
    by (simp add: bmssp-lookup-dist-normalize-dist)
next
  case (Suc fuel)
  obtain S beta P1 where pull:
    bp-pull bmssp-block-size bmssp-bound P = (S, beta, P1)
  by (cases bp-pull bmssp-block-size bmssp-bound P) auto
  let ?pulled = filter (λx. x ∈ S ∧ x ∉ set settled) vertices
  show ?case
  proof (cases ?pulled = [])
    case True
    then show ?thesis
      using pull Suc.prem1 by (simp add: bmssp-lookup-dist-normalize-dist)
  next
    case False
    let ?settled' = ?pulled @ settled
    have v-settled': v ∈ set ?settled'
      using Suc.prem1 by simp
    obtain updates ds' where relaxed:
      bmssp-relax-vertices G ?settled' ?pulled ds = (updates, ds')

```

```

    by force
  have lookup-ds':
    bmssp-lookup-dist ds' v = bmssp-lookup-dist ds v
    using bmssp-relax-vertices-lookup-settled[OF v-settled', of G ?pulled ds]
    relaxed by simp
  have distinct-ds':
    distinct (map fst ds')
    using bmssp-relax-vertices-preserves-distinct-dist[OF Suc.prem1(1),
      of G ?settled' ?pulled] relaxed by simp
  have rec:
    bmssp-lookup-dist
      (bmssp-loop fuel G vertices ?settled' ds'
        (bmssp-apply-updates updates P1)) v =
      bmssp-lookup-dist ds' v
    by (rule Suc.IH[OF distinct-ds' v-settled'])
  have loop-eq:
    bmssp-loop (Suc fuel) G vertices settled ds P =
    bmssp-loop fuel G vertices ?settled' ds'
      (bmssp-apply-updates updates P1)
    using pull False relaxed by (simp add: Let-def)
  show ?thesis
    using loop-eq rec lookup-ds' by simp
qed
qed

lemma bmssp-loop-preserves-label-witnesses:
  assumes wf: nat-graph-well-formed G
  and witnesses: bmssp-label-witnesses G src settled ds
  shows  $\exists$  settled'. set settled  $\subseteq$  set settled'  $\wedge$ 
    bmssp-label-witnesses G src settled'
    (bmssp-loop fuel G vertices settled ds P)
  using witnesses
proof (induction fuel arbitrary: settled ds P)
  case 0
  have set-norm: set (bmssp-normalize-dist ds) = set ds
    unfolding bmssp-normalize-dist-def by simp
  have witnesses-norm:
    bmssp-label-witnesses G src settled (bmssp-normalize-dist ds)
    using 0.prem1 set-norm unfolding bmssp-label-witnesses-def by auto
  show ?case
proof (intro exI conjI)
  show set settled  $\subseteq$  set settled
    by simp
  show bmssp-label-witnesses G src settled
    (bmssp-loop 0 G vertices settled ds P)
    using witnesses-norm by simp
qed
next
  case (Suc fuel)

```

```

obtain  $S$   $\beta$   $P1$  where  $pull$ :
   $bp\text{-}pull$   $bmssp\text{-}block\text{-}size$   $bmssp\text{-}bound$   $P = (S, \beta, P1)$ 
  by ( $cases$   $bp\text{-}pull$   $bmssp\text{-}block\text{-}size$   $bmssp\text{-}bound$   $P$ )  $auto$ 
let  $?pulled = filter (\lambda x. x \in S \wedge x \notin set\ settled)$   $vertices$ 
show  $?case$ 
proof ( $cases$   $?pulled = []$ )
  case  $True$ 
  have  $set\text{-}norm$ :  $set (bmssp\text{-}normalize\text{-}dist\ ds) = set\ ds$ 
    unfolding  $bmssp\text{-}normalize\text{-}dist\text{-}def$  by  $simp$ 
  have  $witnesses\text{-}norm$ :
     $bmssp\text{-}label\text{-}witnesses\ G\ src\ settled (bmssp\text{-}normalize\text{-}dist\ ds)$ 
    using  $Suc.premis\ set\text{-}norm$  unfolding  $bmssp\text{-}label\text{-}witnesses\text{-}def$  by  $auto$ 
  have  $loop\text{-}eq$ :
     $bmssp\text{-}loop (Suc\ fuel)\ G\ vertices\ settled\ ds\ P =$ 
     $bmssp\text{-}normalize\text{-}dist\ ds$ 
    using  $pull\ True$  by  $simp$ 
  show  $?thesis$ 
proof ( $intro\ exI\ conjI$ )
  show  $set\ settled \subseteq set\ settled$ 
    by  $simp$ 
  show  $bmssp\text{-}label\text{-}witnesses\ G\ src\ settled$ 
    ( $bmssp\text{-}loop (Suc\ fuel)\ G\ vertices\ settled\ ds\ P$ )
    using  $witnesses\text{-}norm\ loop\text{-}eq$  by  $simp$ 
qed
next
  case  $False$ 
  let  $?settled' = ?pulled @ settled$ 
  have  $settled\text{-}subset$ :  $set\ settled \subseteq set\ ?settled'$ 
    by  $auto$ 
  have  $witnesses\text{-}settled'$ :
     $bmssp\text{-}label\text{-}witnesses\ G\ src\ ?settled'\ ds$ 
    by ( $rule\ bmssp\text{-}label\text{-}witnesses\text{-}mono\text{-}settled$ 
      [ $OF\ Suc.premis\ settled\text{-}subset$ ])
  have  $pulled\text{-}subset$ :  $set\ ?pulled \subseteq set\ ?settled'$ 
    by  $simp$ 
  obtain  $updates\ ds'$  where  $relaxed$ :
     $bmssp\text{-}relax\text{-}vertices\ G\ ?settled'\ ?pulled\ ds = (updates, ds')$ 
    by  $force$ 
  have  $witnesses\text{-}ds'$ :
     $bmssp\text{-}label\text{-}witnesses\ G\ src\ ?settled'\ ds'$ 
    using  $bmssp\text{-}relax\text{-}vertices\text{-}preserves\text{-}label\text{-}witnesses$ 
    [ $OF\ wf\ witnesses\text{-}settled'\ pulled\text{-}subset$ ]  $relaxed$  by  $simp$ 
  obtain  $settled''$  where  $settled''$ :
     $set\ ?settled' \subseteq set\ settled''$ 
     $bmssp\text{-}label\text{-}witnesses\ G\ src\ settled''$ 
    ( $bmssp\text{-}loop\ fuel\ G\ vertices\ ?settled'\ ds'$ 
      ( $bmssp\text{-}apply\text{-}updates\ updates\ P1$ ))
    using  $Suc.IH[OF\ witnesses\text{-}ds']$  by  $blast$ 
  have  $loop\text{-}eq$ :

```



```

    bmssp-loop (Suc fuel) G vertices settled ds P =
      bmssp-loop fuel G vertices ?settled' ds'
      (bmssp-apply-updates updates P1)
    using pull False relaxed by (simp add: Let-def)
  have settled-to-final: set settled  $\subseteq$  set settled''
    using settled-subset settled''(1) by blast
  show ?thesis
  proof (intro exI conjI)
    show set settled  $\subseteq$  set settled''
      by (rule settled-to-final)
    show bmssp-label-witnesses G src settled''
      (bmssp-loop (Suc fuel) G vertices settled ds P)
      using settled''(2) loop-eq by simp
  qed
qed
qed

lemma bmssp-distances-label-witnesses:
  assumes wf: nat-graph-well-formed G
  shows  $\exists$  settled. bmssp-label-witnesses G src settled
    (bmssp-distances G src)
  proof -
    let ?vertices = bmssp-vertices G src
    let ?P0 = bp-empty bmssp-block-size bmssp-bound
    let ?P1 =
      bp-regularized-local-insert src (bmssp-partition-key src 0) ?P0
    let ?fuel = Suc (length ?vertices * Suc (length G))
    have initial: bmssp-label-witnesses G src  $\square$  [(src, 0)]
      by (rule bmssp-label-witnesses-initial)
    obtain settled where loop:
      bmssp-label-witnesses G src settled
      (bmssp-loop ?fuel G ?vertices  $\square$  [(src, 0)] ?P1)
    using bmssp-loop-preserves-label-witnesses[OF wf initial] by blast
  have unfold:
    bmssp-distances G src =
      bmssp-loop ?fuel G ?vertices  $\square$  [(src, 0)] ?P1
    unfolding bmssp-distances-def by (simp add: Let-def)
  show ?thesis
  proof
    show bmssp-label-witnesses G src settled (bmssp-distances G src)
      using loop unfold by simp
  qed
qed

lemma bmssp-distances-output-simple-walk:
  assumes wf: nat-graph-well-formed G
  and mem: (v, d)  $\in$  set (bmssp-distances G src)
  obtains p where
    p  $\neq$   $\square$ 

```

```

    hd p = src
    last p = v
    distinct p
    exec-walk (bmssp-vertices G src) (nat-graph-edge-list G) p
    exec-walk-weight (nat-graph-weight G) p = real d
proof –
  obtain settled where witnesses:
    bmssp-label-witnesses G src settled (bmssp-distances G src)
  using bmssp-distances-label-witnesses[OF wf] by blast
  then obtain p where
    p ≠ []
    hd p = src
    last p = v
    distinct p
    exec-walk (bmssp-vertices G src) (nat-graph-edge-list G) p
    exec-walk-weight (nat-graph-weight G) p = real d
  using mem unfolding bmssp-label-witnesses-def by blast
  then show ?thesis
  using that by blast
qed

lemma bmssp-distances-distinct-keys:
  distinct (map fst (bmssp-distances G src))
proof –
  let ?vertices = bmssp-vertices G src
  let ?P0 = bp-empty bmssp-block-size bmssp-bound
  let ?P1 =
    bp-result-of
      (c-bp-regularized-local-insert src (bmssp-partition-key src 0) ?P0)
  let ?fuel = Suc (length ?vertices * Suc (length G))
  have unfold:
    bmssp-distances G src =
      bmssp-loop ?fuel G ?vertices [] [(src, 0)] ?P1
  unfolding bmssp-distances-def by (simp add: Let-def)
  have distinct-loop:
    distinct
      (map fst (bmssp-loop ?fuel G ?vertices [] [(src, 0)] ?P1))
  by (rule bmssp-loop-preserves-distinct-output) simp
  show ?thesis
  using unfold distinct-loop by simp
qed

lemma bmssp-distances-keys-subset-bmssp-vertices:
  set (map fst (bmssp-distances G src)) ⊆ set (bmssp-vertices G src)
proof –
  let ?vertices = bmssp-vertices G src
  let ?P0 = bp-empty bmssp-block-size bmssp-bound
  let ?P1 =
    bp-result-of

```

```

    (c-bp-regularized-local-insert src (bmssp-partition-key src 0) ?P0)
  let ?fuel = Suc (length ?vertices * Suc (length G))
  have initial: set (map fst [(src, 0)])  $\subseteq$  set ?vertices
    unfolding bmssp-vertices-def by simp
  have targets:
     $\bigwedge a\ b\ w. (a, b, w) \in \text{set } G \implies b \in \text{set } ?vertices$ 
    by (rule bmssp-edge-target-in-vertices)
  have set (map fst
    (bmssp-loop ?fuel G ?vertices [] [(src, 0)] ?P1))
     $\subseteq$  set ?vertices
    by (rule bmssp-loop-output-keys-subset-vertices[OF initial targets])
  then show ?thesis
    unfolding bmssp-distances-def by (simp add: Let-def)
qed

```

```

lemma bmssp-distances-encode-own-keys:
  bmssp-distances G src =
    encode-dist-assoc-list (map fst (bmssp-distances G src))
    (executable-label-of (bmssp-distances G src))
proof -
  let ?vertices = bmssp-vertices G src
  let ?P0 = bp-empty bmssp-block-size bmssp-bound
  let ?P1 =
    bp-result-of
      (c-bp-regularized-local-insert src (bmssp-partition-key src 0) ?P0)
  let ?fuel = Suc (length ?vertices * Suc (length G))
  have loop:
    bmssp-loop ?fuel G ?vertices [] [(src, 0)] ?P1 =
      encode-dist-assoc-list
        (map fst (bmssp-loop ?fuel G ?vertices [] [(src, 0)] ?P1))
        (executable-label-of
          (bmssp-loop ?fuel G ?vertices [] [(src, 0)] ?P1))
    by (rule bmssp-loop-output-encode) simp
  show ?thesis
    using loop unfolding bmssp-distances-def by (simp add: Let-def)
qed

```

```

lemma nat-graph-finite-weighted-digraph:
  assumes wf: nat-graph-well-formed G
  and src: src  $\in$  nat-graph-vertices G
  shows finite-weighted-digraph
    (nat-graph-vertices G) (nat-graph-edges G) (nat-graph-weight G) src
proof
  show finite (nat-graph-vertices G)
    unfolding nat-graph-vertices-def by simp
  show src  $\in$  nat-graph-vertices G
    by (rule src)
  show  $\bigwedge u\ v. (u, v) \in \text{nat-graph-edges } G \implies$ 
     $u \in \text{nat-graph-vertices } G \wedge v \in \text{nat-graph-vertices } G$ 

```

using *nat-graph-edge-in-vertices* by *blast*
 show $\bigwedge u v. (u, v) \in \text{nat-graph-edges } G \implies$
 $0 \leq \text{nat-graph-weight } G \ u \ v$
 by *simp*
 qed

lemma *nat-graph-reachable-iff-exec-reachable*:
 assumes *wf*: *nat-graph-well-formed* *G*
 and *src*: *src* $\in$ *nat-graph-vertices* *G*
 shows *exec-reachable* (*nat-graph-vertex-list* *G*) (*nat-graph-edge-list* *G*) *a* *b*
 $\longleftrightarrow$ *nat-graph-reachable* *G* *a* *b*
proof –
 interpret *concrete*: *finite-weighted-digraph*
nat-graph-vertices *G* *nat-graph-edges* *G* *nat-graph-weight* *G* *src*
 by (rule *nat-graph-finite-weighted-digraph*[*OF wf src*])
 have *exec*:
exec-reachable (*nat-graph-vertex-list* *G*) (*nat-graph-edge-list* *G*) *a* *b*
 $\longleftrightarrow$ *concrete.reachable* *a* *b*
 by (rule *concrete.exec-reachable-iff-reachable*
 [*OF nat-graph-vertex-list-set nat-graph-edge-list-set*])
 show ?thesis
 using *exec* **unfolding** *nat-graph-reachable-def* .
 qed

lemma *nat-graph-exec-dist-eq-dist*:
 assumes *wf*: *nat-graph-well-formed* *G*
 and *src*: *src* $\in$ *nat-graph-vertices* *G*
 and *reach*: *nat-graph-reachable* *G* *a* *b*
 shows *exec-dist* (*nat-graph-vertex-list* *G*) (*nat-graph-edge-list* *G*)
 (*nat-graph-weight* *G*) *a* *b* =
nat-graph-dist *G* *a* *b*
proof –
 interpret *concrete*: *finite-weighted-digraph*
nat-graph-vertices *G* *nat-graph-edges* *G* *nat-graph-weight* *G* *src*
 by (rule *nat-graph-finite-weighted-digraph*[*OF wf src*])
 have *reach'*: *concrete.reachable* *a* *b*
 using *reach* **unfolding** *nat-graph-reachable-def* .
 show ?thesis
unfolding *nat-graph-dist-def*
 by (rule *concrete.exec-dist-eq-dist*[*OF nat-graph-vertex-list-set*
nat-graph-edge-list-set reach'])
 qed

lemma *nat-graph-weight-integral*:
 $\exists n. \text{nat-graph-weight } G \ u \ v = \text{real } n$
proof (cases *map-of*
 (*map* ($\lambda e. ((\text{nat-edge-source } e, \text{nat-edge-target } e),$
 $\text{real } (\text{nat-edge-weight } e)))$ *G*) (*u*, *v*))
 case *None*

```

    then show ?thesis
      unfolding nat-graph-weight-def by simp
next
case (Some c)
then have  $((u, v), c) \in \text{set}$ 
  (map  $(\lambda e. ((\text{nat-edge-source } e, \text{nat-edge-target } e),$ 
    real (nat-edge-weight e))) G)
  by (rule map-of-SomeD)
then obtain e where  $e \in \text{set } G$ 
  and  $c: c = \text{real } (\text{nat-edge-weight } e)$ 
  by auto
then show ?thesis
  unfolding nat-graph-weight-def using Some by auto
qed

```

```

lemma exec-walk-weight-nat-graph-integral:
   $\exists n. \text{exec-walk-weight } (\text{nat-graph-weight } G) \text{ } p = \text{real } n$ 
proof (induction p)
case Nil
then show ?case
  by simp
next
case (Cons x xs)
show ?case
proof (cases xs)
case Nil
then show ?thesis
  by simp
next
case (Cons y ys)
obtain n where  $w: \text{nat-graph-weight } G \text{ } x \text{ } y = \text{real } n$ 
  using nat-graph-weight-integral[of G x y] by blast
obtain m where tail:
   $\text{exec-walk-weight } (\text{nat-graph-weight } G) \text{ } (y \# \text{ys}) = \text{real } m$ 
  using Cons.IH Cons by blast
have  $\text{exec-walk-weight } (\text{nat-graph-weight } G) \text{ } (x \# y \# \text{ys}) =$ 
   $\text{real } (n + m)$ 
  using w tail by simp
then show ?thesis
  using Cons by blast
qed
qed

```

```

lemma min-list-integral-nonempty:
  assumes  $xs \neq []$ 
  and  $\bigwedge x. x \in \text{set } xs \implies \exists n. x = \text{real } n$ 
  shows  $\exists n. \text{min-list } xs = \text{real } n$ 
  using assms
proof (induction xs)

```

```

    case Nil
    then show ?case
      by simp
next
case (Cons x xs)
obtain n where x: x = real n
  using Cons.prem1(2) by auto
show ?case
proof (cases xs)
  case Nil
  then show ?thesis
    using x by simp
next
case (Cons y ys)
have tail-nonempty: xs ≠ []
  using Cons by simp
have tail-integral:  $\bigwedge z. z \in \text{set } xs \implies \exists m. z = \text{real } m$ 
  using Cons.prem1 by simp
obtain m where tail: min-list xs = real m
  using Cons.IH[OF tail-nonempty tail-integral] by blast
have min-list (x # xs) = min x (min-list xs)
  using Cons by simp
also have ... = real (min n m)
  using x tail by simp
finally have min-list (x # xs) = real (min n m) .
then show ?thesis
  by blast
qed
qed

lemma exec-dist-nat-graph-integral:
   $\exists n. \text{exec-dist } vs \text{ es } (\text{nat-graph-weight } G) \ a \ b = \text{real } n$ 
proof (cases exec-walk-weights vs es (nat-graph-weight G) a b)
  case Nil
  then show ?thesis
    unfolding exec-dist-def by simp
next
case (Cons w ws)
have integral:
   $\bigwedge x. x \in \text{set } (w \# ws) \implies \exists n. x = \text{real } n$ 
proof -
  fix x
  assume x ∈ set (w # ws)
  then obtain p where
    p ∈ set (exec-simple-walks-betw vs es a b)
    and x: x = exec-walk-weight (nat-graph-weight G) p
  unfolding Cons[symmetric] exec-walk-weights-def by auto
then show  $\exists n. x = \text{real } n$ 
  using exec-walk-weight-nat-graph-integral[of G p] by blast

```

```

qed
have nonempty:  $w \# ws \neq []$ 
  by simp
obtain n where min-list ( $w \# ws$ ) = real n
  using min-list-integral-nonempty[OF nonempty integral] by blast
then show ?thesis
  unfolding exec-dist-def Cons by simp
qed

lemma nat-graph-dist-integral:
  assumes wf: nat-graph-well-formed G
    and src:  $src \in \text{nat-graph-vertices } G$ 
    and reach: nat-graph-reachable G a b
  shows  $\exists n. \text{nat-graph-dist } G a b = \text{real } n$ 
proof -
  have exec-eq:
    exec-dist (nat-graph-vertex-list G) (nat-graph-edge-list G)
      (nat-graph-weight G) a b =
      nat-graph-dist G a b
  by (rule nat-graph-exec-dist-eq-dist[OF wf src reach])
  obtain n where exec-dist (nat-graph-vertex-list G) (nat-graph-edge-list G)
    (nat-graph-weight G) a b = real n
  using exec-dist-nat-graph-integral by blast
  then have nat-graph-dist G a b = real n
  using exec-eq by simp
  then show ?thesis
  by blast
qed

lemma real-nat-floor-integral:
  assumes  $\exists n::\text{nat}. x = \text{real } n$ 
  shows  $\text{real } (\text{nat } (\text{floor } x)) = x$ 
  using assms by auto

locale nat-graph-instance =
  fixes G :: nat-graph
    and src :: nat
  assumes wf: nat-graph-well-formed G
    and src-in:  $src \in \text{nat-graph-vertices } G$ 
begin

interpretation concrete: finite-weighted-digraph
  nat-graph-vertices G nat-graph-edges G nat-graph-weight G src
  by (rule nat-graph-finite-weighted-digraph[OF wf src-in])

lemma exec-reachable-iff-reachable:
  exec-reachable (nat-graph-vertex-list G) (nat-graph-edge-list G) a b
 $\longleftrightarrow$  concrete.reachable a b
  by (rule concrete.exec-reachable-iff-reachable)

```

[*OF nat-graph-vertex-list-set nat-graph-edge-list-set*])

lemma *exec-dist-eq-dist*:

assumes *concrete.reachable a b*

shows *exec-dist (nat-graph-vertex-list G) (nat-graph-edge-list G)*
(nat-graph-weight G) a b =

concrete.dist a b

by (*rule concrete.exec-dist-eq-dist[OF nat-graph-vertex-list-set*
nat-graph-edge-list-set assms])

lemma *bmssp-vertices-carrier*:

set (bmssp-vertices G src) = nat-graph-vertices G

using *bmssp-vertices-set-if-source[OF src-in]* .

lemma *bmssp-distances-keys-subset-carrier*:

set (map fst (bmssp-distances G src)) ⊆ nat-graph-vertices G

using *bmssp-distances-keys-subset-bmssp-vertices[of G src]*
bmssp-vertices-carrier by simp

lemma *bmssp-distances-encode-filtered-vertex-list*:

bmssp-distances G src =

encode-dist-assoc-list

(filter (λv. v ∈ set (map fst (bmssp-distances G src)))

(nat-graph-vertex-list G))

(executable-label-of (bmssp-distances G src))

proof –

let *?U = set (map fst (bmssp-distances G src))*

have *filter-set*:

set (filter (λv. v ∈ ?U) (nat-graph-vertex-list G)) =

set (map fst (bmssp-distances G src))

using *bmssp-distances-keys-subset-carrier by auto*

have *own*:

bmssp-distances G src =

encode-dist-assoc-list (map fst (bmssp-distances G src))

(executable-label-of (bmssp-distances G src))

by (*rule bmssp-distances-encode-own-keys*)

have *encode-eq*:

encode-dist-assoc-list

(filter (λv. v ∈ ?U) (nat-graph-vertex-list G))

(executable-label-of (bmssp-distances G src)) =

encode-dist-assoc-list (map fst (bmssp-distances G src))

(executable-label-of (bmssp-distances G src))

by (*rule encode-dist-assoc-list-cong-set[OF filter-set]*)

show *?thesis*

using *own encode-eq by simp*

qed

lemma *bmssp-distances-output-abstract-simple-walk*:

assumes *mem: (v, d) ∈ set (bmssp-distances G src)*

obtains p **where**
 concrete.simple-walk-betw $\text{src } p \ v$
 concrete.walk-weight $p = \text{real } d$
proof –
 obtain p **where** p :
 $p \neq []$
 hd $p = \text{src}$
 last $p = v$
 distinct p
 exec-walk (*bmssp-vertices* $G \ \text{src}$) (*nat-graph-edge-list* G) p
 exec-walk-weight (*nat-graph-weight* G) $p = \text{real } d$
 by (*rule bmssp-distances-output-simple-walk*[*OF wf mem*])
 have *walk-p*: *concrete.walk* p
 using $p(5)$
 by (*simp add*: *concrete.exec-walk-iff-walk*
 [*OF bmssp-vertices-carrier nat-graph-edge-list-set*])
 have *simple*: *concrete.simple-walk-betw* $\text{src } p \ v$
 using $p(1-4)$ *walk-p*
 unfolding *concrete.simple-walk-betw-def* *concrete.walk-betw-def* **by** *blast*
 have *weight*: *concrete.walk-weight* $p = \text{real } d$
 using $p(6)$ **by** *simp*
 show *?thesis*
 by (*rule that*[*OF simple weight*])
qed

lemma *bmssp-distances-output-reachable*:
 assumes $(v, d) \in \text{set } (\text{bmssp-distances } G \ \text{src})$
 shows *nat-graph-reachable* $G \ \text{src } v$
proof –
 obtain p **where** *concrete.simple-walk-betw* $\text{src } p \ v$
 by (*rule bmssp-distances-output-abstract-simple-walk*[*OF assms*])
 then have *concrete.reachable* $\text{src } v$
 unfolding *concrete.reachable-def* **by** *blast*
 then show *?thesis*
 unfolding *nat-graph-reachable-def* .
qed

lemma *bmssp-distances-output-dist-le*:
 assumes *mem*: $(v, d) \in \text{set } (\text{bmssp-distances } G \ \text{src})$
 shows *nat-graph-dist* $G \ \text{src } v \leq \text{real } d$
proof –
 obtain p **where** *simple*: *concrete.simple-walk-betw* $\text{src } p \ v$
 and *weight*: *concrete.walk-weight* $p = \text{real } d$
 by (*rule bmssp-distances-output-abstract-simple-walk*[*OF mem*])
 have *weight-mem*:
 $\text{real } d \in \text{concrete.simple-walk-weights } \text{src } v$
 proof –
 have *concrete.walk-weight* $p \in \text{concrete.walk-weight '}$
 $\{p. \text{concrete.simple-walk-betw } \text{src } p \ v\}$

```

    using simple by blast
  then show ?thesis
    using weight unfolding concrete.simple-walk-weights-def by simp
qed
have concrete.dist src v  $\leq$  real d
  unfolding concrete.dist-def
  by (rule Min-le[OF concrete.finite-simple-walk-weights weight-mem])
then show ?thesis
  unfolding nat-graph-dist-def by simp
qed

lemma bmssp-label-witnesses-lookup-abstract-simple-walk:
  assumes witnesses: bmssp-label-witnesses G src settled ds
    and lookup: bmssp-lookup-dist ds v = Some d
  obtains p where
    concrete.simple-walk-betw src p v
    concrete.walk-weight p = real d
proof -
  have mem:  $(v, d) \in \text{set } ds$ 
    by (rule bmssp-lookup-dist-Some-mem[OF lookup])
  then obtain p where p:
    p  $\neq []$ 
    hd p = src
    last p = v
    distinct p
    exec-walk (bmssp-vertices G src) (nat-graph-edge-list G) p
    exec-walk-weight (nat-graph-weight G) p = real d
    using witnesses unfolding bmssp-label-witnesses-def by blast
  have walk-p: concrete.walk p
    using p(5)
  by (simp add: concrete.exec-walk-iff-walk
    [OF bmssp-vertices-carrier nat-graph-edge-list-set])
  have simple: concrete.simple-walk-betw src p v
    using p(1-4) walk-p
    unfolding concrete.simple-walk-betw-def concrete.walk-betw-def by blast
  have weight: concrete.walk-weight p = real d
    using p(6) by simp
  show ?thesis
    by (rule that[OF simple weight])
qed

lemma bmssp-label-witnesses-lookup-reachable:
  assumes witnesses: bmssp-label-witnesses G src settled ds
    and lookup: bmssp-lookup-dist ds v = Some d
  shows nat-graph-reachable G src v
proof -
  obtain p where concrete.simple-walk-betw src p v
    by (rule bmssp-label-witnesses-lookup-abstract-simple-walk
      [OF witnesses lookup])

```

```

then have concrete.reachable src v
  unfolding concrete.reachable-def by blast
then show ?thesis
  unfolding nat-graph-reachable-def .
qed

lemma bmssp-label-witnesses-lookup-dist-le:
  assumes witnesses: bmssp-label-witnesses G src settled ds
    and lookup: bmssp-lookup-dist ds v = Some d
  shows nat-graph-dist G src v ≤ real d
proof -
  obtain p where simple: concrete.simple-walk-betw src p v
    and weight: concrete.walk-weight p = real d
  by (rule bmssp-label-witnesses-lookup-abstract-simple-walk
      [OF witnesses lookup])
  have weight-mem:
    real d ∈ concrete.simple-walk-weights src v
  proof -
    have concrete.walk-weight p ∈ concrete.walk-weight ‘
      {p. concrete.simple-walk-betw src p v}
    using simple by blast
    then show ?thesis
      using weight unfolding concrete.simple-walk-weights-def by simp
  qed
  have concrete.dist src v ≤ real d
    unfolding concrete.dist-def
  by (rule Min-le[OF concrete.finite-simple-walk-weights weight-mem])
  then show ?thesis
    unfolding nat-graph-dist-def by simp
qed

lemma bmssp-shortest-walk-first-unsettled-lookup-le-dist:
  assumes sp: concrete.shortest-walk src p u
    and u-unsettled: u ∉ set settled
    and source-zero: bmssp-source-zero src ds
    and settled-lookup: bmssp-settled-have-lookups settled ds
    and exact: bmssp-settled-exact G src settled ds
    and frontier: bmssp-frontier-relaxed G src settled ds
  obtains y dy where
    y ∈ set p
    y ∉ set settled
    bmssp-lookup-dist ds y = Some dy
    real dy ≤ nat-graph-dist G src y
    nat-graph-dist G src y ≤ nat-graph-dist G src u
proof -
  have p-ne: p ≠ [] and hd-p: hd p = src and last-p: last p = u
    using concrete.shortest-walk-hd[OF sp] by blast+
  have last-idx: length p - 1 < length p
    using p-ne by simp

```

```

have last-nth: p ! (length p - 1) = u
  using last-conv-nth[OF p-ne] last-p by simp
let ?P =  $\lambda i. i < \text{length } p \wedge p ! i \notin \text{set settled}$ 
have ex-unsettled:  $\exists i. ?P i$ 
  using last-idx last-nth u-unsettled by blast
define i where i = (LEAST i. ?P i)
have i-props:  $i < \text{length } p \wedge p ! i \notin \text{set settled}$ 
  using LeastI-ex[OF ex-unsettled] unfolding i-def by blast+
have before-settled:
   $\bigwedge j. j < i \implies p ! j \in \text{set settled}$ 
proof -
  fix j
  assume j-lt:  $j < i$ 
  then have j-len:  $j < \text{length } p$ 
    using i-props by simp
  show  $p ! j \in \text{set settled}$ 
  proof (rule ccontr)
    assume p ! j  $\notin \text{set settled}$ 
    then have ?P j
      using j-len by blast
    then have  $i \leq j$ 
      unfolding i-def by (rule Least-le)
    then show False
      using j-lt by simp
  qed
qed
let ?y = p ! i
have y-mem:  $?y \in \text{set } p$ 
  using nth-mem[OF i-props(1)] .
have y-dist-u:  $\text{nat-graph-dist } G \text{ src } ?y \leq \text{nat-graph-dist } G \text{ src } u$ 
proof -
  have concrete.dist src  $?y \leq \text{concrete.dist src } u$ 
    using concrete.shortest-walk-prefix-dist-le[OF sp y-mem] .
  then show ?thesis
    unfolding nat-graph-dist-def by simp
qed
have y-lookup-le:  $\exists dy. \text{bmssp-lookup-dist } ds ?y = \text{Some } dy \wedge$ 
   $\text{real } dy \leq \text{nat-graph-dist } G \text{ src } ?y$ 
proof (cases i)
  case 0
  then have y-src:  $?y = \text{src}$ 
    using hd-p p-ne by (cases p) auto
  have lookup:  $\text{bmssp-lookup-dist } ds ?y = \text{Some } 0$ 
    using source-zero y-src unfolding bmssp-source-zero-def by simp
  have  $\text{real } 0 \leq \text{nat-graph-dist } G \text{ src } ?y$ 
    using concrete.dist-reft-zero[OF src-in] y-src
    unfolding nat-graph-dist-def by simp
  then show ?thesis
    using lookup by blast

```

```

next
  case (Suc j)
  let ?x = p ! j
  have j-lt-i:  $j < i$ 
    using Suc by simp
  have j-len:  $j < \text{length } p$ 
    using j-lt-i i-props by simp
  have x-settled:  $?x \in \text{set settled}$ 
    by (rule before-settled[OF j-lt-i])
  obtain dx where lookup-x:  $\text{bmssp-lookup-dist } ds \text{ } ?x = \text{Some } dx$ 
    using settled-lookup x-settled
    unfolding bmssp-settled-have-lookups-def by blast
  have exact-x:  $\text{real } dx = \text{nat-graph-dist } G \text{ src } ?x$ 
    using exact x-settled lookup-x
    unfolding bmssp-settled-exact-def by blast
  have suc-i:  $\text{Suc } j = i$ 
    using Suc by simp
  have edge-idx:  $\text{Suc } j < \text{length } p$ 
    using i-props suc-i by simp
  have tight:  $\text{concrete.tight-edge-step } ?x \text{ } ?y$ 
    using concrete.shortest-walk-successively-tight[OF sp]
    unfolding successively-conv-nth
    using edge-idx suc-i by blast
  have edge:  $(?x, ?y) \in \text{nat-graph-edges } G$ 
    using tight unfolding concrete.tight-edge-step-def by blast
  obtain w where edge-w:  $(?x, ?y, w) \in \text{set } G$ 
    using edge unfolding nat-graph-edges-def nat-graph-edge-list-def by auto
  obtain dy where lookup-y:  $\text{bmssp-lookup-dist } ds \text{ } ?y = \text{Some } dy$ 
    and dy-le:  $\text{real } dy \leq \text{real } dx + \text{real } w$ 
    using frontier edge-w x-settled i-props(2) lookup-x exact-x
    unfolding bmssp-frontier-relaxed-def by blast
  have weight-eq:  $\text{nat-graph-weight } G \text{ } ?x \text{ } ?y = \text{real } w$ 
    by (rule nat-graph-weight-of-edge[OF wf edge-w])
  have dist-y-eq:
     $\text{nat-graph-dist } G \text{ src } ?y =$ 
     $\text{nat-graph-dist } G \text{ src } ?x + \text{real } w$ 
    using tight weight-eq
    unfolding concrete.tight-edge-step-def nat-graph-dist-def by simp
  have real dy  $\leq \text{nat-graph-dist } G \text{ src } ?y$ 
    using dy-le exact-x dist-y-eq by simp
  then show ?thesis
    using lookup-y by blast
qed
obtain dy where lookup-y:  $\text{bmssp-lookup-dist } ds \text{ } ?y = \text{Some } dy$ 
  and dy-le:  $\text{real } dy \leq \text{nat-graph-dist } G \text{ src } ?y$ 
  using y-lookup-le by blast
show thesis
  by (rule that[OF y-mem i-props(2) lookup-y dy-le y-dist-u])
qed

```

lemma *bmssp-partition-state-pulled-label-le-dist*:

assumes *match*: *bmssp-partition-state-match* *vertices settled ds P*

and *source-zero*: *bmssp-source-zero src ds*

and *settled-lookup*: *bmssp-settled-have-lookups settled ds*

and *witnesses*: *bmssp-label-witnesses G src settled ds*

and *exact*: *bmssp-settled-exact G src settled ds*

and *frontier*: *bmssp-frontier-relaxed G src settled ds*

and *pull*: *pull-separates (bp-view P) bmssp-block-size B S beta*
(bp-view P1)

and *uS*: $u \in S$

and *lookup-u*: *bmssp-lookup-dist ds u = Some du*

shows $\text{real } du \leq \text{nat-graph-dist } G \text{ src } u$

proof –

have *u-unsettled*: $u \notin \text{set settled}$

by (rule *bmssp-partition-state-pulled-not-settled*[*OF match pull uS*])

have *reach-u*: *concrete.reachable src u*

using *bmssp-label-witnesses-lookup-reachable*[*OF witnesses lookup-u*]

unfolding *nat-graph-reachable-def* .

obtain *p* **where** *sp*: *concrete.shortest-walk src p u*

using *concrete.shortest-walk-exists*[*OF reach-u*] **by** *blast*

obtain *y dy* **where** *y-mem*: $y \in \text{set } p$

and *y-unsettled*: $y \notin \text{set settled}$

and *lookup-y*: *bmssp-lookup-dist ds y = Some dy*

and *dy-le-dist*: $\text{real } dy \leq \text{nat-graph-dist } G \text{ src } y$

and *dist-y-le-u*: $\text{nat-graph-dist } G \text{ src } y \leq \text{nat-graph-dist } G \text{ src } u$

by (rule *bmssp-shortest-walk-first-unsettled-lookup-le-dist*
[*OF sp u-unsettled source-zero settled-lookup exact frontier*])

have *du-le-dy*: $du \leq dy$

proof (*cases y ∈ S*)

case *True*

have *S-subset*: $S \subseteq \text{keys-of } (bp\text{-view } P)$

by (rule *pull-separates-subset*[*OF pull*])

have *finite-S*: *finite S*

by (rule *finite-subset*[*OF S-subset*]) *simp*

have *card-S*: $\text{card } S \leq 1$

using *pull-separates-card*[*OF pull*] **unfolding** *bmssp-block-size-def* .

have $u = y$

by (rule *finite-card-le-one-member-eq*[*OF finite-S card-S uS True*])

then show *?thesis*

using *lookup-u lookup-y* **by** *simp*

next

case *False*

show *?thesis*

by (rule *bmssp-partition-state-pull-residual-label-le*
[*OF match pull uS lookup-u lookup-y y-unsettled False*])

qed

have $\text{real } du \leq \text{real } dy$

using *du-le-dy* **by** *simp*

also have $\dots \leq \text{nat-graph-dist } G \text{ src } y$
 by (rule dy-le-dist)
 also have $\dots \leq \text{nat-graph-dist } G \text{ src } u$
 by (rule dist-y-le-u)
 finally show ?thesis .
 qed

lemma *bmssp-partition-state-pulled-label-exact:*
 assumes *match: bmssp-partition-state-match vertices settled ds P*
 and *source-zero: bmssp-source-zero src ds*
 and *settled-lookup: bmssp-settled-have-lookups settled ds*
 and *witnesses: bmssp-label-witnesses G src settled ds*
 and *exact: bmssp-settled-exact G src settled ds*
 and *frontier: bmssp-frontier-relaxed G src settled ds*
 and *pull: pull-separates (bp-view P) bmssp-block-size B S beta*
 (*bp-view P1*)
 and *uS: u ∈ S*
 and *lookup-u: bmssp-lookup-dist ds u = Some du*
 shows *real du = nat-graph-dist G src u*
proof –
 have *lower: nat-graph-dist G src u ≤ real du*
 by (rule *bmssp-label-witnesses-lookup-dist-le*
 [*OF witnesses lookup-u*])
 have *upper: real du ≤ nat-graph-dist G src u*
 by (rule *bmssp-partition-state-pulled-label-le-dist*
 [*OF match source-zero settled-lookup witnesses exact frontier*
pull uS lookup-u])
 show ?thesis
 using *lower upper* by simp
 qed

lemma *bmssp-settled-exact-partition-step:*
 assumes *match: bmssp-partition-state-match vertices old-settled ds P*
 and *source-zero: bmssp-source-zero src ds*
 and *settled-lookup: bmssp-settled-have-lookups old-settled ds*
 and *witnesses: bmssp-label-witnesses G src old-settled ds*
 and *exact: bmssp-settled-exact G src old-settled ds*
 and *frontier: bmssp-frontier-relaxed G src old-settled ds*
 and *pull: pull-separates (bp-view P) bmssp-block-size B S beta*
 (*bp-view P1*)
 and *pulled-def:*
pulled = filter (λx. x ∈ S ∧ x ∉ set old-settled)
vertices
 and *settled-def: settled = pulled @ old-settled*
 and *relaxed: bmssp-relax-vertices G settled pulled ds = (updates, ds')*
 shows *bmssp-settled-exact G src settled ds'*
proof –
 have *pulled-exact:*
 $\bigwedge v d. \llbracket v \in \text{set pulled}; \text{bmssp-lookup-dist } ds \text{ } v = \text{Some } d \rrbracket$

```

     $\implies \text{real } d = \text{nat-graph-dist } G \text{ src } v$ 
proof –
  fix  $v \ d$ 
  assume  $v\text{-pulled}: v \in \text{set pulled}$ 
  and  $\text{lookup-v}: \text{bmssp-lookup-dist } ds \ v = \text{Some } d$ 
  have  $vS: v \in S$ 
  using  $v\text{-pulled}$  unfolding  $\text{pulled-def}$  by  $\text{simp}$ 
  show  $\text{real } d = \text{nat-graph-dist } G \text{ src } v$ 
  by ( $\text{rule bmssp-partition-state-pulled-label-exact}$ 
    [ $OF \text{ match source-zero settled-lookup witnesses exact frontier}$ 
       $\text{pull } vS \text{ lookup-v}$ ])
qed
show  $?thesis$ 
  by ( $\text{rule bmssp-settled-exact-step}$ 
    [ $OF \text{ exact pulled-exact settled-def relaxed}$ ])
qed

lemma  $\text{bmssp-dijkstra-state-step-bridge}$ :
fixes  $\text{old-settled settled vertices} :: \text{nat list}$ 
and  $\text{pulled} :: \text{nat list}$ 
and  $\text{updates} :: (\text{nat} \times \text{real}) \text{ list}$ 
assumes  $\text{state}: \text{bmssp-dijkstra-state } G \text{ src vertices old-settled } ds \ P$ 
and  $\text{edge-targets}$ :
   $\bigwedge a \ b \ w. (a, b, w) \in \text{set } G \implies b \in \text{set vertices}$ 
and  $\text{pulled-def}$ :
   $\text{pulled} = \text{filter } (\lambda x. x \in S \wedge x \notin \text{set old-settled})$ 
   $\text{vertices}$ 
and  $\text{settled-def}: \text{settled} = \text{pulled} @ \text{old-settled}$ 
and  $\text{relaxed}: \text{bmssp-relax-vertices } G \text{ settled pulled } ds = (\text{updates}, ds')$ 
and  $P2\text{-def}: P2 = \text{bmssp-apply-updates updates } P1$ 
and  $\text{distinct-vertices}: \text{distinct vertices}$ 
and  $\text{ord}: \text{bp-ordered-invariant } P$ 
and  $\text{upper}: \text{partition-upper-bound } (\text{bp-view } P) \ B$ 
and  $\text{pull}: \text{bp-pull bmssp-block-size } B \ P = (S, \text{beta}, P1)$ 
shows  $\text{bmssp-dijkstra-state } G \text{ src vertices settled } ds' \ P2$ 
proof –
  have  $\text{match}: \text{bmssp-partition-state-match vertices old-settled } ds \ P$ 
  and  $\text{zero}: \text{bmssp-source-zero src } ds$ 
  and  $\text{settled-lookup}: \text{bmssp-settled-have-lookups old-settled } ds$ 
  and  $\text{witnesses}: \text{bmssp-label-witnesses } G \text{ src old-settled } ds$ 
  and  $\text{exact}: \text{bmssp-settled-exact } G \text{ src old-settled } ds$ 
  and  $\text{frontier}: \text{bmssp-frontier-relaxed } G \text{ src old-settled } ds$ 
  using  $\text{state}$  unfolding  $\text{bmssp-dijkstra-state-def}$  by  $\text{auto}$ 
  have  $\text{distinct-ds}: \text{distinct } (\text{map fst } ds)$ 
  using  $\text{match}$  unfolding  $\text{bmssp-partition-state-match-def}$  by  $\text{simp}$ 
  have  $\text{updates-def}: \text{updates} = \text{fst } (\text{bmssp-relax-vertices } G \text{ settled pulled } ds)$ 
  using  $\text{relaxed}$  by  $\text{simp}$ 
  have  $\text{sep}$ :
     $\text{pull-separates } (\text{bp-view } P) \ \text{bmssp-block-size } B \ S \ \text{beta } (\text{bp-view } P1)$ 

```



```

  by (rule bmssp-loop-partition-step-bridge(1)
    [OF pulled-def updates-def P2-def wf distinct-vertices ord upper
      pull])
  have match':
    bmssp-partition-state-match vertices settled ds' P2
  by (rule bmssp-partition-state-match-step-bridge
    [OF match edge-targets pulled-def settled-def relaxed P2-def wf
      distinct-vertices ord upper pull])
  have zero': bmssp-source-zero src ds'
  by (rule bmssp-source-zero-step[OF zero distinct-ds relaxed])
  have settled-lookup':
    bmssp-settled-have-lookups settled ds'
  by (rule bmssp-settled-have-lookups-step
    [OF settled-lookup match sep pulled-def settled-def relaxed])
  have settled-subset: set old-settled  $\subseteq$  set settled
  using settled-def by auto
  have witnesses-settled:
    bmssp-label-witnesses G src settled ds
  by (rule bmssp-label-witnesses-mono-settled
    [OF witnesses settled-subset])
  have pulled-subset: set pulled  $\subseteq$  set settled
  using settled-def by simp
  have witnesses':
    bmssp-label-witnesses G src settled ds'
  proof -
    have bmssp-label-witnesses G src settled
      (snd (bmssp-relax-vertices G settled pulled ds))
    by (rule bmssp-relax-vertices-preserves-label-witnesses
      [OF wf witnesses-settled pulled-subset])
    then show ?thesis
      using relaxed by simp
  qed
  have exact': bmssp-settled-exact G src settled ds'
  by (rule bmssp-settled-exact-partition-step
    [OF match zero settled-lookup witnesses exact frontier sep
      pulled-def settled-def relaxed])
  have frontier': bmssp-frontier-relaxed G src settled ds'
  by (rule bmssp-frontier-relaxed-step
    [OF frontier distinct-ds settled-def relaxed])
  show ?thesis
    unfolding bmssp-dijkstra-state-def
    using match' zero' settled-lookup' witnesses' exact' frontier' by simp
  qed

lemma bmssp-dijkstra-state-step-upper-bound:
  fixes old-settled settled vertices :: nat list
  and pulled :: nat list
  and updates :: (nat  $\times$  real) list
  assumes state: bmssp-dijkstra-state G src vertices old-settled ds P

```

```

and wf: nat-graph-well-formed G
and pulled-def:
  pulled =
    filter (λx. x ∈ S ∧ x ∉ set old-settled)
      vertices
and settled-def: settled = pulled @ old-settled
and relaxed: bmssp-relax-vertices G settled pulled ds = (updates, ds')
and P2-def: P2 = bmssp-apply-updates updates P1
and distinct-vertices: distinct vertices
and ord: bp-ordered-invariant P
and upper: partition-upper-bound (bp-view P) bmssp-bound
and pull: bp-pull bmssp-block-size bmssp-bound P = (S, beta, P1)
shows partition-upper-bound (bp-view P2) bmssp-bound
proof –
  have witnesses-old: bmssp-label-witnesses G src old-settled ds
    using state unfolding bmssp-dijkstra-state-def by simp
  have updates-def: updates = fst (bmssp-relax-vertices G settled pulled ds)
    using relaxed by simp
  have distinct-updates: distinct (map fst updates)
    by (rule bmssp-loop-partition-step-bridge(5)
      [OF pulled-def updates-def P2-def wf distinct-vertices ord upper pull])
  have settled-subset: set old-settled ⊆ set settled
    using settled-def by auto
  have witnesses:
    bmssp-label-witnesses G src settled ds
    by (rule bmssp-label-witnesses-mono-settled
      [OF witnesses-old settled-subset])
  have pulled-subset: set pulled ⊆ set settled
    using settled-def by simp
  have update-lt:
    ∧x b. (x, b) ∈ set updates ⇒ b < bmssp-bound
  proof –
    fix x b
    assume (x, b) ∈ set updates
    then have update-rel:
      (x, b) ∈ set
        (fst (bmssp-relax-vertices G settled pulled ds))
      using relaxed by simp
    have distinct-rel:
      distinct
        (map fst (fst (bmssp-relax-vertices G settled pulled ds)))
      using distinct-updates relaxed by simp
    show b < bmssp-bound
      by (rule bmssp-relax-vertices-update-lt-bound
        [OF wf witnesses pulled-subset distinct-rel update-rel])
  qed
show ?thesis
  by (rule bmssp-loop-partition-step-bridge(9)
    [OF pulled-def updates-def P2-def wf distinct-vertices ord upper

```

pull update-lt])

qed

definition *bmssp-singleton-bucket-shape* ::
nat bucketed-partition $\Rightarrow$ *bool* **where**
bmssp-singleton-bucket-shape *P* $\longleftrightarrow$
bp-block-size *P* = *bmssp-block-size* $\wedge$
 $(\forall b \in \text{set } (bp\text{-buckets } P)).$
 $\exists p. bp\text{-bucket-entries } b = [p] \wedge bp\text{-marker } b = \text{snd } p)$

lemma *bmssp-bucketize-sorted-entries-aux-singleton-shape*:
 $\forall b \in \text{set } (bp\text{-bucketize-sorted-entries-aux fuel } 1 \text{ } xs).$
 $\exists p. bp\text{-bucket-entries } b = [p] \wedge bp\text{-marker } b = \text{snd } p$

proof (*induction fuel arbitrary: xs*)

case 0

then show ?case

by simp

next

case (Suc fuel)

show ?case

proof (*cases xs*)

case Nil

then show ?thesis

by simp

next

case (Cons p ps)

have tail:

$\forall b \in \text{set } (bp\text{-bucketize-sorted-entries-aux fuel } 1 \text{ } (\text{drop } 1 \text{ } xs)).$

$\exists p. bp\text{-bucket-entries } b = [p] \wedge bp\text{-marker } b = \text{snd } p$

by (rule Suc.IH)

show ?thesis

proof

fix b

assume b:

$b \in \text{set } (bp\text{-bucketize-sorted-entries-aux } (Suc \text{ fuel}) \text{ } 1 \text{ } xs)$

have b-cases:

$b = bp\text{-make-bucket } [p] \vee$

$b \in \text{set } (bp\text{-bucketize-sorted-entries-aux fuel } 1 \text{ } ps)$

using b Cons by auto

then show $\exists p. bp\text{-bucket-entries } b = [p] \wedge bp\text{-marker } b = \text{snd } p$

proof

assume b-def: $b = bp\text{-make-bucket } [p]$

show ?thesis

unfolding b-def bp-make-bucket-def

by (rule exI[of - p]) simp

next

assume $b \in \text{set } (bp\text{-bucketize-sorted-entries-aux fuel } 1 \text{ } ps)$

then show ?thesis

using tail Cons by simp

qed
 qed
 qed
 qed

lemma *bmssp-bucketize-entries-singleton-shape*:
 $\forall b \in \text{set } (bp\text{-bucketize-entries } bmssp\text{-block-size } xs).$
 $\exists p. bp\text{-bucket-entries } b = [p] \wedge bp\text{-marker } b = \text{snd } p$
unfolding *bp-bucketize-entries-def bp-bucketize-sorted-entries-def*
bmssp-block-size-def
by (*rule bmssp-bucketize-sorted-entries-aux-singleton-shape*)

lemma *bmssp-rebucket-singleton-shape*:
assumes *block*: *bp-block-size* *P* = *bmssp-block-size*
shows *bmssp-singleton-bucket-shape* (*bp-rebucket* *P*)
unfolding *bmssp-singleton-bucket-shape-def bp-rebucket-def*
using *block* *bmssp-bucketize-entries-singleton-shape* [*of bp-entries P*]
by *simp*

lemma *bmssp-singleton-bucket-shape-drop-empty-prefix-id*:
assumes *shape*:
 $\forall b \in \text{set } bs. \exists p. bp\text{-bucket-entries } b = [p] \wedge bp\text{-marker } b = \text{snd } p$
shows *bp-drop-empty-prefix* *bs* = *bs*
using *shape*
by (*induction bs*) *auto*

lemma *bmssp-singleton-bucket-shape-flat-length*:
assumes *shape*:
 $\forall b \in \text{set } bs. \exists p. bp\text{-bucket-entries } b = [p] \wedge bp\text{-marker } b = \text{snd } p$
shows *length* (*bp-bucket-entries-flat* *bs*) = *length* *bs*
using *shape*
proof (*induction bs*)
case *Nil*
then show *?case*
unfolding *bp-bucket-entries-flat-def* **by** *simp*
next
case (*Cons b bs*)
obtain *p* **where** *b-entries*: *bp-bucket-entries* *b* = [*p*]
and *b-marker*: *bp-marker* *b* = *snd p*
proof –
have $\exists p. bp\text{-bucket-entries } b = [p] \wedge bp\text{-marker } b = \text{snd } p$
using *Cons.prem*s **by** *simp*
then show *?thesis*
by (*elim exE conjE*) (*rule that*)
qed
have *tail*:
 $\forall b \in \text{set } bs. \exists p. bp\text{-bucket-entries } b = [p] \wedge$
 $bp\text{-marker } b = \text{snd } p$
using *Cons.prem*s **by** *simp*

```

have length (bp-bucket-entries-flat bs) = length bs
  by (rule Cons.IH[OF tail])
then show ?case
  using b-entries unfolding bp-bucket-entries-flat-def by simp
qed

lemma bmssp-singleton-bucket-shape-entries-empty-buckets:
  assumes shape: bmssp-singleton-bucket-shape P
  and empty: bp-entries P = []
  shows bp-buckets P = []
proof (cases bp-buckets P)
  case Nil
  then show ?thesis .
next
  case (Cons b bs)
  obtain p where b-entries: bp-bucket-entries b = [p]
  using shape Cons unfolding bmssp-singleton-bucket-shape-def by auto
  then have bp-entries P ≠ []
  using Cons unfolding bp-entries-def bp-bucket-entries-flat-def by simp
  then show ?thesis
  using empty by simp
qed

lemma bmssp-singleton-bucket-shape-trim:
  assumes shape: bmssp-singleton-bucket-shape P
  shows bmssp-trim-empty-prefix P = P
proof -
  have buckets:
    ∀ b ∈ set (bp-buckets P).
      ∃ p. bp-bucket-entries b = [p] ∧ bp-marker b = snd p
  using shape unfolding bmssp-singleton-bucket-shape-def by simp
  have bp-drop-empty-prefix (bp-buckets P) = bp-buckets P
  by (rule bmssp-singleton-bucket-shape-drop-empty-prefix-id[OF buckets])
  then show ?thesis
  unfolding bmssp-trim-empty-prefix-def by simp
qed

lemma bmssp-singleton-bucket-shape-delete-first-bucket:
  assumes buckets: bp-buckets P = b # bs
  and shape: bmssp-singleton-bucket-shape P
  and distinct: distinct (map fst (bp-entries P))
  shows bmssp-singleton-bucket-shape
    (bmssp-trim-empty-prefix (bp-delete-keys (bp-bucket-keys b) P))
proof -
  obtain p where b-entries: bp-bucket-entries b = [p]
  and b-marker: bp-marker b = snd p
  using buckets shape unfolding bmssp-singleton-bucket-shape-def by auto
  have tail-shape:
    ∀ c ∈ set bs. ∃ q. bp-bucket-entries c = [q] ∧ bp-marker c = snd q

```

```

    using buckets shape unfolding bmssp-singleton-bucket-shape-def by auto
  have tail-unchanged:
    map (bp-delete-keys-from-bucket (bp-bucket-keys b)) bs = bs
  proof (rule map-idI)
    fix c
    assume c: c ∈ set bs
    obtain q where c-entries: bp-bucket-entries c = [q]
    and c-marker: bp-marker c = snd q
  proof -
    have ∃ q. bp-bucket-entries c = [q] ∧ bp-marker c = snd q
    using tail-shape c by simp
    then show ?thesis
    by (elim exE conjE) (rule that)
  qed
  have fst-ne: fst q ≠ fst p
  proof
    assume eq: fst q = fst p
    have entries:
      bp-entries P = p # bp-bucket-entries-flat bs
    using buckets b-entries unfolding bp-entries-def
      bp-bucket-entries-flat-def by simp
    have q-flat: q ∈ set (bp-bucket-entries-flat bs)
    using c c-entries
      unfolding bp-bucket-entries-flat-def
      by (induction bs) auto
    then have fst q ∈ set (map fst (bp-bucket-entries-flat bs))
    by simp
    then show False
    using distinct eq entries by simp
  qed
  have fst q ∉ bp-bucket-keys b
    using fst-ne b-entries unfolding bp-bucket-keys-def bp-entry-keys-def
    by simp
  then have keep-c:
    filter (λr. fst r ∉ bp-bucket-keys b)
      (bp-bucket-entries c) = bp-bucket-entries c
    using c-entries by simp
  then show bp-delete-keys-from-bucket (bp-bucket-keys b) c = c
    unfolding bp-delete-keys-from-bucket-def by simp
  qed
  have deleted-head:
    bp-delete-keys-from-bucket (bp-bucket-keys b) b =
      b ⧸ bp-bucket-entries := []
  unfolding bp-delete-keys-from-bucket-def b-entries
    bp-bucket-keys-def bp-entry-keys-def by simp
  have buckets-deleted:
    bp-buckets (bp-delete-keys (bp-bucket-keys b) P) =
      b ⧸ bp-bucket-entries := [] # bs
  using buckets tail-unchanged deleted-head

```

```

unfolding bp-delete-keys-def by simp
have trimmed-buckets:
  bp-buckets
  (bmssp-trim-empty-prefix (bp-delete-keys (bp-bucket-keys b) P)) =
  bs
proof –
  have tail-drop: bp-drop-empty-prefix bs = bs
  by (rule bmssp-singleton-bucket-shape-drop-empty-prefix-id[OF tail-shape])
  show ?thesis
  using buckets-deleted tail-drop
  unfolding bmssp-trim-empty-prefix-def by simp
qed
have block:
  bp-block-size
  (bmssp-trim-empty-prefix (bp-delete-keys (bp-bucket-keys b) P)) =
  bmssp-block-size
  using shape unfolding bmssp-singleton-bucket-shape-def
  bmssp-trim-empty-prefix-def bp-delete-keys-def by simp
show ?thesis
  unfolding bmssp-singleton-bucket-shape-def
  using block trimmed-buckets tail-shape by simp
qed

lemma bmssp-singleton-bucket-shape-delete-all-small:
  assumes shape: bmssp-singleton-bucket-shape P
  and small: length (bp-entries P) ≤ bmssp-block-size
  shows bmssp-singleton-bucket-shape
  (bmssp-trim-empty-prefix
  (bp-delete-keys (bp-entry-keys (bp-entries P)) P))
proof –
  have block: bp-block-size P = bmssp-block-size
  using shape unfolding bmssp-singleton-bucket-shape-def by simp
  have singleton:
    ∀ b ∈ set (bp-buckets P).
    ∃ p. bp-bucket-entries b = [p] ∧ bp-marker b = snd p
  using shape unfolding bmssp-singleton-bucket-shape-def by simp
  have entries-len:
    length (bp-entries P) = length (bp-buckets P)
  proof –
  have length (bp-bucket-entries-flat (bp-buckets P)) =
    length (bp-buckets P)
  by (rule bmssp-singleton-bucket-shape-flat-length[OF singleton])
  then show ?thesis
  unfolding bp-entries-def .
qed
have buckets-len: length (bp-buckets P) ≤ 1
  using small entries-len unfolding bmssp-block-size-def by simp
have trimmed-empty:
  bp-drop-empty-prefix

```

```

    (bp-buckets (bp-delete-keys (bp-entry-keys (bp-entries P)) P)) = []
proof (cases bp-buckets P)
  case Nil
  then show ?thesis
    unfolding bp-delete-keys-def by simp
next
  case (Cons b bs)
  then have bs-empty: bs = []
    using buckets-len by (cases bs) simp-all
  obtain p where b-entries: bp-bucket-entries b = [p]
  proof -
    have  $\exists p. \text{bp-bucket-entries } b = [p] \wedge \text{bp-marker } b = \text{snd } p$ 
      using singleton Cons by simp
    then show ?thesis
      by (elim exE conjE) (rule that)
  qed
  have key-in:
    fst p  $\in$  bp-entry-keys (bp-entries P)
    using Cons b-entries
    unfolding bp-entries-def bp-bucket-entries-flat-def bp-entry-keys-def
    by simp
  have deleted:
    bp-delete-keys-from-bucket (bp-entry-keys (bp-entries P)) b =
      b(bp-bucket-entries := [])
    unfolding bp-delete-keys-from-bucket-def b-entries
    using key-in by simp
  show ?thesis
    using Cons bs-empty deleted unfolding bp-delete-keys-def by simp
  qed
  have block-deleted:
    bp-block-size (bp-delete-keys (bp-entry-keys (bp-entries P)) P) =
      bmssp-block-size
    using block unfolding bp-delete-keys-def by simp
  show ?thesis
    unfolding bmssp-singleton-bucket-shape-def bmssp-trim-empty-prefix-def
    using block-deleted trimmed-empty by simp
  qed

lemma bmssp-singleton-bucket-shape-pull-trim:
  assumes shape: bmssp-singleton-bucket-shape P
  and ord: bp-ordered-invariant P
  and pull: bp-pull bmssp-block-size B P = (S, beta, P1)
  shows bmssp-singleton-bucket-shape (bmssp-trim-empty-prefix P1)
proof (cases bp-can-first-bucket-pull bmssp-block-size P)
  case True
  then obtain b c bs where buckets: bp-buckets P = b # c # bs
    by (rule bp-can-first-bucket-pullE) blast
  have first:
    bp-first-bucket-pull bmssp-block-size B P = (S, beta, P1)

```



```

    using True pull unfolding bp-pull-def by simp
  have P1-def:  $P1 = bp\_delete\_keys (bp\_bucket\_keys\ b)\ P$ 
    using first buckets unfolding bp-first-bucket-pull-def
    by (auto simp: Let-def)
  have distinct:  $distinct\ (map\ fst\ (bp\_entries\ P))$ 
    using ord unfolding bp-ordered-invariant-def bp-invariant-def
    bp-distinct-keys-def by blast
  show ?thesis
    unfolding P1-def
    by (rule bmssp-singleton-bucket-shape-delete-first-bucket
        [OF - shape distinct])
        (use buckets in simp)
next
case False
have conservative:
  bp-conservative-pull bmssp-block-size  $B\ P = (S, beta, P1)$ 
  using False pull unfolding bp-pull-def by simp
have S-def:  $S = bp\_pull\_set\ bmssp\_block\_size\ P$ 
  and P1-def:  $P1 = bp\_delete\_keys\ S\ P$ 
  using conservative unfolding bp-conservative-pull-def
  by (auto simp: Let-def)
show ?thesis
proof (cases  $length\ (bp\_entries\ P) \leq bmssp\_block\_size$ )
case True
  have S-all:  $S = bp\_entry\_keys\ (bp\_entries\ P)$ 
    using S-def True unfolding bp-pull-set-def by simp
  show ?thesis
    unfolding P1-def S-all
    by (rule bmssp-singleton-bucket-shape-delete-all-small
        [OF shape True])
next
case False
  have S-empty:  $S = \{\}$ 
    using S-def False unfolding bp-pull-set-def by simp
  have P1-eq:  $P1 = P$ 
    unfolding P1-def S-empty bp-delete-keys-def
    bp-delete-keys-from-bucket-def by simp
  show ?thesis
    using shape
    unfolding P1-eq
    by (simp add: bmssp-singleton-bucket-shape-trim[OF shape])
qed
qed

lemma bmssp-singleton-bucket-shape-regularized-insert:
  assumes shape: bmssp-singleton-bucket-shape  $P$ 
  shows bmssp-singleton-bucket-shape
    ( $bp\_result\_of\ (c\_bp\_regularized\_local\_insert\ x\ b\ P)$ )
proof -

```

```

have block: bp-block-size P = bmssp-block-size
  using shape unfolding bmssp-singleton-bucket-shape-def by simp
have local-block:
  bp-block-size (bp-local-insert-state x b P) = bmssp-block-size
  using block
  by (simp add: bp-local-insert-state-def bp-delete-key-def Let-def)
show ?thesis
  unfolding c-bp-regularized-local-insert-result
    bp-regularized-local-insert-def
  by (rule bmssp-rebucket-singleton-shape[OF local-block])
qed

lemma bmssp-insert-updates-singleton-shape:
  assumes shape: bmssp-singleton-bucket-shape P
  shows bmssp-singleton-bucket-shape (bmssp-insert-updates xs P)
  using shape
proof (induction xs arbitrary: P)
  case Nil
  then show ?case
    by simp
next
  case (Cons xb xs)
  obtain x b where xb: xb = (x, b)
  by force
  have step-shape:
    bmssp-singleton-bucket-shape
      (bp-result-of (c-bp-regularized-local-insert x b P))
  by (rule bmssp-singleton-bucket-shape-regularized-insert[OF Cons.prem])
  show ?case
    unfolding xb bmssp-insert-updates.simps
  by (rule Cons.IH[OF step-shape])
qed

lemma bmssp-bucketed-batch-prepend-empty-singleton-shape:
  assumes shape: bmssp-singleton-bucket-shape P
  and empty: bp-entries P = []
  shows bmssp-singleton-bucket-shape
    (bp-result-of (c-bp-paper-batch-prepend xs P))
proof (cases xs)
  case Nil
  then show ?thesis
    using shape empty
    unfolding c-bp-paper-batch-prepend-result
      bp-bucketed-batch-prepend-state-def
    by simp
next
  case (Cons p ps)
  have buckets-empty: bp-buckets P = []
  by (rule bmssp-singleton-bucket-shape-entries-empty-buckets

```

```

    [OF shape empty])
  have block: bp-block-size P = bmssp-block-size
    using shape unfolding bmssp-singleton-bucket-shape-def by simp
  have bucket-shape:
     $\forall b \in \text{set } (bp\text{-bucketize-entries } (bp\text{-block-size } P) \text{ } xs).$ 
     $\exists p. bp\text{-bucket-entries } b = [p] \wedge bp\text{-marker } b = \text{snd } p$ 
    using block bmssp-bucketize-entries-singleton-shape[of xs] by simp
  show ?thesis
    using block buckets-empty bucket-shape Cons
    unfolding c-bp-paper-batch-prepend-result
      bp-bucketed-batch-prepend-state-def
      bmssp-singleton-bucket-shape-def
    by (simp add: Let-def)
qed

lemma bmssp-apply-updates-singleton-shape:
  assumes shape: bmssp-singleton-bucket-shape (bmssp-trim-empty-prefix P)
  shows bmssp-singleton-bucket-shape (bmssp-apply-updates xs P)
proof -
  let ?P0 = bmssp-trim-empty-prefix P
  show ?thesis
  proof (cases bp-entries ?P0 = [])
    case True
    have result:
       $bmssp\text{-apply-updates } xs \ P =$ 
       $bp\text{-result-of } (c\text{-bp-paper-batch-prepend } xs \ ?P0)$ 
      using True unfolding bmssp-apply-updates-def by simp
    have shape':
       $bmssp\text{-singleton-bucket-shape}$ 
       $(bp\text{-result-of } (c\text{-bp-paper-batch-prepend } xs \ ?P0))$ 
      by (rule bmssp-bucketed-batch-prepend-empty-singleton-shape
        [OF shape True])
    show ?thesis
      using result shape' by simp
  next
    case False
    have result:
       $bmssp\text{-apply-updates } xs \ P = bmssp\text{-insert-updates } xs \ ?P0$ 
      using False unfolding bmssp-apply-updates-def by simp
    have shape':
       $bmssp\text{-singleton-bucket-shape } (bmssp\text{-insert-updates } xs \ ?P0)$ 
      by (rule bmssp-insert-updates-singleton-shape[OF shape])
    show ?thesis
      using result shape' by simp
  qed
qed

```

```

lemma bmssp-partition-key-inject:
  assumes eq: bmssp-partition-key v d = bmssp-partition-key w e

```

```

shows  $v = w \wedge d = e$ 
proof -
  have  $d\text{-eq}$ :  $d = e$ 
    using  $\text{eq bmssp-partition-key-floor[of } v \text{ } d]$ 
       $\text{bmssp-partition-key-floor[of } w \text{ } e]$ 
    by  $\text{simp}$ 
  have  $\text{frac-eq}$ :
     $\text{real } v / \text{real } (\text{Suc } v) = \text{real } w / \text{real } (\text{Suc } w)$ 
    using  $\text{eq } d\text{-eq unfolding bmssp-partition-key-def by simp}$ 
  have  $\text{frac-v}$ :
     $\text{real } v / \text{real } (\text{Suc } v) = 1 - \text{inverse } (\text{real } (\text{Suc } v))$ 
    by  $(\text{simp add: field-simps})$ 
  have  $\text{frac-w}$ :
     $\text{real } w / \text{real } (\text{Suc } w) = 1 - \text{inverse } (\text{real } (\text{Suc } w))$ 
    by  $(\text{simp add: field-simps})$ 
  have  $\text{inv-eq}$ :
     $\text{inverse } (\text{real } (\text{Suc } v)) = \text{inverse } (\text{real } (\text{Suc } w))$ 
    using  $\text{frac-eq frac-v frac-w by linarith}$ 
  then have  $\text{real } (\text{Suc } v) = \text{real } (\text{Suc } w)$ 
    by  $\text{simp}$ 
  then have  $v = w$ 
    by  $\text{simp}$ 
  then show  $?thesis$ 
    using  $d\text{-eq by simp}$ 
qed

```

```

lemma  $\text{distinct-map-fst-mem-eq}$ :
  assumes  $\text{distinct: distinct (map fst xs)}$ 
    and  $p: p \in \text{set } xs$ 
    and  $q: q \in \text{set } xs$ 
    and  $\text{fst-eq: fst } p = \text{fst } q$ 
  shows  $p = q$ 
  using  $\text{assms}$ 
proof  $(\text{induction xs})$ 
  case  $\text{Nil}$ 
    then show  $?case$ 
      by  $\text{simp}$ 
  next
    case  $(\text{Cons } r \text{ } xs)$ 
    show  $?case$ 
    proof  $(\text{cases } p = r)$ 
    case  $\text{True}$ 
      have  $q = r$ 
    proof  $(\text{rule ccontr})$ 
      assume  $q \neq r$ 
      then have  $q \in \text{set } xs$ 
        using  $\text{Cons.premis(3) by simp}$ 
      then have  $\text{fst } q \in \text{set (map fst xs)}$ 
        by  $\text{force}$ 
    qed
  qed

```

```

    moreover have  $\text{fst } r \notin \text{set } (\text{map } \text{fst } xs)$ 
      using  $\text{Cons.prem}(1)$  by simp
    ultimately show  $\text{False}$ 
      using  $\text{Cons.prem}(4)$  True by simp
  qed
  then show ?thesis
    using True by simp
next
  case False
  then have  $p\text{-tail}: p \in \text{set } xs$ 
    using  $\text{Cons.prem}(2)$  by simp
  show ?thesis
  proof (cases  $q = r$ )
    case True
    have  $\text{fst } p \in \text{set } (\text{map } \text{fst } xs)$ 
      using  $p\text{-tail}$  by force
    moreover have  $\text{fst } r \notin \text{set } (\text{map } \text{fst } xs)$ 
      using  $\text{Cons.prem}(1)$  by simp
    ultimately show ?thesis
      using  $\text{Cons.prem}(4)$  True by simp
  next
    case False
    have  $q\text{-tail}: q \in \text{set } xs$ 
      using False  $\text{Cons.prem}(3)$  by simp
    have  $\text{distinct-tail}: \text{distinct } (\text{map } \text{fst } xs)$ 
      using  $\text{Cons.prem}(1)$  by simp
    show ?thesis
      by (rule  $\text{Cons.IH}[OF \text{distinct-tail } p\text{-tail } q\text{-tail } \text{Cons.prem}(4)]$ )
  qed
qed
qed
qed

lemma bmssp-partition-entries-value-inj:
  assumes match: bmssp-partition-state-match settled ds P
    and ord: bp-ordered-invariant P
  shows inj-on snd (set (bp-entries P))
proof (rule inj-onI)
  fix  $p \ q$ 
  assume  $p: p \in \text{set } (\text{bp-entries } P)$ 
    and  $q: q \in \text{set } (\text{bp-entries } P)$ 
    and snd-eq:  $\text{snd } p = \text{snd } q$ 
  have distinct-ds:  $\text{distinct } (\text{map } \text{fst } ds)$ 
    and keys: bmssp-partition-keys-match settled ds P
    and vals: bmssp-partition-values-match settled ds (bp-view P)
    using match unfolding bmssp-partition-state-match-def by auto
  have values-consistent:
     $\bigwedge r. r \in \text{set } (\text{bp-entries } P) \implies$ 
       $\text{value-of } (\text{bp-view } P) (\text{fst } r) = \text{snd } r$ 
  using ord

```

```

unfolding bp-ordered-invariant-def bp-invariant-def
  bp-values-consistent-def bp-view-def
by simp
have p-key: fst p  $\in$  keys-of (bp-view P)
  using p unfolding bp-view-def bp-entry-keys-def by auto
have q-key: fst q  $\in$  keys-of (bp-view P)
  using q unfolding bp-view-def bp-entry-keys-def by auto
obtain dp where p-lookup: bmssp-lookup-dist ds (fst p) = Some dp
  and p-unsettled: fst p  $\notin$  set settled
proof –
  have key: fst p  $\in$  set (map fst ds) – set settled
    using keys p-key unfolding bmssp-partition-keys-match-def by simp
  then obtain dp where mem: (fst p, dp)  $\in$  set ds
    by force
  have lookup: bmssp-lookup-dist ds (fst p) = Some dp
    by (rule bmssp-lookup-dist-Some-if-distinct-mem
      [OF distinct-ds mem])
  show ?thesis
    using that lookup key by blast
qed
obtain dq where q-lookup: bmssp-lookup-dist ds (fst q) = Some dq
  and q-unsettled: fst q  $\notin$  set settled
proof –
  have key: fst q  $\in$  set (map fst ds) – set settled
    using keys q-key unfolding bmssp-partition-keys-match-def by simp
  then obtain dq where mem: (fst q, dq)  $\in$  set ds
    by force
  have lookup: bmssp-lookup-dist ds (fst q) = Some dq
    by (rule bmssp-lookup-dist-Some-if-distinct-mem
      [OF distinct-ds mem])
  show ?thesis
    using that lookup key by blast
qed
have p-value:
  snd p = bmssp-partition-key (fst p) dp
  using vals p-lookup p-unsettled values-consistent[OF p]
  unfolding bmssp-partition-values-match-def by simp
have q-value:
  snd q = bmssp-partition-key (fst q) dq
  using vals q-lookup q-unsettled values-consistent[OF q]
  unfolding bmssp-partition-values-match-def by simp
have fst-eq: fst p = fst q
  using bmssp-partition-key-inject[of fst p dp fst q dq]
  snd-eq p-value q-value by auto
have distinct-entries: distinct (map fst (bp-entries P))
  using ord unfolding bp-ordered-invariant-def bp-invariant-def
  bp-distinct-keys-def by blast
show p = q
  by (rule distinct-map-fst-mem-eq

```

[OF distinct-entries p q fst-eq])

qed

lemma *bmssp-can-first-bucket-pull-if-singleton-shape*:
assumes *shape*: *bmssp-singleton-bucket-shape* P
and *ord*: *bp-ordered-invariant* P
and *values-inj*: *inj-on* *snd* (*set* (*bp-entries* P))
and *many*: *length* (*bp-entries* P) > *bmssp-block-size*
shows *bp-can-first-bucket-pull* *bmssp-block-size* P

proof –
have *singleton*:
 $\forall b \in \text{set } (bp\text{-buckets } P).$
 $\exists p. bp\text{-bucket-entries } b = [p] \wedge bp\text{-marker } b = \text{snd } p$
using *shape* **unfolding** *bmssp-singleton-bucket-shape-def* **by** *simp*
have *distinct-entries*: *distinct* (*map* *fst* (*bp-entries* P))
using *ord* **unfolding** *bp-ordered-invariant-def* *bp-invariant-def*
bp-distinct-keys-def **by** *blast*
obtain b *rest* **where** *buckets-outer*: *bp-buckets* $P = b \# \text{rest}$
proof (*cases* *bp-buckets* P)
case *Nil*
then have *bp-entries* $P = []$
unfolding *bp-entries-def* *bp-bucket-entries-flat-def* **by** *simp*
then show *?thesis*
using *many* **unfolding** *bmssp-block-size-def* **by** *simp*

next
case (*Cons* b *rest*)
then show *?thesis*
by (*rule that*)

qed

obtain p **where** *b-entries*: *bp-bucket-entries* $b = [p]$
and *b-marker*: *bp-marker* $b = \text{snd } p$
using *singleton* *buckets-outer* **by** *auto*

obtain c *bs* **where** *rest*: *rest* = $c \# bs$

proof (*cases* *rest*)
case *Nil*
then have *bp-entries* $P = [p]$
using *buckets-outer* *b-entries*
unfolding *bp-entries-def* *bp-bucket-entries-flat-def* **by** *simp*
then show *?thesis*
using *many* **unfolding** *bmssp-block-size-def* **by** *simp*

next
case (*Cons* c *bs*)
then show *?thesis*
by (*rule that*)

qed

have *buckets*: *bp-buckets* $P = b \# c \# bs$
using *buckets-outer* *rest* **by** *simp*

obtain q **where** *c-entries*: *bp-bucket-entries* $c = [q]$
and *c-marker*: *bp-marker* $c = \text{snd } q$

```

    using singleton buckets by auto
  have p-entry:  $p \in \text{set } (bp\text{-entries } P)$ 
    using buckets b-entries
    unfolding bp-entries-def bp-bucket-entries-flat-def by simp
  have q-entry:  $q \in \text{set } (bp\text{-entries } P)$ 
    using buckets c-entries
    unfolding bp-entries-def bp-bucket-entries-flat-def by simp
  have p-neq-q:  $p \neq q$ 
  proof
    assume  $p = q$ 
    then have fst  $p = \text{fst } q$ 
      by simp
    moreover have map fst  $(bp\text{-entries } P) =$ 
       $\text{fst } p \# \text{fst } q \# \text{map fst } (bp\text{-bucket-entries-flat } bs)$ 
      using buckets b-entries c-entries
      unfolding bp-entries-def bp-bucket-entries-flat-def by simp
    ultimately show False
      using distinct-entries by simp
  qed
  have snd-ne:  $\text{snd } p \neq \text{snd } q$ 
    using values-inj p-entry q-entry p-neq-q unfolding inj-on-def by blast
  have boundary:
     $\forall r \in \text{set } (bp\text{-bucket-entries } b). \text{snd } r \leq bp\text{-marker } c$ 
    using ord buckets
    unfolding bp-ordered-invariant-def
      bp-bucket-boundaries-state-ok-def
    by simp
  have below: bp-bucket-below-bound  $b$   $(bp\text{-marker } c)$ 
  proof -
    have  $\text{snd } p < \text{snd } q$ 
      using boundary snd-ne unfolding b-entries c-marker by auto
    then show ?thesis
      unfolding bp-bucket-below-bound-def b-entries c-marker by simp
  qed
  have len-b:  $\text{length } (bp\text{-bucket-entries } b) \leq bmssp\text{-block-size}$ 
    unfolding b-entries bmssp-block-size-def by simp
  have tail-nonempty:  $bp\text{-bucket-entries-flat } (c \# bs) \neq []$ 
    using c-entries unfolding bp-bucket-entries-flat-def by simp
  show ?thesis
    unfolding bp-can-first-bucket-pull-def buckets
    using many len-b below tail-nonempty by simp
  qed

lemma bmssp-pull-nonempty-if-keys-nonempty:
  assumes shape:  $bmssp\text{-singleton-bucket-shape } P$ 
  and ord: bp-ordered-invariant  $P$ 
  and values-inj:  $\text{inj-on } \text{snd } (\text{set } (bp\text{-entries } P))$ 
  and pull:  $bp\text{-pull } bmssp\text{-block-size } B \text{ } P = (S, \text{beta}, P1)$ 
  and keys-nonempty:  $\text{keys-of } (bp\text{-view } P) \neq \{\}$ 

```



```

shows  $S \neq \{\}$ 
proof (cases bp-can-first-bucket-pull bmssp-block-size P)
  case True
    obtain b c bs where buckets: bp-buckets  $P = b \# c \# bs$ 
    by (rule bp-can-first-bucket-pullE[OF True])
    obtain p where b-entries: bp-bucket-entries  $b = [p]$ 
    using shape buckets unfolding bmssp-singleton-bucket-shape-def by auto
    have first:
      bp-first-bucket-pull bmssp-block-size B  $P = (S, \text{beta}, P1)$ 
      using True pull unfolding bp-pull-def by simp
    have  $S = \text{bp-bucket-keys } b$ 
      using first buckets unfolding bp-first-bucket-pull-def
      by (simp add: Let-def)
    then show ?thesis
      using b-entries unfolding bp-bucket-keys-def bp-entry-keys-def by simp
  next
    case False
      note not-can = False
      have conservative:
        bp-conservative-pull bmssp-block-size B  $P = (S, \text{beta}, P1)$ 
        using False pull unfolding bp-pull-def by simp
      have S-def:  $S = \text{bp-pull-set } bmssp\text{-block-size } P$ 
        using conservative unfolding bp-conservative-pull-def
        by (simp add: Let-def)
      show ?thesis
      proof (cases length (bp-entries P)  $\leq$  bmssp-block-size)
        case True
          have bp-entry-keys (bp-entries P)  $\neq \{\}$ 
            using keys-nonempty unfolding bp-view-def by simp
          then show ?thesis
            using S-def True unfolding bp-pull-set-def by simp
        next
          case False
            have can: bp-can-first-bucket-pull bmssp-block-size P
              by (rule bmssp-can-first-bucket-pull-if-singleton-shape
                [OF shape ord values-inj])
              (use False in simp)
            then show ?thesis
              using not-can by simp
      qed
    qed

```

definition *bmssp-dijkstra-loop-state* ::
 $\text{nat list} \Rightarrow \text{nat list} \Rightarrow$
 $\text{nat-dist} \Rightarrow \text{nat bucketed-partition} \Rightarrow \text{bool}$ **where**
bmssp-dijkstra-loop-state vertices settled ds $P \longleftrightarrow$
 $\text{bmssp-dijkstra-state } G \text{ src vertices settled ds } P \wedge$
 $\text{bp-ordered-invariant } P \wedge$
 $\text{partition-upper-bound (bp-view } P) \text{ bmssp-bound}$

lemma *bmssp-dijkstra-loop-state-initial:*
assumes *src-vertices:* $src \in \text{set vertices}$
shows *bmssp-dijkstra-loop-state vertices* $\sqsubseteq [(src, 0)]$
 (bp-result-of
 (c-bp-regularized-local-insert src (bmssp-partition-key src 0)
 (bp-empty bmssp-block-size bmssp-bound)))
proof –
have *dijkstra:*
bmssp-dijkstra-state G src vertices $\sqsubseteq [(src, 0)]$
 (bp-result-of
 (c-bp-regularized-local-insert src (bmssp-partition-key src 0)
 (bp-empty bmssp-block-size bmssp-bound)))
by (rule *bmssp-dijkstra-state-initial*[*OF src-vertices*])
have *ord:*
bp-ordered-invariant
 (bp-result-of
 (c-bp-regularized-local-insert src (bmssp-partition-key src 0)
 (bp-empty bmssp-block-size bmssp-bound)))
by (rule *bmssp-initial-partition-bridge*(1))
have *upper:*
partition-upper-bound
 (bp-view
 (bp-result-of
 (c-bp-regularized-local-insert src (bmssp-partition-key src 0)
 (bp-empty bmssp-block-size bmssp-bound))))
bmssp-bound
by (rule *bmssp-initial-partition-bridge*(2))
show *?thesis*
unfolding *bmssp-dijkstra-loop-state-def*
using *dijkstra ord upper* **by** *simp*
qed

lemma *bmssp-dijkstra-loop-state-step-bridge:*
fixes *old-settled settled vertices* $:: \text{nat list}$
and *pulled* $:: \text{nat list}$
and *updates* $:: (\text{nat} \times \text{real}) \text{ list}$
assumes *loop-state:*
bmssp-dijkstra-loop-state vertices old-settled ds P
and *edge-targets:*
 $\bigwedge a \ b \ w. (a, b, w) \in \text{set } G \implies b \in \text{set vertices}$
and *pulled-def:*
pulled =
 filter ($\lambda x. x \in S \wedge x \notin \text{set old-settled}$)
 vertices
and *settled-def:* *settled* = *pulled* @ *old-settled*
and *relaxed:* *bmssp-relax-vertices G settled pulled ds* = (*updates*, *ds'*)
and *P2-def:* *P2* = *bmssp-apply-updates updates P1*

and *wf*: *nat-graph-well-formed* *G*
and *distinct-vertices*: *distinct vertices*
and *pull*: *bp-pull* *bmssp-block-size* *bmssp-bound* *P* = (*S*, *beta*, *P1*)
shows *bmssp-dijkstra-loop-state vertices settled ds' P2*
proof –
have *state*: *bmssp-dijkstra-state* *G* *src vertices old-settled ds P*
and *ord*: *bp-ordered-invariant* *P*
and *upper*: *partition-upper-bound* (*bp-view* *P*) *bmssp-bound*
using *loop-state unfolding* *bmssp-dijkstra-loop-state-def* **by** *auto*
have *state'*:
bmssp-dijkstra-state *G* *src vertices settled ds' P2*
by (*rule* *bmssp-dijkstra-state-step-bridge*
[*OF state edge-targets pulled-def settled-def relaxed P2-def*
distinct-vertices ord upper pull])
have *updates-def*: *updates* = *fst* (*bmssp-relax-vertices* *G* *settled pulled ds*)
using *relaxed* **by** *simp*
have *ord'*:
bp-ordered-invariant *P2*
by (*rule* *bmssp-loop-partition-step-bridge*(8)
[*OF pulled-def updates-def P2-def wf distinct-vertices ord upper*
pull])
have *upper'*:
partition-upper-bound (*bp-view* *P2*) *bmssp-bound*
by (*rule* *bmssp-dijkstra-state-step-upper-bound*
[*OF state wf pulled-def settled-def relaxed P2-def distinct-vertices*
ord upper pull])
show *?thesis*
unfolding *bmssp-dijkstra-loop-state-def*
using *state' ord' upper'* **by** *simp*
qed

lemma *distinct-subset-length-le*:
assumes *xs*: *distinct xs*
and *ys*: *distinct ys*
and *subset*: *set xs* $\subseteq$ *set ys*
shows *length xs* $\leq$ *length ys*
proof –
have *length xs* = *card* (*set xs*)
using *xs* **by** (*simp add: distinct-card*)
also have $\dots \leq$ *card* (*set ys*)
by (*rule card-mono*) (*use subset in simp-all*)
also have $\dots =$ *length ys*
using *ys* **by** (*simp add: distinct-card*)
finally show *?thesis* .
qed

lemma *bmssp-dijkstra-loop-state-settled-subset-vertices*:
assumes *loop-state*: *bmssp-dijkstra-loop-state vertices settled ds P*
shows *set settled* $\subseteq$ *set vertices*

```

proof
  fix  $v$ 
  assume  $v$ -settled:  $v \in \text{set settled}$ 
  have  $\text{state}$ :  $\text{bmssp-dijkstra-state } G \text{ src vertices settled } ds \ P$ 
    using  $\text{loop-state unfolding bmssp-dijkstra-loop-state-def}$  by  $\text{simp}$ 
  have  $\text{match}$ :  $\text{bmssp-partition-state-match vertices settled } ds \ P$ 
    and  $\text{have-lookups}$ :  $\text{bmssp-settled-have-lookups settled } ds$ 
    using  $\text{state unfolding bmssp-dijkstra-state-def}$  by  $\text{auto}$ 
  obtain  $d$  where  $\text{lookup}$ :  $\text{bmssp-lookup-dist } ds \ v = \text{Some } d$ 
    using  $\text{have-lookups } v\text{-settled}$ 
    unfolding  $\text{bmssp-settled-have-lookups-def}$  by  $\text{blast}$ 
  have  $v \in \text{set } (\text{map fst } ds)$ 
    by  $(\text{rule bmssp-lookup-dist-mem-key}[OF \text{lookup}])$ 
  then show  $v \in \text{set vertices}$ 
    using  $\text{match unfolding bmssp-partition-state-match-def}$  by  $\text{blast}$ 
qed

```

```

lemma  $\text{bmssp-loop-quiescent-state}$ :
  assumes  $\text{loop-state}$ :  $\text{bmssp-dijkstra-loop-state vertices settled } ds \ P$ 
    and  $\text{shape}$ :  $\text{bmssp-singleton-bucket-shape } P$ 
    and  $\text{edge-targets}$ :
       $\bigwedge a \ b \ w. (a, b, w) \in \text{set } G \implies b \in \text{set vertices}$ 
    and  $\text{distinct-vertices}$ :  $\text{distinct vertices}$ 
    and  $\text{distinct-settled}$ :  $\text{distinct settled}$ 
    and  $\text{fuel}$ :  $\text{fuel} > \text{length vertices} - \text{length settled}$ 
  shows  $\exists \text{settled}' \ ds' \ P' \ S \ \text{beta } P1.$ 
     $\text{bmssp-loop fuel } G \text{ vertices settled } ds \ P = \text{bmssp-normalize-dist } ds' \wedge$ 
     $\text{bmssp-dijkstra-loop-state vertices settled}' \ ds' \ P' \wedge$ 
     $\text{bmssp-singleton-bucket-shape } P' \wedge$ 
     $\text{distinct settled}' \wedge$ 
     $\text{set settled} \subseteq \text{set settled}' \wedge$ 
     $\text{bp-pull bmssp-block-size bmssp-bound } P' = (S, \text{beta}, P1) \wedge$ 
     $\text{filter } (\lambda x. x \in S \wedge x \notin \text{set settled}') \text{ vertices} = []$ 
  using  $\text{loop-state shape distinct-settled fuel}$ 
proof  $(\text{induction fuel arbitrary: settled } ds \ P)$ 
  case  $0$ 
    then show  $?case$ 
      by  $\text{simp}$ 
  next
    case  $(\text{Suc fuel})$ 
    obtain  $S \ \text{beta } P1$  where  $\text{pull}$ :
       $\text{bp-pull bmssp-block-size bmssp-bound } P = (S, \text{beta}, P1)$ 
      by  $(\text{cases bp-pull bmssp-block-size bmssp-bound } P) \text{ auto}$ 
    let  $?pulled =$ 
       $\text{filter } (\lambda x. x \in S \wedge x \notin \text{set settled}) \text{ vertices}$ 
    show  $?case$ 
    proof  $(\text{cases } ?pulled = [])$ 
      case  $\text{True}$ 
      have  $\text{loop-eq}$ :

```

```

    bmssp-loop (Suc fuel) G vertices settled ds P =
      bmssp-normalize-dist ds
    using pull True by simp
  show ?thesis
proof (intro exI conjI)
  show bmssp-loop (Suc fuel) G vertices settled ds P =
    bmssp-normalize-dist ds
    by (rule loop-eq)
  show bmssp-dijkstra-loop-state vertices settled ds P
    by (rule Suc.prem1)
  show bmssp-singleton-bucket-shape P
    by (rule Suc.prem2)
  show distinct settled
    by (rule Suc.prem3)
  show set settled  $\subseteq$  set settled
    by simp
  show bp-pull bmssp-block-size bmssp-bound P = (S, beta, P1)
    by (rule pull)
  show filter ( $\lambda x. x \in S \wedge x \notin \text{set settled}$ ) vertices = []
    by (rule True)
qed
next
case False
let ?settled = ?pulled @ settled
obtain updates ds' where relaxed:
  bmssp-relax-vertices G ?settled ?pulled ds = (updates, ds')
  by force
let ?P2 = bmssp-apply-updates updates P1
have pulled-def:
  ?pulled =
    filter ( $\lambda x. x \in S \wedge x \notin \text{set settled}$ ) vertices
  by simp
have settled-def: ?settled = ?pulled @ settled
  by simp
have P2-def: ?P2 = bmssp-apply-updates updates P1
  by simp
have loop-state':
  bmssp-dijkstra-loop-state vertices ?settled ds' ?P2
  by (rule bmssp-dijkstra-loop-state-step-bridge
    [OF Suc.prem1 edge-targets pulled-def settled-def relaxed
      P2-def wf distinct-vertices pull])
have state: bmssp-dijkstra-state G src vertices settled ds P
  and ord: bp-ordered-invariant P
  and upper: partition-upper-bound (bp-view P) bmssp-bound
  using Suc.prem1 unfolding bmssp-dijkstra-loop-state-def by auto
have trim-shape:
  bmssp-singleton-bucket-shape (bmssp-trim-empty-prefix P1)
  by (rule bmssp-singleton-bucket-shape-pull-trim
    [OF Suc.prem2 ord pull])

```

```

have shape':
  bmssp-singleton-bucket-shape ?P2
  by (rule bmssp-apply-updates-singleton-shape[OF trim-shape])
have updates-def:
  updates = fst (bmssp-relax-vertices G ?settled ?pulled ds)
  using relaxed by simp
have pulled-len: length ?pulled ≤ 1
  by (rule bmssp-loop-partition-step-bridge(4)
      [OF pulled-def updates-def P2-def wf distinct-vertices ord upper
        pull])
have pulled-one: length ?pulled = 1
  using False pulled-len by (cases ?pulled) auto
have pulled-distinct: distinct ?pulled
  using pulled-len by (cases ?pulled) auto
have pulled-disjoint: set ?pulled ∩ set settled = {}
  by auto
have distinct-settled': distinct ?settled
  using pulled-distinct pulled-disjoint Suc.prems(3) by simp
have settled-subset-vertices:
  set settled ⊆ set vertices
  by (rule bmssp-dijkstra-loop-state-settled-subset-vertices
      [OF Suc.prems(1)])
have old-lt-vertices: length settled < length vertices
proof -
  obtain x where x-pulled: x ∈ set ?pulled
    using False by (cases ?pulled) auto
  have x-vertices: x ∈ set vertices
    using x-pulled by simp
  have x-not-settled: x ∉ set settled
    using x-pulled by simp
  have proper: set settled < set vertices
    using settled-subset-vertices x-vertices x-not-settled
    unfolding psubset-eq by blast
  have card (set settled) < card (set vertices)
    by (rule psubset-card-mono) (use proper in simp-all)
  then show ?thesis
    using Suc.prems(3) distinct-vertices
    by (simp add: distinct-card)
qed
have diff-step:
  Suc (length vertices - Suc (length settled)) =
    length vertices - length settled
  by (rule Suc-diff-Suc[OF old-lt-vertices])
have fuel':
  fuel > length vertices - length ?settled
proof -
  have fuel > length vertices - Suc (length settled)
    using Suc.prems(4) diff-step by simp
  then show ?thesis

```

```

    using pulled-one by simp
  qed
  obtain settled'' ds'' P'' S' beta' P1' where final:
    bmssp-loop fuel G vertices ?settled ds' ?P2 =
      bmssp-normalize-dist ds''
    bmssp-dijkstra-loop-state vertices settled'' ds'' P''
    bmssp-singleton-bucket-shape P''
    distinct settled''
    set ?settled  $\subseteq$  set settled''
    bp-pull bmssp-block-size bmssp-bound P'' = (S', beta', P1')
    filter ( $\lambda x. x \in S' \wedge x \notin \text{set settled''}$ ) vertices = []
  proof -
    have  $\exists$  settled-out ds-out P-out S-out beta-out P1-out.
      bmssp-loop fuel G vertices ?settled ds' ?P2 =
        bmssp-normalize-dist ds-out  $\wedge$ 
        bmssp-dijkstra-loop-state vertices settled-out ds-out P-out  $\wedge$ 
        bmssp-singleton-bucket-shape P-out  $\wedge$ 
        distinct settled-out  $\wedge$ 
        set ?settled  $\subseteq$  set settled-out  $\wedge$ 
        bp-pull bmssp-block-size bmssp-bound P-out =
          (S-out, beta-out, P1-out)  $\wedge$ 
        filter ( $\lambda x. x \in S\text{-out} \wedge x \notin \text{set settled-out}$ ) vertices = []
      by (rule Suc.IH[OF loop-state' shape' distinct-settled' fuel'])
    then show ?thesis
      by (elim exE conjE) (rule that)
  qed
  have loop-eq:
    bmssp-loop (Suc fuel) G vertices settled ds P =
      bmssp-loop fuel G vertices ?settled ds' ?P2
    using pull False relaxed by (simp add: Let-def)
  show ?thesis
  proof (intro exI conjI)
    show bmssp-loop (Suc fuel) G vertices settled ds P =
      bmssp-normalize-dist ds''
      using loop-eq final(1) by simp
    show bmssp-dijkstra-loop-state vertices settled'' ds'' P''
      by (rule final(2))
    show bmssp-singleton-bucket-shape P''
      by (rule final(3))
    show distinct settled''
      by (rule final(4))
    show set settled  $\subseteq$  set settled''
      using final(5) by auto
    show bp-pull bmssp-block-size bmssp-bound P'' = (S', beta', P1')
      by (rule final(6))
    show filter ( $\lambda x. x \in S' \wedge x \notin \text{set settled''}$ ) vertices = []
      by (rule final(7))
  qed
qed

```

qed

lemma *bmssp-dijkstra-loop-state-lookup-exact-if-queue-empty:*
assumes *loop-state: bmssp-dijkstra-loop-state vertices settled ds P*
and *empty: keys-of (bp-view P) = {}*
and *lookup: bmssp-lookup-dist ds v = Some d*
shows *real d = nat-graph-dist G src v*
proof –
have *state: bmssp-dijkstra-state G src vertices settled ds P*
using *loop-state unfolding bmssp-dijkstra-loop-state-def* **by** *simp*
have *match: bmssp-partition-state-match vertices settled ds P*
and *exact: bmssp-settled-exact G src settled ds*
using *state unfolding bmssp-dijkstra-state-def* **by** *auto*
have *keys: bmssp-partition-keys-match settled ds P*
using *match unfolding bmssp-partition-state-match-def* **by** *blast*
have *v-key: v ∈ set (map fst ds)*
by *(rule bmssp-lookup-dist-mem-key[OF lookup])*
have *v-settled: v ∈ set settled*
using *keys empty v-key unfolding bmssp-partition-keys-match-def* **by** *blast*
show *?thesis*
using *exact v-settled lookup unfolding bmssp-settled-exact-def* **by** *blast*
qed

lemma *bmssp-dijkstra-loop-state-normalize-exact-if-queue-empty:*
assumes *loop-state: bmssp-dijkstra-loop-state vertices settled ds P*
and *empty: keys-of (bp-view P) = {}*
and *mem: (v, d) ∈ set (bmssp-normalize-dist ds)*
shows *real d = nat-graph-dist G src v*
proof –
have *state: bmssp-dijkstra-state G src vertices settled ds P*
using *loop-state unfolding bmssp-dijkstra-loop-state-def* **by** *simp*
have *match: bmssp-partition-state-match vertices settled ds P*
using *state unfolding bmssp-dijkstra-state-def* **by** *auto*
have *distinct-ds: distinct (map fst ds)*
using *match unfolding bmssp-partition-state-match-def* **by** *simp*
have *mem-ds: (v, d) ∈ set ds*
using *mem unfolding bmssp-normalize-dist-def* **by** *simp*
have *lookup: bmssp-lookup-dist ds v = Some d*
by *(rule bmssp-lookup-dist-Some-if-distinct-mem[OF distinct-ds mem-ds])*
show *?thesis*
by *(rule bmssp-dijkstra-loop-state-lookup-exact-if-queue-empty*
[OF loop-state empty lookup])
qed

lemma *bmssp-dijkstra-loop-state-normalize-reachable:*
assumes *loop-state: bmssp-dijkstra-loop-state vertices settled ds P*
and *mem: (v, d) ∈ set (bmssp-normalize-dist ds)*
shows *nat-graph-reachable G src v*
proof –


```

have state: bmssp-dijkstra-state G src vertices settled ds P
  using loop-state unfolding bmssp-dijkstra-loop-state-def by simp
have match: bmssp-partition-state-match vertices settled ds P
  and witnesses: bmssp-label-witnesses G src settled ds
  using state unfolding bmssp-dijkstra-state-def by auto
have distinct-ds: distinct (map fst ds)
  using match unfolding bmssp-partition-state-match-def by simp
have mem-ds: (v, d) ∈ set ds
  using mem unfolding bmssp-normalize-dist-def by simp
have lookup: bmssp-lookup-dist ds v = Some d
  by (rule bmssp-lookup-dist-Some-if-distinct-mem[OF distinct-ds mem-ds])
show ?thesis
  by (rule bmssp-label-witnesses-lookup-reachable[OF witnesses lookup])
qed

```

```

lemma bmssp-dijkstra-loop-state-reachable-lookup-if-queue-empty:
  assumes loop-state: bmssp-dijkstra-loop-state vertices settled ds P
    and empty: keys-of (bp-view P) = {}
    and reach: nat-graph-reachable G src v
  shows ∃ d. bmssp-lookup-dist ds v = Some d
proof (rule ccontr)
  assume no-lookup: ¬ (∃ d. bmssp-lookup-dist ds v = Some d)
  have v-unsettled: v ∉ set settled
  proof
    assume v-settled: v ∈ set settled
    have state: bmssp-dijkstra-state G src vertices settled ds P
      using loop-state unfolding bmssp-dijkstra-loop-state-def by simp
    have settled-lookup: bmssp-settled-have-lookups settled ds
      using state unfolding bmssp-dijkstra-state-def by simp
    then obtain d where bmssp-lookup-dist ds v = Some d
      using v-settled unfolding bmssp-settled-have-lookups-def by blast
    then show False
      using no-lookup by blast
  qed
  have state: bmssp-dijkstra-state G src vertices settled ds P
    using loop-state unfolding bmssp-dijkstra-loop-state-def by simp
  have match: bmssp-partition-state-match vertices settled ds P
    and source-zero: bmssp-source-zero src ds
    and settled-lookup: bmssp-settled-have-lookups settled ds
    and exact: bmssp-settled-exact G src settled ds
    and frontier: bmssp-frontier-relaxed G src settled ds
    using state unfolding bmssp-dijkstra-state-def by auto
  have reach': concrete.reachable src v
    using reach unfolding nat-graph-reachable-def .
  obtain p where sp: concrete.shortest-walk src p v
    using concrete.shortest-walk-exists[OF reach'] by blast
  obtain y dy where y-unsettled: y ∉ set settled
    and lookup-y: bmssp-lookup-dist ds y = Some dy
    by (rule bmssp-shortest-walk-first-unsettled-lookup-le-dist)

```

$[OF\ sp\ v\text{-unsettled}\ source\text{-zero}\ settled\text{-lookup}\ exact\ frontier])$
 blast
 have keys: *bmssp-partition-keys-match settled ds P*
 using match unfolding *bmssp-partition-state-match-def* by blast
 have y-key: $y \in set\ (map\ fst\ ds)$
 by (rule *bmssp-lookup-dist-mem-key[OF lookup-y]*)
 have $y \in keys\text{-of}\ (bp\text{-view}\ P)$
 using keys y-key y-unsettled
 unfolding *bmssp-partition-keys-match-def* by blast
 then show False
 using empty by simp
 qed

lemma *bmssp-dijkstra-loop-state-normalize-complete-if-queue-empty*:
 assumes loop-state: *bmssp-dijkstra-loop-state vertices settled ds P*
 and empty: $keys\text{-of}\ (bp\text{-view}\ P) = \{\}$
 and reach: *nat-graph-reachable G src v*
 shows $\exists d. (v, d) \in set\ (bmssp\text{-normalize-dist}\ ds)$
 proof –
 obtain d where lookup: *bmssp-lookup-dist ds v = Some d*
 using *bmssp-dijkstra-loop-state-reachable-lookup-if-queue-empty*
 [OF loop-state empty reach]
 by blast
 have mem-ds: $(v, d) \in set\ ds$
 by (rule *bmssp-lookup-dist-Some-mem[OF lookup]*)
 have $(v, d) \in set\ (bmssp\text{-normalize-dist}\ ds)$
 using mem-ds unfolding *bmssp-normalize-dist-def* by simp
 then show ?thesis
 by blast
 qed

lemma *bmssp-dijkstra-loop-state-quiescent-pull-empty*:
 assumes loop-state: *bmssp-dijkstra-loop-state vertices settled ds P*
 and pull: *bp-pull bmssp-block-size bmssp-bound P = (S, beta, P1)*
 and stopped:
 $filter\ (\lambda x. x \in S \wedge x \notin set\ settled)\ vertices = []$
 shows $S = \{\}$
 proof
 show $S \subseteq \{\}$
 proof
 fix x
 assume xS: $x \in S$
 have state: *bmssp-dijkstra-state G src vertices settled ds P*
 and ord: *bp-ordered-invariant P*
 and upper: *partition-upper-bound (bp-view P) bmssp-bound*
 using loop-state unfolding *bmssp-dijkstra-loop-state-def* by auto
 have match: *bmssp-partition-state-match vertices settled ds P*
 using state unfolding *bmssp-dijkstra-state-def* by simp
 have sep:

pull-separates (*bp-view P*) *bmssp-block-size* *bmssp-bound S* *beta*
 (*bp-view P1*)
 by (*rule bmssp-pull-refines-pull-separates*[*OF ord upper pull*])
 have *S-keys*: $S \subseteq \text{keys-of } (\text{bp-view } P)$
 by (*rule pull-separates-subset*[*OF sep*])
 have *keys-vertices*: $\text{keys-of } (\text{bp-view } P) \subseteq \text{set vertices}$
 by (*rule bmssp-partition-state-match-keys-subset*[*OF match*])
 have *keys*: *bmssp-partition-keys-match* settled *ds P*
 using *match unfolding* *bmssp-partition-state-match-def* by *blast*
 have *x-vertices*: $x \in \text{set vertices}$
 using *S-keys keys-vertices xS* by *blast*
 have *x-unsettled*: $x \notin \text{set settled}$
 using *keys S-keys xS unfolding* *bmssp-partition-keys-match-def* by *blast*
 have $x \in \text{set}$
 (*filter* ($\lambda x. x \in S \wedge x \notin \text{set settled}$) *vertices*)
 using *xS x-vertices x-unsettled* by *simp*
 then show $x \in \{\}$
 using *stopped* by *simp*
 qed
 next
 show $\{\} \subseteq S$
 by *simp*
 qed

lemma *bmssp-dijkstra-loop-state-keys-empty-if-pull-empty*:
 assumes *loop-state*: *bmssp-dijkstra-loop-state* *vertices* settled *ds P*
 and *shape*: *bmssp-singleton-bucket-shape P*
 and *pull*: *bp-pull* *bmssp-block-size* *bmssp-bound P* = (*S*, *beta*, *P1*)
 and *S-empty*: $S = \{\}$
 shows $\text{keys-of } (\text{bp-view } P) = \{\}$
proof (*rule ccontr*)
 assume *nonempty*: $\text{keys-of } (\text{bp-view } P) \neq \{\}$
 have *state*: *bmssp-dijkstra-state G* *src* *vertices* settled *ds P*
 and *ord*: *bp-ordered-invariant P*
 using *loop-state unfolding* *bmssp-dijkstra-loop-state-def* by *auto*
 have *match*: *bmssp-partition-state-match* *vertices* settled *ds P*
 using *state unfolding* *bmssp-dijkstra-state-def* by *simp*
 have *values-inj*: *inj-on snd* (*set* (*bp-entries P*))
 by (*rule bmssp-partition-entries-value-inj*[*OF match ord*])
 have $S \neq \{\}$
 by (*rule bmssp-pull-nonempty-if-keys-nonempty*
 [*OF shape ord values-inj pull nonempty*])
 then show *False*
 using *S-empty* by *simp*
 qed

lemma *bmssp-dijkstra-loop-state-quiescent-queue-empty*:
 assumes *loop-state*: *bmssp-dijkstra-loop-state* *vertices* settled *ds P*
 and *shape*: *bmssp-singleton-bucket-shape P*

and *pull*: *bp-pull* *bmssp-block-size* *bmssp-bound* *P* = (*S*, *beta*, *P1*)
and *stopped*:
 $\text{filter } (\lambda x. x \in S \wedge x \notin \text{set settled}) \text{ vertices} = []$
shows *keys-of* (*bp-view* *P*) = {}
proof –
have *S-empty*: *S* = {}
by (*rule* *bmssp-dijkstra-loop-state-quiescent-pull-empty*
 $[OF \text{ loop-state pull stopped}]$)
show ?thesis
by (*rule* *bmssp-dijkstra-loop-state-keys-empty-if-pull-empty*
 $[OF \text{ loop-state shape pull S-empty}]$)
qed

definition *bmssp-executable-distances-fuel* :: nat **where**
bmssp-executable-distances-fuel =
 $\text{Suc } (\text{length } (\text{bmssp-vertices } G \text{ src}) * \text{Suc } (\text{length } G))$

definition *bmssp-executable-initial-partition* :: nat *bucketed-partition* **where**
bmssp-executable-initial-partition =
 bp-result-of
 $(\text{c-bp-regularized-local-insert } \text{src } (\text{bmssp-partition-key } \text{src } 0))$
 $(\text{bp-empty } \text{bmssp-block-size } \text{bmssp-bound})$

lemma *bmssp-distances-unfold-executable*:
 $\text{bmssp-distances } G \text{ src} =$
 $\text{bmssp-loop } \text{bmssp-executable-distances-fuel } G (\text{bmssp-vertices } G \text{ src}) []$
 $[(\text{src}, 0)] \text{bmssp-executable-initial-partition}$
unfolding *bmssp-distances-def* *bmssp-executable-distances-fuel-def*
bmssp-executable-initial-partition-def
by (*simp* *add*: *Let-def*)

lemma *bmssp-executable-initial-partition-singleton-shape*:
 $\text{bmssp-singleton-bucket-shape } \text{bmssp-executable-initial-partition}$
proof –
have *empty-shape*:
 $\text{bmssp-singleton-bucket-shape } (\text{bp-empty } \text{bmssp-block-size } \text{bmssp-bound})$
unfolding *bmssp-singleton-bucket-shape-def* *bp-empty-def* **by** *simp*
show ?thesis
unfolding *bmssp-executable-initial-partition-def*
by (*rule* *bmssp-singleton-bucket-shape-regularized-insert*
 $[OF \text{ empty-shape}]$)
qed

lemma *bmssp-dijkstra-loop-state-initial-executable*:
 $\text{bmssp-dijkstra-loop-state } (\text{bmssp-vertices } G \text{ src}) [] [(\text{src}, 0)]$
 $\text{bmssp-executable-initial-partition}$
proof –
have *src-vertices*: $\text{src} \in \text{set } (\text{bmssp-vertices } G \text{ src})$
using *bmssp-vertices-set*[*of* *G src*] **by** *simp*

```

show ?thesis
  unfolding bmssp-executable-initial-partition-def
  by (rule bmssp-dijkstra-loop-state-initial[OF src-vertices])
qed

lemma bmssp-distances-quiescent-state-executable:
obtains settled ds P S beta P1 where
  bmssp-distances G src = bmssp-normalize-dist ds
  bmssp-dijkstra-loop-state (bmssp-vertices G src) settled ds P
  bmssp-singleton-bucket-shape P
  bp-pull bmssp-block-size bmssp-bound P = (S, beta, P1)
  filter ( $\lambda x. x \in S \wedge x \notin \text{set settled}$ )
    (bmssp-vertices G src) = []
proof –
  let ?vertices = bmssp-vertices G src
  have fuel-gt:
    bmssp-executable-distances-fuel > length ?vertices – length ([] :: nat list)
  proof (cases ?vertices)
    case Nil
    then show ?thesis
      unfolding bmssp-executable-distances-fuel-def by simp
    next
      case (Cons v vs)
      then show ?thesis
        unfolding bmssp-executable-distances-fuel-def by simp
  qed
have distinct-empty: distinct ([] :: nat list)
  by simp
have quiescent:
   $\exists$  settled ds P S beta P1.
    bmssp-loop bmssp-executable-distances-fuel G ?vertices []
    [(src, 0)] bmssp-executable-initial-partition =
      bmssp-normalize-dist ds  $\wedge$ 
    bmssp-dijkstra-loop-state ?vertices settled ds P  $\wedge$ 
    bmssp-singleton-bucket-shape P  $\wedge$ 
    distinct settled  $\wedge$ 
    set []  $\subseteq$  set settled  $\wedge$ 
    bp-pull bmssp-block-size bmssp-bound P = (S, beta, P1)  $\wedge$ 
    filter ( $\lambda x. x \in S \wedge x \notin \text{set settled}$ ) ?vertices = []
  by (rule bmssp-loop-quiescent-state
    [OF bmssp-dijkstra-loop-state-initial-executable
      bmssp-executable-initial-partition-singleton-shape
      bmssp-edge-target-in-vertices bmssp-vertices-distinct
      distinct-empty fuel-gt])
obtain settled ds P S beta P1 where final:
  bmssp-loop bmssp-executable-distances-fuel G ?vertices []
  [(src, 0)] bmssp-executable-initial-partition =
    bmssp-normalize-dist ds
  bmssp-dijkstra-loop-state ?vertices settled ds P

```

bmssp-singleton-bucket-shape P
 bp-pull bmssp-block-size bmssp-bound $P = (S, \text{beta}, P1)$
 filter $(\lambda x. x \in S \wedge x \notin \text{set settled})$?vertices = []
 using quiescent by blast
 have distances-eq: bmssp-distances G src = bmssp-normalize-dist ds
 using final(1) unfolding bmssp-distances-unfold-executable .
 show ?thesis
 by (rule that[OF distances-eq final(2) final(3) final(4) final(5)])
 qed

theorem *bmssp-correct-instance*:

shows

$\forall (v, d) \in \text{set (bmssp-distances } G \text{ src)}.$
 $\text{real } d = \text{nat-graph-dist } G \text{ src } v$
 $\forall v \in \text{nat-graph-vertices } G. \text{ nat-graph-reachable } G \text{ src } v \longrightarrow$
 $(\exists d. (v, d) \in \text{set (bmssp-distances } G \text{ src)})$

proof –

obtain settled ds P S beta $P1$ **where** final:

bmssp-distances G src = bmssp-normalize-dist ds
 bmssp-dijkstra-loop-state (bmssp-vertices G src) settled ds P
 bmssp-singleton-bucket-shape P
 bp-pull bmssp-block-size bmssp-bound $P = (S, \text{beta}, P1)$
 filter $(\lambda x. x \in S \wedge x \notin \text{set settled})$
 (bmssp-vertices G src) = []
 by (rule bmssp-distances-quiescent-state-executable)

have empty: keys-of (bp-view P) = {}

by (rule bmssp-dijkstra-loop-state-quiescent-queue-empty
 [OF final(2) final(3) final(4) final(5)])

show $\forall (v, d) \in \text{set (bmssp-distances } G \text{ src)}.$

$\text{real } d = \text{nat-graph-dist } G \text{ src } v$

proof

fix p

assume $p\text{-mem}$: $p \in \text{set (bmssp-distances } G \text{ src)}$

obtain v d **where** $p\text{-eq}$: $p = (v, d)$

by force

have norm-mem: $(v, d) \in \text{set (bmssp-normalize-dist } ds)$

using $p\text{-mem}$ $p\text{-eq}$ final(1) **by** simp

have exact: $\text{real } d = \text{nat-graph-dist } G \text{ src } v$

by (rule bmssp-dijkstra-loop-state-normalize-exact-if-queue-empty
 [OF final(2) empty norm-mem])

show case p of $(v, d) \Rightarrow \text{real } d = \text{nat-graph-dist } G \text{ src } v$

using $p\text{-eq}$ exact **by** simp

qed

show $\forall v \in \text{nat-graph-vertices } G. \text{ nat-graph-reachable } G \text{ src } v \longrightarrow$

$(\exists d. (v, d) \in \text{set (bmssp-distances } G \text{ src)})$

proof

fix v

assume vV : $v \in \text{nat-graph-vertices } G$

show $\text{nat-graph-reachable } G \text{ src } v \longrightarrow$

```

    (∃ d. (v, d) ∈ set (bmssp-distances G src))
  proof
    assume reach: nat-graph-reachable G src v
    obtain d where norm-mem: (v, d) ∈ set (bmssp-normalize-dist ds)
    using bmssp-dijkstra-loop-state-normalize-complete-if-queue-empty
    [OF final(2) empty reach] by blast
    have (v, d) ∈ set (bmssp-distances G src)
    using norm-mem final(1) by simp
    then show ∃ d. (v, d) ∈ set (bmssp-distances G src)
    by blast
  qed
qed
qed

```

```

lemma bmssp-distances-executable-integral-on-keys:
  real-label-integral-on (set (map fst (bmssp-distances G src)))
  (executable-label-of (bmssp-distances G src))
  by (rule executable-label-integral-on-keys)
  (rule bmssp-distances-distinct-keys)

```

end

```

theorem bmssp-correct-executable:
  assumes nat-graph-well-formed G
  and src ∈ nat-graph-vertices G
  shows
    ∀ (v, d) ∈ set (bmssp-distances G src).
      real d = nat-graph-dist G src v
    ∀ v ∈ nat-graph-vertices G. nat-graph-reachable G src v ⟶
      (∃ d. (v, d) ∈ set (bmssp-distances G src))
  proof -
    interpret inst: nat-graph-instance G src
    by standard (rule assms(1), rule assms(2))
    have sound:
      ∀ (v, d) ∈ set (bmssp-distances G src).
        real d = nat-graph-dist G src v
    by (rule inst.bmssp-correct-instance(1))
    have complete:
      ∀ v ∈ nat-graph-vertices G. nat-graph-reachable G src v ⟶
        (∃ d. (v, d) ∈ set (bmssp-distances G src))
    by (rule inst.bmssp-correct-instance(2))
    show ∀ (v, d) ∈ set (bmssp-distances G src).
      real d = nat-graph-dist G src v
    by (rule sound)
    show ∀ v ∈ nat-graph-vertices G. nat-graph-reachable G src v ⟶
      (∃ d. (v, d) ∈ set (bmssp-distances G src))
    by (rule complete)
  qed

```

```

end
theory BMSSP-Executable-Refinement
  imports BMSSP-Executable-Refinement-Internal
begin

```

39 Executable Refinement

This theory is the public import point for the executable refinement chain. The detailed refinement proof lives in *BMSSP-Executable-Refinement-Internal*. The aliases below keep the outward-facing facts under the public theory name.

```

lemmas bmssp-distances-distinct-keys =
  BMSSP-Executable-Refinement-Internal.bmssp-distances-distinct-keys

```

```

lemmas bmssp-correct-executable =
  BMSSP-Executable-Refinement-Internal.bmssp-correct-executable

```

```

end
theory BMSSP-Executable-Headline
  imports BMSSP-Executable-Refinement
begin

```

40 Executable Headline

This theory states the headline correctness theorem of the entire development. It is the entry point for a reader who wants one statement that summarises what is proved, and it is the theorem that should be consulted before any other. The body of the proof is a short forward reference to the verified executable contract.

The headline applies to the executable entry point *bmssp-distances*, which takes a finite directed graph of natural vertices with natural-valued edge weights and a source vertex, and returns a finite association list mapping each reachable vertex to its shortest-path distance from the source. The graph representation *nat-graph*, the well-formedness predicate *nat-graph-well-formed*, the vertex set *nat-graph-vertices*, the reachability relation *nat-graph-reachable*, and the shortest distance function *nat-graph-dist* are all defined earlier in the session and used unchanged here.

The theorem below is intentionally a single complete statement. It exposes soundness, completeness, and uniqueness of output keys together, so readers do not have to reconstruct the exact map property from several smaller auxiliary facts.

```

theorem bmssp-correct-strong:
  assumes well-formed: nat-graph-well-formed G
    and src-in: src ∈ nat-graph-vertices G
  shows

```


$\text{distinct } (\text{map fst } (\text{bmssp-distances } G \text{ src})) \wedge$
 $\text{set } (\text{map fst } (\text{bmssp-distances } G \text{ src})) =$
 $\{v \in \text{nat-graph-vertices } G. \text{ nat-graph-reachable } G \text{ src } v\} \wedge$
 $(\forall (v, d) \in \text{set } (\text{bmssp-distances } G \text{ src}).$
 $\text{real } d = \text{nat-graph-dist } G \text{ src } v)$

proof –

interpret *inst*: *nat-graph-instance* *G src*

by *standard* (*rule well-formed*, *rule src-in*)

have *distinct-keys*:

$\text{distinct } (\text{map fst } (\text{bmssp-distances } G \text{ src}))$

by (*rule BMSSP-Executable-Refinement.bmssp-distances-distinct-keys*)

have *output-subset*:

$\text{set } (\text{map fst } (\text{bmssp-distances } G \text{ src})) \subseteq$
 $\{v \in \text{nat-graph-vertices } G. \text{ nat-graph-reachable } G \text{ src } v\}$

proof

fix *v*

assume *v-key*: $v \in \text{set } (\text{map fst } (\text{bmssp-distances } G \text{ src}))$

then obtain *d* **where** *mem*: $(v, d) \in \text{set } (\text{bmssp-distances } G \text{ src})$

by *force*

have *v-vertices*: $v \in \text{nat-graph-vertices } G$

using *inst.bmssp-distances-keys-subset-carrier v-key* **by** *blast*

have *reachable*: $\text{nat-graph-reachable } G \text{ src } v$

by (*rule inst.bmssp-distances-output-reachable[OF mem]*)

show $v \in \{v \in \text{nat-graph-vertices } G. \text{ nat-graph-reachable } G \text{ src } v\}$

using *v-vertices reachable* **by** *simp*

qed

have *reachable-subset*:

$\{v \in \text{nat-graph-vertices } G. \text{ nat-graph-reachable } G \text{ src } v\} \subseteq$
 $\text{set } (\text{map fst } (\text{bmssp-distances } G \text{ src}))$

proof

fix *v*

assume $v \in \{v \in \text{nat-graph-vertices } G. \text{ nat-graph-reachable } G \text{ src } v\}$

then have *v-vertices*: $v \in \text{nat-graph-vertices } G$

and *reachable*: $\text{nat-graph-reachable } G \text{ src } v$

by *simp-all*

obtain *d* **where** $(v, d) \in \text{set } (\text{bmssp-distances } G \text{ src})$

using *BMSSP-Executable-Refinement.bmssp-correct-executable(2)*

$[\text{OF well-formed src-in}]$ *v-vertices reachable* **by** *blast*

then show $v \in \text{set } (\text{map fst } (\text{bmssp-distances } G \text{ src}))$

by *force*

qed

have *exact-distances*:

$\forall (v, d) \in \text{set } (\text{bmssp-distances } G \text{ src}).$
 $\text{real } d = \text{nat-graph-dist } G \text{ src } v$

by (*rule BMSSP-Executable-Refinement.bmssp-correct-executable(1)*)

$[\text{OF well-formed src-in}]$

have *key-set*:

$\text{set } (\text{map fst } (\text{bmssp-distances } G \text{ src})) =$
 $\{v \in \text{nat-graph-vertices } G. \text{ nat-graph-reachable } G \text{ src } v\}$

```

    using output-subset reachable-subset by blast
  show ?thesis
    using distinct-keys key-set exact-distances by blast
qed

```

The proof combines the underlying executable theorem $\llbracket \text{nat-graph-well-formed } ?G; ?src \in \text{nat-graph-vertices } ?G \rrbracket \implies \forall (v, d) \in \text{set } (\text{bmssp-distances } ?G ?src). \text{ real } d = \text{nat-graph-dist } ?G ?src v$

$\llbracket \text{nat-graph-well-formed } ?G; ?src \in \text{nat-graph-vertices } ?G \rrbracket \implies \forall v \in \text{nat-graph-vertices } ?G. \text{ nat-graph-reachable } ?G ?src v \longrightarrow (\exists d. (v, d) \in \text{set } (\text{bmssp-distances } ?G ?src))$ with the distinct-key and output-reachability facts from the executable refinement layer. The two clauses of $\llbracket \text{nat-graph-well-formed } ?G; ?src \in \text{nat-graph-vertices } ?G \rrbracket \implies \forall (v, d) \in \text{set } (\text{bmssp-distances } ?G ?src). \text{ real } d = \text{nat-graph-dist } ?G ?src v$

$\llbracket \text{nat-graph-well-formed } ?G; ?src \in \text{nat-graph-vertices } ?G \rrbracket \implies \forall v \in \text{nat-graph-vertices } ?G. \text{ nat-graph-reachable } ?G ?src v \longrightarrow (\exists d. (v, d) \in \text{set } (\text{bmssp-distances } ?G ?src))$ give exact distances for every returned pair and completeness for every reachable vertex; the local key-set argument above adds the converse inclusion and the distinctness statement.

The proof is deliberately structured rather than written as a one-line automation script. The intention is to make the proof obligation visible and to expose how the headline is derived from the verified executable contract. The automated steps are confined to small set and tuple manipulations after the named refinement facts have been supplied. Tracing the proof reveals exactly which named theorem proves what.

In short, the entire development reduces to:

- *bmssp-distances* computes a finite list of pairs.
- For every well-formed graph and every source vertex of that graph, every returned pair gives the exact shortest-path distance from the source.
- For every well-formed graph and every source vertex of that graph, every reachable vertex appears in the output, and no unreachable vertex appears.

The remaining theories prove these facts. This file states them.

```

end
theory BMSSP-Verified-Runtime
  imports BMSSP-Top-Level-Bounds
begin

```

41 Runtime Bound for a Verified-Correct BMSSP Run

The top-level theory *BMSSP-Top-Level-Bounds* proves two facts that, taken separately, leave a gap a reader is right to worry about.

- *exact-concrete-top-level-run-correct-and-refined-bound-log-params-fixed-degree* shows that *one* costed run is simultaneously correct and cost-bounded: for a single *exact-concrete-top-level-run* producing output label d' , completed set U and cost c , we have $U = V$, *sssp-correct* d' , and a closed-form bound on c . This is a per-run statement in closed form.
- *bmssp-runtime-bigo-target* lifts the closed-form bound to the asymptotic headline $O(m * (\ln n) \text{ powr } (2/3))$, but it is stated about *T-bmssp D* alone — the *least cost* over valid runs. As a number, *T-bmssp D N* carries no evidence that the run achieving it computes shortest paths.

The asymptotic headline therefore speaks about a least-cost integer, while the correctness theorem speaks about an individual run; nothing yet says the run whose cost realises the asymptotic bound is the same run that is proved correct. This theory closes that gap. Its central observation is that *T-bmssp D N*, being a least cost, is realised by an *actual* run (by *exact-concrete-top-level-time-witness*), and that this realising run is governed by the per-run correctness theorem. Consequently the asymptotic running-time bound is a statement about runs that are themselves verified to return exact single-source shortest-path distances.

context *strict-tie-breaking-digraph*
begin

First, the least-cost value *T-bmssp D N* is realised by a concrete top-level run, and that run satisfies full single-source shortest-path correctness. The hypothesis is exactly the one already used throughout the top-level development: at least one valid run exists for the given size.

theorem *T-bmssp-realised-by-verified-run:*

assumes *all-reachable*: $\bigwedge v. v \in V \implies \text{reachable } s \ v$

and *degree*: *edge-outdegree-le D*

and *run-exists*: *exact-concrete-top-level-cost D N c*

shows $\exists d' \ U.$

exact-concrete-top-level-run D N d' U (T-bmssp D N) $\wedge$
U = V $\wedge$ sssp-correct d'

proof —

have *cost-T:*

exact-concrete-top-level-cost D N (exact-concrete-top-level-time D N)

```

    by (rule exact-concrete-top-level-time-witness[OF run-exists])
  then obtain d' U where run:
    exact-concrete-top-level-run D N d' U (exact-concrete-top-level-time D N)
  unfolding exact-concrete-top-level-cost-def by blast
  have correct:
    U = V ∧ sssp-correct d' ∧
    exact-concrete-top-level-time D N ≤ bmssp-refined-graph-time-bound
    (λ-. Suc D * sssp-log-one-third-param N)
    (λ-. sssp-log-two-thirds-param N)
    (λ-. sssp-log-one-third-param N)
    (λ-. sssp-log-one-third-param N)
    (λ-. sssp-log-two-thirds-param N)
    (λ-. edge-count) vertex-count
  by (rule exact-concrete-top-level-run-correct-and-refined-bound-log-params-fixed-
degree
    [OF all-reachable degree run])
  show ?thesis
    using run correct unfolding T-bmssp-def by blast
qed

The realising run exists for all but finitely many sizes: the same eventual-
existence fact that powers the asymptotic proof (eventually-exact-concrete-
top-level-cost-from-empty-top-pivots) guarantees a valid run, hence a verified
realising run, eventually in  $N$ .

theorem eventually-T-bmssp-realised-by-verified-run:
  assumes all-reachable:  $\bigwedge v. v \in V \implies \text{reachable } s \ v$ 
  and degree: edge-outdegree-le D
  shows eventually
    (λN.  $\exists d' U.$ 
      exact-concrete-top-level-run D N d' U (T-bmssp D N) ∧
      U = V ∧ sssp-correct d') at-top
proof –
  have runs:
    eventually (λN.  $\exists c. \text{exact-concrete-top-level-cost } D \ N \ c$ ) at-top
  by (rule eventually-exact-concrete-top-level-cost-from-empty-top-pivots)
  show ?thesis
proof (rule eventually-mono[OF runs])
  fix N
  assume  $\exists c. \text{exact-concrete-top-level-cost } D \ N \ c$ 
  then obtain c where c: exact-concrete-top-level-cost D N c by blast
  show  $\exists d' U.$ 
    exact-concrete-top-level-run D N d' U (T-bmssp D N) ∧
    U = V ∧ sssp-correct d'
  by (rule T-bmssp-realised-by-verified-run[OF all-reachable degree c])
qed
qed
end

The public headline of this theory lives in the same asymptotic locale

```

as *bmssp-runtime-headline-instance.bmssp-runtime-bigo-target*. It states the deterministic BMSSP running-time bound together with the witness that the running time belongs to runs returning exact shortest-path distances: for all but finitely many sizes the checked running time *T-bmssp N* is the cost of a concrete run whose output is *sssp-correct* and whose completed set is the whole vertex set.

context *bmssp-runtime-headline-instance*
begin

theorem *verified-bmssp-runtime-bigo-target*:

shows $(\lambda n. \text{real } (T\text{-bmssp } n)) \in$
 $O(\lambda n. \text{real } (m \ n) * (\ln (\text{real } n + 2)) \text{ powr } (2 / 3)) \wedge$
eventually
 $(\lambda N. \exists d' U.$
 $\text{bounded.exact-concrete-top-level-run } D \ N \ d' \ U \ (T\text{-bmssp } N) \wedge$
 $U = V \wedge \text{bounded.sssp-correct } d') \text{ at-top}$

proof

show $(\lambda n. \text{real } (T\text{-bmssp } n)) \in$
 $O(\lambda n. \text{real } (m \ n) * (\ln (\text{real } n + 2)) \text{ powr } (2 / 3))$
by (*rule bmssp-runtime-bigo-target*)
next
have *eventually*
 $(\lambda N. \exists d' U.$
 $\text{bounded.exact-concrete-top-level-run } D \ N \ d' \ U$
 $(\text{bounded.T-bmssp } D \ N) \wedge$
 $U = V \wedge \text{bounded.sssp-correct } d') \text{ at-top}$
by (*rule bounded.eventually-T-bmssp-realised-by-verified-run*
 $[OF \text{ bounded.all-vertices-reachable bounded.bounded-edge-outdegree}]$)
then show *eventually*
 $(\lambda N. \exists d' U.$
 $\text{bounded.exact-concrete-top-level-run } D \ N \ d' \ U \ (T\text{-bmssp } N) \wedge$
 $U = V \wedge \text{bounded.sssp-correct } d') \text{ at-top}$
unfolding *T-bmssp-def* .

qed

Read together, the two conjuncts of *verified-bmssp-runtime-bigo-target* say that the $O(\lambda n. \text{real } (m \ n) * \ln (\text{real } n + 2) \text{ powr } (2 / 3))$ running-time bound is not a statement about an abstract least cost in isolation: the cost it bounds is, for all but finitely many sizes, achieved by an explicit BMSSP run that provably computes the exact single-source shortest-path distance function on every reachable vertex. This is the sense in which the entry delivers a single algorithm that is at once correct and within the deterministic sorting-barrier running time.

end

end

theory *BMSSP-Runtime-Instance*

imports *BMSSP-Verified-Runtime BMSSP-Executable-Base-Case*

begin

42 A Concrete Non-Vacuous Instance of the Runtime Locale

The asymptotic running-time headline lives in the *bounded-reduced-positive-instance* locale, whose interpretation yields the closed theorem *bmssp-runtime-headline-instance.bmssp-runtime-bigo-target*. A locale theorem only carries content once the locale is shown to be inhabited. This theory exhibits a concrete, non-trivial graph—the unit-weight directed path $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ —and discharges every assumption of that locale, obtaining the $O(m * (\ln n) \text{ powr } (2 / 3))$ bound for it as a closed, assumption-free statement. This certifies that the runtime headline is not vacuously true.

Uniqueness of shortest walks—the only non-routine locale assumption—is obtained from the verified simple-walk enumerator *exec-simple-walks-betw*: on this graph the enumerator returns exactly one simple walk to each vertex, so any two shortest walks to a vertex coincide.

definition *path-vs* :: *nat list* **where**
path-vs = [0, 1, 2, 3]

definition *path-es* :: (*nat* × *nat*) *list* **where**
path-es = [(0, 1), (1, 2), (2, 3)]

definition *path-V* :: *nat set* **where**
path-V = *set path-vs*

definition *path-E* :: (*nat* × *nat*) *set* **where**
path-E = *set path-es*

definition *path-weight* :: *nat* ⇒ *nat* ⇒ *real* **where**
path-weight *u v* = (if (*u, v*) ∈ *path-E* then 1 else 0)

lemma *path-vs-V*: *set path-vs* = *path-V*
unfolding *path-V-def* **by** (*rule refl*)

lemma *path-es-E*: *set path-es* = *path-E*
unfolding *path-E-def* **by** (*rule refl*)

lemma *path-positive-weight*: (*u, v*) ∈ *path-E* ⇒ 0 < *path-weight* *u v*
unfolding *path-weight-def* **by** *simp*

lemma *path-finite-weighted-digraph*:
finite-weighted-digraph path-V path-E path-weight 0
unfolding *finite-weighted-digraph-def*
path-V-def path-vs-def path-E-def path-es-def path-weight-def
by *auto*

interpretation *pfw*: *finite-weighted-digraph path-V path-E path-weight 0*
by (*rule path-finite-weighted-digraph*)

The enumerator returns exactly one simple walk from 0 to each vertex, checked by evaluation of the generated code.

lemma *path-walk-lists*:

exec-simple-walks-betw path-vs path-es 0 0 = $[[0]]$
exec-simple-walks-betw path-vs path-es 0 (Suc 0) = $[[0, 1]]$
exec-simple-walks-betw path-vs path-es 0 2 = $[[0, 1, 2]]$
exec-simple-walks-betw path-vs path-es 0 3 = $[[0, 1, 2, 3]]$
by *eval+*

lemma *path-simple-walk-sets*:

$\{p. \text{pfw.simple-walk-betw } 0 \ p \ 0\} = \{\{0\}\}$
 $\{p. \text{pfw.simple-walk-betw } 0 \ p \ (\text{Suc } 0)\} = \{\{0, 1\}\}$
 $\{p. \text{pfw.simple-walk-betw } 0 \ p \ 2\} = \{\{0, 1, 2\}\}$
 $\{p. \text{pfw.simple-walk-betw } 0 \ p \ 3\} = \{\{0, 1, 2, 3\}\}$
using *pfw.set-exec-simple-walks-betw*[*OF path-vs-V path-es-E, symmetric*]
by (*simp-all add: path-walk-lists*)

lemma *path-walk-vertex*:

assumes *pfw.simple-walk-betw 0 p v*
shows $v \in \text{path-}V$

proof –

have *walk-p*: *pfw.walk p* **and** *p-ne*: $p \neq []$ **and** *last-p*: *last p = v*
using *assms unfolding pfw.simple-walk-betw-def pfw.walk-betw-def* **by** *auto*
have $v \in \text{set } p$
using *p-ne last-p* **by** (*metis last-in-set*)
moreover **have** $\text{set } p \subseteq \text{path-}V$
using *walk-p* **by** (*rule pfw.walk-set-subset*)
ultimately show *?thesis* **by** *blast*

qed

lemma *path-unique-shortest-walk*:

assumes *pfw.shortest-walk 0 p v* **and** *pfw.shortest-walk 0 q v*
shows $p = q$

proof –

have *p*: *pfw.simple-walk-betw 0 p v* **and** *q*: *pfw.simple-walk-betw 0 q v*
using *assms unfolding pfw.shortest-walk-def* **by** *blast+*
have *vV*: $v \in \text{path-}V$
using *p* **by** (*rule path-walk-vertex*)
have *pm*: $p \in \{r. \text{pfw.simple-walk-betw } 0 \ r \ v\}$
using *p* **by** *simp*
have *qm*: $q \in \{r. \text{pfw.simple-walk-betw } 0 \ r \ v\}$
using *q* **by** *simp*
from *vV* **consider** $v = 0 \mid v = \text{Suc } 0 \mid v = 2 \mid v = 3$
unfolding *path-V-def path-vs-def* **by** *auto*
then show $p = q$

```

proof cases
  case 1 with pm qm show ?thesis by (simp add: path-simple-walk-sets)
next
  case 2 with pm qm show ?thesis by (simp add: path-simple-walk-sets)
next
  case 3 with pm qm show ?thesis by (simp add: path-simple-walk-sets)
next
  case 4 with pm qm show ?thesis by (simp add: path-simple-walk-sets)
qed
qed

```

lemma *path-all-reachable*:

assumes $v \in \text{path-}V$

shows *pfw.reachable* 0 *v*

proof –

from *assms* **consider** $v = 0 \mid v = \text{Suc } 0 \mid v = 2 \mid v = 3$

unfolding *path-V-def path-vs-def* **by** *auto*

then show ?*thesis*

proof *cases*

case 1

have $[0] \in \{r. \text{pfw.simple-walk-betw } 0 \ r \ 0\}$

by (*simp add: path-simple-walk-sets*)

with 1 **show** ?*thesis* **unfolding** *pfw.reachable-def* **by** *auto*

next

case 2

have $[0, 1] \in \{r. \text{pfw.simple-walk-betw } 0 \ r \ (\text{Suc } 0)\}$

by (*simp add: path-simple-walk-sets*)

with 2 **show** ?*thesis* **unfolding** *pfw.reachable-def* **by** *auto*

next

case 3

have $[0, 1, 2] \in \{r. \text{pfw.simple-walk-betw } 0 \ r \ 2\}$

by (*simp add: path-simple-walk-sets*)

with 3 **show** ?*thesis* **unfolding** *pfw.reachable-def* **by** *auto*

next

case 4

have $[0, 1, 2, 3] \in \{r. \text{pfw.simple-walk-betw } 0 \ r \ 3\}$

by (*simp add: path-simple-walk-sets*)

with 4 **show** ?*thesis* **unfolding** *pfw.reachable-def* **by** *auto*

qed

qed

interpretation *pusd*: *unique-shortest-digraph path-V path-E path-weight 0*

by *unfold-locales (rule path-unique-shortest-walk)*

lemma *path-edge-outdegree*: *pusd.edge-outdegree-le 1*

unfolding *pusd.edge-outdegree-le-def*

proof

fix *u* :: *nat*

assume $u \in \text{path-}V$


```

then consider  $u = 0 \mid u = \text{Suc } 0 \mid u = 2 \mid u = 3$ 
  unfolding path-V-def path-vs-def by auto
then show  $\text{card } (\text{pusd.outgoing-edges } \{u\}) \leq 1$ 
proof cases
  case 1
  then have  $\text{pusd.outgoing-edges } \{u\} = \{(0, 1)\}$ 
    unfolding pusd.outgoing-edges-def by (auto simp: path-E-def path-es-def)
  then show ?thesis by simp
next
  case 2
  then have  $\text{pusd.outgoing-edges } \{u\} = \{(1, 2)\}$ 
    unfolding pusd.outgoing-edges-def by (auto simp: path-E-def path-es-def)
  then show ?thesis by simp
next
  case 3
  then have  $\text{pusd.outgoing-edges } \{u\} = \{(2, 3)\}$ 
    unfolding pusd.outgoing-edges-def by (auto simp: path-E-def path-es-def)
  then show ?thesis by simp
next
  case 4
  then have  $\text{pusd.outgoing-edges } \{u\} = \{\}$ 
    unfolding pusd.outgoing-edges-def by (auto simp: path-E-def path-es-def)
  then show ?thesis by simp
qed
qed

```

interpretation *pbr*: *bounded-reduced-positive-instance path-V path-E path-weight 0 1*

by *unfold-locales*
 (use *path-positive-weight path-all-reachable path-edge-outdegree* in *auto*)

The closed running-time bound for the concrete path instance. Because it is obtained by interpreting *bounded-reduced-positive-instance* on an explicit graph, it has no remaining locale hypotheses: it is an assumption-free witness that the BMSSP runtime headline is satisfiable by a genuine non-trivial digraph.

lemmas *path-runtime-bigo-target* =
pbr.runtime-headline.bmssp-runtime-bigo-target

The strengthened witness. Re-exporting *verified-bmssp-runtime-bigo-target* on the same concrete path instance yields a closed, assumption-free statement that couples the $O(m * (\ln n) \text{ powr } (2 / 3))$ running-time bound with the guarantee that the bounded running time is, for all but finitely many sizes, the cost of an explicit run returning the exact single-source shortest-path distances on the whole vertex set. This certifies that the runtime headline is non-vacuous in the strong sense: the cost it bounds belongs to genuinely verified-correct BMSSP runs, not merely to an abstract least-cost integer.

```

lemmas path-verified-runtime-bigo-target =
  pbr.runtime-headline.verified-bmssp-runtime-bigo-target

end
theory BMSSP-Runtime-Instance-Branching
  imports BMSSP-Verified-Runtime BMSSP-Executable-Base-Case
begin

```

43 A Second, Structurally Different Non-Vacuous Instance

The path instance in *BMSSP-Runtime-Instance* certifies the runtime locale on a single chain. To show that inhabitation does not rely on the degenerate linear shape, this theory discharges the same locale on a structurally different graph: the out-star $1 \leftarrow 0 \rightarrow 2$ with two leaves, in which the source has outdegree two and there are two incomparable shortest paths. The proof structure mirrors the path instance: the verified simple-walk enumerator pins down the unique shortest walk to each vertex.

```

definition star-vs :: nat list where
  star-vs = [0, 1, 2]

```

```

definition star-es :: (nat × nat) list where
  star-es = [(0, 1), (0, 2)]

```

```

definition star-V :: nat set where
  star-V = set star-vs

```

```

definition star-E :: (nat × nat) set where
  star-E = set star-es

```

```

definition star-weight :: nat ⇒ nat ⇒ real where
  star-weight u v = (if (u, v) ∈ star-E then 1 else 0)

```

```

lemma star-vs-V: set star-vs = star-V
  unfolding star-V-def by (rule refl)

```

```

lemma star-es-E: set star-es = star-E
  unfolding star-E-def by (rule refl)

```

```

lemma star-positive-weight: (u, v) ∈ star-E ⇒ 0 < star-weight u v
  unfolding star-weight-def by simp

```

```

lemma star-finite-weighted-digraph:
  finite-weighted-digraph star-V star-E star-weight 0
  unfolding finite-weighted-digraph-def
    star-V-def star-vs-def star-E-def star-es-def star-weight-def
  by auto

```

interpretation *sfw*: *finite-weighted-digraph star-V star-E star-weight 0*
by (*rule star-finite-weighted-digraph*)

lemma *star-walk-lists*:

exec-simple-walks-betw star-vs star-es 0 0 = $[[0]]$
exec-simple-walks-betw star-vs star-es 0 (Suc 0) = $[[0, 1]]$
exec-simple-walks-betw star-vs star-es 0 2 = $[[0, 2]]$
by *eval+*

lemma *star-simple-walk-sets*:

$\{p. \text{sfw.simple-walk-betw } 0 \ p \ 0\} = \{\{0\}\}$
 $\{p. \text{sfw.simple-walk-betw } 0 \ p \ (\text{Suc } 0)\} = \{\{0, 1\}\}$
 $\{p. \text{sfw.simple-walk-betw } 0 \ p \ 2\} = \{\{0, 2\}\}$
using *sfw.set-exec-simple-walks-betw* [*OF star-vs-V star-es-E, symmetric*]
by (*simp-all add: star-walk-lists*)

lemma *star-walk-vertex*:

assumes *sfw.simple-walk-betw 0 p v*
shows $v \in \text{star-}V$

proof –

have *walk-p*: *sfw.walk p* **and** *p-ne*: $p \neq []$ **and** *last-p*: *last p = v*
using *assms unfolding sfw.simple-walk-betw-def sfw.walk-betw-def* **by** *auto*
have $v \in \text{set } p$
using *p-ne last-p* **by** (*metis last-in-set*)
moreover **have** $\text{set } p \subseteq \text{star-}V$
using *walk-p* **by** (*rule sfw.walk-set-subset*)
ultimately show *?thesis* **by** *blast*

qed

lemma *star-unique-shortest-walk*:

assumes *sfw.shortest-walk 0 p v* **and** *sfw.shortest-walk 0 q v*
shows $p = q$

proof –

have *p*: *sfw.simple-walk-betw 0 p v* **and** *q*: *sfw.simple-walk-betw 0 q v*
using *assms unfolding sfw.shortest-walk-def* **by** *blast+*
have *vV*: $v \in \text{star-}V$
using *p* **by** (*rule star-walk-vertex*)
have *pm*: $p \in \{r. \text{sfw.simple-walk-betw } 0 \ r \ v\}$
using *p* **by** *simp*
have *qm*: $q \in \{r. \text{sfw.simple-walk-betw } 0 \ r \ v\}$
using *q* **by** *simp*
from *vV* **consider** $v = 0 \mid v = \text{Suc } 0 \mid v = 2$
unfolding *star-V-def star-vs-def* **by** *auto*
then show $p = q$
proof *cases*
case 1 with *pm qm* **show** *?thesis* **by** (*simp add: star-simple-walk-sets*)
next
case 2 with *pm qm* **show** *?thesis* **by** (*simp add: star-simple-walk-sets*)

```

next
  case 3 with pm qm show ?thesis by (simp add: star-simple-walk-sets)
qed
qed

```

lemma *star-all-reachable*:

```

assumes  $v \in \text{star-}V$ 
shows  $\text{sfw.reachable } 0 \ v$ 
proof -
  from assms consider  $v = 0 \mid v = \text{Suc } 0 \mid v = 2$ 
  unfolding star-V-def star-vs-def by auto
  then show ?thesis
  proof cases
    case 1
    have  $[0] \in \{r. \text{sfw.simple-walk-betw } 0 \ r \ 0\}$ 
    by (simp add: star-simple-walk-sets)
    with 1 show ?thesis unfolding sfw.reachable-def by auto
  next
    case 2
    have  $[0, 1] \in \{r. \text{sfw.simple-walk-betw } 0 \ r \ (\text{Suc } 0)\}$ 
    by (simp add: star-simple-walk-sets)
    with 2 show ?thesis unfolding sfw.reachable-def by auto
  next
    case 3
    have  $[0, 2] \in \{r. \text{sfw.simple-walk-betw } 0 \ r \ 2\}$ 
    by (simp add: star-simple-walk-sets)
    with 3 show ?thesis unfolding sfw.reachable-def by auto
  qed
qed

```

interpretation *susd*: unique-shortest-digraph *star-V* *star-E* *star-weight 0*
 by unfold-locales (rule star-unique-shortest-walk)

lemma *star-edge-outdegree*: *susd.edge-outdegree-le 2*

```

unfolding susd.edge-outdegree-le-def
proof
  fix  $u :: \text{nat}$ 
  assume  $u \in \text{star-}V$ 
  then consider  $u = 0 \mid u = \text{Suc } 0 \mid u = 2$ 
  unfolding star-V-def star-vs-def by auto
  then show  $\text{card } (\text{susd.outgoing-edges } \{u\}) \leq 2$ 
  proof cases
    case 1
    then have  $\text{susd.outgoing-edges } \{u\} = \{(0, 1), (0, 2)\}$ 
    unfolding susd.outgoing-edges-def by (auto simp: star-E-def star-es-def)
    then show ?thesis by (simp add: card-insert-if)
  next
    case 2
    then have  $\text{susd.outgoing-edges } \{u\} = \{\}$ 

```

```

    unfolding susd.outgoing-edges-def by (auto simp: star-E-def star-es-def)
  then show ?thesis by simp
next
case 3
then have susd.outgoing-edges {u} = {}
  unfolding susd.outgoing-edges-def by (auto simp: star-E-def star-es-def)
then show ?thesis by simp
qed
qed

interpretation sbr: bounded-reduced-positive-instance star-V star-E star-weight 0 2
  by unfold-locales
  (use star-positive-weight star-all-reachable star-edge-outdegree in auto)

  As with the path, the out-star yields the closed, assumption-free running-
  time bound and its verified-correct strengthening. The two instances share
  no structural feature beyond the locale assumptions, so together they witness
  that the BMSSP runtime headline holds across genuinely different graph
  shapes.

lemmas star-runtime-bigo-target =
  sbr.runtime-headline.bmssp-runtime-bigo-target

lemmas star-verified-runtime-bigo-target =
  sbr.runtime-headline.verified-bmssp-runtime-bigo-target

end
theory BMSSP-Path-Family
  imports BMSSP-Verified-Runtime
begin

```

44 A Size-Parametric BMSSP Running-Time Bound over a Growing Graph Family

The headline *bmssp-runtime-headline-instance.bmssp-runtime-bigo-target* is stated for a graph fixed by the locale, with the asymptotic variable feeding only the internal logarithmic schedule; the edge-count factor in its target is a constant. This theory removes that limitation by exhibiting a genuine *family* of graphs whose size grows without bound: the unit-weight directed path P_k on vertices $0, \dots, k$ with edges $i \rightarrow i + 1$. For this family $\text{card } (E_k) = k$ and $\text{card } (V_k) = k + 1$ both grow with k , so k is a true graph-size variable, and we prove the BMSSP running time is $O(k \cdot (\ln k)^{2/3})$.

44.1 The path family

definition *path-k-V* :: *nat* $\Rightarrow$ *nat set* **where**
path-k-V $k = \{0..k\}$

definition $\text{path-}k\text{-}E :: \text{nat} \Rightarrow (\text{nat} \times \text{nat}) \text{ set}$ **where**

$\text{path-}k\text{-}E \ k = (\lambda i. (i, \text{Suc } i)) \text{ ` } \{0..<k\}$

definition $\text{path-}k\text{-weight} :: \text{nat} \Rightarrow \text{nat} \Rightarrow \text{nat} \Rightarrow \text{real}$ **where**

$\text{path-}k\text{-weight} \ k \ u \ v = (\text{if } (u, v) \in \text{path-}k\text{-}E \ k \text{ then } 1 \text{ else } 0)$

44.2 Structural and cardinality lemmas

lemma $\text{path-}k\text{-}V\text{-eq}$: $\text{path-}k\text{-}V \ k = \{0..k\}$

unfolding $\text{path-}k\text{-}V\text{-def}$ **..**

lemma $\text{path-}k\text{-}E\text{-iff}$: $(u, v) \in \text{path-}k\text{-}E \ k \longleftrightarrow v = \text{Suc } u \wedge u < k$

unfolding $\text{path-}k\text{-}E\text{-def}$ **by** *auto*

lemma $\text{path-}k\text{-no-incoming-0}$: $(u, 0) \notin \text{path-}k\text{-}E \ k$

by (*simp add: path-}k\text{-}E\text{-iff}*)

lemma $\text{path-}k\text{-incoming-unique}$:

$(u, j) \in \text{path-}k\text{-}E \ k \longleftrightarrow (0 < j \wedge j \leq k \wedge u = j - 1)$

by (*auto simp: path-}k\text{-}E\text{-iff}*)

lemma $\text{path-}k\text{-card-}V$: $\text{card } (\text{path-}k\text{-}V \ k) = \text{Suc } k$

unfolding $\text{path-}k\text{-}V\text{-eq}$ **by** *simp*

lemma $\text{path-}k\text{-card-}E$: $\text{card } (\text{path-}k\text{-}E \ k) = k$

proof –

have $\text{inj-on } (\lambda i. (i, \text{Suc } i)) \ \{0..<k\}$

by (*rule inj-onI*) *auto*

then show *?thesis*

unfolding $\text{path-}k\text{-}E\text{-def}$ **by** (*simp add: card-image*)

qed

lemma $\text{path-}k\text{-finite-}V$: $\text{finite } (\text{path-}k\text{-}V \ k)$

unfolding $\text{path-}k\text{-}V\text{-eq}$ **by** *simp*

lemma $\text{path-}k\text{-finite-}E$: $\text{finite } (\text{path-}k\text{-}E \ k)$

unfolding $\text{path-}k\text{-}E\text{-def}$ **by** *simp*

44.3 Easy locale obligations, uniform in k

lemma $\text{path-}k\text{-finite-weighted-digraph}$:

$\text{finite-weighted-digraph } (\text{path-}k\text{-}V \ k) (\text{path-}k\text{-}E \ k) (\text{path-}k\text{-weight } k) \ 0$

proof

show $\text{finite } (\text{path-}k\text{-}V \ k)$

by (*rule path-}k\text{-finite-}V*)

show $0 \in \text{path-}k\text{-}V \ k$

unfolding $\text{path-}k\text{-}V\text{-eq}$ **by** *simp*

show $\bigwedge u \ v. (u, v) \in \text{path-}k\text{-}E \ k \implies u \in \text{path-}k\text{-}V \ k \wedge v \in \text{path-}k\text{-}V \ k$

by (*auto simp: path-}k\text{-}E\text{-iff path-}k\text{-}V\text{-eq}*)

show $\bigwedge u v. (u, v) \in \text{path-}k\text{-}E\ k \implies 0 \leq \text{path-}k\text{-weight}\ k\ u\ v$
by (*simp add: path-k-weight-def*)
qed

lemma *path-k-positive-weight*:
 $(u, v) \in \text{path-}k\text{-}E\ k \implies 0 < \text{path-}k\text{-weight}\ k\ u\ v$
by (*simp add: path-k-weight-def*)

44.4 The unconditional cost-recurrence bound (FLOOR)

The function *path-size-cost* is exactly the closed cost expression that bounds a BMSSP run on P_k at schedule parameter k , with the graph cardinalities substituted: edge slot k and vertex slot $\text{Suc}\ k$. Its asymptotic bound holds unconditionally — no run existence and no shortest-walk uniqueness are required — and already exhibits a genuine size-parametric $O(k \cdot (\ln k)^{2/3})$ growth in which both the edge count k and the vertex count $\text{Suc}\ k$ grow with the asymptotic variable.

definition *path-size-cost* :: $\text{nat} \Rightarrow \text{nat}$ **where**
path-size-cost $k =$

bmssp-refined-graph-time-bound
 $(\lambda-. \text{Suc}\ 1 * \text{sssp-log-one-third-param}\ k)$
 $(\lambda-. \text{sssp-log-two-thirds-param}\ k)$
 $(\lambda-. \text{sssp-log-one-third-param}\ k)$
 $(\lambda-. \text{sssp-log-one-third-param}\ k)$
 $(\lambda-. \text{sssp-log-two-thirds-param}\ k)$
 $(\lambda-. k) (\text{Suc}\ k)$

theorem *path-size-cost-bigo-size*:

$(\lambda k. \text{real} (\text{path-size-cost}\ k))$
 $\in O(\lambda k. \text{real}\ k * (\ln (\text{real}\ k + 2)) \text{ powr } (2 / 3))$

proof —

have $(\lambda k. \text{real} (\text{path-size-cost}\ k))$
 $\in O(\lambda k. \text{sssp-time-target}\ (\lambda k. k)\ k)$

proof (*rule bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-degree-slack*

[**where** $D = 1$ **and** $Cn = 2$ **and** $Cm = 1$
and $v = \lambda k. \text{Suc}\ k$ **and** $m' = \lambda k. k$ **and** $m = \lambda k. k$])

show $0 < (2 :: \text{real})$ **by** *simp*

show $0 < (1 :: \text{real})$ **by** *simp*

show *eventually* $(\lambda k. \text{real} (\text{Suc}\ k) \leq 2 * \text{real}\ k)$ *at-top*

by (*rule eventually-at-top-linorderI[of 1]*) *simp*

show *eventually* $(\lambda k. \text{real}\ k \leq 1 * \text{real}\ k)$ *at-top*

by *simp*

show *eventually*

$(\lambda k. \text{path-size-cost}\ k \leq$
bmssp-refined-graph-time-bound
 $(\lambda-. \text{Suc}\ 1 * \text{sssp-log-one-third-param}\ k)$
 $(\lambda-. \text{sssp-log-two-thirds-param}\ k)$

```

      (λ-. sssp-log-one-third-param k)
      (λ-. sssp-log-one-third-param k)
      (λ-. sssp-log-two-thirds-param k)
      (λ-. k) (Suc k)) at-top
    by (simp add: path-size-cost-def)
  qed
  then show ?thesis
    unfolding sssp-time-target-def sssp-log-factor-def by simp
  qed

```

44.5 The forced-degenerate (empty-pivot) regime is provably avoided

The only unconditional source of run existence in the development, *strict-tie-breaking-digraph.eventually-exact-concrete-top-level-cost-from-empty-top-pivots*, fires through *unique-shortest-digraph.find-pivots-pivots-capped-singleton-empty-if-params-exceed-vertex-count*, whose hypothesis requires the schedule parameter to *exceed* the vertex count. On P_k at schedule k this hypothesis is $\text{Suc } k < \text{sssp-log-one-third-param } k$, which fails for all large k because the one-third logarithmic factor grows far slower than k . Hence the empty-pivot no-op cannot be the run whose cost the family bound governs: any such run does genuine top-level work.

lemma *sssp-log-one-third-param-le-linear:*

eventually (λk. sssp-log-one-third-param k ≤ Suc k) at-top

proof (rule *eventually-at-top-linorderI*[of 1])

fix $k :: \text{nat}$ **assume** $k1: 1 \leq k$

have $\text{pos}: (0::\text{real}) < \text{real } k + 2$ **by** *simp*

have $\text{ge1}: 1 \leq \ln (\text{real } k + 2)$

proof –

have $\text{exp } 1 \leq (3::\text{real})$ **using** *exp-le* **by** *simp*

also have $(3::\text{real}) \leq \text{real } k + 2$ **using** $k1$ **by** *simp*

finally have $\text{exp } 1 \leq \text{real } k + 2$.

then show ?thesis **using** *ln-ge-iff*[of $\text{real } k + 2$] **by** *simp*

qed

have $\text{ln-le}: \ln (\text{real } k + 2) \leq \text{real } k + 1$

using *ln-le-minus-one*[OF pos] **by** *simp*

have $\text{sssp-log-factor-one-third } k = (\ln (\text{real } k + 2)) \text{ powr } (1/3)$

unfolding *sssp-log-factor-one-third-def* ..

also have $\dots \leq (\ln (\text{real } k + 2)) \text{ powr } 1$

by (rule *powr-mono*[OF - ge1]) *simp*

also have $\dots = \ln (\text{real } k + 2)$ **by** *simp*

also have $\dots \leq \text{real } k + 1$ **using** ln-le .

finally have $\text{factor-le}: \text{sssp-log-factor-one-third } k \leq \text{real } k + 1$.

have $\text{ceil-nonneg}: 0 \leq \lceil \text{sssp-log-factor-one-third } k \rceil$

proof –

have $(0::\text{real}) \leq \text{sssp-log-factor-one-third } k$

using *sssp-log-factor-one-third-pos*[of k] **by** (rule *order-less-imp-le*)

then have $\lceil (0::\text{real}) \rceil \leq \lceil \text{sssp-log-factor-one-third } k \rceil$


```

    by (rule ceiling-mono)
  then show ?thesis by simp
qed
have real (sssp-log-one-third-param k) =
  real (nat ⌈sssp-log-factor-one-third k⌉)
  unfolding sssp-log-one-third-param-def ..
also have ... = real-of-int ⌈sssp-log-factor-one-third k⌉
  using ceil-nonneg by simp
also have ... ≤ real k + 1
proof -
  have ⌈sssp-log-factor-one-third k⌉ ≤ ⌈real k + 1⌉
    by (rule ceiling-mono[OF factor-le])
  also have ... = int k + 1 by simp
  finally show ?thesis by simp
qed
finally have real (sssp-log-one-third-param k) ≤ real (Suc k)
  by simp
then show sssp-log-one-third-param k ≤ Suc k
  by linarith
qed

theorem path-family-not-forced-degenerate:
  eventually
    (λk. ¬ (card (path-k-V k) < sssp-log-one-third-param k)) at-top
proof -
  have eventually (λk. sssp-log-one-third-param k ≤ Suc k) at-top
    by (rule sssp-log-one-third-param-le-linear)
  then show ?thesis
    by (auto elim: eventually-mono simp: path-k-card-V)
qed

```

44.6 Unique shortest walks on the path family

context fixes $k :: nat$ begin

interpretation pk : finite-weighted-digraph
 $path-k-V\ k\ path-k-E\ k\ path-k-weight\ k\ 0$
 by (rule path-k-finite-weighted-digraph)

lemma $path-k-canonical-walk$:

$j \leq k \implies pk.simple-walk-betw\ 0\ [0..<Suc\ j]\ j$

proof (induction j)

case 0

have $(0::nat) \in path-k-V\ k$

by (simp add: path-k-V-eq)

then have $pk.walk\ [0]$ by simp

then show ?case

by (simp add: pk.simple-walk-betw-def pk.walk-betw-def)

next

```

case (Suc j)
from Suc.premis have jk:  $j \leq k$  and jlt:  $j < k$  by simp-all
have ih:  $pk.simple-walk-betw\ 0\ [0..<Suc\ j]\ j$ 
  using Suc.IH jk by blast
have edge:  $(j, Suc\ j) \in path-k-E\ k$ 
  using jlt by (simp add: path-k-E-iff)
have fresh:  $Suc\ j \notin set\ [0..<Suc\ j]$  by simp
have  $pk.simple-walk-betw\ 0\ ([0..<Suc\ j] @ [Suc\ j])\ (Suc\ j)$ 
  using  $pk.simple-walk-snoc[OF\ ih\ edge\ fresh]$  .
moreover have  $[0..<Suc\ j] @ [Suc\ j] = [0..<Suc\ (Suc\ j)]$ 
  by (simp add: upt-Suc-append)
ultimately show ?case by simp
qed

```

lemma *path-k-all-reachable*:

$v \in path-k-V\ k \implies pk.reachable\ 0\ v$

proof –

assume $v \in path-k-V\ k$

then have $v \leq k$ by (simp add: path-k-V-eq)

then have $pk.simple-walk-betw\ 0\ [0..<Suc\ v]\ v$
 by (rule path-k-canonical-walk)

then show $pk.reachable\ 0\ v$

unfolding $pk.reachable-def$ by blast

qed

lemma *path-k-simple-walk-unique*:

$pk.simple-walk-betw\ 0\ p\ j \implies j \leq k \implies p = [0..<Suc\ j]$

proof (induction j arbitrary: p rule: less-induct)

case (less j)

from less.premis have sw: $pk.simple-walk-betw\ 0\ p\ j$ and jk: $j \leq k$
 by blast+

from sw have walk-p: $pk.walk\ p$ and ne: $p \neq []$ and hd-p: $hd\ p = 0$
 and last-p: $last\ p = j$ and dist-p: $distinct\ p$

unfolding $pk.simple-walk-betw-def\ pk.walk-betw-def$ by auto

show ?case

proof (cases j)

case 0

have len1: $length\ p = 1$

proof (rule ccontr)

assume $length\ p \neq 1$

from ne have $length\ p \geq 1$ by (cases p) auto

with $\langle length\ p \neq 1 \rangle$ have len2: $2 \leq length\ p$ by linarith

have idx: $Suc\ (length\ p - 2) < length\ p$ using len2 by linarith

have e: $(p\ !\ (length\ p - 2), p\ !\ Suc\ (length\ p - 2)) \in path-k-E\ k$
 using $pk.walk-nth-edge[OF\ walk-p\ idx]$.

have suceq: $Suc\ (length\ p - 2) = length\ p - 1$ using len2 by linarith

have $p\ !\ (length\ p - 1) = last\ p$

using ne by (simp add: last-conv-nth)

with last-p 0 have $p\ !\ (length\ p - 1) = 0$ by simp

```

    with  $e$  suceq have  $(p ! (\text{length } p - 2), 0) \in \text{path-}k\text{-}E \ k$  by simp
    with  $\text{path-}k\text{-no-incoming-}0$  show False by simp
qed
from  $\text{len1}$  ne obtain  $x$  where  $p = [x]$  by (cases  $p$ ) auto
with  $\text{hd-}p$  have  $p = [0]$  by simp
then show ?thesis using 0 by simp
next
case (Suc  $j'$ )
have  $j'k: j' \leq k$  using  $jk \text{ Suc}$  by simp
have  $j'lt: j' < j$  using  $\text{Suc}$  by simp
have  $\text{ne1}: \text{length } p \neq 1$ 
proof
  assume  $\text{length } p = 1$ 
  with  $\text{ne}$  obtain  $x$  where  $p = [x]$  by (cases  $p$ ) auto
  with  $\text{hd-}p \text{ last-}p \text{ Suc}$  show False by simp
qed
from  $\text{ne}$  have  $\text{length } p \geq 1$  by (cases  $p$ ) auto
with  $\text{ne1}$  have  $\text{len2}: 2 \leq \text{length } p$  by linarith
have  $\text{idx}: \text{Suc } (\text{length } p - 2) < \text{length } p$  using  $\text{len2}$  by linarith
have  $e: (p ! (\text{length } p - 2), p ! \text{Suc } (\text{length } p - 2)) \in \text{path-}k\text{-}E \ k$ 
  using  $\text{pk.walk-nth-edge}[OF \text{ walk-}p \text{ idx}]$  .
have  $\text{suceq}: \text{Suc } (\text{length } p - 2) = \text{length } p - 1$  using  $\text{len2}$  by linarith
have  $\text{last-nth}: p ! (\text{length } p - 1) = \text{Suc } j'$ 
proof -
  have  $p ! (\text{length } p - 1) = \text{last } p$ 
    using  $\text{ne}$  by (simp add:  $\text{last-conv-nth}$ )
  with  $\text{last-}p \text{ Suc}$  show ?thesis by simp
qed
from  $e$  suceq  $\text{last-nth}$ 
have  $\text{edge-last}: (p ! (\text{length } p - 2), \text{Suc } j') \in \text{path-}k\text{-}E \ k$  by simp
have  $\text{nth-n2}: p ! (\text{length } p - 2) = j'$ 
  using  $\text{edge-last}$  unfolding  $\text{path-}k\text{-incoming-unique}$  by simp
have  $\text{idx2}: \text{length } p - 2 < \text{length } p$  using  $\text{len2}$  by linarith
have  $\text{pref}: \text{pk.simple-walk-betw } 0 \ (\text{take } (\text{Suc } (\text{length } p - 2)) \ p)$ 
   $(p ! (\text{length } p - 2))$ 
  using  $\text{pk.simple-walk-prefix}[OF \text{ sw } \text{idx2}]$  .
have  $\text{pref2}: \text{pk.simple-walk-betw } 0 \ (\text{take } (\text{length } p - 1) \ p) \ j'$ 
  using  $\text{pref suceq nth-n2}$  by simp
have  $\text{take-eq}: \text{take } (\text{length } p - 1) \ p = [0..<\text{Suc } j']$ 
  using  $\text{less.IH}[OF \ j'lt \text{pref2 } j'k]$  .
have  $\text{pbl}: p = \text{butlast } p \ @ \ [\text{last } p]$ 
  using  $\text{append-butlast-last-id}[OF \ \text{ne}]$  by simp
have  $\text{bl}: \text{butlast } p = \text{take } (\text{length } p - 1) \ p$ 
  by (rule  $\text{butlast-conv-take}$ )
have  $p = [0..<\text{Suc } j'] \ @ \ [\text{Suc } j']$ 
  using  $\text{pbl bl last-}p \text{ take-eq } \text{Suc}$  by simp
then show ?thesis using  $\text{Suc}$  by (simp add:  $\text{upt-Suc-append}$ )
qed
qed

```

lemma *path-k-unique-shortest-walk*:
 $pk.shortest-walk\ 0\ p\ v \implies pk.shortest-walk\ 0\ q\ v \implies p = q$
proof –
 assume *sp*: *pk.shortest-walk 0 p v* and *sq*: *pk.shortest-walk 0 q v*
 have *swp*: *pk.simple-walk-betw 0 p v*
 using *sp* unfolding *pk.shortest-walk-def* by *blast*
 have *swq*: *pk.simple-walk-betw 0 q v*
 using *sq* unfolding *pk.shortest-walk-def* by *blast*
 have *walk-p*: *pk.walk p* and *ne-p*: $p \neq []$ and *last-pv*: *last p = v*
 using *swp* unfolding *pk.simple-walk-betw-def* *pk.walk-betw-def* by *auto*
 have *v-in*: $v \in set\ p$
 using *last-in-set*[*OF ne-p*] *last-pv* by *simp*
 have $set\ p \subseteq path-k-V\ k$
 using *pk.walk-set-subset*[*OF walk-p*] .
 with *v-in* have $v \in path-k-V\ k$ by *blast*
 then have *vk*: $v \leq k$ by (*simp add: path-k-V-eq*)
 have $p = [0..<Suc\ v]$
 using *path-k-simple-walk-unique*[*OF swp vk*] .
 moreover have $q = [0..<Suc\ v]$
 using *path-k-simple-walk-unique*[*OF swq vk*] .
 ultimately show $p = q$ by *simp*
qed

lemma *path-k-bounded-instance*:
bounded-reduced-positive-instance (*path-k-V k*) (*path-k-E k*)
 (*path-k-weight k*) 0 1
proof –
 interpret *usd*: *unique-shortest-digraph*
 path-k-V k path-k-E k path-k-weight k 0
 by *unfold-locales* (*rule path-k-unique-shortest-walk*)
 have *outdeg*: *usd.edge-outdegree-le 1*
 unfolding *usd.edge-outdegree-le-def*
proof
 fix *u* :: *nat*
 assume $u \in path-k-V\ k$
 show $card\ (usd.outgoing-edges\ \{u\}) \leq 1$
 proof (*cases u < k*)
 case *True*
 then have $usd.outgoing-edges\ \{u\} = \{(u, Suc\ u)\}$
 unfolding *usd.outgoing-edges-def* by (*auto simp: path-k-E-iff*)
 then show ?*thesis* by *simp*
 next
 case *False*
 then have $usd.outgoing-edges\ \{u\} = \{\}$
 unfolding *usd.outgoing-edges-def* by (*auto simp: path-k-E-iff*)
 then show ?*thesis* by *simp*
 qed
qed

```

show ?thesis
proof
  show  $\bigwedge u v. (u, v) \in \text{path-}k\text{-}E\ k \implies 0 < \text{path-}k\text{-weight}\ k\ u\ v$ 
    by (rule path-k-positive-weight)
  show  $\bigwedge v. v \in \text{path-}k\text{-}V\ k \implies \text{pk.reachable}\ 0\ v$ 
    by (rule path-k-all-reachable)
  show  $\text{usd.edge-outdegree-le}\ 1$ 
    by (rule outdeg)
qed
qed
end

```

44.7 The verified algorithm-level running-time bound (GOLD)

The genuine, size-parametric running-time bound for the path family. Unlike the unconditional floor *path-size-cost-bigo-size*, which bounds the closed cost expression with the graph cardinalities substituted, this theorem bounds the *verified BMSSP running time path-T-bmssp* — the least concrete top-level cost realised by an actual run on P_k at schedule parameter k . The single honest hypothesis is run existence: for all but finitely many k at least one costed top-level run exists on P_k . Under it the verified running time is $O(k \cdot (\ln k)^{2/3})$, with both the edge count k and the vertex count $\text{Suc}\ k$ growing with the asymptotic variable.

definition *path-T-bmssp* :: $\text{nat} \Rightarrow \text{nat}$ **where**
path-T-bmssp $k = \text{strict-tie-breaking-digraph.T-bmssp}$
 (*path-k-V* k) (*path-k-E* k) (*path-k-weight* k) $0\ 1\ k$

theorem *bmssp-path-family-runtime-bigo-size*:

assumes *runs-exist*:

eventually ($\lambda k. \exists c. \text{strict-tie-breaking-digraph.exact-concrete-top-level-cost}$
 (*path-k-V* k) (*path-k-E* k) (*path-k-weight* k) $0\ 1\ k\ c$) *at-top*

shows ($\lambda k. \text{real} (\text{path-T-bmssp}\ k)$)
 $\in O(\lambda k. \text{real}\ k * (\ln (\text{real}\ k + 2)) \text{powr}\ (2 / 3))$

proof —

have ($\lambda k. \text{real} (\text{path-T-bmssp}\ k)$)
 $\in O(\lambda k. \text{sssp-time-target}\ (\lambda k. k)\ k)$

proof (rule *bmssp-refined-cost-bound-bigo-sssp-time-target-log-params-bounded-degree-slack*

[**where** $D = 1$ **and** $Cn = 2$ **and** $Cm = 1$
and $v = \lambda k. \text{Suc}\ k$ **and** $m' = \lambda k. k$ **and** $m = \lambda k. k$])

show $0 < (2 :: \text{real})$ **by** *simp*

show $0 < (1 :: \text{real})$ **by** *simp*

show *eventually* ($\lambda k. \text{real} (\text{Suc}\ k) \leq 2 * \text{real}\ k$) *at-top*

by (rule *eventually-at-top-linorderI*[of 1]) *simp*

show *eventually* ($\lambda k. \text{real}\ k \leq 1 * \text{real}\ k$) *at-top*

by *simp*

```

show eventually
  ( $\lambda k$ . path-T-bmssp  $k \leq$ 
    bmssp-refined-graph-time-bound
      ( $\lambda$ -. Suc 1 * sssp-log-one-third-param  $k$ )
      ( $\lambda$ -. sssp-log-two-thirds-param  $k$ )
      ( $\lambda$ -. sssp-log-one-third-param  $k$ )
      ( $\lambda$ -. sssp-log-one-third-param  $k$ )
      ( $\lambda$ -. sssp-log-two-thirds-param  $k$ )
      ( $\lambda$ -.  $k$ ) (Suc  $k$ )) at-top
  using runs-exist
proof eventually-elim
  case (elim  $k$ )
  then obtain  $c$  where cost:
    strict-tie-breaking-digraph.exact-concrete-top-level-cost
      (path-k-V  $k$ ) (path-k-E  $k$ ) (path-k-weight  $k$ ) 0 1  $k$   $c$ 
  by blast
interpret bpi: bounded-reduced-positive-instance
  path-k-V  $k$  path-k-E  $k$  path-k-weight  $k$  0 1
  by (rule path-k-bounded-instance)
have cost': bpi.exact-concrete-top-level-cost 1  $k$   $c$ 
  using cost .
have key: bpi.T-bmssp 1  $k \leq$ 
  bmssp-refined-graph-time-bound
    ( $\lambda$ -. Suc 1 * sssp-log-one-third-param  $k$ )
    ( $\lambda$ -. sssp-log-two-thirds-param  $k$ )
    ( $\lambda$ -. sssp-log-one-third-param  $k$ )
    ( $\lambda$ -. sssp-log-one-third-param  $k$ )
    ( $\lambda$ -. sssp-log-two-thirds-param  $k$ )
    ( $\lambda$ -. bpi.edge-count) bpi.vertex-count
  by (rule bpi.T-bmssp-refined-bound-log-params-fixed-degree
    [OF bpi.all-vertices-reachable bpi.bounded-edge-outdegree cost'])
have ec: bpi.edge-count =  $k$ 
  by (simp add: bpi.edge-count-def path-k-card-E)
have vc: bpi.vertex-count = Suc  $k$ 
  by (simp add: bpi.vertex-count-def path-k-card-V)
have tb: bpi.T-bmssp 1  $k$  = path-T-bmssp  $k$ 
  by (simp add: path-T-bmssp-def)
show ?case
  using key[unfolded ec vc] tb by simp
qed
qed
then show ?thesis
  unfolding sssp-time-target-def sssp-log-factor-def by simp
qed

end
theory BMSSP-Cost-Totality
  imports BMSSP-Path-Family
begin

```

45 On the Totality of the Exact-Concrete Cost Relation

This theory records a precise, machine-checked structural fact about the exact-concrete cost relation *unique-shortest-digraph.exact-concrete-bmssp* and its companion loop relation *unique-shortest-digraph.exact-concrete-partition-loop-state*.

The intended goal was to discharge the run-existence hypothesis *runs-exist* of *bmssp-path-family-runtime-bigo-size* unconditionally, by constructing a terminating partition-loop witness for the path family. The construction reduces, via *unique-shortest-digraph.exact-concrete-bmssp-Suc-exists-from-finite-initial-loop-cases*, to building a partition-loop state whose loop iterates over the successive half-open distance ranges of the pivot tree.

The lemmas below show why that loop cannot iterate more than once under a sound label. The single inductive rule *unique-shortest-digraph.exact-concrete-partition-loop-state* uses for a genuine step (Exact-Concrete-State-Step) carries the hypothesis that the recursive child output equals the incremental range tree *range-tree P a (Fin b)*. But child correctness, available whenever the carried label is sound, pins that same child output to the cumulative bounded tree *bound-tree P (Fin b)*. Since *range-tree P a (Fin b)* is by definition the set difference *bound-tree P (Fin b) – bound-tree P (Fin a)*, the two can agree only when the lower prefix *bound-tree P (Fin a)* is empty. For a reachable singleton source rooted at *s* this prefix is empty exactly when the lower bound is zero, because the source itself has distance zero and therefore lies in every prefix with a strictly positive bound.

The consequence is that a sound exact-concrete partition loop admits a genuine step only at lower bound zero; any second genuine step, which would necessarily occur at a strictly positive lower bound, is impossible. A complete top-level run, whose returned bound is *Infinity*, requires the loop to terminate in the Done case with an emptied partition; with non-empty pivots that demands the single permitted step to leave no residual frontier, i.e. its one child must already solve the entire bounded tree. These observations isolate the obstruction to the unconditional totality result precisely in the shape of the State-Step rule.

Consequence for the path family. On the unit-weight directed path P_k the top-level schedule parameter $p = \text{sssp-log-one-third-param } k$ grows only like $\lceil (\ln k)^{1/3} \rceil$, so $p < k + 1 = \text{the vertex count for every } k \geq 1$; the empty-pivot existence route (which would need *vertex-count* $< p$) therefore never applies, the top pivot set is non-empty, and the loop is forced into a genuine step. By the rigidity above that step sits at lower bound 0, and its single child — bottoming out at the truncating base case *base-case-vertices*, which keeps at most the k closest vertices — settles only an initial prefix of the path, stranding the remaining vertices in a tail loop that has no

applicable rule. Hence *no complete top-level run exists on P_k for $k \geq 2$.*

In particular the hypothesis *runs-exist* of *bmssp-path-family-runtime-bigo-size* is *unsatisfiable* for the path family at every $k \geq 2$. That theorem remains a valid conditional statement, but as a claim about an actual BMSSP run on P_k it is vacuous: the algorithm-level size-parametric running-time bound is not witnessed by any run of the present exact-concrete relation. The *unconditional floor path-size-cost-bigo-size*, which bounds the closed cost *expression* with the graph cardinalities substituted, is unaffected and remains genuinely true. Closing the gap would require relaxing the State-Step rule to mirror the (more permissive) operational loop *operational-partition-loop*, which leaves the child output free rather than equating it with the incremental range tree; that changes the cost relation on which the entire runtime development depends, and is a design decision recorded here rather than undertaken.

context *unique-shortest-digraph*
begin

Step rigidity. In a sound exact-concrete loop step the recursive child output is the cumulative bounded tree, so equating it with the incremental range tree forces the lower prefix tree to be empty.

lemma *exact-concrete-step-forces-empty-prefix:*

assumes *sound: sound-label d*
and *pre: bmssp-pre-full d P (Fin Bmax)*
and *P-reaches: $\bigwedge x. x \in P \implies \text{reachable } s \ x$*
and *pull-split: $S\text{-pull} = \text{split-below } d \ P \ \text{beta}$*
and *beta-bound: bound-le (Fin beta) (Fin Bmax)*
and *child-pre: bmssp-pre-full d S-pull (Fin beta)*
and *a-le-b: $a \leq b$*
and *child:*
exact-concrete-bmssp Δ M-of t h k cap l d S-pull (Fin beta)
d-child (Fin b) U-child c-child
and *U-child-eg: $U\text{-child} = \text{range-tree } P \ a \ (\text{Fin } b)$*
shows *bound-tree P (Fin a) = $\{\}$*

proof –

have *S-pull-reaches: $\bigwedge x. x \in S\text{-pull} \implies \text{reachable } s \ x$*
using *pull-split P-reaches unfolding split-below-def by blast*
have *child-post:*
bmssp-post-full d S-pull (Fin beta) d-child (Fin b) U-child
by *(rule exact-concrete-bmssp-correct[OF child sound child-pre S-pull-reaches])*
have *pre-beta: bmssp-pre-full d P (Fin beta)*
using *bmssp-pre-full-bound-mono[OF pre beta-bound] .*
have *cover-beta: complete-tree-cover d P (Fin beta)*
using *bmssp-pre-full-complete-tree-cover[OF pre-beta] .*
have *child-post-split:*
bmssp-post-full d (split-below d P beta) (Fin beta) d-child (Fin b) U-child
using *child-post pull-split by simp*
have *U-child-tree: $U\text{-child} = \text{bound-tree } P \ (\text{Fin } b)$*


```

    using pull-recursive-post-lifts-bound-tree[OF cover-beta child-post-split] .
  have eq: range-tree P a (Fin b) = bound-tree P (Fin b)
    using U-child-eq U-child-tree by simp
  have sub: bound-tree P (Fin a)  $\subseteq$  bound-tree P (Fin b)
    using a-le-b by (intro bound-tree-bound-mono) simp
  show ?thesis
    using eq sub unfolding range-tree-eq-bound-diff by blast
qed

end

```

```

context strict-tie-breaking-digraph
begin

```

Source persistence. The source lies in every prefix tree with a strictly positive bound, because its true distance is zero.

```

lemma source-in-positive-prefix:
  assumes a-pos:  $0 < a$ 
  shows  $s \in \text{bound-tree } \{s\} \text{ (Fin } a)$ 
proof -
  have reach: reachable s s
    by (rule reachable-refl[OF source-in-V])
  have dist0: dist s s = 0
    by (rule dist-refl-zero[OF source-in-V])
  have below: below-bound (dist s s) (Fin a)
    using dist0 a-pos by simp
  show ?thesis
    by (rule source-in-own-bound-tree[OF - reach below]) simp
qed

```

The obstruction, combining step rigidity with source persistence. For the singleton source a genuine exact-concrete loop step is impossible at any strictly positive lower bound. Equivalently every genuine step must occur at lower bound zero, so a sound loop performs at most one genuine step.

```

lemma no-genuine-exact-concrete-step-at-positive-lower-bound:

```

```

  assumes sound: sound-label d
  and pre: bmssp-pre-full d {s} (Fin Bmax)
  and pull-split: S-pull = split-below d {s} beta
  and beta-bound: bound-le (Fin beta) (Fin Bmax)
  and child-pre: bmssp-pre-full d S-pull (Fin beta)
  and a-pos:  $0 < a$ 
  and a-le-b:  $a \leq b$ 
  and child:
    exact-concrete-bmssp  $\Delta$  M-of t h k cap l d S-pull (Fin beta)
    d-child (Fin b) U-child c-child
  and U-child-eq: U-child = range-tree {s} a (Fin b)
  shows False
proof -
  have P-reaches:  $\bigwedge x. x \in \{s\} \implies \text{reachable } s \ x$ 

```

```

using reachable-refl[OF source-in-V] by simp
have empty: bound-tree {s} (Fin a) = {}
by (rule exact-concrete-step-forces-empty-prefix
    [OF sound pre P-reaches pull-split beta-bound child-pre a-le-b child
     U-child-eq])
have s ∈ bound-tree {s} (Fin a)
by (rule source-in-positive-prefix[OF a-pos])
then show False
using empty by simp
qed

end

end

```

References

- [1] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. DOI: <https://doi.org/10.1007/BF01386390>.
- [2] R. Duan, J. Mao, X. Mao, X. Shu, and L. Yin. Breaking the sorting barrier for directed single-source shortest paths. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, 2025. DOI: <https://doi.org/10.1145/3717823.3718179>.
- [3] M. L. Fredman and R. E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3):596–615, 1987. DOI: <https://doi.org/10.1145/28869.28874>.